



BSI Standards Publication

Information technology — Programming languages — C

National foreword

This British Standard is the UK implementation of ISO/IEC 9899:2024. It supersedes BS ISO/IEC 9899:2018, which is withdrawn.

The UK participation in its preparation was entrusted to Technical Committee IST/5, Programming languages, their environments and system software interfaces.

A list of organizations represented on this committee can be obtained on request to its committee manager.

Contractual and legal considerations

This publication has been prepared in good faith, however no representation, warranty, assurance or undertaking (express or implied) is or will be made, and no responsibility or liability is or will be accepted by BSI in relation to the adequacy, accuracy, completeness or reasonableness of this publication. All and any such responsibility and liability is expressly disclaimed to the full extent permitted by the law.

This publication is provided as is, and is to be used at the recipient's own risk.

The recipient is advised to consider seeking professional guidance with respect to its use of this publication.

This publication is not intended to constitute a contract. Users are responsible for its correct application.

© The British Standards Institution 2024
Published by BSI Standards Limited 2024

ISBN 978 0 539 16035 2

ICS 35.060

Compliance with a British Standard cannot confer immunity from legal obligations.

This British Standard was published under the authority of the Standards Policy and Strategy Committee on 30 November 2024.

Amendments/corrigenda issued since publication

Date	Text affected
------	---------------

Licensed to National Library of Scotland, National Library of Scotland downloaded on 16/11/2024. For status of this document please check <https://knowledge.bsigroup.com>. Reproduction and distribution is prohibited.



International Standard

ISO/IEC 9899

Information technology — Programming languages — C

Technologies de l'information — Langages de programmation — C

**Fifth edition
2024-10**



COPYRIGHT PROTECTED DOCUMENT

© ISO/IEC 2024

All rights reserved. Unless otherwise specified, or required in the context of its implementation, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from either ISO at the address below or ISO's member body in the country of the requester.

ISO copyright office
CP 401 • Ch. de Blandonnet 8
CH-1214 Vernier, Geneva
Phone: +41 22 749 01 11
Email: copyright@iso.org
Website: www.iso.org

Published in Switzerland

Contents

Foreword	xii
Introduction	xiii
1 Scope	1
2 Normative references	2
3 Terms, definitions, and symbols	3
4 Conformance	9
5 Environment	11
5.1 Introduction	11
5.2 Conceptual models	11
5.2.1 Translation environment	11
5.2.2 Execution environments	12
5.3 Environmental considerations	19
5.3.1 Character sets	19
5.3.2 Multibyte characters	20
5.3.3 Character display semantics	21
5.3.4 Signals and interrupts	21
5.3.5 Environmental limits	21
6 Language	35
6.1 Notation	35
6.2 Concepts	35
6.2.1 Scopes of identifiers, type names, and compound literals	35
6.2.2 Linkages of identifiers	36
6.2.3 Name spaces of identifiers	37
6.2.4 Storage durations of objects	37
6.2.5 Types	38
6.2.6 Representations of types	42
6.2.7 Compatible type and composite type	43
6.2.8 Alignment of objects	45
6.2.9 Encodings	45
6.3 Conversions	46
6.3.1 Introduction	46
6.3.2 Arithmetic operands	46

6.3.3	Other operands	49
6.4	Lexical elements	52
6.4.1	General	52
6.4.2	Keywords	53
6.4.3	Identifiers	54
6.4.4	Universal character names	56
6.4.5	Constants	57
6.4.6	String literals	67
6.4.7	Punctuators	68
6.4.8	Header names	69
6.4.9	Preprocessing numbers	70
6.4.10	Comments	70
6.5	Expressions	72
6.5.1	General	72
6.5.2	Primary expressions	73
6.5.3	Postfix operators	74
6.5.4	Unary operators	81
6.5.5	Cast operators	83
6.5.6	Multiplicative operators	84
6.5.7	Additive operators	85
6.5.8	Bitwise shift operators	86
6.5.9	Relational operators	86
6.5.10	Equality operators	87
6.5.11	Bitwise AND operator	88
6.5.12	Bitwise exclusive OR operator	88
6.5.13	Bitwise inclusive OR operator	89
6.5.14	Logical AND operator	89
6.5.15	Logical OR operator	89
6.5.16	Conditional operator	90
6.5.17	Assignment operators	91
6.5.18	Comma operator	94
6.6	Constant expressions	95
6.7	Declarations	97
6.7.1	General	97
6.7.2	Storage-class specifiers	98
6.7.3	Type specifiers	103
6.7.4	Type qualifiers	120
6.7.5	Function specifiers	124
6.7.6	Alignment specifier	125
6.7.7	Declarators	126

6.7.8	Type names	132
6.7.9	Type definitions	133
6.7.10	Type inference	134
6.7.11	Initialization	136
6.7.12	Static assertions	142
6.7.13	Attributes	142
6.8	Statements and blocks	152
6.8.1	General	152
6.8.2	Labeled statements	153
6.8.3	Compound statement	153
6.8.4	Expression and null statements	153
6.8.5	Selection statements	154
6.8.6	Iteration statements	155
6.8.7	Jump statements	156
6.9	External definitions	159
6.9.1	General	159
6.9.2	Function definitions	159
6.9.3	External object definitions	161
6.10	Preprocessing directives	163
6.10.1	General	163
6.10.2	Conditional inclusion	165
6.10.3	Source file inclusion	169
6.10.4	Binary resource inclusion	171
6.10.5	Macro replacement	178
6.10.6	Line control	185
6.10.7	Diagnostic directives	186
6.10.8	Pragma directive	186
6.10.9	Null directive	187
6.10.10	Predefined macro names	187
6.10.11	Pragma operator	189
6.11	Future language directions	190
6.11.1	Floating types	190
6.11.2	Linkages of identifiers	190
6.11.3	External names	190
6.11.4	Character escape sequences	190
6.11.5	Storage-class specifiers	190
6.11.6	Pragma directives	190
6.11.7	Predefined macro names	190

7 Library

191

7.1	Introduction	191
7.1.1	Definitions of terms	191
7.1.2	Standard headers	191
7.1.3	Reserved identifiers	192
7.1.4	Use of library functions	193
7.2	Diagnostics <assert.h>	195
7.2.1	General	195
7.2.2	Program diagnostics	195
7.3	Complex arithmetic <complex.h>	196
7.3.1	Introduction	196
7.3.2	Conventions	196
7.3.3	Branch cuts	197
7.3.4	The CX_LIMITED_RANGE pragma	197
7.3.5	Trigonometric functions	197
7.3.6	Hyperbolic functions	199
7.3.7	Exponential and logarithmic functions	200
7.3.8	Power and absolute-value functions	201
7.3.9	Manipulation functions	202
7.4	Character handling <ctype.h>	205
7.4.1	General	205
7.4.2	Character classification functions	205
7.4.3	Character case mapping functions	207
7.5	Errors <errno.h>	209
7.6	Floating-point environment <fenv.h>	210
7.6.1	General	210
7.6.2	The FENV_ACCESS pragma	212
7.6.3	The FENV_ROUND pragma	213
7.6.4	The FENV_DEC_ROUND pragma	215
7.6.5	Floating-point exceptions	216
7.6.6	Rounding and other control modes	218
7.6.7	Environment	221
7.7	Characteristics of floating types <float.h>	223
7.8	Format conversion of integer types <inttypes.h>	224
7.8.1	General	224
7.8.2	Macros for format specifiers	224
7.8.3	Functions for greatest-width integer types	225
7.9	Alternative spellings <iso646.h>	227
7.10	Characteristics of integer types <limits.h>	228
7.11	Localization <locale.h>	229
7.11.1	General	229

7.11.2	The setlocale function	229
7.11.3	Numeric formatting convention inquiry	230
7.12	Mathematics <math.h>	235
7.12.1	General	235
7.12.2	Treatment of error conditions	238
7.12.3	The FP_CONTRACT pragma	239
7.12.4	Classification macros	239
7.12.5	Trigonometric functions	242
7.12.6	Hyperbolic functions	247
7.12.7	Exponential and logarithmic functions	249
7.12.8	Power and absolute-value functions	257
7.12.9	Error and gamma functions	260
7.12.10	Nearest integer functions	262
7.12.11	Remainder functions	266
7.12.12	Manipulation functions	268
7.12.13	Maximum, minimum, and positive difference functions	270
7.12.14	Fused multiply-add	275
7.12.15	Functions that round result to narrower type	275
7.12.16	Quantum and quantum exponent functions	277
7.12.17	Decimal re-encoding functions	279
7.12.18	Comparison macros	281
7.13	Non-local jumps <setjmp.h>	284
7.13.1	General	284
7.13.2	Save calling environment	284
7.13.3	Restore calling environment	284
7.14	Signal handling <signal.h>	286
7.14.1	General	286
7.14.2	Specify signal handling	286
7.14.3	Send signal	288
7.15	Alignment <stdalign.h>	289
7.16	Variable arguments <stdarg.h>	290
7.16.1	General	290
7.16.2	Variable argument list access macros	290
7.17	Atomics <stdatomic.h>	294
7.17.1	Introduction	294
7.17.2	Initialization	295
7.17.3	Order and consistency	296
7.17.4	Fences	298
7.17.5	Lock-free property	299
7.17.6	Atomic integer types	300

7.17.7	Operations on atomic types	301
7.17.8	Atomic flag type and operations	303
7.18	Bit and byte utilities <stdbit.h>	305
7.18.1	General	305
7.18.2	Endian	305
7.18.3	Count Leading Zeros	306
7.18.4	Count Leading Ones	306
7.18.5	Count Trailing Zeros	306
7.18.6	Count Trailing Ones	307
7.18.7	First Leading Zero	307
7.18.8	First Leading One	308
7.18.9	First Trailing Zero	308
7.18.10	First Trailing One	309
7.18.11	Count Zeros	310
7.18.12	Count Ones	310
7.18.13	Single-bit Check	310
7.18.14	Bit Width	311
7.18.15	Bit Floor	311
7.18.16	Bit Ceiling	312
7.19	Boolean type and values <stdbool.h>	313
7.20	Checked Integer Arithmetic <stdckdint.h>	314
7.20.1	General	314
7.20.2	Checked Integer Operation Type-generic Macros	314
7.21	Common definitions <stddef.h>	315
7.21.1	General	315
7.21.2	The unreachable macro	316
7.21.3	The nullptr_t type	317
7.22	Integer types <stdint.h>	318
7.22.1	General	318
7.22.2	Integer types	318
7.22.3	Widths of specified-width integer types	320
7.22.4	Width of other integer types	320
7.22.5	Macros for integer constants	321
7.22.6	Maximal and minimal values of integer types	321
7.23	Input/output <stdio.h>	322
7.23.1	Introduction	322
7.23.2	Streams	324
7.23.3	Files	325
7.23.4	Operations on files	326
7.23.5	File access functions	328

7.23.6	Formatted input/output functions	331
7.23.7	Character input/output functions	349
7.23.8	Direct input/output functions	352
7.23.9	File positioning functions	353
7.23.10	Error-handling functions	355
7.24	General utilities <stdlib.h>	357
7.24.1	General	357
7.24.2	Numeric conversion functions	357
7.24.3	Pseudo-random sequence generation functions	364
7.24.4	Memory management functions	365
7.24.5	Communication with the environment	368
7.24.6	Searching and sorting utilities	371
7.24.7	Integer arithmetic functions	372
7.24.8	Multibyte/wide character conversion functions	373
7.24.9	Multibyte/wide string conversion functions	374
7.24.10	Alignment of memory	375
7.25	_Noreturn <stdnoreturn.h>	377
7.26	String handling <string.h>	378
7.26.1	String function conventions	378
7.26.2	Copying functions	378
7.26.3	Concatenation functions	380
7.26.4	Comparison functions	381
7.26.5	Search functions	382
7.26.6	Miscellaneous functions	385
7.27	Type-generic math <tgmath.h>	387
7.28	Threads <threads.h>	392
7.28.1	Introduction	392
7.28.2	Initialization functions	393
7.28.3	Condition variable functions	393
7.28.4	Mutex functions	395
7.28.5	Thread functions	397
7.28.6	Thread-specific storage functions	399
7.29	Date and time <time.h>	402
7.29.1	Components of time	402
7.29.2	Time manipulation functions	403
7.29.3	Time conversion functions	406
7.30	Unicode utilities <uchar.h>	412
7.30.1	General	412
7.30.2	Restartable multibyte/wide character conversion functions	412
7.31	Extended multibyte and wide character utilities <wchar.h>	417

7.31.1 Introduction 417

7.31.2 Formatted wide character input/output functions 418

7.31.3 Wide character input/output functions 431

7.31.4 General wide string utilities 435

 7.31.4.1 General 435

 7.31.4.2 Wide string numeric conversion functions 435

 7.31.4.3 Wide string copying functions 440

 7.31.4.4 Wide string concatenation functions 441

 7.31.4.5 Wide string comparison functions 441

 7.31.4.6 Wide string search functions 443

 7.31.4.7 Miscellaneous functions 446

7.31.5 Wide character time conversion functions 446

7.31.6 Extended multibyte/wide character conversion utilities 447

 7.31.6.1 General 447

 7.31.6.2 Single-byte/wide character conversion functions 447

 7.31.6.3 Conversion state functions 448

 7.31.6.4 Restartable multibyte/wide character conversion functions 448

 7.31.6.5 Restartable multibyte/wide string conversion functions 450

7.32 Wide character classification and mapping utilities <wctype.h> 452

 7.32.1 Introduction 452

 7.32.2 Wide character classification utilities 452

 7.32.2.1 General 452

 7.32.2.2 Wide character classification functions 452

 7.32.2.3 Extensible wide character classification functions 455

 7.32.3 Wide character case mapping utilities 456

 7.32.3.1 Wide character case mapping functions 456

 7.32.3.2 Extensible wide character case mapping functions 456

7.33 Future library directions 458

 7.33.1 General 458

 7.33.2 Complex arithmetic <complex.h> 458

 7.33.3 Character handling <ctype.h> 458

 7.33.4 Errors <errno.h> 458

 7.33.5 Floating-point environment <fenv.h> 458

 7.33.6 Characteristics of floating types <float.h> 458

 7.33.7 Format conversion of integer types <inttypes.h> 458

 7.33.8 Localization <locale.h> 458

 7.33.9 Mathematics <math.h> 458

 7.33.10 Signal handling <signal.h> 459

 7.33.11 Atomics <stdatomic.h> 459

 7.33.12 Boolean type and values <stdbool.h> 459

7.33.13 Bit and byte utilities <stdbit.h>	459
7.33.14 Checked Arithmetic Functions <stdckdint.h>	459
7.33.15 Integer types <stdint.h>	459
7.33.16 Input/output <stdio.h>	459
7.33.17 General utilities <stdlib.h>	459
7.33.18 String handling <string.h>	460
7.33.19 Date and time <time.h>	460
7.33.20 Threads <threads.h>	460
7.33.21 Extended multibyte and wide character utilities <wchar.h>	460
7.33.22 Wide character classification and mapping utilities <wctype.h>	460
Annex A (informative) Language syntax summary	461
Annex B (informative) Library summary	476
Annex C (informative) Sequence points	516
Annex D (informative) Universal character names for identifiers	517
Annex E (informative) Implementation limits	519
Annex F (normative) ISO/IEC 60559 floating-point arithmetic	522
Annex G (normative) ISO/IEC 60559-compatible complex arithmetic	553
Annex H (normative) ISO/IEC 60559 interchange and extended types	564
Annex I (informative) Common warnings	597
Annex J (informative) Portability issues	598
Annex K (normative) Bounds-checking interfaces	636
Annex L (normative) Analyzability	684
Annex M (informative) Change History	685
Bibliography	691
Index	692

Foreword

ISO (the International Organization for Standardization) and IEC (the International Electrotechnical Commission) form the specialized system for worldwide standardization. National bodies that are members of ISO or IEC participate in the development of International Standards through technical committees established by the respective organization to deal with particular fields of technical activity. ISO and IEC technical committees collaborate in fields of mutual interest. Other international organizations, governmental and non-governmental, in liaison with ISO and IEC, also take part in the work.

The procedures used to develop this document and those intended for its further maintenance are described in the ISO/IEC Directives, Part 1. In particular, the different approval criteria needed for the different types of document should be noted. This document was drafted in accordance with the editorial rules of the ISO/IEC Directives, Part 2 (see www.iso.org/directives or www.iec.ch/members_experts/refdocs).

ISO and IEC draw attention to the possibility that the implementation of this document may involve the use of (a) patent(s). ISO and IEC take no position concerning the evidence, validity or applicability of any claimed patent rights in respect thereof. As of the date of publication of this document, ISO and IEC had not received notice of (a) patent(s) which may be required to implement this document. However, implementers are cautioned that this may not represent the latest information, which may be obtained from the patent database available at www.iso.org/patents and patents.iec.ch. ISO and IEC shall not be held responsible for identifying any or all such patent rights.

Any trade name used in this document is information given for the convenience of users and does not constitute an endorsement.

For an explanation of the voluntary nature of standards, the meaning of ISO specific terms and expressions related to conformity assessment, as well as information about ISO's adherence to the World Trade Organization (WTO) principles in the Technical Barriers to Trade (TBT) see www.iso.org/iso/foreword.html. In the IEC, see www.iec.ch/understanding-standards.

This document was prepared by Joint Technical Committee ISO/IEC JTC1, Information technology, Subcommittee SC22, Programming languages, their environments and system software interfaces.

This fifth edition cancels and replaces the fourth edition (ISO/IEC 9899:2018), which has been technically revised. The main changes are contained in Annex M.

Any feedback or questions on this document should be directed to the user's national standards body. A complete listing of these bodies can be found at www.iso.org/members.html and www.iec.ch/national-committees.

Introduction

With the introduction of new devices and extended character sets, new features could be added to future editions of this document. Subclauses in the language and library clauses warn implementers and programmers of usages which, though valid in themselves, could conflict with future additions.

Certain features are *obsolescent*, which means that they could be considered for withdrawal in future revisions of this document. They are retained because of their widespread use, but their use in new implementations (for implementation features) or new programs (for language [6.11] or library features [7.33]) is discouraged.

This document is divided into four major subdivisions:

- preliminary elements (Clauses 1–4);
- the characteristics of environments that translate and execute C programs (Clause 5);
- the language syntax, constraints, and semantics (Clause 6);
- the library facilities (Clause 7).

In any given subsequent clause or subclause, there are section delineations in **bold** to describe the semantics, restrictions, and behaviors of programs for this language and potentially the use of its library clauses in this document:

- **Syntax**
which pertains to the spelling and organization of the language and library;
- **Constraints**
which detail and enumerate various requirements for the correct interpretation of the language and library, typically during translation;
- **Semantics**
which explain the behavior of language features and similar constructs;
- **Description**
which explain the behavior of library usage and similar constructs;
- **Returns**
which describes the effects of constructs provided back to a user of the library;
- **Runtime-constraints**
which detail and enumerate various requirements that are expected to be checked and which shall not be violated, typically during execution;
- **Environmental limits**
which list limitations an implementation may impose on a library or language construct which might otherwise be unlimited;
- **Recommended practice**
which provides guidance and important considerations for implementers of this document.

Examples are provided to illustrate possible forms of the constructions described. Footnotes are provided to emphasize consequences of the rules described in that subclause or elsewhere in this document. References are used to refer to other related subclauses. Recommendations are provided to give advice or guidance to implementers. Annexes define optional features, provide additional

information and summarize the information contained in this document. A bibliography lists documents that were referred to during the preparation of this document.

The language clause (Clause 6) is derived from “The C Reference Manual”[15].

The library clause (Clause 7) is based on the 1984 */usr/group Standard*[16].

Information technology — Programming languages — C

1. Scope

This document specifies the form and establishes the interpretation of programs written in the C programming language. It is designed to promote the portability of C programs among a variety of data-processing systems. It is intended for use by implementers and programmers. It specifies:

- the representation of C programs;
- the syntax and constraints of the C language;
- the semantic rules for interpreting C programs;
- the representation of input data to be processed by C programs;
- the representation of output data produced by C programs;
- the restrictions and limits imposed by a conforming implementation of C.

This document does not specify:

- the mechanism by which C programs are transformed for use by a data-processing system;
- the mechanism by which C programs are invoked for use by a data-processing system;
- the mechanism by which input data are transformed for use by a C program;
- the mechanism by which output data are transformed after being produced by a C program;
- the size or complexity of a program and its data that will exceed the capacity of any specific data-processing system or the capacity of a particular processor;
- all minimal requirements of a data-processing system that is capable of supporting a conforming implementation.

Annex J gives an overview of portability issues that a C program can encounter.

2. Normative references

The following documents are referred to in the text in such a way that some or all their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

ISO/IEC 2382:2015, *Information technology — Vocabulary*.

ISO 4217, *Codes for the representation of currencies*.

ISO 8601 series, *Data elements and interchange formats — Information interchange — Representation of dates and times*.

ISO/IEC 10646, *Information technology — Universal Coded Character Set (UCS)*.

ISO/IEC 60559:2020, *Information technology — Microprocessor Systems — Floating-Point arithmetic*.

ISO 80000-2, *Quantities and units — Part 2: Mathematics*.

The Unicode Consortium. *Unicode Standard Annex, UAX #44, Unicode Character Database [online]*. Edited by Ken Whistler. Available at <https://www.unicode.org/reports/tr44>.

The Unicode Consortium. *The Unicode Standard, Derived Core Properties*. Available at <https://www.unicode.org/Public/UCD/latest/ucd/DerivedCoreProperties.txt>.

3. Terms, definitions, and symbols

For the purposes of this document, the terms and definitions given in ISO/IEC 2382, ISO 80000-2, and the following apply.

ISO and IEC maintain terminology databases for use in standardization at the following addresses:

- ISO Online browsing platform: available at <https://www.iso.org/obp>
- IEC Electropedia: available at <https://www.electropedia.org/>

Additional terms are defined where they appear in *italic* type or on the left side of a syntax rule. Terms explicitly defined in this document are not to be presumed to refer implicitly to similar terms defined elsewhere.

3.1

access (verb)

⟨execution-time action⟩ read or modify the value of an object

Note 1 to entry: Where only one of these two actions is meant, “read” or “modify” is used.

Note 2 to entry: “Modify” includes the case where the new value being stored is the same as the previous value.

Note 3 to entry: Expressions that are not evaluated do not access objects.

3.2

alignment

requirement that objects of a particular type be located on storage boundaries with addresses that are particular multiples of a byte (3.7) address

3.3

argument

actual argument

DEPRECATED: actual parameter

expression in the comma-separated list bounded by the parentheses in a function call expression, or a sequence of preprocessing tokens in the comma-separated list bounded by the parentheses in a function-like macro invocation

3.4

arithmetically negate

produce the negative of a given number

Note 1 to entry: For a floating-point number (5.3.5.3.3), this changes the sign; for an integer, this is equivalent to subtracting from zero.

3.5

behavior

external appearance or action

3.5.1

implementation-defined behavior

unspecified behavior where each implementation documents how the choice is made

Note 1 to entry: J.3 gives an overview over properties of C programs that lead to implementation-defined behavior.

EXAMPLE An example of implementation-defined behavior is the propagation of the high-order bit when a signed integer is shifted right.

3.5.2

locale-specific behavior

behavior that depends on local conventions of nationality, culture, and language that each implementation documents

Note 1 to entry: J.4 gives an overview over properties of C programs that lead to locale-specific behavior.

EXAMPLE An example of locale-specific behavior is whether the `islower` function returns true for characters other than the 26 lowercase Latin letters.

3.5.3

undefined behavior

behavior, upon use of a nonportable or erroneous program construct or of erroneous data, for which this document imposes no requirements

Note 1 to entry: Possible undefined behavior ranges from ignoring the situation completely with unpredictable results, to behaving during translation or program execution in a documented manner characteristic of the environment (with or without the issuance of a diagnostic message), to terminating a translation or execution (with the issuance of a diagnostic message).

Note 2 to entry: J.2 gives an overview over properties of C programs that lead to undefined behavior.

Note 3 to entry: Any other behavior during execution of a program is only affected as a direct consequence of the concrete behavior that occurs when encountering the erroneous or non-portable program construct or data. In particular, all observable behavior (5.2.2.4) appears as specified in this document when it happens before an operation with undefined behavior in the execution of the program.

EXAMPLE An example of undefined behavior is the behavior on dereferencing a null pointer.

3.5.4

unspecified behavior

behavior, that results from the use of an unspecified value, or other behavior upon which this document provides two or more possibilities and imposes no further requirements on which is chosen in any instance

Note 1 to entry: J.1 gives an overview over properties of C programs that lead to unspecified behavior.

EXAMPLE An example of unspecified behavior is the order in which the arguments to a function are evaluated.

3.6

bit

unit of data storage in the execution environment large enough to hold an object that can have one of two values

Note 1 to entry: It is possible that each individual bit of an object is not addressable.

3.7

byte

addressable unit of data storage large enough to hold any member of the basic character set of the execution environment

Note 1 to entry: Each individual byte of an object is uniquely addressable.

3.8

low-order bit

the least significant bit within a byte

3.9

high-order bit

the most significant bit within a byte

3.10

character

⟨abstract⟩ member of a set of elements used for the organization, control, or representation of data

3.10.1

character

single-byte character

⟨C⟩ bit representation that fits in a byte

3.10.2

multibyte character

sequence of one or more bytes representing a member of the extended character set of either the source or the execution environment

Note 1 to entry: The extended character set is a superset of the basic character set.

3.10.3

wide character

value representable by an object of type `wchar_t`, capable of representing any character in the current locale

3.11

constraint

restriction, either syntactic or semantic, by which the exposition of language elements is interpreted

3.12

correctly rounded result

representation in the result format that is nearest in value, subject to the current rounding mode, to what the result would be given unlimited range and precision

Note 1 to entry: In this document, the words “correctly rounded” may apply to an operation that produces a correctly rounded result, or to input for such an operation.

Note 2 to entry: ISO/IEC 60559 or implementation-defined rules apply for extreme magnitude results if the result format contains infinity.

3.13

diagnostic message

message belonging to an implementation-defined subset of the implementation’s message output

3.14

forward reference

reference to a subsequent subclause in this document that contains additional information relevant to the subclause containing the reference

3.15

implementation

particular set of software, running in a particular translation environment under particular control options, that performs translation of programs for, and supports execution of functions in, a particular execution environment

3.16

implementation limit

restriction imposed upon programs by the implementation

3.17

memory location

either an object of scalar type, or a maximal sequence of adjacent bit-fields all having nonzero width

Note 1 to entry: Two threads of execution can update and access separate memory locations without interfering with each other.

Note 2 to entry: A bit-field and an adjacent non-bit-field member are in separate memory locations. The same applies to two bit-fields, if one is declared inside a nested structure declaration and the other is not, or if the two are separated by a zero-length bit-field declaration, or if they are separated by a non-bit-field member declaration. It is not safe to concurrently update two non-atomic bit-fields in the same structure if all members declared between them are also (nonzero-length) bit-fields, no matter what the sizes of those intervening bit-fields happen to be.

EXAMPLE A structure declared as

```
struct {  
    char a;  
    int b:5, c:11, :0, d:8;  
    struct { int ee:8; } e;  
}
```

contains four separate memory locations: The member **a**, and bit-fields **d** and **e.ee** are each separate memory locations, and can be modified concurrently without interfering with each other. The bit-fields **b** and **c** together constitute the fourth memory location. The bit-fields **b** and **c** cannot be concurrently modified, but **b** and **a**, for example, can be.

3.18

object

region of data storage in the execution environment, the contents of which can represent values

Note 1 to entry: When referenced, an object can be interpreted as having a particular type; see 6.3.3.1.

3.19

parameter

formal parameter

DEPRECATED: formal argument

object declared as part of a function declaration or definition that acquires a value on entry to the function, or an identifier from the comma-separated list bounded by the parentheses immediately following the macro name in a function-like macro definition

3.20

recommended practice

specification that is strongly recommended as being in keeping with the intent of the standard, but that may be impractical for some implementations

3.21

runtime-constraint

requirement on a program when calling a library function

Note 1 to entry: Despite the similar terms, a runtime-constraint is not a kind of constraint as defined by 3.11, and it is not necessary for it to be diagnosed at translation time.

Note 2 to entry: Implementations that support the extensions in Annex K are required to verify that the runtime-constraints for a library function are not violated by the program; see K.3.1.4.

Note 3 to entry: Implementations that support Annex L are permitted to invoke a runtime-constraint handler when they perform a trap.

3.22

value

precise meaning of the contents of an object when interpreted as having a specific type

3.22.1

implementation-defined value

unspecified value where each implementation documents how the choice is made

3.22.2

unspecified value

valid value of the relevant type where this document imposes no requirements on which value is chosen in any instance

3.23

indeterminate representation

object representation that either represents an unspecified value or is a non-value representation

3.24

non-value representation

an object representation that does not represent a value of the object type

3.25

perform a trap

interrupt execution of the program such that no further operations are performed

Note 1 to entry: Fetching a non-value representation permits an implementation to perform a trap but is not required to (see 6.2.6.1).

Note 2 to entry: Implementations that support Annex L are permitted to invoke a runtime-constraint handler when they perform a trap.

3.26

$\lceil x \rceil$

ceiling of x

the least integer greater than or equal to x

EXAMPLE $\lceil 2.4 \rceil$ is 3, $\lceil -2.4 \rceil$ is -2 .

3.27

$\lfloor x \rfloor$

floor of x

the greatest integer less than or equal to x

EXAMPLE $\lfloor 2.4 \rfloor$ is 2, $\lfloor -2.4 \rfloor$ is -3 .

3.28

wraparound

the process by which a value is reduced modulo 2^N , where N is the width of the resulting type

3.29

out-of-bounds store

(attempted) access (3.1) that, at run time, for a given computational state, would modify (or, for an object declared **volatile**, fetch) one or more bytes that lie outside the bounds permitted by this document.

3.30

bounded undefined behavior

undefined behavior (3.5.3) that does not perform an out-of-bounds store.

Note 1 to entry: The behavior can perform a trap.

Note 2 to entry: Any values produced can be unspecified values, and the representation of objects that are written to can become indeterminate.

3.31

critical undefined behavior

undefined behavior that is not bounded undefined behavior.

Note 1 to entry: The behavior can perform an out-of-bounds store or perform a trap.

4. Conformance

In this document, “shall” is to be interpreted as a requirement on an implementation or on a program; conversely, “shall not” is to be interpreted as a prohibition.

If a “shall” or “shall not” requirement that appears outside of a constraint or runtime-constraint is violated, the behavior is undefined. Undefined behavior is otherwise indicated in this document by the words “undefined behavior” or by the omission of any explicit definition of behavior. There is no difference in emphasis among these three; they all describe “behavior that is undefined”.

A program that is correct in all other aspects, operating on correct data, containing unspecified behavior shall be a correct program and act in accordance with 5.2.2.4.

The implementation shall not successfully translate a preprocessing translation unit containing a **#error** preprocessing directive unless it is part of a group skipped by conditional inclusion.

A *strictly conforming program* shall use only those features of the language and library specified in this document. It shall not produce output dependent on any unspecified, undefined, or implementation-defined behavior, and shall not exceed any minimum implementation limit.

EXAMPLE A strictly conforming program can use conditional features (see 6.10.10.4) provided the use is guarded by an appropriate conditional inclusion preprocessing directive using the related macro. For example:

```
#ifdef __STDC_IEC_60559_BFP__ /* FE_UPWARD defined */
    /* ... */
    fesetround(FE_UPWARD);
    /* ... */
#endif
```

The two forms of *conforming implementation* are hosted and freestanding. A *conforming hosted implementation* shall accept any strictly conforming program. A *conforming freestanding implementation* shall accept any strictly conforming program in which the use of the features specified in the library clause (Clause 7) is confined to the contents of the standard headers `<float.h>`, `<iso646.h>`, `<limits.h>`, `<stdalign.h>`, `<stdarg.h>`, `<stdbit.h>`, `<stdbool.h>`, `<stddef.h>`, `<stdint.h>`, and `<stdnoreturn.h>`. Additionally, a *conforming freestanding implementation* shall accept any strictly conforming program where:

- the features specified in the header `<string.h>` are used, except the following functions: **strcoll**, **strdup**, **strerror**, **strndup**, **strtok**, **strxfrm**; and/or,
- the selected function **memalign** from `<stdlib.h>` is used.

A conforming implementation may have extensions (including additional library functions), provided they do not alter the behavior of any strictly conforming program.¹⁾

The strictly conforming programs that shall be accepted by a conforming freestanding implementation that defines `__STDC_IEC_60559_BFP__` or `__STDC_IEC_60559_DFP__` can also use features in the contents of the standard headers `<fenv.h>`, `<math.h>`, and the **strto*** floating-point numeric conversion functions (7.24.2) of the standard header `<stdlib.h>`, provided the program does not set the state of the **FENV_ACCESS** pragma to “on”.

All identifiers that are reserved when `<stdlib.h>` is included in a hosted implementation are reserved when it is included in a freestanding implementation.

A *conforming program* is one that is acceptable to a conforming implementation.²⁾

¹⁾This implies that a conforming implementation reserves no identifiers other than those explicitly reserved in this document.

²⁾Strictly conforming programs are intended to be maximally portable among conforming implementations. Conforming programs can depend upon nonportable features of a conforming implementation.

An implementation shall be accompanied by a document that defines all implementation-defined and locale-specific characteristics and all extensions.

Forward references: conditional inclusion (6.10.2), error directive (6.10.7), characteristics of floating types `<float.h>` (7.7), alternative spellings `<iso646.h>` (7.9), sizes of integer types `<limits.h>` (7.10), alignment `<stdalign.h>` (7.15), variable arguments `<stdarg.h>` (7.16), boolean type and values `<stdbool.h>` (7.19), common definitions `<stddef.h>` (7.21), integer types `<stdint.h>` (7.22), `<stdnoreturn.h>` (7.25).

5. Environment

5.1 Introduction

An implementation translates C source files and executes C programs in two data-processing-system environments, which will be called the *translation environment* and the *execution environment* in this document. Their characteristics define and constrain the results of executing conforming C programs constructed according to the syntactic and semantic rules for conforming implementations.

NOTE In this clause, only a few of many possible forward references have been noted.

5.2 Conceptual models

5.2.1 Translation environment

5.2.1.1 Program structure

A C program is not required to be translated in its entirety at the same time. The text of the program is kept in units called *source files*, (or *preprocessing files*) in this document. A source file together with all the headers and source files included via the preprocessing directive `#include` is known as a *preprocessing translation unit*. After preprocessing, a preprocessing translation unit is called a *translation unit*. Previously translated translation units can be preserved individually or in libraries. The separate translation units of a program communicate by (for example) calls to functions whose identifiers have external linkage, manipulation of objects whose identifiers have external linkage, or manipulation of data files. Translation units may be separately translated and then later linked to produce an executable program.

Forward references: linkages of identifiers (6.2.2), external definitions (6.9), preprocessing directives (6.10).

5.2.1.2 Translation phases

The precedence among the syntax rules of translation is specified by the following phases.³⁾

1. Physical source file multibyte characters are mapped, in an implementation-defined manner, to the source character set (introducing new-line characters for end-of-line indicators) if necessary.
2. Each instance of a backslash character (\) immediately followed by a new-line character is deleted, splicing physical source lines to form logical source lines. Only the last backslash on any physical source line shall be eligible for being part of such a splice. A source file that is not empty shall end in a new-line character, which shall not be immediately preceded by a backslash character before any such splicing takes place.
3. The source file is decomposed into preprocessing tokens⁴⁾ and sequences of white-space characters (including comments). A source file shall not end in a partial preprocessing token or in a partial comment. Each comment is replaced by one space character. New-line characters are retained. Whether each nonempty sequence of white-space characters other than new-line is retained or replaced by one space character is implementation-defined.
4. Preprocessing directives are executed, macro invocations are expanded, and `_Pragma` unary operator expressions are executed. If a character sequence that matches the syntax of a universal character name is produced by token concatenation (6.10.5.4), the behavior is undefined. A `#include` preprocessing directive causes the named header or source file to be processed from phase 1 through phase 4, recursively. All preprocessing directives are then deleted.

³⁾This requires implementations to behave as if these separate phases occur, even though many are typically folded together in practice. Source files, translation units, and translated translation units necessarily can be stored as files or through/within any other implementation-defined medium. There is no expectation of any one-to-one correspondence between these entities and any external representation. The description is conceptual only, and does not specify any particular implementation.

⁴⁾As described in 6.4, the process of dividing a source file's characters into preprocessing tokens is context-dependent. For example, see the handling of `<` within a `#include` preprocessing directive.

5. Each source character set member and escape sequence in character constants and string literals is converted to the corresponding member of the execution character set. Each instance of a source character or escape sequence for which there is no corresponding member is converted in an implementation-defined manner to some member of the execution character set other than the null (wide) character.⁵⁾
6. Adjacent string literal tokens are concatenated.
7. White-space characters separating tokens are no longer significant. Each preprocessing token is converted into a token. The resulting tokens are syntactically and semantically analyzed and translated as a translation unit.
8. All external object and function references are resolved. Library components are linked to satisfy external references to functions and objects not defined in the current translation. All such translator output is collected into a program image which contains information needed for execution in its execution environment.

Forward references: universal character names (6.4.4), lexical elements (6.4), preprocessing directives (6.10), external definitions (6.9).

5.2.1.3 Diagnostics

A conforming implementation shall produce at least one diagnostic message (identified in an implementation-defined manner) if a preprocessing translation unit or translation unit contains a violation of any syntax rule or constraint, even if the behavior is also explicitly specified as undefined or implementation-defined. Diagnostic messages are not required to be produced in other circumstances.

EXAMPLE An implementation is required to issue a diagnostic for the translation unit:

```
char i;  
int i;
```

because in those cases where wording in this document describes the behavior for a construct as being both a constraint error and resulting in undefined behavior, the constraint error is still required to be diagnosed.

Recommended practice

An implementation is encouraged to identify the nature of, and where possible localize, each violation. Of course, an implementation is free to produce any number of diagnostic messages, often referred to as warnings, as long as a valid program is still correctly translated. It can also successfully translate an invalid program. Annex I lists a few of the more common warnings.

5.2.2 Execution environments

5.2.2.1 General

Two execution environments are defined: *freestanding* and *hosted*. In both cases, *program startup* occurs when a designated C function is called by the execution environment. All objects with static storage duration shall be *initialized* (set to their initial values) before program startup. The manner and timing of such initialization are otherwise unspecified. *Program termination* returns control to the execution environment.

Forward references: storage durations of objects (6.2.4), initialization (6.7.11).

5.2.2.2 Freestanding environment

In a freestanding environment (in which C program execution can take place without any benefit of an operating system), the name and type of the function called at program startup are implementation-defined. Any library facilities available to a freestanding program, other than the minimal set required by Clause 4, are implementation-defined.

The effect of program termination in a freestanding environment is implementation-defined.

⁵⁾An implementation can convert each instance of the same non-corresponding source character to a different member of the execution character set.

5.2.2.3 Hosted environment

5.2.2.3.1 General

A hosted environment is not required to be provided, but shall conform to the following specifications if present.

5.2.2.3.2 Program startup

The function called at program startup is named **main**. The implementation declares no prototype for this function. It shall be defined with a return type of **int** and with no parameters:

```
int main(void) { /* ... */ }
```

or with two parameters (referred to here as **argc** and **argv**, though any names can be used, as they are local to the function in which they are declared):

```
int main(int argc, char *argv[]) { /* ... */ }
```

or equivalent;⁶⁾ or in some other implementation-defined manner.

If they are declared, the parameters to the **main** function shall obey the following constraints:

- The value of **argc** shall be nonnegative.
- **argv[argc]** shall be a null pointer.
- If the value of **argc** is greater than zero, the array members **argv[0]** through **argv[argc-1]** inclusive shall contain pointers to strings, which are given implementation-defined values by the host environment prior to program startup. The intent is to supply to the program information determined prior to program startup from elsewhere in the hosted environment. If the host environment is not capable of supplying strings with letters in both uppercase and lowercase, the implementation shall ensure that the strings are received in lowercase.
- If the value of **argc** is greater than zero, the string pointed to by **argv[0]** represents the *program name*; **argv[0][0]** shall be the null character if the program name is not available from the host environment. If the value of **argc** is greater than one, the strings pointed to by **argv[1]** through **argv[argc-1]** represent the *program parameters*.
- The parameters **argc** and **argv** and the strings pointed to by the **argv** array shall be modifiable by the program, and retain their last-stored values between program startup and program termination.

5.2.2.3.3 Program execution

In a hosted environment, a program can use all the functions, macros, type definitions, and objects described in the library clause (Clause 7).

5.2.2.3.4 Program termination

If the return type of the **main** function is a type compatible with **int**, a return from the initial call to the **main** function is equivalent to calling the **exit** function with the value returned by the **main** function as its argument;⁷⁾ reaching the **}** that terminates the **main** function returns a value of 0. If the return type is not compatible with **int**, the termination status returned to the host environment is unspecified.

Forward references: definition of terms (7.1.1), the **exit** function (7.24.5.4).

⁶⁾Thus, **int** can be replaced by a typedef name defined as **int**, or the type of **argv** can be written as **char ** argv**, or the return type can be specified by **typeof(1)**, and so on.

⁷⁾In accordance with 6.2.4, the lifetimes of objects with automatic storage duration declared in **main** will have ended in the former case, even where they would not have in the latter.

5.2.2.4 Program semantics

The semantic descriptions in this document describe the behavior of an abstract machine in which issues of optimization are irrelevant.

An access to an object through the use of an lvalue of volatile-qualified type is a *volatile access*. A volatile access to an object, modifying an object, modifying a file, or calling a function that does any of those operations are all *side effects*,⁸⁾ which are changes in the state of the execution environment.

Evaluation of an expression in general includes both value computations and initiation of side effects. Value computation for an lvalue expression includes determining the identity of the designated object.

Sequenced before is an asymmetric, transitive, pair-wise relation between evaluations executed by a single thread, which induces a partial order among those evaluations. Given any two evaluations *A* and *B*, if *A* is sequenced before *B*, then the execution of *A* shall precede the execution of *B*. (Conversely, if *A* is sequenced before *B*, then *B* is *sequenced after A*.) If *A* is not sequenced before or after *B*, then *A* and *B* are *unsequenced*. Evaluations *A* and *B* are *indeterminately sequenced* when *A* is sequenced either before or after *B*, but it is unspecified which.⁹⁾ The presence of a *sequence point* between the evaluation of expressions *A* and *B* implies that every value computation and side effect associated with *A* is sequenced before every value computation and side effect associated with *B*. (A summary of the sequence points is given in Annex C.)

In the abstract machine, all expressions are evaluated as specified by the semantics. An actual implementation is not required to evaluate part of an expression if it can deduce that its value is not used and that no needed side effects are produced (including any caused by calling a function or through volatile access to an object).

When the processing of the abstract machine is interrupted by receipt of a signal, the values of objects that are neither lock-free atomic objects nor of type **volatile sig_atomic_t** are unspecified, as is the state of the dynamic floating-point environment. The representation of any object modified by the handler that is neither a lock-free atomic object nor of type **volatile sig_atomic_t** becomes indeterminate when the handler exits, as does the state of the dynamic floating-point environment if it is modified by the handler and not restored to its original state.

The least requirements on a conforming implementation are:

- Volatile accesses to objects are evaluated strictly according to the rules of the abstract machine.
- At program termination, all data written into files shall be identical to the result that execution of the program according to the abstract semantics would have produced.
- The input and output dynamics of interactive devices shall take place as specified in 7.23.3. The intent of these requirements is that unbuffered or line-buffered output appear as soon as possible, to ensure that prompting messages appear prior to a program waiting for input.

This is the *observable behavior* of the program.

What constitutes an interactive device is implementation-defined.

More stringent correspondences between abstract and actual semantics may be defined by each implementation.

EXAMPLE 1 An implementation can define a one-to-one correspondence between abstract and actual semantics: at every sequence point, the values of the actual objects would agree with those specified by the abstract semantics. The keyword **volatile** would then be redundant.

Alternatively, an implementation can perform various optimizations within each translation unit, such that the actual semantics would agree with the abstract semantics only when making function calls across translation

⁸⁾ISO/IEC 60559 requires certain user-accessible status flags and control modes. Floating-point operations implicitly set the status flags; modes affect result values of floating-point operations. Implementations that support such floating-point state are required to regard changes to it as side effects — see Annex F for details. The floating-point environment library <fenv.h> provides a programming facility for indicating when these side effects matter, freeing the implementations in other cases.

⁹⁾The executions of unsequenced evaluations can interleave. Indeterminately sequenced evaluations cannot interleave, but can be executed in any order.

unit boundaries. In such an implementation, at the time of each function entry and function return where the calling function and the called function are in different translation units, the values of all externally linked objects and of all objects accessible via pointers therein would agree with the abstract semantics. Furthermore, at the time of each such function entry the values of the parameters of the called function and of all objects accessible via pointers therein would agree with the abstract semantics. In this type of implementation, objects referred to by interrupt service routines activated by the **signal** function would require explicit specification of **volatile** storage, as well as other implementation-defined restrictions.

EXAMPLE 2 In executing the fragment

```
char c1, c2;
/* ... */
c1 = c1 + c2;
```

the “integer promotions” require that the abstract machine promote the value of each variable to **int** size and then add the two **ints** and truncate the sum. Provided the addition of two **chars** can be done without integer overflow, or with integer overflow wrapping silently to produce the correct result, the actual execution need only produce the same result, possibly omitting the promotions.

EXAMPLE 3 Similarly, in the fragment

```
float f1, f2;
double d;
/* ... */
f1 = f2 * d;
```

the multiplication can be executed using **float** arithmetic if the implementation can ascertain that the result would be the same as if it were executed using **double** arithmetic (for example, if **d** were replaced by the constant **2.0**, which has type **double**).

EXAMPLE 4 Implementations employing wide registers are expected to take care to honor appropriate semantics. Values are independent of whether they are represented in a register or in memory. For example, an implicit *spilling* of a register can not alter the value. Also, an explicit *store and load* rounds to the precision the precision of the storage type. In particular, casts and assignments perform their specified conversion. For the fragment

```
double d1, d2;
float f;
d1 = f = expression;
d2 = (float) expression;
```

the values assigned to **d1** and **d2** are required to have been converted to **float**.

EXAMPLE 5 Rearrangement for floating-point expressions is often restricted because of limitations in precision as well as range. The implementation cannot generally apply the mathematical associative rules for addition or multiplication, nor the distributive rule, because of roundoff error, even in the absence of overflow and underflow. Likewise, implementations cannot generally replace decimal constants to rearrange expressions. In the following fragment, rearrangements suggested by mathematical rules for real numbers are often not valid (see F.9).

```
double x, y, z;
/* ... */
x = (x * y) * z; // not equivalent to x *= y * z;
z = (x - y) + y; // not equivalent to z = x;
z = x + x * y;   // not equivalent to z = x * (1.0 + y);
y = x / 5.0;     // not equivalent to y = x * 0.2;
```

EXAMPLE 6 To illustrate the grouping behavior of expressions, in the following fragment

```
int a, b;
/* ... */
a = a + 32760 + b + 5;
```

the expression statement behaves exactly the same as

```
a = ((a + 32760) + b) + 5;
```

due to the associativity and precedence of these operators. Thus, the result of the sum (**a** + 32760) is next added to **b**, and that result is then added to 5 which results in the value assigned to **a**. On a machine in which integer overflows produce an explicit trap and in which the range of values representable by an **int** is [−32768, +32767], the implementation cannot rewrite this expression as

```
a = ((a + b) + 32765);
```

since if the values for **a** and **b** were, respectively, −32754 and −15, the sum **a** + **b** would produce a trap while the original expression would not; nor can the expression be rewritten either as

```
a = ((a + 32765) + b);
```

or

```
a = (a + (b + 32765));
```

since the values for **a** and **b** can be, respectively, 4 and −8 or −17 and 12. However, on an implementation in which integer overflow silently generates some value and where positive and negative integer overflows cancel, the preceding expression statement can be rewritten by the implementation in any of the previously specified ways because the same result will occur.

EXAMPLE 7 The grouping of an expression does not completely determine its evaluation. In the following fragment:

```
#include <stdio.h>
int sum;
char *p;
/* ... */
sum = sum * 10 - '0' + (*p++ = getchar());
```

the expression statement is grouped as if it were written as

```
sum = (((sum * 10) - '0') + ((*p++) = (getchar())));
```

but the actual increment of **p** can occur at any time between the previous sequence point and the next sequence point (the ;), and the call to **getchar** can occur at any point prior to the need of its returned value.

Forward references: expressions (6.5.1), type qualifiers (6.7.4), statements (6.8), floating-point environment <fenv.h> (7.6), the **signal** function (7.14), files (7.23.3).

5.2.2.5 Multi-threaded executions and data races

Under a hosted implementation that does not define **__STDC_NO_THREADS__**, a program can have more than one *thread of execution* (or *thread*) running concurrently. The execution of each thread proceeds as defined by the remainder of this document. The execution of the entire program consists of an execution of all its threads.¹⁰⁾ Under a freestanding implementation, it is implementation-defined whether a program can have more than one thread of execution.

The value of an object visible to a thread *T* at a particular point is the initial value of the object, a value stored in the object by *T*, or a value stored in the object by another thread, according to the rules in the rest of this subclause.

NOTE 1 In some cases, there can instead be undefined behavior. Much of this section is motivated by the desire to support atomic operations with explicit and detailed visibility constraints. However, it also implicitly supports a simpler view for more restricted programs.

Two expression evaluations *conflict* if one of them modifies a memory location and the other one reads or modifies the same memory location.

¹⁰⁾The execution can usually be viewed as an interleaving of all the threads. However, some kinds of atomic operations, for example, allow executions inconsistent with a simple interleaving as described in this subclause.

The library defines *atomic operations* (7.17) and operations on mutexes (7.28.4) that are specially identified as synchronization operations. These operations play a special role in making assignments in one thread visible to another. A *synchronization operation* on one or more memory locations is one of an *acquire operation*, a *release operation*, both an acquire and release operation, or a *consume operation*. A synchronization operation without an associated memory location is a *fence* and can be either an acquire fence, a release fence, or both an acquire and release fence. In addition, there are *relaxed atomic operations*, which are not synchronization operations, and atomic *read-modify-write operations*, which have special characteristics.

NOTE 2 For example, a call that acquires a mutex will perform an acquire operation on the locations composing the mutex. Correspondingly, a call that releases the same mutex will perform a release operation on those same locations. Informally, performing a release operation on *A* forces prior side effects on other memory locations to become visible to other threads that later perform an acquire or consume operation on *A*. Relaxed atomic operations are not included as synchronization operations although, like synchronization operations, they cannot contribute to data races.

All modifications to a particular atomic object *M* occur in some particular total order, called the *modification order* of *M*. If *A* and *B* are modifications of an atomic object *M*, and *A* happens before *B*, then *A* shall precede *B* in the modification order of *M*, which is defined later in this subclause.

NOTE 3 This states that the modification orders are expected to respect the “happens before” relation.

NOTE 4 There is a separate order for each atomic object. There is no requirement that these can be combined into a single total order for all objects. In general this will be impossible since different threads can observe modifications to different variables in inconsistent orders.

A *release sequence* headed by a release operation *A* on an atomic object *M* is a maximal contiguous sub-sequence of side effects in the modification order of *M*, where the first operation is *A* and every subsequent operation either is performed by the same thread that performed the release or is an atomic read-modify-write operation.

Certain library calls *synchronize with* other library calls performed by another thread. In particular, an atomic operation *A* that performs a release operation on an object *M* synchronizes with an atomic operation *B* that performs an acquire operation on *M* and reads a value written by any side effect in the release sequence headed by *A*.

NOTE 5 Except in the specified cases, reading a later value does not necessarily ensure visibility as described later in this subclause. Such a requirement would sometimes interfere with efficient implementation.

NOTE 6 The specifications of the synchronization operations define when one reads the value written by another. For atomic variables, the definition is clear. All operations on a given mutex occur in a single total order. Each mutex acquisition “reads the value written” by the last mutex release.

An evaluation *A* carries a dependency¹¹⁾ to an evaluation *B* if:

- the value of *A* is used as an operand of *B*, unless:
 - *B* is an invocation of the **kill_dependency** macro,
 - *A* is the left operand of a **&&** or **||** operator,
 - *A* is the left operand of a **?:** operator, or
 - *A* is the left operand of a **,** operator;
- or
- *A* writes a scalar object or bit-field *M*, *B* reads from *M* the value written by *A*, and *A* is sequenced before *B*, or
- for some evaluation *X*, *A* carries a dependency to *X* and *X* carries a dependency to *B*.

An evaluation *A* is *dependency-ordered before*¹²⁾ an evaluation *B* if:

¹¹⁾The “carries a dependency” relation is a subset of the “sequenced before” relation, and is similarly strictly intra-thread.

¹²⁾The “dependency-ordered before” relation is analogous to the “synchronizes with” relation, but uses release/consume in place of release/acquire.

- A performs a release operation on an atomic object M , and, in another thread, B performs a consume operation on M and reads a value written by any side effect in the release sequence headed by A , or
- for some evaluation X , A is dependency-ordered before X and X carries a dependency to B .

An evaluation A *inter-thread happens before* an evaluation B if A synchronizes with B , A is dependency-ordered before B , or, for some evaluation X :

- A synchronizes with X and X is sequenced before B ,
- A is sequenced before X and X inter-thread happens before B , or
- A inter-thread happens before X and X inter-thread happens before B .

NOTE 7 The “inter-thread happens before” relation describes arbitrary concatenations of “sequenced before”, “synchronizes with”, and “dependency-ordered before” relationships, with two exceptions. The first exception is that a concatenation is not permitted to end with “dependency-ordered before” followed by “sequenced before”. The reason for this limitation is that a consume operation participating in a “dependency-ordered before” relationship provides ordering only with respect to operations to which this consume operation carries a dependency. The reason that this limitation applies only to the end of such a concatenation is that any subsequent release operation will provide the required ordering for a prior consume operation. The second exception is that a concatenation is not permitted to consist entirely of “sequenced before”. The reasons for this limitation are (1) to permit “inter-thread happens before” to be transitively closed and (2) the “happens before” relation, defined subsequently in this subclause, provides for relationships consisting entirely of “sequenced before”.

An evaluation A *happens before* an evaluation B if A is sequenced before B or A inter-thread happens before B . The implementation shall ensure that no program execution demonstrates a cycle in the “happens before” relation.

NOTE 8 This cycle would otherwise be possible only through the use of consume operations.

A *visible side effect* A on an object M with respect to a value computation B of M satisfies the conditions:

- A happens before B , and
- there is no other side effect X to M such that A happens before X and X happens before B .

The value of a non-atomic scalar object M , as determined by evaluation B , shall be the value stored by the visible side effect A .

NOTE 9 If there is ambiguity about which side effect to a non-atomic object is visible, then there is a data race and the behavior is undefined.

NOTE 10 This states that operations on ordinary variables are not visibly reordered. This is not detectable without data races, but ensures that data races, as defined here, and with suitable restrictions on the use of atomics, correspond to data races in a simple interleaved (sequentially consistent) execution.

The value of an atomic object M , as determined by evaluation B , shall be the value stored by some side effect A that modifies M , where B does not happen before A .

NOTE 11 The set of side effects from which a given evaluation can take its value is also restricted by the rest of the rules described here, and in particular, by the coherence requirements subsequently in this subclause.

If an operation A that modifies an atomic object M happens before an operation B that modifies M , then A shall be earlier than B in the modification order of M .

NOTE 12 Such a requirement is known as “write-write coherence”.

If a value computation A of an atomic object M happens before a value computation B of M , and A takes its value from a side effect X on M , then the value computed by B shall either be the value stored by X or the value stored by a side effect Y on M , where Y follows X in the modification order of M .

NOTE 13 Such a requirement is known as “read-read coherence”.

If a value computation A of an atomic object M happens before an operation B on M , then A shall take its value from a side effect X on M , where X precedes B in the modification order of M .

NOTE 14 Such a requirement is known as “read-write coherence”.

If a side effect X on an atomic object M happens before a value computation B of M , then the evaluation B shall take its value from X or from a side effect Y that follows X in the modification order of M .

NOTE 15 Such a requirement is known as “write-read coherence”.

NOTE 16 This effectively disallows compiler reordering of atomic operations to a single object, even if both operations are “relaxed” loads. By doing so, it effectively makes the “cache coherence” guarantee provided by most hardware available to C atomic operations.

NOTE 17 The value observed by a load of an atomic object depends on the “happens before” relation, which in turn depends on the values observed by loads of atomic objects. The intended reading is that there exists an association of atomic loads with modifications they observe that, together with suitably chosen modification orders and the “happens before” relation derived as described previously, satisfy the resulting constraints as imposed here.

The execution of a program contains a *data race* if it contains two conflicting actions in different threads, at least one of which is not atomic, and neither happens before the other. Any such data race results in undefined behavior.

NOTE 18 It can be shown that programs that correctly use simple mutexes and `memory_order_seq_cst` operations to prevent all data races, and use no other synchronization operations, behave as though the operations executed by their constituent threads were simply interleaved, with each value computation of an object being the last value stored in that interleaving. This is normally referred to as “sequential consistency”. However, this applies only to data-race-free programs, and data-race-free programs cannot observe most program transformations that do not change single-threaded program semantics. In fact, most single-threaded program transformations continue to be allowed, since any program that behaves differently as a result of such transformations necessarily has undefined behavior even before such a transformation is applied.

NOTE 19 Compiler transformations that introduce assignments to a potentially shared memory location that would not be modified by the abstract machine are generally precluded by this document, since such an assignment can overwrite another assignment by a different thread in cases in which an abstract machine execution would not have encountered a data race. This includes implementations of data member assignment that overwrite adjacent members in separate memory locations. Reordering of atomic loads in cases in which the atomics in question can alias is also generally precluded, since this can violate the coherence requirements.

NOTE 20 Transformations that introduce a speculative read of a potentially shared memory location possibly will not preserve the semantics of the program as defined in this document, since they potentially introduce a data race. However, they are typically valid in the context of an optimizing compiler that targets a specific machine with well-defined semantics for data races. They would be invalid for a hypothetical machine that is not tolerant of races or provides hardware race detection.

5.3 Environmental considerations

5.3.1 Character sets

Two sets of characters and their associated *collating sequences* shall be defined: the set in which source files are written (the *source character set*), and the set interpreted in the execution environment (the *execution character set*). Each set is further divided into a *basic character set*, whose contents are given by this subclause, and a set of zero or more locale-specific members (which are not members of the basic character set) called *extended characters*. The combined set is also called the *extended character set*. The values of the members of the execution character set are implementation-defined.

In a character constant or string literal, members of the execution character set shall be represented by corresponding members of the source character set or by *escape sequences* consisting of the backslash \ followed by one or more characters. A byte with all bits set to 0, called the *null character*, shall exist in the basic execution character set; it is used to terminate a character string.

Both the basic source and basic execution character sets shall have the following members: the 26 *uppercase letters* of the *Latin alphabet*

A B C D E F G H I J K L M
N O P Q R S T U V W X Y Z

the 26 *lowercase letters* of the Latin alphabet

a b c d e f g h i j k l m
n o p q r s t u v w x y z

the 10 decimal *digit*

0 1 2 3 4 5 6 7 8 9

the following 32 *graphic characters*

! " # % & ' () * + , - . / :
; < = > ? [\] ^ _ { | } ~
@ \$ %

the space character, and control characters representing horizontal tab, vertical tab, and form feed. The representation of each member of the source and execution basic character sets shall fit in a byte. In both the source and execution basic character sets, the value of each character after 0 in the preceding list of decimal digits shall be one greater than the value of the previous. In source files, there shall be some way of indicating the end of each line of text; this document treats such an end-of-line indicator as if it were a single new-line character. In the basic execution character set, there shall be control characters representing alert, backspace, carriage return, and new line. If any other characters are encountered in a source file (except in an identifier, a character constant, a string literal, a header name, a comment, or a preprocessing token that is never converted to a token), the behavior is undefined.

A *letter* is an uppercase letter or a lowercase letter as defined previously in this subclause; in this document the term does not include other characters that are letters in other alphabets.

The universal character name construct provides a way to name other characters.

Forward references: universal character names (6.4.4), character constants (6.4.5.5), preprocessing directives (6.10), string literals (6.4.6), comments (6.4.10), string (7.1.1).

5.3.2 Multibyte characters

The source character set can contain multibyte characters, used to represent members of the extended character set. The execution character set can also contain multibyte characters, which are not required to have the same encoding as for the source character set. For both character sets, the following shall hold:

- The basic character set shall be present and each character shall be encoded as a single byte.
- The presence, meaning, and representation of any additional members is locale-specific.
- A multibyte character set can have a *state-dependent encoding*, wherein each sequence of multi-byte characters begins in an *initial shift state* and enters other locale-specific *shift states* when specific multibyte characters are encountered in the sequence. While in the initial shift state, all single-byte characters retain their usual interpretation and do not alter the shift state. The interpretation for subsequent bytes in the sequence is a function of the current shift state.
- A byte with all bits zero shall be interpreted as a null character independent of shift state. Such a byte shall not occur as part of any other multibyte character.

For source files, the following shall hold:

- An identifier, comment, string literal, character constant, or header name shall begin and end in the initial shift state.
- An identifier, comment, string literal, character constant, or header name shall consist of a sequence of valid multibyte characters.

5.3.3 Character display semantics

The *active position* is that location on a display device where the next character output by the **fputc** function would appear. The intent of writing a printing character (as defined by the **isprint** function) to a display device is to display a graphic representation of that character at the active position and then advance the active position to the next position on the current line. The direction of writing is locale-specific. If the active position is at the final position of a line (if there is one), the behavior of the display device is unspecified.

Alphabetic escape sequences representing non-graphic characters in the execution character set are intended to produce actions on display devices as follows:

- \a (*alert*) Produces an audible or visible alert without changing the active position.
- \b (*backspace*) Moves the active position to the previous position on the current line. If the active position is at the initial position of a line, the behavior of the display device is unspecified.
- \f (*form feed*) Moves the active position to the initial position at the start of the next logical page.
- \n (*new line*) Moves the active position to the initial position of the next line.
- \r (*carriage return*) Moves the active position to the initial position of the current line.
- \t (*horizontal tab*) Moves the active position to the next horizontal tabulation position on the current line. If the active position is at or past the last defined horizontal tabulation position, the behavior of the display device is unspecified.
- \v (*vertical tab*) Moves the active position to the initial position of the next vertical tabulation position. If the active position is at or past the last defined vertical tabulation position, the behavior of the display device is unspecified.

Each of these escape sequences shall produce a unique implementation-defined value which can be stored in a single **char** object. The external representations in a text file are not necessarily identical to the internal representations, and are outside the scope of this document.

Forward references: the **isprint** function (7.4.2.9), the **fputc** function (7.23.7.3).

5.3.4 Signals and interrupts

Functions shall be implemented such that they can be interrupted at any time by a signal, or can be called by a signal handler, or both, with no alteration to earlier, but still active, invocations' control flow (after the interruption), function return values, or objects with automatic storage duration. All such objects shall be maintained outside the *function image* (the instructions that compose the executable representation of a function) on a per-invocation basis.

5.3.5 Environmental limits

5.3.5.1 General

Both the translation and execution environments constrain the implementation of language translators and libraries. The following summarizes the language-related environmental limits on a conforming implementation; the library-related limits are discussed in Clause 7.

5.3.5.2 Translation limits

The implementation shall be able to translate and execute a program that uses but does not exceed the following limitations for these constructs and entities:¹³⁾

- 127 nesting levels of blocks
- 63 nesting levels of conditional inclusion
- 12 pointer, array, and function declarators (in any combinations) modifying an arithmetic, structure, union, or **void** type in a declaration

¹³⁾Implementations are encouraged to avoid imposing fixed translation limits whenever possible.

- 63 nesting levels of parenthesized declarators within a full declarator
- 63 nesting levels of parenthesized expressions within a full expression
- 63 significant initial characters in an internal identifier or a macro name (each universal character name or extended source character is considered a single character)
- 31 significant initial characters in an external identifier (each universal character name specifying a short identifier of 00FFFF or less is considered 6 characters, each universal character name specifying a short identifier of 010000 or more is considered 10 characters, and each extended source character is considered the same number of characters as the corresponding universal character name, if any)¹⁴⁾
- 4095 external identifiers in one translation unit
- 511 identifiers with block scope declared in one block
- 4095 macro identifiers simultaneously defined in one preprocessing translation unit
- 127 parameters in one function definition
- 127 arguments in one function call
- 127 parameters in one macro definition
- 127 arguments in one macro invocation
- 4095 characters in a logical source line
- 4095 characters in a string literal (after concatenation)
- 32767 bytes in an object (in a hosted environment only)
- 15 nesting levels for **#included** files
- 1023 **case** labels for a **switch** statement (excluding those for any nested **switch** statements)
- 1023 members in a single structure or union
- 1023 enumeration constants in a single enumeration
- 63 levels of nested structure or union definitions in a single member declaration list

5.3.5.3 Numerical limits

5.3.5.3.1 General

An implementation is required to document all the limits specified in this subclause, which are specified in the headers `<limits.h>` and `<float.h>`. Additional limits are specified in `<stdint.h>`.

Forward references: integer types `<stdint.h>` (7.22).

5.3.5.3.2 Characteristics of integer types `<limits.h>`

The values given subsequently shall be replaced by constant expressions suitable for use in conditional expression inclusion preprocessing directives. Their implementation-defined values shall be equal or greater to those shown.

- width for an object of type **bool**¹⁵⁾

BOOL_WIDTH	1
------------	---

- number of bits for smallest object that is not a bit-field (byte)

¹⁴⁾See “future language directions” (6.11.3).

¹⁵⁾This value is exact.

CHAR_BIT

8

The macros **CHAR_WIDTH**, **SCHAR_WIDTH**, and **UCHAR_WIDTH** that represent the width of the types **char**, **signed char** and **unsigned char** shall expand to the same value as **CHAR_BIT**.

- width for an object of type **unsigned short int**

USHRT_WIDTH

16

The macro **SHRT_WIDTH** represents the width of the type **short int** and shall expand to the same value as **USHRT_WIDTH**.

- width for an object of type **unsigned int**

UINT_WIDTH

16

The macro **INT_WIDTH** represents the width of the type **int** and shall expand to the same value as **UINT_WIDTH**.

- width for an object of type **unsigned long int**

ULONG_WIDTH

32

The macro **LONG_WIDTH** represents the width of the type **long int** and shall expand to the same value as **ULONG_WIDTH**.

- width for an object of type **unsigned long long int**

ULLONG_WIDTH

64

The macro **LLONG_WIDTH** represents the width of the type **long long int** and shall expand to the same value as **ULLONG_WIDTH**.

- maximum width of a bit-precise integer type

BITINT_MAXWIDTH

/* see the following */

The macro **BITINT_MAXWIDTH** represents the maximum width N supported by the declaration of a bit-precise integer (6.2.5) in the type specifier **_BitInt(N)**. The value **BITINT_MAXWIDTH** shall expand to a value that is greater than or equal to the value of **ULLONG_WIDTH**.

- maximum number of bytes in a multibyte character, for any supported locale

MB_LEN_MAX

1

For all unsigned integer types for which `<limits.h>` or `<stdint.h>` define a macro with suffix **_WIDTH** holding its width N , there is a macro with suffix **_MAX** holding the maximal value $2^N - 1$ that is representable by the type and that has the same type as would an expression that is an object of the corresponding type converted according to the integer promotions. If the value is in the range of the type **uintmax_t** (7.22.2.6) the macro is suitable for use in conditional expression inclusion preprocessing directives.

For all signed integer types for which `<limits.h>` or `<stdint.h>` define a macro with suffix **_WIDTH** holding its width N , there are macros with suffix **_MIN** and **_MAX** holding the minimal and maximal values -2^{N-1} and $2^{N-1} - 1$ that are representable by the type and that have the same type as would an expression that is an object of the corresponding type converted according to the integer promotions. If the values are in the range of the type **intmax_t** (7.22.2.6) the macros are suitable for use in conditional expression inclusion preprocessing directives.

If an object of type **char** can hold negative values, the value of **CHAR_MIN** shall be the same as that of **SCHAR_MIN** and the value of **CHAR_MAX** shall be the same as that of **SCHAR_MAX**. Otherwise, the value of **CHAR_MIN** shall be 0 and the value of **CHAR_MAX** shall be the same as that of **UCHAR_MAX** (see 6.2.5.).

Forward references: representations of types (6.2.6), conditional inclusion (6.10.2), integer types `<stdint.h>` (7.22).

5.3.5.3.3 Characteristics of floating types `<float.h>`

The characteristics of floating types are defined in terms of a model that describes a representation of floating-point numbers and allows other values. The characteristics provide information about an implementation's floating-point arithmetic.¹⁶⁾ An implementation that defines `__STDC_IEC_60559_BFP__` or `__STDC_IEC_559__` shall implement floating types and arithmetic conforming to ISO/IEC 60559 as specified in Annex F of this document. An implementation that defines `__STDC_IEC_60559_COMPLEX__` or `__STDC_IEC_559_COMPLEX__` shall implement complex types and arithmetic conforming to ISO/IEC 60559 as specified in Annex G of this document.

The following parameters are used to define the model for each floating type:

- s sign (± 1)
- b base or radix of exponent representation (an integer > 1)
- e exponent (an integer between a minimum $e_{\min}$ and a maximum $e_{\max}$)
- p precision (the number of base- b digits in the significand)
- f_k nonnegative integers less than b (the significand digits)

For each floating type, the parameters b , p , $e_{\min}$, and $e_{\max}$ are fixed constants.

For each floating type, a *floating-point number* (x) is defined by the following model:

$$x = sb^e \sum_{k=1}^p f_k b^{-k}, \quad e_{\min} \leq e \leq e_{\max}$$

Model floating-point numbers x with $f_1 > 0$ are called *normalized floating-point numbers*.

Model floating-point numbers $x \neq 0$ with $f_1 = 0$ and $e = e_{\min}$ are called *subnormal floating-point numbers*.

Model floating-point numbers $x \neq 0$ with $f_1 = 0$ and $e > e_{\min}$ are called *unnormalized floating-point numbers*.

Model floating-point numbers x with all $f_k = 0$ are zeros.

Floating types shall be able to represent signed zeros or an unsigned zero and all normalized floating-point numbers. In addition, floating types may be able to contain other kinds of floating-point numbers, such as subnormal floating-point numbers and unnormalized floating-point numbers, and values that are not floating-point numbers, such as NaNs and (signed and unsigned) infinities.

NOTE 1 Some implementations have types that include finite numbers with range and/or precision that are not covered by the model.

A NaN is a value signifying Not-a-Number. A *quiet NaN* propagates through almost every arithmetic operation without raising a floating-point exception; a *signaling NaN* generally raises a floating-point exception when occurring as an arithmetic operand.

NOTE 2 ISO/IEC 60559 specifies quiet and signaling NaNs. For implementations that do not support ISO/IEC 60559, the terms quiet NaN and signaling NaN are intended to apply to values with similar behavior.

Wherever values are unsigned, any requirement in this document to get the sign shall produce an unspecified sign, and any requirement to set the sign shall be ignored, unless otherwise specified.

NOTE 3 Bit representations of floating-point values can include a sign bit, even if the values can be regarded as unsigned; ISO/IEC 60559 NaNs are such values.

Whether and in what cases subnormal numbers are treated as zeros is implementation-defined. Subnormal numbers that in some cases are treated by arithmetic operations as zeros are properly

¹⁶⁾The floating-point model is intended to clarify the description of each floating-point characteristic and does not require the floating-point arithmetic of the implementation to be identical.

classified as subnormal. However, object representations that could represent subnormal numbers but that are always treated by arithmetic operations as zeros are non-canonical zeros, and the values are properly classified as zero, not subnormal. ISO/IEC 60559 arithmetic (with default exception handling) always treats subnormal numbers as nonzero.

A value is negative if and only if it compares less than 0. Thus, negative zeros and NaNs are not negative values.

An implementation can prefer particular representations of values that have multiple representations in a floating type, 6.2.6.1 notwithstanding. The preferred representations of a floating type, including unique representations of values in the type, are called *canonical*. A floating type can also contain *non-canonical* representations, for example, redundant representations of some or all its values, or representations that are extraneous to the floating-point model. Typically, floating-point operations deliver results with canonical representations. ISO/IEC 60559 operations deliver results with canonical representations, unless specified otherwise.

NOTE 4 The library operations **iscanonical** and **canonicalize** distinguish canonical (preferred) representations, but this distinction alone does not imply that canonical and non-canonical representations are of different values.

NOTE 5 Some of the values in the ISO/IEC 60559 decimal formats have non-canonical representations (as well as a canonical representation).

The minimum range of representable values for a floating type is the most negative finite floating-point number representable in that type through the most positive finite floating-point number representable in that type. In addition, if negative infinity is representable in a type, the range of that type is extended to all negative real numbers; likewise, if positive infinity is representable in a type, the range of that type is extended to all positive real numbers.

The accuracy of the floating-point operations (+, -, *, /) and of most of the library functions in `<math.h>` and `<complex.h>` that return floating-point results is implementation-defined, as is the accuracy of the conversion between floating-point internal representations and string representations performed by the library functions in `<stdio.h>`, `<stdlib.h>`, and `<wchar.h>`. The implementation may state that the accuracy is unknown. Decimal floating-point operations have stricter requirements.

All integer values in the `<float.h>` header, except **FLT_ROUNDS**, shall be constant expressions suitable for use in conditional expression inclusion preprocessing directives; all floating values shall be arithmetic constant expressions. All except **CR_DECIMAL_DIG** (F.5), **DECIMAL_DIG**, **DEC_EVAL_METHOD**, **FLT_EVAL_METHOD**, **FLT_RADIX**, and **FLT_ROUNDS** have separate names for all floating types. The floating-point model representation is provided for all values except **DEC_EVAL_METHOD**, **FLT_EVAL_METHOD** and **FLT_ROUNDS**.

The remainder of this subclause specifies characteristics of standard floating types.

The rounding mode for floating-point addition for standard floating types is characterized by the implementation-defined value of **FLT_ROUNDS**. Evaluation of **FLT_ROUNDS** correctly reflects any execution-time change of rounding mode through the function **fesetround** in `<fenv.h>`.

- −1 indeterminable
- 0 toward zero
- 1 to nearest, ties to even
- 2 toward positive infinity
- 3 toward negative infinity
- 4 to nearest, ties away from zero

All other values for **FLT_ROUNDS** characterize implementation-defined rounding behavior.

Whether a type has the same base (*b*), precision (*p*), and exponent range ($e_{\min} \leq e \leq e_{\max}$) as an ISO/IEC 60559 format is characterized by the implementation-defined values of **FLT_IS_IEC_60559**, **DBL_IS_IEC_60559** and **LDBL_IS_IEC_60559** (this does not imply conformance to Annex F):

- 0 type does not have the precision and exponent range of an ISO/IEC 60559 format
- 1 type has the precision and exponent range of an ISO/IEC 60559 format

NOTE 6 Outside of the normalized floating-point numbers, the representability of values (e.g. negative zero) of the ISO/IEC 60559 format is not implied.

The values of floating type yielded by operators subject to the usual arithmetic conversions, including the values yielded by the implicit conversion of operands, and the values of floating constants are evaluated to a format whose range and precision can be greater than required by the type. Such a format is called an *evaluation format*. In all cases, assignment and cast operators yield values in the format of the type. The extent to which evaluation formats are used is characterized by the value of **FLT_EVAL_METHOD**:¹⁷⁾

- −1 indeterminate;
- 0 evaluate all operations and constants just to the range and precision of the type;
- 1 evaluate operations and constants of type **float** and **double** to the range and precision of the **double** type, evaluate **long double** operations and constants to the range and precision of the **long double** type;
- 2 evaluate all operations and constants to the range and precision of the **long double** type.

All other negative values for **FLT_EVAL_METHOD** characterize implementation-defined behavior. The value of **FLT_EVAL_METHOD** does not characterize values returned by function calls (see 6.8.7.5, F.6).

The presence or absence of subnormal numbers is characterized by the implementation-defined values of **FLT_HAS_SUBNORM**, **DBL_HAS_SUBNORM**, and **LDBL_HAS_SUBNORM**:

- −1 indeterminate
- 0 absent (type does not support subnormal numbers)
- 1 present (type does support subnormal numbers)

The use of **FLT_HAS_SUBNORM**, **DBL_HAS_SUBNORM**, and **LDBL_HAS_SUBNORM** macros is an obsolescent feature.

Each of the signaling NaN macros

FLT_SNAN DBL_SNAN LDBL_SNAN
--

is defined if and only if the respective type contains signaling NaNs. They expand to a constant expression of the respective type representing a signaling NaN. If an optional unary + or - operator followed by a signaling NaN macro is used as an initializer that is evaluated at translation time, the object is initialized with a signaling NaN value.

The macro

INFINITY

¹⁷⁾The evaluation method determines evaluation formats of expressions involving all floating types, not just real types. For example, if **FLT_EVAL_METHOD** is 1, then the product of two **float _Complex** operands is represented in the **double _Complex** format, and its parts are evaluated to **double**.

is defined if and only if the implementation supports an infinity for the type **float**. It expands to a constant expression of type **float** representing positive or unsigned infinity.

The macro

NAN

is defined if and only if the implementation supports quiet NaNs for the **float** type. It expands to a constant expression of type **float** representing a quiet NaN.

The values given in the following list shall be replaced by constant expressions with implementation-defined values that are greater or equal in magnitude (absolute value) to those shown, with the same sign:

- radix of exponent representation, b

FLT_RADIX	2
------------------	---

- number of base-**FLT_RADIX** digits in the floating-point significand, p

FLT_MANT_DIG
DBL_MANT_DIG
LDBL_MANT_DIG

- number of decimal digits, n , such that any floating-point number with p radix b digits can be rounded to a floating-point number with n decimal digits and back again without change to the value,

$$\begin{cases} p \log_{10} b & \text{if } b \text{ is a power of } 10 \\ \lceil 1 + p \log_{10} b \rceil & \text{otherwise} \end{cases}$$

FLT_DECIMAL_DIG	6
DBL_DECIMAL_DIG	10
LDBL_DECIMAL_DIG	10

- number of decimal digits, n , such that any floating-point number in the widest of the supported floating types and the supported ISO/IEC 60559 encodings with $p_{\max}$ radix b digits can be rounded to a floating-point number with n decimal digits and back again without change to the value,

$$\begin{cases} p_{\max} \log_{10} b & \text{if } b \text{ is a power of } 10 \\ \lceil 1 + p_{\max} \log_{10} b \rceil & \text{otherwise} \end{cases}$$

DECIMAL_DIG	10
--------------------	----

This is an obsolescent feature, see 7.33.9.

- number of decimal digits, q , such that any floating-point number with q decimal digits can be rounded into a floating-point number with p radix b digits and back again without change to the q decimal digits,

$$\begin{cases} p \log_{10} b & \text{if } b \text{ is a power of } 10 \\ \lfloor (p - 1) \log_{10} b \rfloor & \text{otherwise} \end{cases}$$

FLT_DIG	6
DBL_DIG	10
LDBL_DIG	10

- minimum negative integer such that **FLT_RADIX** raised to one less than that power is a normalized floating-point number, $e_{\min}$

FLT_MIN_EXP
DBL_MIN_EXP
LDBL_MIN_EXP

- minimum negative integer such that 10 raised to that power is in the range of normalized floating-point numbers, $\lceil \log_{10} b^{e_{\min}} - 1 \rceil$

FLT_MIN_10_EXP	-37
DBL_MIN_10_EXP	-37
LDBL_MIN_10_EXP	-37

- maximum integer such that **FLT_RADIX** raised to one less than that power is a representable finite floating-point number; if that representable finite floating-point number is normalized, the value of the macro is $e_{\max}$

FLT_MAX_EXP
DBL_MAX_EXP
LDBL_MAX_EXP

- maximum integer such that 10 raised to that power is in the range of representable finite floating-point numbers, $\lfloor \log_{10} ((1 - b^{-p}) b^{e_{\max}}) \rfloor$

FLT_MAX_10_EXP	+37
DBL_MAX_10_EXP	+37
LDBL_MAX_10_EXP	+37

The values given in the following list shall be replaced by constant expressions with implementation-defined values that are greater than or equal to those shown:

- maximum representable finite floating-point number; if that number is normalized, its value is $(1 - b^{-p}) b^{e_{\max}}$

FLT_MAX	1E+37
DBL_MAX	1E+37
LDBL_MAX	1E+37

- maximum normalized floating-point number, $(1 - b^{-p}) b^{e_{\max}}$

FLT_NORM_MAX	1E+37
DBL_NORM_MAX	1E+37
LDBL_NORM_MAX	1E+37

The values given in the following list shall be replaced by constant expressions with implementation-defined (positive) values that are less than or equal to those shown:

- the difference between 1 and the least normalized value greater than 1 that is representable in the given floating type, b^{1-p}

FLT_EPSILON	1E-5
DBL_EPSILON	1E-9
LDBL_EPSILON	1E-9

- minimum normalized positive floating-point number, $b^{e_{\min}-1}$

FLT_MIN	1E-37
DBL_MIN	1E-37
LDBL_MIN	1E-37

- minimum positive floating-point number

FLT_TRUE_MIN	1E-37
DBL_TRUE_MIN	1E-37
LDBL_TRUE_MIN	1E-37

Recommended practice

Conversion between real floating type and decimal character sequence with at most **T_DECIMAL_DIG** digits should be correctly rounded, where *T* is the macro prefix for the type. This assures conversion from real floating type to decimal character sequence with **T_DECIMAL_DIG** digits and back, using to-nearest rounding, is the identity function.

EXAMPLE 1 The following describes an artificial floating-point representation that meets the minimum requirements of this document, and the appropriate values in a `<float.h>` header for type **float**:

$$x = s16^e \sum_{k=1}^6 f_k 16^{-k}, \quad -31 \leq e \leq +32$$

FLT_RADIX	16
FLT_MANT_DIG	6
FLT_EPSILON	9.53674316E-07F
FLT_DECIMAL_DIG	9
FLT_DIG	6
FLT_MIN_EXP	-31
FLT_MIN	2.93873588E-39F
FLT_MIN_10_EXP	-38
FLT_MAX_EXP	+32
FLT_MAX	3.40282347E+38F
FLT_MAX_10_EXP	+38

EXAMPLE 2 The following describes floating-point representations that also meet the requirements for binary32 and binary64 numbers in ISO/IEC 60559,¹⁸⁾ and the appropriate values in a `<float.h>` header for types **float** and **double**. The decimal floating constants can possibly not give correct values (and hence are not appropriate values in a `<float.h>` header) if **FLT_EVAL_METHOD** is not 0 or if a translation-time rounding mode other than the ISO/IEC 60559 default is supported (either as the default or as a constant rounding mode set by an **FENV_ROUND** pragma). The hexadecimal floating constants are correct in all such cases because their values are exactly representable in the type.

$$x_f = s2^e \sum_{k=1}^{24} f_k 2^{-k}, \quad -125 \leq e \leq +128$$

$$x_d = s2^e \sum_{k=1}^{53} f_k 2^{-k}, \quad -1021 \leq e \leq +1024$$

FLT_IS_IEC_60559	1
FLT_RADIX	2
FLT_MANT_DIG	24
FLT_EPSILON	1.19209290E-07F // decimal constant
FLT_EPSILON	0X1P-23F // hex constant
FLT_DECIMAL_DIG	9
FLT_DIG	6

¹⁸⁾The floating-point model in ISO/IEC 60559 sums powers of *b* from zero, so the values of the exponent limits are one less than shown here.

FLT_MIN_EXP	-125	
FLT_MIN	1.17549435E-38F	// decimal constant
FLT_MIN	0X1P-126F	// hex constant
FLT_TRUE_MIN	1.40129846E-45F	// decimal constant
FLT_TRUE_MIN	0X1P-149F	// hex constant
FLT_HAS_SUBNORM	1	
FLT_MIN_10_EXP	-37	
FLT_MAX_EXP	+128	
FLT_MAX	3.40282347E+38F	// decimal constant
FLT_MAX	0X1.fffffeP127F	// hex constant
FLT_MAX_10_EXP	+38	
DBL_MANT_DIG	53	
DBL_IS_IEC_60559	1	
DBL_EPSILON	2.2204460492503131E-16	// decimal constant
DBL_EPSILON	0X1P-52	// hex constant
DBL_DECIMAL_DIG	17	
DBL_DIG	15	
DBL_MIN_EXP	-1021	
DBL_MIN	2.2250738585072014E-308	// decimal constant
DBL_MIN	0X1P-1022	// hex constant
DBL_TRUE_MIN	4.9406564584124654E-324	// decimal constant
DBL_TRUE_MIN	0X1P-1074	// hex constant
DBL_HAS_SUBNORM	1	
DBL_MIN_10_EXP	-307	
DBL_MAX_EXP	+1024	
DBL_MAX	1.7976931348623157E+308	// decimal constant
DBL_MAX	0X1.ffffffffffffFP1023	// hex constant
DBL_MAX_10_EXP	+308	

Forward references: conditional inclusion (6.10.2), predefined macro names (6.10.10), complex arithmetic <complex.h> (7.3), extended multibyte and wide character utilities <wchar.h> (7.31), floating-point environment <fenv.h> (7.6), general utilities <stdlib.h> (7.24), input/output <stdio.h> (7.23), mathematics <math.h> (7.12), ISO/IEC 60559 floating-point arithmetic (Annex F), ISO/IEC 60559-compatible complex arithmetic (Annex G).

5.3.5.3.4 Characteristics of decimal floating types in <float.h>

This subclause specifies macros in <float.h> that provide characteristics of decimal floating types (an optional feature) in terms of the model presented in 5.3.5.3.3. An implementation shall provide these macros if and only if it defines `__STDC_IEC_60559_DFP__`. The prefixes `DEC32_`, `DEC64_`, and `DEC128_` denote the types `_Decimal32`, `_Decimal64`, and `_Decimal128` respectively.

`DEC_EVAL_METHOD` is the decimal floating-point analog of `FLT_EVAL_METHOD` (5.3.5.3.3). Its implementation-defined value characterizes the use of evaluation formats for decimal floating types:

- 1indeterminable;
- 0evaluate all operations and constants just to the range and precision of the type;
- 1evaluate operations and constants of type `_Decimal32` and `_Decimal64` to the range and precision of the `_Decimal64` type, evaluate `_Decimal128` operations and constants to the range and precision of the `_Decimal128` type;
- 2evaluate all operations and constants to the range and precision of the `_Decimal128` type.

Each of the decimal signaling NaN macros

DEC32_SNAN
DEC64_SNAN
DEC128_SNAN

The macro

DEC_INFINITY

The macro

DEC NAN

The integer values given in the following lists shall be replaced by constant expressions suitable for use in conditional expression inclusion preprocessing directives:

- For the standard floating types, this value is implementation-defined and is specified by the macro **FLT_RADIX**. For the decimal floating types there is no corresponding macro, since the value 10 is an inherent property of the types. Wherever **FLT_RADIX** appears in a description of a function that has versions that operate on decimal floating types, it is noted that for the decimal floating-point versions the value used is implicitly 10, rather than **FLT_RADIX**.

- | | |
|-----------------|----|
| DEC32_MANT_DIG | 7 |
| DEC64_MANT_DIG | 16 |
| DEC128_MANT_DIG | 34 |

- | | |
|----------------|-------|
| DEC32_MIN_EXP | -94 |
| DEC64_MIN_EXP | -382 |
| DEC128_MIN_EXP | -6142 |

- | | |
|----------------|------|
| DEC32_MAX_EXP | 97 |
| DEC64_MAX_EXP | 385 |
| DEC128_MAX_EXP | 6145 |

- [illegible]

- | | |
|----------------|---------|
| DEC32_EPSILON | 1E-6DF |
| DEC64_EPSILON | 1E-15DD |
| DEC128_EPSILON | 1E-33DL |

- minimum normalized positive decimal floating-point number

DEC32_MIN	1E-95DF
DEC64_MIN	1E-383DD
DEC128_MIN	1E-6143DL

- minimum positive subnormal decimal floating-point number

[illegible]

For decimal floating-point arithmetic, it is often convenient to consider an alternate equivalent model where the significand is represented with integer rather than fraction digits. With s , b , e , p , and f_k as defined in 5.3.5.3.3, a floating-point number x is defined by the model:

$$x = s \cdot b^{(e-p)} \sum_{k=1}^p f_k \cdot b^{(p-k)}$$

With b fixed to 10, a decimal floating-point number x is thus:

$$x = s \cdot 10^{(e-p)} \sum_{k=1}^p f_k \cdot 10^{(p-k)}$$

The *quantum exponent* is $q = e - p$ and the *coefficient* is $c = f_1 f_2 \cdots f_p$, which is an integer between 0 and $10^p - 1$, inclusive. Thus, $x = s \cdot c \cdot 10^q$ is represented by the triple of integers (s, c, q) . The *quantum* of x is 10^q , which is the value of a unit in the last place of the coefficient. Table 5.1 shows the range of quantum exponents.

Table 5.1 — Quantum exponent ranges

Type	_Decimal32	_Decimal64	_Decimal128
Maximum Quantum Exponent ($q_{\max}$)	90	369	6111
Minimum Quantum Exponent ($q_{\min}$)	-101	-398	-6176

For binary floating-point arithmetic following ISO/IEC 60559, representations in the model described in 5.3.5.3.3 that have the same numerical value are indistinguishable in the arithmetic. However, for decimal floating-point arithmetic, representations that have the same numerical value but different quantum exponents, e.g. $(+1, 10, -1)$ representing 1.0 and $(+1, 100, -2)$ representing 1.00, are distinguishable. To facilitate exact fixed-point calculation, operation results that are of decimal floating type have a *preferred quantum exponent*, as specified in ISO/IEC 60559, which is determined by the quantum exponents of the operands if they have decimal floating types (or by specific rules for conversions from other types). Table 5.2 gives rules for determining preferred quantum exponents for results of ISO/IEC 60559 operations, and for other operations specified in this document. When exact, these operations produce a result with their preferred quantum exponent, or as close to it as possible within the limitations of the type. When inexact, these operations produce a result with the least possible quantum exponent. For example, the preferred quantum exponent for addition is the minimum of the quantum exponents of the operands. Hence $(+1, 123, -2) + (+1, 4000, -3) = (+1, 5230, -3)$ or $1.23 + 4.000 = 5.230$.

Table 5.2 shows, for each operation delivering a result in decimal floating-point format, how the preferred quantum exponents of the operands, $Q(\mathbf{x})$, $Q(\mathbf{y})$, etc., determine the preferred quantum exponent of the operation result, provided the table formula is defined for the arguments. For the cases where the formula is undefined and the function result is $\pm\infty$, the preferred quantum exponent is immaterial because the quantum exponent of $\pm\infty$ is defined to be infinity. For the

other cases where the formula is undefined and the function result is finite, the preferred quantum exponent is unspecified.

NOTE Although unspecified in ISO/IEC 60559, a preferred quantum exponent of 0 for the cases where the formula is undefined and the function result is finite would be a reasonable implementation choice.

Table 5.2 — Preferred quantum exponents

Operation	Preferred quantum exponent of result
roundeven, round, trunc, ceil, floor, rint, nearbyint	$\max(Q(\mathbf{x}), 0)$
nextup, nextdown, nextafter, nexttoward	least possible
remainder	$\min(Q(\mathbf{x}), Q(\mathbf{y}))$
fmin, fmax, fminimum, fmaximum, fminimum_mag, fmaximum_mag, fminimum_num, fmaximum_num, fminimum_mag_num, fmaximum_mag_num	$Q(\mathbf{x})$ if $\mathbf{x}$ gives the result, $Q(\mathbf{y})$ if $\mathbf{y}$ gives the result
scalbn, scalbln	$Q(\mathbf{x}) + \mathbf{n}$
ldexp	$Q(\mathbf{x}) + \mathbf{p}$
logb	0
postfix ++ operator, postfix -- operator, prefix ++ operator, prefix -- operator	$\min(Q(\mathbf{x}), 0)$
+, d32add, d64add	$\min(Q(\mathbf{x}), Q(\mathbf{y}))$
-, d32sub, d64sub	$\min(Q(\mathbf{x}), Q(\mathbf{y}))$
*, d32mul, d64mul	$Q(\mathbf{x}) + Q(\mathbf{y})$
/, d32div, d64div	$Q(\mathbf{x}) - Q(\mathbf{y})$
sqrt, d32sqrt, d64sqrt	$\lfloor Q(\mathbf{x})/2 \rfloor$
fma, d32fma, d64fma	$\min(Q(\mathbf{x}) + Q(\mathbf{y}), Q(\mathbf{z}))$
conversion from integer type	0
exact conversion from non-decimal floating type	0
inexact conversion from non-decimal floating type	least possible
conversion between decimal floating types	$Q(\mathbf{x})$
cx returned by canonicalize	$Q(\mathbf{x})$
strtto, wcsto, scanf , floating constants of decimal floating type	see 7.24.2.7
-(x), +(x)	$Q(\mathbf{x})$
fabs	$Q(\mathbf{x})$
copysign	$Q(\mathbf{x})$
quantize	$Q(\mathbf{y})$
quantum	$Q(\mathbf{x})$

*encptr returned by encodedec , encodebin	$Q(*xptr)$
*xptr returned by decodedec , decodebin	$Q(*encptr)$
fmod	$\min(Q(x), Q(y))$
fdim	$\min((Q(x), Q(y)) \text{ if } x > y, 0 \text{ if } x \leq y)$
cbrt	$\lfloor Q(x)/3 \rfloor$
hypot	$\min(Q(x), Q(y))$
pow	$\lfloor y \times Q(x) \rfloor$
modf	$Q(value)$
*iptr returned by modf	$\max(Q(value), 0)$
frexp	$Q(value)$ if $value = 0$, $-(\text{length of coefficient of value})$ otherwise
*res returned by setpayload , setpayloadsig	0 if pl does not represent a valid payload, not applicable otherwise (NaN returned)
getpayload	0 if *x is a NaN, unspecified otherwise
compoundn	$\lfloor n \times \min(0, Q(x)) \rfloor$
pown	$\lfloor n \times Q(x) \rfloor$
powr	$\lfloor y \times Q(x) \rfloor$
rootn	$\lfloor Q(x)/n \rfloor$
rsqrt	$-\lfloor Q(x)/2 \rfloor$
transcendental functions	0

A function family listed in Table 5.2 indicates the functions for all decimal floating types, where the function family is represented by the name of the functions without a suffix. For example, **ceil** indicates the functions **ceild32**, **ceild64**, and **ceild128**.

Forward references: extended multibyte and wide character utilities <wchar.h> (7.31), floating-point environment <fenv.h> (7.6), general utilities <stdlib.h> (7.24), input/output <stdio.h> (7.23), mathematics <math.h> (7.12), type-generic mathematics <tgmath.h> (7.27), ISO/IEC 60559 floating-point arithmetic (Annex F).

6. Language

6.1 Notation

In the syntax notation used in this clause, syntactic categories (nonterminals) are indicated by *italic type*, and literal words and character set members (terminals) by **bold type**. A colon (:) following a nonterminal introduces its definition. Alternative definitions are listed on separate lines, except when prefaced by the words “one of”. An optional symbol is indicated by the subscript “opt”, so that

$$\{ \textit{expression}_{\text{opt}} \}$$

indicates an optional expression enclosed in braces.

When syntactic categories are referred to in the main text, they are not italicized and words are separated by spaces instead of hyphens.

A summary of the language syntax is given in Annex A.

6.2 Concepts

6.2.1 Scopes of identifiers, type names, and compound literals

An identifier can denote:

- a standard attribute, an attribute prefix, or an attribute name;
- an object;
- a function;
- a tag or a member of a structure, union, or enumeration;
- a typedef name;
- a label name;
- a macro name;
- or, a macro parameter.

The same identifier can denote different entities at different points in the program. A member of an enumeration is called an *enumeration constant*. Macro names and macro parameters are not considered further here, because prior to the semantic phase of program translation any occurrences of macro names in the source file are replaced by the preprocessing token sequences that constitute their macro definitions.

For each different entity that an identifier designates, the identifier is *visible* (i.e. can be used) only within a region of program text called its *scope*. Different entities designated by the same identifier either have different scopes or are in different name spaces. There are four kinds of scopes: function, file, block, and function prototype. (A *function prototype* is a declaration of a function.)

A label name is the only kind of identifier that has *function scope*. It can be used (in a **goto** statement) anywhere in the function in which it appears, and is declared implicitly by its syntactic appearance (followed by a : and a statement).

Every other identifier has scope determined by the placement of its declaration (in a declarator or type specifier). If the declarator or type specifier that declares the identifier appears outside of any block or list of parameters, the identifier has *file scope*, which terminates at the end of the translation unit. If the declarator or type specifier that declares the identifier appears inside a block or within the list of parameter declarations in a function definition, the identifier has *block scope*, which terminates at the end of the associated block. If the declarator or type specifier that declares

the identifier appears within the list of parameter declarations in a function prototype (not part of a function definition), the identifier has *function prototype scope*, which terminates at the end of the function declarator. If an identifier designates two different entities in the same name space, the scopes can overlap. If so, the scope of one entity (the *inner scope*) will end strictly before the scope of the other entity (the *outer scope*). Within the inner scope, the identifier designates the entity declared in the inner scope; the entity declared in the outer scope is *hidden* (and not visible) within the inner scope.

Unless explicitly stated otherwise, where this document uses the term “identifier” to refer to an entity (as opposed to the syntactic construct), it refers to the entity in the relevant name space whose declaration is visible at the point the identifier occurs.

Two identifiers have the *same scope* if and only if their scopes terminate at the same point.

Structure, union, and enumeration tags have scope that begins just after the appearance of the tag in a type specifier that declares the tag. Each enumeration constant has scope that begins just after the appearance of its defining enumerator in an enumerator list. An ordinary identifier that has an underspecified definition has scope that starts when the definition is completed; if the same ordinary identifier declares another entity with a scope that encloses the current block, that declaration is hidden as soon as the inner declarator is completed.¹⁹⁾ Any other identifier has scope that begins just after the completion of its declarator.

As a special case, a type name (which is not a declaration of an identifier) is considered to have a scope that begins just after the place within the type name where the omitted identifier would appear were it not omitted. A compound literal (which is an expression that provides access to an anonymous object) is associated with the scope of the type name used in its definition; that scope is either file scope, function prototype scope, or block scope.

Forward references: declarations (6.7), function calls (6.5.3.3), function calls (6.5.3.6), function definitions (6.9.2), identifiers (6.4.3), macro replacement (6.10.5), name spaces of identifiers (6.2.3), source file inclusion (6.10.3), statements and blocks (6.8).

6.2.2 Linkages of identifiers

An identifier declared in different scopes or in the same scope more than once can be made to refer to the same object or function by a process called *linkage*.²⁰⁾ There are three kinds of linkage: external, internal, and none.

In the set of translation units and libraries that constitutes an entire program, each declaration of a particular identifier with *external linkage* denotes the same object or function. Within one translation unit, each declaration of an identifier with *internal linkage* denotes the same object or function. Each declaration of an identifier with *no linkage* denotes a unique entity.

If the declaration of a file scope identifier for:

- an object contains any of the storage-class specifiers **static** or **constexpr**;
- or, a function contains the storage-class specifier **static**,

then the identifier has internal linkage.²¹⁾

For an identifier declared with the storage-class specifier **extern** in a scope in which a prior declaration of that identifier is visible,²²⁾ if the prior declaration specifies internal or external linkage, the linkage of the identifier at the later declaration is the same as the linkage specified at the prior declaration. If no prior declaration is visible, or if the prior declaration specifies no linkage, then the identifier has external linkage.

If the declaration of an identifier for a function has no storage-class specifier, its linkage is determined exactly as if it were declared with the storage-class specifier **extern**. If the declaration of an identifier

¹⁹⁾That means, that the outer declaration is not visible for the initializer.

²⁰⁾There is no linkage between different identifiers.

²¹⁾A function declaration can contain the storage-class specifier **static** only if it is at file scope; see 6.7.2.

²²⁾As specified in 6.2.1, the later declaration can hide the prior declaration.

for an object has file scope and does not contain the storage-class specifier **static** or **constexpr**, its linkage is external.

The following identifiers have no linkage: an identifier declared to be anything other than an object or a function; an identifier declared to be a function parameter; a block scope identifier for an object declared without the storage-class specifier **extern**.

If, within a translation unit, the same identifier appears with both internal and external linkage, the behavior is undefined.

Forward references: declarations (6.7), expressions (6.5.1), external definitions (6.9), statements (6.8).

6.2.3 Name spaces of identifiers

If more than one declaration of a particular identifier is visible at any point in a translation unit, the syntactic context disambiguates uses that refer to different entities. Thus, there are separate *name spaces* for various categories of identifiers, as follows:

- *label names* (disambiguated by the syntax of the label declaration and use);
- the *tags* of structures, unions, and enumerations (disambiguated by following any²³⁾ of the keywords **struct**, **union**, or **enum**);
- the *members* of structures or unions; each structure or union has a separate name space for its members (disambiguated by the type of the expression used to access the member via the **.** or **->** operator);
- standard attributes and attribute prefixes (disambiguated by the syntax of the attribute specifier and name of the attribute token) (6.7.13);
- the trailing identifier in an attribute prefixed token; each attribute prefix has a separate name space for the implementation-defined attributes that it introduces (disambiguated by the attribute prefix and the trailing identifier token);
- all other identifiers, called *ordinary identifiers* (declared in ordinary declarators or as enumeration constants).

Forward references: enumeration specifiers (6.7.3.3), labeled statements (6.8.2), structure and union specifiers (6.7.3.2), structure and union members (6.5.3.4), tags (6.7.3.4), the **goto** statement (6.8.7.2).

6.2.4 Storage durations of objects

An object has a *storage duration* that determines its lifetime. There are four storage durations: static, thread, automatic, and allocated. Allocated storage is described in 7.24.4.

The *lifetime* of an object is the portion of program execution during which storage is guaranteed to be reserved for it. An object exists, has a constant address,²⁴⁾ and retains its last-stored value throughout its lifetime.²⁵⁾ If an object is referred to outside of its lifetime, the behavior is undefined. If a pointer value is used in an evaluation after the object the pointer points to (or just past) reaches the end of its lifetime, the behavior is undefined. The representation of a pointer object becomes indeterminate when the object the pointer points to (or just past) reaches the end of its lifetime.

An object whose identifier is declared without the storage-class specifier **thread_local**, and either with external or internal linkage or with the storage-class specifier **static**, has *static storage duration*. Its lifetime is the entire execution of the program and its stored value is initialized only once, prior to program startup.

An object whose identifier is declared with the storage-class specifier **thread_local** has *thread storage duration*. Its explicit or implicit initializer is evaluated prior to program execution, its lifetime

²³⁾There is only one name space for tags even though three are possible.

²⁴⁾The term “constant address” means that two pointers to the object constructed at possibly different times will compare equal. The address can be different during two different executions of the same program.

²⁵⁾In the case of a volatile object, the last store is not required to be explicit in the program.

is the entire execution of the thread for which it is created, and its stored value is initialized with the previously determined value when the thread is started. There is a distinct object per thread, and use of the declared name in an expression refers to the object associated with the thread evaluating the expression. The result of attempting to indirectly access an object with thread storage duration from a thread other than the one with which the object is associated is implementation-defined.

An object whose identifier is declared with no linkage and without the storage-class specifier **static** has *automatic storage duration*, as do some compound literals. The result of attempting to indirectly access an object with automatic storage duration from a thread other than the one with which the object is associated is implementation-defined.

For such an object that does not have a variable length array type, its lifetime extends from entry into the block with which it is associated until execution of that block ends in any way. (Entering an enclosed block or calling a function suspends, but does not end, execution of the current block.) If the block is entered recursively, a new instance of the object is created each time. The initial representation of the object is indeterminate. If an initialization is specified for the object and it is not specified with **constexpr**, it is performed each time the declaration or compound literal is reached in the execution of the block; if it is specified with **constexpr** the initializer is evaluated once at translation time and the new instance of the object is initialized to that fixed value each time the specification is reached; otherwise, the representation of the object becomes indeterminate each time the declaration is reached.

For such an object that does have a variable length array type, its lifetime extends from the declaration of the object until execution of the program leaves the scope of the declaration.²⁶⁾ If the scope is entered recursively, a new instance of the object is created each time. The initial representation of the object is indeterminate.

A non-lvalue expression with structure or union type, where the structure or union contains a member with array type (including, recursively, members of all contained structures and unions) refers to an object with automatic storage duration and *temporary lifetime*.²⁷⁾ Its lifetime begins when the expression is evaluated and its initial value is the value of the expression. Its lifetime ends when the evaluation of the containing full expression ends. Any attempt to modify an object with temporary lifetime results in undefined behavior. An object with temporary lifetime behaves as if it were declared with the type of its value for the purposes of effective type. Such an object may not have a unique address.

Forward references: array declarators (6.7.7.3), compound literals (6.5.3.6), declarators (6.7.7), function calls (6.5.3.3), initialization (6.7.11), statements (6.8), effective type (6.5.1).

6.2.5 Types

The meaning of a value stored in an object or returned by a function is determined by the *type* of the expression used to access it. (An identifier declared to be an object is the simplest such expression; the type is specified in the declaration of the identifier.) Types are partitioned into *object types* (types that describe objects) and *function types* (types that describe functions). At various points within a translation unit an object type can be *incomplete*²⁸⁾ (lacking sufficient information to determine the size of objects of that type) or *complete* (having sufficient information).²⁹⁾

An object declared as type **bool** is large enough to store the values **false** and **true**.

An object declared as type **char** is large enough to store any member of the basic execution character set. If a member of the basic execution character set is stored in a **char** object, its value is guaranteed to be nonnegative. If any other character is stored in a **char** object, the resulting value is implementation-defined but shall be within the range of values that can be represented in that type.

²⁶⁾ Leaving the innermost block containing the declaration, or jumping to a point in that block or an embedded block prior to the declaration, leaves the scope of the declaration.

²⁷⁾ The address of such an object is taken implicitly when an array member is accessed.

²⁸⁾ An incomplete type can only be used when the size of an object of that type is not needed. It is not needed, for example, when a typedef name is declared to be a specifier for a structure or union, or when a pointer to or a function returning a structure or union is being declared. The specification has to be complete before such a function is called or defined.

²⁹⁾ A type can be incomplete or complete throughout an entire translation unit, or it can change states at different points within a translation unit.

There are five *standard signed integer types*, designated as **signed char**, **short int**, **int**, **long int**, and **long long int**. (These and other types can be designated in several additional ways, as described in 6.7.3.)

A *bit-precise signed integer type* is designated as **_BitInt(*N*)** where *N* is an integer constant expression that specifies the number of bits that are used to represent the type, including the sign bit. Each value of *N* designates a distinct type.³⁰⁾

There may also be implementation-defined *extended signed integer types*.³¹⁾ The standard signed integer types, bit-precise signed integer types, and extended signed integer types are collectively called *signed integer types*.³²⁾

An object declared as type **signed char** occupies the same amount of storage as a “plain” **char** object. A “plain” **int** object has the natural size suggested by the architecture of the execution environment (large enough to contain any value in the range **INT_MIN** to **INT_MAX** as defined in the header `<limits.h>`).

For each of the signed integer types, there is a *corresponding* (but different) *unsigned integer type* (designated with the keyword **unsigned**) that uses the same amount of storage (including sign information) and has the same alignment requirements. The type **bool** and the unsigned integer types that correspond to the standard signed integer types are the *standard unsigned integer types*. The unsigned integer types that correspond to the extended signed integer types are the *extended unsigned integer types*. In addition to the unsigned integer types that correspond to the bit-precise signed integer types there is the type **unsigned _BitInt(1)**, which uses one bit to represent the type. Collectively, **unsigned _BitInt(1)** and the unsigned integer types that correspond to the bit-precise signed integer types are the *bit-precise unsigned integer types*. The standard unsigned integer types, bit-precise unsigned integer types, and extended unsigned integer types are collectively called *unsigned integer types*.³³⁾

The standard signed integer types and standard unsigned integer types are collectively called the *standard integer types*; the bit-precise signed integer types and bit-precise unsigned integer types are collectively called the *bit-precise integer types*; the extended signed integer types and extended unsigned integer types are collectively called the *extended integer types*.

For any two integer types with the same signedness and different integer conversion rank (see 6.3.2.1), the range of values of the type with smaller integer conversion rank is a subrange of the values of the other type.

The range of nonnegative values of a signed integer type is a subrange of the corresponding unsigned integer type, and the representation of the same value in each type is the same.³⁴⁾ The range of representable values for the unsigned type is 0 to $2^N - 1$ (inclusive). A computation involving unsigned operands can never produce an overflow, because arithmetic for the unsigned type is performed modulo 2^N .

There are three *standard floating types*, designated as **float**, **double**, and **long double**.³⁵⁾ The set of values of the type **float** is a subset of the set of values of the type **double**; the set of values of the type **double** is a subset of the set of values of the type **long double**.

There are three *decimal floating types*, designated as **_Decimal32**, **_Decimal64**, and **_Decimal128**. Respectively, they have the ISO/IEC 60559 formats: decimal32,³⁶⁾ decimal64, and decimal128. (Decimal floating types are a conditional feature that implementations may not support; see 6.10.10.4.)

³⁰⁾ Thus, **_BitInt(3)** is not the same type as **_BitInt(4)**.

³¹⁾ Implementation-defined keywords have the form of an identifier reserved for any use as described in 7.1.3.

³²⁾ Any statement in this document about signed integer types also applies to the bit-precise signed integer types and the extended signed integer types, unless otherwise noted.

³³⁾ Any statement in this document about unsigned integer types also applies to the bit-precise unsigned integer types and the extended unsigned integer types, unless otherwise specified.

³⁴⁾ The same representation and alignment requirements are meant to imply interchangeability as arguments to functions, return values from functions, and members of unions.

³⁵⁾ See “future language directions” (6.11.1).

³⁶⁾ ISO/IEC 60559 specifies decimal32 as a data-interchange format that does not require arithmetic support; however, **_Decimal32** is a fully supported arithmetic type.

The standard floating types and the decimal floating types are collectively called the *real floating types*.

There are three *complex types*, designated as **float _Complex**, **double _Complex**, and **long double _Complex**.³⁷⁾ (Complex types are a conditional feature that implementations may not support; see 6.10.10.4.) The real floating and complex types are collectively called the *floating types*.

For each floating type there is a *corresponding real type*, which is always a real floating type. For real floating types, it is the same type. For complex types, it is the type given by deleting the keyword **_Complex** from the type name.

Each complex type has the same representation and alignment requirements as an array type containing exactly two elements of the corresponding real type; the first element is equal to the real part, and the second element to the imaginary part, of the complex number.

The type **char**, the signed and unsigned integer types, and the floating types are collectively called the *basic types*. The basic types are complete object types. Even if the implementation defines two or more basic types to have the same representation, they are nevertheless distinct types.

NOTE An implementation can define new keywords that provide alternative ways to designate a basic (or any other) type; this does not violate the requirement that all basic types be different. Implementation-defined keywords have the form of an identifier reserved for any use as described in 7.1.3.

The three types **char**, **signed char**, and **unsigned char** are collectively called the *character types*. The implementation shall define **char** to have the same range, representation, and behavior as either **signed char** or **unsigned char**.³⁸⁾

An *enumeration* comprises a set of named integer constant values. Each distinct enumeration constitutes a different *enumerated type*.

The type **char**, the signed and unsigned integer types, and the enumerated types are collectively called *integer types*. The integer and real floating types are collectively called *real types*.

Integer and floating types are collectively called *arithmetic types*. Each arithmetic type belongs to one *type domain*: the *real type domain* comprises the real types, the *complex type domain* comprises the complex types.

The **void** type comprises an empty set of values; it is an incomplete object type that cannot be completed.

Any number of *derived types* can be constructed from the object and function types, as follows:

- An *array type* describes a contiguously allocated nonempty set of objects with a particular member object type, called the *element type*. The element type shall be complete whenever the array type is specified. Array types are characterized by their element type and by the number of elements in the array. An array type is said to be derived from its element type, and if its element type is *T*, the array type is sometimes called “array of *T*”. The construction of an array type from an element type is called “array type derivation”.
- A *structure type* describes a sequentially allocated nonempty set of member objects (and, in certain circumstances, an incomplete array), each of which has an optionally specified name and possibly distinct type.
- A *union type* describes an overlapping nonempty set of member objects, each of which has an optionally specified name and possibly distinct type.
- A *function type* describes a function with specified return type. A function type is characterized by its return type and the number and types of its parameters. A function type is said to be derived from its return type, and if its return type is *T*, the function type is sometimes called “function returning *T*”. The construction of a function type from a return type is called “function type derivation”.

³⁷⁾ A specification for imaginary types is in Annex G.

³⁸⁾ **CHAR_MIN**, defined in `<limits.h>`, will have one of the values 0 or **SCHAR_MIN**, and this can be used to distinguish the two options. Irrespective of the choice made, **char** is a separate type from the other two and is not compatible with either.

- A *pointer type* can be derived from a function type or an object type, called the *referenced type*. A pointer type describes an object whose value provides a reference to an entity of the referenced type. A pointer type derived from the referenced type *T* is sometimes called “pointer to *T*”. The construction of a pointer type from a referenced type is called “pointer type derivation”. A pointer type is a complete object type.
- An *atomic type* describes the type designated by the construct **_Atomic**(*type-name*). (Atomic types are a conditional feature that implementations may not support; see 6.10.10.4.)

These methods of constructing derived types can be applied recursively.

Arithmetic types, pointer types, and the **uintptr_t** type are collectively called *scalar types*. Array and structure types are collectively called *aggregate types*.³⁹⁾

An array type of unknown size is an incomplete type. It is completed, for an identifier of that type, by specifying the size in a later declaration (with internal or external linkage). A structure or union type of unknown content (as described in 6.7.3.4) is an incomplete type. It is completed, for all declarations of that type, by declaring the same structure or union tag with its defining content later in the same scope.

A complete type shall have a size that is less than or equal to **SIZE_MAX**. A type has *known constant size* if it is complete and is not a variable length array type.

Array, function, and pointer types are collectively called *derived declarator types*. A *declarator type derivation* from a type *T* is the construction of a derived declarator type from *T* by the application of an array-type, a function-type, or a pointer-type derivation to *T*.

A type is characterized by its *type category*, which is either the outermost derivation of a derived type (as noted previously in this subclause in the construction of derived types), or the type itself if the type consists of no derived types.

Any type so far mentioned is an *unqualified type*. Each unqualified type has several *qualified versions* of its type,⁴⁰⁾ corresponding to the combinations of one, two, or all three of the **const**, **volatile**, and **restrict** qualifiers. The qualified or unqualified versions of a type are distinct types that belong to the same type category and have the same representation and alignment requirements.⁴¹⁾ An array and its element type are always considered to be identically qualified.⁴²⁾ Any other derived type is not qualified by the qualifiers (if any) of the type from which it is derived.

Further, there is the **_Atomic** qualifier. The presence of the **_Atomic** qualifier designates an atomic type. The size, representation, and alignment of an atomic type may not be the same as those of the corresponding unqualified type. Therefore, this document explicitly uses the phrase “atomic, qualified, or unqualified type” whenever the atomic version of a type is permitted along with the other qualified versions of a type. The phrase “qualified or unqualified type”, without specific mention of atomic, does not include the atomic types.

A pointer to **void** shall have the same representation and alignment requirements as a pointer to a character type.⁴¹⁾ Similarly, pointers to qualified or unqualified versions of compatible types shall have the same representation and alignment requirements. All pointers to structure types shall have the same representation and alignment requirements as each other. All pointers to union types shall have the same representation and alignment requirements as each other. Pointers to other types may not have the same representation or alignment requirements.

EXAMPLE 1 The type designated as “**float ***” has type “pointer to **float**”. Its type category is pointer, not a floating type. The const-qualified version of this type is designated as “**float * const**” whereas the type designated as “**const float ***” is not a qualified type — its type is “pointer to const-qualified **float**” and is a pointer to a qualified type.

³⁹⁾Note that aggregate type does not include union type because an object with union type can only contain one member at a time.

⁴⁰⁾See 6.7.4 regarding qualified array and function types.

⁴¹⁾The same representation and alignment requirements are meant to imply interchangeability as arguments to functions, return values from functions, and members of unions.

⁴²⁾This does not apply to the **_Atomic** qualifier. Qualifiers do not have any direct effect on the array type itself, but affect conversion rules for pointer types that reference an array type.

EXAMPLE 2 The type designated as “**struct tag** (*[5])(**float**)” has type “array of pointer to function returning **struct tag**”. The array has length five and the function has a single parameter of type **float**. Its type category is array.

Forward references: compatible type and composite type (6.2.7), declarations (6.7).

6.2.6 Representations of types

6.2.6.1 General

The representations of all types are unspecified except as stated in this subclause.

Except for bit-fields, objects are composed of contiguous sequences of one or more bytes, the number, order, and encoding of which are either explicitly specified or implementation-defined.

Values stored in unsigned bit-fields and objects of type **unsigned char** shall be represented using a pure binary notation.

Values stored in non-bit-field objects of any other object type are represented using $n \times \text{CHAR_BIT}$ bits, where n is the size of an object of that type, in bytes. An object that has the value can be copied into an object of type **unsigned char** [n] (e.g. by **memcpy**); the resulting set of bytes is called the *object representation* of the value. Values stored in bit-fields consist of m bits, where m is the size specified for the bit-field. The object representation is the set of m bits the bit-field comprises in the addressable storage unit holding it. Two values (other than NaNs) with the same object representation compare equal, but values that compare equal may have different object representations.

Certain object representations do not represent a value of the object type. If such a representation is read by an lvalue expression that does not have character type, the behavior is undefined. If such a representation is produced by a side effect that modifies all or any part of the object by an lvalue expression that does not have character type, the behavior is undefined.⁴³⁾ Such a representation is called a non-value representation.

When a value is stored in an object of structure or union type, including in a member object, the bytes of the object representation that correspond to any padding bytes take unspecified values (e.g. structure and union assignment can fail to copy any padding bits). The object representation of a structure or union object is never a non-value representation, even though the byte range corresponding to a member of the structure or union object can be a non-value representation for that member.

When a value is stored in a member of an object of union type, the bytes of the object representation that do not correspond to that member but do correspond to other members take unspecified values.

Where an operator is applied to a value that has more than one object representation, which object representation is used shall not affect the value of the result.⁴⁴⁾ Where a value is stored in an object using a type that has more than one object representation for that value, it is unspecified which representation is used, but a non-value representation shall not be generated.

Loads and stores of objects with atomic types are done with **memory_order_seq_cst** semantics.

Forward references: declarations (6.7), expressions (6.5.1), lvalues, arrays, and function designators (6.3.3.1), order and consistency (7.17.3).

6.2.6.2 Integer types

For unsigned integer types the bits of the object representation shall be divided into two groups: value bits and padding bits. If there are N value bits, each bit shall represent a different power of 2 between 1 and 2^{N-1} , so that objects of that type shall be capable of representing values from 0 to $2^N - 1$ using a pure binary representation; this shall be known as the value representation. The values of any padding bits are unspecified. The number of value bits N is called the *width* of the unsigned integer type. The type **bool** shall have one value bit and **(sizeof(bool)*CHAR_BIT) - 1**

⁴³⁾Thus, an automatic variable can be initialized to a non-value representation without causing undefined behavior, but the value of the variable cannot be used until a proper value is stored in it.

⁴⁴⁾It is possible for objects **x** and **y** with the same effective type **T** to have the same value when they are accessed as objects of type **T**, but to have different values in other contexts. In particular, if **==** is defined for type **T**, then **x == y** does not imply that **memcmp(&x, &y, sizeof(T)) == 0**. Furthermore, **x == y** does not necessarily imply that **x** and **y** have the same value; other operations on values of type **T** can distinguish between them.

padding bits. Otherwise, there is no requirement to have any padding bits; **unsigned char** shall not have any padding bits.

For signed integer types, the bits of the object representation shall be divided into three groups: value bits, padding bits, and the sign bit. If the corresponding unsigned type has width N , the signed type uses the same number of N bits, its *width*, as value bits and sign bit. $N - 1$ are value bits and the remaining bit is the sign bit. Each bit that is a value bit shall have the same value as the same bit in the object representation of the corresponding unsigned type. If the sign bit is zero, it shall not affect the resulting value. If the sign bit is one, it has value $-(2^{N-1})$. There can be padding bits; **signed char** shall not have any padding bits.

The values of any padding bits are unspecified. A valid object representation of a signed integer type where the sign bit is zero is a valid object representation of the corresponding unsigned type, and shall represent the same value. For any integer type, the object representation where all the bits are zero shall be a representation of the value zero in that type.

The *precision* of an integer type is the number of value bits.

NOTE 1 Some combinations of padding bits can generate non-value representations, for example, if one padding bit is a parity bit. Regardless, no arithmetic operation on valid values can generate a non-value representation other than as part of an exceptional condition such as an integer overflow. All other combinations of padding bits are alternative object representations of the value specified by the value bits.

NOTE 2 The sign representation defined in this document is called *two's complement*. Previous editions of this document (specifically ISO/IEC 9899:2018 and prior editions) additionally allowed other sign representations.

NOTE 3 For unsigned integer types the width and precision are the same, while for signed integer types the width is one greater than the precision.

6.2.7 Compatible type and composite type

Two types are *compatible types* if they are the same. Additional rules for determining whether two types are compatible are described in 6.7.3 for type specifiers, in 6.7.4 for type qualifiers, and in 6.7.7 for declarators.⁴⁵⁾ Moreover, two complete structure, union, or enumerated types declared with the same tag are compatible if members satisfy the following requirements:

- there shall be a one-to-one correspondence between their members such that each pair of corresponding members are declared with compatible types;
- if one member of the pair is declared with an alignment specifier, the other is declared with an equivalent alignment specifier;
- and, if one member of the pair is declared with a name, the other is declared with the same name.

For two structures, corresponding members shall be declared in the same order. For two unions declared in the same translation unit, corresponding members shall be declared in the same order. For two structures or unions, corresponding bit-fields shall have the same widths. For two enumerations, corresponding members shall have the same values; if one has a fixed underlying type, then the other shall have a compatible fixed underlying type. For determining type compatibility, anonymous structures and unions are considered a regular member of the containing structure or union type, and the type of an anonymous structure or union is considered compatible to the type of another anonymous structure or union, respectively, if their members fulfill the preceding requirements.

Furthermore, two structure, union, or enumerated types declared in separate translation units are compatible in the following cases:

- both are declared without tags and they fulfill the preceding requirements;
- both have the same tag and are completed somewhere in their respective translation units and they fulfill the preceding requirements;

⁴⁵⁾Two types are not expected to be identical to be compatible.

- both have the same tag and at least one of the two types is not completed in its translation unit.

Otherwise, the structure, union, or enumerated types are incompatible.⁴⁶⁾

All declarations that refer to the same object or function shall have compatible type; otherwise, the behavior is undefined.

A *composite type* can be constructed from two types that are compatible. If both types are the same type, the composite type is this type. Otherwise, it is a type that is compatible with both and satisfies the following conditions:

- If both types are structure types or both types are union types, the composite type is determined recursively by forming the composite types of their members.
- If both types are array types, the following rules are applied:
 - If one type is an array of known constant size, the composite type is an array of that size.
 - Otherwise, if one type is a variable length array whose size is specified by an expression that is not evaluated, the behavior is undefined.
 - Otherwise, if one type is a variable length array whose size is specified, the composite type is a variable length array of that size.
 - Otherwise, if one type is a variable length array of unspecified size, the composite type is a variable length array of unspecified size.
 - Otherwise, both types are arrays of unknown size and the composite type is an array of unknown size.

The element type of the composite type is the composite type of the two element types.

- If both types are function types, the type of each parameter in the composite parameter type list is the composite type of the corresponding parameters.
- If one of the types has a standard attribute, the composite type also has that attribute.
- If both types are enumerated types, the composite type is an enumerated type.
- If one type is an enumerated type and the other is an integer type other than an enumerated type, it is implementation-defined whether or not the composite type is an enumerated type.

These rules apply recursively to the types from which the two types are derived.

If any of the original types satisfies all requirements of the composite type, it is unspecified whether the composite type is one of these types or a different type that satisfies the requirements.⁴⁷⁾

For an identifier with internal or external linkage declared in a scope in which a prior declaration of that identifier is visible,⁴⁸⁾ if the prior declaration specifies internal or external linkage, the type of the identifier at the later declaration becomes the composite type.

EXAMPLE Given the following two file scope declarations:

```
int f(int (*)(char *), double (*)[3]);
int f(int (*)(char *), double (*)[]);
```

The resulting composite type for the function is:

```
int f(int (*)(char *), double (*)[3]);
```

Forward references: array declarators (6.7.7.3).

⁴⁶⁾A structure, union, or enumerated type without a tag or an incomplete structure, union or enumerated type is not compatible with any other structure, union or enum type declared in the same translation unit.

⁴⁷⁾The notion of “same type” affects redeclarations of typedef names and tags in the same scope.

⁴⁸⁾As specified in 6.2.1, the later declaration can hide the prior declaration.

6.2.8 Alignment of objects

Complete object types have alignment requirements which place restrictions on the addresses at which objects of that type can be allocated. An alignment is an implementation-defined integer value representing the number of bytes between successive addresses at which a given object can be allocated. An object type imposes an alignment requirement on every object of that type: stricter alignment can be requested using the **alignas** keyword.

A *fundamental alignment* is a valid alignment less than or equal to **alignof(max_align_t)**. Fundamental alignments shall be supported by the implementation for objects of all storage durations. The alignment requirements of the following types shall be fundamental alignments:

- all atomic, qualified, or unqualified basic types;
- all atomic, qualified, or unqualified enumerated types;
- all atomic, qualified, or unqualified pointer types;
- all array types whose element type has a fundamental alignment requirement;
- all types specified in Clause 7 as complete object types;
- all structure or union types whose elements have types with fundamental alignment requirements and none of whose elements have an alignment specifier specifying an alignment that is not a fundamental alignment.

An *extended alignment* is represented by an alignment greater than **alignof(max_align_t)**. It is implementation-defined whether any extended alignments are supported and the storage durations for which they are supported. A type having an extended alignment requirement is an *over-aligned* type.⁴⁹⁾

Alignments are represented as values of the type **size_t**. Valid alignments include only fundamental alignments, plus an additional implementation-defined set of values, which can be empty. Every valid alignment value shall be a nonnegative integral power of two.

Alignments have an order from *weaker* to *stronger* or *stricter* alignments. Stricter alignments have larger alignment values. An address that satisfies an alignment requirement also satisfies any weaker valid alignment requirement.

The alignment requirement of a complete type can be queried using an **alignof** expression. The types **char**, **signed char**, and **unsigned char** shall have the weakest alignment requirement.

Comparing alignments is meaningful and provides the obvious results:

- Two alignments are equal when their numeric values are equal.
- Two alignments are different when their numeric values are not equal.
- When an alignment is larger than another it represents a stricter alignment.

6.2.9 Encodings

The *literal encoding* is an implementation-defined mapping of the characters of the execution character set to the values in a character constant (6.4.5.5) or string literal (6.4.6). It shall support a mapping from all the basic execution character set values into the implementation-defined encoding. It can contain multibyte character sequences (5.3.2).

The *wide literal encoding* is an implementation-defined mapping of the characters of the execution character set to the values in a **wchar_t** character constant (6.4.5.5) or a **wchar_t** string literal (6.4.6). It shall support a mapping from all the basic execution character set values into the implementation-defined encoding. The mapping shall produce values identical to the literal encoding for all the basic execution character set values if an implementation does not define **__STDC_MB_MIGHT_NEQ_WC__**. One or more values may map to one or more values of the extended execution character set.

⁴⁹⁾ Every over-aligned type is, or contains, a structure or union type with a member to which an extended alignment has been applied.

6.3 Conversions

6.3.1 Introduction

Several operators convert operand values from one type to another automatically. This subclause specifies the result required from such an *implicit conversion*, as well as those that result from a cast operation (an *explicit conversion*). The list in 6.3.2.8 summarizes the conversions performed by most ordinary operators; it is supplemented as required by the discussion of each operator in 6.5.1.

Unless explicitly stated otherwise, conversion of an operand value to a compatible type causes no change to the value or the representation.

Forward references: cast operators (6.5.5).

6.3.2 Arithmetic operands

6.3.2.1 Boolean, characters, and integers

Every integer type has an *integer conversion rank* defined as follows:

- No two signed integer types shall have the same rank, even if they have the same representation.
- The rank of a signed integer type shall be greater than the rank of any signed integer type with less precision.
- The rank of **long long int** shall be greater than the rank of **long int**, which shall be greater than the rank of **int**, which shall be greater than the rank of **short int**, which shall be greater than the rank of **signed char**.
- The rank of a bit-precise signed integer type shall be greater than the rank of any standard integer type with less width or any bit-precise integer type with less width.
- The rank of any unsigned integer type shall equal the rank of the corresponding signed integer type, if any.
- The rank of any standard integer type shall be greater than the rank of any extended integer type with the same width or bit-precise integer type with the same width.
- The rank of any bit-precise integer type relative to an extended integer type of the same width is implementation-defined.
- The rank of **char** shall equal the rank of **signed char** and **unsigned char**.
- The rank of **bool** shall be less than the rank of all other standard integer types.
- The rank of any enumerated type shall equal the rank of the compatible integer type (see 6.7.3.3).
- The rank of any extended signed integer type relative to another extended signed integer type with the same precision is implementation-defined, but still subject to the other rules for determining the integer conversion rank.
- For all integer types **T1**, **T2**, and **T3**, if **T1** has greater rank than **T2** and **T2** has greater rank than **T3**, then **T1** has greater rank than **T3**.

The following can be used in an expression wherever an **int** or **unsigned int** can be used:

- An object or expression with an integer type (other than **int** or **unsigned int**) whose integer conversion rank is less than or equal to the rank of **int** and **unsigned int**.
- A bit-field of type **bool**, **int**, **signed int**, or **unsigned int**.

The value from a bit-field of a bit-precise integer type is converted to the corresponding bit-precise integer type. If the original type is not a bit-precise integer type (6.2.5): if an **int** can represent all values of the original type (as restricted by the width, for a bit-field), the value is converted to an **int**;⁵⁰⁾ otherwise, it is converted to an **unsigned int**. These are called the *integer promotions*. All other types are unchanged by the integer promotions.

NOTE The integer promotions are applied only:

1. as part of the usual arithmetic conversions,
2. to certain argument expressions,
3. to the operands of the unary **+**, **-**, and **~** operators,
4. and to both operands of the shift operators,

as specified by their respective subclauses.

The integer promotions preserve value including sign. As discussed earlier, whether a “plain” **char** can hold negative values is implementation-defined.

Forward references: enumeration specifiers (6.7.3.3), structure and union specifiers (6.7.3.2).

6.3.2.2 Boolean type

When any scalar value is converted to **bool**, the result is **false** if the value is a zero (for arithmetic types), null (for pointer types), or the scalar has type **nullptr_t**; otherwise, the result is **true**.

6.3.2.3 Signed and unsigned integers

When a value with integer type is converted to another integer type other than **bool**, if the value can be represented by the new type, it is unchanged.

Otherwise, if the new type is unsigned, the value is converted by repeatedly adding or subtracting one more than the maximum value that can be represented in the new type until the value is in the range of the new type.⁵¹⁾

Otherwise, the new type is signed and the value cannot be represented in it; either the result is implementation-defined or an implementation-defined signal is raised.

6.3.2.4 Real floating and integer

When a finite value of standard floating type is converted to an integer type other than **bool**, the fractional part is discarded (i.e. the value is truncated toward zero). If the value of the integral part cannot be represented by the integer type, the behavior is undefined.⁵²⁾

When a finite value of decimal floating type is converted to an integer type other than **bool**, the fractional part is discarded (i.e. the value is truncated toward zero). If the value of the integral part cannot be represented by the integer type, the “invalid” floating-point exception shall be raised and the result of the conversion is unspecified.

When a value of integer type is converted to a standard floating type, if the value being converted can be represented exactly in the new type, it is unchanged. If the value being converted is in the range of values that can be represented but cannot be represented exactly, the result is either the nearest higher or nearest lower representable value, chosen in an implementation-defined manner. If the value being converted is outside the range of values that can be represented, the behavior is undefined. Results of some implicit conversions can be represented in greater range and precision than that required by the new type (see 6.3.2.8 and 6.8.7.5).

When a value of integer type is converted to a decimal floating type, if the value being converted can be represented exactly in the new type, it is unchanged. If the value being converted cannot be represented exactly, the result shall be correctly rounded with exceptions raised as specified in ISO/IEC 60559.

⁵⁰⁾E.g. **unsigned _BitInt(7): 2** is a bit-field that can hold the values 0, 1, 2, 3, and converts to **unsigned _BitInt(7)**.

⁵¹⁾The rules describe arithmetic on the mathematical value, not the value of a given type of expression.

⁵²⁾The remaindering operation performed when a value of integer type is converted to unsigned type is not necessary to be performed when a value of real floating type is converted to unsigned type. Thus, the range of portable real floating values is $(-1, \text{Utype_MAX} + 1)$.

6.3.2.5 Real floating types

When a value of real floating type is converted to a real floating type, if the value being converted can be represented exactly in the new type, it is unchanged.

When a value of real floating type is converted to a standard floating type, if the value being converted is in the range of values that can be represented but cannot be represented exactly, the result is either the nearest higher or nearest lower representable value, chosen in an implementation-defined manner. If the value being converted is outside the range of values that can be represented, the behavior is undefined.

When a value of real floating type is converted to a decimal floating type, if the value being converted cannot be represented exactly, the result is correctly rounded with exceptions raised as specified in ISO/IEC 60559.

Results of some implicit conversions may be represented in greater range and precision than that required by the new type (see 6.3.2.8 and 6.8.7.5).

6.3.2.6 Complex types

When a value of complex type is converted to another complex type, both the real and imaginary parts follow the conversion rules for the corresponding real types.

6.3.2.7 Real and complex

When a value of real type is converted to a complex type, the real part of the complex result value is determined by the rules of conversion to the corresponding real type and the imaginary part of the complex result value is a positive zero or an unsigned zero.

When a value of complex type is converted to a real type other than **bool**,⁵³⁾ the imaginary part of the complex value is discarded and the value of the real part is converted according to the conversion rules for the corresponding real type.

6.3.2.8 Usual arithmetic conversions

Many operators that expect operands of arithmetic type cause conversions and yield result types in a similar way. The purpose is to determine a *common real type* for the operands and result. For the specified operands, each operand is converted, without change of type domain, to a type whose corresponding real type is the common real type. Unless explicitly stated otherwise, the common real type is also the corresponding real type of the result, whose type domain is the type domain of the operands if they are the same, and complex otherwise. This pattern is called the *usual arithmetic conversions*:

If one operand has decimal floating type, the other operand shall not have standard floating, complex, or imaginary type.

First, if the type of either operand is **_Decimal128**, the other operand is converted to **_Decimal128**.

Otherwise, if the type of either operand is **_Decimal64**, the other operand is converted to **_Decimal64**.

Otherwise, if the type of either operand is **_Decimal32**, the other operand is converted to **_Decimal32**.

Otherwise, if the corresponding real type of either operand is **long double**, the other operand is converted, without change of type domain, to a type whose corresponding real type is **long double**.

Otherwise, if the corresponding real type of either operand is **double**, the other operand is converted, without change of type domain, to a type whose corresponding real type is **double**.

⁵³⁾See 6.3.2.2.

Otherwise, if the corresponding real type of either operand is **float**, the other operand is converted, without change of type domain, to a type whose corresponding real type is **float**.⁵⁴⁾

Otherwise, if any of the two types is an enumeration, it is converted to its underlying type. Then, the integer promotions are performed on both operands. Next, the following rules are applied to the promoted operands:

If both operands have the same type, then no further conversion is needed.

Otherwise, if both operands have signed integer types or both have unsigned integer types, the operand with the type of lesser integer conversion rank is converted to the type of the operand with greater rank.

Otherwise, if the operand that has unsigned integer type has rank greater or equal to the rank of the type of the other operand, then the operand with signed integer type is converted to the type of the operand with unsigned integer type.

Otherwise, if the type of the operand with signed integer type can represent all the values of the type of the operand with unsigned integer type, then the operand with unsigned integer type is converted to the type of the operand with signed integer type.

Otherwise, both operands are converted to the unsigned integer type corresponding to the type of the operand with signed integer type.

The values of floating operands and of the results of floating expressions may be represented in greater range and precision than that required by the type; the types are not changed thereby. See 5.3.5.3.3 regarding evaluation formats.

EXAMPLE One consequence of `_BitInt` being exempt from the integer promotion rules (6.3.2) is that a `_BitInt` operand of a binary operator is not always promoted to an `int` or `unsigned int` as part of the usual arithmetic conversions. Instead, a lower-ranked operand is converted to the higher-rank operand type and the result of the operation is the higher-ranked type.

```
_BitInt(2) a2 = 1;
_BitInt(3) a3 = 2;
_BitInt(33) a33 = 1;
signed char c = 3;

a2 * a3; /* As part of the multiplication, a2 is converted to
         _BitInt(3) and the result type is _BitInt(3). */
a2 * c; /* As part of the multiplication, c is promoted to int,
         a2 is converted to int and the result type is int. */
a33 * c; /* As part of the multiplication, c is promoted to int.
         Then, provided int has a width of at most 32,
         it is converted to _BitInt(33) and the result type
         is _BitInt(33). */

void func(_BitInt(8) a8, _BitInt(24) a24) {
    /* Cast one of the operands to 32-bits to guarantee the
       result of the multiplication can contain all possible values. */
    _BitInt(32) a32 = a8 * (_BitInt(32))a24;
}
```

6.3.3 Other operands

6.3.3.1 Lvalues, arrays, and function designators

An *lvalue* is an expression (with an object type other than **void**) that potentially designates an object;⁵⁵⁾ if an lvalue does not designate an object when it is evaluated, the behavior is undefined.

⁵⁴⁾For example, addition of a **double** `_Complex` and a **float** entails just the conversion of the **float** operand to **double** (and yields a **double** `_Complex` result).

⁵⁵⁾The name “lvalue” comes originally from the assignment expression **E1** = **E2**, in which the left operand **E1** is required to be a (modifiable) lvalue. It is perhaps better considered as representing an object “locator value”. What is sometimes called

When an object is said to have a particular type, the type is specified by the lvalue used to designate the object. A *modifiable lvalue* is an lvalue that does not have array type, does not have an incomplete type, does not have a const-qualified type, and if it is a structure or union, does not have any member (including, recursively, any member or element of all contained aggregates or unions) with a const-qualified type.

Except when it is the operand of the **sizeof** operator, or the **typeof** operators, the unary **&** operator, the **++** operator, the **--** operator, or the left operand of the **.** operator or an assignment operator, an lvalue that does not have array type is converted to the value stored in the designated object (and is no longer an lvalue); this is called *lvalue conversion*. If the lvalue has qualified type, the value has the unqualified version of the type of the lvalue; additionally, if the lvalue has atomic type, the value has the non-atomic version of the type of the lvalue; otherwise, the value has the type of the lvalue. If the lvalue has an incomplete type and does not have array type, the behavior is undefined. If the lvalue designates an object of automatic storage duration that could have been declared with the **register** storage class (never had its address taken), and that object is uninitialized (not declared with an initializer and no assignment to it has been performed prior to use), the behavior is undefined.

Except when it is the operand of the **sizeof** operator, or **typeof** operators, or the unary **&** operator, or is a string literal used to initialize an array, an expression that has type “array of *type*” is converted to an expression with type “pointer to *type*” that points to the initial element of the array object and is not an lvalue. If the array object has register storage class, the behavior is undefined.

A *function designator* is an expression that has function type. Except when it is the operand of the **sizeof** operator,⁵⁶⁾ a **typeof** operator, or the unary **&** operator, a function designator with type “function returning *type*” is converted to an expression that has type “pointer to function returning *type*”.

Forward references: address and indirection operators (6.5.4.3), assignment operators (6.5.17), common definitions `<stddef.h>` (7.21), initialization (6.7.11), postfix increment and decrement operators (6.5.3.5), prefix increment and decrement operators (6.5.4.2), the **sizeof** and **alignof** operators (6.5.4.5), structure and union members (6.5.3.4).

6.3.3.2 void

The (nonexistent) value of a *void expression* (an expression that has type **void**) shall not be used in any way, and implicit or explicit conversions (except to **void**) shall not be applied to such an expression. If an expression of any other type is evaluated as a void expression, its value or designator is discarded. (A void expression is evaluated for its side effects.)

6.3.3.3 Pointers

A pointer to **void** can be converted to or from a pointer to any object type. A pointer to any object type can be converted to a pointer to **void** and back again; the result shall compare equal to the original pointer.

For any qualifier *q*, a pointer to a non-*q*-qualified type can be converted to a pointer to the *q*-qualified version of the type; the values stored in the original and converted pointers shall compare equal.

An integer constant expression with the value 0, such an expression cast to type **void ***, or the predefined constant **nullptr** is called a *null pointer constant*.⁵⁷⁾ If a null pointer constant or a value of the type **nullptr_t** (which is necessarily the value **nullptr**) is converted to a pointer type, the resulting pointer, called a *null pointer*, is guaranteed to compare unequal to a pointer to any object or function.

Conversion of a null pointer to another pointer type yields a null pointer of that type. Any two null pointers shall compare equal.

An integer can be converted to any pointer type. Except as previously specified, the result is

“rvalue” is in this document described as the “value of an expression”.

An obvious example of an lvalue is an identifier of an object. As a further example, if **E** is a unary expression that is a pointer to an object, ***E** is an lvalue that designates the object to which **E** points.

⁵⁶⁾Because this conversion does not occur, the operand of the **sizeof** operator remains a function designator and violates the constraints in 6.5.4.5

⁵⁷⁾The macro **NULL** is defined in `<stddef.h>` (and other headers) as a null pointer constant; see 7.21.

implementation-defined, possibly not correctly aligned, can possibly not point to an entity of the referenced type, and can produce an indeterminate representation when stored into an object.⁵⁸⁾

Any pointer type can be converted to an integer type. Except as previously specified, the result is implementation-defined. If the result cannot be represented in the integer type, the behavior is undefined. The result is not required to be in the range of values of any integer type.

A pointer to an object type can be converted to a pointer to a different object type. If the resulting pointer is not correctly aligned⁵⁹⁾ for the referenced type, the behavior is undefined. Otherwise, when converted back again, the result shall compare equal to the original pointer. When a pointer to an object is converted to a pointer to a character type, the result points to the lowest addressed byte of the object. Successive increments of the result, up to the size of the object, yield pointers to the remaining bytes of the object.

A pointer to a function of one type can be converted to a pointer to a function of another type and back again; the result shall compare equal to the original pointer. If a converted pointer is used to call a function whose type is not compatible with the referenced type, the behavior is undefined.

6.3.3.4 `uintptr_t`

The type `uintptr_t` can be converted to `void`, `bool` or to a pointer type; the result is a `void` expression, `false`, or a null pointer value, respectively.

A null pointer constant or value of type `uintptr_t` can be converted to `uintptr_t`.

Forward references: cast operators (6.5.5), equality operators (6.5.10), integer types capable of holding object pointers (7.22.2.5), simple assignment (6.5.17.2), the `uintptr_t` type (7.21.3).

⁵⁸⁾The mapping functions for converting a pointer to an integer or an integer to a pointer are intended to be consistent with the addressing structure of the execution environment.

⁵⁹⁾In general, the concept “correctly aligned” is transitive: if a pointer to type *A* is correctly aligned for a pointer to type *B*, which in turn is correctly aligned for a pointer to type *C*, then a pointer to type *A* is correctly aligned for a pointer to type *C*.

6.4 Lexical elements

6.4.1 General

Syntax

token:

keyword
identifier
constant
string-literal
punctuator

preprocessing-token:

header-name
identifier
pp-number
character-constant
string-literal
punctuator

each universal character name that cannot be one of the above
each non-white-space character that cannot be one of the above

Constraints

Each preprocessing token that is converted to a token shall have the lexical form of a keyword, an identifier, a constant, a string literal, or a punctuator. A single universal character name shall match one of the other preprocessing token categories.

Semantics

A *token* is the minimal lexical element of the language in translation phases 7 and 8 (5.2.1.2). The categories of tokens are: keywords, identifiers, constants, string literals, and punctuators. A preprocessing token is the minimal lexical element of the language in translation phases 3 through 6. The categories of preprocessing tokens are: header names, identifiers, preprocessing numbers, character constants, string literals, punctuators, and both single universal character names as well as single non-white-space characters that do not lexically match the other preprocessing token categories.⁶⁰⁾ If a ' or a " character matches the last category, the behavior is undefined. Preprocessing tokens can be separated by *white space*; this consists of comments (described later), or *white-space characters* (space, horizontal tab, new-line, vertical tab, and form-feed), or both. As described in 6.10, in certain circumstances during translation phase 4, white space (or the absence thereof) serves as more than preprocessing token separation. White space may appear within a preprocessing token only as part of a header name or between the quotation characters in a character constant or string literal.

If the input stream has been parsed into preprocessing tokens up to a given character, the next preprocessing token is the longest sequence of characters that could constitute a preprocessing token. There is one exception to this rule: header name preprocessing tokens are recognized only within **#include** and **#embed** preprocessing directives, in **__has_include** and **__has_embed** expressions, as well as in implementation-defined locations within **#pragma** directives. In such contexts, a sequence of characters that could be either a header name or a string literal is recognized as the former.

EXAMPLE 1 The program fragment **1Ex** is parsed as a preprocessing number token (one that is not a valid floating or integer constant token), even though a parse as the pair of preprocessing tokens **1** and **Ex** can produce a valid expression (for example, if **Ex** were a macro defined as **+1**). Similarly, the program fragment **1E1** is parsed as a preprocessing number (one that is a valid floating constant token), whether or not **E** is a macro name.

EXAMPLE 2 The program fragment **x+++++y** is parsed as **x ++ ++ + y**, which violates a constraint on increment operators, even though the parse **x ++ + ++ y** can yield a correct expression.

⁶⁰⁾An additional category, placemarkers, is used internally in translation phase 4 (see 6.10.5.4); it cannot occur in source files.

Forward references: character constants (6.4.5.5), comments (6.4.10), expressions (6.5.1), floating constants (6.4.5.3), header names (6.4.8), macro replacement (6.10.5), postfix increment and decrement operators (6.5.3.5), prefix increment and decrement operators (6.5.4.2), preprocessing directives (6.10), preprocessing numbers (6.4.9), string literals (6.4.6).

6.4.2 Keywords

Syntax

keyword: one of

alignas	do	int	struct	while
alignof	double	long	switch	_Atomic
auto	else	nullptr	thread_local	_BitInt
bool	enum	register	true	_Complex
break	extern	restrict	typedef	_Decimal128
case	false	return	typeof	_Decimal32
char	float	short	typeof_unqual	_Decimal64
const	for	signed	union	_Generic
constexpr	goto	sizeof	unsigned	_Imaginary
continue	if	static	void	_Noreturn
default	inline	static_assert	volatile	

Semantics

The previously listed tokens (case sensitive) are reserved (in translation phases 7 and 8) for use as keywords except in an attribute token, and shall not be used otherwise. The keyword **_Imaginary** is reserved for specifying imaginary types.⁶¹⁾

Table 6.1 provides alternate spellings for certain keywords. These can be used wherever the keyword can.⁶²⁾

Table 6.1 — Keywords and their spellings

Keyword	Alternative Spelling
alignas	_Alignas
alignof	_Alignof
bool	_Bool
static_assert	_Static_assert
thread_local	_Thread_local

The spelling of these keywords, their alternate forms, and of **false** and **true** inside expressions that are subject to the **#** and **##** preprocessing operators is unspecified.⁶³⁾

⁶¹⁾One possible specification for imaginary types appears in Annex G.

⁶²⁾These alternative keywords are obsolescent features and should not be used for new code and development.

⁶³⁾The intent of this specification is to allow but not force the implementation of the corresponding feature by means of a predefined macro.

6.4.3 Identifiers

6.4.3.1 General

Syntax

identifier:

identifier-start
identifier identifier-continue

identifier-start:

nondigit
XID_Start character
universal character name of class XID_Start

identifier-continue:

digit
nondigit
XID_Continue character
universal character name of class XID_Continue

nondigit: one of

_ a b c d e f g h i j k l m
n o p q r s t u v w x y z
A B C D E F G H I J K L M
N O P Q R S T U V W X Y Z

digit: one of

0 1 2 3 4 5 6 7 8 9

Semantics

An XID_Start character is an implementation-defined character whose corresponding code point in ISO/IEC 10646 has the XID_Start property. An XID_Continue character is an implementation-defined character whose corresponding code point in ISO/IEC 10646 has the XID_Continue property. An identifier is a sequence of one identifier start character followed by 0 or more identifier continue characters, which designates one or more entities as described in 6.2.1. It is implementation-defined if a \$ (U+0024, DOLLAR SIGN) may be used as a nondigit character. Lowercase and uppercase letters are distinct. There is no specific limit on the maximum length of an identifier.

The character classes XID_Start and XID_Continue are Derived Core Properties as described by UAX #44.⁶⁴⁾ Each character and universal character name in an identifier shall designate a character whose encoding in ISO/IEC 10646 has the XID_Continue property. The initial character (which can be a universal character name) shall designate a character whose encoding in ISO/IEC 10646 has the XID_Start property. An identifier shall conform to Normalization Form C as specified in ISO/IEC 10646. Annex D provides an overview of the conforming identifiers.

NOTE 1 Uppercase and lowercase letters are considered different for all identifiers.

NOTE 2 In translation phase 4, the term identifier also includes those preprocessing tokens (6.4.9) differentiated as keywords (6.4.2) in the later translation phase 7 (5.2.1.2).

⁶⁴⁾On systems that cannot accept extended characters in external identifiers, an encoding of the universal character name can be used in forming such identifiers. For example, some otherwise unused character or sequence of characters can be used to encode the **u** in a universal character name.

When preprocessing tokens are converted to tokens during translation phase 7, if a preprocessing token could be converted to either a keyword or an identifier, it is converted to a keyword except in an attribute token.

Some identifiers are reserved.

- All identifiers that begin with a double underscore (__) or begin with an underscore (_) followed by an uppercase letter are reserved for any use, except those identifiers which are lexically identical to keywords.⁶⁵⁾
- All identifiers that begin with an underscore are reserved for use as identifiers with file scope in both the ordinary and tag name spaces.

Other identifiers may be reserved, see 7.1.3.

If the program declares or defines an identifier in a context in which it is reserved (other than as allowed by 7.1.4), the behavior is undefined.

If the program defines a reserved identifier or standard attribute token described in 6.7.13.2 as a macro name, or removes (with **#undef**) any macro definition of an identifier in the first group listed previously or standard attribute token described in 6.7.13.2, the behavior is undefined.

Some identifiers may be potentially reserved. A *potentially reserved identifier* is an identifier which is not reserved unless made so by an implementation providing the identifier (7.1.3) but is anticipated to become reserved by an implementation or a future version of this document. An identifier that this document describes as optional:

- If it is defined as a macro it is reserved.
- Otherwise, if the definition is given in clauses 1 to 6 it is reserved.
- Otherwise, it is potentially reserved.

Recommended practice

Implementations are encouraged to issue a diagnostic message when a potentially reserved identifier is declared or defined for any use that is not implementation-compatible (see subsequent description in this subclause) in a context where the potentially reserved identifier may be reserved under a conforming implementation. This brings attention to a potential conflict when porting a program to a future edition of this document.

An implementation-compatible use of a potentially reserved identifier is a declaration of an external name where the name is provided by the implementation as an external name and where the declaration declares an object or function with a type that is compatible with the type of the object or function provided by the implementation under that name.

Implementation limits

As discussed in 5.3.5.2, an implementation may limit the number of significant initial characters in an identifier; the limit for an *external name* (an identifier that has external linkage) may be more restrictive than that for an *internal name* (a macro name or an identifier that does not have external linkage). The number of significant characters in an identifier is implementation-defined.

Any identifiers that differ in a significant character are different identifiers. If two identifiers differ only in nonsignificant characters, the behavior is undefined.

Forward references: universal character names (6.4.4), macro replacement (6.10.5), reserved library identifiers (7.1.3), use of library functions (7.1.4), attributes (6.7.13.2).

6.4.3.2 Predefined identifiers

Semantics

The identifier **__func__** shall be implicitly declared by the translator as if, immediately following the opening brace of each function definition, the declaration

⁶⁵⁾This allows a reserved identifier that matches the spelling of a keyword to be used as a macro name by the program.

```
static const char __func__[] = "function-name";
```

appeared, where *function-name* is the name of the lexically-enclosing function.⁶⁶

This name is encoded as if the implicit declaration had been written in the source character set and then translated into the execution character set as indicated in translation phase 5.

EXAMPLE The following code fragment can be used as an example:

```
#include <stdio.h>
void myfunc(void)
{
    printf("%S\n", __func__);
    /* ... */
}
```

Each time the function is called, it will print to the standard output stream:

```
myfunc
```

Forward references: function definitions (6.9.2).

6.4.4 Universal character names

Syntax

universal-character-name:

\u *hex-quad*

\U *hex-quad hex-quad*

hex-quad:

hexadecimal-digit hexadecimal-digit hexadecimal-digit hexadecimal-digit

Constraints

A universal character name shall not designate a code point where the hexadecimal value is:

- in the range D800 through DFFF inclusive; or
- greater than 10FFFF.⁶⁷⁾

A universal character name outside the c-char sequence of a character constant, or the s-char sequence of a string literal shall not designate a control character or a character in the basic character set.

Description

Universal character names can be used in identifiers, character constants, and string literals to designate characters that are not in the basic character set.

Semantics

A universal character name designates the character in ISO/IEC 10646 whose code point is the hexadecimal value represented by the sequence of hexadecimal digits in the universal character name.

⁶⁶Since the name `__func__` is reserved for any use by the implementation (7.1.3), if any other identifier is explicitly declared using the name `__func__`, the behavior is undefined.

67) The disallowed characters are the characters in the basic character set and the code positions reserved by ISO/IEC 10646 for control characters, the character DELETE, the S-zone (reserved for use by UTF-16), and characters too large to be encoded by ISO/IEC 10646. Disallowed universal character escape sequences can still be specified with hexadecimal and octal escape sequences (6.4.5.5).

6.4.5 Constants

6.4.5.1 General

Syntax

constant:

integer-constant
floating-constant
enumeration-constant
character-constant
predefined-constant

Constraints

Each constant shall have a type and the value of a constant shall be in the range of representable values for its type.

Semantics

Each constant has a type, determined by its form and value, as detailed later.

6.4.5.2 Integer constants

Syntax

integer-constant:

decimal-constant integer-suffix_{opt}
octal-constant integer-suffix_{opt}
hexadecimal-constant integer-suffix_{opt}
binary-constant integer-suffix_{opt}

decimal-constant:

nonzero-digit
decimal-constant ' _{opt} *digit*

octal-constant:

0
octal-constant ' _{opt} *octal-digit*

hexadecimal-constant:

hexadecimal-prefix hexadecimal-digit-sequence

binary-constant:

binary-prefix binary-digit
binary-constant ' _{opt} *binary-digit*

hexadecimal-prefix: one of

0x 0X

binary-prefix: one of

0b 0B

nonzero-digit: one of
1 2 3 4 5 6 7 8 9

octal-digit: one of
0 1 2 3 4 5 6 7

hexadecimal-digit-sequence:
hexadecimal-digit
hexadecimal-digit-sequence ' _{opt} *hexadecimal-digit*

hexadecimal-digit: one of
0 1 2 3 4 5 6 7 8 9
a b c d e f
A B C D E F

binary-digit: one of
0 1

integer-suffix:
*unsigned-suffix long-suffix*_{opt}
unsigned-suffix long-long-suffix
unsigned-suffix bit-precise-int-suffix
*long-suffix unsigned-suffix*_{opt}
*long-long-suffix unsigned-suffix*_{opt}
*bit-precise-int-suffix unsigned-suffix*_{opt}

bit-precise-int-suffix: one of
wb WB

unsigned-suffix: one of
u U

long-suffix: one of
l L

long-long-suffix: one of
ll LL

Description

An integer constant begins with a digit, but has no period or exponent part. It can have a prefix that specifies its base and a suffix that specifies its type. An optional separating single quote character

(') in an integer or floating constant is called a digit separator. Digit separators are ignored when determining the value of the constant.

EXAMPLE 1 The following integer constants use digit separators; the comment associated with each constant shows the equivalent constant without digit separators.

```
0b11'10'11'01 /* 0b11101101 */
'1'2 /* character constant '1' followed by integer constant 2,
      not the integer constant 12 */
11'22 /* 1122 */
0x'FFFF'FFFF /* invalid hexadecimal constant ( ' cannot appear after 0x) */
0x1'2'3'4AB'C'D /* 0x1234ABCD */
```

A decimal constant begins with a nonzero digit and consists of a sequence of decimal digits. An octal constant consists of the prefix **0** optionally followed by a sequence of the digits **0** through **7** only. A hexadecimal constant consists of the prefix **0x** or **0X** followed by a sequence of the decimal digits and the letters **a** (or **A**) through **f** (or **F**) with values 10 through 15 respectively. A binary constant consists of the prefix **0b** or **0B** followed by a sequence of the digits **0** or **1**.

Semantics

The value of a decimal constant is computed base 10; that of an octal constant, base 8; that of a hexadecimal constant, base 16; that of a binary constant, base 2. The lexically first digit is the most significant.

The type of an integer constant is the first of the corresponding Table 6.2 in which its value can be represented.

If an integer constant that does not have suffixes **wb**, **WB**, **uwb**, or **UWB** cannot be represented by any type in its list, it may have an extended integer type, if the extended integer type can represent its value. If all the types in the list for the constant are signed, the extended integer type shall be signed. If all the types in the list for the constant are unsigned, the extended integer type shall be unsigned. If the list contains both signed and unsigned types, the extended integer type may be signed or unsigned. If an integer constant cannot be represented by any type in its list and has no extended integer type, then the integer constant has no type.

EXAMPLE 2 The **wb** suffix results in an **_BitInt** that includes space for the sign bit even if the value of the constant is positive or was specified in binary, octal, or hexadecimal notation.

```
-3wb /* Yields a _BitInt(3) that is then arithmetically negated;
      two value bits, one sign bit */
-0x3wb /* Yields a _BitInt(3) that is then arithmetically negated;
        two value bits, one sign bit */
3wb /* Yields a _BitInt(3); two value bits, one sign bit */
3uwb /* Yields an unsigned _BitInt(2) */
-3uwb /* Yields an unsigned _BitInt(2) that is then arithmetically
        negated, resulting in wraparound */
```

Forward references: preprocessing numbers (6.4.9), numeric conversion functions (7.24.2).

6.4.5.3 Floating constants

Syntax

floating-constant:

decimal-floating-constant
hexadecimal-floating-constant

decimal-floating-constant:

fractional-constant *exponent-part*_{opt} *floating-suffix*_{opt}
digit-sequence *exponent-part* *floating-suffix*_{opt}

Table 6.2 — Relationship between constants, suffixes, and types

Suffix	Decimal Constant	Octal, Hexadecimal or Binary Constant
none	<code>int</code> <code>long int</code> <code>long long int</code>	<code>int</code> <code>unsigned int</code> <code>long int</code> <code>unsigned long int</code> <code>long long int</code> <code>unsigned long long int</code>
<code>u</code> or <code>U</code>	<code>unsigned int</code> <code>unsigned long int</code> <code>unsigned long long int</code>	<code>unsigned int</code> <code>unsigned long int</code> <code>unsigned long long int</code>
<code>l</code> or <code>L</code>	<code>long int</code> <code>long long int</code>	<code>long int</code> <code>unsigned long int</code> <code>long long int</code> <code>unsigned long long int</code>
Both <code>u</code> or <code>U</code> and <code>l</code> or <code>L</code>	<code>unsigned long int</code> <code>unsigned long long int</code>	<code>unsigned long int</code> <code>unsigned long long int</code>
<code>ll</code> or <code>LL</code>	<code>long long int</code>	<code>long long int</code> <code>unsigned long long int</code>
Both <code>u</code> or <code>U</code> and <code>ll</code> or <code>LL</code>	<code>unsigned long long int</code>	<code>unsigned long long int</code>
<code>wb</code> or <code>WB</code>	<code>_BitInt(N)</code> where the width <code>N</code> is the smallest <code>N</code> greater than 1 which can accommodate the value and the sign bit.	<code>_BitInt(N)</code> where the width <code>N</code> is the smallest <code>N</code> greater than 1 which can accommodate the value and the sign bit.
Both <code>u</code> or <code>U</code> and <code>wb</code> or <code>WB</code>	<code>unsigned _BitInt(N)</code> where the width <code>N</code> is the smallest <code>N</code> greater than 0 which can accommodate the value.	<code>unsigned _BitInt(N)</code> where the width <code>N</code> is the smallest <code>N</code> greater than 0 which can accommodate the value.

hexadecimal-floating-constant:
 hexadecimal-prefix hexadecimal-fractional-constant
 binary-exponent-part floating-suffix_{opt}
 hexadecimal-prefix hexadecimal-digit-sequence
 binary-exponent-part floating-suffix_{opt}

fractional-constant:
 digit-sequence_{opt} . digit-sequence
 digit-sequence .

exponent-part:
 e sign_{opt} digit-sequence
 E sign_{opt} digit-sequence

sign: one of
 + -

digit-sequence:

digit
digit-sequence ' _{opt} *digit*

hexadecimal-fractional-constant:

*hexadecimal-digit-sequence*_{opt} . *hexadecimal-digit-sequence*
hexadecimal-digit-sequence .

binary-exponent-part:

p *sign*_{opt} *digit-sequence*
P *sign*_{opt} *digit-sequence*

floating-suffix: one of

f l F L df dd dL DF DD DL

Constraints

A floating suffix **df**, **dd**, **dL**, **DF**, **DD**, or **DL** shall not be used in a hexadecimal floating constant.

Description

A floating constant has a *significant part* that can be followed by an *exponent part* and a suffix that specifies its type. The components of the significant part can include a digit sequence representing the whole-number part, followed by a period (.), followed by a digit sequence representing the fraction part. Digit separators (6.4.5.2) are ignored when determining the value of the constant. The components of the exponent part are an **e**, **E**, **p**, or **P** followed by an exponent consisting of an optionally signed digit sequence. Either the whole-number part or the fraction part has to be present; for decimal floating constants, either the period or the exponent part has to be present.

Semantics

The significant part is interpreted as a (decimal or hexadecimal) rational number; the digit sequence in the exponent part is interpreted as a decimal integer. For decimal floating constants, the exponent indicates the power of 10 by which the significant part is to be scaled. For hexadecimal floating constants, the exponent indicates the power of 2 by which the significant part is to be scaled. For decimal floating constants, and also for hexadecimal floating constants when **FLT_RADIX** is not a power of 2, the result is either the nearest representable value, or the larger or smaller representable value immediately adjacent to the nearest representable value, chosen in an implementation-defined manner. For hexadecimal floating constants when **FLT_RADIX** is a power of 2, the result is correctly rounded.

An unsuffixed floating constant has type **double**. If suffixed by a floating suffix it has a type according to Table 6.3.

Table 6.3 — Suffixes for floating constants

Suffix	Type
f, F	float
l, L	long double
df, DF	_Decimal32
dd, DD	_Decimal64

dL, DL	_Decimal128
---------------	--------------------

The values of floating constants may be represented in greater range and precision than that required by the type (determined by the suffix); the types are not changed thereby. See 5.3.5.3.3 regarding evaluation formats.⁶⁸⁾

Floating constants of decimal floating type that have the same numerical value but different quantum exponents have distinguishable internal representations. The value shall be correctly rounded as specified in ISO/IEC 60559. The coefficient *c* and the quantum exponent *q* of a finite converted decimal floating-point number (see 5.3.5.3.4) are determined as follows:

- *q* is set to the value of *sign_{opt} digit-sequence* in the exponent part, if any, or to 0, otherwise.
- If there is a fractional constant, *q* is decreased by the number of digits to the right of the period and the period is removed to form a digit sequence.
- *c* is set to the value of the digit sequence (after any period has been removed).
- Rounding required because of insufficient precision or range in the type of the result will round *c* to the full precision available in the type, and will adjust *q* accordingly within the limits of the type, provided the rounding does not yield an infinity (in which case the result is an appropriately signed internal representation of infinity). If the full precision of the type would require *q* to be smaller than the minimum for the type, then *q* is pinned at the minimum and *c* is adjusted through the subnormal range accordingly, perhaps to zero.

Floating constants are converted to internal format as if at translation-time. The conversion of a floating constant shall not raise an exceptional condition or a floating-point exception at execution time. All floating constants of the same source form⁶⁹⁾ shall convert to the same internal format and, provided they are subject to the same translation-time rounding direction (either the default or a constant rounding mode set by an **FENV_ROUND** or **FENV_DEC_ROUND** pragma), to the same value.

EXAMPLE Following are floating constants of type **_Decimal64** and their values as triples (*s*, *c*, *q*). For **_Decimal64**, the precision (maximum coefficient length) is 16 and the quantum exponent range is $-398 \leq q \leq 369$.

0.dd	(+1, 0, 0)
0.00dd	(+1, 0, -2)
123.dd	(+1, 123, 0)
1.23E3dd	(+1, 123, 1)
1.23E+3dd	(+1, 123, 1)
12.3E+7dd	(+1, 123, 6)
12.0dd	(+1, 120, -1)
12.3dd	(+1, 123, -1)
0.00123dd	(+1, 123, -5)
1.23E-12dd	(+1, 123, -14)
1234.5E-4dd	(+1, 12345, -5)
0E+7dd	(+1, 0, 7)
12345678901234567890.dd	(+1, 1234567890123457, 4) assuming default rounding and DEC_EVAL_METHOD is 0 or 1 ⁷⁰⁾
1234E-400dd	(+1, 12, -398) assuming default rounding and DEC_EVAL_METHOD is 0 or 1
1234E-402dd	(+1, 0, -398) assuming default rounding and DEC_EVAL_METHOD is 0 or 1
1000.dd	(+1, 1000, 0)

⁶⁸⁾Hexadecimal floating constants can be used to obtain exact values in the semantic type that are independent of the evaluation format. Casts produce values in the semantic type, though depend on the rounding mode and can raise the inexact floating-point exception.

⁶⁹⁾**1.23**, **1.230**, **123e-2**, **123e-02**, and **1.23L** are all different source forms and thus can convert to a different internal format and value (though they can use the same internal format and value).

⁷⁰⁾That is, assuming the default translation rounding-direction mode is not changed by an **FENV_DEC_ROUND** pragma (7.6.4).

.0001dd	(+1, 1, -4)
1000.e0dd	(+1, 1000, 0)
.0001e0dd	(+1, 1, -4)
1000.0dd	(+1, 10000, -1)
0.0001dd	(+1, 1, -4)
1000.00dd	(+1, 100 000, -2)
00.0001dd	(+1, 1, -4)
001000.dd	(+1, 1000, 0)
001000.0dd	(+1, 10000, -1)
001000.00dd	(+1, 100 000, -2)
00.00dd	(+1, 0, -2)
00.dd	(+1, 0, 0)
.00dd	(+1, 0, -2)
00.00e-5dd	(+1, 0, -7)
00.e-5dd	(+1, 0, -5)
.00e-5dd	(+1, 0, -7)

Recommended practice

The implementation should produce a diagnostic message if a hexadecimal constant cannot be represented exactly in its evaluation format; the implementation should then proceed with the translation of the program.

The translation-time conversion of floating constants should match the execution-time conversion of character strings by library functions, such as **strtod**, given matching inputs suitable for both conversions, the same result format, and default execution-time rounding.⁷¹⁾

NOTE Floating constants do not include a sign and are arithmetically negated by the unary **-** operator (6.5.4.4) which arithmetically negates the rounded value of the constant. In contrast, the numeric conversion functions in the **strto** family (7.24.2.6, 7.24.2.7) can include the sign as part of the input value and convert and round the arithmetically negated input; implementations conforming to Annex F have this behavior. Negating before rounding and negating after rounding can yield different results, depending on the rounding direction and whether the results are correctly rounded. For example, the results are the same when both are correctly rounded using rounding to nearest or rounding toward zero, but the results are different when they are inexact and correctly rounded using rounding toward positive infinity or rounding toward negative infinity.

Conversions yielding exact results are not affected by the order of negating and rounding. For types with radix 10, decimal floating constants expressed within the precision and range of the evaluation format convert exactly. For types whose radix is a power of 2, hexadecimal floating constants expressed within the precision and range of the evaluation format convert exactly.

Forward references: preprocessing numbers (6.4.9), numeric conversion functions (7.24.2), the **strto** function family (7.24.2.6, 7.24.2.7).

6.4.5.4 Enumeration constants

Syntax

enumeration-constant:
identifier

Semantics

An identifier declared as an enumeration constant for an enumeration without a fixed underlying type has either type **int** or the enumerated type, as defined in 6.7.3.3. An identifier declared as an enumeration constant for an enumeration with a fixed underlying type has the associated enumerated type.

An enumeration constant can be used in an expression (or constant expression) wherever a value of an integer type can be used.

Forward references: enumeration specifiers (6.7.3.3).

⁷¹⁾The specification for the library functions recommends more accurate conversion than required for floating constants (see 7.24.2.6).

Syntax

$$encoding_prefix_{opt} \cdot c_char_sequence$$

u8 u U L

c-char
c-char-sequence c-char

any member of the source character set except
the single-quote ' , backslash \, or new-line character
escape-sequence

```
simple-escape-sequence
octal-escape-sequence
hexadecimal-escape-sequence
universal-character-name
```

```
\' \" \? \\
\ a \ b \ f \ n \ r \ t \ v
```

$$\begin{aligned} & \backslash \textit{octal-digit} \\ & \backslash \textit{octal-digit} \textit{octal-digit} \\ & \backslash \textit{octal-digit} \textit{octal-digit} \textit{octal-digit} \end{aligned}$$

`\x hexadecimal-digit`
hexadecimal-escape-sequence hexadecimal-digit

An *integer character constant* is a sequence of one or more multibyte characters enclosed in single-quotes, as in `'x'`. A *UTF-8 character constant* is the same, except prefixed by `u8`. A *wchar_t character constant* is prefixed by the letter `L`. A *UTF-16 character constant* is prefixed by the letter `u`. A *UTF-32 character constant* is prefixed by the letter `U`. Collectively, `wchar_t`, UTF-16, and UTF-32 character constants are called *wide character constants*. With a few exceptions detailed later, the elements of the sequence are any members of the source character set; they are mapped in an implementation-defined manner to members of the execution character set.

The single-quote `'`, the double-quote `"`, the question-mark `?`, the backslash `\`, and arbitrary integer values are representable according to Table 6.4:

Table 6.4 — Escape sequences

single quote '	\ '
double quote "	\ "
question mark ?	\ ?
backslash \	\\
octal character	\ <i>octal digits</i>
hexadecimal character	\ x <i>hexadecimal digits</i>

The double-quote " and question-mark ? are representable either by themselves or by the escape sequences \" and \?, respectively, but the single-quote ' and the backslash \ shall be represented, respectively, by the escape sequences \ ' and \\\.

The octal digits that follow the backslash in an octal escape sequence are taken to be part of the construction of a single character for an integer character constant or of a single wide character for a wide character constant. The numerical value of the octal integer so formed specifies the value of the desired character or wide character.

The hexadecimal digits that follow the backslash and the letter x in a hexadecimal escape sequence are taken to be part of the construction of a single character for an integer character constant or of a single wide character for a wide character constant. The numerical value of the hexadecimal integer so formed specifies the value of the desired character or wide character.

Each octal or hexadecimal escape sequence is the longest sequence of characters that can constitute the escape sequence.

In addition, characters not in the basic character set are representable by universal character names and certain non-graphic characters are representable by escape sequences consisting of the backslash \ followed by a lowercase letter: \a, \b, \f, \n, \r, \t, and \v.⁷²⁾

Constraints

The value of an octal or hexadecimal escape sequence shall be in the range of representable values for the corresponding type, as dictated by Table 6.5:

Table 6.5 — Types provided by prefixes

Prefix	Corresponding Type
none	unsigned char
u8	char8_t
L	the unsigned type corresponding to wchar_t
u	char16_t
U	char32_t

A UTF-8, UTF-16, or UTF-32 character constant shall not contain more than one character.⁷³⁾ The value shall be representable with a single UTF-8, UTF-16, or UTF-32 code unit, respectively.

Semantics

An integer character constant has type **int**. The value of an integer character constant containing a single character that maps to a single value in the literal encoding (6.2.9) is the numerical value of the representation of the mapped character in the literal encoding interpreted as an integer. The value of an integer character constant containing more than one character (e.g. **'ab'**), or containing a character or escape sequence that does not map to a single value in the literal encoding, is implementation-defined. If an integer character constant contains a single character or escape sequence, its value is the one that results when an object with type **char** whose value is that of the single character or escape sequence is converted to type **int**.

⁷²⁾The semantics of these characters were discussed in 5.3.3. If any other character follows a backslash, the result is not a token and a diagnostic is required. See "future language directions" (6.11.4).

⁷³⁾For example **u8'ab'** violates this constraint.

A UTF-8 character constant has type **char8_t**. If the UTF-8 character constant is not produced through a hexadecimal or octal escape sequence, the value of a UTF-8 character constant is equal to its ISO/IEC 10646 code point value, provided that the code point value can be encoded as a single UTF-8 code unit. Otherwise, the value of the UTF-8 character constant is the numeric value specified in the hexadecimal or octal escape sequence.

A UTF-16 character constant has type **char16_t** which is an unsigned integer type defined in the `<uchar.h>` header. If the UTF-16 character constant is not produced through a hexadecimal or octal escape sequence, the value of a UTF-16 character constant is equal to its ISO/IEC 10646 code point value, provided that the code point value can be encoded as a single UTF-16 code unit. Otherwise, the value of the UTF-16 character constant is the numeric value specified in the hexadecimal or octal escape sequence.

A UTF-32 character constant has type **char32_t** which is an unsigned integer type defined in the `<uchar.h>` header. If the UTF-32 character constant is not produced through a hexadecimal or octal escape sequence, the value of a UTF-32 character constant is equal to its ISO/IEC 10646 code point value, provided that the code point value can be encoded as a single UTF-32 code unit. Otherwise, the value of the UTF-32 character constant is the numeric value specified in the hexadecimal or octal escape sequence.

A **wchar_t** character constant has type **wchar_t**, an integer type defined in the `<stddef.h>` header. The value of a **wchar_t** character constant containing a single multibyte character that maps to a single member of the extended execution character set is the wide character corresponding to that multibyte character in the implementation-defined wide literal encoding (6.2.9). The value of a **wchar_t** character constant containing more than one multibyte character or a single multibyte character that maps to multiple members of the extended execution character set, or containing a multibyte character or escape sequence not represented in the extended execution character set, is implementation-defined.

EXAMPLE 1 The construction `'\0'` is commonly used to represent the null character.

EXAMPLE 2 Implementations that use eight bits for objects that have type **char** can furnish certain values in a variety of ways. In an implementation in which type **char** has the same range of values as **signed char**, the integer character constant `'\xFF'` has the value `-1`; if type **char** has the same range of values as **unsigned char**, the character constant `'\xFF'` has the value `+255`.

EXAMPLE 3 Even if eight bits are used for objects that have type **char**, the construction `'\x123'` specifies an integer character constant containing only one character, since a hexadecimal escape sequence is terminated only by a non-hexadecimal character. To specify an integer character constant containing the two characters whose values are `'\x12'` and `'3'`, the construction `'\0223'` can be used, since an octal escape sequence is terminated after three octal digits. (The value of this two-character integer character constant is implementation-defined.)

EXAMPLE 4 Even if 12 or more bits are used for objects that have type **wchar_t**, the construction `L'\1234'` specifies the implementation-defined value that results from the combination of the values `0123` and `'4'`.

Forward references: common definitions `<stddef.h>` (7.21), the **mbtowc** function (7.24.8.3), Unicode utilities `<uchar.h>` (7.30).

6.4.5.6 Predefined constants

Syntax

predefined-constant:
false
true
nullptr

Description

Some keywords represent constants of a specific value and type.

The keywords **false** and **true** are constants of type **bool** with a value of **0** for **false** and **1** for **true**.⁷⁴⁾

⁷⁴⁾The constants **false** and **true** promote to type **int**, see 6.3.2.1. When used for arithmetic, in translation phase 4 (5.2.1.2),

The keyword **nullptr** represents a null pointer constant. Details of its type are described in 7.21.3.

6.4.6 String literals

Syntax

string-literal:

*encoding-prefix*_{opt} " *s-char-sequence*_{opt} "

s-char-sequence:

s-char

s-char-sequence *s-char*

s-char:

any member of the source character set except
the double-quote " , backslash \ , or new-line character
escape-sequence

Constraints

If a sequence of adjacent string literal tokens includes prefixed string literal tokens, the prefixed tokens shall all have the same prefix.

Description

A *character string literal* is a sequence of zero or more multibyte characters enclosed in double-quotes, as in "**xyz**". A *UTF-8 string literal* is the same, except prefixed by **u8**. A **wchar_t** *string literal* is the same, except prefixed by **L**. A *UTF-16 string literal* is the same, except prefixed by **u**. A *UTF-32 string literal* is the same, except prefixed by **U**. Collectively, **wchar_t**, UTF-16, and UTF-32 string literals are called *wide string literals*.

The same considerations apply to each element of the sequence in a string literal as if it were in an integer character constant (for a character or UTF-8 string literal) or a wide character constant (for a wide string literal), except that the single-quote ' is representable either by itself or by the escape sequence \', but the double-quote " shall be represented by the escape sequence \".

Semantics

In translation phase 6 (5.2.1.2), the multibyte character sequences specified by any sequence of adjacent character and identically-prefixed string literal tokens are concatenated into a single multibyte character sequence. If any of the tokens has an encoding prefix, the resulting multibyte character sequence is treated as having the same prefix; otherwise, it is treated as a character string literal.

In translation phase 7 (5.2.1.2), a byte or code of value zero is appended to each multibyte character sequence that results from a string literal or literals.⁷⁵⁾ The multibyte character sequence is then used to initialize an array of static storage duration and length just sufficient to contain the sequence. For character string literals, the array elements have type **char**, and are initialized with the individual bytes of the multibyte character sequence corresponding to the literal encoding (6.2.9). For UTF-8 string literals, the array elements have type **char8_t**, and are initialized with the characters of the multibyte character sequence, as encoded in UTF-8. For wide string literals prefixed by the letter **L**, the array elements have type **wchar_t** and are initialized with the sequence of wide characters corresponding to the wide literal encoding. For wide string literals prefixed by the letter **u** or **U**, the array elements have type **char16_t** or **char32_t**, respectively, and are initialized sequence of wide characters corresponding to UTF-16 and UTF-32 encoded text, respectively. The value of a string literal containing a multibyte character or escape sequence not represented in the execution

they are signed values and the result of such arithmetic is consistent with the results of later translation phases.

⁷⁵⁾A string literal may not be a string (see 7.1.1), because a null character can be embedded in it by a \0 escape sequence.

character set is implementation-defined. Any hexadecimal escape sequence or octal escape sequence specified in a **u8**, **u**, or **U** string specifies a single **char8_t**, **char16_t**, or **char32_t** value and can result in the full character sequence not being valid UTF-8, UTF-16, or UTF-32.

It is unspecified whether these arrays are distinct provided their elements have the appropriate values. If the program attempts to modify such an array, the behavior is undefined.

EXAMPLE 1 This pair of adjacent character string literals

```
"\x12" "3"
```

produces a single character string literal containing the two characters whose values are `'\x12'` and `'3'`, because escape sequences are converted into single members of the execution character set just prior to adjacent string literal concatenation.

EXAMPLE 2 Each of the sequences of adjacent string literal tokens

```
"a" "b" L"c"
"a" L"b" "c"
L"a" "b" L"c"
L"a" L"b" L"c"
```

is equivalent to the string literal

```
L"abc"
```

Likewise, each of the sequences

```
"a" "b" u"c"
"a" u"b" "c"
u"a" "b" u"c"
u"a" u"b" u"c"
```

is equivalent to

```
u"abc"
```

Forward references: common definitions `<stddef.h>` (7.21), the `mbstowcs` function (7.24.9.2), Unicode utilities `<uchar.h>` (7.30).

6.4.7 Punctuators

Syntax

punctuator: one of

```
[ ] ( ) { } . ->
++ -- & * + - ~ !
/ % << >> < > <= >= == != ^ | && ||
? : :: ; ...
= *= /= %= += -= <<= >>= &= ^= |=
, # ##
<: :> <% %> %: %::%
```

Semantics

A punctuator is a symbol that has independent syntactic and semantic significance. Depending on context, it may specify an operation to be performed (which in turn may yield a value or a function designator, produce a side effect, or some combination thereof) in which case it is known as an *operator* (other forms of operator also exist in some contexts). An *operand* is an entity on which an operator acts.

In all aspects of the language, the six tokens⁷⁶⁾

<: :> <% %> %: %: %:

behave, respectively, the same as the six tokens

[] { } # ##

except for their spelling.⁷⁷⁾

Forward references: expressions (6.5.1), declarations (6.7), preprocessing directives (6.10), statements (6.8).

6.4.8 Header names

Syntax

header-name:

< *h-char-sequence* >
" *q-char-sequence* "

h-char-sequence:

h-char
h-char-sequence h-char

h-char:

any member of the source character set except
the new-line character and >

q-char-sequence:

q-char
q-char-sequence q-char

q-char:

any member of the source character set except
the new-line character and "

Semantics

The sequences in both forms of header names are mapped in an implementation-defined manner to headers or external source file names as specified in 6.10.3.

If the characters ' , \ , " , // , or /* occur in the sequence between the < and > delimiters, the behavior is undefined. Similarly, if the characters ' , \ , // , or /* occur in the sequence between the " delimiters, the behavior is undefined.⁷⁸⁾

Header name preprocessing tokens are recognized only within **#include** and **#embed** preprocessing directives, in **__has_include** and **__has_embed** expressions, as well as in implementation-defined locations within **#pragma** directives.⁷⁹⁾

EXAMPLE The following sequence of characters:

⁷⁶⁾These tokens are sometimes called "digraphs".

⁷⁷⁾Thus [and <: behave differently when "stringized" (see 6.10.5.3), but can otherwise be freely interchanged.

⁷⁸⁾Thus, sequences of characters that resemble escape sequences cause undefined behavior.

⁷⁹⁾For an example of a header name preprocessing token used in a **#pragma** directive, see 6.10.11.

```
0x3<1/a.h>1e2
#include <1/a.h>
#define const.member@$
```

forms the following sequence of preprocessing tokens (with each individual preprocessing token delimited by a { on the left and a } on the right).

```
{0x3}{<}{1}{/}{a}{.}{h}{>}{1e2}
#{include} {<1/a.h>}
#{define} {const}{.}{member}{@}{$}
```

Forward references: source file inclusion (6.10.3).

6.4.9 Preprocessing numbers

Syntax

pp-number:

digit
. *digit*
pp-number *identifier-continue*
pp-number ' *digit*
pp-number ' *nondigit*
pp-number e *sign*
pp-number E *sign*
pp-number p *sign*
pp-number P *sign*
pp-number .

Description

A preprocessing number begins with a digit optionally preceded by a period (.) and may be followed by valid identifier characters and the character sequences **e+**, **e-**, **E+**, **E-**, **p+**, **p-**, **P+**, or **P-**.

Preprocessing number tokens lexically include all floating and integer constant tokens.

Semantics

A preprocessing number does not have type or a value; it acquires both after a successful conversion (as part of translation phase 7 (5.2.1.2)) to a floating constant token or an integer constant token.

6.4.10 Comments

Except within a character constant, a string literal, or a comment, the characters `/*` introduce a comment. The contents of such a comment are examined only to identify multibyte characters and to find the characters `*/` that terminate it.⁸⁰⁾

Except within a character constant, a string literal, or a comment, the characters `//` introduce a comment that includes all multibyte characters up to, but not including, the next new-line character. The contents of such a comment are examined only to identify multibyte characters and to find the terminating new-line character.

EXAMPLE

```
"a//b"           // four-character string literal
#include "//e"     // undefined behavior
// */           // comment, not syntax error
f = g/**//h;      // equivalent to f = g / h;
//\
i();              // part of a two-line comment
/\
```

⁸⁰⁾Thus, `/* ... */` comments do not nest.

```
/ j();           // part of a two-line comment
#define glue(x,y) x##y
glue(/,/ ) k();  // syntax error, not comment
/**/ l();        // equivalent to l();
m = n/**/o
    + p;          // equivalent to m = n + p;
```

6.5 Expressions

6.5.1 General

An *expression* is a sequence of operators and operands that specifies computation of a value, or that designates an object or a function, or that generates side effects, or that performs a combination thereof. The value computations of the operands of an operator are sequenced before the value computation of the result of the operator.

If a side effect on a scalar object is unsequenced relative to either a different side effect on the same scalar object or a value computation using the value of the same scalar object, the behavior is undefined. If there are multiple allowable orderings of the subexpressions of an expression, the behavior is undefined if such an unsequenced side effect occurs in any of the orderings.⁸¹⁾

The grouping of operators and operands is indicated by the syntax.⁸²⁾ Except as specified later, side effects and value computations of subexpressions are unsequenced.⁸³⁾

Some operators (the unary operator `~`, and the binary operators `<<`, `>>`, `&`, `^`, and `|`, collectively described as *bitwise operators*) are required to have operands that have integer type. These operators yield values that depend on the internal representations of integers, and have implementation-defined and undefined aspects for signed types.

If an *exceptional condition* occurs during the evaluation of an expression (that is, if the result is not mathematically defined or not in the range of representable values for its type), the behavior is undefined.

The *effective type* of an object for an access to its stored value is the declared type of the object, if any.⁸⁴⁾ If a value is stored into an object having no declared type through an lvalue having a type that is not a non-atomic character type, then the type of the lvalue becomes the effective type of the object for that access and for subsequent accesses that do not modify the stored value. If a value is copied into an object having no declared type using `memcpy` or `memmove`, or is copied as an array of character type, then the effective type of the modified object for that access and for subsequent accesses that do not modify the value is the effective type of the object from which the value is copied, if it has one. For all other accesses to an object having no declared type, the effective type of the object is simply the type of the lvalue used for the access.

An object shall have its stored value accessed only by an lvalue expression that has one of the following types:⁸⁵⁾

- a type compatible with the effective type of the object,
- a qualified version of a type compatible with the effective type of the object,

⁸¹⁾This paragraph renders undefined statement expressions such as

```
i = ++i + 1;
a[i++] = i;
```

while allowing

```
i = i + 1;
a[i] = i;
```

⁸²⁾The syntax specifies the precedence of operators in the evaluation of an expression, which is the same as the order of the major subclauses of this subclause, highest precedence first. Thus, for example, the expressions allowed as the operands of the binary `+` operator (6.5.7) are those expressions defined in 6.5.2 through 6.5.7. The exceptions are cast expressions (6.5.5) as operands of unary operators (6.5.4), and an operand contained between any of the following pairs of operators: grouping parentheses `()` (6.5.2), generic selection parentheses `()` (6.5.2.1), subscripting brackets `code[]` (6.5.3.2), function-call parentheses `()` (6.5.3.3), and the conditional operator `?:` (6.5.16).

Within each major subclause, the operators have the same precedence. Left- or right-associativity is indicated in each subclause by the syntax for the expressions discussed therein.

⁸³⁾In an expression that is evaluated more than once during the execution of a program, unsequenced and indeterminately sequenced evaluations of its subexpressions can be performed inconsistently in different evaluations.

⁸⁴⁾Allocated objects have no declared type.

⁸⁵⁾The intent of this list is to specify those circumstances in which an object can or cannot be aliased.

- the signed or unsigned type compatible with the underlying type of the effective type of the object,
- the signed or unsigned type compatible with a qualified version of the underlying type of the effective type of the object,
- an aggregate or union type that includes one of the aforementioned types among its members (including, recursively, a member of a subaggregate or contained union), or
- a character type.

A floating expression may be *contracted*, that is, evaluated as though it were a single operation, thereby omitting rounding errors implied by the source code and the expression evaluation method.⁸⁶⁾

The **FP_CONTRACT** pragma in `<math.h>` provides a way to disallow contracted expressions. Otherwise, whether and how expressions are contracted is implementation-defined.⁸⁷⁾

Operators involving decimal floating types are evaluated according to the semantics of ISO/IEC 60559, including production of results with the preferred quantum exponent as specified in ISO/IEC 60559.

Forward references: the **FP_CONTRACT** pragma (7.12.3), copying functions (7.26.2).

6.5.2 Primary expressions

Syntax

primary-expression:

identifier
constant
string-literal
(expression)
generic-selection

Constraints

The identifier in an identifier primary expression shall have a visible declaration as an ordinary identifier that declares an object or a function.⁸⁸⁾

Semantics

An identifier primary expression designating an object is an lvalue. An identifier primary expression designating a function is a function designator.

A constant is a primary expression. Its type depends on its form and value, as detailed in 6.4.5.

A string literal is a primary expression. It is an lvalue with type as detailed in 6.4.6.

A *parenthesized expression* is a primary expression. Its type, value, and semantics are identical to those of the unparenthesized expression.

A generic selection is a primary expression. Its type, value, and semantics depend on the selected generic association, as detailed in the following subclause.

Forward references: declarations (6.7).

⁸⁶⁾The intermediate operations in the contracted expression are evaluated as if to infinite range and precision, while the final operation is rounded to the format determined by the expression evaluation method. A contracted expression can also omit the raising of floating-point exceptions.

⁸⁷⁾This license is specifically intended to allow implementations to exploit fast machine instructions that combine multiple C operators. As contractions potentially undermine predictability, and can even decrease accuracy for containing expressions, their use needs to be well-defined and clearly documented.

⁸⁸⁾An identifier designating an enumeration constant is a primary expression through the constant production, not the identifier production.

6.5.2.1 Generic selection

Syntax

generic-selection:
 _Generic (*assignment-expression* , *generic-assoc-list*)
generic-assoc-list:
 generic-association
 generic-assoc-list , *generic-association*
generic-association:
 type-name : *assignment-expression*
 default : *assignment-expression*

Constraints

A generic selection shall have no more than one **default** generic association. The type name in a generic association shall specify a complete object type other than a variably modified type. No two generic associations in the same generic selection shall specify compatible types. The type of the controlling expression is the type of the expression as if it had undergone an lvalue conversion,⁸⁹⁾ array to pointer conversion, or function to pointer conversion. That type shall be compatible with at most one of the types named in the generic association list. If a generic selection has no **default** generic association, its controlling expression shall have type compatible with exactly one of the types named in its generic association list.

Semantics

The controlling expression of a generic selection is not evaluated. If a generic selection has a generic association with a type name that is compatible with the type of the controlling expression, then the result expression of the generic selection is the expression in that generic association. Otherwise, the result expression of the generic selection is the expression in the **default** generic association. None of the expressions from any other generic association of the generic selection is evaluated.

The type and value of a generic selection are identical to those of its result expression. It is an lvalue, a function designator, or a void expression if its result expression is, respectively, an lvalue, a function designator, or a void expression.

EXAMPLE A **cbrt** type-generic macro can be implemented as follows:

```
#define cbrt(X) _Generic((X),  
                        long double: cbrtL,  
                        default: cbrt,  
                        float: cbrtF  
                        )(X)
```

7.27 shows how such a macro can be implemented with the required rounding properties.

6.5.3 Postfix operators

6.5.3.1 General

Syntax

postfix-expression:
 primary-expression
 postfix-expression [*expression*]
 postfix-expression (*argument-expression-list*_{opt})
 postfix-expression . *identifier*
 postfix-expression -> *identifier*
 postfix-expression ++
 postfix-expression --
 compound-literal

⁸⁹⁾An lvalue conversion drops type qualifiers.

argument-expression-list:

assignment-expression

argument-expression-list , *assignment-expression*

6.5.3.2 Array subscripting

Constraints

One of the expressions shall have type “pointer to complete object *type*”, the other expression shall have integer type, and the result has type “*type*”.

Semantics

A postfix expression followed by an expression in square brackets `[]` is a subscripted designation of an element of an array object. The definition of the subscript operator `[]` is that **E1[E2]** is identical to `((*(E1)+(E2)))`. Because of the conversion rules that apply to the binary `+` operator, if **E1** is an array object (equivalently, a pointer to the initial element of an array object) and **E2** is an integer, **E1[E2]** designates the **E2**-th element of **E1** (counting from zero).

Successive subscript operators designate an element of a multidimensional array object. If **E** is an *n*-dimensional array ($n \geq 2$) with dimensions $i \times j \times \dots \times k$, then **E** (used as other than an lvalue) is converted to a pointer to an $(n - 1)$ -dimensional array with dimensions $j \times \dots \times k$. If the unary `*` operator is applied to this pointer explicitly, or implicitly as a result of subscripting, the result is the referenced $(n - 1)$ -dimensional array, which itself is converted into a pointer if used as other than an lvalue. It follows from this that arrays are stored in row-major order (last subscript varies fastest).

EXAMPLE The following snippet has an array object defined by the declaration:

```
int x[3][5];
```

Here **x** is a 3×5 array of objects of type `int`; more precisely, **x** is an array of three element objects, each of which is an array of five objects of type `int`. In the expression **x[i]**, which is equivalent to `((*(x)+(i)))`, **x** is first converted to a pointer to the initial array of five objects of type `int`. Then **i** is adjusted according to the type of **x**, which conceptually entails multiplying **i** by the size of the object to which the pointer points, namely an array of five `int` objects. The results are added and indirection is applied to yield an array of five objects of type `int`. When used in the expression **x[i][j]**, that array is in turn converted to a pointer to the first of the objects of type `int`, so **x[i][j]** yields an `int`.

Forward references: additive operators (6.5.7), address and indirection operators (6.5.4.3), array declarators (6.7.7.3).

6.5.3.3 Function calls

Constraints

The expression that denotes the called function⁹⁰⁾ shall have type pointer to function returning `void` or returning a complete object type other than an array type.

The number of arguments shall agree with the number of parameters. Each argument shall have a type such that its value may be assigned to an object with the unqualified version of the type of its corresponding parameter

Semantics

A postfix expression followed by parentheses `()` containing a possibly empty, comma-separated list of expressions is a function call. The postfix expression denotes the called function. The list of expressions specifies the arguments to the function.

An argument can be an expression of any complete object type. In preparing for the call to a function, the arguments are evaluated, and each parameter is assigned the value of the corresponding argument.⁹¹⁾

⁹⁰⁾Most often, this is the result of converting an identifier that is a function designator.

⁹¹⁾A function can change the values of its parameters, but these changes cannot affect the values of the arguments. On the other hand, it is possible to pass a pointer to an object, and the function can then change the value of the object pointed to. A

If the expression that denotes the called function has type pointer to function returning an object type, the function call expression has the same type as that object type, and has the value determined as specified in 6.8.7.5. Otherwise, the function call has type **void**.

The arguments are implicitly converted, as if by assignment, to the types of the corresponding parameters, taking the type of each parameter to be the unqualified version of its declared type. The ellipsis notation in a function prototype declarator causes argument type conversion to stop after the last declared parameter, if present. The integer promotions are performed on each trailing argument, and trailing arguments that have type **float** are promoted to **double**. These are called the *default argument promotions*. No other conversions are performed implicitly.

If the function is defined with a type that is not compatible with the type (of the expression) pointed to by the expression that denotes the called function, the behavior is undefined.

There is a sequence point after the evaluations of the function designator and the actual arguments but before the actual call. Every evaluation in the calling function (including other function calls) that is not otherwise specifically sequenced before or after the execution of the body of the called function is indeterminately sequenced with respect to the execution of the called function.⁹²⁾

Recursive function calls shall be permitted, both directly and indirectly through any chain of other functions.

EXAMPLE In the function call

```
(*pf[f1()]) (f2(), f3() + f4())
```

the functions **f1**, **f2**, **f3**, and **f4** can be called in any order. All side effects are completed before the function pointed to by **pf[f1()]** is called.

Forward references: function declarators (6.7.7.4), function definitions (6.9.2), the **return** statement (6.8.7.5), simple assignment (6.5.17.2).

6.5.3.4 Structure and union members

Constraints

The first operand of the **.** operator shall have an atomic, qualified, or unqualified structure or union type, and the second operand shall name a member of that type.

The first operand of the **->** operator shall have type “pointer to atomic, qualified, or unqualified structure” or “pointer to atomic, qualified, or unqualified union”, and the second operand shall name a member of the type pointed to.

Semantics

A postfix expression followed by the **.** operator and an identifier designates a member of a structure or union object. The value is that of the named member,⁹³⁾ and is an lvalue if the first expression is an lvalue. If the first expression has qualified type, the result has the so-qualified version of the type of the designated member.

A postfix expression followed by the **->** operator and an identifier designates a member of a structure or union object. The value is that of the named member of the object to which the first expression points, and is an lvalue.⁹⁴⁾ If the first expression is a pointer to a qualified type, the result has the so-qualified version of the type of the designated member.

Accessing a member of an atomic structure or union object results in undefined behavior.⁹⁵⁾

parameter declared to have array or function type is adjusted to have a pointer type as described in 6.7.7.4.

⁹²⁾In other words, function executions do not interleave with each other.

⁹³⁾If the member used to read the contents of a union object is not the same as the member last used to store a value in the object the appropriate part of the object representation of the value is reinterpreted as an object representation in the new type as described in 6.2.6 (a process sometimes called type punning). This can possibly be a non-value representation.

⁹⁴⁾If **&E** is a valid pointer expression (where **&** is the address of operator, which generates a pointer to its operand), the expression **(&E) ->MOS** is the same as **E.MOS**.

⁹⁵⁾For example, a data race would occur if access to the entire structure or union in one thread conflicts with access to a member from another thread, where at least one access is a modification. Members can be safely accessed using a non-atomic object which is assigned to or from the atomic object.

EXAMPLE 1 If $\mathbf{f}$ is a function returning a structure or union, and $\mathbf{x}$ is a member of that structure or union, $\mathbf{f}().\mathbf{x}$ is a valid postfix expression but is not an lvalue.

```
struct s { int i; const int ci; };
struct s s;
const struct s cs;
volatile struct s vs;
```

```
s.i      int
s.ci     const int
cs.i     const int
cs.ci    const int
vs.i     volatile int
vs.ci    volatile const int
```

```

union {
    struct {
        int    alltypes;
    } n;
    struct {
        int    type;
        int    intnode;
    } ni;
    struct {
        int    type;
        double doublenode;
    } nf;
} u;

u.nf.type = 1;
u.nf.doublenode = 3.14;
/* ... */
if (u.n.alltypes == 1)
    if (sin(u.nf.doublenode) == 0.0)
        /* ... */

```

```

struct t1 { int m; };
struct t2 { int m; };
int f(struct t1 *p1, struct t2 *p2)
{
    if (p1->m < 0)
        p2->m = -p2->m;
    return p1->m;
}

int g()
{
    union {
        struct t1 s1;
        struct t2 s2;
    };
}

```

```
    } u;  
    /* ... */  
    return f(&u.s1, &u.s2);  
}
```

Forward references: address and indirection operators (6.5.4.3), structure and union specifiers (6.7.3.2).

6.5.3.5 Postfix increment and decrement operators

Constraints

The operand of the postfix increment or decrement operator shall have atomic, qualified, or unqualified real or pointer type, and shall be a modifiable lvalue.

Semantics

The result of the postfix ++ operator is the value of the operand. As a side effect, the value of the operand object is incremented (that is, the value 1 of the appropriate type is added to it). See the discussions of additive operators and compound assignment for information on constraints, types, and conversions and the effects of operations on pointers. The value computation of the result is sequenced before the side effect of updating the stored value of the operand. With respect to an indeterminately sequenced function call, the operation of postfix ++ is a single evaluation. Postfix ++ on an object with atomic type is a read-modify-write operation with **memory_order_seq_cst** memory order semantics.⁹⁶⁾

The postfix -- operator is analogous to the postfix ++ operator, except that the value of the operand is decremented (that is, the value 1 of the appropriate type is subtracted from it).

Forward references: additive operators (6.5.7), compound assignment (6.5.17.3).

6.5.3.6 Compound literals

Syntax

```
compound-literal:  
    ( storage-class-specifiersopt type-name ) braced-initializer  
  
storage-class-specifiers:  
    storage-class-specifier  
    storage-class-specifiers storage-class-specifier
```

Constraints

The type name shall specify a complete object type or an array of unknown size, but not a variable length array type.

All the constraints for initializer lists in 6.7.11 also apply to compound literals.

If the compound literal is associated with file scope or block scope (see 6.2.1) the storage-class specifiers **SC** (possibly empty),⁹⁷⁾ type name **T**, and initializer list, if any, shall be such that they are

⁹⁶⁾Where a pointer to an atomic object can be formed and **E** has integer type, **E++** is equivalent to the following code sequence where *T* is the type of **E**:

```
T *addr = &E;  
T old = *addr;  
T new;  
do {  
    new = old + 1;  
} while (!atomic_compare_exchange_strong(addr, &old, new));
```

with **old** being the result of the operation.
Special care is necessary if **E** has floating type; see 6.5.17.3.
⁹⁷⁾If the storage-class specifiers contain the same storage-class specifier more than once, the following constraint is violated.

valid specifiers for an object definition in file scope or block scope, respectively, of the following form,

```
SC typeof(T) ID = { IL };
```

where **ID** is an identifier that is unique for the whole program and where **IL** is a (possibly empty) initializer list with nested structure, designators, values and types as the initializer list of the compound literal. All the constraints for storage-class specifiers in 6.7.2 also apply correspondingly to compound literals. If the compound literal is associated with function prototype scope, constraints as if in block scope apply.

Semantics

For a *compound literal* associated with function prototype scope:

- the type is determined as if in block scope and no object is created;
- if it is a compound literal constant it is evaluated at translation time;
- if it is not a compound literal constant, neither the compound literal as a whole nor any of the initializers are evaluated.

Otherwise, a compound literal provides access to an unnamed object whose value, type, storage duration, initializer, and other properties are as if given by the definition syntax in the constraints.

If the storage duration is automatic, the lifetime of the instance of the unnamed object is the current execution of the enclosing block.⁹⁸⁾

If the storage-class specifiers are absent or contain **constexpr**, **static**, **register**, or **thread_local** the behavior is as if the object were declared and initialized in the corresponding scope with these storage-class specifiers; if another storage-class specifier is present, the behavior is undefined. If the storage-class specifier **constexpr** is present, the initializer is evaluated at translation time. Otherwise, if the storage duration is automatic, the initializer is evaluated at each evaluation of the compound literal; if the storage duration is static or thread the initializer is (as if) evaluated once prior to program startup.

The value of the compound literal is that of an lvalue corresponding to the unnamed object.

All the semantic rules for initializer lists in 6.7.11 also apply to compound literals.⁹⁹⁾

String literals, and compound literals with **const**-qualified types, including those specified with **constexpr**, are not required to designate distinct objects.¹⁰⁰⁾

EXAMPLE 1 The following 2 functions can be used as an example:

```
int f(int*);
int g(char * para[f((int[27]){ 0, })]) {
    /* ... */
    return 0;
}
```

Here, each call to **g** creates an unnamed object of type **int**[27] to determine the variably modified type of **para** for the duration of the call. During that determination, a pointer to the object is passed into a call to the function **f**. If a pointer to the object is kept by **f**, access to that object is possible during the whole execution of the call to **g**. The lifetime of the object ends with the end of the call to **g**; for any access after that, the behavior is undefined.

EXAMPLE 2 The file scope definition

⁹⁸⁾This differs from a cast expression. For example, a cast specifies a conversion to scalar types or **void** only, and the result of a cast expression is not an lvalue.

⁹⁹⁾For example, subobjects without explicit initializers are initialized to zero.

¹⁰⁰⁾This allows implementations to share storage for string literals and constant compound literals with the same or overlapping representations.


```
int *p = (int []){2, 4};
```

initializes **p** to point to the first element of an array of two **int**s, the first having the value two and the second having the value four. The expressions in this compound literal are expected to be constant. The unnamed object has static storage duration.

EXAMPLE 3 In contrast, in

```
void f(void)
{
    int *p;
    /*...*/
    p = (int [2]){*p};
    /*...*/
}
```

p is assigned the address of the first element of an array of two **int**s, the first having the value previously pointed to by **p** and the second, zero. The initializer expression in this compound literal is not required to be constant. The unnamed object has automatic storage duration.

EXAMPLE 4 Initializers with designations can be combined with compound literals. Structure objects created using compound literals can be passed to functions without depending on member order:

```
drawline((struct point){.x=1, .y=1},
        (struct point){.x=3, .y=4});
```

Or, if **drawline** instead expected pointers to **struct point**:

```
drawline(&(struct point){.x=1, .y=1},
        &(struct point){.x=3, .y=4});
```

EXAMPLE 5 A read-only compound literal can be specified through constructions like:

```
(const float []){1e0, 1e1, 1e2, 1e3, 1e4, 1e5, 1e6}
```

EXAMPLE 6 The following three expressions have different meanings:

```
"/tmp/fileXXXXXX"  
(char []){"/tmp/fileXXXXXX"}  
(const char []){"/tmp/fileXXXXXX"}
```

The first always has static storage duration and has type array of `char`, but can be modifiable; the last two have automatic storage duration when they occur within the body of a function, and the first of these two is modifiable.

EXAMPLE 7 Like string literals, const-qualified compound literals can be placed into read-only memory and can even be shared. For example,

```
(const char []){"abc"} == "abc"
```

can yield 1 if the literals' storage is shared.

EXAMPLE 8 Since compound literals are unnamed, a single compound literal cannot specify a circularly linked object. For example, there is no way to write a self-referential compound literal that can be used as the function argument in place of the named object `endless_zeros` in the following snippet:

```
struct int_list { int car; struct int_list *cdr; };
struct int_list endless_zeros = {0, &endless_zeros};
eval(endless_zeros);
```

EXAMPLE 9 Each compound literal creates only a single object in a given scope:

```

struct s { int i; };

int f (void)
{
    struct s *p = 0, *q;
    int j = 0;

    again:
        q = p, p = &((struct s){ j++ });
        if (j < 2) goto again;

    return p == q && q->i == 1;
}

```

The function `f()` always returns the value 1.

If an iteration statement were used instead of an explicit `goto` and a label, the lifetime of the unnamed object would be the body of the loop only, and on entry next time around `p` would have indeterminate representation, which would result in undefined behavior.

Forward references: type names (6.7.8), initialization (6.7.11).

6.5.4 Unary operators

6.5.4.1 General

Syntax

unary-expression:

```

    postfix-expression
    ++ unary-expression
    -- unary-expression
    unary-operator cast-expression
    sizeof unary-expression
    sizeof ( type-name )
    alignof ( type-name )

```

unary-operator: one of

```

& * + - ~ !

```

6.5.4.2 Prefix increment and decrement operators

Constraints

The operand of the prefix increment or decrement operator shall have atomic, qualified, or unqualified real or pointer type, and shall be a modifiable lvalue.

Semantics

The value of the operand of the prefix `++` operator is incremented. The result is the new value of the operand after incrementation. The expression `++E` is equivalent to `(E+=1)`, where the value 1 is of the appropriate type. See the discussions of additive operators and compound assignment for information on constraints, types, side effects, and conversions and the effects of operations on pointers.

The prefix `--` operator is analogous to the prefix `++` operator, except that the value of the operand is decremented.

Forward references: additive operators (6.5.7), compound assignment (6.5.17.3).

6.5.4.3 Address and indirection operators

Constraints

The operand of the unary **&** operator shall be either a function designator, the result of a **[]** or unary ***** operator, or an lvalue that designates an object that is not a bit-field and is not declared with the **register** storage-class specifier.

The operand of the unary ***** operator shall have pointer type.

Semantics

The unary **&** operator yields the address of its operand. If the operand has type “*type*”, the result has type “*pointer to type*”. If the operand is the result of a unary ***** operator, neither that operator nor the **&** operator is evaluated and the result is as if both were omitted, except that the constraints on the operators still apply and the result is not an lvalue. Similarly, if the operand is the result of a **[]** operator, neither the **&** operator nor the unary ***** that is implied by the **[]** is evaluated and the result is as if the **&** operator were removed and the **[]** operator were changed to a **+** operator. Otherwise, the result is a pointer to the object or function designated by its operand.

The unary ***** operator denotes indirection. If the operand points to a function, the result is a function designator; if it points to an object, the result is an lvalue designating the object. If the operand has type “*pointer to type*”, the result has type “*type*”. If an invalid value has been assigned to the pointer, the behavior of the unary ***** operator is undefined.¹⁰¹⁾

Forward references: storage-class specifiers (6.7.2), structure and union specifiers (6.7.3.2).

6.5.4.4 Unary arithmetic operators

Constraints

The operand of the unary **+** or **-** operator shall have arithmetic type; of the **~** operator, integer type; of the **!** operator, scalar type.

Semantics

The result of the unary **+** operator is the value of its (promoted) operand. The integer promotions are performed on the operand, and the result has the promoted type.

The result of the unary **-** operator is the negative of its (promoted) operand. The integer promotions are performed on the operand, and the result has the promoted type.

The result of the **~** operator is the bitwise complement of its (promoted) operand (that is, each bit in the result is set if and only if the corresponding bit in the converted operand is not set). The integer promotions are performed on the operand, and the result has the promoted type. If the promoted type is an unsigned type, the expression **~E** is equivalent to the maximum value representable in that type minus **E**.

The result of the logical negation operator **!** is 0 if the value of its operand compares unequal to 0, 1 if the value of its operand compares equal to 0. The result has type **int**. The expression **!E** is equivalent to **(0==E)**.

6.5.4.5 The **sizeof** and **alignof** operators

Constraints

The **sizeof** operator shall not be applied to an expression that has function type or an incomplete type, to the parenthesized name of such a type, or to an expression that designates a bit-field member. The **alignof** operator shall not be applied to a function type or an incomplete type.

¹⁰¹⁾ Thus, **&*E** is equivalent to **E** (even if **E** is a null pointer), and **&(E1[E2])** to **((E1)+(E2))**. It is always true that if **E** is a function designator or an lvalue that is a valid operand of the unary **&** operator, ***&E** is a function designator or an lvalue equal to **E**. If ***P** is an lvalue and **T** is the name of an object pointer type, ***(T)P** is an lvalue that has a type compatible with that to which **T** points.

Among the invalid values for dereferencing a pointer by the unary ***** operator are a null pointer, an address inappropriately aligned for the type of object pointed to, and the address of an object after the end of its lifetime.

Semantics

The **sizeof** operator yields the size (in bytes) of its operand, which can be an expression or the parenthesized name of a type. The size is determined from the type of the operand. The result is an integer. If the type of the operand is a variable length array type, the operand is evaluated; otherwise, the operand is not evaluated and the result is an integer constant.

The **alignof** operator yields the alignment requirement of its operand type. The operand is not evaluated and the result is an integer constant expression. When applied to an array type, the result is the alignment requirement of the element type.

When **sizeof** is applied to an operand that has type **char**, **unsigned char**, or **signed char**, (or a qualified version thereof) the result is 1. When applied to an operand that has array type, the result is the total number of bytes in the array.¹⁰²⁾ When applied to an operand that has structure or union type, the result is the total number of bytes in such an object, including internal and trailing padding.

The value of the result of both operators is implementation-defined, and its type (an unsigned integer type) is **size_t**, defined in `<stddef.h>` (and other headers).

EXAMPLE 1 A principal use of the **sizeof** operator is in communication with routines such as storage allocators and I/O systems. A storage-allocation function can accept a size (in bytes) of an object to allocate and return a pointer to **void**. For example:

```
extern void *alloc(size_t);
double *dp = alloc(sizeof *dp);
```

The implementation of the **alloc** function presumably ensures that its return value is aligned suitably for conversion to a pointer to **double**.

EXAMPLE 2 Another use of the **sizeof** operator is to compute the number of elements in an array:

```
sizeof array / sizeof array[0]
```

EXAMPLE 3 In this example, the size of a variable length array is computed and returned from a function:

```
#include <stddef.h>

size_t fsize3(int n)
{
    char b[n+3];    // variable length array
    return sizeof b; // execution time sizeof
}

int main(void)
{
    size_t size;
    size = fsize3(10); // fsize3 returns 13
    return 0;
}
```

Forward references: common definitions `<stddef.h>` (7.21), declarations (6.7), structure and union specifiers (6.7.3.2), type names (6.7.8), array declarators (6.7.7.3).

6.5.5 Cast operators

Syntax

cast-expression:

```
unary-expression
( type-name ) cast-expression
```

¹⁰²⁾When applied to a parameter declared to have array or function type, the **sizeof** operator yields the size of the adjusted (pointer) type (see 6.9.2).

Constraints

Unless the type name specifies a void type, the type name shall specify atomic, qualified, or unqualified scalar type, and the operand shall have scalar type.

Conversions that involve pointers, other than where permitted by the constraints of 6.5.17.2, shall be specified by means of an explicit cast.

A pointer type shall not be converted to any floating type. A floating type shall not be converted to any pointer type. The type **nullptr_t** shall not be converted to any type other than **void**, **bool** or a pointer type. If the target type is **nullptr_t**, the cast expression shall be a null pointer constant or have type **nullptr_t**.

Semantics

Size expressions and typeof operators contained in a type name used with a cast operator are evaluated whenever the cast expression is evaluated.

Preceding an expression by a parenthesized type name converts the value of the expression to the unqualified, non-atomic version of the named type. This construction is called a *cast*.¹⁰³⁾ A cast that specifies no conversion has no effect on the type or value of an expression.

If the value of the expression is represented with greater range or precision than required by the type named by the cast (6.3.2.8), then the cast specifies a conversion even if the type of the expression is the same as the named type and removes any extra range and precision.

Forward references: equality operators (6.5.10), function declarators (6.7.7.4), simple assignment (6.5.17.2), type names (6.7.8).

6.5.6 Multiplicative operators

Syntax

multiplicative-expression:

```
cast-expression
multiplicative-expression * cast-expression
multiplicative-expression / cast-expression
multiplicative-expression % cast-expression
```

Constraints

Each of the operands shall have arithmetic type. The operands of the % operator shall have integer type.

If either operand has decimal floating type, the other operand shall not have standard floating type, complex type, or imaginary type.

Semantics

The usual arithmetic conversions are performed on the operands.

The result of the binary * operator is the product of the operands.

The result of the / operator is the quotient from the division of the first operand by the second; the result of the % operator is the remainder. In both operations, if the value of the second operand is zero, the behavior is undefined.

When integers are divided, the result of the / operator is the algebraic quotient with any fractional part discarded.¹⁰⁴⁾ If the quotient **a/b** is representable, the expression **(a/b)*b + a%b** shall equal **a**; otherwise, the behavior of both **a/b** and **a%b** is undefined.

¹⁰³⁾ A cast does not yield an lvalue.

¹⁰⁴⁾ This is often called “truncation toward zero”.

6.5.7 Additive operators

Syntax

additive-expression:

multiplicative-expression

additive-expression + *multiplicative-expression*

additive-expression - *multiplicative-expression*

Constraints

For addition, either both operands shall have arithmetic type, or one operand shall be a pointer to a complete object type and the other shall have integer type. (Incrementing is equivalent to adding 1.)

For subtraction, one of the following shall hold:

- both operands have arithmetic type;
- both operands are pointers to qualified or unqualified versions of compatible complete object types; or
- the left operand is a pointer to a complete object type and the right operand has integer type.

(Decrementing is equivalent to subtracting 1.)

If either operand has decimal floating type, the other operand shall not have standard floating type, complex type, or imaginary type.

Semantics

If both operands have arithmetic type, the usual arithmetic conversions are performed on them.

The result of the binary + operator is the sum of the operands.

The result of the binary - operator is the difference resulting from the subtraction of the second operand from the first.

For the purposes of these operators, a pointer to an object that is not an element of an array behaves the same as a pointer to the first element of an array of length one with the type of the object as its element type.

When an expression that has integer type is added to or subtracted from a pointer, the result has the type of the pointer operand. If the pointer operand points to an element of an array object, and the array is large enough, the result points to an element offset from the original element such that the difference of the subscripts of the resulting and original array elements equals the integer expression. In other words, if the expression **P** points to the *i*-th element of an array object, the expressions **(P)+N** (equivalently, **N+(P)**) and **(P)-N** (where **N** has the value *n*) point to, respectively, the *i* + *n*-th and *i* - *n*-th elements of the array object, provided they exist. Moreover, if the expression **P** points to the last element of an array object, the expression **(P)+1** points one past the last element of the array object, and if the expression **Q** points one past the last element of an array object, the expression **(Q)-1** points to the last element of the array object. If the pointer operand and the result do not point to elements of the same array object or one past the last element of the array object, the behavior is undefined. If the addition or subtraction produces an overflow, the behavior is undefined. If the result points one past the last element of the array object, it shall not be used as the operand of a unary * operator that is evaluated.

When two pointers are subtracted, both shall point to elements of the same array object, or one past the last element of the array object; the result is the difference of the subscripts of the two array elements. The size of the result is implementation-defined, and its type (a signed integer type) is **ptrdiff_t** defined in the <stddef.h> header. If the result is not representable in an object of that type, the behavior is undefined. In other words, if the expressions **P** and **Q** point to, respectively, the *i*-th and *j*-th elements of an array object, the expression **(P) - (Q)** has the value *i* - *j* provided the value fits in an object of type **ptrdiff_t**. Moreover, if the expression **P** points either to an element of

an array object or one past the last element of an array object, and the expression **Q** points to the last element of the same array object, the expression $((Q)+1) - (P)$ has the same value as $((Q) - (P)) + 1$ and as $-((P) - ((Q)+1))$, and has the value zero if the expression **P** points one past the last element of the array object, even though the expression $(Q)+1$ does not point to an element of the array object.

NOTE Another way to approach pointer arithmetic is first to convert the pointer(s) to character pointer(s): In this scheme the integer expression added to or subtracted from the converted pointer is first multiplied by the size of the object originally pointed to, and the resulting pointer is converted back to the original type. For pointer subtraction, the result of the difference between the character pointers is similarly divided by the size of the object originally pointed to. When viewed in this way, an implementation need only provide one extra byte (which can overlap another object in the program) just after the end of the object to satisfy the “one past the last element” requirements.

EXAMPLE Pointer arithmetic is well defined with pointers to variable length array types.

```
{
    int n = 4, m = 3;
    int a[n][m];
    int (*p)[m] = a; // p == &a[0]
    p += 1;          // p == &a[1]
    (*p)[2] = 99;    // a[1][2] == 99
    n = p - a;       // n == 1
}
```

If array **a** in the preceding example were declared to be an array of known constant size, and pointer **p** were declared to be a pointer to an array of the same known constant size (pointing to **a**), the results would be the same.

Forward references: array declarators (6.7.7.3), common definitions <stddef.h> (7.21).

6.5.8 Bitwise shift operators

Syntax

shift-expression:
 additive-expression
shift-expression << *additive-expression*
shift-expression >> *additive-expression*

Constraints

Each of the operands shall have integer type.

Semantics

The integer promotions are performed on each of the operands. The type of the result is that of the promoted left operand. If the value of the right operand is negative or is greater than or equal to the width of the promoted left operand, the behavior is undefined.

The result of **E1** << **E2** is **E1** left-shifted **E2** bit positions; vacated bits are filled with zeros. If **E1** has an unsigned type, the value of the result is $E1 \times 2^{E2}$, wrapped around. If **E1** has a signed type and nonnegative value, and $E1 \times 2^{E2}$ is representable in the result type, then that is the resulting value; otherwise, the behavior is undefined.

The result of **E1** >> **E2** is **E1** right-shifted **E2** bit positions. If **E1** has an unsigned type or if **E1** has a signed type and a nonnegative value, the value of the result is the integral part of the quotient of $E1/2^{E2}$. If **E1** has a signed type and a negative value, the resulting value is implementation-defined.

6.5.9 Relational operators

Syntax

relational-expression:
 shift-expression

relational-expression < *shift-expression*
relational-expression > *shift-expression*
relational-expression <= *shift-expression*
relational-expression >= *shift-expression*

Constraints

One of the following shall hold:

- both operands have real type; or
- both operands are pointers to qualified or unqualified versions of compatible object types.

If either operand has decimal floating type, the other operand shall not have standard floating type.

Semantics

If both of the operands have arithmetic type, the usual arithmetic conversions are performed. Positive zeros compare equal to negative zeros.

For the purposes of these operators, a pointer to an object that is not an element of an array behaves the same as a pointer to the first element of an array of length one with the type of the object as its element type.

When two pointers are compared, the result depends on the relative locations in the address space of the objects pointed to. If two pointers to object types both point to the same object, or both point one past the last element of the same array object, they compare equal. If the objects pointed to are members of the same aggregate object, pointers to structure members declared later compare greater than pointers to members declared earlier in the structure, and pointers to array elements with larger subscript values compare greater than pointers to elements of the same array with lower subscript values. All pointers to members of the same union object compare equal. If the expression **P** points to an element of an array object and the expression **Q** points to the last element of the same array object, the pointer expression **Q+1** compares greater than **P**. In all other cases, the behavior is undefined.

Each of the operators < (less than), > (greater than), <= (less than or equal to), and >= (greater than or equal to) shall yield 1 if the specified relation is true and 0 if it is false.¹⁰⁵⁾ The result has type **int**.

6.5.10 Equality operators

Syntax

equality-expression:

relational-expression
equality-expression == *relational-expression*
equality-expression != *relational-expression*

Constraints

One of the following shall hold:

- both operands have arithmetic type;
- both operands are pointers to qualified or unqualified versions of compatible types;
- one operand is a pointer to an object type and the other is a pointer to a qualified or unqualified version of **void**;
- both operands have type **nullptr_t**;

¹⁰⁵⁾ The expression **a<b<c** is not interpreted as in ordinary mathematics. As the syntax indicates, it means **(a<b)<c**; in other words, “if **a** is less than **b**, compare 1 to **c**; otherwise, compare 0 to **c**”.

- one operand has type **nullptr_t** and the other is a null pointer constant; or,
- one operand is a pointer and the other is a null pointer constant or has type **nullptr_t**.

If either operand has decimal floating type, the other operand shall not have standard floating type, complex type, or imaginary type.

Semantics

The **==** (equal to) and **!=** (not equal to) operators are analogous to the relational operators except for their lower precedence.¹⁰⁶⁾ Each of the operators yields 1 if the specified relation is true and 0 if it is false. The result has type **int**. For any pair of operands, exactly one of the relations is true.

If both of the operands have arithmetic type, the usual arithmetic conversions are performed. Positive zeros compare equal to negative zeros. Values of complex types are equal if and only if both their real parts are equal and also their imaginary parts are equal. Any two values of arithmetic types from different type domains are equal if and only if the results of their conversions to the (complex) result type determined by the usual arithmetic conversions are equal. If both operands have type **nullptr_t** or one operand has type **nullptr_t** and the other is a null pointer constant, they compare equal.

Otherwise, at least one operand is a pointer. If one operand is a pointer and the other is a null pointer constant or has type **nullptr_t**, they compare equal if the former is a null pointer. If one operand is a pointer to an object type and the other is a pointer to a qualified or unqualified version of **void**, the former is converted to the type of the latter.

Two pointers compare equal if and only if both are null pointers, both are pointers to the same object (including a pointer to an object and a subobject at its beginning) or function, both are pointers to one past the last element of the same array object, or one is a pointer to one past the end of one array object and the other is a pointer to the start of a different array object that happens to immediately follow the first array object in the address space.¹⁰⁷⁾

For the purposes of these operators, a pointer to an object that is not an element of an array behaves the same as a pointer to the first element of an array of length one with the type of the object as its element type.

6.5.11 Bitwise AND operator

Syntax

AND-expression:

equality-expression
AND-expression **&** *equality-expression*

Constraints

Each of the operands shall have integer type.

Semantics

The usual arithmetic conversions are performed on the operands.

The result of the binary **&** operator is the bitwise AND of the operands (that is, each bit in the result is set if and only if each of the corresponding bits in the converted operands is set).

6.5.12 Bitwise exclusive OR operator

Syntax

exclusive-OR-expression:

AND-expression

¹⁰⁶⁾ Because of the precedences, **a < b == c < d** is 1 whenever **a < b** and **c < d** have the same truth-value.

¹⁰⁷⁾ Two objects can be adjacent in memory because they are adjacent elements of a larger array or adjacent members of a structure with no padding between them, or because the implementation chose to place them so, even though they are unrelated. If prior invalid pointer operations (such as accesses outside array bounds) produced undefined behavior, subsequent comparisons also produce undefined behavior.

exclusive-OR-expression $\wedge$ *AND-expression*

Constraints

Each of the operands shall have integer type.

Semantics

The usual arithmetic conversions are performed on the operands.

The result of the $\wedge$ operator is the bitwise exclusive OR of the operands (that is, each bit in the result is set if and only if exactly one of the corresponding bits in the converted operands is set).

6.5.13 Bitwise inclusive OR operator

Syntax

inclusive-OR-expression:

exclusive-OR-expression

inclusive-OR-expression $\mid$ *exclusive-OR-expression*

Constraints

Each of the operands shall have integer type.

Semantics

The usual arithmetic conversions are performed on the operands.

The result of the $\mid$ operator is the bitwise inclusive OR of the operands (that is, each bit in the result is set if and only if at least one of the corresponding bits in the converted operands is set).

6.5.14 Logical AND operator

Syntax

logical-AND-expression:

inclusive-OR-expression

logical-AND-expression $\&\&$ *inclusive-OR-expression*

Constraints

Each of the operands shall have scalar type.

Semantics

The $\&\&$ operator shall yield 1 if both of its operands compare unequal to 0; otherwise, it yields 0. The result has type **int**.

Unlike the bitwise binary $\&$ operator, the $\&\&$ operator guarantees left-to-right evaluation; if the second operand is evaluated, there is a sequence point between the evaluations of the first and second operands. If the first operand compares equal to 0, the second operand is not evaluated.

6.5.15 Logical OR operator

Syntax

logical-OR-expression:

logical-AND-expression

logical-OR-expression $\mid\mid$ *logical-AND-expression*

Constraints

Each of the operands shall have scalar type.

Semantics

The `||` operator shall yield 1 if either of its operands compare unequal to 0; otherwise, it yields 0. The result has type **int**.

Unlike the bitwise `|` operator, the `||` operator guarantees left-to-right evaluation; if the second operand is evaluated, there is a sequence point between the evaluations of the first and second operands. If the first operand compares unequal to 0, the second operand is not evaluated.

6.5.16 Conditional operator**Syntax**

conditional-expression:

logical-OR-expression

logical-OR-expression ? expression : conditional-expression

Constraints

The first operand shall have scalar type.

One of the following shall hold for the second and third operands:¹⁰⁸⁾

- both operands have arithmetic type;
- both operands have compatible structure or union type;
- both operands have void type;
- both operands are pointers to qualified or unqualified versions of compatible types;
- both operands have **nullptr_t** type;
- one operand is a pointer and the other is a null pointer constant or has type **nullptr_t**; or
- one operand is a pointer to an object type and the other is a pointer to a qualified or unqualified version of **void**.

If either of the second or third operands has decimal floating type, the other operand shall not have standard floating type, complex type, or imaginary type.

Semantics

The first operand is evaluated; there is a sequence point between its evaluation and the evaluation of the second or third operand (whichever is evaluated). The second operand is evaluated only if the first compares unequal to 0; the third operand is evaluated only if the first compares equal to 0; the result is the value of the second or third operand (whichever is evaluated), converted to the type described subsequently in this subclause.¹⁰⁹⁾

If the second and third operands have arithmetic type, the result type is the same as if the usual arithmetic conversions were applied to both operands. If both the operands have structure or union type, the result is the composite type. If both operands have void type, the result has void type.

If both the second and third operands are pointers, the result type is a pointer to a type qualified with all the type qualifiers of the types referenced by both operands; if one is a null pointer constant (other than a pointer) or has type **nullptr_t** and the other is a pointer, the result type is the pointer type; if both the second and third operands have **nullptr_t** type, the result also has that type. Furthermore, if both operands are pointers to compatible types or to differently qualified versions of compatible types, the result type is a pointer to an appropriately qualified version of the composite type; if one operand is a null pointer constant, the result has the type of the other operand; otherwise,

¹⁰⁸⁾ If a second or third operand of type **nullptr_t** is used and the other operand is not a pointer and does not have type **nullptr_t** itself, a constraint is violated even if that other operand is a null pointer constant such as **0**.

¹⁰⁹⁾ A conditional expression does not yield an lvalue.

one operand is a pointer to **void** or a qualified version of **void**, in which case the result type is a pointer to an appropriately qualified version of **void**.

If one operand is a pointer to a variably modified type and the other operand is a null pointer constant or has type **nullptr_t**, the behavior is undefined if the type depends on an array size expression that is not evaluated.

EXAMPLE The common type that results when the second and third operands are pointers is determined in two independent stages. The appropriate qualifiers, for example, do not depend on whether the two pointers have compatible types.

Given the declarations

```
const void *c_vp;
void *vp;
const int *c_ip;
volatile int *v_ip;
int *ip;
const char *c_cp;
```

the third column in the Table 6.6 is the common type that is the result of a conditional expression in which the first two columns are the second and third operands (in either order):

Table 6.6 — Common type for conditional expression evaluations

c_vp	c_ip	const void *
v_ip	0	volatile int *
c_ip	v_ip	const volatile int *
vp	c_cp	const void *
ip	c_ip	const int *
vp	ip	void *

6.5.17 Assignment operators

6.5.17.1 General

Syntax

assignment-expression:

conditional-expression

unary-expression assignment-operator assignment-expression

assignment-operator: one of

= *= /= %= += -= <<= >>= &= ^= |=

Constraints

An assignment operator shall have a modifiable lvalue as its left operand.

Semantics

An assignment operator stores a value in the object designated by the left operand. An assignment expression has the value of the left operand after the assignment,¹¹⁰⁾ but is not an lvalue. The type of an assignment expression is the type the left operand would have after lvalue conversion. The side effect of updating the stored value of the left operand is sequenced after the value computations of the left and right operands. The evaluations of the operands are unsequenced.

¹¹⁰⁾The implementation is permitted to read the object to determine the value but is not required to, even when the object has volatile-qualified type.

6.5.17.2 Simple assignment

Constraints

One of the following shall hold:¹¹¹⁾

- the left operand has atomic, qualified, or unqualified arithmetic type, and the right operand has arithmetic type;
- the left operand has an atomic, qualified, or unqualified version of a structure or union type compatible with the type of the right operand;
- the left operand has atomic, qualified, or unqualified pointer type, and (considering the type the left operand would have after lvalue conversion) both operands are pointers to qualified or unqualified versions of compatible types, and the type pointed to by the left operand has all the qualifiers of the type pointed to by the right operand;
- the left operand has atomic, qualified, or unqualified pointer type, and (considering the type the left operand would have after lvalue conversion) one operand is a pointer to an object type, and the other is a pointer to a qualified or unqualified version of **void**, and the type pointed to by the left operand has all the qualifiers of the type pointed to by the right operand;
- the left operand has an atomic, qualified, or unqualified version of the **nullptr_t** type and the right operand is a null pointer constant or its type is **nullptr_t**;
- the left operand is an atomic, qualified, or unqualified pointer, and the right operand is a null pointer constant or its type is **nullptr_t**; or
- the left operand has type atomic, qualified, or unqualified **bool**, and the right operand is a pointer or its type is **nullptr_t**.

Semantics

In *simple assignment* (**=**), the value of the right operand is converted to the type of the assignment expression and replaces the value stored in the object designated by the left operand.¹¹²⁾

If the value being stored in an object is read from another object that overlaps in any way the storage of the first object, then the two objects shall occupy exactly the same storage and shall have qualified or unqualified versions of a compatible type; otherwise, the behavior is undefined.

EXAMPLE 1 In the program fragment

```
int f(void);
char c;
/* ... */
if ((c = f()) == -1)
    /* ... */
```

the **int** value returned by the function can be truncated when stored in the **char**, and then converted back to **int** width prior to the comparison. In an implementation in which “plain” **char** has the same range of values as **unsigned char** (and **char** is narrower than **int**), the result of the conversion cannot be negative, so the operands of the comparison can never compare equal. Therefore, for full portability, the variable **c** would be declared as **int**.

EXAMPLE 2 In the fragment:

```
char c;
int i;
long l;

l = (c = i);
```

¹¹¹⁾The asymmetric appearance of these constraints with respect to type qualifiers is due to the conversion (specified in 6.3.3.1) that changes lvalues to “the value of the expression” and thus removes any type qualifiers that were applied to the type category of the expression (for example, it removes **const** but not **volatile** from the type **int volatile * const**).

¹¹²⁾As described in 6.2.6.1, a store to an object with atomic type is done with **memory_order_seq_cst** semantics.

the value of **i** is converted to the type of the assignment expression **c = i**, that is, **char** type. The value of the expression enclosed in parentheses is then converted to the type of the outer assignment expression, that is, **long int** type.

EXAMPLE 3 The following fragment can be used as an example:

```
const char **cpp;
char *p;
const char c = 'A';

cpp = &p;           // constraint violation
*cpp = &c;          // valid
*p = 0;             // valid
```

The first assignment is unsafe because it would allow the following valid code to attempt to change the value of the **const** object **c**.

6.5.17.3 Compound assignment

Constraints

For the operators **+=** and **-=** only, either the left operand shall be an atomic, qualified, or unqualified pointer to a complete object type, and the right shall have integer type; or the left operand shall have atomic, qualified, or unqualified arithmetic type, and the right shall have arithmetic type.

For the other operators, the left operand shall have atomic, qualified, or unqualified arithmetic type, and (considering the type the left operand would have after lvalue conversion) each operand shall have arithmetic type consistent with those allowed by the corresponding binary operator.

If either operand has decimal floating type, the other operand shall not have standard floating type, complex type, or imaginary type.

Semantics

A *compound assignment* of the form **E1 op= E2** is equivalent to the simple assignment expression **E1 = E1 op (E2)**, except that the lvalue **E1** is evaluated only once, and with respect to an indeterminately sequenced function call, the operation of a compound assignment is a single evaluation. If **E1** has an atomic type, compound assignment is a read-modify-write operation with **memory_order_seq_cst** memory order semantics.

NOTE Where a pointer to an atomic object can be formed and **E1** and **E2** have integer type, this is equivalent to the following code sequence where **T1** is the type of **E1** and **T2** is the type of **E2**:

```
T1 *addr = &E1;
T2 val = (E2);
T1 old = *addr;
T1 new;
do {
    new = old op val;
} while (!atomic_compare_exchange_strong(addr, &old, new));
```

with **new** being the result of the operation.

If **E1** or **E2** has floating type, then exceptional conditions or floating-point exceptions encountered during discarded evaluations of **new** would also be discarded to satisfy the equivalence of **E1 op= E2** and **E1 = E1 op (E2)**. For example, if Annex F is in effect, the floating types involved have ISO/IEC 60559 binary formats, and **FLT_EVAL_METHOD** is 0, the equivalent code would be:

```
#include <fenv.h>
#pragma STDC FENV_ACCESS ON
/* ... */
fenv_t fenv;
T1 *addr = &E1;
T2 val = E2;
T1 old = *addr;
T1 new;
```



```
    feholdexcept(&fenv);
    for (;;) {
        new = old op val;
        if (atomic_compare_exchange_strong(addr, &old, new))
            break;
        feclearexcept(FE_ALL_EXCEPT);
    }
    feupdateenv(&fenv);
```

If `FLT_EVAL_METHOD` is not 0, then *T2* is expected to be a type with the range and precision to which **E2** is evaluated to satisfy the equivalence.

6.5.18 Comma operator

Syntax

expression:

assignment-expression
expression , *assignment-expression*

Semantics

The left operand of a comma operator is evaluated as a void expression; there is a sequence point between its evaluation and that of the right operand. Then the right operand is evaluated; the result has its type and value.¹¹³⁾

EXAMPLE As indicated by the syntax, the comma operator (as described in this subclause) cannot appear in contexts where a comma is used to separate items in a list (such as arguments to functions or lists of initializers). On the other hand, it can be used within a parenthesized expression or within the second expression of a conditional operator in such contexts. In the function call

```
f(a, (t=3, t+2), c)
```

the function has three arguments, the second of which has the value 5.

Forward references: initialization (6.7.11).

¹¹³⁾A comma operator does not yield an lvalue.

6.6 Constant expressions

Syntax

constant-expression:

conditional-expression

Description

A constant expression can be evaluated during translation rather than runtime, and accordingly can be used in any place that a constant can be.

Constraints

Constant expressions shall not contain assignment, increment, decrement, function-call, or comma operators, except when they are contained within a subexpression that is not evaluated.¹¹⁴⁾

Each constant expression shall evaluate to a constant that is in the range of representable values for its type.

Semantics

An expression that evaluates to a constant is required in several contexts. If a floating expression is evaluated in the translation environment, the arithmetic range and precision shall be at least as great as if the expression were being evaluated in the execution environment.¹¹⁵⁾

A compound literal with storage-class specifier **constexpr** is a *compound literal constant*, as is a postfix expression that applies the **.** member access operator to a compound literal constant of structure or union type, even recursively. A compound literal constant is a constant expression with the type and value of the unnamed object.

An identifier that is:

- an enumeration constant,
- a predefined constant, or
- declared with storage-class specifier **constexpr** and has an object type,

is a *named constant*, as is a postfix expression that applies the **.** member access operator to a named constant of structure or union type, even recursively. For enumeration and predefined constants, their value and type are defined in the respective clauses; for **constexpr** objects, such a named constant is a constant expression with the type and value of the declared object.

An *integer constant expression*¹¹⁶⁾ shall have integer type and shall only have operands that are integer constants, named and compound literal constants of integer type, character constants, **sizeof** expressions whose results are integer constants, **alignof** expressions, and floating, named, or compound literal constants of arithmetic type that are the immediate operands of casts. Cast operators in an integer constant expression shall only convert arithmetic types to integer types, except as part of an operand to the **typeof** operators, **sizeof** operator, or **alignof** operator.

More latitude is permitted for constant expressions in initializers. Such a constant expression shall be, or evaluate to, one of the following:

- a named constant,
- a compound literal constant,

¹¹⁴⁾The operand of a **typeof** (6.7.3.6), **sizeof**, or **alignof** operator is usually not evaluated (6.5.4.5).

¹¹⁵⁾The use of evaluation formats as characterized by **FLT_EVAL_METHOD** and **DEC_EVAL_METHOD** also applies to evaluation in the translation environment.

¹¹⁶⁾An integer constant expression is required in contexts such as the size of a bit-field member of a structure, the value of an enumeration constant, and the size of a non-variable length array. Further constraints that apply to the integer constant expressions used in conditional-inclusion preprocessing directives are discussed in 6.10.2.

- an arithmetic constant expression,
- a null pointer constant,
- an address constant, or
- an address constant for a complete object type plus or minus an integer constant expression.

An *arithmetic constant expression* shall have arithmetic type and shall only have operands that are integer constants, floating constants, named or compound literal constants of arithmetic type, character constants, **sizeof** expressions whose results are integer constants, and **alignof** expressions. Cast operators in an arithmetic constant expression shall only convert arithmetic types to arithmetic types, except as part of an operand to the typeof operators, **sizeof** operator, or **alignof** operator.

An *address constant* is a null pointer,¹¹⁷⁾ a pointer to an lvalue designating an object of static storage duration, or a pointer to a function designator; it shall be created explicitly using the unary **&** operator or an integer constant cast to pointer type, or implicitly using an expression of array or function type.

The array-subscript **[]** and member-access **->** operator, the address **&** and indirection ***** unary operators, and pointer casts can be used in the creation of an address constant, but the value of an object shall not be accessed by use of these operators.¹¹⁸⁾

A *structure or union constant* is a named constant or compound literal constant with structure or union type, respectively.

An implementation may accept other forms of constant expressions, called *extended constant expressions*. It is implementation-defined whether extended constant expressions are usable in the same manner as the constant expressions defined in this document, including whether or not extended integer constant expressions are considered to be integer constant expressions.¹¹⁹⁾

Starting from a structure or union constant, the member-access **.** operator can be used to form a named constant or compound literal constant as described previously in this subclause.

If the member-access operator **.** accesses a member of a union constant, the accessed member shall be the same as the member that is initialized by the union constant's initializer.

The semantic rules for the evaluation of a constant expression are the same as for nonconstant expressions.¹²⁰⁾

Forward references: array declarators (6.7.7.3), initialization (6.7.11).

¹¹⁷⁾A named constant or compound literal constant of integer type and value zero is a null pointer constant. A named constant or compound literal constant with a pointer type and a value null is a null pointer but not a null pointer constant; it can only be used to initialize a pointer object if its type implicitly converts to the target type.

¹¹⁸⁾Named constants or compound literal constants with arithmetic type, including names of **constexpr** objects, are valid in offset computations such as array subscripts or in pointer casts, as long as the expressions in which they occur form integer constant expressions. In contrast, names of other objects, even if **const**-qualified and with static storage duration, are not valid.

¹¹⁹⁾For example, in the declaration **int arr_or_vla[(int)+1.0];**, while possible to be computed by some implementations as an array with a size of one, it is implementation-defined whether this results in a variable length array declaration or a declaration of an array of known constant size of automatic storage duration. The choice depends on whether **(int)+1.0** is an extended integer constant expression.

¹²⁰⁾Thus, in the following initialization,

```
static int i = 2 || 1 / 0;
```

the expression is a valid integer constant expression with value one.

6.7 Declarations

6.7.1 General

Syntax

declaration:

```
declaration-specifiers init-declarator-listopt ;  
attribute-specifier-sequence declaration-specifiers init-declarator-list ;  
static_assert-declaration  
attribute-declaration
```

declaration-specifiers:

```
declaration-specifier attribute-specifier-sequenceopt  
declaration-specifier declaration-specifiers
```

declaration-specifier:

```
storage-class-specifier  
type-specifier-qualifier  
function-specifier
```

init-declarator-list:

```
init-declarator  
init-declarator-list , init-declarator
```

init-declarator:

```
declarator  
declarator = initializer
```

attribute-declaration:

```
attribute-specifier-sequence ;
```

Constraints

If a declaration other than a `static_assert` or attribute declaration does not include an `init declarator list`, its declaration specifiers shall include one of the following:

- a struct or union specifier or enum specifier that includes a tag, with the declaration being of a form specified in 6.7.3.4 to declare that tag;
- an enum specifier that includes an enumerator list.

EXAMPLE 1 The following are invalid, because the declared tag or enumeration constants are in a nested construct, rather than a declaration specifier of the declaration being of one of the given forms:

```
struct { struct s2 { int x2a; } x2b; };  
typedef (struct s3 { int x3; });  
alignas (struct s4 { int x4; }) int;  
typedef (struct s5 *);  
typedef (enum { E6 });  
struct { void (*p)(struct s7 *); };
```

If an identifier has no linkage, there shall be no more than one declaration of the identifier (in a declarator or type specifier) with the same scope and in the same name space, except that:

- a typedef name can be redefined to denote the same type as it currently does, provided that type is not a variably modified type;
- enumeration constants and tags can be redeclared as specified in 6.7.3.3 and 6.7.3.4, respectively.

All declarations in the same scope that refer to the same object or function shall specify compatible types.

Semantics

A declaration specifies the interpretation and properties of a set of identifiers. A *definition* of an identifier is a declaration for that identifier that for:

- an object, causes storage to be reserved for that object,
- a function, includes the function body,¹²¹⁾
- an enumeration constant, is the first (or only) declaration of the identifier, or
- a typedef name, is the first (or only) declaration of the identifier.

The declaration specifiers consist of a sequence of specifiers, followed by an optional attribute specifier sequence. The declaration specifiers indicate the linkage, storage duration, and part of the type of the entities that the declarators denote. The init declarator list is a comma-separated sequence of declarators, each of which can have additional type information, or an initializer, or both. The declarators contain the identifiers (if any) being declared. The optional attribute specifier sequence in a declaration appertains to each of the entities declared by the declarators of the init declarator list.

If an identifier for an object is declared with no linkage, the type for the object shall be complete by the end of its declarator, or by the end of its init-declarator if it has an initializer. In the case of function parameters, it is the adjusted type (see 6.7.7.4) that is required to be complete.

The optional attribute specifier sequence terminating a sequence of declaration specifiers appertains to the type determined by the preceding sequence of declaration specifiers. The attribute specifier sequence affects the type only for the declaration it appears in, not other declarations involving the same type.

Except where specified otherwise, the meaning of an attribute declaration is implementation-defined.

EXAMPLE 2 In the declaration for an entity, attributes appertaining to that entity can appear at the start of the declaration and after the identifier for that declaration.

```
[[deprecated]] void f [[deprecated]] (void); // valid
```

A declaration such that the declaration specifiers contain no type specifier or that is declared with **constexpr** is said to be *underspecified*. If such a declaration is not a definition, if it declares no or more than one ordinary identifier, if the declared identifier already has a declaration in the same scope, if the declared entity is not an object, or if anywhere within the sequence of tokens making up the declaration identifiers that are not ordinary are declared, the behavior is implementation-defined.¹²²⁾

EXAMPLE 3 As declarations using **constexpr** are underspecified, the following has implementation-defined behavior because tokens within the declaration declare **s** which is not an ordinary identifier:

```
constexpr typeof(struct s *) x = 0;
```

Forward references: declarators (6.7.7), enumeration specifiers (6.7.3.3), initialization (6.7.11), storage-class specifiers (6.7.2), type inference (6.7.10), type names (6.7.8), type qualifiers (6.7.4).

6.7.2 Storage-class specifiers

Syntax

storage-class-specifier:

- auto**
- constexpr**
- extern**
- register**

¹²¹⁾Function definitions have a different syntax, described in 6.9.2.
¹²²⁾It is recommended that implementations that accept such declarations follow the semantics of the corresponding feature in ISO/IEC 14882.

```
static
thread_local
typedef
```

Constraints

At most, one storage-class specifier can be given in the declaration specifiers in a declaration, except that:

- **thread_local** can appear with **static** or **extern**,
- **auto** can appear with all the others except **typedef**,¹²³⁾ and
- **constexpr** can appear with **auto**, **register**, or **static**.

In the declaration of an object with block scope, if the declaration specifiers include **thread_local**, they shall also include either **static** or **extern**. If **thread_local** appears in any declaration of an object, it shall be present in every declaration of that object.

thread_local shall not appear in the declaration specifiers of a function declaration. **auto** shall only appear in the declaration specifiers of an identifier with file scope or along with other storage-class specifiers if the type is to be inferred from an initializer.

An object declared with storage-class specifier **constexpr** or any of its members, even recursively, shall not have an atomic type, or a variably modified type, or a type that is **volatile** or **restrict** qualified. An initializer of floating type shall be evaluated with the translation-time floating-point environment. The declaration shall be a definition and shall have an initializer.¹²⁴⁾ The value of any constant expressions or of any character in a string literal of the initializer shall be exactly representable in the corresponding target type; no change of value shall be applied.¹²⁵⁾

If an object or subobject declared with storage-class specifier **constexpr** has pointer, integer, or arithmetic type, any explicit initializer value for it shall be null,¹²⁶⁾ an integer constant expression, or an arithmetic constant expression, respectively. If the object declared has real floating type, the initializer shall have integer or real floating type. If the object declared has imaginary type, the initializer shall have imaginary type. If the initializer has decimal floating type, the object declared shall have decimal floating type and the conversion shall preserve the quantum of the initializer. If the initializer has real type and a signaling NaN value, the unqualified versions of the type of the initializer and the corresponding real type of the object declared shall be compatible.

EXAMPLE 1 Although in the following the expression **A.p** is not a null pointer constant, only a constant expression with pointer type and a null pointer value, the member-wise initialization of **B** with **A** is valid.

```
struct s { void *p; };
constexpr struct s A = { nullptr };
constexpr struct s B = A;
```

EXAMPLE 2 Pointers can be initialized to eligible constant expressions, such as a null pointer constant:

```
constexpr int *p = {}; // Default initialization with a null pointer
```

EXAMPLE 3

¹²³⁾See “future language directions” (6.11.5).

¹²⁴⁾All assignment expressions of such an initializer, if any, are constant expressions or string literals, see 6.7.11.

¹²⁵⁾In the context of arithmetic conversions, 6.3.2 describes the details of changes of value that occur if values of arithmetic expressions are stored in the objects that for example have a different signedness, excess precision or quantum exponent. Whenever such a change of value is necessary, the constraint is violated.

¹²⁶⁾The named constant or compound literal constant corresponding to an object declared with storage-class specifier **constexpr** and pointer type is a constant expression with a value null, and thus a null pointer and an address constant. Thus, such a named constant is a valid initializer for other **constexpr** declarations, provided the pointer types match accordingly. However, even if it has type **void*** it is not a null pointer constant.

```
void f (void) {
    constexpr float f = 1.0f;
    constexpr float g = 3.0f;
    fesetround(FE_TOWARDSZERO); // does not affect
                                // the following initialization
                                // of "h"
    constexpr float h = f / g;
    // ...
}
```

Semantics

Storage-class specifiers specify various properties of identifiers and declared features:

- storage duration (**static** in block scope, **thread_local**, **auto**, **register**),
- linkage (**extern**, **static** and **constexpr** in file scope, **typedef**),
- value (**constexpr**), and
- type (**typedef**).

The meanings of the various linkages and storage durations were discussed in 6.2.2 and 6.2.4, **typedef** is discussed in 6.7.9, and type inference using **auto** is discussed in 6.7.10.

A declaration of an identifier for an object with storage-class specifier **register** suggests that access to the object be as fast as possible. The extent to which such suggestions are effective is implementation-defined.¹²⁷⁾

The declaration of an identifier for a function that has block scope shall have no explicit storage-class specifier other than **extern**.

If an aggregate or union object is declared with a storage-class specifier other than **typedef**, the properties resulting from the storage-class specifier, except with respect to linkage, also apply to the members of the object, including recursively for any aggregate or union member objects.

If **auto** appears with another storage-class specifier, or if it appears in a declaration at file scope, it is ignored for the purposes of determining a storage duration or linkage. In this case, it indicates only that the declared type can be inferred.

An object declared with a storage-class specifier **constexpr** has its value permanently fixed at translation-time; if not yet present, a **const**-qualification is implicitly added to the object’s type. The declared identifier is considered a constant expression of the respective kind, see 6.6.

NOTE 1 An object declared in block scope with a storage-class specifier **constexpr** and without **static** has automatic storage duration, the identifier has no linkage, and each instance of the object has a unique address obtainable with **&** (if it is not declared with the **register** specifier), if any. Such an object in file scope has static storage duration, the corresponding identifier has internal linkage, and each translation unit that sees the same textual definition implements a separate object with a distinct address.

NOTE 2 The constraints for **constexpr** objects are intended to enforce checks for portability at translation time.

```
constexpr unsigned int minusOne = -1;      // constraint violation
constexpr unsigned int uint_max = -1U;     // ok
constexpr double onethird      = 1.0/3.0;  // possible constraint violation
constexpr double onethirdtrunc = (double)(1.0/3.0); // ok
constexpr _Decimal32 small     = DEC64_TRUE_MIN * 0; // constraint violation
```

¹²⁷⁾The implementation can treat any **register** declaration simply as an **auto** declaration. However, whether or not addressable storage is used, the address of any part of an object declared with storage-class specifier **register** cannot be computed, either explicitly (by use of the unary **&** operator as discussed in 6.5.4.3) or implicitly (by converting an array name to a pointer as discussed in 6.3.3.1). Thus, the only operator that can be applied to an array declared with storage-class specifier **register** is **sizeof** and the **typeof** operators.

If a truncation of excess precision changes the value in the initializer of **onethird**, a constraint is violated and a diagnostic is required. In contrast to that, the explicit conversion in the initializer for **onethirdtrunc** ensures that the definition is valid. Similarly, the initializer of **small** has a quantum exponent that is larger than the largest possible quantum exponent for **_Decimal32**.

NOTE 3 Similarly, implementation-defined behavior related to the **char** type of the elements of the string literal `"\xFF"` can cause constraint violations at translation time:

```
constexpr char string[]      = { "\xFF", }; // ok
constexpr char8_t u8string[] = { u8"\xFF", }; // ok
constexpr unsigned char ucstring[] = { "\xFF", }; // possible constraint
                                                    // violation
```

In both the string and **ucstring** initializers, the initializer is a (brace-enclosed) string literal of type **char**. If the type **char** is capable of representing negative values and its width is 8, then the preceding code is equivalent to:

```
constexpr char string[]      = { -1, 0, }; // ok
constexpr char8_t u8string[] = { 255, 0, }; // ok
constexpr unsigned char ucstring[] = { -1, 0, }; // constraint violation
```

The hexadecimal escape sequence results in a value of **255**. For an initializer of type **char**, it is converted to a signed 8-bit integer, making a value of **-1**. A negative value does not fit within the range of values for **unsigned char**, and therefore the initialization of **ucstring** is a constraint violation under the previously stated implementation conditions. In the case where **char** is not capable of representing negative values, the original snippet is equivalent to the following and there is no constraint violation.

```
constexpr char string[]      = { 255, 0, }; // ok
constexpr char8_t u8string[] = { 255, 0, }; // ok
constexpr unsigned char ucstring[] = { 255, 0, }; // ok
```

EXAMPLE 4 An identifier declared with the **constexpr** specifier can have its value used in constant expressions:

```
constexpr int K = 47;
enum {
    A = K,           // valid, constant initialization
};
constexpr int L = K; // valid, constexpr initialization
static int b = K + 1; // valid, static initialization
int array[K];        // not a VLA
```

EXAMPLE 5 This example illustrates **constexpr** initializations involving different type domains, decimal and non-decimal floating types, NaNs and infinities, and quanta in decimal floating types.

```
#include <float.h>
#include <complex.h>

constexpr float _Complex fc1 = 1.0; // ok
constexpr float _Complex fc2 = 0.1; // constraint violation, unless double
                                     // has the same precision as float
                                     // and is evaluated with the same
                                     // precision
constexpr float _Complex fc3 = 3*I; // ok

constexpr double d1 = (double _Complex)1.0; // constraint violation
constexpr float f1 = (long double)INFINITY; // ok
constexpr float f2 = (long double)NaN;      // ok, quiet NaNs in real floating
                                             // types are considered the same
                                             // value, regardless of payloads

constexpr double d2 = DBL_SNaN; // ok
constexpr double d3 = FLT_SNaN; // constraint violation, even if float
                                     // and double have the same format
```

```

constexpr double _Complex dc1 = DBL_SNaN;           // ok
constexpr double _Complex dc2 = CMPLX(DBL_SNaN, 0.); // ok
constexpr double _Complex dc3 = CMPLX(0., DBL_SNaN); // ok

constexpr _Decimal32 d321 = 1.0;           // ok
constexpr _Decimal32 d322 = 1;             // ok
constexpr _Decimal32 d323 = INFINITY;      // ok
constexpr _Decimal32 d324 = NAN;           // ok
constexpr _Decimal64 d641 = DEC64_SNaN;    // ok
constexpr _Decimal64 d642 = DEC32_SNaN;    // constraint violation
constexpr float f3 = 1.DF;                 // constraint violation
constexpr float f4 = DEC_INFINITY;         // constraint violation
constexpr double d4 = DEC_NAN;             // constraint violation
constexpr _Decimal32 d325 = DEC64_TRUE_MIN * 0; // constraint violation,
                                                // quantum not preserved

#ifdef __STDC_IEC_60559_COMPLEX__
constexpr double d5 = (double _Imaginary)0.0; // constraint violation
constexpr double d6 = (double _Imaginary)0.0; // constraint violation
constexpr double _Imaginary di1 = 0.0*I;      // ok
constexpr double _Imaginary di2 = 0.0;        // constraint violation
#endif

```

EXAMPLE 6 An object declared with the **constexpr** specifier stores the exact value of its initializer, no implicit value change is applied:

```

#include <float.h>

constexpr int A      = 42LL;           // valid, 42 always fits in an int
constexpr signed short B = ULLONG_MAX; // constraint violation, value never
                                         // fits
constexpr float C      = 47u;          // valid, exactly representable
                                         // in float

#if FLT_MANT_DIG > 24
constexpr float D = 5369000000;        // constraint violation if float is
                                         // ISO/IEC 60559 binary32
#endif

#if (FLT_MANT_DIG == DBL_MANT_DIG) \
    && (0 <= FLT_EVAL_METHOD) \
    && (FLT_EVAL_METHOD <= 1)
constexpr float E = 1.0 / 3.0;          // only valid if double expressions
                                         // and float objects have the same
                                         // precision
#endif

#if FLT_EVAL_METHOD == 0
constexpr float F = 1.0f / 3.0f;        // valid, same type and precision
#else
constexpr float F = (float)(1.0f / 3.0f); // needs cast to truncate the
                                         // excess precision
#endif

```

EXAMPLE 7 This recursively applies to initializers for all elements of an aggregate object declared with the **constexpr** specifier:

```

constexpr static unsigned short array[] = {
    3000,    // valid, fits in unsigned short range
    300000,  // constraint violation if short is 16-bit
    -1       // constraint violation, target type is unsigned
}

```

```
};

struct S {
    int x, y;
};
constexpr struct S s = {
    .x = INT_MAX,          // valid
    .y = UINT_MAX,        // constraint violation
};
```

Forward references: type definitions (6.7.9), type inference (6.7.10).

6.7.3 Type specifiers

6.7.3.1 General

Syntax

type-specifier:

```
void
char
short
int
long
float
double
signed
unsigned
_BitInt ( constant-expression )
bool
_Complex
_Decimal32
_Decimal64
_Decimal128
atomic-type-specifier
struct-or-union-specifier
enum-specifier
typedef-name
typeof-specifier
```

Constraints

Except where the type is inferred (6.7.10), at least one type specifier shall be given in the declaration specifiers in each declaration, and in the specifier-qualifier list in each member declaration and type name. Each list of type specifiers shall be one of the following multisets (delimited by commas, when there is more than one multiset per item); the type specifiers can occur in any order, possibly intermixed with the other declaration specifiers.

- **void**
- **char**
- **signed char**
- **unsigned char**
- **short, signed short, short int, or signed short int**
- **unsigned short, or unsigned short int**
- **int, signed, or signed int**
- **unsigned, or unsigned int**

- **long**, **signed long**, **long int**, or **signed long int**
- **unsigned long**, or **unsigned long int**
- **long long**, **signed long long**, **long long int**, or **signed long long int**
- **unsigned long long**, or **unsigned long long int**
- **_BitInt**(*constant-expression*), or **signed _BitInt**(*constant-expression*)
- **unsigned _BitInt**(*constant-expression*)
- **float**
- **double**
- **long double**
- **_Decimal32**
- **_Decimal64**
- **_Decimal128**
- **bool**
- **float _Complex**
- **double _Complex**
- **long double _Complex**
- atomic type specifier
- struct or union specifier
- enum specifier
- typedef name
- typeof specifier

The type specifier **_Complex** shall not be used if the implementation does not support complex types, and the type specifiers **_Decimal32**, **_Decimal64**, and **_Decimal128** shall not be used if the implementation does not support decimal floating types (see 6.10.10.4).

The parenthesized constant expression that follows the **_BitInt** keyword shall be an integer constant expression *N* that specifies the width (6.2.6.2) of the type. The value of *N* for **unsigned _BitInt** shall be greater than or equal to 1. The value of *N* for **_BitInt** shall be greater than or equal to 2. The value of *N* shall be less than or equal to the value of **BITINT_MAXWIDTH** (see 5.3.5.3.2).

Semantics

Specifiers for structures, unions, enumerations, atomic types, and typeof specifiers are discussed in 6.7.3.2 through 6.7.3.6. Declarations of typedef names are discussed in 6.7.9. The characteristics of the other types are discussed in 6.2.5.

For a declaration such that the declaration specifiers contain no type specifier a mechanism to infer the type from an initializer is discussed in 6.7.10. In such a declaration, optional elements, if any, of a sequence of declaration specifiers appertain to the inferred type (for qualifiers and attribute specifiers) or to the declared objects (for alignment specifiers).

Each of the comma-separated multisets designates the same type, except that for bit-fields, it is implementation-defined whether the specifier **int** designates the same type as **signed int** or the same type as **unsigned int**.

Forward references: atomic type specifiers (6.7.3.5), enumeration specifiers (6.7.3.3), structure and union specifiers (6.7.3.2), tags (6.7.3.4), type definitions (6.7.9).

6.7.3.2 Structure and union specifiers

Syntax

struct-or-union-specifier:

struct-or-union attribute-specifier-sequence_{opt} identifier_{opt} { member-declaration-list }

struct-or-union attribute-specifier-sequence_{opt} identifier

struct-or-union:

struct
union

member-declaration-list:

member-declaration
member-declaration-list member-declaration

member-declaration:

attribute-specifier-sequence_{opt} specifier-qualifier-list member-declarator-list_{opt} ;
static_assert-declaration

specifier-qualifier-list:

type-specifier-qualifier attribute-specifier-sequence_{opt}
type-specifier-qualifier specifier-qualifier-list

type-specifier-qualifier:

type-specifier
type-qualifier
alignment-specifier

member-declarator-list:

member-declarator
member-declarator-list , member-declarator

member-declarator:

declarator
declarator_{opt} : constant-expression

Constraints

A member declaration that does not declare an anonymous structure or anonymous union shall contain a member declarator list.

A structure or union shall not contain a member with incomplete or function type (hence, a structure shall not contain an instance of itself, but can contain a pointer to an instance of itself), except that the last member of a structure with more than one named member can have incomplete array type; such a structure (and any union containing, possibly recursively, a member that is such a structure) shall not be a member of a structure or an element of an array.

The expression that specifies the width of a bit-field shall be an integer constant expression with a nonnegative value that does not exceed the width of an object of the type that would be specified were the colon and expression omitted.¹²⁸⁾ If the value is zero, the declaration shall have no declarator.

A bit-field shall have a type that is a qualified or unqualified **bool**, **signed int**, **unsigned int**, a bit-precise integer type, or other implementation-defined type. It is implementation-defined whether atomic types are permitted.

An attribute specifier sequence shall not appear in a struct-or-union specifier without a member

¹²⁸⁾While the number of bits in a **bool** object is at least **CHAR_BIT**, the width of a **bool** is just 1 bit.

declaration list, except in a declaration of the form:

struct-or-union attribute-specifier-sequence identifier ;

The attributes in the attribute specifier sequence, if any, are thereafter considered attributes of the **struct** or **union** whenever it is named.

Semantics

As discussed in 6.2.5, a structure is a type consisting of a sequence of members, whose storage is allocated in an ordered sequence, and a union is a type consisting of a sequence of members whose storage overlap.

Structure and union specifiers have the same form. The keywords **struct** and **union** indicate that the type being specified is, respectively, a structure type or a union type.

The optional attribute specifier sequence in a struct-or-union specifier appertains to the structure or union type being declared. The optional attribute specifier sequence in a member declaration appertains to each of the members declared by the member declarator list; it shall not appear if the optional member declarator list is omitted. The optional attribute specifier sequence in a specifier qualifier list appertains to the type denoted by the preceding type specifier qualifiers. The attribute specifier sequence affects the type only for the member declaration or type name it appears in, not other types or declarations involving the same type.

The member declaration list is a sequence of declarations for the members of the structure or union. If the member declaration list does not contain any named members, either directly or via an anonymous structure or anonymous union, the behavior is undefined.¹²⁹⁾

A member of a structure or union can have any complete object type other than a variably modified type.¹³⁰⁾ In addition, a member can be declared to consist of a specified number of bits (including a sign bit, if any). Such a member is called a *bit-field*;¹³¹⁾ its width is preceded by a colon.

A bit-field is interpreted as having a signed or unsigned integer type consisting of the specified number of bits.¹³²⁾ If the value 0 or 1 is stored into a nonzero-width bit-field of type **bool**, the value of the bit-field shall compare equal to the value stored; a **bool** bit-field has the semantics of a **bool**.

An implementation can allocate any addressable storage unit large enough to hold a bit-field. If enough space remains, a bit-field that immediately follows another bit-field in a structure shall be packed into adjacent bits of the same unit. If insufficient space remains, whether a bit-field that does not fit is put into the next unit or overlaps adjacent units is implementation-defined. The order of allocation of bit-fields within a unit (high-order to low-order or low-order to high-order) is implementation-defined. The alignment of the addressable storage unit is unspecified.

A bit-field declaration with no declarator, but only a colon and a width, indicates an unnamed bit-field.¹³³⁾ As a special case, a bit-field structure member with a width of zero indicates that no further bit-field is to be packed into the unit in which the previous bit-field, if any, was placed.

An unnamed member whose type specifier is a structure specifier with no tag is called an *anonymous structure*; an unnamed member whose type specifier is a union specifier with no tag is called an *anonymous union*. The members of an anonymous structure or union are members of the containing structure or union, keeping their structure or union layout. This applies recursively if the containing structure or union is also anonymous.

Each non-bit-field member of a structure or union object is aligned in an implementation-defined manner appropriate to its type.

¹²⁹⁾For further rules affecting compatibility and completeness of structure or union types, see 6.2.7 and 6.7.3.4.

¹³⁰⁾A structure or union cannot contain a member with a variably modified type because member names are not ordinary identifiers as defined in 6.2.3.

¹³¹⁾The unary & (address-of) operator cannot be applied to a bit-field object; thus, there are no pointers to or arrays of bit-field objects.

¹³²⁾As specified in 6.7.3, if the actual type specifier used is **int** or a typedef-name defined as **int**, then it is implementation-defined whether the bit-field is signed or unsigned. This includes an **int** type specifier produced using the **typeof** specifiers (6.7.3.6).

¹³³⁾An unnamed bit-field structure member is useful for padding to conform to externally imposed layouts.

Within a structure object, the non-bit-field members and the units in which bit-fields reside have addresses that increase in the order in which they are declared. A pointer to a structure object, suitably converted, points to its initial member (or if that member is a bit-field, then to the unit in which it resides), and vice versa. There can be unnamed padding within a structure object, but not at its beginning.

The size of a union is sufficient to contain the largest of its members. The value of at most one of the members can be stored in a union object at any time. A pointer to a union object, suitably converted, points to each of its members (or if a member is a bit-field, then to the unit in which it resides), and vice versa. The members of a union object overlap in such a way that pointers to them when converted to pointers to character type point to the same byte. There can be unnamed padding at the end of a union object, but not at its beginning.

There can be unnamed padding at the end of a structure or union.

As a special case, the last member of a structure with more than one named member can have an incomplete array type; this is called a *flexible array member*. In most situations, the flexible array member is ignored. In particular, the size of the structure is as if the flexible array member were omitted except that it may have more trailing padding than the omission would imply. However, when a `.` (or `->`) operator has a left operand that is (a pointer to) a structure with a flexible array member and the right operand names that member, it behaves as if that member were replaced with the longest array (with the same element type) that would not make the structure larger than the object being accessed; the offset of the array shall remain that of the flexible array member, even if this would differ from that of the replacement array. If this array would have no elements, it behaves as if it had one element but the behavior is undefined if any attempt is made to access that element or to generate a pointer one past it.

EXAMPLE 1 The following declarations illustrate the behavior when an attribute is written on a tag declaration:

```
struct [[deprecated]] S; // valid, [[deprecated]] appertains to struct S
void f(struct S *s);    // valid, the struct S type has the [[deprecated]]
                        // attribute
struct S {              // valid, struct S inherits the [[deprecated]] attribute
    int a;                // from the previous declaration
};
void g(struct [[deprecated]] S s); // invalid
```

EXAMPLE 2 The following illustrates anonymous structures and unions:

```
struct v {
    union {               // anonymous union
        struct { int i, j; }; // anonymous structure
        struct { long k, l; } w;
    };
    int m;
} v1;

v1.i = 2; // valid
v1.k = 3; // invalid: inner structure is not anonymous
v1.w.k = 5; // valid
```

EXAMPLE 3 After the declaration:

```
struct s { int n; double d[]; };
```

the structure **struct** **s** has a flexible array member **d**. A typical way to use this is:

```
int m = /* some value */;
struct s *p = malloc(sizeof(struct s) + sizeof(double [m]));
```

and assuming that the call to **malloc** succeeds, the object pointed to by **p** behaves, for most purposes, as if **p** had been declared as:


```
struct { int n; double d[m]; } *p;
```

(there are circumstances in which this equivalence is broken; in particular, the offsets of member **d** can sometimes not be the same).

Following the declaration of the previous example:

```
struct s t1 = { 0 };           // valid
struct s t2 = { 1, { 4.2 } }; // invalid
t1.n = 4;                     // valid
t1.d[0] = 4.2;                // can be undefined behavior
```

The initialization of **t2** is invalid (and violates a constraint) because **struct s** is treated as if it did not contain member **d**. The assignment to **t1.d[0]** has probably undefined behavior, but it is possible that

```
sizeof(struct s) >= offsetof(struct s, d) + sizeof(double)
```

in which case the assignment would be legitimate. Nevertheless, it cannot appear in strictly conforming code.

After the further declaration:

```
struct ss { int n; };
```

the expressions:

```
sizeof(struct s) >= sizeof(struct ss)
sizeof(struct s) >= offsetof(struct s, d)
```

are always equal to 1.

If **sizeof(double)** is 8, then after the following code is executed:

```
struct s *s1;
struct s *s2;
s1 = malloc(sizeof(struct s) + 64);
s2 = malloc(sizeof(struct s) + 46);
```

and assuming that the calls to **malloc** succeed, the objects pointed to by **s1** and **s2** behave, for most purposes, as if the identifiers had been declared as:

```
struct { int n; double d[8]; } *s1;
struct { int n; double d[5]; } *s2;
```

Following the further successful assignments:

```
s1 = malloc(sizeof(struct s) + 10);
s2 = malloc(sizeof(struct s) + 6);
```

they then behave as if the declarations were:

```
struct { int n; double d[1]; } *s1, *s2;
```

and:

```
double *dp;
dp = &(s1->d[0]); // valid
*dp = 42;         // valid
dp = &(s2->d[0]); // valid
*dp = 42;         // undefined behavior
```

The assignment:

```
*s1 = *s2;
```

only copies the member **n**; if any of the array elements are within the first **sizeof(struct s)** bytes of the structure, they are set to an indeterminate representation, that can sometimes not coincide with a copy of the representation of the elements of the source array.

EXAMPLE 4 Because members of anonymous structures and unions are considered to be members of the containing structure or union, **struct s** in the following example has more than one named member and thus the use of a flexible array member is valid:

```
struct s {
    struct { int i; };
    int a[];
};
```

Forward references: declarators (6.7.7), tags (6.7.3.4).

6.7.3.3 Enumeration specifiers

Syntax

enum-specifier:

```
enum attribute-specifier-sequenceopt identifieropt enum-type-specifieropt
    { enumerator-list }
enum attribute-specifier-sequenceopt identifieropt enum-type-specifieropt
    { enumerator-list , }
enum identifier enum-type-specifieropt
```

enumerator-list:

```
enumerator
enumerator-list , enumerator
```

enumerator:

```
enumeration-constant attribute-specifier-sequenceopt
enumeration-constant attribute-specifier-sequenceopt = constant-expression
```

enum-type-specifier:

```
: specifier-qualifier-list
```

All enumerations have an *underlying type*. The underlying type can be explicitly specified using an enum type specifier and is its *fixed underlying type*. If it is not explicitly specified, the underlying type is the enumeration's compatible type, which is either **char** or a standard or extended signed or unsigned integer type.

Constraints

For an enumeration with a fixed underlying type, the integer constant expression defining the value of the enumeration constant shall be representable in that fixed underlying type. If the value of an enumeration constant without a defining constant expression for an enumeration with fixed underlying type is obtained by adding 1 to the previous enumeration constant, the value of that previous enumeration constant shall not be the maximum value of the underlying type.

For an enumeration without a fixed underlying type, the expression that defines the value of an enumeration constant shall be an integer constant expression. For all the integer constant expressions which make up the values of the enumeration constants, there shall be a type capable of representing all the values that is a standard or extended signed or unsigned integer type, or **char**.

If an enum type specifier is present, then the longest possible sequence of tokens that can be interpreted as a specifier qualifier list is interpreted as part of the enum type specifier. It shall name an integer type that is neither an enumeration nor a bit-precise integer type. The underlying type of the enumeration is the unqualified, non-atomic version of the type specified by the type specifiers in the specifier qualifier list.¹³⁴⁾

¹³⁴⁾The specifier qualifier list is not a context listed in 6.7.6 as permitted for alignment specifiers, so the presence of an alignment specifier in the list violates a constraint.

An enum specifier of the form

enum *identifier* *enum-type-specifier*

shall not appear except in a declaration of the form

enum *identifier* *enum-type-specifier* ;

unless it is immediately followed by an opening brace, an enumerator list (with an optional ending comma), and a closing brace.

If two enum specifiers that include an enum type specifier declare the same type, the underlying types shall be compatible.

An enumeration with a fixed underlying type shall be defined with an enum type specifier. No enum specifier for an enumeration without a fixed underlying type shall include an enum type specifier.

Semantics

The optional attribute specifier sequence in the **enum** specifier appertains to the enumeration; the attributes in that attribute specifier sequence are thereafter considered attributes of the enumeration whenever it is named. The optional attribute specifier sequence in the enumerator appertains to that enumerator.

The identifiers in an enumerator list are declared as constants of the types specified later in this subclause and can appear wherever such are permitted.¹³⁵⁾ An enumerator with = defines its enumeration constant as the value of the constant expression. If the first enumerator has no =, the value of its enumeration constant is zero. Each subsequent enumerator with no = defines its enumeration constant as the value of the constant expression obtained by adding 1 to the value of the previous enumeration constant. (The use of enumerators with = may produce enumeration constants with values that duplicate other values in the same enumeration.) The enumerators of an enumeration are also known as its members.

The type for the members of an enumeration is called the *enumeration member type*.

During the processing of each enumeration constant in the enumerator list, the type of the enumeration constant shall be:

- the previously declared type, if it is a redeclaration of the same enumeration constant; or,
- the enumerated type, for an enumeration with fixed underlying type; or,
- **int**, if there are no previous enumeration constants in the enumerator list and no explicit = with a defining integer constant expression; or,
- **int**, if given explicitly with = and the value of the integer constant expression is representable by an **int**; or,
- the type of the integer constant expression, if given explicitly with = and if the value of the integer constant expression is not representable by **int**; or,
- the type of the value from the previous enumeration constant with one added to it. If such an integer constant expression would overflow or wraparound the value of the previous enumeration constant from the addition of one, the type takes on either:
 - a suitably sized signed integer type, excluding the bit-precise signed integer types, capable of representing the value of the previous enumeration constant plus one; or,
 - a suitably sized unsigned integer type, excluding the bit-precise unsigned integer types, capable of representing the value of the previous enumeration constant plus one.

¹³⁵⁾ Thus, the identifiers of enumeration constants declared in the same scope are all required to be distinct from each other and from other identifiers declared in ordinary declarators.

A signed integer type is chosen if the previous enumeration constant being added is of signed integer type. An unsigned integer type is chosen if the previous enumeration constant is of unsigned integer type. If there is no suitably sized integer type described previously which can represent the new value, then the enumeration has no type which can represent all its values.¹³⁶⁾

For all enumerations without a fixed underlying type, each enumerated type shall be compatible with **char** or a signed or an unsigned integer type that is not **bool** or a bit-precise integer type. The choice of type is implementation-defined,¹³⁷⁾ but shall be capable of representing the values of all the members of the enumeration.¹³⁸⁾

Enumeration constants can be redefined in the same scope with the same value as part of a redeclaration of the same enumerated type.

The enumeration member type for an enumerated type without fixed underlying type upon completion is:

- **int** if all the values of the enumeration are representable as an **int**; or,
- the enumerated type.¹³⁹⁾

The enumeration member type for an enumerated type with fixed underlying type is the enumerated type. The enumerated type is compatible with the underlying type of the enumeration. After possible lvalue conversion a value of the enumerated type behaves the same as the value with the underlying type, in particular with all aspects of promotion, conversion, and arithmetic. Conversion to the enumerated type has the same semantics as conversion to the underlying type.¹⁴⁰⁾

EXAMPLE 1 The following fragment:

```
enum hue { chartreuse, burgundy, claret=20, winedark };
enum hue col, *cp;
col = claret;
cp = &col;
if (*cp != burgundy)
    /* ... */
```

makes **hue** the tag of an enumeration, and then declares **col** as an object that has that type and **cp** as a pointer to an object that has that type. The enumerated values are in the set {0, 1, 20, 21}.

EXAMPLE 2 Even if the value of an enumeration constant is generated by the implicit addition of one, an enumeration with a fixed underlying type does not exhibit typical overflow behavior:

```
#include <limits.h>

enum us : unsigned short{
    us_max = USHRT_MAX,
    us_violation, /* Constraint violation:
                   USHRT_MAX + 1 would wraparound. */
    us_violation_2 = us_max + 1, /* Maybe constraint violation:
                                   USHRT_MAX + 1 can be promoted to "int", and
                                   result is too wide for the
                                   underlying type. */
    us_wraparound_to_zero = (unsigned short)(USHRT_MAX + 1) /* Okay:
```

¹³⁶⁾Therefore, a constraint has been violated.

¹³⁷⁾An implementation can delay the choice of which integer type until all enumeration constants have been seen.

¹³⁸⁾For further rules affecting compatibility and completeness of enumerated types see 6.2.7 and 6.7.3.4.

¹³⁹⁾The integer type selected during processing of the enumerator list (before completion) of the enumeration can sometimes not be the same as the compatible implementation-defined integer type selected for the completed enumeration.

¹⁴⁰⁾This means in particular that if the compatible type is **bool**, values of the enumerated type behave in all aspects the same as **bool**, conversion to the enumerated type behaves the same as **bool** (6.3.2.2), and the members only have values **false** and **true**. If it is a signed integer type and the constant expression of an enumeration constant overflows, a constraint for constant expressions (6.6) is violated.

```

conversion done in constant expression
before conversion to underlying type:
unsigned semantics okay. */

};

enum ui : unsigned int {
    ui_max = UINT_MAX,
    ui_violation, /* Constraint violation:
                   UINT_MAX + 1 would wraparound. */
    ui_no_violation = ui_max + 1, /* Okay: Arithmetic performed as typical
                                   unsigned integer arithmetic: conversion
                                   from a value that is already 0 to 0. */
    ui_wraparound_to_zero = (unsigned int)(UINT_MAX + 1) /* Okay: conversion
                                                           done in constant expression before conversion to
                                                           underlying type: unsigned semantics okay. */
};

int main () {
    // Same as return 0;
    return ui_wraparound_to_zero + us_wraparound_to_zero;
}

```

EXAMPLE 3 The following fragment:

```

#include <limits.h>

enum E1: short;
enum E2: short;
enum E3; /* Constraint violation: E3 forward declaration. */
enum E4 : unsigned long long;

enum E1 : short { m11, m12 };
enum E1 x = m11;

enum E2 : long { m21, m22 }; /* Constraint violation: different underlying types
*/

enum E3 {
    m31,
    m32,
    m33 = sizeof(enum E3) /* Constraint violation: E3 is not complete here. */
};
enum E3 : int; /* Constraint violation: E3 previously had no underlying type */

enum E4 : unsigned long long {
    m40 = sizeof(enum E4),
    m41 = ULLONG_MAX,
    m42 /* Constraint violation: unrepresentable value (wraparound) */
};

enum E5 y; /* Constraint violation: incomplete type */
enum E6 : long int z; /* Constraint violation: enum-type-specifier
                      with identifier in declarator */
enum E7 : long int = 0; /* Syntax violation:
                        enum-type-specifier with initializer */

```

demonstrates many of the properties of multiple declarations of enumerations with underlying types. Particularly, **enum E3** is declared and defined without an underlying type first, therefore a redeclaration with an underlying type second is a violation. Because it not complete at that time within its enumerator list, `sizeof(enum E3)` is a constraint violation within the **enum E3** definition. **enum E4** is complete as it is being defined, therefore `sizeof(enum E4)` is not a constraint violation.

EXAMPLE 4 The following fragment:

```
enum no_underlying {
    a0
};

int main (void) {
    int a = _Generic(a0,
        int: 2,
        unsigned char: 1,
        default: 0
    );
    int b = _Generic((enum no_underlying)a0,
        int: 2,
        unsigned char: 1,
        default: 0
    );
    return a + b;
}
```

demonstrates the implementation-defined nature of the underlying type of enumerations using generic selection (6.5.2.1). The value of **a** after its initialization is 2. The value of **b** after its initialization is implementation-defined: the enumeration is compatible with a type large enough to fit the values of its enumeration constants due to the constraints. Because the only value is 0 for **a0**, **b** can hold any of 2, 1, or 0.

The following fragment is similar, but uses a fixed underlying type:

```
enum underlying : unsigned char {
    b0
};

int main (void) {
    int a = _Generic(b0,
        int: 2,
        unsigned char: 1,
        default: 0
    );
    int b = _Generic((enum underlying)b0,
        int: 2,
        unsigned char: 1,
        default: 0
    );
    return 0;
}
```

Here, we are guaranteed that **a** and **b** are both initialized to 1. This makes enumerations with a fixed underlying type more portable.

EXAMPLE 5 Enumerations with a fixed underlying type have braces and the enumerator list specified as part of their declaration if they are not a standalone declaration:

```
void f1 (enum a : long b); /* Constraint violation */
void f2 (enum c : long { x } d);
enum e : int f3(); /* Constraint violation */

typedef enum t u; /* Constraint violation: forward declaration of t. */
typedef enum v : short W; /* Constraint violation */
typedef enum q : short { s } R;

struct s1 {
    int x;
    enum e : int : 1; /* Constraint violation */
    int y;
}
```

```
};

enum forward; /* Constraint violation */
extern enum forward fwd_val0; /* Constraint violation: incomplete type */
extern enum forward* fwd_ptr0; /* Constraint violation: enums cannot be
                                used like other incomplete types */
extern int* fwd_ptr0; /* Constraint violation: incompatible
                      with incomplete type. */

enum forward1 : int;
extern enum forward1 fwd_val1;
extern int fwd_val1;
extern enum forward1* fwd_ptr1;
extern int* fwd_ptr1;

int main () {
    enum e : short;
    enum e : short f = 0; /* Constraint violation */
    enum g : short { y } h = y;
    return 0;
}
```

EXAMPLE 6 Enumerations with a fixed underlying type are complete when the enum type specifier for that specific enumeration is complete. The enumeration **e** in this snippet:

```
enum e : typeof ((enum e : short { A })0, (short)0);
```

enum e is considered complete by the first opening brace within the **typeof** in this snippet.

Forward references: generic selection (6.5.2.1), tags (6.7.3.4), declarations (6.7), declarators (6.7.7), function declarators (6.7.7.4), type names (6.7.8).

6.7.3.4 Tags

Constraints

Where two declarations that use the same tag declare the same type, they shall both use the same choice of **struct**, **union**, or **enum**. If two declarations of the same type have a member-declaration or enumerator-list, one shall not be nested within the other and both declarations shall fulfill all requirements of compatible types (6.2.7) with the additional requirement that corresponding members of structure or union types shall have the same (and not merely compatible) types.

A type specifier of the form

```
enum identifier
```

without an enumerator list shall only appear after the type it specifies is complete.

A type specifier of the form

```
struct-or-union attribute-specifier-sequenceopt identifier
```

shall not contain an attribute specifier sequence.¹⁴¹⁾

Semantics

All declarations of structure, union, or enumerated types that have the same scope and use the same tag declare the same type.

Irrespective of whether there is a tag or what other declarations of the type are in the same translation unit, the type (except enumerated types with a fixed underlying type) is incomplete until immediately after the closing brace of the list defining the content for the first time and complete thereafter.

Enumerated types with fixed underlying type (6.7.3.3) are complete immediately after their first

¹⁴¹⁾As specified in 6.7.3.2, the type specifier can be followed by a ; or a member declaration list.

associated enum type specifier ends.

EXAMPLE 1 The following example shows allowed redeclarations of the same structure, union, or enumerated type in the same scope:

```

struct foo { struct { int x; }; };
struct foo { struct { int x; }; };
union bar { int x; float y; };
union bar { int x; float y; };
typedef struct q { int x; } q_t;
typedef struct q { int x; } q_t;
void foo(void)
{
    struct S { int x; };
    struct T { struct S s; };
    struct S { int x; };
    struct T { struct S s; };
}
enum X { A = 1, B = 1 + 1 };
enum X { B = 2, A = 1 };

enum Q { C = 1 };
enum Q { C = C };           // ok!

```

EXAMPLE 2 The following example shows invalid redeclarations of the same structure, union, or enumerated type in the same scope:

```

struct foo { int (*p)[3]; };
struct foo { int (*p)[]; };    // member has different type

union bar { int x; float y; };
union bar { int z; float y; }; // member has different name

union purr { int x; float y; };
union purr { float y; int x; }; // members have different order
// "purr" only valid if each union "purr" is in
// two different translation units

typedef struct { int x; } q_t;
typedef struct { int x; } q_t; // not the same type

struct S { int x; };

void foo(void)
{
    struct T { struct S s; };
    struct S { int x; };
    struct T { struct S s; }; // struct S not the same type
}

enum X { A = 1, B = 2 };
enum X { A = 1, B = 3 };    // different enumeration constant

enum R { C = 1 };
enum Q { C = 1 };          // conflicting enumeration constant

```

Two declarations of structure, union, or enumerated types which are in different scopes or use different tags declare distinct types. Each declaration of a structure, union, or enumerated type which does not include a tag declares a distinct type.

A type specifier of the form

struct-or-union *attribute-specifier-sequence*_{opt} *identifier*_{opt} { *member-declaration-list* }

or

enum *attribute-specifier-sequence*_{opt} *identifier*_{opt} *enum-type-specifier*_{opt} { *enumerator-list* }

or

enum *attribute-specifier-sequence*_{opt} *identifier*_{opt} *enum-type-specifier*_{opt} { *enumerator-list* , }

declares a structure, union, or enumerated type. The list defines the *structure content*, *union content*, or *enumeration content*. If an identifier is provided,¹⁴²⁾ the type specifier also declares the identifier to be the tag of that type. The optional attribute specifier sequence appertains to the structure, union, or enumerated type being declared; the attributes in that attribute specifier sequence are thereafter considered attributes of the structure, union, or enumerated type whenever it is named.

A declaration of the form

struct-or-union *attribute-specifier-sequence*_{opt} *identifier* ;

or

enum *identifier* *enum-type-specifier* ;

specifies a structure, union, or enumerated type and declares the identifier as a tag of that type.¹⁴³⁾ The optional attribute specifier sequence appertains to the structure or union type being declared; the attributes in that attribute specifier sequence are thereafter considered attributes of the structure or union type whenever it is named.

If a type specifier of the form

struct-or-union *attribute-specifier-sequence*_{opt} *identifier*

occurs other than as part of one of the preceding forms, and no other declaration of the identifier as a tag is visible, then it declares an incomplete structure or union type, and declares the identifier as the tag of that type.¹⁴³⁾

If a type specifier of the form

struct-or-union *attribute-specifier-sequence*_{opt} *identifier*

or

enum *identifier*

occurs other than as part of one of the preceding forms, and a declaration of the identifier as a tag is visible, then it specifies the same type as that other declaration, and does not redeclare the tag.

EXAMPLE 3 This mechanism allows declaration of a self-referential structure.

```
struct tnode {
    int count;
    struct tnode *left, *right;
};
```

specifies a structure that contains an integer and two pointers to objects of the same type. Once this declaration has been given, the declaration

```
struct tnode s, *sp;
```

declares **s** to be an object of the given type and **sp** to be a pointer to an object of the given type. With these declarations, the expression **sp->left** refers to the left **struct tnode** pointer of the object to which **sp** points; the expression **s.right->count** designates the **count** member of the right **struct tnode** pointed to from **s**.

The following alternative formulation uses the **typedef** mechanism:

¹⁴²⁾If there is no identifier, the type can, within the translation unit, only be referred to by the declaration of which it is a part. Of course, when the declaration is of a typedef name, subsequent declarations can make use of that typedef name to declare objects having the specified structure, union, or enumerated type.

¹⁴³⁾A similar construction for an **enum** that does not contain a fixed underlying type does not exist. Enumerations with a fixed underlying type are always complete after the enum type specifier.

```
typedef struct tnode TNODE;
struct tnode {
    int count;
    TNODE *left, *right;
};
TNODE s, *sp;
```

EXAMPLE 4 To illustrate the use of prior declaration of a tag to specify a pair of mutually referential structures, the declarations

```
struct s1 { struct s2 *s2p; /* ... */ }; // D1
struct s2 { struct s1 *s1p; /* ... */ }; // D2
```

specify a pair of structures that contain pointers to each other. If **s2** were already declared as a tag in an enclosing scope, the declaration **D1** would refer to *it*, not to the tag **s2** declared in **D2**. To eliminate this context sensitivity, the declaration

```
struct s2;
```

can be inserted ahead of **D1**. This declares a new tag **s2** in the inner scope; the declaration **D2** then completes the specification of the new type.

Forward references: declarators (6.7.7), type definitions (6.7.9).

6.7.3.5 Atomic type specifiers

Syntax

atomic-type-specifier:

```
_Atomic ( type-name )
```

Constraints

Atomic type specifiers shall not be used if the implementation does not support atomic types (see 6.10.10.4).

The type name in an atomic type specifier shall not refer to an array type, a function type, an atomic type, or a qualified type.

Semantics

The properties associated with atomic types are meaningful only for expressions that are lvalues. If the **_Atomic** keyword is immediately followed by a left parenthesis, it is interpreted as a type specifier (with a type name), not as a type qualifier.

6.7.3.6 Typedef specifiers

Syntax

typedef-specifier:

```
typedef ( typedef-specifier-argument )
typedef_unqual ( typedef-specifier-argument )
```

typedef-specifier-argument:

```
expression
type-name
```

The **typedef** and **typedef_unqual** tokens are collectively called the *typedef operators*.

Constraints

The typedef operators shall not be applied to an expression that designates a bit-field member.

Semantics

The `typeof` specifier applies the `typeof` operators to an *expression* (6.5.1) or a type name. If the `typeof` operators are applied to an expression, they yield the type of their operand.¹⁴⁴⁾ Otherwise, they designate the same type as the type name with any nested `typeof` specifier evaluated.¹⁴⁵⁾ If the type of the operand is a variably modified type, the operand is evaluated; otherwise, the operand is not evaluated.

The result of the `typeof_unqual` operator is the non-atomic unqualified version of the type that would result from the `typeof` operator.¹⁴⁶⁾ The `typeof` operator preserves all qualifiers.

EXAMPLE 1 Type of an expression.

```
typeof(1+1) main () {
    return 0;
}
```

is equivalent to this program:

```
int main () {
    return 0;
}
```

EXAMPLE 2 The following program:

```
const _Atomic int purr = 0;
const int meow = 1;
const char* const animals[] = {
    "aardvark",
    "bluejay",
    "catte",
};

typeof_unqual(meow) main (int argc, char* argv[]) {
    typeof_unqual(purr)      plain_purr;
    typeof(_Atomic typeof(meow)) atomic_meow;
    typeof(animals)          animals_array;
    typeof_unqual(animals)   animals2_array;
    return 0;
}
```

is equivalent to this program:

```
const _Atomic int purr = 0;
const int meow = 1;
const char* const animals[] = {
    "aardvark",
    "bluejay",
    "catte",
};

int main (int argc, char* argv[]) {
    int      plain_purr;
    const _Atomic int atomic_meow;
    const char* const animals_array[3];
    const char*      animals2_array[3];
    return 0;
}
```

¹⁴⁴⁾When applied to a parameter declared to have array or function type, the `typeof` operators yield the adjusted (pointer) type (see 6.9.2).

¹⁴⁵⁾If the `typeof` specifier argument is itself a `typeof` specifier, the operand will be evaluated before evaluating the current `typeof` operator. This happens recursively until a `typeof` specifier is no longer the operand.

¹⁴⁶⁾`_Atomic (type-name)`, with parentheses, is considered an `_Atomic`-qualified type.

```
}

```

EXAMPLE 3 The equivalence between **sizeof** and **typeof**'s deduction of the type means this program has no constraint violations:

```
int main (int argc, char* argv[]) {
    static_assert(sizeof(typeof('p')) == sizeof(int));
    static_assert(sizeof(typeof('p')) == sizeof('p'));
    static_assert(sizeof(typeof((char)'p')) == sizeof(char));
    static_assert(sizeof(typeof((char)'p')) == sizeof((char)'p'));
    static_assert(sizeof(typeof("meow")) == sizeof(char[5]));
    static_assert(sizeof(typeof("meow")) == sizeof("meow"));
    static_assert(sizeof(typeof(argc)) == sizeof(int));
    static_assert(sizeof(typeof(argc)) == sizeof(argc));
    static_assert(sizeof(typeof(argv)) == sizeof(char**));
    static_assert(sizeof(typeof(argv)) == sizeof(argv));

    static_assert(sizeof(typeof_unqual('p')) == sizeof(int));
    static_assert(sizeof(typeof_unqual('p')) == sizeof('p'));
    static_assert(sizeof(typeof_unqual((char)'p')) == sizeof(char));
    static_assert(sizeof(typeof_unqual((char)'p')) == sizeof((char)'p'));
    static_assert(sizeof(typeof_unqual("meow")) == sizeof(char[5]));
    static_assert(sizeof(typeof_unqual("meow")) == sizeof("meow"));
    static_assert(sizeof(typeof_unqual(argc)) == sizeof(int));
    static_assert(sizeof(typeof_unqual(argc)) == sizeof(argc));
    static_assert(sizeof(typeof_unqual(argv)) == sizeof(char**));
    static_assert(sizeof(typeof_unqual(argv)) == sizeof(argv));
    return 0;
}
```

EXAMPLE 4 The following program with nested **typeof(...)**:

```
int main (int argc, char*[]) {
    float val = 6.0f;
    return (typeof(typeof_unqual(typeof(argc))))val;
}
```

is equivalent to this program:

```
int main (int argc, char*[]) {
    float val = 6.0f;
    return (int)val;
}
```

EXAMPLE 5 Variable length arrays with **typeof** operators performs the operation at execution time rather than translation time.

```
#include <stddef.h>

size_t vla_size (int n) {
    typedef char vla_type[n + 3];
    vla_type b; // variable length array
    return sizeof(
        typeof_unqual(b)
    ); // execution-time sizeof, translation-time typeof operation
}

int main () {
    return (int)vla_size(10); // vla_size returns 13
}
```

EXAMPLE 6 Nested typeof operators, arrays, and pointers do not perform array to pointer decay.

```
int main () {
    typeof(typeof(const char*))[4] y = {
        "a",
        "b",
        "c",
        "d"
    }; // 4-element array of "pointer to const char"
    return 0;
}
```

EXAMPLE 7 Function, pointer, and array types can be substituted with typeof operations.

```
void f(int);

typeof(f(5)) g(double x) {           // g has type "void(double)"
    printf("value %g\n", x);
}

typeof(g)* h;                        // h has type "void(*) (double)"
typeof(true ? g : nullptr) k;       // k has type "void(*) (double)"

void j(double A[5], typeof(A)* B); // j has type "void(double*, double**)"

extern typeof(double[]) D;           // D has an incomplete type
typeof(D) C = { 0.7, 99 };           // C has type "double[2]"

typeof(D) D = { 5, 8.9, 0.1, 99 }; // D is now completed to "double[4]"
typeof(D) E;                        // E has type "double[4]" from D's completed type
```

6.7.4 Type qualifiers

6.7.4.1 General

Syntax

type-qualifier:

const
restrict
volatile
_Atomic

Constraints

Types other than pointer types whose referenced type is an object type and (possibly multi-dimensional) array types with such pointer types as element type shall not be restrict-qualified.

The **_Atomic** qualifier shall not be used if the implementation does not support atomic types (see 6.10.10.4).

The type modified by the **_Atomic** qualifier shall not be an array type or a function type.

Semantics

The properties associated with qualified types are meaningful only for expressions that are lvalues.¹⁴⁷⁾

If the same qualifier appears more than once in the same specifier-qualifier list or as declaration specifiers, either directly, via one or more typeof specifiers, or via one or more typedefs, the behavior is the same as if it appeared only once. If other qualifiers appear along with the **_Atomic** qualifier

¹⁴⁷⁾The implementation can place a **const** object that is not **volatile** in a read-only region of storage. Moreover, the implementation is not expected to allocate storage for such an object if its address is never used.

the resulting type is the so-qualified atomic type.

If an attempt is made to modify an object defined with a const-qualified type through use of an lvalue with non-const-qualified type, the behavior is undefined. If an attempt is made to refer to an object defined with a volatile-qualified type through use of an lvalue with non-volatile-qualified type, the behavior is undefined.¹⁴⁸⁾

An object that has volatile-qualified type can be modified in ways unknown to the implementation or have other unknown side effects. Therefore, any expression referring to such an object shall be evaluated strictly according to the rules of the abstract machine, as described in 5.2.2.4. Furthermore, at every sequence point the value last stored in the object shall agree with that prescribed by the abstract machine, except as modified by the unknown factors mentioned previously.¹⁴⁹⁾ What constitutes an access to an object that has volatile-qualified type is implementation-defined.

An object that is accessed through a restrict-qualified pointer has a special association with that pointer. This association, defined in 6.7.4.2, requires that all accesses to that object use, directly or indirectly, the value of that pointer.¹⁵⁰⁾ The intended use of the **restrict** qualifier (like the **register** storage class) is to promote optimization, and deleting all instances of the qualifier from all preprocessing translation units composing a conforming program does not change its meaning (i.e. observable behavior), unless **_Generic** is used to distinguish whether or not a type has that qualifier.

If the specification of an array type includes any type qualifiers, both the array and the element type are so-qualified.¹⁵¹⁾ If the specification of a function type includes any type qualifiers, the behavior is undefined.¹⁵¹⁾

For two qualified types to be compatible, both shall have the identically qualified version of a compatible type; the order of type qualifiers within a list of specifiers or qualifiers does not affect the specified type.

EXAMPLE 1 An object declared

```
extern const volatile int real_time_clock;
```

can be modifiable by hardware, but cannot be assigned to, incremented, or decremented.

EXAMPLE 2 The following declarations and expressions illustrate the behavior when type qualifiers modify an aggregate type:

```
const struct s { int mem; } cs = { 1 };
struct s ncs; // the object ncs is modifiable
typedef int A[2][3];
const A a = {{4, 5, 6}, {7, 8, 9}}; // array of array of const int
int *pi;
const int *pci;

ncs = cs; // valid
cs = ncs; // violates modifiable lvalue constraint for =
pi = &ncs.mem; // valid
pi = &cs.mem; // violates type constraints for =
pci = &cs.mem; // valid
pi = a[0]; // invalid: a[0] has type "const int *"
```

EXAMPLE 3 The declaration

¹⁴⁸⁾This applies to those objects that behave as if they were defined with qualified types, even if they are never actually defined as objects in the program (such as an object at a memory-mapped input/output address).

¹⁴⁹⁾A **volatile** declaration can be used to describe an object corresponding to a memory-mapped input/output port or an object accessed by an asynchronously interrupting function. Actions on objects so declared are not allowed to be "optimized out" by an implementation or reordered except as permitted by the rules for evaluating expressions.

¹⁵⁰⁾For example, a statement that assigns a value returned by **malloc** to a single pointer establishes this association between the allocated object and the pointer.

¹⁵¹⁾This can occur with **typedef** s. This rule does not apply to the **_Atomic** qualifier, and that qualifiers do not have any direct effect on the array type itself, but affect conversion rules for pointer types that reference an array type.


```
_Atomic volatile int *p;
```

specifies that **p** has the type “pointer to volatile atomic int”, a pointer to a volatile-qualified atomic type.

6.7.4.2 Formal definition of restrict

Let **D** be a declaration of an ordinary identifier that provides a means of designating an object **P** as a restrict-qualified pointer to type **T**.

If **D** appears inside a block and does not have storage class **extern**, let **B** denote the block. If **D** appears in the list of parameter declarations of a function definition, let **B** denote the associated block. Otherwise, let **B** denote the block of **main** (or the block of whatever function is called at program startup in a freestanding environment).

In what follows, a pointer expression **E** is said to be *based* on object **P** if (at some sequence point in the execution of **B** prior to the evaluation of **E**) modifying **P** to point to a copy of the array object into which it formerly pointed would change the value of **E**.¹⁵²⁾ “based” is defined only for expressions with pointer types.

During each execution of **B**, let **L** be any lvalue that has **&L** based on **P**. If **L** is used to access the value of the object **X** that it designates, and **X** is also modified (by any means), then the following requirements apply: **T** shall not be const-qualified. Every other lvalue used to access the value of **X** shall also have its address based on **P**. Every access that modifies **X** shall be considered also to modify **P**, for the purposes of this subclause. If **P** is assigned the value of a pointer expression **E** that is based on another restricted pointer object **P2**, associated with block **B2**, then either the execution of **B2** shall begin before the execution of **B**, or the execution of **B2** shall end prior to the assignment. If these requirements are not met, then the behavior is undefined.

Here an execution of **B** means that portion of the execution of the program that would correspond to the lifetime of an object with scalar type and automatic storage duration associated with **B**.

A translator is free to ignore any or all aliasing implications of uses of **restrict**.

EXAMPLE 1 The file scope declarations

```
int * restrict a;  
int * restrict b;  
extern int c[];
```

assert that if an object is accessed using one of **a**, **b**, or **c**, and that object is modified anywhere in the program, then it is never accessed using either of the other two.

EXAMPLE 2 The function parameter declarations in the following example

```
void f(int n, int * restrict p, int * restrict q)  
{  
    while (n-- > 0)  
        *p++ = *q++;  
}
```

assert that, during each execution of the function, if an object is accessed through one of the pointer parameters, then it is not also accessed through the other. The translator can make this no-aliasing inference based on the parameter declarations alone, without analyzing the function body.

The benefit of the **restrict** qualifiers is that they enable a translator to make an effective dependence analysis of function **f** without examining any of the calls of **f** in the program. The cost is that the programmer has to examine all those calls to ensure that none give undefined behavior. For example, the second call of **f** in **g** has undefined behavior because each of **d[1]** through **d[49]** is accessed through both **p** and **q**.

```
void g(void)  
{
```

¹⁵²⁾In other words, **E** depends on the value of **P** itself rather than on the value of an object referenced indirectly through **P**. For example, if identifier **p** has type **(int **restrict)**, then the pointer expressions **p** and **p+1** are based on the restricted pointer object designated by **p**, but the pointer expressions ***p** and **p[1]** are not.

```

extern int d[100];
f(50, d + 50, d); // valid
f(50, d + 1, d); // undefined behavior
}

```

EXAMPLE 3 The function parameter declarations

```

void h(int n, int * restrict p, int * restrict q, int * restrict r)
{
    int i;
    for (i = 0; i < n; i++)
        p[i] = q[i] + r[i];
}

```

illustrate how an unmodified object can be aliased through two restricted pointers. If **a** and **b** are disjoint arrays, a call of the form **h(100, a, b, b)** has defined behavior, because array **b** is not modified within function **h**.

EXAMPLE 4 The rule limiting assignments between restricted pointers does not distinguish between a function call and an equivalent nested block. With one exception, only “outer-to-inner” assignments between restricted pointers declared in nested blocks have defined behavior.

```

{
    int * restrict p1;
    int * restrict q1;
    p1 = q1; // undefined behavior
    {
        int * restrict p2 = p1; // valid
        int * restrict q2 = q1; // valid
        p1 = q2;                // undefined behavior
        p2 = q2;                // undefined behavior
    }
}

```

The one exception allows the value of a restricted pointer to be carried out of the block in which it (or, more precisely, the ordinary identifier used to designate it) is declared when that block finishes execution. For example, this permits **new_vector** to return a **vector**.

```

typedef struct { int n; float * restrict v; } vector;
vector new_vector(int n)
{
    vector t;
    t.n = n;
    t.v = malloc(n * sizeof(float));
    return t;
}

```

EXAMPLE 5 Suppose that a programmer knows that references of the form **p[i]** and **q[j]** are never aliases in the body of a function:

```

void f(int n, int *p, int *q) { /* ... */ }

```

There are several ways that this information can be conveyed to a translator using the **restrict** qualifier. Example 2 shows the most effective way, qualifying all pointer parameters, and can be used provided that neither **p** nor **q** becomes based on the other in the function body. A potentially effective alternative is:

```

void f(int n, int * restrict p, int * const q) { /* ... */ }

```

Again, it is possible for a translator to make the no-aliasing inference based on the parameter declarations alone, though now subtler reasoning is used: that the **const**-qualification of **q** precludes it becoming based on **p**. There is also a requirement that **q** is not modified, so this alternative cannot be used for the function in Example 2, as written.

EXAMPLE 6 Another potentially effective alternative is:

```
void f(int n, int *p, int const * restrict q) { /* ... */ }
```

Again, it is possible for a translator to make the no-aliasing inference based on the parameter declarations alone, though now even subtler reasoning is used: that this combination of **restrict** and **const** means that objects referenced using **q** cannot be modified, and so no modified object can be referenced using both **p** and **q**.

EXAMPLE 7 The least effective alternative is:

```
void f(int n, int * restrict p, int *q) { /* ... */ }
```

Here the translator can make the no-aliasing inference only by analyzing the body of the function and proving that **q** cannot become based on **p**. Some translator designs can choose to exclude this analysis, given availability of the more effective alternatives described previously. Such a translator is required to assume that aliases are present because assuming that aliases are not present can result in an incorrect translation. Also, a translator that attempts the analysis can potentially not succeed in all cases and consequently need to conservatively assume that aliases are present.

6.7.5 Function specifiers

Syntax

function-specifier:

```
inline
_Noreturn
```

Constraints

Function specifiers shall be used only in the declaration of an identifier for a function.

An inline definition of a function with external linkage shall not contain, anywhere in the tokens making up the function definition, a definition of a modifiable object with static or thread storage duration, and shall not contain, anywhere in the tokens making up the function definition, a reference to an identifier with internal linkage.

In a hosted environment, no function specifier(s) shall appear in a declaration of **main**.

Semantics

A function specifier can appear more than once; the behavior is the same as if it appeared only once.

A function declared with an **inline** function specifier is an *inline function*. Making a function an inline function suggests that calls to the function be as fast as possible.¹⁵³⁾ The extent to which such suggestions are effective is implementation-defined.¹⁵⁴⁾

Any function with internal linkage can be an inline function. For a function with external linkage, the following restrictions apply: If a function is declared with an **inline** function specifier, then it shall also be defined in the same translation unit. If all the file scope declarations for a function in a translation unit include the **inline** function specifier without **extern**, then the definition in that translation unit is an *inline definition*. An inline definition does not provide an external definition for the function and does not forbid an external definition in another translation unit. Inline definitions provide an alternative to external definitions, which a translator can use to implement any call to the function in the same translation unit. It is unspecified whether a call to the function uses the inline definition or the external definition.¹⁵⁵⁾

¹⁵³⁾By using, for example, an alternative to the usual function call mechanism, such as “inline substitution”. Inline substitution is not textual substitution, nor does it create a new function. Therefore, for example, the expansion of a macro used within the body of the function uses the definition it had at the point the function body appears, and not where the function is called; and identifiers refer to the declarations in scope where the body occurs. Likewise, the function has a single address, regardless of the number of inline definitions that occur in addition to the external definition.

¹⁵⁴⁾For example, an implementation can possibly never perform inline substitution, or can only perform inline substitutions to calls in the scope of an **inline** declaration.

¹⁵⁵⁾Since an inline definition is distinct from the corresponding external definition and from any other corresponding inline

A function declared with a **_Noreturn** function specifier shall not return to its caller. The attribute **[[noreturn]]** provides similar semantics. The **_Noreturn** function specifier is an obsolescent feature (6.7.13.7).

Recommended practice

The implementation should produce a diagnostic message for a function declared with a **_Noreturn** function specifier that appears to be capable of returning to its caller.

EXAMPLE 1 The declaration of an inline function with external linkage can result in either an external definition, or a definition available for use only within the translation unit. A file scope declaration with **extern** creates an external definition. The following example shows an entire translation unit.

```
inline double fahr(double t)
{
    return (9.0 * t) / 5.0 + 32.0;
}

inline double cels(double t)
{
    return (5.0 * (t - 32.0)) / 9.0;
}

extern double fahr(double); // creates an external definition

double convert(int is_fahr, double temp)
{
    /* A translator can perform inline substitutions */
    return is_fahr ? cels(temp) : fahr(temp);
}
```

The definition of **fahr** is an external definition because **fahr** is also declared with **extern**, but the definition of **cels** is an inline definition. Because **cels** has external linkage and is referenced, an external definition has to appear in another translation unit (see 6.9); the inline definition and the external definition are distinct and either can be used for the call.

EXAMPLE 2 The following inline definitions are invalid:

```
static int a;
typeof (a) inline f() { return 0; }
typeof ((int) { 0 }) inline g() { return 0; }
```

Forward references: function definitions (6.9.2).

6.7.6 Alignment specifier

Syntax

alignment-specifier:

```
alignas ( type-name )
alignas ( constant-expression )
```

Constraints

An alignment specifier shall appear only in the declaration specifiers of a declaration, or in the *specifier-qualifier* list of a member declaration, or in the type name of a compound literal. An alignment specifier shall not be used in conjunction with either of the storage-class specifiers **typedef** or **register**, nor in a declaration of a function or bit-field.

The constant expression shall be an integer constant expression. It shall evaluate to a valid fundamental alignment, or to a valid extended alignment supported by the implementation for an object

definitions in other translation units, all corresponding objects with static storage duration are also distinct in each of the definitions.

of the storage duration (if any) being declared, or to zero.

An object shall not be declared with an over-aligned type with an extended alignment requirement not supported by the implementation for an object of that storage duration.

The combined effect of all alignment specifiers in a declaration shall not specify an alignment that is less strict than the alignment that would otherwise be required for the type of the object or member being declared.

Semantics

The first form is equivalent to **alignas**(**alignof**(*type-name*)).

The alignment requirement of the declared object or member is taken to be the specified alignment. An alignment specification of zero has no effect.¹⁵⁶⁾ When multiple alignment specifiers occur in a declaration, the effective alignment requirement is the strictest specified alignment.

If the definition of an object has an alignment specifier, any other declaration of that object shall either specify equivalent alignment or have no alignment specifier. If the definition of an object does not have an alignment specifier, any other declaration of that object shall also have no alignment specifier. If declarations of an object in different translation units have different alignment specifiers, the behavior is undefined.

6.7.7 Declarators

6.7.7.1 General

Syntax

declarator:

*pointer*_{opt} *direct-declarator*

direct-declarator:

identifier *attribute-specifier-sequence*_{opt}

(*declarator*)

array-declarator *attribute-specifier-sequence*_{opt}

function-declarator *attribute-specifier-sequence*_{opt}

array-declarator:

direct-declarator [*type-qualifier-list*_{opt} *assignment-expression*_{opt}]

direct-declarator [**static** *type-qualifier-list*_{opt} *assignment-expression*]

direct-declarator [*type-qualifier-list* **static** *assignment-expression*]

direct-declarator [*type-qualifier-list*_{opt} *]

function-declarator:

direct-declarator (*parameter-type-list*_{opt})

pointer:

* *attribute-specifier-sequence*_{opt} *type-qualifier-list*_{opt}

* *attribute-specifier-sequence*_{opt} *type-qualifier-list*_{opt} *pointer*

type-qualifier-list:

type-qualifier

type-qualifier-list *type-qualifier*

parameter-type-list:

parameter-list

parameter-list , ...

...

parameter-list:

parameter-declaration

parameter-list , *parameter-declaration*

¹⁵⁶⁾ An alignment specification of zero also does not affect other alignment specifications in the same declaration.

parameter-declaration:

*attribute-specifier-sequence*_{opt} *declaration-specifiers* *declarator*
*attribute-specifier-sequence*_{opt} *declaration-specifiers* *abstract-declarator*_{opt}

Semantics

Each declarator declares an identifier for a single object, function, or type, within a declaration. The preceding specifiers indicate the type, storage class, or other properties of the identifier or identifiers being declared. Each declarator specifies one declaration and names it and/or modifies the type of the specifiers with operators such as *** (pointer to) and *()* (function returning).

A *full declarator* is a declarator that is not part of another declarator. If, in the nested sequence of declarators in a full declarator, there is a declarator specifying a variable length array type, the type specified by the full declarator is said to be *variably modified*. Furthermore, any type derived by declarator type derivation from a variably modified type is itself variably modified.

In the following subclauses, consider a declaration

T D1

where **T** contains the declaration specifiers that specify a type *T* (such as **int**) and **D1** is a declarator that contains an identifier *ident*. The type specified for the identifier *ident* in the various forms of declarator is described inductively using this notation.

If, in the declaration “**T D1**”, **D1** has the form

identifier *attribute-specifier-sequence*_{opt}

then the type specified for *ident* is *T* and the optional attribute specifier sequence appertains to the entity that is declared.

If, in the declaration “**T D1**”, **D1** has the form

(D)

then *ident* has the type specified by the declaration “**T D**”. Thus, a declarator in parentheses is identical to the unparenthesized declarator, but the binding of complicated declarators can be altered by parentheses.

Implementation limits

As discussed in 5.3.5.2, an implementation may limit the number of pointer, array, and function declarators that modify an arithmetic, structure, union, or **void** type, either directly or via one or more **typedefs**.

Forward references: array declarators (6.7.7.3), type definitions (6.7.9).

6.7.7.2 Pointer declarators

Semantics

If, in the declaration “**T D1**”, **D1** has the form

*** *attribute-specifier-sequence*_{opt} *type-qualifier-list*_{opt} **D**

and the type specified for *ident* in the declaration “**T D**” is “*derived-declarator-type-list T*”, then the type specified for *ident* is “*derived-declarator-type-list type-qualifier-list* pointer to *T*”. For each type qualifier in the list, *ident* is a so-qualified pointer. The optional attribute specifier sequence appertains to the pointer and not the object pointed to.

For two pointer types to be compatible, both shall be identically qualified and both shall be pointers to compatible types.

EXAMPLE The following pair of declarations demonstrates the difference between a “variable pointer to a constant value” and a “constant pointer to a variable value”.

```
const int *ptr_to_constant;
int *const constant_ptr;
```


The contents of any object pointed to by **ptr_to_constant** cannot be modified through that pointer, but **ptr_to_constant** itself can be changed to point to another object. Similarly, the contents of the **int** pointed to by **constant_ptr** can be modified, but **constant_ptr** itself always points to the same location.

The declaration of the constant pointer **constant_ptr** can be clarified by including a definition for the type “pointer to **int**”.

```
typedef int *int_ptr;
const int_ptr constant_ptr;
```

declares **constant_ptr** as an object that has type “const-qualified pointer to **int**”.

6.7.7.3 Array declarators

Constraints

In addition to optional type qualifiers and the keyword **static**, the [and] can delimit an expression or *. If they delimit an expression (which specifies the size of an array), the expression shall have an integer type. If the expression is a constant expression, it shall have a value greater than zero. The element type shall not be an incomplete or function type. The optional type qualifiers and the keyword **static** shall appear only in a declaration of a function parameter with an array type, and then only in the outermost array type derivation.

If an identifier is declared as having a variably modified type, it shall be an ordinary identifier (as defined in 6.2.3), have no linkage, and have either block scope or function prototype scope. If an identifier is declared to be an object with static or thread storage duration, it shall not have a variable length array type.

Semantics

If, in the declaration “**T D1**”, **D1** has one of the forms:

```
D [ type-qualifier-listopt assignment-expressionopt ] attribute-specifier-sequenceopt
D [ static type-qualifier-listopt assignment-expression ] attribute-specifier-sequenceopt
D [ type-qualifier-list static assignment-expression ] attribute-specifier-sequenceopt
D [ type-qualifier-listopt * ] attribute-specifier-sequenceopt
```

and the type specified for *ident* in the declaration “**T D**” is “*derived-declarator-type-list T*”, then the type specified for *ident* is “*derived-declarator-type-list array of T*”.¹⁵⁷⁾¹⁵⁸⁾ The optional attribute specifier sequence appertains to the array. (See 6.7.7.4 for the meaning of the optional type qualifiers and the keyword **static**.)

If the size is not present, the array type is an incomplete type. If the size is * instead of being an expression, the array type is a *variable length array* type of unspecified size, which can only be used as part of the nested sequence of declarators or abstract declarators for a parameter declaration, not including anything inside an array size expression in one of those declarators;¹⁵⁹⁾ such arrays are nonetheless complete types. If the size is an integer constant expression and the element type has a known constant size, the array type is not a variable length array type; otherwise, the array type is a *variable length array* type. (Variable length arrays with automatic storage duration are a conditional feature that implementations may support; see 6.10.10.4.)

If the size is an expression that is not an integer constant expression: if it occurs in a declaration at function prototype scope, it is treated as if it were replaced by *; otherwise, each time it is evaluated it shall have a value greater than zero. The size of each instance of a variable length array type does not change during its lifetime. Where a size expression is part of the operand of a **typeof** or **sizeof** operator and changing the value of the size expression would not affect the result of the operator, it is unspecified whether or not the size expression is evaluated. Where a size expression is part of the operand of an **alignof** operator, that expression is not evaluated.

For two array types to be compatible, both shall have compatible element types, and if both size specifiers are present, and are integer constant expressions, then both size specifiers shall have

¹⁵⁷⁾When several “array of” specifications are adjacent, a multidimensional array is declared.

¹⁵⁸⁾The array is considered identically qualified to **T** according to 6.2.5.

¹⁵⁹⁾They can be used only in function declarations that are not definitions (see 6.7.7.4 and 6.9.2).

the same constant value. If the two array types are used in a context which requires them to be compatible, the behavior is undefined if the two size specifiers evaluate to unequal values.

EXAMPLE 1

```
float fa[11], *afp[17];
```

declares an array of **float** numbers and an array of pointers to **float** numbers.

EXAMPLE 2 There is a distinction between the declarations

```
extern int *x;
extern int y[];
```

The first declares **x** to be a pointer to **int**; the second declares **y** to be an array of **int** of unknown size (an incomplete type), the storage for which is defined elsewhere.

EXAMPLE 3 The following declarations demonstrate the compatibility rules for variably modified types.

```
extern int n;
extern int m;

void fcompat(void)
{
    int a[n][6][m];
    int (*p)[4][n+1];
    int c[n][n][6][m];
    int (*r)[n][n][n+1];
    p = a;      // invalid: not compatible because 4 != 6
    r = c;      // compatible, but defined behavior only if
                // n == 6 and m == n+1
}
```

EXAMPLE 4 All valid declarations of variably modified (VM) types are either at block scope or function prototype scope. Array objects declared with the **thread_local**, **static**, or **extern** storage-class specifier cannot have a variable length array (VLA) type. However, an object declared with the **static** storage-class specifier can have a VM type (that is, a pointer to a VLA type). Finally, only ordinary identifiers can be declared with a VM type and identifiers with VM type cannot, therefore, be members of structures or unions.

```
extern int n;
int A[n];           // invalid: file scope VLA
extern int (*p2)[n]; // invalid: file scope VM
int B[100];         // valid: file scope but not VM

void fvla(int m, int C[m][m]); // valid: VLA with prototype scope

void fvla(int m, int C[m][m]) // valid: adjusted to auto pointer to VLA
{
    typedef int VLA[m][m];    // valid: block scope typedef VLA

    struct tag {
        int (*y)[n];          // invalid: y not ordinary identifier
        int z[n];             // invalid: z not ordinary identifier
    };
    int D[m];                // valid: auto VLA
    static int E[m];          // invalid: static block scope VLA
    extern int F[m];          // invalid: F has linkage and is VLA
    int (*s)[m];              // valid: auto pointer to VLA
    extern int (*r)[m];        // invalid: r has linkage and points to VLA
    static int (*q)[m] = &B;   // valid: q is a static block pointer to VLA
}
```

EXAMPLE 5 The following is invalid, because the use of `[*]` is inside an array size expression rather than directly part of the nested sequence of abstract declarators for a parameter declaration:

```
void f(int (*)[sizeof(int (*)[*])]);
```

Forward references: function declarators (6.7.7.4), function definitions (6.9.2), initialization (6.7.11).

6.7.7.4 Function declarators

Constraints

A function declarator shall not specify a return type that is a function type or an array type.

The only storage-class specifier that shall occur in a parameter declaration is **register**.

After adjustment, the parameters in a parameter type list in a function declarator that is part of a definition of that function shall not have incomplete type.

Semantics

If, in the declaration “**T D1**”, **D1** has the form

D (*parameter-type-list*_{opt}) *attribute-specifier-sequence*_{opt}

and the type specified for *ident* in the declaration “**T D**” is “*derived-declarator-type-list T*”, then the type specified for *ident* is “*derived-declarator-type-list* function returning the unqualified, non-atomic version of *T*”. The optional attribute specifier sequence appertains to the function type.

A parameter type list specifies the types of, and can declare identifiers for, the parameters of the function.

A declaration of a parameter as “array of *type*” shall be adjusted to “qualified pointer to *type*”, where the type qualifiers (if any) are those specified within the `[` and `]` of the array type derivation. If the keyword **static** also appears within the `[` and `]` of the array type derivation, then for each call to the function, the value of the corresponding actual argument shall provide access to the first element of an array with at least as many elements as specified by the size expression.

A declaration of a parameter as “function returning *type*” shall be adjusted to “pointer to function returning *type*”, as in 6.3.3.1.

If the list terminates with an ellipsis (...), no information about the number or types of the parameters after the comma is supplied.¹⁶⁰⁾

The special case of an unnamed parameter of type **void** as the only item in the list specifies that the function has no parameters.

If, in a parameter declaration, an identifier can be treated either as a typedef name or as a parameter name, it shall be taken as a typedef name.

If the function declarator is not part of a definition of that function, parameters can have incomplete type and can use the `[*]` notation in their sequences of declarator specifiers to specify variable length array types.

The storage-class specifier in the declaration specifiers for a parameter declaration, if present, is ignored unless the declared parameter is one of the members of the parameter type list for a function definition. The optional attribute specifier sequence in a parameter declaration appertains to the parameter.

For a function declarator without a parameter type list: the effect is as if it were declared with a parameter type list consisting of the keyword **void**. A function declarator provides a prototype for the function.

For two function types to be compatible, both shall specify compatible return types. Moreover, the parameter type lists shall agree in the number of parameters and in use of the final ellipsis; corresponding parameters shall have compatible types. In the determination of type compatibility and of a composite type, each parameter declared with function or array type is taken as having the

¹⁶⁰⁾The macros defined in the `<stdarg.h>` header (7.16) can be used to access arguments that correspond to the ellipsis.

adjusted type and each parameter declared with qualified type is taken as having the unqualified version of its declared type.

EXAMPLE 1 The declaration

```
int f(void), *fip(), (*pfi());
```

declares a function **f** with no parameters returning an **int**, a function **fip** with no parameters returning a pointer to an **int**, and a pointer **pfi** to a function with no parameters returning an **int**. It is especially useful to compare the last two. The binding of ***fip()** is ***(fip())**, so that the declaration suggests, and the same construction in an expression requires, the calling of a function **fip**, and then using indirection through the pointer result to yield an **int**. In the declarator **(*pfi)()**, the extra parentheses are necessary to indicate that indirection through a pointer to a function yields a function designator, which is then used to call the function; it returns an **int**.

If the declaration occurs outside of any function, the identifiers have file scope and external linkage. If the declaration occurs inside a function, the identifiers of the functions **f** and **fip** have block scope and either internal or external linkage (depending on what file scope declarations for these identifiers are visible), and the identifier of the pointer **pfi** has block scope and no linkage.

EXAMPLE 2 The declaration

```
int (*apfi[3])(int *x, int *y);
```

declares an array **apfi** of three pointers to functions returning **int**. Each of these functions has two parameters that are pointers to **int**. The identifiers **x** and **y** are declared for descriptive purposes only and go out of scope at the end of the declaration of **apfi**.

EXAMPLE 3 The declaration

```
int (*fpfi(int (*)(long), int))(int, ...);
```

declares a function **fpfi** that returns a pointer to a function returning an **int**. The function **fpfi** has two parameters: a pointer to a function returning an **int** (with one parameter of type **long int**), and an **int**. The pointer returned by **fpfi** points to a function that has one **int** parameter and accepts zero or more additional arguments of any type.

EXAMPLE 4 The following prototype has a variably modified parameter.

```
void addscalar(int n, int m,
              double a[n][n*m+300], double x);

int main(void)
{
    double b[4][308];
    addscalar(4, 2, b, 2.17);
    return 0;
}

void addscalar(int n, int m,
              double a[n][n*m+300], double x)
{
    for (int i = 0; i < n; i++)
        for (int j = 0, k = n*m+300; j < k; j++)
            // a is a pointer to a VLA with n*m+300 elements
            a[i][j] += x;
}
```

EXAMPLE 5 The following are all compatible function prototype declarators.

```
double maximum(int n, int m, double a[n][m]);
double maximum(int n, int m, double a[*][*]);
double maximum(int n, int m, double a[ ][*]);
double maximum(int n, int m, double a[ ][m]);
```

as are:

```
void f(double (* restrict a)[5]);
void f(double a[restrict][5]);
void f(double a[restrict 3][5]);
void f(double a[restrict static 3][5]);
```

The last declaration also specifies that the argument corresponding to **a** in any call to **f** can be expected to be a non-null pointer to the first of at least three arrays of 5 doubles, which the others do not.

Forward references: function definitions (6.9.2), type names (6.7.8).

6.7.8 Type names

Syntax

type-name:

*specifier-qualifier-list abstract-declarator*_{opt}
abstract-declarator:
 pointer
 *pointer*_{opt} *direct-abstract-declarator*
direct-abstract-declarator:
 (*abstract-declarator*)
 array-abstract-declarator *attribute-specifier-sequence*_{opt}
 function-abstract-declarator *attribute-specifier-sequence*_{opt}
array-abstract-declarator:
 *direct-abstract-declarator*_{opt} [*type-qualifier-list*_{opt} *assignment-expression*_{opt}]
 *direct-abstract-declarator*_{opt} [**static** *type-qualifier-list*_{opt} *assignment-expression*]
 *direct-abstract-declarator*_{opt} [*type-qualifier-list* **static** *assignment-expression*]
 *direct-abstract-declarator*_{opt} [*]

function-abstract-declarator:
 *direct-abstract-declarator*_{opt} (*parameter-type-list*_{opt})

Semantics

In several contexts, it is necessary to specify a type. This is accomplished using a *type name*, which is syntactically a declaration for a function or an object of that type that omits the identifier.¹⁶¹⁾ The optional attribute specifier sequence in a direct abstract declarator appertains to the preceding array or function type. The attribute specifier sequence affects the type only for the declaration it appears in, not other declarations involving the same type.

EXAMPLE The constructions

```
(a)      int
(b)      int *
(c)      int *[3]
(d)      int (*)[3]
(e)      int (*)[*]
(f)      int *()
(g)      int (*)(void)
(h)      int (*const [])(unsigned int, ...)
```

name respectively the types

- (a) **int**,
- (b) pointer to **int**,
- (c) array of three pointers to **int**,

¹⁶¹⁾As indicated by the syntax, empty parentheses in a type name are interpreted as “function with no parameters”, rather than redundant parentheses around the omitted identifier.

- (d) pointer to an array of three **int** s,
- (e) pointer to a variable length array of an unspecified number of **int** s,
- (f) function with no parameters returning a pointer to **int**,
- (g) pointer to function with no parameters returning an **int**, and
- (h) array of an unspecified number of constant pointers to functions, each with one parameter that has type **unsigned int** and an unspecified number of other parameters, returning an **int**.

6.7.9 Type definitions

Syntax

typedef-name:
 identifier

Constraints

If a typedef name specifies a variably modified type then it shall have block scope.

Semantics

In a declaration whose storage-class specifier is **typedef**, each declarator defines an identifier to be a typedef name that denotes the type specified for the identifier in the way described in 6.7.7. Any array size expressions associated with variable length array declarators and typeof operators are evaluated each time the declaration of the typedef name is reached in the order of execution. A **typedef** declaration does not introduce a new type, only a synonym for the type so specified. That is, in the following declarations:

```
typedef T type_ident;
type_ident D;
```

type_ident is defined as a typedef name with the type specified by the declaration specifiers in **T** (known as *T*), and the identifier in **D** has the type “*derived-declarator-type-list T*” where the *derived-declarator-type-list* is specified by the declarators of **D**. A typedef name shares the same name space as other identifiers declared in ordinary declarators. If the identifier is redeclared in an enclosed block, the type of the inner declaration shall not be inferred (6.7.10).

EXAMPLE 1 After

```
typedef int MILES, KCLICKSP();
typedef struct { double hi, lo; } range;
```

the constructions

```
MILES distance;
extern KCLICKSP *metricp;
range x;
range z, *zp;
```

are all valid declarations. The type of **distance** is **int**, that of **metricp** is “pointer to function with no parameters returning **int**”, and that of **x** and **z** is the specified structure; **zp** is a pointer to such a structure. The object **distance** has a type compatible with any other **int** object.

EXAMPLE 2 After the declarations

```
typedef struct s1 { int x; } t1, *tp1;
typedef struct s2 { int x; } t2, *tp2;
```

type **t1** and the type pointed to by **tp1** are compatible. Type **t1** is also compatible with type **struct s1**, but not compatible with the types **struct s2**, **t2**, the type pointed to by **tp2**, or **int**.

EXAMPLE 3 The following obscure constructions

```
typedef signed int t;
typedef int plain;
struct tag {
    unsigned t:4;
    const t:5;
    plain r:5;
};
```

declare a typedef name **t** with type **signed int**, a typedef name **plain** with type **int**, and a structure with three bit-field members, one named **t** that contains values in the range [0, 15], an unnamed const-qualified bit-field which (if it can be accessed) would contain values in the range [−16, +15], and one named **r** that contains values in one of the ranges [0, 31] or [−16, +15]. (The choice of range is implementation-defined.) The first two bit-field declarations differ in that **unsigned** is a type specifier (which forces **t** to be the name of a structure member), while **const** is a type qualifier (which modifies **t** which is still visible as a typedef name). If these declarations are followed in an inner scope by

```
t f(t (t));
long t;
```

then a function **f** is declared with type “function returning **signed int** with one unnamed parameter with type pointer to function returning **signed int** with one unnamed parameter with type **signed int**”, and an identifier **t** with type **long int**.

EXAMPLE 4 On the other hand, typedef names can be used to improve code readability. All three of the following declarations of the **signal** function specify exactly the same type, the first without making use of any typedef names.

```
typedef void fv(int), (*pfv)(int);

void (*signal(int, void (*)(int)))(int);
fv *signal(int, fv *);
pfv signal(int, pfv);
```

EXAMPLE 5 If a typedef name denotes a variable length array type, the length of the array is fixed at the time the typedef name is defined, not each time it is used:

```
void copyt(int n)
{
    typedef int B[n]; // B is n ints, n evaluated now
    n += 1;
    B a; // a is n ints, n without += 1
    int b[n]; // a and b are different sizes
    for (int i = 1; i < n; i++)
        a[i-1] = b[i];
}
```

6.7.10 Type inference

Constraints

A declaration for which the type is inferred shall contain the storage-class specifier **auto**.

Semantics

For such a declaration that is the definition of an object the init-declarator shall have the form

direct-declarator = *assignment-expression*

The inferred type of the declared object is the type of the assignment expression after lvalue, array to pointer or function to pointer conversion, additionally qualified by qualifiers and amended by attributes as they appear in the declaration specifiers, if any.¹⁶²⁾ Implementations can accept a direct

¹⁶²⁾The scope rules as described in 6.2.1 also prohibit the use of the identifier of the declarator within the assignment expression.

declarator that is not of the form

identifier attribute-specifier-sequence_{opt}

optionally enclosed in balanced pairs of parentheses; if a direct declarator of a different form is accepted, the behavior is implementation-defined.¹⁶³⁾

NOTE A declaration that also defines a structure or union type has implementation-defined behavior. Here, the identifier **x** which is not ordinary but in the name space of the structure type is declared.

```
auto p = (struct { int x; } *)0;
```

Even a forward declaration of a structure tag

```
struct s;
auto p = (struct s { int x; } *)0;
```

would not change that situation. A direct use of the structure definition as the type specifier ensures portability of the declaration.

```
struct s { int x; } * p = 0;
```

The following also has implementation-defined behavior:

```
auto alignas (struct s *) x = 0;
```

EXAMPLE 1 The following file scope definitions:

```
static auto a = 3.5;
auto p = &a;
```

are interpreted as if they had been written as:

```
static double a = 3.5;
double * p = &a;
```

So effectively **a** is a **double** and **p** is a **double***. The restrictions on the syntax of such declarations does not allow the declarator to be ***p**, but that the final type here nevertheless is a pointer type.

EXAMPLE 2 The scope of the identifier for which the type is inferred only starts after the end of the initializer (6.2.1), so the assignment expression cannot use the identifier to refer to the object or function that is declared, for example to take its address. Any use of the identifier in the initializer is invalid, even if an entity with the same name exists in an outer scope.

```
{
    double a = 7;
    double b = 9;
    {
        double b = b * b; // undefined, uses uninitialized
                          // variable without address
        printf("%g\n", a); // valid, uses "a" from outer scope, prints 7
        auto a = a * a; // invalid, "a" from outer scope is not
                       // visible during initialization
    }
    {
        auto b = a * a; // valid, uses "a" from outer scope
        auto a = b;    // valid, "a" from outer scope not visible now
        // ...
        printf("%g\n", a); // valid, uses "a" from inner scope, prints 49
    }
}
```

¹⁶³⁾It is recommended that implementations that accept different forms of direct declarators follow the syntax and semantics of the corresponding feature in ISO/IEC 14882.


```
        // ...
    }
```

EXAMPLE 3 In the following, declarations of **pA** and **qA** are valid. The type of **A** after array-to-pointer conversion is a pointer type, and **qA** is a pointer to array.

```
double A[3] = { 0 };
auto pA = A;
auto qA = &A;
```

EXAMPLE 4 Type inference can be used to capture the type of a call to a type-generic function. It ensures that the same type as the argument **x** is used.

```
#include <tgmath.h>
auto y = cos(x);
```

If instead the type of **y** is explicitly specified to a different type than **x**, a diagnosis of the mismatch is not enforced.

EXAMPLE 5 A type-generic macro that generalizes the **div** functions (7.24.7.2) is defined and used as follows.

```
#define div(X, Y) _Generic((X)+(Y),\
    int: div,\
    long: ldiv,\
    long long: lldiv)((X), (Y))
auto z = div(x, y);
auto q = z.quot;
auto r = z.rem;
```

EXAMPLE 6 Definitions of objects with inferred type are valid in all contexts that allow the initializer syntax as described. In particular they can be used to ensure type safety of **for**-loop controlling expressions.

```
for (auto i = j; i < 2*j; ++i) {
    // ...
}
```

Here, regardless of the integer rank or signedness of the type of **j**, **i** will have the non-atomic unqualified version of **j**'s type. So, after lvalue conversion and possible promotion, the two operands of the **<** operator in the controlling expression are guaranteed to have the same type, and, in particular, the same signedness.

6.7.11 Initialization

Syntax

braced-initializer:

```
{ }
{ initializer-list }
{ initializer-list , }
```

initializer:

```
assignment-expression
braced-initializer
```

initializer-list:

```
designationopt initializer
initializer-list , designationopt initializer
```

designation:

```
designator-list =
```

designator-list:

designator
designator-list designator

designator:

[*constant-expression*]
 . *identifier*

An empty brace pair ({}) is called an *empty initializer* and is referred to as *empty initialization*.

Constraints

No initializer shall attempt to provide a value for an object not contained within the entity being initialized.

The type of the entity to be initialized shall be an array of unknown size or a complete object type. An entity of variable length array type shall not be initialized except by an empty initializer. An array of unknown size shall not be initialized by an empty initializer.

All the expressions in an initializer for an object that has static or thread storage duration or is declared with the **constexpr** storage-class specifier shall be constant expressions or string literals.

If the declaration of an identifier has block scope, and the identifier has external or internal linkage, the declaration shall have no initializer for the identifier.

If a designator has the form

[*constant-expression*]

then the current object (defined subsequently in this subclause) shall have array type and the expression shall be an integer constant expression. If the array is of unknown size, any nonnegative value is valid.

If a designator has the form

. *identifier*

then the current object (defined subsequently in this subclause) shall have structure or union type and the identifier shall be the name of a member of that type.

Semantics

An initializer specifies the initial value stored in an object. For objects with atomic type additional restrictions apply, see 7.17.2 and 7.17.8.

Except where explicitly stated otherwise, for the purposes of this subclause unnamed members of objects of structure and union type do not participate in initialization. Unnamed members of structure objects have indeterminate representation even after initialization.

If an object that has automatic storage duration is not initialized explicitly, its representation is indeterminate. If an object that has static or thread storage duration is not initialized explicitly, or any object is initialized with an empty initializer, then it is subject to *default initialization*, which initializes an object as follows:

- if it has pointer type, it is initialized to a null pointer;
- if it has decimal floating type, it is initialized to positive zero, and the quantum exponent is implementation-defined;¹⁶⁴⁾
- if it has arithmetic type, and it does not have decimal floating type, it is initialized to (positive or unsigned) zero;
- if it is an aggregate, every member is initialized (recursively) according to these rules, and any padding is initialized to zero bits;

¹⁶⁴⁾A representation with all bits zero results in a decimal floating-point zero with the most negative exponent.

- if it is a union, the first named member is initialized (recursively) according to these rules, and any padding is initialized to zero bits.

The initializer for a scalar shall be a single expression, optionally enclosed in braces, or it shall be an empty initializer. If the initializer is not the empty initializer, the initial value of the object is that of the expression (after conversion); the same type constraints and conversions as for simple assignment apply, taking the type of the scalar to be the unqualified version of its declared type.

The rest of this subclause deals with initializers for objects that have aggregate or union type.

The initializer for a structure or union object shall be either an initializer list as described subsequently in this subclause, or a single expression that has compatible structure or union type. In the latter case, the initial value of the object, including unnamed members, is that of the expression.¹⁶⁵⁾

An array of character type can be initialized by a character string literal or UTF-8 string literal, optionally enclosed in braces. Successive bytes of the string literal (including the terminating null character if there is room or if the array is of unknown size) initialize the elements of the array.

An array with element type compatible with a qualified or unqualified **wchar_t**, **char16_t**, or **char32_t** can be initialized by a wide string literal with the corresponding encoding prefix (**L**, **u**, or **U**, respectively), optionally enclosed in braces. Successive wide characters of the wide string literal (including the terminating null wide character if there is room or if the array is of unknown size) initialize the elements of the array.

Otherwise, the initializer for an object that has aggregate or union type shall be a brace-enclosed list of initializers for the elements or named members.

Each brace-enclosed initializer list has an associated *current object*. When no designations are present, subobjects of the current object are initialized in order according to the type of the current object: array elements in increasing subscript order, structure members in declaration order, and the first named member of a union.¹⁶⁶⁾ In contrast, a designation causes the following initializer to begin initialization of the subobject described by the designator. Initialization then continues forward in order, beginning with the next subobject after that described by the designator.¹⁶⁷⁾

Each designator list begins its description with the current object associated with the closest surrounding brace pair. Each item in the designator list (in order) specifies a particular member of its current object and changes the current object for the next designator (if any) to be that member.¹⁶⁸⁾ The current object that results at the end of the designator list is the subobject to be initialized by the following initializer.

The initialization shall occur in initializer list order, each initializer provided for a particular subobject overriding any previously listed initializer for the same subobject;¹⁶⁹⁾ all subobjects that are not initialized explicitly are subject to default initialization.

If the aggregate or union contains elements or members that are aggregates or unions, these rules apply recursively to the subaggregates or contained unions. If the initializer of a subaggregate or contained union begins with a left brace, the initializers enclosed by that brace and its matching right brace initialize the elements or members of the subaggregate or the contained union. Otherwise, only enough initializers from the list are taken to account for the elements or members of the subaggregate or the first member of the contained union; any remaining initializers are left to initialize the next element or member of the aggregate of which the current subaggregate or contained union is a part.

If there are fewer initializers in a brace-enclosed list than there are elements or members of an

¹⁶⁵⁾If the object being initialized does not have automatic storage duration, this case violates a constraint unless the expression is a named constant or compound literal constant (6.6).

¹⁶⁶⁾If the initializer list for a subaggregate or contained union does not begin with a left brace, its subobjects are initialized as usual, but the subaggregate or contained union does not become the current object: current objects are associated only with brace-enclosed initializer lists.

¹⁶⁷⁾After a union member is initialized, the next object is not the next member of the union; instead, it is the next subobject of an object containing the union.

¹⁶⁸⁾Thus, a designator can only specify a strict subobject of the aggregate or union that is associated with the surrounding brace pair. Note, too, that each separate designator list is independent.

¹⁶⁹⁾Any initializer for the subobject which is overridden and so not used to initialize that subobject can potentially not be evaluated at all.

aggregate, or fewer characters in a string literal used to initialize an array of known size than there are elements in the array, the remainder of the aggregate is subject to default initialization.

If an array of unknown size is initialized, its size is determined by the largest indexed element with an explicit initializer. The array type is completed at the end of its initializer list.

The evaluations of the initialization list expressions are indeterminately sequenced with respect to one another and thus the order in which any side effects occur is unspecified.¹⁷⁰⁾

EXAMPLE 1 Provided that `<complex.h>` has been included, the declarations

```
int i = 3.5;
double complex c = 5 + 3 * I;
```

define and initialize **i** with the value 3 and **c** with the value $5.0 + i3.0$.

EXAMPLE 2 The declaration

```
int x[] = { 1, 3, 5 };
```

defines and initializes **x** as a one-dimensional array object that has three elements, as no size was specified and there are three initializers.

EXAMPLE 3 The declaration

```
int y[4][3] = {
    { 1, 3, 5 },
    { 2, 4, 6 },
    { 3, 5, 7 },
};
```

is a definition with a fully bracketed initialization: 1, 3, and 5 initialize the first row of **y** (the array object **y[0]**), namely **y[0][0]**, **y[0][1]**, and **y[0][2]**. Likewise the next two lines initialize **y[1]** and **y[2]**. The initializer ends early, so **y[3]** is initialized with zeros. Precisely the same effect can be achieved by

```
int y[4][3] = {
    1, 3, 5, 2, 4, 6, 3, 5, 7
};
```

The initializer for **y[0]** does not begin with a left brace, so three items from the list are used. Likewise the next three are taken successively for **y[1]** and **y[2]**.

EXAMPLE 4 The declaration

```
int z[4][3] = {
    { 1 }, { 2 }, { 3 }, { 4 }
};
```

initializes the first column of **z** as specified and initializes the rest with zeros.

EXAMPLE 5 The declaration

```
struct { int a[3], b; } w[] = { { 1 }, 2 };
```

is a definition with an inconsistently bracketed initialization. It defines an array with two element structures: **w[0].a[0]** is 1 and **w[1].a[0]** is 2; all the other elements are zero.

EXAMPLE 6 The declaration

```
short q[4][3][2] = {
    { 1 },
    { 2, 3 },
    { 4, 5, 6 }
};
```

¹⁷⁰⁾In particular, the evaluation order can be the same or different as the order of subobject initialization.

```
};
```

contains an incompletely but consistently bracketed initialization. It defines a three-dimensional array object: **q[0][0][0]** is 1, **q[1][0][0]** is 2, **q[1][0][1]** is 3, and 4, 5, and 6 initialize **q[2][0][0]**, **q[2][0][1]**, and **q[2][1][0]**, respectively; all the rest are zero. The initializer for **q[0][0]** does not begin with a left brace, so up to six items from the current list can be used. There is only one, so the values for the remaining five elements are initialized with zero. Likewise, the initializers for **q[1][0]** and **q[2][0]** do not begin with a left brace, so each uses up to six items, initializing their respective two-dimensional subaggregates. If there had been more than six items in any of the lists, a diagnostic message would have been issued. The same initialization result can be achieved by:

```
short q[4][3][2] = {
    1, 0, 0, 0, 0, 0,
    2, 3, 0, 0, 0, 0,
    4, 5, 6
};
```

or by:

```
short q[4][3][2] = {
    {
        { 1 },
    },
    {
        { 2, 3 },
    },
    {
        { 4, 5 },
        { 6 },
    },
};
```

in a fully bracketed form.

The fully bracketed and minimally bracketed forms of initialization are, in general, less likely to cause confusion.

EXAMPLE 7 One form of initialization that completes array types involves typedef names. Given the declaration

```
typedef int A[]; // OK - declared with block scope
```

the declaration

```
A a = { 1, 2 }, b = { 3, 4, 5 };
```

is identical to

```
int a[] = { 1, 2 }, b[] = { 3, 4, 5 };
```

due to the rules for incomplete types.

EXAMPLE 8 The declaration

```
char s[] = "abc", t[3] = "abc";
```

defines “plain” **char** array objects **s** and **t** whose elements are initialized with character string literals. This declaration is identical to

```
char s[] = { 'a', 'b', 'c', '\0' },
t[] = { 'a', 'b', 'c' };
```

The contents of the arrays are modifiable. On the other hand, the declaration

```
char *p = "abc";
```

defines **p** with type “pointer to **char**” and initializes it to point to an object with type “array of **char**” with length 4 whose elements are initialized with a character string literal. If an attempt is made to use **p** to modify the contents of the array, the behavior is undefined.

EXAMPLE 9 Arrays can be initialized to correspond to the elements of an enumeration by using designators:

```
enum { member_one, member_two };
const char *nm[] = {
    [member_two] = "member two",
    [member_one] = "member one",
};
```

EXAMPLE 10 Structure members can be initialized to nonzero values without depending on their order:

```
div_t answer = {.quot = 2, .rem = -1 };
```

EXAMPLE 11 Designators can be used to provide explicit initialization when unadorned initializer lists are difficult to understand:

```
struct { int a[3], b; } w[] =
    { [0].a = {1}, [1].a[0] = 2 };
```

EXAMPLE 12

```
struct T {
    int k;
    int l;
};

struct S {
    int i;
    struct T t;
};

struct T x = {.l = 43, .k = 42, };

void f(void)
{
    struct S l = { 1, .t = x, .t.l = 41, };
}
```

The value of **l.t.k** is 42, because implicit initialization does not override explicit initialization.

EXAMPLE 13 Space can be “allocated” from both ends of an array by using a single designator:

```
int a[A_MAX] = {
    1, 3, 5, 7, 9, [A_MAX-5] = 8, 6, 4, 2, 0
};
```

In the preceding snippet, if **A_MAX** is greater than ten, there will be some zero-valued elements in the middle; if it is less than ten, some of the values provided by the first five initializers will be overridden by the second five.

EXAMPLE 14 Any member of a union can be initialized:

```
union { /* ... */ } u = {.any_member = 42 };
```

Forward references: common definitions `<stddef.h>` (7.21).

6.7.12 Static assertions

Syntax

static_assert-declaration:

```
static_assert ( constant-expression , string-literal ) ;
static_assert ( constant-expression ) ;
```

Constraints

The constant expression shall compare unequal to 0.

Semantics

The constant expression shall be an integer constant expression. If the value of the constant expression compares unequal to 0, the declaration has no effect. Otherwise, the constraint is violated and the implementation shall produce a diagnostic message which should include the text of the string literal, if present.

Forward references: diagnostics (7.2).

6.7.13 Attributes

6.7.13.1 Introduction

Attributes specify additional information for various source constructs such as types, objects, identifiers, or blocks. They are identified by an *attribute token*, which can either be a *attribute prefixed token* (for implementation-specific attributes) or a *standard attribute* specified by an identifier (for attributes specified in this document).

Support for any of the standard attributes specified in this document is implementation-defined and optional. For an attribute token (including an attribute prefixed token) not specified in this document, the behavior is implementation-defined. Any attribute token that is not supported by the implementation is ignored.

Attributes are said to appertain to some source construct, identified by the syntactic context where they appear, and for each individual attribute, the corresponding clause constrains the syntactic context in which this appurtenance is valid. The attribute specifier sequence appertaining to some source construct shall contain only attributes that are allowed to apply to that source construct.

In all aspects of the language, a standard attribute specified by this document as an identifier **attr** and an identifier of the form **__attr__** shall behave the same when used as an attribute token, except for the spelling.¹⁷¹⁾

For all standard attributes specified by this document, the current value when its token sequence is given to the **__has_c_attribute** conditional inclusion expression (6.10.2) is written in the associated subclause for that attribute. A history of those values can be found in Table M.1.

Recommended practice

It is recommended that implementations support all standard attributes as defined in this document.

6.7.13.2 General

Syntax

attribute-specifier-sequence:

```
attribute-specifier-sequenceopt attribute-specifier
```

attribute-specifier:

```
[ [ attribute-list ] ]
```

attribute-list:

```
attributeopt  
attribute-list , attributeopt
```

¹⁷¹⁾ Thus, the attributes **[[nodiscard]]** and **[[__nodiscard__]]** can be freely interchanged. Implementations are encouraged to behave similarly for attribute tokens (including attribute prefixed tokens) they provide.

attribute:
*attribute-token attribute-argument-clause*_{opt}

attribute-token:
standard-attribute
attribute-prefixed-token

standard-attribute:
identifier

attribute-prefixed-token:
attribute-prefix :: identifier

attribute-prefix:
identifier

attribute-argument-clause:
*(balanced-token-sequence*_{opt} *)*

balanced-token-sequence:
balanced-token
balanced-token-sequence balanced-token

balanced-token:
*(balanced-token-sequence*_{opt} *)*
*[balanced-token-sequence*_{opt} *]*
*{ balanced-token-sequence*_{opt} *}*
any token other than a parenthesis, a bracket, or a brace

Constraints

The identifier in a standard attribute shall be one of:

deprecated	maybe_unused	noreturn	unsequenced
fallthrough	nodiscard	_Noreturn	reproducible

Semantics

An attribute specifier that contains no attributes has no effect. The order in which attribute tokens appear in an attribute list is not significant. If a keyword (6.4.2) that satisfies the syntactic requirements of an identifier (6.4.3) is contained in an attribute token, it is considered an identifier. A strictly conforming program using a standard attribute remains strictly conforming in the absence of that attribute.¹⁷²⁾

NOTE For each standard attribute, the form of the balanced token sequence, if any, will be specified.

Recommended practice

Each implementation should choose a distinctive name for the attribute prefix in an attribute prefixed token. Implementations should not define attributes without an attribute prefix unless it is a standard attribute as specified in this document.

EXAMPLE 1 Suppose that an implementation chooses the attribute prefix **hal** and provides specific attributes named **daisy** and **rosie**.

```
[[deprecated, hal::daisy]] double nine1000(double);
[[deprecated]] [[hal::daisy]] double nine1000(double);
[[deprecated]] double nine1000 [[hal::daisy]] (double);
```

Then all the following declarations should be equivalent aside from the spelling:

```
[[__deprecated__, __hal__::__daisy__]] double nine1000(double);
[[__deprecated__]] [[__hal__::__daisy__]] double nine1000(double);
[[__deprecated__]] double nine1000 [[__hal__::__daisy__]] (double);
```

¹⁷²⁾Standard attributes specified by this document can be parsed but ignored by an implementation without changing the semantics of a correct program; the same is not true for attributes not specified by this document.

These use the alternate spelling that is required for all standard attributes and recommended for prefixed attributes. These can be better-suited for use in header files, where the use of the alternate spelling avoids naming conflicts with user-provided macros.

EXAMPLE 2 For the same implementation, the following two declarations are equivalent, because the ordering inside attribute lists is not important.

```
[[hal::daisy, hal::rosie]] double nine999(double);
[[hal::rosie, hal::daisy]] double nine999(double);
```

On the other hand the following two declarations are not equivalent, because the ordering of different attribute specifiers can affect the semantics.

```
[[hal::daisy]] [[hal::rosie]] double nine999(double);
[[hal::rosie]] [[hal::daisy]] double nine999(double); // can have different semantics
```

6.7.13.3 The **nodiscard** attribute

Constraints

The **nodiscard** attribute shall be applied to a function or to the definition of a structure, union, or enumerated type. If an attribute argument clause is present, it shall have the form:

(*string-literal*)

Semantics

The **__has_c_attribute** conditional inclusion expression (6.10.2) shall return the value 202311L when given **nodiscard** as the pp-tokens operand if the implementation supports the attribute.

A name or entity declared without the **nodiscard** attribute can later be redeclared with the attribute and vice versa. An entity is considered marked after the first declaration that marks it.

Recommended practice

A **nodiscard** call is a function call expression that calls a function previously declared with attribute **nodiscard**, or whose return type is a structure, union, or enumerated type marked with attribute **nodiscard**. Evaluation of a **nodiscard** call as a void expression (6.8.4) is discouraged unless explicitly cast to **void**. Implementations are encouraged to issue a diagnostic in such cases. This is typically because immediately discarding the return value of a **nodiscard** call has surprising consequences.

The diagnostic message should include text provided by the string literal within the attribute argument clause of any **nodiscard** attribute applied to the name or entity.

EXAMPLE 1

```
struct [[nodiscard]] error_info { /*...*/ };
struct error_info enable_missile_safety_mode(void);
void launch_missiles(void);
void test_missiles(void) {
    enable_missile_safety_mode();
    launch_missiles();
}
```

A diagnostic for the call to **enable_missile_safety_mode** is encouraged.

EXAMPLE 2

```
[[nodiscard]] int important_func(void);
void call(void) {
    int i = important_func();
}
```

No diagnostic for the call to **important_func** is encouraged despite the value of **i** not being used.

EXAMPLE 3

```
[[nodiscard("armer needs to check armed state")]]
bool arm_detonator(int within);

void call(void) {
    arm_detonator(3);
    detonate();
}
```

A diagnostic for the call to **arm_detonator** using the string literal "armer needs to check armed state" from the attribute argument clause is encouraged.

6.7.13.4 The **maybe_unused** attribute

Constraints

The **maybe_unused** attribute shall be applied to the declaration of a structure, a union, a **typedef** name, an object, a structure or union member, a function, an enumeration, an enumerator, or a label. No attribute argument clause shall be present.

Semantics

The **maybe_unused** attribute indicates that a name or entity is possibly intentionally unused.

The **__has_c_attribute** conditional inclusion expression (6.10.2) shall return the value 202311L when given **maybe_unused** as the pp-tokens operand if the implementation supports the attribute.

A name or entity declared without the **maybe_unused** attribute can later be redeclared with the attribute and vice versa. An entity is considered marked with the attribute after the first declaration that marks it.

Recommended practice

For an entity marked **maybe_unused**, implementations are encouraged not to emit a diagnostic that the entity is unused, or that the entity is used despite the presence of the attribute.

EXAMPLE

```
[[maybe_unused]] void f([[maybe_unused]] int i) {
    [[maybe_unused]] int j = i + 100;
    assert(j);
}
```

Implementations are encouraged not to diagnose that **j** is unused, even if **NDEBUG** is defined.

6.7.13.5 The **deprecated** attribute

Constraints

The **deprecated** attribute shall be applied to the declaration of a structure, a union, a **typedef** name, an object, a structure or union member, a function, an enumeration, or an enumerator.

If an attribute argument clause is present, it shall have the form:

(*string-literal*)

Semantics

The **deprecated** attribute can be used to mark names and entities whose use is still allowed, but is discouraged for some reason.¹⁷³⁾

The **__has_c_attribute** conditional inclusion expression (6.10.2) shall return the value 202311L when given **deprecated** as the pp-tokens operand if the implementation supports the attribute.

A name or entity declared without the **deprecated** attribute can later be redeclared with the attribute and vice versa. An entity is considered marked with the attribute after the first declaration that marks it.

¹⁷³⁾In particular, **deprecated** is appropriate for names and entities that are obsolescent, insecure, unsafe, or otherwise unfit for purpose.

Recommended practice

Implementations should use the **deprecated** attribute to produce a diagnostic message in case the program refers to a name or entity other than to declare it, after a declaration that specifies the attribute, when the reference to the name or entity is not within the context of a related deprecated entity. The diagnostic message should include text provided by the string literal within the attribute argument clause of any **deprecated** attribute applied to the name or entity.

EXAMPLE

```
struct [[deprecated]] S {
    int a;
};

enum [[deprecated]] E1 {
    one
};

enum E2 {
    two [[deprecated("use 'three' instead")]],
    three
};

[[deprecated]] typedef int Foo;

void f1(struct S s) { // Diagnose use of S
    int i = one; // Diagnose use of E1
    int j = two; // Diagnose use of two: "use 'three' instead"
    int k = three;
    Foo f; // Diagnose use of Foo
}

[[deprecated]] void f2(struct S s) {
    int i = one;
    int j = two;
    int k = three;
    Foo f;
}

struct [[deprecated]] T {
    Foo f;
    struct S s;
};
```

Implementations are encouraged to diagnose the use of deprecated entities within a context which is not itself deprecated, as indicated for function **f1**, but not to diagnose within function **f2** and **struct T**, as they are themselves deprecated.

6.7.13.6 The **fallthrough** attribute

Constraints

The attribute token **fallthrough** shall only appear in an attribute declaration (6.7); such a declaration is a *fallthrough declaration*. No attribute argument clause shall be present. A **fallthrough** declaration can only appear within an enclosing **switch** statement (6.8.5.3). The next block item (6.8.3) that would be encountered after a **fallthrough** declaration shall be a **case** label or **default** label associated with the innermost enclosing **switch** statement and, if the **fallthrough** declaration is contained in an iteration statement, the next statement shall be part of the same execution of the secondary block of the innermost enclosing iteration statement.

Semantics

The **__has_c_attribute** conditional inclusion expression (6.10.2) shall return the value 202311L when given **fallthrough** as the pp-tokens operand if the implementation supports the attribute.

Recommended practice

The use of a fallthrough declaration is intended to suppress a diagnostic that an implementation can otherwise issue for a **case** or **default** label that is reachable from another **case** or **default** label along some path of execution. Implementations are encouraged to issue a diagnostic if a fallthrough declaration is not dynamically reachable.

EXAMPLE

```
void f(int n) {
    void g(void), h(void), i(void);
    switch (n) {
        case 1: /* diagnostic on fallthrough discouraged */
        case 2:
            g();
            [[fallthrough]];
        case 3: /* diagnostic on fallthrough discouraged */
            do {
                [[fallthrough]]; /* constraint violation: next statement is not
                                part of the same secondary block execution */
            } while(false);
        case 6:
            do {
                [[fallthrough]]; /* constraint violation: next statement is not
                                part of the same secondary block execution */
            } while (n--);
        case 7:
            while (false) {
                [[fallthrough]]; /* constraint violation: next statement is not
                                part of the same secondary block execution */
            }
        case 5:
            h();
        case 4: /* fallthrough diagnostic encouraged */
            i();
            [[fallthrough]]; /* constraint violation */
    }
}
```

6.7.13.7 The **noreturn** and **_Noreturn** attributes

Description

When **_Noreturn** is used as an attribute token (instead of a function specifier), the constraints and semantics are identical to that of the **noreturn** attribute token. Use of **_Noreturn** as an attribute token is an obsolescent feature.¹⁷⁴⁾

Constraints

The **noreturn** attribute shall be applied to a function. No attribute argument clause shall be present.

Semantics

The first declaration of a function shall specify the **noreturn** attribute if any declaration of that function specifies the **noreturn** attribute. If a function is declared with the **noreturn** attribute in one translation unit and the same function is declared without the **noreturn** attribute in another translation unit, the behavior is undefined.

If a function **f** is called where **f** was previously declared with the **noreturn** attribute and **f** eventually returns, the behavior is undefined.

The **__has_c_attribute** conditional inclusion expression (6.10.2) shall return the value 202311L when given **noreturn** as the pp-tokens operand if the implementation supports the attribute.

¹⁷⁴⁾ **[_Noreturn]** and **[[noreturn]]** are equivalent attributes to support code that includes `<stdnoreturn.h>`, because that header defines **noreturn** as a macro that expands to **_Noreturn**.

Recommended practice

The implementation should produce a diagnostic message for a function declared with a **noreturn** attribute that appears to be capable of returning to its caller.

EXAMPLE

```
[[noreturn]] void f(void) {
    abort(); // ok
}

[[noreturn]] void g(int i) { // causes undefined behavior if i <= 0
    if (i > 0) abort();
}

[[noreturn]] int h(void);
```

Implementations are encouraged to diagnose the definition of **g()** because it is capable of returning to its caller. Implementations are similarly encouraged to diagnose the declaration of **h()** because it appears capable of returning to its caller due to the non-**void** return type.

6.7.13.8 Standard attributes for function types

6.7.13.8.1 General

Constraints

The identifier in a standard function type attribute shall be one of:

unsequenced **reproducible**

An attribute for a function type shall be applied to a function declarator¹⁷⁵⁾ or to a type specifier that has a function type. The corresponding attribute is a property of the function type.¹⁷⁶⁾ No attribute argument clause shall be present.

Description

The main purpose of the function type properties and attributes defined in this clause is to provide the translator with information about the access of objects by a function such that certain properties of function calls can be deduced; the properties distinguish read operations (stateless and independent) and write operations (effectless, idempotent and reproducible) or a combination of both (unsequenced). Although semantically attached to a function type, the attributes described are not part of the prototype of such a function, and redeclarations and conversions that drop such an attribute are valid and constitute compatible types. Conversely, if a definition that does not have the asserted property is accessed by a function declaration or a function pointer with a type that has the attribute, the behavior is undefined.¹⁷⁷⁾

To allow reordering of calls to functions as they are described here, possible access to objects with a lifetime that starts before or ends after a call has to be restricted; effects on all objects that are accessed during a function call are restricted to the same thread as the call and the based-on relation between pointer parameters and lvalues (6.7.4.2) models the fact that objects do not change inadvertently during the call. In the following, an operation is said to be sequenced *during* a function call if it is sequenced after the start of the function call¹⁷⁸⁾ and before the call terminates. An object definition of an object *X* in a function *f* *escapes* if an access to *X* happens while no call to *f* is active. An object is *local* to a call to a function *f* if its lifetime starts and ends during the call or if it is defined by *f*

¹⁷⁵⁾That is, they appear in the attributes right after the closing parenthesis of the parameter list, independently of whether the function type is, for example, used directly to declare a function or whether it is used in a pointer to function type.

¹⁷⁶⁾If several declarations of the same function or function pointer are visible, regardless whether an attribute is present at several or just one of the declarators, it is attached to the type of the corresponding function definition, function pointer object, or function pointer value.

¹⁷⁷⁾That is, the fact that a function has one of these properties is in general not determined by the specification of the translation unit in which it is found; other translation units and specific run time conditions also condition the possible assertion of the properties.

¹⁷⁸⁾The initializations of the parameters is sequenced during the function call.

but does not escape. A function call and an object *X* *synchronize* if all accesses to *X* that are not sequenced during the call happen before or after the call. Execution state that is described in the library clause, such as the floating-point environment, conversion state, locale, input/output streams, external files or **errno** are considered as objects for the purposes of these attributes; operations that access this state, even indirectly, are considered as lvalue conversions for the purposes of these attributes, and operations that allow to change this state are considered as store operations, for the purposes of these attributes.

A function definition *f* is *stateless* if any definition of an object of static or thread storage duration in *f* or in a function that is called by *f* is **const** but not **volatile** qualified.

An object *X* is *observed* by a function call if both synchronize, if *X* is not local to the call, if *X* has a lifetime that starts before the function call and if an access of *X* is sequenced during the call; the last value of *X*, if any, that is stored before the call is said to be the value of *X* that is observed by the call. A function pointer value *f* is *independent* if for any object *X* that is observed by some call to *f* through an lvalue that is not based on a parameter of the call, then all accesses to *X* in all calls to *f* during the same program execution observe the same value; otherwise if the access is based on a pointer parameter, there shall be a unique such pointer parameter *P* such that any access to *X* shall be to an lvalue that is based on *P*. A function definition is independent if the derived function pointer value is independent.

A store operation to an object *X* that is sequenced during a function call such that both synchronize is said to be *observable* if *X* is not local to the call, if the lifetime of *X* ends after the call, if the stored value is different from the value observed by the call, if any, and if it is the last value written before the termination of the call. An evaluation of a function call¹⁷⁹⁾ is *effectless* if any store operation that is sequenced during the call is the modification of an object that synchronizes with the call; if additionally the operation is observable, there shall be a unique pointer parameter *P* of the function such that any access to *X* shall be to an lvalue that is based on *P*. A function pointer value *f* is effectless if any evaluation of a function call that calls *f* is effectless. A function definition is effectless if the derived function pointer value is effectless.

An evaluation *E* is *idempotent* if a second evaluation of *E* can be sequenced immediately after the original one without changing the resulting value, if any, or the observable state of the execution. A function pointer value *f* is idempotent if any evaluation of a function call¹⁸⁰⁾ that calls *f* is idempotent. A function definition is idempotent if the derived function pointer value is idempotent.

A function is *reproducible* if it is effectless and idempotent; it is *unsequenced* if it is stateless, effectless, idempotent and independent.¹⁸¹⁾

NOTE 1 The synchronization requirements with respect to any accessed object *X* for the independence of functions provide boundaries up to which a function call can safely be reordered without changing the semantics of the program. If *X* is **const** but not **volatile** qualified the reordering is unconstrained. If it is an object that is conditioned in an initialization phase, for a single threaded program a synchronization is provided by the sequenced before relation and the reordering can, in principle, move the call just after the initialization. For a multi-threaded program, synchronization guarantees can be given by calls to synchronizing functions of the `<threads.h>` header or by an appropriate call to **atomic_thread_fence** at the end of the initialization phase. If a function is known to be independent or effectless, adding **restrict** qualifications to the declarations of all pointer parameters does not change the semantics of any call. Similarly, changing the memory order to **memory_order_relaxed** for all atomic operations during a call to such a function preserves semantics.

NOTE 2 In general the functions provided by the `<math.h>` header do not have the properties that are defined previously in this subclause; many of them change the floating-point state or **errno** when they encounter an error (so they have observable side effects) and the results of most of them depend on execution-wide state such as the rounding direction mode (so they are not independent). Whether a particular C library function is reproducible or unsequenced additionally often depends on properties of the implementation, such as

¹⁷⁹⁾This considers the evaluation of the function call itself, not the evaluation of a full function call expression. Such an evaluation is sequenced after all evaluations that determine *f* and the call arguments, if any, have been performed.

¹⁸⁰⁾This considers the evaluation of the function call itself, not the evaluation of a full function call expression. Such an evaluation is sequenced after all evaluations that determine *f* and the call arguments, if any, have been performed.

¹⁸¹⁾A function call of an unsequenced function can be executed as early as the function pointer value, the values of the arguments and all objects that are accessible through them, and all values of globally accessible state have been determined, and it can be executed as late as the arguments and the objects they possibly target are unchanged and as any of its return value or modified pointed-to arguments are accessed.

implementation-defined behavior for certain error conditions.

Recommended practice

If possible, it is recommended that implementations diagnose if an attribute of this clause is applied to a function definition that does not have the corresponding property. It is recommended that applications that assert the independent or effectless properties for functions qualify pointer parameters with **restrict**.

Forward references: errors <errno.h> (7.5), floating-point environment <fenv.h> (7.6), localization <locale.h> (7.11), mathematics <math.h> (7.12), fences (7.17.4), input/output <stdio.h> (7.23), threads <threads.h> (7.28), extended multibyte and wide character utilities <wchar.h> (7.31).

6.7.13.8.2 The reproducible type attribute

Description

The **reproducible** type attribute asserts that a function or pointed-to function with that type is reproducible.

The **__has_c_attribute** conditional inclusion expression (6.10.2) shall return the value 202311L when given **reproducible** as the pp-tokens operand if the implementation supports the attribute.

EXAMPLE The attribute in the following function declaration asserts that two consecutive calls to the function will result in the same return value. Changes to the abstract state during the call are possible as long as they are not observable, but no other side effects will occur. Thus the function definition can for example use local objects of static or thread storage duration to keep track of the arguments for which the function has been called and cache their computed return values.

```
size_t hash(char const[static 32]) [[reproducible]];
```

6.7.13.8.3 The unsequenced type attribute

Description

The **unsequenced** type attribute asserts that a function or pointed-to function with that type is unsequenced.

The **__has_c_attribute** conditional inclusion expression (6.10.2) shall return the value 202311L when given **unsequenced** as the pp-tokens operand if the implementation supports the attribute.

NOTE 1 The unsequenced type attribute asserts strong properties for such a function, in particular that certain sequencing requirements for function calls can be relaxed without affecting the state of the abstract machine. Thereby, calls to such functions are natural candidates for optimization techniques such as common subexpression elimination, local memoization or lazy evaluation.

NOTE 2 A proof of validity of the annotation of a function type with the **unsequenced** attribute can depend on the property of whether a derived function pointer escapes the translation unit or not. For a function with internal linkage where no function pointer escapes the translation unit, all calling contexts are known and it is possible, in principle, to prove that no control flow exists such that a library function is called with arguments that trigger an exceptional condition. For a function with external linkage such a proof is potentially not possible and such a function therefore only is used when it can be ensured that no exceptional condition results from the provided arguments.

NOTE 3 The unsequenced property does not necessarily imply that the function is reentrant or that calls can be executed concurrently. This is because an unsequenced function can read from and write to objects of static storage duration, as long as no change is observable after a call terminates.

EXAMPLE 1 The attribute in the following function declaration asserts that it doesn't depend on any modifiable state of the abstract machine. Calls to the function can be executed out of sequence before the return value is needed and two calls to the function with the same argument value will result in the same return value.

```
bool tendency(signed char) [[unsequenced]];
```

Therefore such a call for a given argument value needs only to be executed once and the returned value can be reused when appropriate. For example, calls for all possible argument values can be executed during program startup and tabulated.

EXAMPLE 2 The attribute in the following function declaration asserts that it doesn't depend on any modifiable state of the abstract machine. Within the same thread, calls to the function can be executed out of sequence before the return value is needed and two calls to the function will result in the same pointer return value. Therefore such a call needs only to be executed once in a given thread and the returned pointer value can be reused when appropriate. For example, a single call can be executed during thread startup and the return value **p** and the value of the object ***p** of type **toto** **const** can be cached.

```
typedef struct toto toto;
toto const* toto_zero(void) [[unsequenced]];
```

EXAMPLE 3 The unsequenced property of a function *f* can be locally asserted within a function *g* that uses it. For example the library function **sqrt** is in general not unsequenced because a negative argument will raise a domain error and because the result can depend on the rounding mode. Nevertheless in contexts similar to the following function a user can prove that it will not be called with invalid arguments, and, that the floating-point environment has the same value for all calls.

```
#include <math.h>
#include <fenv.h>

inline double distance (double const x[static 2]) [[reproducible]] {
    #pragma STDC FP_CONTRACT OFF
    #pragma STDC FENV_ROUND FE_TONEAREST
    // We assert that sqrt will not be called with invalid arguments
    // and the result only depends on the argument value.
    extern typeof(sqrt) [[unsequenced]] sqrt;
    return sqrt(x[0]*x[0] + x[1]*x[1]);
}
```

The function **distance** potentially has the side effect of changing the floating-point environment. Nevertheless the floating environment is thread local, thus a change to that state outside the function is sequenced with the change within and additionally the observed value is restored when the function returns. Thus this side effect is not observable for a caller. Overall the function **distance** is stateless, effectless and idempotent and in particular it is reproducible as the attribute indicates. Because the function can be called in a context where the floating-point environment has different state, **distance** is not independent and thus it is also not unsequenced. Nevertheless, adding an unsequenced attribute where this is justified can introduce optimization opportunities.

```
double g (double y[static 1], double const x[static 2]) {
    // We assert that distance will not see different states of the floating
    // point environment.
    extern double distance (double const x[static 2]) [[unsequenced]];
    y[0] = distance(x);
    ...
    return distance(x);    // replacement by y[0] is valid
}
```

6.8 Statements and blocks

6.8.1 General

Syntax

statement:

labeled-statement
unlabeled-statement

unlabeled-statement:

expression-statement
*attribute-specifier-sequence*_{opt} *primary-block*
*attribute-specifier-sequence*_{opt} *jump-statement*

primary-block:

compound-statement
selection-statement
iteration-statement

secondary-block:

statement

Semantics

A *statement* specifies an action to be performed. Except as indicated, statements are executed in sequence. The optional attribute specifier sequence appertains to the respective statement.

A *block* is either a primary block, a secondary block, or the block associated with a function definition; it allows a set of declarations and statements to be grouped into one syntactic unit. Whenever a block *B* appears in the syntax production as part of the definition of an enclosing block *A*, scopes of identifiers and lifetimes of objects that are associated with *B* do not extend to the parts of *A* that are outside of *B*. The initializers of objects that have automatic storage duration, and any size expressions and typeof operators in declarations of ordinary identifiers with block scope, are evaluated and the values are stored in the objects (the representation of objects without an initializer becomes indeterminate) each time the declaration is reached in the order of execution, as if it were a statement, and within each declaration in the order that declarators appear.

A *full expression* is an expression that is not part of another expression, nor part of a declarator or abstract declarator. There is also an implicit full expression in which the non-constant size expressions for a variably modified type are evaluated; within that full expression, the evaluation of different size expressions are unsequenced with respect to one another. There is a sequence point between the evaluation of a full expression and the evaluation of the next full expression to be evaluated.

NOTE Each of the following is a full expression:

- a full declarator for a variably modified type,
- an initializer that is not part of a compound literal,
- the expression in an expression statement,
- the controlling expression of a selection statement (**if** or **switch**),
- the controlling expression of a **while** or **do** statement,
- each of the (optional) expressions of a **for** statement,
- the (optional) expression in a **return** statement.

While a constant expression satisfies the definition of a full expression, evaluating it does not depend on nor produce any side effects, so the sequencing implications of being a full expression are not relevant to a constant expression.

Forward references: expression and null statements (6.8.4), selection statements (6.8.5), iteration statements (6.8.6), the **return** statement (6.8.7.5).

6.8.2 Labeled statements

Syntax

label:

*attribute-specifier-sequence*_{opt} *identifier* :
*attribute-specifier-sequence*_{opt} **case** *constant-expression* :
*attribute-specifier-sequence*_{opt} **default** :

labeled-statement:

label statement

Constraints

A **case** or **default** label shall appear only in a **switch** statement. Further constraints on such labels are discussed under the **switch** statement.

Label names shall be unique within a function.

Semantics

Any statement or declaration in a compound statement can be preceded by a prefix that declares an identifier as a label name. The optional attribute specifier sequence appertains to the label. Labels in themselves do not alter the flow of control, which continues unimpeded across them.

Forward references: the **goto** statement (6.8.7.2), the **switch** statement (6.8.5.3) .

6.8.3 Compound statement

Syntax

compound-statement:

{ *block-item-list*_{opt} }

block-item-list:

block-item
block-item-list block-item

block-item:

declaration
unlabeled-statement
label

Semantics

A *compound statement* that is a function body together with the parameter type list and the optional attribute specifier sequence between them forms the block associated with the function definition in which it appears. Otherwise, it is a block that is different from any other block. A label shall be translated as if it were followed by a null statement.

6.8.4 Expression and null statements

Syntax

expression-statement:

*expression*_{opt} ;
attribute-specifier-sequence expression ;

Semantics

The attribute specifier sequence appertains to the expression. The expression in an expression statement is evaluated as a void expression for its side effects.¹⁸²⁾

A *null statement* (consisting of just a semicolon) performs no operations.

¹⁸²⁾Such as assignments, and function calls which have side effects.

EXAMPLE 1 If a function call is evaluated as an expression statement for its side effects only, the discarding of its value can be made explicit by converting the expression to a void expression by means of a cast:

```
int p(int);
/* ... */
(void)p(0);
```

EXAMPLE 2 In the program fragment

```
char *s;
/* ... */
while (*s++ != '\0')
    ;
```

a null statement is used to supply an empty loop body to the iteration statement.

Forward references: iteration statements (6.8.6).

6.8.5 Selection statements

6.8.5.1 General

Syntax

selection-statement:

```
if ( expression ) secondary-block
if ( expression ) secondary-block else secondary-block
switch ( expression ) secondary-block
```

Semantics

A selection statement selects among a set of secondary blocks depending on the value of a controlling expression.

6.8.5.2 The if statement

Constraints

The controlling expression of an **if** statement shall have scalar type.

Semantics

In both forms, the first substatement is executed if the expression compares unequal to 0. In the **else** form, the second substatement is executed if the expression compares equal to 0. If the first substatement is reached via a label, the second substatement is not executed.

An **else** is associated with the lexically nearest preceding **if** that is allowed by the syntax.

6.8.5.3 The switch statement

Constraints

The controlling expression of a **switch** statement shall have integer type.

If a **switch** statement has an associated **case** or **default** label within the scope of an identifier with a variably modified type, the entire **switch** statement shall be within the scope of that identifier.¹⁸³⁾

The expression of each **case** label shall be an integer constant expression and no two of the **case** constant expressions associated to the same **switch** statement shall have the same value after conversion. There can be at most one **default** label associated to a **switch** statement. (Any enclosed **switch** statement can have a **default** label or **case** constant expressions with values that duplicate **case** constant expressions in the enclosing **switch** statement.)

¹⁸³⁾That is, the declaration either precedes the **switch** statement, or it follows the last **case** or **default** label associated with the **switch** that is in the block containing the declaration.

Semantics

A **switch** statement causes control to jump to, into, or past the statement that is the *switch body*, depending on the value of a controlling expression, and on the presence of a **default** label and the values of any **case** labels on or in the switch body. A **case** or **default** label is accessible only within the closest enclosing **switch** statement.

The integer promotions are performed on the controlling expression. The constant expression in each **case** label is converted to the promoted type of the controlling expression. If a converted value matches that of the promoted controlling expression, control jumps to the statement or declaration following the matched **case** label. Otherwise, if there is a **default** label, control jumps to the statement or declaration following the **default** label. If no converted **case** constant expression matches and there is no **default** label, no part of the switch body is executed.

Implementation limits

As discussed in 5.3.5.2, the implementation may limit the number of **case** values in a **switch** statement.

EXAMPLE In the artificial program fragment

```
switch (expr)
{
    int i = 4;
    f(i);
case 0:
    i = 17;
    /* falls through into default code */
default:
    printf("%d\n", i);
}
```

the object whose identifier is **i** exists with automatic storage duration (within the block) but is never initialized, and thus if the controlling expression has a nonzero value, the call to the **printf** function will access an object with an indeterminate representation. Similarly, the call to the function **f** cannot be reached.

6.8.6 Iteration statements

6.8.6.1 General

Syntax

iteration-statement:

```
while ( expression ) secondary-block
do secondary-block while ( expression ) ;
for ( expressionopt ; expressionopt ; expressionopt ) secondary-block
for ( declaration expressionopt ; expressionopt ) secondary-block
```

Constraints

The controlling expression of an iteration statement shall have scalar type.

Semantics

An iteration statement causes a secondary block called the *loop body* to be executed repeatedly until the controlling expression compares equal to 0. The repetition occurs regardless of whether the loop body is entered from the iteration statement or by a jump.¹⁸⁴⁾

An iteration statement may be assumed by the implementation to terminate if its controlling expression is not a constant expression,¹⁸⁵⁾ and none of the following operations are performed in its

¹⁸⁴⁾Code jumped over is not executed. In particular, the controlling expression of a **for** or **while** statement is not evaluated before entering the loop body, nor is *clause-1* (6.8.6.4) of a **for** statement.

¹⁸⁵⁾An omitted controlling expression is replaced by a nonzero constant, which is a constant expression.

body, controlling expression or (in the case of a **for** statement) its *expression-3*.¹⁸⁶⁾

- input/output operations
- accessing a volatile object
- synchronization or atomic operations.

6.8.6.2 The **while** statement

The evaluation of the controlling expression takes place before each execution of the loop body.

6.8.6.3 The **do** statement

The evaluation of the controlling expression takes place after each execution of the loop body.

6.8.6.4 The **for** statement

The statement

```
for (clause-1; expression-2; expression-3) statement
```

behaves as follows: The expression *expression-2* is the controlling expression that is evaluated before each execution of the loop body. The expression *expression-3* is evaluated as a void expression after each execution of the loop body. If *clause-1* is a declaration, the scope of any identifiers it declares is the remainder of the declaration and the entire loop, including the other two expressions; it is reached in the order of execution before the first evaluation of the controlling expression. If *clause-1* is an expression, it is evaluated as a void expression before the first evaluation of the controlling expression.¹⁸⁷⁾

Both *clause-1* and *expression-3* can be omitted. An omitted *expression-2* is replaced by a nonzero constant.

6.8.7 Jump statements

6.8.7.1 General

Syntax

```
jump-statement:  
    goto identifier ;  
    continue ;  
    break ;  
    return expressionopt ;
```

Semantics

A jump statement causes an unconditional jump to another place.

6.8.7.2 The **goto** statement

Constraints

The identifier in a **goto** statement shall name a label located somewhere in the enclosing function. A **goto** statement shall not jump from outside the scope of an identifier having a variably modified type to inside the scope of that identifier.

¹⁸⁶⁾This is intended to allow compiler transformations such as removal of empty loops even when termination cannot be proven.

¹⁸⁷⁾Thus, *clause-1* specifies initialization for the loop, possibly declaring one or more variables for use in the loop; the controlling expression, *expression-2*, specifies an evaluation made before each iteration, such that execution of the loop continues until the expression compares equal to 0; and *expression-3* specifies an operation (such as incrementing) that is performed after each iteration.

Semantics

A **goto** statement causes an unconditional jump to the statement prefixed by the named label in the enclosing function.

EXAMPLE 1 It is sometimes convenient to jump into the middle of a complicated set of statements. The following outline presents one possible approach to a problem based on these three assumptions:

1. The general initialization code accesses objects only visible to the current function.
2. The general initialization code is too large to warrant duplication.
3. The code to determine the next operation is at the head of the loop. (To allow it to be reached by **continue** statements, for example.)

```

/* ... */
goto first_time;
for (;;) {
    // determine next operation
    /* ... */
    if (need to reinitialize) {
        // reinitialize-only code
        /* ... */
        first_time:
            // general initialization code
            /* ... */
            continue;
    }
    // handle other operations
    /* ... */
}

```

EXAMPLE 2 A **goto** statement which jumps past any declarations of objects with variably modified types is not conforming. A jump within the scope, however, is valid.

```

goto lab3;           // invalid: going INTO scope of VLA.
{
    double a[n];
    a[j] = 4.4;
lab3:
    a[j] = 3.3;
    goto lab4;       // valid: going WITHIN scope of VLA.
    a[j] = 5.5;
lab4:
    a[j] = 6.6;
}
goto lab4;           // invalid: going INTO scope of VLA.

```

6.8.7.3 The **continue** statement

Constraints

A **continue** statement shall appear only in or as a loop body.

Semantics

A **continue** statement causes a jump to the loop-continuation portion of the innermost enclosing iteration statement; that is, to the end of the loop body. More precisely, in each of the statements

```

while (/* ... */) {
    /* ... */
    continue;
    /* ... */
contin:
}

```

```

do {
    /* ... */
    continue;
    /* ... */
contin:
} while (/* ... */);

```

```

for (/* ... */) {
    /* ... */
    continue;
    /* ... */
contin:
}

```

unless the **continue** statement shown is in an enclosed iteration statement (in which case it is interpreted within that statement), it is equivalent to **goto contin;**¹⁸⁸⁾

6.8.7.4 The **break** statement

Constraints

A **break** statement shall appear only in or as a switch body or loop body.

Semantics

A **break** statement terminates execution of the innermost enclosing **switch** or iteration statement.

6.8.7.5 The **return** statement

Constraints

A **return** statement with an expression shall not appear in a function whose return type is **void**. A **return** statement without an expression shall only appear in a function whose return type is **void**.

Semantics

A **return** statement terminates execution of the current function and returns control to its caller. A function can have any number of **return** statements.

If a **return** statement with an expression is executed, the value of the expression is returned to the caller as the value of the function call expression. If the expression has a type different from the return type of the function in which it appears, the value is converted as if by assignment to an object having the return type of the function.¹⁸⁹⁾

EXAMPLE In:

```
struct s { double i; } f(void);
union {
    struct {
        int f1;
        struct s f2;
    } u1;
    struct {
        struct s f3;
        int f4;
    } u2;
} g;

struct s f(void)
{
    return g.u1.f2;
}

/* ... */
g.u2.f3 = f();
```

there is no undefined behavior, although there would be if the assignment were done directly (without using a function call to fetch the value).

¹⁸⁸⁾Following the **contin:** label in the 2nd example is a null statement. The null statement in the first and third example is implied by the label (6.8.3).
¹⁸⁹⁾The **return** statement is not an assignment. The overlap restriction of 6.5.17.2 does not apply to the case of function return. The representation of floating-point values can have wider range or precision than implied by the type; a cast can be used to remove this extra range and precision.

6.9 External definitions

6.9.1 General

Syntax

translation-unit:

external-declaration
translation-unit external-declaration

external-declaration:

function-definition
declaration

Constraints

The storage-class specifier **register** shall not appear in the declaration specifiers in an external declaration. The storage-class specifier **auto** shall only appear in the declaration specifiers in an external declaration if the type is inferred.

There shall be no more than one external definition for each identifier declared with internal linkage in a translation unit. Moreover, if an identifier declared with internal linkage is used in an expression there shall be exactly one external definition for the identifier in the translation unit, unless it is:

- part of the operand of a **sizeof** operator whose result is an integer constant;
- part of the operand of an **alignof** operator whose result is an integer constant;
- part of the controlling expression of a generic selection;
- part of the expression in a generic association that is not the result expression of its generic selection;
- or, part of the operand of any typeof operator whose result is not a variably modified type.

Semantics

As discussed in 5.2.1.1, the unit of program text after preprocessing is a translation unit, which consists of a sequence of external declarations. These are described as “external” because they appear outside any function (and hence have file scope). As discussed in 6.7, a declaration that also causes storage to be reserved for an object or a function named by the identifier is a definition.

An *external definition* is an external declaration that is also a definition of a function (other than an inline definition) or an object. If an identifier declared with external linkage is used in an expression (other than as part of the operand of a typeof operator whose result is not a variably modified type, part of the controlling expression of a generic selection, part of the expression in a generic association that is not the result expression of its generic selection, or part of a **sizeof** or **alignof** operator whose result is an integer constant expression), somewhere in the entire program there shall be exactly one external definition for the identifier; otherwise, there shall be no more than one.¹⁹⁰⁾

6.9.2 Function definitions

Syntax

function-definition:

attribute-specifier-sequence_{opt} declaration-specifiers declarator function-body

function-body:

compound-statement

¹⁹⁰⁾ Thus, if an identifier declared with external linkage is not used in an expression, there need be no external definition for it.

Constraints

The identifier declared in a function definition (which is the name of the function) shall have a function type, as specified by the declarator portion of the function definition.

The return type of a function shall be **void** or a complete object type other than array type.

The storage-class specifier, if any, in the declaration specifiers shall be either **extern** or **static**.

If the parameter list consists of a single parameter of type **void**, the parameter declarator shall not include an identifier.

Variable length array types of unspecified size shall not be used as part of a parameter declaration in a function definition.

Semantics

The optional attribute specifier sequence in a function definition appertains to the function.

The declarator in a function definition specifies the name of the function being defined and the types (and optionally the names) of all the parameters; the declarator also serves as a function prototype for later calls to the same function in the same translation unit. The type of each parameter is adjusted as described in 6.7.7.4.

If a function that accepts a variable number of arguments is defined without a parameter type list that ends with the ellipsis notation, the behavior is undefined.

The parameter type list, the attribute specifier sequence of the declarator that follows the parameter type list, and the compound statement of the function body form a single block.¹⁹¹⁾ Each parameter has automatic storage duration; its identifier, if any,¹⁹²⁾ is an lvalue.¹⁹³⁾ The layout of the storage for parameters is unspecified.

On entry to the function, the size expressions of each variably modified parameter and type of operators used in declarations of parameters are evaluated and the value of each argument expression is converted to the type of the corresponding parameter as if by assignment. (Array expressions and function designators as arguments were converted to pointers before the call.)

After all parameters have been assigned, the compound statement of the function body is executed.

Unless otherwise specified, if the **}** that terminates the function body is reached, and the value of the function call is used by the caller, the behavior is undefined.

NOTE In a function definition, the return type of the function and its prototype cannot be inherited from a typedef:

```
typedef int F(void);           // type F is "function with no parameters
                               // returning int"
F f, g;                       // f and g both have type compatible with F
F f { /* ... */ }             // WRONG: syntax/constraint error
F g() { /* ... */ }           // WRONG: declares that g returns a function
int f(void) { /* ... */ }     // RIGHT: f has type compatible with F
int g() { /* ... */ }         // RIGHT: g has type compatible with F
F *e(void) { /* ... */ }      // e returns a pointer to a function
F *((e))(void) { /* ... */ }  // same: parentheses irrelevant
int (*fp)(void);              // fp points to a function that has type F
F *Fp;                         // Fp points to a function that has type F
```

EXAMPLE 1 In the following:

```
extern int max(int a, int b)
{
```

¹⁹¹⁾The visibility scope of a parameter in a function definition starts when its declaration is completed, extends to following parameter declarations, to possible attributes that follow the parameter type list, and then to the entire function body. The lifetime of each instance of a parameter starts when the declaration is evaluated starting a call and ends when that call terminates.

¹⁹²⁾A parameter that has no declared name is inaccessible within the function body.

¹⁹³⁾A parameter identifier cannot be redeclared in the function body except in an enclosed block.

```

    return a > b ? a: b;
}

```

extern is the storage-class specifier and **int** is the type specifier; **max(int a, int b)** is the function declarator; and

```

{ return a > b ? a: b; }

```

is the function body.

EXAMPLE 2 To pass one function to another, one can say

```

int f(void);
/* ... */
g(f);

```

Then the definition of **g** can read

```

void g(int (*funcp)(void))
{
    /* ... */
    (*funcp)(); /* or funcp(); ...*/
}

```

or, equivalently,

```

void g(int func(void))
{
    /* ... */
    func(); /* or (*func)(); ...*/
}

```

6.9.3 External object definitions

Semantics

If the declaration of an identifier for an object has file scope and an initializer, or has file scope and storage-class specifier **thread_local**, the declaration is an external definition for the identifier.

A declaration of an identifier for an object that has file scope without an initializer, and without the storage-class specifier **extern** or **thread_local**, constitutes a *tentative definition*. If a translation unit contains one or more tentative definitions for an identifier, and the translation unit contains no external definition for that identifier, then the behavior is exactly as if the translation unit contains a file scope declaration of that identifier with an empty initializer and a type determined as follows:

- if the composite type as of the end of the translation unit is an array of unknown size, then an array of size one with the composite element type;
- otherwise, the composite type at the end of the translation unit.

If the declaration of an identifier for an object is a tentative definition and has internal linkage, the declared type shall not be an incomplete type.

EXAMPLE 1

```
int i1 = 1;           // definition, external linkage
static int i2 = 2;    // definition, internal linkage
extern int i3 = 3;     // definition, external linkage
int i4;               // tentative definition, external linkage
static int i5;         // tentative definition, internal linkage

int i1;               // valid tentative definition, refers to previous
int i2;               // 6.2.2 renders undefined, linkage disagreement
int i3;               // valid tentative definition, refers to previous
int i4;               // valid tentative definition, refers to previous
int i5;               // 6.2.2 renders undefined, linkage disagreement

extern int i1;         // refers to previous, whose linkage is external
extern int i2;         // refers to previous, whose linkage is internal
extern int i3;         // refers to previous, whose linkage is external
extern int i4;         // refers to previous, whose linkage is external
extern int i5;         // refers to previous, whose linkage is internal
```

EXAMPLE 2 If at the end of the translation unit containing

```
int i[];
```

the array `i` still has incomplete type, the implicit initializer causes it to have one element, which is set to zero on program startup.

6.10 Preprocessing directives

6.10.1 General

Syntax

preprocessing-file:

*group*_{opt}

group:

group-part

group group-part

group-part:

if-section

control-line

text-line

non-directive

if-section:

*if-group elif-groups*_{opt} *else-group*_{opt} *endif-line*

if-group:

if *constant-expression new-line group*_{opt}

ifdef *identifier new-line group*_{opt}

ifndef *identifier new-line group*_{opt}

elif-groups:

elif-group

elif-groups elif-group

elif-group:

elif *constant-expression new-line group*_{opt}

elifdef *identifier new-line group*_{opt}

elifndef *identifier new-line group*_{opt}

else-group:

else *new-line group*_{opt}

endif-line:

endif *new-line*

control-line:

include *pp-tokens new-line*

embed *pp-tokens new-line*

define *identifier replacement-list new-line*

define *identifier lparen identifier-list*_{opt} **)** *replacement-list new-line*

define *identifier lparen ...) replacement-list new-line*

define *identifier lparen identifier-list , ...) replacement-list new-line*

undef *identifier new-line*

line *pp-tokens new-line*

error *pp-tokens*_{opt} *new-line*

warning *pp-tokens*_{opt} *new-line*

pragma *pp-tokens*_{opt} *new-line*

new-line

text-line:

*pp-tokens*_{opt} *new-line*

non-directive:

pp-tokens new-line

lparen:

a (character not immediately preceded by white space

replacement-list:

	<i>pp-tokens</i> _{opt}
<i>pp-tokens:</i>	<i>preprocessing-token</i> <i>pp-tokens preprocessing-token</i>
<i>new-line:</i>	the new-line character
<i>identifier-list:</i>	<i>identifier</i> <i>identifier-list</i> , <i>identifier</i>
<i>pp-parameter:</i>	<i>pp-parameter-name pp-parameter-clause</i> _{opt}
<i>pp-parameter-name:</i>	<i>pp-standard-parameter</i> <i>pp-prefixed-parameter</i>
<i>pp-standard-parameter:</i>	<i>identifier</i>
<i>pp-prefixed-parameter:</i>	<i>identifier :: identifier</i>
<i>pp-parameter-clause:</i>	(<i>pp-balanced-token-sequence</i> _{opt})
<i>pp-balanced-token-sequence:</i>	<i>pp-balanced-token</i> <i>pp-balanced-token-sequence pp-balanced-token</i>
<i>pp-balanced-token:</i>	(<i>pp-balanced-token-sequence</i> _{opt}) [<i>pp-balanced-token-sequence</i> _{opt}] { <i>pp-balanced-token-sequence</i> _{opt} } any pp-token other than a parenthesis, a bracket, or a brace
<i>embed-parameter-sequence:</i>	<i>pp-parameter</i> <i>embed-parameter-sequence pp-parameter</i>

Description
A *preprocessing directive* consists of a sequence of preprocessing tokens that satisfies the following constraints: The first token in the sequence is a # preprocessing token that (at the start of translation phase 4) is either the first character in the source file (optionally after white space containing no new-line characters) or that follows white space containing at least one new-line character. The last

token in the sequence is the first new-line character that follows the first token in the sequence.¹⁹⁴⁾ A new-line character ends the preprocessing directive even if it occurs within what would otherwise be an invocation of a function-like macro.

A text line shall not begin with a **#** preprocessing token. A non-directive shall not begin with any of the directive names appearing in the syntax.

Some preprocessing directives take additional information using preprocessor parameters. A *preprocessing parameter* (pp-parameter) shall be either a *preprocessor prefixed parameter* (identified by a pp-prefixed-parameter, for implementation-defined preprocessor parameters) or a *preprocessor standard parameter* (identified with a pp-standard-parameter, for pp-parameters specified by this document).

In all aspects, a preprocessor standard parameter specified by this document as an identifier **pp_param** and an identifier of the form **__pp_param__** shall behave the same when used as a preprocessor parameter, except for the spelling.

EXAMPLE 1 Thus, the preprocessor parameters on the two binary resource inclusion directives (6.10.4):

```
#embed "boop.h" limit(5)
#embed "boop.h" __limit__(5)
```

behave the same, and can be freely interchanged. Implementations are encouraged to behave similarly for preprocessor parameters (including preprocessor prefixed parameters) they provide.

When in a group that is skipped (6.10.2), the directive syntax is relaxed to allow any sequence of preprocessing tokens to occur between the directive name and the following new-line character.

Constraints

The only white-space characters that shall appear between preprocessing tokens within a preprocessing directive (from just after the introducing **#** preprocessing token through just before the terminating new-line character) are space and horizontal-tab (including spaces that have replaced comments or possibly other white-space characters in translation phase 3).

A preprocessor parameter shall be either a preprocessor standard parameter, or an implementation-defined preprocessor prefixed parameter.¹⁹⁵⁾

Semantics

The implementation can process and skip sections of source files conditionally, include other source files, and replace macros. These capabilities are called *preprocessing*, because conceptually they occur before translation of the resulting translation unit.

The preprocessing tokens within a preprocessing directive are not subject to macro expansion unless otherwise stated.

EXAMPLE 2 In:

```
#define EMPTY
EMPTY # include <file.h>
```

the sequence of preprocessing tokens on the second line is *not* a preprocessing directive, because it does not begin with a **#** at the start of translation phase 4 (5.2.1.2), even though it will do so after the macro **EMPTY** has been replaced.

The execution of a non-directive preprocessing directive results in undefined behavior.

6.10.2 Conditional inclusion

Syntax

defined-macro-expression:

¹⁹⁴⁾ Thus, preprocessing directives are commonly called "lines". These "lines" have no other syntactic significance, as all white space is equivalent except in certain situations during preprocessing (see the **#** character string literal creation operator in 6.10.5.3, for example).

¹⁹⁵⁾ An unrecognized preprocessor prefixed parameter is a constraint violation, except within `has_embed` expressions (6.10.2).

```

defined identifier
defined ( identifier )
h-preprocessing-token:
    any preprocessing-token other than >
h-pp-tokens:
    h-preprocessing-token
    h-pp-tokens h-preprocessing-token
header-name-tokens:
    string-literal
    < h-pp-tokens >
has-include-expression:
    __has_include ( header-name )
    __has_include ( header-name-tokens )
has-embed-expression:
    __has_embed ( header-name embed-parameter-sequenceopt )
    __has_embed ( header-name-tokens pp-balanced-token-sequenceopt )
has-c-attribute-express:
    __has_c_attribute ( pp-tokens )

```

The **#if** and **#elif** directives are collectively known as the *conditional expression inclusion preprocessing directives*. The conditional expression inclusion preprocessing directives, **#ifdef**, **#ifndef**, **#elifdef**, and **#elifndef** directives are collectively known as the *conditional inclusion preprocessing directives*.

Constraints

The expression that controls conditional inclusion shall be an integer constant expression except that: identifiers (including those lexically identical to keywords) are interpreted as described subsequently in this subclause¹⁹⁶⁾ and it can contain zero or more defined macro expressions, **has_include** expressions, **has_embed** expressions, and/or **has_c_attribute** expressions as unary operator expressions.

A defined macro expression evaluates to 1 if the identifier is currently defined as a macro name (that is, if it is predefined or if it has been the subject of a **#define** preprocessing directive without an intervening **#undef** directive with the same subject identifier), 0 if it is not.

The second form of the **has_include** expression and **has_embed** expression is considered only if the first form does not match, in which case the preprocessing tokens are processed just as in normal text.

The header or source file identified by the parenthesized preprocessing token sequence in each contained **has_include** expression is searched for as if that preprocessing token were the **pp-tokens** in a **#include** directive, except that no further macro expansion is performed. Such a directive shall satisfy the syntactic requirements of a **#include** directive. The **has_include** expression evaluates to 1 if the search for the source file succeeds, and to 0 if the search fails.

The resource (6.10.4) identified by the header-name preprocessing token sequence in each contained **has_embed** expression is searched for as if those preprocessing token were the **pp-tokens** in a **#embed** directive, except that no further macro expansion is performed. Such a directive shall satisfy the syntactic requirements of a **#embed** directive. The **has_embed** expression evaluates to the same value as the following predefined macros (6.10.10.2):

- **__STDC_EMBED_NOT_FOUND__**, if the search fails or if any of the embed parameters in the embed parameter sequence specified are not supported by the implementation for the **#embed** directive; or,
- **__STDC_EMBED_FOUND__**, if the search for the resource succeeds and all embed parameters in

¹⁹⁶⁾ Because the controlling constant expression is evaluated during translation phase 4, all identifiers either are or are not macro names — there simply are no keywords, enumeration constants, etc.

the embed parameter sequence specified are supported by the implementation for the **#embed** directive and the resource is not empty; or,

- **__STDC_EMBED_EMPTY__**, if the search for the resource succeeds and all embed parameters in the embed parameter sequence specified are supported by the implementation for the **#embed** directive and the resource is empty.

NOTE 1 Unrecognized preprocessor prefixed parameters in has_embed expressions are not a constraint violation and instead cause the expression to be evaluated to 0, as specified previously.

Each has_c_attribute expression is replaced by a nonzero pp-number matching the form of an integer constant if the implementation supports an attribute with the name specified by interpreting the pp-tokens as an attribute token, and by 0 otherwise. The pp-tokens shall match the form of an attribute token.

Each preprocessing token that remains (in the list of preprocessing tokens that will become the controlling expression) after all macro replacements have occurred shall be in the lexical form of a token (6.4).

Semantics

The **#ifdef**, **#ifndef**, **#elifdef**, and **#elifndef** directives, and the **defined** conditional inclusion operator, shall treat **__has_include**, **__has_embed** and **__has_c_attribute** as if they were the name of defined macros. The identifiers **__has_include**, **__has_embed**, and **__has_c_attribute** shall not appear in any context not mentioned in this subclause.

Preprocessing directives of the forms

```
# if  constant-expression new-line groupopt
# elif constant-expression new-line groupopt
```

check whether the controlling constant expression evaluates to nonzero.

Prior to evaluation, macro invocations in the list of preprocessing tokens that will become the controlling constant expression are replaced (except for those macro names modified by the **defined** unary operator), just as in normal text. If the token **defined** is generated as a result of this replacement process or use of the **defined** unary operator does not match one of the two specified forms prior to macro replacement, the behavior is undefined. After all replacements due to macro expansion and evaluations of defined macro expressions, has_include expressions, has_embed expressions, and has_c_attribute expressions have been performed, all remaining identifiers other than **true** (including those lexically identical to keywords such as **false**) are replaced with the pp-number 0, **true** is replaced with pp-number 1, and then each preprocessing token is converted into a token. The resulting tokens compose the controlling constant expression which is evaluated according to the rules of 6.6. For the purposes of this token conversion and evaluation, all signed integer types and all unsigned integer types act as if they have the same representation as, respectively, the types **intmax_t** and **uintmax_t** defined in the header `<stdint.h>`. This includes interpreting character constants, which can involve converting escape sequences into execution character set members. Whether the numeric value for these character constants matches the value obtained when an identical character constant occurs in an expression (other than within a **#if** or **#elif** directive) is implementation-defined. Whether a single-character character constant may have a negative value is implementation-defined.

NOTE 2 On an implementation where **INT_MAX** is **0x7FFF** and **UINT_MAX** is **0xFFFF**, the constant **0x8000** is signed and positive within a **#if** expression even though it would be unsigned in translation phase 7 (5.2.1.2).

NOTE 3 The constant expression in the following **#if** directive and **if** statement is not guaranteed to evaluate to the same value in these two contexts.

```
#if 'z' - 'a' == 25
if ('z' - 'a' == 25)
```

Preprocessing directives of the forms

```
# ifdef identifier new-line groupopt
```

```
# ifndef identifier new-line groupopt
# elifdef identifier new-line groupopt
# elifndef identifier new-line groupopt
```

check whether the identifier is or is not currently defined as a macro name. Their conditions are equivalent to **#if defined identifier**, **#if !defined identifier**, **#elif defined identifier**, and **#elif !defined identifier** respectively.

Each directive's condition is checked in order. If it evaluates to false (zero), the group that it controls is skipped: directives are processed only through the name that determines the directive to keep track of the level of nested conditionals; the rest of the directives' preprocessing tokens are ignored, as are the other preprocessing tokens in the group. Only the first group whose control condition evaluates to true (nonzero) is processed; any following groups are skipped and their controlling directives are processed as if they were in a group that is skipped. If none of the conditions evaluates to true, and there is a **#else** directive, the group controlled by the **#else** is processed; lacking a **#else** directive, all the groups until the **#endif** are skipped.¹⁹⁷⁾

EXAMPLE 1 This demonstrates a way to include a header file only if it is available.

```
#if __has_include(<optional.h>)
#   include <optional.h>
#   define have_optional 1
#elif __has_include(<experimental/optional.h>)
#   include <experimental/optional.h>
#   define have_optional 1
#   define have_experimental_optional 1
#endif
#ifndef have_optional
#   define have_optional 0
#endif
```

EXAMPLE 2

```
/* Fallback for compilers not yet implementing this feature. */
#ifndef __has_c_attribute
#define __has_c_attribute(x) 0
#endif /* __has_c_attribute */

#if __has_c_attribute(fallthrough)
/* Standard attribute is available, use it. */
#define FALLTHROUGH [[fallthrough]]
#elif __has_c_attribute(vendor::fallthrough)
/* Vendor attribute is available, use it. */
#define FALLTHROUGH [[vendor::fallthrough]]
#else
/* Fallback implementation. */
#define FALLTHROUGH
#endif
```

EXAMPLE 3

```
#ifdef __STDC__
#define TITLE "ISO C Compilation"
#elifndef __cplusplus
#define TITLE "Non-ISO C Compilation"
#else
/* C++ */
#define TITLE "C++ Compilation"
#endif
```

¹⁹⁷⁾As indicated by the syntax, no preprocessing tokens are allowed to follow a **#else** or **#endif** directive before the terminating new-line character. However, comments can appear anywhere in a source file, including within a preprocessing directive.

EXAMPLE 4 A combination of `__FILE__` (6.10.10.2) and `__has_embed` can be used to check for support of specific implementation extensions for the `#embed` (6.10.4) directive's parameters.

```
#if __has_embed(__FILE__ ext::token(0xB055))
#define DESCRIPTION "Supports extended token embed parameter"
#else
#define DESCRIPTION "Does not support extended token embed parameter"
#endif
```

EXAMPLE 5 The following snippet uses `__has_embed` to check for support of a specific implementation-defined embed parameter, and otherwise uses standard behavior to produce the same effect.

```
void parse_into_s(short* ptr, unsigned char* ptr_bytes, unsigned long long size);

int main () {
    #if __has_embed ("bits.bin" ds9000::element_type(short))
        /* Implementation extension: create short integers from the */
        /* translation environment resource into */
        /* a sequence of integer constants */
        short meow[] = {
            #embed "bits.bin" ds9000::element_type(short)
        };
    #elif __has_embed ("bits.bin")
        /* no support for implementation-specific */
        /* ds9000::element_type(short) parameter */
        const unsigned char meow_bytes[] = {
            #embed "bits.bin"
        };
        short meow[sizeof(meow_bytes) / sizeof(short)] = {};
        /* parse meow_bytes into short values by-hand! */
        parse_into_s(meow, meow_bytes, sizeof(meow_bytes));
    #else
        #error "cannot find bits.bin resource"
    #endif
    return (int)(meow[0] + meow[(sizeof(meow) / sizeof(*meow)) - 1]);
}
```

EXAMPLE 6 If the search for the resource is successful, this resource is always considered empty due to the `limit(0)` embed parameter, including in `__has_embed` expressions.

```
int main () {
    #if __has_embed(<infinite-resource> limit(0)) == 2
        // if <infinite-resource> exists, this
        // token sequence is always taken.
        return 0;
    #else
        // the 'infinite-resource' resource does not exist
        #error "The resource does not exist"
    #endif
}
```

Forward references: macro replacement (6.10.5), source file inclusion (6.10.3), mandatory macros (6.10.10.2), largest integer types (7.22.2.6).

6.10.3 Source file inclusion

Constraints

A `#include` directive shall identify a header or source file that can be processed by the implementation.

A preprocessing directive of the form

searches a sequence of implementation-defined places for a header identified uniquely by the specified sequence between the < and > delimiters, and causes the replacement of that directive by the entire contents of the header. How the places are specified or the header identified is implementation-defined.

```
# include "q-char-sequence" new-line
```

causes the replacement of that directive by the entire contents of the source file identified by the specified sequence between the " delimiters. The named source file is searched for in an implementation-defined manner. If this search is not supported, or if the search fails, the directive is reprocessed as if it read

with the identical contained sequence (including > characters, if any) from the original directive.

```
# include pp-tokens new-line
```

(that does not match one of the two previous forms) is permitted. The preprocessing tokens after **include** in the directive are processed just as in normal text. (Each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens.) The directive resulting after all replacements shall match one of the two previous forms.¹⁹⁸⁾ The method by which a sequence of preprocessing tokens between a < and a > preprocessing token pair or a pair of " characters is combined into a single header name preprocessing token is implementation-defined.

The implementation shall provide unique mappings for sequences consisting of one or more nondigits or digits (6.4.3.1) followed by a period (.) and a single nondigit. The first character shall not be a digit. The implementation can ignore distinctions of alphabetical case and restrict the mapping to eight significant characters before the period.

A **#include** preprocessing directive may appear in a source file that has been read because of a **#include** directive in another file, up to an implementation-defined nesting limit (see 5.3.5.2).

EXAMPLE 1 The most common uses of **#include** preprocessing directives are as in the following:

EXAMPLE 2 This illustrates macro-replaced `#include` directives:

```
#if VERSION == 1
    #define INCFILE "vers1.h"
#elif VERSION == 2
    #define INCFILE "vers2.h"    // and so on
#else
    #define INCFILE "versN.h"
#endif
#include INCFILE
```

Forward references: macro replacement (6.10.5).

¹⁹⁸ Adjacent string literals are not concatenated into a single string literal (see the translation phases in 5.2.1.2); thus, an expansion that results in two string literals is an invalid directive.

6.10.4 Binary resource inclusion

6.10.4.1 `#embed` preprocessing directive

Description

A *resource* is a source of data accessible from the translation environment. An *embed parameter* is a single preprocessor parameter in the *embed parameter sequence*. It has an *implementation resource width*, which is the implementation-defined size in bits of the located resource. It also has a *resource width*, which is either:

- the number of bits as computed from the optionally-provided **limit** embed parameter (6.10.4.2), if present; or,
- the implementation resource width.

An *embed parameter sequence* is a whitespace-delimited list of preprocessor parameters which can modify the result of the replacement for the **#embed** preprocessing directive.

Constraints

An **#embed** directive shall identify a resource that can be processed by the implementation as a binary data sequence given the provided embed parameters.

Embed parameters not specified in this document shall be implementation-defined. Implementation-defined embed parameters may change the subsequently-defined semantics of the directive; otherwise, **#embed** directives which do not contain implementation-defined embed parameters shall behave as described in this document.

A resource is considered empty when its resource width is zero.

Let *embed element width* be either:

- an integer constant expression greater than zero determined by an implementation-defined embed parameter; or,
- **CHAR_BIT** (5.3.5.3.2).

The result of $(\text{resource width}) \% (\text{embed element width})$ shall be zero.¹⁹⁹⁾

Semantics

The expansion of a **#embed** directive is a token sequence formed from the list of integer constant expressions described later in this subclause. The group of tokens for each integer constant expression in the list is separated in the token sequence from the group of tokens for the previous integer constant expression in the list by a comma. The sequence neither begins nor ends in a comma. If the list of integer constant expressions is empty, the token sequence is empty. The directive is replaced by its expansion and, with the presence of certain embed parameters, additional or replacement token sequences.

A preprocessing directive of the form

embed < *h-char-sequence* > *embed-parameter-sequence*_{opt} *new-line*

searches a sequence of implementation-defined places for a resource identified uniquely by the specified sequence between the < and >. The search for the named resource is done in an implementation-defined manner.

A preprocessing directive of the form

embed " *q-char-sequence* " *embed-parameter-sequence*_{opt} *new-line*

searches a sequence of implementation-defined places for a resource identified uniquely by the specified sequence between the " delimiters. The search for the named resource is done in an

¹⁹⁹⁾This constraint helps ensure data is neither filled with padding values nor truncated in a given environment, and helps ensure the data is portable with respect to usages of **memcpy** (7.26.2.1) with character type arrays initialized from the data.

implementation-defined manner. If this search is not supported, or if the search fails, the directive is reprocessed as if it read

embed < *h-char-sequence* > *embed-parameter-sequence*_{opt} *new-line*

with the identical contained *q-char-sequence* (including > characters, if any) from the original directive.

Either form of the **#embed** directive specified previously behaves as specified later in this subclause. The values of the integer constant expressions in the expanded sequence are determined by an implementation-defined mapping of the resource's data. Each integer constant expression's value is in the range from 0 to $(2^{\text{embed element width}}) - 1$, inclusive.²⁰⁰⁾ If:

- the list of integer constant expressions is used to initialize an array of a type compatible with **unsigned char**, or compatible with **char** if **char** cannot hold negative values; and,
- the embed element width is equal to **CHAR_BIT** (5.3.5.3.2),

then the contents of the initialized elements of the array are as-if the resource's binary data is **fread** (7.23.8.1) into the array at translation time.

A preprocessing directive of the form

embed *pp-tokens new-line*

(that does not match one of the two previous forms) is permitted. The preprocessing tokens after **embed** in the directive are processed just as in normal text. (Each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens.) The directive resulting after all replacements shall match one of the two previous forms.²⁰¹⁾ The method by which a sequence of preprocessing tokens between a < and a > preprocessing token pair or a pair of " characters is combined into a single resource name preprocessing token is implementation-defined.

An embed parameter with a preprocessor parameter token that is one of the following is a *standard embed parameter*:

limit	prefix	suffix	if_empty
-------	--------	--------	----------

The significance of these standard embed parameters is specified later in this subclause.

Recommended practice

The **#embed** directive is meant to translate binary data in a resource to a sequence of integer constant expressions in a way that preserves the value of the resource's bit stream where possible.

A mechanism similar to, but distinct from, the implementation-defined search paths used for source file inclusion (6.10.3) is encouraged.

Implementations should take into account translation-time bit and byte orders as well as execution-time bit and byte orders to more appropriately represent the resource's binary data from the directive. This maximizes the chance that, if the resource referenced at translation time through the **#embed** directive is the same one accessed through execution-time means, the data that is e.g. **fread** or similar into contiguous storage will compare bit-for-bit equal to an array of character type initialized from an **#embed** directive's expanded contents.

EXAMPLE 1 Placing a small image resource.

```
#include <stddef.h>

void have_you_any_wool(const unsigned char*, size_t);

int main (int, char*[]) {
```

²⁰⁰⁾For example, an embed element width of 8 will yield a range of values from 0 to 255, inclusive.
²⁰¹⁾Adjacent string literals are not concatenated into a single string literal (see the translation phases in 5.2.1.2); thus, an expansion that results in two string literals is an invalid directive.

```

static const unsigned char baa_baa[] = {
#embed "black_sheep.ico"
};

have_you_any_wool(baa_baa, sizeof(baa_baa));

return 0;
}

```

EXAMPLE 2 This snippet:

```

int main (int, char*[]) {
    static const unsigned char coefficients[] = {
#embed "only_8_bits.bin" // potential constraint violation
    };

    return 0;
}

```

can violate the constraint that $(resource\ width) \% (embed\ element\ width)$ is 0. There is a chance that the implementation-defined 8 bit width of the resource is not evenly divisible by the embed element width (e.g. on a system where **CHAR_BIT** is 16). Issuing a diagnostic in this case can aid in portability by calling attention to potentially incompatible expectations between implementations and their resources.

EXAMPLE 3 Initialization of non-arrays.

```

int main () {
    /* Braces can be kept or elided as per normal initialization rules */
    int i = {
#embed "i.dat"
    }; /* valid if i.dat produces 1 value,
        i value is [0, 2(embed element width)) */
    int i2 =
#embed "i.dat"
    ; /* valid if i.dat produces 1 value,
        i2 value is [0, 2(embed element width)) */
    struct s {
        double a, b, c;
        struct { double e, f, g; };
        double h, i, j;
    };
    struct s x = {
    /* initializes each element in order according to initialization
    rules with comma-separated list of integer constant expressions
    inside of braces */
#embed "s.dat"
    };
    return 0;
}

```

Non-array types can still be initialized since the directive produces a comma-delimited list of integer constant expressions, a single integer constant expression, or nothing.

EXAMPLE 4 Equivalency of bit sequence and bit order between a translation-time read and an execution-time read of the same resource/file.

```

#include <string.h>
#include <stddef.h>
#include <stdio.h>

int main(void) {
    static const unsigned char embed_data[] = {

```

```

#embed <data.dat>
    };

    const size_t f_size = sizeof(embed_data);
    unsigned char f_data[f_size];
    FILE* f_source = fopen("data.dat", "rb");
    if (f_source == nullptr)
        return 1;
    char* f_ptr = (char*)&f_data[0];
    if (fread(f_ptr, 1, f_size, f_source) != f_size) {
        fclose(f_source);
        return 1;
    }
    fclose(f_source);

    int is_same = memcmp(&embed_data[0], f_ptr, f_size);
    // if both operations refers to the same resource/file at
    // execution time and translation time, "is_same" should be 0
    return is_same == 0 ? 0 : 1;
}

```

6.10.4.2 limit parameter

Constraints

The **limit** standard embed parameter can appear zero times or one time in the embed parameter sequence. Its preprocessor argument clause shall be present and have the form:

(*constant-expression*)

and shall be an integer constant expression. The integer constant expression shall not evaluate to a value less than 0.

The token **defined** shall not appear within the constant expression.

Semantics

The embed parameter with a preprocessor parameter token limit denotes a balanced preprocessing token sequence that will be used to compute the resource width. Independently of any macro replacement done previously (e.g. when matching the form of **#embed**), the constant expression is evaluated after the balanced preprocessing token sequence is processed as in normal text, using the rules specified for conditional inclusion (6.10.2), with the exception that any defined macro expressions are not permitted.

The resource width is:

- 0, if the integer constant expression evaluates to 0; or,
- the implementation resource width if it is less than the embed element width multiplied by the integer constant expression; or,
- the embed element width multiplied by the integer constant expression, if it is less than or equal to the implementation resource width.

EXAMPLE 1 Checking the first 4 elements of a sound resource.

```

#include <assert.h>

int main (int, char*[]) {
    static const char sound_signature[] = {
#embed <sdk/jump.wav> limit(2+2)
    };
    static_assert((sizeof(sound_signature) / sizeof(*sound_signature)) == 4,
        "There should only be 4 elements in this array.");
}

```

```

        // verify PCM WAV resource
        assert(sound_signature[0] == 'R');
        assert(sound_signature[1] == 'I');
        assert(sound_signature[2] == 'F');
        assert(sound_signature[3] == 'F');
        assert(sizeof(sound_signature) == 4);

        return 0;
    }

```

EXAMPLE 2 Similar to a previous example, except it illustrates macro expansion specifically done for the `limit(...)` parameter.

```

#include <assert.h>

#define TWO_PLUS_TWO 2+2

int main (int, char*[]) {
    const char sound_signature[] = {
        /* the token sequence within the parentheses
        for the "limit" parameter undergoes macro
        expansion, at least once, resulting in
        #embed <sdk/jump.wav> limit(2+2)
        */
        #embed <sdk/jump.wav> limit(TWO_PLUS_TWO)
    };
    static_assert((sizeof(sound_signature) / sizeof(*sound_signature)) == 4,
        "There should only be 4 elements in this array.");

    // verify PCM WAV resource
    assert(sound_signature[0] == 'R');
    assert(sound_signature[1] == 'I');
    assert(sound_signature[2] == 'F');
    assert(sound_signature[3] == 'F');
    assert(sizeof(sound_signature) == 4);

    return 0;
}

```

EXAMPLE 3 A potential constraint violation since an environment that has a **CHAR_BIT** whose value is greater than 24 can possibly not get enough data from the resource.

```

int main (int, char*[]) {
    const unsigned char arr[] = {
        #embed "24_bits.bin" limit(1) // can be a constraint violation
    };

    return 0;
}

```

EXAMPLE 4 Resources interfacing with certain implementations can receive an infinite stream of data, such as the (hypothetical) `</owo/uwurandom>` resource used in the following snippet:

```

int main (int, char*[]) {
    const unsigned char arr[] = {
        #embed </owo/uwurandom> limit(513)
    };

    return 0;
}

```

The **limit** parameter can help process only a portion of that information and prevent exhaustion of an implementation’s internal resources when processing such data.

6.10.4.3 **suffix parameter**

Constraints

The **suffix** standard embed parameter can appear zero times or one time in the embed parameter sequence. Its preprocessor argument clause shall be present and have the form:

(*pp-balanced-token-sequence*_{opt})

Semantics

The embed parameter with a preprocessing parameter token **suffix** denotes a balanced preprocessing token sequence within its preprocessor argument clause that will be placed immediately after the result of the associated **#embed** directive’s expansion.

If the resource is empty, then **suffix** has no effect and is ignored.

EXAMPLE Extra elements added to array initializer.

```
#include <string.h>

#ifndef SHADER_TARGET
#define SHADER_TARGET "edith-impl.glsl"
#endif

extern char* null_term_shader_data;

void fill_in_data () {
    const char internal_data[] = {
#embed SHADER_TARGET \
        suffix(, )
        0
    };

    strcpy(null_term_shader_data, internal_data);
}
```

6.10.4.4 **prefix parameter**

Constraints

The **prefix** standard embed parameter can appear zero times or one time in the embed parameter sequence. Its preprocessor parameter clause shall be present and have the form:

(*pp-balanced-token-sequence*_{opt})

Semantics

The embed parameter with a preprocessor parameter token **prefix** denotes a balanced preprocessing token sequence within its preprocessor argument clause that will be placed immediately before the result of the associated **#embed** directive’s expansion, if any.

If the resource is empty, then **prefix** has no effect and is ignored.

EXAMPLE A null-terminated character array with prefixed and suffixed additional tokens when the resource is not empty, providing null termination and a byte order mark.

```
#include <assert.h>
#include <string.h>

#ifndef SHADER_TARGET
#define SHADER_TARGET "ches.glsl"
#endif

extern char* merp;
```

```

void init_data () {
    const char whl[] = {
#embed SHADER_TARGET \
        prefix(0xEF, 0xBB, 0xBF, ) /* UTF-8 BOM */ \
        suffix(,)
        0
    };
    // always null terminated,
    // contains BOM if not-empty
    int is_good = (sizeof(whl) == 1 && whl[0] == '\0')
    || (whl[0] == '\xEF' && whl[1] == '\xBB'
    && whl[2] == '\xBF' && whl[sizeof(whl) - 1] == '\0');
    assert(is_good);
    strcpy(merp, whl);
}

```

6.10.4.5 if_empty parameter

Constraints

The **if_empty** standard embed parameter can appear zero times or one time in the embed parameter sequence. Its preprocessor argument clause shall be present and have the form:

(*pp-balanced-token-sequence*_{opt})

Semantics

The embed parameter with a preprocessing parameter token **if_empty** denotes a balanced preprocessing token sequence within its preprocessor argument clause that will replace the **#embed** directive entirely.

If the resource is not empty, then **if_empty** has no effect and is ignored.

EXAMPLE 1 If the search for the resource is successful, this resource is always considered empty due to the **limit(0)** embed parameter. This program always returns 0, even if the resource is searched for and found successfully by the implementation and has an implementation resource width greater than 0.

```

int main () {
    return
#embed <some_resource> limit(0) prefix(1) if_empty(0)
;
    // becomes:
    // return 0;
}

```

EXAMPLE 2 An example similar to using the suffix **embed** parameter, but changed slightly.

```

#include <string.h>

#ifndef SHADER_TARGET
#define SHADER_TARGET "edith-impl.glsl"
#endif

extern char* null_term_shader_data;

void fill_in_data () {
    const char internal_data[] = {
#embed SHADER_TARGET \
        suffix(, 0) \
        if_empty(0)
    };

    strcpy(null_term_shader_data, internal_data);
}

```



```
}

```

EXAMPLE 3 This resource is considered empty due to the `limit(0)` embed parameter, meaning an `if_empty` expression replaces the directive as specified previously. A constraint is still violated if the search for the resource is unsuccessful.

```
int main () {
    return
        #embed <infinite-resource> limit(0) if_empty(45540)
    ;
}
```

becomes:

```
int main () {
    return 45540;
}
```

6.10.5 Macro replacement

6.10.5.1 General

Constraints

Two replacement lists are identical if and only if the preprocessing tokens in both have the same number, ordering, spelling, and white-space separation, where all white-space separations are considered identical.

An identifier currently defined as an object-like macro shall not be redefined by another `#define` preprocessing directive unless the second definition is an object-like macro definition and the two replacement lists are identical. Likewise, an identifier currently defined as a function-like macro shall not be redefined by another `#define` preprocessing directive unless the second definition is a function-like macro definition that has the same number and spelling of parameters, and the two replacement lists are identical.

There shall be white space between the identifier and the replacement list in the definition of an object-like macro.

If the identifier-list in the macro definition does not end with an ellipsis, the number of arguments (including those arguments consisting of no preprocessing tokens) in an invocation of a function-like macro shall equal the number of parameters in the macro definition. Otherwise, there shall be at least as many arguments in the invocation as there are parameters in the macro definition (excluding the ...). There shall exist a) preprocessing token that terminates the invocation.

The identifiers `__VA_ARGS__` and `__VA_OPT__` shall occur only in the replacement-list of a function-like macro that uses the ellipsis notation in the parameters.

A parameter identifier in a function-like macro shall be uniquely declared within its scope.

Semantics

The identifier immediately following the `define` is called the *macro name*. There is one name space for macro names. Any white-space characters preceding or following the replacement list of preprocessing tokens are not considered part of the replacement list for either form of macro.

If a # preprocessing token, followed by an identifier, occurs lexically at the point at which a preprocessing directive can begin, the identifier is not subject to macro replacement.

A preprocessing directive of the form

```
# define identifier replacement-list new-line
```

defines an *object-like macro* that causes each subsequent instance of the macro name²⁰²⁾ to be replaced

²⁰²⁾Since, by macro-replacement time, all character constants and string literals are preprocessing tokens, not sequences possibly containing identifier-like subsequences (see 5.2.1.2, translation phases), they are never scanned for macro names or parameters.

by the replacement list of preprocessing tokens that constitute the remainder of the directive. The replacement list is then rescanned for more macro names as specified later in this subclause.

A preprocessing directive of the form

```
# define identifier lparen identifier-listopt ) replacement-list new-line
# define identifier lparen ... ) replacement-list new-line
# define identifier lparen identifier-list , ... ) replacement-list new-line
```

defines a *function-like macro* with parameters, whose use is similar syntactically to a function call. The parameters are specified by the optional list of identifiers, whose scope extends from their declaration in the identifier list until the new-line character that terminates the **#define** preprocessing directive. Each subsequent instance of the function-like macro name followed by a (as the next preprocessing token introduces the sequence of preprocessing tokens that is replaced by the replacement list in the definition (an invocation of the macro). The replaced sequence of preprocessing tokens is terminated by the matching) preprocessing token, skipping intervening matched pairs of left and right parenthesis preprocessing tokens. Within the sequence of preprocessing tokens making up an invocation of a function-like macro, new-line is considered a normal white-space character.

The sequence of preprocessing tokens bounded by the outside-most matching parentheses forms the list of arguments for the function-like macro. The individual arguments within the list are separated by comma preprocessing tokens, but comma preprocessing tokens between matching inner parentheses do not separate arguments. If there are sequences of preprocessing tokens within the list of arguments that would otherwise act as preprocessing directives,²⁰³⁾ the behavior is undefined.

If there is a ... in the identifier-list in the macro definition, then the trailing arguments (if any), including any separating comma preprocessing tokens, are merged to form a single item: the *variable arguments*. The number of arguments so combined is such that, following merger, the number of arguments is one more than the number of parameters in the macro definition (excluding the ...), except that if there are as many arguments as named parameters, the macro invocation behaves as if a comma token has been appended to the argument list such that variable arguments are formed that contain no pp-tokens.

6.10.5.2 Argument substitution

Syntax

va-opt-replacement:

```
__VA_OPT__ ( pp-tokensopt )
```

Description

Argument substitution is a process during macro expansion in which identifiers corresponding to the parameters of the macro definition and the special constructs **__VA_ARGS__** and **__VA_OPT__** are replaced with token sequences from the arguments of the macro invocation and possibly of the argument of the feature **__VA_OPT__**. The latter process allows to control a substitute token sequence that is only expanded if the argument list that corresponds to a trailing ... of the parameter list is present and has a non-empty substitution.

Constraints

The identifier **__VA_OPT__** shall always occur as part of the preprocessing token sequence *va-opt-replacement*; its closing) is determined by skipping intervening pairs of matching left and right parentheses in its pp-tokens. The pp-tokens of a *va-opt-replacement* shall not contain **__VA_OPT__**. The pp-tokens shall form a valid replacement list for the current function-like macro.

Semantics

After the arguments for the invocation of a function-like macro have been identified, argument substitution takes place. A *va-opt-replacement* is treated as if it were a parameter. For each parameter in the replacement list that is neither preceded by a # or ## preprocessing token nor followed by a

²⁰³⁾Despite the name, a non-directive is a preprocessing directive.

preprocessing token, the preprocessing tokens naming the parameter are replaced by a token sequence determined as follows:

- If the parameter is of the form **va-opt-replacement**, the replacement preprocessing tokens are the preprocessing token sequence for the corresponding argument, as specified later in this subclause.
- Otherwise, the replacement preprocessing tokens are the preprocessing tokens of the corresponding argument after all macros contained therein have been expanded. The argument's preprocessing tokens are completely macro replaced before being substituted as if they formed the rest of the preprocessing file with no other preprocessing tokens being available.

EXAMPLE 1

```
#define LPAREN() (  
#define G(Q) 42  
#define F(R, X, ...) __VA_OPT__(G R X) )  
int x = F(LPAREN(), 0, <:-); // replaced by int x = 42;
```

An identifier **__VA_ARGS__** that occurs in the replacement list is treated as if it were a parameter, and the variable arguments form the preprocessing tokens used to replace it.

The preprocessing token sequence for the corresponding argument of a **va-opt-replacement** is defined as follows. If a (hypothetical) substitution of **__VA_ARGS__** as neither an operand of **#** nor **##** consists of no preprocessing tokens, the argument consists of a single placemaker preprocessing token (6.10.5.4, 6.10.5.5). Otherwise, the argument consists of the results of the expansion of the contained pp-tokens as the replacement list of the current function-like macro before removal of placemaker tokens, rescanning, and further replacement.

NOTE The placemaker tokens are removed before stringization (6.10.5.3), and can be removed by rescanning and further replacement (6.10.5.5).

EXAMPLE 2

```
#define F(...)      f(0 __VA_OPT__(,) __VA_ARGS__)  
#define G(X, ...)  f(0, X __VA_OPT__(,) __VA_ARGS__)  
#define SDEF(sname, ...) S name __VA_OPT__(= { __VA_ARGS__ })  
#define EMP  
  
F(a, b, c)        // replaced by f(0, a, b, c)  
F()               // replaced by f(0)  
F(EMP)            // replaced by f(0)  
  
G(a, b, c)        // replaced by f(0, a, b, c)  
G(a, )            // replaced by f(0, a)  
G(a)              // replaced by f(0, a)  
  
SDEF(foo);        // replaced by S foo;  
SDEF(bar, 1, 2);  // replaced by S bar = { 1, 2 };  
  
#define H1(X, ...) X __VA_OPT__(##) __VA_ARGS__  
// error: ## on line above  
// cannot appear at the beginning of a replacement  
// list (6.10.5.4)  
  
#define H2(X, Y, ...) __VA_OPT__(X ## Y,) __VA_ARGS__  
  
H2(a, b, c, d)    // replaced by ab, c, d  
  
#define H3(X, ...) #__VA_OPT__(X##X X##X)  
H3(, 0)           // replaced by ""
```

```

#define H4(X, ...)    __VA_OPT__(a X ## X) ## b
H4(, 1)                // replaced by a b

#define H5A(...)      __VA_OPT__()/**/__VA_OPT__()
#define H5B(X)        a ## X ## b
#define H5C(X)        H5B(X)
H5C(H5A())             // replaced by ab

```

6.10.5.3 The # operator

Constraints

Each **#** preprocessing token in the replacement list for a function-like macro shall be followed by a parameter as the next preprocessing token in the replacement list.

Semantics

If, in the replacement list, a parameter is immediately preceded by a **#** preprocessing token, both are replaced by a single character string literal preprocessing token that contains the spelling of the preprocessing token sequence for the corresponding argument (excluding placemaker tokens).

Let the *stringizing argument* be the preprocessing token sequence for the corresponding argument with placemaker tokens removed. Each occurrence of white space between the stringizing argument's preprocessing tokens becomes a single space character in the character string literal. White space before the first preprocessing token and after the last preprocessing token composing the stringizing argument is deleted. Otherwise, the original spelling of each preprocessing token in the stringizing argument is retained in the character string literal, except for special handling for producing the spelling of string literals and character constants: a **** character is inserted before each **"** and **** character of a character constant or string literal (including the delimiting **"** characters), except that it is implementation-defined whether a **** character is inserted before the **** character beginning a universal character name.

If the replacement that results is not a valid character string literal, the behavior is undefined. The character string literal corresponding to an empty stringizing argument is **""**. The order of evaluation of **#** and **##** operators is unspecified.

6.10.5.4 The ## operator

Constraints

A **##** preprocessing token shall not occur at the beginning or at the end of a replacement list for either form of macro definition.

Semantics

If, in the replacement list of a function-like macro, a parameter is immediately preceded or followed by a **##** preprocessing token, the parameter is replaced by the corresponding argument's preprocessing token sequence; however, if an argument consists of no preprocessing tokens, the parameter is replaced by a *placemaker* preprocessing token instead.²⁰⁴⁾

For both object-like and function-like macro invocations, before the replacement list is reexamined for more macro names to replace, each instance of a **##** preprocessing token in the replacement list (not from an argument) is deleted and the preceding preprocessing token is concatenated with the following preprocessing token. Placemaker preprocessing tokens are handled specially: concatenation of two placemarkers results in a single placemaker preprocessing token, and concatenation of a placemaker with a non-placemaker preprocessing token results in the non-placemaker preprocessing token. If the result is not a valid preprocessing token, the behavior is undefined. The resulting token is available for further macro replacement. The order of evaluation of **##** operators is unspecified.

EXAMPLE In the following fragment:

²⁰⁴⁾Placemaker preprocessing tokens do not appear in the syntax because they are temporary entities that exist only within translation phase 4 (5.2.1.2).

```
#define hash_hash # ## #
#define mkstr(a) # a
#define in_between(a) mkstr(a)
#define join(c, d) in_between(c hash_hash d)

char p[] = join(x, y); // equivalent to
                      // char p[] = "x ## y";
```

The expansion produces, at various stages:

```
join(x, y)

in_between(x hash_hash y)

in_between(x ## y)

mkstr(x ## y)

"x ## y"
```

In other words, expanding **hash_hash** produces a new token, consisting of two adjacent sharp signs, but this new token is not the **##** operator.

6.10.5.5 Rescanning and further replacement

After all parameters in the replacement list have been substituted and **#** and **##** processing has taken place, all placemaker preprocessing tokens are removed. The resulting preprocessing token sequence is then rescanned, along with all subsequent preprocessing tokens of the source file, for more macro names to replace.

If the name of the macro being replaced is found during this scan of the replacement list (not including the rest of the source file’s preprocessing tokens), it is not replaced. Furthermore, if any nested replacements encounter the name of the macro being replaced, it is not replaced. These nonreplaced macro name preprocessing tokens are no longer available for further replacement even if they are later (re)examined in contexts in which that macro name preprocessing token would otherwise have been replaced.

The resulting completely macro-replaced preprocessing token sequence is not processed as a preprocessing directive even if it resembles one, but all pragma unary operator expressions within it are then processed as specified in 6.10.11.

EXAMPLE There are cases where it is not clear whether a replacement is nested or not. For example, given the following macro definitions:

```
#define f(a) a*g
#define g(a) f(a)
```

the invocation

```
f(2)(9)
```

can expand to either

```
2*f(9)
```

or

```
2*9*g
```

Strictly conforming programs are not permitted to depend on such unspecified behavior.

6.10.5.6 Scope of macro definitions

A macro definition lasts (independent of block structure) until a corresponding **#undef** directive is

encountered or (if none is encountered) until the end of the preprocessing translation unit. Macro definitions have no significance after translation phase 4.

A preprocessing directive of the form

```
# undef identifier new-line
```

causes the specified identifier no longer to be defined as a macro name. It is ignored if the specified identifier is not currently defined as a macro name.

EXAMPLE 1 The simplest use of this facility is to define a “manifest constant”, as in

```
#define TABSIZE 100

int table[TABSIZE];
```

EXAMPLE 2 The following defines a function-like macro whose value is the maximum of its arguments. It has the advantages of working for any compatible types of the arguments and of generating in-line code without the overhead of function calling. It has the disadvantages of evaluating one or the other of its arguments a second time (including side effects) and generating more code than a function if invoked several times. It also cannot have its address taken, as it has none.

```
#define max(a, b) ((a) > (b) ? (a) : (b))
```

The parentheses ensure that the arguments and the resulting expression are bound properly.

EXAMPLE 3 To illustrate the rules for redefinition and reexamination, the sequence

```
#define x      3
#define f(a)   f(x * (a))
#undef x
#define x      2
#define g      f
#define z      z[0]
#define h      g(~
#define m(a)   a(w)
#define w      0,1
#define t(a)   a
#define p()    int
#define q(x)   x
#define r(x,y) x ## y
#define str(x) # x

f(y+1) + f(f(z)) % t(t(g)(0) + t)(1);
g(x+(3,4)-w) | h 5) & m
    (f)^m(m);
p() i[q()] = { q(1), r(2,3), r(4,), r(,5), r(,) };
char c[2][6] = { str(hello), str() };
```

results in

```
f(2 * (y+1)) + f(2 * (f(2 * (z[0])))) % f(2 * (0)) + t(1);
f(2 * (2+(3,4)-0,1)) | f(2 * (~ 5)) & f(2 * (0,1))^m(0,1);
int i[] = { 1, 23, 4, 5, };
char c[2][6] = { "hello", "" };
```

EXAMPLE 4 To illustrate the rules for creating character string literals and concatenating tokens, the sequence

```
#define str(s)      # s
#define xstr(s)     str(s)
#define debug(s, t) printf("x" # s " = %d, x" # t " = %s", \
                          x ## s, x ## t)

#define INCFILE(n)  vers ## n
#define glue(a, b)  a ## b
```

```

#define xglue(a, b) glue(a, b)
#define HIGHLOW      "hello"
#define LOW          LOW " ", world"

debug(1, 2);
fputs(str(strncmp("abc\0d", "abc", '\4') // this goes away
      == 0) str(: @\n), s);
#include xstr(INCFIL(2).h)
glue(HIGH, LOW);
xglue(HIGH, LOW)

```

results in

```

printf("x" "1" "=" %d, x" "2" "=" %s", x1, x2);
fputs(
  "strncmp(\"abc\\0d\", \"abc\", '\\4') == 0" ": @\n",
  s);
#include "vers2.h"      (after macro replacement, before file access)
"hello";
"hello" " ", world"

```

or, after concatenation of the character string literals,

```

printf("x1= %d, x2= %s", x1, x2);
fputs(
  "strncmp(\"abc\\0d\", \"abc\", '\\4') == 0: @\n",
  s);
#include "vers2.h"      (after macro replacement, before file access)
"hello";
"hello, world"

```

Space around the # and ## tokens in the macro definition is optional.

EXAMPLE 5 To illustrate the rules for placemarkers preprocessing tokens, the sequence

```

#define t(x,y,z) x ## y ## z
int j[] = { t(1,2,3), t(4,5), t(6,,7), t(8,9,),
            t(10,,), t(,11,), t(,12), t(,,) };

```

results in

```

int j[] = { 123, 45, 67, 89,
            10, 11, 12, };

```

EXAMPLE 6 To demonstrate the redefinition rules, the following sequence is valid.

```

#define OBJ_LIKE      (1-1)
#define OBJ_LIKE      /* white space */ (1-1) /* other */
#define FUNC_LIKE(a)  ( a )
#define FUNC_LIKE( a ) (      /* note the white space */ \
                          a /* other stuff on this line
                          */)

```

But the following redefinitions of the preceding definitions are invalid:

```

#define OBJ_LIKE      (0)      // different token sequence
#define OBJ_LIKE      (1 - 1) // different white space
#define FUNC_LIKE(b) ( a )     // different parameter usage
#define FUNC_LIKE(b) ( b )     // different parameter spelling

```

EXAMPLE 7 Finally, to show the variable argument list macro facilities:


```
#define debug(...)      fprintf(stderr, __VA_ARGS__)
#define showlist(...)   puts(#__VA_ARGS__)
#define report(test, ...) ((test)?puts(#test):\
    printf(__VA_ARGS__))
debug("Flag");
debug("X = %d\n", x);
showlist(The first, second, and third items.);
report(x>y, "x is %d but y is %d", x, y);
```

results in

```
fprintf(stderr, "Flag");
fprintf(stderr, "X = %d\n", x);
puts("The first, second, and third items.");
((x>y)?puts("x>y"):
    printf("x is %d but y is %d", x, y));
```

6.10.6 Line control

Constraints

The string literal of a **#line** directive, if present, shall be a character string literal.

Semantics

The *line number* of the current source line is one greater than the number of new-line characters read or introduced in translation phase 1 (5.2.1.2) while processing the source file to the current token.

If a preprocessing token (in particular **__LINE__**) spans two or more physical lines, it is unspecified which of those line numbers is associated with that token. If a preprocessing directive spans two or more physical lines, it is unspecified which of those line numbers is associated with the preprocessing directive. If a macro invocation spans multiple physical lines, it is unspecified which of those line numbers is associated with that invocation. The line number of a preprocessing token is independent of the context (in particular, as a macro argument or in a preprocessing directive). The line number of a **__LINE__** in a macro body is the line number of the macro invocation.

A preprocessing directive of the form

```
# line digit-sequence new-line
```

causes the implementation to behave as if the following sequence of source lines begins with a source line that has a line number as specified by the digit sequence (interpreted as a decimal integer, ignoring any optional digit separators (6.4.5.2) in the digit sequence). The digit sequence shall not specify zero, nor a number greater than 2147483647.

A preprocessing directive of the form

```
# line digit-sequence " s-char-sequenceopt " new-line
```

sets the presumed line number similarly and changes the presumed name of the source file to be the contents of the character string literal.

A preprocessing directive of the form

```
# line pp-tokens new-line
```

(that does not match one of the two previous forms) is permitted. The preprocessing tokens after **line** on the directive are processed just as in normal text (each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens). The directive resulting after all replacements shall match one of the two previous forms and is then processed as appropriate.²⁰⁵⁾

²⁰⁵⁾ Because a new-line is explicitly included as part of the **#line** directive, the number of new-line characters read while processing to the first *pp-token* can be different depending on whether the implementation uses a one-pass preprocessor. Therefore, there are two possible values for the line number following a directive of the form **#line** **__LINE__** *new-line*.

Recommended practice

The line number associated with a *pp-token* should be the line number of the first character of the *pp-token*. The line number associated with a preprocessing directive should be the line number of the line with the first `#` token. The line number associated with a macro invocation should be the line number of the first character of the macro name in the invocation.

6.10.7 Diagnostic directives

Semantics

A preprocessing directive of either form

```
# error pp-tokensopt new-line
# warning pp-tokensopt new-line
```

causes the implementation to produce a diagnostic message that includes the specified sequence of preprocessing tokens.

6.10.8 Pragma directive

Semantics

A preprocessing directive of the form

```
# pragma pp-tokensopt new-line
```

where the preprocessing token **STDC** does not immediately follow **pragma** in the directive (prior to any macro replacement)²⁰⁶⁾ causes the implementation to behave in an implementation-defined manner. The behavior can cause translation to fail or cause the translator or the resulting program to behave in a non-conforming manner. Any such **pragma** that is not recognized by the implementation is ignored.

If the preprocessing token **STDC** does immediately follow **pragma** in the directive (prior to any macro replacement), then no macro replacement is performed on the directive, and the directive shall have one of the following forms²⁰⁷⁾ whose meanings are described elsewhere:

standard-pragma:

```
# pragma STDC FP_CONTRACT on-off-switch
# pragma STDC FENV_ACCESS on-off-switch
# pragma STDC FENV_DEC_ROUND dec-direction
# pragma STDC FENV_ROUND direction
# pragma STDC CX_LIMITED_RANGE on-off-switch
```

on-off-switch: one of

ON OFF DEFAULT

direction: one of

FE_DOWNWARD FE_TONEAREST FE_TONEARESTFROMZERO
FE_TOWARDZERO FE_UPWARD FE_DYNAMIC

dec-direction: one of

FE_DEC_DOWNWARD FE_DEC_TONEAREST FE_DEC_TONEARESTFROMZERO
FE_DEC_TOWARDZERO FE_DEC_UPWARD FE_DEC_DYNAMIC

²⁰⁶⁾An implementation is not required to perform macro replacement in pragmas, but it is permitted except for in standard pragmas (where **STDC** immediately follows **pragma**). If the result of macro replacement in a non-standard pragma has the same form as a standard pragma, the behavior is still implementation-defined; an implementation is permitted to behave as if it were the standard pragma, but is not required to.

²⁰⁷⁾See “future language directions” (6.11.6).

Recommended practice

Implementations are encouraged to diagnose unrecognized pragmas.

Forward references: the **FP_CONTRACT** pragma (7.12.3), the **FENV_ACCESS** pragma (7.6.2), the **FENV_DEC_ROUND** pragma (7.6.4), the **FENV_ROUND** pragma (7.6.3), the **CX_LIMITED_RANGE** pragma (7.3.4).

6.10.9 Null directive**Semantics**

A preprocessing directive of the form

new-line

has no effect.

6.10.10 Predefined macro names**6.10.10.1 General**

The values of the predefined macros listed in the following subclauses²⁰⁸⁾ (except for **__FILE__** and **__LINE__**) remain constant throughout the translation unit.

None of the following macro names in this subclause nor the identifiers **defined**, **__has_c_attribute**, **__has_include**, or **__has_embed** shall be the subject of a **#define** or a **#undef** preprocessing directive. Any other predefined macro names: shall begin with a leading underscore followed by an uppercase letter; or, a second underscore; or, shall be any of the identifiers **alignas**, **alignof**, **bool**, **false**, **static_assert**, **thread_local**, or **true**.

The implementation shall not predefine the macro **__cplusplus**, nor shall it define it in any standard header.

Forward references: standard headers (7.1.2).

6.10.10.2 Mandatory macros

The following macro names shall be defined by the implementation:

__DATE__ The date of translation of the preprocessing translation unit: a character string literal of the form "**Mmm dd yyyy**", where the names of the months are the same as those generated by the **asctime** function, and the first character of **dd** is a space character if the value is less than 10. If the date of translation is not available, an implementation-defined valid date shall be supplied.

__FILE__ The presumed name of the current source file (a character string literal).²⁰⁹⁾

__LINE__ The presumed line number (within the current source file) of the current source line (an integer constant).²⁰⁹⁾

__STDC__ The integer constant **1**, intended to indicate a conforming implementation.

__STDC_EMBED_NOT_FOUND__, **__STDC_EMBED_FOUND__**, **__STDC_EMBED_EMPTY__** The integer constants **0**, **1**, and **2**, respectively.

__STDC_HOSTED__ The integer constant **1** if the implementation is a hosted implementation or the integer constant **0** if it is not.

__STDC_UTF_16__ The integer constant **1**, intended to indicate that values of type **char16_t** are UTF-16 encoded.

__STDC_UTF_32__ The integer constant **1**, intended to indicate that values of type **char32_t** are UTF-32 encoded.

²⁰⁸⁾See "future language directions" (6.11.7).

²⁰⁹⁾The presumed source file name and line number can be changed by the **#line** directive.

___STDC_VERSION___ The integer constant 202311L.²¹⁰⁾

___TIME___ The time of translation of the preprocessing translation unit: a character string literal of the form "**hh:mm:ss**" as in the time generated by the **asctime** function. If the time of translation is not available, an implementation-defined valid time shall be supplied.

Forward references: the **asctime** function (7.29.3.2).

6.10.10.3 Environment macros

The following macro names are conditionally defined by the implementation:

___STDC_ISO_10646___ An integer constant of the form **yyyymmL** (for example, **202012L**). If this symbol is defined, then every character in the Unicode required set, when stored in an object of type **wchar_t**, has the same value as the short identifier of that character. The *Unicode required set* consists of all the characters that are defined by ISO/IEC 10646, along with all amendments and technical corrigenda, as of the specified year and month. If some other encoding is used, the macro shall not be defined and the actual encoding used is implementation-defined.

___STDC_MB_MIGHT_NEQ_WC___ The integer constant **1**, intended to indicate that, in the encoding for **wchar_t**, a member of the basic character set is not required to have a code value equal to its value when used as the lone character in an integer character constant.

Forward references: common definitions (7.21), Unicode utilities (7.30).

6.10.10.4 Conditional feature macros

The following macro names are conditionally defined by the implementation:

___STDC_ANALYZABLE___ The integer constant **1**, if the implementation conforms to the specifications in Annex L (Analyzability).

___STDC_IEC_60559_BFP___ The integer constant 202311L, intended to indicate conformance to Annex F (ISO/IEC 60559 floating-point arithmetic) for binary floating-point arithmetic.

___STDC_IEC_559___ The integer constant **1**, intended to indicate conformance to the specifications in Annex F (ISO/IEC 60559 floating-point arithmetic) for binary floating-point arithmetic. Use of this macro is an obsolescent feature.

___STDC_IEC_60559_DFP___ The integer constant 202311L, intended to indicate support of decimal floating types and conformance to Annex F (ISO/IEC 60559 floating-point arithmetic) for decimal floating-point arithmetic.

___STDC_IEC_60559_COMPLEX___ The integer constant 202311L, intended to indicate conformance to the specifications in Annex G (ISO/IEC 60559 compatible complex arithmetic).

___STDC_IEC_60559_TYPES___ The integer constant 202311L, intended to indicate conformance to the specification in Annex H (ISO/IEC 60559 interchange and extended types).

___STDC_IEC_559_COMPLEX___ The integer constant **1**, intended to indicate adherence to the specifications in Annex G (ISO/IEC 60559 compatible complex arithmetic). Use of this macro is an obsolescent feature.

___STDC_LIB_EXT1___ The integer constant 202311L, intended to indicate support for the extensions defined in Annex K (Bounds-checking interfaces).

___STDC_NO_ATOMICS___ The integer constant **1**, intended to indicate that the implementation does not support atomic types (including the **_Atomic** type qualifier) and the `<stdatomic.h>` header.

²¹⁰⁾See Annex M for the values in previous editions of this document. The intention is that this will remain an integer constant of type **long int** that is increased with each edition of this document.

___STDC_NO_COMPLEX___ The integer constant **1**, intended to indicate that the implementation does not support complex types or the `<complex.h>` header.

___STDC_NO_THREADS___ The integer constant **1**, intended to indicate that the implementation does not support the `<threads.h>` header.

___STDC_NO_VLA___ The integer constant **1**, intended to indicate that the implementation does not support variable length arrays with automatic storage duration. Parameters declared with variable length array types are adjusted and then define objects of automatic storage duration with pointer types. Thus, support for such declarations is mandatory.

NOTE The intention for the macros ___STDC_LIB_EXT1___, ___STDC_IEC_60559_BFP___, ___STDC_IEC_60559_DFP___, ___STDC_IEC_60559_COMPLEX___, and ___STDC_IEC_60559_TYPES___, with the value 202311L, is that this will remain an integer constant of type **long int** that is increased with each edition of this document.

An implementation that defines ___STDC_NO_COMPLEX___ shall not define ___STDC_IEC_60559_COMPLEX___ or ___STDC_IEC_559_COMPLEX___.

6.10.11 Pragma operator

Semantics

A unary operator expression of the form:

_Pragma (*string-literal*)

is processed as follows: The string literal is *destringized* by deleting any encoding prefix, deleting the leading and trailing double-quotes, replacing each escape sequence `\` by a double-quote, and replacing each escape sequence `\\` by a single backslash. The resulting sequence of characters is processed through translation phase 3 to produce preprocessing tokens that are executed as if they were the *pp-tokens* in a pragma directive. The original four preprocessing tokens in the unary operator expression are removed.

EXAMPLE A directive of the form:

```
#pragma listing on "..\listing.dir"
```

can also be expressed as:

```
_Pragma ("listing on \"..\listing.dir\"")
```

The latter form is processed in the same way whether it appears literally as shown, or results from macro replacement, as in:

```
#define LISTING(x) PRAGMA(listing on #x)
#define PRAGMA(x) _Pragma(#x)

LISTING (..\listing.dir)
```

6.11 Future language directions

6.11.1 Floating types

Future standardization can include additional floating types, including those with greater range, precision, or both than **long double**.

6.11.2 Linkages of identifiers

Declaring an identifier with internal linkage at file scope without the **static** or **constexpr** storage-class specifier is an obsolescent feature.

6.11.3 External names

Restriction of the significance of an external name to fewer than 255 characters (considering each universal character name or extended source character as a single character) is an obsolescent feature that is a concession to existing implementations.

6.11.4 Character escape sequences

Lowercase letters as escape sequences are reserved for future standardization. Other characters may be used in extensions.

6.11.5 Storage-class specifiers

The placement of a storage-class specifier other than at the beginning of the declaration specifiers in a declaration is an obsolescent feature.

Future standardization can change the **auto** storage-class specifier to a type specifier.

6.11.6 Pragma directives

Pragmas whose first preprocessing token is **STDC** are reserved for future standardization.

6.11.7 Predefined macro names

Macro names beginning with **__STDC__** are reserved for future standardization.

Uses of the **__STDC_IEC_559__** and **__STDC_IEC_559_COMPLEX__** macros are obsolescent features.

7. Library

7.1 Introduction

7.1.1 Definitions of terms

A *string* is a contiguous sequence of characters terminated by and including the first null character. The term *multibyte string* is sometimes used instead to emphasize special processing given to multibyte characters contained in the string or to avoid confusion with a wide string. A *pointer to a string* is a pointer to its initial (lowest addressed) character. The *length of a string* is the number of bytes preceding the null character and the *value of a string* is the sequence of the values of the contained characters, in order.

The *decimal-point character* is the character used by functions that convert floating-point numbers to or from character sequences to denote the beginning of the fractional part of such character sequences.²¹¹⁾ It is represented in the text and examples by a period, but can be changed by the **setlocale** function.

A *null wide character* is a wide character with code value zero.

A *wide string* is a contiguous sequence of wide characters terminated by and including the first null wide character. A *pointer to a wide string* is a pointer to its initial (lowest addressed) wide character. The *length of a wide string* is the number of wide characters preceding the null wide character and the *value of a wide string* is the sequence of code values of the contained wide characters, in order.

A *shift sequence* is a contiguous sequence of bytes within a multibyte string that (potentially) causes a change in shift state (see 5.3.2). A shift sequence shall not have a corresponding wide character; it is instead taken to be an adjunct to an adjacent multibyte character.²¹²⁾ In this clause, “*white-space character*” refers to (execution) white-space character as defined by **isspace**. “*White-space wide character*” refers to (execution) white-space wide character as defined by **iswspace**.

Forward references: character handling (7.4), the **setlocale** function (7.11.2).

7.1.2 Standard headers

Each library function is declared in a *header*,²¹³⁾ whose contents are made available by the **#include** preprocessing directive. The header declares a set of related functions, plus any types and additional macros needed to facilitate the use of such related functions. In addition to the provisions given in this clause, an implementation that defines **__STDC_IEC_60559_BFP__**, **__STDC_IEC_559__**, or **__STDC_IEC_60559_DFP__** shall conform to the specifications in Annex F, one that defines **__STDC_IEC_60559_COMPLEX__** or **__STDC_IEC_559_COMPLEX__** shall conform to the specifications in Annex G, one that defines **__STDC_IEC_60559_TYPES__** shall conform to the specifications in Annex H and one that defines **__STDC_LIB_EXT1__** shall conform to the specifications in Annex K, and those Annexes should be read as if they were merged into the parallel structure of named subclauses of this clause. Declarations of types described here, in Annex H, or in Annex K, shall not include type qualifiers, unless explicitly stated otherwise.

An implementation that does not support decimal floating types (6.10.10.4) may not support interfaces or aspects of interfaces that are specific to these types.

The standard headers are²¹⁴⁾

²¹¹⁾The functions that make use of the decimal-point character are the numeric conversion functions (7.24.2, 7.31.4.2) and the formatted input/output functions (7.23.6, 7.31.2).

²¹²⁾For state-dependent encodings, the values for **MB_CUR_MAX** and **MB_LEN_MAX** are thus required to be large enough to count all the bytes in any complete multibyte character plus at least one adjacent shift sequence of maximum length. Whether these counts provide for more than one shift sequence is the implementation's choice.

²¹³⁾A header is not necessarily a source file, nor are the < and > delimited sequences in header names necessarily valid source file names.

²¹⁴⁾The headers <complex.h>, <stdatomic.h>, and <threads.h> are conditional features that implementations can support but are not required to; see 6.10.10.4.

<assert.h>	<setjmp.h>	<stdlib.h>
<complex.h>	<signal.h>	<stdnoreturn.h>
<ctype.h>	<stdalign.h>	<string.h>
<errno.h>	<stdarg.h>	<tgmath.h>
<fenv.h>	<stdatomic.h>	<threads.h>
<float.h>	<stdbit.h>	<time.h>
<inttypes.h>	<stdbool.h>	<uchar.h>
<iso646.h>	<stdckdint.h>	<wchar.h>
<limits.h>	<stddef.h>	<wctype.h>
<locale.h>	<stdint.h>	
<math.h>	<stdio.h>	

If a file with the same name as one of the preceding entries in < and > delimited sequences, not provided as part of the implementation, is placed in any of the standard places that are searched for included source files, the behavior is undefined.

Standard headers can be included in any order; each can be included more than once in a given scope, with no effect different from being included only once, except that the effect of including <assert.h> depends on the definition of **NDEBUG** (see 7.2). If used, a header shall be included outside of any external declaration or definition, and it shall first be included before the first reference to any of the functions or objects it declares, or to any of the types or macros it defines. However, if an identifier is declared or defined in more than one header, the second and subsequent associated headers can be included after the initial reference to the identifier. The program shall not have any macros with names lexically identical to keywords currently defined prior to the inclusion of the header or when any macro defined in the header is expanded.

Some standard headers define or declare identifiers that had not been present in previous versions of this document. To allow implementations and users to adapt to that situation, they also define a version macro for feature test of the form **__STDC_VERSION__ XXXX_H**, where XXXX is the all-caps spelling of the corresponding header <xxxx.h>. These version macros expand to one of many values detailed in the subclauses of Annex M such as 202311L.

Any definition of an object-like macro described in this clause or Annex F, Annex G, Annex H, or Annex K shall expand to code that is fully protected by parentheses where necessary, so that it groups in an arbitrary expression as if it were a single identifier.

Any declaration of a library function shall have external linkage.

A summary of the contents of the standard headers is given in Annex B.

Forward references: diagnostics (7.2).

7.1.3 Reserved identifiers

Each header declares or defines all identifiers listed in its associated subclause, and optionally declares or defines identifiers listed in its associated future library directions subclause and identifiers which are always reserved either for any use or for use as file scope identifiers.

- All potentially reserved identifiers (including ones listed in the future library directions) that are provided by an implementation with an external definition are reserved for any use. An implementation shall not provide an external definition of a potentially reserved identifier unless that identifier is reserved for a use where it would have external linkage.²¹⁵⁾ All other potentially reserved identifiers that are provided by an implementation (including in the form of a macro) are reserved for any use when the associated header is included. No other potentially reserved identifiers are reserved.²¹⁶⁾
- Each macro name in any of the following subclauses (including the future library directions) is reserved for use as specified if any of its associated headers is included; unless explicitly

²¹⁵⁾All library functions have external linkage.
²¹⁶⁾A potentially reserved identifier becomes a reserved identifier when an implementation begins using it or a future standard reserves it, but is otherwise available for use by the programmer.

stated otherwise (see 7.1.4).

- All identifiers with external linkage in any of the following subclauses (including the future library directions) and **errno** are always reserved for use as identifiers with external linkage.²¹⁷⁾
- Each identifier with file scope listed in any of the following subclauses (including the future library directions) is reserved for use as a macro name and as an identifier with file scope in the same name space if any of its associated headers is included.

7.1.4 Use of library functions

Each of the following statements applies unless explicitly stated otherwise in the detailed descriptions that follow:

- If an argument to a function has an invalid value (such as a value outside the domain of the function, or a pointer outside the address space of the program, or a null pointer, or a pointer to non-modifiable storage when the corresponding parameter is not const-qualified) or a type (after default argument promotion) not expected by a function with a variable number of arguments, the behavior is undefined.
- If a function argument is described as being an array, the pointer passed to the function shall have a value such that all address computations and accesses to objects (that would be valid if the pointer did point to the first element of such an array) are valid.²¹⁸⁾
- Any function declared in a header may be additionally implemented as a function-like macro defined in the header, so if a library function is declared explicitly when its header is included, one of the techniques shown later in the next subclause can be used to ensure the declaration is not affected by such a macro. Any macro definition of a function can be suppressed locally by enclosing the name of the function in parentheses, because the name is then not followed by the left parenthesis that indicates expansion of a macro function name. For the same syntactic reason, it is permitted to take the address of a library function even if it is also defined as a macro.²¹⁹⁾ The use of **#undef** to remove any macro definition will also ensure that an actual function is referred to.
- Any invocation of a library function that is implemented as a macro shall expand to code that evaluates each of its arguments exactly once, fully protected by parentheses where necessary, so it is generally safe to use arbitrary expressions as arguments.²²⁰⁾
- Likewise, those function-like macros described in the following subclauses may be invoked in an expression anywhere a function with a compatible return type could be called.²²¹⁾
- All object-like macros listed as expanding to integer constant expressions shall additionally be suitable for use in conditional expression inclusion preprocessing directives.

²¹⁷⁾The list of reserved identifiers with external linkage includes **math_errhandling**, **setjmp**, **va_copy**, and **va_end**.

²¹⁸⁾This includes, for example, passing a valid pointer that points one-past-the-end of an array along with a size of 0, or using any valid pointer with a size of 0.

²¹⁹⁾This means that an implementation is required to provide an actual function for each library function, even if it also provides a macro for that function.

²²⁰⁾However, such macros can sometimes not contain the sequence points that the corresponding function calls do.

²²¹⁾Because external identifiers and some macro names beginning with an underscore are reserved, implementations can provide special semantics for such names. For example, the identifier **_BUILTIN_abs** can be used to indicate generation of in-line code for the **abs** function. Thus, the appropriate header can specify

```
#define abs(x) _BUILTIN_abs(x)
```

for a compiler whose code generator will accept it.

In this manner, a user desiring to guarantee that a given library function such as **abs** will be a genuine function can write

```
#undef abs
```

whether the implementation's header provides a macro implementation of **abs** or a built-in implementation. The prototype for the function, which precedes and is hidden by any macro definition, is thereby revealed also.

Provided that a library function can be declared without reference to any type defined in a header, it is also permissible to declare the function and use it without including its associated header.

There is a sequence point immediately before a library function returns.

The functions in the standard library are not guaranteed to be reentrant and may modify objects with static or thread storage duration.²²²⁾

Unless explicitly stated otherwise in the detailed descriptions that follow, library functions shall prevent data races as follows: A library function shall not directly or indirectly access objects accessible by threads other than the current thread unless the objects are accessed directly or indirectly via the function’s arguments. A library function shall not directly or indirectly modify objects accessible by threads other than the current thread unless the objects are accessed directly or indirectly via the function’s non-const arguments.²²³⁾ Implementations may share their own internal objects between threads if the objects are not visible to users and are protected against data races.

Unless otherwise specified, library functions shall perform all operations solely within the current thread if those operations have effects that are visible to users.²²⁴⁾

EXAMPLE The function `atoi` can be used in any of several ways:

- by use of its associated header (possibly generating a macro expansion)

```
#include <stdlib.h>
const char *str;
/* ... */
i = atoi(str);
```

- by use of its associated header (assuredly generating a true function reference)

```
#include <stdlib.h>
#undef atoi
const char *str;
/* ... */
i = atoi(str);
```

or

```
#include <stdlib.h>
const char *str;
/* ... */
i = (atoi)(str);
```

- by explicit declaration

```
extern int atoi(const char *);
const char *str;
/* ... */
i = atoi(str);
```

²²²⁾Thus, a signal handler cannot, in general, call standard library functions.
²²³⁾This means, for example, that an implementation is not permitted to use a `static` object for internal purposes without synchronization because it can cause a data race even in programs that do not explicitly share objects between threads. Similarly, an implementation of `memcpy` is not permitted to copy bytes beyond the specified length of the destination object and then restore the original values because it can cause a data race if the program shared those bytes between threads.
²²⁴⁾This allows implementations to parallelize operations if there are no visible side effects.

7.2 Diagnostics <assert.h>

7.2.1 General

The header <assert.h> defines the **assert** and `__STDC_VERSION_ASSERT_H__` macros and refers to another macro,

```
NDEBUG
```

which is *not* defined by <assert.h>. If **NDEBUG** is defined as a macro name at the point in the source file where <assert.h> is included, the **assert** macro is defined simply as

```
#define assert(...) ((void)0)
```

The **assert** macro is redefined according to the current state of **NDEBUG** each time that <assert.h> is included.

The **assert** macro shall be implemented as a macro with an ellipsis parameter, not as an actual function. If the macro definition is suppressed to access an actual function, the behavior is undefined.

NOTE Nevertheless, when **NDEBUG** is not defined, the macro acts as a function taking one parameter as indicated by the prototype as given later in this subclause (7.2.2.1). For both **assert()** and **assert(1, 1)**, the number of arguments does not agree with the number of parameters. A diagnostic is required by 6.5.3.3.

The macro

```
__STDC_VERSION_ASSERT_H__
```

is an integer constant expression with a value equivalent to 202311L.

7.2.2 Program diagnostics

7.2.2.1 The assert macro

Synopsis

```
#include <assert.h>
void assert(scalar expression);
```

Description

The **assert** macro puts diagnostic tests into programs; it expands to a void expression. When it is executed, if **expression** (which shall have a scalar type) is false (that is, compares equal to 0), the **assert** macro writes information about the particular invocation that failed (including the text of the argument, the name of the source file, the source line number, and the name of the enclosing function — the latter are respectively the values of the preprocessing macros `__FILE__` and `__LINE__` and of the identifier `__func__`) on the standard error stream in an implementation-defined format.²²⁵⁾

It then calls the **abort** function.

Returns

The **assert** macro returns no value.

Forward references: the **abort** function (7.24.5.1).

²²⁵⁾ The message written can be of the form:

```
Assertion failed: expression, function abc, file xyz, line nnn.
```

7.3 Complex arithmetic <complex.h>

7.3.1 Introduction

The header <complex.h> defines macros and declares functions that support complex arithmetic.²²⁶⁾ Implementations that define the macro `__STDC_NO_COMPLEX__` may not provide this header nor support any of its facilities.

The macro

```
__STDC_VERSION_COMPLEX_H__
```

is an integer constant expression with a value equivalent to 202311L.

Each synopsis, other than for the **CMPLX** macros, specifies a family of functions consisting of a principal function with one or more **double complex** parameters and a **double complex** or **double** return value; and other functions with the same name but with **f** and **l** suffixes which are corresponding functions with **float** and **long double** parameters and return values.

The macro

```
complex
```

expands to `_Complex`; the macro

```
_Complex_I
```

expands to an arithmetic constant expression of type **float** `_Complex`, with the value of the imaginary unit.²²⁷⁾

The macros

```
imaginary
```

and

```
_Imaginary_I
```

are defined if and only if the implementation supports imaginary types;²²⁸⁾ and, if defined, they expand to `_Imaginary` and an arithmetic constant expression of type **float** `_Imaginary` with the value of the imaginary unit.

The macro

```
I
```

expands to either `_Imaginary_I` or `_Complex_I`. If `_Imaginary_I` is not defined, **I** shall expand to `_Complex_I`.

Notwithstanding the provisions of 7.1.3, a program can undefine and perhaps then redefine the macros **complex**, **imaginary**, and **I**.

Forward references: the **CMPLX** macros (7.3.9.3), ISO/IEC 60559-compatible complex arithmetic (Annex G).

7.3.2 Conventions

Values are interpreted as radians, not degrees. An implementation may set **errno** but is not required to do so.

²²⁶⁾See “future library directions” (7.33.2).
²²⁷⁾The imaginary unit is a number *i* such that *i*² = −1.
²²⁸⁾A specification for imaginary types is in Annex G.

7.3.3 Branch cuts

Some of the following functions have branch cuts, across which the function is discontinuous. For implementations with a signed zero (including all ISO/IEC 60559 implementations) that follow the specifications of Annex G, the sign of zero distinguishes one side of a cut from another so that the function is continuous (except for format limitations) as the cut is approached from either side. For example, for the square root function, which has a branch cut along the negative real axis, the top of the cut, with imaginary part $+0$, maps to the positive imaginary axis, and the bottom of the cut, with imaginary part -0 , maps to the negative imaginary axis.

Implementations that do not support a signed zero (see Annex F) cannot distinguish the sides of branch cuts. These implementations shall map a cut so that the function is continuous as the cut is approached coming around the finite endpoint of the cut in a counter clockwise direction. (Branch cuts for the functions specified here have just one finite endpoint.) For example, in the square root function, coming counter clockwise around the finite endpoint of the cut along the negative real axis approaches the cut from above, so that the cut maps to the positive imaginary axis.

7.3.4 The `CX_LIMITED_RANGE` pragma

Synopsis

```
#include <complex.h>
#pragma STDC CX_LIMITED_RANGE on-off-switch
```

Description

The usual mathematical formulas for complex multiply, divide, and absolute value are problematic because of their treatment of infinities and because of undue overflow and underflow. The `CX_LIMITED_RANGE` pragma can be used to inform the implementation that (where the state is “on”) the usual mathematical formulas are acceptable.²²⁹⁾ The pragma can occur either outside external declarations or preceding all explicit declarations and statements inside a compound statement. When outside external declarations, the pragma takes effect from its occurrence until another `CX_LIMITED_RANGE` pragma is encountered, or until the end of the translation unit. When inside a compound statement, the pragma takes effect from its occurrence until another `CX_LIMITED_RANGE` pragma is encountered (including within a nested compound statement), or until the end of the compound statement; at the end of a compound statement the state for the pragma is restored to its condition just before the compound statement. If this pragma is used in any other context, the behavior is undefined. The default state for the pragma is “off”.

7.3.5 Trigonometric functions

7.3.5.1 The `cacos` functions

Synopsis

```
#include <complex.h>
double complex cacos(double complex z);
float complex cacosf(float complex z);
long double complex cacosl(long double complex z);
```

Description

The `cacos` functions compute the complex arc cosine of z , with branch cuts outside the interval $[-1, +1]$ along the real axis.

²²⁹⁾ The purpose of the pragma is to allow the implementation to use the formulas:

$$\begin{aligned}(x + iy) \times (u + iv) &= (xu - yv) + i(yu + xv) \\ (x + iy) / (u + iv) &= [(xu + yv) + i(yu - xv)] / (u^2 + v^2) \\ |x + iy| &= \sqrt{x^2 + y^2}\end{aligned}$$

where the programmer can determine they are safe.

Returns

The **ccos** functions return the complex arc cosine value, in the range of a strip mathematically unbounded along the imaginary axis and in the interval $[0, \pi]$ along the real axis.

7.3.5.2 The casin functions

Synopsis

```
#include <complex.h>
double complex casin(double complex z);
float complex casinf(float complex z);
long double complex casinl(long double complex z);
```

Description

The **casin** functions compute the complex arc sine of **z**, with branch cuts outside the interval $[-1, +1]$ along the real axis.

Returns

The **casin** functions return the complex arc sine value, in the range of a strip mathematically unbounded along the imaginary axis and in the interval $[-\frac{\pi}{2}, +\frac{\pi}{2}]$ along the real axis.

7.3.5.3 The catan functions

Synopsis

```
#include <complex.h>
double complex catan(double complex z);
float complex catanf(float complex z);
long double complex catanl(long double complex z);
```

Description

The **catan** functions compute the complex arc tangent of **z**, with branch cuts outside the interval $[-i, +i]$ along the imaginary axis.

Returns

The **catan** functions return the complex arc tangent value, in the range of a strip mathematically unbounded along the imaginary axis and in the interval $[-\frac{\pi}{2}, +\frac{\pi}{2}]$ along the real axis.

7.3.5.4 The ccos functions

Synopsis

```
#include <complex.h>
double complex ccos(double complex z);
float complex ccosf(float complex z);
long double complex ccosl(long double complex z);
```

Description

The **ccos** functions compute the complex cosine of **z**.

Returns

The **ccos** functions return the complex cosine value.

7.3.5.5 The csin functions

Synopsis

```
#include <complex.h>
double complex csin(double complex z);
float complex csinf(float complex z);
long double complex csinl(long double complex z);
```


Description

The **csin** functions compute the complex sine of **z**.

Returns

The **csin** functions return the complex sine value.

7.3.5.6 The ctan functions**Synopsis**

```
#include <complex.h>
double complex ctan(double complex z);
float complex ctanf(float complex z);
long double complex ctanl(long double complex z);
```

Description

The **ctan** functions compute the complex tangent of **z**.

Returns

The **ctan** functions return the complex tangent value.

7.3.6 Hyperbolic functions**7.3.6.1 The cacosh functions****Synopsis**

```
#include <complex.h>
double complex cacosh(double complex z);
float complex cacoshf(float complex z);
long double complex cacoshl(long double complex z);
```

Description

The **cacosh** functions compute the complex arc hyperbolic cosine of **z**, with a branch cut at values less than 1 along the real axis.

Returns

The **cacosh** functions return the complex arc hyperbolic cosine value, in the range of a half-strip of nonnegative values along the real axis and in the interval $[-i\pi, +i\pi]$ along the imaginary axis.

7.3.6.2 The casinh functions**Synopsis**

```
#include <complex.h>
double complex casinh(double complex z);
float complex casinhf(float complex z);
long double complex casinhl(long double complex z);
```

Description

The **casinh** functions compute the complex arc hyperbolic sine of **z**, with branch cuts outside the interval $[-i, +i]$ along the imaginary axis.

Returns

The **casinh** functions return the complex arc hyperbolic sine value, in the range of a strip mathematically unbounded along the real axis and in the interval $[-\frac{i\pi}{2}, +\frac{i\pi}{2}]$ along the imaginary axis.

7.3.6.3 The catanh functions**Synopsis**

```
#include <complex.h>
```

```
double complex catanh(double complex z);
float complex catanhf(float complex z);
long double complex catanhl(long double complex z);
```

Description

The **catanh** functions compute the complex arc hyperbolic tangent of **z**, with branch cuts outside the interval $[-1, +1]$ along the real axis.

Returns

The **catanh** functions return the complex arc hyperbolic tangent value, in the range of a strip mathematically unbounded along the real axis and in the interval $[-\frac{i\pi}{2}, +\frac{i\pi}{2}]$ along the imaginary axis.

7.3.6.4 The ccosh functions

Synopsis

```
#include <complex.h>
double complex ccosh(double complex z);
float complex ccoshf(float complex z);
long double complex ccoshl(long double complex z);
```

Description

The **ccosh** functions compute the complex hyperbolic cosine of **z**.

Returns

The **ccosh** functions return the complex hyperbolic cosine value.

7.3.6.5 The csinh functions

Synopsis

```
#include <complex.h>
double complex csinh(double complex z);
float complex csinhf(float complex z);
long double complex csinhl(long double complex z);
```

Description

The **csinh** functions compute the complex hyperbolic sine of **z**.

Returns

The **csinh** functions return the complex hyperbolic sine value.

7.3.6.6 The ctanh functions

Synopsis

```
#include <complex.h>
double complex ctanh(double complex z);
float complex ctanhf(float complex z);
long double complex ctanhl(long double complex z);
```

Description

The **ctanh** functions compute the complex hyperbolic tangent of **z**.

Returns

The **ctanh** functions return the complex hyperbolic tangent value.

7.3.7 Exponential and logarithmic functions

7.3.7.1 The cexp functions

Synopsis

```
#include <complex.h>
double complex cexp(double complex z);
float complex cexpf(float complex z);
long double complex cexpl(long double complex z);
```

Description

The **cexp** functions compute the complex base- e exponential of **z**.

Returns

The **cexp** functions return the complex base- e exponential value.

7.3.7.2 The clog functions

Synopsis

```
#include <complex.h>
double complex clog(double complex z);
float complex clogf(float complex z);
long double complex clogl(long double complex z);
```

Description

The **clog** functions compute the complex natural (base- e) logarithm of **z**, with a branch cut along the negative real axis.

Returns

The **clog** functions return the complex natural logarithm value, in the range of a strip mathematically unbounded along the real axis and in the interval $[-i\pi, +i\pi]$ along the imaginary axis.

7.3.8 Power and absolute-value functions

7.3.8.1 The cabs functions

Synopsis

```
#include <complex.h>
double cabs(double complex z);
float cabsf(float complex z);
long double cabsl(long double complex z);
```

Description

The **cabs** functions compute the complex absolute value (also called norm, modulus, or magnitude) of **z**.

Returns

The **cabs** functions return the complex absolute value.

7.3.8.2 The cpow functions

Synopsis

```
#include <complex.h>
double complex cpow(double complex x, double complex y);
float complex cpowf(float complex x, float complex y);
long double complex cpowl(long double complex x, long double complex y);
```

Description

The **cpow** functions compute the complex power function x^y , with a branch cut for the first parameter along the negative real axis.

Returns

The **cpow** functions return the complex power function value.

7.3.8.3 The **csqrt** functions

Synopsis

```
#include <complex.h>
double complex csqrt(double complex z);
float complex csqrtf(float complex z);
long double complex csqrtl(long double complex z);
```

Description

The **csqrt** functions compute the complex square root of **z**, with a branch cut along the negative real axis.

Returns

The **csqrt** functions return the complex square root value, in the range of the right half-plane (including the imaginary axis).

7.3.9 Manipulation functions

7.3.9.1 The **carg** functions

Synopsis

```
#include <complex.h>
double carg(double complex z);
float cargf(float complex z);
long double cargl(long double complex z);
```

Description

The **carg** functions compute the argument (also called phase (which is an angle)) of **z**, with a branch cut along the negative real axis.

Returns

The **carg** functions return the value of the argument in the interval $[-\pi, +\pi]$.

7.3.9.2 The **cimag** functions

Synopsis

```
#include <complex.h>
double cimag(double complex z);
float cimagf(float complex z);
long double cimagl(long double complex z);
```

Description

The **cimag** functions compute the imaginary part of **z**.²³⁰⁾

Returns

The **cimag** functions return the imaginary part value (as a real).

7.3.9.3 The **CMPLX** macros

Synopsis

```
#include <complex.h>
double complex CMPLX(double x, double y);
float complex CMPLXF(float x, float y);
```

²³⁰⁾For a complex variable **z**, **z** and **CMPLX(creal(z), cimag(z))** are equivalent expressions. If imaginary types are supported, **z** and **creal(z)+cimag(z)*I** are equivalent expressions.

```
long double complex CMPLXL(long double x, long double y);
```

Description

The **CMPLX** macros expand to an expression of the specified complex type, with the real part having the (converted) value of **x** and the imaginary part having the (converted) value of **y**. The resulting expression shall be suitable for use as an initializer for an object with static or thread storage duration, provided both arguments are likewise suitable. The resulting expression shall be an arithmetic constant expression, provided both arguments are arithmetic constant expressions.

Returns

The **CMPLX** macros return the complex value $x + iy$.

NOTE These macros act as if the implementation supported imaginary types and the definitions were:

```
#define CMPLX(x, y) ((double complex)((double)(x) + \
    _Imaginary_I * (double)(y)))
#define CMPLXF(x, y) ((float complex)((float)(x) + \
    _Imaginary_I * (float)(y)))
#define CMPLXL(x, y) ((long double complex)((long double)(x) + \
    _Imaginary_I * (long double)(y)))
```

7.3.9.4 The conj functions

Synopsis

```
#include <complex.h>
double complex conj(double complex z);
float complex conjf(float complex z);
long double complex conjl(long double complex z);
```

Description

The **conj** functions compute the complex conjugate of **z**, by arithmetically negating its imaginary part.

Returns

The **conj** functions return the complex conjugate value.

7.3.9.5 The cproj functions

Synopsis

```
#include <complex.h>
double complex cproj(double complex z);
float complex cprojf(float complex z);
long double complex cprojl(long double complex z);
```

Description

The **cproj** functions compute a projection of **z** onto the Riemann sphere where **z** projects to **z** except that all complex infinities (even those with one infinite part and one NaN part) project to positive infinity on the real axis. If **z** has an infinite part, then **cproj(z)** is equivalent to

```
INFINITY + I * copysign(0.0, cimag(z))
```

Returns

The **cproj** functions return the value of the projection onto the Riemann sphere.

7.3.9.6 The creal functions

Synopsis

```
#include <complex.h>
double creal(double complex z);
float crealf(float complex z);
long double creall(long double complex z);
```

Description

The **creal** functions compute the real part of **z**.²³¹⁾

Returns

The **creal** functions return the real part value.

²³¹⁾For a complex variable **z**, **z** and **CMPLX(creal(z), cimag(z))** are equivalent expressions. If imaginary types are supported, **z** and **creal(z)+cimag(z)*I** are equivalent expressions.

7.4 Character handling <ctype.h>

7.4.1 General

The header <ctype.h> declares several functions useful for classifying and mapping characters.²³²⁾ In all cases the argument is an **int**, the value of which shall be representable as an **unsigned char** or shall equal the value of the macro **E0F**. If the argument has any other value, the behavior is undefined.

The behavior of these functions is affected by the current locale. Those functions that have locale-specific aspects only when not in the "C" locale are noted subsequently in this subclause.

The term *printing character* refers to a member of a locale-specific set of characters, each of which occupies one printing position on a display device; the term *control character* refers to a member of a locale-specific set of characters that are not printing characters.²³³⁾ All letters and digits are printing characters.

Forward references:

E0F (7.23.1), localization (7.11).

7.4.2 Character classification functions

7.4.2.1 General

The functions in this subclause return nonzero (true) if and only if the value of the argument **c** conforms to that in the description of the function.

7.4.2.2 The **isalnum** function

Synopsis

```
#include <ctype.h>
int isalnum(int c);
```

Description

The **isalnum** function tests for any character for which **isalpha** or **isdigit** is true.

7.4.2.3 The **isalpha** function

Synopsis

```
#include <ctype.h>
int isalpha(int c);
```

Description

The **isalpha** function tests for any character for which **isupper** or **islower** is true, or any character that is one of a locale-specific set of alphabetic characters for which none of **iscntrl**, **isdigit**, **ispunct**, or **isspace** is true.²³⁴⁾ In the "C" locale, **isalpha** returns true only for the characters for which **isupper** or **islower** is true.

7.4.2.4 The **isblank** function

Synopsis

```
#include <ctype.h>
int isblank(int c);
```

²³²⁾See "future library directions" (7.33.3).

²³³⁾In an implementation that uses the seven-bit US ASCII character set, the printing characters are those whose values lie from 0x20 (space) through 0x7E (tilde); the control characters are those whose values lie from 0 (NUL) through 0x1F (US), and the character 0x7F (DEL).

²³⁴⁾The functions **islower** and **isupper** test true or false separately for each of these additional characters; all four combinations are possible.

Description

The **isblank** function tests for any character that is a standard blank character or is one of a locale-specific set of characters for which **isspace** is true and that is used to separate words within a line of text. The standard blank characters are the following: space (' '), and horizontal tab ('\t'). In the "C" locale, **isblank** returns true only for the standard blank characters.

7.4.2.5 The **isctrl** function

Synopsis

```
#include <ctype.h>
int isctrl(int c);
```

Description

The **isctrl** function tests for any control character.

7.4.2.6 The **isdigit** function

Synopsis

```
#include <ctype.h>
int isdigit(int c);
```

Description

The **isdigit** function tests for any decimal-digit character (as defined in 5.3.1).

7.4.2.7 The **isgraph** function

Synopsis

```
#include <ctype.h>
int isgraph(int c);
```

Description

The **isgraph** function tests for any printing character except space (' ').

7.4.2.8 The **islower** function

Synopsis

```
#include <ctype.h>
int islower(int c);
```

Description

The **islower** function tests for any character that is a lowercase letter or is one of a locale-specific set of characters for which none of **isctrl**, **isdigit**, **ispunct**, or **isspace** is true. In the "C" locale, **islower** returns true only for the lowercase letters (as defined in 5.3.1).

7.4.2.9 The **isprint** function

Synopsis

```
#include <ctype.h>
int isprint(int c);
```

Description

The **isprint** function tests for any printing character including space (' ').

7.4.2.10 The **ispunct** function

Synopsis

```
#include <ctype.h>
int ispunct(int c);
```

Description

The **ispunct** function tests for any printing character that is one of a locale-specific set of punctuation characters for which neither **isspace** nor **isalnum** is true. In the "C" locale, **ispunct** returns true for every printing character for which neither **isspace** nor **isalnum** is true.

7.4.2.11 The **isspace** function

Synopsis

```
#include <ctype.h>
int isspace(int c);
```

Description

The **isspace** function tests for any character that is a standard white-space character or is one of a locale-specific set of characters for which **isalnum** is false. The standard white-space characters are the following: space (' '), form feed ('\f'), new-line ('\n'), carriage return ('\r'), horizontal tab ('\t'), and vertical tab ('\v'). In the "C" locale, **isspace** returns true only for the standard white-space characters.

7.4.2.12 The **isupper** function

Synopsis

```
#include <ctype.h>
int isupper(int c);
```

Description

The **isupper** function tests for any character that is an uppercase letter or is one of a locale-specific set of characters for which none of **iscntrl**, **isdigit**, **ispunct**, or **isspace** is true. In the "C" locale, **isupper** returns true only for the uppercase letters (as defined in 5.3.1).

7.4.2.13 The **isxdigit** function

Synopsis

```
#include <ctype.h>
int isxdigit(int c);
```

Description

The **isxdigit** function tests for any hexadecimal-digit character (as defined in 6.4.5.2).

7.4.3 Character case mapping functions

7.4.3.1 The **tolower** function

Synopsis

```
#include <ctype.h>
int tolower(int c);
```

Description

The **tolower** function converts an uppercase letter to a corresponding lowercase letter.

Returns

If the argument is a character for which **isupper** is true and there are one or more corresponding characters, as specified by the current locale, for which **islower** is true, the **tolower** function returns one of the corresponding characters (always the same one for any given locale); otherwise, the argument is returned unchanged.

7.4.3.2 The **toupper** function

Synopsis

```
#include <ctype.h>
int toupper(int c);
```

Description

The **toupper** function converts a lowercase letter to a corresponding uppercase letter.

Returns

If the argument is a character for which **islower** is true and there are one or more corresponding characters, as specified by the current locale, for which **isupper** is true, the **toupper** function returns one of the corresponding characters (always the same one for any given locale); otherwise, the argument is returned unchanged.

7.5 Errors <errno.h>

The header <errno.h> defines several macros, all relating to the reporting of error conditions.
The macros are

EDOM
EILSEQ
ERANGE

which expand to integer constant expressions with type **int**, distinct positive values, and which are suitable for use in conditional expression inclusion preprocessing directives; and

errno

which expands to a modifiable lvalue²³⁵⁾ that has type **int** and thread storage duration, the value of which is set to a positive error number by several library functions. If a macro definition is suppressed to access an actual object, or a program defines an identifier with the name **errno**, the behavior is undefined.

The value of **errno** in the initial thread is zero at program startup (the initial representation of the object designated by **errno** in other threads is indeterminate), but is never set to zero by any library function.²³⁶⁾ The value of **errno** may be set to nonzero by a library function call whether or not there is an error, provided the use of **errno** is not documented in the description of the function in this document.

Additional macro definitions, beginning with **E** and a digit or **E** and an uppercase letter,²³⁷⁾ may also be specified by the implementation.

²³⁵⁾The macro **errno** is not expected to be the identifier of an object. Expansion to a modifiable lvalue resulting from a function call (for example, **(*errno())**) is a viable implementation strategy.

²³⁶⁾Thus, a program that uses **errno** for error checking would set it to zero before a library function call, then inspect it before a subsequent library function call. Of course, a library function can save the value of **errno** on entry and then set it to zero, as long as the original value is restored if **errno**'s value is still zero just before the return.

²³⁷⁾See "future library directions" (7.33.4).

7.6 Floating-point environment <fenv.h>

7.6.1 General

The header <fenv.h> defines several macros, and declares types and functions that provide access to the floating-point environment. The *floating-point environment* refers collectively to any floating-point status flags and control modes supported by the implementation.²³⁸⁾

A *floating-point status flag* is a system variable whose value is set (but never cleared) when a *floating-point exception* is raised, which occurs as a side effect of exceptional floating-point arithmetic to provide auxiliary information.²³⁹⁾ A *floating-point control mode* is a system variable whose value may be set by the user to affect the subsequent behavior of floating-point arithmetic.

A floating-point control mode may be *constant* (7.6.3) or *dynamic*. The *dynamic floating-point environment* includes the dynamic floating-point control modes and the floating-point status flags.

The dynamic floating-point environment has thread storage duration. The initial state for a thread’s dynamic floating-point environment is the current state of the dynamic floating-point environment of the thread that creates it. It is initialized at the time of the thread’s creation.

Certain programming conventions support the intended model of use for the dynamic floating-point environment:²⁴⁰⁾

- a function call does not alter its caller’s floating-point control modes, clear its caller’s floating-point status flags, nor depend on the state of its caller’s floating-point status flags unless the function is so documented;
- a function call is assumed to require default floating-point control modes, unless its documentation promises otherwise;
- a function call is assumed to have the potential for raising floating-point exceptions, unless its documentation promises otherwise.

The feature test macro `__STDC_VERSION_FENV_H__` expands to the token 202311L.

The type

```
fenv_t
```

represents the entire dynamic floating-point environment.

The type

```
femode_t
```

represents the collection of dynamic floating-point control modes supported by the implementation, including the dynamic rounding direction mode.

The type

```
fexcept_t
```

represents the floating-point status flags collectively, including any status the implementation associates with the flags.

Each of the macros

²³⁸⁾This header is designed to support the floating-point exception status flags and rounding-direction control modes required by ISO/IEC 60559, and other similar floating-point state information. It is also designed to facilitate code portability among all systems.

²³⁹⁾A floating-point status flag is not an object and can be set more than once within an expression.

²⁴⁰⁾With these conventions, a programmer can safely assume default floating-point control modes (or be unaware of them). The responsibilities associated with accessing the floating-point environment fall on the programmer or program that does so explicitly.

```
FE_DIVBYZERO
FE_INEXACT
FE_INVALID
FE_OVERFLOW
FE_UNDERFLOW
```

is defined if and only if the implementation supports the floating-point exception by means of the functions in 7.6.5.²⁴¹⁾ Additional implementation-defined floating-point exceptions, with macro definitions beginning with **FE_** and an uppercase letter,²⁴²⁾ may also be specified by the implementation. The defined macros expand to integer constant expressions with values such that bitwise ORs of all combinations of the macros result in distinct values, and furthermore, bitwise ANDs of all combinations of the macros result in zero.²⁴³⁾

Decimal floating-point operations and ISO/IEC 60559 binary floating-point operations (Annex F) access the same floating-point exception status flags.

The macro

```
FE_DFL_MODE
```

represents the default state for the collection of dynamic floating-point control modes supported by the implementation – and has type “pointer to const-qualified **femode_t**”. Additional implementation-defined states for the dynamic mode collection, with macro definitions beginning with **FE_** and an uppercase letter, and having type “pointer to const-qualified **femode_t**”, may also be specified by the implementation.

The macro

```
FE_ALL_EXCEPT
```

is the bitwise OR of all floating-point exception macros defined by the implementation. If no such macros are defined, **FE_ALL_EXCEPT** shall be defined as **0**.

Each of the macros

```
FE_DOWNWARD
FE_TONEAREST
FE_TONEARESTFROMZERO
FE_TOWARDZERO
FE_UPWARD
```

is defined if and only if the implementation supports getting and setting the represented rounding direction by means of the **fegetround** and **fesetround** functions. The defined macros expand to integer constant expressions whose values are distinct nonnegative values. Additional implementation-defined rounding directions, with macro definitions beginning with **FE_** and an uppercase letter,²⁴⁴⁾ may also be specified by the implementation.²⁴⁵⁾

If the implementation supports decimal floating-point types, each of the macros

```
FE_DEC_DOWNWARD
FE_DEC_TONEAREST
FE_DEC_TONEARESTFROMZERO
FE_DEC_TOWARDZERO
```

²⁴¹⁾The implementation supports a floating-point exception if there are circumstances where a call to at least one of the functions in 7.6.5, using the macro as the appropriate argument, will succeed. It is not necessary for all the functions to succeed all the time.

²⁴²⁾See “future library directions” (7.33.5).

²⁴³⁾The macros are typically distinct powers of two.

²⁴⁴⁾See “future library directions” (7.33.5).

²⁴⁵⁾Even though the rounding direction macros can expand to constants corresponding to the values of **FLT_ROUNDS**, they are not required to do so.

FE_DEC_UPWARD

is defined for use with the `fe_dec_getround` and `fe_dec_setround` functions for getting and setting the dynamic rounding direction mode for decimal floating-point operations. The decimal rounding direction affects all (inexact) operations that produce a result of decimal floating type and all operations that produce an integer or character sequence result and have an operand of decimal floating type, unless stated otherwise. The macros expand to integer constant expressions whose values are distinct nonnegative values.

During translation, constant rounding direction modes for decimal floating-point arithmetic are in effect where specified. Elsewhere, during translation the decimal rounding direction mode is **FE_DEC_TONEAREST**.

At program startup the dynamic rounding direction mode for decimal floating-point arithmetic is initialized to **FE_DEC_TONEAREST**.

The macro

FE_DFL_ENV

represents the default dynamic floating-point environment — the one installed at program startup — and has type “pointer to const-qualified `fenv_t`”. It can be used as an argument to `<fenv.h>` functions that manage the dynamic floating-point environment.

Additional implementation-defined environments, with macro definitions beginning with **FE_** and an uppercase letter,²⁴⁶⁾ and having type “pointer to const-qualified `fenv_t`”, may also be specified by the implementation.

7.6.2 The FENV_ACCESS pragma

Synopsis

```
#include <fenv.h>
#pragma STDC FENV_ACCESS on-off-switch
```

Description

The **FENV_ACCESS** pragma provides a means to inform the implementation when a program can access the floating-point environment to test floating-point status flags or run under non-default floating-point control modes.²⁴⁷⁾ The pragma shall occur either outside external declarations or preceding all explicit declarations and statements inside a compound statement. When outside external declarations, the pragma takes effect from its occurrence until another **FENV_ACCESS** pragma is encountered, or until the end of the translation unit. When inside a compound statement, the pragma takes effect from its occurrence until another **FENV_ACCESS** pragma is encountered (including within a nested compound statement), or until the end of the compound statement. At the end of a compound statement, the state for the pragma is restored to its condition just before the compound statement. If this pragma is used in any other context, the behavior is undefined. If part of a program tests floating-point status flags or establishes or is executed with non-default floating-point mode settings using any means other than the **FENV_ROUND** pragmas, but was translated with the state for the **FENV_ACCESS** pragma “off”, the behavior is undefined. The default state (“on” or “off”) for the pragma is implementation-defined. (When execution passes from a part of the program translated with **FENV_ACCESS** “off” to a part translated with **FENV_ACCESS** “on”, the state of the floating-point status flags is unspecified and the floating-point control modes have their default settings.)

EXAMPLE

```
#include <fenv.h>
```

²⁴⁶⁾See “future library directions” (7.33.5).
²⁴⁷⁾The purpose of the **FENV_ACCESS** pragma is to allow certain optimizations that can subvert flag tests and mode changes (e.g. global common subexpression elimination, code motion, and constant folding). In general, if the state of **FENV_ACCESS** is “off”, the translator can assume that the flags are not tested, and that default modes are in effect, except where specified otherwise by an **FENV_ROUND** pragma.


```

void f(double x)
{
    #pragma STDC FENV_ACCESS ON
    void g(double);
    void h(double);
    /* ... */
    g(x + 1);
    h(x + 1);
    /* ... */
}

```

If the function **g** can depend on status flags set as a side effect of the first **x + 1**, or if the second **x + 1** can depend on control modes set as a side effect of the call to function **g**, then the program has to contain an appropriately placed invocation of **#pragma STDC FENV_ACCESS ON** as shown.²⁴⁸⁾

7.6.3 The FENV_ROUND pragma

Synopsis

```

#include <fenv.h>
#pragma STDC FENV_ROUND direction
#pragma STDC FENV_ROUND FE_DYNAMIC

```

Description

The **FENV_ROUND** pragma provides a means to specify a constant rounding direction for floating-point operations for standard floating types within a translation unit or compound statement. The pragma shall occur either outside external declarations or before all explicit declarations and statements inside a compound statement. When outside external declarations, the pragma takes effect from its occurrence until another **FENV_ROUND** pragma is encountered, or until the end of the translation unit. When inside a compound statement, the pragma takes effect from its occurrence until another **FENV_ROUND** pragma is encountered (including within a nested compound statement), or until the end of the compound statement; at the end of a compound statement the static rounding mode is restored to its condition just before the compound statement. If this pragma is used in any other context, its behavior is undefined.

direction shall be: one of the names of the supported rounding direction macros for operations for standard floating types (7.6), to specify a constant rounding mode; or, **FE_DYNAMIC**, to specify dynamic rounding. If any other value is specified, the behavior is undefined. If no **FENV_ROUND** pragma is in effect, or the specified *direction* is **FE_DYNAMIC**, rounding is according to the mode specified by the dynamic floating-point environment, which is the dynamic rounding mode that was established either at thread creation or by a call to **fesetround**, **fesetmode**, **fesetenv**, or **feupdateenv**. If the *direction* **FE_DYNAMIC** is specified and **FENV_ACCESS** is “off”, the translator may assume that the default rounding mode is in effect.

The **FENV_ROUND** pragma affects operations for standard floating types. Within the scope of an **FENV_ROUND** pragma establishing a constant rounding mode, floating-point operators, implicit conversions (including the conversion of a value represented in a format wider than its semantic types to its semantic type, as done by classification macros), and invocations of functions indicated in Table 7.1, for which macro replacement has not been suppressed (7.1.4), shall be evaluated according to the specified constant rounding mode (as though no constant mode was specified and the corresponding dynamic rounding mode had been established by a call to **fesetround**). Invocations of functions for which macro replacement has been suppressed and invocations of functions other than those indicated in Table 7.1 shall not be affected by constant rounding modes – they are affected by (and affect) only the dynamic mode. Floating constants (6.4.5.3) of a standard floating type that occur in the scope of a constant rounding mode shall be interpreted according to that mode.

²⁴⁸⁾The side effects impose a temporal ordering that requires two evaluations of **x + 1**. On the other hand, without the **#pragma STDC FENV_ACCESS ON** pragma, and assuming the default state is “off”, just one evaluation of **x + 1** would suffice.

Table 7.1 — Functions affected by constant rounding modes – for standard floating types

Header	Function families
<math.h>	acos, acospi, asin, asinpi, atan, atan2, atan2pi, atanpi
<math.h>	cos, cospi, sin, sinpi, tan, tanpi
<math.h>	acosh, asinh, atanh
<math.h>	cosh, sinh, tanh
<math.h>	exp, exp10, exp10m1, exp2, exp2m1, expm1
<math.h>	log, log10, log10p1, log1p, log2, log2p1, logp1
<math.h>	scalbn, scalbln, ldexp
<math.h>	cbrt, compoundn, hypot, pow, pown, powr, rootn, rsqrt, sqrt
<math.h>	erf, erfc
<math.h>	lgamma, tgamma
<math.h>	rint, nearbyint, lrint, llrint
<math.h>	fdim
<math.h>	fma
<math.h>	fadd, dadd, fsub, dsub, fmul, dmul, fdiv, ddiv, fma, dfma, fsqrt, dsqrt
<stdlib.h>	atof, strfrom, strt0
<wchar.h>	wcsto
<stdio.h>	printf and scanf families
<wchar.h>	wprintf and wscanf families

A function family listed in Table 7.1 indicates the functions for all standard floating types, where the function family is represented by the name of the functions without a suffix. For example, **acos** indicates the functions **acos**, **acosf**, and **acosl**.

NOTE Constant rounding modes can be implemented using dynamic rounding modes as illustrated in the following example, except that this method does not interpret inexact floating constants according to the constant rounding mode as required.

```
{
    #pragma STDC FENV_ROUND direction
    // compiler inserts:
    // #pragma STDC FENV_ACCESS ON
    // int __savedrnd;
    // __savedrnd = __swapround(direction);
    ... operations affected by constant rounding mode ...
    // compiler inserts:
    // __savedrnd = __swapround(__savedrnd);
    ... operations not affected by constant rounding mode ...
    // compiler inserts:
    // __savedrnd = __swapround(__savedrnd);
    ... operations affected by constant rounding mode ...
    // compiler inserts:
    // __swapround(__savedrnd);
}
```

```
}

```

where `__swapround` is defined by:

```
static inline int __swapround(const int new) {
    const int old = fegetround();
    fesetround(new);
    return old;
}
```

7.6.4 The FENV_DEC_ROUND pragma

Synopsis

```
#include <fenv.h>
#ifdef __STDC_IEC_60559_DFP__
#pragma STDC FENV_DEC_ROUND dec-direction
#endif
```

Description

The **FENV_DEC_ROUND** pragma is a decimal floating-point analog of the **FENV_ROUND** pragma. If **FLT_RADIX** is not 10, the **FENV_DEC_ROUND** pragma affects operators, functions, and floating constants only for decimal floating types. The affected functions are listed in Table 7.2.

If **FLT_RADIX** is 10, whether the **FENV_ROUND** and **FENV_DEC_ROUND** pragmas alter the rounding direction of both standard and decimal floating-point operations is implementation-defined. *dec-direction* shall be one of the decimal rounding direction macro names (**FE_DEC_DOWNWARD**, **FE_DEC_TONEAREST**, **FE_DEC_TONEARESTFROMZERO**, **FE_DEC_TOWARDZERO**, and **FE_DEC_UPWARD**) defined in 7.6, to specify a constant rounding mode, or **FE_DEC_DYNAMIC**, to specify dynamic rounding. The corresponding dynamic rounding mode can be established by a call to **fe_dec_setround**.

Table 7.2 — Functions affected by constant rounding modes – for decimal floating types

Header	Function families
<math.h>	acos, acospi, asin, asinpi, atan, atan2, atan2pi, atanpi
<math.h>	cos, cospi, sin, sinpi, tan, tanpi
<math.h>	acosh, asinh, atanh
<math.h>	cosh, sinh, tanh
<math.h>	exp, exp10, exp10m1, exp2, exp2m1, expm1
<math.h>	log, log10, log10p1, log1p, log2, log2p1, logp1
<math.h>	scalbn, scalbln, ldexp
<math.h>	cbrt, compoundn, hypot, pow, pown, powr, rootn, rsqrt, sqrt
<math.h>	erf, erfc
<math.h>	lgamma, tgamma
<math.h>	rint, nearbyint, lrint, llrint
<math.h>	quantize
<math.h>	fdim
<math.h>	fma

<math.h>	d32add, d64add, d32sub, d64sub, d32mul, d64mul, d32div, d64div, d32fma, d64fma, d32sqrt, d64sqrt
<stdlib.h>	strfrom, strtol
<wchar.h>	wcsto
<stdio.h>	printf and scanf families
<wchar.h>	wprintf and wscanf families

A function family listed in Table 7.2 indicates the functions for all decimal floating types, where the function family is represented by the name of the functions without a suffix. For example, **acos** indicates the functions **acosd32**, **acosd64**, and **acosd128**.

7.6.5 Floating-point exceptions

7.6.5.1 General

The following functions provide access to the floating-point status flags.²⁴⁹⁾ The **int** input argument for the functions represents a subset of floating-point exceptions, and can be zero or the bitwise OR of one or more floating-point exception macros, for example **FE_OVERFLOW** | **FE_INEXACT**. For other argument values, the behavior of these functions is undefined.

7.6.5.2 The **feclearexcept** function

Synopsis

```
#include <fenv.h>
int feclearexcept(int excepts);
```

Description

The **feclearexcept** function attempts to clear the supported floating-point exceptions represented by its argument.

Returns

The **feclearexcept** function returns zero if the **excepts** argument is zero or if all the specified exceptions were successfully cleared. Otherwise, it returns a nonzero value.

7.6.5.3 The **fegetexceptflag** function

Synopsis

```
#include <fenv.h>
int fegetexceptflag(fexcept_t *flagp, int excepts);
```

Description

The **fegetexceptflag** function attempts to store an implementation-defined representation of the states of the floating-point status flags indicated by the argument **excepts** in the object pointed to by the argument **flagp**.

Returns

The **fegetexceptflag** function returns zero if the representation was successfully stored. Otherwise, it returns a nonzero value.

7.6.5.4 The **feraiseexcept** function

²⁴⁹⁾The functions **fetestexcept**, **feraiseexcept**, and **feclearexcept** support the basic abstraction of flags that are either set or clear. An implementation can endow floating-point status flags with more information — for example, the address of the code which first raised the floating-point exception; the functions **fegetexceptflag** and **fesetexceptflag** deal with the full content of flags.

Synopsis

```
#include <fenv.h>
int feraiseexcept(int excepts);
```

Description

The **feraiseexcept** function attempts to raise the supported floating-point exceptions represented by its argument.²⁵⁰⁾ The order in which these floating-point exceptions are raised is unspecified, except as stated in F.8.7. Whether the **feraiseexcept** function additionally raises the “inexact” floating-point exception whenever it raises the “overflow” or “underflow” floating-point exception is implementation-defined.

Returns

The **feraiseexcept** function returns zero if the **excepts** argument is zero or if all the specified exceptions were successfully raised. Otherwise, it returns a nonzero value.

Recommended practice

Implementation extensions associated with raising a floating-point exception (for example, enabled traps or ISO/IEC 60559 alternate exception handling) should be honored by this function.

7.6.5.5 The fesetexcept function

Synopsis

```
#include <fenv.h>
int fesetexcept(int excepts);
```

Description

The **fesetexcept** function attempts to set the supported floating-point exception flags represented by its argument. This function does not clear any floating-point exception flags. This function changes the state of the floating-point exception flags, but does not cause any other side effects that can be associated with raising floating-point exceptions.²⁵¹⁾

Returns

The **fesetexcept** function returns zero if all the specified exceptions were successfully set or if the **excepts** argument is zero. Otherwise, it returns a nonzero value.

7.6.5.6 The fesetexceptflag function

Synopsis

```
#include <fenv.h>
int fesetexceptflag(const fexcept_t *flagp, int excepts);
```

Description

The **fesetexceptflag** function attempts to set the floating-point status flags indicated by the argument **excepts** to the states stored in the object pointed to by **flagp**. The value of ***flagp** shall have been set by a previous call to **fegetexceptflag** whose second argument represented at least those floating-point exceptions represented by the argument **excepts**. Like **fesetexcept**, this function does not raise floating-point exceptions, but only sets the state of the flags.

Returns

The **fesetexceptflag** function returns zero if the **excepts** argument is zero or if all the specified flags were successfully set to the appropriate state. Otherwise, it returns a nonzero value.

²⁵⁰⁾The effect is intended to be similar to that of floating-point exceptions raised by arithmetic operations. Hence, implementation extensions associated with raising a floating-point exception (for example, enabled traps or ISO/IEC 60559 alternate exception handling) should be honored. The specification in F.8.7 is in the same spirit.

²⁵¹⁾Implementation extensions like traps for floating-point exceptions and ISO/IEC 60559 exception handling do not occur.

7.6.5.7 The `fetestexceptflag` function

Synopsis

```
#include <fenv.h>
int fetestexceptflag(const fexcept_t *flagp, int excepts);
```

Description

The `fetestexceptflag` function determines which of a specified subset of the floating-point exception flags are set in the object pointed to by `flagp`. The value of `*flagp` shall have been set by a previous call to `fegetexceptflag` whose second argument represented at least those floating-point exceptions represented by the argument `excepts`. The `excepts` argument specifies the floating-point status flags to be queried.

Returns

The `fetestexceptflag` function returns the value of the bitwise OR of the floating-point exception macros included in `excepts` corresponding to the floating-point exceptions set in `*flagp`.

7.6.5.8 The `fetestexcept` function

Synopsis

```
#include <fenv.h>
int fetestexcept(int excepts);
```

Description

The `fetestexcept` function determines which of a specified subset of the floating-point exception flags are currently set. The `excepts` argument specifies the floating-point status flags to be queried.²⁵²⁾

Returns

The `fetestexcept` function returns the value of the bitwise OR of the floating-point exception macros corresponding to the currently set floating-point exceptions included in `excepts`.

EXAMPLE Call `f` if “invalid” is set, then `g` if “overflow” is set:

```
#include <fenv.h>
/* ... */
{
    #pragma STDC FENV_ACCESS ON
    int set_excepts;
    feclearexcept(FE_INVALID | FE_OVERFLOW);
    // maybe raise exceptions
    set_excepts = fetestexcept(FE_INVALID | FE_OVERFLOW);
    if (set_excepts & FE_INVALID) f();
    if (set_excepts & FE_OVERFLOW) g();
    /* ... */
}
```

7.6.6 Rounding and other control modes

7.6.6.1 General

The `fegetround` and `fesetround` functions provide control of rounding direction modes. The `fegetmode` and `fesetmode` functions manage all the implementation’s dynamic floating-point control modes collectively.

7.6.6.2 The `fegetmode` function

Synopsis

²⁵²⁾This mechanism allows testing several floating-point exceptions with just one function call.

```
#include <fenv.h>
int fegetmode(femode_t *modep);
```

Description

The **fegetmode** function attempts to store all the dynamic floating-point control modes in the object pointed to by **modep**.

Returns

The **fegetmode** function returns zero if the modes were successfully stored. Otherwise, it returns a nonzero value.

7.6.6.3 The fegetround function

Synopsis

```
#include <fenv.h>
int fegetround(void);
```

Description

The **fegetround** function gets the current value of the dynamic rounding direction mode.

Returns

The **fegetround** function returns the value of the rounding direction macro representing the current dynamic rounding direction or a negative value if there is no such rounding direction macro or the current dynamic rounding direction is not determinable.

7.6.6.4 The fe_dec_getround function

Synopsis

```
#include <fenv.h>
#ifdef __STDC_IEC_60559_DFP__
int fe_dec_getround(void);
#endif
```

Description

The **fe_dec_getround** function gets the current value of the dynamic rounding direction mode for decimal floating-point operations.

Returns

The **fe_dec_getround** function returns the value of the rounding direction macro representing the current dynamic rounding direction for decimal floating-point operations, or a negative value if there is no such rounding macro or the current rounding direction is not determinable.

7.6.6.5 The fesetmode function

Synopsis

```
#include <fenv.h>
int fesetmode(const femode_t *modep);
```

Description

The **fesetmode** function attempts to establish the dynamic floating-point modes represented by the object pointed to by **modep**. The argument **modep** shall point to an object set by a call to **fegetmode**, or equal **FE_DFL_MODE** or a dynamic floating-point mode state macro defined by the implementation.

Returns

The **fesetmode** function returns zero if the modes were successfully established. Otherwise, it returns a nonzero value.

7.6.6.6 The fesetround function

Synopsis

```
#include <fenv.h>
int fesetround(int rnd);
```

Description

The **fesetround** function establishes the rounding direction represented by its argument **rnd**. If the argument is not equal to the value of a rounding direction macro, the rounding direction is not changed.

Returns

The **fesetround** function returns zero if and only if the dynamic rounding direction mode was set to the requested rounding direction.

EXAMPLE The following is a way to save, set, and restore the rounding direction, including reporting an error and abort if setting the rounding direction fails.

```
#include <fenv.h>
#include <assert.h>

void f(int rnd_dir)
{
    #pragma STDC FENV_ACCESS ON
    int save_round;
    int setround_ok;
    save_round = fegetround();
    setround_ok = fesetround(rnd_dir);
    assert(setround_ok == 0);
    /* ... */
    fesetround(save_round);
    /* ... */
}
```

7.6.6.7 The fe_dec_setround function

Synopsis

```
#include <fenv.h>
#ifdef __STDC_IEC_60559_DFP__
int fe_dec_setround(int rnd);
#endif
```

Description

The **fe_dec_setround** function sets the dynamic rounding direction mode for decimal floating-point operations to be the rounding direction represented by its argument **rnd**. If the argument is not equal to the value of a decimal rounding direction macro, the rounding direction is not changed.

If **FLT_RADIX** is not 10, the rounding direction altered by the **fesetround** function is independent of the rounding direction altered by the **fe_dec_setround** function; otherwise if **FLT_RADIX** is 10, whether the **fesetround** and **fe_dec_setround** functions alter the rounding direction of both standard and decimal floating-point operations is implementation-defined.

Returns

The **fe_dec_setround** function returns a zero value if and only if the argument is equal to a decimal rounding direction macro (that is, if and only if the dynamic rounding direction mode for decimal floating-point operations was set to the requested rounding direction).

7.6.7 Environment

7.6.7.1 General

The functions in this section manage the floating-point environment — status flags and control modes — as one entity.

7.6.7.2 The `fegetenv` function

Synopsis

```
#include <fenv.h>
int fegetenv(fenv_t *envp);
```

Description

The **fegetenv** function attempts to store the current dynamic floating-point environment in the object pointed to by **envp**.

Returns

The **fegetenv** function returns zero if the environment was successfully stored. Otherwise, it returns a nonzero value.

7.6.7.3 The `feholdexcept` function

Synopsis

```
#include <fenv.h>
int feholdexcept(fenv_t *envp);
```

Description

The **feholdexcept** function saves the current dynamic floating-point environment in the object pointed to by **envp**, clears the floating-point status flags, and then installs a *non-stop* (continue on floating-point exceptions) mode, if available, for all floating-point exceptions.²⁵³⁾

Returns

The **feholdexcept** function returns zero if and only if non-stop floating-point exception handling was successfully installed.

7.6.7.4 The `fesetenv` function

Synopsis

```
#include <fenv.h>
int fesetenv(const fenv_t *envp);
```

Description

The **fesetenv** function attempts to establish the dynamic floating-point environment represented by the object pointed to by **envp**. The argument **envp** shall point to an object set by a call to **fegetenv** or **feholdexcept**, or equal a dynamic floating-point environment macro. **fesetenv** merely installs the state of the floating-point status flags represented through its argument, and does not raise these floating-point exceptions.

Returns

The **fesetenv** function returns zero if the environment was successfully established. Otherwise, it returns a nonzero value.

7.6.7.5 The `feupdateenv` function

²⁵³⁾ISO/IEC 60559 systems have a default non-stop mode, and typically at least one other mode for trap handling or aborting; if the system provides only the non-stop mode then installing it is trivial. For such systems, the **feholdexcept** function can be used in conjunction with the **feupdateenv** function to write routines that hide spurious floating-point exceptions from their callers.

Synopsis

```
#include <fenv.h>
int feupdateenv(const fenv_t *envp);
```

Description

The **feupdateenv** function attempts to save the currently raised floating-point exceptions in its automatic storage, install the dynamic floating-point environment represented by the object pointed to by **envp**, and then raise the saved floating-point exceptions. The argument **envp** shall point to an object set by a call to **feholdexcept** or **fegetenv**, or equal a dynamic floating-point environment macro.

Returns

The **feupdateenv** function returns zero if all the actions were successfully carried out. Otherwise, it returns a nonzero value.

EXAMPLE Hide spurious underflow floating-point exceptions:

```
#include <fenv.h>
double f(double x)
{
    #pragma STDC FENV_ACCESS ON
    double result;
    fenv_t save_env;
    if (feholdexcept(&save_env))
        return /* indication of an environmental problem */;
    // compute result
    if (/* test spurious underflow */)
        if (feclearexcept(FE_UNDERFLOW))
            return /* indication of an environmental problem */;
    if (feupdateenv(&save_env))
        return /* indication of an environmental problem */;
    return result;
}
```

7.7 Characteristics of floating types <float.h>

The header <float.h> defines several macros that expand to various limits and parameters of the real floating types.

The macro

`__STDC_VERSION_FLOAT_H__`

is an integer constant expression with a value equivalent to 202311L.

The rest of the macros, their meanings, and the constraints (or restrictions) on their values are listed in 5.3.5.3.3 and 5.3.5.3.4. A summary is given in Annex E.

7.8 Format conversion of integer types <inttypes.h>

7.8.1 General

The header <inttypes.h> includes the header <stdint.h> and extends it with additional facilities provided by hosted implementations.

It defines the macro

```
__STDC_VERSION__INTTYPES_H__
```

which is an integer constant expression with a value equivalent to 202311L.

It declares functions for manipulating greatest-width integers and converting numeric character strings to greatest-width integers, and it declares the type

```
imaxdiv_t
```

which is a structure type that is the type of the value returned by the `imaxdiv` function. For each type declared in <stdint.h>, it defines corresponding macros for conversion specifiers for use with the formatted input/output functions.²⁵⁴⁾

Forward references: integer types <stdint.h> (7.22), formatted input/output functions (7.23.6), formatted wide character input/output functions (7.31.2).

7.8.2 Macros for format specifiers

Each of the following object-like macros expands to a character string literal containing a conversion specifier, possibly modified by a length modifier, suitable for use within the format argument of a formatted input/output function when converting the corresponding integer type. These macro names have the general form of **PRI** (character string literals for the `fprintf` and `fwprintf` family) or **SCN** (character string literals for the `fscanf` and `fwscanf` family),²⁵⁵⁾ followed by the conversion specifier, followed by a name corresponding to a similar type name in 7.22.2. In these names, *N* represents the width of the type as described in 7.22.2. For example, **PRIdFAST32** can be used in a format string to print the value of an integer of type `int_fast32_t`. The functions in the `fprintf` and `fwprintf` families shall behave as if they use `va_arg` with a type argument naming the type resulting from applying the default argument promotions to the type corresponding to the macro and then convert the result of the `va_arg` expansion to the type corresponding to the macro.

The `fprintf` macros for signed integers are:

PRIdN	PRIdLEASTN	PRIdFASTN	PRIdMAX	PRIdPTR
PRiN	PRiLEASTN	PRiFASTN	PRiMAX	PRiPTR

The `fprintf` macros for unsigned integers are:

PRIBN	PRIBLEASTN	PRIBFASTN	PRIBMAX	PRIBPTR
PRIoN	PRIoLEASTN	PRIoFASTN	PRIoMAX	PRIoPTR
PRIU	PRIULEASTN	PRIUFASTN	PRIU	PRiPTR
PRIXN	PRIXLEASTN	PRIXFASTN	PRIXMAX	PRIXPTR
PRiXN	PRiXLEASTN	PRiXFASTN	PRiXMAX	PRiXPTR

The following `fprintf` macros for unsigned integer types are optional:

PRIBN	PRIBLEASTN	PRIBFASTN	PRIBMAX	PRIBPTR
--------------	-------------------	------------------	----------------	----------------

They shall be defined if the implementation supports the **B** specifier as indicated in 7.23.6.2 and 7.31.2.2; otherwise they shall not be defined.

The `fscanf` macros for signed integers are:

SCNdN	SCNdLEASTN	SCNdFASTN	SCNdMAX	SCNdPTR
SCNiN	SCNiLEASTN	SCNiFASTN	SCNiMAX	SCNiPTR

²⁵⁴⁾See “future library directions” (7.33.7).
²⁵⁵⁾For any given type, the corresponding macros for `fprintf` and `fscanf` functions can be distinct.

The **fscanf** macros for unsigned integers are:

SCNbN	SCNbLEASTN	SCNbFASTN	SCNbMAX	SCNbPTR
SCNoN	SCNoLEASTN	SCNoFASTN	SCNoMAX	SCNoPTR
SCNuN	SCNuLEASTN	SCNuFASTN	SCNuMAX	SCNuPTR
SCNxN	SCNxLEASTN	SCNxFASTN	SCNxMAX	SCNxPTR

For each type that the implementation provides in `<stdint.h>`, the corresponding **fprintf** macros shall be defined and the corresponding **fscanf** macros shall be defined unless the implementation does not have a suitable **fscanf** length modifier for the type.

EXAMPLE

```
#include <inttypes.h>
#include <wchar.h>
int main(void)
{
    uintmax_t i = UINTMAX_MAX;    // this type always exists
    wprintf(L"The largest integer value is %020"
           PRIxMAX "\n", i);
    return 0;
}
```

7.8.3 Functions for greatest-width integer types

7.8.3.1 The **imaxabs** function

Synopsis

```
#include <inttypes.h>
intmax_t imaxabs(intmax_t j);
```

Description

The **imaxabs** function computes the absolute value of an integer **j**. If the result cannot be represented, the behavior is undefined.²⁵⁶⁾

Returns

The **imaxabs** function returns the absolute value.

7.8.3.2 The **imaxdiv** function

Synopsis

```
#include <inttypes.h>
imaxdiv_t imaxdiv(intmax_t numer, intmax_t denom);
```

Description

The **imaxdiv** function computes **numer** / **denom** and **numer** % **denom** in a single operation.

Returns

The **imaxdiv** function returns a structure of type **imaxdiv_t** comprising both the quotient and the remainder. The structure shall contain (in either order) the members **quot** (the quotient) and **rem** (the remainder), each of which has type **intmax_t**. If either part of the result cannot be represented, the behavior is undefined.

7.8.3.3 The **strtoimax** and **strtoumax** functions

Synopsis

```
#include <inttypes.h>
intmax_t strtoimax(const char * restrict nptr, char ** restrict endptr, int base);
uintmax_t strtoumax(const char * restrict nptr, char ** restrict endptr, int base);
```

²⁵⁶⁾The absolute value of the most negative number is not representable.

Description

The **strtoimax** and **strtoumax** functions are equivalent to the **strtol**, **strtoll**, **strtoul**, and **strtoull** functions, except that the initial portion of the string is converted to **intmax_t** and **uintmax_t** representation, respectively.

Returns

The **strtoimax** and **strtoumax** functions return the converted value, if any. If no conversion could be performed, zero is returned. If the correct value is outside the range of representable values, **INTMAX_MAX**, **INTMAX_MIN**, or **UINTMAX_MAX** is returned (according to the return type and sign of the value, if any), and the value of the macro **ERANGE** is stored in **errno**.

Forward references: the **strtol**, **strtoll**, **strtoul**, and **strtoull** functions (7.24.2.8).

7.8.3.4 The **wcstoimax** and **wcstoumax** functions

Synopsis

```
#include <stddef.h>           // for wchar_t
#include <inttypes.h>
intmax_t wcstoimax(const wchar_t *restrict nptr, wchar_t **restrict endptr, int base);
uintmax_t wcstoumax(const wchar_t *restrict nptr, wchar_t **restrict endptr, int base);
```

Description

The **wcstoimax** and **wcstoumax** functions are equivalent to the **wcstol**, **wcstoll**, **wcstoul**, and **wcstoull** functions except that the initial portion of the wide string is converted to **intmax_t** and **uintmax_t** representation, respectively.

Returns

The **wcstoimax** function returns the converted value, if any. If no conversion could be performed, zero is returned. If the correct value is outside the range of representable values, **INTMAX_MAX**, **INTMAX_MIN**, or **UINTMAX_MAX** is returned (according to the return type and sign of the value, if any), and the value of the macro **ERANGE** is stored in **errno**.

Forward references: the **wcstol**, **wcstoll**, **wcstoul**, and **wcstoull** functions (7.31.4.2.4).

7.9 Alternative spellings <iso646.h>

The header <iso646.h> defines the following eleven macros (on the left) that expand to the corresponding tokens (on the right):

and	&&
and_eq	&=
bitand	&
bitor	
compl	~
not	!
not_eq	!=
or	
or_eq	 =
xor	^
xor_eq	^=

7.10 Characteristics of integer types <limits.h>

The header <limits.h> defines several macros that expand to various limits and parameters of the standard integer types.

The macro

```
__STDC_VERSION_LIMITS_H__
```

is an integer constant expression with a value equivalent to 202311L.

The rest of the macros, their meanings, and the constraints (or restrictions) on their values are listed in 5.3.5.3.2. A summary is given in Annex E.

7.11 Localization <locale.h>

7.11.1 General

The header <locale.h> declares two functions, one type, and defines several macros.

The type is

```
struct lconv
```

which contains members related to the formatting of numeric values. The structure shall contain at least the following members, in any order. The semantics of the members and their normal ranges are explained in 7.11.3.1. In the "C" locale, the members shall have the values specified in the comments.

```
char *decimal_point;    // "."
char *thousands_sep;   // ""
char *grouping;         // ""
char *mon_decimal_point; // ""
char *mon_thousands_sep; // ""
char *mon_grouping;     // ""
char *positive_sign;    // ""
char *negative_sign;    // ""
char *currency_symbol;  // ""
char frac_digits;       // CHAR_MAX
char p_cs_precedes;     // CHAR_MAX
char n_cs_precedes;     // CHAR_MAX
char p_sep_by_space;    // CHAR_MAX
char n_sep_by_space;    // CHAR_MAX
char p_sign_posn;       // CHAR_MAX
char n_sign_posn;       // CHAR_MAX
char *int_curr_symbol;  // ""
char int_frac_digits;   // CHAR_MAX
char int_p_cs_precedes; // CHAR_MAX
char int_n_cs_precedes; // CHAR_MAX
char int_p_sep_by_space; // CHAR_MAX
char int_n_sep_by_space; // CHAR_MAX
char int_p_sign_posn;   // CHAR_MAX
char int_n_sign_posn;   // CHAR_MAX
```

The macros defined are **NULL** (described in 7.21); and

```
LC_ALL
LC_COLLATE
LC_CTYPE
LC_MONETARY
LC_NUMERIC
LC_TIME
```

which expand to integer constant expressions with distinct values, suitable for use as the first argument to the **setlocale** function.²⁵⁷⁾ Additional macro definitions, beginning with the characters **LC_** and an uppercase letter,²⁵⁸⁾ may also be specified by the implementation.

7.11.2 The **setlocale** function

Synopsis

```
#include <locale.h>
char *setlocale(int category, const char *locale);
```

²⁵⁷⁾ISO/IEC 9945 specifies locale and charmap formats that can be used to specify locales for C.

²⁵⁸⁾See "future library directions" (7.33.8).

Description

The **setlocale** function selects the appropriate portion of the program’s locale as specified by the **category** and **locale** arguments. The **setlocale** function may be used to change or query the program’s entire current locale or portions thereof. The value **LC_ALL** for **category** names the program’s entire locale; the other values for **category** name only a portion of the program’s locale. **LC_COLLATE** affects the behavior of the **strcoll** and **strxfrm** functions. **LC_CTYPE** affects the behavior of the character handling functions²⁵⁹⁾ and the multibyte and wide character functions. **LC_MONETARY** affects the monetary formatting information returned by the **localeconv** function. **LC_NUMERIC** affects the decimal-point character for the formatted input/output functions and the string conversion functions, as well as the nonmonetary formatting information returned by the **localeconv** function. **LC_TIME** affects the behavior of the **strftime** and **wcsftime** functions.

A value of **"C"** for **locale** specifies the minimal environment for C translation; a value of **""** for **locale** specifies the locale-specific native environment. Other implementation-defined strings may be passed as the second argument to **setlocale**.

At program startup, the equivalent of

```
setlocale(LC_ALL, "C");
```

is executed.

A call to the **setlocale** function may introduce a data race with other calls to the **setlocale** function or with calls to functions that are affected by the current locale. The implementation shall behave as if no library function calls the **setlocale** function.

Returns

If a pointer to a string is given for **locale** and the selection can be honored, the **setlocale** function returns a pointer to the string associated with the specified **category** for the new locale. If the selection cannot be honored, the **setlocale** function returns a null pointer and the program’s locale is not changed.

A null pointer for **locale** causes the **setlocale** function to return a pointer to the string associated with the **category** for the program’s current locale; the program’s locale is not changed.²⁶⁰⁾

The pointer to string returned by the **setlocale** function is such that a subsequent call with that string value and its associated category will restore that part of the program’s locale. The string pointed to shall not be modified by the program. The behavior is undefined if the returned value is used after a subsequent call to the **setlocale** function, or after the thread which called the **setlocale** function to obtain the returned value has exited.

Forward references: formatted input/output functions (7.23.6), multibyte/wide character conversion functions (7.24.8), multibyte/wide string conversion functions (7.24.9), numeric conversion functions (7.24.2), the **strcoll** function (7.26.4.4), the **strftime** function (7.29.3.6), the **strxfrm** function (7.26.4.6).

7.11.3 Numeric formatting convention inquiry

7.11.3.1 The **localeconv** function

Synopsis

```
#include <locale.h>
struct lconv *localeconv(void);
```

Description

The **localeconv** function sets the components of an object with type **struct lconv** with values appropriate for the formatting of numeric quantities (monetary and otherwise) according to the rules of the current locale.

²⁵⁹⁾The only functions in 7.4 whose behavior is not affected by the current locale are **isdigit** and **isxdigit**.
²⁶⁰⁾The implementation is thus required to arrange to encode in a string the various categories due to a heterogeneous locale when **category** has the value **LC_ALL**.

The members of the structure with type **char *** are pointers to strings, any of which (except **decimal_point**) can point to "", to indicate that the value is not available in the current locale or is of zero length. Apart from **grouping** and **mon_grouping**, the strings shall start and end in the initial shift state. The members with type **char** are nonnegative numbers, any of which can be **CHAR_MAX** to indicate that the value is not available in the current locale. The members include the following:

char *decimal_point

The decimal-point character used to format nonmonetary quantities.

char *thousands_sep

The character used to separate groups of digits before the decimal-point character in formatted nonmonetary quantities.

char *grouping

A string whose elements indicate the size of each group of digits in formatted nonmonetary quantities.

char *mon_decimal_point

The decimal-point used to format monetary quantities.

char *mon_thousands_sep

The separator for groups of digits before the decimal-point in formatted monetary quantities.

char *mon_grouping

A string whose elements indicate the size of each group of digits in formatted monetary quantities.

char *positive_sign

The string used to indicate a nonnegative-valued formatted monetary quantity.

char *negative_sign

The string used to indicate a negative-valued formatted monetary quantity.

char *currency_symbol

The local currency symbol applicable to the current locale.

char frac_digits

The number of fractional digits (those after the decimal-point) to be displayed in a locally formatted monetary quantity.

char p_cs_precedes

Set to 1 or 0 if the **currency_symbol** respectively precedes or succeeds the value for a nonnegative locally formatted monetary quantity.

char n_cs_precedes

Set to 1 or 0 if the **currency_symbol** respectively precedes or succeeds the value for a negative locally formatted monetary quantity.

char p_sep_by_space

Set to a value indicating the separation of the **currency_symbol**, the sign string, and the value for a nonnegative locally formatted monetary quantity.

char n_sep_by_space

Set to a value indicating the separation of the **currency_symbol**, the sign string, and the value for a negative locally formatted monetary quantity.

char p_sign_posn

Set to a value indicating the positioning of the **positive_sign** for a nonnegative locally formatted monetary quantity.

char n_sign_posn

Set to a value indicating the positioning of the **negative_sign** for a negative locally formatted monetary quantity.

char *int_curr_symbol

The international currency symbol applicable to the current locale. The first three characters contain the alphabetic international currency symbol in accordance with those specified in ISO 4217. The fourth character (immediately preceding the null character) is the character used to separate the international currency symbol from the monetary quantity.

char int_frac_digits

The number of fractional digits (those after the decimal-point) to be displayed in an internationally formatted monetary quantity.

char int_p_cs_precedes

Set to 1 or 0 if the **int_curr_symbol** respectively precedes or succeeds the value for a nonnegative internationally formatted monetary quantity.

char int_n_cs_precedes

Set to 1 or 0 if the **int_curr_symbol** respectively precedes or succeeds the value for a negative internationally formatted monetary quantity.

char int_p_sep_by_space

Set to a value indicating the separation of the **int_curr_symbol**, the sign string, and the value for a nonnegative internationally formatted monetary quantity.

char int_n_sep_by_space

Set to a value indicating the separation of the **int_curr_symbol**, the sign string, and the value for a negative internationally formatted monetary quantity.

char int_p_sign_posn

Set to a value indicating the positioning of the **positive_sign** for a nonnegative internationally formatted monetary quantity.

char int_n_sign_posn

Set to a value indicating the positioning of the **negative_sign** for a negative internationally formatted monetary quantity.

The elements of **grouping** and **mon_grouping** are interpreted according to the following:

CHAR_MAX No further grouping is to be performed.

0 The previous element is to be repeatedly used for the remainder of the digits.

other The integer value is the number of digits that compose the current group. The next element is examined to determine the size of the next group of digits before the current group.

The values of **p_sep_by_space**, **n_sep_by_space**, **int_p_sep_by_space**, and **int_n_sep_by_space** are interpreted according to the following:

0 No space separates the currency symbol and value.

1 If the currency symbol and sign string are adjacent, a space separates them from the value; otherwise, a space separates the currency symbol from the value.

2 If the currency symbol and sign string are adjacent, a space separates them; otherwise, a space separates the sign string from the value.

For **int_p_sep_by_space** and **int_n_sep_by_space**, the fourth character of **int_curr_symbol** is used instead of a space.

The values of **p_sign_posn**, **n_sign_posn**, **int_p_sign_posn**, and **int_n_sign_posn** are interpreted according to the following:

- 0 Parentheses surround the quantity and currency symbol.
- 1 The sign string precedes the quantity and currency symbol.
- 2 The sign string succeeds the quantity and currency symbol.
- 3 The sign string immediately precedes the currency symbol.
- 4 The sign string immediately succeeds the currency symbol.

The implementation shall behave as if no library function calls the **localeconv** function.

Returns

The **localeconv** function returns a pointer to the filled-in object. The structure pointed to by the return value shall not be modified by the program, but may be overwritten by a subsequent call to the **localeconv** function. In addition, calls to the **setlocale** function with categories **LC_ALL**, **LC_MONETARY**, or **LC_NUMERIC** may overwrite the contents of the structure.

EXAMPLE 1 Table 7.3 illustrates rules which can be used by four countries to format — potentially irrelevant, imaginary, and/or historical — monetary quantities.

Table 7.3 — Formatting rule examples

Country	Local format		International format	
	Positive	Negative	Positive	Negative
Country1	1.234,56 mk	-1.234,56 mk	FIM 1.234,56	FIM -1.234,56
Country2	L.1.234	-L.1.234	ITL 1.234	-ITL 1.234
Country3	ƒ 1.234,56	ƒ -1.234,56	NLG 1.234,56	NLG -1.234,56
Country4	SFr.s.1,234.56	SFr.s.1,234.56C	CHF 1,234.56	CHF 1,234.56C

For these four countries, the respective values for the monetary members of the structure returned by **localeconv** can be as displayed in Table 7.4:

Table 7.4 — Formatting values for example countries

	Country1	Country2	Country3	Country4
mon_decimal_point	","	""	","	."
mon_thousands_sep	."	."	."	."
mon_grouping	"\3"	"\3"	"\3"	"\3"
positive_sign	""	""	""	""
negative_sign	"_"	"_"	"_"	"ç"
currency_symbol	"mk"	"L."	"\u0192"	"SFrs."
frac_digits	2	0	2	2
p_cs_precedes	0	1	1	1
n_cs_precedes	0	1	1	1
p_sep_by_space	1	0	1	0
n_sep_by_space	1	0	2	0
p_sign_posn	1	1	1	1
n_sign_posn	1	1	4	2
int_curr_symbol	"FIM "	"ITL "	"NLG "	"CHF "
int_frac_digits	2	0	2	2
int_p_cs_precedes	1	1	1	1
int_n_cs_precedes	1	1	1	1
int_p_sep_by_space	1	1	1	1
int_n_sep_by_space	2	1	2	1
int_p_sign_posn	1	1	1	1
int_n_sign_posn	4	1	4	2

EXAMPLE 2 Table 7.5 illustrates how the **cs_precedes**, **sep_by_space**, and **sign_posn** members affect the formatted value.

Table 7.5 — Select member formatting affect

p_cs_precedes	p_sign_posn	p_sep_by_space		
		0	1	2
0	0	(1.25\$)	(1.25 \$)	(1.25\$)
	1	+1.25\$	+1.25 \$	+ 1.25\$
	2	1.25\$+	1.25 \$+	1.25\$ +
	3	1.25+\$	1.25 +\$	1.25+ \$
	4	1.25\$+	1.25 \$+	1.25\$ +
1	0	(\$1.25)	(\$ 1.25)	(\$1.25)
	1	+\$1.25	+\$ 1.25	+ \$1.25
	2	\$1.25+	\$ 1.25+	\$1.25 +
	3	+\$1.25	+\$ 1.25	+ \$1.25
	4	+\$1.25	+\$ 1.25	\$ +1.25

7.12 Mathematics <math.h>

7.12.1 General

The header <math.h> declares two types and many mathematical functions and defines several macros. Most synopses specify a family of functions consisting of a principal function with one or more **double** parameters, a **double** return value, or both; and other functions with the same name but with **f** and **l** suffixes, which are corresponding functions with **float** and **long double** parameters, return values, or both.²⁶¹⁾ Integer arithmetic functions and conversion functions are discussed later.

The feature test macro `__STDC_VERSION_MATH_H__` expands to the token 202311L.

The types

float_t
double_t

are floating types such that the values of **float** and **double** are subsets of the values of **float_t** and **double_t**, respectively, and such that the values of **float_t** are a subset of the values of **double_t**. If `FLT_EVAL_METHOD` equals 0, **float_t** and **double_t** are **float** and **double**, respectively; if `FLT_EVAL_METHOD` equals 1, they are both **double**; if `FLT_EVAL_METHOD` equals 2, they are both **long double**; and for other values of `FLT_EVAL_METHOD`, they are otherwise implementation-defined.²⁶²⁾ If they are not real floating types, the behavior is implementation-defined.

The types

_Decimal32_t
_Decimal64_t

are decimal floating types at least as wide as **_Decimal32** and **_Decimal64**, respectively, and such that **_Decimal64_t** is at least as wide as **_Decimal32_t**. They are present only if the implementation defines `__STDC_IEC_60559_DFP__` and additionally the user code defines `__STDC_WANT_IEC_60559_EXT__` before any inclusion of <math.h>. If `DEC_EVAL_METHOD` equals 0, **_Decimal32_t** and **_Decimal64_t** are **_Decimal32** and **_Decimal64**, respectively; if `DEC_EVAL_METHOD` equals 1, they are both **_Decimal64**; if `DEC_EVAL_METHOD` equals 2, they are both **_Decimal128**; and for other values of `DEC_EVAL_METHOD`, they are otherwise implementation-defined.

The macro

HUGE_VAL

expands to a **double** arithmetic constant expression, not necessarily representable as a **float**, whose value is the maximum value returned by library functions when a floating result of type **double** overflows under the default rounding mode, either maximum finite number in the type or positive or unsigned infinity. The macros

HUGE_VALF
HUGE_VALL

are respectively **float** and **long double** analogs of **HUGE_VAL**.²⁶³⁾

The macros in this paragraph are only present if the implementation defines `__STDC_IEC_60559_DFP__` and additionally the user code defines `__STDC_WANT_IEC_60559_EXT__` before any inclusion of <math.h>. The macro

²⁶¹⁾ Particularly on systems with wide expression evaluation, a <math.h> function can pass arguments and return values in wider format than the synopsis prototype indicates.

²⁶²⁾ The types **float_t** and **double_t** are intended to be the implementation's most efficient types at least as wide as **float** and **double**, respectively. For `FLT_EVAL_METHOD` equal 0, 1, or 2, the type **float_t** is the narrowest type used by the implementation to evaluate floating expressions.

²⁶³⁾ **HUGE_VAL**, **HUGE_VALF**, and **HUGE_VALL** can be positive infinities in an implementation that supports infinities.

HUGE_VAL_D32

expands to an arithmetic constant expression of type `_Decimal32` representing positive infinity. The macros

HUGE_VAL_D64
HUGE_VAL_D128

are respectively `_Decimal64` and `_Decimal128` analogs of `HUGE_VAL_D32`.

The macro

INFINITY

is defined if and only if the implementation supports an infinity for the type `float`. It expands to an arithmetic constant expression of type `float` representing positive or unsigned infinity.

The macro

DEC_INFINITY

expands to an arithmetic constant expression of type `_Decimal32` representing positive infinity.

The macro

NAN

is defined if and only if the implementation supports quiet NaNs for the `float` type. It expands to an arithmetic constant expression of type `float` representing a quiet NaN.

The macro

DEC_NAN

expands to an arithmetic constant expression of type `_Decimal32` representing a quiet NaN.

Use of the macros `INFINITY`, `DEC_INFINITY`, `NAN`, and `DEC_NAN` in `<math.h>` is an obsolescent feature. Instead, use the same macros in `<float.h>`.

The *number classification macros*

FP_INFINITE
FP_NAN
FP_NORMAL
FP_SUBNORMAL
FP_ZERO

represent mutually exclusive kinds of floating-point values. They expand to integer constant expressions with distinct values. Additional implementation-defined floating-point classifications, with macro definitions beginning with `FP_` and an uppercase letter, may also be specified by the implementation.

The *math rounding direction macros*

FP_INT_UPWARD
FP_INT_DOWNWARD
FP_INT_TOWARDZERO
FP_INT_TONEARESTFROMZERO
FP_INT_TONEAREST

represent the rounding directions of the functions `ceil`, `floor`, `trunc`, `round`, and `roundeven`, respectively, that convert to integral values in floating-point formats. They expand to integer

constant expressions with distinct values suitable for use as the second argument to the **fromfp**, **ufromfp**, **fromfpx**, and **ufromfpx** functions.

The macro

FP_FAST_FMA

is optionally defined. If defined, it indicates that the **fma** function generally executes about as fast as, or faster than, a multiply and an add of **double** operands.²⁶⁴⁾ The macros

FP_FAST_FMAF
FP_FAST_FMAL

are, respectively, **float** and **long double** analogs of **FP_FAST_FMA**. If defined, these macros expand to the integer constant 1.

The macros

FP_FAST_FMA_D32
FP_FAST_FMA_D64
FP_FAST_FMA_D128

are, respectively, **_Decimal32**, **_Decimal64**, and **_Decimal128** analogs of **FP_FAST_FMA**.

Each of the macros

FP_FAST_FADD	FP_FAST_DSUBL	FP_FAST_FDIVL	FP_FAST_FFMA
FP_FAST_FADDL	FP_FAST_FML	FP_FAST_FDIVL	FP_FAST_FFMAL
FP_FAST_DADDL	FP_FAST_FMLL	FP_FAST_FSQRT	FP_FAST_DFMA
FP_FAST_FSUB	FP_FAST_DMULL	FP_FAST_FSQRTL	
FP_FAST_FSUBL	FP_FAST_FDIV	FP_FAST_DSQRTL	

is optionally defined. If defined, it indicates that the corresponding function generally executes about as fast, or faster, than the corresponding operation or function of the argument type with result type the same as the argument type followed by conversion to the narrower type. For **FP_FAST_FFMA**, **FP_FAST_FFMAL**, and **FP_FAST_DFMA**, the comparison is to a call to **fma** or **fmal** followed by a conversion, not to separate multiply, add, and conversion. If defined, these macros expand to the integer constant 1.

The macros

FP_FAST_D32ADDD64	FP_FAST_D32MULD64	FP_FAST_D32FMAD64
FP_FAST_D32ADDD128	FP_FAST_D32MULD128	FP_FAST_D32FMAD128
FP_FAST_D64ADDD128	FP_FAST_D64MULD128	FP_FAST_D64FMAD128
FP_FAST_D32SUBD64	FP_FAST_D32DIVD64	FP_FAST_D32SQRTD64
FP_FAST_D32SUBD128	FP_FAST_D32DIVD128	FP_FAST_D32SQRTD128
FP_FAST_D64SUBD128	FP_FAST_D64DIVD128	FP_FAST_D64SQRTD128

are analogs of **FP_FAST_FADD**, **FP_FAST_FADDL**, **FP_FAST_DADDL**, etc., for decimal floating types.

The macros

FP_ILOGB0
FP_ILOGBNAN

expand to integer constant expressions whose values are returned by **ilogb(x)** if **x** is zero or NaN, respectively. The value of **FP_ILOGB0** shall be either **INT_MIN** or **-INT_MAX**. The value of **FP_ILOGBNAN** shall be either **INT_MAX** or **INT_MIN**.

²⁶⁴⁾Typically, the **FP_FAST_FMA** macro is defined if and only if the **fma** function is implemented directly with a hardware multiply-add instruction. Software implementations are expected to be substantially slower.

The macros

FP_LLOGB0 FP_LLOGBNAN
--

expand to integer constant expressions whose values are returned by `llogb(x)` if `x` is zero or NaN, respectively. The value of `FP_LLOGB0` shall be `LONG_MIN` if the value of `FP_ILOGB0` is `INT_MIN`, and shall be `-LONG_MAX` if the value of `FP_ILOGB0` is `-INT_MAX`. The value of `FP_LLOGBNAN` shall be `LONG_MAX` if the value of `FP_ILOGBNAN` is `INT_MAX`, and shall be `LONG_MIN` if the value of `FP_ILOGBNAN` is `INT_MIN`.

The macros

MATH_ERRNO MATH_ERREXCEPT
--

expand to the integer constants 1 and 2, respectively; the macro

math_errhandling

expands to an expression that has type `int` and the value `MATH_ERRNO`, `MATH_ERREXCEPT`, the bitwise OR of both, or 0; the value shall not be 0 in a hosted implementation. The value of `math_errhandling` is constant for the duration of the program. It is unspecified whether `math_errhandling` is a macro or an identifier with external linkage. If a macro definition is suppressed or a program defines an identifier with the name `math_errhandling`, the behavior is undefined. If the expression `math_errhandling & MATH_ERREXCEPT` can be nonzero, the implementation shall define the macros `FE_DIVBYZERO`, `FE_INVALID`, and `FE_OVERFLOW` in `<fenv.h>`.

7.12.2 Treatment of error conditions

The behavior of each of the functions in `<math.h>` is specified for all representable values of its input arguments, except where explicitly stated otherwise. Each function shall execute as if it were a single operation without raising `SIGFPE` and without generating any of the floating-point exceptions “invalid”, “divide-by-zero”, or “overflow” except to reflect the result of the function.

For all functions, a *domain error* occurs if and only if an input argument is outside the domain over which the mathematical function is defined. The description of each function lists any required domain errors; an implementation may define additional domain errors, provided that such errors are consistent with the mathematical definition of the function.²⁶⁵⁾ Whether a signaling NaN input causes a domain error is implementation-defined. On a domain error, the function returns an implementation-defined value; if the integer expression `math_errhandling & MATH_ERRNO` is nonzero, the integer expression `errno` acquires the value `EDOM`; if the integer expression `math_errhandling & MATH_ERREXCEPT` is nonzero, the “invalid” floating-point exception is raised.

Similarly, a *pole error* (also known as a singularity or infinitary) occurs if and only if the mathematical function has an exact infinite result as the finite input argument(s) are approached in the limit (for example, `log(0.0)`). The description of each function lists any required pole errors; an implementation may define additional pole errors, provided that such errors are consistent with the mathematical definition of the function. On a pole error, the function returns an implementation-defined value; if the integer expression `math_errhandling & MATH_ERRNO` is nonzero, the integer expression `errno` acquires the value `ERANGE`; if the integer expression `math_errhandling & MATH_ERREXCEPT` is nonzero, the “divide-by-zero” floating-point exception is raised.

Likewise, a *range error* occurs if and only if the result overflows or underflows, as defined below. The description of each function lists any required range errors; an implementation may define additional range errors, provided that such errors are consistent with the mathematical definition of

²⁶⁵⁾In an implementation that supports infinities, this allows an infinity as an argument to be a domain error if the mathematical domain of the function does not include the infinity.

the function and are the result of either overflow or underflow. Range errors that are required or implementation-defined shall or may be reported, as specified in this subclause, respectively.

A floating result overflows if a finite result value with ordinary accuracy²⁶⁶⁾ would have magnitude (absolute value) too large for the representation with full precision in the specified type. A result that is exactly an infinity does not overflow. If a floating result overflows and default rounding is in effect, then the function returns the value of the macro **HUGE_VAL**, **HUGE_VALF**, or **HUGE_VALL** according to the return type, with the same sign as the correct value of the function; however, for the types with reduced-precision representations of numbers beyond the overflow threshold, the function may return a representation of the result with less than full precision for the type. If a floating result overflows and default rounding is in effect and the integer expression **math_errhandling & MATH_ERRNO** is nonzero, then the integer expression **errno** acquires the value **ERANGE**. If a floating result overflows, and the integer expression **math_errhandling & MATH_ERREXCEPT** is nonzero, the “overflow” floating-point exception is raised (regardless of whether default rounding is in effect).

The result underflows if a nonzero result value with ordinary accuracy would have magnitude (absolute value) less than the minimum normalized number in the type; however a zero result that is specified to be an exact zero does not underflow. Also, a result with ordinary accuracy and the magnitude of the minimum normalized number may underflow.²⁶⁷⁾ If the result underflows, the function returns an implementation-defined value whose magnitude is no greater than the smallest normalized positive number in the specified type; if the integer expression **math_errhandling & MATH_ERRNO** is nonzero, whether **errno** acquires the value **ERANGE** is implementation-defined; if the integer expression **math_errhandling & MATH_ERREXCEPT** is nonzero, whether the “underflow” floating-point exception is raised is implementation-defined.

If a domain, pole, or range error occurs and the integer expression **math_errhandling & MATH_ERRNO** is zero,²⁶⁸⁾ then **errno** shall either be set to the value corresponding to the error or left unmodified. If no such error occurs, **errno** shall be left unmodified regardless of the setting of **math_errhandling**.

7.12.3 The FP_CONTRACT pragma

Synopsis

```
#include <math.h>
#pragma STDC FP_CONTRACT on-off-switch
```

Description

The **FP_CONTRACT** pragma can be used to allow (if the state is “on”) or disallow (if the state is “off”) the implementation to contract expressions (6.5.1). Each pragma can occur either outside external declarations or preceding all explicit declarations and statements inside a compound statement. When outside external declarations, the pragma takes effect from its occurrence until another **FP_CONTRACT** pragma is encountered, or until the end of the translation unit. When inside a compound statement, the pragma takes effect from its occurrence until another **FP_CONTRACT** pragma is encountered (including within a nested compound statement), or until the end of the compound statement; at the end of a compound statement the state for the pragma is restored to its condition just before the compound statement. If this pragma is used in any other context, the behavior is undefined. The default state (“on” or “off”) for the pragma is implementation-defined.

7.12.4 Classification macros

7.12.4.1 General

Floating-point values can be classified as NaN, infinite, normal, subnormal, or zero, or into other implementation-defined categories. Numbers whose magnitude is at least $b^{e_{\min}-1}$ (the minimum magnitude of normalized floating-point numbers in the type) and at most $(1 - b^{-p})b^{e_{\max}}$ (the

²⁶⁶⁾Ordinary accuracy is determined by the implementation. It refers to the accuracy of the function where results are not compromised by extreme magnitude.

²⁶⁷⁾The term underflow here is intended to encompass both “gradual underflow” as in ISO/IEC 60559 and also “flush-to-zero” underflow. ISO/IEC 60559 underflow can occur in cases where the magnitude of the rounded result (accurate to the full precision of the type) equals the minimum normalized number in the format.

²⁶⁸⁾Math errors are being indicated by the floating-point exception flags rather than by **errno**.

maximum magnitude of normalized floating-point numbers in the type), where b , p , $e_{\min}$, and $e_{\max}$ are as in 5.3.5.3.3, are classified as normal. Larger magnitude finite numbers represented with full precision in the type may also be classified as normal. Nonzero numbers whose magnitude is less than $b^{e_{\min}-1}$ are classified as subnormal.

In the synopses in this subclause, *real-floating* indicates that the argument shall be an expression of real floating type.

7.12.4.2 The `fpclassify` macro

Synopsis

```
#include <math.h>
int fpclassify(real-floating x);
```

Description

The `fpclassify` macro classifies its argument value as NaN, infinite, normal, subnormal, zero, or into another implementation-defined category. First, an argument represented in a format wider than its semantic type is converted to its semantic type. Then classification is based on the type of the argument.²⁶⁹⁾

Returns

The `fpclassify` macro returns the value of the number classification macro appropriate to the value of its argument.

7.12.4.3 The `iscanonical` macro

Synopsis

```
#include <math.h>
int iscanonical(real-floating x);
```

Description

The `iscanonical` macro determines whether its argument value is canonical (5.3.5.3.3). First, an argument represented in a format wider than its semantic type is converted to its semantic type. Then, determination is based on the type of the argument.

Returns

The `iscanonical` macro returns a nonzero value if and only if its argument is canonical.

7.12.4.4 The `isfinite` macro

Synopsis

```
#include <math.h>
int isfinite(real-floating x);
```

Description

The `isfinite` macro determines whether its argument has a finite value (zero, subnormal, or normal, and not infinite or NaN). First, an argument represented in a format wider than its semantic type is converted to its semantic type. Then determination is based on the type of the argument.

Returns

The `isfinite` macro returns a nonzero value if and only if its argument has a finite value.

7.12.4.5 The `isinf` macro

²⁶⁹⁾Since an expression can be evaluated with more range and precision than its type has, it is important to know the type that classification is based on. For example, a normal `long double` value can become subnormal when converted to `double`, and zero when converted to `float`.

Synopsis

```
#include <math.h>
int isinf(real-floating x);
```

Description

The **isinf** macro determines whether its argument value is (positive or negative) infinity. First, an argument represented in a format wider than its semantic type is converted to its semantic type. Then determination is based on the type of the argument.

Returns

The **isinf** macro returns a nonzero value if and only if its argument has an infinite value.

7.12.4.6 The isnan macro

Synopsis

```
#include <math.h>
int isnan(real-floating x);
```

Description

The **isnan** macro determines whether its argument value is a NaN. First, an argument represented in a format wider than its semantic type is converted to its semantic type. Then determination is based on the type of the argument.²⁷⁰⁾

Returns

The **isnan** macro returns a nonzero value if and only if its argument has a NaN value.

7.12.4.7 The isnormal macro

Synopsis

```
#include <math.h>
int isnormal(real-floating x);
```

Description

The **isnormal** macro determines whether its argument value is normal (neither zero, subnormal, infinite, nor NaN). First, an argument represented in a format wider than its semantic type is converted to its semantic type. Then determination is based on the type of the argument.

Returns

The **isnormal** macro returns a nonzero value if and only if its argument has a normal value.

7.12.4.8 The signbit macro

Synopsis

```
#include <math.h>
int signbit(real-floating x);
```

Description

The **signbit** macro determines whether the sign of its argument value is negative.²⁷¹⁾ If the argument value is an unsigned zero, its sign is regarded as positive. Otherwise, if the argument value is unsigned, the result value (zero or nonzero) is implementation-defined.

²⁷⁰⁾For the **isnan** macro, the type for determination does not matter unless the implementation supports NaNs in the evaluation type but not in the semantic type.

²⁷¹⁾The **signbit** macro determines the sign of all values, including infinities, zeros, and NaNs.

Returns

The **signbit** macro returns a nonzero value if and only if the sign of its argument value is determined to be negative.

7.12.4.9 The **issignaling** macro

Synopsis

```
#include <math.h>
int issignaling(real-floating x);
```

Description

The **issignaling** macro determines whether its argument value is a signaling NaN.

Returns

The **issignaling** macro returns a nonzero value if and only if its argument is a signaling NaN.²⁷²⁾

7.12.4.10 The **issubnormal** macro

Synopsis

```
#include <math.h>
int issubnormal(real-floating x);
```

Description

The **issubnormal** macro determines whether its argument value is subnormal. First, an argument represented in a format wider than its semantic type is converted to its semantic type. Then determination is based on the type of the argument.

Returns

The **issubnormal** macro returns a nonzero value if and only if its argument is subnormal.

7.12.4.11 The **iszero** macro

Synopsis

```
#include <math.h>
int iszero(real-floating x);
```

Description

The **iszero** macro determines whether its argument value is (positive, negative, or unsigned) zero. First, an argument represented in a format wider than its semantic type is converted to its semantic type. Then, determination is based on the type of the argument.

Returns

The **iszero** macro returns a nonzero value if and only if its argument is zero.

7.12.5 Trigonometric functions

7.12.5.1 The **acos** functions

Synopsis

```
#include <math.h>
double acos(double x);
float acosf(float x);
long double acosl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 acosd32(_Decimal32 x);
```

²⁷²⁾F.3 specifies that **issignaling** (and all the other classification macros), raise no floating-point exception if the argument is a variable, or any other expression whose value is represented in the format of its semantic type, even if the value is a signaling NaN.

```
_Decimal64 acosd64(_Decimal64 x);
_Decimal128 acosd128(_Decimal128 x);
#endif
```

Description

The **acos** functions compute the principal value of the arc cosine of **x**. A domain error occurs for arguments not in the interval $[-1, +1]$.

Returns

The **acos** functions return $\arccos x$ in the interval $[0, \pi]$ radians.

7.12.5.2 The asin functions

Synopsis

```
#include <math.h>
double asin(double x);
float asinf(float x);
long double asinl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 asind32(_Decimal32 x);
_Decimal64 asind64(_Decimal64 x);
_Decimal128 asind128(_Decimal128 x);
#endif
```

Description

The **asin** functions compute the principal value of the arc sine of **x**. A domain error occurs for arguments not in the interval $[-1, +1]$. A range error occurs if nonzero **x** is too close to zero.

Returns

The **asin** functions return $\arcsin x$ in the interval $[-\frac{\pi}{2}, +\frac{\pi}{2}]$ radians.

7.12.5.3 The atan functions

Synopsis

```
#include <math.h>
double atan(double x);
float atanf(float x);
long double atanl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 atand32(_Decimal32 x);
_Decimal64 atand64(_Decimal64 x);
_Decimal128 atand128(_Decimal128 x);
#endif
```

Description

The **atan** functions compute the principal value of the arc tangent of **x**. A range error occurs if nonzero **x** is too close to zero.

Returns

The **atan** functions return $\arctan x$ in the interval $[-\frac{\pi}{2}, +\frac{\pi}{2}]$ radians.

7.12.5.4 The atan2 functions

Synopsis

```
#include <math.h>
double atan2(double y, double x);
float atan2f(float y, float x);
long double atan2l(long double y, long double x);
#ifdef __STDC_IEC_60559_DFP__
```

```
_Decimal32 atan2d32(_Decimal32 y, _Decimal32 x);
_Decimal64 atan2d64(_Decimal64 y, _Decimal64 x);
_Decimal128 atan2d128(_Decimal128 y, _Decimal128 x);
#endif
```

Description

The **atan2** functions compute the value of the arc tangent of **y/x**, using the signs of both arguments to determine the quadrant of the return value. A domain error may occur if both arguments are zero. A range error occurs if **x** is positive and nonzero $\frac{y}{x}$ is too close to zero.

Returns

The **atan2** functions return $\arctan(y/x)$ in the interval $[-\pi, +\pi]$ radians.

7.12.5.5 The cos functions

Synopsis

```
#include <math.h>
double cos(double x);
float cosf(float x);
long double cosl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 cosd32(_Decimal32 x);
_Decimal64 cosd64(_Decimal64 x);
_Decimal128 cosd128(_Decimal128 x);
#endif
```

Description

The **cos** functions compute the cosine of **x** (measured in radians).

Returns

The **cos** functions return $\cos x$.

7.12.5.6 The sin functions

Synopsis

```
#include <math.h>
double sin(double x);
float sinf(float x);
long double sinl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 sind32(_Decimal32 x);
_Decimal64 sind64(_Decimal64 x);
_Decimal128 sind128(_Decimal128 x);
#endif
```

Description

The **sin** functions compute the sine of **x** (measured in radians). A range error occurs if nonzero **x** is too close to zero.

Returns

The **sin** functions return $\sin x$.

7.12.5.7 The tan functions

Synopsis

```
#include <math.h>
double tan(double x);
float tanf(float x);
long double tanl(long double x);
```

```
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 tand32(_Decimal32 x);
_Decimal64 tand64(_Decimal64 x);
_Decimal128 tand128(_Decimal128 x);
#endif
```

Description

The **tan** functions return the tangent of **x** (measured in radians). A range error occurs if nonzero **x** is too close to zero.

Returns

The **tan** functions return $\tan x$.

7.12.5.8 The acospi functions

Synopsis

```
#include <math.h>
double acospi(double x);
float acospi(float x);
long double acospi(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 acospid32(_Decimal32 x);
_Decimal64 acospid64(_Decimal64 x);
_Decimal128 acospid128(_Decimal128 x);
#endif
```

Description

The **acospi** functions compute the principal value of the arc cosine of **x**, divided by π , thus measuring the angle in half-revolutions. A domain error occurs for arguments not in the interval $[-1, +1]$.

Returns

The **acospi** functions return $\arccos(x)/\pi$ in the interval $[0, 1]$.

7.12.5.9 The asinpi functions

Synopsis

```
#include <math.h>
double asinpi(double x);
float asinpi(float x);
long double asinpi(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 asinpid32(_Decimal32 x);
_Decimal64 asinpid64(_Decimal64 x);
_Decimal128 asinpid128(_Decimal128 x);
#endif
```

Description

The **asinpi** functions compute the principal value of the arc sine of **x**, divided by π , thus measuring the angle in half-revolutions. A domain error occurs for arguments not in the interval $[-1, +1]$. A range error occurs if nonzero **x** is too close to zero.

Returns

The **asinpi** functions return $\arcsin(x)/\pi$ in the interval $[-\frac{1}{2}, +\frac{1}{2}]$.

7.12.5.10 The atanpi functions

Synopsis

```
#include <math.h>
```

```
double atanpi(double x);
float atanpif(float x);
long double atanpil(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 atanpid32(_Decimal32 x);
_Decimal64 atanpid64(_Decimal64 x);
_Decimal128 atanpid128(_Decimal128 x);
#endif
```

Description

The **atanpi** functions compute the principal value of the arc tangent of **x**, divided by π , thus measuring the angle in half-revolutions. A range error occurs if nonzero **x** is too close to zero.

Returns

The **atanpi** functions return $\arctan(\mathbf{x})/\pi$. in the interval $[-\frac{1}{2}, +\frac{1}{2}]$.

7.12.5.11 The atan2pi functions

Synopsis

```
#include <math.h>
double atan2pi(double y, double x);
float atan2pif(float y, float x);
long double atan2pil(long double y, long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 atan2pid32(_Decimal32 y, _Decimal32 x);
_Decimal64 atan2pid64(_Decimal64 y, _Decimal64 x);
_Decimal128 atan2pid128(_Decimal128 y, _Decimal128 x);
#endif
```

Description

The **atan2pi** functions compute the angle, measured in half-revolutions, subtended at the origin by the point (**x**, **y**) and the positive x-axis. Thus, the **atan2pi** functions compute $\arctan(\frac{y}{x})/\pi$, in the range $[-1, +1]$. A domain error may occur if both arguments are zero. A range error occurs if **x** is positive and nonzero $\frac{y}{x}$ is too close to zero.

Returns

The **atan2pi** functions return the computed angle, in the interval $[-1, +1]$.

7.12.5.12 The cospi functions

Synopsis

```
#include <math.h>
double cospi(double x);
float cospif(float x);
long double cospil(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 cospid32(_Decimal32 x);
_Decimal64 cospid64(_Decimal64 x);
_Decimal128 cospid128(_Decimal128 x);
#endif
```

Description

The **cospi** functions compute the cosine of $\pi \times \mathbf{x}$, thus regarding **x** as a measurement in half-revolutions.

Returns

The **cospi** functions return $\cos(\pi \times \mathbf{x})$.

7.12.5.13 The `sinpi` functions

Synopsis

```
#include <math.h>
double sinpi(double x);
float sinpif(float x);
long double sinpil(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 sinpid32(_Decimal32 x);
_Decimal64 sinpid64(_Decimal64 x);
_Decimal128 sinpid128(_Decimal128 x);
#endif
```

Description

The `sinpi` functions compute the sine of $\pi \times x$, thus regarding x as a measurement in half-revolutions. A range error occurs if nonzero x is too close to zero.

Returns

The `sinpi` functions return $\sin(\pi \times x)$.

7.12.5.14 The `tanpi` functions

Synopsis

```
#include <math.h>
double tanpi(double x);
float tanpif(float x);
long double tanpil(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 tanpid32(_Decimal32 x);
_Decimal64 tanpid64(_Decimal64 x);
_Decimal128 tanpid128(_Decimal128 x);
#endif
```

Description

The `tanpi` functions compute the tangent of $\pi \times x$, thus regarding x as a measurement in half-revolutions. A range error occurs if nonzero x is too close to zero. A pole error may occur if $|x|$ is $(n + 0.5)$ for integer n .

Returns

The `tanpi` functions return $\tan(\pi \times x)$.

7.12.6 Hyperbolic functions

7.12.6.1 The `acosh` functions

Synopsis

```
#include <math.h>
double acosh(double x);
float acoshf(float x);
long double acoshl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 acoshd32(_Decimal32 x);
_Decimal64 acoshd64(_Decimal64 x);
_Decimal128 acoshd128(_Decimal128 x);
#endif
```

Description

The `acosh` functions compute the (nonnegative) arc hyperbolic cosine of x . A domain error occurs for arguments less than 1.

Returns

The **acosh** functions return $\operatorname{arcosh} x$ in the interval $[0, +\infty]$.

7.12.6.2 The **asinh** functions

Synopsis

```
#include <math.h>
double asinh(double x);
float asinhf(float x);
long double asinhl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 asinhd32(_Decimal32 x);
_Decimal64 asinhd64(_Decimal64 x);
_Decimal128 asinhd128(_Decimal128 x);
#endif
```

Description

The **asinh** functions compute the arc hyperbolic sine of **x**. A range error occurs if nonzero **x** is too close to zero.

Returns

The **asinh** functions return $\operatorname{arsinh} x$.

7.12.6.3 The **atanh** functions

Synopsis

```
#include <math.h>
double atanh(double x);
float atanhf(float x);
long double atanhhl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 atanhhd32(_Decimal32 x);
_Decimal64 atanhhd64(_Decimal64 x);
_Decimal128 atanhhd128(_Decimal128 x);
#endif
```

Description

The **atanh** functions compute the arc hyperbolic tangent of **x**. A domain error occurs for arguments not in the interval $[-1, +1]$. A pole error may occur if the argument equals **-1** or **+1**. A range error occurs if nonzero **x** is too close to zero.

Returns

The **atanh** functions return $\operatorname{artanh} x$.

7.12.6.4 The **cosh** functions

Synopsis

```
#include <math.h>
double cosh(double x);
float coshf(float x);
long double coshl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 coshd32(_Decimal32 x);
_Decimal64 coshd64(_Decimal64 x);
_Decimal128 coshd128(_Decimal128 x);
#endif
```

Description

The **cosh** functions compute the hyperbolic cosine of **x**. A range error occurs if the magnitude of finite **x** is too large.

Returns

The **cosh** functions return $\cosh x$.

7.12.6.5 The sinh functions**Synopsis**

```
#include <math.h>
double sinh(double x);
float sinhf(float x);
long double sinhl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 sinhd32(_Decimal32 x);
_Decimal64 sinhd64(_Decimal64 x);
_Decimal128 sinhd128(_Decimal128 x);
#endif
```

Description

The **sinh** functions compute the hyperbolic sine of **x**. A range error occurs if the magnitude of finite **x** is too large or if nonzero **x** is too close to zero.

Returns

The **sinh** functions return $\sinh x$.

7.12.6.6 The tanh functions**Synopsis**

```
#include <math.h>
double tanh(double x);
float tanhf(float x);
long double tanhl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 tanhd32(_Decimal32 x);
_Decimal64 tanhd64(_Decimal64 x);
_Decimal128 tanhd128(_Decimal128 x);
#endif
```

Description

The **tanh** functions compute the hyperbolic tangent of **x**. A range error occurs if nonzero **x** is too close to zero.

Returns

The **tanh** functions return $\tanh x$.

7.12.7 Exponential and logarithmic functions**7.12.7.1 The exp functions****Synopsis**

```
#include <math.h>
double exp(double x);
float expf(float x);
long double expl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 expd32(_Decimal32 x);
_Decimal64 expd64(_Decimal64 x);
_Decimal128 expd128(_Decimal128 x);
```

```
#endif
```

Description

The **exp** functions compute the base-*e* exponential of **x**. A range error occurs if the magnitude of finite **x** is too large.

Returns

The **exp** functions return e^x .

7.12.7.2 The **exp10** functions

Synopsis

```
#include <math.h>
double exp10(double x);
float exp10f(float x);
long double exp10l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 exp10d32(_Decimal32 x);
_Decimal64 exp10d64(_Decimal64 x);
_Decimal128 exp10d128(_Decimal128 x);
#endif
```

Description

The **exp10** functions compute the base-10 exponential of **x**. A range error occurs if the magnitude of finite **x** is too large.

Returns

The **exp10** functions return 10^x .

7.12.7.3 The **exp10m1** functions

Synopsis

```
#include <math.h>
double exp10m1(double x);
float exp10m1f(float x);
long double exp10m1l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 exp10m1d32(_Decimal32 x);
_Decimal64 exp10m1d64(_Decimal64 x);
_Decimal128 exp10m1d128(_Decimal128 x);
#endif
```

Description

The **exp10m1** functions compute the base-10 exponential of the argument, minus 1. A range error occurs if positive finite **x** is too large or if nonzero **x** is too close to zero.

Returns

The **exp10m1** functions return $10^x - 1$.

7.12.7.4 The **exp2** functions

Synopsis

```
#include <math.h>
double exp2(double x);
float exp2f(float x);
long double exp2l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 exp2d32(_Decimal32 x);
_Decimal64 exp2d64(_Decimal64 x);
```

```
_Decimal128 exp2d128(_Decimal128 x);
#endif
```

Description

The **exp2** functions compute the base-2 exponential of **x**. A range error occurs if the magnitude of finite **x** is too large.

Returns

The **exp2** functions return 2^x .

7.12.7.5 The exp2m1 functions

Synopsis

```
#include <math.h>
double exp2m1(double x);
float exp2m1f(float x);
long double exp2m1l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 exp2m1d32(_Decimal32 x);
_Decimal64 exp2m1d64(_Decimal64 x);
_Decimal128 exp2m1d128(_Decimal128 x);
#endif
```

Description

The **exp2m1** functions compute the base-2 exponential of the argument, minus 1. A range error occurs if positive finite **x** is too large or if nonzero **x** is too close to zero.

Returns

The **exp2m1** functions return $2^x - 1$.

7.12.7.6 The expm1 functions

Synopsis

```
#include <math.h>
double expm1(double x);
float expm1f(float x);
long double expm1l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 expm1d32(_Decimal32 x);
_Decimal64 expm1d64(_Decimal64 x);
_Decimal128 expm1d128(_Decimal128 x);
#endif
```

Description

The **expm1** functions compute the base-*e* exponential of the argument, minus 1. A range error occurs if positive finite **x** is too large or if nonzero **x** is too close to zero.²⁷³⁾

Returns

The **expm1** functions return $e^x - 1$.

7.12.7.7 The frexp functions

Synopsis

```
#include <math.h>
double frexp(double value, int *p);
float frexpf(float value, int *p);
long double frexpl(long double value, int *p);
```

²⁷³⁾For small magnitude **x**, **expm1(x)** is expected to be more accurate than **exp(x) - 1**.

```
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 frexpd32(_Decimal32 value, int *p);
_Decimal64 frexpd64(_Decimal64 value, int *p);
_Decimal128 frexpd128(_Decimal128 value, int *p);
#endif
```

Description

The **frexp** functions break a floating-point number into a normalized fraction and an integer exponent. They store the integer in the **int** object pointed to by **p**. If the return type of the function is a standard floating type, the exponent is an integral power of 2. If the return type of the function is a decimal floating type, the exponent is an integral power of 10.

Returns

If **value** is not a floating-point number or if the integral power is outside the range of **int**, the results are unspecified. Otherwise, the **frexp** functions return the value **x**, such that **x** has a magnitude in the interval $[\frac{1}{2}, 1)$ or zero, and **value** equals $x \times 2^{*p}$, when the return type of the function is a standard floating type; or **x** has a magnitude in the interval $[1/10, 1)$ or zero, and **value** equals $x \times 10^{*p}$, when the return type of the function is a decimal floating type. If **value** is zero, both parts of the result are zero.

7.12.7.8 The **ilogb** functions

Synopsis

```
#include <math.h>
int ilogb(double x);
int ilogbf(float x);
int ilogbl(long double x);
#ifdef __STDC_IEC_60559_DFP__
int ilogbd32(_Decimal32 x);
int ilogbd64(_Decimal64 x);
int ilogbd128(_Decimal128 x);
#endif
```

Description

The **ilogb** functions extract the exponent of **x** as a signed **int** value. If **x** is zero they compute the value **FP_ILOGB0**; if **x** is infinite they compute the value **INT_MAX**; if **x** is a NaN they compute the value **FP_ILOGBNAN**; otherwise, they are equivalent to calling the corresponding **logb** function and converting the returned value to type **int**. A domain error or range error may occur if **x** is zero, infinite, or NaN. If the correct value is outside the range of the return type, the numeric result is unspecified and a domain error or range error may occur.

Returns

The **ilogb** functions return the exponent of **x** as a signed **int** value.

Forward references: the **logb** functions (7.12.7.17).

7.12.7.9 The **ldexp** functions

Synopsis

```
#include <math.h>
double ldexp(double x, int p);
float ldexpf(float x, int p);
long double ldexpl(long double x, int p);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 ldexpd32(_Decimal32 x, int p);
_Decimal64 ldexpd64(_Decimal64 x, int p);
_Decimal128 ldexpd128(_Decimal128 x, int p);
#endif
```

Description

The **ldexp** functions multiply a floating-point number by an integral power of 2 when the return type of the function is a standard floating type, or by an integral power of 10 when the return type of the function is a decimal floating type. A range error occurs for some finite **x**, depending on **p**.

Returns

The **ldexp** functions return $x \times 2^p$ when the return type of the function is a standard floating type, or return $x \times 10^p$ when the return type of the function is a decimal floating type.

7.12.7.10 The llogb functions**Synopsis**

```
#include <math.h>
long int llogb(double x);
long int llogbf(float x);
long int llogbl(long double x);
#ifdef __STDC_IEC_60559_DFP__
long int llogbd32(_Decimal32 x);
long int llogbd64(_Decimal64 x);
long int llogbd128(_Decimal128 x);
#endif
```

Description

The **llogb** functions extract the exponent of **x** as a signed **long int** value. If **x** is zero they compute the value **FP_LLOGB0**; if **x** is infinite they compute the value **LONG_MAX**; if **x** is a NaN they compute the value **FP_LLOGBNAN**; otherwise, they are equivalent to calling the corresponding **logb** function and converting the returned value to type **long int**. A domain error or range error may occur if **x** is zero, infinite, or NaN. If the correct value is outside the range of the return type, the numeric result is unspecified.

Returns

The **llogb** functions return the exponent of **x** as a signed **long int** value.

Forward references: the **logb** functions (7.12.7.17).

7.12.7.11 The log functions**Synopsis**

```
#include <math.h>
double log(double x);
float logf(float x);
long double logl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 logd32(_Decimal32 x);
_Decimal64 logd64(_Decimal64 x);
_Decimal128 logd128(_Decimal128 x);
#endif
```

Description

The **log** functions compute the base-*e* (natural) logarithm of **x**. A domain error occurs if the argument is less than zero. A pole error may occur if the argument is zero.

Returns

The **log** functions return $\log_e x$.

7.12.7.12 The log10 functions**Synopsis**

```
#include <math.h>
```

```
double log10(double x);
float log10f(float x);
long double log10l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 log10d32(_Decimal32 x);
_Decimal64 log10d64(_Decimal64 x);
_Decimal128 log10d128(_Decimal128 x);
#endif
```

Description

The **log10** functions compute the base-10 (common) logarithm of **x**. A domain error occurs if the argument is less than zero. A pole error may occur if the argument is zero.

Returns

The **log10** functions return $\log_{10} x$.

7.12.7.13 The log10p1 functions

Synopsis

```
#include <math.h>
double log10p1(double x);
float log10p1f(float x);
long double log10p1l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 log10p1d32(_Decimal32 x);
_Decimal64 log10p1d64(_Decimal64 x);
_Decimal128 log10p1d128(_Decimal128 x);
#endif
```

Description

The **log10p1** functions compute the base-10 logarithm of 1 plus the argument. A domain error occurs if the argument is less than -1 . A pole error may occur if the argument equals -1 . A range error occurs if nonzero **x** is too close to zero.

Returns

The **log10p1** functions return $\log_{10}(1 + x)$.

7.12.7.14 The log1p and logp1 functions

Synopsis

```
#include <math.h>
double log1p(double x);
float log1pf(float x);
long double log1pl(long double x);
double logp1(double x);
float logp1f(float x);
long double logp1l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 log1pd32(_Decimal32 x);
_Decimal64 log1pd64(_Decimal64 x);
_Decimal128 log1pd128(_Decimal128 x);
_Decimal32 logp1d32(_Decimal32 x);
_Decimal64 logp1d64(_Decimal64 x);
_Decimal128 logp1d128(_Decimal128 x);
#endif
```


Description

The **log1p** functions are equivalent to the **logp1** functions.²⁷⁴⁾ These functions compute the base-*e* (natural) logarithm of 1 plus the argument.²⁷⁵⁾ A domain error occurs if the argument is less than -1 . A pole error may occur if the argument equals -1 . A range error occurs if nonzero *x* is too close to zero.

Returns

The **log1p** and **logp1** functions return $\log_e(1 + x)$.

7.12.7.15 The log2 functions**Synopsis**

```
#include <math.h>
double log2(double x);
float log2f(float x);
long double log2l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 log2d32(_Decimal32 x);
_Decimal64 log2d64(_Decimal64 x);
_Decimal128 log2d128(_Decimal128 x);
#endif
```

Description

The **log2** functions compute the base-2 logarithm of *x*. A domain error occurs if the argument is less than zero. A pole error may occur if the argument is zero.

Returns

The **log2** functions return $\log_2 x$.

7.12.7.16 The log2p1 functions**Synopsis**

```
#include <math.h>
double log2p1(double x);
float log2p1f(float x);
long double log2p1l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 log2p1d32(_Decimal32 x);
_Decimal64 log2p1d64(_Decimal64 x);
_Decimal128 log2p1d128(_Decimal128 x);
#endif
```

Description

The **log2p1** functions compute the base-2 logarithm of 1 plus the argument. A domain error occurs if the argument is less than -1 . A pole error may occur if the argument equals -1 . A range error occurs if nonzero *x* is too close to zero.

Returns

The **log2p1** functions return $\log_2(1+x)$.

7.12.7.17 The logb functions**Synopsis**

```
#include <math.h>
double logb(double x);
float logbf(float x);
```

²⁷⁴⁾The **logp1** functions are preferred for name consistency with the **log10p1** and **log2p1** functions.

²⁷⁵⁾For small magnitude *x*, **logp1**(*x*) is expected to be more accurate than **log**(1 + *x*).

```

long double logbl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 logbd32(_Decimal32 x);
_Decimal64 logbd64(_Decimal64 x);
_Decimal128 logbd128(_Decimal128 x);
#endif

```

Description

The **logb** functions extract the exponent of **x**, as a signed integer value in floating-point format. If **x** is subnormal it is treated as though it were normalized; thus, for positive finite **x**,

$$1 \leq x \times b^{-\text{logb}(x)} < b$$

where $b = \text{FLT_RADIX}$ if the return type of the function is a standard floating type, or $b = 10$ if the return type of the function is a decimal floating type. A domain error or pole error may occur if the argument is zero.

Returns

The **logb** functions return the signed exponent of **x**.

7.12.7.18 The modf functions

Synopsis

```

#include <math.h>
double modf(double value, double *iptr);
float modff(float value, float *iptr);
long double modfl(long double value, long double *iptr);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 modfd32(_Decimal32 x, _Decimal32 *iptr);
_Decimal64 modfd64(_Decimal64 x, _Decimal64 *iptr);
_Decimal128 modfd128(_Decimal128 x, _Decimal128 *iptr);
#endif

```

Description

The **modf** functions break the argument **value** into integral and fractional parts, each of which has the same type and sign as the argument. They store the integral part (in floating-point format) in the object pointed to by **iptr**.

Returns

The **modf** functions return the signed fractional part of **value**.

7.12.7.19 The scalbn and scalbln functions

Synopsis

```

#include <math.h>
double scalbn(double x, int n);
float scalbnf(float x, int n);
long double scalbnl(long double x, int n);
double scalbln(double x, long int n);
float scalblnf(float x, long int n);
long double scalblnl(long double x, long int n);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 scalbnd32(_Decimal32 x, int n);
_Decimal64 scalbnd64(_Decimal64 x, int n);
_Decimal128 scalbnd128(_Decimal128 x, int n);
_Decimal32 scalblnd32(_Decimal32 x, long int n);
_Decimal64 scalblnd64(_Decimal64 x, long int n);
_Decimal128 scalblnd128(_Decimal128 x, long int n);
#endif

```

Description

The **scalbn** and **scalbln** functions compute $x \times b^n$, where $b = \text{FLT_RADIX}$ if the return type of the function is a standard floating type, or $b = 10$ if the return type of the function is a decimal floating type. A range error occurs for some finite x , depending on n .

Returns

The **scalbn** and **scalbln** functions return $x \times b^n$.

7.12.8 Power and absolute-value functions**7.12.8.1 The cbrt functions****Synopsis**

```
#include <math.h>
double cbrt(double x);
float cbrtf(float x);
long double cbrtl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 cbrtd32(_Decimal32 x);
_Decimal64 cbrtd64(_Decimal64 x);
_Decimal128 cbrtd128(_Decimal128 x);
#endif
```

Description

The **cbrt** functions compute the real cube root of x .

Returns

The **cbrt** functions return $x^{\frac{1}{3}}$.

7.12.8.2 The compoundn functions**Synopsis**

```
#include <math.h>
double compoundn(double x, long long int n);
float compoundnf(float x, long long int n);
long double compoundnl(long double x, long long int n);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 compoundnd32(_Decimal32 x, long long int n);
_Decimal64 compoundnd64(_Decimal64 x, long long int n);
_Decimal128 compoundnd128(_Decimal128 x, long long int n);
#endif
```

Description

The **compoundn** functions compute 1 plus x , raised to the power n . A domain error occurs if $x < -1$. Depending on n , a range error occurs if either positive finite x is too large or if x is too near but not equal to -1 . A pole error may occur if x equals -1 and $n < 0$.

Returns

The **compoundn** functions return $(1 + x)^n$.

7.12.8.3 The fabs functions**Synopsis**

```
#include <math.h>
double fabs(double x);
float fabsf(float x);
long double fabsl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fabsd32(_Decimal32 x);
```

```
_Decimal64 fabsd64(_Decimal64 x);
_Decimal128 fabsd128(_Decimal128 x);
#endif
```

Description
The **fabs** functions compute the absolute value of **x**.

Returns
The **fabs** functions return $|x|$.

7.12.8.4 The hypot functions

Synopsis

```
#include <math.h>
double hypot(double x, double y);
float hypotf(float x, float y);
long double hypotl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 hypotd32(_Decimal32 x, _Decimal32 y);
_Decimal64 hypotd64(_Decimal64 x, _Decimal64 y);
_Decimal128 hypotd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description
The **hypot** functions compute the square root of the sum of the squares of **x** and **y**, without undue overflow or underflow. A range error occurs for some finite arguments.

Returns
The **hypot** functions return $\sqrt{x^2 + y^2}$.

7.12.8.5 The pow functions

Synopsis

```
#include <math.h>
double pow(double x, double y);
float powf(float x, float y);
long double powl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 powd32(_Decimal32 x, _Decimal32 y);
_Decimal64 powd64(_Decimal64 x, _Decimal64 y);
_Decimal128 powd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description
The **pow** functions compute **x** raised to the power **y**. A domain error occurs if **x** is finite and less than zero and **y** is finite and not an integer value. A domain error may occur if **x** is zero and **y** is zero. Depending on **y**, a range error occurs if either the magnitude of nonzero finite **x** is too large or too near zero. A domain error or pole error may occur if **x** is zero and **y** is less than zero.

Returns
The **pow** functions return x^y .

7.12.8.6 The pown functions

Synopsis

```
#include <math.h>
double pown(double x, long long int n);
float pownf(float x, long long int n);
```

```

long double pownl(long double x, long long int n);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 pownd32(_Decimal32 x, long long int n);
_Decimal64 pownd64(_Decimal64 x, long long int n);
_Decimal128 pownd128(_Decimal128 x, long long int n);
#endif

```

Description

The **pown** functions compute x raised to the n^{th} power. A pole error may occur if x equals 0 and $n < 0$. Depending on n , a range error occurs if either the magnitude of nonzero finite x is too large or too near zero.

Returns

The **pown** functions return x^n .

7.12.8.7 The powr functions

Synopsis

```

#include <math.h>
double powr(double y, double x);
float powrf(float y, float x);
long double powrl(long double y, long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 powrd32(_Decimal32 y, _Decimal32 x);
_Decimal64 powrd64(_Decimal64 y, _Decimal64 x);
_Decimal128 powrd128(_Decimal128 y, _Decimal128 x);
#endif

```

Description

The **powr** functions compute x raised to the power y as $e^{y \log_e x}$.²⁷⁶⁾ A domain error occurs if $x < 0$ or if x and y are both zero. Depending on y , a range error occurs if either positive nonzero finite x is too large or too near zero. A pole error may occur if x equals zero and finite $y < 0$.

Returns

The **powr** functions return $e^{y \log_e x}$.

7.12.8.8 The rootn functions

Synopsis

```

#include <math.h>
double rootn(double x, long long int n);
float rootnf(float x, long long int n);
long double rootnl(long double x, long long int n);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 rootnd32(_Decimal32 x, long long int n);
_Decimal64 rootnd64(_Decimal64 x, long long int n);
_Decimal128 rootnd128(_Decimal128 x, long long int n);
#endif

```

Description

The **rootn** functions compute the principal n^{th} root of x . A domain error occurs if n is 0 or if $x < 0$ and n is even. If n is -1 , a range error occurs if either the magnitude of nonzero finite x is too large or too near zero. A pole error may occur if x equals zero and $n < 0$.

²⁷⁶⁾ Restricting the domain to that of the formula $e^{y \log_e x}$ is intended to better meet expectations for a continuous power function and to allow implementations with fewer tests for special cases.

Returns

The **rootn** functions return $x^{\frac{1}{n}}$.

7.12.8.9 The **rsqrt** functions

Synopsis

```
#include <math.h>
double rsqrt(double x);
float rsqrtf(float x);
long double rsqrtl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 rsqrt32(_Decimal32 x);
_Decimal64 rsqrt64(_Decimal64 x);
_Decimal128 rsqrt128(_Decimal128 x);
#endif
```

Description

The **rsqrt** functions compute the reciprocal of the nonnegative square root of the argument. A domain error occurs if the argument is less than zero. A pole error may occur if the argument equals zero.

Returns

The **rsqrt** functions return $\frac{1}{\sqrt{x}}$.

7.12.8.10 The **sqrt** functions

Synopsis

```
#include <math.h>
double sqrt(double x);
float sqrtf(float x);
long double sqrtl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 sqrt32(_Decimal32 x);
_Decimal64 sqrt64(_Decimal64 x);
_Decimal128 sqrt128(_Decimal128 x);
#endif
```

Description

The **sqrt** functions compute the nonnegative square root of **x**. A domain error occurs if the argument is less than zero.

Returns

The **sqrt** functions return $\sqrt{x}$.

7.12.9 Error and gamma functions

7.12.9.1 The **erf** functions

Synopsis

```
#include <math.h>
double erf(double x);
float erff(float x);
long double erfl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 erfd32(_Decimal32 x);
_Decimal64 erfd64(_Decimal64 x);
_Decimal128 erfd128(_Decimal128 x);
#endif
```

Description

The **erf** functions compute the error function of **x**. A range error occurs if nonzero **x** is too close to zero.

Returns

The **erf** functions return $\text{erf } x = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$.

7.12.9.2 The erfc functions**Synopsis**

```
#include <math.h>
double erfc(double x);
float erfcf(float x);
long double erfcl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 erfc32(_Decimal32 x);
_Decimal64 erfc64(_Decimal64 x);
_Decimal128 erfc128(_Decimal128 x);
#endif
```

Description

The **erfc** functions compute the complementary error function of **x**. A range error occurs if positive finite **x** is too large.

Returns

The **erfc** functions return $\text{erfc } x = 1 - \text{erf } x = \frac{2}{\sqrt{\pi}} \int_x^\infty e^{-t^2} dt$.

7.12.9.3 The lgamma functions**Synopsis**

```
#include <math.h>
double lgamma(double x);
float lgammaf(float x);
long double lgammal(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 lgammad32(_Decimal32 x);
_Decimal64 lgammad64(_Decimal64 x);
_Decimal128 lgammad128(_Decimal128 x);
#endif
```

Description

The **lgamma** functions compute the natural logarithm of the absolute value of gamma of **x**. A range error occurs if positive finite **x** is too large. A pole error may occur if **x** is a negative integer or zero.

Returns

The **lgamma** functions return $\log_e |\Gamma(x)|$.

7.12.9.4 The tgamma functions**Synopsis**

```
#include <math.h>
double tgamma(double x);
float tgammaf(float x);
long double tgammal(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 tgam32(_Decimal32 x);
_Decimal64 tgam64(_Decimal64 x);
#endif
```



```
_Decimal128 tgamma128(_Decimal128 x);
#endif
```

Description

The **tgamma** functions compute the gamma function of **x**. A domain error or pole error may occur if **x** is a negative integer or zero. A range error occurs for some finite **x** less than zero, if positive finite **x** is too large, or nonzero **x** is too close to zero.

Returns

The **tgamma** functions return $\Gamma(x)$.

7.12.10 Nearest integer functions

7.12.10.1 The **ceil** functions

Synopsis

```
#include <math.h>
double ceil(double x);
float ceilf(float x);
long double ceill(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 ceild32(_Decimal32 x);
_Decimal64 ceild64(_Decimal64 x);
_Decimal128 ceild128(_Decimal128 x);
#endif
```

Description

The **ceil** functions compute the smallest integer value not less than **x**.

Returns

The **ceil** functions return $\lceil x \rceil$, expressed as a floating-point number.

7.12.10.2 The **floor** functions

Synopsis

```
#include <math.h>
double floor(double x);
float floorf(float x);
long double floorl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 floord32(_Decimal32 x);
_Decimal64 floord64(_Decimal64 x);
_Decimal128 floord128(_Decimal128 x);
#endif
```

Description

The **floor** functions compute the largest integer value not greater than **x**.

Returns

The **floor** functions return $\lfloor x \rfloor$, expressed as a floating-point number.

7.12.10.3 The **nearbyint** functions

Synopsis

```
#include <math.h>
double nearbyint(double x);
float nearbyintf(float x);
long double nearbyintl(long double x);
#ifdef __STDC_IEC_60559_DFP__
```

```

_Decimal32 nearbyintd32(_Decimal32 x);
_Decimal64 nearbyintd64(_Decimal64 x);
_Decimal128 nearbyintd128(_Decimal128 x);
#endif

```

Description

The **nearbyint** functions round their argument to an integer value in floating-point format, using the current rounding direction and without raising the “inexact” floating-point exception.

Returns

The **nearbyint** functions return the rounded integer value.

7.12.10.4 The rint functions

Synopsis

```

#include <math.h>
double rint(double x);
float rintf(float x);
long double rintl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 rintd32(_Decimal32 x);
_Decimal64 rintd64(_Decimal64 x);
_Decimal128 rintd128(_Decimal128 x);
#endif

```

Description

The **rint** functions differ from the **nearbyint** functions (7.12.10.3) only in that the **rint** functions may raise the “inexact” floating-point exception if the result differs in value from the argument.

Returns

The **rint** functions return the rounded integer value.

7.12.10.5 The lrint and llrint functions

Synopsis

```

#include <math.h>
long int lrint(double x);
long int lrintf(float x);
long int lrintl(long double x);
long long int llrint(double x);
long long int llrintf(float x);
long long int llrintl(long double x);
#ifdef __STDC_IEC_60559_DFP__
long int lrintd32(_Decimal32 x);
long int lrintd64(_Decimal64 x);
long int lrintd128(_Decimal128 x);
long long int llrintd32(_Decimal32 x);
long long int llrintd64(_Decimal64 x);
long long int llrintd128(_Decimal128 x);
#endif

```

Description

The **lrint** and **llrint** functions round their argument to the nearest integer value, rounding according to the current rounding direction. If the rounded value is outside the range of the return type, the numeric result is unspecified and a domain error or range error may occur.

Returns

The **lrint** and **llrint** functions return the rounded integer value.

7.12.10.6 The round functions

Synopsis

```
#include <math.h>
double round(double x);
float roundf(float x);
long double roundl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 roundd32(_Decimal32 x);
_Decimal64 roundd64(_Decimal64 x);
_Decimal128 roundd128(_Decimal128 x);
#endif
```

Description

The **round** functions round their argument to the nearest integer value in floating-point format, rounding halfway cases away from zero, regardless of the current rounding direction.

Returns

The **round** functions return the rounded integer value.

7.12.10.7 The lround and llround functions

Synopsis

```
#include <math.h>
long int lround(double x);
long int lroundf(float x);
long int lroundl(long double x);
long long int llround(double x);
long long int llroundf(float x);
long long int llroundl(long double x);
#ifdef __STDC_IEC_60559_DFP__
long int lroundd32(_Decimal32 x);
long int lroundd64(_Decimal64 x);
long int lroundd128(_Decimal128 x);
long long int llroundd32(_Decimal32 x);
long long int llroundd64(_Decimal64 x);
long long int llroundd128(_Decimal128 x);
#endif
```

Description

The **lround** and **llround** functions round their argument to the nearest integer value, rounding halfway cases away from zero, regardless of the current rounding direction. If the rounded value is outside the range of the return type, the numeric result is unspecified and a domain error or range error may occur.

Returns

The **lround** and **llround** functions return the rounded integer value.

7.12.10.8 The roundeven functions

Synopsis

```
#include <math.h>
double roundeven(double x);
float roundevenf(float x);
long double roundevenl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 roundevend32(_Decimal32 x);
_Decimal64 roundevend64(_Decimal64 x);
_Decimal128 roundevend128(_Decimal128 x);
#endif
```

Description

The **roundeven** functions round their argument to the nearest integer value in floating-point format, rounding halfway cases to even (that is, to the nearest value that is an even integer), regardless of the current rounding direction.

Returns

The **roundeven** functions return the rounded integer value.

7.12.10.9 The trunc functions

Synopsis

```
#include <math.h>
double trunc(double x);
float truncf(float x);
long double trunc_l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 truncd32(_Decimal32 x);
_Decimal64 truncd64(_Decimal64 x);
_Decimal128 truncd128(_Decimal128 x);
#endif
```

Description

The **trunc** functions round their argument to the integer value, in floating format, nearest to but no larger in magnitude than the argument.

Returns

The **trunc** functions return the truncated integer value.

7.12.10.10 The fromfp and ufromfp functions

Synopsis

```
#include <math.h>
double fromfp(double x, int rnd, unsigned int width);
float fromfpf(float x, int rnd, unsigned int width);
long double fromfpl(long double x, int rnd, unsigned int width);
double ufromfp(double x, int rnd, unsigned int width);
float ufromfpf(float x, int rnd, unsigned int width);
long double ufromfpl(long double x, int rnd, unsigned int width);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fromfpd32(_Decimal32 x, int rnd, unsigned int width);
_Decimal64 fromfpd64(_Decimal64 x, int rnd, unsigned int width);
_Decimal128 fromfpd128(_Decimal128 x, int rnd, unsigned int width);
_Decimal32 ufromfpd32(_Decimal32 x, int rnd, unsigned int width);
_Decimal64 ufromfpd64(_Decimal64 x, int rnd, unsigned int width);
_Decimal128 ufromfpd128(_Decimal128 x, int rnd, unsigned int width);
#endif
```

Description

The **fromfp** and **ufromfp** functions round **x**, using the math rounding direction indicated by **rnd**, to a signed or unsigned integer, respectively. If **width** is nonzero and the resulting integer is within the range

- $[-2^{(\text{width}-1)}, 2^{(\text{width}-1)} - 1]$, for signed
- $[0, 2^{\text{width}} - 1]$, for unsigned

then the functions return the integer value (represented in floating type). Otherwise, if **width** is zero or **x** does not round to an integer within the range, the functions return a NaN (of the type of the **x** argument, if available), else the value of **x**, and a domain error occurs. If the value of the **rnd** argument is not equal to the value of a math rounding direction macro (7.12), the direction of rounding is unspecified. The **fromfp** and **ufromfp** functions do not raise the “inexact” floating-point exception.

Returns

The **fromfp** and **ufromfp** functions return the rounded integer value.

EXAMPLE 1 Upward rounding of **double x** to type **int**, without raising the “inexact” floating-point exception, is achieved by

```
(int)fromfp(x, FP_INT_UPWARD, INT_WIDTH)
```

EXAMPLE 2 Unsigned integer wrapping is not performed in

```
ufromfp(-3.0, FP_INT_UPWARD, UINT_WIDTH) /* domain error */
```

7.12.10.11 The **fromfpx** and **ufromfpx** functions

Synopsis

```
#include <math.h>
double fromfpx(double x, int rnd, unsigned int width);
float fromfpxf(float x, int rnd, unsigned int width);
long double fromfpxl(long double x, int rnd, unsigned int width);
double ufromfpx(double x, int rnd, unsigned int width);
float ufromfpxf(float x, int rnd, unsigned int width);
long double ufromfpxl(long double x, int rnd, unsigned int width);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fromfpxd32(_Decimal32 x, int rnd, unsigned int width);
_Decimal64 fromfpxd64(_Decimal64 x, int rnd, unsigned int width);
_Decimal128 fromfpxd128(_Decimal128 x, int rnd, unsigned int width);
_Decimal32 ufromfpxd32(_Decimal32 x, int rnd, unsigned int width);
_Decimal64 ufromfpxd64(_Decimal64 x, int rnd, unsigned int width);
_Decimal128 ufromfpxd128(_Decimal128 x, int rnd, unsigned int width);
#endif
```

Description

The **fromfpx** and **ufromfpx** functions differ from the **fromfp** and **ufromfp** functions, respectively, only in that the **fromfpx** and **ufromfpx** functions raise the “inexact” floating-point exception if a rounded result not exceeding the specified width differs in value from the argument **x**.

Returns

The **fromfpx** and **ufromfpx** functions return the rounded integer value.

NOTE Conversions to integer types that are not required to raise the inexact exception can be done simply by rounding to integral value in floating type and then converting to the target integer type. For example, the conversion of **long double x** to **uint64_t**, using upward rounding, is done by

```
(uint64_t)ceil(x)
```

7.12.11 Remainder functions

7.12.11.1 The **fmod** functions

Synopsis

```
#include <math.h>
double fmod(double x, double y);
float fmodf(float x, float y);
```

```

long double fmodl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fmodd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmodd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmodd128(_Decimal128 x, _Decimal128 y);
#endif

```

Description

The **fmod** functions compute the floating-point remainder of **x/y**.

Returns

The **fmod** functions return the value $x - ny$, for some integer n such that, if **y** is nonzero, the result has the same sign as **x** and magnitude less than the magnitude of **y**. If **y** is zero, whether a domain error occurs or the **fmod** functions return zero is implementation-defined.

7.12.11.2 The remainder functions

Synopsis

```

#include <math.h>
double remainder(double x, double y);
float remainderf(float x, float y);
long double remainderl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 remainderd32(_Decimal32 x, _Decimal32 y);
_Decimal64 remainderd64(_Decimal64 x, _Decimal64 y);
_Decimal128 remainderd128(_Decimal128 x, _Decimal128 y);
#endif

```

Description

The **remainder** functions compute the remainder **x REM y** required by ISO/IEC 60559.²⁷⁷⁾

Returns

The **remainder** functions return **x REM y**. If **y** is zero, whether a domain error occurs or the functions return zero is implementation-defined.

7.12.11.3 The remquo functions

Synopsis

```

#include <math.h>
double remquo(double x, double y, int *quo);
float remquof(float x, float y, int *quo);
long double remquol(long double x, long double y, int *quo);

```

Description

The **remquo** functions compute the same remainder as the **remainder** functions. In the object pointed to by **quo** they store a value whose magnitude is congruent modulo 2^n to the magnitude of the integral quotient of **x/y**, where n is an implementation-defined integer greater than or equal to 3. If the value stored is not zero, its sign is the sign of **x/y**.

Returns

The **remquo** functions return **x REM y**. If **y** is zero, the value stored in the object pointed to by **quo** is unspecified and whether a domain error occurs or the functions return zero is implementation-defined.

NOTE There are no decimal floating-point versions of the **remquo** functions.

²⁷⁷⁾When $y \neq 0$, the remainder $r = x \text{ REM } y$ is defined regardless of the rounding mode by the mathematical relation $r = x - ny$, where n is the integer nearest the exact value of $\frac{x}{y}$; whenever $|n - \frac{x}{y}| = \frac{1}{2}$, then n is even. If $r = 0$, its sign shall be that of x . This definition is applicable for all implementations.

7.12.12 Manipulation functions

7.12.12.1 The copysign functions

Synopsis

```
#include <math.h>
double copysign(double x, double y);
float copysignf(float x, float y);
long double copysignl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 copysign32(_Decimal32 x, _Decimal32 y);
_Decimal64 copysign64(_Decimal64 x, _Decimal64 y);
_Decimal128 copysign128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

The **copysign** functions produce a value with the magnitude of **x** and the sign of **y**. If **x** or **y** is an unsigned value, the sign (if any) of the result is implementation-defined. On implementations that represent a signed zero but do not treat negative zero consistently in arithmetic operations, the **copysign** functions should regard the sign of zero as positive.

Returns

The **copysign** functions return a value with the magnitude of **x** and the sign of **y**.

7.12.12.2 The nan functions

Synopsis

```
#include <math.h>
double nan(const char *tagp);
float nanf(const char *tagp);
long double nanl(const char *tagp);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 nand32(const char *tagp);
_Decimal64 nand64(const char *tagp);
_Decimal128 nand128(const char *tagp);
#endif
```

Description

The **nan**, **nanf**, and **nanl** functions convert the string pointed to by **tagp** according to the following rules. The call **nan("n-char-sequence")** is equivalent to **strtod("NAN(n-char-sequence)", nullptr)**; the call **nan("")** is equivalent to **strtod("NAN()", nullptr)**. If **tagp** does not point to an empty string or an n-char sequence, the call is equivalent to **strtod("NAN", nullptr)**. Calls to **nanf** and **nanl** are equivalent to the corresponding calls to **strtof** and **strtold**.

Returns

The **nan** functions return a quiet NaN, if available, with content indicated through **tagp**. If the implementation does not support quiet NaNs, the functions return zero.

Forward references: the **strtod**, **strtof**, and **strtold** functions (7.24.2.6).

7.12.12.3 The nextafter functions

Synopsis

```
#include <math.h>
double nextafter(double x, double y);
float nextafterf(float x, float y);
long double nextafterl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 nextafterd32(_Decimal32 x, _Decimal32 y);
_Decimal64 nextafterd64(_Decimal64 x, _Decimal64 y);
#endif
```



```
_Decimal128 nextafterd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

The **nextafter** functions determine the next representable value, in the return type of the function, after **x** in the direction of **y**, where **x** and **y** are first converted to the return type of the function.²⁷⁸⁾ The **nextafter** functions return **y** if **x** equals **y**.

A range error occurs if the magnitude of **x** is the largest finite value representable in the type and the result is infinite or not representable in the type. If **x** $\neq$ **y**, a range error occurs for either subnormal or zero results.

Returns

The **nextafter** functions return the next representable value in the specified format after **x** in the direction of **y**.

7.12.12.4 The nexttoward functions

Synopsis

```
#include <math.h>
double nexttoward(double x, long double y);
float nexttowardf(float x, long double y);
long double nexttowardl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 nexttowarddd32(_Decimal32 x, _Decimal128 y);
_Decimal64 nexttowarddd64(_Decimal64 x, _Decimal128 y);
_Decimal128 nexttowarddd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

The **nexttoward** functions are equivalent to the **nextafter** functions except that the second parameter has type **long double** or **_Decimal128** and the functions return **y** converted to the return type of the function if **x** equals **y**.²⁷⁹⁾

Returns

The **nexttoward** functions return the next representable value in the specified format after **x** in the direction of **y**.

7.12.12.5 The nextup functions

Synopsis

```
#include <math.h>
double nextup(double x);
float nextupf(float x);
long double nextupl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 nextupd32(_Decimal32 x);
_Decimal64 nextupd64(_Decimal64 x);
_Decimal128 nextupd128(_Decimal128 x);
#endif
```

Description

The **nextup** functions determine the next representable value, in the return type of the function, greater than **x**. If **x** is the negative number of least magnitude in the type of **x**, **nextup(x)** is -0 if the type has signed zeros and is 0 otherwise. If **x** is zero, **nextup(x)** is the positive number of least

²⁷⁸⁾The argument values are converted to the return type of the function, even by a macro implementation of the function.

²⁷⁹⁾The result of the **nexttoward** functions is determined in the return type of the function, without loss of range or precision in a floating second argument.

magnitude in the type of **x**. If **x** is the positive number (finite or infinite) of maximum magnitude in the type, **nextup(x)** is **x**.

Returns

The nextup functions return the next representable value in the specified type greater than **x**.

7.12.12.6 The nextdown functions

Synopsis

```
#include <math.h>
double nextdown(double x);
float nextdownf(float x);
long double nextdownl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 nextdownd32(_Decimal32 x);
_Decimal64 nextdownd64(_Decimal64 x);
_Decimal128 nextdownd128(_Decimal128 x);
#endif
```

Description

The **nextdown** functions determine the next representable value, in the return type of the function, less than **x**. If **x** is the positive number of least magnitude in the type of **x**, **nextdown(x)** is **+0** if the type has signed zeros and is **0** otherwise. If **x** is zero, **nextdown(x)** is the negative number of least magnitude in the type of **x**. If **x** is the negative number (finite or infinite) of maximum magnitude in the type, **nextdown(x)** is **x**.

Returns

The **nextdown** functions return the next representable value in the specified type less than **x**.

7.12.12.7 The canonicalize functions

Synopsis

```
#include <math.h>
int canonicalize(double *cx, const double *x);
int canonicalizef(float *cx, const float *x);
int canonicalizel(long double *cx, const long double *x);
#ifdef __STDC_IEC_60559_DFP__
int canonicalized32(_Decimal32 *cx, const _Decimal32 *x);
int canonicalized64(_Decimal64 *cx, const _Decimal64 *x);
int canonicalized128(_Decimal128 *cx, const _Decimal128 *x);
#endif
```

Description

The **canonicalize** functions attempt to produce a canonical version of the floating-point representation in the object pointed to by the argument **x**, as if to a temporary object of the specified type, and store the canonical result in the object pointed to by the argument **cx**.²⁸⁰⁾ If the input ***x** is a signaling NaN, the **canonicalize** functions are intended to store a canonical quiet NaN. If a canonical result is not produced the object pointed to by **cx** is unchanged.

Returns

The **canonicalize** functions return zero if a canonical result is stored in the object pointed to by **cx**. Otherwise they return a nonzero value.

7.12.13 Maximum, minimum, and positive difference functions

7.12.13.1 The fdim functions

²⁸⁰⁾Arguments **x** and **cx** can point to the same object.

Synopsis

```
#include <math.h>
double fdim(double x, double y);
float fdimf(float x, float y);
long double fdiml(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fdimd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fdimd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fdimd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

The **fdim** functions determine the *positive difference* between their arguments:

$$\begin{cases} x - y & \text{if } x > y \\ +0 & \text{if } x \leq y \end{cases}$$

A range error occurs for some finite arguments.

Returns

The **fdim** functions return the positive difference value.

7.12.13.2 The **fmax** functions

Synopsis

```
#include <math.h>
double fmax(double x, double y);
float fmaxf(float x, float y);
long double fmaxl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fmaxd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaxd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaxd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

The **fmax** functions determine the maximum numeric value of their arguments.²⁸¹⁾

Returns

The **fmax** functions return the maximum numeric value of their arguments.

7.12.13.3 The **fmin** functions

Synopsis

```
#include <math.h>
double fmin(double x, double y);
float fminf(float x, float y);
long double fminl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fmind32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmind64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmind128(_Decimal128 x, _Decimal128 y);
#endif
```

²⁸¹⁾Quiet NaN arguments are treated as missing data: if one argument is a quiet NaN and the other numeric, then the **fmax** functions choose the numeric value. See F.10.10.2.

Description

The **fmin** functions determine the minimum numeric value of their arguments.²⁸²⁾

Returns

The **fmin** functions return the minimum numeric value of their arguments.

NOTE The **fmax** and **fmin** functions are similar to the **fmaximum_num** and **fminimum_num** functions, though can differ in which signed zero is returned when the arguments are differently signed zeros and in their treatment of signaling NaNs (see F.10.10.5).

7.12.13.4 The **fmaximum** functions

Synopsis

```
#include <math.h>
double fmaximum(double x, double y);
float fmaximumf(float x, float y);
long double fmaximuml(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fmaximumd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaximumd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaximumd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

The **fmaximum** functions determine the maximum value of their arguments. For these functions, +0 is considered greater than −0. These functions differ from the **fmaximum_num** functions only in their treatment of NaN arguments (see F.10.10.4, F.10.10.5).

Returns

The **fmaximum** functions return the maximum value of their arguments.

7.12.13.5 The **fminimum** functions

Synopsis

```
#include <math.h>
double fminimum(double x, double y);
float fminimumf(float x, float y);
long double fminimuml(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fminimumd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fminimumd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fminimumd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

The **fminimum** functions determine the minimum value of their arguments. For these functions, −0 is considered less than +0. These functions differ from the **fminimum_num** functions only in their treatment of NaN arguments (see F.10.10.4, F.10.10.5).

Returns

The **fminimum** functions return the minimum value of their arguments.

7.12.13.6 The **fmaximum_mag** functions

Synopsis

```
#include <math.h>
double fmaximum_mag(double x, double y);
float fmaximum_magf(float x, float y);
```

²⁸²⁾The **fmin** functions are analogous to the **fmax** functions in their treatment of quiet NaNs.

```

long double fmaximum_magl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fmaximum_magd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaximum_magd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaximum_magd128(_Decimal128 x, _Decimal128 y);
#endif

```

Description

The **fmaximum_mag** functions determine the value of the argument of maximum magnitude: **x** if $|x| > |y|$, **y** if $|y| > |x|$, and **fmaximum(x, y)** otherwise. These functions differ from the **fmaximum_mag_num** functions only in their treatment of NaN arguments (see F.10.10.4, F.10.10.5).

Returns

The **fmaximum_mag** functions return the value of the argument of maximum magnitude.

7.12.13.7 The fminimum_mag functions

Synopsis

```

#include <math.h>
double fminimum_mag(double x, double y);
float fminimum_magf(float x, float y);
long double fminimum_magl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fminimum_magd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fminimum_magd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fminimum_magd128(_Decimal128 x, _Decimal128 y);
#endif

```

Description

The **fminimum_mag** functions determine the value of the argument of minimum magnitude: **x** if $|x| < |y|$, **y** if $|y| < |x|$, and **fminimum(x, y)** otherwise. These functions differ from the **fminimum_mag_num** functions only in their treatment of NaN arguments (see F.10.10.4, F.10.10.5).

Returns

The **fminimum_mag** functions return the value of the argument of minimum magnitude.

7.12.13.8 The fmaximum_num functions

Synopsis

```

#include <math.h>
double fmaximum_num(double x, double y);
float fmaximum_numf(float x, float y);
long double fmaximum_numl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fmaximum_numd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaximum_numd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaximum_numd128(_Decimal128 x, _Decimal128 y);
#endif

```

Description

The **fmaximum_num** functions determine the maximum value of their numeric arguments. They determine the number if one argument is a number and the other is a NaN. These functions differ from the **fmaximum** functions only in their treatment of NaN arguments (see F.10.10.4, F.10.10.5).

Returns

The **fmaximum_num** functions return the maximum value of their numeric arguments.

7.12.13.9 The fminimum_num functions

Synopsis

```
#include <math.h>
double fminimum_num(double x, double y);
float fminimum_numf(float x, float y);
long double fminimum_numl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fminimum_numd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fminimum_numd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fminimum_numd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

The **fminimum_num** functions determine the minimum value of their numeric arguments. They determine the number if one argument is a number and the other is a NaN. These functions differ from the **fminimum** functions only in their treatment of NaN arguments (see F.10.10.4, F.10.10.5).

Returns

The **fminimum_num** functions return the minimum value of their numeric arguments.

7.12.13.10 The **fmaximum_mag_num** functions

Synopsis

```
#include <math.h>
double fmaximum_mag_num(double x, double y);
float fmaximum_mag_numf(float x, float y);
long double fmaximum_mag_numl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fmaximum_mag_numd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaximum_mag_numd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaximum_mag_numd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

The **fmaximum_mag_num** functions determine the value of a numeric argument of maximum magnitude. They determine the number if one argument is a number and the other is a NaN. These functions differ from the **fmaximum_mag** functions only in their treatment of NaN arguments (see F.10.10.4, F.10.10.5).

Returns

The **fmaximum_mag_num** functions return the value of a numeric argument of maximum magnitude.

7.12.13.11 The **fminimum_mag_num** functions

Synopsis

```
#include <math.h>
double fminimum_mag_num(double x, double y);
float fminimum_mag_numf(float x, float y);
long double fminimum_mag_numl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fminimum_mag_numd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fminimum_mag_numd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fminimum_mag_numd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

The **fminimum_mag_num** functions determine the value of a numeric argument of minimum magnitude. They determine the number if one argument is a number and the other is a NaN. These

functions differ from the `fminimum_mag` functions only in their treatment of NaN arguments (see F.10.10.4, F.10.10.5).

Returns

The `fminimum_mag_num` functions return the value of a numeric argument of minimum magnitude.

7.12.14 Fused multiply-add

7.12.14.1 The `fma` functions

Synopsis

```
#include <math.h>
double fma(double x, double y, double z);
float fmaf(float x, float y, float z);
long double fmal(long double x, long double y, long double z);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fmad32(_Decimal32 x, _Decimal32 y, _Decimal32 z);
_Decimal64 fmad64(_Decimal64 x, _Decimal64 y, _Decimal64 z);
_Decimal128 fmad128(_Decimal128 x, _Decimal128 y, _Decimal128 z);
#endif
```

Description

The `fma` functions compute $(x \times y) + z$, rounded as one ternary operation: they compute the value (as if) to infinite precision and round once to the return type, according to the current rounding mode. A range error occurs for some finite arguments. A domain error occurs for some infinite arguments.

Returns

The `fma` functions return $(x \times y) + z$, rounded as one ternary operation.

7.12.15 Functions that round result to narrower type

7.12.15.1 General

The functions in this subclause round their results to the return type, which is typically narrower²⁸³⁾ than the parameter types.

7.12.15.2 Add and round to narrower type

Synopsis

```
#include <math.h>
float fadd(double x, double y);
float faddl(long double x, long double y);
double daddl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 d32addd64(_Decimal64 x, _Decimal64 y);
_Decimal32 d32addd128(_Decimal128 x, _Decimal128 y);
_Decimal64 d64addd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

These functions compute the sum of $x + y$, rounded to the return type of the function. They compute the sum (as if) to infinite precision and round once to the return type, according to the current rounding mode. A range error occurs for some finite arguments. A domain error may occur for infinite arguments.

Returns

These functions return the sum of $x + y$, rounded to the return type of the function.

²⁸³⁾In some cases the destination type can sometimes not be narrower than the parameter types. For example, `double` potentially is not narrower than `long double`.

7.12.15.3 Subtract and round to narrower type**Synopsis**

```
#include <math.h>
float fsub(double x, double y);
float fsubl(long double x, long double y);
double dsubl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 d32subd64(_Decimal64 x, _Decimal64 y);
_Decimal32 d32subd128(_Decimal128 x, _Decimal128 y);
_Decimal64 d64subd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

These functions compute the difference of $x - y$, rounded to the return type of the function. They compute the difference (as if) to infinite precision and round once to the return type, according to the current rounding mode. A range error occurs for some finite arguments. A domain error may occur for infinite arguments.

Returns

These functions return the difference of $x - y$, rounded to the return type of the function.

7.12.15.4 Multiply and round to narrower type**Synopsis**

```
#include <math.h>
float fmul(double x, double y);
float fmul1(long double x, long double y);
double dmul1(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 d32muld64(_Decimal64 x, _Decimal64 y);
_Decimal32 d32muld128(_Decimal128 x, _Decimal128 y);
_Decimal64 d64muld128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

These functions compute the product $x \times y$, rounded to the return type of the function. They compute the product (as if) to infinite precision and round once to the return type, according to the current rounding mode. A range error occurs for some finite arguments. A domain error occurs for one infinite argument and one zero argument.

Returns

These functions return the product of $x \times y$, rounded to the return type of the function.

7.12.15.5 Divide and round to narrower type**Synopsis**

```
#include <math.h>
float fdiv(double x, double y);
float fdivl(long double x, long double y);
double ddivl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 d32divd64(_Decimal64 x, _Decimal64 y);
_Decimal32 d32divd128(_Decimal128 x, _Decimal128 y);
_Decimal64 d64divd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

These functions compute the quotient $x \div y$, rounded to the return type of the function. They compute the quotient (as if) to infinite precision and round once to the return type, according to the current rounding mode. A range error occurs for some finite arguments. A domain error occurs for either both arguments infinite or both arguments zero. A pole error occurs for a finite x and a zero y .

Returns

These functions return the quotient $x \div y$, rounded to the return type of the function.

7.12.15.6 Fused multiply-add and round to narrower type**Synopsis**

```
#include <math.h>
float ffsma(double x, double y, double z);
float ffsma(long double x, long double y, long double z);
double dffma(long double x, long double y, long double z);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 d32ffmad64(_Decimal64 x, _Decimal64 y, _Decimal64 z);
_Decimal32 d32ffmad128(_Decimal128 x, _Decimal128 y, _Decimal128 z);
_Decimal64 d64ffmad128(_Decimal128 x, _Decimal128 y, _Decimal128 z);
#endif
```

Description

These functions compute $(x \times y) + z$ (as if) to infinite precision and round once to the return type, according to the current rounding mode. A range error occurs for some finite arguments. A domain error may occur for an infinite argument.

Returns

These functions return $(x \times y) + z$, rounded to the return type of the function.

7.12.15.7 Square root rounded to narrower type**Synopsis**

```
#include <math.h>
float fsqrt(double x);
float fsqrtl(long double x);
double dsqrtl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 d32sqrtd64(_Decimal64 x);
_Decimal32 d32sqrtd128(_Decimal128 x);
_Decimal64 d64sqrtd128(_Decimal128 x);
#endif
```

Description

These functions compute the square root of x , rounded to the return type of the function. They compute the square root (as if) to infinite precision and round once to the return type, according to the current rounding mode. A range error occurs for some finite positive arguments. A domain error occurs if the argument is less than zero.

Returns

These functions return the nonnegative square root of x , rounded to the return type of the function.

7.12.16 Quantum and quantum exponent functions**7.12.16.1 The quantizedN functions****Synopsis**

```
#include <math.h>
#ifdef __STDC_IEC_60559_DFP__
```

```
_Decimal32 quantized32(_Decimal32 x, _Decimal32 y);
_Decimal64 quantized64(_Decimal64 x, _Decimal64 y);
_Decimal128 quantized128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

The **quantized***N* functions compute, if possible, a value with the numerical value of **x** and the quantum exponent of **y**. If the quantum exponent is being increased, the value shall be correctly rounded; if the result does not have the same value as **x**, the “inexact” floating-point exception shall be raised. If the quantum exponent is being decreased and the significand of the result has more digits than the type would allow, the result is NaN, the “invalid” floating-point exception is raised, and a domain error occurs. If one or both operands are NaN the result is NaN. Otherwise if only one operand is infinite, the result is NaN, the “invalid” floating-point exception is raised, and a domain error occurs. If both operands are infinite, the result is **DEC_INFINITY** with the sign of **x**, converted to the return type of the function. The **quantized***N* functions do not raise the “overflow” and “underflow” floating-point exceptions.

Returns

The **quantized***N* functions return a value with the numerical value of **x** (except for any rounding) and the quantum exponent of **y**.

7.12.16.2 The samequantumdN functions

Synopsis

```
#include <math.h>
#ifdef __STDC_IEC_60559_DFP__
bool samequantumd32(_Decimal32 x, _Decimal32 y);
bool samequantumd64(_Decimal64 x, _Decimal64 y);
bool samequantumd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

The **samequantumd***N* functions determine if the quantum exponents of **x** and **y** are the same. If both **x** and **y** are NaN, or both infinite, they have the same quantum exponents; if exactly one operand is infinite or exactly one operand is NaN, they do not have the same quantum exponents. The **samequantumd***N* functions raise no floating-point exception.

Returns

The **samequantumd***N* functions return nonzero (**true**) when **x** and **y** have the same quantum exponents, zero (**false**) otherwise.

7.12.16.3 The quantumdN functions

Synopsis

```
#include <math.h>
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 quantumd32(_Decimal32 x);
_Decimal64 quantumd64(_Decimal64 x);
_Decimal128 quantumd128(_Decimal128 x);
#endif
```

Description

The **quantumd***N* functions compute the quantum (5.3.5.3.4) of a finite argument. If **x** is infinite, the result is $+\infty$.

Returns

The **quantumd***N* functions return the quantum of **x**.

7.12.16.4 The `llquantexpdN` functions

Synopsis

```
#include <math.h>
#ifdef __STDC_IEC_60559_DFP__
long long int llquantexpd32(_Decimal32 x);
long long int llquantexpd64(_Decimal64 x);
long long int llquantexpd128(_Decimal128 x);
#endif
```

Description

The `llquantexpdN` functions compute the quantum exponent (5.3.5.3.4) of a finite argument. If `x` is infinite or NaN, they compute `LLONG_MIN`, the “invalid” floating-point exception is raised, and a domain error occurs.

Returns

The `llquantexpdN` functions return the quantum exponent of `x`.

7.12.17 Decimal re-encoding functions

7.12.17.1 General

ISO/IEC 60559 specifies two different schemes to encode significands in the object representation of a decimal floating-point object: one based on decimal encoding (which packs three decimal digits into 10 bits), the other based on binary encoding (as a binary integer). An implementation may use either of these encoding schemes for its decimal floating types. The re-encoding functions in this subclause provide conversions between external decimal data with a given encoding scheme and the implementation’s corresponding decimal floating type.

7.12.17.2 The `encodedecdN` functions

Synopsis

```
#include <math.h>
#ifdef __STDC_IEC_60559_DFP__
void encodedecd32(unsigned char encptr[restrict static 4],
    const _Decimal32 * restrict xptr);
void encodedecd64(unsigned char encptr[restrict static 8],
    const _Decimal64 * restrict xptr);
void encodedecd128(unsigned char encptr[restrict static 16],
    const _Decimal128 * restrict xptr);
#endif
```

Description

The `encodedecdN` functions convert `*xptr` into an ISO/IEC 60559 decimal N encoding in the encoding scheme based on decimal encoding of the significand and store the resulting encoding as an $N/8$ element array, with 8 bits per array element, in the object pointed to by `encptr`. The order of bytes in the array follows the endianness specified with `__STDC_ENDIAN_NATIVE__` (7.18.2). These functions preserve the value of `*xptr` and raise no floating-point exceptions. If `*xptr` is non-canonical, these functions can possibly produce a canonical encoding.

Returns

The `encodedecdN` functions return no value.

7.12.17.3 The `decodedecdN` functions

Synopsis

```
#include <math.h>
#ifdef __STDC_IEC_60559_DFP__
void decodedecd32(_Decimal32 * restrict xptr,
    const unsigned char encptr[restrict static 4]);
```

```

void decodedecd64(_Decimal64 * restrict xptr,
    const unsigned char encptr[restrict static 8]);
void decodedecd128(_Decimal128 * restrict xptr,
    const unsigned char encptr[restrict static 16]);
#endif

```

Description

The **decodedecd N** functions interpret the $N/8$ element array pointed to by **encptr** as an ISO/IEC 60559 decimal N encoding, with 8 bits per array element, in the encoding scheme based on decimal encoding of the significand. The order of bytes in the array follows the endianness specified with **__STDC_ENDIAN_NATIVE__** (7.18.2). These functions convert the given encoding into a value of the decimal floating type, and store the result in the object pointed to by **xptr**. These functions preserve the encoded value and raise no floating-point exceptions. If the encoding is non-canonical, these functions can possibly produce a canonical representation.

Returns

The **decodedecd N** functions return no value.

7.12.17.4 The encodebind N functions

Synopsis

```

#include <math.h>
#ifdef __STDC_IEC_60559_DFP__
void encodebind32(unsigned char encptr[restrict static 4],
    const _Decimal32 * restrict xptr);
void encodebind64(unsigned char encptr[restrict static 8],
    const _Decimal64 * restrict xptr);
void encodebind128(unsigned char encptr[restrict static 16],
    const _Decimal128 * restrict xptr);
#endif

```

Description

The **encodebind N** functions convert ***xptr** into an ISO/IEC 60559 decimal N encoding in the encoding scheme based on binary encoding of the significand and store the resulting encoding as an $N/8$ element array, with 8 bits per array element, in the object pointed to by **encptr**. The order of bytes in the array follows the endianness specified with **__STDC_ENDIAN_NATIVE__** (7.18.2). These functions preserve the value of ***xptr** and raise no floating-point exceptions. If ***xptr** is non-canonical, these functions can possibly produce a canonical encoding.

Returns

The **encodebind N** functions return no value.

7.12.17.5 The decodebind N functions

Synopsis

```

#include <math.h>
#ifdef __STDC_IEC_60559_DFP__
void decodebind32(_Decimal32 * restrict xptr,
    const unsigned char encptr[restrict static 4]);
void decodebind64(_Decimal64 * restrict xptr,
    const unsigned char encptr[restrict static 8]);
void decodebind128(_Decimal128 * restrict xptr,
    const unsigned char encptr[restrict static 16]);
#endif

```

Description

The **decodebind N** functions interpret the $N/8$ element array pointed to by **encptr** as an ISO/IEC 60559 decimal N encoding, with 8 bits per array element, in the encoding scheme based on

binary encoding of the significand. The order of bytes in the array follows the endianness specified with `__STDC_ENDIAN_NATIVE__` (7.18.2). These functions convert the given encoding into a value of decimal floating type, and store the result in the object pointed to by `xptr`. These functions preserve the encoded value and raise no floating-point exceptions. If the encoding is non-canonical, these functions can produce a canonical representation.

Returns

The `decodebindN` functions return no value.

7.12.18 Comparison macros

7.12.18.1 General

The relational and equality operators support the usual mathematical relationships between numeric values. For any ordered pair of numeric values exactly one of the relationships — *less*, *greater*, and *equal* — is true. Relational operators may raise the “invalid” floating-point exception when argument values are NaNs. For a NaN and a numeric value, or for two NaNs, just the *unordered* relationship is true.²⁸⁴⁾ 7.12.18.2 through 7.12.18.7 provide macros that are quiet versions of the relational operators: the macros do not raise the “invalid” floating-point exception as an effect of quiet NaN arguments. The comparison macros facilitate writing efficient code that accounts for quiet NaNs without suffering the “invalid” floating-point exception. In the synopses in this subclause, *real-floating* indicates that the argument shall be an expression of real floating type²⁸⁵⁾ (both arguments are not required to have the same type).²⁸⁶⁾ If either argument has decimal floating type, the other argument shall have decimal floating type as well.

7.12.18.2 The `isgreater` macro

Synopsis

```
#include <math.h>
int isgreater(real-floating x, real-floating y);
```

Description

The `isgreater` macro determines whether its first argument is greater than its second argument. The value of `isgreater(x,y)` is always equal to `(x) > (y)`.

However, unlike `(x) > (y)`, `isgreater(x,y)` does not raise the “invalid” floating-point exception when `x` and `y` are unordered and neither is a signaling NaN.

Returns

The `isgreater` macro returns the value of `(x) > (y)`.

7.12.18.3 The `isgreaterequal` macro

Synopsis

```
#include <math.h>
int isgreaterequal(real-floating x, real-floating y);
```

Description

The `isgreaterequal` macro determines whether its first argument is greater than or equal to its second argument. The value of `isgreaterequal(x,y)` is always equal to `(x) >= (y)`.

However, unlike `(x) >= (y)`, `isgreaterequal(x,y)` does not raise the “invalid” floating-point exception when `x` and `y` are unordered and neither is a signaling NaN.

²⁸⁴⁾ISO/IEC 60559 requires that the built-in relational operators raise the “invalid” floating-point exception if the operands compare unordered, as an error indicator for programs written without consideration of NaNs; the result in these cases is false.

²⁸⁵⁾If any argument is of integer type, or any other type that is not a real floating type, the behavior is undefined.

²⁸⁶⁾Whether an argument represented in a format wider than its semantic type is converted to the semantic type is unspecified.

Returns

The **isgreaterequal** macro returns the value of **(x) >= (y)**.

7.12.18.4 The **isless** macro

Synopsis

```
#include <math.h>
int isless(real-floating x, real-floating y);
```

Description

The **isless** macro determines whether its first argument is less than its second argument. The value of **isless(x,y)** is always equal to **(x) < (y)**.

However, unlike **(x) < (y)**, **isless(x,y)** does not raise the “invalid” floating-point exception when **x** and **y** are unordered and neither is a signaling NaN.

Returns

The **isless** macro returns the value of **(x) < (y)**.

7.12.18.5 The **islessequal** macro

Synopsis

```
#include <math.h>
int islessequal(real-floating x, real-floating y);
```

Description

The **islessequal** macro determines whether its first argument is less than or equal to its second argument. The value of **islessequal(x,y)** is always equal to **(x) <= (y)**.

However, unlike **(x) <= (y)**, **islessequal(x,y)** does not raise the “invalid” floating-point exception when **x** and **y** are unordered and neither is a signaling NaN.

Returns

The **islessequal** macro returns the value of **(x) <= (y)**.

7.12.18.6 The **islessgreater** macro

Synopsis

```
#include <math.h>
int islessgreater(real-floating x, real-floating y);
```

Description

The **islessgreater** macro determines whether its first argument is less than or greater than its second argument. The **islessgreater(x,y)** macro is similar to **(x) < (y) || (x) > (y)**.

However, **islessgreater(x,y)** does not raise the “invalid” floating-point exception when **x** and **y** are unordered and neither is a signaling NaN (nor does it evaluate **x** and **y** twice).

Returns

The **islessgreater** macro returns the value of **(x) < (y) || (x) > (y)**.

7.12.18.7 The **isunordered** macro

Synopsis

```
#include <math.h>
int isunordered(real-floating x, real-floating y);
```


Description

The **isunordered** macro determines whether its arguments are unordered. It does not raise the “invalid” floating-point exception when **x** and **y** are unordered and neither is a signaling NaN.

Returns

The **isunordered** macro returns 1 if its arguments are unordered and 0 otherwise.

7.12.18.8 The *iseqsig* macro**Synopsis**

```
#include <math.h>
int iseqsig(real-floating x, real-floating y);
```

Description

The **iseqsig** macro determines whether its arguments are equal. If an argument is a NaN, a domain error occurs for the macro, as if a domain error occurred for a function (7.12.2).

Returns

The **iseqsig** macro returns 1 if its arguments are equal and 0 otherwise.

7.13 Non-local jumps <setjmp.h>

7.13.1 General

The header <setjmp.h> defines the macros **setjmp** and `__STDC_VERSION_SETJMP_H__`, and declares one function and one type, for bypassing the normal function call and return discipline.²⁸⁷⁾

The macro

```
__STDC_VERSION_SETJMP_H__
```

is an integer constant expression with a value equivalent to 202311L.

The type declared is

```
jmp_buf
```

which is an array type suitable for holding the information needed to restore a calling environment. The environment of an invocation of the **setjmp** macro consists of information sufficient for a call to the **longjmp** function to return execution to the correct block and invocation of that block, were it called recursively. It does not include the state of the floating-point environment, of open files, or of any other component of the abstract machine.

It is unspecified whether **setjmp** is a macro or an identifier declared with external linkage. If a macro definition is suppressed to access an actual function, or a program defines an external identifier with the name **setjmp**, the behavior is undefined.

7.13.2 Save calling environment

7.13.2.1 The setjmp macro

Synopsis

```
#include <setjmp.h>
int setjmp(jmp_buf env);
```

Description

The **setjmp** macro saves its calling environment in its **jmp_buf** argument for later use by the **longjmp** function.

Returns

If the return is from a direct invocation, the **setjmp** macro returns the value zero. If the return is from a call to the **longjmp** function, the **setjmp** macro returns a nonzero value.

Environmental limits

An invocation of the **setjmp** macro shall appear only in one of the following contexts:

- the entire controlling expression of a selection or iteration statement;
- one operand of a relational or equality operator with the other operand an integer constant expression, with the resulting expression being the entire controlling expression of a selection or iteration statement;
- the operand of a unary **!** operator with the resulting expression being the entire controlling expression of a selection or iteration statement; or
- the entire expression of an expression statement (possibly cast to **void**).

If the invocation appears in any other context, the behavior is undefined.

7.13.3 Restore calling environment

7.13.3.1 The longjmp function

²⁸⁷⁾These functions are useful for dealing with unusual conditions encountered in a low-level function of a program.

Synopsis

```
#include <setjmp.h>
[[noreturn]] void longjmp(jmp_buf env, int val);
```

Description

The **longjmp** function restores the environment saved by the most recent invocation of the **setjmp** macro in the same invocation of the program with the corresponding **jmp_buf** argument. If there has been no such invocation, or if the invocation was from another thread of execution, or if the function containing the invocation of the **setjmp** macro has terminated execution²⁸⁸⁾ in the interim, or if the invocation of the **setjmp** macro was within the scope of an identifier with variably modified type and execution has left that scope in the interim, the behavior is undefined.

All accessible objects have values, and all other components of the abstract machine²⁸⁹⁾ have state, as of the time the **longjmp** function was called, except that the representation of objects of automatic storage duration that are local to the function containing the invocation of the corresponding **setjmp** macro that do not have volatile-qualified type and have been changed between the **setjmp** invocation and **longjmp** call is indeterminate.

Returns

After **longjmp** is completed, thread execution continues as if the corresponding invocation of the **setjmp** macro had just returned the value specified by **val**. The **longjmp** function cannot cause the **setjmp** macro to return the value 0; if **val** is 0, the **setjmp** macro returns the value 1.

EXAMPLE The **longjmp** function that returns control back to the point of the **setjmp** invocation can cause memory associated with a variable length array object to be squandered.

```
#include <setjmp.h>
jmp_buf buf;
void g(int n);
void h(int n);
int n = 6;

void f(void)
{
    int x[n];           // valid: f is not terminated
    setjmp(buf);
    g(n);
}

void g(int n)
{
    int a[n];           // a can remain allocated
    h(n);
}

void h(int n)
{
    int b[n];           // b can remain allocated
    longjmp(buf, 2);    // can cause memory loss
}
```

²⁸⁸⁾For example, by executing a **return** statement or because another **longjmp** call has caused a transfer to a **setjmp** invocation in a function earlier in the set of nested calls.

²⁸⁹⁾This includes, but is not limited to, the floating-point environment and the state of open files.

7.14 Signal handling <signal.h>

7.14.1 General

The header <signal.h> declares a type and two functions and defines several macros, for handling various *signals* (conditions that may be reported during program execution).

The type defined is

```
sig_atomic_t
```

which is the (possibly volatile-qualified) integer type of an object that can be accessed as an atomic entity, even in the presence of asynchronous interrupts.

The macros defined are

```
SIG_DFL
SIG_ERR
SIG_IGN
```

which expand to constant expressions with distinct values that have type compatible with the second argument to, and the return value of, the **signal** function, and whose values compare unequal to the address of any declarable function; and the following, which expand to positive integer constant expressions with type **int** and distinct values that are the signal numbers, each corresponding to the specified condition:

- SIGABRT** abnormal termination, such as is initiated by the **abort** function
- SIGFPE** an erroneous arithmetic operation, such as zero divide or an operation resulting in overflow
- SIGILL** detection of an invalid function image, such as an invalid instruction
- SIGINT** receipt of an interactive attention signal
- SIGSEGV** an invalid access to storage
- SIGTERM** a termination request sent to the program

An implementation is not required to generate any of these signals, except as a result of explicit calls to the **raise** function. Additional signals and pointers to undeclarable functions, with macro definitions beginning, respectively, with the letters **SIG** and an uppercase letter or with **SIG_** and an uppercase letter,²⁹⁰⁾ may also be specified by the implementation. The complete set of signals, their semantics, and their default handling is implementation-defined; all signal numbers shall be positive.

7.14.2 Specify signal handling

7.14.2.1 The signal function

Synopsis

```
#include <signal.h>
void (*signal(int sig, void (*func)(int)))(int);
```

Description

The **signal** function chooses one of three ways in which receipt of the signal number **sig** is to be subsequently handled. If the value of **func** is **SIG_DFL**, default handling for that signal will occur. If the value of **func** is **SIG_IGN**, the signal will be ignored. Otherwise, **func** shall point to a function to be called when that signal occurs. An invocation of such a function because of a signal, or

²⁹⁰⁾See “future library directions” (7.33.10). The names of the signal numbers reflect the following terms (respectively): abort, floating-point exception, illegal instruction, interrupt, segmentation violation, and termination.

(recursively) of any further functions called by that invocation (other than functions in the standard library),²⁹¹⁾ is called a *signal handler*.

When a signal occurs and **func** points to a function, it is implementation-defined whether the equivalent of **signal(sig, SIG_DFL);** is executed or the implementation prevents some implementation-defined set of signals (at least including **sig**) from occurring until the current signal handling has completed; in the case of **SIGILL**, the implementation may alternatively define that no action is taken. Then the equivalent of **(*func)(sig);** is executed. If and when the function returns, if the value of **sig** is **SIGFPE**, **SIGILL**, **SIGSEGV**, or any other implementation-defined value corresponding to a computational exception, the behavior is undefined; otherwise the program will resume execution at the point it was interrupted.

If the signal occurs as the result of calling the **abort** or **raise** function, the signal handler shall not call the **raise** function.

If the signal occurs other than as the result of calling the **abort** or **raise** function, the behavior is undefined if the signal handler refers to any object with static or thread storage duration that is not a lock-free atomic object and that is not declared with the **constexpr** storage-class specifier other than by assigning a value to an object declared as **volatile sig_atomic_t**, or the signal handler calls any function in the standard library other than

- the **abort** function,
- the **_Exit** function,
- the **quick_exit** function,
- the functions in `<stdatomic.h>` (except where explicitly stated otherwise) when the atomic arguments are lock-free,
- the **atomic_is_lock_free** function with any atomic argument, or
- the **signal** function with the first argument equal to the signal number corresponding to the signal that caused the invocation of the handler. Furthermore, if such a call to the **signal** function results in a **SIG_ERR** return, the object designated by **errno** has an indeterminate representation.²⁹²⁾

At program startup, the equivalent of

```
signal(sig, SIG_IGN);
```

may be executed for some signals selected in an implementation-defined manner; the equivalent of

```
signal(sig, SIG_DFL);
```

is executed for all other signals defined by the implementation.

Use of this function in a multi-threaded program results in undefined behavior. The implementation shall behave as if no library function calls the **signal** function.

Returns

If the request can be honored, the **signal** function returns the value of **func** for the most recent successful call to **signal** for the specified signal **sig**. Otherwise, a value of **SIG_ERR** is returned and a positive value is stored in **errno**.

Forward references: the **abort** function (7.24.5.1), the **exit** function (7.24.5.4), the **_Exit** function (7.24.5.5), the **quick_exit** function (7.24.5.7).

²⁹¹⁾This includes functions called indirectly via standard library functions (e.g. a **SIGABRT** handler called via the **abort** function).

²⁹²⁾If any signal is generated by an asynchronous signal handler, the behavior is undefined.

7.14.3 Send signal

7.14.3.1 The raise function

Synopsis

```
#include <signal.h>
int raise(int sig);
```

Description

The **raise** function carries out the actions described in 7.14.2.1 for the signal **sig**. If a signal handler is called, the **raise** function shall not return until after the signal handler does.

Returns

The **raise** function returns zero if successful, nonzero if unsuccessful.

7.15 Alignment `<stdalign.h>`

The header `<stdalign.h>` provides no content.

7.16 Variable arguments <stdarg.h>

7.16.1 General

The header <stdarg.h> declares a type and defines five macros, for advancing through a list of arguments whose number and types are not known to the called function when it is translated.

The macro

```
__STDC_VERSION_STDARG_H__
```

is an integer constant expression with a value equivalent to 202311L.

A function may be called with a variable number of arguments of varying types if its parameter type list ends with an ellipsis.

The type declared is

```
va_list
```

which is a complete object type suitable for holding information needed by the macros **va_start**, **va_arg**, **va_end**, and **va_copy**. If access to the varying arguments is desired, the called function shall declare an object (generally referred to as **ap** in this subclause) having type **va_list**. The object **ap** may be passed as an argument to another function; if that function invokes the **va_arg** macro with parameter **ap**, the representation of **ap** in the calling function is indeterminate and shall be passed to the **va_end** macro prior to any further reference to **ap**.²⁹³⁾

7.16.2 Variable argument list access macros

7.16.2.1 General

The **va_start** and **va_arg** macros described in this subclause shall be implemented as macros, not functions. It is unspecified whether **va_copy** and **va_end** are macros or identifiers declared with external linkage. If a macro definition is suppressed to access an actual function, or a program defines an external identifier with the same name, the behavior is undefined. Each invocation of the **va_start** and **va_copy** macros shall be matched by a corresponding invocation of the **va_end** macro in the same function.

7.16.2.2 The va_arg macro

Synopsis

```
#include <stdarg.h>
type va_arg(va_list ap, type);
```

Description

The **va_arg** macro expands to an expression that has the specified type and the value of the next argument in the call. The parameter **ap** shall have been initialized by the **va_start** or **va_copy** macro (without an intervening invocation of the **va_end** macro for the same **ap**). Each invocation of the **va_arg** macro modifies **ap** so that the values of successive arguments are returned in turn. The behavior is undefined if there is no actual next argument. The parameter *type* shall be an object type name. If *type* is not compatible with the type of the actual next argument (as promoted according to the default argument promotions), the behavior is undefined, except for the following cases:

- both types are pointers to qualified or unqualified versions of compatible types;
- one type is compatible with a signed integer type, the other type is compatible with the corresponding unsigned integer type, and the value is representable in both types;
- one type is pointer to qualified or unqualified **void** and the other is a pointer to a qualified or unqualified character type;

²⁹³⁾ A pointer to a **va_list** can be created and passed to another function, in which case the original function can make further use of the original list after the other function returns.

- or, the type of the next argument is **nullptr_t** and *type* is a pointer type that has the same representation and alignment requirements as a pointer to a character type.²⁹⁴⁾

Returns

The first invocation of the **va_arg** macro after that of the **va_start** macro returns the value of the first argument without an explicit parameter, which matches the position of the `...` in the parameter list. Successive invocations return the values of the remaining arguments in succession.

7.16.2.3 The **va_copy** macro

Synopsis

```
#include <stdarg.h>
void va_copy(va_list dest, va_list src);
```

Description

The **va_copy** macro initializes **dest** as a copy of **src**, as if the **va_start** macro had been applied to **dest** followed by the same sequence of uses of the **va_arg** macro as had previously been used to reach the present state of **src**. Neither the **va_copy** nor **va_start** macro shall be invoked to reinitialize **dest** without an intervening invocation of the **va_end** macro for the same **dest**.

Returns

The **va_copy** macro returns no value.

7.16.2.4 The **va_end** macro

Synopsis

```
#include <stdarg.h>
void va_end(va_list ap);
```

Description

The **va_end** macro facilitates a normal return from the function whose variable argument list was referred to by the expansion of the **va_start** macro, or the function containing the expansion of the **va_copy** macro, that initialized the **va_list** **ap**. The **va_end** macro may modify **ap** so that it is no longer usable (without being reinitialized by the **va_start** or **va_copy** macro). If there is no corresponding invocation of the **va_start** or **va_copy** macro, or if the **va_end** macro is not invoked before the return, the behavior is undefined.

Returns

The **va_end** macro returns no value.

7.16.2.5 The **va_start** macro

Synopsis

```
#include <stdarg.h>
void va_start(va_list ap, ...);
```

Description

The **va_start** macro shall be invoked before any access to the unnamed arguments.

The **va_start** macro initializes **ap** for subsequent use by the **va_arg** and **va_end** macros. Neither the **va_start** nor **va_copy** macro shall be invoked to reinitialize **ap** without an intervening invocation of the **va_end** macro for the same **ap**.

Only the first argument passed to **va_start** is evaluated. If any additional arguments expand to include unbalanced parentheses, or a preprocessing token that does not convert to a token, the behavior is undefined.

²⁹⁴⁾Such types are in particular pointers to qualified or unqualified versions of **void**.

NOTE The macro allows additional arguments to be passed for **va_start** for compatibility with older versions of the library only.

Returns

The **va_start** macro returns no value.

Recommended practice

Additional arguments beyond the first given to the **va_start** macro may be expanded and used in unspecified contexts where they are unevaluated. For example, an implementation diagnoses potentially erroneous input for an invocation of **va_start** such as:

```
#include <stdarg.h>

void miaou (...) {
    va_list vl;
    va_start(vl, 1, 3.0, "12", xd); // diagnostic encouraged
    /* ... */
    va_end(vl);
}
```

Simultaneously, **va_start** usage consistent with older revisions of this document should not produce a diagnostic:

```
#include <stdarg.h>

void neigh (int last_arg, ...) {
    va_list vl;
    va_start(vl, last_arg); // no diagnostic
    /* ... */
    va_end(vl);
}
```

EXAMPLE 1 The function **f1** gathers into an array a list of arguments that are pointers to strings (but not more than **MAXARGS** arguments), then passes the array as a single argument to function **f2**. The number of pointers is specified by the first argument to **f1**.

```
#include <stdarg.h>
#define MAXARGS 31

void f1(int n_ptrs, ...)
{
    va_list ap;
    char *array[MAXARGS];
    int ptr_no = 0;

    if (n_ptrs > MAXARGS)
        n_ptrs = MAXARGS;
    va_start(ap);
    while (ptr_no < n_ptrs)
        array[ptr_no++] = va_arg(ap, char *);
    va_end(ap);
    f2(n_ptrs, array);
}
```

Each call to **f1** is required to have visible the definition of the function or a declaration such as

```
void f1(int, ...);
```

EXAMPLE 2 The function **f3** is similar, but saves the status of the variable argument list after the indicated number of arguments; after **f2** has been called once with the whole list, the trailing part of the list is gathered again and passed to function **f4**.

```

#include <stdarg.h>
#define MAXARGS 31

void f3(int n_ptrs, int f4_after, ...)
{
    va_list ap, ap_save;
    char *array[MAXARGS];
    int ptr_no = 0;
    if (n_ptrs > MAXARGS)
        n_ptrs = MAXARGS;
    va_start(ap);
    while (ptr_no < n_ptrs) {
        array[ptr_no++] = va_arg(ap, char *);
        if (ptr_no == f4_after)
            va_copy(ap_save, ap);
    }
    va_end(ap);
    f2(n_ptrs, array);

    // Now process the saved copy.

    n_ptrs -= f4_after;
    ptr_no = 0;
    while (ptr_no < n_ptrs)
        array[ptr_no++] = va_arg(ap_save, char *);
    va_end(ap_save);
    f4(n_ptrs, array);
}

```

EXAMPLE 3 The function **f5** is similar to **f1**, but instead of passing an explicit number of strings as the first argument, the argument list is terminated with a null pointer.

```

#include <stdarg.h>

#define MAXARGS 31

void f5(...)
{
    va_list ap;
    char *array[MAXARGS];
    int ptr_no = 0;
    va_start(ap);
    while (ptr_no < MAXARGS)
    {
        char *ptr = va_arg(ap, char *);
        if (!ptr)
            break;
        array[ptr_no++] = ptr;
    }
    va_end(ap);
    f6(ptr_no, array);
}

```

Each call to **f5** is required to have visible the definition of the function or a declaration such as

```
void f5(...);
```

and implicitly requires the last argument to be a null pointer.

7.17 Atomics <stdatomic.h>

7.17.1 Introduction

The header <stdatomic.h> defines several macros and declares several types and functions for performing atomic operations on data shared between threads.²⁹⁵⁾

Implementations that define the macro `__STDC_NO_ATOMICS__` may not provide this header nor support any of its facilities.

The macro

```
__STDC_VERSION_STDATOMIC_H__
```

is an integer constant expression with a value equivalent to 202311L.

The macros defined are the *atomic lock-free macros*

```
ATOMIC_BOOL_LOCK_FREE
ATOMIC_CHAR_LOCK_FREE
ATOMIC_CHAR8_T_LOCK_FREE
ATOMIC_CHAR16_T_LOCK_FREE
ATOMIC_CHAR32_T_LOCK_FREE
ATOMIC_WCHAR_T_LOCK_FREE
ATOMIC_SHORT_LOCK_FREE
ATOMIC_INT_LOCK_FREE
ATOMIC_LONG_LOCK_FREE
ATOMIC_LLONG_LOCK_FREE
ATOMIC_POINTER_LOCK_FREE
```

which expand to constant expressions suitable for use in conditional expression inclusion preprocessing directives and which indicate the lock-free property of the corresponding atomic types (both signed and unsigned); and

```
ATOMIC_FLAG_INIT
```

which expands to an initializer for an object of type `atomic_flag`.

The types include

```
memory_order
```

which is an enumerated type whose enumerators identify memory ordering constraints;

```
atomic_flag
```

which is a structure type representing a lock-free, primitive atomic flag; and several atomic analogs of integer types.

In the following synopses:

- An *A* refers to an atomic type.
- A *C* refers to its corresponding non-atomic type.
- An *M* refers to the type of the other argument for arithmetic operations. For atomic integer types, *M* is *C*. For atomic pointer types, *M* is `ptrdiff_t`.
- The functions not ending in `_explicit` have the same semantics as the corresponding `_explicit` function with `memory_order_seq_cst` for the `memory_order` argument.

²⁹⁵⁾See “future library directions” (7.33.11).

It is unspecified whether any generic function declared in `<stdatomic.h>` is a macro or an identifier declared with external linkage. If a macro definition is suppressed to access an actual function, or a program defines an external identifier with the name of a generic function, the behavior is undefined.

NOTE Many operations are volatile-qualified. The “volatile as device register” semantics have not changed in the standard. This qualification means that volatility is preserved when applying these operations to volatile objects.

7.17.2 Initialization

7.17.2.1 General

An atomic object with automatic storage duration that is not initialized or such an object with allocated storage duration initially has an indeterminate representation; equally, a non-atomic store to any byte of the representation (either directly or, for example, by calls to `memcpy` or `memset`) makes any atomic object have an indeterminate representation. Explicit or default initialization for atomic objects with static or thread storage duration that do not have the type `atomic_flag` is guaranteed to produce a valid state.²⁹⁶⁾

Concurrent access to an atomic object before it is set to a valid state, even via an atomic operation, constitutes a data race. If a signal occurs other than as the result of calling the `abort` or `raise` functions, the behavior is undefined if the signal handler reads or modifies an atomic object that has an indeterminate representation.

EXAMPLE The following definition ensure valid states for `guide` and `head` regardless if these are found in file scope or block scope. Thus any atomic operation that is performed on them after their initialization has been met is well defined.

```
_Atomic int guide = 42;
static _Atomic(void*) head;
```

7.17.2.2 The `atomic_init` generic function

Synopsis

```
#include <stdatomic.h>
void atomic_init(volatile A *obj, C value);
```

Description

The `atomic_init` generic function initializes the atomic object pointed to by `obj` to the value `value`, while also initializing any additional state that the implementation may need to carry for the atomic object. If the object has no declared type, after the call the effective type is the atomic type `A`.

Although this function initializes an atomic object, it does not avoid data races; concurrent access to the object being initialized, even via an atomic operation, constitutes a data race.

If a signal occurs other than as the result of calling the `abort` or `raise` functions, the behavior is undefined if the signal handler calls the `atomic_init` generic function.

Returns

The `atomic_init` generic function returns no value.

EXAMPLE

```
atomic_int guide;
atomic_init(&guide, 42);
```

²⁹⁶⁾See “future library directions” (7.33.11).

7.17.3 Order and consistency

7.17.3.1 General

The enumerated type **memory_order** specifies the detailed regular (non-atomic) memory synchronization operations as defined in 5.2.2.5 and may provide for operation ordering. Its enumeration constants are as follows:²⁹⁷⁾

memory_order_relaxed
memory_order_consume
memory_order_acquire
memory_order_release
memory_order_acq_rel
memory_order_seq_cst

For **memory_order_relaxed**, no operation orders memory.

For **memory_order_release**, **memory_order_acq_rel**, and **memory_order_seq_cst**, a store operation performs a release operation on the affected memory location.

For **memory_order_acquire**, **memory_order_acq_rel**, and **memory_order_seq_cst**, a load operation performs an acquire operation on the affected memory location.

For **memory_order_consume**, a load operation performs a consume operation on the affected memory location.

There shall be a single total order *S* on all **memory_order_seq_cst** operations, consistent with the “happens before” order and modification orders for all affected locations, such that each **memory_order_seq_cst** operation *B* that loads a value from an atomic object *M* observes one of the following values:

- the result of the last modification *A* of *M* that precedes *B* in *S*, if it exists, or
- if *A* exists, the result of some modification of *M* that is not **memory_order_seq_cst** and that does not happen before *A*, or
- if *A* does not exist, the result of some modification of *M* that is not **memory_order_seq_cst**.

NOTE 1 Although it is not explicitly required that *S* include lock operations, it can always be extended to an order that does include lock and unlock operations, since the ordering between those is already included in the “happens before” ordering.

NOTE 2 Atomic operations specifying **memory_order_relaxed** are relaxed only with respect to memory ordering. Implementations still guarantee that any given atomic access to a particular atomic object is indivisible with respect to all other atomic accesses to that object.

For an atomic operation *B* that reads the value of an atomic object *M*, if there is a **memory_order_seq_cst** fence *X* sequenced before *B*, then *B* observes either the last **memory_order_seq_cst** modification of *M* preceding *X* in the total order *S* or a later modification of *M* in its modification order.

For atomic operations *A* and *B* on an atomic object *M*, where *A* modifies *M* and *B* takes its value, if there is a **memory_order_seq_cst** fence *X* such that *A* is sequenced before *X* and *B* follows *X* in *S*, then *B* observes either the effects of *A* or a later modification of *M* in its modification order.

For atomic modifications *A* and *B* of an atomic object *M*, *B* occurs later than *A* in the modification order of *M* if:

- there is a **memory_order_seq_cst** fence *X* such that *A* is sequenced before *X*, and *X* precedes *B* in *S*, or
- there is a **memory_order_seq_cst** fence *Y* such that *Y* is sequenced before *B*, and *A* precedes *Y* in *S*, or

²⁹⁷⁾See “future library directions” (7.33.11).

- there are **memory_order_seq_cst** fences *X* and *Y* such that *A* is sequenced before *X*, *Y* is sequenced before *B*, and *X* precedes *Y* in *S*.

Atomic read-modify-write operations shall always read the last value (in the modification order) stored before the write associated with the read-modify-write operation.

An atomic store shall only store a value that has been computed from constants and program input values by a finite sequence of program evaluations, such that each evaluation observes the stored values of the objects as computed by the last prior assignment in the sequence. The ordering of evaluations in this sequence shall be such that

- If an evaluation *B* observes a value computed by *A* in a different thread, then *B* does not happen before *A*.
- If an evaluation *A* is included in the sequence, then all evaluations that assign to the same object and happen before *A* are also included.

NOTE 3 The second requirement disallows “out-of-thin-air”, or “speculative” stores of atomics when relaxed atomics are used. Since unordered operations are involved, evaluations can appear in this sequence out of thread order. For example, with *x* and *y* initially zero,

```
// Thread 1:
r1 = atomic_load_explicit(&y, memory_order_relaxed);
atomic_store_explicit(&x, r1, memory_order_relaxed);

// Thread 2:
r2 = atomic_load_explicit(&x, memory_order_relaxed);
atomic_store_explicit(&y, 42, memory_order_relaxed);
```

is allowed to produce **r1 == 42 && r2 == 42**. The sequence of evaluations justifying this consists of:

```
atomic_store_explicit(&y, 42, memory_order_relaxed);
r1 = atomic_load_explicit(&y, memory_order_relaxed);
atomic_store_explicit(&x, r1, memory_order_relaxed);
r2 = atomic_load_explicit(&x, memory_order_relaxed);
```

On the other hand,

```
// Thread 1:
r1 = atomic_load_explicit(&y, memory_order_relaxed);
atomic_store_explicit(&x, r1, memory_order_relaxed);

// Thread 2:
r2 = atomic_load_explicit(&x, memory_order_relaxed);
atomic_store_explicit(&y, r2, memory_order_relaxed);
```

is not allowed to produce **r1 == 42 && r2 == 42**, since there is no sequence of evaluations that results in the computation of 42. In the absence of “relaxed” operations and read-modify-write operations with weaker than **memory_order_acq_rel** ordering, the second requirement has no impact.

Recommended practice

The requirements do not forbid **r1 == 42 && r2 == 42** in the following example, with *x* and *y* initially zero:

```
// Thread 1:
r1 = atomic_load_explicit(&x, memory_order_relaxed);
if (r1 == 42)
    atomic_store_explicit(&y, r1, memory_order_relaxed);

// Thread 2:
```

```
r2 = atomic_load_explicit(&y, memory_order_relaxed);
if (r2 == 42)
    atomic_store_explicit(&x, 42, memory_order_relaxed);
```

However, this is not useful behavior, and implementations should not allow it. Implementations should make atomic stores visible to atomic loads within a reasonable amount of time.

7.17.3.2 The `kill_dependency` macro

Synopsis

```
#include <stdatomic.h>
type kill_dependency(type y);
```

Description

The `kill_dependency` macro terminates a dependency chain; the argument does not carry a dependency to the return value.

Returns

The `kill_dependency` macro returns the value of `y`.

7.17.4 Fences

7.17.4.1 General

This subclause introduces synchronization primitives called *fences*. Fences can have acquire semantics, release semantics, or both. A fence with acquire semantics is called an *acquire fence*; a fence with release semantics is called a *release fence*.

A release fence *A* synchronizes with an acquire fence *B* if there exist atomic operations *X* and *Y*, both operating on some atomic object *M*, such that *A* is sequenced before *X*, *X* modifies *M*, *Y* is sequenced before *B*, and *Y* reads the value written by *X* or a value written by any side effect in the hypothetical release sequence *X* would head if it were a release operation.

A release fence *A* synchronizes with an atomic operation *B* that performs an acquire operation on an atomic object *M* if there exists an atomic operation *X* such that *A* is sequenced before *X*, *X* modifies *M*, and *B* reads the value written by *X* or a value written by any side effect in the hypothetical release sequence *X* would head if it were a release operation.

An atomic operation *A* that is a release operation on an atomic object *M* synchronizes with an acquire fence *B* if there exists some atomic operation *X* on *M* such that *X* is sequenced before *B* and reads the value written by *A* or a value written by any side effect in the release sequence headed by *A*.

7.17.4.2 The `atomic_thread_fence` function

Synopsis

```
#include <stdatomic.h>
void atomic_thread_fence(memory_order order);
```

Description

Depending on the value of `order`, this operation:

- has no effects, if `order == memory_order_relaxed`;
- is an acquire fence, if `order == memory_order_acquire` or `order == memory_order_consume`;
- is a release fence, if `order == memory_order_release`;
- is both an acquire fence and a release fence, if `order == memory_order_acq_rel`;

— is a sequentially consistent acquire and release fence, if **order** == **memory_order_seq_cst**.

Returns

The **atomic_thread_fence** function returns no value.

7.17.4.3 The **atomic_signal_fence** function

Synopsis

```
#include <stdatomic.h>
void atomic_signal_fence(memory_order order);
```

Description

Equivalent to **atomic_thread_fence(order)**, except that the resulting ordering constraints are established only between a thread and a signal handler executed in the same thread.

NOTE 1 The **atomic_signal_fence** function can be used to specify the order in which actions performed by the thread become visible to the signal handler.

NOTE 2 Compiler optimizations and reorderings of loads and stores are inhibited in the same way as with **atomic_thread_fence**, but the hardware fence instructions that **atomic_thread_fence** would have inserted are not emitted.

Returns

The **atomic_signal_fence** function returns no value.

7.17.5 Lock-free property

7.17.5.1 General

The atomic lock-free macros indicate the lock-free property of integer and address atomic types. A value of 0 indicates that the type is never lock-free; a value of 1 indicates that the type is sometimes lock-free; a value of 2 indicates that the type is always lock-free.

Recommended practice

Operations that are lock-free should also be *address-free*. That is, atomic operations on the same memory location via two different addresses will communicate atomically. The implementation should not depend on any per-process state. This restriction enables communication via memory mapped into a process more than once and memory shared between two processes.

7.17.5.2 The **atomic_is_lock_free** generic function

Synopsis

```
#include <stdatomic.h>
bool atomic_is_lock_free(const volatile A *obj);
```

Description

The **atomic_is_lock_free** generic function indicates whether atomic operations on objects of the type pointed to by **obj** are lock-free.

Returns

The **atomic_is_lock_free** generic function returns nonzero (true) if and only if atomic operations on objects of the type pointed to by the argument are lock-free. In any given program execution, the result of the lock-free query shall be consistent for all pointers of the same type.²⁹⁸⁾

²⁹⁸⁾ **obj** can be a null pointer.

7.17.6 Atomic integer types

For each line in Table 7.6,²⁹⁹⁾ the atomic type name is declared as a type that has the same representation and alignment requirements as the corresponding direct type.³⁰⁰⁾

Table 7.6 — Type name equivalency

Atomic type name	Direct type
<code>atomic_bool</code>	<code>_Atomic bool</code>
<code>atomic_char</code>	<code>_Atomic char</code>
<code>atomic_schar</code>	<code>_Atomic signed char</code>
<code>atomic_uchar</code>	<code>_Atomic unsigned char</code>
<code>atomic_short</code>	<code>_Atomic short</code>
<code>atomic_ushort</code>	<code>_Atomic unsigned short</code>
<code>atomic_int</code>	<code>_Atomic int</code>
<code>atomic_uint</code>	<code>_Atomic unsigned int</code>
<code>atomic_long</code>	<code>_Atomic long</code>
<code>atomic_ulong</code>	<code>_Atomic unsigned long</code>
<code>atomic_llong</code>	<code>_Atomic long long</code>
<code>atomic_ullong</code>	<code>_Atomic unsigned long long</code>
<code>atomic_char8_t</code>	<code>_Atomic char8_t</code>
<code>atomic_char16_t</code>	<code>_Atomic char16_t</code>
<code>atomic_char32_t</code>	<code>_Atomic char32_t</code>
<code>atomic_wchar_t</code>	<code>_Atomic wchar_t</code>
<code>atomic_int_least8_t</code>	<code>_Atomic int_least8_t</code>
<code>atomic_uint_least8_t</code>	<code>_Atomic uint_least8_t</code>
<code>atomic_int_least16_t</code>	<code>_Atomic int_least16_t</code>
<code>atomic_uint_least16_t</code>	<code>_Atomic uint_least16_t</code>
<code>atomic_int_least32_t</code>	<code>_Atomic int_least32_t</code>
<code>atomic_uint_least32_t</code>	<code>_Atomic uint_least32_t</code>
<code>atomic_int_least64_t</code>	<code>_Atomic int_least64_t</code>
<code>atomic_uint_least64_t</code>	<code>_Atomic uint_least64_t</code>
<code>atomic_int_fast8_t</code>	<code>_Atomic int_fast8_t</code>
<code>atomic_uint_fast8_t</code>	<code>_Atomic uint_fast8_t</code>
<code>atomic_int_fast16_t</code>	<code>_Atomic int_fast16_t</code>
<code>atomic_uint_fast16_t</code>	<code>_Atomic uint_fast16_t</code>
<code>atomic_int_fast32_t</code>	<code>_Atomic int_fast32_t</code>
<code>atomic_uint_fast32_t</code>	<code>_Atomic uint_fast32_t</code>
<code>atomic_int_fast64_t</code>	<code>_Atomic int_fast64_t</code>
<code>atomic_uint_fast64_t</code>	<code>_Atomic uint_fast64_t</code>
<code>atomic_intptr_t</code>	<code>_Atomic intptr_t</code>
<code>atomic_uintptr_t</code>	<code>_Atomic uintptr_t</code>
<code>atomic_size_t</code>	<code>_Atomic size_t</code>
<code>atomic_ptrdiff_t</code>	<code>_Atomic ptrdiff_t</code>
<code>atomic_intmax_t</code>	<code>_Atomic intmax_t</code>
<code>atomic_uintmax_t</code>	<code>_Atomic uintmax_t</code>

Conversions to `atomic_bool` behave the same as conversions to `bool`.

Recommended practice

The representation of an atomic integer type is not required to have the same size as the corresponding regular type but it should have the same size whenever possible, as it eases effort required to port existing code.

²⁹⁹⁾See “future library directions” (7.33.11).
³⁰⁰⁾The same representation and alignment requirements are meant to imply interchangeability as arguments to functions, return values from functions, and members of unions.

7.17.7 Operations on atomic types

7.17.7.1 General

There are only a few kinds of operations on atomic types, though there are many instances of those kinds. This subclause specifies each general kind.

7.17.7.2 The `atomic_store` generic functions

Synopsis

```
#include <stdatomic.h>
void atomic_store(volatile A *object, C desired);
void atomic_store_explicit(volatile A *object, C desired, memory_order order);
```

Description

The **order** argument shall not be `memory_order_acquire`, `memory_order_consume`, nor `memory_order_acq_rel`. Atomically replace the value pointed to by **object** with the value of **desired**. Memory is affected according to the value of **order**.

Returns

The `atomic_store` generic functions return no value.

7.17.7.3 The `atomic_load` generic functions

Synopsis

```
#include <stdatomic.h>
C atomic_load(const volatile A *object);
C atomic_load_explicit(const volatile A *object, memory_order order);
```

Description

The **order** argument shall not be `memory_order_release` nor `memory_order_acq_rel`. Memory is affected according to the value of **order**.

Returns

Atomically returns the value pointed to by **object**.

7.17.7.4 The `atomic_exchange` generic functions

Synopsis

```
#include <stdatomic.h>
C atomic_exchange(volatile A *object, C desired);
C atomic_exchange_explicit(volatile A *object, C desired, memory_order order);
```

Description

Atomically replace the value pointed to by **object** with **desired**. Memory is affected according to the value of **order**. These operations are read-modify-write operations (5.2.2.5).

Returns

Atomically returns the value pointed to by **object** immediately before the effects.

7.17.7.5 The `atomic_compare_exchange` generic functions

Synopsis

```
#include <stdatomic.h>
bool atomic_compare_exchange_strong(volatile A *object, C *expected, C desired);
bool atomic_compare_exchange_strong_explicit(volatile A *object, C *expected,
C desired, memory_order success, memory_order failure);
bool atomic_compare_exchange_weak(volatile A *object, C *expected, C desired);
bool atomic_compare_exchange_weak_explicit(volatile A *object, C *expected,
C desired, memory_order success, memory_order failure);
```

Description

The **failure** argument shall not be **memory_order_release** nor **memory_order_acq_rel**. The **failure** argument shall be no stronger than the success argument.

Atomically, compares the contents of the memory pointed to by **object** for equality with that pointed to by **expected**, and if true, replaces the contents of the memory pointed to by **object** with **desired**, and if false, updates the contents of the memory pointed to by **expected** with that pointed to by **object**. Further, if the comparison is true, memory is affected according to the value of **success**, and if the comparison is false, memory is affected according to the value of **failure**. These operations are atomic read-modify-write operations (5.2.2.5).

NOTE 1 For example, the effect of **atomic_compare_exchange_strong** is

```
if (memcmp(object, expected, sizeof(*object)) == 0)
    memcpy(object, &desired, sizeof(*object));
else
    memcpy(expected, object, sizeof(*object));
```

A weak compare-and-exchange operation can fail spuriously. That is, even when the contents of memory referred to by **expected** and **object** are equal, it can return zero and store back to **expected** the same memory contents that were originally there.

NOTE 2 This spurious failure enables implementation of compare-and-exchange on a broader class of machines; e.g. load-locked store-conditional machines.

EXAMPLE A consequence of spurious failure is that nearly all uses of weak compare-and-exchange will be in a loop.

```
exp = atomic_load(&cur);
do {
    des = function(exp);
} while (!atomic_compare_exchange_weak(&cur, &exp, des));
```

When a compare-and-exchange is in a loop, the weak version will yield better performance on some platforms. When a weak compare-and-exchange would require a loop and a strong one would not, the strong one is preferable.

Returns

The result of the comparison.

7.17.7.6 The **atomic_fetch** and **modify** generic functions

The following operations perform arithmetic and bitwise computations. All these operations are applicable to an object of any atomic integer type other than **_Atomic_bool**, **atomic_bool**, or the atomic version of an enumeration with underlying type **bool**. The key, operator, and computation correspondence is:

key	op	computation
add	+	addition
sub	-	subtraction
or		bitwise inclusive or
xor	^	bitwise exclusive or
and	&	bitwise and

Synopsis

```
#include <stdatomic.h>
C atomic_fetch_key(volatile A *object, M operand);
C atomic_fetch_key_explicit(volatile A *object, M operand, memory_order order);
```

Description

Atomically replaces the value pointed to by **object** with the result of the computation applied to the value pointed to by **object** and the given operand. Memory is affected according to the value of **order**. These operations are atomic read-modify-write operations (5.2.2.5). For signed integer types, arithmetic performs silent wraparound on integer overflow; there are no undefined results. For address types, the result may be an undefined address, but the operations otherwise have no undefined behavior.

Returns

Atomically, the value pointed to by **object** immediately before the effects.

7.17.8 Atomic flag type and operations**7.17.8.1 General**

The **atomic_flag** type provides the classic test-and-set functionality. It has two states, set and clear. Operations on an object of type **atomic_flag** shall be lock free.

NOTE Hence, as per 7.17.5, the operations should also be address-free. No other type requires lock-free operations, so the **atomic_flag** type is the minimum hardware-implemented type needed to conform to this document. The remaining types can be emulated with **atomic_flag**, though with less than ideal properties.

The macro **ATOMIC_FLAG_INIT** can be used to initialize an **atomic_flag** to the clear state. An **atomic_flag** that is not explicitly initialized with **ATOMIC_FLAG_INIT** has initially an indeterminate representation.

EXAMPLE

```
atomic_flag guard = ATOMIC_FLAG_INIT;
```

7.17.8.2 The atomic_flag_test_and_set functions**Synopsis**

```
#include <stdatomic.h>
bool atomic_flag_test_and_set(volatile atomic_flag *object);
bool atomic_flag_test_and_set_explicit(volatile atomic_flag *object,
    memory_order order);
```

Description

Atomically places the atomic flag pointed to by **object** in the set state and returns the value corresponding to the immediately preceding state. Memory is affected according to the value of **order**. These operations are atomic read-modify-write operations (5.2.2.5).

Returns

The **atomic_flag_test_and_set** functions return the value that corresponds to the state of the atomic flag immediately before the effects. The return value true corresponds to the set state and the return value false corresponds to the clear state.

7.17.8.3 The atomic_flag_clear functions**Synopsis**

```
#include <stdatomic.h>
void atomic_flag_clear(volatile atomic_flag *object);
void atomic_flag_clear_explicit(volatile atomic_flag *object,
    memory_order order);
```

Description

The **order** argument shall not be **memory_order_acquire** nor **memory_order_acq_rel**. Atomically places the atomic flag pointed to by **object** into the clear state. Memory is affected according to the value of **order**.

Returns

The **atomic_flag_clear** functions return no value.

7.18 Bit and byte utilities <stdbit.h>

7.18.1 General

The header <stdbit.h> defines the following macros, types, and functions, to work with the byte and bit representation of many types, typically integer types. This header makes available the **size_t** type name (7.21) and any **uintN_t**, **intN_t**, **uint_leastN_t**, or **int_leastN_t** type names defined by the implementation (7.22).

The macro

```
__STDC_VERSION_STDBIT_H__
```

is an integer constant expression with a value equivalent to 202311L.

The *most significant index* is the 0-based index counting from the most significant bit, 0, to the least significant bit, $w - 1$, where w is the width of the type that is having its most significant index computed.

The *least significant index* is the 0-based index counting from the least significant bit, 0, to the most significant bit, $w - 1$, where w is the width of the type that is having its least significant index computed.

It is unspecified whether any generic function declared in <stdbit.h> is a macro or an identifier declared with external linkage. If a macro definition is suppressed to access an actual function, or a program defines an external identifier with the name of a generic function, the behavior is unspecified.

7.18.2 Endian

Two common methods of byte ordering in multi-byte scalar types are *little-endian* and *big-endian*. Little-endian is a format for storage or transmission of binary data in which the least significant byte is placed first, with the rest in ascending order. Or, that the least significant byte is stored at the smallest memory address. Big-endian is a format for storage or transmission of binary data in which the most significant byte is placed first, with the rest in descending order. Or, that the most significant byte is stored at the smallest memory address. Other byte orderings are also possible.

The macros are:

```
__STDC_ENDIAN_LITTLE__
```

which represents a method of byte order storage in which the least significant byte is placed first and the rest are in ascending order, and is an integer constant expression;

```
__STDC_ENDIAN_BIG__
```

which represents a method of byte order storage in which the most significant byte is placed first and the rest are in descending order, and is an integer constant expression;

```
__STDC_ENDIAN_NATIVE__ /* see following description */
```

which represents the method of byte order storage for the execution environment and is an integer constant expression. **__STDC_ENDIAN_NATIVE__** describes the endianness of the execution environment with respect to bit-precise integer types, standard integer types, and extended integer types which do not have padding bits.

__STDC_ENDIAN_NATIVE__ shall expand to an integer constant expression whose value is equivalent to the value of **__STDC_ENDIAN_LITTLE__** if the execution environment is little-endian. Otherwise, **__STDC_ENDIAN_NATIVE__** shall expand to an integer constant expression whose value is equivalent to the value of **__STDC_ENDIAN_BIG__** if the execution environment is big-endian. If the execution environment is neither little-endian nor big-endian, it then has some other implementation-defined byte order and the macro **__STDC_ENDIAN_NATIVE__** shall expand to an integer constant expression whose value is different from the values of

`__STDC_ENDIAN_LITTLE__` and `__STDC_ENDIAN_BIG__`. The values of the integer constant expressions for `__STDC_ENDIAN_LITTLE__` and `__STDC_ENDIAN_BIG__` are not equal.

7.18.3 Count Leading Zeros

Synopsis

```
#include <stdbit.h>
unsigned int stdc_leading_zeros_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_leading_zeros_us(unsigned short value) [[unsequenced]];
unsigned int stdc_leading_zeros_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_leading_zeros_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_leading_zeros_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_leading_zeros(generic_value_type value) [[unsequenced]];
```

Returns

Returns the number of consecutive 0 bits in **value**, starting from the most significant bit.

The type-generic function (marked by its *generic_value_type* argument) returns the appropriate value based on the type of the input value, so long as it is a:

- standard unsigned integer type, excluding **bool**;
- extended unsigned integer type;
- or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.4 Count Leading Ones

Synopsis

```
#include <stdbit.h>
unsigned int stdc_leading_ones_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_leading_ones_us(unsigned short value) [[unsequenced]];
unsigned int stdc_leading_ones_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_leading_ones_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_leading_ones_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_leading_ones(generic_value_type value) [[unsequenced]];
```

Returns

Returns the number of consecutive 1 bits in **value**, starting from the most significant bit.

The type-generic function (marked by its *generic_value_type* argument) returns the appropriate value based on the type of the input value, so long as it is a:

- standard unsigned integer type, excluding **bool**;
- extended unsigned integer type;
- or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.5 Count Trailing Zeros

Synopsis

```
#include <stdbit.h>
unsigned int stdc_trailing_zeros_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_trailing_zeros_us(unsigned short value) [[unsequenced]];
unsigned int stdc_trailing_zeros_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_trailing_zeros_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_trailing_zeros_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_trailing_zeros(generic_value_type value) [[unsequenced]];
```

Returns

Returns the number of consecutive 0 bits in **value**, starting from the least significant bit.

The type-generic function (marked by its *generic_value_type* argument) returns the appropriate value based on the type of the input value, so long as it is a:

- standard unsigned integer type, excluding **bool**;
- extended unsigned integer type;
- or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.6 Count Trailing Ones

Synopsis

```
#include <stdbit.h>
unsigned int stdc_trailing_ones_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_trailing_ones_us(unsigned short value) [[unsequenced]];
unsigned int stdc_trailing_ones_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_trailing_ones_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_trailing_ones_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_trailing_ones(generic_value_type value) [[unsequenced]];
```

Returns

Returns the number of consecutive 1 bits in **value**, starting from the least significant bit.

The type-generic function (marked by its *generic_value_type* argument) returns the appropriate value based on the type of the input value, so long as it is a:

- standard unsigned integer type, excluding **bool**;
- extended unsigned integer type;
- or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.7 First Leading Zero

Synopsis

```
#include <stdbit.h>
unsigned int stdc_first_leading_zero_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_first_leading_zero_us(unsigned short value) [[unsequenced]];
unsigned int stdc_first_leading_zero_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_first_leading_zero_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_first_leading_zero_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_first_leading_zero(generic_value_type value) [[unsequenced]];
```

Returns

Returns the most significant index of the first 0 bit in **value**, plus 1. If it is not found, this function returns 0.

The type-generic function (marked by its *generic_value_type* argument) returns the appropriate value based on the type of the input value, so long as it is a:

- standard unsigned integer type, excluding **bool**;
- extended unsigned integer type;
- or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.8 First Leading One

Synopsis

```
#include <stdbit.h>
unsigned int stdc_first_leading_one_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_first_leading_one_us(unsigned short value) [[unsequenced]];
unsigned int stdc_first_leading_one_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_first_leading_one_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_first_leading_one_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_first_leading_one(generic_value_type value) [[unsequenced]];
```

Returns

Returns the most significant index of the first 1 bit in **value**, plus 1. If it is not found, this function returns 0.

The type-generic function (marked by its *generic_value_type* argument) returns the appropriate value based on the type of the input value, so long as it is a:

- standard unsigned integer type, excluding **bool**;
- extended unsigned integer type;
- or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.9 First Trailing Zero

Synopsis

```
#include <stdbit.h>
unsigned int stdc_first_trailing_zero_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_first_trailing_zero_us(unsigned short value) [[unsequenced]];
unsigned int stdc_first_trailing_zero_ui(unsigned int value) [[unsequenced]];
unsigned int
stdc_first_trailing_zero_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_first_trailing_zero_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_first_trailing_zero(generic_value_type value) [[unsequenced]];
```

Returns

Returns the least significant index of the first 0 bit in **value**, plus 1. If it is not found, this function returns 0.

The type-generic function (marked by its *generic_value_type* argument) returns the appropriate value based on the type of the input value, so long as it is a:

- standard unsigned integer type, excluding **bool**;
- extended unsigned integer type;
- or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.10 First Trailing One

Synopsis

```
#include <stdbit.h>
unsigned int stdc_first_trailing_one_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_first_trailing_one_us(unsigned short value) [[unsequenced]];
unsigned int stdc_first_trailing_one_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_first_trailing_one_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_first_trailing_one_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_first_trailing_one(generic_value_type value) [[unsequenced]];
```

Returns

Returns the least significant index of the first 1 bit in **value**, plus 1. If it is not found, this function returns 0.

The type-generic function (marked by its *generic_value_type* argument) returns the appropriate value based on the type of the input value, so long as it is a:

- standard unsigned integer type, excluding **bool**;
- extended unsigned integer type;
- or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.11 Count Zeros

Synopsis

```
#include <stdbit.h>
unsigned int stdc_count_zeros_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_count_zeros_us(unsigned short value) [[unsequenced]];
unsigned int stdc_count_zeros_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_count_zeros_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_count_zeros_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_count_zeros(generic_value_type value) [[unsequenced]];
```

Returns

Returns the total number of 0 bits within the given **value**.

The type-generic function (marked by its *generic_value_type* argument) returns the previously described result for a given input value so long as the *generic_value_type* is a:

- standard unsigned integer type, excluding **bool**;
- extended unsigned integer type;
- or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.12 Count Ones

Synopsis

```
#include <stdbit.h>
unsigned int stdc_count_ones_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_count_ones_us(unsigned short value) [[unsequenced]];
unsigned int stdc_count_ones_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_count_ones_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_count_ones_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_count_ones(generic_value_type value) [[unsequenced]];
```

Returns

Returns the total number of 1 bits within the given **value**.

The type-generic function (marked by its *generic_value_type* argument) returns the previously described result for a given input value so long as the *generic_value_type* is a:

- standard unsigned integer type, excluding **bool**;
- extended unsigned integer type;
- or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.13 Single-bit Check

Synopsis

```
#include <stdbit.h>
bool stdc_has_single_bit_uc(unsigned char value) [[unsequenced]];
bool stdc_has_single_bit_us(unsigned short value) [[unsequenced]];
bool stdc_has_single_bit_ui(unsigned int value) [[unsequenced]];
bool stdc_has_single_bit_ul(unsigned long int value) [[unsequenced]];
bool stdc_has_single_bit_ull(unsigned long long int value) [[unsequenced]];
bool stdc_has_single_bit(generic_value_type value) [[unsequenced]];
```

Returns

The **stdc_has_single_bit** functions return **true** if and only if there is a single 1 bit in **value**.

The type-generic function (marked by its *generic_value_type* argument) returns the previously described result for a given input value so long as the *generic_value_type* is a:

- standard unsigned integer type, excluding **bool**;
- extended unsigned integer type;
- or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

7.18.14 Bit Width

Synopsis

```
#include <stdbit.h>
unsigned int stdc_bit_width_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_bit_width_us(unsigned short value) [[unsequenced]];
unsigned int stdc_bit_width_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_bit_width_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_bit_width_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_bit_width(generic_value_type value) [[unsequenced]];
```

Description

The **stdc_bit_width** functions compute the smallest number of bits needed to store **value**.

Returns

The **stdc_bit_width** functions return 0 if **value** is 0. Otherwise, they return $1 + \lfloor \log_2(\text{value}) \rfloor$.

The type-generic function (marked by its *generic_value_type* argument) returns the previously described result for a given input value so long as the *generic_value_type* is a:

- standard unsigned integer type, excluding **bool**;
- extended unsigned integer type;
- or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.15 Bit Floor

Synopsis

```
#include <stdbit.h>
unsigned char stdc_bit_floor_uc(unsigned char value) [[unsequenced]];
unsigned short stdc_bit_floor_us(unsigned short value) [[unsequenced]];
unsigned int stdc_bit_floor_ui(unsigned int value) [[unsequenced]];
unsigned long int stdc_bit_floor_ul(unsigned long int value) [[unsequenced]];
unsigned long long int
stdc_bit_floor_ull(unsigned long long int value) [[unsequenced]];
generic_value_type stdc_bit_floor(generic_value_type value) [[unsequenced]];
```

Description

The **stdc_bit_floor** functions compute the largest integral power of 2 that is not greater than **value**.

Returns

The **stdc_bit_floor** functions return 0 if **value** is 0. Otherwise, they return the largest integral power of 2 that is not greater than **value**.

The type-generic function (marked by its *generic_value_type* argument) returns the previously described result for a given input value so long as the *generic_value_type* is a:

- standard unsigned integer type, excluding **bool**;
- extended unsigned integer type;
- or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

7.18.16 Bit Ceiling

Synopsis

```
#include <stdbit.h>
unsigned char stdc_bit_ceil_uc(unsigned char value) [[unsequenced]];
unsigned short stdc_bit_ceil_us(unsigned short value) [[unsequenced]];
unsigned int stdc_bit_ceil_ui(unsigned int value) [[unsequenced]];
unsigned long int stdc_bit_ceil_ul(unsigned long int value) [[unsequenced]];
unsigned long long int
stdc_bit_ceil_ull(unsigned long long int value) [[unsequenced]];
generic_value_type stdc_bit_ceil(generic_value_type value) [[unsequenced]];
```

Description

The **stdc_bit_ceil** functions compute the smallest integral power of 2 that is not less than **value**. If the computation does not fit in the given return type, they return 0.

Returns

The **stdc_bit_ceil** functions return the smallest integral power of 2 that is not less than **value** or 0 if such a value is not representable in the return type.

The type-generic function (marked by its *generic_value_type* argument) returns the previously described result for a given input value so long as the *generic_value_type* is a:

- standard unsigned integer type, excluding **bool**;
- extended unsigned integer type;
- or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

7.19 Boolean type and values `<stdbool.h>`

The header `<stdbool.h>` provides the obsolescent macro `__bool_true_false_are_defined` which expands to the integer constant `1`.

7.20 Checked Integer Arithmetic <stdckdint.h>

7.20.1 General

The header <stdckdint.h> defines several macros for performing checked integer arithmetic.
The macro

```
__STDC_VERSION__STDCKDINT_H__
```

is an integer constant expression with a value equivalent to 202311L.

7.20.2 Checked Integer Operation Type-generic Macros

Synopsis

```
#include <stdckdint.h>
bool ckd_add(type1 *result, type2 a, type3 b);
bool ckd_sub(type1 *result, type2 a, type3 b);
bool ckd_mul(type1 *result, type2 a, type3 b);
```

Description

These type-generic macros perform addition, subtraction, or multiplication of the mathematical values of **a** and **b**, storing the result of the operation in ***result**, (that is, ***result** is assigned the result of computing **a + b**, **a - b**, or **a * b**). Each operation is performed as if both operands were represented in a signed integer type with infinite range, and the result was then converted from this integer type to *type1*.

Both *type2* and *type3* shall be any integer type other than “plain” **char**, **bool**, a bit-precise integer type, or an enumerated type, and they can be the same. ***result** shall be a modifiable lvalue of any integer type other than “plain” **char**, **bool**, a bit-precise integer type, or an enumerated type.

Recommended practice

It is recommended to produce a diagnostic message if *type2* or *type3* are not suitable integer types, or if ***result** is not a modifiable lvalue of a suitable integer type.

Returns

If these type-generic macros return **false**, the value assigned to ***result** correctly represents the mathematical result of the operation. Otherwise, these type-generic macros return **true**. In this case, the value assigned to ***result** is the mathematical result of the operation wrapped around to the width of ***result**.

EXAMPLE If **a** and **b** are values of type **signed int**, and **result** is a **signed long**, then

```
ckd_sub(&result, a, b);
```

indicates if **a - b** can be expressed as a **signed long**. If **signed long** has a greater width than **signed int**, this is the case and this macro invocation returns **false**.

7.21 Common definitions <stddef.h>

7.21.1 General

The header <stddef.h> defines the following macros and declares the following types. Some are also defined in other headers, as noted in their respective subclauses.

The macro

```
__STDC_VERSION_STDDEF_H__
```

is an integer constant expression with a value equivalent to 202311L.

The types are

```
ptrdiff_t
```

which is the signed integer type of the result of subtracting two pointers;

```
size_t
```

which is the unsigned integer type of the result of the **sizeof** operator;

```
max_align_t
```

which is an object type whose alignment is the greatest fundamental alignment;

```
wchar_t
```

which is an integer type whose range of values can represent distinct codes for all members of the largest extended character set specified among the supported locales; the null character shall have the code value zero. Each member of the basic character set shall have a code value equal to its value when used as the lone character in an integer character constant if an implementation does not define `__STDC_MB_MIGHT_NEQ_WC__`; and,

```
nullptr_t
```

which is the type of the **nullptr** predefined constant, see the subsequent description in the following subclauses.

The macros are

```
NULL
```

which expands to an implementation-defined null pointer constant;

```
unreachable()
```

which expands to a void expression whose behavior is undefined if it is reached during execution; and

```
offsetof(type, member-designator)
```

which expands to an integer constant expression that has type **size_t**, the value of which is the offset in bytes, to the subobject (designated by *member-designator*), from the beginning of any object of type *type*. The type and member designator shall be such that given

```
static type t;
```

then the expression `&(t.member-designator)` evaluates to an address constant. If the specified type name contains a comma not between matching parentheses or if the specified member is a bit-field, the behavior is undefined.

Recommended practice

The types used for `size_t` and `ptrdiff_t` should not have an integer conversion rank greater than that of `signed long int` unless the implementation supports objects large enough to make this necessary.

7.21.2 The unreachable macro

Synopsis

```
#include <stddef.h>
void unreachable(void);
```

Description

An invocation of the function-like macro `unreachable` indicates that the particular flow control that leads to the invocation will never be taken; it receives no arguments and expands to a void expression. The program execution shall not reach such an invocation.

Returns

If a macro invocation `unreachable()` is reached during execution, the behavior is undefined.

EXAMPLE 1 The following program assumes that each execution is provided with at least one command line argument. The behavior of an execution with no arguments is undefined.

```
#include <stddef.h>
#include <stdio.h>

int main (int argc, char* argv[static argc + 1]) {
    if (argc <= 2)
        unreachable();
    else
        return printf("%s: we see %s", argv[0], argv[1]);

    return puts("this should never be reached");
}
```

Here, the `static` array size expression and the annotation of the control flow with `unreachable` indicates that the pointed-to parameter array `argv` will hold at least three elements, regardless of the circumstances. A possible optimization is that the resulting executable never performs the comparison and unconditionally executes a tail call to `printf` that never returns to the `main` function. In particular, the entire call and reference to `puts` can be omitted from the executable. No diagnostic is expected.

Because `argv` and `argc`'s values are controlled by the implementation and cannot be deterministically accounted for by the program, this program runs a high risk of engaging in completely undefined behavior.

EXAMPLE 2 The following code expresses the expectation that the argument to the function will be one of the three enumerator values despite an enumeration type allowing other, non-enumerated values to be passed. Some implementations can diagnose the lack of a `return` statement after the `switch`, but use of the `unreachable` macro signals information to the implementation that this scenario should not be possible, allowing for better diagnostic and optimization properties.

```
enum Colors { Red, Green, Blue };
int get_channel_index(enum Colors c) {
    switch (c) {
        case Red: return 0;
        case Green: return 1;
        case Blue: return 2;
    }
    unreachable();
}
```

```
}
```

7.21.3 The `nullptr_t` type

Synopsis

```
#include <stddef.h>
typedef typeof_unqual(nullptr) nullptr_t;
```

Description

The `nullptr_t` type is the type of the `nullptr` predefined constant. It has only a very limited use in contexts where this type is needed to distinguish `nullptr` from other expression types. It is an unqualified complete scalar type that is different from all pointer or arithmetic types and is neither an atomic or array type and has exactly one value, `nullptr`. Default or empty initialization of an object of this type is equivalent to an initialization by `nullptr`.

The size and alignment of `nullptr_t` is the same as for a pointer to character type. An object representation of the value `nullptr` is the same as the object representation of a null pointer value of type `void*`. An lvalue conversion of an object of type `nullptr_t` with such an object representation has the value `nullptr`; if the object representation is different, the behavior is undefined.³⁰¹⁾

NOTE Because it is considered to be a scalar type, `nullptr_t` can appear in many context where `(void*)0` would be valid, for example,

- as the operand of `alignas`, `sizeof` or `typeof` operators,
- as the operand of an implicit or explicit conversion to a pointer type,
- as the assignment expression in an assignment or initialization of an object of type `nullptr_t`,
- as an argument to a parameter of type `nullptr_t` or in a variable argument list,
- as a void expression,
- as the operand of an implicit or explicit conversion to `bool`,
- as an operand of a `_Generic` primary expression,
- as an operand of the `!`, `&&`, `||` or conditional operators, or
- as the controlling expression of an `if` or iteration statement.

³⁰¹⁾ Thus, during the whole program execution an object of type `nullptr_t` evaluates to the assumed value `nullptr`.

7.22 Integer types <stdint.h>

7.22.1 General

The header <stdint.h> declares sets of integer types having specified widths, and defines corresponding sets of macros.³⁰²⁾ It also defines macros that specify limits of integer types corresponding to types defined in other standard headers.

Types are defined in the following categories:

- integer types having certain exact widths;
- integer types having at least certain specified widths;
- fastest integer types having at least certain specified widths;
- integer types wide enough to hold pointers to objects;
- integer types having greatest width.

(Some of these types may denote the same type.)

Corresponding macros specify limits of the declared types and construct suitable constants.

For each type described herein that the implementation provides,³⁰³⁾ <stdint.h> shall declare that typedef name and define the associated macros. Conversely, for each type described herein that the implementation does not provide, <stdint.h> shall not declare that typedef name nor shall it define the associated macros. An implementation shall provide those types described as “required”, but may not provide any of the others (described as “optional”). None of the types shall be defined as a synonym for a bit-precise integer type.

The feature test macro `__STDC_VERSION_STDINT_H__` expands to the token 202311L.

7.22.2 Integer types

7.22.2.1 General

When typedef names differing only in the absence or presence of the initial **u** are defined, they shall denote corresponding signed and unsigned types as described in 6.2.5; an implementation providing one of these corresponding types shall also provide the other.

In the following descriptions, the symbol *N* represents an unsigned decimal integer with no leading zeros (e.g. 8 or 24, but not 04 or 048).

7.22.2.2 Exact-width integer types

The typedef name `intN_t` designates a signed integer type with width *N* and no padding bits. Thus, `int8_t` denotes such a signed integer type with a width of exactly 8 bits.

The typedef name `uintN_t` designates an unsigned integer type with width *N* and no padding bits. Thus, `uint24_t` denotes such an unsigned integer type with a width of exactly 24 bits.

If an implementation provides standard or extended integer types with a particular width and no padding bits, it shall define the corresponding typedef names.

7.22.2.3 Minimum-width integer types

The typedef name `int_leastN_t` designates a signed integer type with a width of at least *N*, such that no signed integer type with lesser size has at least the specified width. Thus, `int_least32_t` denotes a signed integer type with a width of at least 32 bits.

The typedef name `uint_leastN_t` designates an unsigned integer type with a width of at least *N*, such that no unsigned integer type with lesser size has at least the specified width. Thus, `uint_least16_t` denotes an unsigned integer type with a width of at least 16 bits.

³⁰²⁾See “future library directions” (7.33.15).

³⁰³⁾Some of these types can denote implementation-defined extended integer types.

If the typedef name **intN_t** is defined, **int_leastN_t** designates the same type. If the typedef name **uintN_t** is defined, **uint_leastN_t** designates the same type.

The following types are required:

int_least8_t	uint_least8_t
int_least16_t	uint_least16_t
int_least32_t	uint_least32_t
int_least64_t	uint_least64_t

All other types of this form are optional.

7.22.2.4 Fastest minimum-width integer types

Each of the following types designates an integer type that is usually fastest³⁰⁴⁾ to operate with among all integer types that have at least the specified width.

The typedef name **int_fastN_t** designates the fastest signed integer type with a width of at least *N*. The typedef name **uint_fastN_t** designates the fastest unsigned integer type with a width of at least *N*.

The following types are required:

int_fast8_t	uint_fast8_t
int_fast16_t	uint_fast16_t
int_fast32_t	uint_fast32_t
int_fast64_t	uint_fast64_t

All other types of this form are optional.

7.22.2.5 Integer types capable of holding object pointers

The following type designates a signed integer type, other than a bit-precise integer type, with the property that any valid pointer to **void** can be converted to this type, then converted back to pointer to **void**, and the result will compare equal to the original pointer:

intptr_t

The following type designates an unsigned integer type, other than a bit-precise integer type, with the property that any valid pointer to **void** can be converted to this type, then converted back to pointer to **void**, and the result will compare equal to the original pointer:

uintptr_t

These types are optional.

7.22.2.6 Greatest-width integer types

The following type designates a signed integer type, other than a bit-precise integer type, capable of representing any value of any signed integer type with the possible exceptions of signed bit-precise integer types and of signed extended integer types that are wider than **long long** and that are referred by the type definition for an exact width integer type:

intmax_t

The following type designates the unsigned integer type that corresponds to **intmax_t**:³⁰⁵⁾

³⁰⁴⁾The designated type is not guaranteed to be fastest for all purposes; if the implementation has no clear grounds for choosing one type over another, it will simply pick some integer type satisfying the signedness and width requirements.

³⁰⁵⁾Thus this type is capable of representing any value of any unsigned integer type with the possible exception of bit-precise integer types and particular extended integer types that are wider than **unsigned long long**.

<code>uintmax_t</code>

These types are required.

7.22.3 Widths of specified-width integer types

7.22.3.1 General

The following object-like macros specify the width of the types declared in `<stdint.h>`. Each macro name corresponds to a similar type name in 7.22.2.

Each instance of any defined macro shall be replaced by a constant expression suitable for use in `#if` preprocessing directives. Its implementation-defined value shall be equal to or greater than the value given in the subsequent subclauses, except where stated to be exactly the given value. An implementation shall define only the macros corresponding to those typedef names it actually provides.³⁰⁶⁾

7.22.3.2 Width of exact-width integer types

<code>INTN_WIDTH</code>	exactly <i>N</i>
<code>UINTN_WIDTH</code>	exactly <i>N</i>

7.22.3.3 Width of minimum-width integer types

<code>INT_LEASTN_WIDTH</code>	exactly <code>UINT_LEASTN_WIDTH</code>
<code>UINT_LEASTN_WIDTH</code>	<i>N</i>

7.22.3.4 Width of fastest minimum-width integer types

<code>INT_FASTN_WIDTH</code>	exactly <code>UINT_FASTN_WIDTH</code>
<code>UINT_FASTN_WIDTH</code>	<i>N</i>

7.22.3.5 Width of integer types capable of holding object pointers

<code>INTPTR_WIDTH</code>	exactly <code>UINTPTR_WIDTH</code>
<code>UINTPTR_WIDTH</code>	16

7.22.3.6 Width of greatest-width integer types

<code>INTMAX_WIDTH</code>	exactly <code>UINTMAX_WIDTH</code>
<code>UINTMAX_WIDTH</code>	64

7.22.4 Width of other integer types

7.22.4.1 General

The following object-like macros specify the width of integer types corresponding to types defined in other standard headers.

Each instance of these macros shall be replaced by a constant expression suitable for use in `#if` preprocessing directives. Its implementation-defined value shall be equal to or greater than the corresponding value given in the following subclauses. An implementation shall define only the macros corresponding to those typedef names it actually provides.³⁰⁷⁾

7.22.4.2 Width of `ptrdiff_t`

<code>PTRDIFF_WIDTH</code>	16
----------------------------	----

³⁰⁶⁾The exact-width and pointer-holding integer types are optional.
³⁰⁷⁾A freestanding implementation is not expected to provide all these types.

7.22.4.3 Width of `sig_atomic_t`

<code>SIG_ATOMIC_WIDTH</code>	8
-------------------------------	---

7.22.4.4 Width of `size_t`

<code>SIZE_WIDTH</code>	16
-------------------------	----

7.22.4.5 Width of `wchar_t`

<code>WCHAR_WIDTH</code>	8
--------------------------	---

7.22.4.6 Width of `wint_t`

<code>WINT_WIDTH</code>	16
-------------------------	----

7.22.5 Macros for integer constants

7.22.5.1 General

The following function-like macros expand to integer constants suitable for initializing objects that have integer types corresponding to types defined in `<stdint.h>`. Each macro name corresponds to a similar type name in 7.22.2.3 or 7.22.2.6.

The argument in any instance of these macros shall be an unsuffixed integer constant (as defined in 6.4.5.2) with a value that does not exceed the limits for the corresponding type.

Each invocation of one of these macros shall expand to an integer constant expression. The type of the expression shall have the same type as would an expression of the corresponding type converted according to the integer promotions. The value of the expression shall be that of the argument. If the value is in the range of the type `intmax_t` (for a signed type) or the type `uintmax_t` (for an unsigned type), see 7.22.2.6, the expression is suitable for use in conditional expression inclusion preprocessing directives.

7.22.5.2 Macros for minimum-width integer constants

The macro `INTN_C(value)` expands to an integer constant expression corresponding to the type `int_leastN_t`. The macro `UINTN_C(value)` expands to an integer constant expression corresponding to the type `uint_leastN_t`. For example, if `uint_least64_t` is a name for the type `unsigned long long int`, then `UINT64_C(0x123)` can expand to the integer constant `0x123ULL`.

7.22.5.3 Macros for greatest-width integer constants

The following macro expands to an integer constant expression having the value specified by its argument and the type `intmax_t`:

<code>INTMAX_C(value)</code>

The following macro expands to an integer constant expression having the value specified by its argument and the type `uintmax_t`:

<code>UINTMAX_C(value)</code>

7.22.6 Maximal and minimal values of integer types

For all integer types for which there is a macro with suffix `_WIDTH` holding the width, maximum macros with suffix `_MAX` and, for all signed types, minimum macros with suffix `_MIN` are defined as by 5.3.5.3. If it is unspecified if a type is signed or unsigned and the implementation has it as an unsigned type, a minimum macro with extension `_MIN` and value `0` of the corresponding type, converted according to the integer promotions, is defined.

7.23 Input/output <stdio.h>

7.23.1 Introduction

The header <stdio.h> defines several macros, and declares three types and many functions for performing input and output.

The macro

```
__STDC_VERSION_STDIO_H__
```

is an integer constant expression with a value equivalent to 202311L.

The types declared are **size_t** (described in 7.21);

```
FILE
```

which is an object type capable of recording all the information needed to control a stream, including its file position indicator, a pointer to its associated buffer (if any), an *error indicator* that records whether a read/write error has occurred, and an *end-of-file indicator* that records whether the end of the file has been reached; and

```
fpos_t
```

which is a complete object type other than an array type capable of recording all the information needed to specify uniquely every position within a file.

The macros are **NULL** (described in 7.21);

```
_IOFBF
_IOLBF
_IONBF
```

which expand to integer constant expressions with distinct values, suitable for use as the third argument to the **setvbuf** function;

```
BUFSIZ
```

which expands to an integer constant expression that is the size of the buffer used by the **setbuf** function;

```
EOF
```

which expands to an integer constant expression, with type **int** and a negative value, that is returned by several functions to indicate *end-of-file*, that is, no more input from a stream;

```
FOPEN_MAX
```

which expands to an integer constant expression that is the minimum number of files that the implementation guarantees can be open simultaneously;

```
FILENAME_MAX
```

which expands to an integer constant expression that is the size needed for an array of **char** large enough to hold the longest file name string that the implementation guarantees can be opened or, if the implementation imposes no practical limit on the length of file name strings, the recommended size of an array intended to hold a file name string;³⁰⁸⁾

³⁰⁸⁾Of course, file name string contents are subject to other system-specific constraints; therefore *all* possible strings of length **FILENAME_MAX** cannot be expected to be opened successfully.

_PRINTF_NAN_LEN_MAX

which expands to an integer constant expression (suitable for use in conditional expression inclusion preprocessing directives) that is the maximum number of characters output for any

[-]NAN (*n-char-sequence*)

sequence.³⁰⁹⁾ If an implementation has no support for NaNs, it shall be 0. **_PRINTF_NAN_LEN_MAX** shall be less than 64;

L_tmpnam

which expands to an integer constant expression that is the size needed for an array of **char** large enough to hold a temporary file name string generated by the **tmpnam** function;

SEEK_CUR
SEEK_END
SEEK_SET

which expand to integer constant expressions with distinct values, suitable for use as the third argument to the **fseek** function;

TMP_MAX

which expands to an integer constant expression that is the minimum number of unique file names that can be generated by the **tmpnam** function;

stderr
stdin
stdout

which are expressions of type “pointer to **FILE**” that point to the **FILE** objects associated, respectively, with the standard error, input, and output streams.

The header `<wchar.h>` declares functions for wide character input and output. The wide character input/output functions described in that subclause provide operations analogous to most of those described here, except that the fundamental units internal to the program are wide characters. The external representation (in the file) is a sequence of generalized multibyte characters, as described further in 7.23.3.

The input/output functions are given the following collective terms:

- The *wide character input functions* — those functions described in 7.31 that perform input into wide characters and wide strings: **fgetwc**, **fgetws**, **getwc**, **getwchar**, **fwscanf**, **wscanf**, **vfwscanf**, and **vwscanf**.
- The *wide character output functions* — those functions described in 7.31 that perform output from wide characters and wide strings: **fputwc**, **fputws**, **putwc**, **putwchar**, **fwprintf**, **wprintf**, **vfwprintf**, and **vwprintf**.
- The *wide character input/output functions* — the union of the **ungetwc** function, the wide character input functions, and the wide character output functions.
- The *byte input/output functions* — those functions described in this subclause that perform input/output: **fgetc**, **fgets**, **fprintf**, **fputc**, **fputs**, **fread**, **fscanf**, **fwrite**, **getc**, **getchar**, **printf**, **putc**, **putchar**, **puts**, **scanf**, **ungetc**, **vfprintf**, **vfscanf**, **vprintf**, and **vscanf**.

Forward references: files (7.23.3), the **fseek** function (7.23.9.2), streams (7.23.2), the **tmpnam** function (7.23.4.4), `<wchar.h>` (7.31).

³⁰⁹⁾If the implementation only uses the [-]NAN style, then **_PRINTF_NAN_LEN_MAX** would have the value 4.

7.23.2 Streams

Input and output, whether to or from physical devices such as terminals and tape drives, or whether to or from files supported on structured storage devices, are mapped into logical data *streams*, whose properties are more uniform than their various inputs and outputs. Two forms of mapping are supported, for *text streams* and for *binary streams*.³¹⁰⁾

A text stream is an ordered sequence of characters composed into *lines*, each line consisting of zero or more characters plus a terminating new-line character. Whether the last line requires a terminating new-line character is implementation-defined. Characters can be added, altered, or deleted on input and output to conform to differing conventions for representing text in the host environment. Thus, there is not required to be a one-to-one correspondence between the characters in a stream and those in the external representation. Data read in from a text stream will necessarily compare equal to the data that were earlier written out to that stream only if:

- the data consist only of printing characters and the control characters horizontal tab and new-line;
- no new-line character is immediately preceded by space characters;
- and, the last character is a new-line character.

Whether space characters that are written out immediately before a new-line character appear when read in is implementation-defined.

A binary stream is an ordered sequence of characters that can transparently record internal data. Data read in from a binary stream shall compare equal to the data that were earlier written out to that stream, under the same implementation. Such a stream may, however, have an implementation-defined number of null characters appended to the end of the stream.

Each stream has an *orientation*. After a stream is associated with an external file, but before any operations are performed on it, the stream is unoriented. Once a wide character input/output function has been applied to an unoriented stream, the stream becomes a *wide-oriented stream*. Similarly, once a byte input/output function has been applied to an unoriented stream, the stream becomes a *byte-oriented stream*. Only a call to the **freopen** function or the **fwide** function can otherwise alter the orientation of a stream. (A successful call to **freopen** removes any orientation.)³¹¹⁾

Byte input/output functions shall not be applied to a wide-oriented stream and wide character input/output functions shall not be applied to a byte-oriented stream. The remaining stream operations do not affect, and are not affected by, a stream's orientation, except for the following additional restrictions:

- Binary wide-oriented streams have the file-positioning restrictions ascribed to both text and binary streams.
- For wide-oriented streams, after a successful call to a file-positioning function that leaves the file position indicator prior to the end-of-file, a wide character output function can overwrite a partial multibyte character; any file contents beyond the byte(s) written can henceforth not consist of valid multibyte characters.

Each wide-oriented stream has an associated **mbstate_t** object that stores the current parse state of the stream. A successful call to **fgetpos** stores a representation of the value of this **mbstate_t** object as part of the value of the **fpos_t** object. A later successful call to **fsetpos** using the same stored **fpos_t** value restores the value of the associated **mbstate_t** object as well as the position within the controlled stream.

Each stream has an associated lock that is used to prevent data races when multiple threads of execution access a stream, and to restrict the interleaving of stream operations performed by multiple

³¹⁰⁾ An implementation does not need to distinguish between text streams and binary streams. In such an implementation, there need be no new-line characters in a text stream nor any limit to the length of a line.

³¹¹⁾ The three predefined streams **stdin**, **stdout**, and **stderr** are unoriented at program startup.

threads. Only one thread can hold this lock at a time. The lock is reentrant: a single thread can hold the lock multiple times at a given time.

All functions that read, write, position, or query the position of a stream lock the stream before accessing it. They release the lock associated with the stream when the access is complete.

Environmental limits

An implementation shall support text files with lines containing at least 254 characters, including the terminating new-line character. The value of the macro **BUFSIZ** shall be at least 256.

Forward references: the **freopen** function (7.23.5.4), the **fwide** function (7.31.3.5), **mbstate_t** (7.31.1), the **fgetpos** function (7.23.9.1), the **fsetpos** function (7.23.9.3).

7.23.3 Files

A stream is associated with an external file (which can be a physical device) by *opening* a file, which can involve *creating* a new file. Creating an existing file causes its former contents to be discarded, if necessary. If a file can support positioning requests (such as a disk file, as opposed to a terminal), then a *file position indicator* associated with the stream is positioned at the start (character number zero) of the file, unless the file is opened with append mode in which case it is implementation-defined whether the file position indicator is initially positioned at the beginning or the end of the file. The file position indicator is maintained by subsequent reads, writes, and positioning requests, to facilitate an orderly progression through the file.

Binary files are not truncated, except as defined in 7.23.5.3. Whether a write on a text stream causes the associated file to be truncated beyond that point is implementation-defined.

When a stream is *unbuffered*, characters are intended to appear from the source or at the destination as soon as possible. Otherwise characters may be accumulated and transmitted to or from the host environment as a block. When a stream is *fully buffered*, characters are intended to be transmitted to or from the host environment as a block when a buffer is filled. When a stream is *line buffered*, characters are intended to be transmitted to or from the host environment as a block when a new-line character is encountered. Furthermore, characters are intended to be transmitted as a block to the host environment when a buffer is filled, when input is requested on an unbuffered stream, or when input is requested on a line buffered stream that requires the transmission of characters from the host environment. Support for these characteristics is implementation-defined, and can be affected via the **setbuf** and **setvbuf** functions.

A file may be disassociated from a controlling stream by *closing* the file. Output streams are flushed (any unwritten buffer contents are transmitted to the host environment) before the stream is disassociated from the file. The lifetime of a **FILE** object ends when the associated file is closed (including the standard text streams). Whether a file of zero length (on which no characters have been written by an output stream) actually exists is implementation-defined.

The file may be subsequently reopened, by the same or another program execution, and its contents reclaimed or modified (if it can be repositioned at its start). If the **main** function returns to its original caller, or if the **exit** function is called, all open files are closed (hence all output streams are flushed) before program termination. Other paths to program termination, such as calling the **abort** function, are not required to close all files properly.

The address of the **FILE** object used to control a stream can be significant; a copy of a **FILE** object is not required to serve in place of the original.

At program startup, three text streams are predefined and are already opened — *standard input* (for reading conventional input), *standard output* (for writing conventional output), and *standard error* (for writing diagnostic output). As initially opened, the standard error stream is not fully buffered; the standard input and standard output streams are fully buffered if and only if the stream can be determined not to refer to an interactive device.

Functions that open additional (nontemporary) files require a *file name*, which is a string. The rules for composing valid file names are implementation-defined. Whether the same file can be simultaneously open multiple times is also implementation-defined.

Although both text and binary wide-oriented streams are conceptually sequences of wide characters, the external file associated with a wide-oriented stream is a sequence of multibyte characters, generalized as follows:

- Multibyte encodings within files can contain embedded null bytes (unlike multibyte encodings valid for use internal to the program).
- A file is not required to begin nor end in the initial shift state.³¹²⁾

Moreover, the encodings used for multibyte characters can differ among files. Both the nature and choice of such encodings are implementation-defined.

The wide character input functions read multibyte characters from the stream and convert them to wide characters as if they were read by successive calls to the **fgetwc** function. Each conversion occurs as if by a call to the **mbrtowc** function, with the conversion state described by the stream's own **mbstate_t** object. The byte input functions read characters from the stream as if by successive calls to the **fgetc** function.

The wide character output functions convert wide characters to multibyte characters and write them to the stream as if they were written by successive calls to the **fputwc** function. Each conversion occurs as if by a call to the **wcrtomb** function, with the conversion state described by the stream's own **mbstate_t** object. The byte output functions write characters to the stream as if by successive calls to the **fputc** function.

In some cases, some of the byte input/output functions also perform conversions between multibyte characters and wide characters. These conversions also occur as if by calls to the **mbrtowc** and **wcrtomb** functions.

An *encoding error* occurs if the character sequence presented to the underlying **mbrtowc** function does not form a valid (generalized) multibyte character, or if the code value passed to the underlying **wcrtomb** does not correspond to a valid (generalized) multibyte character. The wide character input/output functions and the byte input/output functions store the value of the macro **EILSEQ** in **errno** if and only if an encoding error occurs.

Environmental limits

The value of **FOPEN_MAX** shall be at least eight, including the three standard text streams.

Forward references: the **exit** function (7.24.5.4), the **fgetc** function (7.23.7.1), the **fopen** function (7.23.5.3), the **fputc** function (7.23.7.3), the **setbuf** function (7.23.5.5), the **setvbuf** function (7.23.5.6), the **fgetwc** function (7.31.3.1), the **fputwc** function (7.31.3.3), conversion state (7.31.6), the **mbrtowc** function (7.31.6.4.3), the **wcrtomb** function (7.31.6.4.4).

7.23.4 Operations on files

7.23.4.1 The remove function

Synopsis

```
#include <stdio.h>
int remove(const char *filename);
```

Description

The **remove** function causes the file whose name is the string pointed to by **filename** to be no longer accessible by that name. A subsequent attempt to open that file using that name will fail, unless it is created anew. If the file is open, the behavior of the **remove** function is implementation-defined.

Returns

The **remove** function returns zero if the operation succeeds, nonzero if it fails.

³¹²⁾Setting the file position indicator to end-of-file, as with **fseek(file, 0, SEEK_END)**, has undefined behavior for a binary stream (because of possible trailing null characters) or for any stream with state-dependent encoding that does not assuredly end in the initial shift state.

7.23.4.2 The rename function

Synopsis

```
#include <stdio.h>
int rename(const char *old, const char *new);
```

Description

The **rename** function causes the file whose name is the string pointed to by **old** to be henceforth known by the name given by the string pointed to by **new**. The file named **old** is no longer accessible by that name. If a file named by the string pointed to by **new** exists prior to the call to the **rename** function, the behavior is implementation-defined.

Returns

The **rename** function returns zero if the operation succeeds, nonzero if it fails,³¹³⁾ in which case if the file existed previously it is still known by its original name.

7.23.4.3 The tmpfile function

Synopsis

```
#include <stdio.h>
FILE *tmpfile(void);
```

Description

The **tmpfile** function creates a temporary binary file that is different from any other existing file and that will automatically be removed when it is closed or at program termination. If the program terminates abnormally, whether an open temporary file is removed is implementation-defined. The file is opened for update with "**wb+**" mode.

Recommended practice

It should be possible to open at least **TMP_MAX** temporary files during the lifetime of the program (this limit may be shared with **tmpnam**) and there should be no limit on the number simultaneously open other than this limit and any limit on the number of open files (**FOPEN_MAX**).

Returns

The **tmpfile** function returns a pointer to the stream of the file that it created. If the file cannot be created, the **tmpfile** function returns a null pointer.

Forward references: the **fopen** function (7.23.5.3).

7.23.4.4 The tmpnam function

Synopsis

```
#include <stdio.h>
char *tmpnam(char *s);
```

Description

The **tmpnam** function generates a string that is a valid file name and that is not the same as the name of an existing file.³¹⁴⁾ The function is potentially capable of generating at least **TMP_MAX** different strings, but any or all of them can already be in use by existing files and thus not be suitable return values.

The **tmpnam** function generates a different string each time it is called.

³¹³⁾ Among the reasons the implementation can cause the **rename** function to fail are that the file is open or that it is necessary to copy its contents to effectuate its renaming.

³¹⁴⁾ Files created using strings generated by the **tmpnam** function are temporary only in the sense that their names are not expected to collide with those generated by conventional naming rules for the implementation. It is still necessary to use the **remove** function to remove such files when their use is ended, and before program termination.

Calls to the **tmpnam** function with a null pointer argument may introduce data races with each other. The implementation shall behave as if no library function calls the **tmpnam** function.

Returns

If no suitable string can be generated, the **tmpnam** function returns a null pointer. Otherwise, if the argument is a null pointer, the **tmpnam** function leaves its result in an internal static object and returns a pointer to that object (subsequent calls to the **tmpnam** function may modify the same object). If the argument is not a null pointer, it is assumed to point to an array of at least **L_tmpnam** chars; the **tmpnam** function writes its result in that array and returns the argument as its value.

Environmental limits

The value of the macro **TMP_MAX** shall be at least 25.

7.23.5 File access functions

7.23.5.1 The **fclose** function

Synopsis

```
#include <stdio.h>
int fclose(FILE *stream);
```

Description

A successful call to the **fclose** function causes the stream pointed to by **stream** to be flushed and the associated file to be closed. Any unwritten buffered data for the stream are delivered to the host environment to be written to the file; any unread buffered data are discarded. Whether the call succeeds or not, the stream is disassociated from the file and any buffer set by the **setbuf** or **setvbuf** function is disassociated from the stream (and deallocated if it was automatically allocated).

Returns

The **fclose** function returns zero if the stream was successfully closed, or **EOF** if any errors were detected.

7.23.5.2 The **fflush** function

Synopsis

```
#include <stdio.h>
int fflush(FILE *stream);
```

Description

If **stream** points to an output stream or an update stream in which the most recent operation was not input, the **fflush** function causes any unwritten data for that stream to be delivered to the host environment to be written to the file; otherwise, the behavior is undefined.

If **stream** is a null pointer, the **fflush** function performs this flushing action on all streams for which the behavior is defined previously in this subclause.

Returns

The **fflush** function sets the error indicator for the stream and returns **EOF** if a write error occurs, otherwise it returns zero.

Forward references: the **fopen** function (7.23.5.3).

7.23.5.3 The **fopen** function

Synopsis

```
#include <stdio.h>
FILE *fopen(const char * restrict filename, const char * restrict mode);
```

Description

The **fopen** function opens the file whose name is the string pointed to by **filename**, and associates a stream with it.

The argument **mode** points to a string. If the string is one of the following, the file is open in the indicated mode. Otherwise, the behavior is undefined.³¹⁵⁾

r	open text file for reading
w	truncate to zero length or create text file for writing
wx	create text file for writing
a	append; open or create text file for writing at end-of-file
rb	open binary file for reading
wb	truncate to zero length or create binary file for writing
wbx	create binary file for writing
ab	append; open or create binary file for writing at end-of-file
r+	open text file for update (reading and writing)
w+	truncate to zero length or create text file for update
w+x	create text file for update
a+	append; open or create text file for update, writing at end-of-file
r+b or rb+	open binary file for update (reading and writing)
w+b or wb+	truncate to zero length or create binary file for update
w+bx or wb+x	create binary file for update
a+b or ab+	append; open or create binary file for update, writing at end-of-file

Opening a file with read mode (**'r'** as the first character in the **mode** argument) fails if the file does not exist or cannot be read.

Opening a file with exclusive mode (**'x'** as the last character in the **mode** argument) fails if the file already exists or cannot be created. The check for the existence of the file and the creation of the file if it does not exist is atomic with respect to other threads and other concurrent program executions. If the implementation is not capable of performing the check for the existence of the file and the creation of the file atomically, it shall fail instead of performing a non-atomic check and creation.

Opening a file with append mode (**'a'** as the first character in the **mode** argument) causes all subsequent writes to the file to be forced to the then current end-of-file at the point of buffer flush or actual write, regardless of intervening calls to the **fseek**, **fsetpos**, or **rewind** functions. Incrementing the current end-of-file by the amount of data written is atomic with respect to other threads writing to the same file provided the file was also opened in append mode. If the implementation is not capable of incrementing the current end-of-file atomically, it shall fail instead of performing non-atomic end-of-file writes. In some implementations, opening a binary file with append mode (**'b'** as the second or third character in the previously described list of **mode** argument values) may initially position the file position indicator for the stream beyond the last data written, because of null character padding.

When a file is opened with update mode (**'+'** as the second or third character in the previously described list of **mode** argument values), both input and output can be performed on the associated stream. However, output shall not be directly followed by input without an intervening call to the **fflush** function or to a file positioning function (**fseek**, **fsetpos**, or **rewind**), and input shall not be directly followed by output without an intervening call to a file positioning function, unless the input operation encounters end-of-file. Opening (or creating) a text file with update mode may instead open (or create) a binary stream in some implementations.

When opened, a stream is fully buffered if and only if it can be determined not to refer to an interactive device. The error and end-of-file indicators for the stream are cleared.

³¹⁵⁾ If the string begins with one of the listed mode sequences, the implementation can choose to ignore the remaining characters, or it can use them to select different kinds of a file (some of which can not conform to the properties in 7.23.2).

Returns

The **fopen** function returns a pointer to the object controlling the stream. If the open operation fails, **fopen** returns a null pointer.

Forward references: file positioning functions (7.23.9).

7.23.5.4 The freopen function

Synopsis

```
#include <stdio.h>
FILE *freopen(const char * restrict filename, const char * restrict mode,
FILE * restrict stream);
```

Description

The **freopen** function opens the file whose name is the string pointed to by **filename** and associates the stream pointed to by **stream** with it. The **mode** argument is used just as in the **fopen** function.³¹⁶⁾

If **filename** is a null pointer, the **freopen** function attempts to change the mode of the stream to that specified by **mode**, as if the name of the file currently associated with the stream had been used. It is implementation-defined which changes of mode are permitted (if any), and under what circumstances.

The **freopen** function first attempts to close any file that is associated with the specified stream. Failure to close the file is ignored. The error and end-of-file indicators for the stream are cleared.

Returns

The **freopen** function returns a null pointer if the open operation fails. Otherwise, **freopen** returns the value of **stream**.

7.23.5.5 The setbuf function

Synopsis

```
#include <stdio.h>
void setbuf(FILE * restrict stream, char * restrict buf);
```

Description

Except that it returns no value, the **setbuf** function is equivalent to the **setvbuf** function invoked with the values **_IOFBF** for **mode** and **BUFSIZ** for **size**, or (if **buf** is a null pointer), with the value **_IONBF** for **mode**.

Returns

The **setbuf** function returns no value.

Forward references: the **setvbuf** function (7.23.5.6).

7.23.5.6 The setvbuf function

Synopsis

```
#include <stdio.h>
int setvbuf(FILE * restrict stream, char * restrict buf, int mode, size_t size);
```

Description

The **setvbuf** function may be used only after the stream pointed to by **stream** has been associated with an open file and before any other operation (other than an unsuccessful call to **setvbuf**) is performed on the stream. The argument **mode** determines how **stream** will be buffered, as follows:

³¹⁶⁾The primary use of the **freopen** function is to change the file associated with a standard text stream (**stderr**, **stdin**, or **stdout**), as those identifiers are not required to be modifiable lvalues to which the value returned by the **fopen** function can be assigned.

- _IOFBF** causes input/output to be fully buffered;
- _IOLBF** causes input/output to be line buffered;
- _IONBF** causes input/output to be unbuffered.

If **buf** is not a null pointer, the array it points to can be used instead of a buffer allocated by the **setvbuf** function³¹⁷⁾ and the argument **size** specifies the size of the array; otherwise, **size** may determine the size of a buffer allocated by the **setvbuf** function. The members of the array at any time have unspecified values.

Returns

The **setvbuf** function returns zero on success, or nonzero if an invalid value is given for **mode** or if the request cannot be honored.

7.23.6 Formatted input/output functions

7.23.6.1 General

The formatted input/output functions shall behave as if there is a sequence point after the actions associated with each specifier.³¹⁸⁾

7.23.6.2 The **fprintf** function

Synopsis

```
#include <stdio.h>
int fprintf(FILE * restrict stream, const char * restrict format, ...);
```

Description

The **fprintf** function writes output to the stream pointed to by **stream**, under control of the string pointed to by **format** that specifies how subsequent arguments are converted for output. If there are insufficient arguments for the format, the behavior is undefined. If the format is exhausted while arguments remain, the excess arguments are evaluated (as always) but are otherwise ignored. The **fprintf** function returns when the end of the format string is encountered.

The format shall be a multibyte character sequence, beginning and ending in its initial shift state. The format is composed of zero or more directives: ordinary multibyte characters (not %), which are copied unchanged to the output stream; and conversion specifications, each of which results in fetching zero or more subsequent arguments, converting them, if applicable, according to the corresponding conversion specifier, and then writing the result to the output stream.

Each conversion specification is introduced by the character %. After the %, the following appear in sequence:

- Zero or more *flags* (in any order) that modify the meaning of the conversion specification.
- An optional minimum *field width*. If the converted value has fewer characters than the field width, it is padded with spaces (by default) on the left (or right, if the left adjustment flag, described later, has been given) to the field width. The field width takes the form of an asterisk * (described later) or a nonnegative decimal integer.³¹⁹⁾
- An optional *precision* that gives the minimum number of digits to appear for the b, B, d, i, o, u, x, and X conversions, the number of digits to appear after the decimal-point character for a, A, e, E, f, and F conversions, the maximum number of significant digits for the g and G conversions, or the maximum number of bytes to be written for s conversions. The precision takes the form of a period (.) followed either by an asterisk * (described later) or by an optional nonnegative decimal integer; if only the period is specified, the precision is taken as zero. If a precision appears with any other conversion specifier, the behavior is undefined.

³¹⁷⁾The buffer has to have a lifetime at least as great as the open stream, so not closing the stream before a buffer that has automatic storage duration is deallocated upon block exit results in undefined behavior.

³¹⁸⁾The **fprintf** functions perform writes to memory for the %n specifier.

³¹⁹⁾0 is taken as a flag, not as the beginning of a field width.

- An optional *length modifier* that specifies the size of the argument.
- A *conversion specifier* character that specifies the type of conversion to be applied.

As noted previously, a field width, or precision, or both, may be indicated by an asterisk. In this case, an **int** argument supplies the field width or precision. The arguments specifying field width, or precision, or both, shall appear (in that order) before the argument (if any) to be converted. A negative field width argument is taken as a - flag followed by a positive field width. A negative precision argument is taken as if the precision were omitted.

The flag characters and their meanings are:

- The result of the conversion is left-justified within the field. (It is right-justified if this flag is not specified.)
- + The result of a signed conversion always begins with a plus or minus sign. (It begins with a sign only when a value with a negative sign is converted if this flag is not specified.)³²⁰⁾
- space* If the first character of a signed conversion is not a sign, or if a signed conversion results in no characters, a space is prefixed to the result. If the *space* and + flags both appear, the *space* flag is ignored.
- # The result is converted to an “alternative form”. For o conversion, it increases the precision, if and only if necessary, to force the first digit of the result to be a zero (if the value and precision are both 0, a single 0 is printed). For b conversion, a nonzero result has **0b** prefixed to it. For the optional **B** conversion as described later in this subclause, a nonzero result has **0B** prefixed to it. For x (or X) conversion, a nonzero result has 0x (or 0X) prefixed to it. For a, A, e, E, f, F, g, and G conversions, the result of converting a floating-point number always contains a decimal-point character, even if no digits follow it. (Normally, a decimal-point character appears in the result of these conversions only if a digit follows it.) For g and G conversions, trailing zeros are *not* removed from the result. For other conversions, the behavior is undefined.
- 0 For b, B, d, i, o, u, x, X, a, A, e, E, f, F, g, and G conversions, leading zeros (following any indication of sign or base) are used to pad to the field width rather than performing space padding, except when converting an infinity or NaN. If the 0 and - flags both appear, the 0 flag is ignored. For b, B, d, i, o, u, x, and X conversions, if a precision is specified, the 0 flag is ignored. For other conversions, the behavior is undefined.

The length modifiers and their meanings are:

- hh Specifies that a following b, B, d, i, o, u, x, or X conversion specifier applies to a **signed char** or **unsigned char** argument (the argument will have been promoted according to the integer promotions, but its value shall be converted to **signed char** or **unsigned char** before printing); or that a following n conversion specifier applies to a pointer to a **signed char** argument.
- h Specifies that a following b, B, d, i, o, u, x, or X conversion specifier applies to a **short int** or **unsigned short int** argument (the argument will have been promoted according to the integer promotions, but its value shall be converted to **short int** or **unsigned short int** before printing); or that a following n conversion specifier applies to a pointer to a **short int** argument.
- l (ell) Specifies that a following b, B, d, i, o, u, x, or X conversion specifier applies to a **long int** or **unsigned long int** argument; that a following n conversion specifier applies to a pointer to a **long int** argument; that a following c conversion specifier applies to a **wint_t** argument; that a following s conversion specifier applies to a pointer to a **wchar_t** argument; or has no effect on a following a, A, e, E, f, F, g, or G conversion specifier.

³²⁰⁾The results of all floating conversions of a negative zero, and of negative values that round to zero, include a minus sign.

ll (ell-ell)	Specifies that a following <i>b</i> , <i>B</i> , <i>d</i> , <i>i</i> , <i>o</i> , <i>u</i> , <i>x</i> , or <i>X</i> conversion specifier applies to a long long int or unsigned long long int argument; or that a following <i>n</i> conversion specifier applies to a pointer to a long long int argument.
j	Specifies that a following <i>b</i> , <i>B</i> , <i>d</i> , <i>i</i> , <i>o</i> , <i>u</i> , <i>x</i> , or <i>X</i> conversion specifier applies to an intmax_t or uintmax_t argument; or that a following <i>n</i> conversion specifier applies to a pointer to an intmax_t argument.
z	Specifies that a following <i>b</i> , <i>B</i> , <i>d</i> , <i>i</i> , <i>o</i> , <i>u</i> , <i>x</i> , or <i>X</i> conversion specifier applies to a size_t or the corresponding signed integer type argument; or that a following <i>n</i> conversion specifier applies to a pointer to a signed integer type corresponding to size_t argument.
t	Specifies that a following <i>b</i> , <i>B</i> , <i>d</i> , <i>i</i> , <i>o</i> , <i>u</i> , <i>x</i> , or <i>X</i> conversion specifier applies to a ptrdiff_t or the corresponding unsigned integer type argument; or that a following <i>n</i> conversion specifier applies to a pointer to a ptrdiff_t argument.
wN	Specifies that a following <i>b</i> , <i>B</i> , <i>d</i> , <i>i</i> , <i>o</i> , <i>u</i> , <i>x</i> , or <i>X</i> conversion specifier applies to an integer argument with a specific width where <i>N</i> is a positive decimal integer with no leading zeros (the argument will have been promoted according to the integer promotions, but its value shall be converted to the unpromoted type); or that a following <i>n</i> conversion specifier applies to a pointer to an integer type argument with a width of <i>N</i> bits. All minimum-width integer types (7.22.2.3) and exact-width integer types (7.22.2.2) defined in the header <code><stdint.h></code> shall be supported. Other supported values of <i>N</i> are implementation-defined.
wfN	Specifies that a following <i>b</i> , <i>B</i> , <i>d</i> , <i>i</i> , <i>o</i> , <i>u</i> , <i>x</i> , or <i>X</i> conversion specifier applies to a fastest minimum-width integer argument with a specific width where <i>N</i> is a positive decimal integer with no leading zeros (the argument will have been promoted according to the integer promotions, but its value shall be converted to the unpromoted type); or that a following <i>n</i> conversion specifier applies to a pointer to a fastest minimum-width integer type argument with a width of <i>N</i> bits. All fastest minimum-width integer types (7.22.2.4) defined in the header <code><stdint.h></code> shall be supported. Other supported values of <i>N</i> are implementation-defined.
L	Specifies that a following <i>a</i> , <i>A</i> , <i>e</i> , <i>E</i> , <i>f</i> , <i>F</i> , <i>g</i> , or <i>G</i> conversion specifier applies to a long double argument.
H	Specifies that a following <i>a</i> , <i>A</i> , <i>e</i> , <i>E</i> , <i>f</i> , <i>F</i> , <i>g</i> , or <i>G</i> conversion specifier applies to a _Decimal32 argument.
D	Specifies that a following <i>a</i> , <i>A</i> , <i>e</i> , <i>E</i> , <i>f</i> , <i>F</i> , <i>g</i> , or <i>G</i> conversion specifier applies to a _Decimal64 argument.
DD	Specifies that a following <i>a</i> , <i>A</i> , <i>e</i> , <i>E</i> , <i>f</i> , <i>F</i> , <i>g</i> , or <i>G</i> conversion specifier applies to a _Decimal128 argument.

If a length modifier appears with any conversion specifier other than as specified previously, the behavior is undefined.

The conversion specifiers and their meanings are:

- d, i* The **int** argument is converted to signed decimal in the style *[-]dddd*. The precision specifies the minimum number of digits to appear; if the value being converted can be represented in fewer digits, it is expanded with leading zeros. The default precision is 1. The result of converting a zero value with a precision of zero is no characters.
- b, B, o, u, x, X* The **unsigned int** argument is converted to unsigned binary (*b* or *B*), unsigned octal (*o*), unsigned decimal (*u*), or unsigned hexadecimal notation (*x* or *X*) in the style *dddd*; the letters *abcdef* are used for *x* conversion and the letters *ABCDEF* for *X* conversion. The precision specifies the minimum number of digits to appear; if the value being converted can be represented in fewer digits, it is expanded with leading zeros. The default precision

is 1. The result of converting a zero value with a precision of zero is no characters. The specifier B is optional and provides the same functionality as b, except for the # flag as previously specified. The **PRIB** macros from `<inttypes.h>` shall only be defined if the implementation follows the specification as given here.

f, F A **double** argument representing a floating-point number is converted to decimal notation in the style `[-]ddd.ddd`, where the number of digits after the decimal-point character is equal to the precision specification. If the precision is missing, it is taken as 6; if the precision is zero and the # flag is not specified, no decimal-point character appears. If a decimal-point character appears, at least one digit appears before it. The value is rounded to the appropriate number of digits.

A **double** argument representing an infinity is converted in one of the styles `[-]inf` or `[-]infinity` — which style is implementation-defined. A **double** argument representing a NaN is converted in one of the styles `[-]nan` or `[-]nan(n-char-sequence)` — which style, and the meaning of any *n-char-sequence*, is implementation-defined. The F conversion specifier produces INF, INFINITY, or NAN instead of inf, infinity, or nan, respectively.³²¹⁾

e, E A **double** argument representing a floating-point number is converted in the style `[-]d.ddde±dd`, where there is one digit (which is nonzero if the argument is nonzero) before the decimal-point character and the number of digits after it is equal to the precision; if the precision is missing, it is taken as 6; if the precision is zero and the # flag is not specified, no decimal-point character appears. The value is rounded to the appropriate number of digits. The E conversion specifier produces a number with E instead of e introducing the exponent. The exponent always contains at least two digits, and only as many more digits as necessary to represent the exponent. If the value is zero, the exponent is zero.

A **double** argument representing an infinity or NaN is converted in the style of an f or F conversion specifier.

g, G A **double** argument representing a floating-point number is converted in style f or e (or in style F or E in the case of a G conversion specifier), depending on the value converted and the precision. Let *P* equal the precision if nonzero, 6 if the precision is omitted, or 1 if the precision is zero. Then, if a conversion with style E would have an exponent of *X*:

if $P > X \geq -4$, the conversion is with style f (or F) and precision $P - (X + 1)$.

otherwise, the conversion is with style e (or E) and precision $P - 1$.

Finally, unless the # flag is used, any trailing zeros are removed from the fractional portion of the result and the decimal-point character is removed if there is no fractional portion remaining.

A **double** argument representing an infinity or NaN is converted in the style of an f or F conversion specifier.

a, A A **double** argument representing a floating-point number is converted in the style `[-]0xh.hhhhp±d`, where there is one hexadecimal digit (which is nonzero if the argument is a normalized floating-point number and is otherwise unspecified) before the decimal-point character³²²⁾ and the number of hexadecimal digits after it is equal to the precision; if the

³²¹⁾When applied to infinite and NaN values, the -, +, and space flag characters have their usual meaning; the # and 0 flag characters have no effect.

³²²⁾Binary implementations can choose the hexadecimal digit to the left of the decimal-point character so that subsequent digits align to nibble (4-bit) boundaries. This implementation choice affects numerical values printed with a precision *P* that is insufficient to represent all values exactly. Implementations with different conventions about the most significant hexadecimal digit will round at different places, affecting the numerical value of the hexadecimal result. For example, possible printed output for the code

```
#include <stdio.h>
/* ... */
double x = 123.0;
printf("%.1a", x);
```

include "0x1.fp+6" and "0xf.6p+3" whose numerical values are 124 and 123, respectively. Portable code seeking identical numerical results on different platforms should avoid precisions *P* that require rounding.

precision is missing and **FLT_RADIX** is a power of 2, then the precision is sufficient for an exact representation of the value; if the precision is missing and **FLT_RADIX** is not a power of 2, then the precision is sufficient to distinguish³²³⁾ values of type **double**, except that trailing zeros may be omitted; if the precision is zero and the **#** flag is not specified, no decimal-point character appears. The letters **abcdef** are used for a conversion and the letters **ABCDEF** for A conversion. The A conversion specifier produces a number with **X** and **P** instead of **x** and **p**. The exponent always contains at least one digit, and only as many more digits as necessary to represent the decimal exponent of 2. If the value is zero, the exponent is zero.

A **double** argument representing an infinity or NaN is converted in the style of an **f** or **F** conversion specifier.

If an **H**, **D**, or **DD** modifier is present and the precision is missing, then for a decimal floating type argument represented by a triple of integers (s, c, q) , where n is the number of significant digits in the coefficient c ,

- if $-(n + 5) \leq q \leq 0$, use style **f** (or style **F** in the case of an A conversion specifier) with formatting precision equal to $-q$,
- otherwise, use style **e** (or style **E** in the case of an A conversion specifier) with formatting precision equal to $n - 1$, with the exceptions that if $c = 0$ then the digit-sequence in the exponent-part shall have the value q (rather than 0), and that the exponent is always expressed with the minimum number of digits required to represent its value (the exponent never contains a leading zero).

If the precision P is present (in the conversion specification) and is zero or at least as large as the precision p (5.3.5.3.3) of the decimal floating type, the conversion is as if the precision were missing. If the precision P is present (and nonzero) and less than the precision p of the decimal floating type, the conversion first obtains an intermediate result as follows, where n is the number of significant digits in the coefficient:

- If $n \leq P$, set the intermediate result to the input.
- If $n > P$, round the input value, according to the current rounding direction for decimal floating-point operations, to P decimal digits, with unbounded exponent range, representing the result with a P -digit integer coefficient when in the form (s, c, q) .

Convert the intermediate result in the manner described previously for the case where the precision is missing.

- c If no **l** length modifier is present, the **int** argument is converted to an **unsigned char**, and the resulting character is written.

If an **l** length modifier is present, the **wint_t** argument is converted as if by a call to the **wcrtomb** function with a pointer to storage of at least **MB_CUR_MAX** bytes, the **wint_t** argument converted to **wchar_t**, and an initial shift state.

- s If no **l** length modifier is present, the argument shall be a pointer to storage of character type.³²⁴⁾ Characters from the storage are written up to (but not including) the terminating null character. If the precision is specified, no more than that many bytes are written. If the precision is not specified or is greater than the size of the storage, the storage shall contain a null character.

If an **l** length modifier is present, the argument shall be a pointer to storage of **wchar_t** type. Wide characters from the storage are converted to multibyte characters (each as if by a call to the **wcrtomb** function, with the conversion state described by an **mbstate_t** object initialized to zero before the first wide character is converted) up to and including

³²³⁾The formatting precision P is sufficient to distinguish values of the source type if $16^P > b^p$ where b (not a power of 2) and p are the base and precision of the source type (5.3.5.3.3). A smaller P potentially suffices depending on the implementation's scheme for determining the digit to the left of the decimal-point character.

³²⁴⁾No special provisions are made for multibyte characters.

a terminating null wide character. The resulting multibyte characters are written up to (but not including) the terminating null character (byte). If no precision is specified, the storage shall contain a null wide character. If a precision is specified, no more than that many bytes are written (including shift sequences, if any), and the storage shall contain a null wide character if, to equal the multibyte character sequence length given by the precision, the function would need to access a wide character one past the end of the array. In no case is a partial multibyte character written.³²⁵⁾

- p The argument shall be a pointer to **void** or a pointer to a character type. The value of the pointer is converted to a sequence of printing characters, in an implementation-defined manner.
- n The argument shall be a pointer to signed integer whose type is specified by the length modifier, if any, for the conversion specification, or shall be **int** if no length modifier is specified for the conversion specification. The number of characters written to the output stream so far by this call to **fprintf** is stored into the integer object pointed to by the argument. No argument is converted, but one is consumed. If the conversion specification includes any flags, a field width, or a precision, the behavior is undefined.
- % A % character is written. No argument is converted. The complete conversion specification shall be %%.

If a conversion specification is invalid, the behavior is undefined.³²⁶⁾ **fprintf** shall behave as if it uses **va_arg** with a type argument naming the type resulting from applying the default argument promotions to the type corresponding to the conversion specification and then converting the result of the **va_arg** expansion to the type corresponding to the conversion specification.³²⁷⁾

In no case does a nonexistent or small field width cause truncation of a field; if the result of a conversion is wider than the field width, the field is expanded to contain the conversion result.

For a and A conversions, if **FLT_RADIX** is a power of 2, the value is correctly rounded to a hexadecimal floating number with the given precision.

Recommended practice

For a and A conversions, if **FLT_RADIX** is not a power of 2 and the result is not exactly representable in the given precision, the result should be one of the two adjacent numbers in hexadecimal floating style with the given precision, with the extra stipulation that the error should have a correct sign for the current rounding direction.

For e, E, f, F, g, and G conversions, if the number of significant decimal digits is at most the maximum value *M* of the **T_DECIMAL_DIG** macros (defined in `<float.h>`), then the result should be correctly rounded.³²⁸⁾ If the number of significant decimal digits is more than *M* but the source value is exactly representable with *M* digits, then the result should be an exact representation with trailing zeros. Otherwise, the source value is bounded by two adjacent decimal strings $L < U$, both having *M* significant digits; the value of the resultant decimal string *D* should satisfy $L \leq D \leq U$, with the extra stipulation that the error should have a correct sign for the current rounding direction.

The uppercase B format specifier is made optional by the previous description, because it used to be available for extensions in previous versions of this document. Implementations that did not use an uppercase B as their own extension before are encouraged to implement it as previously described.

Returns

The **fprintf** function returns the number of characters transmitted, or a negative value if an output or encoding error occurred or if the implementation does not support a specified width length modifier.

³²⁵⁾ Redundant shift sequences can result if multibyte characters have a state-dependent encoding.

³²⁶⁾ See "future library directions" (7.33.16).

³²⁷⁾ The behavior is undefined when the types differ as specified for **va_arg** 7.16.2.2.

³²⁸⁾ For binary-to-decimal conversion, the result format's values are the numbers representable with the given format specifier. The number of significant digits is determined by the format specifier, and in the case of fixed-point conversion by the source value as well.

Environmental limits

The number of characters that can be produced by any single conversion shall be at least 4095.

EXAMPLE 1 To print a date and time in the form "Sunday, July 3, 10:02" followed by π to five decimal places:

```
#include <math.h>
#include <stdio.h>
/* ... */
char *weekday, *month;    // pointers to strings
int day, hour, min;
fprintf(stdout, "%s, %s %d, %.2d:%.2d\n",
        weekday, month, day, hour, min);
fprintf(stdout, "pi = %.5f\n", 4 * atan(1.0));
```

EXAMPLE 2 In this example, multibyte characters do not have a state-dependent encoding, and the members of the extended character set that consist of more than one byte each consist of exactly two bytes, the first of which is denoted here by a $\square$ and the second by an uppercase letter.

Given the following wide string with length seven,

```
static wchar_t wstr[] = L"\squareYabcd\squareW";
```

the seven calls

```
fprintf(stdout, "|1234567890123|\n");
fprintf(stdout, "|%13ls|\n", wstr);
fprintf(stdout, "|%-13.9ls|\n", wstr);
fprintf(stdout, "|%13.10ls|\n", wstr);
fprintf(stdout, "|%13.11ls|\n", wstr);
fprintf(stdout, "|%13.15ls|\n", &wstr[2]);
fprintf(stdout, "|%13lc|\n", (wint_t) wstr[5]);
```

will print the following seven lines:

```
|1234567890123|
|  \squareYabcd\squareW|
|\squareYabcd\squareZ|
|  \squareYabcd\squareZ|
|  \squareYabcd\squareW|
|    abcd\squareW|
|          \squareZ|
```

EXAMPLE 3 Following are representations of **_Decimal64** arguments as triples (s, c, q) and the corresponding character sequences **fprintf** produces with "%Da":

(+1, 123, 0)	123
(-1, 123, 0)	-123
(+1, 123, -2)	1.23
(+1, 123, 1)	1.23e+3
(-1, 123, 1)	-1.23e+3
(+1, 123, -8)	0.00000123
(+1, 123, -9)	1.23e-7
(+1, 120, -8)	0.00000120
(+1, 120, -9)	1.20e-7
(+1, 1234567890123456, 0)	1234567890123456
(+1, 1234567890123456, 1)	1.234567890123456e+16
(+1, 1234567890123456, -1)	123456789012345.6
(+1, 1234567890123456, -21)	0.000001234567890123456
(+1, 1234567890123456, -22)	1.234567890123456e-7
(+1, 0, 0)	0
(-1, 0, 0)	-0
(+1, 0, -6)	0.000000
(+1, 0, -7)	0e-7
(+1, 0, 2)	0e+2
(+1, 5, -6)	0.000005
(+1, 50, -7)	0.0000050
(+1, 5, -7)	5e-7

To illustrate the effects of a precision specification, the sequence:

```
_Decimal32 x = 6543.00DF;          // (+1, 654300, -2)
fprintf(stdout, "%Ha\n", x);
fprintf(stdout, "%.6Ha\n", x);
fprintf(stdout, "%.5Ha\n", x);
fprintf(stdout, "%.4Ha\n", x);
fprintf(stdout, "%.3Ha\n", x);
fprintf(stdout, "%.2Ha\n", x);
fprintf(stdout, "%.1Ha\n", x);
fprintf(stdout, "%.0Ha\n", x);
```

assuming default rounding, results in:

6543.00
6543.00
6543.0
6543
6.54e+3
6.5e+3
7e+3
6543.00

To illustrate the effects of the exponent range, the sequence:

```
_Decimal32 x = 9543210e87DF;        // (+1, 9543210, 87)
_Decimal32 y = 9500000e90DF;        // (+1, 9500000, 90)
fprintf(stdout, "%.6Ha\n", x);
fprintf(stdout, "%.5Ha\n", x);
fprintf(stdout, "%.4Ha\n", x);
fprintf(stdout, "%.3Ha\n", x);
fprintf(stdout, "%.2Ha\n", x);
fprintf(stdout, "%.1Ha\n", x);
fprintf(stdout, "%.1Ha\n", y);
```

assuming default rounding, results in:

9.54321e+93
 9.5432e+93
 9.543e+93
 9.54e+93
 9.5e+93
 1e+94
 1e+97

To further illustrate the effects of the exponent range, the sequence:

```
_Decimal32 x = 9512345e90DF;    // (+1, 9512345, 90)
_Decimal32 y = 9512345e86DF;    // (+1, 9512345, 86)
fprintf(stdout, "%.3Ha\n", x);
fprintf(stdout, "%.2Ha\n", x);
fprintf(stdout, "%.1Ha\n", x);
fprintf(stdout, "%.2Ha\n", y);
```

assuming default rounding, results in:

9.51e+96
 9.5e+96
 1e+97
 9.5e+92

Forward references: conversion state (7.31.6), the `wcrtomb` function (7.31.6.4.4).

7.23.6.3 The `fscanf` function

Synopsis

```
#include <stdio.h>
int fscanf(FILE * restrict stream, const char * restrict format, ...);
```

Description

The `fscanf` function reads input from the stream pointed to by **stream**, under control of the string pointed to by **format** that specifies the admissible input sequences and how they are to be converted for assignment, using subsequent arguments as pointers to the objects to receive the converted input. If there are insufficient arguments for the format, the behavior is undefined. If the format is exhausted while arguments remain, the excess arguments are evaluated (as always) but are otherwise ignored.

The format shall be a multibyte character sequence, beginning and ending in its initial shift state. The format is composed of zero or more directives: one or more white-space characters, an ordinary multibyte character (neither % nor a white-space character), or a conversion specification. Each conversion specification is introduced by the character %. After the %, the following appear in sequence:

- An optional assignment-suppressing character ***.
- An optional decimal integer greater than zero that specifies the maximum field width (in characters).
- An optional *length modifier* that specifies the size of the receiving object.
- A *conversion specifier* character that specifies the type of conversion to be applied.

The `fscanf` function executes each directive of the format in turn. When all directives have been executed, or if a directive fails (as detailed later in this subclause), the function returns. Failures are described as input failures (due to the occurrence of an encoding error or the unavailability of input characters), or matching failures (due to inappropriate input).

A directive composed of white-space character(s) is executed by reading input up to the first non-white-space character (which remains unread), or until no more characters can be read. The directive never fails.

A directive that is an ordinary multibyte character is executed by reading the next characters of the stream. If any of those characters differ from the ones composing the directive, the directive fails and the differing and subsequent characters remain unread. Similarly, if end-of-file, an encoding error, or a read error prevents a character from being read, the directive fails.

A directive that is a conversion specification defines a set of matching input sequences, as described further in this subclause for each specifier. A conversion specification is executed in the following steps:

Input white-space characters are skipped, unless the specification includes a `[`, `c`, or `n` specifier.³²⁹⁾

An input item is read from the stream, unless the specification includes an `n` specifier. An input item is defined as the longest sequence of input characters which does not exceed any specified field width and which is, or is a prefix of, a matching input sequence.³³⁰⁾ The first character, if any, after the input item remains unread. If the length of the input item is zero, the execution of the directive fails; this condition is a matching failure unless end-of-file, an encoding error, or a read error prevented input from the stream, in which case it is an input failure.

Except in the case of a `%` specifier, the input item (or, in the case of a `%n` directive, the count of input characters) is converted to a type appropriate to the conversion specifier. If the input item is not a matching sequence, the execution of the directive fails: this condition is a matching failure. Unless assignment suppression was indicated by a `*`, the result of the conversion is placed in the object pointed to by the first argument following the **format** argument that has not already received a conversion result. If this object does not have an appropriate type, or if the result of the conversion cannot be represented in the object, the behavior is undefined.

The length modifiers and their meanings are:

- hh Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **signed char** or **unsigned char**.
- h Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **short int** or **unsigned short int**.
- l (ell) Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **long int** or **unsigned long int**; that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to an argument with type pointer to **double**; or that a following `c`, `s`, or `[` conversion specifier applies to an argument with type pointer to **wchar_t**.
- ll (ell-ell) Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **long long int** or **unsigned long long int**.
- j Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **intmax_t** or **uintmax_t**.
- z Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **size_t** or the corresponding signed integer type.
- t Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **ptrdiff_t** or the corresponding unsigned integer type.
- wN Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument which is a pointer to an integer with a specific width where *N* is a positive decimal integer with no leading zeros. All minimum-width integer types (7.22.2.3) and exact-width integer types (7.22.2.2) defined in the header `<stdint.h>` shall be supported. Other supported values of *N* are implementation-defined.

³²⁹⁾These white-space characters are not counted against a specified field width.
³³⁰⁾**fscanf** pushes back at most one input character onto the input stream. Therefore, some sequences that are acceptable to **strtod**, **strtol**, etc., are unacceptable to **fscanf**.

wfN	Specifies that a following b, d, i, o, u, x, X, or n conversion specifier applies to an argument which is a pointer to a fastest minimum-width integer with a specific width where N is a positive decimal integer with no leading zeros. All fastest minimum-width integer types (7.22.2.4) defined in the header <code><stdint.h></code> shall be supported. Other supported values of N are implementation-defined.
L	Specifies that a following a, A, e, E, f, F, g, or G conversion specifier applies to an argument with type pointer to long double .
H	Specifies that a following a, A, e, E, f, F, g, or G conversion specifier applies to an argument with type pointer to _Decimal32 .
D	Specifies that a following a, A, e, E, f, F, g, or G conversion specifier applies to an argument with type pointer to _Decimal64 .
DD	Specifies that a following a, A, e, E, f, F, g, or G conversion specifier applies to an argument with type pointer to _Decimal128 .

If a length modifier appears with any conversion specifier other than as specified previously, the behavior is undefined.

In the following, the type of the corresponding argument for a conversion specifier shall be a pointer to a type determined by the length modifiers, if any, or specified by the conversion specifier. The conversion specifiers and their meanings are:

d	Matches an optionally signed decimal integer, whose format is the same as expected for the subject sequence of the strtoul function with the value 10 for the base argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to int .
b	Matches an optionally signed binary integer, whose format is the same as expected for the subject sequence of the strtoul function with the value 2 for the base argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to unsigned int .
i	Matches an optionally signed integer, whose format is the same as expected for the subject sequence of the strtoul function with the value 0 for the base argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to int .
o	Matches an optionally signed octal integer, whose format is the same as expected for the subject sequence of the strtoul function with the value 8 for the base argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to unsigned int .
u	Matches an optionally signed decimal integer, whose format is the same as expected for the subject sequence of the strtoul function with the value 10 for the base argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to unsigned int .
x	Matches an optionally signed hexadecimal integer, whose format is the same as expected for the subject sequence of the strtoul function with the value 16 for the base argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to unsigned int .
a, e, f, g	Matches an optionally signed floating-point number, infinity, or NaN, whose format is the same as expected for the subject sequence of the strtod function. Unless a length modifier is specified, the corresponding argument shall be a pointer to float .
c	Matches a sequence of characters of exactly the number specified by the field width (1 if no field width is present in the directive).

331)

If no `l` length modifier is present, the corresponding argument shall be a pointer to **char**, **signed char**, **unsigned char**, or **void** that points to storage large enough to accept the sequence. No null character is added.

If an `l` length modifier is present, the input shall be a sequence of multibyte characters that begins in the initial shift state. Each multibyte character in the sequence is converted to a wide character as if by a call to the `mbrtowc` function, with the conversion state described by an `mbstate_t` object initialized to zero before the first multibyte character is converted. The corresponding argument shall be a pointer to storage of **wchar_t** large enough to accept the resulting sequence of wide characters. No null wide character is added.

s Matches a sequence of non-white-space characters.³³¹⁾

If no `l` length modifier is present, the corresponding argument shall be a pointer to **char**, **signed char**, **unsigned char**, or **void** that points to storage large enough to accept the sequence and a terminating null character, which will be added automatically.

If an `l` length modifier is present, the input shall be a sequence of multibyte characters that begins in the initial shift state. Each multibyte character is converted to a wide character as if by a call to the `mbrtowc` function, with the conversion state described by an `mbstate_t` object initialized to zero before the first multibyte character is converted. The corresponding argument shall be a pointer to storage of **wchar_t** large enough to accept the sequence and the terminating null wide character, which will be added automatically.

[Matches a nonempty sequence of characters from a set of expected characters (the *scanset*).³³¹⁾

If no `l` length modifier is present, the corresponding argument shall be a pointer to **char**, **signed char**, **unsigned char**, or **void** that points to storage large enough to accept the sequence and a terminating null character, which will be added automatically.

If an `l` length modifier is present, the input shall be a sequence of multibyte characters that begins in the initial shift state. Each multibyte character is converted to a wide character as if by a call to the `mbrtowc` function, with the conversion state described by an `mbstate_t` object initialized to zero before the first multibyte character is converted. The corresponding argument shall be a pointer that points to storage of **wchar_t** large enough to accept the sequence and the terminating null wide character, which will be added automatically.

The conversion specifier includes all subsequent characters in the **format** string, up to and including the matching right bracket (`]`). The characters between the brackets (the *scanlist*) compose the scanset, unless the character after the left bracket is a circumflex (`^`), in which case the scanset contains all characters that do not appear in the scanlist between the circumflex and the right bracket. If the conversion specifier begins with `[]` or `[^]`, the right bracket character is in the scanlist and the next following right bracket character is the matching right bracket that ends the specification; otherwise the first following right bracket character is the one that ends the specification. If a `-` character is in the scanlist and is not the first, nor the second where the first character is a `^`, nor the last character, the behavior is implementation-defined.

p Matches an implementation-defined set of sequences, which should be the same as the set of sequences that may be produced by the `%p` conversion of the `fprintf` function. The corresponding argument shall be a pointer to a pointer of **void**. The input item is converted to a pointer value in an implementation-defined manner. If the input item is a value converted earlier during the same program execution, the pointer that results shall compare equal to that value; otherwise the behavior of the `%p` conversion is undefined.

³³¹⁾No special provisions are made for multibyte characters in the matching rules used by the `c`, `s`, and `[` conversion specifiers — the extent of the input field is determined on a byte-by-byte basis. The resulting field is nevertheless a sequence of multibyte characters that begins in the initial shift state.

- n** No input is consumed. The corresponding argument shall be a pointer of a signed integer type. The number of characters read from the input stream so far by this call to the **fscanf** function is stored into the integer object pointed to by the argument. Execution of a **%n** directive does not increment the assignment count returned at the completion of execution of the **fscanf** function. No argument is converted, but one is consumed. If the conversion specification includes an assignment-suppressing character or a field width, the behavior is undefined.
- %** Matches a single **%** character; no conversion or assignment occurs. The complete conversion specification shall be **%%**.

If a conversion specification is invalid, the behavior is undefined.³³²⁾

The conversion specifiers **A**, **E**, **F**, **G**, and **X** are also valid and behave the same as, respectively, **a**, **e**, **f**, **g**, and **x**.

Trailing white-space characters (including new-line characters) are left unread unless matched by a directive. The success of literal matches and suppressed assignments is not directly determinable other than via the **%n** directive.

Returns

The **fscanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure or if the implementation does not support a specific width length modifier.

EXAMPLE 1 The call:

```
#include <stdio.h>
/* ... */
int n, i; float x; char name[50];
n = fscanf(stdin, "%d%f%s", &i, &x, name);
```

with the input line:

```
25 54.32E-1 thompson
```

will assign to **n** the value 3, to **i** the value 25, to **x** the value 5.432, and to **name** the sequence **thompson\0**.

EXAMPLE 2 The call:

```
#include <stdio.h>
/* ... */
int i; float x; char name[50];
fscanf(stdin, "%2d%f%d %[0123456789]", &i, &x, name);
```

with input:

```
56789 0123 56a72
```

will assign to **i** the value 56 and to **x** the value 789.0, will skip 0123, and will assign to **name** the sequence **56\0**. The next character read from the input stream will be **a**.

EXAMPLE 3 To accept repeatedly from **stdin** a quantity, a unit of measure, and an item name:

```
#include <stdio.h>
/* ... */
int count; float quant; char units[21], item[21];
do {
    count = fscanf(stdin, "%f%20s of %20s", &quant, units, item);
```

³³²⁾See "future library directions" (7.33.16).

```
fscanf(stdin,"%*[^\\n]");
} while (!feof(stdin) && !ferror(stdin));
```

If the `stdin` stream contains the following lines:

```
2 quarts of oil
-12.8degrees Celsius
lots of luck
10.0LBS      of
dirt
100ergs of energy
```

the execution of the preceding example will be analogous to the following assignments:

```
quant = 2; strcpy(units, "quarts"); strcpy(item, "oil");
count = 3;
quant = -12.8; strcpy(units, "degrees");
count = 2; // "C" fails to match "o"
count = 0; // "l" fails to match "%f"
quant = 10.0; strcpy(units, "LBS"); strcpy(item, "dirt");
count = 3;
count = 0; // "100e" fails to match "%f"
count = EOF;
```

EXAMPLE 4 In:

```
#include <stdio.h>
/* ... */
int d1, d2, n1, n2, i;
i = sscanf("123", "%d%n%d", &d1, &n1, &n2, &d2);
```

the value 123 is assigned to `d1` and the value 3 to `n1`. Because `%n` can never get an input failure, the value of 3 is also assigned to `n2`. The value of `d2` is not affected. The value 1 is assigned to `i`.

EXAMPLE 5 The call:

```
#include <stdio.h>
/* ... */
int n, i;
n = sscanf("foo %bar 42", "foo%bar%d", &i);
```

will assign to `n` the value 1 and to `i` the value 42 because input white-space characters are skipped for both the `%` and `d` conversion specifiers.

EXAMPLE 6 In these examples, multibyte characters do have a state-dependent encoding, and the members of the extended character set that consist of more than one byte each consist of exactly two bytes, the first of which is denoted here by a `□` and the second by an uppercase letter, but are only recognized as such when in the alternate shift state. The shift sequences are denoted by `↑` and `↓`, in which the first causes entry into the alternate shift state.

After the call:

```
#include <stdio.h>
/* ... */
char str[50];
fscanf(stdin, "%s", str);
```

with the input line:

```
a↑□□Y↓ bc
```

`str` will contain `↑□X□Y↓\0` assuming that none of the bytes of the shift sequences (or of the multibyte characters, in the more general case) appears to be a single-byte white-space character.

In contrast, after the call:

```
#include <stdio.h>
#include <stddef.h>
/* ... */
wchar_t wstr[50];
fscanf(stdin, "a%ls", wstr);
```

with the same input line, **wstr** will contain the two wide characters that correspond to **□X** and **□Y** and a terminating null wide character.

However, the call:

```
#include <stdio.h>
#include <stddef.h>
/* ... */
wchar_t wstr[50];
fscanf(stdin, "a↑□X↓%ls", wstr);
```

with the same input line will return zero due to a matching failure against the **↓** sequence in the format string.

Assuming that the first byte of the multibyte character **□X** is the same as the first byte of the multibyte character **□Y**, after the call:

```
#include <stdio.h>
#include <stddef.h>
/* ... */
wchar_t wstr[50];
fscanf(stdin, "a↑□Y↓%ls", wstr);
```

with the same input line, zero will again be returned, but **stdin** will be left with a partially consumed multibyte character.

Forward references: the **strtod**, **strtof**, and **strtold** functions (7.24.2.6), the **strtol**, **strtoll**, **strtoul**, and **strtoull** functions (7.24.2.8), conversion state (7.31.6), the **wcrtomb** function (7.31.6.4.4).

7.23.6.4 The printf function

Synopsis

```
#include <stdio.h>
int printf(const char * restrict format, ...);
```

Description

The **printf** function is equivalent to **fprintf** with the argument **stdout** interposed before the arguments to **printf**.

Returns

The **printf** function returns the number of characters transmitted, or a negative value if an output or encoding error occurred.

7.23.6.5 The scanf function

Synopsis

```
#include <stdio.h>
int scanf(const char * restrict format, ...);
```

Description

The **scanf** function is equivalent to **fscanf** with the argument **stdin** interposed before the arguments to **scanf**.

Returns

The **scanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **scanf** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.23.6.6 The **snprintf** function

Synopsis

```
#include <stdio.h>
int snprintf(char * restrict s, size_t n, const char * restrict format, ...);
```

Description

The **snprintf** function is equivalent to **fprintf**, except that the output is written into an array (specified by argument **s**) rather than to a stream. If **n** is zero, nothing is written, and **s** can be a null pointer. Otherwise, output characters beyond the **n-1st** are discarded rather than being written to the array, and a null character is written at the end of the characters actually written into the array. If copying takes place between objects that overlap, the behavior is undefined.

Returns

The **snprintf** function returns the number of characters that would have been written had **n** been sufficiently large, not counting the terminating null character, or a negative value if an encoding error occurred. Thus, the null-terminated output has been completely written if and only if the returned value is both nonnegative and less than **n**.

7.23.6.7 The **sprintf** function

Synopsis

```
#include <stdio.h>
int sprintf(char * restrict s, const char * restrict format, ...);
```

Description

The **sprintf** function is equivalent to **fprintf**, except that the output is written into an array (specified by the argument **s**) rather than to a stream. A null character is written at the end of the characters written; it is not counted as part of the returned value. If copying takes place between objects that overlap, the behavior is undefined.

Returns

The **sprintf** function returns the number of characters written in the array, not counting the terminating null character, or a negative value if an encoding error occurred.

7.23.6.8 The **sscanf** function

Synopsis

```
#include <stdio.h>
int sscanf(const char * restrict s, const char * restrict format, ...);
```

Description

The **sscanf** function is equivalent to **fscanf**, except that input is obtained from a string (specified by the argument **s**) rather than from a stream. Reaching the end of the string is equivalent to encountering end-of-file for the **fscanf** function. If copying takes place between objects that overlap, the behavior is undefined.

Returns

The **sscanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **sscanf** function returns the number of input

items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.23.6.9 The `vfprintf` function

Synopsis

```
#include <stdarg.h>
#include <stdio.h>
int vfprintf(FILE * restrict stream, const char * restrict format, va_list arg);
```

Description

The `vfprintf` function is equivalent to `fprintf`, with the variable argument list replaced by **arg**, which shall have been initialized by the `va_start` macro (and possibly subsequent `va_arg` invocations). The `vfprintf` function does not invoke the `va_end` macro.³³³⁾

Returns

The `vfprintf` function returns the number of characters transmitted, or a negative value if an output or encoding error occurred.

EXAMPLE The following shows the use of the `vfprintf` function in a general error-reporting routine.

```
#include <stdarg.h>
#include <stdio.h>

void error(char *function_name, char *format, ...)
{
    va_list args;

    va_start(args, format);
    // print out name of function causing error
    fprintf(stderr, "ERROR in %s: ", function_name);
    // print out remainder of message
    vfprintf(stderr, format, args);
    va_end(args);
}
```

7.23.6.10 The `vscanf` function

Synopsis

```
#include <stdarg.h>
#include <stdio.h>
int vscanf(FILE * restrict stream, const char * restrict format, va_list arg);
```

Description

The `vscanf` function is equivalent to `fscanf`, with the variable argument list replaced by **arg**, which shall have been initialized by the `va_start` macro (and possibly subsequent `va_arg` calls). The `vscanf` function does not invoke the `va_end` macro.³³³⁾

Returns

The `vscanf` function returns the value of the macro `EOF` if an input failure occurs before the first conversion (if any) has completed. Otherwise, the `vscanf` function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.23.6.11 The `vprintf` function

³³³⁾As the functions `vfprintf`, `vscanf`, `vprintf`, `vscanf`, `vsprintf`, `vsnprintf`, `vsprintf`, and `vsscanf` invoke the `va_arg` macro, **arg** after the return has an indeterminate representation.

Synopsis

```
#include <stdarg.h>
#include <stdio.h>
int vprintf(const char * restrict format, va_list arg);
```

Description

The **vprintf** function is equivalent to **printf**, with the variable argument list replaced by **arg**, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vprintf** function does not invoke the **va_end** macro.³³³⁾

Returns

The **vprintf** function returns the number of characters transmitted, or a negative value if an output or encoding error occurred.

7.23.6.12 The vscanf function

Synopsis

```
#include <stdarg.h>
#include <stdio.h>
int vscanf(const char * restrict format, va_list arg);
```

Description

The **vscanf** function is equivalent to **scanf**, with the variable argument list replaced by **arg**, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vscanf** function does not invoke the **va_end** macro.³³³⁾

Returns

The **vscanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **vscanf** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.23.6.13 The vsnprintf function

Synopsis

```
#include <stdarg.h>
#include <stdio.h>
int vsnprintf(char * restrict s, size_t n, const char * restrict format, va_list arg);
```

Description

The **vsnprintf** function is equivalent to **snprintf**, with the variable argument list replaced by **arg**, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vsnprintf** function does not invoke the **va_end** macro.³³³⁾ If copying takes place between objects that overlap, the behavior is undefined.

Returns

The **vsnprintf** function returns the number of characters that would have been written had **n** been sufficiently large, not counting the terminating null character, or a negative value if an encoding error occurred. Thus, the null-terminated output has been completely written if and only if the returned value is both nonnegative and less than **n**.

7.23.6.14 The vsprintf function

Synopsis

```
#include <stdarg.h>
#include <stdio.h>
```

```
int vsprintf(char * restrict s, const char * restrict format, va_list arg);
```

Description

The **vsprintf** function is equivalent to **sprintf**, with the variable argument list replaced by **arg**, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vsprintf** function does not invoke the **va_end** macro.³³³ If copying takes place between objects that overlap, the behavior is undefined.

Returns

The **vsprintf** function returns the number of characters written in the array, not counting the terminating null character, or a negative value if an encoding error occurred.

7.23.6.15 The vsscanf function

Synopsis

```
#include <stdarg.h>
#include <stdio.h>
int vsscanf(const char * restrict s, const char * restrict format, va_list arg);
```

Description

The **vsscanf** function is equivalent to **sscanf**, with the variable argument list replaced by **arg**, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vsscanf** function does not invoke the **va_end** macro.³³³

Returns

The **vsscanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **vsscanf** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.23.7 Character input/output functions

7.23.7.1 The fgetc function

Synopsis

```
#include <stdio.h>
int fgetc(FILE *stream);
```

Description

If the end-of-file indicator for the input stream pointed to by **stream** is not set and a next character is present, the **fgetc** function obtains that character as an **unsigned char** converted to an **int** and advances the associated file position indicator for the stream (if defined).

Returns

If the end-of-file indicator for the stream is set, or if the stream is at end-of-file, the end-of-file indicator for the stream is set and the **fgetc** function returns **EOF**. Otherwise, the **fgetc** function returns the next character from the input stream pointed to by **stream**. If a read error occurs, the error indicator for the stream is set and the **fgetc** function returns **EOF**.³³⁴

7.23.7.2 The fgets function

Synopsis

```
#include <stdio.h>
char *fgets(char * restrict s, int n, FILE * restrict stream);
```

³³⁴) An end-of-file and a read error can be distinguished by use of the **feof** and **ferror** functions.

Description

The **fgets** function reads at most one less than the number of characters specified by **n** from the stream pointed to by **stream** into the array pointed to by **s**. No additional characters are read after a new-line character (which is retained) or after end-of-file. A null character is written immediately after the last character read into the array. If **n** is negative or zero, the behavior is undefined.

Returns

The **fgets** function returns **s** if successful. If end-of-file is encountered and no characters have been read into the array, the contents of the array remain unchanged and a null pointer is returned. If a read error occurs during the operation, the members of the array have unspecified values and a null pointer is returned.

7.23.7.3 The **fputc** function

Synopsis

```
#include <stdio.h>
int fputc(int c, FILE *stream);
```

Description

The **fputc** function writes the character specified by **c** (converted to an **unsigned char**) to the output stream pointed to by **stream**, at the position indicated by the associated file position indicator for the stream (if defined), and advances the indicator appropriately. If the file cannot support positioning requests, or if the stream was opened with append mode, the character is appended to the output stream.

Returns

The **fputc** function returns the character written. If a write error occurs, the error indicator for the stream is set and **fputc** returns **EOF**.

7.23.7.4 The **fputs** function

Synopsis

```
#include <stdio.h>
int fputs(const char * restrict s, FILE * restrict stream);
```

Description

The **fputs** function writes the string pointed to by **s** to the stream pointed to by **stream**. The terminating null character is not written.

Returns

The **fputs** function returns **EOF** if a write error occurs; otherwise it returns a nonnegative value.

7.23.7.5 The **getc** function

Synopsis

```
#include <stdio.h>
int getc(FILE *stream);
```

Description

The **getc** function is equivalent to **fgetc**, except that if it is implemented as a macro, it may evaluate **stream** more than once, so the argument should never be an expression with side effects.

Returns

The **getc** function returns the next character from the input stream pointed to by **stream**. If the stream is at end-of-file, the end-of-file indicator for the stream is set and **getc** returns **EOF**. If a read error occurs, the error indicator for the stream is set and **getc** returns **EOF**.

7.23.7.6 The `getchar` function

Synopsis

```
#include <stdio.h>
int getchar(void);
```

Description

The `getchar` function is equivalent to `getc` with the argument `stdin`.

Returns

The `getchar` function returns the next character from the input stream pointed to by `stdin`. If the stream is at end-of-file, the end-of-file indicator for the stream is set and `getchar` returns `EOF`. If a read error occurs, the error indicator for the stream is set and `getchar` returns `EOF`.

7.23.7.7 The `putc` function

Synopsis

```
#include <stdio.h>
int putc(int c, FILE *stream);
```

Description

The `putc` function is equivalent to `fputc`, except that if it is implemented as a macro, it may evaluate `stream` more than once, so that argument should never be an expression with side effects.

Returns

The `putc` function returns the character written. If a write error occurs, the error indicator for the stream is set and `putc` returns `EOF`.

7.23.7.8 The `putchar` function

Synopsis

```
#include <stdio.h>
int putchar(int c);
```

Description

The `putchar` function is equivalent to `putc` with the second argument `stdout`.

Returns

The `putchar` function returns the character written. If a write error occurs, the error indicator for the stream is set and `putchar` returns `EOF`.

7.23.7.9 The `puts` function

Synopsis

```
#include <stdio.h>
int puts(const char *s);
```

Description

The `puts` function writes the string pointed to by `s` to the stream pointed to by `stdout`, and appends a new-line character to the output. The terminating null character is not written.

Returns

The `puts` function returns `EOF` if a write error occurs; otherwise it returns a nonnegative value.

7.23.7.10 The `ungetc` function

Synopsis

```
#include <stdio.h>
int ungetc(int c, FILE *stream);
```

Description

The **ungetc** function pushes the character specified by **c** (converted to an **unsigned char**) back onto the input stream pointed to by **stream**. Pushed-back characters will be returned by subsequent reads on that stream in the reverse order of their pushing. A successful intervening call (with the stream pointed to by **stream**) to a file positioning function (**fseek**, **fsetpos**, or **rewind**) discards any pushed-back characters for the stream. The external storage corresponding to the stream is unchanged.

One character of pushback is guaranteed. If the **ungetc** function is called too many times on the same stream without an intervening read or file positioning operation on that stream, the operation can fail.

If the value of **c** equals that of the macro **EOF**, the operation fails and the input stream is unchanged.

A successful call to the **ungetc** function clears the end-of-file indicator for the stream. The value of the file position indicator for the stream after reading or discarding all pushed-back characters shall be the same as it was before the characters were pushed back.³³⁵⁾ For a text stream, the value of its file position indicator after a successful call to the **ungetc** function is unspecified until all pushed-back characters are read or discarded. For a binary stream, its file position indicator is decremented by each successful call to the **ungetc** function; if its value was zero before a call, it has an indeterminate representation after the call.³³⁶⁾

Returns

The **ungetc** function returns the character pushed back after conversion, or **EOF** if the operation fails.

Forward references: file positioning functions (7.23.9).

7.23.8 Direct input/output functions

7.23.8.1 The fread function

Synopsis

```
#include <stdio.h>
size_t fread(void * restrict ptr, size_t size, size_t nmemb,
FILE * restrict stream);
```

Description

The **fread** function reads, into the array pointed to by **ptr**, up to **nmemb** elements whose size is specified by **size**, from the stream pointed to by **stream**. For each object, **size** calls are made to the **fgetc** function and the results stored, in the order read, in an array of **unsigned char** exactly overlaying the object. The file position indicator for the stream (if defined) is advanced by the number of characters successfully read. If an error occurs, the resulting representation of the file position indicator for the stream is indeterminate. If a partial element is read, its representation is indeterminate.

Returns

The **fread** function returns the number of elements successfully read, which can be less than **nmemb** if a read error or end-of-file is encountered. If **size** or **nmemb** is zero, **fread** returns zero and the contents of the array and the state of the stream remain unchanged.

7.23.8.2 The fwrite function

Synopsis

³³⁵⁾A file positioning function can further modify the file position indicator after discarding any pushed-back characters.
³³⁶⁾See “future library directions” (7.33.16).


```
#include <stdio.h>
size_t fwrite(const void * restrict ptr, size_t size, size_t nmemb,
FILE * restrict stream);
```

Description

The **fwrite** function writes, from the array pointed to by **ptr**, up to **nmemb** elements whose size is specified by **size**, to the stream pointed to by **stream**. For each object, **size** calls are made to the **fputc** function, taking the values (in order) from an array of **unsigned char** exactly overlaying the object. The file position indicator for the stream (if defined) is advanced by the number of characters successfully written. If an error occurs, the resulting representation of the file position indicator for the stream is indeterminate.

Returns

The **fwrite** function returns the number of elements successfully written, which will be less than **nmemb** only if a write error is encountered. If **size** or **nmemb** is zero, **fwrite** returns zero and the state of the stream remains unchanged.

7.23.9 File positioning functions

7.23.9.1 The **fgetpos** function

Synopsis

```
#include <stdio.h>
int fgetpos(FILE * restrict stream, fpos_t * restrict pos);
```

Description

The **fgetpos** function stores the current values of the parse state (if any) and file position indicator for the stream pointed to by **stream** in the object pointed to by **pos**. The values stored contain unspecified information usable by the **fsetpos** function for repositioning the stream to its position at the time of the call to the **fgetpos** function.

Returns

If successful, the **fgetpos** function returns zero; on failure, the **fgetpos** function returns nonzero and stores an implementation-defined positive value in **errno**.

Forward references: the **fsetpos** function (7.23.9.3).

7.23.9.2 The **fseek** function

Synopsis

```
#include <stdio.h>
int fseek(FILE *stream, long int offset, int whence);
```

Description

The **fseek** function sets the file position indicator for the stream pointed to by **stream**. If a read or write error occurs, the error indicator for the stream is set and **fseek** fails.

For a binary stream, the new position, measured in characters from the beginning of the file, is obtained by adding **offset** to the position specified by **whence**. The specified position is the beginning of the file if **whence** is **SEEK_SET**, the current value of the file position indicator if **SEEK_CUR**, or end-of-file if **SEEK_END**. A binary stream can fail to meaningfully support **fseek** calls with a **whence** value of **SEEK_END**.

For a text stream, either **offset** shall be zero, or **offset** shall be a value returned by an earlier successful call to the **ftell** function on a stream associated with the same file and **whence** shall be **SEEK_SET**.

After determining the new position, a successful call to the **fseek** function undoes any effects of the **ungetc** function on the stream, clears the end-of-file indicator for the stream, and then establishes

the new position. After a successful **fseek** call, the next operation on an update stream may be either input or output.

Returns

The **fseek** function returns nonzero only for a request that cannot be satisfied.

Forward references: the **ftell** function (7.23.9.4).

7.23.9.3 The **fsetpos** function

Synopsis

```
#include <stdio.h>
int fsetpos(FILE *stream, const fpos_t *pos);
```

Description

The **fsetpos** function sets the **mbstate_t** object (if any) and file position indicator for the stream pointed to by **stream** according to the value of the object pointed to by **pos**, which shall be a value obtained from an earlier successful call to the **fgetpos** function on a stream associated with the same file. If a read or write error occurs, the error indicator for the stream is set and **fsetpos** fails.

A successful call to the **fsetpos** function undoes any effects of the **ungetc** function on the stream, clears the end-of-file indicator for the stream, and then establishes the new parse state and position. After a successful **fsetpos** call, the next operation on an update stream may be either input or output.

Returns

If successful, the **fsetpos** function returns zero; on failure, the **fsetpos** function returns nonzero and stores an implementation-defined positive value in **errno**.

7.23.9.4 The **ftell** function

Synopsis

```
#include <stdio.h>
long int ftell(FILE *stream);
```

Description

The **ftell** function obtains the current value of the file position indicator for the stream pointed to by **stream**. For a binary stream, the value is the number of characters from the beginning of the file. For a text stream, its file position indicator contains unspecified information, usable by the **fseek** function for returning the file position indicator for the stream to its position at the time of the **ftell** call; the difference between two such return values is not necessarily a meaningful measure of the number of characters written or read.

Returns

If successful, the **ftell** function returns the current value of the file position indicator for the stream. On failure, the **ftell** function returns **-1L** and stores an implementation-defined positive value in **errno**.

7.23.9.5 The **rewind** function

Synopsis

```
#include <stdio.h>
void rewind(FILE *stream);
```

Description

The **rewind** function sets the file position indicator for the stream pointed to by **stream** to the beginning of the file. It is equivalent to

```
(void) fseek(stream, 0L, SEEK_SET)
```

except that the error indicator for the stream is also cleared.

Returns

The **rewind** function returns no value.

7.23.10 Error-handling functions

7.23.10.1 The **clearerr** function

Synopsis

```
#include <stdio.h>
void clearerr(FILE *stream);
```

Description

The **clearerr** function clears the end-of-file and error indicators for the stream pointed to by **stream**.

Returns

The **clearerr** function returns no value.

7.23.10.2 The **feof** function

Synopsis

```
#include <stdio.h>
int feof(FILE *stream);
```

Description

The **feof** function tests the end-of-file indicator for the stream pointed to by **stream**.

Returns

The **feof** function returns nonzero if and only if the end-of-file indicator is set for **stream**.

7.23.10.3 The **ferror** function

Synopsis

```
#include <stdio.h>
int ferror(FILE *stream);
```

Description

The **ferror** function tests the error indicator for the stream pointed to by **stream**.

Returns

The **ferror** function returns nonzero if and only if the error indicator is set for **stream**.

7.23.10.4 The **perror** function

Synopsis

```
#include <stdio.h>
void perror(const char *s);
```

Description

The **perror** function maps the error number in the integer expression **errno** to an error message. It writes a sequence of characters to the standard error stream thus: first (if **s** is not a null pointer and the character pointed to by **s** is not the null character), the string pointed to by **s** followed by a colon (:) and a space; then an appropriate error message string followed by a new-line character.

The contents of the error message strings are the same as those returned by the **strerror** function with argument **errno**.

Returns

The **perror** function returns no value.

Forward references: the **strerror** function (7.26.6.3).

7.24 General utilities <stdlib.h>

7.24.1 General

The header <stdlib.h> declares several types and functions of general utility, and defines several macros.³³⁷⁾

The feature test macro `__STDC_VERSION_STDLIB_H` expands to the token 202311L.

The types declared are `size_t` and `wchar_t` (both described in 7.21), `once_flag` (described in 7.28),

div_t

which is a structure type that is the type of the value returned by the `div` function,

ldiv_t

which is a structure type that is the type of the value returned by the `ldiv` function, and

lldiv_t

which is a structure type that is the type of the value returned by the `lldiv` function.

The macros defined are `NULL` (described in 7.21); `ONCE_FLAG_INIT` (described in 7.28);

EXIT_FAILURE

and

EXIT_SUCCESS

which expand to integer constant expressions that can be used as the argument to the `exit` function to return unsuccessful or successful termination status, respectively, to the host environment;

RAND_MAX

which expands to an integer constant expression that is the maximum value returned by the `rand` function; and

MB_CUR_MAX

which expands to a positive integer expression with type `size_t` that is the maximum number of bytes in a multibyte character for the extended character set specified by the current locale (category `LC_CTYPE`), which is never greater than `MB_LEN_MAX`.

The function

#include <stdlib.h>
void call_once(once_flag *flag, void (*func)(void));

is described in 7.28.2.

7.24.2 Numeric conversion functions

7.24.2.1 General

The functions `atof`, `atoi`, `atol`, and `atoll` are not required to affect the value of the integer expression `errno` on an error. If the value of the result cannot be represented, the behavior is undefined.

7.24.2.2 The `atof` function

³³⁷⁾See “future library directions” (7.33.17).

Synopsis

```
#include <stdlib.h>
double atof(const char *nptr);
```

Description

The **atof** function converts the initial portion of the string pointed to by **nptr** to **double** representation. Except for the behavior on error, it is equivalent to

```
strtod(nptr, nullptr)
```

Returns

The **atof** function returns the converted value.

Forward references: the **strtod**, **strtof**, and **strtold** functions (7.24.2.6).

7.24.2.3 The **atoi**, **atol**, and **atoll** functions

Synopsis

```
#include <stdlib.h>
int atoi(const char *nptr);
long int atol(const char *nptr);
long long int atoll(const char *nptr);
```

Description

The **atoi**, **atol**, and **atoll** functions convert the initial portion of the string pointed to by **nptr** to **int**, **long int**, and **long long int** representation, respectively. Except for the behavior on error, they are equivalent to

```
atoi: (int)strtoul(nptr, nullptr, 10)
atol: strtoul(nptr, nullptr, 10)
atoll: strtoll(nptr, nullptr, 10)
```

Returns

The **atoi**, **atol**, and **atoll** functions return the converted value.

Forward references: the **strtoul**, **strtoll**, **strtoul**, and **strtoull** functions (7.24.2.8).

7.24.2.4 The **strfromd**, **strfromf**, and **strfroml** functions

Synopsis

```
#include <stdlib.h>
int strfromd(char * restrict s, size_t n, const char * restrict format,
double fp);
int strfromf(char * restrict s, size_t n, const char * restrict format,
float fp);
int strfroml(char * restrict s, size_t n, const char * restrict format,
long double fp);
```

Description

The **strfromd**, **strfromf**, and **strfroml** functions are equivalent to **snprintf(s, n, format, fp)** (7.23.6.6), except that the default argument promotions are not applied and the format string shall only contain the character %, an optional precision that does not contain an asterisk *, and one of the conversion specifiers a, A, e, E, f, F, g, or G, which applies to the type (**double**, **float**, or **long double**) indicated by the function suffix (rather than by a length modifier). Use of these functions with any other format string results in undefined behavior.

Returns

The **strfromd**, **strfromf**, and **strfroml** functions return the number of characters that would have been written had **n** been sufficiently large, not counting the terminating null character. Thus, the null-terminated output has been completely written if and only if the returned value is both nonnegative and less than **n**.

7.24.2.5 The **strfromdN** functions

Synopsis

```
#include <stdlib.h>
#ifdef __STDC_IEC_60559_DFP__
int strfromd32(char * restrict s, size_t n, const char * restrict format,
               _Decimal32 fp);
int strfromd64(char * restrict s, size_t n, const char * restrict format,
               _Decimal64 fp);
int strfromd128(char * restrict s, size_t n, const char * restrict format,
                _Decimal128 fp);
#endif
```

Description

The **strfromdN** functions are equivalent to **snprintf(s, n, format, fp)** (7.23.6.6), except the format string contains only the character %, an optional precision that does not contain an asterisk *, and one of the conversion specifiers a, A, e, E, f, F, g, or G, which applies to the type (**_Decimal32**, **_Decimal64**, or **_Decimal128**) indicated by the function suffix (rather than by a length modifier). Use of these functions with any other format string results in undefined behavior.

Returns

The **strfromdN** functions return the number of characters that would have been written had **n** been sufficiently large, not counting the terminating null character. Thus, the null-terminated output has been completely written if and only if the returned value is both nonnegative and less than **n**.

7.24.2.6 The **strtod**, **strtof**, and **strtold** functions

Synopsis

```
#include <stdlib.h>
double strtod(const char * restrict nptr, char ** restrict endptr);
float strtof(const char * restrict nptr, char ** restrict endptr);
long double strtold(const char * restrict nptr, char ** restrict endptr);
```

Description

The **strtod**, **strtof**, and **strtold** functions convert the initial portion of the string pointed to by **nptr** to **double**, **float**, and **long double** representation, respectively. First, they decompose the input string into three parts: an initial, possibly empty, sequence of white-space characters, a subject sequence resembling a floating constant or representing an infinity or NaN; and a final string of one or more unrecognized characters, including the terminating null character of the input string. Then, they attempt to convert the subject sequence to a floating-point number, and return the result.

The expected form of the subject sequence is an optional plus or minus sign, then one of the following:

- a nonempty sequence of decimal digits optionally containing a decimal-point character, then an optional exponent part as defined in 6.4.5.3, excluding any digit separators (6.4.5.2);
- a 0x or 0X, then a nonempty sequence of hexadecimal digits optionally containing a decimal-point character, then an optional binary exponent part as defined in 6.4.5.3, excluding any digit separators;
- INF or INFINITY, ignoring case

— NAN or NAN(*n-char-sequence_{opt}*), ignoring case in the NAN part, where:

n-char-sequence:

digit

nondigit

n-char-sequence digit

n-char-sequence nondigit

The subject sequence is defined as the longest initial subsequence of the input string, starting with the first non-white-space character, that is of the expected form. The subject sequence contains no characters if the input string is not of the expected form.

If the subject sequence has the expected form for a floating-point number, the sequence of characters starting with the first digit or the decimal-point character (whichever occurs first) is interpreted as a floating constant according to the rules of 6.4.5.3, except that the decimal-point character is used in place of a period, and that if neither an exponent part nor a decimal-point character appears in a decimal floating-point number, or if a binary exponent part does not appear in a hexadecimal floating-point number, an exponent part of the appropriate type with value zero is assumed to follow the last digit in the string. If the subject sequence begins with a minus sign, the sequence is interpreted as arithmetically negated.³³⁸⁾

A character sequence INF or INFINITY is interpreted as an infinity, if representable in the return type, else like a floating constant that is too large for the range of the return type. A character sequence NAN or NAN(*n-char-sequence_{opt}*) is interpreted as a quiet NaN, if supported in the return type, else like a subject sequence part that does not have the expected form; the meaning of the *n-char* sequence is implementation-defined.³³⁹⁾ A pointer to the final string is stored in the object pointed to by **endptr**, provided that **endptr** is not a null pointer.

If the subject sequence has the hexadecimal form and **FLT_RADIX** is a power of 2, the value resulting from the conversion is correctly rounded.

In other than the "C" locale, additional locale-specific subject sequence forms may be accepted.

If the subject sequence is empty or does not have the expected form, no conversion is performed; the value of **nptr** is stored in the object pointed to by **endptr**, provided that **endptr** is not a null pointer.

Recommended practice

If the subject sequence has the hexadecimal form, **FLT_RADIX** is not a power of 2, and the result is not exactly representable, the result should be one of the two numbers in the appropriate internal format that are adjacent to the hexadecimal floating source value, with the extra stipulation that the error should have a correct sign for the current rounding direction.

If the subject sequence has the decimal form and at most *M* significant digits, where *M* is the maximum value of the **T_DECIMAL_DIG** macros (defined in <float.h>), the result should be correctly rounded. If the subject sequence *D* has the decimal form and more than *M* significant digits, consider the two bounding, adjacent decimal strings *L* and *U*, both having *M* significant digits, such that the values of *L*, *D*, and *U* satisfy $L \leq D \leq U$. The result should be one of the (equal or adjacent) values that would be obtained by correctly rounding *L* and *U* according to the current rounding direction, with the extra stipulation that the error with respect to *D* should have a correct sign for the current rounding direction.³⁴⁰⁾

Returns

The functions return the converted value, if any. If no conversion could be performed, positive or unsigned zero is returned.

³³⁸⁾ It is unspecified whether a minus-signed sequence is converted to a negative number directly or by arithmetically negating the value resulting from converting the corresponding unsigned sequence (see F.5); the two methods can yield different results if rounding is toward positive or negative infinity. In either case, the functions honor the sign of zero if floating-point arithmetic supports signed zeros.

³³⁹⁾ An implementation can use the *n-char* sequence to determine extra information to be represented in the NaN's significand.

³⁴⁰⁾ *M* is sufficiently large that *L* and *U* will usually correctly round to the same internal floating value, but if not will correctly round to adjacent values.

If the correct value overflows and default rounding is in effect (7.12.2), plus or minus **HUGE_VAL**, **HUGE_VALF**, or **HUGE_VALL** is returned (according to the return type and sign of the value); if the integer expression **math_errhandling & MATH_ERRNO** is nonzero, the integer expression **errno** acquires the value of **ERANGE**; if the integer expression **math_errhandling & MATH_ERREXCEPT** is nonzero, the “overflow” floating-point exception is raised.

If the result underflows (7.12.2), the functions return a value whose magnitude is no greater than the smallest normalized positive number in the return type; if the integer expression **math_errhandling & MATH_ERRNO** is nonzero, whether **errno** acquires the value **ERANGE** is implementation-defined; if the integer expression **math_errhandling & MATH_ERREXCEPT** is nonzero, whether the “underflow” floating-point exception is raised is implementation-defined.

7.24.2.7 The strtodN functions

Synopsis

```
#include <stdlib.h>
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 strtod32(const char * restrict nptr, char ** restrict endptr);
_Decimal64 strtod64(const char * restrict nptr, char ** restrict endptr);
_Decimal128 strtod128(const char * restrict nptr, char ** restrict endptr);
#endif
```

Description

The **strtodN** functions convert the initial portion of the string pointed to by **nptr** to decimal floating type representation. First, they decompose the input string into three parts: an initial, possibly empty, sequence of white-space characters; a subject sequence resembling a floating constant or representing an infinity or NaN; and a final string of one or more unrecognized characters, including the terminating null character of the input string. Then, they attempt to convert the subject sequence to a floating-point number, and return the result.

The expected form of the subject sequence is an optional plus or minus sign, then one of the following:

- a nonempty sequence of decimal digits optionally containing a decimal-point character, then an optional exponent part as defined in 6.4.5.3, excluding any digit separators (6.4.5.2)
- a **0x** or **0X**, then a nonempty sequence of hexadecimal digits optionally containing a decimal-point character, then an optional binary exponent part as defined in 6.4.5.3, excluding any digit separators (6.4.5.2)
- **INF** or **INFINITY**, ignoring case
- **NAN** or **NAN(*d-char-sequence_{opt}*)**, ignoring case in the **NAN** part, where:

d-char-sequence:

```
digit
nondigit
d-char-sequence digit
d-char-sequence nondigit
```

The subject sequence is defined as the longest initial subsequence of the input string, starting with the first non-white-space character, that is of the expected form. The subject sequence contains no characters if the input string is not of the expected form.

If the subject sequence has the expected form for a floating-point number, the sequence of characters starting with the first digit or the decimal-point character (whichever occurs first) is interpreted as a floating constant according to the rules of 6.4.5.3, except that the decimal-point character is used

in place of a period, and that if neither an exponent part nor a decimal-point character appears in a decimal floating-point number, or if a binary exponent part does not appear in a hexadecimal floating-point number, an exponent part of the appropriate type with value zero is assumed to follow the last digit in the string. If the subject sequence begins with a minus sign, the sequence is interpreted as arithmetically negated before rounding and the sign *s* is set to -1 , else *s* is set to 1 .

If the subject sequence has the expected form for a decimal floating-point number, the value resulting from the conversion is correctly rounded and the coefficient *c* and the quantum exponent *q* are determined by the rules in 6.4.5.3 for a decimal floating constant of decimal type.

If the subject sequence has the expected form for a hexadecimal floating-point number, the value resulting from the conversion is correctly rounded provided the subject sequence has at most *M* significant hexadecimal digits, where $M \geq \lceil (P - 1)/4 \rceil + 1$ is implementation-defined, and *P* is the maximum precision of the supported radix-2 floating types and binary non-arithmetic interchange formats.³⁴¹⁾ If all subject sequences of hexadecimal form are correctly rounded, *M* may be regarded as infinite. If the subject sequence has more than *M* significant hexadecimal digits, the implementation may first round to *M* significant hexadecimal digits according to the applicable decimal rounding direction mode, signaling exceptions as though converting from a wider format, then correctly round the result of the shortened hexadecimal input to the result type. The preferred quantum exponent for the result is 0 if the hexadecimal number is exactly represented in the decimal type; the preferred quantum exponent for the result is the least possible if the hexadecimal number is not exactly represented in the decimal type.

A character sequence INF or INFINITY is interpreted as an infinity. A character sequence NAN or NAN(*d-char-sequence_{opt}*), is interpreted as a quiet NaN; the meaning of the d-char sequence is implementation-defined.³⁴²⁾ A pointer to the final string is stored in the object pointed to by **endptr**, provided that **endptr** is not a null pointer.

In other than the "C" locale, additional locale-specific subject sequence forms may be accepted.

If the subject sequence is empty or does not have the expected form, no conversion is performed; the value of **nptr** is stored in the object pointed to by **endptr**, provided that **endptr** is not a null pointer.

Returns

The **strtodN** functions return the converted value, if any. If no conversion could be performed, the value of the triple $(+1, 0, 0)$ is returned. If the correct value overflows:

- the value of the macro **ERANGE** is stored in **errno** if the integer expression **math_errhandling & MATH_ERRNO** is nonzero;
- the "overflow" floating-point exception is raised if the integer expression **math_errhandling & MATH_ERREXCEPT** is nonzero.

If the result underflows (7.12.2), whether **errno** acquires the value **ERANGE** if the integer expression **math_errhandling & MATH_ERRNO** is nonzero is implementation-defined; if the integer expression **math_errhandling & MATH_ERREXCEPT** is nonzero, whether the "underflow" floating-point exception is raised is implementation-defined.

EXAMPLE Following are subject sequences of the decimal form and the resulting triples (*s*, *c*, *q*) produced by **strtod64**. For **_Decimal64**, the precision (maximum coefficient length) is 16 and the quantum exponent range is $-398 \leq q \leq 369$.

"0"	(+1, 0, 0)
"0.00"	(+1, 0, -2)
"123"	(+1, 123, 0)
"-123"	(-1, 123, 0)
"1.23E3"	(+1, 123, 1)
"1.23E+3"	(+1, 123, 1)

³⁴¹⁾Non-arithmetic interchange formats are an optional feature in Annex H.
³⁴²⁾An implementation can use the d-char sequence to determine extra information to be represented in the NaN's significand.

"12.3E+7"	(+1, 123, 6)
"12.0"	(+1, 120, -1)
"12.3"	(+1, 123, -1)
"0.00123"	(+1, 123, -5)
"-1.23E-12"	(-1, 123, -14)
"1234.5E-4"	(+1, 12345, -5)
"-0"	(-1, 0, 0)
"-0.00"	(-1, 0, -2)
"0E+7"	(+1, 0, 7)
"-0E-7"	(-1, 0, -7)
"12345678901234567890"	(+1, 1234567890123457, 4) or (+1, 1234567890123456, 4) depending on rounding mode
"1234E-400"	(+1, 12, -398) or (+1, 13, -398) depending on rounding mode
"1234E-402"	(+1, 0, -398) or (+1, 1, -398) depending on rounding mode
"1000."	(+1, 1000, 0)
".0001"	(+1, 1, -4)
"1000.e0"	(+1, 1000, 0)
".0001e0"	(+1, 1, -4)
"1000.0"	(+1, 10000, -1)
"0.0001"	(+1, 1, -4)
"1000.00"	(+1, 100000, -2)
"00.0001"	(+1, 1, -4)
"001000."	(+1, 1000, 0)
"001000.0"	(+1, 10000, -1)
"001000.00"	(+1, 100000, -2)
"00.00"	(+1, 0, -2)
"00."	(+1, 0, 0)
".00"	(+1, 0, -2)
"00.00e-5"	(+1, 0, -7)
"00.e-5"	(+1, 0, -5)
".00e-5"	(+1, 0, -7)
"0x1.8p+4"	(+1, 24, 0)
"infinite"	infinity, and a pointer to "infinite" is stored in the object pointed to by endptr , provided endptr is not a null pointer

7.24.2.8 The **strtol**, **strtoll**, **strtoul**, and **strtoull** functions

Synopsis

```
#include <stdlib.h>
long int strtol(const char * restrict nptr, char ** restrict endptr, int base);
long long int strtoll(const char * restrict nptr, char ** restrict endptr,
    int base);
unsigned long int strtoul(const char * restrict nptr, char ** restrict endptr,
    int base);
unsigned long long int strtoull(const char * restrict nptr,
    char ** restrict endptr, int base);
```

Description

The **strtol**, **strtoll**, **strtoul**, and **strtoull** functions convert the initial portion of the string pointed to by **nptr** to **long int**, **long long int**, **unsigned long int**, and **unsigned long long int** representation, respectively. First, they decompose the input string into three parts: an initial, possibly empty, sequence of white-space characters, a subject sequence resembling an integer represented in some radix determined by the value of **base**, and a final string of one or more unrecognized characters, including the terminating null character of the input string. Then, they attempt to convert the subject sequence to an integer, and return the result.

If the value of **base** is zero, the expected form of the subject sequence is that of an integer constant as described in 6.4.5.2, optionally preceded by a plus or minus sign, but not including an integer suffix or any optional digit separators. If the value of **base** is between 2 and 36 (inclusive), the expected form of the subject sequence is a sequence of letters and digits representing an integer with the radix

specified by **base**, optionally preceded by a plus or minus sign, but not including an integer suffix or any optional digit separators. The letters from a (or A) through z (or Z) are ascribed the values 10 through 35; only letters and digits whose ascribed values are less than that of **base** are permitted. If the value of **base** is 2, the characters **0b** or **0B** can optionally precede the sequence of letters and digits, following the sign if present. If the value of **base** is 16, the characters **0x** or **0X** can optionally precede the sequence of letters and digits, following the sign if present.

The subject sequence is defined as the longest initial subsequence of the input string, starting with the first non-white-space character, that is of the expected form. The subject sequence contains no characters if the input string is empty or consists entirely of white-space characters, or if the first non-white-space character is other than a sign or a permissible letter or digit.

If the subject sequence has the expected form and the value of **base** is zero, the sequence of characters starting with the first digit is interpreted as an integer constant according to the rules of 6.4.5.2. If the subject sequence has the expected form and the value of **base** is between 2 and 36, it is used as the base for conversion, ascribing to each letter its value as previously given. If the subject sequence begins with a minus sign, the resulting value is the negative of the converted value; for functions whose return type is an unsigned integer type this action is performed in the return type. A pointer to the final string is stored in the object pointed to by **endptr**, provided that **endptr** is not a null pointer.

In other than the "C" locale, additional locale-specific subject sequence forms may be accepted.

If the subject sequence is empty or does not have the expected form, no conversion is performed; the value of **nptr** is stored in the object pointed to by **endptr**, provided that **endptr** is not a null pointer.

Returns

The **strtoul**, **strtoll**, **strtoul**, and **strtoull** functions return the converted value, if any. If no conversion could be performed, zero is returned. If the correct value is outside the range of representable values, **LONG_MIN**, **LONG_MAX**, **LLONG_MIN**, **LLONG_MAX**, **ULONG_MAX**, or **ULLONG_MAX** is returned (according to the return type and sign of the value, if any), and the value of the macro **ERANGE** is stored in **errno**.

7.24.3 Pseudo-random sequence generation functions

7.24.3.1 The rand function

Synopsis

```
#include <stdlib.h>
int rand(void);
```

Description

The **rand** function computes a sequence of pseudo-random integers in the range 0 to **RAND_MAX** inclusive.

The **rand** function is not required to avoid data races with other calls to pseudo-random sequence generation functions. The implementation shall behave as if no library function calls the **rand** function.

NOTE There are no guarantees as to the quality of the random sequence produced and some implementations are known to produce sequences with distressingly non-random low-order bits. Applications with particular requirements should use a generator that is known to be sufficient for their needs.

Returns

The **rand** function returns a pseudo-random integer.

Environmental limits

The value of the **RAND_MAX** macro shall be at least 32767.

7.24.3.2 The srand function

Synopsis

```
#include <stdlib.h>
void srand(unsigned int seed);
```

Description

The **srand** function uses the argument as a seed for a new sequence of pseudo-random numbers to be returned by subsequent calls to **rand**. If **srand** is then called with the same seed value, the sequence of pseudo-random numbers shall be repeated. If **rand** is called before any calls to **srand** have been made, the same sequence shall be generated as when **srand** is first called with a seed value of 1.

The **srand** function is not required to avoid data races with other calls to pseudo-random sequence generation functions. The implementation shall behave as if no library function calls the **srand** function.

Returns

The **srand** function returns no value.

EXAMPLE The following functions define a portable implementation of **rand** and **srand**.

```
static unsigned long int next = 1;

int rand(void)    // RAND_MAX assumed to be 32767
{
    next = next * 1103515245 + 12345;
    return (unsigned int)(next/65536) % 32768;
}

void srand(unsigned int seed)
{
    next = seed;
}
```

7.24.4 Memory management functions

7.24.4.1 General

The order and contiguity of storage allocated by successive calls to the **aligned_alloc**, **calloc**, **malloc**, and **realloc** functions is unspecified. The pointer returned if the allocation succeeds is suitably aligned so that it can be assigned to a pointer to any type of object with a fundamental alignment requirement and size less than or equal to the size requested. It can then be used to access such an object or an array of such objects in the space allocated (until the space is explicitly deallocated). The lifetime of an allocated object extends from the allocation until the deallocation. Each such allocation shall yield a pointer to an object disjoint from any other object. The pointer returned points to the start (lowest byte address) of the allocated space. If the space cannot be allocated, a null pointer is returned. If the size of the space requested is zero, the behavior is implementation-defined: either a null pointer is returned to indicate an error, or the behavior is as if the size were some nonzero value, except that the returned pointer shall not be used to access an object.

For purposes of determining the existence of a data race, memory allocation functions behave as though they accessed only memory locations accessible through their arguments and not other static duration storage. These functions may, however, visibly modify the storage that they allocate or deallocate. Calls to these functions that allocate or deallocate a particular region of memory shall occur in a single total order, and each such deallocation call shall synchronize with the next allocation (if any) in this order.

7.24.4.2 The **aligned_alloc** function

Synopsis

```
#include <stdlib.h>
```



```
void *aligned_alloc(size_t alignment, size_t size);
```

Description

The **aligned_alloc** function allocates space for an object whose alignment is specified by **alignment**,³⁴³⁾ whose size is specified by **size**, and whose representation is indeterminate. If the value of **alignment** is not a valid alignment supported by the implementation the function shall fail by returning a null pointer.

Returns

The **aligned_alloc** function returns either a null pointer or a pointer to the allocated space.

7.24.4.3 The calloc function

Synopsis

```
#include <stdlib.h>
void *calloc(size_t nmemb, size_t size);
```

Description

The **calloc** function allocates space for an array of **nmemb** objects, each of whose size is **size**. The space is initialized to all bits zero.³⁴⁴⁾

Returns

The **calloc** function returns either a pointer to the allocated space or a null pointer if the space cannot be allocated or if the product **nmemb** * **size** would wraparound **size_t**.

7.24.4.4 The free function

Synopsis

```
#include <stdlib.h>
void free(void *ptr);
```

Description

The **free** function causes the space pointed to by **ptr** to be deallocated, that is, made available for further allocation. If **ptr** is a null pointer, no action occurs. Otherwise, if the argument does not match a pointer earlier returned by a memory management function, or if the space has been deallocated by a call to **free** or **realloc**, the behavior is undefined.

Returns

The **free** function returns no value.

7.24.4.5 The free_sized function

Synopsis

```
#include <stdlib.h>
void free_sized(void *ptr, size_t size);
```

Description

If **ptr** is a null pointer or the result obtained from a call to **malloc**, **realloc**, or **calloc**, where **size** size is equal to the requested allocation size, this function is equivalent to **free(ptr)**. Otherwise, the behavior is undefined. The result of an **aligned_alloc** call may not be passed to **free_sized**.

NOTE A conforming implementation can ignore **size** and call **free**.

³⁴³⁾The alignment requirements from 7.24.4 also apply even if the requested alignment is less strict.
³⁴⁴⁾This is not expected to be the same as the representation of floating-point zero or a null pointer constant.

Recommended practice

Implementations may provide extensions to query the usable size of an allocation, or to determine the usable size of the allocation that would result if a request for some other size were to succeed. Such implementations should allow passing the resulting usable size as the **size** parameter, and provide functionality equivalent to **free** in such cases.

Returns

The **free_sized** function returns no value.

7.24.4.6 The free_aligned_sized function**Synopsis**

```
#include <stdlib.h>
void free_aligned_sized(void *ptr, size_t alignment, size_t size);
```

Description

If **ptr** is a null pointer or the result obtained from a call to **aligned_alloc**, where **alignment** is equal to the requested allocation alignment and **size** is equal to the requested allocation size, this function is equivalent to **free(ptr)**. Otherwise, the behavior is undefined. The result of an **malloc**, **calloc**, or **realloc** call may not be passed to **free_aligned_sized**.

NOTE A conforming implementation can ignore **alignment** and **size** and call **free**.

Recommended practice

Implementations may provide extensions to query the usable size of an allocation, or to determine the usable size of the allocation that would result if a request for some other size were to succeed. Such implementations should allow passing the resulting usable size as the **size** parameter, and provide functionality equivalent to **free** in such cases.

Returns

The **free_aligned_sized** function returns no value.

7.24.4.7 The malloc function**Synopsis**

```
#include <stdlib.h>
void *malloc(size_t size);
```

Description

The **malloc** function allocates space for an object whose size is specified by **size** and whose representation is indeterminate.

Returns

The **malloc** function returns either a null pointer or a pointer to the allocated space.

7.24.4.8 The realloc function**Synopsis**

```
#include <stdlib.h>
void *realloc(void *ptr, size_t size);
```

Description

The **realloc** function deallocates the old object pointed to by **ptr** and returns a pointer to a new object that has the size specified by **size**. The contents of the new object shall be the same as that of the old object prior to deallocation, up to the lesser of the new and old sizes. Any bytes in the new object beyond the size of the old object have unspecified values.

If **ptr** is a null pointer, the **realloc** function behaves like the **malloc** function for the specified size.

Otherwise, if **ptr** does not match a pointer earlier returned by a memory management function, or if the space has been deallocated by a call to the **free** or **realloc** function, or if the **size** is zero, the behavior is undefined. If memory for the new object is not allocated, the old object is not deallocated and its value is unchanged.

Returns

The **realloc** function returns a pointer to the new object (which can have the same value as a pointer to the old object), or a null pointer if the new object has not been allocated.

7.24.5 Communication with the environment

7.24.5.1 The **abort** function

Synopsis

```
#include <stdlib.h>
[[noreturn]] void abort(void);
```

Description

The **abort** function causes abnormal program termination to occur, unless the signal **SIGABRT** is being caught and the signal handler does not return. Whether open streams with unwritten buffered data are flushed, open streams are closed, or temporary files are removed is implementation-defined. An implementation-defined form of the status *unsuccessful termination* is returned to the host environment by means of the function call **raise(SIGABRT)**.

Returns

The **abort** function does not return to its caller.

7.24.5.2 The **atexit** function

Synopsis

```
#include <stdlib.h>
int atexit(void (*func)(void));
```

Description

The **atexit** function registers the function pointed to by **func**, to be called without arguments at normal program termination.³⁴⁵⁾ It is unspecified whether a call to the **atexit** function that does not happen before the **exit** function is called will succeed.

Environmental limits

The implementation shall support the registration of at least 32 functions.

Returns

The **atexit** function returns zero if the registration succeeds, nonzero if it fails.

Forward references: the **at_quick_exit** function (7.24.5.3), the **exit** function (7.24.5.4).

7.24.5.3 The **at_quick_exit** function

Synopsis

```
#include <stdlib.h>
int at_quick_exit(void (*func)(void));
```

Description

The **at_quick_exit** function registers the function pointed to by **func**, to be called without arguments should **quick_exit** be called.³⁴⁶⁾ It is unspecified whether a call to the **at_quick_exit**

³⁴⁵⁾The **atexit** function registrations are distinct from the **at_quick_exit** registrations, so applications potentially need to call both registration functions with the same argument.

³⁴⁶⁾The **at_quick_exit** function registrations are distinct from the **atexit** registrations, so applications potentially need to call both registration functions with the same argument.

function that does not happen before the **quick_exit** function is called will succeed.

Environmental limits

The implementation shall support the registration of at least 32 functions.

Returns

The **at_quick_exit** function returns zero if the registration succeeds, nonzero if it fails.

Forward references: the **quick_exit** function (7.24.5.7).

7.24.5.4 The exit function

Synopsis

```
#include <stdlib.h>
[[noreturn]] void exit(int status);
```

Description

The **exit** function causes normal program termination to occur. No functions registered by the **at_quick_exit** function are called. If a program calls the **exit** function more than once, or calls the **quick_exit** function in addition to the **exit** function, the behavior is undefined.

First, all functions registered by the **atexit** function are called, in the reverse order of their registration,³⁴⁷⁾ except that a function is called after any previously registered functions that had already been called at the time it was registered. If, during the call to any such function, a call to the **longjmp** function is made that would terminate the call to the registered function, the behavior is undefined.

Next, all open streams with unwritten buffered data are flushed, all open streams are closed, and all files created by the **tmpfile** function are removed.

Finally, control is returned to the host environment. If the value of **status** is zero or **EXIT_SUCCESS**, an implementation-defined form of the status *successful termination* is returned. If the value of **status** is **EXIT_FAILURE**, an implementation-defined form of the status *unsuccessful termination* is returned. Otherwise the status returned is implementation-defined.

Returns

The **exit** function cannot return to its caller.

7.24.5.5 The _Exit function

Synopsis

```
#include <stdlib.h>
[[noreturn]] void _Exit(int status);
```

Description

The **_Exit** function causes normal program termination to occur and control to be returned to the host environment. No functions registered by the **atexit** function, the **at_quick_exit** function, or signal handlers registered by the **signal** function are called. The status returned to the host environment is determined in the same way as for the **exit** function (7.24.5.4). Whether open streams with unwritten buffered data are flushed, open streams are closed, or temporary files are removed is implementation-defined.

Returns

The **_Exit** function cannot return to its caller.

7.24.5.6 The getenv function

Synopsis

³⁴⁷⁾Each function is called as many times as it was registered, and in the correct order with respect to other registered functions.

```
#include <stdlib.h>
char *getenv(const char *name);
```

Description

The **getenv** function searches an *environment list*, provided by the host environment, for a string that matches the string pointed to by **name**. The set of environment names and the method for altering the environment list are implementation-defined. The **getenv** function is not required to avoid data races with other threads of execution that modify the environment list.³⁴⁸⁾

The implementation shall behave as if no library function calls the **getenv** function.

Returns

The **getenv** function returns a pointer to a string associated with the matched list member. The string pointed to shall not be modified by the program, but can be overwritten by a subsequent call to the **getenv** function. If the specified **name** cannot be found, a null pointer is returned.

7.24.5.7 The quick_exit function

Synopsis

```
#include <stdlib.h>
[[noreturn]] void quick_exit(int status);
```

Description

The **quick_exit** function causes normal program termination to occur. No functions registered by the **atexit** function or signal handlers registered by the **signal** function are called. If a program calls the **quick_exit** function more than once, or calls the **exit** function in addition to the **quick_exit** function, the behavior is undefined. If a signal is raised while the **quick_exit** function is executing, the behavior is undefined.

The **quick_exit** function first calls all functions registered by the **at_quick_exit** function, in the reverse order of their registration,³⁴⁹⁾ except that a function is called after any previously registered functions that had already been called at the time it was registered. If, during the call to any such function, a call to the **longjmp** function is made that would terminate the call to the registered function, the behavior is undefined.

Then control is returned to the host environment by means of the function call **_Exit(status)**.

Returns

The **quick_exit** function cannot return to its caller.

7.24.5.8 The system function

Synopsis

```
#include <stdlib.h>
int system(const char *string);
```

Description

If **string** is a null pointer, the **system** function determines whether the host environment has a *command processor*. If **string** is not a null pointer, the **system** function passes the string pointed to by **string** to that command processor to be executed in a manner which the implementation shall document; this can then cause the program calling **system** to behave in a non-conforming manner or to terminate.

³⁴⁸⁾Many implementations provide non-standard functions that modify the environment list.
³⁴⁹⁾Each function is called as many times as it was registered, and in the correct order with respect to other registered functions.

Returns

If the argument is a null pointer, the **system** function returns nonzero only if a command processor is available. If the argument is not a null pointer, and the **system** function does return, it returns an implementation-defined value.

7.24.6 Searching and sorting utilities**7.24.6.1 General**

These utilities make use of a comparison function to search or sort arrays of unspecified type. Where an argument declared as **size_t nmemb** specifies the length of the array for a function, **nmemb** can have the value zero on a call to that function; the comparison function is not called, a search finds no matching element, and sorting performs no rearrangement. Pointer arguments on such a call shall still have valid values, as described in 7.1.4.

The implementation shall ensure that the second argument of the comparison function (when called from **bsearch**), or both arguments (when called from **qsort**), are pointers to elements of the array.³⁵⁰ The first argument when called from **bsearch** shall equal **key**.

The comparison function shall not alter the contents of the array. The implementation may reorder elements of the array between calls to the comparison function, but shall not alter the contents of any individual element.

When the same objects (consisting of **size** bytes, irrespective of their current positions in the array) are passed more than once to the comparison function, the results shall be consistent with one another. That is, for **qsort** they shall define a total ordering on the array, and for **bsearch** the same object shall always compare the same way with the key.

A sequence point occurs immediately before and immediately after each call to the comparison function, and also between any call to the comparison function and any movement of the objects passed as arguments to that call.

7.24.6.2 The bsearch generic function**Synopsis**

```
#include <stdlib.h>
QVoid *bsearch(const void *key, QVoid *base, size_t nmemb, size_t size,
               int (*compar)(const void *, const void *));
```

Description

The **bsearch** generic function searches an array of **nmemb** objects, the initial element of which is pointed to by **base**, for an element that matches the object pointed to by **key**. The size of each element of the array is specified by **size**.

The comparison function pointed to by **compar** is called with two arguments that point to the **key** object and to an array element, in that order. The function shall return an integer less than, equal to, or greater than zero if the **key** object is considered, respectively, to be less than, to match, or to be greater than the array element. The array shall consist of: all the elements that compare less than, all the elements that compare equal to, and all the elements that compare greater than the **key** object, in that order.³⁵¹

Returns

The **bsearch** generic function returns a pointer to a matching element of the array, or a null pointer if no match is found. If two elements compare as equal, which element is matched is unspecified.

³⁵⁰) That is, if the value passed is **p**, then the following expressions are always nonzero:

```
((char *)p - (char *)base) % size == 0
(char *)p >= (char *)base
(char *)p < (char *)base + nmemb * size
```

³⁵¹) In practice, the entire array is sorted according to the comparison function.

The **bsearch** function is generic in the qualification of the type pointed to by the argument **base**. If this argument is a pointer to a **const**-qualified object type, the returned pointer will be a pointer to **const**-qualified **void**. Otherwise, the argument shall be a pointer to an unqualified object type or a null pointer constant,³⁵²⁾ and the returned pointer will be a pointer to unqualified **void**.

The external declaration of **bsearch** has the concrete type:

```
void * (const void *, const void *, size_t, size_t,
       int (*) (const void *, const void *))
```

which supports all correct uses. If a macro definition of this generic function is suppressed to access an actual function, the external declaration with this concrete type is visible.³⁵³⁾

7.24.6.3 The **qsort** function

Synopsis

```
#include <stdlib.h>
void qsort(void *base, size_t nmemb, size_t size,
           int (*compar)(const void *, const void *));
```

Description

The **qsort** function sorts an array of **nmemb** objects, the initial element of which is pointed to by **base**. The size of each object is specified by **size**.

The contents of the array are sorted into ascending order according to a comparison function pointed to by **compar**, which is called with two arguments that point to the objects being compared. The function shall return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

If two elements compare as equal, their order in the resulting sorted array is unspecified.

Returns

The **qsort** function returns no value.

7.24.7 Integer arithmetic functions

7.24.7.1 The **abs**, **labs**, and **llabs** functions

Synopsis

```
#include <stdlib.h>
int abs(int j);
long int labs(long int j);
long long int llabs(long long int j);
```

Description

The **abs**, **labs**, and **llabs** functions compute the absolute value of an integer **j**. If the result cannot be represented, the behavior is undefined.³⁵⁴⁾

Returns

The **abs**, **labs**, and **llabs**, functions return the absolute value.

7.24.7.2 The **div**, **ldiv**, and **lldiv** functions

Synopsis

```
#include <stdlib.h>
div_t div(int number, int denom);
ldiv_t ldiv(long int number, long int denom);
```

³⁵²⁾If the argument is a null pointer and the call is executed, the behavior is undefined.
³⁵³⁾This is an obsolescent feature.
³⁵⁴⁾The absolute value of the most negative number is not representable.

```
lldiv_t lldiv(long long int numer, long long int denom);
```

Description

The **div**, **ldiv**, and **lldiv** functions compute **numer/denom** and **numer%denom** in a single operation.

Returns

The **div**, **ldiv**, and **lldiv** functions return a structure of type **div_t**, **ldiv_t**, and **lldiv_t**, respectively, comprising both the quotient and the remainder. The structures shall contain (in either order) the members **quot** (the quotient) and **rem** (the remainder), each of which has the same type as the arguments **numer** and **denom**. If either part of the result cannot be represented, the behavior is undefined.

7.24.8 Multibyte/wide character conversion functions

7.24.8.1 General

The behavior of the multibyte character functions is affected by the **LC_CTYPE** category of the current locale. For a state-dependent encoding, each of the **mbtowc** and **wctomb** functions is placed into its initial conversion state prior to the first call to the function and can be returned to that state by a call for which its character pointer argument, **s**, is a null pointer. Subsequent calls with **s** as other than a null pointer cause the internal conversion state of the function to be altered as necessary. It is implementation-defined whether internal conversion state has thread storage duration, and whether a newly created thread has the same state as the current thread at the time of creation, or the initial conversion state. A call with **s** as a null pointer causes these functions to return a nonzero value if encodings have state dependency, and zero otherwise.³⁵⁵ It is implementation-defined whether these functions avoid data races with other calls to the same function.

Changing the **LC_CTYPE** category causes the internal object describing the conversion state of the **mbtowc** and **wctomb** functions to have an indeterminate representation.

7.24.8.2 The **mblen** function

Synopsis

```
#include <stdlib.h>
int mblen(const char *s, size_t n);
```

Description

If **s** is not a null pointer, the **mblen** function determines the number of bytes contained in the multibyte character pointed to by **s**. Except that the conversion state of the **mbtowc** function is not affected, it is equivalent to

```
mbtowc((wchar_t *)0, (const char *)0, 0);
mbtowc((wchar_t *)0, s, n);
```

Returns

If **s** is a null pointer, the **mblen** function returns a nonzero or zero value, if multibyte character encodings, respectively, do or do not have state-dependent encodings. If **s** is not a null pointer, the **mblen** function either returns 0 (if **s** points to the null character), or returns the number of bytes that are contained in the multibyte character (if the next **n** or fewer bytes form a valid multibyte character), or returns -1 (if they do not form a valid multibyte character).

Forward references: the **mbtowc** function (7.24.8.3).

7.24.8.3 The **mbtowc** function

Synopsis

```
#include <stdlib.h>
```

³⁵⁵ If the locale employs special bytes to change the shift state, these bytes do not produce separate wide character codes, but are grouped with an adjacent multibyte character.


```
int mbtowc(wchar_t * restrict pwc, const char * restrict s, size_t n);
```

Description

If **s** is not a null pointer, the **mbtowc** function inspects at most **n** bytes beginning with the byte pointed to by **s** to determine the number of bytes needed to complete the next multibyte character (including any shift sequences). If the function determines that the next multibyte character is complete and valid, it determines the value of the corresponding wide character and then, if **pwc** is not a null pointer, stores that value in the object pointed to by **pwc**. If the corresponding wide character is the null wide character, the function is left in the initial conversion state.

The implementation shall behave as if no library function calls the **mbtowc** function.

Returns

If **s** is a null pointer, the **mbtowc** function returns a nonzero or zero value, if multibyte character encodings, respectively, do or do not have state-dependent encodings. If **s** is not a null pointer, the **mbtowc** function either returns 0 (if **s** points to the null character), or returns the number of bytes that are contained in the converted multibyte character (if the next **n** or fewer bytes form a valid multibyte character), or returns -1 (if they do not form a valid multibyte character).

In no case will the value returned be greater than **n** or the value of the **MB_CUR_MAX** macro.

7.24.8.4 The **wctomb** function

Synopsis

```
#include <stdlib.h>
int wctomb(char *s, wchar_t wc);
```

Description

The **wctomb** function determines the number of bytes needed to represent the multibyte character corresponding to the wide character given by **wc** (including any shift sequences), and stores the multibyte character representation in the array whose first element is pointed to by **s** (if **s** is not a null pointer). At most **MB_CUR_MAX** characters are stored. If **wc** is a null wide character, a null byte is stored, preceded by any shift sequence needed to restore the initial shift state, and the function is left in the initial conversion state.

The implementation shall behave as if no library function calls the **wctomb** function.

Returns

If **s** is a null pointer, the **wctomb** function returns a nonzero or zero value, if multibyte character encodings, respectively, do or do not have state-dependent encodings. If **s** is not a null pointer, the **wctomb** function returns -1 if the value of **wc** does not correspond to a valid multibyte character, or returns the number of bytes that are contained in the multibyte character corresponding to the value of **wc**.

In no case will the value returned be greater than the value of the **MB_CUR_MAX** macro.

7.24.9 Multibyte/wide string conversion functions

7.24.9.1 General

The behavior of the multibyte string functions is affected by the **LC_CTYPE** category of the current locale.

7.24.9.2 The **mbstowcs** function

Synopsis

```
#include <stdlib.h>
size_t mbstowcs(wchar_t * restrict pwcs, const char * restrict s, size_t n);
```

Description

The **mbstowcs** function converts a sequence of multibyte characters that begins in the initial shift state from the array pointed to by **s** into a sequence of corresponding wide characters and stores not more than **n** wide characters into the array pointed to by **pwcs**. No multibyte characters that follow a null character (which is converted into a null wide character) will be examined or converted. Each multibyte character is converted as if by a call to the **mbtowc** function, except that the conversion state of the **mbtowc** function is not affected.

No more than **n** elements will be modified in the array pointed to by **pwcs**. If copying takes place between objects that overlap, the behavior is undefined.

Returns

If an invalid multibyte character is encountered, the **mbstowcs** function returns **(size_t)(-1)**. Otherwise, the **mbstowcs** function returns the number of array elements modified, not including a terminating null wide character, if any.³⁵⁶⁾

7.24.9.3 The wcstombs function**Synopsis**

```
#include <stdlib.h>
size_t wcstombs(char * restrict s, const wchar_t * restrict pwcs, size_t n);
```

Description

The **wcstombs** function converts a sequence of wide characters from the array pointed to by **pwcs** into a sequence of corresponding multibyte characters that begins in the initial shift state, and stores these multibyte characters into the array pointed to by **s**, stopping if a multibyte character would exceed the limit of **n** total bytes or if a null character is stored. Each wide character is converted as if by a call to the **wctomb** function, except that the conversion state of the **wctomb** function is not affected.

No more than **n** bytes will be modified in the array pointed to by **s**. If copying takes place between objects that overlap, the behavior is undefined.

Returns

If a wide character is encountered that does not correspond to a valid multibyte character, the **wcstombs** function returns **(size_t)(-1)**. Otherwise, the **wcstombs** function returns the number of bytes modified, not including a terminating null character, if any.³⁵⁶⁾

7.24.10 Alignment of memory**7.24.10.1 The memalignment function****Synopsis**

```
#include <stdlib.h>
size_t memalignment(const void *p);
```

Description

The **memalignment** function accepts a pointer to any object and returns the maximum alignment satisfied by its address value. The alignment may be an extended alignment and may also be beyond the range supported by the implementation for explicit use by **alignas**.³⁵⁷⁾ If so, it will satisfy all alignments usable by the implementation. The value returned can be compared to the result of **alignof**, and if it is greater or equal, the alignment requirement for the type operand is satisfied.

³⁵⁶⁾The array will not be null-terminated if the value returned is **n**.

³⁵⁷⁾The actual alignment of an object can be stricter than the alignment requested for an object by **alignas** or (implicitly) by an allocation function, but will always satisfy it.

Returns

The alignment of the pointer **p**, which is a power of two. If **p** is a null pointer, an alignment of zero is returned.

NOTE An alignment of zero indicates that the tested pointer cannot be used to access an object of any type.

7.25 `_Noreturn` <stdnoreturn.h>

The header <stdnoreturn.h> defines the macro

<code>noreturn</code>

which expands to `_Noreturn`.

The `noreturn` macro and the <stdnoreturn.h> header are obsolescent features.

7.26 String handling <string.h>

7.26.1 String function conventions

The header <string.h> declares one type, several functions, several type-generic functions, and defines two macros useful for manipulating arrays of character type and other objects treated as arrays of character type.³⁵⁸⁾ The type is **size_t** and one of the macros is **NULL** (both described in 7.21). Various methods are used for determining the lengths of the arrays, but in all cases a **char *** or **void *** argument points to the initial (lowest addressed) character of the array. If an array is accessed beyond the end of an object, the behavior is undefined.

The macro

```
__STDC_VERSION_STRING_H__
```

is an integer constant expression with a value equivalent to 202311L.

Where an argument declared as **size_t n** specifies the length of the array for a function, **n** can have the value zero on a call to that function. Unless explicitly stated otherwise in the description of a particular function in this subclause, pointer arguments on such a call shall still have valid values, as described in 7.1.4. On such a call, a function that locates a character finds no occurrence, a function that compares two character sequences returns zero, and a function that copies characters copies zero characters.

For all functions in this subclause, each character shall be interpreted as if it had the type **unsigned char** (and therefore every possible object representation is valid and has a different value).

7.26.2 Copying functions

7.26.2.1 The memcpy function

Synopsis

```
#include <string.h>
void *memcpy(void * restrict s1, const void * restrict s2, size_t n);
```

Description

The **memcpy** function copies **n** characters from the object pointed to by **s2** into the object pointed to by **s1**. If copying takes place between objects that overlap, the behavior is undefined.

Returns

The **memcpy** function returns the value of **s1**.

7.26.2.2 The memccpy function

Synopsis

```
#include <string.h>
void *memccpy(void * restrict s1, const void * restrict s2, int c, size_t n);
```

Description

The **memccpy** function copies characters from the object pointed to by **s2** into the object pointed to by **s1**, stopping after the first occurrence of character **c** (converted to an **unsigned char**) is copied, or after **n** characters are copied, whichever comes first. If copying takes place between objects that overlap, the behavior is undefined.

Returns

The **memccpy** function returns a pointer to the character after the copy of **c** in **s1**, or a null pointer if **c** was not found in the first **n** characters of **s2**.

7.26.2.3 The memmove function

³⁵⁸⁾See “future library directions” (7.33.18).

Synopsis

```
#include <string.h>
void *memmove(void *s1, const void *s2, size_t n);
```

Description

The **memmove** function copies **n** characters from the object pointed to by **s2** into the object pointed to by **s1**. Copying takes place as if the **n** characters from the object pointed to by **s2** are first copied into a temporary array of **n** characters that does not overlap the objects pointed to by **s1** and **s2**, and then the **n** characters from the temporary array are copied into the object pointed to by **s1**.

Returns

The **memmove** function returns the value of **s1**.

7.26.2.4 The strcpy function

Synopsis

```
#include <string.h>
char *strcpy(char * restrict s1, const char * restrict s2);
```

Description

The **strcpy** function copies the string pointed to by **s2** (including the terminating null character) into the array pointed to by **s1**. If copying takes place between objects that overlap, the behavior is undefined.

Returns

The **strcpy** function returns the value of **s1**.

7.26.2.5 The strncpy function

Synopsis

```
#include <string.h>
char *strncpy(char * restrict s1, const char * restrict s2, size_t n);
```

Description

The **strncpy** function copies not more than **n** characters (characters that follow a null character are not copied) from the array pointed to by **s2** to the array pointed to by **s1**.³⁵⁹⁾ If copying takes place between objects that overlap, the behavior is undefined.

If the array pointed to by **s2** is a string that is shorter than **n** characters, null characters are appended to the copy in the array pointed to by **s1**, until **n** characters in all have been written.

Returns

The **strncpy** function returns the value of **s1**.

7.26.2.6 The strdup function

Synopsis

```
#include <string.h>
char *strdup(const char *s);
```

Description

The **strdup** function creates a copy of the string pointed to by **s** in a space allocated as if by a call to **malloc**.

³⁵⁹⁾Thus, if there is no null character in the first **n** characters of the array pointed to by **s2**, the result will not be null-terminated.

Returns

The **strdup** function returns a pointer to the first character of the duplicate string. The returned pointer can be passed to **free**. If no space can be allocated the **strdup** function returns a null pointer.

7.26.2.7 The **strndup** function

Synopsis

```
#include <string.h>
char *strndup(const char *s, size_t n);
```

Description

The **strndup** function creates a string initialized with no more than **n** initial characters of the array pointed to by **s** and up to the first null character, whichever comes first, in a space allocated as if by a call to **malloc**. If the array pointed to by **s** does not contain a null within the first **n** characters, a null is appended to the copy of the array.

Returns

The **strndup** function returns a pointer to the first character of the created string. The returned pointer can be passed to **free**. If space cannot be allocated the **strndup** function returns a null pointer.

7.26.3 Concatenation functions

7.26.3.1 The **strcat** function

Synopsis

```
#include <string.h>
char *strcat(char * restrict s1, const char * restrict s2);
```

Description

The **strcat** function appends a copy of the string pointed to by **s2** (including the terminating null character) to the end of the string pointed to by **s1**. The initial character of **s2** overwrites the null character at the end of **s1**. If copying takes place between objects that overlap, the behavior is undefined.

Returns

The **strcat** function returns the value of **s1**.

7.26.3.2 The **strncat** function

Synopsis

```
#include <string.h>
char *strncat(char * restrict s1, const char * restrict s2, size_t n);
```

Description

The **strncat** function appends not more than **n** characters (a null character and characters that follow it are not appended) from the array pointed to by **s2** to the end of the string pointed to by **s1**. The initial character of **s2** overwrites the null character at the end of **s1**. A terminating null character is always appended to the result.³⁶⁰⁾ If copying takes place between objects that overlap, the behavior is undefined.

Returns

The **strncat** function returns the value of **s1**.

Forward references: the **strlen** function (7.26.6.4).

³⁶⁰⁾Thus, the maximum number of characters that can end up in the array pointed to by **s1** is **strlen(s1)+n+1**.

7.26.4 Comparison functions

7.26.4.1 General

The sign of a nonzero value returned by the comparison functions `memcmp`, `strcmp`, and `strncmp` is determined by the sign of the difference between the values of the first pair of characters (both interpreted as `unsigned char`) that differ in the objects being compared.

7.26.4.2 The `memcmp` function

Synopsis

```
#include <string.h>
int memcmp(const void *s1, const void *s2, size_t n);
```

Description

The `memcmp` function compares the first `n` characters of the object pointed to by `s1` to the first `n` characters of the object pointed to by `s2`.³⁶¹⁾

Returns

The `memcmp` function returns an integer greater than, equal to, or less than zero, accordingly as the object pointed to by `s1` is greater than, equal to, or less than the object pointed to by `s2`.

7.26.4.3 The `strcmp` function

Synopsis

```
#include <string.h>
int strcmp(const char *s1, const char *s2);
```

Description

The `strcmp` function compares the string pointed to by `s1` to the string pointed to by `s2`.

Returns

The `strcmp` function returns an integer greater than, equal to, or less than zero, accordingly as the string pointed to by `s1` is greater than, equal to, or less than the string pointed to by `s2`.

7.26.4.4 The `strcoll` function

Synopsis

```
#include <string.h>
int strcoll(const char *s1, const char *s2);
```

Description

The `strcoll` function compares the string pointed to by `s1` to the string pointed to by `s2`, both interpreted as appropriate to the `LC_COLLATE` category of the current locale.

Returns

The `strcoll` function returns an integer greater than, equal to, or less than zero, accordingly as the string pointed to by `s1` is greater than, equal to, or less than the string pointed to by `s2` when both are interpreted as appropriate to the current locale.

7.26.4.5 The `strncmp` function

Synopsis

```
#include <string.h>
int strncmp(const char *s1, const char *s2, size_t n);
```

³⁶¹⁾The unused bytes used as padding for purposes of alignment within structure objects take on unspecified values when a value is stored in the object (see 6.2.6.1). Strings shorter than their allocated space and unions can also cause problems in comparison.

Description

The `strncmp` function compares not more than `n` characters (characters that follow a null character are not compared) from the array pointed to by `s1` to the array pointed to by `s2`.

Returns

The `strncmp` function returns an integer greater than, equal to, or less than zero, accordingly as the possibly null-terminated array pointed to by `s1` is greater than, equal to, or less than the possibly null-terminated array pointed to by `s2`.

7.26.4.6 The `strxfrm` function

Synopsis

```
#include <string.h>
size_t strxfrm(char * restrict s1, const char * restrict s2, size_t n);
```

Description

The `strxfrm` function transforms the string pointed to by `s2` and places the resulting string into the array pointed to by `s1`. The transformation is such that if the `strcmp` function is applied to two transformed strings, it returns a value greater than, equal to, or less than zero, corresponding to the result of the `strcmp` function applied to the same two original strings. No more than `n` characters are placed into the resulting array pointed to by `s1`, including the terminating null character. If `n` is zero, `s1` is permitted to be a null pointer. If copying takes place between objects that overlap, the behavior is undefined.

Returns

The `strxfrm` function returns the length of the transformed string (not including the terminating null character). If the value returned is `n` or more, the members of the array pointed to by `s1` have an indeterminate representation.

EXAMPLE The value of the following expression is the size of the array needed to hold the transformation of the string pointed to by `s`.

```
1 + strxfrm(nullptr, s, 0)
```

7.26.5 Search functions

7.26.5.1 Introduction

The stateless search functions in this section (`memchr`, `strchr`, `strpbrk`, `strrchr`, `strstr`) are *generic functions*. These functions are generic in the qualification of the array to be searched and will return a result pointer to an element with the same qualification as the passed array. If the array to be searched is `const`-qualified, the result pointer will be to a `const`-qualified element. If the array to be searched is not `const`-qualified,³⁶²⁾ the result pointer will be to an unqualified element.

The external declarations of these generic functions have a concrete function type that returns a pointer to an unqualified element (of type `char` when specified as *QChar*, and `void` when specified as *QVoid*), and accepts a pointer to a `const`-qualified array of the same type to search. This signature supports all correct uses. If a macro definition of any of these generic functions is suppressed to access an actual function, the external declaration with the corresponding concrete type is visible.³⁶³⁾

The `volatile` and `restrict` qualifiers are not accepted on the elements of the array to search.

7.26.5.2 The `memchr` generic function

Synopsis

```
#include <string.h>
QVoid *memchr(QVoid *s, int c, size_t n);
```

³⁶²⁾The null pointer constant is not a pointer to a `const`-qualified type, and therefore the result expression has the type of a pointer to an unqualified element; however, evaluating such a call is undefined.

³⁶³⁾This is an obsolescent feature.

Description

The **memchr** generic function locates the first occurrence of **c** (converted to an **unsigned char**) in the initial **n** characters (each interpreted as **unsigned char**) of the object pointed to by **s**. The implementation shall behave as if it reads the characters sequentially and stops as soon as a matching character is found.

Returns

The **memchr** generic function returns a pointer to the located character, or a null pointer if the character does not occur in the object.

7.26.5.3 The **strchr** generic function

Synopsis

```
#include <string.h>
QChar *strchr(QChar *s, int c);
```

Description

The **strchr** generic function locates the first occurrence of **c** (converted to a **char**) in the string pointed to by **s**. The terminating null character is considered to be part of the string.

Returns

The **strchr** generic function returns a pointer to the located character, or a null pointer if the character does not occur in the string.

7.26.5.4 The **strcspn** function

Synopsis

```
#include <string.h>
size_t strcspn(const char *s1, const char *s2);
```

Description

The **strcspn** function computes the length of the maximum initial segment of the string pointed to by **s1** which consists entirely of characters *not* from the string pointed to by **s2**.

Returns

The **strcspn** function returns the length of the segment.

7.26.5.5 The **strpbrk** generic function

Synopsis

```
#include <string.h>
QChar *strpbrk(QChar *s1, const char *s2);
```

Description

The **strpbrk** generic function locates the first occurrence in the string pointed to by **s1** of any character from the string pointed to by **s2**.

Returns

The **strpbrk** generic function returns a pointer to the character, or a null pointer if no character from **s2** occurs in **s1**.

7.26.5.6 The **strrchr** generic function

Synopsis

```
#include <string.h>
QChar *strrchr(QChar *s, int c);
```

Description

The **strrchr** generic function locates the last occurrence of **c** (converted to a **char**) in the string pointed to by **s**. The terminating null character is considered to be part of the string.

Returns

The **strrchr** generic function returns a pointer to the character, or a null pointer if **c** does not occur in the string.

7.26.5.7 The **strspn** function

Synopsis

```
#include <string.h>
size_t strspn(const char *s1, const char *s2);
```

Description

The **strspn** function computes the length of the maximum initial segment of the string pointed to by **s1** which consists entirely of characters from the string pointed to by **s2**.

Returns

The **strspn** function returns the length of the segment.

7.26.5.8 The **strstr** generic function

Synopsis

```
#include <string.h>
QChar *strstr(QChar *s1, const char *s2);
```

Description

The **strstr** generic function locates the first occurrence in the string pointed to by **s1** of the sequence of characters (excluding the terminating null character) in the string pointed to by **s2**.

Returns

The **strstr** generic function returns a pointer to the located string, or a null pointer if the string is not found. If **s2** points to a string with zero length, the function returns **s1**.

7.26.5.9 The **strtok** function

Synopsis

```
#include <string.h>
char *strtok(char * restrict s1, const char * restrict s2);
```

Description

A sequence of calls to the **strtok** function breaks the string pointed to by **s1** into a sequence of tokens, each of which is delimited by a character from the string pointed to by **s2**. The first call in the sequence has a non-null first argument; subsequent calls in the sequence have a null first argument. If any of the subsequent calls in the sequence is made by a different thread than the first, the behavior is undefined. The separator string pointed to by **s2** can be different from call to call.

The first call in the sequence searches the string pointed to by **s1** for the first character that is *not* contained in the current separator string pointed to by **s2**. If no such character is found, then there are no tokens in the string pointed to by **s1** and the **strtok** function returns a null pointer. If such a character is found, it is the start of the first token.

The **strtok** function then searches from there for a character that *is* contained in the current separator string. If no such character is found, the current token extends to the end of the string pointed to by **s1**, and subsequent searches for a token will return a null pointer. If such a character is found, it is overwritten by a null character, which terminates the current token. The **strtok** function saves a pointer to the following character, from which the next search for a token will start.

Each subsequent call, with a null pointer as the value of the first argument, starts searching from the saved pointer and behaves as described previously.

The **strtok** function is not required to avoid data races with other calls to the **strtok** function.³⁶⁴⁾ The implementation shall behave as if no library function calls the **strtok** function.

Returns

The **strtok** function returns a pointer to the first character of a token, or a null pointer if there is no token.

EXAMPLE

```
#include <string.h>
static char str[] = "?a???b,,,#c";
char *t;

t = strtok(str, "?");      // t points to the token "a"
t = strtok(nullptr, ",");  // t points to the token "???b"
t = strtok(nullptr, "#,"); // t points to the token "c"
t = strtok(nullptr, "?");  // t is a null pointer
```

Forward references: The **strtok_s** function (K.3.7.4.1).

7.26.6 Miscellaneous functions

7.26.6.1 The **memset** function

Synopsis

```
#include <string.h>
void *memset(void *s, int c, size_t n);
```

Description

The **memset** function copies the value of **c** (converted to an **unsigned char**) into each of the first **n** characters of the object pointed to by **s**.

Returns

The **memset** function returns the value of **s**.

7.26.6.2 The **memset_explicit** function

Synopsis

```
#include <string.h>
void *memset_explicit(void *s, int c, size_t n);
```

Description

The **memset_explicit** function copies the value of **c** (converted to an **unsigned char**) into each of the first **n** characters of the object pointed to by **s**. The purpose of this function is to make sensitive information stored in the object inaccessible.³⁶⁵⁾

Returns

The **memset_explicit** function returns the value of **s**.

7.26.6.3 The **strerror** function

Synopsis

```
#include <string.h>
char *strerror(int errnum);
```

³⁶⁴⁾The **strtok_s** function can be used instead to avoid data races.

³⁶⁵⁾The intention is that the memory store is always performed (i.e. never elided), regardless of optimizations. This is in contrast to calls to the **memset** function (7.26.6.1)

Description

The **strerror** function maps the number in **errnum** to a message string. Typically, the values for **errnum** come from **errno**, but **strerror** shall map any value of type **int** to a message.

The **strerror** function is not required to avoid data races with other calls to the **strerror** function.³⁶⁶⁾ The implementation shall behave as if no library function calls the **strerror** function.

Returns

The **strerror** function returns a pointer to the string, the contents of which are locale-specific. The array pointed to shall not be modified by the program. The behavior is undefined if the returned value is used after a subsequent call to the **strerror** function, or after the thread which called the function to obtain the returned value has exited.

Forward references: The **strerror_s** function (K.3.7.5.2).

7.26.6.4 The **strlen** function

Synopsis

```
#include <string.h>
size_t strlen(const char *s);
```

Description

The **strlen** function computes the length of the string pointed to by **s**.

Returns

The **strlen** function returns the number of characters that precede the terminating null character.

³⁶⁶⁾The **strerror_s** function can be used instead to avoid data races.

7.27 Type-generic math <tmath.h>

The header <tmath.h> includes the headers <math.h> and <complex.h> and defines several type-generic macros.

The feature test macro `__STDC_VERSION_TGMATH_H__` expands to the token 202311L.

This clause specifies a many-to-one correspondence of functions in <math.h> and <complex.h> with *type-generic macros*.³⁶⁷⁾ Use of a type-generic macro invokes a corresponding function whose type is determined by the types of the arguments for particular parameters called the *generic parameters*.³⁶⁸⁾

Of the <math.h> and <complex.h> functions without an **f** (**float**) or **l** (**long double**) suffix, several have one or more parameters whose corresponding real type is **double**. For each such function, except the functions that round result to narrower type (7.12.15) (which are covered subsequently in this subclause) and **modf**, there is a corresponding type-generic macro. The parameters whose corresponding real type is **double** in the function synopsis are generic parameters.

Some of the <math.h> functions for decimal floating types have no unsuffixed counterpart. Of these functions with a **d64** suffix, some have one or more parameters whose type is **_Decimal64**. For each such function, except **decodedecd64**, **encodedecd64**, **decodebind64**, and **encodebind64**, there is a corresponding type-generic macro. The parameters whose real type is **_Decimal64** in the function synopsis are generic parameters.

If arguments for generic parameters of a type-generic macro are such that some argument has a corresponding real type that is of standard floating type and another argument is of decimal floating type, the behavior is undefined.

Except for the macros for functions that round result to a narrower type (7.12.15), use of a type-generic macro invokes a function whose generic parameters have the corresponding real type determined by the types of the arguments for the generic parameters as follows:

- Arguments of integer type are regarded as having type **_Decimal64** if any argument has decimal floating type, and as having type **double** otherwise.
- If the function has exactly one generic parameter, the type determined is the corresponding real type of the argument for the generic parameter.
- If the function has exactly two generic parameters, the type determined is the corresponding real type determined by the usual arithmetic conversions (6.3.2.8) applied to the arguments for the generic parameters.
- If the function has more than two generic parameters, the type determined is the corresponding real type determined by repeatedly applying the usual arithmetic conversions, first to the first two arguments for generic parameters, then to that result type and the next argument for a generic parameter, and so forth until the usual arithmetic conversions have been applied to the last argument for a generic parameter.

If neither <math.h> and <complex.h> define a function whose generic parameters have the determined corresponding real type, the behavior is undefined.

For each unsuffixed function in <math.h> for which there is a function in <complex.h> with the same name except for a **c** prefix, the corresponding type-generic macro (for both functions) has the same name as the function in <math.h>. The corresponding type-generic macro for **fabs** and **cabs** is **fabs**. Table 7.8 shows the associations between functions and type-generic macros.

³⁶⁷⁾ Like other function-like macros in standard libraries, each type-generic macro can be suppressed to make available the corresponding ordinary function.

³⁶⁸⁾ If the type of the argument is not compatible with the type of the parameter for the selected function, the behavior is undefined.

Table 7.8 — Functions and type-generic macro relations

<math.h> function	<complex.h> function	type-generic macro
acos	cacos	acos
asin	casin	asin
atan	catan	atan
acosh	cacosh	acosh
asinh	casinh	asinh
atanh	catanh	atanh
cos	ccos	cos
sin	csin	sin
tan	ctan	tan
cosh	ccosh	cosh
sinh	csinh	sinh
tanh	ctanh	tanh
exp	cexp	exp
log	clog	log
pow	cpow	pow
sqrt	csqrt	sqrt
fabs	cabs	fabs

If at least one argument for a generic parameter is complex, then use of the macro invokes a complex function; otherwise, use of the macro invokes a real function.

For each unsuffixed function in <math.h> without a **c**-prefixed counterpart in <complex.h> (except functions that round result to narrower type, **modf**, and **canonicalize**), the corresponding type-generic macro has the same name as the function. These type-generic macros are:

acospi	exp2	fmod	log2	rootn
asinpi	expm1	frexp	logb	roundeven
atan2pi	fdim	fromfp_x	logp1	round
atan2	floor	fromfp	lrint	rsqrt
atanpi	fmax	hypot	lround	scalbln
cbrt	fmaximum	ilogb	nearbyint	scalbn
ceil	fmaximum_mag	ldexp	nextafter	sinpi
compoundn	fmaximum_num	lgamma	nextdown	tanpi
copysign	fmaximum_mag_num	llogb	nexttoward	tgamma
cospi	fma	llrint	nextup	trunc
erfc	fmin	llround	pown	ufromfp_x
erf	fminimum	log10p1	powr	ufromfp
exp10m1	fminimum_mag	log10	remainder	
exp10	fminimum_num	log1p	remquo	
exp2m1	fminimum_mag_num	log2p1	rint	

If all arguments for generic parameters are real, then use of the macro invokes a real function (provided <math.h> defines a function of the determined type); otherwise, use of the macro is undefined.

For each unsuffixed function in <complex.h> that is not a **c**-prefixed counterpart to a function in <math.h>, the corresponding type-generic macro has the same name as the function. These type-generic macros are:

carg	cimag	conj	cproj	creal
-------------	--------------	-------------	--------------	--------------

Use of the macro with any argument of standard floating or complex type invokes a complex function. Use of the macro with an argument of decimal floating type is undefined.

The functions that round result to a narrower type have type-generic macros whose names are obtained by omitting any suffix from the function names. Thus, the macros with **f** or **d** prefix are:

fadd	fsub	fmul	fdiv	ffma	fsqrt
dadd	dsub	dmul	ddiv	dfma	dsqrt

and the macros with **d32** or **d64** prefix are:

d32add	d32sub	d32mul	d32div	d32fma	d32sqrt
d64add	d64sub	d64mul	d64div	d64fma	d64sqrt

All arguments shall be real. If the macro prefix is **f** or **d**, use of an argument of decimal floating type is undefined. If the macro prefix is **d32** or **d64**, use of an argument of standard floating type is undefined. The function invoked is determined as follows:

- If any argument has type **_Decimal128**, or if the macro prefix is **d64**, the function invoked has the name of the macro, with a **d128** suffix.
- Otherwise, if the macro prefix is **d32**, the function invoked has the name of the macro, with a **d64** suffix.
- Otherwise, if any argument has type **long double**, or if the macro prefix is **d**, the function invoked has the name of the macro, with an **l** suffix.
- Otherwise, the function invoked has the name of the macro (with no suffix).

For each **d64**-suffixed function in `<math.h>`, except **decodedecd64**, **encodedecd64**, **decodebind64**, and **encodebind64**, that does not have an unsuffixed counterpart, the corresponding type-generic macro has the name of the function, but without the suffix. These type-generic macros are displayed in Table 7.9:

Table 7.9 — Decimal functions and type-generic macro relations

<code><math.h></code> function	type-generic macro
quantizedN	quantize
samequantumdN	samequantum
quantumdN	quantum
llquantexpdN	llquantexp

Use of the macro with an argument of standard floating or complex type or with only integer type arguments is undefined.

A type-generic macro corresponding to a function indicated in Table 7.1 is affected by constant rounding modes (7.6.5).

NOTE The type-generic macro definition in the example in 6.5.2.1 does not conform to this specification. A conforming macro can be implemented as follows:

```
#define cbrt(X) \
_Generic((X), \
    long double: _Roundwise_cbrtl, \
    default: _Roundwise_cbrt, \
    float: _Roundwise_cbrtf \
)(X)
```

where **_Roundwise_cbrtl**, **_Roundwise_cbrt**, and **_Roundwise_cbrtf** are pointers to functions that are equivalent to **cbrtl**, **cbrt**, and **cbrtf**, respectively, but that are guaranteed to be affected by constant rounding modes (7.6.3).

EXAMPLE With the declarations

```
#include <tgmath.h>
int n;
float f;
double d;
long double ld;
float complex fc;
double complex dc;
long double complex ldc;
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 d32;
_Decimal64 d64;
_Decimal128 d128;
#endif
```

functions invoked by use of type-generic macros are shown in the Table 7.10:

Table 7.10 — Generic macro use to underlying invocation

macro use	invocation
<code>exp(n)</code>	<code>exp(n)</code> , the function
<code>acosh(f)</code>	<code>acoshf(f)</code>
<code>sin(d)</code>	<code>sin(d)</code> , the function
<code>atan(ld)</code>	<code>atanl(ld)</code>
<code>log(fc)</code>	<code>clogf(fc)</code>
<code>sqrt(dc)</code>	<code>csqrt(dc)</code>
<code>pow(ldc, f)</code>	<code>cpowl(ldc, f)</code>
<code>remainder(n, n)</code>	<code>remainder(n, n)</code> , the function
<code>nextafter(d, f)</code>	<code>nextafter(d, f)</code> , the function
<code>nexttoward(f, ld)</code>	<code>nexttowardf(f, ld)</code>
<code>copysign(n, ld)</code>	<code>copysignl(n, ld)</code>
<code>ceil(fc)</code>	undefined
<code>rint(dc)</code>	undefined
<code>fmaximum(ldc, ld)</code>	undefined
<code>carg(n)</code>	<code>carg(n)</code> , the function
<code>cproj(f)</code>	<code>cprojf(f)</code>
<code>creal(d)</code>	<code>creal(d)</code> , the function
<code>cimag(ld)</code>	<code>cimagl(ld)</code>
<code>fabs(fc)</code>	<code>cabsf(fc)</code>
<code>carg(dc)</code>	<code>carg(dc)</code> , the function
<code>cproj(ldc)</code>	<code>cprojl(ldc)</code>
<code>fsub(f, ld)</code>	<code>fsubl(f, ld)</code>
<code>fdiv(d, n)</code>	<code>fdiv(d, n)</code> , the function
<code>dfma(f, d, ld)</code>	<code>dfmal(f, d, ld)</code>
<code>dadd(f, f)</code>	<code>daddl(f, f)</code>
<code>dsqrt(dc)</code>	undefined
<code>exp(d64)</code>	<code>expd64(d64)</code>
<code>sqrt(d32)</code>	<code>sqrtd32(d32)</code>
<code>fmaximum(d64, d128)</code>	<code>fmaximumd128(d64, d128)</code>
<code>pow(d32, n)</code>	<code>powd64(d32, n)</code>
<code>remainder(d64, d)</code>	undefined
<code>creal(d64)</code>	undefined
<code>remquo(d32, d32, &n)</code>	undefined
<code>llquantexp(d)</code>	undefined
<code>quantize(dc)</code>	undefined
<code>samequantum(n, n)</code>	undefined
<code>d32sub(d32, d128)</code>	<code>d32subd128(d32, d128)</code>
<code>d32div(d64, n)</code>	<code>d32divd64(d64, n)</code>
<code>d64fma(d32, d64, d128)</code>	<code>d64fmad128(d32, d64, d128)</code>

d64add(d32, d32)	d64addd128(d32, d32)
d64sqrt(d)	undefined
dadd(n, d64)	undefined

7.28 Threads <threads.h>

7.28.1 Introduction

The header <threads.h> includes the header <time.h>, defines macros, and declares types, enumeration constants, and functions that support multiple threads of execution.³⁶⁹⁾

Implementations that define the macro `__STDC_NO_THREADS__` may not provide this header nor support any of its facilities.

The macros are

```
ONCE_FLAG_INIT
```

which expands to a value that can be used to initialize an object of type `once_flag`; and

```
TSS_DTOR_ITERATIONS
```

which expands to an integer constant expression representing the maximum number of times that destructors will be called when a thread terminates.

The types are

```
cnd_t
```

which is a complete object type that holds an identifier for a condition variable;

```
thrd_t
```

which is a complete object type that holds an identifier for a thread;

```
tss_t
```

which is a complete object type that holds an identifier for a thread-specific storage pointer;

```
mtx_t
```

which is a complete object type that holds an identifier for a mutex;

```
tss_dtor_t
```

which is the function pointer type `void (*)(void*)`, used for a destructor for a thread-specific storage pointer;

```
thrd_start_t
```

which is the function pointer type `int (*)(void*)` that is passed to `thrd_create` to create a new thread; and

```
once_flag
```

which is a complete object type that holds a flag for use by `call_once`.

The enumeration constants are

```
mtx_plain
```

which is passed to `mtx_init` to create a mutex object that does not support timeout;

```
mtx_recursive
```

³⁶⁹⁾See “future library directions” (7.33.20).

which is passed to `mtx_init` to create a mutex object that supports recursive locking;

mtx_timed

which is passed to `mtx_init` to create a mutex object that supports timeout;

thrd_timedout

which is returned by a timed wait function to indicate that the time specified in the call was reached without acquiring the requested resource;

thrd_success

which is returned by a function to indicate that the requested operation succeeded;

thrd_busy

which is returned by a function to indicate that the requested operation failed because a resource requested by a test and return function is already in use;

thrd_error

which is returned by a function to indicate that the requested operation failed; and

thrd_nomem

which is returned by a function to indicate that the requested operation failed because it was unable to allocate memory.

Forward references: date and time (7.29).

7.28.2 Initialization functions

7.28.2.1 The `call_once` function

Synopsis

```
#include <threads.h>
void call_once(once_flag *flag, void (*func)(void));
```

Description

The `call_once` function uses the `once_flag` pointed to by `flag` to ensure that `func` is called exactly once, the first time the `call_once` function is called with that value of `flag`. Completion of an effective call to the `call_once` function synchronizes with all subsequent calls to the `call_once` function with the same value of `flag`.

Returns

The `call_once` function returns no value.

7.28.3 Condition variable functions

7.28.3.1 The `cnd_broadcast` function

Synopsis

```
#include <threads.h>
int cnd_broadcast(cnd_t *cnd);
```

Description

The **cnd_broadcast** function unblocks all the threads that are blocked on the condition variable pointed to by **cond** at the time of the call. If no threads are blocked on the condition variable pointed to by **cond** at the time of the call, the function does nothing.

Returns

The **cnd_broadcast** function returns **thrd_success** on success, or **thrd_error** if the request could not be honored.

7.28.3.2 The **cnd_destroy** function

Synopsis

```
#include <threads.h>
void cnd_destroy(cnd_t *cond);
```

Description

The **cnd_destroy** function releases all resources used by the condition variable pointed to by **cond**. The **cnd_destroy** function requires that no threads be blocked waiting for the condition variable pointed to by **cond**.

Returns

The **cnd_destroy** function returns no value.

7.28.3.3 The **cnd_init** function

Synopsis

```
#include <threads.h>
int cnd_init(cnd_t *cond);
```

Description

The **cnd_init** function creates a condition variable. If it succeeds it sets the object pointed to by **cond** to a value that uniquely identifies the newly created condition variable. A thread that calls **cnd_wait** on a newly created condition variable will block.

Returns

The **cnd_init** function returns **thrd_success** on success, or **thrd_nomem** if no memory could be allocated for the newly created condition, or **thrd_error** if the request could not be honored.

7.28.3.4 The **cnd_signal** function

Synopsis

```
#include <threads.h>
int cnd_signal(cnd_t *cond);
```

Description

The **cnd_signal** function unblocks one of the threads that are blocked on the condition variable pointed to by **cond** at the time of the call. If no threads are blocked on the condition variable at the time of the call, the function does nothing and returns success.

Returns

The **cnd_signal** function returns **thrd_success** on success or **thrd_error** if the request could not be honored.

7.28.3.5 The **cnd_timedwait** function

Synopsis

```
#include <threads.h>
int cnd_timedwait(cnd_t * restrict cond, mtx_t * restrict mtx,
    const struct timespec * restrict ts);
```


Description

The **cnd_timedwait** function atomically unlocks the mutex pointed to by **mtx** and blocks until the condition variable pointed to by **cond** is signaled by a call to **cnd_signal** or to **cnd_broadcast**, or until after the **TIME_UTC**-based calendar time pointed to by **ts**, or until it is unblocked due to an unspecified reason. When the calling thread becomes unblocked it locks the object pointed to by **mtx** before it returns. The **cnd_timedwait** function requires that the mutex pointed to by **mtx** be locked by the calling thread.

Returns

The **cnd_timedwait** function returns **thrd_success** upon success, or **thrd_timedout** if the time specified in the call was reached without acquiring the requested resource, or **thrd_error** if the request could not be honored.

7.28.3.6 The cnd_wait function**Synopsis**

```
#include <threads.h>
int cnd_wait(cnd_t *cond, mtx_t *mtx);
```

Description

The **cnd_wait** function atomically unlocks the mutex pointed to by **mtx** and blocks until the condition variable pointed to by **cond** is signaled by a call to **cnd_signal** or to **cnd_broadcast**, or until it is unblocked due to an unspecified reason. When the calling thread becomes unblocked it locks the mutex pointed to by **mtx** before it returns. The **cnd_wait** function requires that the mutex pointed to by **mtx** be locked by the calling thread.

Returns

The **cnd_wait** function returns **thrd_success** on success or **thrd_error** if the request could not be honored.

7.28.4 Mutex functions**7.28.4.1 General**

For purposes of determining the existence of a data race, lock and unlock operations behave as atomic operations. All lock and unlock operations on a particular mutex occur in some particular total order.

NOTE This total order can be viewed as the modification order of the mutex.

7.28.4.2 The mtx_destroy function**Synopsis**

```
#include <threads.h>
void mtx_destroy(mtx_t *mtx);
```

Description

The **mtx_destroy** function releases any resources used by the mutex pointed to by **mtx**. No threads can be blocked waiting for the mutex pointed to by **mtx**.

Returns

The **mtx_destroy** function returns no value.

7.28.4.3 The mtx_init function**Synopsis**

```
#include <threads.h>
int mtx_init(mtx_t *mtx, int type);
```

Description

The `mtx_init` function creates a mutex object with properties indicated by `type`, which shall have one of these values:

- `mtx_plain` for a simple non-recursive mutex,
- `mtx_timed` for a non-recursive mutex that supports timeout,
- `mtx_plain | mtx_recursive` for a simple recursive mutex, or
- `mtx_timed | mtx_recursive` for a recursive mutex that supports timeout.

If the `mtx_init` function succeeds, it sets the mutex pointed to by `mtx` to a value that uniquely identifies the newly created mutex.

Returns

The `mtx_init` function returns `thrd_success` on success, or `thrd_error` if the request could not be honored.

7.28.4.4 The `mtx_lock` function

Synopsis

```
#include <threads.h>
int mtx_lock(mtx_t *mtx);
```

Description

The `mtx_lock` function blocks until it locks the mutex pointed to by `mtx`. If the mutex is non-recursive, it shall not be locked by the calling thread. Prior calls to `mtx_unlock` on the same mutex synchronize with this operation.

Returns

The `mtx_lock` function returns `thrd_success` on success, or `thrd_error` if the request could not be honored.

7.28.4.5 The `mtx_timedlock` function

Synopsis

```
#include <threads.h>
int mtx_timedlock(mtx_t * restrict mtx, const struct timespec * restrict ts);
```

Description

The `mtx_timedlock` function endeavors to block until it locks the mutex pointed to by `mtx` or until after the `TIME_UTC`-based calendar time pointed to by `ts`. The specified mutex shall support timeout. If the operation succeeds, prior calls to `mtx_unlock` on the same mutex synchronize with this operation.

Returns

The `mtx_timedlock` function returns `thrd_success` on success, or `thrd_timedout` if the time specified was reached without acquiring the requested resource, or `thrd_error` if the request could not be honored.

7.28.4.6 The `mtx_trylock` function

Synopsis

```
#include <threads.h>
int mtx_trylock(mtx_t *mtx);
```

Description

The **mtx_trylock** function endeavors to lock the mutex pointed to by **mtx**. If the mutex is already locked, the function returns without blocking. If the operation succeeds, prior calls to **mtx_unlock** on the same mutex synchronize with this operation.

Returns

The **mtx_trylock** function returns **thrd_success** on success, or **thrd_busy** if the resource requested is already in use, or **thrd_error** if the request could not be honored. **mtx_trylock** can spuriously fail to lock an unused resource, in which case it returns **thrd_busy**.

7.28.4.7 The mtx_unlock function**Synopsis**

```
#include <threads.h>
int mtx_unlock(mtx_t *mtx);
```

Description

The **mtx_unlock** function unlocks the mutex pointed to by **mtx**. The mutex pointed to by **mtx** shall be locked by the calling thread.

Returns

The **mtx_unlock** function returns **thrd_success** on success or **thrd_error** if the request could not be honored.

7.28.5 Thread functions**7.28.5.1 The thrd_create function****Synopsis**

```
#include <threads.h>
int thrd_create(thrd_t *thr, thrd_start_t func, void *arg);
```

Description

The **thrd_create** function creates a new thread executing **func(arg)**. If the **thrd_create** function succeeds, it sets the object pointed to by **thr** to the identifier of the newly created thread. (A thread's identifier can be reused for a different thread once the original thread has exited and either been detached or joined to another thread.) The completion of the **thrd_create** function synchronizes with the beginning of the execution of the new thread.

Returning from **func** has the same behavior as invoking **thrd_exit** with the value returned from **func**.

Returns

The **thrd_create** function returns **thrd_success** on success, or **thrd_nomem** if no memory could be allocated for the thread requested, or **thrd_error** if the request could not be honored.

7.28.5.2 The thrd_current function**Synopsis**

```
#include <threads.h>
thrd_t thrd_current(void);
```

Description

The **thrd_current** function identifies the thread that called it.

Returns

The **thrd_current** function returns the identifier of the thread that called it.

7.28.5.3 The `thrd_detach` function

Synopsis

```
#include <threads.h>
int thrd_detach(thrd_t thr);
```

Description

The `thrd_detach` function tells the operating system to dispose of any resources allocated to the thread identified by `thr` when that thread terminates. The thread identified by `thr` shall not have been previously detached or joined with another thread.

Returns

The `thrd_detach` function returns `thrd_success` on success or `thrd_error` if the request could not be honored.

7.28.5.4 The `thrd_equal` function

Synopsis

```
#include <threads.h>
int thrd_equal(thrd_t thr0, thrd_t thr1);
```

Description

The `thrd_equal` function will determine whether the thread identified by `thr0` refers to the thread identified by `thr1`.

Returns

The `thrd_equal` function returns zero if the thread `thr0` and the thread `thr1` refer to different threads. Otherwise the `thrd_equal` function returns a nonzero value.

7.28.5.5 The `thrd_exit` function

Synopsis

```
#include <threads.h>
[[noreturn]] void thrd_exit(int res);
```

Description

For every thread-specific storage key which was created with a non-null destructor and for which the value is non-null, `thrd_exit` sets the value associated with the key to a null pointer value and then invokes the destructor with its previous value. The order in which destructors are invoked is unspecified.

If after this process there remain keys with both non-null destructors and values, the implementation repeats this process up to `TSS_DTOR_ITERATIONS` times.

Following this, the `thrd_exit` function terminates execution of the calling thread and sets its result code to `res`.

The program terminates normally after the last thread has been terminated. The behavior is as if the program called the `exit` function with the status `EXIT_SUCCESS` at thread termination time.

Returns

The `thrd_exit` function returns no value.

7.28.5.6 The `thrd_join` function

Synopsis

```
#include <threads.h>
int thrd_join(thrd_t thr, int *res);
```

Description

The **thrd_join** function joins the thread identified by **thr** with the current thread by blocking until the other thread has terminated. If the parameter **res** is not a null pointer, it stores the thread's result code in the integer pointed to by **res**. The termination of the other thread synchronizes with the completion of the **thrd_join** function. The thread identified by **thr** shall not have been previously detached or joined with another thread.

Returns

The **thrd_join** function returns **thrd_success** on success or **thrd_error** if the request could not be honored.

7.28.5.7 The **thrd_sleep** function

Synopsis

```
#include <threads.h>
int thrd_sleep(const struct timespec *duration, struct timespec *remaining);
```

Description

The **thrd_sleep** function suspends execution of the calling thread until either the interval specified by **duration** has elapsed or a signal which is not being ignored is received. If interrupted by a signal and the **remaining** argument is not null, the amount of time remaining (the requested interval minus the time actually slept) is stored in the interval it points to. The **duration** and **remaining** arguments can point to the same object.

The suspension time can be longer than requested because the interval is rounded up to an integer multiple of the sleep resolution or because of the scheduling of other activity by the system. But, except for the case of being interrupted by a signal, the suspension time will not be less than that specified, as measured by the system clock **TIME_UTC**.

Returns

The **thrd_sleep** function returns zero if the requested time has elapsed, **-1** if it has been interrupted by a signal, or a negative value (which can also be **-1**) if it fails.

7.28.5.8 The **thrd_yield** function

Synopsis

```
#include <threads.h>
void thrd_yield(void);
```

Description

The **thrd_yield** function endeavors to permit other threads to run, even if the current thread would ordinarily continue to run.

Returns

The **thrd_yield** function returns no value.

7.28.6 Thread-specific storage functions

7.28.6.1 The **tss_create** function

Synopsis

```
#include <threads.h>
int tss_create(tss_t *key, tss_dtor_t dtor);
```

Description

The **tss_create** function creates a thread-specific storage pointer with destructor **dtor**, which can be null.

A null pointer value is associated with the newly created key in all existing threads. Upon subsequent thread creation, the value associated with all keys is initialized to a null pointer value in the new thread.

Destructors associated with thread-specific storage are not invoked at program termination.

The **tss_create** function shall not be called from within a destructor.

Returns

If the **tss_create** function is successful, it sets the thread-specific storage pointed to by **key** to a value that uniquely identifies the newly created pointer and returns **thrd_success**; otherwise, **thrd_error** is returned and the thread-specific storage pointed to by **key** is set to an indeterminate representation.

7.28.6.2 The **tss_delete** function

Synopsis

```
#include <threads.h>
void tss_delete(tss_t key);
```

Description

The **tss_delete** function releases any resources used by the thread-specific storage identified by **key**. The **tss_delete** function shall only be called with a value for **key** that was returned by a call to **tss_create** before the thread commenced executing destructors.

If **tss_delete** is called while another thread is executing destructors, whether this will affect the number of invocations of the destructor associated with **key** on that thread is unspecified.

Calling **tss_delete** will not result in the invocation of any destructors.

Returns

The **tss_delete** function returns no value.

7.28.6.3 The **tss_get** function

Synopsis

```
#include <threads.h>
void *tss_get(tss_t key);
```

Description

The **tss_get** function returns the value for the current thread held in the thread-specific storage identified by **key**. The **tss_get** function shall only be called with a value for **key** that was returned by a call to **tss_create** before the thread commenced executing destructors.

Returns

The **tss_get** function returns the value for the current thread if successful, or zero if unsuccessful.

7.28.6.4 The **tss_set** function

Synopsis

```
#include <threads.h>
int tss_set(tss_t key, void *val);
```

Description

The **tss_set** function sets the value for the current thread held in the thread-specific storage identified by **key** to **val**. The **tss_set** function shall only be called with a value for **key** that was returned by a call to **tss_create** before the thread commenced executing destructors.

This action will not invoke the destructor associated with the key on the value being replaced.

Returns

The **tss_set** function returns **thrd_success** on success or **thrd_error** if the request could not be honored.

7.29 Date and time <time.h>

7.29.1 Components of time

The header <time.h> defines several macros, and declares types and functions for manipulating time. Many functions deal with a *calendar time* that represents the current date (according to the Gregorian calendar) and time. Some functions deal with *local time*, which is the calendar time expressed for some specific time zone, and with *Daylight Saving Time*, which is a temporary change in the algorithm for determining local time. The local time zone and Daylight Saving Time are implementation-defined.

The feature test macro `__STDC_VERSION_TIME_H__` expands to the token 202311L. The other macros defined are `NULL` (described in 7.21);

`CLOCKS_PER_SEC`

which expands to an expression with type `clock_t` (described later in this subclause) that is the number per second of the value returned by the `clock` function;

`TIME_UTC`
`TIME_MONOTONIC`

which expand to integer constants greater than 0 designating the calendar time and monotonic time bases, respectively. Additional time base macro definitions, beginning with `TIME_` and an uppercase letter, may also be specified by the implementation;³⁷⁰⁾ and,

`TIME_ACTIVE`
`TIME_THREAD_ACTIVE`

which, if defined, expand to integer values, designating overall execution and thread-specific active processing time bases, respectively.

The definition of macros for time bases other than `TIME_UTC` are optional. If defined, the corresponding time bases are supported by `timespec_get` and `timespec_getres`, and their values are positive. If defined, the value of the optional macro `TIME_ACTIVE` shall be different from the constants `TIME_UTC` and `TIME_MONOTONIC` and shall not change during the same program invocation. The optional macro `TIME_THREAD_ACTIVE` shall not be defined if the implementation does not support threads; its value shall be different from `TIME_UTC`, `TIME_MONOTONIC`, and `TIME_ACTIVE`, it shall be the same for all expansions of the macro for the same thread, and the value provided for one thread shall not be used by a different thread as the `base` argument of `timespec_get` or `timespec_getres`.

The types declared are `size_t` (described in 7.21);

`clock_t`

and

`time_t`

which are real types capable of representing times;

`struct timespec`

which holds an interval specified in seconds and nanoseconds (which may represent a calendar time based on a particular epoch); and

`struct tm`

³⁷⁰⁾See future library directions (7.33). Implementations can define additional time bases, but are only required to support a real time clock based on UTC.

which holds the components of a calendar time, called the *broken-down time*.

The range and precision of times representable in **clock_t** and **time_t** are implementation-defined. The **timespec** structure shall contain at least the following members, in any order. The semantics of the members and their normal ranges are expressed in the comments.³⁷¹⁾

```
time_t      tv_sec; // whole seconds --  $\geq 0$ 
/* see the following */ tv_nsec; // nanoseconds -- [0, 999999999]
```

The **tv_nsec** member shall be an implementation-defined signed integer type capable of representing the range [0, 999999999].

The **tm** structure shall contain at least the following members, in any order.³⁷²⁾ The semantics of the members and their normal ranges are expressed in the comments.

```
int tm_sec; // seconds after the minute -- [0, 60]
int tm_min; // minutes after the hour -- [0, 59]
int tm_hour; // hours since midnight -- [0, 23]
int tm_mday; // day of the month -- [1, 31]
int tm_mon; // months since January -- [0, 11]
int tm_year; // years since 1900
int tm_wday; // days since Sunday -- [0, 6]
int tm_yday; // days since January 1 -- [0, 365]
int tm_isdst; // Daylight Saving Time flag
```

The value of **tm_isdst** is positive if Daylight Saving Time is in effect, zero if Daylight Saving Time is not in effect, and negative if the information is not available.

7.29.2 Time manipulation functions

7.29.2.1 The clock function

Synopsis

```
#include <time.h>
clock_t clock(void);
```

Description

The **clock** function determines the processor time used.

Returns

The **clock** function returns the implementation's best approximation of the active processing time associated with the program execution since the beginning of an implementation-defined era related only to the program invocation. To determine the time in seconds, the value returned by the **clock** function should be divided by the value of the macro **CLOCKS_PER_SEC**. If the processor time used is not available, the function returns the value (**clock_t**) (-1). If the value cannot be represented, the function returns an unspecified value.³⁷³⁾

7.29.2.2 The difftime function

Synopsis

```
#include <time.h>
double difftime(time_t time1, time_t time0);
```

Description

The **difftime** function computes the difference between two calendar times: **time1** - **time0**.

³⁷¹⁾ The **tv_sec** member is a linear count of seconds and potentially does not have the normal semantics of a **time_t**.

³⁷²⁾ The range [0, 60] for **tm_sec** allows for a positive leap second.

³⁷³⁾ This can be due to overflow of the **clock_t** type.

Returns

The `difftime` function returns the difference expressed in seconds as a `double`.

7.29.2.3 The `mktime` function

Synopsis

```
#include <time.h>
time_t mktime(struct tm *timeptr);
```

Description

The `mktime` function converts the broken-down time, expressed as local time, in the structure pointed to by `timeptr` into a calendar time value with the same encoding as that of the values returned by the `time` function. The original values of the `tm_wday` and `tm_yday` components of the structure are ignored, and the original values of the other components are not restricted to the ranges indicated previously. If the local time to be used for the conversion is one that includes Daylight Saving Time adjustments, a positive or zero value for `tm_isdst` causes the `mktime` function to perform the conversion as if Daylight Saving Time, respectively, is or is not in effect for the specified time. A negative value causes it to attempt to determine whether Daylight Saving Time is in effect for the specified time; if it determines that Daylight Saving Time is in effect it produces the same result as an equivalent call with a positive `tm_isdst` value, otherwise it produces the same result as an equivalent call with a `tm_isdst` value of zero.³⁷⁴⁾ On successful completion, the components of the structure are set to the same values that would be returned by a call to the `localtime` function with the calculated calendar time as its argument.

Returns

The `mktime` function returns the specified calendar time encoded as a value of type `time_t`. If the calendar time cannot be represented in the `time_t` encoding used for the return value or the value to be returned in the `tm_year` component of the structure pointed to by `timeptr` cannot be represented as an `int`, the function returns the value `(time_t)(-1)` and does not change the value of the `tm_wday` component of the structure.

EXAMPLE What day of the week is July 4, 2001?

```
#include <stdio.h>
#include <time.h>
static const char *const wday[] = {
    "Sunday", "Monday", "Tuesday", "Wednesday",
    "Thursday", "Friday", "Saturday", "-unknown-"
};
struct tm time_str;
/* ... */

time_str.tm_year = 2001 - 1900;
time_str.tm_mon = 7 - 1;
time_str.tm_mday = 4;
time_str.tm_hour = 0;
time_str.tm_min = 0;
time_str.tm_sec = 1;
time_str.tm_isdst = -1;
time_str.tm_wday = 7;
mktime(&time_str);
printf("%s\n", wday[time_str.tm_wday]);
```

7.29.2.4 The `timegm` function

Synopsis

³⁷⁴⁾If the broken-down time specifies a time that is either skipped over or repeated when a transition to or from Daylight Saving Time occurs, it is unspecified whether the `mktime` function produces the same result as an equivalent call with a positive `tm_isdst` value or as an equivalent call with a `tm_isdst` value of zero.

```
#include <time.h>
time_t timegm(struct tm *timeptr);
```

Description

The **timegm** function converts the broken-down time, expressed as UTC time, in the structure pointed to by **timeptr** into a calendar time value with the same encoding as that of the values returned by the **time** function. The original values of the **tm_wday** and **tm_yday** components of the structure are ignored, and the original values of the other components are not restricted to the ranges indicated previously. On successful completion, the values of the **tm_wday** and **tm_yday** components of the structure are set appropriately, and the other components are set to represent the specified calendar time, but with their values forced to the ranges indicated previously; the final value of **tm_mday** is not set until **tm_mon** and **tm_year** are determined.

Returns

The **timegm** function returns the specified calendar time encoded as a value of type **time_t**. If the calendar time cannot be represented in the **time_t** encoding used for the return value or the value to be returned in the **tm_year** component of the structure pointed to by **timeptr** cannot be represented as an **int**, the function returns the value **(time_t)(-1)** and does not change the value of the **tm_wday** component of the structure.

7.29.2.5 The **time** function

Synopsis

```
#include <time.h>
time_t time(time_t *timer);
```

Description

The **time** function determines the current calendar time. The encoding of the value is unspecified.

Returns

The **time** function returns the implementation's best approximation to the current calendar time. The value **(time_t)(-1)** is returned if the calendar time is not available. If **timer** is not a null pointer, the return value is also assigned to the object it points to.

7.29.2.6 The **timespec_get** function

Synopsis

```
#include <time.h>
int timespec_get(struct timespec *ts, int base);
```

Description

The **timespec_get** function sets the interval pointed to by **ts** to hold the current calendar time based on the specified time base.

If **base** is **TIME_UTC**, the **tv_sec** member is set to the number of seconds since an implementation-defined *epoch*, truncated to a whole value and the **tv_nsec** member is set to the integral number of nanoseconds, rounded to the resolution of the system clock.³⁷⁵⁾ The optional time base **TIME_MONOTONIC** is the same, but the reference point is an implementation-defined time point; different program invocations can refer to the same or different reference points.³⁷⁶⁾ For the same program invocation, the results of two calls to **timespec_get** with **TIME_MONOTONIC** such that the first happens before the second shall not be decreasing. It is implementation-defined if **TIME_MONOTONIC** accounts for time during which the execution environment is suspended.³⁷⁷⁾ For the optional time

³⁷⁵⁾ Although a **struct timespec** object describes times with nanosecond resolution, the available resolution is system dependent and can even be greater than 1 second.

³⁷⁶⁾ Commonly, this reference point is the boot time of the execution environment or the start of the execution.

³⁷⁷⁾ The execution environment can, for example, lack the ability to track physical time that elapsed during suspension in a low power consumption mode.

bases **TIME_ACTIVE** and **TIME_THREAD_ACTIVE** the result is similar, but the call measures the amount of active processing time associated with the whole program invocation or with the calling thread, respectively.

Returns

If the **timespec_get** function is successful it returns the nonzero value **base**; otherwise, it returns zero.

Recommended practice

It is recommended practice that timing results of calls to **timespec_get** with **TIME_ACTIVE**, if defined, and of calls to **clock** are as close to each other as their types, value ranges, and resolutions (obtained with **timespec_getres** and **CLOCKS_PER_SEC**, respectively) allow. Because of its wider value range and improved indications on error, **timespec_get** with time base **TIME_ACTIVE** should be used instead of **clock** by new code whenever possible.

7.29.2.7 The **timespec_getres** function

Synopsis

```
#include <time.h>
int timespec_getres(struct timespec *ts, int base);
```

Description

If **ts** is non-null and **base** is supported by the **timespec_get** function, the **timespec_getres** function returns the resolution of the time provided by the **timespec_get** function for **base** in the **timespec** structure pointed to by **ts**. For each supported **base**, multiple calls to the **timespec_getres** function during the same program execution shall have identical results.

Returns

If the value **base** is supported by the **timespec_get** function, the **timespec_getres** function returns the nonzero value **base**; otherwise, it returns zero.

7.29.3 Time conversion functions

7.29.3.1 General

Functions with a **_r** suffix place the result of the conversion into the buffer referred by **buf** and return that pointer. These functions and the function **strftime** shall not be subject to data races, unless the time or calendar state is changed in a multi-thread execution.³⁷⁸⁾

Functions **gmtime** and **localtime** are the same as their counterparts suffixed with **_r**. In place of the parameter **buf**, they use a pointer to one or two broken-down time structures. Similarly, an array of **char** is commonly used by **asctime** and **ctime**. Execution of any of the functions that return a pointer to one of these objects may overwrite the information returned from any previous call to one of these functions that uses the same object. These functions are not reentrant and are not required to avoid data races with each other. Accessing the returned pointer after the thread that called the function that returned it has exited results in undefined behavior. The implementation shall behave as if no other library functions call these functions.

7.29.3.2 The **asctime** function

Synopsis

```
#include <time.h>
[[deprecated]] char *asctime(const struct tm *timeptr);
```

Description

This function is obsolescent and should be avoided in new code.

³⁷⁸⁾ This does not mean that these functions are forbidden to read global state that describes the time and calendar settings of the execution, such as the **LC_TIME** locale or the implementation-defined specification of the local time zone. Only the setting of that state by **setlocale** or by means of implementation-defined functions can constitute races.

The **asctime** function converts the broken-down time in the structure pointed to by **timeptr** into a string in the form

```
Sun Sep 16 01:03:52 1973\n\0
```

using the equivalent of the following algorithm.

```
[[deprecated]] char *asctime(const struct tm *timeptr)
{
    static const char wday_name[7][3] = {
        "Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"
    };
    static const char mon_name[12][3] = {
        "Jan", "Feb", "Mar", "Apr", "May", "Jun",
        "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"
    };
    static char result[26];

    snprintf(result, 26, "%.3s %.3s%3d %.2d:%.2d:%.2d %d\n",
        wday_name[timeptr->tm_wday],
        mon_name[timeptr->tm_mon],
        timeptr->tm_mday, timeptr->tm_hour,
        timeptr->tm_min, timeptr->tm_sec,
        1900 + timeptr->tm_year);
    return result;
}
```

If any of the members of the broken-down time contain values that are outside their normal ranges,³⁷⁹⁾ the behavior of the **asctime** function is undefined. Likewise, if the calculated year exceeds four digits or is less than the year 1000, the behavior is undefined.

Returns

The **asctime** function returns a pointer to the string.

7.29.3.3 The **ctime** function

Synopsis

```
#include <time.h>
[[deprecated]] char *ctime(const time_t *timer);
```

Description

This function is obsolescent and should be avoided in new code.

The **ctime** function converts the calendar time pointed to by **timer** to local time in the form of a string. They are equivalent to:

```
asctime(localtime(timer))
```

Returns

The **ctime** function returns the pointer returned by the **asctime** function with that broken-down time as argument.

Forward references: the **localtime** functions (7.29.3.5).

7.29.3.4 The **gmtime** functions

Synopsis

```
#include <time.h>
```

³⁷⁹⁾See 7.29.1.

```
struct tm *gmtime(const time_t *timer);
struct tm *gmtime_r(const time_t *timer, struct tm *buf);
```

Description

The **gmtime** functions convert the calendar time pointed to by **timer** into a broken-down time, expressed as UTC.

Returns

The **gmtime** functions return a pointer to the broken-down time, or a null pointer if the specified time cannot be converted to UTC.

7.29.3.5 The **localtime** functions

Synopsis

```
#include <time.h>
struct tm *localtime(const time_t *timer);
struct tm *localtime_r(const time_t *timer, struct tm *buf);
```

Description

The **localtime** functions convert the calendar time pointed to by **timer** into a broken-down time, expressed as local time.

Returns

The **localtime** functions return a pointer to the broken-down time, or a null pointer if the specified time cannot be converted to local time.

7.29.3.6 The **strftime** function

Synopsis

```
#include <time.h>
size_t strftime(char * restrict s, size_t maxsize, const char * restrict format,
                const struct tm * restrict timeptr);
```

Description

The **strftime** function places characters into the array pointed to by **s** as controlled by the string pointed to by **format**. The format shall be a multibyte character sequence, beginning and ending in its initial shift state. The **format** string consists of zero or more conversion specifiers and ordinary multibyte characters. A conversion specifier consists of a % character, possibly followed by an E or O modifier character (described later), followed by a character that determines the behavior of the conversion specifier. All ordinary multibyte characters (including the terminating null character) are copied unchanged into the array. If copying takes place between objects that overlap, the behavior is undefined. No more than **maxsize** characters are placed into the array.

Each conversion specifier shall be replaced by appropriate characters as described in the following list. The appropriate characters shall be determined using the **LC_TIME** category of the current locale and by the values of zero or more members of the broken-down time structure pointed to by **timeptr**, as specified in brackets in the description. If any of the specified values is outside the normal range, the characters stored are unspecified.

- %a is replaced by the locale’s abbreviated weekday name. [**tm_wday**]
- %A is replaced by the locale’s full weekday name. [**tm_wday**]
- %b is replaced by the locale’s abbreviated month name. [**tm_mon**]
- %B is replaced by the locale’s full month name. [**tm_mon**]
- %c is replaced by the locale’s appropriate date and time representation. [all specified in 7.29.1]
- %C is replaced by the year divided by 100 and truncated to an integer, as a decimal number (**00–99**). [**tm_year**]

- %d is replaced by the day of the month as a decimal number (01–31). [tm_mday]
- %D is equivalent to "%m/%d/%y". [tm_mon, tm_mday, tm_year]
- %e is replaced by the day of the month as a decimal number (1–31); a single digit is preceded by a space. [tm_mday]
- %F is equivalent to "%Y-%m-%d" (the ISO 8601 date format, when the year is between 1000 and 9999 inclusive). [tm_year, tm_mon, tm_mday]
- %g is replaced by the last 2 digits of the week-based year (see further in this subclause) as a decimal number (00–99). [tm_year, tm_wday, tm_yday]
- %G is replaced by the week-based year (see further in this subclause) as a decimal number (e.g. 1997). [tm_year, tm_wday, tm_yday]
- %h is equivalent to "%b". [tm_mon]
- %H is replaced by the hour (24-hour clock) as a decimal number (00–23). [tm_hour]
- %I is replaced by the hour (12-hour clock) as a decimal number (01–12). [tm_hour]
- %j is replaced by the day of the year as a decimal number (001–366). [tm_yday]
- %m is replaced by the month as a decimal number (01–12). [tm_mon]
- %M is replaced by the minute as a decimal number (00–59). [tm_min]
- %n is replaced by a new-line character.
- %p is replaced by the locale's equivalent of the AM/PM designations associated with a 12-hour clock. [tm_hour]
- %r is replaced by the locale's 12-hour clock time. [tm_hour, tm_min, tm_sec]
- %R is equivalent to "%H:%M". [tm_hour, tm_min]
- %S is replaced by the second as a decimal number (00–60). [tm_sec]
- %t is replaced by a horizontal-tab character.
- %T is equivalent to "%H:%M:%S" (the ISO 8601 time format). [tm_hour, tm_min, tm_sec]
- %u is replaced by the ISO 8601 weekday as a decimal number (1–7), where Monday is 1. [tm_wday]
- %U is replaced by the week number of the year (the first Sunday as the first day of week 1) as a decimal number (00–53). [tm_year, tm_wday, tm_yday]
- %V is replaced by the ISO 8601 week number (see further in this subclause) as a decimal number (01–53). [tm_year, tm_wday, tm_yday]
- %w is replaced by the weekday as a decimal number (0–6), where Sunday is 0. [tm_wday]
- %W is replaced by the week number of the year (the first Monday as the first day of week 1) as a decimal number (00–53). [tm_year, tm_wday, tm_yday]
- %x is replaced by the locale's appropriate date representation. [all specified in 7.29.1]
- %X is replaced by the locale's appropriate time representation. [all specified in 7.29.1]
- %y is replaced by the last 2 digits of the year as a decimal number (00–99). [tm_year]
- %Y is replaced by the year as a decimal number (e.g. 1997). [tm_year]
- %z is replaced by the offset from UTC in the ISO 8601 format "-0430" (meaning 4 hours 30 minutes behind UTC, west of Greenwich), or by no characters if no time zone is determinable. [tm_isdst]
- %Z is replaced by the locale's time zone name or abbreviation, or by no characters if no time zone is determinable. [tm_isdst]
- %% is replaced by %.

Some conversion specifiers can be modified by the inclusion of an E or O modifier character to indicate an alternative format or specification. If the alternative format or specification does not exist for the current locale, the modifier is ignored.

%Ec	is replaced by the locale's alternative date and time representation.
%EC	is replaced by the name of the base year (period) in the locale's alternative representation.
%Ex	is replaced by the locale's alternative date representation.
%EX	is replaced by the locale's alternative time representation.
%Ey	is replaced by the offset from %EC (year only) in the locale's alternative representation.
%EY	is replaced by the locale's full alternative year representation.
%Ob	is replaced by the locale's abbreviated alternative month name.
%OB	is replaced by the locale's alternative appropriate full month name.
%Od	is replaced by the day of the month, using the locale's alternative numeric symbols (filled as needed with leading zeros, or with leading spaces if there is no alternative symbol for zero).
%Oe	is replaced by the day of the month, using the locale's alternative numeric symbols (filled as needed with leading spaces).
%OH	is replaced by the hour (24-hour clock), using the locale's alternative numeric symbols.
%OI	is replaced by the hour (12-hour clock), using the locale's alternative numeric symbols.
%Om	is replaced by the month, using the locale's alternative numeric symbols.
%OM	is replaced by the minutes, using the locale's alternative numeric symbols.
%OS	is replaced by the seconds, using the locale's alternative numeric symbols.
%Ou	is replaced by the ISO 8601 weekday as a number in the locale's alternative representation, where Monday is 1.
%OU	is replaced by the week number, using the locale's alternative numeric symbols.
%OV	is replaced by the ISO 8601 week number, using the locale's alternative numeric symbols.
%Ow	is replaced by the weekday as a number, using the locale's alternative numeric symbols.
%OW	is replaced by the week number of the year, using the locale's alternative numeric symbols.
%Oy	is replaced by the last 2 digits of the year, using the locale's alternative numeric symbols.

%g, %G, and %V give values according to the ISO 8601 week-based year. In this system, weeks begin on a Monday and week 1 of the year is the week that includes January 4th, which is also the week that includes the first Thursday of the year, and is also the first week that contains at least four days in the year. If the first Monday of January is the 2nd, 3rd, or 4th, the preceding days are part of the last week of the preceding year; thus, for Saturday 2nd January 1999, %G is replaced by 1998 and %V is replaced by 53. If December 29th, 30th, or 31st is a Monday, it and any following days are part of week 1 of the following year. Thus, for Tuesday 30th December 1997, %G is replaced by 1998 and %V is replaced by 01.

If a conversion specifier is not one of the ones previously specified, the behavior is undefined.

In the "C" locale, the E and O modifiers are ignored and the replacement strings for the following specifiers are:

%a	the first three characters of %A.
%A	one of "Sunday", "Monday", ..., "Saturday".
%b	the first three characters of %B.
%B	one of "January", "February", ..., "December".
%c	equivalent to "%a %b %e %T %Y".
%p	one of "AM" or "PM".
%r	equivalent to "%I:%M:%S %p".
%x	equivalent to "%m/%d/%y".
%X	equivalent to %T.
%Z	implementation-defined.

Returns

If the total number of resulting characters including the terminating null character is not more than **maxsize**, the **strftime** function returns the number of characters placed into the array pointed to by **s** not including the terminating null character. Otherwise, zero is returned and the members of the array have an indeterminate representation.

7.30 Unicode utilities <uchar.h>

7.30.1 General

The header <uchar.h> declares one macro, a few types, and several functions for manipulating Unicode characters.

The macro

```
__STDC_VERSION_UCHAR_H__
```

is an integer constant expression with a value equivalent to 202311L.

The types declared are **mbstate_t** (described in 7.31.1) and **size_t** (described in 7.21);

```
char8_t
```

which is an unsigned integer type used for 8-bit characters and is the same type as **unsigned char**;

```
char16_t
```

which is an unsigned integer type used for 16-bit characters and is the same type as **uint_least16_t** (described in 7.22.2.3); and

```
char32_t
```

which is an unsigned integer type used for 32-bit characters and is the same type as **uint_least32_t** (also described in 7.22.2.3).

7.30.2 Restartable multibyte/wide character conversion functions

7.30.2.1 General

These functions have a parameter, **ps**, of type pointer to **mbstate_t** that points to an object that can completely describe the current conversion state of the associated multibyte character sequence, which the functions alter as necessary. If **ps** is a null pointer, each function uses its own internal **mbstate_t** object instead, which is initialized prior to the first call to the function to the initial conversion state; the functions are not required to avoid data races with other calls to the same function in this case. It is implementation-defined whether the internal **mbstate_t** object has thread storage duration; if it has thread storage duration, it is initialized to the initial conversion state prior to the first call to the function on the new thread. The implementation behaves as if no library function calls these functions with a null pointer for **ps**.

When used in the functions in this subclause, the encoding of **char8_t**, **char16_t**, and **char32_t** objects, and sequences of such objects, is UTF-8, UTF-16, and UTF-32, respectively. Similarly, the encoding of **char** and **wchar_t**, and sequences of such objects, is the execution and wide execution encodings (6.2.9), respectively.

7.30.2.2 The mbrtoc8 function

Synopsis

```
#include <uchar.h>
size_t mbrtoc8(char8_t * restrict pc8, const char * restrict s, size_t n,
               mbstate_t * restrict ps);
```

Description

If **s** is a null pointer, the **mbrtoc8** function is equivalent to the call:

```
mbrtoc8(nullptr, "", 1, ps)
```

In this case, the values of the parameters **pc8** and **n** are ignored.

If **s** is not a null pointer, the **mbrtoc8** function function inspects at most **n** bytes beginning with the byte pointed to by **s** to determine the number of bytes needed to complete the next multibyte character (including any shift sequences). If the function determines that the next multibyte character is complete and valid, it determines the values of the corresponding characters and then, if **pc8** is not a null pointer, stores the value of the first (or only) such character in the object pointed to by **pc8**. Subsequent calls will store successive characters without consuming any additional input until all the characters have been stored. If the corresponding character is the null character, the resulting state described is the initial conversion state.

Returns

The **mbrtoc8** function returns the first of the following that applies (given the current conversion state):

- 0** if the next **n** or fewer bytes complete the multibyte character that corresponds to the null character (which is the value stored).
- between 1 and n inclusive* if the next **n** or fewer bytes complete a valid multibyte character (which is the value stored); the value returned is the number of bytes that complete the multibyte character.
- (size_t)(-3)** if the next character resulting from a previous call has been stored (no bytes from the input have been consumed by this call).
- (size_t)(-2)** if the next **n** bytes contribute to an incomplete (but potentially valid) multibyte character, and all **n** bytes have been processed (no value is stored).³⁸⁰⁾
- (size_t)(-1)** if an encoding error occurs, in which case the next **n** or fewer bytes do not contribute to a complete and valid multibyte character (no value is stored); the value of the macro **EILSEQ** is stored in **errno**, and the conversion state is unspecified.

7.30.2.3 The **c8rtomb** function

Synopsis

```
#include <uchar.h>
size_t c8rtomb(char * restrict s, char8_t c8, mbstate_t * restrict ps);
```

Description

If **s** is a null pointer, the **c8rtomb** function is equivalent to the call

```
c8rtomb(buf, u8'\0', ps)
```

where **buf** is an internal buffer.

If **s** is not a null pointer, the **c8rtomb** function determines the number of bytes needed to represent the multibyte character that corresponds to the character given or completed by **c8** (including any shift sequences), and stores the multibyte character representation in the array whose first element is pointed to by **s**, or stores nothing if **c8** does not represent a complete character. At most **MB_CUR_MAX** bytes are stored. If **c8** is a null character, a null byte is stored, preceded by any shift sequence needed to restore the initial shift state; the resulting state described is the initial conversion state.

Returns

The **c8rtomb** function returns the number of bytes stored in the array object (including any shift sequences). When **c8** is not a valid character, an encoding error occurs: the function stores the value of the macro **EILSEQ** in **errno** and returns **(size_t)(-1)**; the conversion state is unspecified.

7.30.2.4 The **mbrtoc16** function

³⁸⁰⁾When **n** has at least the value of the **MB_CUR_MAX** macro, this case can only occur if **s** points at a sequence of redundant shift sequences (for implementations with state-dependent encodings).

Synopsis

```
#include <uchar.h>
size_t mbrtoc16(char16_t * restrict pc16, const char * restrict s, size_t n,
               mbstate_t * restrict ps);
```

Description

If **s** is a null pointer, the **mbrtoc16** function is equivalent to the call:

```
mbrtoc16(nullptr, "", 1, ps)
```

In this case, the values of the parameters **pc16** and **n** are ignored.

If **s** is not a null pointer, the **mbrtoc16** function inspects at most **n** bytes beginning with the byte pointed to by **s** to determine the number of bytes needed to complete the next multibyte character (including any shift sequences). If the function determines that the next multibyte character is complete and valid, it determines the values of the corresponding wide characters and then, if **pc16** is not a null pointer, stores the value of the first (or only) such character in the object pointed to by **pc16**. Subsequent calls will store successive wide characters without consuming any additional input until all the characters have been stored. If the corresponding wide character is the null wide character, the resulting state described is the initial conversion state.

Returns

The **mbrtoc16** function returns the first of the following that applies (given the current conversion state):

- 0** if the next **n** or fewer bytes complete the multibyte character that corresponds to the null wide character (which is the value stored).
- between 1 and n inclusive* if the next **n** or fewer bytes complete a valid multibyte character (which is the value stored); the value returned is the number of bytes that complete the multibyte character.
- (size_t)(-3)** if the next character resulting from a previous call has been stored (no bytes from the input have been consumed by this call).
- (size_t)(-2)** if the next **n** bytes contribute to an incomplete (but potentially valid) multibyte character, and all **n** bytes have been processed (no value is stored).³⁸¹⁾
- (size_t)(-1)** if an encoding error occurs, in which case the next **n** or fewer bytes do not contribute to a complete and valid multibyte character (no value is stored); the value of the macro **EILSEQ** is stored in **errno**, and the conversion state is unspecified.

7.30.2.5 The c16rtomb function

Synopsis

```
#include <uchar.h>
size_t c16rtomb(char * restrict s, char16_t c16, mbstate_t * restrict ps);
```

Description

If **s** is a null pointer, the **c16rtomb** function is equivalent to the call

```
c16rtomb(buf, u'\0', ps)
```

where **buf** is an internal buffer.

³⁸¹⁾When **n** has at least the value of the **MB_CUR_MAX** macro, this case can only occur if **s** points at a sequence of redundant shift sequences (for implementations with state-dependent encodings).

If **s** is not a null pointer, the **c16rtomb** function determines the number of bytes needed to represent the multibyte character that corresponds to the wide character given or completed by **c16** (including any shift sequences), and stores the multibyte character representation in the array whose first element is pointed to by **s**, or stores nothing if **c16** does not represent a complete character. At most **MB_CUR_MAX** bytes are stored. If **c16** is a null wide character, a null byte is stored, preceded by any shift sequence needed to restore the initial shift state; the resulting state described is the initial conversion state.

Returns

The **c16rtomb** function returns the number of bytes stored in the array object (including any shift sequences). When **c16** is not a valid wide character, an encoding error occurs: the function stores the value of the macro **EILSEQ** in **errno** and returns **(size_t)(-1)**; the conversion state is unspecified.

7.30.2.6 The **mbrtoc32** function

Synopsis

```
#include <uchar.h>
size_t mbrtoc32(char32_t * restrict pc32, const char * restrict s, size_t n,
                mbstate_t * restrict ps);
```

Description

If **s** is a null pointer, the **mbrtoc32** function is equivalent to the call:

```
mbrtoc32(NULLptr, "", 1, ps)
```

In this case, the values of the parameters **pc32** and **n** are ignored.

If **s** is not a null pointer, the **mbrtoc32** function inspects at most **n** bytes beginning with the byte pointed to by **s** to determine the number of bytes needed to complete the next multibyte character (including any shift sequences). If the function determines that the next multibyte character is complete and valid, it determines the values of the corresponding wide characters and then, if **pc32** is not a null pointer, stores the value of the first (or only) such character in the object pointed to by **pc32**. Subsequent calls will store successive wide characters without consuming any additional input until all the characters have been stored. If the corresponding wide character is the null wide character, the resulting state described is the initial conversion state.

Returns

The **mbrtoc32** function returns the first of the following that applies (given the current conversion state):

- 0** if the next **n** or fewer bytes complete the multibyte character that corresponds to the null wide character (which is the value stored).
- between 1 and n inclusive* if the next **n** or fewer bytes complete a valid multibyte character (which is the value stored); the value returned is the number of bytes that complete the multibyte character.
- (size_t)(-3)** if the next character resulting from a previous call has been stored (no bytes from the input have been consumed by this call).
- (size_t)(-2)** if the next **n** bytes contribute to an incomplete (but potentially valid) multibyte character, and all **n** bytes have been processed (no value is stored).³⁸²⁾
- (size_t)(-1)** if an encoding error occurs, in which case the next **n** or fewer bytes do not contribute to a complete and valid multibyte character (no value is stored); the value of the macro **EILSEQ** is stored in **errno**, and the conversion state is unspecified.

³⁸²⁾When **n** has at least the value of the **MB_CUR_MAX** macro, this case can only occur if **s** points at a sequence of redundant shift sequences (for implementations with state-dependent encodings).

7.30.2.7 The `c32rtomb` function

Synopsis

```
#include <uchar.h>
size_t c32rtomb(char * restrict s, char32_t c32, mbstate_t * restrict ps);
```

Description

If `s` is a null pointer, the `c32rtomb` function is equivalent to the call

```
c32rtomb(buf, U'\0', ps)
```

where `buf` is an internal buffer.

If `s` is not a null pointer, the `c32rtomb` function determines the number of bytes needed to represent the multibyte character that corresponds to the wide character given by `c32` (including any shift sequences), and stores the multibyte character representation in the array whose first element is pointed to by `s`. At most `MB_CUR_MAX` bytes are stored. If `c32` is a null wide character, a null byte is stored, preceded by any shift sequence needed to restore the initial shift state; the resulting state described is the initial conversion state.

Returns

The `c32rtomb` function returns the number of bytes stored in the array object (including any shift sequences). When `c32` is not a valid wide character, an encoding error occurs: the function stores the value of the macro `EILSEQ` in `errno` and returns `(size_t)(-1)`; the conversion state is unspecified.

7.31 Extended multibyte and wide character utilities <wchar.h>

7.31.1 Introduction

The header <wchar.h> defines five macros, and declares four data types, one tag, and many functions.³⁸³⁾

The macro

`__STDC_VERSION_WCHAR_H__`

is an integer constant expression with a value equivalent to 202311L.

The types declared are **wchar_t** and **size_t** (both described in 7.21);

mbstate_t

which is a complete object type other than an array type that can hold the conversion state information necessary to convert between sequences of multibyte characters and wide characters;

wint_t

which is an integer type unchanged by default argument promotions that can hold any value corresponding to members of the extended character set, as well as at least one value that does not correspond to any member of the extended character set (see subsequent **WEOF** description),³⁸⁴⁾ and

struct tm

which is declared as an incomplete structure type (the contents are described in 7.29.1).

The macros defined are **NULL** (described in 7.21); **WCHAR_MIN**, **WCHAR_MAX**, and **WCHAR_WIDTH** (described in 7.22); and

WEOF

which expands to a constant expression of type **wint_t** whose value does not correspond to any member of the extended character set.³⁸⁵⁾ It is accepted (and returned) by several functions in this subclause to indicate *end-of-file*, that is, no more input from a stream. It is also used as a wide character value that does not correspond to any member of the extended character set.

The functions declared are grouped as follows:

- Functions that perform input and output of wide characters, or multibyte characters, or both;
- Functions that provide wide string numeric conversion;
- Functions that perform general wide string manipulation;
- Functions for wide string date and time conversion; and
- Functions that provide extended capabilities for conversion between multibyte and wide character sequences.

Arguments to the functions in this subclause may point to arrays containing **wchar_t** values that do not correspond to members of the extended character set. Such values shall be processed according to the specified semantics, except that it is unspecified whether an encoding error occurs if such a value appears in the format string for a function in 7.31.2 or 7.31.5 and the specified semantics do not require that value to be processed by **wcrtomb**.

Unless explicitly stated otherwise, if the execution of a function described in this subclause causes copying to take place between objects that overlap, the behavior is undefined.

³⁸³⁾ See “future library directions” (7.33.21).

³⁸⁴⁾ **wchar_t** and **wint_t** can be the same integer type.

³⁸⁵⁾ The value of the macro **WEOF** can differ from that of **Eof** and the value can be positive.

7.31.2 Formatted wide character input/output functions

7.31.2.1 General

The formatted wide character input/output functions shall behave as if there is a sequence point after the actions associated with each specifier.³⁸⁶⁾

7.31.2.2 The `fwprintf` function

Synopsis

```
#include <stdio.h>
#include <wchar.h>
int fwprintf(FILE * restrict stream, const wchar_t * restrict format, ...);
```

Description

The `fwprintf` function writes output to the stream pointed to by `stream`, under control of the wide string pointed to by `format` that specifies how subsequent arguments are converted for output. If there are insufficient arguments for the format, the behavior is undefined. If the format is exhausted while arguments remain, the excess arguments are evaluated (as always) but are otherwise ignored. The `fwprintf` function returns when the end of the format string is encountered.

The format is composed of zero or more directives: ordinary wide characters (not %), which are copied unchanged to the output stream; and conversion specifications, each of which results in fetching zero or more subsequent arguments, converting them, if applicable, according to the corresponding conversion specifier, and then writing the result to the output stream.

Each conversion specification is introduced by the wide character %. After the %, the following appear in sequence:

- Zero or more *flags* (in any order) that modify the meaning of the conversion specification.
- An optional minimum *field width*. If the converted value has fewer wide characters than the field width, it is padded with spaces (by default) on the left (or right, if the left adjustment flag, described later, has been given) to the field width. The field width takes the form of an asterisk * (described later) or a nonnegative decimal integer.³⁸⁷⁾
- An optional *precision* that gives the minimum number of digits to appear for the b, B, d, i, o, u, x, and X conversions, the number of digits to appear after the decimal-point wide character for a, A, e, E, f, and F conversions, the maximum number of significant digits for the g and G conversions, or the maximum number of wide characters to be written for s conversions. The precision takes the form of a period (.) followed either by an asterisk * (described later) or by an optional nonnegative decimal integer; if only the period is specified, the precision is taken as zero. If a precision appears with any other conversion specifier, the behavior is undefined.
- An optional *length modifier* that specifies the size of the argument.
- A *conversion specifier* wide character that specifies the type of conversion to be applied.

As noted previously, a field width, or precision, or both, may be indicated by an asterisk. In this case, an `int` argument supplies the field width or precision. The arguments specifying field width, or precision, or both, shall appear (in that order) before the argument (if any) to be converted. A negative field width argument is taken as a - flag followed by a positive field width. A negative precision argument is taken as if the precision were omitted.

The flag wide characters and their meanings are:

- The result of the conversion is left-justified within the field. (It is right-justified if this flag is not specified.)

³⁸⁶⁾The `fwprintf` functions perform writes to memory for the %n specifier.

³⁸⁷⁾0 is taken as a flag, not as the beginning of a field width.

- + The result of a signed conversion always begins with a plus or minus sign. (It begins with a sign only when a value with a negative sign is converted if this flag is not specified.)³⁸⁸⁾
- space* If the first wide character of a signed conversion is not a sign, or if a signed conversion results in no wide characters, a space is prefixed to the result. If the *space* and + flags both appear, the *space* flag is ignored.
- # The result is converted to an “alternative form”. For o conversion, it increases the precision, if and only if necessary, to force the first digit of the result to be a zero (if the value and precision are both 0, a single 0 is printed). For b conversion, a nonzero result has **0b** prefixed to it. For the optional B conversion as described later in this subclause, a nonzero result has **0B** prefixed to it. For x (or X) conversion, a nonzero result has **0x** (or **0X**) prefixed to it. For a, A, e, E, f, F, g, and G conversions, the result of converting a floating-point number always contains a decimal-point wide character, even if no digits follow it. (Normally, a decimal-point wide character appears in the result of these conversions only if a digit follows it.) For g and G conversions, trailing zeros are *not* removed from the result. For other conversions, the behavior is undefined.
- 0 For b, B, d, i, o, u, x, X, a, A, e, E, f, F, g, and G conversions, leading zeros (following any indication of sign or base) are used to pad to the field width rather than performing space padding, except when converting an infinity or NaN. If the 0 and - flags both appear, the 0 flag is ignored. For b, B, d, i, o, u, x, and X conversions, if a precision is specified, the 0 flag is ignored. For other conversions, the behavior is undefined.

The length modifiers and their meanings are:

- hh Specifies that a following b, B, d, i, o, u, x, or X conversion specifier applies to a **signed char** or **unsigned char** argument (the argument will have been promoted according to the integer promotions, but its value shall be converted to **signed char** or **unsigned char** before printing); or that a following n conversion specifier applies to a pointer to a **signed char** argument.
- h Specifies that a following b, B, d, i, o, u, x, or X conversion specifier applies to a **short int** or **unsigned short int** argument (the argument will have been promoted according to the integer promotions, but its value shall be converted to **short int** or **unsigned short int** before printing); or that a following n conversion specifier applies to a pointer to a **short int** argument.
- l (ell) Specifies that a following b, B, d, i, o, u, x, or X conversion specifier applies to a **long int** or **unsigned long int** argument; that a following n conversion specifier applies to a pointer to a **long int** argument; that a following c conversion specifier applies to a **wint_t** argument; that a following s conversion specifier applies to a pointer to a **wchar_t** argument; or has no effect on a following a, A, e, E, f, F, g, or G conversion specifier.
- ll (ell-ell) Specifies that a following b, B, d, i, o, u, x, or X conversion specifier applies to a **long long int** or **unsigned long long int** argument; or that a following n conversion specifier applies to a pointer to a **long long int** argument.
- j Specifies that a following b, B, d, i, o, u, x, or X conversion specifier applies to an **intmax_t** or **uintmax_t** argument; or that a following n conversion specifier applies to a pointer to an **intmax_t** argument.
- z Specifies that a following b, B, d, i, o, u, x, or X conversion specifier applies to a **size_t** or the corresponding signed integer type argument; or that a following n conversion specifier applies to a pointer to a signed integer type corresponding to **size_t** argument.

³⁸⁸⁾The results of all floating conversions of a negative zero, and of negative values that round to zero, include a minus sign.

t	Specifies that a following b, B, d, i, o, u, x, or X conversion specifier applies to a ptrdiff_t or the corresponding unsigned integer type argument; or that a following n conversion specifier applies to a pointer to a ptrdiff_t argument.
wN	Specifies that a following b, B, d, i, o, u, x, or X conversion specifier applies to an integer argument with a specific width where <i>N</i> is a positive decimal integer with no leading zeros (the argument will have been promoted according to the integer promotions, but its value shall be converted to the unpromoted type); or that a following n conversion specifier applies to a pointer to an integer type argument with a width of <i>N</i> bits. All minimum-width integer types (7.22.2.3) and exact-width integer types (7.22.2.2) defined in the header <code><stdint.h></code> shall be supported. Other supported values of <i>N</i> are implementation-defined.
wfN	Specifies that a following b, B, d, i, o, u, x, or X conversion specifier applies to a fastest minimum-width integer argument with a specific width where <i>N</i> is a positive decimal integer with no leading zeros (the argument will have been promoted according to the integer promotions, but its value shall be converted to the unpromoted type); or that a following n conversion specifier applies to a pointer to a fastest minimum-width integer type argument with a width of <i>N</i> bits. All fastest minimum-width integer types (7.22.2.4) defined in the header <code><stdint.h></code> shall be supported. Other supported values of <i>N</i> are implementation-defined.
L	Specifies that a following a, A, e, E, f, F, g, or G conversion specifier applies to a long double argument.
H	Specifies that a following a, A, e, E, f, F, g, or G conversion specifier applies to a _Decimal32 argument.
D	Specifies that a following a, A, e, E, f, F, g, or G conversion specifier applies to a _Decimal64 argument.
DD	Specifies that a following a, A, e, E, f, F, g, or G conversion specifier applies to a _Decimal128 argument.

If a length modifier appears with any conversion specifier other than as specified previously, the behavior is undefined.

The conversion specifiers and their meanings are:

d, i	The int argument is converted to signed decimal in the style <i>[-]dddd</i> . The precision specifies the minimum number of digits to appear; if the value being converted can be represented in fewer digits, it is expanded with leading zeros. The default precision is 1. The result of converting a zero value with a precision of zero is no wide characters.
b, B, o, u, x, X	The unsigned int argument is converted to unsigned binary (b or B), unsigned octal (o), unsigned decimal (u), or unsigned hexadecimal notation (x or X) in the style <i>dddd</i> ; the letters <code>abcdef</code> are used for x conversion and the letters <code>ABCDEF</code> for X conversion. The precision specifies the minimum number of digits to appear; if the value being converted can be represented in fewer digits, it is expanded with leading zeros. The default precision is 1. The result of converting a zero value with a precision of zero is no wide characters. The specifier B is optional and provides the same functionality as b, except for the # flag as previously specified. The PRIB macros from <code><inttypes.h></code> shall only be defined if the implementation follows the specification as given here.
f, F	A double argument representing a floating-point number is converted to decimal notation in the style <i>[-]ddd.ddd</i> , where the number of digits after the decimal-point wide character is equal to the precision specification. If the precision is missing, it is taken as 6; if the precision is zero and the # flag is not specified, no decimal-point wide character appears. If a decimal-point wide character appears, at least one digit appears before it. The value is rounded to the appropriate number of digits.

A **double** argument representing an infinity is converted in one of the styles *[-]inf* or *[-]infinity* — which style is implementation-defined. A **double** argument representing a NaN is converted in one of the styles *[-]nan* or *[-]nan (n-wchar-sequence)* — which style, and the meaning of any *n-wchar-sequence*, is implementation-defined. The F conversion specifier produces INF, INFINITY, or NAN instead of inf, infinity, or nan, respectively.³⁸⁹⁾

e, E A **double** argument representing a floating-point number is converted in the style *[-]d.ddde±dd*, where there is one digit (which is nonzero if the argument is nonzero) before the decimal-point wide character and the number of digits after it is equal to the precision; if the precision is missing, it is taken as 6; if the precision is zero and the # flag is not specified, no decimal-point wide character appears. The value is rounded to the appropriate number of digits. The E conversion specifier produces a number with E instead of e introducing the exponent. The exponent always contains at least two digits, and only as many more digits as necessary to represent the exponent. If the value is zero, the exponent is zero.

A **double** argument representing an infinity or NaN is converted in the style of an f or F conversion specifier.

g, G A **double** argument representing a floating-point number is converted in style f or e (or in style F or E in the case of a G conversion specifier), depending on the value converted and the precision. Let P equal the precision if nonzero, 6 if the precision is omitted, or 1 if the precision is zero. Then, if a conversion with style E would have an exponent of X :

if $P > X \geq -4$, the conversion is with style f (or F) and precision $P - (X + 1)$.

otherwise, the conversion is with style e (or E) and precision $P - 1$.

Finally, unless the # flag is used, any trailing zeros are removed from the fractional portion of the result and the decimal-point wide character is removed if there is no fractional portion remaining.

A **double** argument representing an infinity or NaN is converted in the style of an f or F conversion specifier.

a, A A **double** argument representing a floating-point number is converted in the style *[-]0xh.hhhhp±d*, where there is one hexadecimal digit (which is nonzero if the argument is a normalized floating-point number and is otherwise unspecified) before the decimal-point wide character³⁹⁰⁾ and the number of hexadecimal digits after it is equal to the precision; if the precision is missing and FLT_RADIX is a power of 2, then the precision is sufficient for an exact representation of the value; if the precision is missing and FLT_RADIX is not a power of 2, then the precision is sufficient to distinguish³⁹¹⁾ values of type **double**, except that trailing zeros may be omitted; if the precision is zero and the # flag is not specified, no decimal-point wide character appears. The letters abcdef are used for a conversion and the letters ABCDEF for A conversion. The A conversion specifier produces a number with X and P instead of x and p. The exponent always contains at least one digit, and only as

³⁸⁹⁾When applied to infinite and NaN values, the -, +, and space flag wide characters have their usual meaning; the # and 0 flag wide characters have no effect.

³⁹⁰⁾Binary implementations can choose the hexadecimal digit to the left of the decimal-point wide character so that subsequent digits align to nibble (4-bit) boundaries. This implementation choice affects numerical values printed with a precision P that is insufficient to represent all values exactly. Implementations with different conventions about the most significant hexadecimal digit will round at different places, affecting the numerical value of the hexadecimal result. For example, possible printed output for the code

```
#include <stdio.h>
/* ... */
double x = 123.0;
printf("%.1a", x);
```

include "0x1.fp+6" and "0xf.6p+3" whose numerical values are 124 and 123, respectively. Portable code seeking identical numerical results on different platforms should avoid precisions P that require rounding.

³⁹¹⁾The formatting precision P is sufficient to distinguish values of the source type if $16^P > b^p$ where b (not a power of 2) and p are the base and precision of the source type (5.3.5.3.3). A smaller P potentially suffices depending on the implementation's scheme for determining the digit to the left of the decimal-point wide character.

many more digits as necessary to represent the decimal exponent of 2. If the value is zero, the exponent is zero.

A **double** argument representing an infinity or NaN is converted in the style of an **f** or **F** conversion specifier.

If an **H**, **D**, or **DD** modifier is present and the precision is missing, then for a decimal floating type argument represented by a triple of integers (s, c, q) , where n is the number of significant digits in the coefficient c ,

- if $-(n + 5) \leq q \leq 0$, use style **f** (or style **F** in the case of an **A** conversion specifier) with formatting precision equal to $-q$,
- otherwise, use style **e** (or style **E** in the case of an **A** conversion specifier) with formatting precision equal to $n - 1$, with the exceptions that if $c = 0$ then the digit-sequence in the exponent-part shall have the value q (rather than 0), and that the exponent is always expressed with the minimum number of digits required to represent its value (the exponent never contains a leading zero).

If the precision P is present (in the conversion specification) and is zero or at least as large as the precision p (5.3.5.3.3) of the decimal floating type, the conversion is as if the precision were missing. If the precision P is present (and nonzero) and less than the precision p of the decimal floating type, the conversion first obtains an intermediate result as follows, where n is the number of significant digits in the coefficient:

- If $n \leq P$, set the intermediate result to the input.
- If $n > P$, round the input value, according to the current rounding direction for decimal floating-point operations, to P decimal digits, with unbounded exponent range, representing the result with a P -digit integer coefficient when in the form (s, c, q) .

Convert the intermediate result in the manner described previously for the case where the precision is missing.

- c** If no **l** length modifier is present, the **int** argument is converted to a wide character as if by calling **btowc** and the resulting wide character is written.
- If an **l** length modifier is present, the **wint_t** argument is converted to **wchar_t** and written.
- s** If no **l** length modifier is present, the argument shall be a pointer to storage of character type containing a multibyte character sequence beginning in the initial shift state. Characters from the storage are converted as if by repeated calls to the **mbrtowc** function, with the conversion state described by an **mbsstate_t** object initialized to zero before the first multibyte character is converted, and written up to (but not including) the terminating null wide character. If the precision is specified, no more than that many wide characters are written. If the precision is not specified or is greater than the size of the converted storage, the converted storage shall contain a null wide character.
- If an **l** length modifier is present, the argument shall be a pointer to storage of **wchar_t** type. Wide characters from the storage are written up to (but not including) a terminating null wide character. If the precision is specified, no more than that many wide characters are written. If the precision is not specified or is greater than the size of the array, the storage shall contain a null wide character.
- p** The argument shall be a pointer to **void** or a pointer to a character type. The value of the pointer is converted to a sequence of printing wide characters, in an implementation-defined manner.
- n** The argument shall be a pointer to signed integer whose type is specified by the length modifier, if any, for the conversion specification, or shall be **int** if no length modifier is specified for the conversion specification. The number of wide characters written to the output stream so far by this call to **fwprintf** is stored into the integer object pointed to

by the argument. No argument is converted, but one is consumed. If the conversion specification includes any flags, a field width, or a precision, the behavior is undefined.

% A % wide character is written. No argument is converted. The complete conversion specification shall be %%.

If a conversion specification is invalid, the behavior is undefined.³⁹²⁾ **fwprintf** shall behave as if it uses **va_arg** with a type argument naming the type resulting from applying the default argument promotions to the type corresponding to the conversion specification and then converting the result of the **va_arg** expansion to the type corresponding to the conversion specification.³⁹³⁾

In no case does a nonexistent or small field width cause truncation of a field; if the result of a conversion is wider than the field width, the field is expanded to contain the conversion result.

For a and A conversions, if **FLT_RADIX** is a power of 2, the value is correctly rounded to a hexadecimal floating number with the given precision.

Recommended practice

For a and A conversions, if **FLT_RADIX** is not a power of 2 and the result is not exactly representable in the given precision, the result should be one of the two adjacent numbers in hexadecimal floating style with the given precision, with the extra stipulation that the error should have a correct sign for the current rounding direction.

For e, E, f, F, g, and G conversions, if the number of significant decimal digits is at most the maximum value *M* of the **T_DECIMAL_DIG** macros (defined in `<float.h>`), then the result should be correctly rounded.³⁹⁴⁾ If the number of significant decimal digits is more than *M* but the source value is exactly representable with *M* digits, then the result should be an exact representation with trailing zeros. Otherwise, the source value is bounded by two adjacent decimal strings $L < U$, both having *M* significant digits; the value of the resultant decimal string *D* should satisfy $L \leq D \leq U$, with the extra stipulation that the error should have a correct sign for the current rounding direction.

The uppercase B format specifier is made optional by the previous description, because it used to be available for extensions in previous versions of this document. Implementations that did not use an uppercase B as their own extension before are encouraged to implement it as previously described.

Returns

The **fwprintf** function returns the number of wide characters transmitted, or a negative value if an output or encoding error occurred or if the implementation does not support a specified width length modifier.

Environmental limits

The number of wide characters that can be produced by any single conversion shall be at least 4095.

EXAMPLE To print a date and time in the form "Sunday, July 3, 10:02" followed by π to five decimal places:

```
#include <math.h>
#include <stdio.h>
#include <wchar.h>
/* ... */
wchar_t *weekday, *month; // pointers to wide strings
int day, hour, min;
fwprintf(stdout, L"%ls, %ls %d, %.2d:%.2d\n",
         weekday, month, day, hour, min);
fwprintf(stdout, L"pi = %.5f\n", 4 * atan(1.0));
```

Forward references: the **btowc** function (7.31.6.2.1), the **mbtowc** function (7.31.6.4.3).

³⁹²⁾See "future library directions" (7.33.21).

³⁹³⁾The behavior is undefined when the types differ as specified for **va_arg** 7.16.2.2.

³⁹⁴⁾For binary-to-decimal conversion, the result format's values are the numbers representable with the given format specifier. The number of significant digits is determined by the format specifier, and in the case of fixed-point conversion by the source value as well.

7.31.2.3 The fwscanf function

Synopsis

```
#include <stdio.h>
#include <wchar.h>
int fwscanf(FILE * restrict stream, const wchar_t * restrict format, ...);
```

Description

The **fwscanf** function reads input from the stream pointed to by **stream**, under control of the wide string pointed to by **format** that specifies the admissible input sequences and how they are to be converted for assignment, using subsequent arguments as pointers to the objects to receive the converted input. If there are insufficient arguments for the format, the behavior is undefined. If the format is exhausted while arguments remain, the excess arguments are evaluated (as always) but are otherwise ignored.

The format is composed of zero or more directives: one or more white-space wide characters, an ordinary wide character (neither % nor a white-space wide character), or a conversion specification. Each conversion specification is introduced by the wide character %. After the %, the following appear in sequence:

- An optional assignment-suppressing wide character *.
- An optional decimal integer greater than zero that specifies the maximum field width (in wide characters).
- An optional *length modifier* that specifies the size of the receiving object.
- A *conversion specifier* wide character that specifies the type of conversion to be applied.

The **fwscanf** function executes each directive of the format in turn. When all directives have been executed, or if a directive fails (as detailed later in this subclause), the function returns. Failures are described as input failures (due to the occurrence of an encoding error or the unavailability of input characters), or matching failures (due to inappropriate input).

A directive composed of white-space wide character(s) is executed by reading input up to the first non-white-space wide character (which remains unread), or until no more wide characters can be read. The directive never fails.

A directive that is an ordinary wide character is executed by reading the next wide character of the stream. If that wide character differs from the directive, the directive fails and the differing and subsequent wide characters remain unread. Similarly, if end-of-file, an encoding error, or a read error prevents a wide character from being read, the directive fails.

A directive that is a conversion specification defines a set of matching input sequences, as described further in this subclause for each specifier. A conversion specification is executed in the following steps:

Input white-space wide characters are skipped, unless the specification includes a [, c, or n specifier.³⁹⁵⁾

An input item is read from the stream, unless the specification includes an n specifier. An input item is defined as the longest sequence of input wide characters which does not exceed any specified field width and which is, or is a prefix of, a matching input sequence.³⁹⁶⁾ The first wide character, if any, after the input item remains unread. If the length of the input item is zero, the execution of the directive fails; this condition is a matching failure unless end-of-file, an encoding error, or a read error prevented input from the stream, in which case it is an input failure.

Except in the case of a % specifier, the input item (or, in the case of a %n directive, the count of input wide characters) is converted to a type appropriate to the conversion specifier. If the input item is

³⁹⁵⁾ These white-space wide characters are not counted against a specified field width.
³⁹⁶⁾ **fwscanf** pushes back at most one input wide character onto the input stream. Therefore, some sequences that are acceptable to **wcstod**, **wcstol**, etc., are unacceptable to **fwscanf**.

not a matching sequence, the execution of the directive fails: this condition is a matching failure. Unless assignment suppression was indicated by a *, the result of the conversion is placed in the object pointed to by the first argument following the **format** argument that has not already received a conversion result. If this object does not have an appropriate type, or if the result of the conversion cannot be represented in the object, the behavior is undefined.

The length modifiers and their meanings are:

hh	Specifies that a following b, d, i, o, u, x, X, or n conversion specifier applies to an argument with type pointer to signed char or unsigned char .
h	Specifies that a following b, d, i, o, u, x, X, or n conversion specifier applies to an argument with type pointer to short int or unsigned short int .
l (ell)	Specifies that a following b, d, i, o, u, x, X, or n conversion specifier applies to an argument with type pointer to long int or unsigned long int ; that a following a, A, e, E, f, F, g, or G conversion specifier applies to an argument with type pointer to double ; or that a following c, s, or l conversion specifier applies to an argument with type pointer to wchar_t .
ll (ell-ell)	Specifies that a following b, d, i, o, u, x, X, or n conversion specifier applies to an argument with type pointer to long long int or unsigned long long int .
j	Specifies that a following b, d, i, o, u, x, X, or n conversion specifier applies to an argument with type pointer to intmax_t or uintmax_t .
z	Specifies that a following b, d, i, o, u, x, X, or n conversion specifier applies to an argument with type pointer to size_t or the corresponding signed integer type.
t	Specifies that a following b, d, i, o, u, x, X, or n conversion specifier applies to an argument with type pointer to ptrdiff_t or the corresponding unsigned integer type.
wN	Specifies that a following b, d, i, o, u, x, X, or n conversion specifier applies to an argument which is a pointer to an integer with a specific width where N is a positive decimal integer with no leading zeros. All minimum-width integer types (7.22.2.3) and exact-width integer types (7.22.2.2) defined in the header <code><stdint.h></code> shall be supported. Other supported values of N are implementation-defined.
wfN	Specifies that a following b, d, i, o, u, x, X, or n conversion specifier applies to an argument which is a pointer to a fastest minimum-width integer with a specific width where N is a positive decimal integer with no leading zeros. All fastest minimum-width integer types (7.22.2.4) defined in the header <code><stdint.h></code> shall be supported. Other supported values of N are implementation-defined.
L	Specifies that a following a, A, e, E, f, F, g, or G conversion specifier applies to an argument with type pointer to long double .
H	Specifies that a following a, A, e, E, f, F, g, or G conversion specifier applies to an argument with type pointer to _Decimal32 .
D	Specifies that a following a, A, e, E, f, F, g, or G conversion specifier applies to an argument with type pointer to _Decimal64 .
DD	Specifies that a following a, A, e, E, f, F, g, or G conversion specifier applies to an argument with type pointer to _Decimal128 .

If a length modifier appears with any conversion specifier other than as specified previously, the behavior is undefined.

In the following, the type of the corresponding argument for a conversion specifier shall be a pointer to a type determined by the length modifiers, if any, or specified by the conversion specifier. The conversion specifiers and their meanings are:

- d Matches an optionally signed decimal integer, whose format is the same as expected for the subject sequence of the **wcstol** function with the value 10 for the **base** argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to **int**.
- b Matches an optionally signed binary integer, whose format is the same as expected for the subject sequence of the **wcstoul** function with the value 2 for the **base** argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to **unsigned int**.
- i Matches an optionally signed integer, whose format is the same as expected for the subject sequence of the **wcstol** function with the value 0 for the **base** argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to **int**.
- o Matches an optionally signed octal integer, whose format is the same as expected for the subject sequence of the **wcstoul** function with the value 8 for the **base** argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to **unsigned int**.
- u Matches an optionally signed decimal integer, whose format is the same as expected for the subject sequence of the **wcstoul** function with the value 10 for the **base** argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to **unsigned int**.
- x Matches an optionally signed hexadecimal integer, whose format is the same as expected for the subject sequence of the **wcstoul** function with the value 16 for the **base** argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to **unsigned int**.
- a, e, f, g Matches an optionally signed floating-point number, infinity, or NaN, whose format is the same as expected for the subject sequence of the **wcstod** function. Unless a length modifier is specified, the corresponding argument shall be a pointer to **float**.
- c Matches a sequence of wide characters of exactly the number specified by the field width (1 if no field width is present in the directive).
 If no **l** length modifier is present, characters from the input field are converted as if by repeated calls to the **wcrtomb** function, with the conversion state described by an **mbstate_t** object initialized to zero before the first wide character is converted. The corresponding argument shall be a pointer to **char**, **signed char**, **unsigned char**, or **void** that points to storage large enough to accept the sequence. No null character is added.
 If an **l** length modifier is present, the corresponding argument shall be a pointer to storage of **wchar_t** large enough to accept the sequence. No null wide character is added.
- s Matches a sequence of non-white-space wide characters.
 If no **l** length modifier is present, characters from the input field are converted as if by repeated calls to the **wcrtomb** function, with the conversion state described by an **mbstate_t** object initialized to zero before the first wide character is converted. The corresponding argument shall be a pointer to **char**, **signed char**, **unsigned char**, or **void** that points to storage large enough to accept the sequence and a terminating null character, which will be added automatically.
 If an **l** length modifier is present, the corresponding argument shall be a pointer to storage of **wchar_t** large enough to accept the sequence and the terminating null wide character, which will be added automatically.
- [Matches a nonempty sequence of wide characters from a set of expected characters (the *scanset*).

If no `l` length modifier is present, characters from the input field are converted as if by repeated calls to the `wcrtomb` function, with the conversion state described by an `mbstate_t` object initialized to zero before the first wide character is converted. The corresponding argument shall be a pointer to `char`, `signed char`, `unsigned char`, or `void` that points to storage large enough to accept the sequence and a terminating null character, which will be added automatically.

If an `l` length modifier is present, the corresponding argument shall be a pointer that points to storage of `wchar_t` large enough to accept the sequence and the terminating null wide character, which will be added automatically.

The conversion specifier includes all subsequent wide characters in the **format** string, up to and including the matching right bracket (`]`). The wide characters between the brackets (the *scanlist*) compose the scanset, unless the wide character after the left bracket is a circumflex (`^`), in which case the scanset contains all wide characters that do not appear in the scanlist between the circumflex and the right bracket. If the conversion specifier begins with `[]` or `[^]`, the right bracket wide character is in the scanlist and the next following right bracket wide character is the matching right bracket that ends the specification; otherwise the first following right bracket wide character is the one that ends the specification. If a `-` wide character is in the scanlist and is not the first, nor the second where the first wide character is a `^`, nor the last character, the behavior is implementation-defined.

- p** Matches an implementation-defined set of sequences, which should be the same as the set of sequences that may be produced by the `%p` conversion of the `fwprintf` function. The corresponding argument shall be a pointer to a pointer of `void`. The input item is converted to a pointer value in an implementation-defined manner. If the input item is a value converted earlier during the same program execution, the pointer that results shall compare equal to that value; otherwise the behavior of the `%p` conversion is undefined.
- n** No input is consumed. The corresponding argument shall be a pointer of a signed integer type. The number of wide characters read from the input stream so far by this call to the `fwscanf` function is stored into the integer object pointed to by the argument. Execution of a `%n` directive does not increment the assignment count returned at the completion of execution of the `fwscanf` function. No argument is converted, but one is consumed. If the conversion specification includes an assignment-suppressing wide character or a field width, the behavior is undefined.
- %** Matches a single `%` wide character; no conversion or assignment occurs. The complete conversion specification shall be `%%`.

If a conversion specification is invalid, the behavior is undefined.³⁹⁷⁾

The conversion specifiers `A`, `E`, `F`, `G`, and `X` are also valid and behave the same as, respectively, `a`, `e`, `f`, `g`, and `x`.

Trailing white-space wide characters (including new-line wide characters) are left unread unless matched by a directive. The success of literal matches and suppressed assignments is not directly determinable other than via the `%n` directive.

Returns

The `fwscanf` function returns the value of the macro `EOF` if an input failure occurs before the first conversion (if any) has completed. Otherwise, the function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure or if the implementation does not support a specific width length modifier.

EXAMPLE 1 The call:

```
#include <stdio.h>
```

³⁹⁷⁾See “future library directions” (7.33.21).

```
#include <wchar.h>
/* ... */
int n, i; float x; wchar_t name[50];
n = fwscanf(stdin, L"%d%f%ls", &i, &x, name);
```

with the input line:

```
25 54.32E-1 thompson
```

will assign to **n** the value 3, to **i** the value 25, to **x** the value 5.432, and to **name** the sequence thompson\0.

EXAMPLE 2 The call:

```
#include <stdio.h>
#include <wchar.h>
/* ... */
int i; float x; double y;
fwscanf(stdin, L"%2d%f%d %lf", &i, &x, &y);
```

with input:

```
56789 0123 56a72
```

will assign to **i** the value 56 and to **x** the value 789.0, will skip past 0123, and will assign to **y** the value 56.0. The next wide character read from the input stream will be a.

Forward references: the **wcstod**, **wcstof**, and **wcstold** functions (7.31.4.2.2), the **wcstol**, **wcstoll**, **wcstoul**, and **wcstoull** functions (7.31.4.2.4), the **wcrtomb** function (7.31.6.4.4).

7.31.2.4 The **swprintf** function

Synopsis

```
#include <wchar.h>
int swprintf(wchar_t * restrict s, size_t n, const wchar_t * restrict format,
...);
```

Description

The **swprintf** function is equivalent to **fwprintf**, except that the argument **s** specifies an array of wide characters into which the generated output is to be written, rather than written to a stream. No more than **n** wide characters are written, including a terminating null wide character, which is always added (unless **n** is zero).

Returns

The **swprintf** function returns the number of wide characters written in the array, not counting the terminating null wide character, or a negative value if an encoding error occurred or if **n** or more wide characters were requested to be written.

7.31.2.5 The **swscanf** function

Synopsis

```
#include <wchar.h>
int swscanf(const wchar_t * restrict s, const wchar_t * restrict format, ...);
```

Description

The **swscanf** function is equivalent to **fwscanf**, except that the argument **s** specifies a wide string from which the input is to be obtained, rather than from a stream. Reaching the end of the wide string is equivalent to encountering end-of-file for the **fwscanf** function.

Returns

The **swscanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **swscanf** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.31.2.6 The vfwprintf function**Synopsis**

```
#include <stdarg.h>
#include <stdio.h>
#include <wchar.h>
int vfwprintf(FILE * restrict stream, const wchar_t * restrict format,
              va_list arg);
```

Description

The **vfwprintf** function is equivalent to **fprintf**, with the variable argument list replaced by **arg**, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** invocations). The **vfwprintf** function does not invoke the **va_end** macro.³⁹⁸⁾

Returns

The **vfwprintf** function returns the number of wide characters transmitted, or a negative value if an output or encoding error occurred.

EXAMPLE The following shows the use of the **vfwprintf** function in a general error-reporting routine.

```
#include <stdarg.h>
#include <stdio.h>
#include <wchar.h>

void error(char *function_name, wchar_t *format, ...)
{
    va_list args;

    va_start(args, format);
    // print out name of function causing error
    fprintf(stderr, L"ERROR in %s: ", function_name);
    // print out remainder of message
    vfwprintf(stderr, format, args);
    va_end(args);
}
```

7.31.2.7 The vfwscanf function**Synopsis**

```
#include <stdarg.h>
#include <stdio.h>
#include <wchar.h>
int vfwscanf(FILE * restrict stream, const wchar_t * restrict format,
             va_list arg);
```

Description

The **vfwscanf** function is equivalent to **fscanf**, with the variable argument list replaced by **arg**, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vfwscanf** function does not invoke the **va_end** macro.³⁹⁸⁾

³⁹⁸⁾As the functions **vfwprintf**, **vswprintf**, **vfwscanf**, **vwprintf**, **vwscanf**, and **vswscanf** invoke the **va_arg** macro, the representation of **arg** after the return is indeterminate.

Returns

The **vfwscanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **vfwscanf** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.31.2.8 The **vswprintf** function

Synopsis

```
#include <stdarg.h>
#include <wchar.h>
int vswprintf(wchar_t * restrict s, size_t n, const wchar_t * restrict format,
              va_list arg);
```

Description

The **vswprintf** function is equivalent to **swprintf**, with the variable argument list replaced by **arg**, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vswprintf** function does not invoke the **va_end** macro.³⁹⁸⁾

Returns

The **vswprintf** function returns the number of wide characters written in the array, not counting the terminating null wide character, or a negative value if an encoding error occurred or if **n** or more wide characters were requested to be generated.

7.31.2.9 The **vswscanf** function

Synopsis

```
#include <stdarg.h>
#include <wchar.h>
int vswscanf(const wchar_t * restrict s, const wchar_t * restrict format,
              va_list arg);
```

Description

The **vswscanf** function is equivalent to **swscanf**, with the variable argument list replaced by **arg**, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vswscanf** function does not invoke the **va_end** macro.³⁹⁸⁾

Returns

The **vswscanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **vswscanf** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.31.2.10 The **vwprintf** function

Synopsis

```
#include <stdarg.h>
#include <wchar.h>
int vwprintf(const wchar_t * restrict format, va_list arg);
```

Description

The **vwprintf** function is equivalent to **wprintf**, with the variable argument list replaced by **arg**, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vwprintf** function does not invoke the **va_end** macro.³⁹⁸⁾

Returns

The **wprintf** function returns the number of wide characters transmitted, or a negative value if an output or encoding error occurred.

7.31.2.11 The **vwscanf** function

Synopsis

```
#include <stdarg.h>
#include <wchar.h>
int vwscanf(const wchar_t * restrict format, va_list arg);
```

Description

The **vwscanf** function is equivalent to **wscanf**, with the variable argument list replaced by **arg**, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vwscanf** function does not invoke the **va_end** macro.³⁹⁸⁾

Returns

The **vwscanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **vwscanf** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.31.2.12 The **wprintf** function

Synopsis

```
#include <wchar.h>
int wprintf(const wchar_t * restrict format, ...);
```

Description

The **wprintf** function is equivalent to **fwprintf** with the argument **stdout** interposed before the arguments to **wprintf**.

Returns

The **wprintf** function returns the number of wide characters transmitted, or a negative value if an output or encoding error occurred.

7.31.2.13 The **wscanf** function

Synopsis

```
#include <wchar.h>
int wscanf(const wchar_t * restrict format, ...);
```

Description

The **wscanf** function is equivalent to **fwscanf** with the argument **stdin** interposed before the arguments to **wscanf**.

Returns

The **wscanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **wscanf** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.31.3 Wide character input/output functions

7.31.3.1 The **fgetwc** function

Synopsis

```
#include <stdio.h>
```

```
#include <wchar.h>
wint_t fgetwc(FILE *stream);
```

Description

If the end-of-file indicator for the input stream pointed to by **stream** is not set and a next wide character is present, the **fgetwc** function obtains that wide character as a **wchar_t** converted to a **wint_t** and advances the associated file position indicator for the stream (if defined).

Returns

If the end-of-file indicator for the stream is set, or if the stream is at end-of-file, the end-of-file indicator for the stream is set and the **fgetwc** function returns **WEOF**. Otherwise, the **fgetwc** function returns the next wide character from the input stream pointed to by **stream**. If a read error occurs, the error indicator for the stream is set and the **fgetwc** function returns **WEOF**. If an encoding error occurs (including too few bytes), the error indicator for the stream is set and the value of the macro **EILSEQ** is stored in **errno** and the **fgetwc** function returns **WEOF**.³⁹⁹⁾

7.31.3.2 The fgetws function

Synopsis

```
#include <stdio.h>
#include <wchar.h>
wchar_t *fgetws(wchar_t * restrict s, int n, FILE * restrict stream);
```

Description

The **fgetws** function reads at most one less than the number of wide characters specified by **n** from the stream pointed to by **stream** into the array pointed to by **s**. No additional wide characters are read after a new-line wide character (which is retained) or after end-of-file. A null wide character is written immediately after the last wide character read into the array. If **n** is negative or zero, the behavior is undefined.

Returns

The **fgetws** function returns **s** if successful. If end-of-file is encountered and no characters have been read into the array, the contents of the array remain unchanged and a null pointer is returned. If a read or encoding error occurs during the operation, the array members have an indeterminate representation and a null pointer is returned.

7.31.3.3 The fputwc function

Synopsis

```
#include <stdio.h>
#include <wchar.h>
wint_t fputwc(wchar_t c, FILE *stream);
```

Description

The **fputwc** function writes the wide character specified by **c** to the output stream pointed to by **stream**, at the position indicated by the associated file position indicator for the stream (if defined), and advances the indicator appropriately. If the file cannot support positioning requests, or if the stream was opened with append mode, the character is appended to the output stream.

Returns

The **fputwc** function returns the wide character written. If a write error occurs, the error indicator for the stream is set and **fputwc** returns **WEOF**. If an encoding error occurs, the error indicator for the stream is set and the value of the macro **EILSEQ** is stored in **errno** and **fputwc** returns **WEOF**.

7.31.3.4 The fputws function

³⁹⁹⁾ An end-of-file and a read error can be distinguished by use of the **feof** and **ferror** functions. Also, **errno** will be set to **EILSEQ** by input/output functions only if an encoding error occurs.

Synopsis

```
#include <stdio.h>
#include <wchar.h>
int fputws(const wchar_t * restrict s, FILE * restrict stream);
```

Description

The **fputws** function writes the wide string pointed to by **s** to the stream pointed to by **stream**. The terminating null wide character is not written.

Returns

The **fputws** function returns **EOF** if a write or encoding error occurs; otherwise, it returns a nonnegative value.

7.31.3.5 The fwide function

Synopsis

```
#include <stdio.h>
#include <wchar.h>
int fwide(FILE *stream, int mode);
```

Description

The **fwide** function determines the orientation of the stream pointed to by **stream**. If **mode** is greater than zero, the function first attempts to make the stream wide oriented. If **mode** is less than zero, the function first attempts to make the stream byte oriented.⁴⁰⁰⁾ Otherwise, **mode** is zero and the function does not alter the orientation of the stream.

Returns

The **fwide** function returns a value greater than zero if, after the call, the stream has wide orientation, a value less than zero if the stream has byte orientation, or zero if the stream has no orientation.

7.31.3.6 The getwc function

Synopsis

```
#include <stdio.h>
#include <wchar.h>
wint_t getwc(FILE *stream);
```

Description

The **getwc** function is equivalent to **fgetwc**, except that if it is implemented as a macro, it may evaluate **stream** more than once, so the argument should never be an expression with side effects.

Returns

The **getwc** function returns the next wide character from the input stream pointed to by **stream**, or **WEOF**.

7.31.3.7 The getwchar function

Synopsis

```
#include <wchar.h>
wint_t getwchar(void);
```

Description

The **getwchar** function is equivalent to **getwc** with the argument **stdin**.

⁴⁰⁰⁾If the orientation of the stream has already been determined, **fwide** does not change it.

Returns

The **getwchar** function returns the next wide character from the input stream pointed to by **stdin**, or **WEOF**.

7.31.3.8 The **putwc** function

Synopsis

```
#include <stdio.h>
#include <wchar.h>
wint_t putwc(wchar_t c, FILE *stream);
```

Description

The **putwc** function is equivalent to **fputwc**, except that if it is implemented as a macro, it may evaluate **stream** more than once, so that argument should never be an expression with side effects.

Returns

The **putwc** function returns the wide character written, or **WEOF**.

7.31.3.9 The **putwchar** function

Synopsis

```
#include <wchar.h>
wint_t putwchar(wchar_t c);
```

Description

The **putwchar** function is equivalent to **putwc** with the second argument **stdout**.

Returns

The **putwchar** function returns the character written, or **WEOF**.

7.31.3.10 The **ungetwc** function

Synopsis

```
#include <stdio.h>
#include <wchar.h>
wint_t ungetwc(wint_t c, FILE *stream);
```

Description

The **ungetwc** function pushes the wide character specified by **c** back onto the input stream pointed to by **stream**. Pushed-back wide characters will be returned by subsequent reads on that stream in the reverse order of their pushing. A successful intervening call (with the stream pointed to by **stream**) to a file positioning function (**fseek**, **fsetpos**, or **rewind**) discards any pushed-back wide characters for the stream. The external storage corresponding to the stream is unchanged.

One wide character of pushback is guaranteed, even if the call to the **ungetwc** function follows just after a call to a formatted wide character input function **fwscanf**, **vfwscanf**, **vwscanf**, or **wscanf**. If the **ungetwc** function is called too many times on the same stream without an intervening read or file positioning operation on that stream, the operation may fail.

If the value of **c** equals that of the macro **WEOF**, the operation fails and the input stream is unchanged.

A successful call to the **ungetwc** function clears the end-of-file indicator for the stream. The value of the file position indicator for the stream after reading or discarding all pushed-back wide characters is the same as it was before the wide characters were pushed back.⁴⁰¹⁾ For a text or binary stream, the value of its file position indicator after a successful call to the **ungetwc** function is unspecified until all pushed-back wide characters are read or discarded.

⁴⁰¹⁾A file positioning function can further modify the file position indicator after discarding any pushed-back wide characters.

Returns

The **ungetwc** function returns the wide character pushed back, or **WEOF** if the operation fails.

7.31.4 General wide string utilities

7.31.4.1 General

The header `<wchar.h>` declares functions for wide string manipulation. Various methods are used for determining the lengths of the arrays, but in all cases a **wchar_t*** argument points to the initial (lowest addressed) element of the array. If an array is accessed beyond the end of an object, the behavior is undefined.

Where an argument declared as **size_t n** determines the length of the array for a function, **n** can have the value zero on a call to that function. Unless explicitly stated otherwise in the description of a particular function in this subclause, pointer arguments on such a call shall still have valid values, as described in 7.1.4. On such a call, a function that locates a wide character finds no occurrence, a function that compares two wide character sequences returns zero, and a function that copies wide characters copies zero wide characters.

7.31.4.2 Wide string numeric conversion functions

7.31.4.2.1 General

This subclause describes wide string analogs of the **strtod** family of functions (7.24.2.6, 7.24.2.7).⁴⁰²⁾

7.31.4.2.2 The **wcstod**, **wcstof**, and **wcstold** functions

Synopsis

```
#include <wchar.h>
double wcstod(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
float wcstof(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
long double wcstold(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
```

Description

The **wcstod**, **wcstof**, and **wcstold** functions convert the initial portion of the wide string pointed to by **nptr** to **double**, **float**, and **long double** representation, respectively. First, they decompose the input string into three parts: an initial, possibly empty, sequence of white-space wide characters, a subject sequence resembling a floating constant or representing an infinity or NaN; and a final wide string of one or more unrecognized wide characters, including the terminating null wide character of the input wide string. Then, they attempt to convert the subject sequence to a floating-point number, and return the result.

The expected form of the subject sequence is an optional plus or minus sign, then one of the following:

⁴⁰²⁾Wide string analogs of the **strfromd** family of functions (7.24.2.6, 7.24.2.7) are not provided because those conversions can be done by using **mbstowcs** (7.24.9.2) to convert the result of **strfromd**, **strfromf**, and similar to wide string. For example, the following converts **double d** to wide string **ws** with at most **n-1** non-null wide characters, using style **g** formatting, and computes the number **nc** of wide characters that would have been written had **n** been sufficiently large, not counting the terminating null wide character.

```
#include <stdlib.h>
const size_t n = 20;
double d;
//...
// convert d to single-byte character string s
char s[n];
int nc = strfromd(s, n, "%g", d);
// convert s (regarded as a multi-byte character
// string) to wide string ws
wchar_t ws[n];
(void)mbstowcs(ws, s, n);
```

- a nonempty sequence of decimal digits optionally containing a decimal-point wide character, then an optional exponent part as defined for the corresponding single-byte characters in 6.4.5.3, excluding any digit separators (6.4.5.2);
- a 0x or 0X, then a nonempty sequence of hexadecimal digits optionally containing a decimal-point wide character, then an optional binary exponent part as defined in 6.4.5.3, excluding any digit separators (6.4.5.2);
- INF or INFINITY, or any other wide string equivalent except for case
- NAN or NAN(*n-wchar-sequence_{opt}*), or any other wide string equivalent except for case in the NAN part, where:
n-wchar-sequence:
 - digit*
 - nondigit*
 - n-wchar-sequence digit*
 - n-wchar-sequence nondigit*

The subject sequence is defined as the longest initial subsequence of the input wide string, starting with the first non-white-space wide character, that is of the expected form. The subject sequence contains no wide characters if the input wide string is not of the expected form.

If the subject sequence has the expected form for a floating-point number, the sequence of wide characters starting with the first digit or the decimal-point wide character (whichever occurs first) is interpreted as a floating constant according to the rules of 6.4.5.3, except that the decimal-point wide character is used in place of a period, and that if neither an exponent part nor a decimal-point wide character appears in a decimal floating-point number, or if a binary exponent part does not appear in a hexadecimal floating-point number, an exponent part of the appropriate type with value zero is assumed to follow the last digit in the string.

If the subject sequence begins with a minus sign, the sequence is interpreted as arithmetically negated.⁴⁰³⁾

A wide character sequence INF or INFINITY is interpreted as an infinity, if representable in the return type, else like a floating constant that is too large for the range of the return type. A wide character sequence NAN or NAN(*n-wchar-sequence_{opt}*) is interpreted as a quiet NaN, if supported in the return type, else like a subject sequence part that does not have the expected form; the meaning of the *n-wchar* sequence is implementation-defined.⁴⁰⁴⁾

A pointer to the final wide string is stored in the object pointed to by **endptr**, provided that **endptr** is not a null pointer.

If the subject sequence has the hexadecimal form and **FLT_RADIX** is a power of 2, the value resulting from the conversion is correctly rounded.

In other than the "C" locale, additional locale-specific subject sequence forms may be accepted.

If the subject sequence is empty or does not have the expected form, no conversion is performed; the value of **nptr** is stored in the object pointed to by **endptr**, provided that **endptr** is not a null pointer.

Recommended practice

If the subject sequence has the hexadecimal form, **FLT_RADIX** is not a power of 2, and the result is not exactly representable, the result should be one of the two numbers in the appropriate internal format that are adjacent to the hexadecimal floating source value, with the extra stipulation that the error should have a correct sign for the current rounding direction.

⁴⁰³⁾It is unspecified whether a minus-signed sequence is converted to a negative number directly or by arithmetically negating the value resulting from converting the corresponding unsigned sequence (see F.5); the two methods can yield different results if rounding is toward positive or negative infinity. In either case, the functions honor the sign of zero if floating-point arithmetic supports signed zeros.

⁴⁰⁴⁾An implementation can use the *n-wchar* sequence to determine extra information to be represented in the NaN's significand.

If the subject sequence has the decimal form and at most M significant digits, where M is the maximum value of the `T_DECIMAL_DIG` macros (defined in `<float.h>`), the result should be correctly rounded. If the subject sequence D has the decimal form and more than M significant digits, consider the two bounding, adjacent decimal strings L and U , both having M significant digits, such that the values of L , D , and U satisfy $L \leq D \leq U$. The result should be one of the (equal or adjacent) values that would be obtained by correctly rounding L and U according to the current rounding direction, with the extra stipulation that the error with respect to D should have a correct sign for the current rounding direction.⁴⁰⁵⁾

Returns

The functions return the converted value, if any. If no conversion could be performed, positive or unsigned zero is returned.

If the correct value overflows and default rounding is in effect (7.12.2), plus or minus **HUGE_VAL**, **HUGE_VALF**, or **HUGE_VALL** is returned (according to the return type and sign of the value); if the integer expression `math_errhandling & MATH_ERRNO` is nonzero, the integer expression `errno` acquires the value of **ERANGE**; if the integer expression `math_errhandling & MATH_ERREXCEPT` is nonzero, the “overflow” floating-point exception is raised.

If the result underflows (7.12.2), the functions return a value whose magnitude is no greater than the smallest normalized positive number in the return type; if the integer expression `math_errhandling & MATH_ERRNO` is nonzero, whether `errno` acquires the value **ERANGE** is implementation-defined; if the integer expression `math_errhandling & MATH_ERREXCEPT` is nonzero, whether the “underflow” floating-point exception is raised is implementation-defined.

7.31.4.2.3 The `wcstodN` functions

Synopsis

```
#include <wchar.h>
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 wcstod32(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
_Decimal64 wcstod64(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
_Decimal128 wcstod128(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
#endif
```

Description

The `wcstodN` functions convert the initial portion of the wide string pointed to by `nptr` to decimal floating type representation. First, they decompose the input wide string into three parts: an initial, possibly empty, sequence of white-space wide characters; a subject sequence resembling a floating constant or representing an infinity or NaN; and a final wide string of one or more unrecognized wide characters, including the terminating null wide character of the input wide string. Then, they attempt to convert the subject sequence to a floating-point number, and return the result.

The expected form of the subject sequence is an optional plus or minus sign, then one of the following:

- a nonempty sequence of decimal digits optionally containing a decimal-point wide character, then an optional exponent part as defined in 6.4.5.3, excluding any digit separators (6.4.5.2)
- a **0x** or **0X**, then a nonempty sequence of hexadecimal digits optionally containing a decimal-point wide character, then an optional binary exponent part as defined in 6.4.5.3, excluding any digit separators (6.4.5.2)
- **INF** or **INFINITY**, ignoring case
- **NAN** or **NAN**(*d-wchar-sequence_{opt}*), ignoring case in the **NAN** part, where:

⁴⁰⁵⁾ M is sufficiently large that L and U will usually correctly round to the same internal floating value, but if not will correctly round to adjacent values.

d-wchar-sequence:

digit
nondigit
d-wchar-sequence digit
d-wchar-sequence nondigit

The subject sequence is defined as the longest initial subsequence of the input wide string, starting with the first non-white-space wide character, that is of the expected form. The subject sequence contains no wide characters if the input wide string is not of the expected form.

If the subject sequence has the expected form for a floating-point number, the sequence of wide characters starting with the first digit or the decimal-point wide character (whichever occurs first) is interpreted as a floating constant according to the rules of 6.4.5.3, except that the decimal-point wide character is used in place of a period, and that if neither an exponent part nor a decimal-point wide character appears in a decimal floating-point number, or if a binary exponent part does not appear in a hexadecimal floating-point number, an exponent part of the appropriate type with value zero is assumed to follow the last digit in the wide string. If the subject sequence begins with a minus sign, the sequence is interpreted as arithmetically negated before rounding and the sign *s* is set to -1 , else *s* is set to 1 .

If the subject sequence has the expected form for a decimal floating-point number, the value resulting from the conversion is correctly rounded and the coefficient *c* and the quantum exponent *q* are determined by the rules in 6.4.5.3 for a decimal floating constant of decimal type.

If the subject sequence has the expected form for a hexadecimal floating-point number, the value resulting from the conversion is correctly rounded provided the subject sequence has at most *M* significant hexadecimal digits, where $M \geq \lceil (P - 1)/4 \rceil + 1$ is implementation-defined, and *P* is the maximum precision of the supported radix-2 floating types and binary non-arithmetic interchange formats.⁴⁰⁶⁾ If all subject sequences of hexadecimal form are correctly rounded, *M* may be regarded as infinite. If the subject sequence has more than *M* significant hexadecimal digits, the implementation may first round to *M* significant hexadecimal digits according to the applicable decimal rounding direction mode, signaling exceptions as though converting from a wider format, then correctly round the result of the shortened hexadecimal input to the result type. The preferred quantum exponent for the result is 0 if the hexadecimal number is exactly represented in the decimal type; the preferred quantum exponent for the result is the least possible if the hexadecimal number is not exactly represented in the decimal type.

A wide character sequence INF or INFINITY is interpreted as an infinity. A wide character sequence NAN or NAN(*d-wchar-sequence_{opt}*), is interpreted as a quiet NaN; the meaning of the *d-wchar* sequence is implementation-defined.⁴⁰⁷⁾ A pointer to the final wide string is stored in the object pointed to by **endptr**, provided that **endptr** is not a null pointer.

In other than the "C" locale, additional locale-specific subject sequence forms may be accepted.

If the subject sequence is empty or does not have the expected form, no conversion is performed; the value of **nptr** is stored in the object pointed to by **endptr**, provided that **endptr** is not a null pointer.

Returns

The **wcstodN** functions return the converted value, if any. If no conversion could be performed, the value of the triple $(+1, 0, 0)$ is returned. If the correct value overflows:

- the value of the macro **ERANGE** is stored in **errno** if the integer expression **math_errhandling** & **MATH_ERRNO** is nonzero;

⁴⁰⁶⁾Non-arithmetic interchange formats are an optional feature in Annex H.

⁴⁰⁷⁾An implementation can use the *d-wchar* sequence to determine extra information to be represented in the NaN's significand.

- the “overflow” floating-point exception is raised if the integer expression **math_errhandling** & **MATH_ERREXCEPT** is nonzero.

If the result underflows (7.12.2), whether **errno** acquires the value **ERANGE** if the integer expression **math_errhandling** & **MATH_ERRNO** is nonzero is implementation-defined; if the integer expression **math_errhandling** & **MATH_ERREXCEPT** is nonzero, whether the “underflow” floating-point exception is raised is implementation-defined.

7.31.4.2.4 The **wcstol**, **wcstoll**, **wcstoul**, and **wcstoull** functions

Synopsis

```
#include <wchar.h>
long int wcstol(const wchar_t * restrict nptr, wchar_t ** restrict endptr,
               int base);
long long int wcstoll(const wchar_t * restrict nptr, wchar_t ** restrict endptr,
                    int base);
unsigned long int wcstoul(const wchar_t * restrict nptr,
                        wchar_t ** restrict endptr, int base);
unsigned long long int wcstoull(const wchar_t * restrict nptr,
                              wchar_t ** restrict endptr, int base);
```

Description

The **wcstol**, **wcstoll**, **wcstoul**, and **wcstoull** functions convert the initial portion of the wide string pointed to by **nptr** to **long int**, **long long int**, **unsigned long int**, and **unsigned long long int** representation, respectively. First, they decompose the input string into three parts: an initial, possibly empty, sequence of white-space wide characters, a subject sequence resembling an integer represented in some radix determined by the value of **base**, and a final wide string of one or more unrecognized wide characters, including the terminating null wide character of the input wide string. Then, they attempt to convert the subject sequence to an integer, and return the result.

If the value of **base** is zero, the expected form of the subject sequence is that of an integer constant as described for the corresponding single-byte characters in 6.4.5.2, optionally preceded by a plus or minus sign, but not including an integer suffix or any optional digit separators (6.4.5.2). If the value of **base** is between 2 and 36 (inclusive), the expected form of the subject sequence is a sequence of letters and digits representing an integer with the radix specified by **base**, optionally preceded by a plus or minus sign, but not including an integer suffix or any optional digit separators. The letters from a (or A) through z (or Z) are ascribed the values 10 through 35; only letters and digits whose ascribed values are less than that of **base** are permitted. If the value of **base** is 2, the characters **0b** or **0B** may optionally precede the sequence of letters and digits, following the sign if present. If the value of **base** is 16, the wide characters **0x** or **0X** may optionally precede the sequence of letters and digits, following the sign if present.

The subject sequence is defined as the longest initial subsequence of the input wide string, starting with the first non-white-space wide character, that is of the expected form. The subject sequence contains no wide characters if the input wide string is empty or consists entirely of white-space wide characters, or if the first non-white-space wide character is other than a sign or a permissible letter or digit.

If the subject sequence has the expected form and the value of **base** is zero, the sequence of wide characters starting with the first digit is interpreted as an integer constant according to the rules of 6.4.5.2. If the subject sequence has the expected form and the value of **base** is between 2 and 36, it is used as the base for conversion, ascribing to each letter its value as previously given. If the subject sequence begins with a minus sign, the resulting value is the negative of the converted value; for functions whose return type is an unsigned integer type this action is performed in the return type. A pointer to the final wide string is stored in the object pointed to by **endptr**, provided that **endptr** is not a null pointer.

In other than the “C” locale, additional locale-specific subject sequence forms may be accepted.

If the subject sequence is empty or does not have the expected form, no conversion is performed; the

value of **nptr** is stored in the object pointed to by **endptr**, provided that **endptr** is not a null pointer.

Returns

The **wcstol**, **wcstoll**, **wcstoul**, and **wcstoull** functions return the converted value, if any. If no conversion could be performed, zero is returned. If the correct value is outside the range of representable values, **LONG_MIN**, **LONG_MAX**, **LLONG_MIN**, **LLONG_MAX**, **ULONG_MAX**, or **ULLONG_MAX** is returned (according to the return type sign of the value, if any), and the value of the macro **ERANGE** is stored in **errno**.

7.31.4.3 Wide string copying functions

7.31.4.3.1 The **wcscpy** function

Synopsis

```
#include <wchar.h>
wchar_t *wcscpy(wchar_t * restrict s1, const wchar_t * restrict s2);
```

Description

The **wcscpy** function copies the wide string pointed to by **s2** (including the terminating null wide character) into the array pointed to by **s1**.

Returns

The **wcscpy** function returns the value of **s1**.

7.31.4.3.2 The **wcsncpy** function

Synopsis

```
#include <wchar.h>
wchar_t *wcsncpy(wchar_t * restrict s1, const wchar_t * restrict s2, size_t n);
```

Description

The **wcsncpy** function copies not more than **n** wide characters (those that follow a null wide character are not copied) from the array pointed to by **s2** to the array pointed to by **s1**.⁴⁰⁸⁾

If the array pointed to by **s2** is a wide string that is shorter than **n** wide characters, null wide characters are appended to the copy in the array pointed to by **s1**, until **n** wide characters in all have been written.

Returns

The **wcsncpy** function returns the value of **s1**.

7.31.4.3.3 The **wmemcpy** function

Synopsis

```
#include <wchar.h>
wchar_t *wmemcpy(wchar_t * restrict s1, const wchar_t * restrict s2, size_t n);
```

Description

The **wmemcpy** function copies **n** wide characters from the object pointed to by **s2** to the object pointed to by **s1**.

Returns

The **wmemcpy** function returns the value of **s1**.

7.31.4.3.4 The **wmemmove** function

⁴⁰⁸⁾Thus, if there is no null wide character in the first **n** wide characters of the array pointed to by **s2**, the result will not be null-terminated.

Synopsis

```
#include <wchar.h>
wchar_t *wmemmove(wchar_t *s1, const wchar_t *s2, size_t n);
```

Description

The **wmemmove** function copies **n** wide characters from the object pointed to by **s2** to the object pointed to by **s1**. Copying takes place as if the **n** wide characters from the object pointed to by **s2** are first copied into a temporary array of **n** wide characters that does not overlap the objects pointed to by **s1** or **s2**, and then the **n** wide characters from the temporary array are copied into the object pointed to by **s1**.

Returns

The **wmemmove** function returns the value of **s1**.

7.31.4.4 Wide string concatenation functions

7.31.4.4.1 The **wcscat** function

Synopsis

```
#include <wchar.h>
wchar_t *wcscat(wchar_t * restrict s1, const wchar_t * restrict s2);
```

Description

The **wcscat** function appends a copy of the wide string pointed to by **s2** (including the terminating null wide character) to the end of the wide string pointed to by **s1**. The initial wide character of **s2** overwrites the null wide character at the end of **s1**.

Returns

The **wcscat** function returns the value of **s1**.

7.31.4.4.2 The **wcsncat** function

Synopsis

```
#include <wchar.h>
wchar_t *wcsncat(wchar_t * restrict s1, const wchar_t * restrict s2, size_t n);
```

Description

The **wcsncat** function appends not more than **n** wide characters (a null wide character and those that follow it are not appended) from the array pointed to by **s2** to the end of the wide string pointed to by **s1**. The initial wide character of **s2** overwrites the null wide character at the end of **s1**. A terminating null wide character is always appended to the result.⁴⁰⁹⁾

Returns

The **wcsncat** function returns the value of **s1**.

7.31.4.5 Wide string comparison functions

7.31.4.5.1 General

Unless explicitly stated otherwise, the functions described in this subclause order two wide characters the same way as two integers of the underlying integer type designated by **wchar_t**.

7.31.4.5.2 The **wcsncmp** function

Synopsis

```
#include <wchar.h>
int wcsncmp(const wchar_t *s1, const wchar_t *s2);
```

⁴⁰⁹⁾Thus, the maximum number of wide characters that can end up in the array pointed to by **s1** is **wcslen(s1)+n+1**.

Description

The **wcscmp** function compares the wide string pointed to by **s1** to the wide string pointed to by **s2**.

Returns

The **wcscmp** function returns an integer greater than, equal to, or less than zero, accordingly as the wide string pointed to by **s1** is greater than, equal to, or less than the wide string pointed to by **s2**.

7.31.4.5.3 The **wcscoll** function

Synopsis

```
#include <wchar.h>
int wcscoll(const wchar_t *s1, const wchar_t *s2);
```

Description

The **wcscoll** function compares the wide string pointed to by **s1** to the wide string pointed to by **s2**, both interpreted as appropriate to the **LC_COLLATE** category of the current locale.

Returns

The **wcscoll** function returns an integer greater than, equal to, or less than zero, accordingly as the wide string pointed to by **s1** is greater than, equal to, or less than the wide string pointed to by **s2** when both are interpreted as appropriate to the current locale.

7.31.4.5.4 The **wcsncmp** function

Synopsis

```
#include <wchar.h>
int wcsncmp(const wchar_t *s1, const wchar_t *s2, size_t n);
```

Description

The **wcsncmp** function compares not more than **n** wide characters (those that follow a null wide character are not compared) from the array pointed to by **s1** to the array pointed to by **s2**.

Returns

The **wcsncmp** function returns an integer greater than, equal to, or less than zero, accordingly as the possibly null-terminated array pointed to by **s1** is greater than, equal to, or less than the possibly null-terminated array pointed to by **s2**.

7.31.4.5.5 The **wcsxfrm** function

Synopsis

```
#include <wchar.h>
size_t wcsxfrm(wchar_t * restrict s1, const wchar_t * restrict s2, size_t n);
```

Description

The **wcsxfrm** function transforms the wide string pointed to by **s2** and places the resulting wide string into the array pointed to by **s1**. The transformation is such that if the **wcscmp** function is applied to two transformed wide strings, it returns a value greater than, equal to, or less than zero, corresponding to the result of the **wcscoll** function applied to the same two original wide strings. No more than **n** wide characters are placed into the resulting array pointed to by **s1**, including the terminating null wide character. If **n** is zero, **s1** is permitted to be a null pointer.

Returns

The **wcsxfrm** function returns the length of the transformed wide string (not including the terminating null wide character). If the value returned is **n** or greater, the members of the array pointed to by **s1** have an indeterminate representation.

EXAMPLE The value of the following expression is the length of the array needed to hold the transformation of the wide string pointed to by **s**:


```
1 + wcsxfrm(nullptr, s, 0)
```

7.31.4.5.6 The wmemcmp function

Synopsis

```
#include <wchar.h>
int wmemcmp(const wchar_t *s1, const wchar_t *s2, size_t n);
```

Description

The **wmemcmp** function compares the first **n** wide characters of the object pointed to by **s1** to the first **n** wide characters of the object pointed to by **s2**.

Returns

The **wmemcmp** function returns an integer greater than, equal to, or less than zero, accordingly as the object pointed to by **s1** is greater than, equal to, or less than the object pointed to by **s2**.

7.31.4.6 Wide string search functions

7.31.4.6.1 Introduction

The stateless search functions in this section (**wcschr**, **wcspbrk**, **wcsrchr**, **wmemchr**, **wcsstr**) are *generic functions*. These functions are generic in the qualification of the array to be searched and will return a result pointer to an element with the same qualification as the passed array. If the array to be searched is **const**-qualified, the result pointer will be to a **const**-qualified element. If the array to be searched is not **const**-qualified,⁴¹⁰⁾ the result pointer will be to an unqualified element.

The external declarations of these generic functions have a concrete function type that returns a pointer to an unqualified element of type **wchar_t** (indicated by *QWchar_t*), and accepts a pointer to a **const**-qualified array of the same type to search. This signature supports all correct uses. If a macro definition of any of these generic functions is suppressed to access an actual function, the external declaration with this concrete type is visible.⁴¹¹⁾

The **volatile** and **restrict** qualifiers are not accepted on the elements of the array to search.

7.31.4.6.2 The wcschr generic function

Synopsis

```
#include <wchar.h>
QWchar_t *wcschr(QWchar_t *s, wchar_t c);
```

Description

The **wcschr** generic function locates the first occurrence of **c** in the wide string pointed to by **s**. The terminating null wide character is considered to be part of the wide string.

Returns

The **wcschr** generic function returns a pointer to the located wide character, or a null pointer if the wide character does not occur in the wide string.

7.31.4.6.3 The wcscspn function

Synopsis

```
#include <wchar.h>
size_t wcscspn(const wchar_t *s1, const wchar_t *s2);
```

⁴¹⁰⁾The null pointer constant is not a pointer to a **const**-qualified type, and therefore the result expression has the type of a pointer to an unqualified element; however, evaluating such a call is undefined.

⁴¹¹⁾This is an obsolescent feature.

Description

The **wcscspn** function computes the length of the maximum initial segment of the wide string pointed to by **s1** which consists entirely of wide characters *not* from the wide string pointed to by **s2**.

Returns

The **wcscspn** function returns the length of the segment.

7.31.4.6.4 The **wcspbrk** generic function

Synopsis

```
#include <wchar.h>
QWchar_t *wcspbrk(QWchar_t *s1, const wchar_t *s2);
```

Description

The **wcspbrk** generic function locates the first occurrence in the wide string pointed to by **s1** of any wide character from the wide string pointed to by **s2**.

Returns

The **wcspbrk** generic function returns a pointer to the wide character in **s1**, or a null pointer if no wide character from **s2** occurs in **s1**.

7.31.4.6.5 The **wcsrchr** generic function

Synopsis

```
#include <wchar.h>
QWchar_t *wcsrchr(QWchar_t *s, wchar_t c);
```

Description

The **wcsrchr** generic function locates the last occurrence of **c** in the wide string pointed to by **s**. The terminating null wide character is considered to be part of the wide string.

Returns

The **wcsrchr** generic function returns a pointer to the wide character, or a null pointer if **c** does not occur in the wide string.

7.31.4.6.6 The **wcsspn** function

Synopsis

```
#include <wchar.h>
size_t wcsspn(const wchar_t *s1, const wchar_t *s2);
```

Description

The **wcsspn** function computes the length of the maximum initial segment of the wide string pointed to by **s1** which consists entirely of wide characters from the wide string pointed to by **s2**.

Returns

The **wcsspn** function returns the length of the segment.

7.31.4.6.7 The **wcsstr** generic function

Synopsis

```
#include <wchar.h>
QWchar_t *wcsstr(QWchar_t *s1, const wchar_t *s2);
```

Description

The **wcsstr** generic function locates the first occurrence in the wide string pointed to by **s1** of the sequence of wide characters (excluding the terminating null wide character) in the wide string pointed to by **s2**.

Returns

The **wcsstr** generic function returns a pointer to the located wide string, or a null pointer if the wide string is not found. If **s2** points to a wide string with zero length, the function returns **s1**.

7.31.4.6.8 The wcstok function

Synopsis

```
#include <wchar.h>
wchar_t *wcstok(wchar_t * restrict s1, const wchar_t * restrict s2,
                wchar_t ** restrict ptr);
```

Description

A sequence of calls to the **wcstok** function breaks the wide string pointed to by **s1** into a sequence of tokens, each of which is delimited by a wide character from the wide string pointed to by **s2**. The third argument points to a caller-provided **wchar_t** pointer into which the **wcstok** function stores information necessary for it to continue scanning the same wide string.

The first call in a sequence has a non-null first argument and stores an initial value in the object pointed to by **ptr**. Subsequent calls in the sequence have a null first argument and the object pointed to by **ptr** is required to have the value stored by the previous call in the sequence, which is then updated. The separator wide string pointed to by **s2** may be different from call to call.

The first call in the sequence searches the wide string pointed to by **s1** for the first wide character that is *not* contained in the current separator wide string pointed to by **s2**. If no such wide character is found, then there are no tokens in the wide string pointed to by **s1** and the **wcstok** function returns a null pointer. If such a wide character is found, it is the start of the first token.

The **wcstok** function then searches from there for a wide character that *is* contained in the current separator wide string. If no such wide character is found, the current token extends to the end of the wide string pointed to by **s1**, and subsequent searches in the same wide string for a token return a null pointer. If such a wide character is found, it is overwritten by a null wide character, which terminates the current token.

In all cases, the **wcstok** function stores sufficient information in the pointer pointed to by **ptr** so that subsequent calls, with a null pointer for **s1** and the unmodified pointer value for **ptr**, shall start searching just past the element overwritten by a null wide character (if any).

Returns

The **wcstok** function returns a pointer to the first wide character of a token, or a null pointer if there is no token.

EXAMPLE

```
#include <wchar.h>
static wchar_t str1[] = L"?a???b,,#c";
static wchar_t str2[] = L"\t \t";
wchar_t *t, *ptr1, *ptr2;

t = wcstok(str1, L"?", &ptr1);    // t points to the token L"a"
t = wcstok(NULLptr, L",", &ptr1); // t points to the token L"???b"
t = wcstok(str2, L" \t", &ptr2);  // t is a null pointer
t = wcstok(NULLptr, L"#", &ptr1); // t points to the token L"c"
t = wcstok(NULLptr, L"?", &ptr1); // t is a null pointer
```

7.31.4.6.9 The wmemchr generic function

Synopsis

```
#include <wchar.h>
QWchar_t *wmemchr(QWchar_t *s, wchar_t c, size_t n);
```

Description

The **wmemchr** generic function locates the first occurrence of **c** in the initial **n** wide characters of the object pointed to by **s**.

Returns

The **wmemchr** generic function returns a pointer to the located wide character, or a null pointer if the wide character does not occur in the object.

7.31.4.7 Miscellaneous functions

7.31.4.7.1 The **wcslen** function

Synopsis

```
#include <wchar.h>
size_t wcslen(const wchar_t *s);
```

Description

The **wcslen** function computes the length of the wide string pointed to by **s**.

Returns

The **wcslen** function returns the number of wide characters that precede the terminating null wide character.

7.31.4.7.2 The **wmemset** function

Synopsis

```
#include <wchar.h>
wchar_t *wmemset(wchar_t *s, wchar_t c, size_t n);
```

Description

The **wmemset** function copies the value of **c** into each of the first **n** wide characters of the object pointed to by **s**.

Returns

The **wmemset** function returns the value of **s**.

7.31.5 Wide character time conversion functions

7.31.5.1 The **wcsftime** function

Synopsis

```
#include <time.h>
#include <wchar.h>
size_t wcsftime(wchar_t * restrict s, size_t maxsize,
                const wchar_t * restrict format, const struct tm * restrict timeptr);
```

Description

The **wcsftime** function is equivalent to the **strftime** function, except that:

- The argument **s** points to the initial element of an array of wide characters into which the generated output is to be placed.
- The argument **maxsize** indicates the limiting number of wide characters.

- The argument **format** is a wide string and the conversion specifiers are replaced by corresponding sequences of wide characters.
- The return value indicates the number of wide characters.

Returns

If the total number of resulting wide characters including the terminating null wide character is not more than **maxsize**, the **wcsftime** function returns the number of wide characters placed into the array pointed to by **s** not including the terminating null wide character. Otherwise, zero is returned and the members of the array have an indeterminate representation.

7.31.6 Extended multibyte/wide character conversion utilities

7.31.6.1 General

The header `<wchar.h>` declares an extended set of functions useful for conversion between multibyte characters and wide characters.

Most of the following functions — those that are listed as “restartable”, 7.31.6.4 and 7.31.6.5 — take as a last argument a pointer to an object of type **mbstate_t** that is used to describe the current *conversion state* from a particular multibyte character sequence to a wide character sequence (or the reverse) under the rules of a particular setting for the **LC_CTYPE** category of the current locale.

The initial conversion state corresponds, for a conversion in either direction, to the beginning of a new multibyte character in the initial shift state. A zero-valued **mbstate_t** object is (at least) one way to describe an initial conversion state. A zero-valued **mbstate_t** object can be used to initiate conversion involving any multibyte character sequence, in any **LC_CTYPE** category setting. If an **mbstate_t** object has been altered by any of the functions described in this subclause, and is then used with a different multibyte character sequence, or in the other conversion direction, or with a different **LC_CTYPE** category setting than on earlier function calls, the behavior is undefined.⁴¹²⁾

On entry, each function takes the described conversion state (either internal or pointed to by an argument) as current. The conversion state described by the referenced object is altered as needed to track the shift state, and the position within a multibyte character, for the associated multibyte character sequence.

7.31.6.2 Single-byte/wide character conversion functions

7.31.6.2.1 The **btowc** function

Synopsis

```
#include <wchar.h>
wint_t btowc(int c);
```

Description

The **btowc** function determines whether **c** constitutes a valid single-byte character in the initial shift state.

Returns

The **btowc** function returns **WEOF** if **c** has the value **EEOF** or if **(unsigned char)c** does not constitute a valid single-byte character in the initial shift state. Otherwise, it returns the wide character representation of that character.

7.31.6.2.2 The **wctob** function

Synopsis

```
#include <wchar.h>
int wctob(wint_t c);
```

⁴¹²⁾Thus, a particular **mbstate_t** object can be used, for example, with both the **mbrtowc** and **mbsrtowcs** functions as long as they are used to step sequentially through the same multibyte character string.

Description

The **wctob** function determines whether **c** corresponds to a member of the extended character set whose multibyte character representation is a single byte when in the initial shift state.

Returns

The **wctob** function returns **EOF** if **c** does not correspond to a multibyte character with length one in the initial shift state. Otherwise, it returns the single-byte representation of that character as an **unsigned char** converted to an **int**.

7.31.6.3 Conversion state functions

7.31.6.3.1 The **mbsinit** function

Synopsis

```
#include <wchar.h>
int mbsinit(const mbstate_t *ps);
```

Description

If **ps** is not a null pointer, the **mbsinit** function determines whether the referenced **mbstate_t** object describes an initial conversion state.

Returns

The **mbsinit** function returns nonzero if **ps** is a null pointer or if the referenced object describes an initial conversion state; otherwise, it returns zero.

7.31.6.4 Restartable multibyte/wide character conversion functions

7.31.6.4.1 General

These functions differ from the corresponding multibyte character functions of 7.24.8 (**mblen**, **mbtowc**, and **wctomb**) in that they have an extra parameter, **ps**, of type pointer to **mbstate_t** that points to an object that can completely describe the current conversion state of the associated multibyte character sequence. If **ps** is a null pointer, each function uses its own internal **mbstate_t** object instead, which is initialized prior to the first call to the function to the initial conversion state; the functions are not required to avoid data races with other calls to the same function in this case. It is implementation-defined whether the internal **mbstate_t** object has thread storage duration; if it has thread storage duration, it is initialized to the initial conversion state prior to the first call to the function on the new thread. The implementation behaves as if no library function calls these functions with a null pointer for **ps**.

Also unlike their corresponding functions, the return value does not represent whether the encoding is state-dependent.

7.31.6.4.2 The **mbrlen** function

Synopsis

```
#include <wchar.h>
size_t mbrlen(const char * restrict s, size_t n, mbstate_t * restrict ps);
```

Description

The **mbrlen** function is equivalent to the call:

```
mbrtowc(nullptr, s, n, ps != nullptr ? ps: &internal)
```

where **internal** is the **mbstate_t** object for the **mbrlen** function, except that the expression designated by **ps** is evaluated only once.

Returns

The **mbrlen** function returns a value between zero and **n**, inclusive, **(size_t)(-2)**, or **(size_t)(-1)**.

Forward references: the `mbrtowc` function (7.31.6.4.3).

7.31.6.4.3 The `mbrtowc` function

Synopsis

```
#include <wchar.h>
size_t mbrtowc(wchar_t * restrict pwc, const char * restrict s, size_t n,
               mbstate_t * restrict ps);
```

Description

If `s` is a null pointer, the `mbrtowc` function is equivalent to the call:

```
mbrtowc(nullptr, "", 1, ps)
```

In this case, the values of the parameters `pwc` and `n` are ignored.

If `s` is not a null pointer, the `mbrtowc` function inspects at most `n` bytes beginning with the byte pointed to by `s` to determine the number of bytes needed to complete the next multibyte character (including any shift sequences). If the function determines that the next multibyte character is complete and valid, it determines the value of the corresponding wide character and then, if `pwc` is not a null pointer, stores that value in the object pointed to by `pwc`. If the corresponding wide character is the null wide character, the resulting state described is the initial conversion state.

Returns

The `mbrtowc` function returns the first of the following that applies (given the current conversion state):

- 0** if the next `n` or fewer bytes complete the multibyte character that corresponds to the null wide character (which is the value stored).
- between 1 and n inclusive* if the next `n` or fewer bytes complete a valid multibyte character (which is the value stored); the value returned is the number of bytes that complete the multibyte character.
- (size_t)(-2)** if the next `n` bytes contribute to an incomplete (but potentially valid) multibyte character, and all `n` bytes have been processed (no value is stored).⁴¹³⁾
- (size_t)(-1)** if an encoding error occurs, in which case the next `n` or fewer bytes do not contribute to a complete and valid multibyte character (no value is stored); the value of the macro `EILSEQ` is stored in `errno`, and the conversion state is unspecified.

7.31.6.4.4 The `wcrtomb` function

Synopsis

```
#include <wchar.h>
size_t wcrtomb(char * restrict s, wchar_t wc, mbstate_t * restrict ps);
```

Description

If `s` is a null pointer, the `wcrtomb` function is equivalent to the call

```
wcrtomb(buf, L'\0', ps)
```

where `buf` is an internal buffer.

If `s` is not a null pointer, the `wcrtomb` function determines the number of bytes needed to represent the multibyte character that corresponds to the wide character given by `wc` (including any shift

⁴¹³⁾When `n` has at least the value of the `MB_CUR_MAX` macro, this case can only occur if `s` points at a sequence of redundant shift sequences (for implementations with state-dependent encodings).

sequences), and stores the multibyte character representation in the array whose first element is pointed to by **s**. At most **MB_CUR_MAX** bytes are stored. If **wc** is a null wide character, a null byte is stored, preceded by any shift sequence needed to restore the initial shift state; the resulting state described is the initial conversion state.

Returns

The **wcrtomb** function returns the number of bytes stored in the array object (including any shift sequences). When **wc** is not a valid wide character, an encoding error occurs: the function stores the value of the macro **EILSEQ** in **errno** and returns **(size_t)(-1)**; the conversion state is unspecified.

7.31.6.5 Restartable multibyte/wide string conversion functions

7.31.6.5.1 General

These functions differ from the corresponding multibyte string functions of 7.24.9 (**mbstowcs** and **wcstombs**) in that they have an extra parameter, **ps**, of type pointer to **mbstate_t** that points to an object that can completely describe the current conversion state of the associated multibyte character sequence. If **ps** is a null pointer, each function uses its own internal **mbstate_t** object instead, which is initialized prior to the first call to the function to the initial conversion state; the functions are not required to avoid data races with other calls to the same function in this case. It is implementation-defined whether the internal **mbstate_t** object has thread storage duration; if it has thread storage duration, it is initialized to the initial conversion state prior to the first call to the function on the new thread. The implementation behaves as if no library function calls these functions with a null pointer for **ps**.

Also unlike their corresponding functions, the conversion source parameter, **src**, has a pointer-to-pointer type. When the function is storing the results of conversions (that is, when **dst** is not a null pointer), the pointer object pointed to by this parameter is updated to reflect the amount of the source processed by that invocation.

7.31.6.5.2 The **mbsrtowcs** function

Synopsis

```
#include <wchar.h>
size_t mbsrtowcs(wchar_t * restrict dst, const char ** restrict src, size_t len,
mbstate_t * restrict ps);
```

Description

The **mbsrtowcs** function converts a sequence of multibyte characters that begins in the conversion state described by the object pointed to by **ps**, from the array indirectly pointed to by **src** into a sequence of corresponding wide characters. If **dst** is not a null pointer, the converted characters are stored into the array pointed to by **dst**. Conversion continues up to and including a terminating null character, which is also stored. Conversion stops earlier in two cases: when a sequence of bytes is encountered that does not form a valid multibyte character, or (if **dst** is not a null pointer) when **len** wide characters have been stored into the array pointed to by **dst**.⁴¹⁴⁾ Each conversion takes place as if by a call to the **mbrtowc** function.

If **dst** is not a null pointer, the pointer object pointed to by **src** is assigned either a null pointer (if conversion stopped due to reaching a terminating null character) or the address just past the last multibyte character converted (if any). If conversion stopped due to reaching a terminating null character and if **dst** is not a null pointer, the resulting state described is the initial conversion state.

Returns

If the input conversion encounters a sequence of bytes that do not form a valid multibyte character, an encoding error occurs: the **mbsrtowcs** function stores the value of the macro **EILSEQ** in **errno** and returns **(size_t)(-1)**; the conversion state is unspecified. Otherwise, it returns the number of multibyte characters successfully converted, not including the terminating null character (if any).

7.31.6.5.3 The **wcsrtombs** function

⁴¹⁴⁾Thus, the value of **len** is ignored if **dst** is a null pointer.

Synopsis

```
#include <wchar.h>
size_t wcsrtombs(char * restrict dst, const wchar_t ** restrict src, size_t len,
                mbstate_t * restrict ps);
```

Description

The **wcsrtombs** function converts a sequence of wide characters from the array indirectly pointed to by **src** into a sequence of corresponding multibyte characters that begins in the conversion state described by the object pointed to by **ps**. If **dst** is not a null pointer, the converted characters are then stored into the array pointed to by **dst**. Conversion continues up to and including a terminating null wide character, which is also stored. Conversion stops earlier in two cases: when a wide character is reached that does not correspond to a valid multibyte character, or (if **dst** is not a null pointer) when the next multibyte character would exceed the limit of **len** total bytes to be stored into the array pointed to by **dst**. Each conversion takes place as if by a call to the **wcrtomb** function.⁴¹⁵⁾

If **dst** is not a null pointer, the pointer object pointed to by **src** is assigned either a null pointer (if conversion stopped due to reaching a terminating null wide character) or the address just past the last wide character converted (if any). If conversion stopped due to reaching a terminating null wide character, the resulting state described is the initial conversion state.

Returns

If conversion stops because a wide character is reached that does not correspond to a valid multibyte character, an encoding error occurs: the **wcsrtombs** function stores the value of the macro **EILSEQ** in **errno** and returns **(size_t)(-1)**; the conversion state is unspecified. Otherwise, it returns the number of bytes in the resulting multibyte character sequence, not including the terminating null character (if any).

⁴¹⁵⁾ If conversion stops because a terminating null wide character has been reached, the bytes stored include those necessary to reach the initial shift state immediately before the null byte.

7.32 Wide character classification and mapping utilities <wctype.h>

7.32.1 Introduction

The header <wctype.h> defines one macro, and declares three data types and many functions.⁴¹⁶⁾
The types declared are `wint_t` described in 7.31.1;

wctrans_t

which is a scalar type that can hold values which represent locale-specific character mappings; and

wctype_t

which is a scalar type that can hold values which represent locale-specific character classifications.
The macro defined is `WEOF` (described in 7.31.1).
The functions declared are grouped as follows:

- Functions that provide wide character classification;
- Extensible functions that provide wide character classification;
- Functions that provide wide character case mapping;
- Extensible functions that provide wide character mapping.

For all functions described in this subclause that accept an argument of type `wint_t`, the value shall be representable as a `wchar_t` or shall equal the value of the macro `WEOF`. If this argument has any other value, the behavior is undefined.
The behavior of these functions is affected by the `LC_CTYPE` category of the current locale.

7.32.2 Wide character classification utilities

7.32.2.1 General

The header <wctype.h> declares several functions useful for classifying wide characters.
The term *printing wide character* refers to a member of a locale-specific set of wide characters, each of which occupies at least one printing position on a display device. The term *control wide character* refers to a member of a locale-specific set of wide characters that are not printing wide characters.

7.32.2.2 Wide character classification functions

7.32.2.2.1 General

The functions in this subclause return nonzero (true) if and only if the value of the argument `wc` conforms to that in the description of the function.
Each of the following functions returns true for each wide character that corresponds (as if by a call to the `wctob` function) to a single-byte character for which the corresponding character classification function from 7.4.2 returns true, except that the `iswgraph` and `iswpunct` functions may differ with respect to wide characters other than `L' '` that are both printing and white-space wide characters.⁴¹⁷⁾
Forward references: the `wctob` function (7.31.6.2.2).

⁴¹⁶⁾See “future library directions” (7.33.22).
⁴¹⁷⁾For example, if the expression `isalpha(wctob(wc))` evaluates to true, then the call `iswalphabet(wc)` also returns true. But, if the expression `isgraph(wctob(wc))` evaluates to true (which cannot occur for `wc == L' '` of course), then either `iswgraph(wc)` or `iswprint(wc)&& iswspace(wc)` is true, but not both.

7.32.2.2.2 The `iswalnum` function

Synopsis

```
#include <wctype.h>
int iswalnum(wint_t wc);
```

Description

The `iswalnum` function tests for any wide character for which `iswalpha` or `iswdigit` is true.

7.32.2.2.3 The `iswalpha` function

Synopsis

```
#include <wctype.h>
int iswalpha(wint_t wc);
```

Description

The `iswalpha` function tests for any wide character for which `iswupper` or `iswlower` is true, or any wide character that is one of a locale-specific set of alphabetic wide characters for which none of `iswcntrl`, `iswdigit`, `iswpunct`, or `iswspace` is true.⁴¹⁸⁾

7.32.2.2.4 The `iswblank` function

Synopsis

```
#include <wctype.h>
int iswblank(wint_t wc);
```

Description

The `iswblank` function tests for any wide character that is a standard blank wide character or is one of a locale-specific set of wide characters for which `iswspace` is true and that is used to separate words within a line of text. The standard blank wide characters are the following: space (`L' '`), and horizontal tab (`L'\t'`). In the "C" locale, `iswblank` returns true only for the standard blank characters.

7.32.2.2.5 The `iswcntrl` function

Synopsis

```
#include <wctype.h>
int iswcntrl(wint_t wc);
```

Description

The `iswcntrl` function tests for any control wide character.

7.32.2.2.6 The `iswdigit` function

Synopsis

```
#include <wctype.h>
int iswdigit(wint_t wc);
```

Description

The `iswdigit` function tests for any wide character that corresponds to a decimal-digit character (as defined in 5.3.1).

⁴¹⁸⁾The functions `iswlower` and `iswupper` test true or false separately for each of these additional wide characters; all four combinations are possible.

7.32.2.2.7 The **iswgraph** function

Synopsis

```
#include <wctype.h>
int iswgraph(wint_t wc);
```

Description

The **iswgraph** function tests for any wide character for which **iswprint** is true and **iswspace** is false.⁴¹⁹⁾

7.32.2.2.8 The **iswlower** function

Synopsis

```
#include <wctype.h>
int iswlower(wint_t wc);
```

Description

The **iswlower** function tests for any wide character that corresponds to a lowercase letter or is one of a locale-specific set of wide characters for which none of **iswcntrl**, **iswdigit**, **iswpunct**, or **iswspace** is true.

7.32.2.2.9 The **iswprint** function

Synopsis

```
#include <wctype.h>
int iswprint(wint_t wc);
```

Description

The **iswprint** function tests for any printing wide character.

7.32.2.2.10 The **iswpunct** function

Synopsis

```
#include <wctype.h>
int iswpunct(wint_t wc);
```

Description

The **iswpunct** function tests for any printing wide character that is one of a locale-specific set of punctuation wide characters for which neither **iswspace** nor **iswalnum** is true.⁴¹⁹⁾

7.32.2.2.11 The **iswspace** function

Synopsis

```
#include <wctype.h>
int iswspace(wint_t wc);
```

Description

The **iswspace** function tests for any wide character that corresponds to a locale-specific set of white-space wide characters for which none of **iswalnum**, **iswgraph**, or **iswpunct** is true.

7.32.2.2.12 The **iswupper** function

Synopsis

```
#include <wctype.h>
int iswupper(wint_t wc);
```

⁴¹⁹⁾The behavior of the **iswgraph** and **iswpunct** functions can differ from their corresponding functions in 7.4.2 with respect to printing, white-space, single-byte execution characters other than ' '.

Description

The **iswupper** function tests for any wide character that corresponds to an uppercase letter or is one of a locale-specific set of wide characters for which none of **iswcntrl**, **iswdigit**, **iswpunct**, or **iswspace** is true.

7.32.2.2.13 The iswxdigit function

Synopsis

```
#include <wctype.h>
int iswxdigit(wint_t wc);
```

Description

The **iswxdigit** function tests for any wide character that corresponds to a hexadecimal-digit character (as defined in 6.4.5.2).

7.32.2.3 Extensible wide character classification functions

7.32.2.3.1 General

The functions **wctype** and **iswctype** provide extensible wide character classification as well as testing equivalent to that performed by the functions described in the previous subclause (7.32.2.2).

7.32.2.3.2 The iswctype function

Synopsis

```
#include <wctype.h>
int iswctype(wint_t wc, wctype_t desc);
```

Description

The **iswctype** function determines whether the wide character **wc** has the property described by **desc**. The current setting of the **LC_CTYPE** category shall be the same as during the call to **wctype** that returned the value **desc**.

Each of the following expressions has a truth-value equivalent to the call to the wide character classification function (7.32.2.2) in the comment that follows the expression:

```
iswctype(wc, wctype("alnum"))    // iswalnum(wc)
iswctype(wc, wctype("alpha"))    // iswalpha(wc)
iswctype(wc, wctype("blank"))    // iswblank(wc)
iswctype(wc, wctype("cntrl"))    // iswcntrl(wc)
iswctype(wc, wctype("digit"))    // iswdigit(wc)
iswctype(wc, wctype("graph"))    // iswgraph(wc)
iswctype(wc, wctype("lower"))    // iswlower(wc)
iswctype(wc, wctype("print"))    // iswprint(wc)
iswctype(wc, wctype("punct"))    // iswpunct(wc)
iswctype(wc, wctype("space"))    // iswspace(wc)
iswctype(wc, wctype("upper"))    // iswupper(wc)
iswctype(wc, wctype("xdigit"))   // iswxdigit(wc)
```

Returns

The **iswctype** function returns nonzero (true) if and only if the value of the wide character **wc** has the property described by **desc**. If **desc** is zero, the **iswctype** function returns zero (false).

Forward references: the **wctype** function (7.32.2.3.3).

7.32.2.3.3 The wctype function

Synopsis

```
#include <wctype.h>
wctype_t wctype(const char *property);
```

Description

The **wctype** function constructs a value with type **wctype_t** that describes a class of wide characters identified by the string argument **property**.

The strings listed in the description of the **iswctype** function shall be valid in all locales as **property** arguments to the **wctype** function.

Returns

If **property** identifies a valid class of wide characters according to the **LC_CTYPE** category of the current locale, the **wctype** function returns a nonzero value that is valid as the second argument to the **iswctype** function; otherwise, it returns zero.

7.32.3 Wide character case mapping utilities

7.32.3.1 Wide character case mapping functions

7.32.3.1.1 The **towlower** function

Synopsis

```
#include <wctype.h>
wint_t tolower(wint_t wc);
```

Description

The **towlower** function converts an uppercase letter to a corresponding lowercase letter.

Returns

If the argument is a wide character for which **iswupper** is true and there are one or more corresponding wide characters, as specified by the current locale, for which **iswlower** is true, the **towlower** function returns one of the corresponding wide characters (always the same one for any given locale); otherwise, the argument is returned unchanged.

7.32.3.1.2 The **towupper** function

Synopsis

```
#include <wctype.h>
wint_t towupper(wint_t wc);
```

Description

The **towupper** function converts a lowercase letter to a corresponding uppercase letter.

Returns

If the argument is a wide character for which **iswlower** is true and there are one or more corresponding wide characters, as specified by the current locale, for which **iswupper** is true, the **towupper** function returns one of the corresponding wide characters (always the same one for any given locale); otherwise, the argument is returned unchanged.

7.32.3.2 Extensible wide character case mapping functions

7.32.3.2.1 General

The functions **wctrans** and **towctrans** provide extensible wide character mapping as well as case mapping equivalent to that performed by the functions described in the previous subclause (7.32.3.1).

7.32.3.2.2 The **towctrans** function

Synopsis

```
#include <wctype.h>
wint_t towctrans(wint_t wc, wctrans_t desc);
```

Description

The **towctrans** function maps the wide character **wc** using the mapping described by **desc**. The current setting of the **LC_CTYPE** category shall be the same as during the call to **wctrans** that returned the value **desc**.

Each of the following expressions behaves the same as the call to the wide character case mapping function (7.32.3.1) in the comment that follows the expression:

```
towctrans(wc, wctrans("tolower"))    // tolower(wc)
towctrans(wc, wctrans("toupper"))    // toupper(wc)
```

Returns

The **towctrans** function returns the mapped value of **wc** using the mapping described by **desc**. If **desc** is zero, the **towctrans** function returns the value of **wc**.

7.32.3.2.3 The wctrans function

Synopsis

```
#include <wctype.h>
wctrans_t wctrans(const char *property);
```

Description

The **wctrans** function constructs a value with type **wctrans_t** that describes a mapping between wide characters identified by the string argument **property**.

The strings listed in the description of the **towctrans** function shall be valid in all locales as **property** arguments to the **wctrans** function.

Returns

If **property** identifies a valid mapping of wide characters according to the **LC_CTYPE** category of the current locale, the **wctrans** function returns a nonzero value that is valid as the second argument to the **towctrans** function; otherwise, it returns zero.

7.33 Future library directions

7.33.1 General

Although grouped under individual headers, all the external names identified as reserved identifiers or potentially reserved identifiers in this subclause remain so regardless of which headers are included in the program.

7.33.2 Complex arithmetic <complex.h>

The function names

cacospi	cexp10m1	clog10	crootn
casinpi	cexp10	clog1p	crsqrt
catanpi	cexp2m1	clog2p1	csinpi
ccompoundn	cexp2	clog2	ctanpi
ccospi	cexpm1	clogp1	ctgamma
cerfc	clgamma	cpown	
cerf	clog10p1	cpowr	

and the same names suffixed with **f** or **l** are potentially reserved identifiers and may be added to the declarations in the <complex.h> header.

7.33.3 Character handling <ctype.h>

Function names that begin with either **is** or **to**, and a lowercase letter are potentially reserved identifiers and may be added to the declarations in the <ctype.h> header.

7.33.4 Errors <errno.h>

Macros that begin with **E** and a digit or **E** and an uppercase letter may be added to the macros defined in the <errno.h> header by a future edition of this document or by an implementation.

7.33.5 Floating-point environment <fenv.h>

Macros that begin with **FE_** and an uppercase letter may be added to the macros defined in the <fenv.h> header by a future edition of this document or by an implementation.

7.33.6 Characteristics of floating types <float.h>

Macros that begin with **DBL_**, **DEC32_**, **DEC64_**, **DEC128_**, **DEC_**, **FLT_**, or **LDBL_** and an uppercase letter are potentially reserved identifiers and may be added to the macros defined in the <float.h> header.

Use of the **DECIMAL_DIG** macro is an obsolescent feature. A similar type-specific macro, such as **LDBL_DECIMAL_DIG**, can be used instead.

The use of **FLT_HAS_SUBNORM**, **DBL_HAS_SUBNORM**, and **LDBL_HAS_SUBNORM** macros is an obsolescent feature.

7.33.7 Format conversion of integer types <inttypes.h>

Macros that begin with either **PRI** or **SCN**, and either a lowercase letter, **B**, or **X** are potentially reserved identifiers and may be added to the macros defined in the <inttypes.h> header.

Function names that begin with **str**, or **wcs** and a lowercase letter are potentially reserved identifiers may be added to the declarations in the <inttypes.h> header.

7.33.8 Localization <locale.h>

Macros that begin with **LC_** and an uppercase letter may be added to the macros defined in the <locale.h> header by a future edition of this document or by an implementation.

7.33.9 Mathematics <math.h>

Macros that begin with **FP_** and an uppercase letter may be added to the macros defined in the <math.h> header by a future edition of this document or by an implementation.

Macros that begin with **MATH_** and an uppercase letter are potentially reserved identifiers and may be added to the macros in the `<math.h>` header.

Function names that begin with **is** and a lowercase letter are potentially reserved identifiers and may be added to the declarations in the `<math.h>` header.

Function names that begin with **cr_** are potentially reserved identifiers and may be added to the `<math.h>` header. The **cr_** prefix is intended to indicate a correctly rounded version of the function.

Use of the macros **INFINITY**, **DEC_INFINITY**, **NAN**, and **DEC_NAN** in `<math.h>` is an obsolescent feature. Instead, use the same macros in `<float.h>`.

7.33.10 Signal handling `<signal.h>`

Macros that begin with either **SIG** and an uppercase letter or **SIG_** and an uppercase letter may be added to the macros defined in the `<signal.h>` header by a future edition of this document or by an implementation.

7.33.11 Atomics `<stdatomic.h>`

Macros that begin with **ATOMIC_** and an uppercase letter are potentially reserved identifiers and may be added to the macros defined in the `<stdatomic.h>` header. Typedef names that begin with either **atomic_** or **memory_**, and a lowercase letter are potentially reserved identifiers and may be added to the declarations in the `<stdatomic.h>` header. Enumeration constants that begin with **memory_order_** and a lowercase letter are potentially reserved identifiers and may be added to the definition of the **memory_order** type in the `<stdatomic.h>` header. Function names that begin with **atomic_** and a lowercase letter are potentially reserved identifiers and may be added to the declarations in the `<stdatomic.h>` header.

7.33.12 Boolean type and values `<stdbool.h>`

The macro **__bool_true_false_are_defined** is an obsolescent feature.

7.33.13 Bit and byte utilities `<stdbit.h>`

Type and function names that begin with **stdc_** are potentially reserved identifiers and may be added to the declarations in the `<stdbit.h>` header.

7.33.14 Checked Arithmetic Functions `<stdckdint.h>`

Type and function names that begin with **ckd_** are potentially reserved identifiers and may be added to the declarations in the `<stdckdint.h>` header.

7.33.15 Integer types `<stdint.h>`

Typedef names beginning with **int** or **uint** and ending with **_t** are potentially reserved identifiers and may be added to the types defined in the `<stdint.h>` header. Macro names beginning with **INT** or **UINT** and ending with **_MAX**, **_MIN**, **_WIDTH**, or **_C** are potentially reserved identifiers and may be added to the macros defined in the `<stdint.h>` header.

7.33.16 Input/output `<stdio.h>`

Lowercase letters may be added to the conversion specifiers and length modifiers in **fprintf** and **fscanf**. Other characters may be used in extensions. The specifier **B** for **printf** may become mandatory in future versions of this document.

The use of **ungetc** on a binary stream where the file position indicator is zero prior to the call is an obsolescent feature.

7.33.17 General utilities `<stdlib.h>`

Function names that begin with **str** or **wcs** and a lowercase letter are potentially reserved identifiers and may be added to the declarations in the `<stdlib.h>` header.

Suppressing the macro definition of **bsearch** to access the actual function is an obsolescent feature.

7.33.18 String handling <string.h>

Function names that begin with **str**, **mem**, or **wcs** and a lowercase letter are potentially reserved identifiers and may be added to the declarations in the <string.h> header.

Suppressing the macro definitions of **memchr**, **strchr**, **strpbrk**, **strrchr**, or **strstr** to access the corresponding actual function is an obsolescent feature.

7.33.19 Date and time <time.h>

Macros beginning with **TIME_** and an uppercase letter may be added to the macros in the <time.h> header by a future edition of this document or by an implementation.

The time bases **TIME_MONOTONIC**, **TIME_ACTIVE** and **TIME_THREAD_ACTIVE** may become mandatory in future versions of this standard.

7.33.20 Threads <threads.h>

Function names, type names, and enumeration constants that begin with either **cnd_**, **mtx_**, **thrd_**, or **tss_**, and a lowercase letter are potentially reserved identifiers and may be added to the declarations in the <threads.h> header.

7.33.21 Extended multibyte and wide character utilities <wchar.h>

Function names that begin with **wcs** and a lowercase letter are potentially reserved identifiers and may be added to the declarations in the <wchar.h> header.

Lowercase letters may be added to the conversion specifiers and length modifiers in **fwprintf** and **fwscanf**. Other characters may be used in extensions.

Suppressing the macro definitions of **wcschr**, **wcspbrk**, **wcsrchr**, **wmemchr**, or **wcsstr** to access the corresponding actual function is an obsolescent feature.

7.33.22 Wide character classification and mapping utilities <wctype.h>

Function names that begin with **is** or **to** and a lowercase letter are potentially reserved identifiers and may be added to the declarations in the <wctype.h> header.

Annex A
(informative)
Language syntax summary

A.1 Notation

The notation is described in 6.1.

A.2 Lexical grammar

A.2.1 Lexical elements

(6.4.1) *token*:

keyword
identifier
constant
string-literal
punctuator

(6.4.1) *preprocessing-token*:

header-name
identifier
pp-number
character-constant
string-literal
punctuator

each universal character name that cannot be one of the above
each non-white-space character that cannot be one of the above

A.2.2 Keywords

(6.4.2) *keyword*: one of

alignas	do	int	struct	while
alignof	double	long	switch	_Atomic
auto	else	nullptr	thread_local	_BitInt
bool	enum	register	true	_Complex
break	extern	restrict	typedef	_Decimal128
case	false	return	typeof	_Decimal32
char	float	short	typeof_unqual	_Decimal64
const	for	signed	union	_Generic
constexpr	goto	sizeof	unsigned	_Imaginary
continue	if	static	void	_Noreturn
default	inline	static_assert	volatile	

A.2.3 Identifiers

(6.4.3.1) *identifier*:

identifier-start
identifier identifier-continue

(6.4.3.1) *identifier-start*:

nondigit
XID_Start character
universal character name of class XID_Start

(6.4.3.1) *identifier-continue*:

digit
nondigit
XID_Continue character
universal character name of class XID_Continue

(6.4.3.1) *nondigit*: one of

–

a b c d e f g h i j k l m
n o p q r s t u v w x y z
A B C D E F G H I J K L M
N O P Q R S T U V W X Y Z

(6.4.3.1) *digit*: one of

0 1 2 3 4 5 6 7 8 9

A.2.4 Universal character names

(6.4.4) *universal-character-name*:
 u *hex-quad*
 U *hex-quad hex-quad*

(6.4.4) *hex-quad*:
 hexadecimal-digit hexadecimal-digit hexadecimal-digit hexadecimal-digit

A.2.5 Constants

(6.4.5.1) *constant*:
 integer-constant
 floating-constant
 enumeration-constant
 character-constant
 predefined-constant

(6.4.5.2) *integer-constant*:
 decimal-constant integer-suffix_{opt}
 octal-constant integer-suffix_{opt}
 hexadecimal-constant integer-suffix_{opt}
 binary-constant integer-suffix_{opt}

(6.4.5.2) *decimal-constant*:
 nonzero-digit
 decimal-constant ' _{opt} digit

(6.4.5.2) *octal-constant*:
 0
 octal-constant ' _{opt} octal-digit

(6.4.5.2) *hexadecimal-constant*:
 hexadecimal-prefix hexadecimal-digit-sequence

(6.4.5.2) *binary-constant*:
 binary-prefix binary-digit
 binary-constant ' _{opt} binary-digit

(6.4.5.2) *hexadecimal-prefix*: one of

0x 0X

(6.4.5.2) *binary-prefix*: one of

0b 0B

(6.4.5.2) *nonzero-digit*: one of

1 2 3 4 5 6 7 8 9

(6.4.5.2) *octal-digit*: one of

0 1 2 3 4 5 6 7

hexadecimal-digit-sequence:
 hexadecimal-digit
 hexadecimal-digit-sequence ' _{opt} hexadecimal-digit

(6.4.5.2) *hexadecimal-digit*: one of

0 1 2 3 4 5 6 7 8 9
a b c d e f
A B C D E F

(6.4.5.2) *binary-digit*: one of

0 1

(6.4.5.2) *integer-suffix*:

unsigned-suffix long-suffix_{opt}
unsigned-suffix long-long-suffix
unsigned-suffix bit-precise-int-suffix
long-suffix unsigned-suffix_{opt}
long-long-suffix unsigned-suffix_{opt}
bit-precise-int-suffix unsigned-suffix_{opt}

(6.4.5.2) *bit-precise-int-suffix*: one of

wb WB

(6.4.5.2) *unsigned-suffix*: one of

u U

(6.4.5.2) *long-suffix*: one of

l L

(6.4.5.2) *long-long-suffix*: one of

ll LL

(6.4.5.3) *floating-constant*:

decimal-floating-constant
hexadecimal-floating-constant

(6.4.5.3) *decimal-floating-constant*:

fractional-constant exponent-part_{opt} floating-suffix_{opt}
digit-sequence exponent-part floating-suffix_{opt}

(6.4.5.3) *hexadecimal-floating-constant*:

hexadecimal-prefix hexadecimal-fractional-constant
binary-exponent-part floating-suffix_{opt}
hexadecimal-prefix hexadecimal-digit-sequence
binary-exponent-part floating-suffix_{opt}

(6.4.5.3) *fractional-constant*:

digit-sequence_{opt} . digit-sequence
digit-sequence .

(6.4.5.3) *exponent-part*:

e *sign_{opt} digit-sequence*
E *sign_{opt} digit-sequence*

(6.4.5.3) *sign*: one of

+ -

(6.4.5.3) *digit-sequence*:

digit
digit-sequence ' _{opt} digit

(6.4.5.3) *hexadecimal-fractional-constant*:

hexadecimal-digit-sequence_{opt} . hexadecimal-digit-sequence
hexadecimal-digit-sequence .

(6.4.5.3) *binary-exponent-part*:

p *sign_{opt} digit-sequence*
P *sign_{opt} digit-sequence*

- (6.4.5.3) *floating-suffix*: one of
f l F L df dd dL DF DD DL
- (6.4.5.4) *enumeration-constant*:
identifier
- (6.4.5.5) *character-constant*:
encoding-prefix_{opt} ' c-char-sequence '
- (6.4.5.5) *encoding-prefix*: one of
u8 u U L
- (6.4.5.5) *c-char-sequence*:
c-char
c-char-sequence c-char
- (6.4.5.5) *c-char*:
any member of the source character set except
the single-quote ' , backslash \ , or new-line character
escape-sequence
- (6.4.5.5) *escape-sequence*:
simple-escape-sequence
octal-escape-sequence
hexadecimal-escape-sequence
universal-character-name
- (6.4.5.5) *simple-escape-sequence*: one of
' \" \\ \a \b \f \n \r \t \v
- (6.4.5.5) *octal-escape-sequence*:
\ octal-digit
\ octal-digit octal-digit
\ octal-digit octal-digit octal-digit
- (6.4.5.5) *hexadecimal-escape-sequence*:
\x hexadecimal-digit
hexadecimal-escape-sequence hexadecimal-digit
- (6.4.5.6) *predefined-constant*:
false
true
nullptr

A.2.6 String literals

- (6.4.6) *string-literal*:
encoding-prefix_{opt} " s-char-sequence_{opt} "
- (6.4.6) *s-char-sequence*:
s-char
s-char-sequence s-char
- (6.4.6) *s-char*:
any member of the source character set except
the double-quote " , backslash \ , or new-line character
escape-sequence

A.2.7 Punctuators

(6.4.7) *punctuator*: one of

```
[ ] ( ) { } . ->
++ -- & * + - ~ !
/ % << >> < > <= >= == != ^ | && ||
? : :: ; ...
= *= /= %= += -= <=> >=> &= ^= |=
, # ##
<: :> <% %> %: %::
```

A.2.8 Header names

(6.4.8) *header-name*:

```
< h-char-sequence >
" q-char-sequence "
```

(6.4.8) *h-char-sequence*:

```
h-char
h-char-sequence h-char
```

(6.4.8) *h-char*:

any member of the source character set except
the new-line character and >

(6.4.8) *q-char-sequence*:

```
q-char
q-char-sequence q-char
```

(6.4.8) *q-char*:

any member of the source character set except
the new-line character and "

A.2.9 Preprocessing numbers

(6.4.9) *pp-number*:

```
digit
. digit
pp-number identifier-continue
pp-number ' digit
pp-number ' nondigit
pp-number e sign
pp-number E sign
pp-number p sign
pp-number P sign
pp-number .
```

A.3 Phrase structure grammar

A.3.1 Expressions

(6.5.2) *primary-expression*:

```
identifier
constant
string-literal
( expression )
generic-selection
```

(6.5.2.1) *generic-selection*:

```
_Generic ( assignment-expression , generic-assoc-list )
```

(6.5.2.1) *generic-assoc-list*:

```
generic-association
generic-assoc-list , generic-association
```

(6.5.2.1) *generic-association*:

type-name : *assignment-expression*
default : *assignment-expression*

(6.5.3.1) *postfix-expression*:

primary-expression
postfix-expression [*expression*]
postfix-expression (*argument-expression-list*_{opt})
postfix-expression . *identifier*
postfix-expression -> *identifier*
postfix-expression ++
postfix-expression --
compound-literal

(6.5.3.1) *argument-expression-list*:

assignment-expression
argument-expression-list , *assignment-expression*

(6.5.3.6) *compound-literal*:

(*storage-class-specifiers*_{opt} *type-name*) *braced-initializer*

(6.5.3.6) *storage-class-specifiers*:

storage-class-specifier
storage-class-specifiers *storage-class-specifier*

(6.5.4.1) *unary-expression*:

postfix-expression
++ *unary-expression*
-- *unary-expression*
unary-operator *cast-expression*
sizeof *unary-expression*
sizeof (*type-name*)
alignof (*type-name*)

(6.5.4.1) *unary-operator*: one of

& * + - ~ !

(6.5.5) *cast-expression*:

unary-expression
(*type-name*) *cast-expression*

(6.5.6) *multiplicative-expression*:

cast-expression
multiplicative-expression * *cast-expression*
multiplicative-expression / *cast-expression*
multiplicative-expression % *cast-expression*

(6.5.7) *additive-expression*:

multiplicative-expression
additive-expression + *multiplicative-expression*
additive-expression - *multiplicative-expression*

(6.5.8) *shift-expression*:

additive-expression
shift-expression << *additive-expression*
shift-expression >> *additive-expression*

(6.5.9) *relational-expression*:

shift-expression
relational-expression < *shift-expression*
relational-expression > *shift-expression*
relational-expression <= *shift-expression*
relational-expression >= *shift-expression*

(6.5.10) *equality-expression*:

relational-expression
equality-expression == *relational-expression*
equality-expression != *relational-expression*

(6.5.11) *AND-expression*:

equality-expression
AND-expression & *equality-expression*

(6.5.12) *exclusive-OR-expression*:

AND-expression
exclusive-OR-expression ^ *AND-expression*

(6.5.13) *inclusive-OR-expression*:

exclusive-OR-expression
inclusive-OR-expression | *exclusive-OR-expression*

(6.5.14) *logical-AND-expression*:

inclusive-OR-expression
logical-AND-expression && *inclusive-OR-expression*

(6.5.15) *logical-OR-expression*:

logical-AND-expression
logical-OR-expression || *logical-AND-expression*

(6.5.16) *conditional-expression*:

logical-OR-expression
logical-OR-expression ? *expression* : *conditional-expression*

(6.5.17.1) *assignment-expression*:

conditional-expression
unary-expression *assignment-operator* *assignment-expression*

(6.5.17.1) *assignment-operator*: one of

= *= /= %= += -= <=> >>= &= ^= |=

(6.5.18) *expression*:

assignment-expression
expression , *assignment-expression*

(6.6) *constant-expression*:

conditional-expression

A.3.2 Declarations

(6.7.1) *declaration*:

declaration-specifiers *init-declarator-list*_{opt} ;
attribute-specifier-sequence *declaration-specifiers* *init-declarator-list* ;
static_assert-declaration
attribute-declaration

(6.7.1) *declaration-specifiers*:

declaration-specifier *attribute-specifier-sequence*_{opt}
declaration-specifier *declaration-specifiers*

(6.7.1) *declaration-specifier*:

storage-class-specifier
type-specifier-qualifier
function-specifier

(6.7.1) *init-declarator-list*:

init-declarator
init-declarator-list , *init-declarator*

(6.7.1) *init-declarator*:

declarator
declarator = *initializer*

(6.7.1) *attribute-declaration*:

attribute-specifier-sequence ;

(6.7.2) *storage-class-specifier*:

auto
constexpr
extern
register
static
thread_local
typedef

(6.7.3.1) *type-specifier*:

void
char
short
int
long
float
double
signed
unsigned
_BitInt (*constant-expression*)
bool
_Complex
_Decimal32
_Decimal64
_Decimal128
atomic-type-specifier
struct-or-union-specifier
enum-specifier
typedef-name
typeof-specifier

(6.7.3.2) *struct-or-union-specifier*:

struct-or-union *attribute-specifier-sequence*_{opt} *identifier*_{opt} { *member-declaration-list* }
struct-or-union *attribute-specifier-sequence*_{opt} *identifier*

(6.7.3.2) *struct-or-union*:

struct
union

(6.7.3.2) *member-declaration-list*:

member-declaration
member-declaration-list *member-declaration*

(6.7.3.2) *member-declaration*:

*attribute-specifier-sequence*_{opt} *specifier-qualifier-list* *member-declarator-list*_{opt} ;
static_assert-declaration

(6.7.3.2) *specifier-qualifier-list*:

type-specifier-qualifier *attribute-specifier-sequence*_{opt}
type-specifier-qualifier *specifier-qualifier-list*

(6.7.3.2) *type-specifier-qualifier*:

type-specifier
type-qualifier
alignment-specifier

(6.7.3.2) *member-declarator-list*:

member-declarator
member-declarator-list , *member-declarator*

(6.7.3.2) *member-declarator*:

declarator
*declarator*_{opt} : *constant-expression*

(6.7.3.3) *enum-specifier*:

enum *attribute-specifier-sequence*_{opt} *identifier*_{opt} *enum-type-specifier*_{opt}
 { *enumerator-list* }
enum *attribute-specifier-sequence*_{opt} *identifier*_{opt} *enum-type-specifier*_{opt}
 { *enumerator-list* , }
enum *identifier* *enum-type-specifier*_{opt}

(6.7.3.3) *enumerator-list*:

enumerator
enumerator-list , *enumerator*

(6.7.3.3) *enumerator*:

enumeration-constant *attribute-specifier-sequence*_{opt}
enumeration-constant *attribute-specifier-sequence*_{opt} = *constant-expression*

(6.7.3.3) *enum-type-specifier*:

: *specifier-qualifier-list*

(6.7.3.5) *atomic-type-specifier*:

_Atomic (*type-name*)

(6.7.3.6) *typeof-specifier*:

typeof (*typeof-specifier-argument*)
typeof_unqual (*typeof-specifier-argument*)

(6.7.3.6) *typeof-specifier-argument*:

expression
type-name

(6.7.4.1) *type-qualifier*:

const
restrict
volatile
_Atomic

(6.7.5) *function-specifier*:

inline
_Noreturn

(6.7.6) *alignment-specifier*:

alignas (*type-name*)
alignas (*constant-expression*)

(6.7.7.1) *declarator*:

*pointer*_{opt} *direct-declarator*

(6.7.7.1) *direct-declarator*:

identifier *attribute-specifier-sequence*_{opt}
(*declarator*)
array-declarator *attribute-specifier-sequence*_{opt}
function-declarator *attribute-specifier-sequence*_{opt}

(6.7.7.1) *array-declarator*:

direct-declarator [*type-qualifier-list*_{opt} *assignment-expression*_{opt}]
direct-declarator [**static** *type-qualifier-list*_{opt} *assignment-expression*]
direct-declarator [*type-qualifier-list* **static** *assignment-expression*]
direct-declarator [*type-qualifier-list*_{opt} *]

(6.7.7.1) *function-declarator*:

direct-declarator (*parameter-type-list*_{opt})

(6.7.7.1) *pointer*:

* *attribute-specifier-sequence*_{opt} *type-qualifier-list*_{opt}
 * *attribute-specifier-sequence*_{opt} *type-qualifier-list*_{opt} *pointer*

(6.7.7.1) *type-qualifier-list*:

type-qualifier
type-qualifier-list *type-qualifier*

(6.7.7.1) *parameter-type-list*:

parameter-list
parameter-list , ...
 ...

(6.7.7.1) *parameter-list*:

parameter-declaration
parameter-list , *parameter-declaration*

(6.7.7.1) *parameter-declaration*:

*attribute-specifier-sequence*_{opt} *declaration-specifiers* *declarator*
*attribute-specifier-sequence*_{opt} *declaration-specifiers* *abstract-declarator*_{opt}

(6.7.8) *type-name*:

specifier-qualifier-list *abstract-declarator*_{opt}

(6.7.8) *abstract-declarator*:

pointer
*pointer*_{opt} *direct-abstract-declarator*

(6.7.8) *direct-abstract-declarator*:

(*abstract-declarator*)
array-abstract-declarator *attribute-specifier-sequence*_{opt}
function-abstract-declarator *attribute-specifier-sequence*_{opt}

(6.7.8) *array-abstract-declarator*:

*direct-abstract-declarator*_{opt} [*type-qualifier-list*_{opt} *assignment-expression*_{opt}]
*direct-abstract-declarator*_{opt} [**static** *type-qualifier-list*_{opt} *assignment-expression*]
*direct-abstract-declarator*_{opt} [*type-qualifier-list* **static** *assignment-expression*]
*direct-abstract-declarator*_{opt} [*]

(6.7.8) *function-abstract-declarator*:

*direct-abstract-declarator*_{opt} (*parameter-type-list*_{opt})

(6.7.9) *typedef-name*:

identifier

(6.7.11) *braced-initializer*:

{ }
 { *initializer-list* }
 { *initializer-list* , }

(6.7.11) *initializer*:

assignment-expression
braced-initializer

(6.7.11) *initializer-list*:

*designation*_{opt} *initializer*
initializer-list , *designation*_{opt} *initializer*

(6.7.11) *designation*:

designator-list =

(6.7.11) *designator-list*:

designator
designator-list *designator*

(6.7.11) *designator*:

[*constant-expression*]
 . *identifier*

(6.7.12) *static_assert-declaration*:

static_assert (*constant-expression* , *string-literal*) ;
static_assert (*constant-expression*) ;

(6.7.13.2) *attribute-specifier-sequence*:

*attribute-specifier-sequence*_{opt} *attribute-specifier*

(6.7.13.2) *attribute-specifier*:

[[*attribute-list*]]

(6.7.13.2) *attribute-list*:

*attribute*_{opt}
attribute-list , *attribute*_{opt}

(6.7.13.2) *attribute*:

attribute-token *attribute-argument-clause*_{opt}

(6.7.13.2) *attribute-token*:

standard-attribute
attribute-prefixed-token

(6.7.13.2) *standard-attribute*:

identifier

(6.7.13.2) *attribute-prefixed-token*:

attribute-prefix :: *identifier*

(6.7.13.2) *attribute-prefix*:

identifier

(6.7.13.2) *attribute-argument-clause*:

(*balanced-token-sequence*_{opt})

(6.7.13.2) *balanced-token-sequence*:

balanced-token
balanced-token-sequence *balanced-token*

(6.7.13.2) *balanced-token*:

(*balanced-token-sequence*_{opt})
 [*balanced-token-sequence*_{opt}]
 { *balanced-token-sequence*_{opt} }
 any token other than a parenthesis, a bracket, or a brace

A.3.3 Statements

(6.8.1) *statement*:

labeled-statement
unlabeled-statement

(6.8.1) *unlabeled-statement*:

expression-statement
*attribute-specifier-sequence*_{opt} *primary-block*
*attribute-specifier-sequence*_{opt} *jump-statement*

(6.8.1) *primary-block*:

compound-statement
selection-statement
iteration-statement

(6.8.1) *secondary-block*:

statement

(6.8.2) *label*:

*attribute-specifier-sequence*_{opt} *identifier* :
*attribute-specifier-sequence*_{opt} **case** *constant-expression* :
*attribute-specifier-sequence*_{opt} **default** :

(6.8.2) *labeled-statement*:

label *statement*

(6.8.3) *compound-statement*:

{ *block-item-list*_{opt} }

(6.8.3) *block-item-list*:

block-item
block-item-list *block-item*

(6.8.3) *block-item*:

declaration
unlabeled-statement
label

(6.8.4) *expression-statement*:

*expression*_{opt} ;
attribute-specifier-sequence *expression* ;

(6.8.5.1) *selection-statement*:

if (*expression*) *secondary-block*
if (*expression*) *secondary-block* **else** *secondary-block*
switch (*expression*) *secondary-block*

(6.8.6.1) *iteration-statement*:

while (*expression*) *secondary-block*
do *secondary-block* **while** (*expression*) ;
for (*expression*_{opt} ; *expression*_{opt} ; *expression*_{opt}) *secondary-block*
for (*declaration* *expression*_{opt} ; *expression*_{opt}) *secondary-block*

(6.8.7.1) *jump-statement*:

goto *identifier* ;
continue ;
break ;
return *expression*_{opt} ;

A.3.4 External definitions

(6.9.1) *translation-unit*:

external-declaration
translation-unit *external-declaration*

(6.9.1) *external-declaration*:

function-definition
declaration

(6.9.2) *function-definition*:

*attribute-specifier-sequence*_{opt} *declaration-specifiers* *declarator* *function-body*

(6.9.2) *function-body*:

compound-statement

A.4 Preprocessing directives

(6.10.1) *preprocessing-file*:

*group*_{opt}

(6.10.1) *group*:

group-part
group *group-part*

(6.10.1) *group-part*:

if-section
control-line
text-line
non-directive

(6.10.1) *if-section*:

if-group elif-groups_{opt} else-group_{opt} endif-line

(6.10.1) *if-group*:

if *constant-expression new-line group_{opt}*
ifdef *identifier new-line group_{opt}*
ifndef *identifier new-line group_{opt}*

(6.10.1) *elif-groups*:

elif-group
elif-groups elif-group

(6.10.1) *elif-group*:

elif *constant-expression new-line group_{opt}*
elifdef *identifier new-line group_{opt}*
elifndef *identifier new-line group_{opt}*

(6.10.1) *else-group*:

else *new-line group_{opt}*

(6.10.1) *endif-line*:

endif *new-line*

(6.10.1) *control-line*:

include *pp-tokens new-line*
embed *pp-tokens new-line*
define *identifier replacement-list new-line*
define *identifier lparen identifier-list_{opt}) replacement-list new-line*
define *identifier lparen ...) replacement-list new-line*
define *identifier lparen identifier-list , ...) replacement-list new-line*
undef *identifier new-line*
line *pp-tokens new-line*
error *pp-tokens_{opt} new-line*
warning *pp-tokens_{opt} new-line*
pragma *pp-tokens_{opt} new-line*
new-line

(6.10.1) *text-line*:

pp-tokens_{opt} new-line

(6.10.1) *non-directive*:

pp-tokens new-line

(6.10.1) *lparen*:

a (character not immediately preceded by white space

(6.10.1) *replacement-list*:

pp-tokens_{opt}

(6.10.1) *pp-tokens*:

preprocessing-token
pp-tokens preprocessing-token

(6.10.1) *new-line*:

the new-line character

(6.10.1) *identifier-list*:

identifier
identifier-list , identifier

- (6.10.1) *pp-parameter*:
 pp-parameter-name pp-parameter-clause_{opt}
- (6.10.1) *pp-parameter-name*:
 pp-standard-parameter
 pp-prefixed-parameter
- (6.10.1) *pp-standard-parameter*:
 identifier
- (6.10.1) *pp-prefixed-parameter*:
 identifier :: identifier
- (6.10.1) *pp-parameter-clause*:
 (*pp-balanced-token-sequence_{opt}*)
- (6.10.1) *pp-balanced-token-sequence*:
 pp-balanced-token
 pp-balanced-token-sequence pp-balanced-token
- (6.10.1) *pp-balanced-token*:
 (*pp-balanced-token-sequence_{opt}*)
 [*pp-balanced-token-sequence_{opt}*]
 { *pp-balanced-token-sequence_{opt}* }
 any *pp-token* other than a parenthesis, a bracket, or a brace
- (6.10.1) *embed-parameter-sequence*:
 pp-parameter
 embed-parameter-sequence pp-parameter
- defined-macro-expression*:
 defined *identifier*
 defined (*identifier*)
- h-preprocessing-token*:
 any *preprocessing-token* other than >
- h-pp-tokens*:
 h-preprocessing-token
 h-pp-tokens h-preprocessing-token
- header-name-tokens*:
 string-literal
 < *h-pp-tokens* >
- has-include-expression*:
 __has_include (*header-name*)
 __has_include (*header-name-tokens*)
- has-embed-expression*:
 __has_embed (*header-name embed-parameter-sequence_{opt}*)
 __has_embed (*header-name-tokens pp-balanced-token-sequence_{opt}*)
- has-c-attribute-express*:
 __has_c_attribute (*pp-tokens*)
- va-opt-replacement*:
 __VA_OPT__ (*pp-tokens_{opt}*)
- (6.10.8) *standard-pragma*:
 # pragma STDC FP_CONTRACT *on-off-switch*
 # pragma STDC FENV_ACCESS *on-off-switch*
 # pragma STDC FENV_DEC_ROUND *dec-direction*
 # pragma STDC FENV_ROUND *direction*
 # pragma STDC CX_LIMITED_RANGE *on-off-switch*
- (6.10.8) *on-off-switch*: one of
 ON OFF DEFAULT

(6.10.8) *direction*: one of

FE_DOWNWARD	FE_TONEAREST	FE_TONEARESTFROMZERO
FE_TOWARDZERO	FE_UPWARD	FE_DYNAMIC

(6.10.8) *dec-direction*: one of

FE_DEC_DOWNWARD	FE_DEC_TONEAREST	FE_DEC_TONEARESTFROMZERO
FE_DEC_TOWARDZERO	FE_DEC_UPWARD	FE_DEC_DYNAMIC

A.5 Floating-point subject sequence

A.5.1 NaN char sequence

(7.24.2.6) *n-char-sequence*:

- digit*
- nondigit*
- n-char-sequence digit*
- n-char-sequence nondigit*

A.5.2 NaN wchar_t sequence

(7.31.4.2.2) *n-wchar-sequence*:

- digit*
- nondigit*
- n-wchar-sequence digit*
- n-wchar-sequence nondigit*

A.6 Decimal floating-point subject sequence

A.6.1 NaN decimal char sequence

(7.24.2.7) *d-char-sequence*:

- digit*
- nondigit*
- d-char-sequence digit*
- d-char-sequence nondigit*

A.6.2 NaN decimal wchar_t sequence

(7.31.4.2.3) *d-wchar-sequence*:

- digit*
- nondigit*
- d-wchar-sequence digit*
- d-wchar-sequence nondigit*

Annex B

(informative)

Library summary

B.1 Diagnostics <assert.h>

__STDC_VERSION_ASSERT_H__

NDEBUG

```
void assert(scalar expression);
```

B.2 Complex <complex.h>

__STDC_NO_COMPLEX__

imaginary

__STDC_VERSION_COMPLEX_H__

_Imaginary_I

complex

I

_Complex_I

```
#pragma STDC CX_LIMITED_RANGE on-off-switch
double complex cacos(double complex z);
float complex cacosf(float complex z);
long double complex cacosl(long double complex z);
double complex casin(double complex z);
float complex casinf(float complex z);
long double complex casinl(long double complex z);
double complex catan(double complex z);
float complex catanf(float complex z);
long double complex catanl(long double complex z);
double complex ccos(double complex z);
float complex ccosf(float complex z);
long double complex ccosl(long double complex z);
double complex csin(double complex z);
float complex csinf(float complex z);
long double complex csinl(long double complex z);
double complex ctan(double complex z);
float complex ctanf(float complex z);
long double complex ctanl(long double complex z);
double complex cacosh(double complex z);
float complex cacoshf(float complex z);
long double complex cacoshl(long double complex z);
double complex casinh(double complex z);
float complex casinhf(float complex z);
long double complex casinhl(long double complex z);
double complex catanh(double complex z);
float complex catanhf(float complex z);
long double complex catanhl(long double complex z);
double complex ccosh(double complex z);
float complex ccoshf(float complex z);
long double complex ccoshl(long double complex z);
double complex csinh(double complex z);
float complex csinhf(float complex z);
long double complex csinhl(long double complex z);
double complex ctanh(double complex z);
float complex ctanhf(float complex z);
long double complex ctanhl(long double complex z);
double complex cexp(double complex z);
float complex cexpf(float complex z);
long double complex cexpl(long double complex z);
```

```

double complex clog(double complex z);
float complex clogf(float complex z);
long double complex clogl(long double complex z);
double cabs(double complex z);
float cabsf(float complex z);
long double cabsl(long double complex z);
double complex cpow(double complex x, double complex y);
float complex cpowf(float complex x, float complex y);
long double complex cpowl(long double complex x, long double complex y);
double complex csqrt(double complex z);
float complex csqrtf(float complex z);
long double complex csqrtl(long double complex z);
double carg(double complex z);
float cargf(float complex z);
long double cargl(long double complex z);
double cimag(double complex z);
float cimagf(float complex z);
long double cimagl(long double complex z);
double complex CMPLX(double x, double y);
float complex CMPLXF(float x, float y);
long double complex CMPLXL(long double x, long double y);
double complex conj(double complex z);
float complex conjf(float complex z);
long double complex conjl(long double complex z);
double complex cproj(double complex z);
float complex cprojf(float complex z);
long double complex cprojl(long double complex z);
double creal(double complex z);
float crealf(float complex z);
long double creall(long double complex z);

```

Only if the implementation defines `__STDC_IEC_60559_TYPES__` and additionally the user code defines `__STDC_WANT_IEC_60559_TYPES_EXT__` before any inclusion of `<complex.h>`:

```

_FloatNx complex cacosfN(_FloatNx complex z);
_FloatNx complex cacosfNx(_FloatNx complex z);
_FloatNx complex casinfn(_FloatNx complex z);
_FloatNx complex casinfNx(_FloatNx complex z);
_FloatNx complex catanfN(_FloatNx complex z);
_FloatNx complex catanfNx(_FloatNx complex z);
_FloatNx complex ccosfN(_FloatNx complex z);
_FloatNx complex ccosfNx(_FloatNx complex z);
_FloatNx complex csinfN(_FloatNx complex z);
_FloatNx complex csinfNx(_FloatNx complex z);
_FloatNx complex ctanfN(_FloatNx complex z);
_FloatNx complex ctanfNx(_FloatNx complex z);
_FloatNx complex cacoshfN(_FloatNx complex z);
_FloatNx complex cacoshfNx(_FloatNx complex z);
_FloatNx complex casinhfN(_FloatNx complex z);
_FloatNx complex casinhfNx(_FloatNx complex z);
_FloatNx complex catanhfN(_FloatNx complex z);
_FloatNx complex catanhfNx(_FloatNx complex z);
_FloatNx complex ccoshfN(_FloatNx complex z);
_FloatNx complex ccoshfNx(_FloatNx complex z);
_FloatNx complex csinhfN(_FloatNx complex z);
_FloatNx complex csinhfNx(_FloatNx complex z);
_FloatNx complex ctanhfN(_FloatNx complex z);
_FloatNx complex ctanhfNx(_FloatNx complex z);
_FloatNx complex cexpfN(_FloatNx complex z);
_FloatNx complex cexpfNx(_FloatNx complex z);
_FloatNx complex clogfN(_FloatNx complex z);

```

```
_FloatNx complex clogfNx(_FloatNx complex z);
_FloatN cabsfN(_FloatN complex z);
_FloatNx cabsfNx(_FloatNx complex z);
_FloatN complex cpowfN(_FloatN complex x, _FloatN complex y);
_FloatNx complex cpowfNx(_FloatNx complex x, _FloatNx complex y);
_FloatN complex csqrtfN(_FloatN complex z);
_FloatNx complex csqrtfNx(_FloatNx complex z);
_FloatN cargfN(_FloatN complex z);
_FloatNx cargfNx(_FloatNx complex z);
_FloatN cimagfN(_FloatN complex z);
_FloatNx cimagfNx(_FloatNx complex z);
_FloatN complex CMPLXFN(_FloatN x, _FloatN y);
_FloatNx complex CMPLXFNx(_FloatNx x, _FloatNx y);
_FloatN complex conjfN(_FloatN complex z);
_FloatNx complex conjfNx(_FloatNx complex z);
_FloatN complex cprojfN(_FloatN complex z);
_FloatNx complex cprojfNx(_FloatNx complex z);
_FloatN crealfN(_FloatN complex z);
_FloatNx crealfNx(_FloatNx complex z);
```

B.3 Character handling <ctype.h>

```
int isalnum(int c);
int isalpha(int c);
int isblank(int c);
int iscntrl(int c);
int isdigit(int c);
int isgraph(int c);
int islower(int c);
int isprint(int c);
int ispunct(int c);
int isspace(int c);
int isupper(int c);
int isxdigit(int c);
int tolower(int c);
int toupper(int c);
```

B.4 Errors <errno.h>

EDOM	EILSEQ	ERANGE	errno
------	--------	--------	-------

Only if the implementation defines `__STDC_LIB_EXT1__` and additionally the user code defines `__STDC_WANT_LIB_EXT1__` before any inclusion of `<errno.h>`:

`errno_t`

B.5 Floating-point environment <fenv.h>

<code>fenv_t</code>	<code>FE_INVALID</code>	<code>FE_TONEARESTFROMZERO</code>
<code>fexcept_t</code>	<code>FE_OVERFLOW</code>	<code>FE_TOWARDZERO</code>
<code>femode_t</code>	<code>FE_UNDERFLOW</code>	<code>FE_UPWARD</code>
<code>FE_DFL_MODE</code>	<code>FE_ALL_EXCEPT</code>	<code>FE_DFL_ENV</code>
<code>FE_DIVBYZERO</code>	<code>FE_DOWNWARD</code>	<code>__STDC_VERSION_FENV_H__</code>
<code>FE_INEXACT</code>	<code>FE_TONEAREST</code>	

```
#pragma STDC FENV_ACCESS on-off-switch
#pragma STDC FENV_ROUND direction
```



```
#pragma STDC FENV_ROUND FE_DYNAMIC
int feclearexcept(int excepts);
int fegetexceptflag(fexcept_t *flagp, int excepts);
int feraiseexcept(int excepts);
int fesetexcept(int excepts);
int fesetexceptflag(const fexcept_t *flagp, int excepts);
int fetestexceptflag(const fexcept_t *flagp, int excepts);
int fetestexcept(int excepts);
int fegetmode(femode_t *modep);
int fegetround(void);
int fesetmode(const femode_t *modep);
int fesetround(int rnd);
int fegetenv(fenv_t *envp);
int feholdexcept(fenv_t *envp);
int fesetenv(const fenv_t *envp);
int feupdateenv(const fenv_t *envp);
```

Only if the implementation defines `__STDC_IEC_60559_DFP__`:

FE_DEC_DOWNWARD

FE_DEC_TONEAREST

FE_DEC_TONEARESTFROMZERO

FE_DEC_TOWARDZERO

FE_DEC_UPWARD

```
#pragma STDC FENV_DEC_ROUND dec-direction
int fe_dec_getround(void);
int fe_dec_setround(int rnd);
```

Only if the implementation follows the recommended practice from F.2.2:

FE_SNANS_ALWAYS_SIGNAL

B.6 Characteristics of floating types <float.h>

B.6.1 Macros

__STDC_VERSION_FLOAT_H__	DBL_DIG	DBL_NORM_MAX
FLT_ROUNDS	LDBL_DIG	LDBL_NORM_MAX
FLT_EVAL_METHOD	FLT_MIN_EXP	FLT_EPSILON
FLT_HAS_SUBNORM	DBL_MIN_EXP	DBL_EPSILON
DBL_HAS_SUBNORM	LDBL_MIN_EXP	LDBL_EPSILON
LDBL_HAS_SUBNORM	FLT_MIN_10_EXP	FLT_MIN
FLT_RADIX	DBL_MIN_10_EXP	DBL_MIN
FLT_MANT_DIG	LDBL_MIN_10_EXP	LDBL_MIN
DBL_MANT_DIG	FLT_MAX_EXP	FLT_SNAN
LDBL_MANT_DIG	DBL_MAX_EXP	DBL_SNAN
FLT_DECIMAL_DIG	LDBL_MAX_EXP	LDBL_SNAN
DBL_DECIMAL_DIG	FLT_MAX_10_EXP	FLT_TRUE_MIN
LDBL_DECIMAL_DIG	DBL_MAX_10_EXP	DBL_TRUE_MIN
DECIMAL_DIG	LDBL_MAX_10_EXP	LDBL_TRUE_MIN
FLT_IS_IEC_60559	FLT_MAX	INFINITY
DBL_IS_IEC_60559	DBL_MAX	NAN
LDBL_IS_IEC_60559	LDBL_MAX	
FLT_DIG	FLT_NORM_MAX	

The following macro is provided only if the program defines `__STDC_WANT_IEC_60559_EXT__` before inclusion of the header <float.h>:

CR_DECIMAL_DIG

B.6.2 Characteristics of decimal floating types

The following macros are provided only if the implementation defines `__STDC_IEC_60559_DFP__`. *N* is 32, 64 and 128.

DEC_EVAL_METHOD	DECN_EPSILON	DECN_MAX	DECN_TRUE_MIN
DEC_INFINITY	DECN_MANT_DIG	DECN_MIN_EXP	DECN_SNaN
DEC_NAN	DECN_MAX_EXP	DECN_MIN	

B.6.3 Characteristics of ISO/IEC 60559 interchange and extended types

The following macros are provided only if the implementation defines `__STDC_IEC_60559_TYPES__` and additionally the user code defines `__STDC_WANT_IEC_60559_TYPES_EXT__` before any inclusion of `<float.h>`.

FLT_N_DECIMAL_DIG	FLT_N_SNaN	FLT_NX_MIN_EXP	DEC_N_SNaN
FLT_N_DIG	FLT_N_TRUE_MIN	FLT_NX_MIN	DEC_N_TRUE_MIN
FLT_N_EPSILON	FLT_NX_DECIMAL_DIG	FLT_NX_SNaN	DEC_NX_EPSILON
FLT_N_MANT_DIG	FLT_NX_DIG	FLT_NX_TRUE_MIN	DEC_NX_MANT_DIG
FLT_N_MAX_10_EXP	FLT_NX_EPSILON	DEC_N_EPSILON	DEC_NX_MAX_EXP
FLT_N_MAX_EXP	FLT_NX_MANT_DIG	DEC_N_MANT_DIG	DEC_NX_MAX
FLT_N_MAX	FLT_NX_MAX_10_EXP	DEC_N_MAX_EXP	DEC_NX_MIN_EXP
FLT_N_MIN_10_EXP	FLT_NX_MAX_EXP	DEC_N_MAX	DEC_NX_MIN
FLT_N_MIN_EXP	FLT_NX_MAX	DEC_N_MIN_EXP	DEC_NX_SNaN
FLT_N_MIN	FLT_NX_MIN_10_EXP	DEC_N_MIN	DEC_NX_TRUE_MIN

B.7 Format conversion of integer types `<inttypes.h>`

`__STDC_VERSION_INTTYPES_H__` `imaxdiv_t`

PRIbN	PRIBLEASTN	PRIBFASTN	PRIBMAX	PRIBPTR
PRIBN	PRIBLEASTN	PRIBFASTN	PRIBMAX	PRIBPTR
PRIdN	PRIDLEASTN	PRIDFASTN	PRIDMAX	PRIDPTR
PRIiN	PRIILEASTN	PRIIFASTN	PRIIMAX	PRIIPTR
PRIoN	PRIOLEASTN	PRIOFASTN	PRIOMAX	PRIOPTR
PRIuN	PRIULEASTN	PRIUFASTN	PRIUMAX	PRIUPTR
PRIxN	PRIXLEASTN	PRIXFASTN	PRIXMAX	PRIXPTR
PRIXN	PRIXLEASTN	PRIXFASTN	PRIXMAX	PRIXPTR
SCNbN	SCNBLEASTN	SCNBFASTN	SCNBMAX	SCNBPTR
SCNdN	SCNDLEASTN	SCNDFASTN	SCNDMAX	SCNDPTR
SCNiN	SCNILEASTN	SCNIFASTN	SCNIMAX	SCNIPTR
SCNoN	SCNOLEASTN	SCNOFASTN	SCNOMAX	SCNOPTR
SCNuN	SCNULEASTN	SCNUFASTN	SCNUMAX	SCNUPTR
SCNxN	SCNXLEASTN	SCNXFASTN	SCNXMAX	SCNXPTR

```
intmax_t imaxabs(intmax_t j);
imaxdiv_t imaxdiv(intmax_t numer, intmax_t denom);
intmax_t strtoumax(const char * restrict nptr, char ** restrict endptr, int base);
uintmax_t strtoumax(const char * restrict nptr, char ** restrict endptr, int base);
intmax_t wcstoumax(const wchar_t * restrict nptr, wchar_t ** restrict endptr, int base);
uintmax_t wcstoumax(const wchar_t * restrict nptr, wchar_t ** restrict endptr, int base);
```

B.8 Alternative spellings `<iso646.h>`

and	bitor	not_eq	xor
and_eq	compl	or	xor_eq
bitand	not	or_eq	

B.9 Sizes of integer types <limits.h>

__STDC_VERSION_LIMITS_H__	ULONG_WIDTH	USHRT_MAX
BITINT_MAXWIDTH	LLONG_WIDTH	INT_MIN
BOOL_WIDTH	ULLONG_WIDTH	INT_MAX
CHAR_BIT	BOOL_MAX	UINT_MAX
CHAR_WIDTH	SCHAR_MIN	LONG_MIN
SCHAR_WIDTH	SCHAR_MAX	LONG_MAX
UCHAR_WIDTH	UCHAR_MAX	ULONG_MAX
SHRT_WIDTH	CHAR_MIN	LLONG_MIN
USHRT_WIDTH	CHAR_MAX	LLONG_MAX
INT_WIDTH	MB_LEN_MAX	ULLONG_MAX
UINT_WIDTH	SHRT_MIN	
LONG_WIDTH	SHRT_MAX	

B.10 Localization <locale.h>

struct lconv	LC_ALL	LC_CTYPE	LC_NUMERIC
NULL	LC_COLLATE	LC_MONETARY	LC_TIME

```
char *setlocale(int category, const char *locale);
struct lconv *localeconv(void);
```

B.11 Mathematics <math.h>

__STDC_VERSION_MATH_H__	FP_INT_TONEARESTFROMZERO	FP_FAST_DDIVL
float_t	FP_INT_TONEAREST	FP_FAST_FSQRT
double_t	FP_FAST_FMA	FP_FAST_FSQRTL
HUGE_VAL	FP_FAST_FMAF	FP_FAST_DSQRTL
HUGE_VALF	FP_FAST_FMAL	FP_FAST_FFMA
HUGE_VALL	FP_FAST_FADD	FP_FAST_FFMAL
INFINITY	FP_FAST_FADDL	FP_FAST_DFMAL
NAN	FP_FAST_DADDL	FP_ILOGB0
FP_INFINITE	FP_FAST_FSUB	FP_ILOGBNAN
FP_NAN	FP_FAST_FSUBL	FP_LLOGB0
FP_NORMAL	FP_FAST_DSUBL	FP_LLOGBNAN
FP_SUBNORMAL	FP_FAST_FMUL	MATH_ERRNO
FP_ZERO	FP_FAST_FMULL	MATH_ERREXCEPT
FP_INT_UPWARD	FP_FAST_DMULL	math_errhandling
FP_INT_DOWNWARD	FP_FAST_FDIV	
FP_INT_TOWARDZERO	FP_FAST_FDIVL	

```
#pragma STDC FP_CONTRACT on-off-switch
int fpclassify(real-floating x);
int iscanonical(real-floating x);
int isfinite(real-floating x);
int isinf(real-floating x);
int isnan(real-floating x);
int isnormal(real-floating x);
int signbit(real-floating x);
int issignaling(real-floating x);
```

```

int issubnormal(real-floating x);
int iszero(real-floating x);
double acos(double x);
float acosf(float x);
long double acosl(long double x);
double asin(double x);
float asinf(float x);
long double asinl(long double x);
double atan(double x);
float atanf(float x);
long double atanl(long double x);
double atan2(double y, double x);
float atan2f(float y, float x);
long double atan2l(long double y, long double x);
double cos(double x);
float cosf(float x);
long double cosl(long double x);
double sin(double x);
float sinf(float x);
long double sinl(long double x);
double tan(double x);
float tanf(float x);
long double tanl(long double x);
double acospi(double x);
float acospi_f(float x);
long double acospi_l(long double x);
double asinpi(double x);
float asinpi_f(float x);
long double asinpi_l(long double x);
double atanpi(double x);
float atanpi_f(float x);
long double atanpi_l(long double x);
double atan2pi(double y, double x);
float atan2pi_f(float y, float x);
long double atan2pi_l(long double y, long double x);
double cospi(double x);
float cospi_f(float x);
long double cospi_l(long double x);
double sinpi(double x);
float sinpi_f(float x);
long double sinpi_l(long double x);
double tanpi(double x);
float tanpi_f(float x);
long double tanpi_l(long double x);
double acosh(double x);
float acoshf(float x);
long double acoshl(long double x);
double asinh(double x);
float asinhf(float x);
long double asinhl(long double x);
double atanh(double x);
float atanhf(float x);
long double atanh_l(long double x);
double cosh(double x);
float coshf(float x);
long double coshl(long double x);
double sinh(double x);
float sinh_f(float x);
long double sinhl(long double x);
double tanh(double x);
float tanhf(float x);

```

```

long double tanhl(long double x);
double exp(double x);
float expf(float x);
long double expl(long double x);
double exp10(double x);
float exp10f(float x);
long double exp10l(long double x);
double exp10m1(double x);
float exp10m1f(float x);
long double exp10m1l(long double x);
double exp2(double x);
float exp2f(float x);
long double exp2l(long double x);
double exp2m1(double x);
float exp2m1f(float x);
long double exp2m1l(long double x);
double expm1(double x);
float expm1f(float x);
long double expm1l(long double x);
double frexp(double value, int *p);
float frexpf(float value, int *p);
long double frexpl(long double value, int *p);
int ilogb(double x);
int ilogbf(float x);
int ilogbl(long double x);
double ldexp(double x, int p);
float ldexpf(float x, int p);
long double ldexpl(long double x, int p);
long int llogb(double x);
long int llogbf(float x);
long int llogbl(long double x);
double log(double x);
float logf(float x);
long double logl(long double x);
double log10(double x);
float log10f(float x);
long double log10l(long double x);
double log10p1(double x);
float log10p1f(float x);
long double log10p1l(long double x);
double log1p(double x);
float log1pf(float x);
long double log1pl(long double x);
double logp1(double x);
float logp1f(float x);
long double logp1l(long double x);
double log2(double x);
float log2f(float x);
long double log2l(long double x);
double log2p1(double x);
float log2p1f(float x);
long double log2p1l(long double x);
double logb(double x);
float logbf(float x);
long double logbl(long double x);
double modf(double value, double *iptr);
float modff(float value, float *iptr);
long double modfl(long double value, long double *iptr);
double scalbn(double x, int n);
float scalbnf(float x, int n);
long double scalbnl(long double x, int n);

```

```

double scalbln(double x, long int n);
float scalblnf(float x, long int n);
long double scalblnl(long double x, long int n);
double cbrt(double x);
float cbrtf(float x);
long double cbrtl(long double x);
double compoundn(double x, long long int n);
float compoundnf(float x, long long int n);
long double compoundnl(long double x, long long int n);
double fabs(double x);
float fabsf(float x);
long double fabsl(long double x);
double hypot(double x, double y);
float hypotf(float x, float y);
long double hypotl(long double x, long double y);
double pow(double x, double y);
float powf(float x, float y);
long double powl(long double x, long double y);
double pown(double x, long long int n);
float pownf(float x, long long int n);
long double pownl(long double x, long long int n);
double powr(double y, double x);
float powrf(float y, float x);
long double powrl(long double y, long double x);
double rootn(double x, long long int n);
float rootnf(float x, long long int n);
long double rootnl(long double x, long long int n);
double rsqrt(double x);
float rsqrtf(float x);
long double rsqrtl(long double x);
double sqrt(double x);
float sqrtf(float x);
long double sqrtl(long double x);
double erf(double x);
float erff(float x);
long double erfl(long double x);
double erfc(double x);
float erfcf(float x);
long double erfcl(long double x);
double lgamma(double x);
float lgammaf(float x);
long double lgammal(long double x);
double tgamma(double x);
float tgammaf(float x);
long double tgammal(long double x);
double ceil(double x);
float ceilf(float x);
long double ceill(long double x);
double floor(double x);
float floorf(float x);
long double floorl(long double x);
double nearbyint(double x);
float nearbyintf(float x);
long double nearbyintl(long double x);
double rint(double x);
float rintf(float x);
long double rintl(long double x);
long int lrint(double x);
long int lrintf(float x);
long int lrintl(long double x);
long long int llrint(double x);

```

```

long long int llrintf(float x);
long long int llrintl(long double x);
double round(double x);
float roundf(float x);
long double roundl(long double x);
long int lround(double x);
long int lroundf(float x);
long int lroundl(long double x);
long long int llround(double x);
long long int llroundf(float x);
long long int llroundl(long double x);
double roundeven(double x);
float roundevenf(float x);
long double roundevenl(long double x);
double trunc(double x);
float truncf(float x);
long double truncf(long double x);
double fromfp(double x, int rnd, unsigned int width);
float fromfpf(float x, int rnd, unsigned int width);
long double fromfpl(long double x, int rnd, unsigned int width);
double ufromfp(double x, int rnd, unsigned int width);
float ufromfpf(float x, int rnd, unsigned int width);
long double ufromfpl(long double x, int rnd, unsigned int width);
double fromfpx(double x, int rnd, unsigned int width);
float fromfpxf(float x, int rnd, unsigned int width);
long double fromfpxl(long double x, int rnd, unsigned int width);
double ufromfpx(double x, int rnd, unsigned int width);
float ufromfpxf(float x, int rnd, unsigned int width);
long double ufromfpxl(long double x, int rnd, unsigned int width);
double fmod(double x, double y);
float fmodf(float x, float y);
long double fmodl(long double x, long double y);
double remainder(double x, double y);
float remainderf(float x, float y);
long double remainderl(long double x, long double y);
double remquo(double x, double y, int *quo);
float remquof(float x, float y, int *quo);
long double remquol(long double x, long double y, int *quo);
double copysign(double x, double y);
float copysignf(float x, float y);
long double copysignl(long double x, long double y);
double nan(const char *tagp);
float nanf(const char *tagp);
long double nanl(const char *tagp);
double nextafter(double x, double y);
float nextafterf(float x, float y);
long double nextafterl(long double x, long double y);
double nexttoward(double x, long double y);
float nexttowardf(float x, long double y);
long double nexttowardl(long double x, long double y);
double nextup(double x);
float nextupf(float x);
long double nextupl(long double x);
double nextdown(double x);
float nextdownf(float x);
long double nextdownl(long double x);
int canonicalize(double *cx, const double *x);
int canonicalizef(float *cx, const float *x);
int canonicalizel(long double *cx, const long double *x);
double fdim(double x, double y);
float fdimf(float x, float y);

```



```

long double fdiml(long double x, long double y);
double fmax(double x, double y);
float fmaxf(float x, float y);
long double fmaxl(long double x, long double y);
double fmin(double x, double y);
float fminf(float x, float y);
long double fminl(long double x, long double y);
double fmaximum(double x, double y);
float fmaximumf(float x, float y);
long double fmaximuml(long double x, long double y);
double fminimum(double x, double y);
float fminimumf(float x, float y);
long double fminimuml(long double x, long double y);
double fmaximum_mag(double x, double y);
float fmaximum_magf(float x, float y);
long double fmaximum_magl(long double x, long double y);
double fminimum_mag(double x, double y);
float fminimum_magf(float x, float y);
long double fminimum_magl(long double x, long double y);
double fmaximum_num(double x, double y);
float fmaximum_numf(float x, float y);
long double fmaximum_numl(long double x, long double y);
double fminimum_num(double x, double y);
float fminimum_numf(float x, float y);
long double fminimum_numl(long double x, long double y);
double fmaximum_mag_num(double x, double y);
float fmaximum_mag_numf(float x, float y);
long double fmaximum_mag_numl(long double x, long double y);
double fminimum_mag_num(double x, double y);
float fminimum_mag_numf(float x, float y);
long double fminimum_mag_numl(long double x, long double y);
double fma(double x, double y, double z);
float fmaf(float x, float y, float z);
long double fmal(long double x, long double y, long double z);
float fadd(double x, double y);
float faddl(long double x, long double y);
double daddl(long double x, long double y);
float fsub(double x, double y);
float fsubl(long double x, long double y);
double dsubl(long double x, long double y);
float fmul(double x, double y);
float fmul_l(long double x, long double y);
double dmul_l(long double x, long double y);
float fdiv(double x, double y);
float fdivl(long double x, long double y);
double ddivl(long double x, long double y);
float fma(double x, double y, double z);
float fmal(long double x, long double y, long double z);
double dfmal(long double x, long double y, long double z);
float fsqrt(double x);
float fsqrtl(long double x);
double dsqrtl(long double x);
int isgreater(real-floating x, real-floating y);
int isgreaterequal(real-floating x, real-floating y);
int isless(real-floating x, real-floating y);
int islessequal(real-floating x, real-floating y);
int islessgreater(real-floating x, real-floating y);
int isunordered(real-floating x, real-floating y);
int iseqsig(real-floating x, real-floating y);

```

Only if the implementation defines `__STDC_IEC_60559_DFP__`:

DEC_INFINITY	FP_FAST_D32SUBD64	FP_FAST_D64DIVD128
DEC_NAN	FP_FAST_D32SUBD128	FP_FAST_D32FMAD64
FP_FAST_FMAD32	FP_FAST_D64SUBD128	FP_FAST_D32FMAD128
FP_FAST_FMAD64	FP_FAST_D32MULD64	FP_FAST_D64FMAD128
FP_FAST_FMAD128	FP_FAST_D32MULD128	FP_FAST_D32SQRTD64
FP_FAST_D32ADDD64	FP_FAST_D64MULD128	FP_FAST_D32SQRTD128
FP_FAST_D32ADDD128	FP_FAST_D32DIVD64	FP_FAST_D64SQRTD128
FP_FAST_D64ADDD128	FP_FAST_D32DIVD128	

```

_Decimal32 acosd32(_Decimal32 x);
_Decimal64 acosd64(_Decimal64 x);
_Decimal128 acosd128(_Decimal128 x);
_Decimal32 asind32(_Decimal32 x);
_Decimal64 asind64(_Decimal64 x);
_Decimal128 asind128(_Decimal128 x);
_Decimal32 atand32(_Decimal32 x);
_Decimal64 atand64(_Decimal64 x);
_Decimal128 atand128(_Decimal128 x);
_Decimal32 atan2d32(_Decimal32 y, _Decimal32 x);
_Decimal64 atan2d64(_Decimal64 y, _Decimal64 x);
_Decimal128 atan2d128(_Decimal128 y, _Decimal128 x);
_Decimal32 cosd32(_Decimal32 x);
_Decimal64 cosd64(_Decimal64 x);
_Decimal128 cosd128(_Decimal128 x);
_Decimal32 sind32(_Decimal32 x);
_Decimal64 sind64(_Decimal64 x);
_Decimal128 sind128(_Decimal128 x);
_Decimal32 tand32(_Decimal32 x);
_Decimal64 tand64(_Decimal64 x);
_Decimal128 tand128(_Decimal128 x);
_Decimal32 acospid32(_Decimal32 x);
_Decimal64 acospid64(_Decimal64 x);
_Decimal128 acospid128(_Decimal128 x);
_Decimal32 asinpid32(_Decimal32 x);
_Decimal64 asinpid64(_Decimal64 x);
_Decimal128 asinpid128(_Decimal128 x);
_Decimal32 atanpid32(_Decimal32 x);
_Decimal64 atanpid64(_Decimal64 x);
_Decimal128 atanpid128(_Decimal128 x);
_Decimal32 atan2pid32(_Decimal32 y, _Decimal32 x);
_Decimal64 atan2pid64(_Decimal64 y, _Decimal64 x);
_Decimal128 atan2pid128(_Decimal128 y, _Decimal128 x);
_Decimal32 cospid32(_Decimal32 x);
_Decimal64 cospid64(_Decimal64 x);
_Decimal128 cospid128(_Decimal128 x);
_Decimal32 sinpid32(_Decimal32 x);
_Decimal64 sinpid64(_Decimal64 x);
_Decimal128 sinpid128(_Decimal128 x);
_Decimal32 tanpid32(_Decimal32 x);
_Decimal64 tanpid64(_Decimal64 x);
_Decimal128 tanpid128(_Decimal128 x);
_Decimal32 acoshd32(_Decimal32 x);
_Decimal64 acoshd64(_Decimal64 x);
_Decimal128 acoshd128(_Decimal128 x);
_Decimal32 asinhd32(_Decimal32 x);
_Decimal64 asinhd64(_Decimal64 x);
_Decimal128 asinhd128(_Decimal128 x);
_Decimal32 atanh32(_Decimal32 x);
_Decimal64 atanh64(_Decimal64 x);
_Decimal128 atanh128(_Decimal128 x);

```

```

_Decimal32 coshd32(_Decimal32 x);
_Decimal64 coshd64(_Decimal64 x);
_Decimal128 coshd128(_Decimal128 x);
_Decimal32 sinhd32(_Decimal32 x);
_Decimal64 sinhd64(_Decimal64 x);
_Decimal128 sinhd128(_Decimal128 x);
_Decimal32 tanhd32(_Decimal32 x);
_Decimal64 tanhd64(_Decimal64 x);
_Decimal128 tanhd128(_Decimal128 x);
_Decimal32 expd32(_Decimal32 x);
_Decimal64 expd64(_Decimal64 x);
_Decimal128 expd128(_Decimal128 x);
_Decimal32 expl0d32(_Decimal32 x);
_Decimal64 expl0d64(_Decimal64 x);
_Decimal128 expl0d128(_Decimal128 x);
_Decimal32 expl0m1d32(_Decimal32 x);
_Decimal64 expl0m1d64(_Decimal64 x);
_Decimal128 expl0m1d128(_Decimal128 x);
_Decimal32 exp2d32(_Decimal32 x);
_Decimal64 exp2d64(_Decimal64 x);
_Decimal128 exp2d128(_Decimal128 x);
_Decimal32 exp2m1d32(_Decimal32 x);
_Decimal64 exp2m1d64(_Decimal64 x);
_Decimal128 exp2m1d128(_Decimal128 x);
_Decimal32 expm1d32(_Decimal32 x);
_Decimal64 expm1d64(_Decimal64 x);
_Decimal128 expm1d128(_Decimal128 x);
_Decimal32 frexpd32(_Decimal32 value, int *p);
_Decimal64 frexpd64(_Decimal64 value, int *p);
_Decimal128 frexpd128(_Decimal128 value, int *p);
int ilogbd32(_Decimal32 x);
int ilogbd64(_Decimal64 x);
int ilogbd128(_Decimal128 x);
_Decimal32 ldexpd32(_Decimal32 x, int p);
_Decimal64 ldexpd64(_Decimal64 x, int p);
_Decimal128 ldexpd128(_Decimal128 x, int p);
long int llogbd32(_Decimal32 x);
long int llogbd64(_Decimal64 x);
long int llogbd128(_Decimal128 x);
_Decimal32 logd32(_Decimal32 x);
_Decimal64 logd64(_Decimal64 x);
_Decimal128 logd128(_Decimal128 x);
_Decimal32 logl0d32(_Decimal32 x);
_Decimal64 logl0d64(_Decimal64 x);
_Decimal128 logl0d128(_Decimal128 x);
_Decimal32 logl0p1d32(_Decimal32 x);
_Decimal64 logl0p1d64(_Decimal64 x);
_Decimal128 logl0p1d128(_Decimal128 x);
_Decimal32 logl1pd32(_Decimal32 x);
_Decimal64 logl1pd64(_Decimal64 x);
_Decimal128 logl1pd128(_Decimal128 x);
_Decimal32 logp1d32(_Decimal32 x);
_Decimal64 logp1d64(_Decimal64 x);
_Decimal128 logp1d128(_Decimal128 x);
_Decimal32 log2d32(_Decimal32 x);
_Decimal64 log2d64(_Decimal64 x);
_Decimal128 log2d128(_Decimal128 x);
_Decimal32 log2p1d32(_Decimal32 x);
_Decimal64 log2p1d64(_Decimal64 x);
_Decimal128 log2p1d128(_Decimal128 x);
_Decimal32 logbd32(_Decimal32 x);

```

```

_Decimal64 logbd64(_Decimal64 x);
_Decimal128 logbd128(_Decimal128 x);
_Decimal32 modfd32(_Decimal32 x, _Decimal32 *iptr);
_Decimal64 modfd64(_Decimal64 x, _Decimal64 *iptr);
_Decimal128 modfd128(_Decimal128 x, _Decimal128 *iptr);
_Decimal32 scalbnd32(_Decimal32 x, int n);
_Decimal64 scalbnd64(_Decimal64 x, int n);
_Decimal128 scalbnd128(_Decimal128 x, int n);
_Decimal32 scalblnd32(_Decimal32 x, long int n);
_Decimal64 scalblnd64(_Decimal64 x, long int n);
_Decimal128 scalblnd128(_Decimal128 x, long int n);
_Decimal32 cbrtd32(_Decimal32 x);
_Decimal64 cbrtd64(_Decimal64 x);
_Decimal128 cbrtd128(_Decimal128 x);
_Decimal32 compoundnd32(_Decimal32 x, long long int n);
_Decimal64 compoundnd64(_Decimal64 x, long long int n);
_Decimal128 compoundnd128(_Decimal128 x, long long int n);
_Decimal32 fabsd32(_Decimal32 x);
_Decimal64 fabsd64(_Decimal64 x);
_Decimal128 fabsd128(_Decimal128 x);
_Decimal32 hypotd32(_Decimal32 x, _Decimal32 y);
_Decimal64 hypotd64(_Decimal64 x, _Decimal64 y);
_Decimal128 hypotd128(_Decimal128 x, _Decimal128 y);
_Decimal32 powd32(_Decimal32 x, _Decimal32 y);
_Decimal64 powd64(_Decimal64 x, _Decimal64 y);
_Decimal128 powd128(_Decimal128 x, _Decimal128 y);
_Decimal32 pownd32(_Decimal32 x, long long int n);
_Decimal64 pownd64(_Decimal64 x, long long int n);
_Decimal128 pownd128(_Decimal128 x, long long int n);
_Decimal32 powrd32(_Decimal32 y, _Decimal32 x);
_Decimal64 powrd64(_Decimal64 y, _Decimal64 x);
_Decimal128 powrd128(_Decimal128 y, _Decimal128 x);
_Decimal32 rootnd32(_Decimal32 x, long long int n);
_Decimal64 rootnd64(_Decimal64 x, long long int n);
_Decimal128 rootnd128(_Decimal128 x, long long int n);
_Decimal32 rsqrt32(_Decimal32 x);
_Decimal64 rsqrt64(_Decimal64 x);
_Decimal128 rsqrt128(_Decimal128 x);
_Decimal32 sqrt32(_Decimal32 x);
_Decimal64 sqrt64(_Decimal64 x);
_Decimal128 sqrt128(_Decimal128 x);
_Decimal32 erfd32(_Decimal32 x);
_Decimal64 erfd64(_Decimal64 x);
_Decimal128 erfd128(_Decimal128 x);
_Decimal32 erfcd32(_Decimal32 x);
_Decimal64 erfcd64(_Decimal64 x);
_Decimal128 erfcd128(_Decimal128 x);
_Decimal32 lgammad32(_Decimal32 x);
_Decimal64 lgammad64(_Decimal64 x);
_Decimal128 lgammad128(_Decimal128 x);
_Decimal32 tgammd32(_Decimal32 x);
_Decimal64 tgammd64(_Decimal64 x);
_Decimal128 tgammd128(_Decimal128 x);
_Decimal32 ceild32(_Decimal32 x);
_Decimal64 ceild64(_Decimal64 x);
_Decimal128 ceild128(_Decimal128 x);
_Decimal32 floord32(_Decimal32 x);
_Decimal64 floord64(_Decimal64 x);
_Decimal128 floord128(_Decimal128 x);
_Decimal32 nearbyintd32(_Decimal32 x);
_Decimal64 nearbyintd64(_Decimal64 x);

```

```

_Decimal128 nearbyintd128(_Decimal128 x);
_Decimal32 rintd32(_Decimal32 x);
_Decimal64 rintd64(_Decimal64 x);
_Decimal128 rintd128(_Decimal128 x);
long int lrintd32(_Decimal32 x);
long int lrintd64(_Decimal64 x);
long int lrintd128(_Decimal128 x);
long long int llrintd32(_Decimal32 x);
long long int llrintd64(_Decimal64 x);
long long int llrintd128(_Decimal128 x);
_Decimal32 roundd32(_Decimal32 x);
_Decimal64 roundd64(_Decimal64 x);
_Decimal128 roundd128(_Decimal128 x);
long int lroundd32(_Decimal32 x);
long int lroundd64(_Decimal64 x);
long int lroundd128(_Decimal128 x);
long long int llroundd32(_Decimal32 x);
long long int llroundd64(_Decimal64 x);
long long int llroundd128(_Decimal128 x);
_Decimal32 roundevend32(_Decimal32 x);
_Decimal64 roundevend64(_Decimal64 x);
_Decimal128 roundevend128(_Decimal128 x);
_Decimal32 truncd32(_Decimal32 x);
_Decimal64 truncd64(_Decimal64 x);
_Decimal128 truncd128(_Decimal128 x);
_Decimal32 fromfpd32(_Decimal32 x, int rnd, unsigned int width);
_Decimal64 fromfpd64(_Decimal64 x, int rnd, unsigned int width);
_Decimal128 fromfpd128(_Decimal128 x, int rnd, unsigned int width);
_Decimal32 ufromfpd32(_Decimal32 x, int rnd, unsigned int width);
_Decimal64 ufromfpd64(_Decimal64 x, int rnd, unsigned int width);
_Decimal128 ufromfpd128(_Decimal128 x, int rnd, unsigned int width);
_Decimal32 fromfpxd32(_Decimal32 x, int rnd, unsigned int width);
_Decimal64 fromfpxd64(_Decimal64 x, int rnd, unsigned int width);
_Decimal128 fromfpxd128(_Decimal128 x, int rnd, unsigned int width);
_Decimal32 ufromfpxd32(_Decimal32 x, int rnd, unsigned int width);
_Decimal64 ufromfpxd64(_Decimal64 x, int rnd, unsigned int width);
_Decimal128 ufromfpxd128(_Decimal128 x, int rnd, unsigned int width);
_Decimal32 fmodd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmodd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmodd128(_Decimal128 x, _Decimal128 y);
_Decimal32 remainderd32(_Decimal32 x, _Decimal32 y);
_Decimal64 remainderd64(_Decimal64 x, _Decimal64 y);
_Decimal128 remainderd128(_Decimal128 x, _Decimal128 y);
_Decimal32 copysignd32(_Decimal32 x, _Decimal32 y);
_Decimal64 copysignd64(_Decimal64 x, _Decimal64 y);
_Decimal128 copysignd128(_Decimal128 x, _Decimal128 y);
_Decimal32 nand32(const char *tagp);
_Decimal64 nand64(const char *tagp);
_Decimal128 nand128(const char *tagp);
_Decimal32 nextafterd32(_Decimal32 x, _Decimal32 y);
_Decimal64 nextafterd64(_Decimal64 x, _Decimal64 y);
_Decimal128 nextafterd128(_Decimal128 x, _Decimal128 y);
_Decimal32 nexttowardd32(_Decimal32 x, _Decimal128 y);
_Decimal64 nexttowardd64(_Decimal64 x, _Decimal128 y);
_Decimal128 nexttowardd128(_Decimal128 x, _Decimal128 y);
_Decimal32 nextupd32(_Decimal32 x);
_Decimal64 nextupd64(_Decimal64 x);
_Decimal128 nextupd128(_Decimal128 x);
_Decimal32 nextdownd32(_Decimal32 x);
_Decimal64 nextdownd64(_Decimal64 x);
_Decimal128 nextdownd128(_Decimal128 x);

```

```

int canonicalized32(_Decimal32 *CX, const _Decimal32 *x);
int canonicalized64(_Decimal64 *CX, const _Decimal64 *x);
int canonicalized128(_Decimal128 *CX, const _Decimal128 *x);
_Decimal32 fdimd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fdimd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fdimd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fmaxd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaxd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaxd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fmind32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmind64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmind128(_Decimal128 x, _Decimal128 y);
_Decimal32 fmaximumd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaximumd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaximumd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fminimumd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fminimumd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fminimumd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fmaximum_magd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaximum_magd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaximum_magd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fminimum_magd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fminimum_magd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fminimum_magd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fmaximum_numd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaximum_numd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaximum_numd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fminimum_numd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fminimum_numd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fminimum_numd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fmaximum_mag_numd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaximum_mag_numd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaximum_mag_numd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fminimum_mag_numd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fminimum_mag_numd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fminimum_mag_numd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fmad32(_Decimal32 x, _Decimal32 y, _Decimal32 z);
_Decimal64 fmad64(_Decimal64 x, _Decimal64 y, _Decimal64 z);
_Decimal128 fmad128(_Decimal128 x, _Decimal128 y, _Decimal128 z);
_Decimal32 d32addd64(_Decimal64 x, _Decimal64 y);
_Decimal32 d32addd128(_Decimal128 x, _Decimal128 y);
_Decimal64 d64addd128(_Decimal128 x, _Decimal128 y);
_Decimal32 d32subd64(_Decimal64 x, _Decimal64 y);
_Decimal32 d32subd128(_Decimal128 x, _Decimal128 y);
_Decimal64 d64subd128(_Decimal128 x, _Decimal128 y);
_Decimal32 d32muld64(_Decimal64 x, _Decimal64 y);
_Decimal32 d32muld128(_Decimal128 x, _Decimal128 y);
_Decimal64 d64muld128(_Decimal128 x, _Decimal128 y);
_Decimal32 d32divd64(_Decimal64 x, _Decimal64 y);
_Decimal32 d32divd128(_Decimal128 x, _Decimal128 y);
_Decimal64 d64divd128(_Decimal128 x, _Decimal128 y);
_Decimal32 d32fmad64(_Decimal64 x, _Decimal64 y, _Decimal64 z);
_Decimal32 d32fmad128(_Decimal128 x, _Decimal128 y, _Decimal128 z);
_Decimal64 d64fmad128(_Decimal128 x, _Decimal128 y, _Decimal128 z);
_Decimal32 d32sqrt64(_Decimal64 x);
_Decimal32 d32sqrt128(_Decimal128 x);
_Decimal64 d64sqrt128(_Decimal128 x);
_Decimal32 quantized32(_Decimal32 x, _Decimal32 y);
_Decimal64 quantized64(_Decimal64 x, _Decimal64 y);
_Decimal128 quantized128(_Decimal128 x, _Decimal128 y);
bool samequantumd32(_Decimal32 x, _Decimal32 y);

```



```

bool samequantumd64(_Decimal64 x, _Decimal64 y);
bool samequantumd128(_Decimal128 x, _Decimal128 y);
_Decimal32 quantumd32(_Decimal32 x);
_Decimal64 quantumd64(_Decimal64 x);
_Decimal128 quantumd128(_Decimal128 x);
long long int llquantexpd32(_Decimal32 x);
long long int llquantexpd64(_Decimal64 x);
long long int llquantexpd128(_Decimal128 x);
void encodedecd32(unsigned char encptr[restrict static 4],
    const _Decimal32 * restrict xptr);
void encodedecd64(unsigned char encptr[restrict static 8],
    const _Decimal64 * restrict xptr);
void encodedecd128(unsigned char encptr[restrict static 16],
    const _Decimal128 * restrict xptr);
void decodedecd32(_Decimal32 * restrict xptr,
    const unsigned char encptr[restrict static 4]);
void decodedecd64(_Decimal64 * restrict xptr,
    const unsigned char encptr[restrict static 8]);
void decodedecd128(_Decimal128 * restrict xptr,
    const unsigned char encptr[restrict static 16]);
void encodebind32(unsigned char encptr[restrict static 4],
    const _Decimal32 * restrict xptr);
void encodebind64(unsigned char encptr[restrict static 8],
    const _Decimal64 * restrict xptr);
void encodebind128(unsigned char encptr[restrict static 16],
    const _Decimal128 * restrict xptr);
void decodebind32(_Decimal32 * restrict xptr,
    const unsigned char encptr[restrict static 4]);
void decodebind64(_Decimal64 * restrict xptr,
    const unsigned char encptr[restrict static 8]);
void decodebind128(_Decimal128 * restrict xptr,
    const unsigned char encptr[restrict static 16]);

```

Only if the implementation defines `__STDC_IEC_60559_BFP__` or `__STDC_IEC_559__` and additionally the user code defines `__STDC_WANT_IEC_60559_EXT__` before any inclusion of `<math.h>`:

```

int totalorder(const double *x, const double *y);
int totalorderf(const float *x, const float *y);
int totalorderl(const long double *x, const long double *y);
int totalordermag(const double *x, const double *y);
int totalordermagf(const float *x, const float *y);
int totalordermagl(const long double *x, const long double *y);
double getpayload(const double *x);
float getpayloadf(const float *x);
long double getpayloadl(const long double *x);
int setpayload(double *res, double pl);
int setpayloadf(float *res, float pl);
int setpayloadl(long double *res, long double pl);
int setpayloadsig(double *res, double pl);
int setpayloadsigf(float *res, float pl);
int setpayloadsigl(long double *res, long double pl);

```

Only if the implementation defines `__STDC_IEC_60559_DFP__` and additionally the user code defines `__STDC_WANT_IEC_60559_EXT__` before any inclusion of `<math.h>`:

`_Decimal32_t` `_Decimal64_t` `HUGE_VAL_D32` `HUGE_VAL_D64` `HUGE_VAL_D128`

```

int totalorderd32(const _Decimal32 *x, const _Decimal32 *y);
int totalorderd64(const _Decimal64 *x, const _Decimal64 *y);
int totalorderd128(const _Decimal128 *x, const _Decimal128 *y);

```



```

int totalordermagd32(const _Decimal32 *x, const _Decimal32 *y);
int totalordermagd64(const _Decimal64 *x, const _Decimal64 *y);
int totalordermagd128(const _Decimal128 *x, const _Decimal128 *y);
_Decimal32 getpayloadadd32(const _Decimal32 *x);
_Decimal64 getpayloadadd64(const _Decimal64 *x);
_Decimal128 getpayloadadd128(const _Decimal128 *x);
int setpayloadadd32(_Decimal32 *res, _Decimal32 pl);
int setpayloadadd64(_Decimal64 *res, _Decimal64 pl);
int setpayloadadd128(_Decimal128 *res, _Decimal128 pl);
int setpayloadsigd32(_Decimal32 *res, _Decimal32 pl);
int setpayloadsigd64(_Decimal64 *res, _Decimal64 pl);
int setpayloadsigd128(_Decimal128 *res, _Decimal128 pl);

```

Only if the implementation defines `__STDC_IEC_60559_TYPES__` and additionally the user code defines `__STDC_WANT_IEC_60559_TYPES_EXT__` before any inclusion of `<math.h>`:

<code>long_double_t</code>	<code>FP_FAST_FMSUBFNX</code>	<code>FP_FAST_DMDIVDNX</code>
<code>_FloatN_t</code>	<code>FP_FAST_FMXSUBFN</code>	<code>FP_FAST_DMxDIVDN</code>
<code>_DecimalN_t</code>	<code>FP_FAST_FMXSUBFNX</code>	<code>FP_FAST_DMxDIVDNX</code>
<code>HUGE_VAL_FN</code>	<code>FP_FAST_DMSUBDN</code>	<code>FP_FAST_FMFMAFN</code>
<code>HUGE_VAL_DN</code>	<code>FP_FAST_DMSUBDNX</code>	<code>FP_FAST_FMFMAFNX</code>
<code>HUGE_VAL_FNX</code>	<code>FP_FAST_DMXSUBDN</code>	<code>FP_FAST_FMXFMAFN</code>
<code>HUGE_VAL_DNX</code>	<code>FP_FAST_DMXSUBDNX</code>	<code>FP_FAST_FMXFMAFNX</code>
<code>FP_FAST_FMAFN</code>	<code>FP_FAST_FMMULFN</code>	<code>FP_FAST_DMFMADN</code>
<code>FP_FAST_FMAFN</code>	<code>FP_FAST_FMMULFNX</code>	<code>FP_FAST_DMFMADNX</code>
<code>FP_FAST_FMAFNX</code>	<code>FP_FAST_FMXMULFN</code>	<code>FP_FAST_DMXFMAFN</code>
<code>FP_FAST_FMAFNX</code>	<code>FP_FAST_FMXMULFNX</code>	<code>FP_FAST_DMXFMAFNX</code>
<code>FP_FAST_FMAFDN</code>	<code>FP_FAST_DMMULDN</code>	<code>FP_FAST_FMSQRTFN</code>
<code>FP_FAST_FMAFDNX</code>	<code>FP_FAST_DMMULDNX</code>	<code>FP_FAST_FMSQRTFNX</code>
<code>FP_FAST_FMXADFN</code>	<code>FP_FAST_DMXMULDN</code>	<code>FP_FAST_FMXSQRTFN</code>
<code>FP_FAST_FMXADFNX</code>	<code>FP_FAST_DMXMULDNX</code>	<code>FP_FAST_FMXSQRTFNX</code>
<code>FP_FAST_DMADDDN</code>	<code>FP_FAST_FMDIVFN</code>	<code>FP_FAST_DMSQRTDN</code>
<code>FP_FAST_DMADDDNX</code>	<code>FP_FAST_FMDIVFNX</code>	<code>FP_FAST_DMSQRTDNX</code>
<code>FP_FAST_DMxADDDN</code>	<code>FP_FAST_FMXDIVFN</code>	<code>FP_FAST_DMXSQRTDN</code>
<code>FP_FAST_DMxADDDNX</code>	<code>FP_FAST_FMXDIVFNX</code>	<code>FP_FAST_DMXSQRTDNX</code>
<code>FP_FAST_FMSUBFN</code>	<code>FP_FAST_DMDIVDN</code>	

```

_FloatN acosfN(_FloatN x);
_FloatNx acosfNx(_FloatNx x);
_DecimalN acosdN(_DecimalN x);
_DecimalNx acosdNx(_DecimalNx x);
_FloatN asinfN(_FloatN x);
_FloatNx asinfNx(_FloatNx x);
_DecimalN asindN(_DecimalN x);
_DecimalNx asindNx(_DecimalNx x);
_FloatN atanfN(_FloatN x);
_FloatNx atanfNx(_FloatNx x);
_DecimalN atandN(_DecimalN x);
_DecimalNx atandNx(_DecimalNx x);
_FloatN atan2fN(_FloatN y, _FloatN x);
_FloatNx atan2fNx(_FloatNx y, _FloatNx x);
_DecimalN atan2dN(_DecimalN y, _DecimalN x);
_DecimalNx atan2dNx(_DecimalNx y, _DecimalNx x);
_FloatN cosfN(_FloatN x);
_FloatNx cosfNx(_FloatNx x);
_DecimalN cosdN(_DecimalN x);
_DecimalNx cosdNx(_DecimalNx x);
_FloatN sinfN(_FloatN x);

```

```

_FloatNx sinfNx(_FloatNx x);
_DecimalN sindN(_DecimalN x);
_DecimalNx sindNx(_DecimalNx x);
_FloatN tanfN(_FloatN x);
_FloatNx tanfNx(_FloatNx x);
_DecimalN tandN(_DecimalN x);
_DecimalNx tandNx(_DecimalNx x);
_FloatN acospiFN(_FloatN x);
_FloatNx acospiFNx(_FloatNx x);
_DecimalN acospiDN(_DecimalN x);
_DecimalNx acospiDNx(_DecimalNx x);
_FloatN asinpiFN(_FloatN x);
_FloatNx asinpiFNx(_FloatNx x);
_DecimalN asinpiDN(_DecimalN x);
_DecimalNx asinpiDNx(_DecimalNx x);
_FloatN atanpiFN(_FloatN x);
_FloatNx atanpiFNx(_FloatNx x);
_DecimalN atanpiDN(_DecimalN x);
_DecimalNx atanpiDNx(_DecimalNx x);
_FloatN atan2piFN(_FloatN y, _FloatN x);
_FloatNx atan2piFNx(_FloatNx y, _FloatNx x);
_DecimalN atan2piDN(_DecimalN y, _DecimalN x);
_DecimalNx atan2piDNx(_DecimalNx y, _DecimalNx x);
_FloatN cospiFN(_FloatN x);
_FloatNx cospiFNx(_FloatNx x);
_DecimalN cospiDN(_DecimalN x);
_DecimalNx cospiDNx(_DecimalNx x);
_FloatN sinpiFN(_FloatN x);
_FloatNx sinpiFNx(_FloatNx x);
_DecimalN sinpiDN(_DecimalN x);
_DecimalNx sinpiDNx(_DecimalNx x);
_FloatN tanpiFN(_FloatN x);
_FloatNx tanpiFNx(_FloatNx x);
_DecimalN tanpiDN(_DecimalN x);
_DecimalNx tanpiDNx(_DecimalNx x);
_FloatN acoshFN(_FloatN x);
_FloatNx acoshFNx(_FloatNx x);
_DecimalN acoshDN(_DecimalN x);
_DecimalNx acoshDNx(_DecimalNx x);
_FloatN asinhFN(_FloatN x);
_FloatNx asinhFNx(_FloatNx x);
_DecimalN asinhDN(_DecimalN x);
_DecimalNx asinhDNx(_DecimalNx x);
_FloatN atanhFN(_FloatN x);
_FloatNx atanhFNx(_FloatNx x);
_DecimalN atanhDN(_DecimalN x);
_DecimalNx atanhDNx(_DecimalNx x);
_FloatN coshFN(_FloatN x);
_FloatNx coshFNx(_FloatNx x);
_DecimalN coshDN(_DecimalN x);
_DecimalNx coshDNx(_DecimalNx x);
_FloatN sinhFN(_FloatN x);
_FloatNx sinhFNx(_FloatNx x);
_DecimalN sinhDN(_DecimalN x);
_DecimalNx sinhDNx(_DecimalNx x);
_FloatN tanhFN(_FloatN x);
_FloatNx tanhFNx(_FloatNx x);
_DecimalN tanhDN(_DecimalN x);
_DecimalNx tanhDNx(_DecimalNx x);
_FloatN expFN(_FloatN x);
_FloatNx expFNx(_FloatNx x);

```

```

_DecimalN expdN(_DecimalN x);
_DecimalNx expdNx(_DecimalNx x);
_FloatN expl0fN(_FloatN x);
_FloatNx expl0fNx(_FloatNx x);
_DecimalN expl0dN(_DecimalN x);
_DecimalNx expl0dNx(_DecimalNx x);
_FloatN expl0m1fN(_FloatN x);
_FloatNx expl0m1fNx(_FloatNx x);
_DecimalN expl0m1dN(_DecimalN x);
_DecimalNx expl0m1dNx(_DecimalNx x);
_FloatN exp2fN(_FloatN x);
_FloatNx exp2fNx(_FloatNx x);
_DecimalN exp2dN(_DecimalN x);
_DecimalNx exp2dNx(_DecimalNx x);
_FloatN exp2m1fN(_FloatN x);
_FloatNx exp2m1fNx(_FloatNx x);
_DecimalN exp2m1dN(_DecimalN x);
_DecimalNx exp2m1dNx(_DecimalNx x);
_FloatN expm1fN(_FloatN x);
_FloatNx expm1fNx(_FloatNx x);
_DecimalN expm1dN(_DecimalN x);
_DecimalNx expm1dNx(_DecimalNx x);
_FloatN frexpfN(_FloatN value, int *exp);
_FloatNx frexpfNx(_FloatNx value, int *exp);
_DecimalN frexpdN(_DecimalN value, int *exp);
_DecimalNx frexpdNx(_DecimalNx value, int *exp);
int ilogbfN(_FloatN x);
int ilogbfNx(_FloatNx x);
int ilogbdN(_DecimalN x);
int ilogbdNx(_DecimalNx x);
_FloatN ldexpfN(_FloatN value, int exp);
_FloatNx ldexpfNx(_FloatNx value, int exp);
_DecimalN ldexpdN(_DecimalN value, int exp);
_DecimalNx ldexpdNx(_DecimalNx value, int exp);
long int llogbfN(_FloatN x);
long int llogbfNx(_FloatNx x);
long int llogbdN(_DecimalN x);
long int llogbdNx(_DecimalNx x);
_FloatN logfN(_FloatN x);
_FloatNx logfNx(_FloatNx x);
_DecimalN logdN(_DecimalN x);
_DecimalNx logdNx(_DecimalNx x);
_FloatN log10fN(_FloatN x);
_FloatNx log10fNx(_FloatNx x);
_DecimalN log10dN(_DecimalN x);
_DecimalNx log10dNx(_DecimalNx x);
_FloatN log10p1fN(_FloatN x);
_FloatNx log10p1fNx(_FloatNx x);
_DecimalN log10p1dN(_DecimalN x);
_DecimalNx log10p1dNx(_DecimalNx x);
_FloatN log1pfN(_FloatN x);
_FloatNx log1pfNx(_FloatNx x);
_FloatN log1p1fN(_FloatN x);
_FloatNx log1p1fNx(_FloatNx x);
_DecimalN log1pdN(_DecimalN x);
_DecimalNx log1pdNx(_DecimalNx x);
_DecimalN logp1dN(_DecimalN x);
_DecimalNx logp1dNx(_DecimalNx x);
_FloatN log2fN(_FloatN x);
_FloatNx log2fNx(_FloatNx x);
_DecimalN log2dN(_DecimalN x);

```

```

_DecimalNx log2dNx(_DecimalNx x);
_FloatN log2plfN(_FloatN x);
_FloatNx log2plfNx(_FloatNx x);
_DecimalW log2pldN(_DecimalW x);
_DecimalNx log2pldNx(_DecimalNx x);
_FloatN logbfN(_FloatN x);
_FloatNx logbfNx(_FloatNx x);
_DecimalW logbdN(_DecimalW x);
_DecimalNx logbdNx(_DecimalNx x);
_FloatN modffN(_FloatN x, _FloatN *iptr);
_FloatNx modffNx(_FloatNx x, _FloatNx *iptr);
_DecimalW modfdN(_DecimalW x, _DecimalW *iptr);
_DecimalNx modfdNx(_DecimalNx x, _DecimalNx *iptr);
_FloatN scalbnfN(_FloatN value, int exp);
_FloatNx scalbnfNx(_FloatNx value, int exp);
_DecimalW scalbndN(_DecimalW value, int exp);
_DecimalNx scalbndNx(_DecimalNx value, int exp);
_FloatN scalblnfN(_FloatN value, long int exp);
_FloatNx scalblnfNx(_FloatNx value, long int exp);
_DecimalW scalblndN(_DecimalW value, long int exp);
_DecimalNx scalblndNx(_DecimalNx value, long int exp);
_FloatN cbrtfN(_FloatN x);
_FloatNx cbrtfNx(_FloatNx x);
_DecimalW cbrtdN(_DecimalW x);
_DecimalNx cbrtdNx(_DecimalNx x);
_FloatN compoundnfN(_FloatN x, long long int n);
_FloatNx compoundnfNx(_FloatNx x, long long int n);
_DecimalW compoundndN(_DecimalW x, long long int n);
_DecimalNx compoundndNx(_DecimalNx x, long long int n);
_FloatN fabsfN(_FloatN x);
_FloatNx fabsfNx(_FloatNx x);
_DecimalW fabsdN(_DecimalW x);
_DecimalNx fabsdNx(_DecimalNx x);
_FloatN hypotfN(_FloatN x, _FloatN y);
_FloatNx hypotfNx(_FloatNx x, _FloatNx y);
_DecimalW hypotdN(_DecimalW x, _DecimalW y);
_DecimalNx hypotdNx(_DecimalNx x, _DecimalNx y);
_FloatN powfN(_FloatN x, _FloatN y);
_FloatNx powfNx(_FloatNx x, _FloatNx y);
_DecimalW powdN(_DecimalW x, _DecimalW y);
_DecimalNx powdNx(_DecimalNx x, _DecimalNx y);
_FloatN pownfN(_FloatN x, long long int n);
_FloatNx pownfNx(_FloatNx x, long long int n);
_DecimalW powndN(_DecimalW x, long long int n);
_DecimalNx powndNx(_DecimalNx x, long long int n);
_FloatN powrfN(_FloatN x, _FloatN y);
_FloatNx powrfNx(_FloatNx x, _FloatNx y);
_DecimalW powrdN(_DecimalW x, _DecimalW y);
_DecimalNx powrdNx(_DecimalNx x, _DecimalNx y);
_FloatN rootnfN(_FloatN x, long long int n);
_FloatNx rootnfNx(_FloatNx x, long long int n);
_DecimalW rootndN(_DecimalW x, long long int n);
_DecimalNx rootndNx(_DecimalNx x, long long int n);
_FloatN rsqrtfN(_FloatN x);
_FloatNx rsqrtfNx(_FloatNx x);
_DecimalW rsqrtndN(_DecimalW x);
_DecimalNx rsqrtndNx(_DecimalNx x);
_FloatN sqrtfN(_FloatN x);
_FloatNx sqrtfNx(_FloatNx x);
_DecimalW sqrtdN(_DecimalW x);
_DecimalNx sqrtdNx(_DecimalNx x);

```

```

_FloatN erffN(_FloatN x);
_FloatNx erffNx(_FloatNx x);
_DecimalN erfdN(_DecimalN x);
_DecimalNx erfdNx(_DecimalNx x);
_FloatN erfcfN(_FloatN x);
_FloatNx erfcfNx(_FloatNx x);
_DecimalN erfcdN(_DecimalN x);
_DecimalNx erfcdNx(_DecimalNx x);
_FloatN lgammafN(_FloatN x);
_FloatNx lgammafNx(_FloatNx x);
_DecimalN lgammadN(_DecimalN x);
_DecimalNx lgammadNx(_DecimalNx x);
_FloatN tgammafN(_FloatN x);
_FloatNx tgammafNx(_FloatNx x);
_DecimalN tgammaN(_DecimalN x);
_DecimalNx tgammaNx(_DecimalNx x);
_FloatN ceilfN(_FloatN x);
_FloatNx ceilfNx(_FloatNx x);
_DecimalN ceildN(_DecimalN x);
_DecimalNx ceildNx(_DecimalNx x);
_FloatN floorfN(_FloatN x);
_FloatNx floorfNx(_FloatNx x);
_DecimalN floordN(_DecimalN x);
_DecimalNx floordNx(_DecimalNx x);
_FloatN nearbyintfN(_FloatN x);
_FloatNx nearbyintfNx(_FloatNx x);
_DecimalN nearbyintdN(_DecimalN x);
_DecimalNx nearbyintdNx(_DecimalNx x);
_FloatN rintfN(_FloatN x);
_FloatNx rintfNx(_FloatNx x);
_DecimalN rintdN(_DecimalN x);
_DecimalNx rintdNx(_DecimalNx x);
long int lrintfN(_FloatN x);
long int lrintfNx(_FloatNx x);
long int lrintdN(_DecimalN x);
long int lrintdNx(_DecimalNx x);
long long int llrintfN(_FloatN x);
long long int llrintfNx(_FloatNx x);
long long int llrintdN(_DecimalN x);
long long int llrintdNx(_DecimalNx x);
_FloatN roundfN(_FloatN x);
_FloatNx roundfNx(_FloatNx x);
_DecimalN rounddN(_DecimalN x);
_DecimalNx rounddNx(_DecimalNx x);
long int lroundfN(_FloatN x);
long int lroundfNx(_FloatNx x);
long int lrounddN(_DecimalN x);
long int lrounddNx(_DecimalNx x);
long long int llroundfN(_FloatN x);
long long int llroundfNx(_FloatNx x);
long long int llrounddN(_DecimalN x);
long long int llrounddNx(_DecimalNx x);
_FloatN roundevenfN(_FloatN x);
_FloatNx roundevenfNx(_FloatNx x);
_DecimalN roundevendN(_DecimalN x);
_DecimalNx roundevendNx(_DecimalNx x);
_FloatN truncfN(_FloatN x);
_FloatNx truncfNx(_FloatNx x);
_DecimalN truncdN(_DecimalN x);
_DecimalNx truncdNx(_DecimalNx x);
_FloatN frompfpN(_FloatN x, int rnd, unsigned int width);

```

```

_FloatNx fromfpfNx(_FloatNx x, int rnd, unsigned int width);
_DecimalN fromfpdN(_DecimalN x, int rnd, unsigned int width);
_DecimalNx fromfpdNx(_DecimalNx x, int rnd, unsigned int width);
_FloatN ufromfpfN(_FloatN x, int rnd, unsigned int width);
_FloatNx ufromfpfNx(_FloatNx x, int rnd, unsigned int width);
_DecimalN ufromfpdN(_DecimalN x, int rnd, unsigned int width);
_DecimalNx ufromfpdNx(_DecimalNx x, int rnd, unsigned int width);
_FloatN fromfpxfN(_FloatN x, int rnd, unsigned int width);
_FloatNx fromfpxfNx(_FloatNx x, int rnd, unsigned int width);
_DecimalN fromfpxdN(_DecimalN x, int rnd, unsigned int width);
_DecimalNx fromfpxdNx(_DecimalNx x, int rnd, unsigned int width);
_FloatN ufromfpxfN(_FloatN x, int rnd, unsigned int width);
_FloatNx ufromfpxfNx(_FloatNx x, int rnd, unsigned int width);
_DecimalN ufromfpxdN(_DecimalN x, int rnd, unsigned int width);
_DecimalNx ufromfpxdNx(_DecimalNx x, int rnd, unsigned int width);
_FloatN fmodfN(_FloatN x, _FloatN y);
_FloatNx fmodfNx(_FloatNx x, _FloatNx y);
_DecimalN fmoddN(_DecimalN x, _DecimalN y);
_DecimalNx fmoddNx(_DecimalNx x, _DecimalNx y);
_FloatN remainderfN(_FloatN x, _FloatN y);
_FloatNx remainderfNx(_FloatNx x, _FloatNx y);
_DecimalN remainderdN(_DecimalN x, _DecimalN y);
_DecimalNx remainderdNx(_DecimalNx x, _DecimalNx y);
_FloatN remquofN(_FloatN x, _FloatN y, int *quo);
_FloatNx remquofNx(_FloatNx x, _FloatNx y, int *quo);
_FloatN copysignfN(_FloatN x, _FloatN y);
_FloatNx copysignfNx(_FloatNx x, _FloatNx y);
_DecimalN copysigndN(_DecimalN x, _DecimalN y);
_DecimalNx copysigndNx(_DecimalNx x, _DecimalNx y);
_FloatN nanfN(const char *tagp);
_FloatNx nanfNx(const char *tagp);
_DecimalN nandN(const char *tagp);
_DecimalNx nandNx(const char *tagp);
_FloatN nextafterfN(_FloatN x, _FloatN y);
_FloatNx nextafterfNx(_FloatNx x, _FloatNx y);
_DecimalN nextafterdN(_DecimalN x, _DecimalN y);
_DecimalNx nextafterdNx(_DecimalNx x, _DecimalNx y);
_FloatN nextupfN(_FloatN x);
_FloatNx nextupfNx(_FloatNx x);
_DecimalN nextupdN(_DecimalN x);
_DecimalNx nextupdNx(_DecimalNx x);
_FloatN nextdownfN(_FloatN x);
_FloatNx nextdownfNx(_FloatNx x);
_DecimalN nextdowndN(_DecimalN x);
_DecimalNx nextdowndNx(_DecimalNx x);
int canonicalizefN(_FloatN *cx, const _FloatN *x);
int canonicalizefNx(_FloatNx *cx, const _FloatNx *x);
int canonicalizedN(_DecimalN *cx, const _DecimalN *x);
int canonicalizedNx(_DecimalNx *cx, const _DecimalNx *x);
_FloatN fdimfN(_FloatN x, _FloatN y);
_FloatNx fdimfNx(_FloatNx x, _FloatNx y);
_DecimalN fdimdN(_DecimalN x, _DecimalN y);
_DecimalNx fdimdNx(_DecimalNx x, _DecimalNx y);
_FloatN fmaximumfN(_FloatN x, _FloatN y);
_FloatNx fmaximumfNx(_FloatNx x, _FloatNx y);
_DecimalN fmaximumdN(_DecimalN x, _DecimalN y);
_DecimalNx fmaximumdNx(_DecimalNx x, _DecimalNx y);
_FloatN fminimumfN(_FloatN x, _FloatN y);
_FloatNx fminimumfNx(_FloatNx x, _FloatNx y);
_DecimalN fminimumdN(_DecimalN x, _DecimalN y);
_DecimalNx fminimumdNx(_DecimalNx x, _DecimalNx y);

```



```

_FloatN fmaximum_magfN(_FloatN x, _FloatN y);
_FloatNx fmaximum_magfNx(_FloatNx x, _FloatNx y);
_DecimalN fmaximum_magdN(_DecimalN x, _DecimalN y);
_DecimalNx fmaximum_magdNx(_DecimalNx x, _DecimalNx y);
_FloatN fminimum_magfN(_FloatN x, _FloatN y);
_FloatNx fminimum_magfNx(_FloatNx x, _FloatNx y);
_DecimalN fminimum_magdN(_DecimalN x, _DecimalN y);
_DecimalNx fminimum_magdNx(_DecimalNx x, _DecimalNx y);
_FloatN fmaximum_numfN(_FloatN x, _FloatN y);
_FloatNx fmaximum_numfNx(_FloatNx x, _FloatNx y);
_DecimalN fmaximum_numdN(_DecimalN x, _DecimalN y);
_DecimalNx fmaximum_numdNx(_DecimalNx x, _DecimalNx y);
_FloatN fminimum_numfN(_FloatN x, _FloatN y);
_FloatNx fminimum_numfNx(_FloatNx x, _FloatNx y);
_DecimalN fminimum_numdN(_DecimalN x, _DecimalN y);
_DecimalNx fminimum_numdNx(_DecimalNx x, _DecimalNx y);
_FloatN fmaximum_mag_numfN(_FloatN x, _FloatN y);
_FloatNx fmaximum_mag_numfNx(_FloatNx x, _FloatNx y);
_DecimalN fmaximum_mag_numdN(_DecimalN x, _DecimalN y);
_DecimalNx fmaximum_mag_numdNx(_DecimalNx x, _DecimalNx y);
_FloatN fminimum_mag_numfN(_FloatN x, _FloatN y);
_FloatNx fminimum_mag_numfNx(_FloatNx x, _FloatNx y);
_DecimalN fminimum_mag_numdN(_DecimalN x, _DecimalN y);
_DecimalNx fminimum_mag_numdNx(_DecimalNx x, _DecimalNx y);
_FloatN fmafN(_FloatN x, _FloatN y, _FloatN z);
_FloatNx fmafNx(_FloatNx x, _FloatNx y, _FloatNx z);
_DecimalN fmadN(_DecimalN x, _DecimalN y, _DecimalN z);
_DecimalNx fmadNx(_DecimalNx x, _DecimalNx y, _DecimalNx z);
_FloatM fMaddfN(_FloatN x, _FloatN y); // M < N
_FloatM fMaddfNx(_FloatNx x, _FloatNx y); // M ≤ N
_FloatMx fMxaddfN(_FloatN x, _FloatN y); // M < N
_FloatMx fMxaddfNx(_FloatNx x, _FloatNx y); // M < N
_DecimalM dMaddN(_DecimalN x, _DecimalN y); // M < N
_DecimalM dMaddNx(_DecimalNx x, _DecimalNx y); // M ≤ N
_DecimalMx dMxaddN(_DecimalN x, _DecimalN y); // M < N
_DecimalMx dMxaddNx(_DecimalNx x, _DecimalNx y); // M < N
_FloatM fMsubfN(_FloatN x, _FloatN y); // M < N
_FloatM fMsubfNx(_FloatNx x, _FloatNx y); // M ≤ N
_FloatMx fMxsubfN(_FloatN x, _FloatN y); // M < N
_FloatMx fMxsubfNx(_FloatNx x, _FloatNx y); // M < N
_DecimalM dMsubdN(_DecimalN x, _DecimalN y); // M < N
_DecimalM dMsubdNx(_DecimalNx x, _DecimalNx y); // M ≤ N
_DecimalMx dMxsubdN(_DecimalN x, _DecimalN y); // M < N
_DecimalMx dMxsubdNx(_DecimalNx x, _DecimalNx y); // M < N
_FloatM fMmulfN(_FloatN x, _FloatN y); // M < N
_FloatM fMmulfNx(_FloatNx x, _FloatNx y); // M ≤ N
_FloatMx fMxmulfN(_FloatN x, _FloatN y); // M < N
_FloatMx fMxmulfNx(_FloatNx x, _FloatNx y); // M < N
_DecimalM dMmuldN(_DecimalN x, _DecimalN y); // M < N
_DecimalM dMmuldNx(_DecimalNx x, _DecimalNx y); // M ≤ N
_DecimalMx dMxmuldN(_DecimalN x, _DecimalN y); // M < N
_DecimalMx dMxmuldNx(_DecimalNx x, _DecimalNx y); // M < N
_FloatM fMdivfN(_FloatN x, _FloatN y); // M < N
_FloatM fMdivfNx(_FloatNx x, _FloatNx y); // M ≤ N
_FloatMx fMxdivfN(_FloatN x, _FloatN y); // M < N
_FloatMx fMxdivfNx(_FloatNx x, _FloatNx y); // M < N
_DecimalM dMdivdN(_DecimalN x, _DecimalN y); // M < N
_DecimalM dMdivdNx(_DecimalNx x, _DecimalNx y); // M ≤ N
_DecimalMx dMxdivdN(_DecimalN x, _DecimalN y); // M < N
_DecimalMx dMxdivdNx(_DecimalNx x, _DecimalNx y); // M < N
_FloatM fMfmfN(_FloatN x, _FloatN y, _FloatN z); // M < N

```



```

_FloatM fmfmafN(_FloatNx x, _FloatNx y, _FloatNx z); //  $M \leq N$ 
_FloatMx fMxfmafN(_FloatN x, _FloatN y, _FloatN z); //  $M < N$ 
_FloatMx fMxfmafNx(_FloatNx x, _FloatNx y, _FloatNx z); //  $M < N$ 
_DecimalM dMfmadN(_DecimalN x, _DecimalN y, _DecimalN z); //  $M < N$ 
_DecimalM dMfmadNx(_DecimalNx x, _DecimalNx y, _DecimalNx z); //  $M \leq N$ 
_DecimalMx dMxfmadN(_DecimalN x, _DecimalN y, _DecimalN z); //  $M < N$ 
_DecimalMx dMxfmadNx(_DecimalNx x, _DecimalNx y, _DecimalNx z); //  $M < N$ 
_FloatM fmsqrtfN(_FloatN x); //  $M < N$ 
_FloatM fmsqrtfNx(_FloatNx x); //  $M \leq N$ 
_FloatMx fMxsqrtfN(_FloatN x); //  $M < N$ 
_FloatMx fMxsqrtfNx(_FloatNx x); //  $M < N$ 
_DecimalM dMsqrtdN(_DecimalN x); //  $M < N$ 
_DecimalM dMsqrtdNx(_DecimalNx x); //  $M \leq N$ 
_DecimalMx dMxsqrtdN(_DecimalN x); //  $M < N$ 
_DecimalMx dMxsqrtdNx(_DecimalNx x); //  $M < N$ 
_DecimalN quantizedN(_DecimalN x, _DecimalN y);
_DecimalNx quantizedNx(_DecimalNx x, _DecimalNx y);
bool samequantumdN(_DecimalN x, _DecimalN y);
bool samequantumdNx(_DecimalNx x, _DecimalNx y);
_DecimalN quantumdN(_DecimalN x);
_DecimalNx quantumdNx(_DecimalNx x);
long long int llquantexpdN(_DecimalN x);
long long int llquantexpdNx(_DecimalNx x);
void encodedecdN(unsigned char * restrict encptr,
    const _DecimalN * restrict xptr);
void decodedecdN(_DecimalN * restrict xptr,
    const unsigned char * restrict encptr);
void encodebindN(unsigned char * restrict encptr,
    const _DecimalN * restrict xptr);
void decodebindN(_DecimalN * restrict xptr,
    const unsigned char * restrict encptr);
int totalorderfN(const _FloatN *x, const _FloatN *y);
int totalorderfNx(const _FloatNx *x, const _FloatNx *y);
int totalorderdN(const _DecimalN *x, const _DecimalN *y);
int totalorderdNx(const _DecimalNx *x, const _DecimalNx *y);
int totalordermagfN(const _FloatN *x, const _FloatN *y);
int totalordermagfNx(const _FloatNx *x, const _FloatNx *y);
int totalordermagdN(const _DecimalN *x, const _DecimalN *y);
int totalordermagdNx(const _DecimalNx *x, const _DecimalNx *y);
_FloatN getpayloadfN(const _FloatN *x);
_FloatNx getpayloadfNx(const _FloatNx *x);
_DecimalN getpayloaddN(const _DecimalN *x);
_DecimalNx getpayloaddNx(const _DecimalNx *x);
int setpayloadfN(_FloatN *res, _FloatN pl);
int setpayloadfNx(_FloatNx *res, _FloatNx pl);
int setpayloaddN(_DecimalN *res, _DecimalN pl);
int setpayloaddNx(_DecimalNx *res, _DecimalNx pl);
int setpayloadsigfN(_FloatN *res, _FloatN pl);
int setpayloadsigfNx(_FloatNx *res, _FloatNx pl);
int setpayloadsigdN(_DecimalN *res, _DecimalN pl);
int setpayloadsigdNx(_DecimalNx *res, _DecimalNx pl);
void encodefN(unsigned char encptr[restrict static N/8],
    const _FloatN * restrict xptr);
void decodefN(_FloatN * restrict xptr,
    const unsigned char encptr[restrict static N/8]);
void fMencfN(unsigned char encMptr[restrict static M/8],
    const unsigned char encNptr[restrict static N/8]);
void dMencdecN(unsigned char encMptr[restrict static M/8],
    const unsigned char encNptr[restrict static N/8]);
void dMencbindN(unsigned char encMptr[restrict static M/8],
    const unsigned char encNptr[restrict static N/8]);

```

B.12 Non-local jumps <setjmp.h>

__STDC_VERSION_SETJMP_H__ jmp_buf

```
int setjmp(jmp_buf env);
[[noreturn]] void longjmp(jmp_buf env, int val);
```

B.13 Signal handling <signal.h>

sig_atomic_t	SIG_IGN	SIGILL	SIGTERM
SIG_DFL	SIGABRT	SIGINT	
SIG_ERR	SIGFPE	SIGSEGV	

```
void (*signal(int sig, void (*func)(int)))(int);
int raise(int sig);
```

B.14 Alignment <stdalign.h>

The header <stdalign.h> provides no content.

B.15 Variable arguments <stdarg.h>

va_list __STDC_VERSION_STDARG_H__

```
type va_arg(va_list ap, type);
void va_copy(va_list dest, va_list src);
void va_end(va_list ap);
void va_start(va_list ap, ...);
```

B.16 Atomics <stdatomic.h>

__STDC_NO_ATOMICS__	atomic_char
__STDC_VERSION_STDATOMIC_H__	atomic_schar
ATOMIC_BOOL_LOCK_FREE	atomic_uchar
ATOMIC_CHAR_LOCK_FREE	atomic_short
ATOMIC_CHAR8_T_LOCK_FREE	atomic_ushort
ATOMIC_CHAR16_T_LOCK_FREE	atomic_int
ATOMIC_CHAR32_T_LOCK_FREE	atomic_uint
ATOMIC_WCHAR_T_LOCK_FREE	atomic_long
ATOMIC_SHORT_LOCK_FREE	atomic_ulong
ATOMIC_INT_LOCK_FREE	atomic_llong
ATOMIC_LONG_LOCK_FREE	atomic_ullong
ATOMIC_LLONG_LOCK_FREE	atomic_char8_t
ATOMIC_POINTER_LOCK_FREE	atomic_char16_t
ATOMIC_FLAG_INIT	atomic_char32_t
memory_order	atomic_wchar_t
atomic_flag	atomic_int_least8_t
memory_order_relaxed	atomic_uint_least8_t
memory_order_consume	atomic_int_least16_t
memory_order_acquire	atomic_uint_least16_t
memory_order_release	atomic_int_least32_t
memory_order_acq_rel	atomic_uint_least32_t
memory_order_seq_cst	atomic_int_least64_t
atomic_bool	atomic_uint_least64_t

<code>atomic_int_fast8_t</code>	<code>atomic_uint_fast64_t</code>
<code>atomic_uint_fast8_t</code>	<code>atomic_intptr_t</code>
<code>atomic_int_fast16_t</code>	<code>atomic_uintptr_t</code>
<code>atomic_uint_fast16_t</code>	<code>atomic_size_t</code>
<code>atomic_int_fast32_t</code>	<code>atomic_ptrdiff_t</code>
<code>atomic_uint_fast32_t</code>	<code>atomic_intmax_t</code>
<code>atomic_int_fast64_t</code>	<code>atomic_uintmax_t</code>

```
void atomic_init(volatile A *obj, C value);
type kill_dependency(type y);
void atomic_thread_fence(memory_order order);
void atomic_signal_fence(memory_order order);
bool atomic_is_lock_free(const volatile A *obj);
void atomic_store(volatile A *object, C desired);
void atomic_store_explicit(volatile A *object, C desired, memory_order order);
C atomic_load(const volatile A *object);
C atomic_load_explicit(const volatile A *object, memory_order order);
C atomic_exchange(volatile A *object, C desired);
C atomic_exchange_explicit(volatile A *object, C desired, memory_order order);
bool atomic_compare_exchange_strong(volatile A *object, C *expected, C desired);
bool atomic_compare_exchange_strong_explicit(volatile A *object, C *expected,
      C desired, memory_order success, memory_order failure);
bool atomic_compare_exchange_weak(volatile A *object, C *expected, C desired);
bool atomic_compare_exchange_weak_explicit(volatile A *object, C *expected,
      C desired, memory_order success, memory_order failure);
C atomic_fetch_key(volatile A *object, M operand);
C atomic_fetch_key_explicit(volatile A *object, M operand, memory_order order);
bool atomic_flag_test_and_set(volatile atomic_flag *object);
bool atomic_flag_test_and_set_explicit(volatile atomic_flag *object,
      memory_order order);
void atomic_flag_clear(volatile atomic_flag *object);
void atomic_flag_clear_explicit(volatile atomic_flag *object,
      memory_order order);
```

B.17 Bit and byte utilities <stdbit.h>

<code>__STDC_ENDIAN_BIG__</code>	<code>__STDC_ENDIAN_NATIVE__</code>
<code>__STDC_ENDIAN_LITTLE__</code>	<code>__STDC_VERSION_STDBIT_H__</code>

```
unsigned int stdc_leading_zeros_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_leading_zeros_us(unsigned short value) [[unsequenced]];
unsigned int stdc_leading_zeros_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_leading_zeros_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_leading_zeros_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_leading_zeros(generic_value_type value) [[unsequenced]];
unsigned int stdc_leading_ones_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_leading_ones_us(unsigned short value) [[unsequenced]];
unsigned int stdc_leading_ones_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_leading_ones_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_leading_ones_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_leading_ones(generic_value_type value) [[unsequenced]];
unsigned int stdc_trailing_zeros_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_trailing_zeros_us(unsigned short value) [[unsequenced]];
unsigned int stdc_trailing_zeros_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_trailing_zeros_ul(unsigned long int value) [[unsequenced]];
unsigned int
```

```

stdc_trailing_zeros_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_trailing_zeros(generic_value_type value) [[unsequenced]];
unsigned int stdc_trailing_ones_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_trailing_ones_us(unsigned short value) [[unsequenced]];
unsigned int stdc_trailing_ones_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_trailing_ones_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_trailing_ones_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_trailing_ones(generic_value_type value) [[unsequenced]];
unsigned int stdc_first_leading_zero_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_first_leading_zero_us(unsigned short value) [[unsequenced]];
unsigned int stdc_first_leading_zero_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_first_leading_zero_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_first_leading_zero_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_first_leading_zero(generic_value_type value) [[unsequenced]];
unsigned int stdc_first_leading_one_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_first_leading_one_us(unsigned short value) [[unsequenced]];
unsigned int stdc_first_leading_one_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_first_leading_one_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_first_leading_one_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_first_leading_one(generic_value_type value) [[unsequenced]];
unsigned int stdc_first_trailing_zero_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_first_trailing_zero_us(unsigned short value) [[unsequenced]];
unsigned int stdc_first_trailing_zero_ui(unsigned int value) [[unsequenced]];
unsigned int
stdc_first_trailing_zero_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_first_trailing_zero_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_first_trailing_zero(generic_value_type value) [[unsequenced]];
unsigned int stdc_first_trailing_one_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_first_trailing_one_us(unsigned short value) [[unsequenced]];
unsigned int stdc_first_trailing_one_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_first_trailing_one_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_first_trailing_one_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_first_trailing_one(generic_value_type value) [[unsequenced]];
unsigned int stdc_count_zeros_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_count_zeros_us(unsigned short value) [[unsequenced]];
unsigned int stdc_count_zeros_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_count_zeros_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_count_zeros_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_count_zeros(generic_value_type value) [[unsequenced]];
unsigned int stdc_count_ones_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_count_ones_us(unsigned short value) [[unsequenced]];
unsigned int stdc_count_ones_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_count_ones_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_count_ones_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_count_ones(generic_value_type value) [[unsequenced]];
bool stdc_has_single_bit_uc(unsigned char value) [[unsequenced]];
bool stdc_has_single_bit_us(unsigned short value) [[unsequenced]];
bool stdc_has_single_bit_ui(unsigned int value) [[unsequenced]];
bool stdc_has_single_bit_ul(unsigned long int value) [[unsequenced]];

```

```
bool stdc_has_single_bit_ull(unsigned long long int value) [[unsequenced]];
bool stdc_has_single_bit(generic_value_type value) [[unsequenced]];
unsigned int stdc_bit_width_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_bit_width_us(unsigned short value) [[unsequenced]];
unsigned int stdc_bit_width_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_bit_width_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_bit_width_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_bit_width(generic_value_type value) [[unsequenced]];
unsigned char stdc_bit_floor_uc(unsigned char value) [[unsequenced]];
unsigned short stdc_bit_floor_us(unsigned short value) [[unsequenced]];
unsigned int stdc_bit_floor_ui(unsigned int value) [[unsequenced]];
unsigned long int stdc_bit_floor_ul(unsigned long int value) [[unsequenced]];
unsigned long long int
stdc_bit_floor_ull(unsigned long long int value) [[unsequenced]];
generic_value_type stdc_bit_floor(generic_value_type value) [[unsequenced]];
unsigned char stdc_bit_ceil_uc(unsigned char value) [[unsequenced]];
unsigned short stdc_bit_ceil_us(unsigned short value) [[unsequenced]];
unsigned int stdc_bit_ceil_ui(unsigned int value) [[unsequenced]];
unsigned long int stdc_bit_ceil_ul(unsigned long int value) [[unsequenced]];
unsigned long long int
stdc_bit_ceil_ull(unsigned long long int value) [[unsequenced]];
generic_value_type stdc_bit_ceil(generic_value_type value) [[unsequenced]];
```

B.18 Boolean type and values <stdbool.h>

__bool_true_false_are_defined

B.19 Checked Integer Operations <stdckdint.h>

__STDC_VERSION__STDCKDINT_H__

```
bool ckd_add(type1 *result, type2 a, type3 b);
bool ckd_sub(type1 *result, type2 a, type3 b);
bool ckd_mul(type1 *result, type2 a, type3 b);
```

B.20 Common definitions <stddef.h>

ptrdiff_t	max_align_t	NULL
nullptr_t	wchar_t	
size_t	__STDC_VERSION__STDDEF_H__	

```
offsetof(type, member-designator)

unreachable()
```

Only if the implementation defines __STDC_LIB_EXT1__ and additionally the user code defines __STDC_WANT_LIB_EXT1__ before any inclusion of <stddef.h>:

rsize_t

B.21 Integer types <stdint.h>

<code>intN_t</code>	<code>INT_LEASTN_WIDTH</code>	<code>PTRDIFF_MIN</code>
<code>uintN_t</code>	<code>UINT_LEASTN_MAX</code>	<code>PTRDIFF_MAX</code>
<code>int_leastN_t</code>	<code>UINT_LEASTN_WIDTH</code>	<code>SIG_ATOMIC_MIN</code>
<code>uint_leastN_t</code>	<code>INT_FASTN_MIN</code>	<code>SIG_ATOMIC_MAX</code>
<code>int_fastN_t</code>	<code>INT_FASTN_MAX</code>	<code>SIG_ATOMIC_WIDTH</code>
<code>uint_fastN_t</code>	<code>INT_FASTN_WIDTH</code>	<code>SIZE_MAX</code>
<code>intptr_t</code>	<code>UINT_FASTN_MAX</code>	<code>SIZE_WIDTH</code>
<code>uintptr_t</code>	<code>UINT_FASTN_WIDTH</code>	<code>WCHAR_MIN</code>
<code>intmax_t</code>	<code>INTPTR_MIN</code>	<code>WCHAR_MAX</code>
<code>uintmax_t</code>	<code>INTPTR_MAX</code>	<code>WCHAR_WIDTH</code>
<code>__STDC_VERSION_STDINT_H__</code>	<code>INTPTR_WIDTH</code>	<code>WINT_MIN</code>
<code>INTN_MIN</code>	<code>UINTPTR_MAX</code>	<code>WINT_MAX</code>
<code>INTN_MAX</code>	<code>UINTPTR_WIDTH</code>	<code>WINT_WIDTH</code>
<code>INTN_WIDTH</code>	<code>INTMAX_MIN</code>	<code>INTN_C(value)</code>
<code>UINTN_MAX</code>	<code>INTMAX_MAX</code>	<code>UINTN_C(value)</code>
<code>UINTN_WIDTH</code>	<code>INTMAX_WIDTH</code>	<code>INTMAX_C(value)</code>
<code>INT_LEASTN_MIN</code>	<code>UINTMAX_MAX</code>	<code>UINTMAX_C(value)</code>
<code>INT_LEASTN_MAX</code>	<code>UINTMAX_WIDTH</code>	

Only if the implementation defines `__STDC_LIB_EXT1__` and additionally the user code defines `__STDC_WANT_LIB_EXT1__` before any inclusion of `<stdint.h>`:

`RSIZE_MAX`

B.22 Input/output `<stdio.h>`

<code>size_t</code>	<code>_IONBF</code>	<code>SEEK_CUR</code>	<code>stdout</code>
<code>FILE</code>	<code>BUFSIZ</code>	<code>SEEK_END</code>	<code>_PRINTF_NAN_LEN_MAX</code>
<code>fpos_t</code>	<code>EOF</code>	<code>SEEK_SET</code>	
<code>NULL</code>	<code>FOPEN_MAX</code>	<code>TMP_MAX</code>	<code>__STDC_VERSION_STDIO_H__</code>
<code>_IOFBF</code>	<code>FILENAME_MAX</code>	<code>stderr</code>	
<code>_IOLBF</code>	<code>L_tmpnam</code>	<code>stdin</code>	

```
int remove(const char *filename);
int rename(const char *old, const char *new);
FILE *tmpfile(void);
char *tmpnam(char *s);
int fclose(FILE *stream);
int fflush(FILE *stream);
FILE *fopen(const char * restrict filename, const char * restrict mode);
FILE *freopen(const char * restrict filename, const char * restrict mode,
FILE * restrict stream);
void setbuf(FILE * restrict stream, char * restrict buf);
int setvbuf(FILE * restrict stream, char * restrict buf, int mode, size_t size);
int printf(const char * restrict format, ...);
int scanf(const char * restrict format, ...);
int snprintf(char * restrict s, size_t n, const char * restrict format, ...);
int sprintf(char * restrict s, const char * restrict format, ...);
int sscanf(const char * restrict s, const char * restrict format, ...);
int vfprintf(FILE * restrict stream, const char * restrict format, va_list arg);
int vfscanf(FILE * restrict stream, const char * restrict format, va_list arg);
int vprintf(const char * restrict format, va_list arg);
int vscanf(const char * restrict format, va_list arg);
int vsnprintf(char * restrict s, size_t n, const char * restrict format, va_list arg);
int vsprintf(char * restrict s, const char * restrict format, va_list arg);
int vsscanf(const char * restrict s, const char * restrict format, va_list arg);
int fgetc(FILE *stream);
```



```

char *fgets(char * restrict s, int n, FILE * restrict stream);
int fputc(int c, FILE *stream);
int fputs(const char * restrict s, FILE * restrict stream);
int getc(FILE *stream);
int getchar(void);
int putc(int c, FILE *stream);
int putchar(int c);
int puts(const char *s);
int ungetc(int c, FILE *stream);
size_t fread(void * restrict ptr, size_t size, size_t nmemb,
    FILE * restrict stream);
size_t fwrite(const void * restrict ptr, size_t size, size_t nmemb,
    FILE * restrict stream);
int fgetpos(FILE * restrict stream, fpos_t * restrict pos);
int fseek(FILE *stream, long int offset, int whence);
int fsetpos(FILE *stream, const fpos_t *pos);
long int ftell(FILE *stream);
void rewind(FILE *stream);
void clearerr(FILE *stream);
int feof(FILE *stream);
int ferror(FILE *stream);
void perror(const char *s);
int fprintf(FILE * restrict stream, const char * restrict format, ...);
int fscanf(FILE * restrict stream, const char * restrict format, ...);

```

Only if the implementation defines `__STDC_LIB_EXT1__` and additionally the user code defines `__STDC_WANT_LIB_EXT1__` before any inclusion of `<stdio.h>`:

`L_tmpnam_s` `TMP_MAX_S` `errno_t` `rsize_t`

```

errno_t tmpfile_s(FILE * restrict * restrict streamptr);
errno_t tmpnam_s(char *s, rsize_t maxsize);
errno_t fopen_s(FILE * restrict * restrict streamptr,
    const char * restrict filename, const char * restrict mode);
errno_t freopen_s(FILE * restrict * restrict newstreamptr,
    const char * restrict filename, const char * restrict mode,
    FILE * restrict stream);
int fprintf_s(FILE * restrict stream, const char * restrict format, ...);
int fscanf_s(FILE * restrict stream, const char * restrict format, ...);
int printf_s(const char * restrict format, ...);
int scanf_s(const char * restrict format, ...);
int snprintf_s(char * restrict s, rsize_t n, const char * restrict format, ...);
int sprintf_s(char * restrict s, rsize_t n, const char * restrict format, ...);
int sscanf_s(const char * restrict s, const char * restrict format, ...);
int vfprintf_s(FILE * restrict stream, const char * restrict format, va_list arg);
int vscanf_s(FILE * restrict stream, const char * restrict format, va_list arg);
int vprintf_s(const char * restrict format, va_list arg);
int vscanf_s(const char * restrict format, va_list arg);
int vsnprintf_s(char * restrict s, rsize_t n, const char * restrict format,
    va_list arg);
int vsprintf_s(char * restrict s, rsize_t n, const char * restrict format,
    va_list arg);
int vsscanf_s(const char * restrict s, const char * restrict format, va_list arg);
char *gets_s(char *s, rsize_t n);

```

B.23 General utilities `<stdlib.h>`

size_t	once_flag	NULL
wchar_t	__STDC_VERSION__	ONCE_FLAG_INIT
div_t	EXIT_FAILURE	RAND_MAX
ldiv_t	EXIT_SUCCESS	
lldiv_t	MB_CUR_MAX	

```

void call_once(once_flag *flag, void (*func)(void));
double atof(const char *nptr);
int atoi(const char *nptr);
long int atol(const char *nptr);
long long int atoll(const char *nptr);
int strfromd(char * restrict s, size_t n, const char * restrict format,
double fp);
int strfromf(char * restrict s, size_t n, const char * restrict format,
float fp);
int strfroml(char * restrict s, size_t n, const char * restrict format,
long double fp);
double strtod(const char * restrict nptr, char ** restrict endptr);
float strttof(const char * restrict nptr, char ** restrict endptr);
long double strtold(const char * restrict nptr, char ** restrict endptr);
long int strtol(const char * restrict nptr, char ** restrict endptr, int base);
long long int strtoll(const char * restrict nptr, char ** restrict endptr,
int base);
unsigned long int strtoul(const char * restrict nptr, char ** restrict endptr,
int base);
unsigned long long int strtoull(const char * restrict nptr,
char ** restrict endptr, int base);
int rand(void);
void srand(unsigned int seed);
void *aligned_alloc(size_t alignment, size_t size);
void *calloc(size_t nmemb, size_t size);
void free(void *ptr);
void free_sized(void *ptr, size_t size);
void free_aligned_sized(void *ptr, size_t alignment, size_t size);
void *malloc(size_t size);
void *realloc(void *ptr, size_t size);
[[noreturn]] void abort(void);
int atexit(void (*func)(void));
int at_quick_exit(void (*func)(void));
[[noreturn]] void exit(int status);
[[noreturn]] void _Exit(int status);
char *getenv(const char *name);
[[noreturn]] void quick_exit(int status);
int system(const char *string);
QVoid *bsearch(const void *key, QVoid *base, size_t nmemb, size_t size,
int (*compar)(const void *, const void *));
void qsort(void *base, size_t nmemb, size_t size,
int (*compar)(const void *, const void *));
int abs(int j);
long int labs(long int j);
long long int llabs(long long int j);
div_t div(int numer, int denom);
ldiv_t ldiv(long int numer, long int denom);
lldiv_t lldiv(long long int numer, long long int denom);
int mblen(const char *s, size_t n);
int mbtowc(wchar_t * restrict pwc, const char * restrict s, size_t n);
int wctomb(char *s, wchar_t wc);
size_t mbstowcs(wchar_t * restrict pwcs, const char * restrict s, size_t n);
size_t wcstombs(char * restrict s, const wchar_t * restrict pwcs, size_t n);
size_t memalignment(const void *p);

```

```
int strfromd32(char * restrict s, size_t n, const char * restrict format,
               _Decimal32 fp);
int strfromd64(char * restrict s, size_t n, const char * restrict format,
               _Decimal64 fp);
int strfromd128(char * restrict s, size_t n, const char * restrict format,
                _Decimal128 fp);
_Decimal32 strtod32(const char * restrict nptr, char ** restrict endptr);
_Decimal64 strtod64(const char * restrict nptr, char ** restrict endptr);
_Decimal128 strtod128(const char * restrict nptr, char ** restrict endptr);
```

```
int strfromfN(char * restrict s, size_t n,
    const char * restrict format, _FloatN fp);
int strfromfNx(char * restrict s, size_t n,
    const char * restrict format, _FloatNx fp);
int strfromdN(char * restrict s, size_t n,
    const char * restrict format, _DecimalN fp);
int strfromdNx(char * restrict s, size_t n,
    const char * restrict format, _DecimalNx fp);
_FloatN strttofN(const char * restrict nptr,
    char ** restrict endptr);
_FloatNx strttofNx(const char * restrict nptr,
    char ** restrict endptr);
_DecimalN strtodN(const char * restrict nptr,
    char ** restrict endptr);
_DecimalNx strtodNx(const char * restrict nptr,
    char ** restrict endptr);
int strfromencfN(char * restrict s, size_t n, const char * restrict format,
    const unsigned char encptr[restrict static N/8]);
int strfromencdecN(char * restrict s, size_t n, const char * restrict format,
    const unsigned char encptr[restrict static N/8]);
int strfromencbindN(char * restrict s, size_t n, const char * restrict format,
    const unsigned char encptr[restrict static N/8]);
void strttoencfN(unsigned char encptr[restrict static N/8],
    const char * restrict nptr, char ** restrict endptr);
void strttoencdecN(unsigned char encptr[restrict static N/8],
    const char * restrict nptr, char ** restrict endptr);
void strttoencbindN(unsigned char encptr[restrict static N/8],
    const char * restrict nptr, char ** restrict endptr);
```

```
errno_t      rsize_t      constraint_handler_t
```

© ISO/IEC 2024 — All rights reserved

```

    int (*compar)(const void *x, const void *y, void *context),
    void *context);
errno_t wctomb_s(int * restrict status, char * restrict s, rsize_t smax,
    wchar_t wc);
errno_t mbstowcs_s(size_t * restrict retval, wchar_t * restrict dst,
    rsize_t dstmax, const char * restrict src, rsize_t len);
errno_t wcstombs_s(size_t * restrict retval, char * restrict dst, rsize_t dstmax,
    const wchar_t * restrict src, rsize_t len);

```

B.24 _Noreturn <stdnoreturn.h>

noreturn

B.25 String handling <string.h>

size_t **NULL**
__STDC_VERSION_STRING_H__

```

void *memcpy(void * restrict s1, const void * restrict s2, size_t n);
void *memccpy(void * restrict s1, const void * restrict s2, int c, size_t n);
void *memmove(void *s1, const void *s2, size_t n);
char *strcpy(char * restrict s1, const char * restrict s2);
char *strncpy(char * restrict s1, const char * restrict s2, size_t n);
char *strdup(const char *s);
char *strndup(const char *s, size_t n);
char *strcat(char * restrict s1, const char * restrict s2);
char *strncat(char * restrict s1, const char * restrict s2, size_t n);
int memcmp(const void *s1, const void *s2, size_t n);
int strcmp(const char *s1, const char *s2);
int strcoll(const char *s1, const char *s2);
int strncmp(const char *s1, const char *s2, size_t n);
size_t strxfrm(char * restrict s1, const char * restrict s2, size_t n);
QVoid *memchr(QVoid *s, int c, size_t n);
QChar *strchr(QChar *s, int c);
size_t strcspn(const char *s1, const char *s2);
QChar *strpbrk(QChar *s1, const char *s2);
QChar *strrchr(QChar *s, int c);
size_t strspn(const char *s1, const char *s2);
QChar *strstr(QChar *s1, const char *s2);
char *strtok(char * restrict s1, const char * restrict s2);
void *memset(void *s, int c, size_t n);
void *memset_explicit(void *s, int c, size_t n);
char *strerror(int errnum);
size_t strlen(const char *s);

```

Only if the implementation defines **__STDC_LIB_EXT1__** and additionally the user code defines **__STDC_WANT_LIB_EXT1__** before any inclusion of <string.h>:

errno_t **rsize_t**

```

errno_t memcpy_s(void * restrict s1, rsize_t s1max, const void * restrict s2,
    rsize_t n);
errno_t memmove_s(void *s1, rsize_t s1max, const void *s2, rsize_t n);
errno_t strcpy_s(char * restrict s1, rsize_t s1max, const char * restrict s2);
errno_t strncpy_s(char * restrict s1, rsize_t s1max, const char * restrict s2,
    rsize_t n);
errno_t strcat_s(char * restrict s1, rsize_t s1max, const char * restrict s2);
errno_t strncat_s(char * restrict s1, rsize_t s1max, const char * restrict s2,
    rsize_t n);

```

```
char *strtok_s(char * restrict s1, rsize_t * restrict s1max,
               const char * restrict s2, char ** restrict ptr);
errno_t memset_s(void *s, rsize_t smax, int c, rsize_t n)
errno_t strerror_s(char *s, rsize_t maxsize, errno_t errnum);
size_t strerrorlen_s(errno_t errnum);
size_t strnlen_s(const char *s, size_t maxsize);
```

B.26 Type-generic math <tgmath.h>

__STDC_VERSION_TGMATH_H__	expm1	nearbyint
acos	fdim	nextafter
asin	floor	nextdown
atan	fmax	nexttoward
acosh	fmaximum	nextup
asinh	fmaximum_mag	pown
atanh	fmaximum_num	powr
cos	fmaximum_mag_num	remainder
sin	fma	remquo
tan	fmin	rint
cosh	fminimum	rootn
sinh	fminimum_mag	roundeven
tanh	fminimum_num	round
exp	fminimum_mag_num	rsqrt
log	fmod	scalbln
pow	frexp	scalbn
sqrt	fromfpx	sinpi
fabs	fromfp	tanpi
acospi	hypot	tgamma
asinpi	ilogb	trunc
atan2pi	ldexp	ufromfpx
atan2	lgamma	ufromfp
atanpi	llogb	fadd
cbrt	llrint	dadd
ceil	llround	fsub
compoundn	log10p1	dsub
copysign	log10	fmul
cospi	log1p	dmul
erfc	log2p1	fdiv
erf	log2	ddiv
exp10m1	logb	ffma
exp10	logp1	dfma
exp2m1	lrint	fsqrt
exp2	lround	dsqrt

Only if the implementation does not define __STDC_NO_COMPLEX__:

carg	cimag	conj	cproj	creal
------	-------	------	-------	-------

Only if the implementation defines __STDC_IEC_60559_DFP__:

d32add	d64sub	d32div	d64fma	quantize	llquantexp
d64add	d32mul	d64div	d32sqrt	samequantum	
d32sub	d64mul	d32fma	d64sqrt	quantum	

Only if the implementation defines __STDC_IEC_60559_TYPES__ and additionally the user code defines __STDC_WANT_IEC_60559_TYPES_EXT__ before any inclusion of <tgmath.h>:

fMadd	fMsub	fMmul	fMdiv	fMfma	fMsqrt
fMxadd	fMxsub	fMxmul	fMxdiv	fMxfma	fMxsqrt
dMadd	dMsub	dMmul	dMdiv	dMfma	dMsqrt
dMxadd	dMxsub	dMxmul	dMxdiv	dMxfma	dMxsqrt

B.27 Threads <threads.h>

__STDC_NO_THREADS__	mtx_t	mtx_timed
ONCE_FLAG_INIT	tss_dtor_t	thrd_timedout
TSS_DTOR_ITERATIONS	thrd_start_t	thrd_success
cnd_t	once_flag	thrd_busy
thrd_t	mtx_plain	thrd_error
tss_t	mtx_recursive	thrd_nomem

```
void call_once(once_flag *flag, void (*func)(void));
int cnd_broadcast(cnd_t *cond);
void cnd_destroy(cnd_t *cond);
int cnd_init(cnd_t *cond);
int cnd_signal(cnd_t *cond);
int cnd_timedwait(cnd_t * restrict cond, mtx_t * restrict mtx,
    const struct timespec * restrict ts);
int cnd_wait(cnd_t *cond, mtx_t *mtx);
void mtx_destroy(mtx_t *mtx);
int mtx_init(mtx_t *mtx, int type);
int mtx_lock(mtx_t *mtx);
int mtx_timedlock(mtx_t * restrict mtx, const struct timespec * restrict ts);
int mtx_trylock(mtx_t *mtx);
int mtx_unlock(mtx_t *mtx);
int thrd_create(thrd_t *thr, thrd_start_t func, void *arg);
thrd_t thrd_current(void);
int thrd_detach(thrd_t thr);
int thrd_equal(thrd_t thr0, thrd_t thr1);
[[noreturn]] void thrd_exit(int res);
int thrd_join(thrd_t thr, int *res);
int thrd_sleep(const struct timespec *duration, struct timespec *remaining);
void thrd_yield(void);
int tss_create(tss_t *key, tss_dtor_t dtor);
void tss_delete(tss_t key);
void *tss_get(tss_t key);
int tss_set(tss_t key, void *val);
```

B.28 Date and time <time.h>

__STDC_VERSION_TIME_H__	TIME_UTC	time_t
NULL	size_t	struct timespec
CLOCKS_PER_SEC	clock_t	struct tm

```
clock_t clock(void);
double difftime(time_t time1, time_t time0);
time_t mktime(struct tm *timeptr);
time_t timegm(struct tm *timeptr);
time_t time(time_t *timer);
int timespec_get(struct timespec *ts, int base);
int timespec_getres(struct timespec *ts, int base);
[[deprecated]] char *asctime(const struct tm *timeptr);
[[deprecated]] char *ctime(const time_t *timer);
struct tm *gmtime(const time_t *timer);
```

```
struct tm *gmtime_r(const time_t *timer, struct tm *buf);
struct tm *localtime(const time_t *timer);
struct tm *localtime_r(const time_t *timer, struct tm *buf);
size_t strftime(char * restrict s, size_t maxsize, const char * restrict format,
               const struct tm * restrict timeptr);
```

Only if supported by the implementation:

TIME_MONOTONIC **TIME_ACTIVE**

Only if threads are supported and it is supported by the implementation:

TIME_THREAD_ACTIVE

Only if the implementation defines **__STDC_LIB_EXT1__** and additionally the user code defines **__STDC_WANT_LIB_EXT1__** before any inclusion of `<time.h>`:

errno_t **rsize_t**

```
errno_t asctime_s(char *s, rsize_t maxsize, const struct tm *timeptr);
errno_t ctime_s(char *s, rsize_t maxsize, const time_t *timer);
struct tm *gmtime_s(const time_t * restrict timer, struct tm * restrict result);
struct tm *localtime_s(const time_t * restrict timer, struct tm * restrict result);
```

B.29 Unicode utilities `<uchar.h>`

__STDC_VERSION_UCHAR_H__ **size_t** **char16_t**
mbstate_t **char8_t** **char32_t**

```
size_t mbrtoc8(char8_t * restrict pc8, const char * restrict s, size_t n,
               mbstate_t * restrict ps);
size_t c8rtomb(char * restrict s, char8_t c8, mbstate_t * restrict ps);
size_t mbrtoc16(char16_t * restrict pc16, const char * restrict s, size_t n,
               mbstate_t * restrict ps);
size_t c16rtomb(char * restrict s, char16_t c16, mbstate_t * restrict ps);
size_t mbrtoc32(char32_t * restrict pc32, const char * restrict s, size_t n,
               mbstate_t * restrict ps);
size_t c32rtomb(char * restrict s, char32_t c32, mbstate_t * restrict ps);
```

B.30 Extended multibyte/wide character utilities `<wchar.h>`

wchar_t **struct tm** **WCHAR_MIN**
size_t **__STDC_VERSION_WCHAR_H__** **WEOF**
mbstate_t **NULL**
wint_t **WCHAR_MAX**

```
int fwprintf(FILE * restrict stream, const wchar_t * restrict format, ...);
int fwsscanf(FILE * restrict stream, const wchar_t * restrict format, ...);
int swprintf(wchar_t * restrict s, size_t n, const wchar_t * restrict format,
            ...);
int swscanf(const wchar_t * restrict s, const wchar_t * restrict format, ...);
int vfwprintf(FILE * restrict stream, const wchar_t * restrict format,
              va_list arg);
int vfwscanf(FILE * restrict stream, const wchar_t * restrict format,
```

```

    va_list arg);
int vswprintf(wchar_t * restrict s, size_t n, const wchar_t * restrict format,
    va_list arg);
int vswscanf(const wchar_t * restrict s, const wchar_t * restrict format,
    va_list arg);
int vwprintf(const wchar_t * restrict format, va_list arg);
int vwscanf(const wchar_t * restrict format, va_list arg);
int wprintf(const wchar_t * restrict format, ...);
int wscanf(const wchar_t * restrict format, ...);
wint_t fgetwc(FILE *stream);
wchar_t *fgetws(wchar_t * restrict s, int n, FILE * restrict stream);
wint_t fputwc(wchar_t c, FILE *stream);
int fputws(const wchar_t * restrict s, FILE * restrict stream);
int fwide(FILE *stream, int mode);
wint_t getwc(FILE *stream);
wint_t getwchar(void);
wint_t putwc(wchar_t c, FILE *stream);
wint_t putwchar(wchar_t c);
wint_t ungetwc(wint_t c, FILE *stream);
double wcstod(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
float wcstof(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
long double wcstold(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
long int wcstol(const wchar_t * restrict nptr, wchar_t ** restrict endptr,
    int base);
long long int wcstoll(const wchar_t * restrict nptr, wchar_t ** restrict endptr,
    int base);
unsigned long int wcstoul(const wchar_t * restrict nptr,
    wchar_t ** restrict endptr, int base);
unsigned long long int wcstoull(const wchar_t * restrict nptr,
    wchar_t ** restrict endptr, int base);
wchar_t *wcscpy(wchar_t * restrict s1, const wchar_t * restrict s2);
wchar_t *wcsncpy(wchar_t * restrict s1, const wchar_t * restrict s2, size_t n);
wchar_t *wmemcpy(wchar_t * restrict s1, const wchar_t * restrict s2, size_t n);
wchar_t *wmemmove(wchar_t *s1, const wchar_t *s2, size_t n);
wchar_t *wcscat(wchar_t * restrict s1, const wchar_t * restrict s2);
wchar_t *wcsncat(wchar_t * restrict s1, const wchar_t * restrict s2, size_t n);
int wcscmp(const wchar_t *s1, const wchar_t *s2);
int wscoll(const wchar_t *s1, const wchar_t *s2);
int wcsncmp(const wchar_t *s1, const wchar_t *s2, size_t n);
size_t wcsxfrm(wchar_t * restrict s1, const wchar_t * restrict s2, size_t n);
int wmemcmp(const wchar_t *s1, const wchar_t *s2, size_t n);
QWchar_t *wcschr(QWchar_t *s, wchar_t c);
size_t wcslen(const wchar_t *s1, const wchar_t *s2);
QWchar_t *wcpbrk(QWchar_t *s1, const wchar_t *s2);
QWchar_t *wcsrchr(QWchar_t *s, wchar_t c);
size_t wcsspncpy(const wchar_t *s1, const wchar_t *s2);
QWchar_t *wcssstr(QWchar_t *s1, const wchar_t *s2);
wchar_t *wcstok(wchar_t * restrict s1, const wchar_t * restrict s2,
    wchar_t ** restrict ptr);
QWchar_t *wmemchr(QWchar_t *s, wchar_t c, size_t n);
size_t wcslen(const wchar_t *s);
wchar_t *wmemset(wchar_t *s, wchar_t c, size_t n);
size_t wcsftime(wchar_t * restrict s, size_t maxsize,
    const wchar_t * restrict format, const struct tm * restrict timeptr);
wint_t btowc(int c);
int wctob(wint_t c);
int mbsinit(const mbstate_t *ps);
size_t mbrlen(const char * restrict s, size_t n, mbstate_t * restrict ps);
size_t mbrtowc(wchar_t * restrict pwc, const char * restrict s, size_t n,
    mbstate_t * restrict ps);
size_t wctomb(char * restrict s, wchar_t wc, mbstate_t * restrict ps);

```



```

size_t mbsrtowcs(wchar_t * restrict dst, const char ** restrict src, size_t len,
                mbstate_t * restrict ps);
size_t wcsrtombs(char * restrict dst, const wchar_t ** restrict src, size_t len,
                mbstate_t * restrict ps);

```

Only if the implementation defines `__STDC_IEC_60559_DFP__`:

```

_Decimal32 wcstod32(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
_Decimal64 wcstod64(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
_Decimal128 wcstod128(const wchar_t * restrict nptr, wchar_t ** restrict endptr);

```

Only if the implementation defines `__STDC_IEC_60559_TYPES__` and additionally the user code defines `__STDC_WANT_IEC_60559_TYPES_EXT__` before any inclusion of `<wchar.h>`:

```

_FloatN wcstofN(const wchar_t * restrict nptr,
                wchar_t ** restrict endptr);
_FloatNx wcstofNx(const wchar_t * restrict nptr,
                 wchar_t ** restrict endptr);
_DecimalN wcstodN(const wchar_t * restrict nptr,
                 wchar_t ** restrict endptr);
_DecimalNx wcstodNx(const wchar_t * restrict nptr,
                   wchar_t ** restrict endptr);
void wcstoencfN(unsigned char encptr[restrict static N/8],
               const wchar_t * restrict nptr, wchar_t ** restrict endptr);
void wcstoencdecN(unsigned char encptr[restrict static N/8],
                 const wchar_t * restrict nptr, wchar_t ** restrict endptr);
void wcstoencbindN(unsigned char encptr[restrict static N/8],
                  const wchar_t * restrict nptr, wchar_t ** restrict endptr);

```

Only if the implementation defines `__STDC_LIB_EXT1__` and additionally the user code defines `__STDC_WANT_LIB_EXT1__` before any inclusion of `<wchar.h>`:

`errno_t`

`rsize_t`

```

int fwprintf_s(FILE * restrict stream, const wchar_t * restrict format, ...);
int fwscanf_s(FILE * restrict stream, const wchar_t * restrict format, ...);
int snwprintf_s(wchar_t * restrict s, rsize_t n, const wchar_t * restrict format,
               ...);
int swprintf_s(wchar_t * restrict s, rsize_t n, const wchar_t * restrict format,
               ...);
int swscanf_s(const wchar_t * restrict s, const wchar_t * restrict format, ...);
int vfwprintf_s(FILE * restrict stream, const wchar_t * restrict format,
               va_list arg);
int vfwscanf_s(FILE * restrict stream, const wchar_t * restrict format,
               va_list arg);
int vsnwprintf_s(wchar_t * restrict s, rsize_t n, const wchar_t * restrict format,
                 va_list arg);
int vswprintf_s(wchar_t * restrict s, rsize_t n, const wchar_t * restrict format,
                 va_list arg);
int vswscanf_s(const wchar_t * restrict s, const wchar_t * restrict format,
               va_list arg);
int vwprintf_s(const wchar_t * restrict format, va_list arg);
int vwscanf_s(const wchar_t * restrict format, va_list arg);
int wprintf_s(const wchar_t * restrict format, ...);
int wscanf_s(const wchar_t * restrict format, ...);
errno_t wcsncpy_s(wchar_t * restrict s1, rsize_t slmax,
                 const wchar_t * restrict s2);
errno_t wcsncpy_s(wchar_t * restrict s1, rsize_t slmax,
                 const wchar_t * restrict s2, rsize_t n);

```

```

errno_t wmemcpy_s(wchar_t * restrict s1, rsize_t slmax,
    const wchar_t * restrict s2, rsize_t n);
errno_t wmemmove_s(wchar_t *s1, rsize_t slmax, const wchar_t *s2, rsize_t n);
errno_t wcscat_s(wchar_t * restrict s1, rsize_t slmax,
    const wchar_t * restrict s2);
errno_t wcsncat_s(wchar_t * restrict s1, rsize_t slmax,
    const wchar_t * restrict s2, rsize_t n);
wchar_t *wcstok_s(wchar_t * restrict s1, rsize_t * restrict slmax,
    const wchar_t * restrict s2, wchar_t ** restrict ptr);
size_t wcsnlen_s(const wchar_t *s, size_t maxsize);
errno_t wcrctomb_s(size_t * restrict retval, char * restrict s, rsize_t smax,
    wchar_t wc, mbstate_t * restrict ps);
errno_t mbsrtowcs_s(size_t * restrict retval, wchar_t * restrict dst,
    rsize_t dstmax, const char ** restrict src, rsize_t len,
    mbstate_t * restrict ps);
errno_t wcsrtombs_s(size_t * restrict retval, char * restrict dst,
    rsize_t dstmax, const wchar_t ** restrict src, rsize_t len,
    mbstate_t * restrict ps);

```

B.31 Wide character classification and mapping utilities <wctype.h>

wint_t	wctrans_t	wctype_t	WEOF
--------	-----------	----------	------

```

int iswalnum(wint_t wc);
int iswalpna(wint_t wc);
int iswblank(wint_t wc);
int iswcntrl(wint_t wc);
int iswdigit(wint_t wc);
int iswgraph(wint_t wc);
int iswlower(wint_t wc);
int iswprint(wint_t wc);
int iswpunct(wint_t wc);
int iswspace(wint_t wc);
int iswupper(wint_t wc);
int iswxdigit(wint_t wc);
int iswctype(wint_t wc, wctype_t desc);
wctype_t wctype(const char *property);
wint_t towlower(wint_t wc);
wint_t towupper(wint_t wc);
wint_t towctrans(wint_t wc, wctrans_t desc);
wctrans_t wctrans(const char *property);

```

Annex C

(informative)

Sequence points

C.1 Known Sequence Points

The following are the sequence points described in 5.2.2.4:

- Between the evaluations of the function designator and actual arguments in a function call and the actual call. (6.5.3.3).
- Between the evaluations of the first and second operands of the following operators: logical AND **&&** (6.5.14); logical OR **||** (6.5.15); comma **,** (6.5.18).
- Between the evaluations of the first operand of the conditional **?:** operator and whichever of the second and third operands is evaluated (6.5.16).
- Between the evaluation of a full expression and the next full expression to be evaluated. The following are full expressions: a full declarator for a variably modified type; an initializer that is not part of a compound literal (6.7.11); the expression in an expression statement (6.8.4); the controlling expression of a selection statement (**if** or **switch**) (6.8.5); the controlling expression of a **while** or **do** statement (6.8.6); each of the (optional) expressions of a **for** statement (6.8.6.4); the (optional) expression in a **return** statement (6.8.7.5).
- Immediately before a library function returns (7.1.4).
- After the actions associated with each formatted input/output function conversion specifier (7.23.6, 7.31.2).
- Immediately before and immediately after each call to a comparison function, and also between any call to a comparison function and any movement of the objects passed as arguments to that call (7.24.6).

Annex D

(informative)

Universal character names for identifiers

D.1 Introduction

This subclause describes the choices made in application of UAX #31 (“Unicode Identifier and Pattern Syntax”) to C of the requirements from UAX #31 and how they do or do not apply to C. For UAX #31, C conforms by meeting the requirements “Default Identifiers” (D.2) and “Equivalent Normalized Identifiers” (D.2). The other requirements, also listed in the following subclauses, are either alternatives not taken or do not apply to C.

D.2 Default Identifiers

D.2.1 General

UAX #31 specifies a default syntax for identifiers based on properties from the Unicode Character Database, UAX #44. The general syntax is

```
<Identifier> := <Start> <Continue>* (<Medial> <Continue>+)*
```

where **<Start>** has the `XID_Start` property, **<Continue>** has the `XID_Continue` property, and **<Medial>** is a list of characters permitted between continue characters. For C we add the character `_` (U+005F, LOW LINE) to the set of permitted Start characters, the Medial set is empty, and the Continue characters are unmodified. In the grammar used in UAX #31, this is

```
<Identifier> := <Start> <Continue>*  
<Start> := XID_Start + U+005F  
<Continue> := <Start> + XID_Continue
```

Additionally, implementations may add the character `$` (U+0024, DOLLAR SIGN) to the set of permitted Start and Continue characters. This is described in the C grammar (6.4.3.1), where *identifier* is formed from *identifier-start* or *identifier* followed by *identifier-continue*.

D.2.2 Restricted Format Characters

If an implementation of UAX #31 wishes to allow format characters such as ZERO WIDTH JOINER or ZERO WIDTH NON-JOINER it shall define a profile allowing them, or describe precisely which combinations are permitted.

C does not allow format characters in identifiers, so this does not apply.

D.2.3 Stable Identifiers

An implementation of UAX #31 may choose to guarantee that identifiers are stable across versions of the Unicode Standard. Once a string qualifies as an identifier it does so in all future versions. C does not make this guarantee, except to the extent that UAX #31 guarantees the stability of the `XID_Start` and `XID_Continue` properties.

D.3 Immutable Identifiers

An implementation may choose to guarantee that the set of identifiers will never change by fixing the set of code points allowed in identifiers forever.

C does not choose to make this guarantee. As scripts are added to Unicode, additional characters in those scripts may become available for use in identifiers.

D.4 Pattern_White_Space and Pattern_Syntax Characters

UAX #31 describes how languages that use or interpret patterns of characters, such as regular expressions or number formats, may describe that syntax with Unicode properties.

C does not do this as part of the language, deferring to library components for such usage of patterns. This requirement does not apply to C.

D.5 Equivalent Normalized Identifiers

UAX #31 requires that implementations describe how identifiers are compared and considered equivalent.

C requires that identifiers be in Normalization Form C and therefore identifiers that compare the same under NFC are equivalent. This is described in 6.4.3.

D.6 Equivalent Case-Insensitive Identifiers

C considers case to be significant in identifier comparison, and does not do any case folding. This requirement does not apply to C

D.7 Filtered Normalized Identifiers

If any characters are excluded from normalization, UAX #31 requires a precise specification of those exclusions.

C does not make any such exclusions.

D.8 Filtered Case-Insensitive Identifiers

C identifiers are case sensitive, and therefore this requirement does not apply.

D.9 Hashtag Identifiers

There are no hashtags in C, so this requirement does not apply.

Annex E

(informative)

Implementation limits

E.1 Introduction

The contents of the header `<limits.h>` are given in the following subclauses. The values shall all be constant expressions suitable for use in conditional expression inclusion preprocessing directives. The components are described further in 5.3.5.3.2.

E.2 Minimum values

For the following macros, the minimum values shown shall be replaced by implementation-defined values.

#define	BOOL_WIDTH	1	// exact value
#define	CHAR_BIT	8	
#define	USHRT_WIDTH	16	
#define	UINT_WIDTH	16	
#define	ULONG_WIDTH	32	
#define	ULLONG_WIDTH	64	
#define	BITINT_MAXWIDTH	ULLONG_WIDTH	// at minimum as large // as unsigned long long
#define	MB_LEN_MAX	1	

For the following macros, the minimum magnitudes shown shall be replaced by implementation-defined magnitudes with the same sign that are deduced from the prior macros as indicated.⁴²⁰⁾

#define	BOOL_MAX	1	// $2^{\text{BOOL_WIDTH}} - 1$
#define	CHAR_MAX	UCHAR_MAX or SCHAR_MAX	
#define	CHAR_MIN	0 or SCHAR_MIN	
#define	CHAR_WIDTH	8	// CHAR_BIT
#define	UCHAR_MAX	255	// $2^{\text{UCHAR_WIDTH}} - 1$
#define	UCHAR_WIDTH	8	// CHAR_BIT
#define	USHRT_MAX	65535	// $2^{\text{USHRT_WIDTH}} - 1$
#define	SCHAR_MAX	+127	// $2^{\text{SCHAR_WIDTH}-1} - 1$
#define	SCHAR_MIN	-128	// $-2^{\text{SCHAR_WIDTH}-1}$
#define	SCHAR_WIDTH	8	// CHAR_BIT
#define	SHRT_MAX	+32767	// $2^{\text{SHRT_WIDTH}-1} - 1$
#define	SHRT_MIN	-32768	// $-2^{\text{SHRT_WIDTH}-1}$
#define	SHRT_WIDTH	16	// USHRT_WIDTH
#define	INT_MAX	+32767	// $2^{\text{INT_WIDTH}-1} - 1$
#define	INT_MIN	-32768	// $-2^{\text{INT_WIDTH}-1}$
#define	INT_WIDTH	16	// UINT_WIDTH
#define	UINT_MAX	65535	// $2^{\text{UINT_WIDTH}} - 1$
#define	LONG_MAX	+2147483647	// $2^{\text{LONG_WIDTH}-1} - 1$
#define	LONG_MIN	-2147483648	// $-2^{\text{LONG_WIDTH}-1}$
#define	LONG_WIDTH	32	// ULONG_WIDTH
#define	LLONG_MAX	+9223372036854775807	// $2^{\text{LLONG_WIDTH}-1} - 1$
#define	LLONG_MIN	-9223372036854775808	// $-2^{\text{LLONG_WIDTH}-1}$
#define	LLONG_WIDTH	64	// ULLONG_WIDTH
#define	ULONG_MAX	4294967295	// $2^{\text{ULONG_WIDTH}} - 1$
#define	ULLONG_MAX	18446744073709551615	// $2^{\text{ULLONG_WIDTH}} - 1$

The contents of the header `<float.h>` are given in the following subclauses. All integer values, except **FLT_ROUNDS**, shall be constant expressions suitable for use in **#if** preprocessing directives;

⁴²⁰⁾For the minimum value of a signed integer type there is no expression consisting of a minus sign and a decimal literal of that same type. The numbers in the table are only given as indications for the values and do not represent suitable expressions to be used for these macros.

all floating values shall be arithmetic constant expressions. The components are described further in 5.3.5.3.3 and 5.3.5.3.4.

The values given in the following list shall be replaced by implementation-defined expressions:

```
#define FLT_EVAL_METHOD
#define FLT_ROUND
#ifdef __STDC_IEC_60559_DFP__
#define DEC_EVAL_METHOD
#endif
```

The values given in the following list shall be replaced by implementation-defined constant expressions that are greater or equal in magnitude (absolute value) to those shown, with the same sign:

```
#define DBL_DECIMAL_DIG 10
#define DBL_DIG 10
#define DBL_MANT_DIG
#define DBL_MAX_10_EXP +37
#define DBL_MAX_EXP
#define DBL_MIN_10_EXP -37
#define DBL_MIN_EXP
#define DECIMAL_DIG 10
#define FLT_DECIMAL_DIG 6
#define FLT_DIG 6
#define FLT_MANT_DIG
#define FLT_MAX_10_EXP +37
#define FLT_MAX_EXP
#define FLT_MIN_10_EXP -37
#define FLT_MIN_EXP
#define FLT_RADIX 2
#define LDBL_DECIMAL_DIG 10
#define LDBL_DIG 10
#define LDBL_MANT_DIG
#define LDBL_MAX_10_EXP +37
#define LDBL_MAX_EXP
#define LDBL_MIN_10_EXP -37
#define LDBL_MIN_EXP
```

The values given in the following list shall be replaced by implementation-defined constant expressions with values that are greater than or equal to those shown:

```
#define DBL_MAX 1E+37
#define DBL_NORM_MAX 1E+37
#define FLT_MAX 1E+37
#define FLT_NORM_MAX 1E+37
#define LDBL_MAX 1E+37
#define LDBL_NORM_MAX 1E+37
```

The values given in the following list shall be replaced by implementation-defined constant expressions with (positive) values that are less than or equal to those shown:

```
#define DBL_EPSILON 1E-9
#define DBL_MIN 1E-37
#define FLT_EPSILON 1E-5
#define FLT_MIN 1E-37
#define LDBL_EPSILON 1E-9
#define LDBL_MIN 1E-37
```

If the implementation supports decimal floating types, the following macros provide the parameters of these types as exact values.

© ISO/IEC 2024 — All rights reserved

Annex F
(normative)
ISO/IEC 60559 floating-point arithmetic

F.1 Introduction

This annex specifies C language support for the *ISO/IEC 60559 floating-point standard*. The *ISO/IEC 60559 floating-point standard* is specifically Floating-point arithmetic (ISO/IEC 60559:2020), also designated as *IEEE Standard for Floating-Point Arithmetic* (IEEE 754–2019). ISO/IEC 60559 generally refers to the floating-point standard, as in ISO/IEC 60559 operation, ISO/IEC 60559 format, etc.

The ISO/IEC 60559 floating-point standard is a minor upgrade to ISO/IEC/IEEE 60559:2011 (IEEE 754-2008). ISO/IEC/IEEE 60559:2011 was a major upgrade to IEC 60559:1989 (IEEE 754–1985), specifying decimal as well as binary floating-point arithmetic.

An implementation that defines `__STDC_IEC_60559_BFP__` to 202311L shall conform to the specifications in this annex for binary floating-point arithmetic and shall also define `__STDC_IEC_559__` to 1.⁴²¹⁾

An implementation that defines `__STDC_IEC_60559_DFP__` to 202311L shall conform to the specifications for decimal floating-point arithmetic in the following subclauses of this annex:

- F.2.2 Infinities and NaNs
- F.3 Operations
- F.4 Floating to integer conversions
- F.6 The **return** statement
- F.7 Contracted expressions
- F.8 Floating-point environment
- F.9 Optimization
- F.10 Mathematics `<math.h>` and `<tgmath.h>`

For the purpose of specifying these conformance requirements, the macros, functions, and values mentioned in the subclauses listed prior are understood to refer to the corresponding macros, functions, and values for decimal floating types. Likewise, the “rounding direction mode” is understood to refer to the rounding direction mode for decimal floating-point arithmetic.

Where a binding between the C language and ISO/IEC 60559 is indicated, the ISO/IEC 60559-specified behavior is adopted by reference, unless stated otherwise.

F.2 Types

F.2.1 General

The C floating types match the ISO/IEC 60559 formats as follows:

- The **float** type matches the ISO/IEC 60559 binary32 format.
- The **double** type matches the ISO/IEC 60559 binary64 format.
- The **long double** type matches the ISO/IEC 60559 binary128 format, else an ISO/IEC 60559 binary64-extended format,⁴²²⁾ else a non-ISO/IEC 60559 extended format, else the ISO/IEC 60559 binary64 format.

Any non-ISO/IEC 60559 extended format used for the **long double** type shall have more precision than ISO/IEC 60559 binary64 and at least the range of ISO/IEC 60559 binary64.⁴²³⁾ The value

⁴²¹⁾Implementations that do not define either of `__STDC_IEC_60559_BFP__` and `__STDC_IEC_559__` are not required to conform to these specifications. New code should not use the obsolescent macro `__STDC_IEC_559__` to test for conformance to this annex.

⁴²²⁾ISO/IEC 60559 binary64-extended formats include the common 80-bit ISO/IEC 60559 format.

⁴²³⁾A non-ISO/IEC 60559 **long double** type provides signed infinities, signed zeros, and NaNs, as its values include all **double** values.

of **FLT_ROUNDS** applies to all ISO/IEC 60559 types supported by the implementation, but is not required to apply to non-ISO/IEC 60559 types.

Recommended practice

The **long double** type should match the ISO/IEC 60559 binary128 format, else an ISO/IEC 60559 binary64-extended format.

F.2.2 Infinities and NaNs

Since negative and positive infinity are representable in ISO/IEC 60559 formats, all real numbers lie within the range of representable values (5.3.5.3.3).

The **NAN** and **INFINITY** macros in `<float.h>` and the **nan** functions in `<math.h>` provide designations for ISO/IEC 60559 quiet NaNs and infinities. The **FLT_SNAN**, **DBL_SNAN**, and **LDBL_SNAN** macros in `<float.h>` provide designations for ISO/IEC 60559 signaling NaNs.

This annex does not require the full support for signaling NaNs specified in ISO/IEC 60559. This annex uses the term NaN, unless explicitly qualified, to denote quiet NaNs. Where specification of signaling NaNs is not provided, the behavior of signaling NaNs is implementation-defined (either treated as an ISO/IEC 60559 quiet NaN or treated as an ISO/IEC 60559 signaling NaN).⁴²⁴⁾

Any operator or `<math.h>` function that raises an “invalid” floating-point exception, if delivering a floating type result, shall return a quiet NaN, unless explicitly specified otherwise.

To support signaling NaNs as specified in ISO/IEC 60559, an implementation should adhere to the following recommended practice.

Recommended practice

Any floating-point operator or `<math.h>` function or macro with a signaling NaN input, unless explicitly specified otherwise, raises an “invalid” floating-point exception.

NOTE Some functions do not propagate quiet NaN arguments. For example, **hypot**(**x**, **y**) returns infinity if **x** or **y** is infinite and the other is a quiet NaN. The recommended practice in this subclause specifies that such functions (and others) raise the “invalid” floating-point exception if an argument is a signaling NaN, which also implies they return a quiet NaN in these cases.

The `<fenv.h>` header defines the macro **FE_SNANS_ALWAYS_SIGNAL** if and only if the implementation follows the recommended practice in this subclause. If defined, **FE_SNANS_ALWAYS_SIGNAL** expands to the integer constant **1**.

F.3 Operations

C operators, functions, and function-like macros provide operations specified by ISO/IEC 60559 as shown in Table F.2. In the table, C functions are represented by the function name without a type suffix. Specifications for the C facilities are provided in the listed clauses. The C specifications are intended to match ISO/IEC 60559, unless stated otherwise.

Table F.2 — Operation binding — mathematical operations

ISO/IEC 60559 operation	C operation	Clause
<code>roundToIntegralTiesToEven</code>	roundeven	7.12.10.8, F.10.7.8
<code>roundToIntegralTiesAway</code>	round	7.12.10.6, F.10.7.6
<code>roundToIntegralTowardZero</code>	trunc	7.12.10.9, F.10.7.9
<code>roundToIntegralTowardPositive</code>	ceil	7.12.10.1, F.10.7.1
<code>roundToIntegralTowardNegative</code>	floor	7.12.10.2, F.10.7.2

⁴²⁴⁾Since NaNs created by ISO/IEC 60559 arithmetic operations are always quiet, quiet NaNs (along with infinities) are sufficient for closure of the arithmetic.

roundToIntegralExact	rint	7.12.10.4, F.10.7.4
nextUp	nextup	7.12.12.5, F.10.9.5
nextDown	nextdown	7.12.12.6, F.10.9.6
getPayload	getpayload	F.10.14.2
setPayload	setpayload	F.10.14.3
setPayloadSignaling	setpayloadsig	F.10.14.4
quantize	quantize	7.12.16.1
sameQuantum	samequantum	7.12.16.2
quantum	quantum	7.12.16.3
encodeDecimal	encodedec	7.12.17.2
decodeDecimal	decodedec	7.12.17.3
encodeBinary	encodebin	7.12.17.4
decodeBinary	decodebin	7.12.17.5
remainder	remainder, remquo	7.12.11.2, F.10.8.2, 7.12.11.3, F.10.8.3
maximum	fmaximum	7.12.13.4, F.10.10.4
minimum	fminimum	7.12.13.5, F.10.10.4
maximumMagnitude	fmaximum_mag	7.12.13.6, F.10.10.4
minimumMagnitude	fminimum_mag	7.12.13.7, F.10.10.4
maximumNumber	fmaximum_num	7.12.13.8, F.10.10.5
minimumNumber	fminimum_num	7.12.13.9, F.10.10.5
maximumMagnitudeNumber	fmaximum_mag_num	7.12.13.10, F.10.10.5
minimumMagnitudeNumber	fminimum_mag_num	7.12.13.11, F.10.10.5
scaleB	scalbn, scalbln	7.12.7.19, F.10.4.19
logB	logb, ilogb, llogb	7.12.7.17, F.10.4.17, 7.12.7.8, F.10.4.8, 7.12.7.10, F.10.4.10
addition	+, fadd, faddl, daddl	6.5.7, 7.12.15.2, F.10.12
subtraction	-, fsub, fsubl, dsubl	6.5.7, 7.12.15.3, F.10.12
multiplication	*, fmul, fmul, dmull	6.5.6, 7.12.15.4, F.10.12
division	/, fdiv, fdivl, ddivl	6.5.6, 7.12.15.5, F.10.12
squareRoot	sqrt, fsqrt, fsqrtl, dsqrtl	7.12.8.10, F.10.5.10, 7.12.15.7, F.10.12
fusedMultiplyAdd	fma, ffma, fmal, dfmal	7.12.14.1, F.10.11.1, 7.12.15.6, F.10.12

convertFromInt	cast and implicit conversion	6.3.2.4, 6.5.5
convertToIntegerTiesToEven convertToIntegerTowardZero convertToIntegerTowardPositive convertToIntegerTowardNegative	fromfp, ufromfp	7.12.10.10, F.10.7.10
convertToIntegerTiesToAway	fromfp, ufromfp, lround, llround	7.12.10.10, F.10.7.10, 7.12.10.7, F.10.7.7
convertToIntegerExactTiesToEven convertToIntegerExactTowardZero convertToIntegerExactTowardPositive convertToIntegerExactTowardNegative convertToIntegerExactTiesToAway	fromfpx, ufromfpx	7.12.10.11, F.10.7.11
convertFormat - different formats	cast and implicit conversions	6.3.2.5, 6.5.5
convertFormat - same format	canonicalize	7.12.12.7, F.10.9.7
convertFromDecimalCharacter	strtod, wcstod, scanf, wscanf, decimal floating constants	7.24.2.6, 7.31.4.2.2, 7.23.6.5, 7.31.2.13, F.5
convertToDecimalCharacter	printf, wprintf, strfromd	7.23.6.4, 7.31.2.12, 7.24.2.4, F.5
convertFromHexCharacter	strtod, wcstod, scanf, wscanf, hexadecimal floating constants	7.24.2.6, 7.31.4.2.2, 7.23.6.5, 7.31.2.13, F.5
convertToHexCharacter	printf, wprintf, strfromd	7.23.6.4, 7.31.2.12, 7.24.2.4, F.5
copy	memcpy, memmove, +(x)	7.26.2.1, 7.26.2.3
negate	-(x)	6.5.4.4
abs	fabs	7.12.8.3, F.10.5.3
copySign	copysign	7.12.12.1, F.10.9.1
compareQuietEqual	==	6.5.10, F.9.4
compareQuietNotEqual	!=	6.5.10, F.9.4
compareSignalingEqual	iseqsig	7.12.18.8, F.10.15.2
compareSignalingGreater	>	6.5.9, F.9.4
compareSignalingGreaterEqual	>=	6.5.9, F.9.4
compareSignalingLess	<	6.5.9, F.9.4
compareSignalingLessEqual	<=	6.5.9, F.9.4
compareSignalingNotEqual	! iseqsig(x)	7.12.18.8, F.10.15.2
compareSignalingNotGreater	! (x > y)	6.5.9, F.9.4
compareSignalingLessUnordered	! (x >= y)	6.5.9, F.9.4
compareSignalingNotLess	! (x < y)	6.5.9, F.9.4
compareSignalingGreaterUnordered	! (x <= y)	6.5.9, F.9.4
compareQuietGreater	isgreater	7.12.18.2

compareQuietGreaterEqual	isgreaterequal	7.12.18.3
compareQuietLess	isless	7.12.18.4
compareQuietLessEqual	islessequal	7.12.18.5
compareQuietUnordered	isunordered	7.12.18.7
compareQuietNotGreater	! isgreater(x, y)	7.12.18.2
compareQuietLessUnordered	! isgreaterequal(x, y)	7.12.18.3
compareQuietNotLess	! isless(x, y)	7.12.18.4
compareQuietGreaterUnordered	! islessequal(x, y)	7.12.18.5
compareQuietOrdered	! isunordered(x, y)	7.12.18.7
class	fpclassify, signbit, issignaling	7.12.4.2, 7.12.4.8, 7.12.4.9
isSignMinus	signbit	7.12.4.8
isNormal	isnormal	7.12.4.7
isFinite	isfinite	7.12.4.4
isZero	iszero	7.12.4.11
isSubnormal	issubnormal	7.12.4.10
isInfinite	isinf	7.12.4.5
isNaN	isnan	7.12.4.6
isSignaling	issignaling	7.12.4.9
isCanonical	iscanonical	7.12.4.3
radix	FLT_RADIX	5.3.5.3.3
totalOrder	totalorder	F.10.13.2
totalOrderMag	totalordermag	F.10.13.3
lowerFlags	feclearexcept	7.6.5.2
raiseFlags	fesetexcept	7.6.5.5
testFlags	fetestexcept	7.6.5.8
testSavedFlags	fetestexceptflag	7.6.5.7
restoreFlags	fesetexceptflag	7.6.5.6
saveAllFlags	fegetexceptflag	7.6.5.3
getBinaryRoundingDirection	fegetround	7.6.6.3
setBinaryRoundingDirection	fesetround	7.6.6.6
saveModes	fegetmode	7.6.6.2

restoreModes	fesetmode	7.6.6.5
defaultModes	fesetmode(FE_DFL_MODE)	7.6.6.5, 7.6

The ISO/IEC 60559 requirement that certain of its operations be provided for operands of different formats (of the same radix) is satisfied by C's usual arithmetic conversions (6.3.2.8) and function-call argument conversions (6.5.3.3). For example, the following operations take **float f** and **double d** inputs and produce a **long double** result:

```
(long double)f * d
powl(f, d)
```

The functions **fmin** and **fmax** have been superseded by **fminimum_num** and **fmaximum_num**. The **fmin** and **fmax** functions provide the minNum and maxNum operations specified in (the superseded) ISO/IEC/IEEE 60559:2011.

Whether C assignment (6.5.17) (and conversion as if by assignment) to the same format is an ISO/IEC 60559 convertFormat or copy operation⁴²⁵⁾ is implementation-defined, even if `<fenv.h>` defines the macro **FE_SNANS_ALWAYS_SIGNAL** (F.2.2). If the return expression of a **return** statement is evaluated to the floating-point format of the return type, it is implementation-defined whether a convertFormat operation is applied to the result of the return expression.

The unary **+** and **-** operators raise no floating-point exceptions, even if the operand is a signaling NaN.

The C classification macros **fpclassify**, **iscanonical**, **isfinite**, **isinf**, **isnan**, **isnormal**, **issignaling**, **issubnormal**, **iszero**, and **signbit** provide the ISO/IEC 60559 operations indicated in Table F.2 provided their arguments are in the format of their semantic type. Then these macros raise no floating-point exceptions, even if an argument is a signaling NaN.

The **signbit** macro, providing the ISO/IEC 60559 **isSignMinus** operation, determines the sign of its argument value as the sign bit of the value's representation. This applies to all values, including NaNs whose sign bit is not generally interpreted by ISO/IEC 60559.

The C **nearbyint** functions (7.12.10.3, F.10.7.3) provide the nearbyinteger function recommended in the Appendix to (superseded) ANSI/IEEE 854–1987.

The C **nextafter** (7.12.12.3, F.10.9.3) and **nexttoward** (7.12.12.4, F.10.9.4) functions provide the **nextafter** function recommended in the Appendix to (superseded) IEC 60559:1989 (but with a minor change to better handle signed zeros).

The macros (7.6) **FE_DOWNWARD**, **FE_TONEAREST**, **FE_TONEARESTFROMZERO**, **FE_TOWARDZERO**, and **FE_UPWARD**, which are used in conjunction with the **fegetround** and **fesetround** functions and the **FENV_ROUND** pragma, represent the ISO/IEC 60559 rounding-direction attributes roundTowardNegative, roundTiesToEven, roundTiesToAway, roundTowardZero, and roundTowardPositive, respectively, for binary floating-point arithmetic. Support for the roundTiesToAway attribute for binary floating-point arithmetic, and hence for the **FE_TONEARESTFROMZERO** macro, is optional.

The C **fegetenv** (7.6.7.2), **feholdexcept** (7.6.7.3), **fesetenv** (7.6.7.4) and **feupdateenv** (7.6.7.5) functions provide a facility to manage the dynamic floating-point environment, comprising the ISO/IEC 60559 status flags and dynamic control modes.

ISO/IEC 60559 requires operations with specified operand and result formats. Therefore, math functions that are bound to ISO/IEC 60559 operations (see Table F.2) shall remove any extra range and precision from arguments or results.

ISO/IEC 60559 requires operations that round their result to formats the same as and wider than the operands, in addition to the operations that round their result to narrower formats (see 7.12.15).

⁴²⁵⁾Where the source and destination formats are the same, convertFormat operations differ from copy operations in that convertFormat operations raise the “invalid” floating-point exception on signaling NaN inputs and do not propagate non-canonical encodings.

Operators (+, -, *, and /) whose evaluation formats are wider than the semantic type (5.3.5.3.3) may not support some of the ISO/IEC 60559 operations, because getting a result in a given format can require a cast that can introduce an extra rounding error. The functions that round result to narrower type (7.12.15) provide the ISO/IEC 60559 operations that round result to same and wider (as well as narrower) formats, in those cases where built-in operators and casts do not. For example, **ddivl(x, y)** computes a correctly rounded **double** divide of **float x** by **float y**, regardless of the evaluation method.

Decimal versions of the **remquo** library function are not provided. (The decimal **remainder** functions provide the remainder operation defined by ISO/IEC 60559.)

The binding for the **convertFormat** operation applies to all conversions among ISO/IEC 60559 formats. Therefore, for implementations that conform to this annex, conversions between decimal floating types and standard floating types with ISO/IEC 60559 formats are correctly rounded and raise floating-point exceptions as specified in ISO/IEC 60559.

ISO/IEC 60559 specifies the **convertFromHexCharacter** and **convertToHexCharacter** operations only for binary floating-point arithmetic.

The integer constant **10** provides the radix operation defined in ISO/IEC 60559 for decimal floating-point arithmetic.

The **fe_dec_getround** (7.6.6.4) and **fe_dec_setround** (7.6.6.7) functions provide the **getDecimalRoundingDirection** and **setDecimalRoundingDirection** operations defined in ISO/IEC 60559 for decimal floating-point arithmetic. The macros (7.6) **FE_DEC_DOWNWARD**, **FE_DEC_TONEAREST**, **FE_DEC_TONEARESTFROMZERO**, **FE_DEC_TOWARDZERO**, and **FE_DEC_UPWARD**, which are used in conjunction with the **fe_dec_getround** and **fe_dec_setround** functions and the **FENV_DEC_ROUND** pragma, represent the ISO/IEC 60559 rounding-direction attributes **roundTowardNegative**, **roundTiesToEven**, **roundTiesToAway**, **roundTowardZero**, and **roundTowardPositive**, respectively, for decimal floating-point arithmetic.

The **llquantexpdN** (7.12.16.4) functions compute the (quantum) exponent q defined in ISO/IEC 60559 for decimal numbers viewed as having integer significands.

The C functions in Table F.3 correspond to mathematical operations recommended by ISO/IEC 60559. However, correct rounding, which ISO/IEC 60559 specifies for its operations, is not required for the C functions in the table. 7.33.9 (potentially) reserves **cr_** prefixed names for functions fully matching the ISO/IEC 60559 mathematical operations. In the table, the C functions are represented by the function name without a type suffix.

Table F.3 — ISO/IEC 60559 operation to C function

ISO/IEC 60559 operation	C function	Clause
exp	exp	7.12.7.1, F.10.4.1
expm1	expm1	7.12.7.6, F.10.4.6
exp2	exp2	7.12.7.4, F.10.4.4
exp2m1	exp2m1	7.12.7.5, F.10.4.5
exp10	exp10	7.12.7.2, F.10.4.2
exp10m1	exp10m1	7.12.7.3, F.10.4.3
log	log	7.12.7.11, F.10.4.11
log2	log2	7.12.7.15, F.10.4.15
log10	log10	7.12.7.12, F.10.4.12
logp1	log1p, logp1	7.12.7.14, F.10.4.14
log2p1	log2p1	7.12.7.16, F.10.4.16
log10p1	log10p1	7.12.7.13, F.10.4.13
hypot	hypot	7.12.8.4, F.10.5.4
rSqrt	rsqrt	7.12.8.9, F.10.5.9
compound	compoundn	7.12.8.2, F.10.5.2
rootn	rootn	7.12.8.8, F.10.5.8

Table F.3 — ISO/IEC 60559 operation to C function

ISO/IEC 60559 operation	C function	Clause
pown	pown	7.12.8.6, F.10.5.6
pow	pow	7.12.8.5, F.10.5.5
powr	powr	7.12.8.7, F.10.5.7
sin	sin	7.12.5.6, F.10.2.6
cos	cos	7.12.5.5, F.10.2.5
tan	tan	7.12.5.7, F.10.2.7
sinPi	sinpi	7.12.5.13, F.10.2.13
cosPi	cospi	7.12.5.12, F.10.2.12
tanPi	tanpi	7.12.5.14, F.10.2.14
asinPi	asinpi	7.12.5.9, F.10.2.9
acosPi	acospi	7.12.5.8, F.10.2.8
atanPi	atanpi	7.12.5.10, F.10.2.10
atan2Pi	atan2pi	7.12.5.11, F.10.2.11
asin	asin	7.12.5.2, F.10.2.2
acos	acos	7.12.5.1, F.10.2.1
atan	atan	7.12.5.3, F.10.2.3
atan2	atan2	7.12.5.4, F.10.2.4
sinh	sinh	7.12.6.5, F.10.3.5
cosh	cosh	7.12.6.4, F.10.3.4
tanh	tanh	7.12.6.6, F.10.3.6
asinh	asinh	7.12.6.2, F.10.3.2
acosh	acosh	7.12.6.1, F.10.3.1
atanh	atanh	7.12.6.3, F.10.3.3

F.4 Floating to integer conversion

If the integer type is **bool**, 6.3.2.2 applies and the conversion raises no floating-point exceptions if the floating-point value is not a signaling NaN. Otherwise, if the floating value is infinite or NaN or if the integral part of the floating value exceeds the range of the integer type, then the “invalid” floating-point exception is raised and the resulting value is unspecified. Otherwise, the resulting value is determined by 6.3.2.4. Conversion of an integral floating value that does not exceed the range of the integer type raises no floating-point exceptions; whether conversion of a non-integral floating value raises the “inexact” floating-point exception is unspecified.⁴²⁶⁾

F.5 Conversions between binary floating types and decimal character sequences

The `<float.h>` header defines the macro

CR_DECIMAL_DIG

if and only if `__STDC_WANT_IEC_60559_EXT__` is defined as a macro at the point in the source file where `<float.h>` is first included. If defined, **CR_DECIMAL_DIG** expands to an integer constant expression suitable for use in conditional expression inclusion preprocessing directives whose value is a number such that conversions between all supported ISO/IEC 60559 binary formats and character sequences with at most **CR_DECIMAL_DIG** significant decimal digits are correctly rounded. The value of **CR_DECIMAL_DIG** shall be at least $M + 3$, where M is the maximum value of the **T_DECIMAL_DIG** macros for ISO/IEC 60559 binary formats. If the implementation correctly rounds for all numbers of significant decimal digits, then **CR_DECIMAL_DIG** shall have the value of the macro **UINTMAX_MAX**.

Conversions of types with ISO/IEC 60559 binary formats to character sequences with more than

⁴²⁶⁾ISO/IEC 60559 recommends that implicit floating-to-integer conversions raise the “inexact” floating-point exception for non-integer in-range values. In those cases where it matters, library functions can be used to effect such conversions with or without raising the “inexact” floating-point exception. See `fromfp`, `ufromfp`, `fromfpx`, `ufromfpx`, `rint`, `lrint`, `llrint`, and `nearbyint` in `<math.h>`.

CR_DECIMAL_DIG significant decimal digits shall correctly round to **CR_DECIMAL_DIG** significant digits and pad zeros on the right.

Conversions from character sequences with more than **CR_DECIMAL_DIG** significant decimal digits to types with ISO/IEC 60559 binary formats shall correctly round to an intermediate character sequence with **CR_DECIMAL_DIG** significant decimal digits, according to the applicable rounding direction, and correctly round the intermediate result (having **CR_DECIMAL_DIG** significant decimal digits) to the destination type. The “inexact” floating-point exception is raised (once) if either conversion is inexact.⁴²⁷⁾ (The second conversion can raise the “overflow” or “underflow” floating-point exception.)

The specification in this subclause assures conversion between ISO/IEC 60559 binary format and decimal character sequence follows all pertinent recommended practice. It also assures conversion from ISO/IEC 60559 format to decimal character sequence with at least **T_DECIMAL_DIG** digits and back, using to-nearest rounding, is the identity function, where *T* is the macro prefix for the format.

Functions such as **strtod** that convert character sequences to floating types honor the rounding direction. Hence, if the rounding direction can be upward or downward, the implementation cannot convert a minus-signed sequence by arithmetically negating the converted unsigned sequence.

NOTE ISO/IEC 60559 specifies that conversion to one-digit character strings using **roundTiesToEven**, when both choices have an odd least significant digit, produce the value with the larger magnitude. This can happen with **9.5e2** whose nearest neighbors are **9.e2** and **1.e3**, both of which have a single odd digit in the significand part.

F.6 The return statement

If the return expression is evaluated in a floating-point format different from the return type, the expression is converted as if by assignment⁴²⁸⁾ to the return type of the function and the resulting value is returned to the caller.

F.7 Contracted expressions

A contracted expression is correctly rounded (once) and treats infinities, NaNs, signed zeros, subnormals, and the rounding directions in a manner consistent with the basic arithmetic operations covered by ISO/IEC 60559.

Recommended practice

A contracted expression should raise floating-point exceptions in a manner generally consistent with the basic arithmetic operations.

F.8 Floating-point environment

F.8.1 General

The floating-point environment defined in `<fenv.h>` includes the ISO/IEC 60559 floating-point exception status flags and rounding-direction control modes. It may also include other floating-point status or modes that the implementation provides as extensions.⁴²⁹⁾

This annex does not include support for ISO/IEC 60559’s optional alternate exception handling. The specification in this annex assumes ISO/IEC 60559 default exception handling: the flag is set, a default result is delivered, and execution continues. Implementations may provide alternate exception handling as an extension.

F.8.2 Environment management

ISO/IEC 60559 requires that floating-point operations implicitly raise floating-point exception status flags, and that rounding control modes can be set explicitly to affect result values of floating-point operations. These changes to the floating-point state are treated as side effects which respect

⁴²⁷⁾ The intermediate conversion is exact only if all input digits after the first **CR_DECIMAL_DIG** digits are 0.

⁴²⁸⁾ Assignment removes any extra range and precision.

⁴²⁹⁾ Dynamic rounding precision and trap enablement modes are examples of such extensions.

sequence points.⁴³⁰⁾

F.8.3 Translation

During translation, constant rounding direction modes (7.6.3) are in effect where specified. Elsewhere, during translation the ISO/IEC 60559 default modes are in effect:

- The rounding direction mode is rounding to nearest.
- The rounding precision mode (if supported) is set so that results are not shortened.
- Trapping or stopping (if supported) is disabled on all floating-point exceptions.

Recommended practice

The implementation should produce a diagnostic message for each translation-time floating-point exception, other than “inexact”,⁴³¹⁾ the implementation should then proceed with the translation of the program.

F.8.4 Execution

At program startup the dynamic floating-point environment is initialized as prescribed by ISO/IEC 60559:

- All floating-point exception status flags are cleared.
- The dynamic rounding direction mode is rounding to nearest.
- The dynamic rounding precision mode (if supported) is set so that results are not shortened.
- Trapping or stopping (if supported) is disabled on all floating-point exceptions.

F.8.5 Constant expressions

An arithmetic constant expression of floating type, other than one in an initializer for an object that has static or thread storage duration or that is declared with storage-class specifier **constexpr**, is evaluated (as if) during execution; thus, it is affected by any operative floating-point control modes and raises floating-point exceptions as required by ISO/IEC 60559 (provided the state for the **FENV_ACCESS** pragma is “on”).⁴³²⁾

EXAMPLE

```
#include <fenv.h>
#pragma STDC FENV_ACCESS ON
void f(void)
{
    constexpr double v = 0.0/0.0; // does not raise an exception
    float w[] = { 0.0/0.0 }; // raises an exception
    static float x = 0.0/0.0; // does not raise an exception
    float y = 0.0/0.0; // raises an exception
    double z = 0.0/0.0; // raises an exception
    /* ... */
}
```

⁴³⁰⁾ If the state for the **FENV_ACCESS** pragma is “off”, the implementation is free to assume the dynamic floating-point control modes will be the default ones and the floating-point status flags will not be tested, which allows certain optimizations (see F.9).

⁴³¹⁾ As floating constants are converted to appropriate internal representations at translation time, their conversion is subject to constant or default rounding modes and raises no execution-time floating-point exceptions (even where the state of the **FENV_ACCESS** pragma is “on”). Library functions, for example **strtod**, provide execution-time conversion of numeric strings.

⁴³²⁾ Where the state for the **FENV_ACCESS** pragma is “on”, results of inexact expressions like **1.0/3.0** are affected by rounding modes set at execution time, and expressions such as **0.0/0.0** and **1.0/0.0** generate execution-time floating-point exceptions. The programmer can achieve the efficiency of translation-time evaluation through static initialization, such as

```
const static double one_third = 1.0/3.0;
```

For the **static** and **constexpr** initializations, the division is done at translation time, raising no (execution-time) floating-point exceptions. On the other hand, for the three automatic initializations the invalid division occurs at execution time.

F.8.6 Initialization

All computation for automatic initialization is done (as if) at execution time; thus, it is affected by any operative modes and raises floating-point exceptions as required by ISO/IEC 60559 (provided the state for the **FENV_ACCESS** pragma is “on”). All computation for initialization of objects that have static or thread storage duration, or that are declared with storage-class specifier **constexpr**, is done (as if) at translation time.

EXAMPLE

```
#include <fenv.h>
#pragma STDC FENV_ACCESS ON
void f(void)
{
    constexpr float t = (float)1.1e75; // does not raise exceptions
    float u[] = { 1.1e75 }; // raises exceptions
    static float v = 1.1e75; // does not raise exceptions
    float w = 1.1e75; // raises exceptions
    double x = 1.1e75; // may raise exceptions
    float y = 1.1e75f; // may raise exceptions
    long double z = 1.1e75; // does not raise exceptions
    /* ... */
}
```

The **constexpr** initialization of **t** and the static initialization of **v** raise no (execution-time) floating-point exceptions because their computation is done at translation time. The automatic initialization of **u** and **w** require an execution-time conversion to **float** of the wider value **1.1e75**, which raises floating-point exceptions. The automatic initializations of **x** and **y** entail execution-time conversion; however, in some expression evaluation methods, the conversions are not to a narrower format, in which case no floating-point exception is raised.⁴³³⁾ The automatic initialization of **z** entails execution-time conversion, but not to a narrower format, so no floating-point exception is raised. The conversions of the floating constants **1.1e75** and **1.1e75f** to their internal representations occur at translation time in all cases.

F.8.7 Changing the environment

Operations defined in 6.5.1 and functions and macros defined for the standard libraries change floating-point status flags and control modes just as indicated by their specifications (including conformance to ISO/IEC 60559). They do not change flags or modes (so as to be detectable by the user) in any other cases.

If the floating-point exceptions represented by the argument to the **feraiseexcept** function in **<fenv.h>** include both “overflow” and “inexact”, then “overflow” is raised before “inexact”. Similarly, if the represented exceptions include both “underflow” and “inexact”, then “underflow” is raised before “inexact”.

F.9 Optimization

F.9.1 General

This section identifies code transformations that can subvert ISO/IEC 60559-specified behavior, and others that do not.

⁴³³⁾Use of **float_t** and **double_t** variables increases the likelihood of translation-time computation. For example, the automatic initialization

```
double_t x = 1.1e75;
```

can be done at translation time, regardless of the expression evaluation method.

F.9.2 Global transformations

Floating-point arithmetic operations and external function calls can entail side effects which optimization shall honor, at least where the state of the **FENV_ACCESS** pragma is “on”. The flags and modes in the floating-point environment can be regarded as global variables; floating-point operations (+, *, etc.) implicitly read the modes and write the flags.

Concern about side effects can inhibit code motion and removal of seemingly useless code. For example, in

```
#include <fenv.h>
#pragma STDC FENV_ACCESS ON
void f(double x)
{
    /* ... */
    for (i = 0; i < n; i++) x + 1;
    /* ... */
}
```

x+1 may raise floating-point exceptions, so cannot be removed. And since the loop body may not execute (maybe $0 \geq n$), **x+1** cannot be moved out of the loop. (Of course these optimizations are valid if the implementation can rule out the nettlesome cases.)

This specification does not require support for trap handlers that maintain information about the order or count of floating-point exceptions. Therefore, between function calls, the side effects due to floating-point exceptions are not required be precise: the actual order and number of occurrences of floating-point exceptions (> 1) may vary from what the source code expresses. Thus, the preceding loop could be treated as

```
if (0 < n) x + 1;
```

F.9.3 Expression transformations

Valid expression transformations shall preserve numerical values.

The equivalences noted in the following description apply to expressions of standard floating types.

$x/2 \leftrightarrow x \times 0.5$	Although similar transformations involving inexact constants generally do not yield equivalent expressions, if the constants are exact then such transformations can be made on ISO/IEC 60559 machines and others that round perfectly.
$1 \times x$ and $x/1 \rightarrow x$	The expressions $1 \times x$, $x/1$, and x may be regarded as equivalent (on ISO/IEC 60559 machines, among others). ⁴³⁴⁾
$x/x \rightarrow 1.0$	The expressions x/x and 1.0 are not equivalent if x can be zero, infinite, or NaN.
$x - y \leftrightarrow x + (-y)$	The expressions $x - y$, $x + (-y)$, and $(-y) + x$ are equivalent (on ISO/IEC 60559 machines, among others).
$x - y \leftrightarrow -(y - x)$	The expressions $x - y$ and $-(y - x)$ are not equivalent because $1 - 1$ is +0 but $-(1 - 1)$ is -0 (in the default rounding direction). ⁴³⁵⁾
$x - x \rightarrow 0.0$	The expressions $x - x$ and 0.0 are not equivalent if x is a NaN or infinite.

⁴³⁴⁾Implementations can have non-required features that invalidate these and other transformations that remove arithmetic operators. Examples include strict support for signaling NaNs (an optional feature) and alternate exception handling (not included in this specification).

⁴³⁵⁾ISO/IEC 60559 prescribes a signed zero to preserve mathematical identities across certain discontinuities. Examples include:

$1/(1/\pm\infty)$ is $\pm\infty$

and

$\text{conj}(\text{csqrt}(z))$ is $\text{csqrt}(\text{conj}(z))$,

for complex z .

$0 \times x \rightarrow 0.0$	The expressions $0 \times x$ and 0.0 are not equivalent if x is a NaN, infinite, or -0 .
$x + 0 \rightarrow x$	The expressions $x + 0$ and x are not equivalent if x is -0 , because $(-0) + (+0)$ yields $+0$ (in the default rounding direction), not -0 .
$x - 0 \rightarrow x$	$(+0) - (+0)$ yields -0 when rounding is downward (toward $-\infty$), but $+0$ otherwise, and $(-0) - (+0)$ always yields -0 ; so, if the state of the FENV_ACCESS pragma is “off”, promising default rounding, then the implementation can replace $x - 0$ by x , even if x may be zero.
$-x \leftrightarrow 0 - x$	The expressions $-x$ and $0 - x$ are not equivalent if x is $+0$, because $-(+0)$ yields -0 , but $0 - (+0)$ yields $+0$ (unless rounding is downward).

For expressions of decimal floating types, transformations shall preserve quantum exponents, as well as numerical values (5.3.5.3.4).

EXAMPLE The computation $1. \times x \rightarrow x$ is valid for decimal floating-point expressions x , but $1.0 \times x \rightarrow x$ is not:

$1. \times 12.34$	$=$	$(+1, 1, 0) \times (+1, 1234, -2)$	<i>yields</i>	$(+1, 1234, -2)$	$=$	12.34
1.0×12.34	$=$	$(+1, 10, -1) \times (+1, 1234, -2)$	<i>yields</i>	$(+1, 12340, -3)$	$=$	12.340

In the second case, the factor 12.34 and the result 12.340 have different quantum exponents, demonstrating that $1.0 \times x$ and x are not equivalent expressions.

F.9.4 Relational operators

$x \neq x \rightarrow \text{false}$	The expression $x \neq x$ is true if x is a NaN.
$x = x \rightarrow \text{true}$	The expression $x = x$ is false if x is a NaN.
$x < y \rightarrow \text{isless}(x, y)$	(and similarly for $\leq, >, \geq$) Though equal, these expressions are not equivalent because of side effects when x or y is a NaN and the state of the FENV_ACCESS pragma is “on”. This transformation, which would be desirable if extra code were required to cause the “invalid” floating-point exception for unordered cases, could be performed provided the state of the FENV_ACCESS pragma is “off”.

The sense of relational operators shall be maintained. This includes handling unordered cases as expressed by the source code.

EXAMPLE

```
// calls g and raises “invalid” if a and b are unordered
if (a < b)
    f();
else
    g();
```

is not equivalent to

```
// calls f and raises “invalid” if a and b are unordered
if (a >= b)
    g();
else
    f();
```

nor to

```
// calls f without raising “invalid” if a and b are unordered
if (isgreaterqual(a,b))
    g();
else
    f();
```


nor, unless the state of the **FENV_ACCESS** pragma is “off”, to

```
// calls g without raising “invalid” if a and b are unordered
if (isless(a,b))
    f();
else
    g();
```

but is equivalent to

```
if (!(a < b))
    g();
else
    f();
```

F.9.5 Constant arithmetic

The implementation shall honor floating-point exceptions raised by execution-time constant arithmetic wherever the state of the **FENV_ACCESS** pragma is “on”. (See F.8.5 and F.8.6.) An operation on constants that raises no floating-point exception can be folded during translation, except, if the state of the **FENV_ACCESS** pragma is “on”, a further check is required to assure that changing the rounding direction to downward does not alter the sign of the result,⁴³⁶⁾ and implementations that support dynamic rounding precision modes shall assure further that the result of the operation raises no floating-point exception when converted to the semantic type of the operation.

F.10 Mathematics <math.h> and <tgmath.h>

F.10.1 General

This subclause contains specifications of <math.h> and <tgmath.h> facilities that are particularly suited for ISO/IEC 60559 implementations.

The Standard C macro **HUGE_VAL** and its **float** and **long double** analogs, **HUGE_VALF** and **HUGE_VALL**, expand to expressions whose values are positive infinities.

For each single-argument function **f** in <math.h> whose mathematical counterpart is symmetric (even), **f(-x)** is **f(x)** for all rounding modes and for all **x** in the (valid) domain of the function. For each single-argument function **f** in <math.h> whose mathematical counterpart is antisymmetric (odd), **f(-x)** is **-f(x)** for the ISO/IEC 60559 rounding modes **roundTiesToEven**, **roundTiesToAway**, and **roundTowardZero**, and for all **x** in the (valid) domain of the function. The **atan2** and **atan2pi** functions are odd in their first argument.

Special cases for functions in <math.h> are covered directly or indirectly by ISO/IEC 60559. The functions that ISO/IEC 60559 specifies directly are identified in F.3. The other functions in <math.h> treat infinities, NaNs, signed zeros, subnormals, and (provided the state of the **FENV_ACCESS** pragma is “on”) the floating-point status flags in a manner consistent with ISO/IEC 60559 operations.

The expression **math_errhandling** & **MATH_ERREXCEPT** shall evaluate to a nonzero value.

The functions bound to operations in ISO/IEC 60559 (F.3) are fully specified by ISO/IEC 60559, including rounding behaviors and floating-point exceptions.

The “invalid” and “divide-by-zero” floating-point exceptions are raised as specified in subsequent subclauses of this annex.

The “overflow” floating-point exception is raised whenever an infinity — or, because of rounding direction, a maximal-magnitude finite number — is returned in lieu of a finite value whose magnitude is too large.

The “underflow” floating-point exception is raised whenever a computed result is tiny⁴³⁷⁾ and the returned result is inexact.

⁴³⁶⁾ **0-0** yields **-0** instead of **+0** just when the rounding direction is downward.

⁴³⁷⁾ Tiny generally indicates having a magnitude in the subnormal range. See ISO/IEC 60559 for details about detecting tininess.

Whether or when library functions not listed in Table F.2 raise the “inexact” floating-point exception is unspecified, unless stated otherwise.

Whether or when library functions not listed in Table F.2 raise a spurious “underflow” floating-point exception is not specified by this annex.⁴³⁸⁾

As implied by F.8.7, library functions do not raise spurious “invalid”, “overflow”, or “divide-by-zero” floating-point exceptions (detectable by the user).

Whether the functions not listed in Table F.2 honor the rounding direction mode is implementation-defined, unless explicitly specified otherwise.

Functions with a NaN argument return a NaN result and raise no floating-point exception, except where explicitly stated otherwise.

The specifications in the following subclauses append to the definitions in `<math.h>`. For families of functions, the specifications apply to all the functions even though only the principal function is shown. Unless otherwise specified, where the symbol “±” occurs in both an argument and the result, the result has the same sign as the argument.

Recommended practice

ISO/IEC 60559 specifies correct rounding for the operations in Table F.2 recommended by ISO/IEC 60559, and thereby preserves useful mathematical properties such as symmetry, monotonicity, and periodicity. The corresponding functions with (potentially) reserved `cr_`-prefixed names (7.33.9) do the same. The C functions in the table, however, are not required to be correctly rounded, but implementations should still preserve as many of these useful mathematical properties as possible.

If a function with one or more NaN arguments returns a NaN result, the result should be the same as one of the NaN arguments (after possible type conversion), except perhaps for the sign.

F.10.2 Trigonometric functions

F.10.2.1 The `acos` functions

- `acos(1)` returns +0.
- `acos(x)` returns a NaN and raises the “invalid” floating-point exception for $|x| > 1$.

F.10.2.2 The `asin` functions

- `asin(±0)` returns ±0.
- `asin(x)` returns a NaN and raises the “invalid” floating-point exception for $|x| > 1$.

F.10.2.3 The `atan` functions

- `atan(±0)` returns ±0.
- `atan(±∞)` returns $\pm \frac{\pi}{2}$.

F.10.2.4 The `atan2` functions

- `atan2(±0, −0)` returns $\pm\pi$.⁴³⁹⁾
- `atan2(±0, +0)` returns ±0.
- `atan2(±0, x)` returns $\pm\pi$ for $x < 0$.
- `atan2(±0, x)` returns ±0 for $x > 0$.

⁴³⁸⁾It is intended that spurious “underflow” and “inexact” floating-point exceptions are raised only if avoiding them would be too costly. 7.12.2 specifies that if `math_errhandling & MATH_ERREXCEPT` is nonzero, then an “underflow” floating-point exception shall not be raised unless an underflow range error occurs.

⁴³⁹⁾`atan2(0, 0)` does not raise the “invalid” floating-point exception, nor does `atan2(y, 0)` raise the “divide-by-zero” floating-point exception.

- **atan2**($y, \pm 0$) returns $-\frac{\pi}{2}$ for $y < 0$.
- **atan2**($y, \pm 0$) returns $\frac{\pi}{2}$ for $y > 0$.
- **atan2**($\pm y, -\infty$) returns $\pm\pi$ for finite $y > 0$.
- **atan2**($\pm y, +\infty$) returns ± 0 for finite $y > 0$.
- **atan2**($\pm\infty, x$) returns $\pm\frac{\pi}{2}$ for finite x .
- **atan2**($\pm\infty, -\infty$) returns $\pm\frac{3\pi}{4}$.
- **atan2**($\pm\infty, +\infty$) returns $\pm\frac{\pi}{4}$.

F.10.2.5 The cos functions

- **cos**(± 0) returns 1.
- **cos**($\pm\infty$) returns a NaN and raises the “invalid” floating-point exception.

F.10.2.6 The sin functions

- **sin**(± 0) returns ± 0 .
- **sin**($\pm\infty$) returns a NaN and raises the “invalid” floating-point exception.

F.10.2.7 The tan functions

- **tan**(± 0) returns ± 0 .
- **tan**($\pm\infty$) returns a NaN and raises the “invalid” floating-point exception.

F.10.2.8 The acospi functions

- **acospi**(+1) returns +0.
- **acospi**(x) returns a NaN and raises the “invalid” floating-point exception for $|x| > 1$.

F.10.2.9 The asinpi functions

- **asinpi**(± 0) returns ± 0 .
- **asinpi**(x) returns a NaN and raises the “invalid” floating-point exception for $|x| > 1$.

F.10.2.10 The atanpi functions

- **atanpi**(± 0) returns ± 0 .
- **atanpi**($\pm\infty$) returns $\pm\frac{1}{2}$.

F.10.2.11 The atan2pi functions

- **atan2pi**($\pm 0, -0$) returns ± 1 .⁴⁴⁰⁾
- **atan2pi**($\pm 0, +0$) returns ± 0 .
- **atan2pi**($\pm 0, x$) returns ± 1 for $x < 0$.
- **atan2pi**($\pm 0, x$) returns ± 0 for $x > 0$.
- **atan2pi**($y, \pm 0$) returns $-\frac{1}{2}$ for $y < 0$.
- **atan2pi**($y, \pm 0$) returns $+\frac{1}{2}$ for $y > 0$.
- **atan2pi**($\pm y, -\infty$) returns ± 1 for finite $y > 0$.
- **atan2pi**($\pm y, +\infty$) returns ± 0 for finite $y > 0$.
- **atan2pi**($\pm\infty, x$) returns $\pm\frac{1}{2}$ for finite x .
- **atan2pi**($\pm\infty, -\infty$) returns $\pm\frac{3}{4}$.
- **atan2pi**($\pm\infty, +\infty$) returns $\pm\frac{1}{4}$.

⁴⁴⁰⁾ **atan2pi**(0, 0) does not raise the “invalid” floating-point exception, nor does **atan2pi**($y, 0$) raise the “divide-by-zero” floating-point exception.

F.10.2.12 The `cospi` functions

- `cospi`(± 0) returns 1.
- `cospi`($n + \frac{1}{2}$) returns +0, for integers n .
- `cospi`($\pm\infty$) returns a NaN and raises the “invalid” floating-point exception.

F.10.2.13 The `sinpi` functions

- `sinpi`(± 0) returns ± 0 .
- `sinpi`($\pm n$) returns ± 0 , for positive integers n .
- `sinpi`($\pm\infty$) returns a NaN and raises the “invalid” floating-point exception.

F.10.2.14 The `tanpi` functions

- `tanpi`(± 0) returns ± 0 .
- `tanpi`(n) returns +0, for positive even and negative odd integers n .
- `tanpi`(n) returns −0, for positive odd and negative even integers n .
- `tanpi`($n + \frac{1}{2}$) returns $+\infty$ and raises the “divide-by-zero” floating-point exception, for even integers n .
- `tanpi`($n + \frac{1}{2}$) returns $-\infty$ and raises the “divide-by-zero” floating-point exception, for odd integers n .
- `tanpi`($\pm\infty$) returns a NaN and raises the “invalid” floating-point exception.

F.10.3 Hyperbolic functions**F.10.3.1 The `acosh` functions**

- `acosh`(1) returns +0.
- `acosh`(x) returns a NaN and raises the “invalid” floating-point exception for $x < 1$.
- `acosh`($+\infty$) returns $+\infty$.

F.10.3.2 The `asinh` functions

- `asinh`(± 0) returns ± 0 .
- `asinh`($\pm\infty$) returns $\pm\infty$.

F.10.3.3 The `atanh` functions

- `atanh`(± 0) returns ± 0 .
- `atanh`(± 1) returns $\pm\infty$ and raises the “divide-by-zero” floating-point exception.
- `atanh`(x) returns a NaN and raises the “invalid” floating-point exception for $|x| > 1$.

F.10.3.4 The `cosh` functions

- `cosh`(± 0) returns 1.
- `cosh`($\pm\infty$) returns $+\infty$.

F.10.3.5 The `sinh` functions

- `sinh`(± 0) returns ± 0 .
- `sinh`($\pm\infty$) returns $\pm\infty$.

F.10.3.6 The tanh functions

- **tanh**(± 0) returns ± 0 .
- **tanh**($\pm \infty$) returns ± 1 .

F.10.4 Exponential and logarithmic functions**F.10.4.1 The exp functions**

- **exp**(± 0) returns 1.
- **exp**($-\infty$) returns +0.
- **exp**($+\infty$) returns $+\infty$.

F.10.4.2 The exp10 functions

- **exp10**(± 0) returns 1.
- **exp10**($-\infty$) returns +0.
- **exp10**($+\infty$) returns $+\infty$.

F.10.4.3 The exp10m1 functions

- **exp10m1**(± 0) returns ± 0 .
- **exp10m1**($-\infty$) returns -1 .
- **exp10m1**($+\infty$) returns $+\infty$.

F.10.4.4 The exp2 functions

- **exp2**(± 0) returns 1.
- **exp2**($-\infty$) returns +0.
- **exp2**($+\infty$) returns $+\infty$.

F.10.4.5 The exp2m1 functions

- **exp2m1**(± 0) returns ± 0 .
- **exp2m1**($-\infty$) returns -1 .
- **exp2m1**($+\infty$) returns $+\infty$.

F.10.4.6 The expm1 functions

- **expm1**(± 0) returns ± 0 .
- **expm1**($-\infty$) returns -1 .
- **expm1**($+\infty$) returns $+\infty$.

F.10.4.7 The frexp functions

- **frexp**($\pm 0, p$) returns ± 0 , and stores 0 in the object pointed to by p .
- **frexp**($\pm \infty, p$) returns $\pm \infty$, and stores an unspecified value in the object pointed to by p .
- **frexp**(NaN, p) stores an unspecified value in the object pointed to by p (and returns a NaN).

frexp raises no floating-point exceptions if **value** is not a signaling NaN.

The returned value is independent of the current rounding direction mode.

On a binary system, the body of the **frexp** function may be

```
{
    *p = (value == 0 || !isfinite(value)) ? 0: (int)(1 + logb(value));
    return scalbn(value, -( *p ));
}
```

F.10.4.8 The `ilogb` functions

When the correct result is representable in the range of the return type, the returned value is exact and is independent of the current rounding direction mode.

If the correct result is outside the range of the return type, the numeric result is unspecified and the “invalid” floating-point exception is raised.

`ilogb(x)`, for x zero, infinite, or NaN, raises the “invalid” floating-point exception and returns the value specified in 7.12.7.8.

F.10.4.9 The `ldexp` functions

On a binary system, **`ldexp(x, exp)`** is equivalent to **`scalbn(x, exp)`**.

F.10.4.10 The `llogb` functions

The **`llogb`** functions are equivalent to the **`ilogb`** functions, except that the **`llogb`** functions determine a result in the **`long int`** type.

F.10.4.11 The `log` functions

- **`log(±0)`** returns $-\infty$ and raises the “divide-by-zero” floating-point exception.
- **`log(1)`** returns $+0$.
- **`log(x)`** returns a NaN and raises the “invalid” floating-point exception for $x < 0$.
- **`log(+∞)`** returns $+\infty$.

F.10.4.12 The `log10` functions

- **`log10(±0)`** returns $-\infty$ and raises the “divide-by-zero” floating-point exception.
- **`log10(1)`** returns $+0$.
- **`log10(x)`** returns a NaN and raises the “invalid” floating-point exception for $x < 0$.
- **`log10(+∞)`** returns $+\infty$.

F.10.4.13 The `log10p1` functions

- **`log10p1(±0)`** returns ± 0 .
- **`log10p1(−1)`** returns $-\infty$ and raises the “divide-by-zero” floating-point exception.
- **`log10p1(x)`** returns a NaN and raises the “invalid” floating-point exception for $x < -1$.
- **`log10p1(+∞)`** returns $+\infty$.

F.10.4.14 The `log1p` and `logp1` functions

- **`log1p(±0)`** returns ± 0 .
- **`log1p(−1)`** returns $-\infty$ and raises the “divide-by-zero” floating-point exception.
- **`log1p(x)`** returns a NaN and raises the “invalid” floating-point exception for $x < -1$.
- **`log1p(+∞)`** returns $+\infty$.

The **`log1p`** functions are equivalent to the **`logp1`** functions.

F.10.4.15 The `log2` functions

- **`log2(±0)`** returns $-\infty$ and raises the “divide-by-zero” floating-point exception.
- **`log2(1)`** returns $+0$.
- **`log2(x)`** returns a NaN and raises the “invalid” floating-point exception for $x < 0$.
- **`log2(+∞)`** returns $+\infty$.

F.10.4.16 The `log2p1` functions

- `log2p1`(± 0) returns ± 0 .
- `log2p1`(-1) returns $-\infty$ and raises the “divide-by-zero” floating-point exception.
- `log2p1`(x) returns a NaN and raises the “invalid” floating-point exception for $x < -1$.
- `log2p1`($+\infty$) returns $+\infty$.

F.10.4.17 The `logb` functions

- `logb`(± 0) returns $-\infty$ and raises the “divide-by-zero” floating-point exception.
- `logb`($\pm \infty$) returns $+\infty$.

The returned value is exact and is independent of the current rounding direction mode.

F.10.4.18 The `modf` functions

- `modf`($\pm x$, *iptr*) returns a result with the same sign as x .
- `modf`($\pm \infty$, *iptr*) returns ± 0 and stores $\pm \infty$ in the object pointed to by *iptr*.
- `modf`(NaN, *iptr*) stores a NaN in the object pointed to by *iptr* (and returns a NaN).

The returned values are exact and are independent of the current rounding direction mode.

`modf` behaves as though implemented by

```
#include <math.h>
#include <fenv.h>
#pragma STDC FENV_ACCESS ON
double modf(double value, double *iptr)
{
    int save_round = fegetround();
    fesetround(FE_TOWARDZERO);
    *iptr = nearbyint(value);
    fesetround(save_round);
    return copysign(
        isinf(value) ? 0.0:
        value - (*iptr), value);
}
```

F.10.4.19 The `scalbn` and `scalbln` functions

- `scalbn`(± 0 , n) returns ± 0 .
- `scalbn`(x , 0) returns x .
- `scalbn`($\pm \infty$, n) returns $\pm \infty$.

If the calculation does not overflow or underflow, the returned value is exact and independent of the current rounding direction mode.

F.10.5 Power and absolute value functions**F.10.5.1 The `cbrt` functions**

- `cbrt`(± 0) returns ± 0 .
- `cbrt`($\pm \infty$) returns $\pm \infty$.

F.10.5.2 The compoundn functions

- **compoundn**($x, 0$) returns 1 for $x \geq -1$ or x a NaN.
- **compoundn**(x, n) returns a NaN and raises the “invalid” floating-point exception for $x < -1$.
- **compoundn**($-1, n$) returns $+\infty$ and raises the divide-by-zero floating-point exception for $n < 0$.
- **compoundn**($-1, n$) returns $+0$ for $n > 0$.
- **compoundn**($+\infty, n$) returns $+\infty$ for $n > 0$.
- **compoundn**($+\infty, n$) returns $+0$ for $n < 0$.

F.10.5.3 The fabs functions

- **fabs**(± 0) returns $+0$.
- **fabs**($\pm\infty$) returns $+\infty$.

fabs(x) raises no floating-point exceptions, even if x is a signaling NaN. The returned value is independent of the current rounding direction mode.

F.10.5.4 The hypot functions

- **hypot**(x, y), **hypot**(y, x), and **hypot**($x, -y$) are equivalent.
- **hypot**($x, \pm 0$) returns the absolute value of x , if x is not a NaN.
- **hypot**($\pm\infty, y$) returns $+\infty$, even if y is a NaN.
- **hypot**(x, NaN) returns a NaN, if x is not $\pm\infty$.

F.10.5.5 The pow functions

- **pow**($\pm 0, y$) returns $\pm\infty$ and raises the “divide-by-zero” floating-point exception for y an odd integer < 0 .
- **pow**($\pm 0, y$) returns $+\infty$ and raises the “divide-by-zero” floating-point exception for $y < 0$, finite, and not an odd integer.
- **pow**($\pm 0, -\infty$) returns $+\infty$.
- **pow**($\pm 0, y$) returns ± 0 for y an odd integer > 0 .
- **pow**($\pm 0, y$) returns $+0$ for $y > 0$ and not an odd integer.
- **pow**($-1, \pm\infty$) returns 1.
- **pow**($+1, y$) returns 1 for any y , even a NaN.
- **pow**($x, \pm 0$) returns 1 for any x , even a NaN.
- **pow**(x, y) returns a NaN and raises the “invalid” floating-point exception for finite $x < 0$ and finite non-integer y .
- **pow**($x, -\infty$) returns $+\infty$ for $|x| < 1$.
- **pow**($x, -\infty$) returns $+0$ for $|x| > 1$.
- **pow**($x, +\infty$) returns $+0$ for $|x| < 1$.
- **pow**($x, +\infty$) returns $+\infty$ for $|x| > 1$.
- **pow**($-\infty, y$) returns -0 for y an odd integer < 0 .
- **pow**($-\infty, y$) returns $+0$ for $y < 0$ and not an odd integer.
- **pow**($-\infty, y$) returns $-\infty$ for y an odd integer > 0 .
- **pow**($-\infty, y$) returns $+\infty$ for $y > 0$ and not an odd integer.
- **pow**($+\infty, y$) returns $+0$ for $y < 0$.
- **pow**($+\infty, y$) returns $+\infty$ for $y > 0$.

F.10.5.6 The pown functions

- **pown**($x, 0$) returns 1 for all x not a signaling NaN.
- **pown**($\pm 0, n$) returns $\pm\infty$ and raises the “divide-by-zero” floating-point exception for odd $n < 0$.
- **pown**($\pm 0, n$) returns $+\infty$ and raises the “divide-by-zero” floating-point exception for even $n < 0$.
- **pown**($\pm 0, n$) returns $+0$ for even $n > 0$.
- **pown**($\pm 0, n$) returns ± 0 for odd $n > 0$.
- **pown**($\pm\infty, n$) is equivalent to **pown**($\pm 0, -n$) for n not 0, except that the “divide-by-zero” floating-point exception is not raised.

F.10.5.7 The powr functions

- **powr**($x, \pm 0$) returns 1 for finite $x > 0$.
- **powr**($\pm 0, y$) returns $+\infty$ and raises the “divide-by-zero” floating-point exception for finite $y < 0$.
- **powr**($\pm 0, -\infty$) returns $+\infty$.
- **powr**($\pm 0, y$) returns $+0$ for $y > 0$.
- **powr**($+1, y$) returns 1 for finite y .
- **powr**($+1, \pm\infty$) returns a NaN and raises the “invalid” floating-point exception.
- **powr**(x, y) returns a NaN and raises the “invalid” floating-point exception for $x < 0$.
- **powr**($\pm 0, \pm 0$) returns a NaN and raises the “invalid” floating-point exception.
- **powr**($+\infty, \pm 0$) returns a NaN and raises the “invalid” floating-point exception.

F.10.5.8 The rootn functions

- **rootn**($\pm 0, n$) returns $\pm\infty$ and raises the “divide-by-zero” floating-point exception for odd $n < 0$.
- **rootn**($\pm 0, n$) returns $+\infty$ and raises the “divide-by-zero” floating-point exception for even $n < 0$.
- **rootn**($\pm 0, n$) returns $+0$ for even $n > 0$.
- **rootn**($\pm 0, n$) returns ± 0 for odd $n > 0$.
- **rootn**($+\infty, n$) returns $+\infty$ for $n > 0$.
- **rootn**($-\infty, n$) returns $-\infty$ for odd $n > 0$.
- **rootn**($-\infty, n$) returns a NaN and raises the “invalid” floating-point exception for even $n > 0$.
- **rootn**($+\infty, n$) returns $+0$ for $n < 0$.
- **rootn**($-\infty, n$) returns -0 for odd $n < 0$.
- **rootn**($-\infty, n$) returns a NaN and raises the “invalid” floating-point exception for even $n < 0$.
- **rootn**($x, 0$) returns a NaN and raises the “invalid” floating-point exception for all x (including NaN).
- **rootn**(x, n) returns a NaN and raises the “invalid” floating-point exception for $x < 0$ and n even.

F.10.5.9 The rsqrt functions

- **rsqrt**(± 0) returns $\pm\infty$ and raises the “divide-by-zero” floating-point exception.
- **rsqrt**(x) returns a NaN and raises the “invalid” floating-point exception for $x < 0$.
- **rsqrt**($+\infty$) returns $+0$.

F.10.5.10 The sqrt functions

- **sqrt**(± 0) returns ± 0 .
- **sqrt**($+\infty$) returns $+\infty$.
- **sqrt**(x) returns a NaN and raises the “invalid” floating-point exception for $x < 0$.

The returned value is dependent on the current rounding direction mode.

F.10.6 Error and gamma functions**F.10.6.1 The erf functions**

- **erf**(± 0) returns ± 0 .
- **erf**($\pm\infty$) returns ± 1 .

F.10.6.2 The erfc functions

- **erfc**($-\infty$) returns 2.
- **erfc**($+\infty$) returns $+0$.

F.10.6.3 The lgamma functions

- **lgamma**(1) returns $+0$.
- **lgamma**(2) returns $+0$.
- **lgamma**(x) returns $+\infty$ and raises the “divide-by-zero” floating-point exception for x a negative integer or zero.
- **lgamma**($-\infty$) returns $+\infty$.
- **lgamma**($+\infty$) returns $+\infty$.

F.10.6.4 The tgamma functions

- **tgamma**(± 0) returns $\pm\infty$ and raises the “divide-by-zero” floating-point exception.
- **tgamma**(x) returns a NaN and raises the “invalid” floating-point exception for x a negative integer.
- **tgamma**($-\infty$) returns a NaN and raises the “invalid” floating-point exception.
- **tgamma**($+\infty$) returns $+\infty$.

F.10.7 Nearest integer functions**F.10.7.1 The ceil functions**

- **ceil**(± 0) returns ± 0 .
- **ceil**($\pm\infty$) returns $\pm\infty$.

The returned value is exact and is independent of the current rounding direction mode.

The **double** version of **ceil** behaves as though implemented by

```
#include <math.h>
#include <fenv.h>
#pragma STDC FENV_ACCESS ON
double ceil(double x)
{
    double result;
    int save_round = fegetround();
    fesetround(FE_UPWARD);
    result = nearbyint(x);
    fesetround(save_round);
    return result;
}
```

F.10.7.2 The floor functions

- **floor**(± 0) returns ± 0 .
- **floor**($\pm \infty$) returns $\pm \infty$.

The returned value is exact and is independent of the current rounding direction mode.

See the sample implementation for **ceil** in F.10.7.1.

F.10.7.3 The nearbyint functions

The **nearbyint** functions use ISO/IEC 60559 rounding according to the current rounding direction. They do not raise the “inexact” floating-point exception if the result differs in value from the argument.

- **nearbyint**(± 0) returns ± 0 (for all rounding directions).
- **nearbyint**($\pm \infty$) returns $\pm \infty$ (for all rounding directions).

F.10.7.4 The rint functions

The **rint** functions differ from the **nearbyint** functions only in that they do raise the “inexact” floating-point exception if the result differs in value from the argument.

F.10.7.5 The lrint and llrint functions

The **lrint** and **llrint** functions provide floating-to-integer conversion as prescribed by ISO/IEC 60559. They round according to the current rounding direction. If the rounded value is outside the range of the return type, the numeric result is unspecified and the “invalid” floating-point exception is raised. When they raise no other floating-point exception and the result differs from the argument, they raise the “inexact” floating-point exception.

F.10.7.6 The round functions

- **round**(± 0) returns ± 0 .
- **round**($\pm \infty$) returns $\pm \infty$.

The returned value is independent of the current rounding direction mode.

The **double** version of **round** behaves as though implemented by:⁴⁴¹⁾

```
#include <math.h>
#include <fenv.h>
#pragma STDC FENV_ACCESS ON
double round(double x)
{
    double result;
```

⁴⁴¹⁾This code does not handle signaling NaNs as required of implementations that define **FE_SNANS_ALWAYS_SIGNAL**.

```

    fenv_t save_env;
    feholdexcept(&save_env);
    result = rint(x);
    if (fetestexcept(FE_INEXACT)) {
        fesetround(FE_TOWARDZERO);
        result = rint(copysign(0.5 + fabs(x), x));
        feclearexcept(FE_INEXACT);
    }
    feupdateenv(&save_env);
    return result;
}

```

F.10.7.7 The **lround** and **llround** functions

The **lround** and **llround** functions differ from the **lrint** and **llrint** functions with the default rounding direction just in that the **lround** and **llround** functions round halfway cases away from zero and are not required to raise the “inexact” floating-point exception for non-integer arguments that round to within the range of the return type.

F.10.7.8 The **roundeven** functions

- **roundeven**(± 0) returns ± 0 .
- **roundeven**($\pm \infty$) returns $\pm \infty$.

The returned value is exact and is independent of the current rounding direction mode.

See the sample implementation for **ceil** in F.10.7.1.

F.10.7.9 The **trunc** functions

The **trunc** functions use ISO/IEC 60559 rounding toward zero (regardless of the current rounding direction).

- **trunc**(± 0) returns ± 0 .
- **trunc**($\pm \infty$) returns $\pm \infty$.

The returned value is exact and is independent of the current rounding direction mode.

F.10.7.10 The **fromfp** and **ufromfp** functions

The **fromfp** and **ufromfp** functions raise the “invalid” floating-point exception and return a NaN if the argument **width** is zero or if the floating-point argument **x** is infinite or NaN or rounds to an integral value that is outside the range determined by the argument **width** (see 7.12.10.10).

These functions do not raise the “inexact” floating-point exception.

F.10.7.11 The **fromfpx** and **ufromfpx** functions

The **fromfpx** and **ufromfpx** functions raise the “invalid” floating-point exception and return a NaN if the argument **width** is zero or if the floating-point argument **x** is infinite or NaN or rounds to an integral value that is outside the range determined by the argument **width** (see 7.12.10.11).

These functions raise the “inexact” floating-point exception if a valid result differs in value from the floating-point argument **x**.

F.10.8 Remainder functions

F.10.8.1 The **fmod** functions

- **fmod**($\pm 0, y$) returns ± 0 for y not zero.
- **fmod**(x, y) returns a NaN and raises the “invalid” floating-point exception for x infinite or y zero (and neither is a NaN).

- **fmod**($x, \pm\infty$) returns x for finite x .

When subnormal results are supported, the returned value is exact and is independent of the current rounding direction mode.

The **double** version of **fmod** behaves as though implemented by

```
#include <math.h>
#include <fenv.h>
#pragma STDC FENV_ACCESS ON
double fmod(double x, double y)
{
    double result;
    result = remainder(fabs(x), (y = fabs(y)));
    if (signbit(result)) result += y;
    return copysign(result, x);
}
```

F.10.8.2 The remainder functions

- **remainder**($\pm 0, y$) returns ± 0 for y not zero.
- **remainder**(x, y) returns a NaN and raises the “invalid” floating-point exception for x infinite or y zero (and neither is a NaN).
- **remainder**($x, \pm\infty$) returns x for finite x .

When subnormal results are supported, the returned value is exact and is independent of the current rounding direction mode.

F.10.8.3 The remquo functions

The **remquo** functions follow the specifications for the **remainder** functions.

If a NaN is returned, the value stored in the object pointed to by **quo** is unspecified.

When subnormal results are supported, the returned value is exact and is independent of the current rounding direction mode.

F.10.9 Manipulation functions

F.10.9.1 The copysign functions

copysign(x, y) raises no floating-point exceptions, even if x or y is a signaling NaN. The returned value is independent of the current rounding direction mode.

F.10.9.2 The nan functions

All ISO/IEC 60559 implementations support quiet NaNs, in all floating formats.

The returned value is exact and is independent of the current rounding direction mode.

F.10.9.3 The nextafter functions

- **nextafter**(x, y) raises the “overflow” and “inexact” floating-point exceptions for x finite and the function value infinite.
- **nextafter**(x, y) raises the “underflow” and “inexact” floating-point exceptions for the function value subnormal or zero and $x \neq y$.

Even though underflow or overflow can occur, the returned value is independent of the current rounding direction mode.

F.10.9.4 The nexttoward functions

No additional requirements beyond those on **nextafter**.

Even though underflow or overflow can occur, the returned value is independent of the current rounding direction mode.

F.10.9.5 The **nextup** functions

- **nextup**($+\infty$) returns $+\infty$.
- **nextup**($-\infty$) returns the largest-magnitude negative finite number in the return type of the function.

nextup(**x**) raises no floating-point exceptions if **x** is not a signaling NaN. The returned value is independent of the current rounding direction mode.

F.10.9.6 The **nextdown** functions

- **nextdown**($-\infty$) returns $-\infty$.
- **nextdown**($+\infty$) returns the largest-magnitude positive finite number in the type of the function.

nextdown(**x**) raises no floating-point exceptions if **x** is not a signaling NaN. The returned value is independent of the current rounding direction mode.

F.10.9.7 The **canonicalize** functions

The **canonicalize** functions produce⁴⁴²⁾ the canonical version of the representation in the object pointed to by the argument **x**. If the input ***x** is a signaling NaN, the “invalid” floating-point exception is raised and a (canonical) quiet NaN (which should be the canonical version of that signaling NaN made quiet) is produced. For quiet NaN, infinity, and finite inputs, the functions raise no floating-point exceptions.

F.10.10 Maximum, minimum, and positive difference functions

F.10.10.1 The **fdim** functions

No additional requirements.

F.10.10.2 The **fmax** functions

If just one argument is a NaN, the **fmax** functions return the other argument (if both arguments are NaNs, the functions return a NaN).

The returned value is exact and is independent of the current rounding direction mode.

The body of the **fmax** function may be:⁴⁴³⁾

```
{
    double r = (isgreater(x, y) || isnan(y)) ? x : y;
    (void) canonicalize(&r, &r);
    return r;
}
```

F.10.10.3 The **fmin** functions

The **fmin** functions are analogous to the **fmax** functions (see F.10.10.2).

The returned value is exact and is independent of the current rounding direction mode.

F.10.10.4 The **fmaximum**, **fminimum**, **fmaximum_mag**, and **fminimum_mag** functions

These functions treat NaNs like other functions in `<math.h>` (see F.10). They differ from the corresponding **fmaximum_num**, **fminimum_num**, **fmaximum_mag_num**, and **fminimum_mag_num** functions only in their treatment of NaNs.

⁴⁴²⁾As if ***x * 1e0** were computed.
⁴⁴³⁾If possible, **fmax** is sensitive to the sign of zero, for example **fmax**($-0.0, +0.0$) ideally returns $+0$. Note also that this implementation does not handle signaling NaNs as required of implementations that define **FE_SNANS_ALWAYS_SIGNAL**.

F.10.10.5 The `fmaximum_num`, `fminimum_num`, `fmaximum_mag_num`, and `fminimum_mag_num` functions

These functions return the number if one argument is a number and the other is a quiet or signaling NaN. If both arguments are NaNs, a quiet NaN is returned. If an argument is a signaling NaN, the “invalid” floating-point exception is raised (even though the function returns the number when the other argument is a number).

F.10.11 Fused multiply-add

F.10.11.1 The `fma` functions

- `fma`(x, y, z) computes $xy + z$, correctly rounded once.
- `fma`(x, y, z) returns a NaN and optionally raises the “invalid” floating-point exception if one of x and y is infinite, the other is zero, and z is a NaN.
- `fma`(x, y, z) returns a NaN and raises the “invalid” floating-point exception if one of x and y is infinite, the other is zero, and z is not a NaN.
- `fma`(x, y, z) returns a NaN and raises the “invalid” floating-point exception if x times y is an exact infinity and z is also an infinity but with the opposite sign.

F.10.12 Functions that round result to narrower type

The functions that round their result to narrower type (7.12.15) are fully specified in ISO/IEC 60559. The returned value is dependent on the current rounding direction mode.

These functions treat zero and infinite arguments like the corresponding operation or function: `+`, `-`, `*`, `/`, `fma`, or `sqrt`.

F.10.13 Total order functions

F.10.13.1 General

This subclause specifies the total order functions required by ISO/IEC 60559.

NOTE These functions are specified only in this annex because the functions for standard floating types depend on details of ISO/IEC 60559 formats that are conditionally supported based on if the relevant feature test macro, `__STDC_IEC_60559_BFP__` or `__STDC_IEC_60559_DFP__`, is defined or not.

F.10.13.2 The `totalorder` functions

Synopsis

```
#define __STDC_WANT_IEC_60559_EXT__
#include <math.h>
#ifdef __STDC_IEC_60559_BFP__
int totalorder(const double *x, const double *y);
int totalorderf(const float *x, const float *y);
int totalorderl(const long double *x, const long double *y);
#endif
#ifdef __STDC_IEC_60559_DFP__
int totalorderd32(const _Decimal32 *x, const _Decimal32 *y);
int totalorderd64(const _Decimal64 *x, const _Decimal64 *y);
int totalorderd128(const _Decimal128 *x, const _Decimal128 *y);
#endif
```

Description

The `totalorder` functions determine whether the total order relationship, defined by ISO/IEC 60559, is true for the ordered pair of `*x`, `*y`. These functions are fully specified in ISO/IEC 60559. These functions are independent of the current rounding direction mode and raise no floating-point exceptions, even if `*x` or `*y` is a signaling NaN.

Returns

The **totalorder** functions return nonzero if and only if the total order relation is true for the ordered pair of ***x**, ***y**.

F.10.13.3 The **totalordermag** functions

Synopsis

```
#define __STDC_WANT_IEC_60559_EXT__
#include <math.h>
#ifdef __STDC_IEC_60559_BFP__
int totalordermag(const double *x, const double *y);
int totalordermagf(const float *x, const float *y);
int totalordermagl(const long double *x, const long double *y);
#endif
#ifdef __STDC_IEC_60559_DFP__
int totalordermagd32(const _Decimal32 *x, const _Decimal32 *y);
int totalordermagd64(const _Decimal64 *x, const _Decimal64 *y);
int totalordermagd128(const _Decimal128 *x, const _Decimal128 *y);
#endif
```

Description

The **totalordermag** functions determine whether the total order relationship, defined by ISO/IEC 60559, is true for the ordered pair of the magnitudes of ***x**, ***y**. These functions are fully specified in ISO/IEC 60559. These functions are independent of the current rounding direction mode and raise no floating-point exceptions, even if ***x** or ***y** is a signaling NaN.

Returns

The **totalordermag** functions return nonzero if and only if the total order relation is true for the ordered pair of the magnitudes of ***x**, ***y**.

F.10.14 Payload functions

F.10.14.1 General

ISO/IEC 60559 defines the payload to be information contained in a quiet or signaling NaN. The payload is intended for implementation-defined diagnostic information about the NaN, such as where or how the NaN was created.⁴⁴⁴⁾ The implementation interprets the payload as a nonnegative integer suitable for use with the functions in this subclause, which get and set payloads. The implementation may restrict which payloads are admissible for the user to set.

NOTE These functions are specified only in this annex because the functions for standard floating types depend on details of ISO/IEC 60559 formats that are conditionally supported based on if the relevant feature test macro, **__STDC_IEC_60559_BFP__** or **__STDC_IEC_60559_DFP__**, is defined or not.

F.10.14.2 The **getpayload** functions

Synopsis

```
#define __STDC_WANT_IEC_60559_EXT__
#include <math.h>
#ifdef __STDC_IEC_60559_BFP__
double getpayload(const double *x);
float getpayloadf(const float *x);
long double getpayloadl(const long double *x);
#endif
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 getpayloadd32(const _Decimal32 *x);
_Decimal64 getpayloadd64(const _Decimal64 *x);
_Decimal128 getpayloadd128(const _Decimal128 *x);
#endif
```

⁴⁴⁴⁾For the purpose of determining value inclusion (as in 6.2.5, 7.12, and H.11), quiet NaN representations can be regarded as having the same value, regardless of payloads.

Description

The **getpayload** functions extract the payload of a quiet or signaling NaN input and return it as a positive-signed floating-point integer. If ***x** is not a NaN, the return result is -1 . These functions raise no floating-point exceptions, even if ***x** is a signaling NaN.

Returns

The **getpayload** functions return the payload of the NaN input as a positive-signed floating-point integer.

F.10.14.3 The **setpayload** functions

Synopsis

```
#define __STDC_WANT_IEC_60559_EXT__
#include <math.h>
#ifdef __STDC_IEC_60559_BFP__
int setpayload(double *res, double pl);
int setpayloadf(float *res, float pl);
int setpayloadl(long double *res, long double pl);
#endif
#ifdef __STDC_IEC_60559_DFP__
int setpayloadd32(_Decimal32 *res, _Decimal32 pl);
int setpayloadd64(_Decimal64 *res, _Decimal64 pl);
int setpayloadd128(_Decimal128 *res, _Decimal128 pl);
#endif
```

Description

The **setpayload** functions create a quiet NaN with the payload specified by **pl** and a zero sign bit and store that NaN in the object pointed to by ***res**. If **pl** is not a floating-point integer representing an admissible payload, ***res** is set to $+0$.

Returns

If the **setpayload** functions stored the specified NaN, they return a zero value, otherwise a nonzero value (and ***res** is set to $+0$).

F.10.14.4 The **setpayloadsig** functions

Synopsis

```
#define __STDC_WANT_IEC_60559_EXT__
#include <math.h>
#ifdef __STDC_IEC_60559_BFP__
int setpayloadsig(double *res, double pl);
int setpayloadsigf(float *res, float pl);
int setpayloadsigl(long double *res, long double pl);
#endif
#ifdef __STDC_IEC_60559_DFP__
int setpayloadsigd32(_Decimal32 *res, _Decimal32 pl);
int setpayloadsigd64(_Decimal64 *res, _Decimal64 pl);
int setpayloadsigd128(_Decimal128 *res, _Decimal128 pl);
#endif
```

Description

The **setpayloadsig** functions create a signaling NaN with the payload specified by **pl** and a zero sign bit and store that NaN in the object pointed to by ***res**. If **pl** is not a floating-point integer representing an admissible payload, ***res** is set to $+0$.

Returns

If the **setpayloadsig** functions stored the specified NaN, they return a zero value, otherwise a nonzero value (and ***res** is set to $+0$).

F.10.15 Comparison macros

F.10.15.1 General

Relational operators and their corresponding comparison macros (7.12.18) produce equivalent result values, even if argument values are represented in wider formats. Thus, comparison macro arguments represented in formats wider than their semantic types are not converted to the semantic types, unless the wide evaluation method converts operands of relational operators to their semantic types. The standard wide evaluation methods characterized by **FLT_EVAL_METHOD** and **DEC_EVAL_METHOD** equal to 1 or 2 (5.3.5.3.3, 5.3.5.3.4), do not convert operands of relational operators to their semantic types.

F.10.15.2 The **iseqsig** macro

The equality operator **==** and the **iseqsig** macro produce equivalent results, except that the **iseqsig** macro raises the “invalid” floating-point exception if an argument is a NaN.

Annex G

(normative)

ISO/IEC 60559-compatible complex arithmetic

G.1 Introduction

This annex supplements Annex F to specify complex arithmetic for compatibility with ISO/IEC 60559 real floating-point arithmetic. An implementation that defines `__STDC_IEC_60559_COMPLEX__` or `__STDC_IEC_559_COMPLEX__` shall conform to the specifications in this annex.⁴⁴⁵⁾

G.2 Types

There is a new keyword `_Imaginary`, which is used to specify imaginary types. It is used as a type specifier within declaration specifiers in the same way as `_Complex` is (thus, `float _Imaginary` is a valid type name).

There are three *imaginary type*, designated as `float _Imaginary`, `double _Imaginary`, and `long double _Imaginary`. The imaginary types (along with the real floating and complex types) are floating types.

For imaginary types, the corresponding real type is given by deleting the keyword `_Imaginary` from the type name.

Each imaginary type has the same representation and alignment requirements as the corresponding real type. The value of an object of imaginary type is the value of the real representation times the imaginary unit.

The *imaginary type domain* comprises the imaginary types.

G.3 Conventions

A complex or imaginary value with at least one infinite part is regarded as an *infinity* (even if its other part is a quiet NaN). A complex or imaginary value is a *finite number* if each of its parts is a finite number (neither infinite nor NaN). A complex or imaginary value is a *zero* if each of its parts is a zero.

G.4 Conversions

G.4.1 Imaginary types

Conversions among imaginary types follow rules analogous to those for real floating types.

G.4.2 Real and imaginary

When a value of imaginary type is converted to a real type other than `bool`,⁴⁴⁶⁾ the result is a positive zero.

When a value of real type is converted to an imaginary type, the result is a positive imaginary zero.

G.4.3 Imaginary and complex

When a value of imaginary type is converted to a complex type, the real part of the complex result value is a positive zero and the imaginary part of the complex result value is determined by the conversion rules for the corresponding real types.

When a value of complex type is converted to an imaginary type, the real part of the complex value is discarded and the value of the imaginary part is converted according to the conversion rules for the corresponding real types.

⁴⁴⁵⁾Implementations that do not define `__STDC_IEC_60559_COMPLEX__` or `__STDC_IEC_559_COMPLEX__` are not required to conform to these specifications. The use of `__STDC_IEC_559_COMPLEX__` for this purpose is obsolescent and should be avoided in new code.

⁴⁴⁶⁾See 6.3.2.2.

G.5 Binary operators

G.5.1 General

The following subclauses supplement 6.5.1 to specify the type of the result for an operation with an imaginary operand.

For most operand types, the value of the result of a binary operator with an imaginary or complex operand is completely determined, with reference to real arithmetic, by the usual mathematical formula. For some operand types, the usual mathematical formula is problematic because of its treatment of infinities and because of undue overflow or underflow; in these cases the result satisfies certain properties (specified in G.5.2), but is not completely determined.

G.5.2 Multiplicative operators

Semantics

If one operand has real type and the other operand has imaginary type, then the result has imaginary type. If both operands have imaginary type, then the result has real type. (If either operand has complex type, then the result has complex type.)

If the operands are not both complex, then the result and floating-point exception behavior of the `*` operator is defined by the usual mathematical formula shown in Table G.1:

Table G.1 — Results of multiplication operations

<code>*</code>	u	iv	$u + iv$
x	xu	$i(xv)$	$(xu) + i(xv)$
iy	$i(yu)$	$(-y)v$	$((-y)v) + i(yu)$
$x + iy$	$(xu) + i(yu)$	$((-y)v) + i(xv)$	

If the second operand is not complex, then the result and floating-point exception behavior of the `/` operator is defined by the usual mathematical formula as in Table G.2:

Table G.2 — Results of division operations

<code>/</code>	u	iv
x	x/u	$i((-x)/v)$
iy	$i(y/u)$	y/v
$x + iy$	$(x/u) + i(y/u)$	$(y/v) + i((-x)/v)$

The `*` and `/` operators satisfy the following infinity properties for all real, imaginary, and complex operands:⁴⁴⁷⁾

- if one operand is an infinity and the other operand is a nonzero finite number or an infinity, then the result of the `*` operator is an infinity;
- if the first operand is an infinity and the second operand is a finite number, then the result of the `/` operator is an infinity;
- if the first operand is a finite number and the second operand is an infinity, then the result of the `/` operator is a zero;
- if the first operand is a nonzero finite number or an infinity and the second operand is a zero, then the result of the `/` operator is an infinity.

If both operands of the `*` operator are complex or if the second operand of the `/` operator is complex, the operator raises floating-point exceptions if appropriate for the calculation of the parts of the result, and may raise spurious floating-point exceptions.

⁴⁴⁷⁾These properties are already implied for those cases covered in Table G.1 and Table G.2, but are required for all cases (at least where the state for `CX_LIMITED_RANGE` is “off”).

EXAMPLE 1 Multiplication of **double _Complex** operands can be implemented as follows. The imaginary unit **I** has imaginary type (see G.6).

```
#include <math.h>
#include <complex.h>

/* Multiply z * w ... */
double complex _Cmultd(double complex z, double complex w)
{
    #pragma STDC FP_CONTRACT OFF
    double a, b, c, d, ac, bd, ad, bc, x, y;
    a = creal(z); b = cimag(z);
    c = creal(w); d = cimag(w);
    ac = a * c;   bd = b * d;
    ad = a * d;   bc = b * c;
    x = ac - bd;  y = ad + bc;
    if (isnan(x) && isnan(y)) {
        /* Recover infinities that computed as NaN+iNaN ... */
        int recalc = 0;
        if (isinf(a) || isinf(b)) { // z is infinite
            /* "Box" the infinity and change NaNs in the other factor to 0 */
            a = copysign(isinf(a) ? 1.0: 0.0, a);
            b = copysign(isinf(b) ? 1.0: 0.0, b);
            if (isnan(c)) c = copysign(0.0, c);
            if (isnan(d)) d = copysign(0.0, d);
            recalc = 1;
        }
        if (isinf(c) || isinf(d)) { // w is infinite
            /* "Box" the infinity and change NaNs in the other factor to 0 */
            c = copysign(isinf(c) ? 1.0: 0.0, c);
            d = copysign(isinf(d) ? 1.0: 0.0, d);
            if (isnan(a)) a = copysign(0.0, a);
            if (isnan(b)) b = copysign(0.0, b);
            recalc = 1;
        }
        if (!recalc && (isinf(ac) || isinf(bd) ||
                       isinf(ad) || isinf(bc))) {
            /* Recover infinities from overflow by changing NaNs to 0 ... */
            if (isnan(a)) a = copysign(0.0, a);
            if (isnan(b)) b = copysign(0.0, b);
            if (isnan(c)) c = copysign(0.0, c);
            if (isnan(d)) d = copysign(0.0, d);
            recalc = 1;
        }
        if (recalc) {
            x = INFINITY * (a * c - b * d);
            y = INFINITY * (a * d + b * c);
        }
    }
    return x + I * y;
}
```

This implementation achieves the required treatment of infinities at the cost of only one **isnan** test in ordinary (finite) cases. It is less than ideal in that undue overflow and underflow can occur.

EXAMPLE 2 Division of two **double _Complex** operands can be implemented as follows.

```
#include <math.h>
#include <complex.h>

/* Divide z / w ... */
double complex _Cdivd(double complex z, double complex w)
```



```
{
    #pragma STDC FP_CONTRACT OFF
    double a, b, c, d, logbw, denom, x, y;
    int ilogbw = 0;
    a = creal(z); b = cimag(z);
    c = creal(w); d = cimag(w);
    logbw = logb(fmaximum_num(fabs(c), fabs(d)));
    if (isfinite(logbw)) {
        ilogbw = (int)logbw;
        c = scalbn(c, -ilogbw); d = scalbn(d, -ilogbw);
    }
    denom = c * c + d * d;
    x = scalbn((a * c + b * d) / denom, -ilogbw);
    y = scalbn((b * c - a * d) / denom, -ilogbw);

    /* Recover infinities and zeros that computed as NaN+iNaN; */
    /* the only cases are nonzero/zero, infinite/finite, and finite/infinite, ... */

    if (isnan(x) && isnan(y)) {
        if ((denom == 0.0) &&
            (!isnan(a) || !isnan(b))) {
            x = copysign(INFINITY, c) * a;
            y = copysign(INFINITY, c) * b;
        }
        else if ((isinf(a) || isinf(b)) &&
            isfinite(c) && isfinite(d)) {
            a = copysign(isinf(a) ? 1.0: 0.0, a);
            b = copysign(isinf(b) ? 1.0: 0.0, b);
            x = INFINITY * (a * c + b * d);
            y = INFINITY * (b * c - a * d);
        }
        else if ((logbw == INFINITY) &&
            isfinite(a) && isfinite(b)) {
            c = copysign(isinf(c) ? 1.0: 0.0, c);
            d = copysign(isinf(d) ? 1.0: 0.0, d);
            x = 0.0 * (a * c + b * d);
            y = 0.0 * (b * c - a * d);
        }
    }
    return x + I * y;
}
```

Scaling the denominator alleviates the main overflow and underflow problem, which is more serious than for multiplication. In the spirit of the preceding multiplication example, this code does not defend against overflow and underflow in the calculation of the numerator. Scaling with the `scalbn` function, instead of with division, provides better roundoff characteristics.

G.5.3 Additive operators

Semantics

If both operands have imaginary type, then the result has imaginary type. (If one operand has real type and the other operand has imaginary type, or if either operand has complex type, then the result has complex type.)

In all cases the result and floating-point exception behavior of a `+` or `-` operator is defined by the usual mathematical formula in Table G.3:

Table G.3 — Results of addition or subtraction operations

+ or -	<i>u</i>	<i>iv</i>	<i>u + iv</i>
<i>x</i>	<i>x ± u</i>	<i>x ± iv</i>	<i>(x ± u) ± iv</i>

iy	$\pm u + iy$	$i(y \pm v)$	$\pm u + i(y \pm v)$
$x + iy$	$(x \pm u) + iy$	$x + i(y \pm v)$	$(x \pm u) + i(y \pm v)$

G.6 Complex arithmetic <complex.h>

G.6.1 General

The macros

imaginary

and

_Imaginary_I

are defined, respectively, as **_Imaginary** and a constant expression of type **float** **_Imaginary** with the value of the imaginary unit. The macro

I

is defined to be **_Imaginary_I** (not **_Complex_I** as stated in 7.3). Notwithstanding the provisions of 7.1.3, a program can undefine and then perhaps redefine the macro **imaginary**.

This subclause contains specifications for the <complex.h> functions that are particularly suited to ISO/IEC 60559 implementations. For families of functions, the specifications apply to all of the functions even though only the principal function is shown. Unless otherwise specified, where the symbol “ $\pm$ ” occurs in both an argument and the result, the result has the same sign as the argument.

The functions are continuous onto both sides of their branch cuts, taking into account the sign of zero. For example, $\text{csqrt}(-2 \pm i0) = \pm i\sqrt{2}$.

Since complex and imaginary values are composed of real values, each function can be regarded as computing real values from real values. Except as noted, the functions treat real infinities, NaNs, signed zeros, subnormals, and the floating-point exception flags in a manner consistent with the specifications for real functions in F.10.⁴⁴⁸⁾

In subsequent subclauses in G.6 “NaN” refers to a quiet NaN. The behavior of signaling NaNs in this annex is implementation-defined.

The functions **cimag**, **conj**, **cproj**, and **creal** are fully specified for all implementations, including ISO/IEC 60559 ones, in 7.3.9. These functions raise no floating-point exceptions.

Each of the functions **cabs** and **carg** is specified by a formula in terms of a real function (whose special cases are covered in Annex F):

$$\begin{aligned}\text{cabs}(x + iy) &= \text{hypot}(x, y) \\ \text{carg}(x + iy) &= \text{atan2}(y, x)\end{aligned}$$

Each of the functions **casin**, **catan**, **ccos**, **csin**, and **ctan** is specified implicitly by a formula in terms of other complex functions (whose special cases are specified below):

$$\begin{aligned}\text{casin}(z) &= -i \text{casinh}(iz) \\ \text{catan}(z) &= -i \text{catanh}(iz) \\ \text{ccos}(z) &= \text{ccosh}(iz) \\ \text{csin}(z) &= -i \text{csinh}(iz) \\ \text{ctan}(z) &= -i \text{ctanh}(iz)\end{aligned}$$

For the other functions, the following subclauses specify behavior for special cases, including treatment of the “invalid” and “divide-by-zero” floating-point exceptions. For families of functions,

⁴⁴⁸⁾As noted in G.3, a complex value with at least one infinite part is regarded as an infinity even if its other part is a quiet NaN.

the specifications apply to all of the functions even though only the principal function is shown. For a function f satisfying $f(\text{conj}(z)) = \text{conj}(f(z))$, the specifications for the upper half-plane imply the specifications for the lower half-plane; if the function f is also either even, $f(-z) = f(z)$, or odd, $f(-z) = -f(z)$, then the specifications for the first quadrant imply the specifications for the other three quadrants.

In the following subclauses, $\text{cis}(y)$ is defined as $\cos(y) + i \sin(y)$.

G.6.2 Trigonometric functions

G.6.2.1 The **cacos** functions

- **cacos**(**conj**(z)) = **conj**(**cacos**(z)).
- **cacos**($\pm 0 + i0$) returns $\frac{\pi}{2} - i0$.
- **cacos**($\pm 0 + i\text{NaN}$) returns $\frac{\pi}{2} + i\text{NaN}$.
- **cacos**($x + i\infty$) returns $\frac{\pi}{2} - i\infty$, for finite x .
- **cacos**($x + i\text{NaN}$) returns $\text{NaN} + i\text{NaN}$ and optionally raises the “invalid” floating-point exception, for nonzero finite x .
- **cacos**($-\infty + iy$) returns $\pi - i\infty$, for positive-signed finite y .
- **cacos**($+\infty + iy$) returns $+0 - i\infty$, for positive-signed finite y .
- **cacos**($-\infty + i\infty$) returns $3\frac{\pi}{4} - i\infty$.
- **cacos**($+\infty + i\infty$) returns $\frac{\pi}{4} - i\infty$.
- **cacos**($\pm\infty + i\text{NaN}$) returns $\text{NaN} \pm i\infty$ (where the sign of the imaginary part of the result is unspecified).
- **cacos**($\text{NaN} + iy$) returns $\text{NaN} + i\text{NaN}$ and optionally raises the “invalid” floating-point exception, for finite y .
- **cacos**($\text{NaN} + i\infty$) returns $\text{NaN} - i\infty$.
- **cacos**($\text{NaN} + i\text{NaN}$) returns $\text{NaN} + i\text{NaN}$.

G.6.3 Hyperbolic functions

G.6.3.1 The **cacosh** functions

- **cacosh**(**conj**(z)) = **conj**(**cacosh**(z)).
- **cacosh**($\pm 0 + i0$) returns $+0 + \frac{i\pi}{2}$.
- **cacosh**($x + i\infty$) returns $+\infty + \frac{i\pi}{2}$, for finite x .
- **cacosh**($0 + i\text{NaN}$) returns $\text{NaN} \pm \frac{i\pi}{2}$ (where the sign of the imaginary part of the result is unspecified).
- **cacosh**($x + i\text{NaN}$) returns $\text{NaN} + i\text{NaN}$ and optionally raises the “invalid” floating-point exception, for finite nonzero x .
- **cacosh**($-\infty + iy$) returns $+\infty + i\pi$, for positive-signed finite y .
- **cacosh**($+\infty + iy$) returns $+\infty + i0$, for positive-signed finite y .
- **cacosh**($-\infty + i\infty$) returns $+\infty + i\frac{3\pi}{4}$.
- **cacosh**($+\infty + i\infty$) returns $+\infty + \frac{i\pi}{4}$.
- **cacosh**($\pm\infty + i\text{NaN}$) returns $+\infty + i\text{NaN}$.

- **cacosh**(NaN + iy) returns NaN + i NaN and optionally raises the “invalid” floating-point exception, for finite y .
- **cacosh**(NaN + $i\infty$) returns $+\infty + i$ NaN.
- **cacosh**(NaN + i NaN) returns NaN + i NaN.

G.6.3.2 The **casinh** functions

- **casinh**(**conj**(z)) = **conj**(**casinh**(z)) and **casinh** is odd.
- **casinh**($+0 + i0$) returns $0 + i0$.
- **casinh**($x + i\infty$) returns $+\infty + \frac{i\pi}{2}$ for positive-signed finite x .
- **casinh**($x + i$ NaN) returns NaN + i NaN and optionally raises the “invalid” floating-point exception, for finite x .
- **casinh**($+\infty + iy$) returns $+\infty + i0$ for positive-signed finite y .
- **casinh**($+\infty + i\infty$) returns $+\infty + \frac{i\pi}{4}$.
- **casinh**($+\infty + i$ NaN) returns $+\infty + i$ NaN.
- **casinh**(NaN + $i0$) returns NaN + $i0$.
- **casinh**(NaN + iy) returns NaN + i NaN and optionally raises the “invalid” floating-point exception, for finite nonzero y .
- **casinh**(NaN + $i\infty$) returns $\pm\infty + i$ NaN (where the sign of the real part of the result is unspecified).
- **casinh**(NaN + i NaN) returns NaN + i NaN.

G.6.3.3 The **catanh** functions

- **catanh**(**conj**(z)) = **conj**(**catanh**(z)) and **catanh** is odd.
- **catanh**($+0 + i0$) returns $+0 + i0$.
- **catanh**($+0 + i$ NaN) returns $+0 + i$ NaN.
- **catanh**($+1 + i0$) returns $+\infty + i0$ and raises the “divide-by-zero” floating-point exception.
- **catanh**($x + i\infty$) returns $+0 + \frac{i\pi}{2}$, for finite positive-signed x .
- **catanh**($x + i$ NaN) returns NaN + i NaN and optionally raises the “invalid” floating-point exception, for nonzero finite x .
- **catanh**($+\infty + iy$) returns $+0 + \frac{i\pi}{2}$, for finite positive-signed y .
- **catanh**($+\infty + i\infty$) returns $+0 + \frac{i\pi}{2}$.
- **catanh**($+\infty + i$ NaN) returns $+0 + i$ NaN.
- **catanh**(NaN + iy) returns NaN + i NaN and optionally raises the “invalid” floating-point exception, for finite y .
- **catanh**(NaN + $i\infty$) returns $\pm 0 + \frac{i\pi}{2}$ (where the sign of the real part of the result is unspecified).
- **catanh**(NaN + i NaN) returns NaN + i NaN.

G.6.3.4 The ccosh functions

- **ccosh**(**conj**(z)) = **conj**(**ccosh**(z)) and **ccosh** is even.
- **ccosh**($+0 + i0$) returns $1 + i0$.
- **ccosh**($+0 + i\infty$) returns NaN $\pm i0$ (where the sign of the imaginary part of the result is unspecified) and raises the “invalid” floating-point exception.
- **ccosh**($+0 + i\text{NaN}$) returns NaN $\pm i0$ (where the sign of the imaginary part of the result is unspecified).
- **ccosh**($x + i\infty$) returns NaN + $i\text{NaN}$ and raises the “invalid” floating-point exception, for finite nonzero x .
- **ccosh**($x + i\text{NaN}$) returns NaN + $i\text{NaN}$ and optionally raises the “invalid” floating-point exception, for finite nonzero x .
- **ccosh**($+\infty + i0$) returns $+\infty + i0$.
- **ccosh**($+\infty + iy$) returns $+\infty \text{ cis}(y)$, for finite nonzero y .
- **ccosh**($+\infty + i\infty$) returns $\pm\infty + i\text{NaN}$ (where the sign of the real part of the result is unspecified) and raises the “invalid” floating-point exception.
- **ccosh**($+\infty + i\text{NaN}$) returns $+\infty + i\text{NaN}$.
- **ccosh**(NaN + $i0$) returns NaN $\pm i0$ (where the sign of the imaginary part of the result is unspecified).
- **ccosh**(NaN + iy) returns NaN + $i\text{NaN}$ and optionally raises the “invalid” floating-point exception, for all nonzero numbers y .
- **ccosh**(NaN + $i\text{NaN}$) returns NaN + $i\text{NaN}$.

G.6.3.5 The csinh functions

- **csinh**(**conj**(z)) = **conj**(**csinh**(z)) and **csinh** is odd.
- **csinh**($+0 + i0$) returns $+0 + i0$.
- **csinh**($+0 + i\infty$) returns $\pm 0 + i\text{NaN}$ (where the sign of the real part of the result is unspecified) and raises the “invalid” floating-point exception.
- **csinh**($+0 + i\text{NaN}$) returns $\pm 0 + i\text{NaN}$ (where the sign of the real part of the result is unspecified).
- **csinh**($x + i\infty$) returns NaN + $i\text{NaN}$ and raises the “invalid” floating-point exception, for positive finite x .
- **csinh**($x + i\text{NaN}$) returns NaN + $i\text{NaN}$ and optionally raises the “invalid” floating-point exception, for finite nonzero x .
- **csinh**($+\infty + i0$) returns $+\infty + i0$.
- **csinh**($+\infty + iy$) returns $+\infty \text{ cis}(y)$, for positive finite y .
- **csinh**($+\infty + i\infty$) returns $\pm\infty + i\text{NaN}$ (where the sign of the real part of the result is unspecified) and raises the “invalid” floating-point exception.
- **csinh**($+\infty + i\text{NaN}$) returns $\pm\infty + i\text{NaN}$ (where the sign of the real part of the result is unspecified).
- **csinh**(NaN + $i0$) returns NaN + $i0$.
- **csinh**(NaN + iy) returns NaN + $i\text{NaN}$ and optionally raises the “invalid” floating-point exception, for all nonzero numbers y .
- **csinh**(NaN + $i\text{NaN}$) returns NaN + $i\text{NaN}$.

G.6.3.6 The ctanh functions

- **ctanh**(**conj**(z)) = **conj**(**ctanh**(z)) and **ctanh** is odd.
- **ctanh**($+0 + i0$) returns $+0 + i0$.
- **ctanh**($0 + i\infty$) returns $0 + i\text{NaN}$ and raises the “invalid” floating-point exception.
- **ctanh**($x + i\infty$) returns $\text{NaN} + i\text{NaN}$ and raises the “invalid” floating-point exception, for finite nonzero x .
- **ctanh**($0 + i\text{NaN}$) returns $0 + i\text{NaN}$.
- **ctanh**($x + i\text{NaN}$) returns $\text{NaN} + i\text{NaN}$ and optionally raises the “invalid” floating-point exception, for finite nonzero x .
- **ctanh**($+\infty + iy$) returns $1 + i0 \sin(2y)$, for positive-signed finite y .
- **ctanh**($+\infty + i\infty$) returns $1 \pm i0$ (where the sign of the imaginary part of the result is unspecified).
- **ctanh**($+\infty + i\text{NaN}$) returns $1 \pm i0$ (where the sign of the imaginary part of the result is unspecified).
- **ctanh**($\text{NaN} + i0$) returns $\text{NaN} + i0$.
- **ctanh**($\text{NaN} + iy$) returns $\text{NaN} + i\text{NaN}$ and optionally raises the “invalid” floating-point exception, for all nonzero numbers y .
- **ctanh**($\text{NaN} + i\text{NaN}$) returns $\text{NaN} + i\text{NaN}$.

G.6.4 Exponential and logarithmic functions**G.6.4.1 The cexp functions**

- **cexp**(**conj**(z)) = **conj**(**cexp**(z)).
- **cexp**($\pm 0 + i0$) returns $1 + i0$.
- **cexp**($x + i\infty$) returns $\text{NaN} + i\text{NaN}$ and raises the “invalid” floating-point exception, for finite x .
- **cexp**($x + i\text{NaN}$) returns $\text{NaN} + i\text{NaN}$ and optionally raises the “invalid” floating-point exception, for finite x .
- **cexp**($+\infty + i0$) returns $+\infty + i0$.
- **cexp**($-\infty + iy$) returns $+0 \text{ cis}(y)$, for finite y .
- **cexp**($+\infty + iy$) returns $+\infty \text{ cis}(y)$, for finite nonzero y .
- **cexp**($-\infty + i\infty$) returns $\pm 0 \pm i0$ (where the signs of the real and imaginary parts of the result are unspecified).
- **cexp**($+\infty + i\infty$) returns $\pm \infty + i\text{NaN}$ and raises the “invalid” floating-point exception (where the sign of the real part of the result is unspecified).
- **cexp**($-\infty + i\text{NaN}$) returns $\pm 0 \pm i0$ (where the signs of the real and imaginary parts of the result are unspecified).
- **cexp**($+\infty + i\text{NaN}$) returns $\pm \infty + i\text{NaN}$ (where the sign of the real part of the result is unspecified).
- **cexp**($\text{NaN} + i0$) returns $\text{NaN} + i0$.
- **cexp**($\text{NaN} + iy$) returns $\text{NaN} + i\text{NaN}$ and optionally raises the “invalid” floating-point exception, for all nonzero numbers y .
- **cexp**($\text{NaN} + i\text{NaN}$) returns $\text{NaN} + i\text{NaN}$.

G.6.4.2 The `clog` functions

- `clog(conj(z)) = conj(clog(z))`.
- `clog(−0 + i0)` returns $−∞ + iπ$ and raises the “divide-by-zero” floating-point exception.
- `clog(+0 + i0)` returns $−∞ + i0$ and raises the “divide-by-zero” floating-point exception.
- `clog(x + i∞)` returns $+∞ + \frac{iπ}{2}$, for finite x .
- `clog(x + iNaN)` returns $NaN + iNaN$ and optionally raises the “invalid” floating-point exception, for finite x .
- `clog(−∞ + iy)` returns $+∞ + iπ$, for finite positive-signed y .
- `clog(+∞ + iy)` returns $+∞ + i0$, for finite positive-signed y .
- `clog(−∞ + i∞)` returns $+∞ + i\frac{3π}{4}$.
- `clog(+∞ + i∞)` returns $+∞ + i\frac{π}{4}$.
- `clog(±∞ + iNaN)` returns $+∞ + iNaN$.
- `clog(NaN + iy)` returns $NaN + iNaN$ and optionally raises the “invalid” floating-point exception, for finite y .
- `clog(NaN + i∞)` returns $+∞ + iNaN$.
- `clog(NaN + iNaN)` returns $NaN + iNaN$.

G.6.5 Power and absolute-value functions

G.6.5.1 The `cpow` functions

The `cpow` functions raise floating-point exceptions if appropriate for the calculation of the parts of the result, and may also raise spurious floating-point exceptions.⁴⁴⁹⁾

G.6.5.2 The `csqrt` functions

- `csqrt(conj(z)) = conj(csqrt(z))`.
- `csqrt(±0 + i0)` returns $+0 + i0$.
- `csqrt(x + i∞)` returns $+∞ + i∞$, for all x (including NaN).
- `csqrt(x + iNaN)` returns $NaN + iNaN$ and optionally raises the “invalid” floating-point exception, for finite x .
- `csqrt(−∞ + iy)` returns $+0 + i∞$, for finite positive-signed y .
- `csqrt(+∞ + iy)` returns $+∞ + i0$, for finite positive-signed y .
- `csqrt(−∞ + iNaN)` returns $NaN ± i∞$ (where the sign of the imaginary part of the result is unspecified).
- `csqrt(+∞ + iNaN)` returns $+∞ + iNaN$.
- `csqrt(NaN + iy)` returns $NaN + iNaN$ and optionally raises the “invalid” floating-point exception, for finite y .
- `csqrt(NaN + iNaN)` returns $NaN + iNaN$.

⁴⁴⁹⁾This allows `cpow(z, c)` to be implemented as `cexp(cclog(z))` without precluding implementations that treat special cases more carefully.

G.7 Type-generic math <tgmath.h>

Type-generic macros that accept complex arguments also accept imaginary arguments. If an argument is imaginary, the macro expands to an expression whose type is real, imaginary, or complex, as appropriate for the particular function: if the argument is imaginary, then the types of **cos**, **cosh**, **fabs**, **carg**, **cimag**, and **creal** are real; the types of **sin**, **tan**, **sinh**, **tanh**, **asin**, **atan**, **asinh**, and **atanh** are imaginary; and the types of the others are complex.

Given an imaginary argument, each of the type-generic macros **cos**, **sin**, **tan**, **cosh**, **sinh**, **tanh**, **asin**, **atan**, **asinh**, **atanh** is specified by a formula in terms of real functions:

```
cos(iy)   = cosh(y)
sin(iy)   = i sinh(y)
tan(iy)   = i tanh(y)
cosh(iy)  = cos(y)
sinh(iy)  = i sin(y)
tanh(iy)  = i tan(y)
asin(iy)  = i asinh(y)
atan(iy)  = i atanh(y)
asinh(iy) = i asin(y)
atanh(iy) = i atan(y)
```

Annex H
(normative)
ISO/IEC 60559 interchange and extended types

H.1 Introduction

This annex specifies extension types for programming language C that have the arithmetic interchange and extended floating-point formats specified in ISO/IEC 60559. This annex also includes functions that support the non-arithmetic interchange formats in that standard. This annex was adapted from ISO/IEC TS 18661-3:2015, Floating-point extensions for C —Interchange and extended types.

An implementation that defines `__STDC_IEC_60559_TYPES__` to 202311L shall conform to the specifications in this annex. An implementation may define `__STDC_IEC_60559_TYPES__` only if it defines `__STDC_IEC_60559_BFP__`, indicating support for ISO/IEC 60559 binary floating-point arithmetic, or defines `__STDC_IEC_60559_DFP__`, indicating support for ISO/IEC 60559 decimal floating-point arithmetic (or defines both). Where a binding between the C language and ISO/IEC 60559 is indicated, the ISO/IEC 60559-specified behavior is adopted by reference, unless stated otherwise.

H.2 Types

H.2.1 General

This clause specifies types that support ISO/IEC 60559 arithmetic interchange and extended formats. The encoding conversion functions (H.11.4) and numeric conversion functions for encodings (H.12.4, H.12.5) support the non-arithmetic interchange formats specified in ISO/IEC 60559.

H.2.2 Interchange floating types

ISO/IEC 60559 specifies interchange formats, and their encodings, which can be used for the exchange of floating-point data between implementations. These formats are identified by their radix (binary or decimal) and their storage width N . Table H.1, Table H.2, Table H.3, and Table H.4 give the C floating-point model parameters⁴⁵⁰⁾ (5.3.5.3.3) for the ISO/IEC 60559 interchange formats, where the function `round()` rounds to the nearest integer.

Table H.1 — Binary interchange format parameters

Parameter	binary16	binary32	binary64	binary128
N , storage width in bits	16	32	64	128
p , precision in binary digits (bits)	11	24	53	113
$e_{\max}$, maximum exponent e	16	128	1024	16384
$e_{\min}$, minimum exponent e	−13	−125	−1021	−16381

Table H.2 — Binary interchange format parameters for arbitrary N

Parameter	binary N ($N \geq 128$)
N , storage width in bits	N , a multiple of 32
p , precision in binary digits (bits)	$N - \text{round}(4 \times \log_2(N)) + 13$
$e_{\max}$, maximum exponent e	$2^{(N-p-1)}$
$e_{\min}$, minimum exponent e	$3 - e_{\max}$

⁴⁵⁰⁾In ISO/IEC 60559, normal floating-point numbers are expressed with the first significant digit to the left of the radix point. Hence the exponent in the C model (shown in Table H.1, Table H.2 Table H.3, and Table H.4) is 1 more than the exponent of the same number in the ISO/IEC 60559 model.

Table H.3 — Decimal interchange format parameters

Parameter	decimal32	decimal64	decimal128
N , storage width in bits	32	64	128
p , precision in decimal digits	7	16	34
$e_{\max}$, maximum exponent e	97	385	6145
$e_{\min}$, minimum exponent e	−94	−382	−6142

Table H.4 — Decimal interchange format parameters for arbitrary N

Parameter	decimal N ($N \geq 32$)
N , storage width in bits	N , a multiple of 32
p , precision in decimal digits	$9 \times (N \div 32) - 2$
$e_{\max}$, maximum exponent e	$3 \times 2^{((N \div 16) + 3)} + 1$
$e_{\min}$, minimum exponent e	$3 - e_{\max}$

EXAMPLE For the binary160 format, $p = 144$, $e_{\max} = 32678$ and $e_{\min} = -32765$. For the decimal160 format, $p = 43$, $e_{\max} = 24577$ and $e_{\min} = -24574$.

Types designated:

_Float N

where N is 16, 32, 64, or ≥ 128 and a multiple of 32; and, types designated

_Decimal N

where $N \geq 32$ and a multiple of 32, are collectively called the *interchange floating types*. Each interchange floating type has the ISO/IEC 60559 interchange format corresponding to its width (N) and radix (2 for **_Float N** , 10 for **_Decimal N**). Each interchange floating type is not compatible with any other type.

An implementation that defines **__STDC_IEC_60559_BFP__** and **__STDC_IEC_60559_TYPES__** shall provide **_Float32** and **_Float64** as interchange floating types with the same representation and alignment requirements as **float** and **double**, respectively. If the implementation's **long double** type supports an ISO/IEC 60559 interchange format of width $N > 64$, then the implementation shall also provide the type **_Float N** as an interchange floating type with the same representation and alignment requirements as **long double**. The implementation may provide other radix-2 interchange floating types **_Float N** ; the set of such types supported is implementation-defined.

An implementation that defines **__STDC_IEC_60559_DFP__** provides the decimal floating types **_Decimal32**, **_Decimal64**, and **_Decimal128** (6.2.5). If the implementation also defines **__STDC_IEC_60559_TYPES__**, it may provide other radix-10 interchange floating types **_Decimal N** ; the set of such types supported is implementation-defined.

H.2.3 Non-arithmetic interchange formats

An implementation supports ISO/IEC 60559 non-arithmetic interchange formats by providing the associated encoding-to-encoding conversion functions (H.11.4.3) in `<math.h>` and the string-from-encoding functions (H.12.4) and string-to-encoding functions (H.12.5) in `<stdlib.h>`.

An implementation that defines **__STDC_IEC_60559_BFP__** and **__STDC_IEC_60559_TYPES__** supports some ISO/IEC 60559 radix-2 interchange formats as arithmetic formats by providing types **_Float N** (as well as **float** and **double**) with those formats. The implementation may support other ISO/IEC 60559 radix-2 interchange formats as non-arithmetic formats; the set of such formats supported is implementation-defined.

An implementation that defines **__STDC_IEC_60559_DFP__** and **__STDC_IEC_60559_TYPES__** supports some ISO/IEC 60559 radix-10 interchange formats as arithmetic formats by providing types

_DecimalN with those formats. The implementations may support other ISO/IEC 60559 radix-10 interchange formats as non-arithmetic formats; the set of such formats supported is implementation-defined.

H.2.4 Extended floating types

For each of its basic formats, ISO/IEC 60559 specifies an extended format whose maximum exponent and precision exceed those of the basic format it is associated with. Extended formats are intended for arithmetic with more precision and exponent range than is available in the basic formats used for the input data. The extra precision and range often mitigate round-off error and eliminate overflow and underflow in intermediate computations. Table H.5 gives the minimum values of these parameters, as defined for the C floating-point model (5.3.5.3.3). For all ISO/IEC 60559 extended (and interchange) formats, $e_{\min} = 3 - e_{\max}$.

Table H.5 — Extended format parameters for floating-point numbers

	Extended formats associated with:				
Parameter	binary32	binary64	binary128	decimal64	decimal128
$p \text{ digits} \geq$	32	64	128	22	40
$e_{\max} \geq$	1024	16384	65536	6145	24577

Types designated **_Float32x**, **_Float64x**, **_Float128x**, **_Decimal64x**, and **_Decimal128x** support the corresponding ISO/IEC 60559 extended formats and are collectively called the *extended floating types*. The set of values of **_Float32x** is a subset of the set of values of **_Float64x**; the set of values of **_Float64x** is a subset of the set of values of **_Float128x**. The set of values of **_Decimal64x** is a subset of the set of values of **_Decimal128x**. Each extended floating type is not compatible with any other type. An implementation that defines **__STDC_IEC_60559_BFP__** and **__STDC_IEC_60559_TYPES__** shall provide **_Float32x**, and may provide one or both of the types **_Float64x** and **_Float128x**. An implementation that defines **__STDC_IEC_60559_DFP__** and **__STDC_IEC_60559_TYPES__** shall provide **_Decimal64x**, and may provide **_Decimal128x**. Which (if any) of the optional extended floating types are provided is implementation-defined.

NOTE 1 ISO/IEC 60559 does not specify an extended format associated with the decimal32 format, nor does this annex specify an extended type associated with the **_Decimal32** type.

NOTE 2 The **_Float32x** type can have the same format as **double**. The **_Decimal64x** type can have the same format as **_Decimal128**.

H.2.5 Classification of real floating types

6.2.5 defines standard floating types as a collective name for the types **float**, **double** and **long double** and it defines decimal floating types as a collective name for the types **_Decimal32**, **_Decimal64**, and **_Decimal128**.

H.2.2 defines interchange floating types and H.2.4 defines extended floating types.

The types **_FloatN** and **_FloatNx** are collectively called *binary floating types*.

This subclause broadens *decimal floating types* to include the types **_DecimalN** and **_DecimalNx**, introduced in this annex, as well as **_Decimal32**, **_Decimal64**, and **_Decimal128**.

This subclause broadens *real floating types* to include all interchange floating types and extended floating types, as well as standard floating types.

Thus, in this annex, real floating types are classified as follows:

- standard floating types, composed of **float**, **double**, **long double**;
- decimal floating types, composed of **_DecimalN**, **_DecimalNx**;

- binary floating types, composed of `_FloatN`, `_FloatNx`;
- interchange floating types, composed of `_FloatN`, `_DecimalN`; and,
- extended floating types, composed of `_FloatNx`, `_DecimalNx`.

NOTE Standard floating types (which have an implementation-defined radix) are not included in either binary floating types (which always have radix 2) or decimal floating types (which always have radix 10).

H.2.6 Complex types

This subclause broadens the C complex types (6.2.5) to also include similar types whose corresponding real parts have binary floating types. For the types `_FloatN` and `_FloatNx`, there are complex types designated respectively as `_FloatN_Complex` and `_FloatNx_Complex`. (Complex types are a conditional feature that implementations are not required to support; see 6.10.10.4.)

H.2.7 Imaginary types

This subclause broadens the C imaginary types (G.2) to also include similar types whose corresponding real parts have binary floating types. For the types `_FloatN` and `_FloatNx`, there are imaginary types designated respectively as `_FloatN_Imaginary` and `_FloatNx_Imaginary`. The imaginary types (along with the real floating and complex types) are floating types. (Annex G, including imaginary types, is a conditional feature that implementations are not required to support; see 6.10.10.4.)

H.3 Characteristics in `<float.h>`

This subclause enhances the `FLT_EVAL_METHOD` and `DEC_EVAL_METHOD` macros to apply to the types introduced in this annex.

If `FLT_RADIX` is 2, the value of `FLT_EVAL_METHOD` (5.3.5.3.3) characterizes the use of evaluation formats for standard floating types and for binary floating types:

- | | |
|--------------|---|
| -1 | indeterminable; |
| 0 | evaluate all operations and constants, whose semantic type comprises a set of values that is a strict subset of the values of <code>float</code> , to the range and precision of <code>float</code> ; evaluate all other operations and constants to the range and precision of the semantic type; |
| 1 | evaluate operations and constants, whose semantic type comprises a set of values that is a strict subset of the values of <code>double</code> , to the range and precision of <code>double</code> ; evaluate all other operations and constants to the range and precision of the semantic type; |
| 2 | evaluate operations and constants, whose semantic type comprises a set of values that is a strict subset of the values of <code>long double</code> , to the range and precision of <code>long double</code> ; evaluate all other operations and constants to the range and precision of the semantic type; |
| <i>N</i> | where <code>_FloatN</code> is a supported interchange floating type, evaluate operations and constants, whose semantic type comprises a set of values that is a strict subset of the values of <code>_FloatN</code> , to the range and precision of <code>_FloatN</code> ; evaluate all other operations and constants to the range and precision of the semantic type; |
| <i>N</i> + 1 | where <code>_FloatNx</code> is a supported extended floating type, evaluate operations and constants, whose semantic type comprises a set of values that is a strict subset of the values of <code>_FloatNx</code> , to the range and precision of <code>_FloatNx</code> ; evaluate all other operations and constants to the range and precision of the semantic type. |

If `FLT_RADIX` is not 2, the use of evaluation formats for operations and constants of binary floating types is implementation-defined.

The implementation-defined value of `DEC_EVAL_METHOD` (5.3.5.3.4) characterizes the use of evaluation formats for decimal floating types:

- 1 indeterminate;
- 0 evaluate all operations and constants just to the range and precision of the type;
- 1 evaluate operations and constants, whose semantic type comprises a set of values that is a strict subset of the values of `_Decimal64`, to the range and precision of `_Decimal64`; evaluate all other operations and constants to the range and precision of the semantic type;
- 2 evaluate operations and constants, whose semantic type comprises a set of values that is a strict subset of the values of `_Decimal128`, to the range and precision of `_Decimal128`; evaluate all other operations and constants to the range and precision of the semantic type;
- N* where `_DecimalN` is a supported interchange floating type, evaluate operations and constants, whose semantic type comprises a set of values that is a strict subset of the values of `_DecimalN`, to the range and precision of `_DecimalN`; evaluate all other operations and constants to the range and precision of the semantic type;
- N* + 1 where `_DecimalNx` is a supported extended floating type, evaluate operations and constants, whose semantic type comprises a set of values that is a strict subset of the values of `_DecimalNx`, to the range and precision of `_DecimalNx`; evaluate all other operations and constants to the range and precision of the semantic type.

This subclause also specifies `<float.h>` macros, analogous to the macros for standard floating types, that characterize binary floating types in terms of the model presented in 5.3.5.3.3. This subclause generalizes the specification of characteristics in 5.3.5.3.4 to include the decimal floating types introduced in this annex. The prefix `FLTN_` indicates the type `_FloatN` or the non-arithmetic binary interchange format of width *N*. The prefix `FLTNX_` indicates the type `_FloatNx`. The prefix `DECN_` indicates the type `_DecimalN` or the non-arithmetic decimal interchange format of width *N*. The prefix `DECNX_` indicates the type `_DecimalNx`. The type parameters *p*, *e*_{max}, and *e*_{min} for extended floating types are for the extended floating type itself, not for the basic format that it extends.

If `__STDC_WANT_IEC_60559_TYPES_EXT__` is defined (by the user) at the point in the code where `<float.h>` is first included, the following applies (H.8). For each interchange or extended floating type that the implementation provides, `<float.h>` shall define the associated macros in the following lists. Conversely, for each such type that the implementation does not provide, `<float.h>` shall not define the associated macros in the following list, except, the implementation shall define the macros `FLTN_DECIMAL_DIG` and `FLTN_DIG` if it supports the ISO/IEC 60559 non-arithmetic binary interchange format of width *N* (H.2.3).

The signaling NaN macros

<code>FLT<i>N</i>_SNAN</code>
<code>DEC<i>N</i>_SNAN</code>
<code>FLT<i>N</i>X_<i>N</i>SNAN</code>
<code>DEC<i>N</i>X_<i>N</i>SNAN</code>

expand to constant expressions of types `_FloatN`, `_DecimalN`, `_FloatNx`, and `_DecimalNx` respectively, representing a signaling NaN. If an optional unary + or - operator followed by a signaling NaN macro is used for initializing an object of the same type that has static or thread storage duration, the object is initialized with a signaling NaN value.

The integer values given in the following lists shall be replaced by integer constant expressions:

- radix of exponent representation, *b* (2 for binary, 10 for decimal)

For the standard floating types, this value is implementation-defined and is specified by the macro `FLT_RADIX`. For the interchange and extended floating types there is no corresponding macro; the radix is an inherent property of the types.

- The number of bits in the floating-point significand, p

FLT_N_MANT_DIG
FLT_NX_MANT_DIG

- The number of digits in the coefficient, p

DEC_N_MANT_DIG
DEC_NX_MANT_DIG

- number of decimal digits, n , such that any floating-point number with p bits can be rounded to a floating-point number with n decimal digits and back again without change to the value, $\lceil 1 + p \log_{10}(2) \rceil$

FLT_N_DECIMAL_DIG
FLT_NX_DECIMAL_DIG

- number of decimal digits, q , such that any floating-point number with q decimal digits can be rounded to a floating-point number with p bits and back again without a change to the q decimal digits, $\lfloor (p - 1) \log_{10}(2) \rfloor$

FLT_N_DIG
FLT_NX_DIG

- minimum negative integer such that the radix raised to one less than that power is a normalized floating-point number, $e_{\min}$

FLT_N_MIN_EXP
FLT_NX_MIN_EXP
DEC_N_MIN_EXP
DEC_NX_MIN_EXP

- minimum negative integer such that 10 raised to that power is in the range of normalized floating-point numbers, $\lceil \log_{10}(2)^{e_{\min} - 1} \rceil$

FLT_N_MIN_10_EXP
FLT_NX_MIN_10_EXP

- maximum positive integer such that the radix raised to one less than that power is a representable finite floating-point number, $e_{\max}$

FLT_N_MAX_EXP
FLT_NX_MAX_EXP
DEC_N_MAX_EXP
DEC_NX_MAX_EXP

- maximum integer such that 10 raised to that power is in the range of representable finite floating-point numbers, $\lfloor \log_{10}((1 - 2^{-p})2^{e_{\max}}) \rfloor$

FLT_N_MAX_10_EXP
FLT_NX_MAX_10_EXP

- maximum representable finite floating-point number, $(1 - b^{-p})b^{e_{\max}}$

FLT_N_MAX
FLT_NX_MAX
DEC_N_MAX
DEC_NX_MAX

- the difference between 1 and the least normalized value greater than 1 that is representable in the given floating type, b^{1-p}

FLT/_EPSILON
FLT/_X_EPSILON
DEC/_EPSILON
DEC/_X_EPSILON

- minimum normalized positive floating-point number, $b^{e_{\min}-1}$

FLT/_MIN
FLT/_X_MIN
DEC/_MIN
DEC/_X_MIN

- minimum positive floating-point number, $b^{e_{\min}-p}$

FLT/_TRUE_MIN
FLT/_X_TRUE_MIN
DEC/_TRUE_MIN
DEC/_X_TRUE_MIN

H.4 Conversions

H.4.1 General

This subclause enhances the usual arithmetic conversions (6.3.2.8) to handle interchange and extended floating types. It supports the ISO/IEC 60559 recommendation against allowing implicit conversions of operands to obtain a common type where the conversion is between types where neither is a subset of (or equivalent to) the other.

This subclause also broadens the operation binding in F.3 for the ISO/IEC 60559 convertFormat operation to apply to ISO/IEC 60559 arithmetic and non-arithmetic formats.

H.4.2 Real floating and integer

When a finite value of interchange or extended floating type is converted to an integer type other than **bool**, the fractional part is discarded (i.e. the value is truncated toward zero). If the value of the integral part cannot be represented by the integer type, the “invalid” floating-point exception shall be raised and the result of the conversion is unspecified.

When a value of integer type is converted to an interchange or extended floating type, if the value being converted can be represented exactly in the new type, it is unchanged. If the value being converted cannot be represented exactly, the result shall be correctly rounded with exceptions raised as specified in ISO/IEC 60559.

H.4.3 Usual arithmetic conversions

If either operand is of floating type, the common real type is determined as follows:

- If one operand has decimal floating type, the other operand shall not have standard floating type, binary floating type, complex type, or imaginary type.
- If only one operand has a floating type, the other operand is converted to the corresponding real type of the operand of floating type.
- If both operands have the same corresponding real type, no further conversion is needed.
- If both operands have floating types and neither of the sets of values of their corresponding real types is a subset of (or equivalent to) the other, the behavior is undefined.

- Otherwise, if both operands are floating types and the sets of values of their corresponding real types are not equivalent, the operand whose set of values of its corresponding real type is a strict subset of the set of values of the corresponding real type of the other operand is converted, without change of type domain, to a type with the corresponding real type of that other operand.
- Otherwise, if both operands are floating types and the sets of values of their corresponding real types are equivalent, then the following rules are applied:
 - If the corresponding real type of either operand is an interchange floating type, the other operand is converted, without change of type domain, to a type whose corresponding real type is that same interchange floating type.
 - Otherwise, if the corresponding real type of either operand is **long double**, the other operand is converted, without change of type domain, to a type whose corresponding real type is **long double**.
 - Otherwise, if the corresponding real type of either operand is **double**, the other operand is converted, without change of type domain, to a type whose corresponding real type is **double**.⁴⁵¹⁾
 - Otherwise, if the corresponding real type of either operand is **_Float128x** or **_Decimal128x**, the other operand is converted, without change of type domain, to a type whose corresponding real type is **_Float128x** or **_Decimal128x**, respectively.
 - Otherwise, if the corresponding real type of either operand is **_Float64x** or **_Decimal64x**, the other operand is converted, without change of type domain, to a type whose corresponding real type is **_Float64x** or **_Decimal64x**, respectively.

H.4.4 Arithmetic and non-arithmetic formats

The operation binding in F.3 for the ISO/IEC 60559 `convertFormat` operation applies to ISO/IEC 60559 arithmetic and non-arithmetic formats as follows:

- For conversions between arithmetic formats supported by floating types (same or different radix) – casts and implicit conversions.
- For same-radix conversions between non-arithmetic interchange formats – encoding-to-encoding conversion functions (H.11.4.3).
- For conversions between non-arithmetic interchange formats (same or different radix) – compositions of string-from-encoding functions (H.12.4) (converting exactly) and string-to-encoding functions (H.12.5).
- For same-radix conversions from interchange formats supported by interchange floating types to non-arithmetic interchange formats – compositions of encode functions (H.11.4.2.2, 7.12.17.2, 7.12.17.4) and encoding-to-encoding functions (H.11.4.3).
- For same radix conversions from non-arithmetic interchange formats to interchange formats supported by interchange floating types – compositions of encoding-to-encoding conversion functions (H.11.4.3) and decode functions (H.11.4.2.3, 7.12.17.3, 7.12.17.5). See the example in H.11.4.3.2.
- For conversions from non-arithmetic interchange formats to arithmetic formats supported by floating types (same or different radix) – compositions of string-from-encoding functions (H.12.4) (converting exactly) and numeric conversion functions **strtod**, etc. (7.24.2.6, 7.24.2.7). See the example in H.12.3.
- For conversions from arithmetic formats supported by floating types to non-arithmetic interchange formats (same or different radix) – compositions of numeric conversion functions **strfromd**, etc. (7.24.2.4, 7.24.2.5) (converting exactly) and string-to-encoding functions (H.12.5).

⁴⁵¹⁾ All cases where **float** is expected to have the same format as another type are covered in the preceding paragraphs.

H.5 Lexical Elements

H.5.1 Keywords

This subclause expands the list of keywords (6.4.2) to also include:

- **_Float***N*, where *N* is 16, 32, 64, or ≥ 128 and a multiple of 32
- **_Float32x**
- **_Float64x**
- **_Float128x**
- **_Decimal***N*, where *N* is 96 or > 128 and a multiple of 32
- **_Decimal64x**
- **_Decimal128x**

H.5.2 Constants

This subclause specifies constants of interchange and extended floating types.

This subclause expands floating-suffix (6.4.5.3) to also include: **fN**, **FN**, **fNx**, **FNx**, **dN**, **DN**, **dNx**, or **DNx**.

A floating suffix **dN**, **DN**, **dNx**, or **DNx** shall not be used in a hexadecimal-floating-constant.

A floating suffix shall not designate a type that the implementation does not provide.

If a floating constant is suffixed by **fN** or **FN**, it has type **_Float***N*. If suffixed by **fNx** or **FNx**, it has type **_FloatNx**. If suffixed by **dN** or **DN**, it has type **_Decimal***N*. If suffixed by **dNx** or **DNx**, it has type **_DecimalNx**.

The quantum exponent of a floating constant of decimal floating type is the same as for the result value of the corresponding **strtodN** or **strtodNx** function (H.12.3) for the same numeric string.

NOTE For *N* = 32, 64, and 128, the suffixes **dN** and **DN** in this subclause for constants of type **_Decimal***N* are equivalent alternatives to the suffixes **df**, **dd**, **dL**, **DF**, **DD**, and **DL** in 6.4.5.3 for the same types.

H.6 Expressions

This subclause expands the specification of expressions to also cover interchange and extended floating types.

Operators involving operands of interchange or extended floating type are evaluated according to the semantics of ISO/IEC 60559, including production of decimal floating-point results with the preferred quantum exponent as specified in ISO/IEC 60559 (see 5.3.5.3.4).

For multiplicative operators (6.5.6), additive operators (6.5.7), relational operators (6.5.9), equality operators (6.5.10), and compound assignment operators (6.5.17.3), if either operand has decimal floating type, the other operand shall not have standard floating type, binary floating type, complex type, or imaginary type.

For conditional operators (6.5.16), if the second or third operand has decimal floating type, the other of those operands shall not have standard floating type, binary floating type, complex type, or imaginary type.

The equivalence of expressions noted in F.9.3 apply to expressions of binary floating types, as well as standard floating types.

H.7 Declarations

This subclause expands the list of type specifiers (6.7.3) to also include:

- **_Float***N*, where *N* is 16, 32, 64, or ≥ 128 and a multiple of 32
- **_Float32x**

- **_Float64x**
- **_Float128x**
- **_DecimalN**, where N is 96 or > 128 and a multiple of 32
- **_Decimal64x**
- **_Decimal128x**

The type specifiers **_FloatN** (where N is 16, 32, 64, or ≥ 128 and a multiple of 32), **_Float32x**, **_Float64x**, **_Float128x**, **_DecimalN** (where N is 96 or > 128 and a multiple of 32), **_Decimal64x**, and **_Decimal128x** shall not be used if the implementation does not support the corresponding types (see 6.10.10.4 and H.2).

This subclause also expands the list under Constraints in 6.7.3 to also include:

- **_FloatN**, where N is 16, 32, 64, or ≥ 128 and a multiple of 32
- **_Float32x**
- **_Float64x**
- **_Float128x**
- **_DecimalN**, where N is 96 or > 128 and a multiple of 32
- **_Decimal64x**
- **_Decimal128x**
- **_FloatN _Complex**, where N is 16, 32, 64, or ≥ 128 and a multiple of 32
- **_Float32x _Complex**
- **_Float64x _Complex**
- **_Float128x _Complex**

H.8 Identifiers in standard headers

The identifiers added to library headers by this annex are defined or declared by their respective headers only if the macro **__STDC_WANT_IEC_60559_TYPES_EXT** is defined (by the user) at the point in the code where the appropriate header is first included.

H.9 Complex arithmetic <complex.h>

This subclause specifies complex functions for corresponding real types that are binary floating types.

Each function synopsis in 7.3 specifies a family of functions including a principal function with one or more **double complex** parameters and a **double complex** or **double** return value. This subclause expands the synopsis to also include other functions, with the same name as the principal function but with **fN** and **fNx** suffixes, which are corresponding functions whose parameters and return values have corresponding real types **_FloatN** and **_FloatNx**.

The following function prototypes are added to the synopses of the respective subclauses in 7.3. For each binary floating type that the implementation provides, <complex.h> shall declare the associated functions (see H.8). Conversely, for each such type that the implementation does not provide, <complex.h> shall not declare the associated functions.

7.3.5 Trigonometric functions

```

_FloatN complex cacosfN(_FloatN complex z);
_FloatNx complex cacosfNx(_FloatNx complex z);
_FloatN complex casinfN(_FloatN complex z);
_FloatNx complex casinfNx(_FloatNx complex z);
_FloatN complex catanfN(_FloatN complex z);
_FloatNx complex catanfNx(_FloatNx complex z);
_FloatN complex ccosfN(_FloatN complex z);
_FloatNx complex ccosfNx(_FloatNx complex z);
_FloatN complex csinfN(_FloatN complex z);
_FloatNx complex csinfNx(_FloatNx complex z);
_FloatN complex ctanfN(_FloatN complex z);
_FloatNx complex ctanfNx(_FloatNx complex z);

```

7.3.6 Hyperbolic functions

```

_FloatN complex cacoshfN(_FloatN complex z);
_FloatNx complex cacoshfNx(_FloatNx complex z);
_FloatN complex casinhfN(_FloatN complex z);
_FloatNx complex casinhfNx(_FloatNx complex z);
_FloatN complex catanhfN(_FloatN complex z);
_FloatNx complex catanhfNx(_FloatNx complex z);
_FloatN complex ccoshfN(_FloatN complex z);
_FloatNx complex ccoshfNx(_FloatNx complex z);
_FloatN complex csinhfN(_FloatN complex z);
_FloatNx complex csinhfNx(_FloatNx complex z);
_FloatN complex ctanhfN(_FloatN complex z);
_FloatNx complex ctanhfNx(_FloatNx complex z);

```

7.3.7 Exponential and logarithmic functions

```

_FloatN complex cexpfN(_FloatN complex z);
_FloatNx complex cexpfNx(_FloatNx complex z);
_FloatN complex clogfN(_FloatN complex z);
_FloatNx complex clogfNx(_FloatNx complex z);

```

7.3.8 Power and absolute value functions

```

_FloatN cabsfN(_FloatN complex z);
_FloatNx cabsfNx(_FloatNx complex z);
_FloatN complex cpowfN(_FloatN complex x, _FloatN complex y);
_FloatNx complex cpowfNx(_FloatNx complex x, _FloatNx complex y);
_FloatN complex csqrtfN(_FloatN complex z);
_FloatNx complex csqrtfNx(_FloatNx complex z);

```

7.3.9 Manipulation functions

```

_FloatN cargfN(_FloatN complex z);
_FloatNx cargfNx(_FloatNx complex z);
_FloatN cimagfN(_FloatN complex z);
_FloatNx cimagfNx(_FloatNx complex z);
_FloatN complex CMPLXFN(_FloatN x, _FloatN y);
_FloatNx complex CMPLXFNX(_FloatNx x, _FloatNx y);
_FloatN complex conjfN(_FloatN complex z);
_FloatNx complex conjfNx(_FloatNx complex z);
_FloatN complex cprojfN(_FloatN complex z);
_FloatNx complex cprojfNx(_FloatNx complex z);
_FloatN crealfN(_FloatN complex z);
_FloatNx crealfNx(_FloatNx complex z);

```

For the functions listed in “future library directions” for `<complex.h>` (7.33.2), the possible suffixes are expanded to also include **fN** and **fNx**.

H.10 Floating-point environment

This subclause broadens the effects of the floating-point environment (7.6) to apply to types and formats specified in this annex.

The same floating-point status flags are used by floating-point operations for all floating types, including those types introduced in this annex, and by conversions for ISO/IEC 60559 non-arithmetic interchange formats.

Both the dynamic rounding direction mode accessed by **fegetround** and **fesetround** and the **FENV_ROUND** rounding control pragma apply to operations for binary floating types, as well as for standard floating types, and also to conversions for radix-2 non-arithmetic interchange formats. Likewise, both the dynamic rounding direction mode accessed by **fe_dec_getround** and **fe_dec_setround** and the **FENV_DEC_ROUND** rounding control pragmas apply to operations for all the decimal floating types, including those decimal floating types introduced in this annex, and to conversions for radix-10 non-arithmetic interchange formats.

Table 7.1, which describes functions affected by constant rounding modes for standard floating types, applies also for binary floating types. Each `<math.h>` function family listed in the table indicates the family of functions of all standard and binary floating types (for example, the **acos** family includes **acosf**, **acosl**, **acosfN**, and **acosfNx** as well as **acos**). The **fMencfN**, **strfromencfN**, and **strtoencfN** functions are also affected by these constant rounding modes.

Table 7.2, which describes functions affected by constant rounding modes for decimal floating types, each `<math.h>` function family indicates the family of functions of all decimal floating types (for example, the **acos** family includes **acosdN** and **acosdNx**). The **dMencbindN**, **dMencdecN**, **strfromencbindN**, **strfromencdecN**, **strtoencbindN**, and **strtoencdecN** functions are also affected by these constant rounding modes.

H.11 Mathematics `<math.h>`

H.11.1 General

This subclause specifies types, functions, and macros for interchange and extended floating types, generally corresponding to those specified in 7.12 and F.10.

All classification macros (7.12.4) and comparison macros (7.12.18) naturally extend to handle interchange and extended floating types. For comparison macros, if neither of the sets of values of the argument formats is a subset of (or equivalent to) the other, the behavior is undefined.

This subclause also specifies encoding conversion functions that are part of support for the non-arithmetic interchange formats in ISO/IEC 60559 (see H.2.3).

Most function synopses in 7.12 specify a family of functions including a principal function with one or more **double** parameters, a **double** return value, or both. The synopses are expanded to also include functions with the same name as the principal function but with **fN**, **fNx**, **dN**, and **dNx** suffixes, which are corresponding functions whose parameters, return values, or both are of types **_FloatN**, **_FloatNx**, **_DecimalN**, and **_DecimalNx**, respectively.

For each interchange or extended floating type that the implementation provides, `<math.h>` shall define the associated types and macros and declare the associated functions (see H.8). Conversely, for each such type that the implementation does not provide, `<math.h>` shall not define the associated types and macros or declare the associated functions unless explicitly specified otherwise.

With the types

```
float_t
double_t
```

in 7.12 are included the type

long_double_t

and for each supported type `_FloatN`, the type

_FloatN_t

and for each supported type `_DecimalN`, the type

_DecimalN_t

These are floating types, such that:

- the values of `long double` are a subset of the values of `long_double_t`;
- the values of `_FloatN` are a subset of the values of `_FloatN_t`;
- the values of `_DecimalN` are a subset of the values of `_DecimalN_t`;
- the values of `double_t` are a subset of the values of `long_double_t`;
- the values of `_FloatM_t` are a subset of the values of `_FloatN_t` if $M < N$;
- the values of `_DecimalM_t` are a subset of the values of `_DecimalN_t` if $M < N$.

If `FLT_RADIX` is 2 and `FLT_EVAL_METHOD` (H.3) is nonnegative, then each of the types corresponding to a standard or binary floating type is the type whose range and precision are specified by `FLT_EVAL_METHOD` to be used for evaluating operations and constants of that standard or binary floating type. If `DEC_EVAL_METHOD` (H.3) is nonnegative, then each of the types corresponding to a decimal floating type is the type whose range and precision are specified by `DEC_EVAL_METHOD` to be used for evaluating operations and constants of that decimal floating type.

EXAMPLE If the supported standard and binary floating types are as shown in Table H.6:

Table H.6 — Example supported types

Type	ISO/IEC 60559 format
<code>_Float16</code>	binary16
<code>float, _Float32</code>	binary32
<code>double, _Float64, _Float32x</code>	binary64
<code>long double, _Float64x</code>	80-bit binary64-extended
<code>_Float128</code>	binary128

then Table H.7 gives the types with `_t` suffixes for various values for a `FLT_EVAL_METHOD` of a given value m :

Table H.7 — `_t` type (vertical) vs. m (horizontal) relation

<code>_t</code> type	m			
	0	1	2	32
<code>_Float16_t</code>	<code>float</code>	<code>double</code>	<code>long double</code>	<code>_Float32</code>
<code>float_t</code>	<code>float</code>	<code>double</code>	<code>long double</code>	<code>float</code>
<code>_Float32_t</code>	<code>_Float32</code>	<code>double</code>	<code>long double</code>	<code>_Float32</code>
<code>double_t</code>	<code>double</code>	<code>double</code>	<code>long double</code>	<code>double</code>
<code>_Float64_t</code>	<code>_Float64</code>	<code>_Float64</code>	<code>long double</code>	<code>_Float64</code>
<code>long_double_t</code>	<code>long double</code>	<code>long double</code>	<code>long double</code>	<code>long double</code>
<code>_Float128_t</code>	<code>_Float128</code>	<code>_Float128</code>	<code>_Float128</code>	<code>_Float128</code>

_t type	<i>m</i>			
	64	128	33	65
_Float16_t	_Float64	_Float128	_Float32x	_Float64x
float_t	_Float64	_Float128	_Float32x	_Float64x
_Float32_t	_Float64	_Float128	_Float32x	_Float64x
double_t	double	_Float128	double	_Float64x
_Float64_t	_Float64	_Float128	_Float64	_Float64x
long_double_t	long double	_Float128	long double	long double
_Float128_t	_Float128	_Float128	_Float128	_Float128

H.11.2 Macros

This subclause adds macros in 7.12 as follows.

The macros

```
HUGE_VAL_FN
HUGE_VAL_DN
HUGE_VAL_FNX
HUGE_VAL_DNX
```

expand to constant expressions of types **_FloatN**, **_DecimalN**, **_FloatNx**, and **_DecimalNx**, respectively, representing positive infinity.

The macros

```
FP_FAST_FMAFN
FP_FAST_FMADN
FP_FAST_FMAFNX
FP_FAST_FMADNX
```

are, respectively, **_FloatN**, **_DecimalN**, **_FloatNx**, and **_DecimalNx** analogues of **FP_FAST_FMA**.

The macros in the following lists are interchange and extended floating type analogues of **FP_FAST_FADD**, **FP_FAST_FADDL**, **FP_FAST_DADDL**, etc.

For $M < N$, the macros

```
FP_FAST_FMADDFN
FP_FAST_FMSUBFN
FP_FAST_FMMULFN
FP_FAST_FMDIVFN
FP_FAST_FMFMAFN
FP_FAST_FMSQRTFN
FP_FAST_DMADDDN
FP_FAST_DMSUBDN
FP_FAST_DMMULDN
FP_FAST_DMDIVDN
FP_FAST_DMFADN
FP_FAST_DMSQRDN
```

characterize the corresponding functions whose arguments are of an interchange floating type of width N and whose return type is an interchange floating type of width M .

For $M \leq N$, the macros

```
FP_FAST_FMAFDNX
FP_FAST_FMSUBFNX
FP_FAST_FMMULFNX
FP_FAST_FMDIVFNX
FP_FAST_FMFMAFNX
FP_FAST_FMSQRTFNX
```

```
FP_FAST_DMADDDNX
FP_FAST_DMSUBDNX
FP_FAST_DMMULDNX
FP_FAST_DMDIVDNX
FP_FAST_DMFMADNX
FP_FAST_DMSQRDNX
```

characterize the corresponding functions whose arguments are of an extended floating type that extends a format of width N and whose return type is an interchange floating type of width M .
For $M < N$, the macros

```
FP_FAST_FMXADDFN
FP_FAST_FMXSUBFN
FP_FAST_FMXMULFN
FP_FAST_FMXDIVFN
FP_FAST_FMXFMAFN
FP_FAST_FMXSQRTFN
FP_FAST_DMxADDDN
FP_FAST_DMXSUBDN
FP_FAST_DMXMULDN
FP_FAST_DMxDIVDN
FP_FAST_DMXFMAFN
FP_FAST_DMXSQRDN
```

characterize the corresponding functions whose arguments are of an interchange floating type of width N and whose return type is an extended floating type that extends a format of width M .
For $M < N$, the macros

```
FP_FAST_FMXADDFNX
FP_FAST_FMXSUBFNX
FP_FAST_FMXMULFNX
FP_FAST_FMXDIVFNX
FP_FAST_FMXFMAFNX
FP_FAST_FMXSQRTFNX
FP_FAST_DMxADDDNX
FP_FAST_DMXSUBDNX
FP_FAST_DMXMULDNX
FP_FAST_DMxDIVDNX
FP_FAST_DMXFMAFNX
FP_FAST_DMXSQRDNX
```

characterize the corresponding functions whose arguments are of an extended floating type that extends a format of width N and whose return type is an extended floating type that extends a format of width M .

H.11.3 Functions

This subclause adds the following functions to the synopses of the respective subclauses in 7.12.
7.12.5 Trigonometric functions

```
_FloatN acosfN(_FloatN x);
_FloatNx acosfNx(_FloatNx x);
_DecimalN acosdN(_DecimalN x);
_DecimalNx acosdNx(_DecimalNx x);

_FloatN asinfN(_FloatN x);
_FloatNx asinfNx(_FloatNx x);
_DecimalN asindN(_DecimalN x);
_DecimalNx asindNx(_DecimalNx x);
```

```

_FloatN atanfN(_FloatN x);
_FloatNx atanfNx(_FloatNx x);
_DecimalW atandN(_DecimalW x);
_DecimalNx atandNx(_DecimalNx x);

_FloatN atan2fN(_FloatN y, _FloatN x);
_FloatNx atan2fNx(_FloatNx y, _FloatNx x);
_DecimalW atan2dN(_DecimalW y, _DecimalW x);
_DecimalNx atan2dNx(_DecimalNx y, _DecimalNx x);

_FloatN cosfN(_FloatN x);
_FloatNx cosfNx(_FloatNx x);
_DecimalW cosdN(_DecimalW x);
_DecimalNx cosdNx(_DecimalNx x);

_FloatN sinfN(_FloatN x);
_FloatNx sinfNx(_FloatNx x);
_DecimalW sindN(_DecimalW x);
_DecimalNx sindNx(_DecimalNx x);

_FloatN tanfN(_FloatN x);
_FloatNx tanfNx(_FloatNx x);
_DecimalW tandN(_DecimalW x);
_DecimalNx tandNx(_DecimalNx x);

_FloatN acospiFN(_FloatN x);
_FloatNx acospiFNx(_FloatNx x);
_DecimalW acospidN(_DecimalW x);
_DecimalNx acospidNx(_DecimalNx x);

_FloatN asinpiFN(_FloatN x);
_FloatNx asinpiFNx(_FloatNx x);
_DecimalW asinpidN(_DecimalW x);
_DecimalNx asinpidNx(_DecimalNx x);

_FloatN atanpiFN(_FloatN x);
_FloatNx atanpiFNx(_FloatNx x);
_DecimalW atanpidN(_DecimalW x);
_DecimalNx atanpidNx(_DecimalNx x);

_FloatN atan2piFN(_FloatN y, _FloatN x);
_FloatNx atan2piFNx(_FloatNx y, _FloatNx x);
_DecimalW atan2pidN(_DecimalW y, _DecimalW x);
_DecimalNx atan2pidNx(_DecimalNx y, _DecimalNx x);

_FloatN cospiFN(_FloatN x);
_FloatNx cospiFNx(_FloatNx x);
_DecimalW cospidN(_DecimalW x);
_DecimalNx cospidNx(_DecimalNx x);

_FloatN sinpiFN(_FloatN x);
_FloatNx sinpiFNx(_FloatNx x);
_DecimalW sinpidN(_DecimalW x);
_DecimalNx sinpidNx(_DecimalNx x);

_FloatN tanpiFN(_FloatN x);
_FloatNx tanpiFNx(_FloatNx x);
_DecimalW tanpidN(_DecimalW x);
_DecimalNx tanpidNx(_DecimalNx x);

```

7.12.6 Hyperbolic functions

```
_FloatN acoshfN(_FloatN x);
_FloatNx acoshfNx(_FloatNx x);
_DecimalN acoshdN(_DecimalN x);
_DecimalNx acoshdNx(_DecimalNx x);

_FloatN asinhfN(_FloatN x);
_FloatNx asinhfNx(_FloatNx x);
_DecimalN asinhdN(_DecimalN x);
_DecimalNx asinhdNx(_DecimalNx x);

_FloatN atanhfN(_FloatN x);
_FloatNx atanhfNx(_FloatNx x);
_DecimalN atanhdN(_DecimalN x);
_DecimalNx atanhdNx(_DecimalNx x);

_FloatN coshfN(_FloatN x);
_FloatNx coshfNx(_FloatNx x);
_DecimalN coshdN(_DecimalN x);
_DecimalNx coshdNx(_DecimalNx x);

_FloatN sinhN(_FloatN x);
_FloatNx sinhNx(_FloatNx x);
_DecimalN sinhdN(_DecimalN x);
_DecimalNx sinhdNx(_DecimalNx x);

_FloatN tanhfN(_FloatN x);
_FloatNx tanhfNx(_FloatNx x);
_DecimalN tanhdN(_DecimalN x);
_DecimalNx tanhdNx(_DecimalNx x);
```

7.12.7 Exponential and logarithmic functions

```
_FloatN expfN(_FloatN x);
_FloatNx expfNx(_FloatNx x);
_DecimalN expdN(_DecimalN x);
_DecimalNx expdNx(_DecimalNx x);

_FloatN exp10fN(_FloatN x);
_FloatNx exp10fNx(_FloatNx x);
_DecimalN exp10dN(_DecimalN x);
_DecimalNx exp10dNx(_DecimalNx x);

_FloatN exp10m1fN(_FloatN x);
_FloatNx exp10m1fNx(_FloatNx x);
_DecimalN exp10m1dN(_DecimalN x);
_DecimalNx exp10m1dNx(_DecimalNx x);

_FloatN exp2fN(_FloatN x);
_FloatNx exp2fNx(_FloatNx x);
_DecimalN exp2dN(_DecimalN x);
_DecimalNx exp2dNx(_DecimalNx x);

_FloatN exp2m1fN(_FloatN x);
_FloatNx exp2m1fNx(_FloatNx x);
_DecimalN exp2m1dN(_DecimalN x);
_DecimalNx exp2m1dNx(_DecimalNx x);

_FloatN expm1fN(_FloatN x);
_FloatNx expm1fNx(_FloatNx x);
```

```

_DecimalN expm1dN(_DecimalN x);
_DecimalNx expm1dNx(_DecimalNx x);

_FloatN frexpfN(_FloatN value, int *exp);
_FloatNx frexpfNx(_FloatNx value, int *exp);
_DecimalN frexpdN(_DecimalN value, int *exp);
_DecimalNx frexpdNx(_DecimalNx value, int *exp);

int ilogbfN(_FloatN x);
int ilogbfNx(_FloatNx x);
int ilogbdN(_DecimalN x);
int ilogbdNx(_DecimalNx x);

_FloatN ldexpfN(_FloatN value, int exp);
_FloatNx ldexpfNx(_FloatNx value, int exp);
_DecimalN ldexpdN(_DecimalN value, int exp);
_DecimalNx ldexpdNx(_DecimalNx value, int exp);

long int llogbfN(_FloatN x);
long int llogbfNx(_FloatNx x);
long int llogbdN(_DecimalN x);
long int llogbdNx(_DecimalNx x);

_FloatN logfN(_FloatN x);
_FloatNx logfNx(_FloatNx x);
_DecimalN logdN(_DecimalN x);
_DecimalNx logdNx(_DecimalNx x);

_FloatN log10fN(_FloatN x);
_FloatNx log10fNx(_FloatNx x);
_DecimalN log10dN(_DecimalN x);
_DecimalNx log10dNx(_DecimalNx x);

_FloatN log10p1fN(_FloatN x);
_FloatNx log10p1fNx(_FloatNx x);
_DecimalN log10p1dN(_DecimalN x);
_DecimalNx log10p1dNx(_DecimalNx x);

_FloatN log1pfN(_FloatN x);
_FloatNx log1pfNx(_FloatNx x);
_FloatN logp1fN(_FloatN x);
_FloatNx logp1fNx(_FloatNx x);
_DecimalN log1pdN(_DecimalN x);
_DecimalNx log1pdNx(_DecimalNx x);
_DecimalN logp1dN(_DecimalN x);
_DecimalNx logp1dNx(_DecimalNx x);

_FloatN log2fN(_FloatN x);
_FloatNx log2fNx(_FloatNx x);
_DecimalN log2dN(_DecimalN x);
_DecimalNx log2dNx(_DecimalNx x);

_FloatN log2p1fN(_FloatN x);
_FloatNx log2p1fNx(_FloatNx x);
_DecimalN log2p1dN(_DecimalN x);
_DecimalNx log2p1dNx(_DecimalNx x);

_FloatN logbfN(_FloatN x);
_FloatNx logbfNx(_FloatNx x);
_DecimalN logbdN(_DecimalN x);
_DecimalNx logbdNx(_DecimalNx x);

```

```

_FloatN modffN(_FloatN x, _FloatN *iptr);
_FloatNx modffNx(_FloatNx x, _FloatNx *iptr);
_DecimalN modfdN(_DecimalN x, _DecimalN *iptr);
_DecimalNx modfdNx(_DecimalNx x, _DecimalNx *iptr);

_FloatN scalbnfN(_FloatN value, int exp);
_FloatNx scalbnfNx(_FloatNx value, int exp);
_DecimalN scalbndN(_DecimalN value, int exp);
_DecimalNx scalbndNx(_DecimalNx value, int exp);

_FloatN scalblnfN(_FloatN value, long int exp);
_FloatNx scalblnfNx(_FloatNx value, long int exp);
_DecimalN scalblndN(_DecimalN value, long int exp);
_DecimalNx scalblndNx(_DecimalNx value, long int exp);

```

7.12.8 Power and absolute-value functions

```

_FloatN cbrtfN(_FloatN x);
_FloatNx cbrtfNx(_FloatNx x);
_DecimalN cbrtdN(_DecimalN x);
_DecimalNx cbrtdNx(_DecimalNx x);

_FloatN compoundnfN(_FloatN x, long long int n);
_FloatNx compoundnfNx(_FloatNx x, long long int n);
_DecimalN compoundndN(_DecimalN x, long long int n);
_DecimalNx compoundndNx(_DecimalNx x, long long int n);

_FloatN fabsfN(_FloatN x);
_FloatNx fabsfNx(_FloatNx x);
_DecimalN fabsdN(_DecimalN x);
_DecimalNx fabsdNx(_DecimalNx x);

_FloatN hypotfN(_FloatN x, _FloatN y);
_FloatNx hypotfNx(_FloatNx x, _FloatNx y);
_DecimalN hypotdN(_DecimalN x, _DecimalN y);
_DecimalNx hypotdNx(_DecimalNx x, _DecimalNx y);

_FloatN powfN(_FloatN x, _FloatN y);
_FloatNx powfNx(_FloatNx x, _FloatNx y);
_DecimalN powdN(_DecimalN x, _DecimalN y);
_DecimalNx powdNx(_DecimalNx x, _DecimalNx y);

_FloatN pownfN(_FloatN x, long long int n);
_FloatNx pownfNx(_FloatNx x, long long int n);
_DecimalN powndN(_DecimalN x, long long int n);
_DecimalNx powndNx(_DecimalNx x, long long int n);

_FloatN powrfN(_FloatN x, _FloatN y);
_FloatNx powrfNx(_FloatNx x, _FloatNx y);
_DecimalN powrdN(_DecimalN x, _DecimalN y);
_DecimalNx powrdNx(_DecimalNx x, _DecimalNx y);

_FloatN rootnfN(_FloatN x, long long int n);
_FloatNx rootnfNx(_FloatNx x, long long int n);
_DecimalN rootndN(_DecimalN x, long long int n);
_DecimalNx rootndNx(_DecimalNx x, long long int n);

_FloatN rsqrtfN(_FloatN x);
_FloatNx rsqrtfNx(_FloatNx x);
_DecimalN rsqrtndN(_DecimalN x);

```

```

_DecimalNx rsqrtfN(_DecimalNx x);

_FloatNx sqrtfN(_FloatNx x);
_FloatNx sqrtfNx(_FloatNx x);
_DecimalNx sqrtfN(_DecimalNx x);
_DecimalNx sqrtfNx(_DecimalNx x);

```

7.12.9 Error and gamma functions

```

_FloatNx erffN(_FloatNx x);
_FloatNx erffNx(_FloatNx x);
_DecimalNx erfdN(_DecimalNx x);
_DecimalNx erfdNx(_DecimalNx x);

_FloatNx erfcfN(_FloatNx x);
_FloatNx erfcfNx(_FloatNx x);
_DecimalNx erfcdN(_DecimalNx x);
_DecimalNx erfcdNx(_DecimalNx x);

_FloatNx lgammafN(_FloatNx x);
_FloatNx lgammafNx(_FloatNx x);
_DecimalNx lgammadN(_DecimalNx x);
_DecimalNx lgammadNx(_DecimalNx x);

_FloatNx tgammafN(_FloatNx x);
_FloatNx tgammafNx(_FloatNx x);
_DecimalNx tgammapN(_DecimalNx x);
_DecimalNx tgammapNx(_DecimalNx x);

```

7.12.10 Nearest integer functions

```

_FloatNx ceilfN(_FloatNx x);
_FloatNx ceilfNx(_FloatNx x);
_DecimalNx ceildN(_DecimalNx x);
_DecimalNx ceildNx(_DecimalNx x);

_FloatNx floorfN(_FloatNx x);
_FloatNx floorfNx(_FloatNx x);
_DecimalNx floordN(_DecimalNx x);
_DecimalNx floordNx(_DecimalNx x);

_FloatNx nearbyintfN(_FloatNx x);
_FloatNx nearbyintfNx(_FloatNx x);
_DecimalNx nearbyintdN(_DecimalNx x);
_DecimalNx nearbyintdNx(_DecimalNx x);

_FloatNx rintfN(_FloatNx x);
_FloatNx rintfNx(_FloatNx x);
_DecimalNx rintdN(_DecimalNx x);
_DecimalNx rintdNx(_DecimalNx x);

long int lrintfN(_FloatNx x);
long int lrintfNx(_FloatNx x);
long int lrintdN(_DecimalNx x);
long int lrintdNx(_DecimalNx x);

long long int llrintfN(_FloatNx x);
long long int llrintfNx(_FloatNx x);
long long int llrintdN(_DecimalNx x);
long long int llrintdNx(_DecimalNx x);

```



```

_FloatN roundfN(_FloatN x);
_FloatNx roundfNx(_FloatNx x);
_DecimalN rounddN(_DecimalN x);
_DecimalNx rounddNx(_DecimalNx x);

long int lroundfN(_FloatN x);
long int lroundfNx(_FloatNx x);
long int lrounddN(_DecimalN x);
long int lrounddNx(_DecimalNx x);

long long int llroundfN(_FloatN x);
long long int llroundfNx(_FloatNx x);
long long int llrounddN(_DecimalN x);
long long int llrounddNx(_DecimalNx x);

_FloatN roundevenfN(_FloatN x);
_FloatNx roundevenfNx(_FloatNx x);
_DecimalN roundevendN(_DecimalN x);
_DecimalNx roundevendNx(_DecimalNx x);

_FloatN truncfN(_FloatN x);
_FloatNx truncfNx(_FloatNx x);
_DecimalN truncdN(_DecimalN x);
_DecimalNx truncdNx(_DecimalNx x);

_FloatN fromfpfN(_FloatN x, int rnd, unsigned int width);
_FloatNx fromfpfNx(_FloatNx x, int rnd, unsigned int width);
_DecimalN fromfpdN(_DecimalN x, int rnd, unsigned int width);
_DecimalNx fromfpdNx(_DecimalNx x, int rnd, unsigned int width);
_FloatN ufromfpfN(_FloatN x, int rnd, unsigned int width);
_FloatNx ufromfpfNx(_FloatNx x, int rnd, unsigned int width);
_DecimalN ufromfpdN(_DecimalN x, int rnd, unsigned int width);
_DecimalNx ufromfpdNx(_DecimalNx x, int rnd, unsigned int width);

_FloatN fromfpxfN(_FloatN x, int rnd, unsigned int width);
_FloatNx fromfpxfNx(_FloatNx x, int rnd, unsigned int width);
_DecimalN fromfpxdN(_DecimalN x, int rnd, unsigned int width);
_DecimalNx fromfpxdNx(_DecimalNx x, int rnd, unsigned int width);
_FloatN ufromfpxfN(_FloatN x, int rnd, unsigned int width);
_FloatNx ufromfpxfNx(_FloatNx x, int rnd, unsigned int width);
_DecimalN ufromfpxdN(_DecimalN x, int rnd, unsigned int width);
_DecimalNx ufromfpxdNx(_DecimalNx x, int rnd, unsigned int width);

```

7.12.11.2 Remainder functions

```

_FloatN fmodfN(_FloatN x, _FloatN y);
_FloatNx fmodfNx(_FloatNx x, _FloatNx y);
_DecimalN fmoddN(_DecimalN x, _DecimalN y);
_DecimalNx fmoddNx(_DecimalNx x, _DecimalNx y);

_FloatN remainderfN(_FloatN x, _FloatN y);
_FloatNx remainderfNx(_FloatNx x, _FloatNx y);
_DecimalN remainderdN(_DecimalN x, _DecimalN y);
_DecimalNx remainderdNx(_DecimalNx x, _DecimalNx y);

_FloatN remquofN(_FloatN x, _FloatN y, int *quo);
_FloatNx remquofNx(_FloatNx x, _FloatNx y, int *quo);

```

7.12.12 Manipulation functions

```

_FloatN copysignfN(_FloatN x, _FloatN y);

```

```

_FloatNx copysignfNx(_FloatNx x, _FloatNx y);
_DecimalN copysigndN(_DecimalN x, _DecimalN y);
_DecimalNx copysigndNx(_DecimalNx x, _DecimalNx y);

_FloatN nanfN(const char *tagp);
_FloatNx nanfNx(const char *tagp);
_DecimalN nandN(const char *tagp);
_DecimalNx nandNx(const char *tagp);

_FloatN nextafterfN(_FloatN x, _FloatN y);
_FloatNx nextafterfNx(_FloatNx x, _FloatNx y);
_DecimalN nextafterdN(_DecimalN x, _DecimalN y);
_DecimalNx nextafterdNx(_DecimalNx x, _DecimalNx y);

_FloatN nextupfN(_FloatN x);
_FloatNx nextupfNx(_FloatNx x);
_DecimalN nextupdN(_DecimalN x);
_DecimalNx nextupdNx(_DecimalNx x);

_FloatN nextdownfN(_FloatN x);
_FloatNx nextdownfNx(_FloatNx x);
_DecimalN nextdowndN(_DecimalN x);
_DecimalNx nextdowndNx(_DecimalNx x);

int canonicalizefN(_FloatN *cx, const _FloatN *x);
int canonicalizefNx(_FloatNx *cx, const _FloatNx *x);
int canonicalizedN(_DecimalN *cx, const _DecimalN *x);
int canonicalizedNx(_DecimalNx *cx, const _DecimalNx *x);

```

7.12.13 Maximum, minimum, and positive difference functions

```

_FloatN fdimfN(_FloatN x, _FloatN y);
_FloatNx fdimfNx(_FloatNx x, _FloatNx y);
_DecimalN fdimdN(_DecimalN x, _DecimalN y);
_DecimalNx fdimdNx(_DecimalNx x, _DecimalNx y);

_FloatN fmaximumfN(_FloatN x, _FloatN y);
_FloatNx fmaximumfNx(_FloatNx x, _FloatNx y);
_DecimalN fmaximumdN(_DecimalN x, _DecimalN y);
_DecimalNx fmaximumdNx(_DecimalNx x, _DecimalNx y);

_FloatN fminimumfN(_FloatN x, _FloatN y);
_FloatNx fminimumfNx(_FloatNx x, _FloatNx y);
_DecimalN fminimumdN(_DecimalN x, _DecimalN y);
_DecimalNx fminimumdNx(_DecimalNx x, _DecimalNx y);

_FloatN fmaximum_magfN(_FloatN x, _FloatN y);
_FloatNx fmaximum_magfNx(_FloatNx x, _FloatNx y);
_DecimalN fmaximum_magdN(_DecimalN x, _DecimalN y);
_DecimalNx fmaximum_magdNx(_DecimalNx x, _DecimalNx y);

_FloatN fminimum_magfN(_FloatN x, _FloatN y);
_FloatNx fminimum_magfNx(_FloatNx x, _FloatNx y);
_DecimalN fminimum_magdN(_DecimalN x, _DecimalN y);
_DecimalNx fminimum_magdNx(_DecimalNx x, _DecimalNx y);

_FloatN fmaximum_numfN(_FloatN x, _FloatN y);
_FloatNx fmaximum_numfNx(_FloatNx x, _FloatNx y);
_DecimalN fmaximum_numdN(_DecimalN x, _DecimalN y);
_DecimalNx fmaximum_numdNx(_DecimalNx x, _DecimalNx y);

```

```

_FloatN fminimum_numfN(_FloatN x, _FloatN y);
_FloatNx fminimum_numfNx(_FloatNx x, _FloatNx y);
_DecimalN fminimum_numdN(_DecimalN x, _DecimalN y);
_DecimalNx fminimum_numdNx(_DecimalNx x, _DecimalNx y);

_FloatN fmaximum_mag_numfN(_FloatN x, _FloatN y);
_FloatNx fmaximum_mag_numfNx(_FloatNx x, _FloatNx y);
_DecimalN fmaximum_mag_numdN(_DecimalN x, _DecimalN y);
_DecimalNx fmaximum_mag_numdNx(_DecimalNx x, _DecimalNx y);

_FloatN fminimum_mag_numfN(_FloatN x, _FloatN y);
_FloatNx fminimum_mag_numfNx(_FloatNx x, _FloatNx y);
_DecimalN fminimum_mag_numdN(_DecimalN x, _DecimalN y);
_DecimalNx fminimum_mag_numdNx(_DecimalNx x, _DecimalNx y);

```

7.12.14.1 Fused multiply-add

```

_FloatN fmafN(_FloatN x, _FloatN y, _FloatN z);
_FloatNx fmafNx(_FloatNx x, _FloatNx y, _FloatNx z);
_DecimalN fmadN(_DecimalN x, _DecimalN y, _DecimalN z);
_DecimalNx fmadNx(_DecimalNx x, _DecimalNx y, _DecimalNx z);

```

7.12.15 Functions that round result to narrower type

```

_FloatM fMaddfN(_FloatN x, _FloatN y); //  $M < N$ 
_FloatM fMaddfNx(_FloatNx x, _FloatNx y); //  $M \leq N$ 
_FloatMx fMxaddfN(_FloatN x, _FloatN y); //  $M < N$ 
_FloatMx fMxaddfNx(_FloatNx x, _FloatNx y); //  $M < N$ 
_DecimalM dMadddN(_DecimalN x, _DecimalN y); //  $M < N$ 
_DecimalM dMadddNx(_DecimalNx x, _DecimalNx y); //  $M \leq N$ 
_DecimalMx dMxadddN(_DecimalN x, _DecimalN y); //  $M < N$ 
_DecimalMx dMxadddNx(_DecimalNx x, _DecimalNx y); //  $M < N$ 

_FloatM fMsubfN(_FloatN x, _FloatN y); //  $M < N$ 
_FloatM fMsubfNx(_FloatNx x, _FloatNx y); //  $M \leq N$ 
_FloatMx fMxsubfN(_FloatN x, _FloatN y); //  $M < N$ 
_FloatMx fMxsubfNx(_FloatNx x, _FloatNx y); //  $M < N$ 
_DecimalM dMsubdN(_DecimalN x, _DecimalN y); //  $M < N$ 
_DecimalM dMsubdNx(_DecimalNx x, _DecimalNx y); //  $M \leq N$ 
_DecimalMx dMxsubdN(_DecimalN x, _DecimalN y); //  $M < N$ 
_DecimalMx dMxsubdNx(_DecimalNx x, _DecimalNx y); //  $M < N$ 

_FloatM fMmulfN(_FloatN x, _FloatN y); //  $M < N$ 
_FloatM fMmulfNx(_FloatNx x, _FloatNx y); //  $M \leq N$ 
_FloatMx fMxmulfN(_FloatN x, _FloatN y); //  $M < N$ 
_FloatMx fMxmulfNx(_FloatNx x, _FloatNx y); //  $M < N$ 
_DecimalM dMmuldN(_DecimalN x, _DecimalN y); //  $M < N$ 
_DecimalM dMmuldNx(_DecimalNx x, _DecimalNx y); //  $M \leq N$ 
_DecimalMx dMxmuldN(_DecimalN x, _DecimalN y); //  $M < N$ 
_DecimalMx dMxmuldNx(_DecimalNx x, _DecimalNx y); //  $M < N$ 

_FloatM fMdivfN(_FloatN x, _FloatN y); //  $M < N$ 
_FloatM fMdivfNx(_FloatNx x, _FloatNx y); //  $M \leq N$ 
_FloatMx fMxdivfN(_FloatN x, _FloatN y); //  $M < N$ 
_FloatMx fMxdivfNx(_FloatNx x, _FloatNx y); //  $M < N$ 
_DecimalM dMdivdN(_DecimalN x, _DecimalN y); //  $M < N$ 
_DecimalM dMdivdNx(_DecimalNx x, _DecimalNx y); //  $M \leq N$ 
_DecimalMx dMxdivdN(_DecimalN x, _DecimalN y); //  $M < N$ 
_DecimalMx dMxdivdNx(_DecimalNx x, _DecimalNx y); //  $M < N$ 

_FloatM fMfafN(_FloatN x, _FloatN y, _FloatN z); //  $M < N$ 

```

```

_FloatM fMfmafNx(_FloatNx x, _FloatNx y, _FloatNx z); //  $M \leq N$ 
_FloatMx fMxfmafN(_FloatN x, _FloatN y, _FloatN z); //  $M < N$ 
_FloatMx fMxfmafNx(_FloatNx x, _FloatNx y, _FloatNx z); //  $M < N$ 
_DecimalM dMfmadN(_DecimalN x, _DecimalN y, _DecimalN z); //  $M < N$ 
_DecimalM dMfmadNx(_DecimalNx x, _DecimalNx y, _DecimalNx z); //  $M \leq N$ 
_DecimalMx dMxfmadN(_DecimalN x, _DecimalN y, _DecimalN z); //  $M < N$ 
_DecimalMx dMxfmadNx(_DecimalNx x, _DecimalNx y, _DecimalNx z); //  $M < N$ 

_FloatM fMsqrtfN(_FloatN x); //  $M < N$ 
_FloatM fMsqrtfNx(_FloatNx x); //  $M \leq N$ 
_FloatMx fMxsqrtfN(_FloatN x); //  $M < N$ 
_FloatMx fMxsqrtfNx(_FloatNx x); //  $M < N$ 
_DecimalM dMsqrtN(_DecimalN x); //  $M < N$ 
_DecimalM dMsqrtNx(_DecimalNx x); //  $M \leq N$ 
_DecimalMx dMxsqrtN(_DecimalN x); //  $M < N$ 
_DecimalMx dMxsqrtNx(_DecimalNx x); //  $M < N$ 

```

7.12.16 Quantum and quantum exponent functions

```

_DecimalN quantizedN(_DecimalN x, _DecimalN y);
_DecimalNx quantizedNx(_DecimalNx x, _DecimalNx y);

bool samequantumdN(_DecimalN x, _DecimalN y);
bool samequantumdNx(_DecimalNx x, _DecimalNx y);

_DecimalN quantumdN(_DecimalN x);
_DecimalNx quantumdNx(_DecimalNx x);

long long int llquantexpdN(_DecimalN x);
long long int llquantexpdNx(_DecimalNx x);

```

7.12.17 Decimal re-encoding functions

```

void encodedecdN(unsigned char * restrict encptr,
                 const _DecimalN * restrict xptr);
void decodedecdN(_DecimalN * restrict xptr,
                 const unsigned char * restrict encptr);
void encodebindN(unsigned char * restrict encptr,
                 const _DecimalN * restrict xptr);
void decodebindN(_DecimalN * restrict xptr,
                 const unsigned char * restrict encptr);

```

F.10.13 Total order functions

```

int totalorderfN(const _FloatN *x, const _FloatN *y);
int totalorderfNx(const _FloatNx *x, const _FloatNx *y);
int totalorderdN(const _DecimalN *x, const _DecimalN *y);
int totalorderdNx(const _DecimalNx *x, const _DecimalNx *y);

int totalordermagfN(const _FloatN *x, const _FloatN *y);
int totalordermagfNx(const _FloatNx *x, const _FloatNx *y);
int totalordermagdN(const _DecimalN *x, const _DecimalN *y);
int totalordermagdNx(const _DecimalNx *x, const _DecimalNx *y);

```

F.10.14 Payload functions

```

_FloatN getpayloadfN(const _FloatN *x);
_FloatNx getpayloadfNx(const _FloatNx *x);
_DecimalN getpayloaddN(const _DecimalN *x);
_DecimalNx getpayloaddNx(const _DecimalNx *x);

```

```

int setpayloadfN(_FloatN *res, _FloatN pl);
int setpayloadfNx(_FloatNx *res, _FloatNx pl);
int setpayloaddN(_DecimalN *res, _DecimalN pl);
int setpayloaddNx(_DecimalNx *res, _DecimalNx pl);

int setpayloadsigfN(_FloatN *res, _FloatN pl);
int setpayloadsigfNx(_FloatNx *res, _FloatNx pl);
int setpayloadsigdN(_DecimalN *res, _DecimalN pl);
int setpayloadsigdNx(_DecimalNx *res, _DecimalNx pl);

```

The specification of the **frexp** functions (7.12.7.7) applies to the functions for binary floating types like those for standard floating types: the exponent is an integral power of 2 and, when applicable, **value** equals $x \times 2^{\text{exp}}$.

The specification of the **ldexp** functions (7.12.7.9) applies to the functions for binary floating types like those for standard floating types: they return $x2^{\text{exp}}$.

The specification of the **logb** functions (7.12.7.17) applies to binary floating types, with $b = 2$.

The specification of the **scalbn** and **scalbln** functions (7.12.7.19) applies to binary floating types, with $b = 2$.

H.11.4 Encoding conversion functions

H.11.4.1 General

This subclause introduces `<math.h>` functions that, together with the numerical conversion functions for encodings in H.12, support the non-arithmetic interchange formats specified by ISO/IEC 60559. Support for these formats is an optional feature of this annex. Implementations that do not support non-arithmetic interchange formats are not required to declare the functions in this subclause.

Non-arithmetic interchange formats are not associated with floating types. Arrays of element type **unsigned char** are used as parameters for conversion functions, to represent encodings in interchange formats that may be non-arithmetic formats.

H.11.4.2 Encode and decode functions

H.11.4.2.1 General

This subclause specifies functions to map representations in binary floating types to and from encodings in **unsigned char** arrays.

H.11.4.2.2 The **encodefN** functions

Synopsis

```

#define __STDC_WANT_IEC_60559_TYPES_EXT__
#include <math.h>

void encodefN(unsigned char encptr[restrict static N/8],
               const _FloatN * restrict xptr);

```

Description

The **encodefN** functions convert ***xptr** into an ISO/IEC 60559 binary N encoding and store the resulting encoding as an $N/8$ element array, with 8 bits per array element, in the object pointed to by **encptr**. The order of bytes in the array follows the endianness specified with **__STDC_ENDIAN_NATIVE__** (7.18.2). These functions preserve the value of ***xptr** and raise no floating-point exceptions. If ***xptr** is non-canonical, these functions can produce a canonical encoding.

Returns

The **encodefN** functions return no value.

H.11.4.2.3 The **decodefN** functions

Synopsis

```
#define __STDC_WANT_IEC_60559_TYPES_EXT__
#include <math.h>

void decodefN(_FloatN * restrict xptr,
              const unsigned char encptr[restrict static N/8]);
```

Description

The **decodefN** functions interpret the $N/8$ element array pointed to by **encptr** as an ISO/IEC 60559 binary N encoding, with 8 bits per array element. The order of bytes in the array follows the endianness specified with **__STDC_ENDIAN_NATIVE__** (7.18.2). These functions convert the given encoding into a representation in the type **_FloatN**, and store the result in the object pointed to by **xptr**. These functions preserve the encoded value and raise no floating-point exceptions. If the encoding is non-canonical, these functions can produce a canonical representation.

Returns

The **decodefN** functions return no value.

See Example in H.11.4.3.2.

H.11.4.3 Encoding-to-encoding conversion functions

H.11.4.3.1 General

An implementation shall declare an **fMencfN** function for each M and N equal to the width of a supported ISO/IEC 60559 arithmetic or non-arithmetic binary interchange format, $M \neq N$. An implementation shall provide both **dMencdecN** and **dMencbindN** functions for each M and N equal to the width of a supported ISO/IEC 60559 arithmetic or non-arithmetic decimal interchange format, $M \neq N$.

H.11.4.3.2 The fMencfN functions

Synopsis

```
#define __STDC_WANT_IEC_60559_TYPES_EXT__
#include <math.h>

void fMencfN(unsigned char encMptr[restrict static M/8],
              const unsigned char encNptr[restrict static N/8]);
```

Description

The **fMencfN** functions convert between ISO/IEC 60559 binary interchange formats. These functions interpret the $N/8$ element array pointed to by **encNptr** as an encoding of width N bits. They convert the encoding to an encoding of width M bits and store the resulting encoding as an $M/8$ element array in the object pointed to by **encMptr**. The conversion rounds and raises floating-point exceptions as specified in ISO/IEC 60559. The order of bytes in the arrays follows the endianness specified with **__STDC_ENDIAN_NATIVE__** (7.18.2).

Returns

These functions return no value.

EXAMPLE If the ISO/IEC 60559 binary16 format is supported as a non-arithmetic format, data in binary16 format can be converted to type **float** as follows:

```
#define __STDC_WANT_IEC_60559_TYPES_EXT__
#include <math.h>
unsigned char b16[2]; // for input binary16 datum
float f; // for result
unsigned char b32[4];
_Float32 f32;
```



```
// store input binary16 datum in array b16
...
f32encf16(b32, b16);
decodef32(&f32, b32);
f = f32;
...
```

H.11.4.3.3 The dMencdecdN and dMencbindN functions

Synopsis

```
#define __STDC_WANT_IEC_60559_TYPES_EXT__
#include <math.h>

void dMencdecdN(unsigned char encMptr[restrict static M/8],
               const unsigned char encNptr[restrict static N/8]);
void dMencbindN(unsigned char encMptr[restrict static M/8],
               const unsigned char encNptr[restrict static N/8]);
```

Description

The **dMencdecdN** and **dMencbindN** functions convert between ISO/IEC 60559 decimal interchange formats that use the same encoding scheme. The **dMencdecdN** functions convert between formats using the encoding scheme based on decimal encoding of the significand. The **dMencbindN** functions convert between formats using the encoding scheme based on binary encoding of the significand. These functions interpret the *N*/8 element array pointed to by **encNptr** as an encoding of width *N* bits. They convert the encoding to an encoding of width *M* bits and store the resulting encoding as an *M*/8 element array in the object pointed to by **encMptr**. The conversion rounds and raises floating-point exceptions as specified in ISO/IEC 60559. The order of bytes in the arrays follows the endianness specified with **__STDC_ENDIAN_NATIVE__** (7.18.2).

Returns

These functions return no value.

H.12 Numeric conversion functions <stdlib.h>

H.12.1 General

This clause expands the specification of numeric conversion functions in <stdlib.h> (7.24.2) to also include conversions of strings from and to interchange and extended floating types. The conversions from floating are provided by functions analogous to the **strfromd** function. The conversions to floating are provided by functions analogous to the **strtod** function.

This clause also specifies functions to convert strings from and to ISO/IEC 60559 interchange format encodings.

For each interchange or extended floating type that the implementation provides, <stdlib.h> shall declare the associated functions specified in the following subclauses in H.12.2 and H.12.3 (see H.8). Conversely, for each such type that the implementation does not provide, <stdlib.h> shall not declare the associated functions.

For each ISO/IEC 60559 arithmetic or non-arithmetic format that the implementation supports, <stdlib.h> shall declare the associated functions specified the following subclauses in H.12.4 and H.12.5 (see H.8). Conversely, for each such format that the implementation does not provide, <stdlib.h> shall not declare the associated functions.

H.12.2 String from floating

This subclause expands 7.24.2.4 and 7.24.2.5 to also include functions for the interchange and extended floating types. It adds to the synopsis in 7.24.2.4 the prototypes

```
int strfromfN(char * restrict s, size_t n,
             const char * restrict format, _FloatN fp);
```



```
int strfromfNx(char * restrict s, size_t n,
               const char * restrict format, _FloatNx fp);
```

It encompasses the prototypes in 7.24.2.5 by replacing them with

```
int strfromdN(char * restrict s, size_t n,
               const char * restrict format, _DecimalW fp);
int strfromdNx(char * restrict s, size_t n,
               const char * restrict format, _DecimalNx fp);
```

The descriptions and returns for the added functions are analogous to the ones in 7.24.2.4 and 7.24.2.5.

H.12.3 String to floating

This subclause expands 7.24.2.6, 7.31.4.2.2, 7.24.2.7, and 7.31.4.2.3 to also include functions for the interchange and extended floating types.

It adds to the synopsis in 7.24.2.6 the prototypes

```
_FloatN strtodN(const char * restrict nptr,
                 char ** restrict endptr);
_FloatNx strtodNx(const char * restrict nptr,
                  char ** restrict endptr);
```

It adds to the synopsis in 7.31.4.2.2 the prototypes

```
_FloatW wcstodN(const wchar_t * restrict nptr,
                 wchar_t ** restrict endptr);
_FloatNx wcstodNx(const wchar_t * restrict nptr,
                  wchar_t ** restrict endptr);
```

It encompasses the prototypes in 7.24.2.7 by replacing them with

```
_DecimalW strtodN(const char * restrict nptr,
                  char ** restrict endptr);
_DecimalNx strtodNx(const char * restrict nptr,
                    char ** restrict endptr);
```

It encompasses the prototypes in 7.31.4.2.3 by replacing them with

```
_DecimalW wcstodN(const wchar_t * restrict nptr,
                   wchar_t ** restrict endptr);
_DecimalNx wcstodNx(const wchar_t * restrict nptr,
                    wchar_t ** restrict endptr);
```

The descriptions and returns for the added functions are analogous to the ones in 7.24.2.6, 7.31.4.2.2, 7.24.2.7, and 7.31.4.2.3.

EXAMPLE If the ISO/IEC 60559 binary128 format is supported as a non-arithmetic format, data in binary128 format can be converted to type `_Decimal128` as follows:

```
#define __STDC_WANT_IEC_60559_TYPES_EXT__
#include <stdlib.h>
#define MAXSIZE 41 // > intermediate hex string length
                  // for the "C" locale
unsigned char b128[16]; // for input binary128 datum
_Decimal128 d128; // for result
char s[MAXSIZE];
// store input binary128 datum in array b128
...
```

```
strfromencf128(s, MAXSIZE, "%a", b128);
d128 = strtod128(s, nullptr);
...
```

Use of `"%a"` for formatting assures an exact conversion of the value in binary format to character sequence. The value of that character sequence will be correctly rounded to `_Decimal128`, as specified previously in this subclause. Assuming a single-byte decimal-point character as in the `"C"` locale, the array `s` for the output of `strfromencf128` need have no greater size than 41, which is the maximum length of strings of the form

`[−]0xh.h...hp ± d`

where there are up to 29 hexadecimal digits `h` and `d` has 5 digits plus 1 for the null character.

H.12.4 String from encoding

H.12.4.1 General

An implementation shall declare the `strfromencfN` function for each `N` equal to the width of a supported ISO/IEC 60559 arithmetic or non-arithmetic binary interchange format. An implementation shall declare both the `strfromencdecN` and `strfromencbindN` functions for each `N` equal to the width of a supported ISO/IEC 60559 arithmetic or non-arithmetic decimal interchange format.

H.12.4.2 The `strfromencfN` functions

Synopsis

```
#define __STDC_WANT_IEC_60559_TYPES_EXT__
#include <stdlib.h>

int strfromencfN(char * restrict s, size_t n, const char * restrict format,
    const unsigned char encptr[restrict static N/8]);
```

Description

The `strfromencfN` functions are similar to the `strfromfN` functions, except the input is the value of the `N/8` element array pointed to by `encptr`, interpreted as an ISO/IEC 60559 binary `N` encoding. The order of bytes in the array follows the endianness specified with `__STDC_ENDIAN_NATIVE__` (7.18.2).

Returns

The `strfromencfN` functions return the same values as corresponding `strfromfN` functions.

H.12.4.3 The `strfromencdecN` and `strfromencbindN` functions

Synopsis

```
#define __STDC_WANT_IEC_60559_TYPES_EXT__
#include <stdlib.h>

int strfromencdecN(char * restrict s, size_t n, const char * restrict format,
    const unsigned char encptr[restrict static N/8]);
int strfromencbindN(char * restrict s, size_t n, const char * restrict format,
    const unsigned char encptr[restrict static N/8]);
```

Description

The `strfromencdecN` functions are similar to the `strfromdN` functions except the input is the value of the `N/8` element array pointed to by `encptr`, interpreted as an ISO/IEC 60559 decimal `N` encoding in the coding scheme based on decimal encoding of the significand. The `strfromencbindN` functions are similar to the `strfromdN` functions except the input is the value of the `N/8` element array pointed to by `encptr`, interpreted as an ISO/IEC 60559 decimal `N` encoding in the coding scheme based on binary encoding of the significand. The order of bytes in the array follows the endianness specified with `__STDC_ENDIAN_NATIVE__` (7.18.2).

Returns

The **strfromencdecN** and **strfromencbindN** functions return the same values as corresponding **strfromdN** functions.

H.12.5 String to encoding**H.12.5.1 General**

An implementation shall declare the **strtoencfN** and **wcstoencfN** functions for each *N* equal to the width of a supported ISO/IEC 60559 arithmetic or non-arithmetic binary interchange format. An implementation shall declare the **strtoencdecN**, **strtoencbindN**, **wcstoencdecN**, and **wcstoencbindN** functions for each *N* equal to the width of a supported ISO/IEC 60559 arithmetic or non-arithmetic decimal interchange format.

H.12.5.2 The strtoencfN functions**Synopsis**

```
#define __STDC_WANT_IEC_60559_TYPES_EXT__
#include <stdlib.h>

void strtoencfN(unsigned char encptr[restrict static N/8],
               const char * restrict nptr, char ** restrict endptr);
```

Description

The **strtoencfN** functions are similar to the **strtofN** functions, except they store an ISO/IEC 60559 encoding of the result as an *N/8* element array in the object pointed to by **encptr**. The order of bytes in the array follows the endianness specified with **__STDC_ENDIAN_NATIVE__** (7.18.2).

Returns

These functions return no value.

H.12.5.3 The wcstoencfN functions**Synopsis**

```
#define __STDC_WANT_IEC_60559_TYPES_EXT__
#include <wchar.h>

void wcstoencfN(unsigned char encptr[restrict static N/8],
               const wchar_t * restrict nptr, wchar_t ** restrict endptr);
```

Description

The **wcstoencfN** functions are similar to the **wcstofN** functions, except they store an ISO/IEC 60559 encoding of the result as an *N/8* element array in the object pointed to by **encptr**. The order of bytes in the array follows the endianness specified with **__STDC_ENDIAN_NATIVE__** (7.18.2).

Returns

These functions return no value.

H.12.5.4 The strtoencdecN and strtoencbindN functions**Synopsis**

```
#define __STDC_WANT_IEC_60559_TYPES_EXT__
#include <stdlib.h>

void strtoencdecN(unsigned char encptr[restrict static N/8],
                 const char * restrict nptr, char ** restrict endptr);
void strtoencbindN(unsigned char encptr[restrict static N/8],
                  const char * restrict nptr, char ** restrict endptr);
```

Description

The **strtoencdecN** and **strtoencbindN** functions are similar to the **strtodN** functions, except they store an ISO/IEC 60559 encoding of the result as an $N/8$ element array in the object pointed to by **encptr**. The **strtoencdecN** functions produce an encoding in the encoding scheme based on decimal encoding of the significand. The **strtoencbindN** functions produce an encoding in the encoding scheme based on binary encoding of the significand. The order of bytes in the array follows the endianness specified with **__STDC_ENDIAN_NATIVE__** (7.18.2).

Returns

These functions return no value.

H.12.5.5 The **wcstoencdecN** and **wcstoencbindN** functions

Synopsis

```
#define __STDC_WANT_IEC_60559_TYPES_EXT__
#include <wchar.h>

void wcstoencdecN(unsigned char encptr[restrict static N/8],
                  const wchar_t * restrict nptr, wchar_t ** restrict endptr);
void wcstoencbindN(unsigned char encptr[restrict static N/8],
                   const wchar_t * restrict nptr, wchar_t ** restrict endptr);
```

Description

The **wcstoencdecN** and **wcstoencbindN** functions are similar to the **wcstodN** functions, except they store an ISO/IEC 60559 encoding of the result as an $N/8$ element array in the object pointed to by **encptr**. The **wcstoencdecN** functions produce an encoding in the encoding scheme based on decimal encoding of the significand. The **wcstoencbindN** functions produce an encoding in the encoding scheme based on binary encoding of the significand. The order of bytes in the array follows the endianness specified with **__STDC_ENDIAN_NATIVE__** (7.18.2).

Returns

These functions return no value.

H.13 Type-generic macros **<tgmath.h>**

This clause enhances the specification of type-generic macros in **<tgmath.h>** (7.27) to apply to interchange and extended floating types, as well as standard floating types.

If arguments for generic parameters of a type-generic macro are such that some argument has a corresponding real type that is a standard floating type or a binary floating type and another argument is of decimal floating type, the behavior is undefined.

The treatment of arguments of integer type in 7.27 is expanded to cases where another argument has extended type. Arguments of integer type are regarded as having type:

- **_Decimal64x**, if any argument has a decimal extended type; otherwise
- **_Float32x**, if any argument has a binary extended type; otherwise
- **_Decimal64**, if any argument has decimal type; otherwise
- **double**

Use of the macros **carg**, **cimag**, **conj**, **cproj**, or **creal** with any argument of standard floating type, binary floating type, complex type, or imaginary type invokes a complex function. Use of the macro with an argument of a decimal floating type results in undefined behavior.

The functions that round results to a narrower type have type-generic macros whose names are obtained by omitting any suffix from the function names. Thus, the macros with **f** or **d** prefix are (as in 7.27):

fadd	fmul	ffma
dadd	dmul	dfma
fsub	fdiv	fsqrt
dsub	ddiv	dsqrt

and the macros with **fM**, **fMx**, **dM**, or **dMx** prefix are:

fMadd	fMxmul	dMfma
fMsub	fMxdiv	dMsqrt
fMmul	fMxfma	dMxadd
fMdiv	fMxsqrt	dMxsub
fMfma	dMadd	dMxmul
fMsqrt	dMsub	dMxdiv
fMxadd	dMmul	dMxfma
fMxsub	dMdiv	dMxsqrt

All arguments are generic. If any argument is not real, use of the macro results in undefined behavior. The following specification uses the notation $type1 \subseteq type2$ to mean the values of $type1$ are a subset of (or the same as) the values of $type2$. The generic parameter T for the function invoked by the macro is determined as follows:

- First, obtain a preliminary type P for the generic parameters: if all arguments are of integer type, then P is **double** if the macro prefix is **f**, **d**, **fNx**, or **fNx** and P is **_Decimal64** if the macro prefix is **dN** or **dNx**; otherwise (if some argument is not of integer type), apply the rules (for determining the corresponding real type of the generic parameters) in 7.27 for macros that do not round result to narrower type, using the usual arithmetic conversion rules in H.4.3, to obtain P .
- If there exists a corresponding function whose generic parameters have type P , then T is P .
- Otherwise, T is determined from P and the macro prefix as follows:
 - For prefix **f**: if P is a standard or binary floating type, then T is the first standard floating type of either **double** or **long double**, such that $P \subseteq T$, if such a type T exists. Otherwise (if no such type T exists or P is a decimal floating type), the behavior is undefined.
 - For prefix **d**: if P is a standard or binary floating type, then T is **long double** if $P \subseteq \text{long double}$. Otherwise (if $P \subseteq \text{long double}$ is **false** or P is a decimal floating type), the behavior is undefined.
 - For prefix **fM**: if P is a standard or binary floating type, then T is **_FloatN** for minimum $N > M$ such that $P \subseteq T$, if such a type T is supported; otherwise T is **_FloatNx** for minimum $N \geq M$ such that $P \subseteq T$, if such a type T is supported. Otherwise (if no such **_FloatN** or **_FloatNx** is supported or P is a decimal floating type), the behavior is undefined.
 - For prefix **fMx**: if P is a standard or binary floating type, then T is **_FloatNx** for minimum $N > M$ such that $P \subseteq T$, if such a type T is supported; otherwise T is **_FloatN** for minimum $N > M$ such that $P \subseteq T$, if such a type T is supported. Otherwise (if no such **_FloatNx** or **_FloatN** is supported or P is a decimal floating type), the behavior is undefined.
 - For prefix **dM**: if P is a decimal floating type, then T is **_DecimalN** for minimum $N > M$ such that $P \subseteq T$, if such a type T is supported; otherwise T is **_DecimalNx** for minimum $N \geq M$ such that $P \subseteq T$. Otherwise (P is a standard or binary floating type), the behavior is undefined.
 - For prefix **dMx**: if P is a decimal floating type, then T is **_DecimalNx** for minimum $N > M$ such that $P \subseteq T$, if such a type T is supported; otherwise T is **_DecimalN** for minimum $N > M$ such that $P \subseteq T$, if such a type T is supported. Otherwise (P is a standard or binary floating type), the behavior is undefined.

EXAMPLE With the declarations

```
#define __STDC_WANT_IEC_60559_TYPES_EXT__

#include <tgmath.h>

int n;
double d;
long double ld;
double complex dc;
_Float32x f32x;
_Float64 f64;
_Float64x f64x;
_Float128 f128;
_Float64x complex f64xc;
```

functions invoked by use of type-generic macros are shown in the following Table H.8 and Table H.9. *type1* $\subseteq$ *type2* means the values of *type1* are a subset of (or the same as) the values of *type2*, and *type1* $\subset$ *type2* means the values of *type1* are a strict subset of the values of *type2*:

Table H.8 — Type-generic macro resolution

macro use	invokes
<code>cos(f64xc)</code>	<code>ccosf64x</code>
<code>pow(dc, f128)</code>	<code>cpowf128</code>
<code>pow(f64, d)</code>	<code>powf64</code>
<code>pow(d, f32x)</code>	<code>pow</code> , the function, if <code>_Float32x</code> $\subseteq$ <code>double</code> , else <code>powf32x</code> if <code>double</code> $\subset$ <code>_Float32x</code> , else undefined
<code>pow(f32, n)</code>	<code>pow</code> , the function
<code>pow(f32x, n)</code>	<code>powf32x</code>

Some type-generic macros that round the result to a narrower type function behave as shown in Table H.9.

Table H.9 — Type-generic narrow rounding examples

macro use	invokes
<code>fsub(d, ld)</code>	<code>fsubl</code>
<code>dsub(d, f32)</code>	<code>dsubl</code>
<code>fmul(dc, d)</code>	undefined
<code>ddiv(ld, f128)</code>	<code>ddivl</code> if <code>_Float128</code> $\subseteq$ <code>long double</code> , else undefined
<code>f32add(f64x, f64)</code>	<code>f32addf64x</code>
<code>f32xsqrt(n)</code>	<code>f32xsqrtf64x</code>
<code>f32mul(f128, f32x)</code>	<code>f32mulf128</code> if <code>_Float32x</code> $\subseteq$ <code>_Float128</code> , else <code>f32mulf32x</code> if <code>_Float128</code> $\subset$ <code>_Float32x</code> , else undefined
<code>f32fma(f32x, n, f32x)</code>	<code>f32fmaf32x</code>
<code>f32add(f32, f32)</code>	<code>f32addf64</code>
<code>f32xsqrt(f32)</code>	<code>f32xsqrtf64x</code> , as prior declarations show <code>_Float64x</code> is supported
<code>f64div(f32x, f32x)</code>	<code>f64divf128</code> if <code>_Float32x</code> $\subseteq$ <code>_Float128</code> , else <code>f64divf64x</code>

Annex I

(informative)

Common warnings

I.1 Introduction

An implementation may generate warnings in many situations to help find a source of unintended behavior during the translation or execution of a program. Many such situations are not specified as part of this document.

I.2 Common situations

The following are a few of the common situations where an implementation may generate a warning:

- A new **struct** or **union** type appears in a function prototype (6.2.1, 6.7.3.4).
- A block with initialization of an object that has automatic storage duration is jumped into (6.2.4).
- An implicit narrowing conversion is encountered, such as the assignment of a **long int** or a **double** to an **int**, or a pointer to **void** to a pointer to any type other than a character type (6.3).
- A hexadecimal floating constant cannot be represented exactly in its evaluation format (6.4.5.3).
- An integer character constant includes more than one character or a wide character constant includes more than one multibyte character (6.4.5.5).
- The characters `/*` are found in a comment (6.4.8).
- An “unordered” binary operator (not comma, `&&`, or `| |`) contains a side effect to an lvalue in one operand, and a side effect to, or an access to the value of, the identical lvalue in the other operand (6.5.1).
- An object is defined but not used (6.7).
- A value is given to an object of an enumerated type other than by assignment of an enumeration constant that is a member of that type, or an enumeration object that has the same type, or the value of a function that returns the same enumerated type (6.7.3.3).
- An aggregate has a partly bracketed initialization (6.7.9).
- A statement cannot be reached (6.8).
- A statement with no apparent effect is encountered (6.8).
- A constant expression is used as the controlling expression of a selection statement (6.8.5).
- An incorrectly formed preprocessing group is encountered while skipping a preprocessing group (6.10.2).
- An unrecognized **#pragma** directive is encountered (6.10.8).

Annex J

(informative)

Portability issues

J.1 Unspecified behavior

The following are unspecified:

- (1) The manner and timing of static initialization (5.2.2).
- (2) The termination status returned to the hosted environment if the return type of `main` is not compatible with `int` (5.2.2.3.4).
- (3) The values of objects that are neither lock-free atomic objects nor of type `volatile sig_atomic_t` and the state of the floating-point environment, when the processing of the abstract machine is interrupted by receipt of a signal (5.2.2.4).
- (4) The behavior of the display device if a printing character is written when the active position is at the final position of a line (5.3.3).
- (5) The behavior of the display device if a backspace character is written when the active position is at the initial position of a line (5.3.3).
- (6) The behavior of the display device if a horizontal tab character is written when the active position is at or past the last defined horizontal tabulation position (5.3.3).
- (7) The behavior of the display device if a vertical tab character is written when the active position is at or past the last defined vertical tabulation position (5.3.3).
- (8) How an extended source character that does not correspond to a universal character name counts toward the significant initial characters in an external identifier (5.3.5.2).
- (9) Many aspects of the representations of types (6.2.6).
- (10) The value of padding bytes when storing values in structures or unions (6.2.6.1).
- (11) The values of bytes that correspond to union members other than the one last stored into (6.2.6.1).
- (12) The representation used when storing a value in an object that has more than one object representation for that value (6.2.6.1).
- (13) The values of any padding bits in integer representations (6.2.6.2).
- (14) Whether two string literals result in distinct arrays (6.4.6).
- (15) The order in which subexpressions are evaluated and the order in which side effects take place, except as specified for the function-call `()`, `&&`, `||`, `?:`, and comma operators (6.5.1).
- (16) The order in which the function designator, arguments, and subexpressions within the arguments are evaluated in a function call (6.5.3.3).
- (17) The order of side effects among compound literal initialization list expressions (6.5.3.6).
- (18) The order in which the operands of an assignment operator are evaluated (6.5.17).
- (19) The alignment of the addressable storage unit allocated to hold a bit-field (6.7.3.2).
- (20) Whether a call to an inline function uses the inline definition or the external definition of the function (6.7.5).
- (21) Whether a size expression is evaluated when it is part of the operand of a `sizeof` operator and changing the value of the size expression would not affect the result of the operator (6.7.7.3).

- (22) The order in which any side effects occur among the initialization list expressions in an initializer (6.7.11).
- (23) The layout of storage for function parameters (6.9.2).
- (24) When a fully expanded macro replacement list contains a function-like macro name as its last preprocessing token and the next preprocessing token from the source file is a `(`, and the fully expanded replacement of that macro ends with the name of the first macro and the next preprocessing token from the source file is again a `(`, whether that is considered a nested replacement (6.10.5).
- (25) The order in which `#` and `##` operations are evaluated during macro substitution (6.10.5.3, 6.10.5.4).
- (26) The line number of a preprocessing token, in particular `__LINE__`, that spans multiple physical lines (6.10.6).
- (27) The line number of a preprocessing directive that spans multiple physical lines (6.10.6).
- (28) The line number of a macro invocation that spans multiple physical or logical lines (6.10.6).
- (29) The line number following a directive of the form `#line __LINE__ new-line` (6.10.6).
- (30) The state of the floating-point status flags when execution passes from a part of the program translated with `FENV_ACCESS` “off” to a part translated with `FENV_ACCESS` “on” (7.6.2).
- (31) The order in which `feraiseexcept` raises floating-point exceptions, except as stated in F.8.7 (7.6.5.4).
- (32) Whether `math_errhandling` is a macro or an identifier with external linkage (7.12).
- (33) The results of the `frexp` functions when the specified value is not a floating-point number (7.12.7.7).
- (34) The numeric result of the `ilogb` functions when the correct value is outside the range of the return type (7.12.7.8, F.10.4.8).
- (35) The result of rounding when the value is out of range (7.12.10.5, 7.12.10.7, F.10.7.5).
- (36) The value stored by the `remquo` functions in the object pointed to by `quo` when `y` is zero (7.12.11.3).
- (37) Whether a comparison macro argument that is represented in a format wider than its semantic type is converted to the semantic type (7.12.18).
- (38) Whether `setjmp` is a macro or an identifier with external linkage (7.13).
- (39) Whether `va_copy` and `va_end` are macros or identifiers with external linkage (7.16.2).
- (40) The hexadecimal digit before the decimal point when a non-normalized floating-point number is printed with an `a` or `A` conversion specifier (7.23.6.2, 7.31.2.2).
- (41) The value of the file position indicator after a successful call to the `ungetc` function for a text stream, or the `ungetwc` function for any stream, until all pushed-back characters are read or discarded (7.23.7.10, 7.31.3.10).
- (42) The details of the value stored by the `fgetpos` function (7.23.9.1).
- (43) The details of the value returned by the `ftell` function for a text stream (7.23.9.4).
- (44) Whether the `strtod`, `strtof`, `strtold`, `wctod`, `wctof`, and `wctold` functions convert a minus-signed sequence to a negative number directly or by arithmetically negating the value resulting from converting the corresponding unsigned sequence (7.24.2.6, 7.31.4.2.2).

- (45) The order and contiguity of storage allocated by successive calls to the **calloc**, **malloc**, **realloc**, and **aligned_alloc** functions (7.24.4).
- (46) The amount of storage allocated by a successful call to the **calloc**, **malloc**, **realloc**, or **aligned_alloc** function when 0 bytes was requested (7.24.4).
- (47) Whether a call to the **atexit** function that does not happen before the **exit** function is called will succeed (7.24.5.2).
- (48) Whether a call to the **at_quick_exit** function that does not happen before the **quick_exit** function is called will succeed (7.24.5.3).
- (49) Which of two elements that compare as equal is matched by the **bsearch** function (7.24.6.2).
- (50) The order of two elements that compare as equal in an array sorted by the **qsort** function (7.24.6.3).
- (51) The order in which destructors are invoked by **thrd_exit** (7.28.5.5).
- (52) Whether calling **tss_delete** on a key while another thread is executing destructors affects the number of invocations of the destructors associated with the key on that thread (7.28.6.2).
- (53) The encoding of the calendar time returned by the **time** function (7.29.2.5).
- (54) The characters stored by the **strftime** or **wcsftime** function if any of the time values being converted is outside the normal range (7.29.3.6, 7.31.5.1).
- (55) Whether an encoding error occurs if a **wchar_t** value that does not correspond to a member of the extended character set appears in the format string for a function in 7.31.2 or 7.31.5 and the specified semantics do not require that value to be processed by **wcrtomb** (7.31.1).
- (56) The conversion state after an encoding error occurs (7.31.6.4.3, 7.31.6.4.4, 7.31.6.5.2, 7.31.6.5.3, 7.30.2.2, 7.30.2.3, 7.30.2.4, 7.30.2.5, 7.30.2.6, 7.30.2.7).
- (57) The resulting value when the “invalid” floating-point exception is raised during ISO/IEC 60559 floating to integer conversion (F.4).
- (58) Whether conversion of non-integer ISO/IEC 60559 floating values to integer raises the “inexact” floating-point exception (F.4).
- (59) Whether or when library functions in `<math.h>` raise the “inexact” floating-point exception in an ISO/IEC 60559 conformant implementation (F.10).
- (60) Whether or when library functions in `<math.h>` raise an undeserved “underflow” floating-point exception in an ISO/IEC 60559 conformant implementation (F.10).
- (61) The exponent value stored by **frexp** for a NaN or infinity (F.10.4.7).
- (62) The numeric result returned by the **lrint**, **llrint**, **lround**, and **llround** functions if the rounded value is outside the range of the return type (F.10.7.5, F.10.7.7).
- (63) The sign of one part of the **complex** result of several math functions for certain special cases in ISO/IEC 60559 compatible implementations (G.6.2.1, G.6.3.2, G.6.3.3, G.6.3.4, G.6.3.5, G.6.3.6, G.6.4.1, G.6.5.2).

J.2 Undefined behavior

The behavior is undefined in the following circumstances:

- (1) A “shall” or “shall not” requirement that appears outside of a constraint is violated (Clause 4).
- (2) A nonempty source file does not end in a new-line character which is not immediately preceded by a backslash character or ends in a partial preprocessing token or comment (5.2.1.2).

- (3) Token concatenation produces a character sequence matching the syntax of a universal character name (5.2.1.2).
- (4) A program in a hosted environment does not define a function named **main** using one of the specified forms (5.2.2.3.2).
- (5) The execution of a program contains a data race (5.2.2.5).
- (6) A character not in the basic source character set is encountered in a source file, except in an identifier, a character constant, a string literal, a header name, a comment, or a preprocessing token that is never converted to a token (5.3.1).
- (7) An identifier, comment, string literal, character constant, or header name contains an invalid multibyte character or does not begin and end in the initial shift state (5.3.2).
- (8) The same identifier has both internal and external linkage in the same translation unit (6.2.2).
- (9) An object is referred to outside of its lifetime (6.2.4).
- (10) The value of a pointer to an object whose lifetime has ended is used (6.2.4).
- (11) The value of an object with automatic storage duration is used while the object has an indeterminate representation (6.2.4, 6.7.11, 6.8).
- (12) A non-value representation is read by an lvalue expression that does not have character type (6.2.6.1).
- (13) A non-value representation is produced by a side effect that modifies any part of the object using an lvalue expression that does not have character type (6.2.6.1).
- (14) Two declarations of the same object or function specify types that are not compatible (6.2.7).
- (15) A program requires the formation of a composite type from a variable length array type whose size is specified by an expression that is not evaluated (6.2.7).
- (16) Conversion to or from an integer type produces a value outside the range that can be represented (6.3.2.4).
- (17) Demotion of one real floating type to another produces a value outside the range that can be represented (6.3.2.5).
- (18) An lvalue does not designate an object when evaluated (6.3.3.1).
- (19) A non-array lvalue with an incomplete type is used in a context that requires the value of the designated object (6.3.3.1).
- (20) An lvalue designating an object of automatic storage duration that could have been declared with the **register** storage class is used in a context that requires the value of the designated object, but the object is uninitialized. (6.3.3.1).
- (21) An lvalue having array type is converted to a pointer to the initial element of the array, and the array object has register storage class (6.3.3.1).
- (22) An attempt is made to use the value of a void expression, or an implicit or explicit conversion (except to **void**) is applied to a void expression (6.3.3.2).
- (23) Conversion of a pointer to an integer type produces a value outside the range that can be represented (6.3.3.3).
- (24) Conversion between two pointer types produces a result that is incorrectly aligned (6.3.3.3).
- (25) A pointer is used to call a function whose type is not compatible with the referenced type (6.3.3.3).

- (26) An unmatched ' or " character is encountered on a logical source line during tokenization (6.4).
- (27) A reserved keyword token is used in translation phase 7 or 8 (5.2.1.2) for some purpose other than as a keyword (6.4.2).
- (28) A universal character name in an identifier does not designate a character whose encoding falls into one of the specified ranges (6.4.3.1).
- (29) The initial character of an identifier is a universal character name designating a digit (6.4.3.1).
- (30) Two identifiers differ only in nonsignificant characters (6.4.3.1).
- (31) The identifier `__func__` is explicitly declared (6.4.3.2).
- (32) The program attempts to modify a string literal (6.4.6).
- (33) The characters ' , \ , " , // , or /* occur in the sequence between the < and > delimiters, or the characters ' , \ , // , or /* occur in the sequence between the " delimiters, in a header name preprocessing token (6.4.8).
- (34) A side effect on a scalar object is unsequenced relative to either a different side effect on the same scalar object or a value computation using the value of the same scalar object (6.5.1).
- (35) An exceptional condition occurs during the evaluation of an expression (6.5.1).
- (36) An object has its stored value accessed other than by an lvalue of an allowable type (6.5.1).
- (37) A function is defined with a type that is not compatible with the type (of the expression) pointed to by the expression that denotes the called function (6.5.3.3).
- (38) A member of an atomic structure or union is accessed (6.5.3.4).
- (39) The operand of the unary * operator has an invalid value (6.5.4.3).
- (40) A pointer is converted to other than an integer or pointer type (6.5.5).
- (41) The value of the second operand of the / or % operator is zero (6.5.6).
- (42) If the quotient **a/b** is not representable, the behavior of both **a/b** and **a%b** (6.5.6).
- (43) Addition or subtraction of a pointer into, or just beyond, an array object and an integer type produces a result that does not point into, or just beyond, the same array object (6.5.7).
- (44) Addition or subtraction of a pointer into, or just beyond, an array object and an integer type produces a result that points just beyond the array object and is used as the operand of a unary * operator that is evaluated (6.5.7).
- (45) Pointers that do not point into, or just beyond, the same array object are subtracted (6.5.7).
- (46) An array subscript is out of range, even if an object is apparently accessible with the given subscript (as in the lvalue expression **a[1][7]** given the declaration **int a[4][5]**) (6.5.7).
- (47) The result of subtracting two pointers is not representable in an object of type **ptrdiff_t** (6.5.7).
- (48) An expression is shifted by a negative number or by an amount greater than or equal to the width of the promoted expression (6.5.8).
- (49) An expression having signed promoted type is left-shifted and either the value of the expression is negative or the result of shifting would not be representable in the promoted type (6.5.8).
- (50) Pointers that do not point to the same aggregate or union (nor just beyond the same array object) are compared using relational operators (6.5.9).

- (51) An object is assigned to an inexactly overlapping object or to an exactly overlapping object with incompatible type (6.5.17.2).
- (52) An expression that is required to be an integer constant expression does not have an integer type; has operands that are not integer constants, named constants, compound literal constants, enumeration constants, character constants, predefined constants, **sizeof** expressions whose results are integer constants, **alignof** expressions, or immediately-cast floating constants; or contains casts (outside operands to **sizeof** and **alignof** operators) other than conversions of arithmetic types to integer types (6.6).
- (53) A constant expression in an initializer is not, or does not evaluate to, one of the following: a named constant, a compound literal constant, an arithmetic constant expression, a null pointer constant, an address constant, or an address constant for a complete object type plus or minus an integer constant expression (6.6).
- (54) An arithmetic constant expression does not have arithmetic type; has operands that are not integer constants, floating constants, named and compound literal constants of arithmetic type, character constants, predefined constants, **sizeof** expressions whose results are integer constants, or **alignof** expressions; or contains casts (outside operands to **sizeof** or **alignof** operators) other than conversions of arithmetic types to arithmetic types (6.6).
- (55) The value of an object is accessed by an array-subscript `[]`, member-access `.` or `->`, address `&`, or indirection `*` operator or a pointer cast in creating an address constant (6.6).
- (56) An identifier for an object is declared with no linkage and the type of the object is incomplete after its declarator, or after its init-declarator if it has an initializer (6.7).
- (57) A function is declared at block scope with an explicit storage-class specifier other than **extern** (6.7.2).
- (58) A structure or union is defined without any named members (including those specified indirectly via anonymous structures and unions) (6.7.3.2).
- (59) An attempt is made to access, or generate a pointer to just past, a flexible array member of a structure when the referenced object provides no elements for that array (6.7.3.2).
- (60) When the complete type is needed, an incomplete structure or union type is not completed in the same scope by another declaration of the tag that defines the content (6.7.3.4).
- (61) An attempt is made to modify an object defined with a const-qualified type through use of an lvalue with non-const-qualified type (6.7.4).
- (62) An attempt is made to refer to an object defined with a volatile-qualified type through use of an lvalue with non-volatile-qualified type (6.7.4).
- (63) The specification of a function type includes any type qualifiers (6.7.4).
- (64) Two qualified types that are required to be compatible do not have the identically qualified version of a compatible type (6.7.4).
- (65) An object which has been modified is accessed through a restrict-qualified pointer to a const-qualified type, or through a restrict-qualified pointer and another pointer that are not both based on the same object (6.7.4.2).
- (66) A restrict-qualified pointer is assigned a value based on another restricted pointer whose associated block neither began execution before the block associated with this pointer, nor ended before the assignment (6.7.4.2).
- (67) A function with external linkage is declared with an **inline** function specifier, but is not also defined in the same translation unit (6.7.5).
- (68) A function declared with a **_Noreturn** function specifier returns to its caller (6.7.5).

- (69) The definition of an object has an alignment specifier and another declaration of that object has a different alignment specifier (6.7.6).
- (70) Declarations of an object in different translation units have different alignment specifiers (6.7.6).
- (71) Two pointer types that are required to be compatible are not identically qualified, or are not pointers to compatible types (6.7.7.2).
- (72) The size expression in an array declaration is not a constant expression and evaluates at program execution time to a nonpositive value (6.7.7.3).
- (73) In a context requiring two array types to be compatible, they do not have compatible element types, or their size specifiers evaluate to unequal values (6.7.7.3).
- (74) A declaration of an array parameter includes the keyword **static** within the [and] and the corresponding argument does not provide access to the first element of an array with at least the specified number of elements (6.7.7.4).
- (75) A storage-class specifier or type qualifier modifies the keyword **void** as a function parameter type list (6.7.7.4).
- (76) In a context requiring two function types to be compatible, they do not have compatible return types, or their parameters disagree in use of the ellipsis terminator or the number and type of parameters (after default argument promotion, when there is no parameter type list) (6.7.7.4).
- (77) The value of an unnamed member of a structure or union is used (6.7.11).
- (78) The initializer for a scalar is neither a single expression, nor an empty initializer, nor a single expression enclosed in braces (6.7.11).
- (79) The initializer for a structure or union object is neither an initializer list nor a single expression that has compatible structure or union type (6.7.11).
- (80) The initializer for an aggregate or union, other than an array initialized by a string literal, is not a brace-enclosed list of initializers for its elements or members (6.7.11).
- (81) A function definition that does not have the asserted property is called by a function declaration or a function pointer with a type that has the **unsequenced** or **reproducible** attribute (6.7.13.8).
- (82) An identifier with external linkage is used, but in the program there does not exist exactly one external definition for the identifier, or the identifier is not used and there exist multiple external definitions for the identifier (6.9).
- (83) A function that accepts a variable number of arguments is defined without a parameter type list that ends with the ellipsis notation (6.9.2).
- (84) The **}** that terminates a function is reached, and the value of the function call is used by the caller (6.9.2).
- (85) An identifier for an object with internal linkage and an incomplete type is declared with a tentative definition (6.9.3).
- (86) A non-directive preprocessing directive is executed (6.10).
- (87) The token **defined** is generated during the expansion of a **#if** or **#elif** preprocessing directive, or the use of the **defined** unary operator does not match one of the two specified forms prior to macro replacement (6.10.2).
- (88) The **#include** preprocessing directive that results after expansion does not match one of the two header name forms (6.10.3).

- (89) The **#embed** preprocessing directive that results after expansion does not match any of the name forms (6.10.4).
- (90) The character sequence in an **#include** preprocessing directive does not start with a letter (6.10.3).
- (91) The character sequence in an **#embed** preprocessing directive does not start with a letter (6.10.4).
- (92) There are sequences of preprocessing tokens within the list of macro arguments that would otherwise act as preprocessing directives (6.10.5).
- (93) The result of the preprocessing operator **#** is not a valid character string literal (6.10.5.3).
- (94) The result of the preprocessing operator **##** is not a valid preprocessing token (6.10.5.4).
- (95) The **#line** preprocessing directive that results after expansion does not match one of the two well-defined forms, or its digit sequence specifies zero or a number greater than 2147483647 (6.10.6).
- (96) A non-**STDC** **#pragma** preprocessing directive that is documented as causing translation failure or some other form of undefined behavior is encountered (6.10.8).
- (97) A **#pragma STDC** preprocessing directive does not match one of the well-defined forms (6.10.8).
- (98) The name of a predefined macro, or the identifier **defined**, is the subject of a **#define** or **#undef** preprocessing directive (6.10.10).
- (99) An attempt is made to copy an object to an overlapping object by use of a library function, other than as explicitly allowed (e.g. **memmove**) (Clause 7).
- (100) A file with the same name as one of the standard headers, not provided as part of the implementation, is placed in any of the standard places that are searched for included source files (7.1.2).
- (101) A header is included within an external declaration or definition (7.1.2).
- (102) A function, object, type, or macro that is specified as being declared or defined by some standard header is used before any header that declares or defines it is included (7.1.2).
- (103) A standard header is included while a macro is defined with the same name as a keyword (7.1.2).
- (104) The program attempts to declare a library function itself, rather than via a standard header, but the declaration does not have external linkage (7.1.2).
- (105) The program declares or defines a reserved identifier, other than as allowed by 7.1.4 (7.1.3).
- (106) The program removes the definition of a macro whose name begins with an underscore and either an uppercase letter or another underscore (7.1.3).
- (107) An argument to a library function has an invalid value or a type not expected by a function with a variable number of arguments (7.1.4).
- (108) The pointer passed to a library function array parameter does not have a value such that all address computations and object accesses are valid (7.1.4).
- (109) The macro definition of **assert** is suppressed to access an actual function (7.2).
- (110) The argument to the **assert** macro does not have a scalar type (7.2).
- (111) The **CX_LIMITED_RANGE**, **FENV_ACCESS**, or **FP_CONTRACT** pragma is used in any context other than outside all external declarations or preceding all explicit declarations and statements inside a compound statement (7.3.4, 7.6.2, 7.12.3).
- (112) The value of an argument to a character handling function is neither equal to the value of **E0F** nor representable as an **unsigned char** (7.4).

- (113) A macro definition of **errno** is suppressed to access an actual object, or the program defines an identifier with the name **errno** (7.5).
- (114) Part of the program tests floating-point status flags, sets floating-point control modes, or runs under non-default mode settings, but was translated with the state for the **FENV_ACCESS** pragma “off” (7.6.2).
- (115) The exception-mask argument for one of the functions that provide access to the floating-point status flags has a nonzero value not obtained by bitwise OR of the floating-point exception macros (7.6.5).
- (116) The **fesetexceptflag** function is used to set floating-point status flags that were not specified in the call to the **fegetexceptflag** function that provided the value of the corresponding **fexcept_t** object (7.6.5.6).
- (117) The argument to **fesetenv** or **feupdateenv** is neither an object set by a call to **fegetenv** or **fehldexcept**, nor is it an environment macro (7.6.7.4, 7.6.7.5).
- (118) The value of the result of an integer arithmetic or conversion function cannot be represented (7.8.3.1, 7.8.3.2, 7.8.3.3, 7.8.3.4, 7.24.7.1, 7.24.7.2, 7.24.2).
- (119) The program modifies the string pointed to by the value returned by the **setlocale** function (7.11.2).
- (120) A pointer returned by the **setlocale** function is used after a subsequent call to the function, or after the calling thread has exited (7.11.2).
- (121) The program modifies the structure pointed to by the value returned by the **localeconv** function (7.11.3.1).
- (122) A macro definition of **math_errhandling** is suppressed or the program defines an identifier with the name **math_errhandling** (7.12).
- (123) An argument to a floating-point classification or comparison macro is not of real floating type (7.12.4, 7.12.18).
- (124) A macro definition of **setjmp** is suppressed to access an actual function, or the program defines an external identifier with the name **setjmp** (7.13).
- (125) An invocation of the **setjmp** macro occurs other than in an allowed context (7.13.3.1).
- (126) The **longjmp** function is invoked to restore a nonexistent environment (7.13.3.1).
- (127) After a **longjmp**, there is an attempt to access the value of an object of automatic storage duration that does not have volatile-qualified type, local to the function containing the invocation of the corresponding **setjmp** macro, that was changed between the **setjmp** invocation and **longjmp** call (7.13.3.1).
- (128) The program specifies an invalid pointer to a signal handler function (7.14.2.1).
- (129) A signal handler returns when the signal corresponded to a computational exception (7.14.2.1).
- (130) A signal handler called in response to **SIGFPE**, **SIGILL**, **SIGSEGV**, or any other implementation-defined value corresponding to a computational exception returns (7.14.2.1).
- (131) A signal occurs as the result of calling the **abort** or **raise** function, and the signal handler calls the **raise** function (7.14.2.1).
- (132) A signal occurs other than as the result of calling the **abort** or **raise** function, and the signal handler refers to an object with static or thread storage duration that is not a lock-free atomic object other than by assigning a value to an object declared as **volatile sig_atomic_t**, or calls any function in the standard library other than the **abort** function, the **_Exit** function, the **quick_exit** function, the functions in **<stdatomic.h>** (except where explicitly stated otherwise) when the atomic arguments are lock-free, the **atomic_is_lock_free** function with any atomic argument, or the **signal** function (for the same signal number) (7.14.2.1).

- (133) The value of **errno** is referred to after a signal occurred other than as the result of calling the **abort** or **raise** function and the corresponding signal handler obtained a **SIG_ERR** return from a call to the **signal** function (7.14.2.1).
- (134) A signal is generated by an asynchronous signal handler (7.14.2.1).
- (135) The **signal** function is used in a multi-threaded program (7.14.2.1).
- (136) A function with a variable number of arguments attempts to access its varying arguments other than through a properly declared and initialized **va_list** object, or before the **va_start** macro is invoked (7.16, 7.16.2.2, 7.16.2.5).
- (137) The macro **va_arg** is invoked using the parameter **ap** that was passed to a function that invoked the macro **va_arg** with the same parameter (7.16).
- (138) A macro definition of **va_start**, **va_arg**, **va_copy**, or **va_end** is suppressed to access an actual function, or the program defines an external identifier with the name **va_copy** or **va_end** (7.16.2).
- (139) The **va_start** or **va_copy** macro is invoked without a corresponding invocation of the **va_end** macro in the same function, or vice versa (7.16.2, 7.16.2.3, 7.16.2.4, 7.16.2.5).
- (140) The **va_arg** macro is invoked when there is no actual next argument, or with a specified type that is not compatible with the promoted type of the actual next argument, with certain exceptions (7.16.2.2).
- (141) The *type* parameter to the **va_arg** macro does not name an object type (7.16.2.2).
- (142) Using a null pointer constant in form of an integer expression as an argument to a ... function and then interpreting it as a **void*** or **char*** (7.16.2.2).
- (143) The **va_copy** or **va_start** macro is invoked to initialize a **va_list** that was previously initialized by either macro without an intervening invocation of the **va_end** macro for the same **va_list** (7.16.2.3, 7.16.2.5).
- (144) The **va_start** macro is invoked with additional arguments that include unbalanced parentheses, or unrecognized preprocessing tokens (7.16.2.5).
- (145) The macro definition of a generic function is suppressed to access an actual function (7.17.1, 7.18).
- (146) The *type* parameter of an **offsetof** macro defines a new type (7.21).
- (147) When program execution reaches an **unreachable()** macro invocation (7.21.2).
- (148) Arbitrarily copying or changing the bytes of or copying from a non-null pointer into a **nullptr_t** object and then reading that object (7.21.3).
- (149) The *member-designator* parameter of an **offsetof** macro is an invalid right operand of the **.** operator for the *type* parameter, or designates a bit-field (7.21).
- (150) The argument in an instance of one of the integer-constant macros is not a decimal, octal, or hexadecimal constant, or it has a value that exceeds the limits for the corresponding type (7.22.5).
- (151) A byte input/output function is applied to a wide-oriented stream, or a wide character input/output function is applied to a byte-oriented stream (7.23.2).
- (152) Use is made of any portion of a file beyond the most recent wide character written to a wide-oriented stream (7.23.2).
- (153) The value of a pointer to a **FILE** object is used after the associated file is closed (7.23.3).

- (154) The stream for the **fflush** function points to an input stream or to an update stream in which the most recent operation was input (7.23.5.2).
- (155) The string pointed to by the **mode** argument in a call to the **fopen** function does not exactly match one of the specified character sequences (7.23.5.3).
- (156) An output operation on an update stream is followed by an input operation without an intervening call to the **fflush** function or a file positioning function, or an input operation on an update stream is followed by an output operation with an intervening call to a file positioning function (7.23.5.3).
- (157) An attempt is made to use the contents of the array that was supplied in a call to the **setvbuf** function (7.23.5.6).
- (158) There are insufficient arguments for the format in a call to one of the formatted input/output functions, or an argument does not have an appropriate type (7.23.6.2, 7.23.6.3, 7.31.2.2, 7.31.2.3).
- (159) The format in a call to one of the formatted input/output functions or to the **strftime** or **wcsftime** function is not a valid multibyte character sequence that begins and ends in its initial shift state (7.23.6.2, 7.23.6.3, 7.29.3.6, 7.31.2.2, 7.31.2.3, 7.31.5.1).
- (160) In a call to one of the formatted output functions, a precision appears with a conversion specifier other than those described (7.23.6.2, 7.31.2.2).
- (161) A conversion specification for a formatted output function uses an asterisk to denote an argument-supplied field width or precision, but the corresponding argument is not provided (7.23.6.2, 7.31.2.2).
- (162) A conversion specification for a formatted output function uses a **#** or **0** flag with a conversion specifier other than those described (7.23.6.2, 7.31.2.2).
- (163) A conversion specification for one of the formatted input/output functions uses a length modifier with a conversion specifier other than those described (7.23.6.2, 7.23.6.3, 7.31.2.2, 7.31.2.3).
- (164) An **s** conversion specifier is encountered by one of the formatted output functions, and the argument is missing the null terminator (unless a precision is specified that does not require null termination) (7.23.6.2, 7.31.2.2).
- (165) An **n** conversion specification for one of the formatted input/output functions includes any flags, an assignment-suppressing character, a field width, or a precision (7.23.6.2, 7.23.6.3, 7.31.2.2, 7.31.2.3).
- (166) A **%** conversion specifier is encountered by one of the formatted input/output functions, but the complete conversion specification is not exactly **%%** (7.23.6.2, 7.23.6.3, 7.31.2.2, 7.31.2.3).
- (167) An invalid conversion specification is found in the format for one of the formatted input/output functions, or the **strftime** or **wcsftime** function (7.23.6.2, 7.23.6.3, 7.29.3.6, 7.31.2.2, 7.31.2.3, 7.31.5.1).
- (168) The number of characters or wide characters transmitted by a formatted output function (or written to an array, or that would have been written to an array) is greater than **INT_MAX** (7.23.6.2, 7.31.2.2).
- (169) The number of input items assigned by a formatted input function is greater than **INT_MAX** (7.23.6.3, 7.31.2.3).
- (170) The result of a conversion by one of the formatted input functions cannot be represented in the corresponding object, or the receiving object does not have an appropriate type (7.23.6.3, 7.31.2.3).

- (171) A **c**, **s**, or **[** conversion specifier is encountered by one of the formatted input functions, and the array pointed to by the corresponding argument is not large enough to accept the input sequence (and a null terminator if the conversion specifier is **s** or **[**) (7.23.6.3, 7.31.2.3).
- (172) A **c**, **s**, or **[** conversion specifier with an **l** qualifier is encountered by one of the formatted input functions, but the input is not a valid multibyte character sequence that begins in the initial shift state (7.23.6.3, 7.31.2.3).
- (173) The input item for a **%p** conversion by one of the formatted input functions is not a value converted earlier during the same program execution (7.23.6.3, 7.31.2.3).
- (174) The **fprintf**, **fscanf**, **printf**, **scanf**, **vsprintf**, **vscanf**, **vsscanf**, **fwprintf**, **fwscanf**, **vswprintf**, **vswscanf**, **wprintf**, or **wscanf** function is called with an improperly initialized **va_list** argument, or the argument is used (other than in an invocation of **va_end**) after the function returns (7.23.6.9, 7.23.6.10, 7.23.6.11, 7.23.6.12, 7.23.6.13, 7.23.6.14, 7.23.6.15, 7.31.2.6, 7.31.2.7, 7.31.2.8, 7.31.2.9, 7.31.2.10, 7.31.2.11).
- (175) The contents of the array supplied in a call to the **fgets** or **fgetws** function are used after a read error occurred (7.23.7.2, 7.31.3.2).
- (176) The **n** parameter is negative or zero for a call to **fgets** or **fgetws**. (7.23.7.2, 7.31.3.2).
- (177) The file position indicator for a binary stream is used after a call to the **ungetc** function where its value was zero before the call (7.23.7.10).
- (178) The file position indicator for a stream is used after an error occurred during a call to the **fread** or **fwrite** function (7.23.8.1, 7.23.8.2).
- (179) A partial element read by a call to the **fread** function is used (7.23.8.1).
- (180) The **fseek** function is called for a text stream with a nonzero offset and either the offset was not returned by a previous successful call to the **ftell** function on a stream associated with the same file or **whence** is not **SEEK_SET** (7.23.9.2).
- (181) The **fsetpos** function is called to set a position that was not returned by a previous successful call to the **fgetpos** function on a stream associated with the same file (7.23.9.3).
- (182) A non-null pointer returned by a call to the **calloc**, **malloc**, **realloc**, or **aligned_alloc** function with a zero requested size is used to access an object (7.24.4).
- (183) The value of a pointer that refers to space deallocated by a call to the **free** or **realloc** function is used (7.24.4).
- (184) The pointer argument to the **free** or **realloc** function is unequal to a null pointer and does not match a pointer earlier returned by a memory management function, or the space has been deallocated by a call to **free** or **realloc** (7.24.4.4, 7.24.4.8).
- (185) The value of the object allocated by the **malloc** function is used (7.24.4.7).
- (186) The values of any bytes in a new object allocated by the **realloc** function beyond the size of the old object are used (7.24.4.8).
- (187) The program calls the **exit** or **quick_exit** function more than once, or calls both functions (7.24.5.4, 7.24.5.7).
- (188) During the call to a function registered with the **atexit** or **at_quick_exit** function, a call is made to the **longjmp** function that would terminate the call to the registered function (7.24.5.4, 7.24.5.7).
- (189) The string set up by the **getenv** or **strerror** function is modified by the program (7.24.5.6, 7.26.6.3).
- (190) A signal is raised while the **quick_exit** function is executing (7.24.5.7).

- (191) A command is executed through the **system** function in a way that is documented as causing termination or some other form of undefined behavior (7.24.5.8).
- (192) A searching or sorting utility function is called with an invalid pointer argument, even if the number of elements is zero (7.24.6).
- (193) The comparison function called by a searching or sorting utility function alters the contents of the array being searched or sorted, or returns ordering values inconsistently (7.24.6).
- (194) The array being searched by the **bsearch** function does not have its elements in proper order (7.24.6.2).
- (195) The current conversion state is used by a multibyte/wide character conversion function after changing the **LC_CTYPE** category (7.24.8).
- (196) A string or wide string utility function is instructed to access an array beyond the end of an object (7.26.1, 7.31.4).
- (197) A string or wide string utility function is called with an invalid pointer argument, even if the length is zero (7.26.1, 7.31.4).
- (198) The contents of the destination array are used after a call to the **strxfrm**, **strftime**, **wcsxfrm**, or **wcsftime** function in which the specified length was too small to hold the entire null-terminated result (7.26.4.6, 7.29.3.6, 7.31.4.5.5, 7.31.5.1).
- (199) A sequence of calls of the **strtok** function is made from different threads (7.26.5.9).
- (200) The first argument in the very first call to the **strtok** or **wcstok** is a null pointer (7.26.5.9, 7.31.4.6.8).
- (201) A pointer returned by the **strerror** function is used after a subsequent call to the function, or after the calling thread has exited (7.26.6.3).
- (202) The type of an argument to a type-generic macro is not compatible with the type of the corresponding parameter of the selected function (7.27).
- (203) Arguments for generic parameters of a type-generic macro are such that some argument has a corresponding real type that is of standard floating type and another argument is of decimal floating type (7.27).
- (204) Arguments for generic parameters of a type-generic macro are such that neither **<math.h>** and **<complex.h>** define a function whose generic parameters have the determined corresponding real type (7.27).
- (205) A complex argument is supplied for a generic parameter of a type-generic macro that has no corresponding complex function (7.27).
- (206) A decimal floating argument is supplied for a generic parameter of a type-generic macro that expects a complex argument (7.27).
- (207) A standard floating or complex argument is supplied for a generic parameter of a type-generic macro that expects a decimal floating type argument (7.27).
- (208) A non-recursive mutex passed to **mtx_lock** is locked by the calling thread (7.28.4.4).
- (209) The mutex passed to **mtx_timedlock** does not support timeout (7.28.4.5).
- (210) The mutex passed to **mtx_unlock** is not locked by the calling thread (7.28.4.7).
- (211) The thread passed to **thrd_detach** or **thrd_join** was previously detached or joined with another thread (7.28.5.3, 7.28.5.6).
- (212) The **tss_create** function is called from within a destructor (7.28.6.1).

- (213) The key passed to **tss_delete**, **tss_get**, or **tss_set** was not returned by a call to **tss_create** before the thread commenced executing destructors (7.28.6.2, 7.28.6.3, 7.28.6.4).
- (214) An attempt is made to access the pointer returned by the time conversion functions after the thread that originally called the function to obtain it has exited (7.29.3).
- (215) At least one member of the broken-down time passed to **asctime** contains a value outside its normal range, or the calculated year exceeds four digits or is less than the year 1000 (7.29.3.2).
- (216) The argument corresponding to an **s** specifier without an **l** qualifier in a call to the **fwprintf** function does not point to a valid multibyte character sequence that begins in the initial shift state (7.31.2.12).
- (217) In a call to the **wcstok** function, the object pointed to by **ptr** does not have the value stored by the previous call for the same wide string (7.31.4.6.8).
- (218) An **mbstate_t** object is used inappropriately (7.31.6).
- (219) The value of an argument of type **wint_t** to a wide character classification or case mapping function is neither equal to the value of **WEOF** nor representable as a **wchar_t** (7.32.1).
- (220) The **iswctype** function is called using a different **LC_CTYPE** category from the one in effect for the call to the **wctype** function that returned the description (7.32.2.3.2).
- (221) The **towctrans** function is called using a different **LC_CTYPE** category from the one in effect for the call to the **wctrans** function that returned the description (7.32.3.2.2).

J.3 Implementation-defined behavior

J.3.1 General

A conforming implementation is required to document its choice of behavior in each of the areas listed in this subclause. The following are implementation-defined:

J.3.2 Translation

- (1) How a diagnostic is identified (3.13, 5.2.1.3).
- (2) Whether each nonempty sequence of white-space characters other than new-line is retained or replaced by one space character in translation phase 3 (5.2.1.2).

J.3.3 Environment

- (1) The mapping between physical source file multibyte characters and the source character set in translation phase 1 (5.2.1.2).
- (2) The name and type of the function called at program startup in a freestanding environment (5.2.2.2).
- (3) The effect of program termination in a freestanding environment (5.2.2.2).
- (4) An alternative manner in which the **main** function may be defined (5.2.2.3.2).
- (5) The values given to the strings pointed to by the **argv** argument to **main** (5.2.2.3.2).
- (6) What constitutes an interactive device (5.2.2.4).
- (7) Whether a program can have more than one thread of execution in a freestanding environment (5.2.2.5).
- (8) The set of signals, their semantics, and their default handling (7.14).
- (9) Signal values other than **SIGFPE**, **SIGILL**, and **SIGSEGV** that correspond to a computational exception (7.14.2.1).

- (10) Signals for which the equivalent of **signal(sig, SIG_IGN)**; is executed at program startup (7.14.2.1).
- (11) The set of environment names and the method for altering the environment list used by the **getenv** function (7.24.5.6).
- (12) The manner of execution of the string by the **system** function (7.24.5.8).

J.3.4 Identifiers

- (1) Which additional multibyte characters may appear in identifiers and their correspondence to universal character names (6.4.3).
- (2) The number of significant initial characters in an identifier (5.3.5.2, 6.4.3).

J.3.5 Characters

- (1) The number of bits in a byte (3.7).
- (2) The values of the members of the execution character set (5.3.1).
- (3) The unique value of the member of the execution character set produced for each of the standard alphabetic escape sequences (5.3.3).
- (4) The value of a **char** object into which has been stored any character other than a member of the basic execution character set (6.2.5).
- (5) Which of **signed char** or **unsigned char** has the same range, representation, and behavior as “plain” **char** (6.2.5, 6.3.2.1).
- (6) The literal encoding, which maps of the characters of the execution character set to the values in a character constant or string literal (6.2.9, 6.4.5.5).
- (7) The wide literal encoding, of the characters of the execution character set to the values in a **wchar_t** character constant or **wchar_t** string literal (6.2.9, 6.4.5.5).
- (8) The mapping of members of the source character set (in character constants and string literals) to members of the execution character set (6.4.5.5, 5.2.1.2).
- (9) The value of an integer character constant containing more than one character or containing a character or escape sequence that does not map to a single-byte execution character (6.4.5.5).
- (10) The value of a wide character constant containing more than one multibyte character or a single multibyte character that maps to multiple members of the extended execution character set, or containing a multibyte character or escape sequence not represented in the extended execution character set (6.4.5.5).
- (11) The current locale used to convert a wide character constant consisting of a single multibyte character that maps to a member of the extended execution character set into a corresponding wide character code (6.4.5.5).
- (12) The current locale used to convert a wide string literal into corresponding wide character codes (6.4.6).
- (13) The value of a string literal containing a multibyte character or escape sequence not represented in the execution character set (6.4.6).
- (14) The encoding of **wchar_t** where the macro **__STDC_ISO_10646__** is not defined (6.10.10.3).

J.3.6 Integers

- (1) Any extended integer types that exist in the implementation (6.2.5).
- (2) The rank of any extended integer type relative to another extended integer type with the same precision (6.3.2.1).
- (3) The result of, or the signal raised by, converting an integer to a signed integer type when the value cannot be represented in an object of that type (6.3.2.3).
- (4) The results of some bitwise operations on signed integers (6.5.1).

J.3.7 Floating-point

- (1) The accuracy of the floating-point operations and of the library functions in `<math.h>` and `<complex.h>` that return floating-point results (5.3.5.3.3).
- (2) The accuracy of the conversions between floating-point internal representations and string representations performed by the library functions in `<stdio.h>`, `<stdlib.h>`, and `<wchar.h>` (5.3.5.3.3).
- (3) The rounding behaviors characterized by non-standard values of **FLT_ROUNDS** (5.3.5.3.3).
- (4) The evaluation methods characterized by non-standard negative values of **FLT_EVAL_METHOD** (5.3.5.3.3).
- (5) The evaluation methods characterized by non-standard negative values of **DEC_EVAL_METHOD** (5.3.5.3.4).
- (6) If decimal floating types are supported (6.2.5).
- (7) The direction of rounding when an integer is converted to a floating-point number that cannot exactly represent the original value (6.3.2.4).
- (8) The direction of rounding when a floating-point number is converted to a narrower floating-point number (6.3.2.5).
- (9) How the nearest representable value or the larger or smaller representable value immediately adjacent to the nearest representable value is chosen for certain floating constants (6.4.5.3).
- (10) Whether and how floating expressions are contracted when not disallowed by the **FP_CONTRACT** pragma (6.5.1).
- (11) The default state for the **FENV_ACCESS** pragma (7.6.2).
- (12) Additional floating-point exceptions, rounding modes, environments, and classifications, and their macro names (7.6, 7.12).
- (13) The default state for the **FP_CONTRACT** pragma (7.12.3).

J.3.8 Constant expressions

- (1) Whether or not an expression not explicitly sanctioned by this document is an extended constant expression, whether or not such extended constant expressions can be used in the same contexts as this document, and whether or not such extended constant expressions can affect potentially detectable semantic changes in the program (6.6).

J.3.9 Arrays and pointers

- (1) The result of converting a pointer to an integer or vice versa (6.3.3.3).
- (2) The size of the result of subtracting two pointers to elements of the same array (6.5.7).

J.3.10 Hints

- (1) The extent to which suggestions made by using the **register** storage-class specifier are effective (6.7.2).
- (2) The extent to which suggestions made by using the **inline** function specifier are effective (6.7.5).

J.3.11 Structures, unions, enumerations, and bit-fields

- (1) Whether a “plain” **int** bit-field is treated as a **signed int** bit-field or as an **unsigned int** bit-field (6.7.3, 6.7.3.2).
- (2) Allowable bit-field types other than **bool**, **signed int**, **unsigned int**, and bit-precise integer types (6.7.3.2).
- (3) Whether atomic types are permitted for bit-fields (6.7.3.2).
- (4) Whether a bit-field can straddle a storage-unit boundary (6.7.3.2).
- (5) The order of allocation of bit-fields within a unit (6.7.3.2).
- (6) The alignment of non-bit-field members of structures (6.7.3.2). This should present no problem unless binary data written by one implementation is read by another.
- (7) The integer type compatible with each enumerated type without fixed underlying type (6.7.3.3).

J.3.12 Qualifiers

- (1) What constitutes an access to an object that has volatile-qualified type (6.7.4).

J.3.13 Types

- (1) A program forms the composite type of an enumerated type and a non-enumeration integer type (6.2.7).
- (2) Whether or not it is supported for a declaration for which a type is inferred to contain a pointer, array, or function declarator (6.7.10).
- (3) Whether or not it is supported for a declaration for which a type is inferred to contain no or more than one declarators (6.7.10).

J.3.14 Preprocessing directives

- (1) The locations within **#pragma** directives where header name preprocessing tokens are recognized (6.4, 6.4.8).
- (2) How sequences in both forms of header names are mapped to headers or external source file names (6.4.8).
- (3) Whether the value of a character constant in a constant expression that controls conditional inclusion matches the value of the same character constant in the execution character set (6.10.2).
- (4) Whether the value of a single-character character constant in a constant expression that controls conditional inclusion may have a negative value (6.10.2).
- (5) The places that are searched for an included `< >` delimited header, and how the places are specified or the header is identified (6.10.3).
- (6) How the named source file is searched for in an included `" "` delimited header name (6.10.3).
- (7) How the named resource file is searched for in an embedded `" "` delimited resource name (6.10.4).

- (8) The method by which preprocessing tokens (possibly resulting from macro expansion) in a **#include** directive are combined into a header name (6.10.3).
- (9) The nesting limit for **#include** processing (6.10.3).
- (10) The method by which preprocessing tokens (possibly resulting from macro expansion) in a **#embed** directive are combined into a resource name (6.10.4).
- (11) The mapping between a resource's data and the values of the integer constant expressions, if any, in the replacement of a **#embed** directive (6.10.4).
- (12) The width of a resource located by the **#embed** directive (6.10.4).
- (13) Whether the **#** operator inserts a **** character before the **** character that begins a universal character name in a character constant or string literal (6.10.5.3).
- (14) The behavior on each recognized non-**STDC** **#pragma** directive (6.10.8).
- (15) The definitions for **__DATE__** and **__TIME__** when respectively, the date and time of translation are not available (6.10.10.2).

J.3.15 Library functions

- (1) Any library facilities available to a freestanding program, other than the minimal set required by Clause 4 (5.2.2.2).
- (2) The format of the diagnostic printed by the **assert** macro (7.2.2.1).
- (3) The representation of the floating-point status flags stored by the **fegetexceptflag** function (7.6.5.3).
- (4) Whether the **feraiseexcept** function raises the "inexact" floating-point exception in addition to the "overflow" or "underflow" floating-point exception (7.6.5.4).
- (5) Strings other than **"C"** and **""** that may be passed as the second argument to the **setlocale** function (7.11.2).
- (6) The types defined for **float_t** and **double_t** when the value of the **FLT_EVAL_METHOD** macro is less than 0 (7.12).
- (7) The types defined for **_Decimal32_t** and **_Decimal64_t** when the value of the **DEC_EVAL_METHOD** macro is less than 0 (7.12).
- (8) Domain errors for the mathematics functions, other than those required by this document (7.12.2).
- (9) The values returned by the mathematics functions on domain errors or pole errors (7.12.2).
- (10) The values returned by the mathematics functions on underflow range errors, whether **errno** is set to the value of the macro **ERANGE** when the integer expression **math_errhandling & MATH_ERRNO** is nonzero, and whether the "underflow" floating-point exception is raised when the integer expression **math_errhandling & MATH_ERREXCEPT** is nonzero. (7.12.2).
- (11) Whether a domain error occurs or zero is returned when an **fmod** function has a second argument of zero (7.12.11.1).
- (12) Whether a domain error occurs or zero is returned when a **remainder** function has a second argument of zero (7.12.11.2).
- (13) The base-2 logarithm of the modulus used by the **remquo** functions in reducing the quotient (7.12.11.3).
- (14) Whether a domain error occurs or zero is returned when a **remquo** function has a second argument of zero (7.12.11.3).

- (15) Whether the equivalent of **signal(sig, SIG_DFL)**; is executed prior to the call of a signal handler, and, if not, the blocking of signals that is performed (7.14.2.1).
- (16) The value of **__STDC_ENDIAN_NATIVE__** if the execution environment is not big-endian or little-endian (7.18.2)
- (17) The null pointer constant to which the macro **NULL** expands (7.21).
- (18) Whether the last line of a text stream requires a terminating new-line character (7.23.2).
- (19) Whether space characters that are written out to a text stream immediately before a new-line character appear when read in (7.23.2).
- (20) The number of null characters that may be appended to data written to a binary stream (7.23.2).
- (21) Whether the file position indicator of an append-mode stream is initially positioned at the beginning or end of the file (7.23.3).
- (22) Whether a write on a text stream causes the associated file to be truncated beyond that point (7.23.3).
- (23) The characteristics of file buffering (7.23.3).
- (24) Whether a zero-length file actually exists (7.23.3).
- (25) The rules for composing valid file names (7.23.3).
- (26) Whether the same file can be simultaneously open multiple times (7.23.3).
- (27) The nature and choice of encodings used for multibyte characters in files (7.23.3).
- (28) The effect of the **remove** function on an open file (7.23.4.1).
- (29) The effect if a file with the new name exists prior to a call to the **rename** function (7.23.4.2).
- (30) Whether an open temporary file is removed upon abnormal program termination (7.23.4.3).
- (31) Which changes of mode are permitted (if any), and under what circumstances (7.23.5.4).
- (32) The style used to print an infinity or NaN, and the meaning of any n-char or n-wchar sequence printed for a NaN (7.23.6.2, 7.31.2.2).
- (33) The output for **%p** conversion in the **fprintf** or **fwprintf** function (7.23.6.2, 7.31.2.2).
- (34) The interpretation of a **-** character that is neither the first nor the last character, nor the second where a **^** character is the first, in the scanlist for **%[** conversion in the **fscanf** or **fwscanf** function (7.23.6.3, 7.31.2.2).
- (35) The set of sequences matched by a **%p** conversion and the interpretation of the corresponding input item in the **fscanf** or **fwscanf** function (7.23.6.3, 7.31.2.3).
- (36) The value to which the macro **errno** is set by the **fgetpos**, **fsetpos**, or **ftell** functions on failure (7.23.9.1, 7.23.9.3, 7.23.9.4).
- (37) The meaning of any n-char or n-wchar sequence in a string representing a NaN that is converted by the **strtod**, **strtof**, **strtold**, **wctod**, **wctof**, or **wctold** function (7.24.2.6, 7.31.4.2.2).
- (38) Whether the **strtod**, **strtof**, **strtold**, **wctod**, **wctof**, or **wctold** function sets **errno** to **ERANGE** when underflow occurs (7.24.2.6, 7.31.4.2.2).
- (39) The meaning of any d-char or d-wchar sequence in a string representing a NaN that is converted by the **strtod32**, **strtod64**, **strtod128**, **wctod32**, **wctod64**, or **wctod128** function (7.24.2.7, 7.31.4.2.3).

- (40) Whether the **strtod32**, **strtod64**, **strtod128**, **wcstod32**, **wcstod64**, or **wcstod128** function sets **errno** to **ERANGE** when underflow occurs (7.24.2.7, 7.31.4.2.3).
- (41) Whether the **calloc**, **malloc**, **realloc**, and **aligned_alloc** functions return a null pointer or a pointer to an allocated object when the size requested is zero (7.24.4).
- (42) Whether open streams with unwritten buffered data are flushed, open streams are closed, or temporary files are removed when the **abort** or **_Exit** function is called (7.24.5.1, 7.24.5.5).
- (43) The termination status returned to the host environment by the **abort**, **exit**, **_Exit**, or **quick_exit** function (7.24.5.1, 7.24.5.4, 7.24.5.5, 7.24.5.7).
- (44) The value returned by the **system** function when its argument is not a null pointer (7.24.5.8).
- (45) Whether the internal state of multibyte/wide character conversion functions has thread-storage duration, and its initial value in newly created threads (7.24.8).
- (46) Whether the multibyte/wide character conversion functions avoid data races with other calls to the same function (7.24.8).
- (47) The range and precision of times representable in **clock_t** and **time_t** (7.29).
- (48) The local time zone and Daylight Saving Time (7.29.1).
- (49) Whether **TIME_MONOTONIC** or **TIME_ACTIVE** are supported time bases (7.29.1).
- (50) Whether **TIME_THREAD_ACTIVE** is a supported time bases (7.29.1, 7.28.1).
- (51) The era for the **clock** function (7.29.2.1).
- (52) The **TIME_UTC** epoch (7.29.2.6).
- (53) The replacement string for the **%Z** specifier to the **strftime**, and **wcsftime** functions in the **"C"** locale (7.29.3.6, 7.31.5.1).
- (54) Whether internal **mbstate_t** objects have thread storage duration (7.30.2, 7.31.6.4, 7.31.6.5).
- (55) Whether the functions in **<math.h>** honor the rounding direction mode in an ISO/IEC 60559 conformant implementation, unless explicitly specified otherwise (F.10).

J.3.16 Architecture

- (1) The values or expressions assigned to the macros specified in the headers **<float.h>**, **<limits.h>**, and **<stdint.h>** (5.3.5.3, 7.22).
- (2) The result of attempting to indirectly access an object with automatic or thread storage duration from a thread other than the one with which it is associated (6.2.4).
- (3) The number, order, and encoding of bytes in any object (when not explicitly specified in this document) (6.2.6.1).
- (4) Whether any extended alignments are supported and the contexts in which they are supported (6.2.8).
- (5) Valid alignment values other than those returned by an **alignof** expression for fundamental types, if any (6.2.8).
- (6) The value of the result of the **sizeof** and **alignof** operators (6.5.4.5).

J.4 Locale-specific behavior

The following characteristics of a hosted environment are locale-specific and are required to be documented by the implementation:

- (1) Additional members of the source and execution character sets beyond the basic character set (5.3.1).
- (2) The presence, meaning, and representation of additional multibyte characters in the execution character set beyond the basic character set (5.3.2).
- (3) The shift states used for the encoding of multibyte characters (5.3.2).
- (4) The direction of writing of successive printing characters (5.3.3).
- (5) The decimal-point character (7.1.1).
- (6) The set of printing characters (7.4, 7.32.2).
- (7) The set of control characters (7.4, 7.32.2).
- (8) The sets of characters tested for by the **isalpha**, **isblank**, **islower**, **ispunct**, **isspace**, **isupper**, **iswalph**, **iswblank**, **iswlower**, **iswpunct**, **iswspace**, or **iswupper** functions (7.4.2.3, 7.4.2.4, 7.4.2.8, 7.4.2.10, 7.4.2.11, 7.4.2.12, 7.32.2.2.3, 7.32.2.2.4, 7.32.2.2.8, 7.32.2.2.10, 7.32.2.2.11, 7.32.2.2.12).
- (9) The native environment (7.11.2).
- (10) Additional subject sequences accepted by the numeric conversion functions (7.24.2, 7.31.4.2).
- (11) The collation sequence of the execution character set (7.26.4.4, 7.31.4.5.3).
- (12) The contents of the error message strings set up by the **strerror** function (7.26.6.3).
- (13) The formats for time and date (7.29.3.6, 7.31.5.1).
- (14) Character mappings that are supported by the **towctrans** function (7.32.1).
- (15) Character classifications that are supported by the **iswctype** function (7.32.1).

J.5 Common extensions

J.5.1 General

The following extensions are widely used in many systems, but are not portable to all implementations. The inclusion of any extension that may cause a strictly conforming program to become invalid renders an implementation nonconforming. Examples of such extensions are new keywords, extra library functions declared in standard headers, or predefined macros with names that do not begin with an underscore.

J.5.2 Environment arguments

In a hosted environment, the **main** function receives a third argument, **char *envp[]**, that points to a null-terminated array of pointers to **char**, each of which points to a string that provides information about the environment for this execution of the program (5.2.2.3.2).

J.5.3 Specialized identifiers

Characters other than the underscore **_**, letters, and digits, that are not part of the basic source character set (such as the dollar sign **\$**, or characters in national character sets) may appear in an identifier (6.4.3).

J.5.4 Lengths and cases of identifiers

All characters in identifiers (with or without external linkage) are significant (6.4.3).

J.5.5 Scopes of identifiers

A function identifier, or the identifier of an object the declaration of which contains the keyword **extern**, has file scope (6.2.1).

J.5.6 Writable string literals

String literals are modifiable (in which case, identical string literals should denote distinct objects) (6.4.6).

J.5.7 Other arithmetic types

Additional arithmetic types, such as **__int128** or **double double**, and their appropriate conversions are defined (6.2.5, 6.3.2). Additional floating types may have more range or precision than **long double**, may be used for evaluating expressions of other floating types, and may be used to define **float_t** or **double_t**. Additional floating types may also have less range or precision than **float**.

J.5.8 Function pointer casts

A pointer to an object or to **void** may be cast to a pointer to a function, allowing data to be invoked as a function (6.5.5).

A pointer to a function may be cast to a pointer to an object or to **void**, allowing a function to be inspected or modified (for example, by a debugger) (6.5.5).

J.5.9 Extended bit-field types

A bit-field may be declared with a type other than **bool**, **unsigned int**, **signed int**, or a bit-precise integer type, with an appropriate maximum width (6.7.3.2).

J.5.10 The **fortran** keyword

The **fortran** function specifier may be used in a function declaration to indicate that calls suitable for FORTRAN should be generated, or that a different representation for the external name is to be generated (6.7.5).

J.5.11 The **asm** keyword

The **asm** keyword may be used to insert assembly language directly into the translator output (6.8). The most common implementation is via a statement of the form:

```
asm (character-string-literal);
```

J.5.12 Type inference

A declaration for which a type is inferred (6.7.10) may additionally accept pointer declarators, function declarators, and may have more than one declarator.

J.5.13 Multiple external definitions

There may be more than one external definition for the identifier of an object, with or without the explicit use of the keyword **extern**; if the definitions disagree, or more than one is initialized, the behavior is undefined (6.9.3).

J.5.14 Predefined macro names

Macro names that do not begin with an underscore, describing the translation and execution environments, are defined by the implementation before translation begins (6.10.10).

J.5.15 Floating-point status flags

If any floating-point status flags are set on normal termination after all calls to functions registered by the **atexit** function have been made (see 7.24.5.4), the implementation writes some diagnostics indicating the fact to the **stderr** stream, if it is still open,

J.5.16 Extra arguments for signal handlers

Handlers for specific signals are called with extra arguments in addition to the signal number (7.14.2.1).

J.5.17 Additional stream types and file-opening modes

Additional mappings from files to streams are supported (7.23.2).

Additional file-opening modes may be specified by characters appended to the **mode** argument of the **fopen** function (7.23.5.3).

J.5.18 Defined file position indicator

The file position indicator is decremented by each successful call to the **ungetc** or **ungetwc** function for a text stream, except if its value was zero before a call (7.23.7.10, 7.31.3.10).

J.5.19 Math error reporting

Functions declared in `<complex.h>` and `<math.h>` raise **SIGFPE** to report errors instead of, or in addition to, setting **errno** or raising floating-point exceptions (7.3, 7.12).

J.6 Reserved identifiers and keywords

J.6.1 General

A lot of identifier preprocessing tokens are used for specific purposes in regular clauses or appendices from translation phase 3 (5.2.1.2) onwards. Using any of these for a purpose different from their description in this document, even if the use is in a context where they are normatively permitted, may have an impact on the portability of code and should thus be avoided.

J.6.2 Rule based identifiers

The following

regular expressions characterize identifiers that are systematically reserved by some clause in this document.

ATOMIC_ [A-Z][a-zA-Z0-9_]*	[a-zA-Z0-9_]*_DECIMAL_DIG
DBL_ [A-Z][a-zA-Z0-9_]*	[a-zA-Z0-9_]*_DIG
DEC128_ [A-Z][a-zA-Z0-9_]*	[a-zA-Z0-9_]*_EPSILON
DEC32_ [A-Z][a-zA-Z0-9_]*	[a-zA-Z0-9_]*_MANT_DIG
DEC64_ [A-Z][a-zA-Z0-9_]*	[a-zA-Z0-9_]*_MAX_10_EXP
DEC_ [A-Z][a-zA-Z0-9_]*	[a-zA-Z0-9_]*_MAX_EXP
E [0-9A-Z][a-zA-Z0-9_]*	[a-zA-Z0-9_]*_MAX
FE_ [A-Z][a-zA-Z0-9_]*	[a-zA-Z0-9_]*_MIN_10_EXP
FLT_ [A-Z][a-zA-Z0-9_]*	[a-zA-Z0-9_]*_MIN_EXP
FP_ [A-Z][a-zA-Z0-9_]*	[a-zA-Z0-9_]*_MIN
INT [a-zA-Z0-9_]*_C	[a-zA-Z0-9_]*_SNAN
INT [a-zA-Z0-9_]*_MAX	[a-zA-Z0-9_]*_TRUE_MIN
INT [a-zA-Z0-9_]*_MIN	_[a-zA-Z][a-zA-Z0-9_]*
INT [a-zA-Z0-9_]*_WIDTH	atomic_ [a-z][a-zA-Z0-9_]*
LC_ [A-Z][a-zA-Z0-9_]*	ckd_ [a-z][a-zA-Z0-9_]*
LDBL_ [A-Z][a-zA-Z0-9_]*	cnd_ [a-z][a-zA-Z0-9_]*
MATH_ [A-Z][a-zA-Z0-9_]*	cr_ [a-z][a-zA-Z0-9_]*
PRI [a-zBX][a-zA-Z0-9_]*	int [a-zA-Z0-9_]*_t
SCN [a-zBX][a-zA-Z0-9_]*	is [a-z][a-zA-Z0-9_]*
SIG [A-Z][a-zA-Z0-9_]*	mem [a-z][a-zA-Z0-9_]*
SIG_ [A-Z][a-zA-Z0-9_]*	mtx_ [a-z][a-zA-Z0-9_]*
TIME_ [A-Z][a-zA-Z0-9_]*	stdc_ [a-z][a-zA-Z0-9_]*
UINT [a-zA-Z0-9_]*_C	str [a-z][a-zA-Z0-9_]*
UINT [a-zA-Z0-9_]*_MAX	thrd_ [a-z][a-zA-Z0-9_]*
UINT [a-zA-Z0-9_]*_WIDTH	to [a-z][a-zA-Z0-9_]*

tss[a-z][a-zA-Z0-9_]*
uint[a-zA-Z0-9_]*_t

wcs[a-z][a-zA-Z0-9_]*

The following

identifiers or keywords match these patterns and have particular semantics provided by this document.

atomic_bool	ATOMIC_LLONG_LOCK_FREE
ATOMIC_BOOL_LOCK_FREE	atomic_load
atomic_char	atomic_load_explicit
atomic_char16_t	atomic_long
ATOMIC_CHAR16_T_LOCK_FREE	ATOMIC_LONG_LOCK_FREE
atomic_char32_t	ATOMIC_POINTER_LOCK_FREE
ATOMIC_CHAR32_T_LOCK_FREE	atomic_ptrdiff_t
atomic_char8_t	atomic_schar
ATOMIC_CHAR8_T_LOCK_FREE	atomic_short
ATOMIC_CHAR_LOCK_FREE	ATOMIC_SHORT_LOCK_FREE
atomic_compare_exchange_strong	atomic_signal_fence
atomic_compare_exchange_strong_explicit	atomic_size_t
atomic_compare_exchange_weak	atomic_store
atomic_compare_exchange_weak_explicit	atomic_store_explicit
atomic_exchange	atomic_thread_fence
atomic_exchange_explicit	atomic_uchar
atomic_fetch_	atomic_uint
atomic_fetch_add	atomic_uintmax_t
atomic_fetch_add_explicit	atomic_uintptr_t
atomic_fetch_and	atomic_uint_fast16_t
atomic_fetch_and_explicit	atomic_uint_fast32_t
atomic_fetch_or	atomic_uint_fast64_t
atomic_fetch_or_explicit	atomic_uint_fast8_t
atomic_fetch_sub	atomic_uint_least16_t
atomic_fetch_sub_explicit	atomic_uint_least32_t
atomic_fetch_xor	atomic_uint_least64_t
atomic_fetch_xor_explicit	atomic_uint_least8_t
atomic_flag	atomic_ullong
atomic_flag_clear	atomic_ulong
atomic_flag_clear_explicit	atomic_ushort
ATOMIC_FLAG_INIT	atomic_wchar_t
atomic_flag_test_and_set	ATOMIC_WCHAR_T_LOCK_FREE
atomic_flag_test_and_set_explicit	BOOL_MAX
atomic_init	CHAR_MAX
atomic_int	CHAR_MIN
atomic_intmax_t	ckd_add
atomic_intptr_t	ckd_div
atomic_int_fast16_t	ckd_mul
atomic_int_fast32_t	ckd_sub
atomic_int_fast64_t	cnd_broadcast
atomic_int_fast8_t	cnd_destroy
atomic_int_least16_t	cnd_init
atomic_int_least32_t	cnd_signal
atomic_int_least64_t	cnd_t
atomic_int_least8_t	cnd_timedwait
ATOMIC_INT_LOCK_FREE	cnd_wait
atomic_is_lock_free	CR_DECIMAL_DIG
atomic_llong	DBL_DECIMAL_DIG

DBL_DIG	FE_DFL_ENV
DBL_EPSILON	FE_DFL_MODE
DBL_HAS_SUBNORM	FE_DIVBYZERO
DBL_IS_IEC_60559	FE_DOWNWARD
DBL_MANT_DIG	FE_DYNAMIC
DBL_MAX	FE_INEXACT
DBL_MAX_10_EXP	FE_INVALID
DBL_MAX_EXP	FE_OVERFLOW
DBL_MIN	FE_SNANS_ALWAYS_SIGNAL
DBL_MIN_10_EXP	FE_TONEAREST
DBL_MIN_EXP	FE_TONEARESTFROMZERO
DBL_NORM_MAX	FE_TOWARDZERO
DBL_SNAN	FE_UNDERFLOW
DBL_TRUE_MIN	FE_UPWARD
DEC128_EPSILON	FILENAME_MAX
DEC128_MANT_DIG	FLT_DECIMAL_DIG
DEC128_MAX	FLT_DIG
DEC128_MAX_EXP	FLT_EPSILON
DEC128_MIN	FLT_EVAL_METHOD
DEC128_MIN_EXP	FLT_HAS_SUBNORM
DEC128_SNAN	FLT_IS_IEC_60559
DEC128_TRUE_MIN	FLT_MANT_DIG
DEC32_EPSILON	FLT_MAX
DEC32_MANT_DIG	FLT_MAX_10_EXP
DEC32_MAX	FLT_MAX_EXP
DEC32_MAX_EXP	FLT_MIN
DEC32_MIN	FLT_MIN_10_EXP
DEC32_MIN_EXP	FLT_MIN_EXP
DEC32_SNAN	FLT_NORM_MAX
DEC32_TRUE_MIN	FLT_RADIX
DEC64_EPSILON	FLT_ROUNDS
DEC64_MANT_DIG	FLT_SNAN
DEC64_MAX	FLT_TRUE_MIN
DEC64_MAX_EXP	FOPEN_MAX
DEC64_MIN	FP_CONTRACT
DEC64_MIN_EXP	FP_FAST_D32ADDD128
DEC64_SNAN	FP_FAST_D32ADDD64
DEC64_TRUE_MIN	FP_FAST_D32DIVD128
DECIMAL_DIG	FP_FAST_D32DIVD64
DEC_EVAL_METHOD	FP_FAST_D32FMAD128
DEC_INFINITY	FP_FAST_D32FMAD64
DEC_NAN	FP_FAST_D32MULD128
EDOM	FP_FAST_D32MULD64
EILSEQ	FP_FAST_D32SQRTD128
EOF	FP_FAST_D32SQRTD64
EOL	FP_FAST_D32SUBD128
ERANGE	FP_FAST_D32SUBD64
EXIT_FAILURE	FP_FAST_D64ADDD128
EXIT_SUCCESS	FP_FAST_D64DIVD128
FE_ALL_EXCEPT	FP_FAST_D64FMAD128
FE_DEC_DOWNWARD	FP_FAST_D64MULD128
FE_DEC_DYNAMIC	FP_FAST_D64SQRTD128
FE_DEC_TONEAREST	FP_FAST_D64SUBD128
FE_DEC_TONEARESTFROMZERO	FP_FAST_DADDL
FE_DEC_TOWARDZERO	FP_FAST_DDIVL
FE_DEC_UPWARD	FP_FAST_DFMAL

FP_FAST_DMULL	INTMAX_MAX
FP_FAST_DSQRTL	INTMAX_MIN
FP_FAST_DSUBL	intmax_t
FP_FAST_FADD	INTMAX_WIDTH
FP_FAST_FADDL	INTPTR_MAX
FP_FAST_FDIV	INTPTR_MIN
FP_FAST_FDIVL	intptr_t
FP_FAST_FFMA	INTPTR_WIDTH
FP_FAST_FFMAL	int_fast16_t
FP_FAST_FMA	int_fast32_t
FP_FAST_FMA128	int_fast64_t
FP_FAST_FMA32	int_fast8_t
FP_FAST_FMA64	int_least16_t
FP_FAST_FMAF	int_least32_t
FP_FAST_FMAL	int_least64_t
FP_FAST_FMUL	int_least8_t
FP_FAST_FMULL	INT_MAX
FP_FAST_FSQRT	INT_MIN
FP_FAST_FSQRTL	INT_WIDTH
FP_FAST_FSUB	isalnum
FP_FAST_FSUBL	isalpha
FP_ILOGB0	isblank
FP_ILOGBNAN	iscanonical
FP_INFINITE	iscntrl
FP_INT_DOWNWARD	isdigit
FP_INT_TONEAREST	iseqsig
FP_INT_TONEARESTFROMZERO	isfinite
FP_INT_TOWARDZERO	isgraph
FP_INT_UPWARD	isgreater
FP_LLOGB0	isgreaterequal
FP_LLOGBNAN	isinf
FP_NAN	isless
FP_NORMAL	islessequal
FP_SUBNORMAL	islessgreater
FP_ZERO	islower
INT16_C	isnan
INT16_MAX	isnormal
INT16_MIN	isprint
int16_t	ispunct
INT16_WIDTH	issignaling
INT32_C	isspace
INT32_MAX	issubnormal
INT32_MIN	isunordered
int32_t	isupper
INT32_WIDTH	iswalnum
INT64_C	iswalpha
INT64_MAX	iswblank
INT64_MIN	iswcntrl
int64_t	iswctype
INT64_WIDTH	iswdigit
INT8_C	iswgraph
INT8_MAX	iswlower
INT8_MIN	iswprint
int8_t	iswpunct
INT8_WIDTH	iswspace
INTMAX_C	iswupper

iswxdigit	mtx_timed
isxdigit	mtx_timedlock
iszero	mtx_trylock
LC_ALL	mtx_unlock
LC_COLLATE	PRib32
LC_CTYPE	PRib64
LC_MONETARY	PRibFAST32
LC_NUMERIC	PRibFAST64
LC_TIME	PRibLEAST32
LDBL_DECIMAL_DIG	PRibLEAST64
LDBL_DIG	PRibMAX
LDBL_EPSILON	PRibPTR
LDBL_HAS_SUBNORM	PRId32
LDBL_IS_IEC_60559	PRId64
LDBL_MANT_DIG	PRIdFAST32
LDBL_MAX	PRIdFAST64
LDBL_MAX_10_EXP	PRIdLEAST32
LDBL_MAX_EXP	PRIdLEAST64
LDBL_MIN	PRIdMAX
LDBL_MIN_10_EXP	PRIdPTR
LDBL_MIN_EXP	PRId32
LDBL_NORM_MAX	PRId64
LDBL_SNAN	PRIdFAST32
LDBL_TRUE_MIN	PRIdFAST64
LLONG_MAX	PRIdLEAST32
LLONG_MIN	PRIdLEAST64
LONG_MAX	PRIdMAX
LONG_MIN	PRIdPTR
MATH_ERREXCEPT	PRIo32
MATH_ERRNO	PRIo64
MB_CUR_MAX	PRIoFAST32
MB_LEN_MAX	PRIoFAST64
memalignment	PRIoLEAST32
memccpy	PRIoLEAST64
memchr	PRIoMAX
memcmp	PRIoPTR
memcpy	PRId32
memcpy_s	PRId64
memmove	PRIdFAST32
memmove_s	PRIdFAST64
memory_order	PRIdLEAST32
memory_order_acquire	PRIdLEAST64
memory_order_acq_rel	PRIdMAX
memory_order_consume	PRIdPTR
memory_order_relaxed	PRId32
memory_order_release	PRId64
memory_order_seq_cst	PRIdFAST32
memset	PRIdFAST64
memset_explicit	PRIdLEAST32
memset_s	PRIdLEAST64
mtx_destroy	PRIdMAX
mtx_init	PRIdPTR
mtx_lock	PRIdIFF_MAX
mtx_plain	PRIdIFF_MIN
mtx_recursive	RAND_MAX
mtx_t	RSIZE_MAX

SCHAR_MAX	stdc_count_zeros_ul
SCHAR_MIN	stdc_count_zeros_ull
SCNbMAX	stdc_count_zeros_us
SCNbPTR	stdc_first_leading_one
SCNdMAX	stdc_first_leading_one_uc
SCNdPTR	stdc_first_leading_one_ui
SCNiMAX	stdc_first_leading_one_ul
SCNiPTR	stdc_first_leading_one_ull
SCNoMAX	stdc_first_leading_one_us
SCNoPTR	stdc_first_leading_zero
SCNuMAX	stdc_first_leading_zero_uc
SCNuPTR	stdc_first_leading_zero_ui
SCNxMAX	stdc_first_leading_zero_ul
SCNxPTR	stdc_first_leading_zero_ull
SHRT_MAX	stdc_first_leading_zero_us
SHRT_MIN	stdc_first_trailing_one
SIGABRT	stdc_first_trailing_one_uc
SIGFPE	stdc_first_trailing_one_ui
SIGILL	stdc_first_trailing_one_ul
SIGINT	stdc_first_trailing_one_ull
SIGSEGV	stdc_first_trailing_one_us
SIGTERM	stdc_first_trailing_zero
SIG_ATOMIC_MAX	stdc_first_trailing_zero_uc
SIG_ATOMIC_MIN	stdc_first_trailing_zero_ui
SIG_ATOMIC_WIDTH	stdc_first_trailing_zero_ul
SIG_DFL	stdc_first_trailing_zero_ull
SIG_ERR	stdc_first_trailing_zero_us
SIG_IGN	stdc_has_single_bit
SIZE_MAX	stdc_has_single_bit_uc
stdc_bit_ceil	stdc_has_single_bit_ui
stdc_bit_ceil_uc	stdc_has_single_bit_ul
stdc_bit_ceil_ui	stdc_has_single_bit_ull
stdc_bit_ceil_ul	stdc_has_single_bit_us
stdc_bit_ceil_ull	stdc_leading_ones
stdc_bit_ceil_us	stdc_leading_ones_uc
stdc_bit_floor	stdc_leading_ones_ui
stdc_bit_floor_uc	stdc_leading_ones_ul
stdc_bit_floor_ui	stdc_leading_ones_ull
stdc_bit_floor_ul	stdc_leading_ones_us
stdc_bit_floor_ull	stdc_leading_zeros
stdc_bit_floor_us	stdc_leading_zeros_uc
stdc_bit_width	stdc_leading_zeros_ui
stdc_bit_width_uc	stdc_leading_zeros_ul
stdc_bit_width_ui	stdc_leading_zeros_ull
stdc_bit_width_ul	stdc_leading_zeros_us
stdc_bit_width_ull	stdc_trailing_ones
stdc_bit_width_us	stdc_trailing_ones_uc
stdc_count_ones	stdc_trailing_ones_ui
stdc_count_ones_uc	stdc_trailing_ones_ul
stdc_count_ones_ui	stdc_trailing_ones_ull
stdc_count_ones_ul	stdc_trailing_ones_us
stdc_count_ones_ull	stdc_trailing_zeros
stdc_count_ones_us	stdc_trailing_zeros_uc
stdc_count_zeros	stdc_trailing_zeros_ui
stdc_count_zeros_uc	stdc_trailing_zeros_ul
stdc_count_zeros_ui	stdc_trailing_zeros_ull

stdc_trailing_zeros_us	thrd_join
strcat	thrd_nomem
strcat_s	thrd_sleep
strchr	thrd_start_t
strcmp	thrd_success
strcoll	thrd_t
strcpy	thrd_timedout
strcpy_s	thrd_yield
strcspn	TIME_ACTIVE
strdup	TIME_MONOTONIC
strerror	TIME_THREAD_ACTIVE
strerrorlen_s	TIME_UTC
strerror_s	TMP_MAX
strfromd	tolower
strfromd128	totalorder
strfromd32	totalorderd128
strfromd64	totalorderd32
strfromencf128	totalorderd64
strfromf	totalorderf
strfroml	totalorderl
strftime	totalordermag
strlen	totalordermagd128
strncat	totalordermagd32
strncat_s	totalordermagd64
strncmp	totalordermagf
strncpy	totalordermagl
strncpy_s	toupper
strndup	towctrans
strnlen_s	tolower
strpbrk	toupper
strrchr	tss_create
strspn	tss_delete
strstr	tss_dtor_t
strtod	tss_get
strtod128	tss_set
strtod32	tss_t
strtod64	UCHAR_MAX
strtof	UINT16_C
strtoimax	UINT16_MAX
strtok	uint16_t
strtok_s	UINT16_WIDTH
strtol	UINT32_C
strtold	UINT32_MAX
strtoll	uint32_t
strtoul	UINT32_WIDTH
strtoull	UINT64_C
strtoumax	UINT64_MAX
struct	uint64_t
strxfrm	UINT64_WIDTH
thrd_busy	UINT8_C
thrd_create	UINT8_MAX
thrd_current	uint8_t
thrd_detach	UINT8_WIDTH
thrd_equal	UINTMAX_C
thrd_error	UINTMAX_MAX
thrd_exit	uintmax_t

UINTMAX_WIDTH	wcstoumax
UINTPTR_MAX	wcsxfrm
uintptr_t	WINT_MAX
UINTPTR_WIDTH	WINT_MIN
uint_fast16_t	_Alignas
uint_fast32_t	_Alignof
uint_fast64_t	_Atomic
uint_fast8_t	_BitInt
uint_least16_t	_Bool
uint_least32_t	_Complex
uint_least64_t	_Complex_I
uint_least8_t	_Decimal128
UINT_MAX	_Decimal128x
UINT_WIDTH	_Decimal32
ULLONG_MAX	_Decimal32_t
ULONG_MAX	_Decimal64
USHRT_MAX	_Decimal64x
WCHAR_MAX	_Decimal64_t
WCHAR_MIN	_Exit
wcscat	_Float128
wcscat_s	_Float128x
wcschr	_Float128_t
wcscmp	_Float16
wcscoll	_Float16_t
wcscpy	_Float32
wcscpy_s	_Float32x
wcscspn	_Float32_t
wcsftime	_Float64
wcslen	_Float64x
wcsncat	_Float64_t
wcsncat_s	_Generic
wcsncmp	_Imaginary
wcsncpy	_Imaginary_I
wcsncpy_s	_IOFBF
wcsnlen_s	_IOLBF
wcspbrk	_IONBF
wcsrchr	_Noreturn
wcsrtombs	_Pragma
wcsrtombs_s	_PRINTF_NAN_LEN_MAX
wcsspn	_Static_assert
wcsstr	_Thread_local
wcstod	__bool_true_false_are_defined
wcstod128	__cplusplus
wcstod32	__DATE__
wcstod64	__deprecated__
wcstof	__fallthrough__
wcstoimax	__FILE__
wcstok	__func__
wcstok_s	__has_c_attribute
wcstol	__has_embed
wcstold	__has_include
wcstoll	__if_empty__
wcstombs	__limit__
wcstombs_s	__LINE__
wcstoul	__maybe_unused__
wcstoull	__nodiscard__

__noreturn__	__STDC_VERSION_FLOAT_H__
__reproducible__	__STDC_VERSION_INTTYPES_H__
__STDC_ANALYZABLE__	__STDC_VERSION_LIMITS_H__
__STDC_EMBED_EMPTY__	__STDC_VERSION_MATH_H__
__STDC_EMBED_FOUND__	__STDC_VERSION_SETJMP_H__
__STDC_EMBED_NOT_FOUND__	__STDC_VERSION_STDARG_H__
__STDC_ENDIAN_BIG__	__STDC_VERSION_STDATOMIC_H__
__STDC_ENDIAN_LITTLE__	__STDC_VERSION_STDBIT_H__
__STDC_ENDIAN_NATIVE__	__STDC_VERSION_STDCKDINT_H__
__STDC_HOSTED__	__STDC_VERSION_STDDEF_H__
__STDC_IEC_559_COMPLEX__	__STDC_VERSION_STDINT_H__
__STDC_IEC_559__	__STDC_VERSION_STDIO_H__
__STDC_IEC_60559_BFP__	__STDC_VERSION_STDLIB_H__
__STDC_IEC_60559_COMPLEX__	__STDC_VERSION_STRING_H__
__STDC_IEC_60559_DFP__	__STDC_VERSION_TGMATH_H__
__STDC_IEC_60559_TYPES__	__STDC_VERSION_TIME_H__
__STDC_ISO_10646__	__STDC_VERSION_UCHAR_H__
__STDC_LIB_EXT1__	__STDC_VERSION_WCHAR_H__
__STDC_MB_MIGHT_NEQ_WC__	__STDC_VERSION__
__STDC_NO_ATOMICS__	__STDC_WANT_IEC_60559_EXT__
__STDC_NO_COMPLEX__	__STDC_WANT_IEC_60559_TYPES_EXT__
__STDC_NO_THREADS__	__STDC_WANT_LIB_EXT1__
__STDC_NO_VLA__	__STDC__
__STDC_UTF_16__	__TIME__
__STDC_UTF_32__	__unsequenced__
__STDC_VERSION_ASSERT_H__	__VA_ARGS__
__STDC_VERSION_COMPLEX_H__	__VA_OPT__
__STDC_VERSION_FENV_H__	__Noreturn__

J.6.3 Particular identifiers or keywords

The following

identifiers or keywords are not covered by the previously listed matching patterns and have particular semantics provided by this document.

abort_handler_s	alignas	asinpif
abort	aligned_alloc	asinpil
abs	alignof	asinpi
acosd128	and_eq	asin
acosd32	and	assert
acosd64	asctime_s	atan2d128
acosf	asctime	atan2d32
acoshd128	asind128	atan2d64
acoshd32	asind32	atan2f
acoshd64	asind64	atan2l
acoshf	asinf	atan2pid128
acoshl	asinhd128	atan2pid32
acosh	asinhd32	atan2pid64
acosl	asinhd64	atan2pif
acospid128	asinhf	atan2pil
acospid32	asinh	atan2pi
acospid64	asinh	atan2
acospif	asinl	atand128
acospil	asinpil128	atand32
acospil	asinpil32	atand64
acospi	asinpil64	atanf
acos		

atanhd128	carg	cexp10
atanhd32	case	cexp2f
atanhd64	casinf	cexp2l
atanhf	casinhf	cexp2m1f
atanhl	casinhhl	cexp2m1l
atanh	casinh	cexp2m1
atanl	casinl	cexp2
atanpid128	casinpif	cexpf
atanpid32	casinpil	cexpl
atanpid64	casinpi	cexpm1f
atanpif	casin	cexpm1l
atanpil	catanf	cexpm1
atanpi	catanhf	cexp
atan	catanhhl	char16_t
atexit	catanh	char32_t
atof	catanl	char8_t
atoi	catanpif	CHAR_BIT
atoll	catanpil	CHAR_WIDTH
atol	catanpi	char
at_quick_exit	catan	cimagf
auto	cbrtd128	cimagl
bitand	cbrtd32	cimag
BITINT_MAXWIDTH	cbrtd64	clearerr
bitor	cbrtf	clgammaf
BOOL_WIDTH	cbrtl	clgammal
bool	cbrt	clgamma
break	ccompoundnf	CLOCKS_PER_SEC
bsearch_s	ccompoundnl	clock_t
bsearch	ccompoundn	clock
btowc	ccosf64x	clog10f
BUFSIZ	ccosf	clog10l
c16rtomb	ccoshf	clog10p1f
c32rtomb	ccoshl	clog10p1l
c8rtomb	ccosh	clog10p1
cabsf	ccosl	clog10
cabsl	ccospif	clog1pf
cabs	ccospil	clog1pl
cacosf	ccospi	clog1p
cacoshf	ccos	clog2f
cacoshl	ceild128	clog2l
cacosh	ceild32	clog2p1f
cacosl	ceild64	clog2p1l
cacospif	ceilf	clog2p1
cacospil	ceill	clog2
cacospi	ceil	clogf
cacos	cerfcf	clogl
calloc	cerfcl	clogp1f
call_once	cerfc	clogp1l
canonicalized128	cerff	clogp1
canonicalized32	cerfl	clog
canonicalized64	cerf	CMPLXF
canonicalizef	cexp10f	CMPLXL
canonicalizel	cexp10l	CMPLX
canonicalize	cexp10m1f	complex
cargf	cexp10m1l	compl
cargl	cexp10m1	compoundnd128

compoundnd32	crsqrtrl	d64sqrtd128
compoundnd64	crsqrtr	d64sqrtr
compoundnf	csinf	d64subd128
compoundnl	csinhf	d64sub
compoundn	csinhl	daddl
conjf	csinh	dadd
conjl	csinl	ddivl
conj	csinpif	ddiv
constexpr	csinpil	decimal_point
constraint_handler_t	csinpi	decodebind128
const	csin	decodebind32
continue	csqrtrf	decodebind64
copysignd128	csqrtrl	decodedecd128
copysignd32	csqrtr	decodedecd32
copysignd64	ctanf	decodedecd64
copysignf	ctanhf	DEFAULT
copysignl	ctanhl	defined
copysign	ctanh	define
cosd128	ctanl	deprecated
cosd32	ctanpif	dfmal
cosd64	ctanpil	dfma
cosf	ctanpi	difftime
coshd128	ctan	div_t
coshd32	ctgammaf	div
coshd64	ctgamma	dmull
coshf	ctgamma	dmul
coshl	ctime_s	double_t
cosh	ctime	double
cosl	currency_symbol	do
cospid128	CX_LIMITED_RANGE	dsqrtrl
cospid32	d32addd128	dsqrtr
cospid64	d32addd64	dsubl
cospif	d32add	dsub
cospil	d32divd128	elifdef
cospil	d32divd64	elifndef
cospi	d32div	elif
cos	d32fmad128	else
cpowf128	d32fmad64	embed
cpowf	d32fma	encodebind128
cpowl	d32muld128	encodebind32
cpownf	d32muld64	encodebind64
cpownl	d32mul	encodedecd128
cpown	d32sqrtd128	encodedecd32
cpowrf	d32sqrtd64	encodedecd64
cpowrl	d32sqrtr	endif
cpowr	d32subd128	enum
cpow	d32subd64	erfcd128
cprojf	d32sub	erfcd32
cprojl	d64addd128	erfcd64
cproj	d64add	erfcf
crealf	d64divd128	erfcl
creall	d64div	erfc
creal	d64fmad128	erfd128
crootnf	d64fma	erfd32
crootnl	d64muld128	erfd64
crootn	d64mul	erff
crsqrtrf		

erfl	f64div	floord128
erf	fabsd128	floord32
errno_t	fabsd32	floord64
errno	fabsd64	floorf
error	fabsf	floorl
exit	fabsl	floor
exp10d128	fabs	fmad128
exp10d32	faddl	fmad32
exp10d64	fadd	fmad64
exp10f	fallthrough	fmaf
exp10l	false	fmal
exp10m1d128	fclose	fmaxd128
exp10m1d32	fdimd128	fmaxd32
exp10m1d64	fdimd32	fmaxd64
exp10m1f	fdimd64	fmaxf
exp10m1l	fdimf	fmaximumd128
exp10m1	fdiml	fmaximumd32
exp10	fdim	fmaximumd64
exp2d128	fdivl	fmaximumf
exp2d32	fdiv	fmaximuml
exp2d64	feclearexcept	fmaximum_magd128
exp2f	fegetenv	fmaximum_magd32
exp2l	fegetexceptflag	fmaximum_magd64
exp2m1d128	fegetmode	fmaximum_magf
exp2m1d32	fegetround	fmaximum_magl
exp2m1d64	feholdexcept	fmaximum_mag_numd128
exp2m1f	femode_t	fmaximum_mag_numd32
exp2m1l	FENV_ACCESS	fmaximum_mag_numd64
exp2m1	FENV_DEC_ROUND	fmaximum_mag_numf
exp2	FENV_ROUND	fmaximum_mag_numl
expd128	fenv_t	fmaximum_mag_num
expd32	feof	fmaximum_mag
expd64	feraiseexcept	fmaximum_numd128
expf	ferror	fmaximum_numd32
expl	fesetenv	fmaximum_numd64
expm1d128	fesetexceptflag	fmaximum_numf
expm1d32	fesetexcept	fmaximum_numl
expm1d64	fesetmode	fmaximum_num
expm1f	fesetround	fmaximum
expm1l	fetestexceptflag	fmaxl
expm1	fetestexcept	fmax
exp	feupdateenv	fma
extern	fexcept_t	fmind128
f32addf64x	fe_dec_getround	fmind32
f32addf64	fe_dec_setround	fmind64
f32add	fflush	fminf
f32fmaf32x	ffmal	fminimumd128
f32fma	ffma	fminimumd32
f32mulf128	fgetc	fminimumd64
f32mulf32x	fgetpos	fminimumf
f32mul	fgets	fminimuml
f32xsqrtf54x	fgetwc	fminimum_magd128
f32xsqrtf64x	fgetws	fminimum_magd32
f32xsqrt	FILE	fminimum_magd64
f64divf128	float_t	fminimum_magf
f64divf64x	float	fminimum_magl

fminimum_mag_numd128	fromfpxf	ilogbd32
fminimum_mag_numd32	fromfpxl	ilogbd64
fminimum_mag_numd64	fromfpx	ilogbf
fminimum_mag_numf	fromfp	ilogbl
fminimum_mag_numl	fscanf_s	ilogb
fminimum_mag_num	fscanf	imaginary
fminimum_mag	fseek	imaxabs
fminimum_numd128	fsetpos	imaxdiv_t
fminimum_numd32	fsqrtl	imaxdiv
fminimum_numd64	fsqrt	include
fminimum_numf	fsubl	INFINITY
fminimum_numl	fsub	inline
fminimum_num	ftell	int_curr_symbol
fminimum	fwide	int_frac_digits
fminl	fwprintf_s	int_n_cs_precedes
fmin	fwprintf	int_n_sep_by_space
fmodd128	fwrite	int_n_sign_posn
fmodd32	fwscanf_s	int_p_cs_precedes
fmodd64	fwscanf	int_p_sep_by_space
fmodf	getchar	int_p_sign_posn
fmodl	getc	I
fmod	getenv_s	jmp_buf
fmull	getenv	kill_dependency
fmul	getpayloadd128	labs
fopen_s	getpayloadd32	lconv
fopen	getpayloadd64	ldexpd128
for	getpayloadf	ldexpd32
fpclassify	getpayloadl	ldexpd64
fpos_t	getpayload	ldexpf
fprintf_s	gets_s	ldexpl
fprintf	gets	ldexp
fputc	getwchar	ldiv_t
fputs	getwc	ldiv
fputwc	gmtime_r	lgammad128
fputws	gmtime_s	lgammad32
frac_digits	gmtime	lgammad64
fread	goto	lgammaf
free_aligned_sized	grouping	lgammal
free_sized	HUGE_VALF	lgamma
free	HUGE_VALL	limit
freopen_s	HUGE_VAL_D128	line
freopen	HUGE_VAL_D32	llabs
frexpd128	HUGE_VAL_D64	lldiv_t
frexpd32	HUGE_VAL	lldiv
frexpd64	hypotd128	llogbd128
frexpf	hypotd32	llogbd32
frexpl	hypotd64	llogbd64
frexp	hypotf	llogbf
fromfpxd128	hypotl	llogbl
fromfpxd32	hypot	llogb
fromfpxd64	ifdef	LLONG_WIDTH
fromfpf	ifndef	llquantexpd128
fromfpl	if_empty	llquantexpd32
fromfpxd128	if	llquantexpd64
fromfpxd32	ignore_handler_s	llquantexp
fromfpxd64	ilogbd128	llrintd128

llrintd32	logp1d128	nanf
llrintd64	logp1d32	nanl
llrintf	logp1d64	nan
llrintl	logp1f	NDEBUG
llrint	logp1l	nearbyintd128
llroundd128	logp1	nearbyintd32
llroundd32	log	nearbyintd64
llroundd64	longjmp	nearbyintf
llroundf	long_double_t	nearbyintl
llroundl	LONG_WIDTH	nearbyint
llround	long	negative_sign
localeconv	lrintd128	nextafterd128
localtime_r	lrintd32	nextafterd32
localtime_s	lrintd64	nextafterd64
localtime	lrintf	nextafterf
log10d128	lrintl	nextafterl
log10d32	lrint	nextafter
log10d64	lroundd128	nextdownd128
log10f	lroundd32	nextdownd32
log10l	lroundd64	nextdownd64
log10p1d128	lroundf	nextdownf
log10p1d32	lroundl	nextdownl
log10p1d64	lround	nextdown
log10p1f	L_tmpnam_s	nexttowardd128
log10p1l	L_tmpnam	nexttowardd32
log10p1	main	nexttowardd64
log10	malloc	nexttowardf
log1pd128	math_errhandling	nexttowardl
log1pd32	max_align_t	nexttoward
log1pd64	maybe_unused	nextupd128
log1pf	mblen	nextupd32
log1pl	mbrlen	nextupd64
log1p	mbrtoc16	nextupf
log2d128	mbrtoc32	nextupl
log2d32	mbrtoc8	nextup
log2d64	mbrtowc	nodiscard
log2f	mbsinit	noreturn
log2l	mbsrtowcs_s	not_eq
log2p1d128	mbsrtowcs	not
log2p1d32	mbstate_t	nullptr_t
log2p1d64	mbstowcs_s	nullptr
log2p1f	mbstowcs	NULL
log2p1l	mbtowc	n_cs_precedes
log2p1	mktime	n_sep_by_space
log2	modfd128	n_sign_posn
logbd128	modfd32	offsetof
logbd32	modfd64	OFF
logbd64	modff	ONCE_FLAG_INIT
logbf	modfl	once_flag
logbl	modf	ON
logb	mon_decimal_point	or_eq
logd128	mon_grouping	or
logd32	mon_thousands_sep	perror
logd64	nand128	positive_sign
logf	nand32	powd128
logl	nand64	powd32

powd64	remquo	SCHAR_WIDTH
powf32x	remquo	SEEK_CUR
powf32	rename	SEEK_END
powf64	reproducible	SEEK_SET
powf	restrict	setbuf
powl	return	setjmp
pownd128	rewind	setlocale
pownd32	rintd128	setpayloadd128
pownd64	rintd32	setpayloadd32
pownf	rintd64	setpayloadd64
pownl	rintf	setpayloadf
pown	rintl	setpayloadl
powrd128	rint	setpayloadsigd128
powrd32	rootnd128	setpayloadsigd32
powrd64	rootnd32	setpayloadsigd64
powrf	rootnd64	setpayloadsigf
powrl	rootnf	setpayloadsigl
powr	rootnl	setpayloadsig
pow	rootn	setpayload
pragma	roundd128	setvbuf
prefix	roundd32	set_constraint_handler_s
printf_s	roundd64	short
printf	roundevend128	SHRT_WIDTH
ptrdiff_t	roundevend32	signal
PTRDIFF_WIDTH	roundevend64	signbit
putchar	roundevenf	signed
putc	roundevenl	sig_atomic_t
puts	roundeven	sind128
putwchar	roundf	sind32
putwc	roundl	sind64
p_cs_precedes	round	sinf
p_sep_by_space	rsize_t	sinhd128
p_sign_posn	rsqrd128	sinhd32
qsort_s	rsqrd32	sinhd64
qsort	rsqrd64	sinhf
quantized128	rsqrtf	sinhl
quantized32	rsqrtl	sinh
quantized64	rsqrt	sinl
quantize	samequantumd128	sinpid128
quantumd128	samequantumd32	sinpid32
quantumd32	samequantumd64	sinpid64
quantumd64	samequantum	sinpif
quantum	scalblnd128	sinpil
quick_exit	scalblnd32	sinpi
raise	scalblnd64	sin
rand	scalblnf	sizeof
realloc	scalblnl	size_t
register	scalbln	SIZE_WIDTH
remainderd128	scalbnd128	snprintf_s
remainderd32	scalbnd32	snprintf
remainderd64	scalbnd64	snwprintf_s
remainderf	scalbnf	sprintf_s
remainderl	scalbnl	sprintf
remainder	scalbn	sqrtd128
remove	scanf_s	sqrtd32
remquof	scanf	sqrtd64

<code>sqrtf</code>	<code>TMP_MAX_S</code>	<code>vfscanf</code>
<code>sqrtl</code>	<code>tm_hour</code>	<code>vfwprintf_s</code>
<code>sqrt</code>	<code>tm_isdst</code>	<code>vfwprintf</code>
<code>srand</code>	<code>tm_mday</code>	<code>vfwscanf_s</code>
<code>sscanf_s</code>	<code>tm_min</code>	<code>vfwscanf</code>
<code>sscanf</code>	<code>tm_mon</code>	<code>void</code>
<code>static_assert</code>	<code>tm_sec</code>	<code>volatile</code>
<code>static</code>	<code>tm_wday</code>	<code>vprintf_s</code>
<code>STDC</code>	<code>tm_yday</code>	<code>vprintf</code>
<code>stderr</code>	<code>tm_year</code>	<code>vscanf_s</code>
<code>stdin</code>	<code>tm</code>	<code>vscanf</code>
<code>stdout</code>	<code>true</code>	<code>vsprintf_s</code>
<code>suffix</code>	<code>truncd128</code>	<code>vsprintf</code>
<code>switch</code>	<code>truncd32</code>	<code>vsnwprintf_s</code>
<code>swprintf_s</code>	<code>truncd64</code>	<code>vsprintf_s</code>
<code>swprintf</code>	<code>truncf</code>	<code>vsprintf</code>
<code>swscanf_s</code>	<code>truncl</code>	<code>vsscanf_s</code>
<code>swscanf</code>	<code>trunc</code>	<code>vsscanf</code>
<code>system</code>	<code>TSS_DTOR_ITERATIONS</code>	<code>vswprintf_s</code>
<code>tand128</code>	<code>tv_nsec</code>	<code>vswprintf</code>
<code>tand32</code>	<code>tv_sec</code>	<code>vswscanf_s</code>
<code>tand64</code>	<code>typedef</code>	<code>vswscanf</code>
<code>tanf</code>	<code>typeof_unqual</code>	<code>vwprintf_s</code>
<code>tanh128</code>	<code>typeof</code>	<code>vwprintf</code>
<code>tanh32</code>	<code>UCHAR_WIDTH</code>	<code>vwscanf_s</code>
<code>tanh64</code>	<code>ufromfpd128</code>	<code>vwscanf</code>
<code>tanhf</code>	<code>ufromfpd32</code>	<code>warning</code>
<code>tanh1</code>	<code>ufromfpd64</code>	<code>wchar_t</code>
<code>tanh</code>	<code>ufromfpf</code>	<code>WCHAR_WIDTH</code>
<code>tanl</code>	<code>ufromfpl</code>	<code>wcrtomb_s</code>
<code>tanpid128</code>	<code>ufromfpxd128</code>	<code>wcrtomb</code>
<code>tanpid32</code>	<code>ufromfpxd32</code>	<code>wctob</code>
<code>tanpid64</code>	<code>ufromfpxd64</code>	<code>wctomb_s</code>
<code>tanpif</code>	<code>ufromfpxf</code>	<code>wctomb</code>
<code>tanpil</code>	<code>ufromfpxl</code>	<code>wctrans_t</code>
<code>tanpi</code>	<code>ufromfpx</code>	<code>wctrans</code>
<code>tan</code>	<code>ufromfp</code>	<code>wctype_t</code>
<code>tgamma128</code>	<code>ULLONG_WIDTH</code>	<code>wctype</code>
<code>tgamma32</code>	<code>ULONG_WIDTH</code>	<code>WEOF</code>
<code>tgamma64</code>	<code>undef</code>	<code>while</code>
<code>tgammaf</code>	<code>ungetc</code>	<code>wint_t</code>
<code>tgamma1</code>	<code>ungetwc</code>	<code>WINT_WIDTH</code>
<code>tgamma</code>	<code>union</code>	<code>wmemchr</code>
<code>thousands_sep</code>	<code>unreachable</code>	<code>wmemcmp</code>
<code>thread_local</code>	<code>unsequenced</code>	<code>wmemcpy_s</code>
<code>timegm</code>	<code>unsigned</code>	<code>wmemcpy</code>
<code>timespec_getres</code>	<code>USHRT_WIDTH</code>	<code>wmemmove_s</code>
<code>timespec_get</code>	<code>va_arg</code>	<code>wmemmove</code>
<code>timespec</code>	<code>va_copy</code>	<code>wmemset</code>
<code>time_t</code>	<code>va_end</code>	<code>wprintf_s</code>
<code>time</code>	<code>va_list</code>	<code>wprintf</code>
<code>tmpfile_s</code>	<code>va_start</code>	<code>wscanf_s</code>
<code>tmpfile</code>	<code>vfprintf_s</code>	<code>wscanf</code>
<code>tmpnam_s</code>	<code>vfprintf</code>	<code>xor_eq</code>
<code>tmpnam</code>	<code>vfscanf_s</code>	<code>xor</code>

Annex K

(normative)

Bounds-checking interfaces

K.1 Background

Traditionally, the C Library has contained many functions that trust the programmer to provide output character arrays big enough to hold the result being produced. Not only do these functions not check that the arrays are big enough, they frequently lack the information needed to perform such checks. While it is possible to write safe, robust, and error-free code using the existing library, the library tends to promote programming styles that lead to mysterious failures if a result is too big for the provided array.

A common programming style is to declare character arrays large enough to handle most practical cases. However, if these arrays are not large enough to handle the resulting strings, data can be written past the end of the array overwriting other data and program structures. The program never gets any indication that a problem exists, and so never has a chance to recover or to fail gracefully.

Worse, this style of programming has compromised the security of computers and networks. Buffer overflows can often be exploited to run arbitrary code with the permissions of the vulnerable (defective) program.

If the programmer writes runtime checks to verify lengths before calling library functions, then those runtime checks frequently duplicate work done inside the library functions, which discover string lengths as a side effect of doing their job.

This annex provides alternative library functions that promote safer, more secure programming. The alternative functions verify that output buffers are large enough for the intended result and return a failure indicator if they are not. Data is never written past the end of an array. All string results are null terminated.

This annex also addresses another problem that complicates writing robust code: functions that are not reentrant because they return pointers to static objects owned by the function. Such functions can be troublesome since a previously returned result can change if the function is called again, perhaps by another thread.

K.2 Scope

This annex specifies a series of optional extensions that can be useful in the mitigation of security vulnerabilities in programs, and comprise new functions, macros, and types declared or defined in existing standard headers.

An implementation that defines `__STDC_LIB_EXT1__` shall conform to the specifications in this annex.⁴⁵²⁾

This annex should be read as if it were merged into the parallel structure of named subclauses of Clause 7.

K.3 Library

K.3.1 Introduction

K.3.1.1 Standard headers

The functions, macros, and types declared or defined in this annex and its subclauses are not declared or defined by their respective headers if `__STDC_WANT_LIB_EXT1__` is defined as a macro which expands to the integer constant `0` at the point in the source file where the appropriate header is first included.

The functions, macros, and types declared or defined in this annex and its subclauses are declared and defined by their respective headers if `__STDC_WANT_LIB_EXT1__` is defined as a macro which expands to the integer constant `1` at the point in the source file where the appropriate header is first

⁴⁵²⁾ Implementations that do not define `__STDC_LIB_EXT1__` are not required to conform to these specifications.

included.⁴⁵³⁾

It is implementation-defined whether the functions, macros, and types declared or defined in this annex and its subclauses are declared or defined by their respective headers if `__STDC_WANT_LIB_EXT1__` is not defined as a macro at the point in the source file where the appropriate header is first included.⁴⁵⁴⁾

Within a preprocessing translation unit, `__STDC_WANT_LIB_EXT1__` shall be defined identically for all inclusions of any headers from this annex. If `__STDC_WANT_LIB_EXT1__` is defined differently for any such inclusion, the implementation shall issue a diagnostic as if a preprocessor error directive were used.

K.3.1.2 Reserved identifiers

Each macro name in any of the following subclauses is reserved for use as specified if it is defined by any of its associated headers when included; unless explicitly stated otherwise (see 7.1.4).

All identifiers with external linkage in any of the following subclauses are reserved for use as identifiers with external linkage if any of them are used by the program. None of them are reserved if none of them are used.

Each identifier with file scope listed in any of the following subclauses is reserved for use as a macro name and as an identifier with file scope in the same name space if it is defined by any of its associated headers when included.

K.3.1.3 Use of `errno`

An implementation can set `errno` for the functions defined in this annex, but is not required to.

K.3.1.4 Runtime-constraint violations

Most functions in this annex include as part of their specification a list of runtime-constraints. These runtime-constraints are requirements on the program using the library.⁴⁵⁵⁾

Implementations shall verify that the runtime-constraints for a function are not violated by the program. If a runtime-constraint is violated, the implementation shall call the currently registered runtime-constraint handler (see `set_constraint_handler_s` in `<stdlib.h>`). Multiple runtime-constraint violations in the same call to a library function result in only one call to the runtime-constraint handler. It is unspecified which one of the multiple runtime-constraint violations cause the handler to be called.

If the runtime-constraints section for a function states an action to be performed when a runtime-constraint violation occurs, the function shall perform the action before calling the runtime-constraint handler. If the runtime-constraints section lists actions that are prohibited when a runtime-constraint violation occurs, then such actions are prohibited to the function both before calling the handler and after the handler returns.

The runtime-constraint handler is permitted not to return. If the handler does return, the library function whose runtime-constraint was violated shall return some indication of failure as given by the returns section in the function's specification.

K.3.2 Errors `<errno.h>`

The header `<errno.h>` defines a type.

The type is

<code>errno_t</code>

⁴⁵³⁾ Future revisions of this document can define meanings for other values of `__STDC_WANT_LIB_EXT1__`.

⁴⁵⁴⁾ 7.1.3 reserves certain names and patterns of names that an implementation can use in headers. All other names are not reserved, and a conforming implementation is not permitted to use them. While some of the names defined in this annex and its subclauses are reserved, others are not. If an unreserved name is defined in a header when `__STDC_WANT_LIB_EXT1__` is defined as `0`, the implementation is not conforming.

⁴⁵⁵⁾ Although runtime-constraints replace many cases of undefined behavior, undefined behavior still exists in this annex. Implementations are free to detect any case of undefined behavior and treat it as a runtime-constraint violation by calling the runtime-constraint handler. This license comes directly from the definition of undefined behavior.

which is type `int`.⁴⁵⁶⁾

K.3.3 Common definitions <stddef.h>

The header `<stddef.h>` defines a type.

The type is

```
    rsize_t
```

which is the type `size_t`.⁴⁵⁷⁾

K.3.4 Integer types <stdint.h>

The header `<stdint.h>` defines a macro.

The macro is

```
    RSIZE_MAX
```

which expands to a value of type `size_t`. It can be an expression that is not constant. Functions that have parameters of type `rsize_t` consider it a runtime-constraint violation if the values of those parameters are greater than `RSIZE_MAX`.

Recommended practice

Extremely large object sizes are frequently a sign that an object’s size was calculated incorrectly. For example, negative numbers appear as very large positive numbers when converted to an unsigned type like `size_t`. Also, some implementations do not support objects as large as the maximum value that can be represented by type `size_t`.

For those reasons, it is sometimes beneficial to restrict the range of object sizes to detect programming errors. For implementations targeting machines with large address spaces, it is recommended that `RSIZE_MAX` be defined as the smaller of the size of the largest object supported or `(SIZE_MAX >> 1)`, even if this limit is smaller than the size of some legitimate, but very large, objects. Implementations targeting machines with small address spaces may wish to define `RSIZE_MAX` as `SIZE_MAX`, which means that there is no object size that is considered a runtime-constraint violation.

K.3.5 Input/output <stdio.h>

K.3.5.1 General

The header `<stdio.h>` defines several macros and two types.

The macros are

```
    L_tmpnam_s
```

which expands to an integer constant expression that is the size needed for an array of `char` large enough to hold a temporary file name string generated by the `tmpnam_s` function;

```
    TMP_MAX_S
```

which expands to an integer constant expression that is the maximum number of unique file names that can be generated by the `tmpnam_s` function.

The types are

```
    errno_t
```

which is type `int`; and

⁴⁵⁶⁾As a matter of programming style, `errno_t` can be used as the type of something that deals only with the values that can be found in `errno`. For example, a function which returns the value of `errno` can be declared as having the return type `errno_t`.

⁴⁵⁷⁾See the description of the `RSIZE_MAX` macro in `<stdint.h>`.

rsizet

which is the type `size_t`.

K.3.5.2 Operations on files

K.3.5.2.1 The `tmpfile_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
errno_t tmpfile_s(FILE * restrict * restrict streamptr);
```

Runtime-constraints

`streamptr` shall not be a null pointer.

If there is a runtime-constraint violation, `tmpfile_s` does not attempt to create a file.

Description

The `tmpfile_s` function creates a temporary binary file that is different from any other existing file and that will automatically be removed when it is closed or at program termination. If the program terminates abnormally, whether an open temporary file is removed is implementation-defined. The file is opened for update with "`wb+`" mode with the meaning that mode has in the `fopen_s` function (including the mode's effect on exclusive access and file permissions).

If the file was created successfully, then the pointer to `FILE` pointed to by `streamptr` will be set to the pointer to the object controlling the opened file. Otherwise, the pointer to `FILE` pointed to by `streamptr` will be set to a null pointer.

Recommended practice

It should be possible to open at least `TMP_MAX_S` temporary files during the lifetime of the program (this limit can be shared with `tmpnam_s`) and there should be no limit on the number simultaneously open other than this limit and any limit on the number of open files (`FOPEN_MAX`).

Returns

The `tmpfile_s` function returns zero if it created the file. If it did not create the file or there was a runtime-constraint violation, `tmpfile_s` returns a nonzero value.

K.3.5.2.2 The `tmpnam_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
errno_t tmpnam_s(char *s, rsizet maxsize);
```

Runtime-constraints

`s` shall not be a null pointer. `maxsize` shall be less than or equal to `RSIZE_MAX`. `maxsize` shall be greater than the length of the generated file name string.

Description

The `tmpnam_s` function generates a string that is a valid file name and that is not the same as the name of an existing file.⁴⁵⁸⁾ The function is potentially capable of generating `TMP_MAX_S` different strings, but any or all of them can already be in use by existing files and thus not be suitable return values. The lengths of these strings shall be less than the value of the `L_tmpnam_s` macro.

The `tmpnam_s` function generates a different string each time it is called.

⁴⁵⁸⁾Files created using strings generated by the `tmpnam_s` function are temporary only in the sense that their names are not expected to collide with those generated by conventional naming rules for the implementation. It is still necessary to use the `remove` function to remove such files when their use is ended, and before program termination.

It is assumed that **s** points to an array of at least **maxsize** characters. This array will be set to the generated string, as specified in the rest of this subclause.

The implementation shall behave as if no library function except **tmpnam** calls the **tmpnam_s** function.⁴⁵⁹⁾

Recommended practice

After a program obtains a file name using the **tmpnam_s** function and before the program creates a file with that name, the possibility exists that someone else can create a file with that same name. To avoid this race condition, the **tmpfile_s** function should be used instead of **tmpnam_s** when possible. One situation that requires the use of the **tmpnam_s** function is when the program needs to create a temporary directory rather than a temporary file.

Implementations should take care in choosing the patterns used for names returned by **tmpnam_s**. For example, making a thread ID part of the names avoids the race condition and possible conflict when multiple programs run simultaneously by the same user generate the same temporary file names.

Returns

If no suitable string can be generated, or if there is a runtime-constraint violation, the **tmpnam_s** function:

- if **s** is not null and **maxsize** is both greater than zero and not greater than **RSIZE_MAX**, writes a null character to **s[0]**
- returns a nonzero value.

Otherwise, the **tmpnam_s** function writes the string in the array pointed to by **s** and returns zero.

Environmental limits

The value of the macro **TMP_MAX_S** shall be at least 25.

K.3.5.3 File access functions

K.3.5.3.1 The **fopen_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
errno_t fopen_s(FILE * restrict * restrict streamptr,
                const char * restrict filename, const char * restrict mode);
```

Runtime-constraints

None of **streamptr**, **filename**, or **mode** shall be a null pointer.

If there is a runtime-constraint violation, **fopen_s** does not attempt to open a file. Furthermore, if **streamptr** is not a null pointer, **fopen_s** sets ***streamptr** to the null pointer.

Description

The **fopen_s** function opens the file whose name is the string pointed to by **filename**, and associates a stream with it.

The **mode** string shall be as described for **fopen**, with the addition that modes starting with the character **'w'** or **'a'** can be preceded by the character **'u'**, see the following:

- | | |
|-----|--|
| uw | truncate to zero length or create text file for writing, default permissions |
| uwX | create text file for writing, default permissions |
| ua | append; open or create text file for writing at end-of-file, default permissions |

⁴⁵⁹⁾ An implementation can have **tmpnam** call **tmpnam_s** (perhaps so there is only one naming convention for temporary files), but this is not required.

<code>uwb</code>	truncate to zero length or create binary file for writing, default permissions
<code>uwbx</code>	create binary file for writing, default permissions
<code>uab</code>	append; open or create binary file for writing at end-of-file, default permissions
<code>uw+</code>	truncate to zero length or create text file for update, default permissions
<code>uw+x</code>	create text file for update, default permissions
<code>ua+</code>	append; open or create text file for update, writing at end-of-file, default permissions
<code>uw+b</code> or <code>uwb+</code>	truncate to zero length or create binary file for update, default permissions
<code>uw+bx</code> or <code>uwb+bx</code>	create binary file for update, default permissions
<code>ua+b</code> or <code>uab+</code>	append; open or create binary file for update, writing at end-of-file, default permissions

Opening a file with exclusive mode (‘x’ as the last character in the **mode** argument) fails if the file already exists or cannot be created.

If the file was opened successfully, then the pointer to **FILE** pointed to by **streamptr** will be set to the pointer to the object controlling the opened file. Otherwise, the pointer to **FILE** pointed to by **streamptr** will be set to a null pointer.

Returns

The **fopen_s** function returns zero if it opened the file. If it did not open the file or if there was a runtime-constraint violation, **fopen_s** returns a nonzero value.

K.3.5.3.2 The **freopen_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
errno_t freopen_s(FILE * restrict * restrict newstreamptr,
                  const char * restrict filename, const char * restrict mode,
                  FILE * restrict stream);
```

Runtime-constraints

None of **newstreamptr**, **mode**, and **stream** shall be a null pointer.

If there is a runtime-constraint violation, **freopen_s** neither attempts to close any file associated with **stream** nor attempts to open a file. Furthermore, if **newstreamptr** is not a null pointer, **fopen_s** sets ***newstreamptr** to the null pointer.

Description

The **freopen_s** function opens the file whose name is the string pointed to by **filename** and associates the stream pointed to by **stream** with it. The **mode** argument has the same meaning as in the **fopen_s** function (including the mode’s effect on exclusive access and file permissions).

If **filename** is a null pointer, the **freopen_s** function attempts to change the mode of the stream to that specified by **mode**, as if the name of the file currently associated with the stream had been used. It is implementation-defined which changes of mode are permitted (if any), and under what circumstances.

The **freopen_s** function first attempts to close any file that is associated with **stream**. Failure to close the file is ignored. The error and end-of-file indicators for the stream are cleared.

If the file was opened successfully, then the pointer to **FILE** pointed to by **newstreamptr** will be set to the value of **stream**. Otherwise, the pointer to **FILE** pointed to by **newstreamptr** will be set to a null pointer.

Returns

The **freopen_s** function returns zero if it opened the file. If it did not open the file or there was a runtime-constraint violation, **freopen_s** returns a nonzero value.

K.3.5.4 Formatted input/output functions

K.3.5.4.1 General

Unless explicitly stated otherwise, if the execution of a function described in this subclause causes copying to take place between objects that overlap, the objects take on unspecified values.

K.3.5.4.2 The **fprintf_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
int fprintf_s(FILE * restrict stream, const char * restrict format, ...);
```

Runtime-constraints

Neither **stream** nor **format** shall be a null pointer. The **%n** specifier⁴⁶⁰⁾ (modified or not by flags, field width, or precision) shall not appear in the string pointed to by **format**. Any argument to **fprintf_s** corresponding to a **%s** specifier shall not be a null pointer.

If there is a runtime-constraint violation, the⁴⁶¹⁾ **fprintf_s** function does not attempt to produce further output, and it is unspecified to what extent **fprintf_s** produced output before discovering the runtime-constraint violation.

Description

The **fprintf_s** function is equivalent to the **fprintf** function except for the previously listed explicit runtime-constraints.

Returns

The **fprintf_s** function returns the number of characters transmitted, or a negative value if an output error, encoding error, or runtime-constraint violation occurred.

K.3.5.4.3 The **fscanf_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
int fscanf_s(FILE * restrict stream, const char * restrict format, ...);
```

Runtime-constraints

Neither **stream** nor **format** shall be a null pointer. Any argument indirected though to store converted input shall not be a null pointer.

If there is a runtime-constraint violation, the⁴⁶²⁾ **fscanf_s** function does not attempt to perform further input, and it is unspecified to what extent **fscanf_s** performed input before discovering the runtime-constraint violation.

Description

The **fscanf_s** function is equivalent to **fscanf** except that the **c**, **s**, and **[** conversion specifiers apply to a pair of arguments (unless assignment suppression is indicated by a *****). The first of these arguments is the same as for **fscanf**. That argument is immediately followed in the argument list by the second argument, which has type **rsize_t** and gives the number of elements in the array

⁴⁶⁰⁾It is not a runtime-constraint violation for the characters **%n** to appear in sequence in the string pointed at by **format** when those characters are not a interpreted as a **%n** specifier. For example, if the entire format string was **%%n**.

⁴⁶¹⁾Because an implementation can treat any undefined behavior as a runtime-constraint violation, an implementation can treat any unsupported specifiers in the string pointed to by **format** as a runtime-constraint violation.

⁴⁶²⁾Because an implementation can treat any undefined behavior as a runtime-constraint violation, an implementation can treat any unsupported specifiers in the string pointed to by **format** as a runtime-constraint violation.

pointed to by the first argument of the pair. If the first argument points to a scalar object, it is considered to be an array of one element.⁴⁶³⁾

A matching failure occurs if the number of elements in a receiving object is insufficient to hold the converted input (including any trailing null character).

Returns

The **fscanf_s** function returns the value of the macro **EOF** if an input failure occurs before any conversion or if there is a runtime-constraint violation. Otherwise, the **fscanf_s** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

EXAMPLE 1 The call:

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
/* ... */
int n, i; float x; char name[50];
n = fscanf_s(stdin, "%d%f%s", &i, &x, name, (rsize_t) 50);
```

with the input line:

```
25 54.32E-1 thompson
```

will assign to **n** the value 3, to **i** the value 25, to **x** the value 5.432, and to **name** the sequence thompson\0.

EXAMPLE 2 The call:

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
/* ... */
int n; char s[5];
n = fscanf_s(stdin, "%s", s, sizeof s);
```

with the input line:

```
hello
```

will assign to **n** the value 0 since a matching failure occurred because the sequence hello\0 requires an array of six characters to store it.

K.3.5.4.4 The printf_s function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
int printf_s(const char * restrict format, ...);
```

Runtime-constraints

format shall not be a null pointer. The %n specifier⁴⁶⁴⁾ (modified or not by flags, field width, or precision) shall not appear in the string pointed to by **format**. Any argument to **printf_s** corresponding to a %s specifier shall not be a null pointer.

⁴⁶³⁾ If the format is known at translation time, an implementation can issue a diagnostic for any argument used to store the result from a c, s, or [conversion specifier if that argument is not followed by an argument of a type compatible with **rsize_t**. A limited amount of checking can be done if even if the format is not known at translation time. For example, an implementation can issue a diagnostic for each argument after **format** that has of type pointer to one of **char**, **signed char**, **unsigned char**, or **void** that is not followed by an argument of a type compatible with **rsize_t**. The diagnostic can warn that unless the pointer is being used with a conversion specifier using the hh length modifier, a length argument is expected to follow the pointer argument. Another useful diagnostic can flag any non-pointer argument following **format** that did not have a type compatible with **rsize_t**.

⁴⁶⁴⁾ It is not a runtime-constraint violation for the characters %n to appear in sequence in the string pointed at by **format** when those characters are not interpreted as a %n specifier. For example, if the entire format string was %%n.

If there is a runtime-constraint violation, the `printf_s` function does not attempt to produce further output, and it is unspecified to what extent `printf_s` produced output before discovering the runtime-constraint violation.

Description

The `printf_s` function is equivalent to the `printf` function except for the previously listed explicit runtime-constraints.

Returns

The `printf_s` function returns the number of characters transmitted, or a negative value if an output error, encoding error, or runtime-constraint violation occurred.

K.3.5.4.5 The `scanf_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
int scanf_s(const char * restrict format, ...);
```

Runtime-constraints

format shall not be a null pointer. Any argument indirected though to store converted input shall not be a null pointer.

If there is a runtime-constraint violation, the `scanf_s` function does not attempt to perform further input, and it is unspecified to what extent `scanf_s` performed input before discovering the runtime-constraint violation.

Description

The `scanf_s` function is equivalent to `fscanf_s` with the argument **stdin** interposed before the arguments to `scanf_s`.

Returns

The `scanf_s` function returns the value of the macro **EOF** if an input failure occurs before any conversion or if there is a runtime-constraint violation. Otherwise, the `scanf_s` function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

K.3.5.4.6 The `snprintf_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
int snprintf_s(char * restrict s, rsize_t n, const char * restrict format, ...);
```

Runtime-constraints

Neither **s** nor **format** shall be a null pointer. **n** shall neither equal zero nor be greater than **RSIZE_MAX**. The `%n` specifier⁴⁶⁵⁾ (modified or not by flags, field width, or precision) shall not appear in the string pointed to by **format**. Any argument to `snprintf_s` corresponding to a `%s` specifier shall not be a null pointer. No encoding error shall occur.

If there is a runtime-constraint violation, then if **s** is not a null pointer and **n** is greater than zero and not greater than **RSIZE_MAX**, then the `snprintf_s` function sets **s[0]** to the null character.

Description

The `snprintf_s` function is equivalent to the `snprintf` function except for the previously listed explicit runtime-constraints.

⁴⁶⁵⁾ It is not a runtime-constraint violation for the characters `%n` to appear in sequence in the string pointed at by **format** when those characters are not interpreted as a `%n` specifier. For example, if the entire format string was `%n`.

The **snprintf_s** function, unlike **sprintf_s**, will truncate the result to fit within the array pointed to by **s**.

Returns

The **snprintf_s** function returns the number of characters that would have been written had **n** been sufficiently large, not counting the terminating null character, or a negative value if a runtime-constraint violation occurred. Thus, the null-terminated output has been completely written if and only if the returned value is both nonnegative and less than **n**.

K.3.5.4.7 The **sprintf_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
int sprintf_s(char * restrict s, rsize_t n, const char * restrict format, ...);
```

Runtime-constraints

Neither **s** nor **format** shall be a null pointer. **n** shall neither equal zero nor be greater than **RSIZE_MAX**. The number of characters (including the trailing null) required for the result to be written to the array pointed to by **s** shall not be greater than **n**. The **%n** specifier⁴⁶⁶) (modified or not by flags, field width, or precision) shall not appear in the string pointed to by **format**. Any argument to **sprintf_s** corresponding to a **%s** specifier shall not be a null pointer. No encoding error shall occur.

If there is a runtime-constraint violation, then if **s** is not a null pointer and **n** is greater than zero and not greater than **RSIZE_MAX**, then the **sprintf_s** function sets **s[0]** to the null character.

Description

The **sprintf_s** function is equivalent to the **sprintf** function except for the parameter **n** and the previously listed explicit runtime-constraints.

The **sprintf_s** function, unlike **snprintf_s**, treats a result too big for the array pointed to by **s** as a runtime-constraint violation.

Returns

If no runtime-constraint violation occurred, the **sprintf_s** function returns the number of characters written in the array, not counting the terminating null character. If an encoding error occurred, **sprintf_s** returns a negative value. If any other runtime-constraint violation occurred, **sprintf_s** returns zero.

K.3.5.4.8 The **sscanf_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
int sscanf_s(const char * restrict s, const char * restrict format, ...);
```

Runtime-constraints

Neither **s** nor **format** shall be a null pointer. Any argument indirected though to store converted input shall not be a null pointer.

If there is a runtime-constraint violation, the **sscanf_s** function does not attempt to perform further input, and it is unspecified to what extent **sscanf_s** performed input before discovering the runtime-constraint violation.

Description

The **sscanf_s** function is equivalent to **fscanf_s**, except that input is obtained from a string (specified by the argument **s**) rather than from a stream. Reaching the end of the string is equivalent

⁴⁶⁶) It is not a runtime-constraint violation for the characters **%n** to appear in sequence in the string pointed at by **format** when those characters are not interpreted as a **%n** specifier. For example, if the entire format string was **%%n**.

to encountering end-of-file for the **fscanf_s** function. If copying takes place between objects that overlap, the objects take on unspecified values.

Returns

The **sscanf_s** function returns the value of the macro **EOF** if an input failure occurs before any conversion or if there is a runtime-constraint violation. Otherwise, the **sscanf_s** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

K.3.5.4.9 The **vfprintf_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdarg.h>
#include <stdio.h>
int vfprintf_s(FILE * restrict stream, const char * restrict format, va_list arg)
    ;
```

Runtime-constraints

Neither **stream** nor **format** shall be a null pointer. The **%n** specifier⁴⁶⁷⁾ (modified or not by flags, field width, or precision) shall not appear in the string pointed to by **format**. Any argument to **vfprintf_s** corresponding to a **%s** specifier shall not be a null pointer.

If there is a runtime-constraint violation, the **vfprintf_s** function does not attempt to produce further output, and it is unspecified to what extent **vfprintf_s** produced output before discovering the runtime-constraint violation.

Description

The **vfprintf_s** function is equivalent to the **vfprintf** function except for the previously listed explicit runtime-constraints.

Returns

The **vfprintf_s** function returns the number of characters transmitted, or a negative value if an output error, encoding error, or runtime-constraint violation occurred.

⁴⁶⁷⁾It is not a runtime-constraint violation for the characters **%n** to appear in sequence in the string pointed at by **format** when those characters are not interpreted as a **%n** specifier. For example, if the entire format string was **%n**.

K.3.5.4.10 The `vfscanf_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdarg.h>
#include <stdio.h>
int vfscanf_s(FILE * restrict stream, const char * restrict format, va_list arg);
```

Runtime-constraints

Neither **stream** nor **format** shall be a null pointer. Any argument indirected though to store converted input shall not be a null pointer.

If there is a runtime-constraint violation, the **vfscanf_s** function does not attempt to perform further input, and it is unspecified to what extent **vfscanf_s** performed input before discovering the runtime-constraint violation.

Description

The **vfscanf_s** function is equivalent to **fscanf_s**, with the variable argument list replaced by **arg**, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** invocations). The **vfscanf_s** function does not invoke the **va_end** macro.⁴⁶⁸⁾

Returns

The **vfscanf_s** function returns the value of the macro **EOF** if an input failure occurs before any conversion or if there is a runtime-constraint violation. Otherwise, the **vfscanf_s** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

K.3.5.4.11 The `vprintf_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdarg.h>
#include <stdio.h>
int vprintf_s(const char * restrict format, va_list arg);
```

Runtime-constraints

format shall not be a null pointer. The `%n` specifier⁴⁶⁹⁾ (modified or not by flags, field width, or precision) shall not appear in the string pointed to by **format**. Any argument to **vprintf_s** corresponding to a `%s` specifier shall not be a null pointer.

If there is a runtime-constraint violation, the **vprintf_s** function does not attempt to produce further output, and it is unspecified to what extent **vprintf_s** produced output before discovering the runtime-constraint violation.

Description

The **vprintf_s** function is equivalent to the **vprintf** function except for the previously listed explicit runtime-constraints.

Returns

The **vprintf_s** function returns the number of characters transmitted, or a negative value if an output error, encoding error, or runtime-constraint violation occurred.

K.3.5.4.12 The `vscanf_s` function

⁴⁶⁸⁾As the functions **vfprintf_s**, **vfscanf_s**, **vprintf_s**, **vscanf_s**, **vsprintf_s**, **vsscanf_s** invoke the **va_arg** macro, the representation of **arg** after the return is indeterminate.

⁴⁶⁹⁾It is not a runtime-constraint violation for the characters `%n` to appear in sequence in the string pointed at by **format** when those characters are not interpreted as a `%n` specifier. For example, if the entire format string was `%%n`.

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdarg.h>
#include <stdio.h>
int vscanf_s(const char * restrict format, va_list arg);
```

Runtime-constraints

format shall not be a null pointer. Any argument indirected though to store converted input shall not be a null pointer.

If there is a runtime-constraint violation, the **vscanf_s** function does not attempt to perform further input, and it is unspecified to what extent **vscanf_s** performed input before discovering the runtime-constraint violation.

Description

The **vscanf_s** function is equivalent to **scanf_s**, with the variable argument list replaced by **arg**, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** invocations). The **vscanf_s** function does not invoke the **va_end** macro.⁴⁷⁰⁾

Returns

The **vscanf_s** function returns the value of the macro **EOF** if an input failure occurs before any conversion or if there is a runtime-constraint violation. Otherwise, the **vscanf_s** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

K.3.5.4.13 The vsnprintf_s function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdarg.h>
#include <stdio.h>
int vsnprintf_s(char * restrict s, rsize_t n, const char * restrict format,
va_list arg);
```

Runtime-constraints

Neither **s** nor **format** shall be a null pointer. **n** shall neither equal zero nor be greater than **RSIZE_MAX**. The **%n** specifier⁴⁷¹⁾ (modified or not by flags, field width, or precision) shall not appear in the string pointed to by **format**. Any argument to **vsnprintf_s** corresponding to a **%s** specifier shall not be a null pointer. No encoding error shall occur.

If there is a runtime-constraint violation, then if **s** is not a null pointer and **n** is greater than zero and not greater than **RSIZE_MAX**, then the **vsnprintf_s** function sets **s[0]** to the null character.

Description

The **vsnprintf_s** function is equivalent to the **vsnprintf** function except for the previously listed explicit runtime-constraints.

The **vsnprintf_s** function, unlike **vsnprintf**, will truncate the result to fit within the array pointed to by **s**.

Returns

The **vsnprintf_s** function returns the number of characters that would have been written had **n** been sufficiently large, not counting the terminating null character, or a negative value if a runtime-constraint violation occurred. Thus, the null-terminated output has been completely written if and

⁴⁷⁰⁾As the functions **vfprintf_s**, **vfscanf_s**, **vprintf_s**, **vsprintf_s**, **vsnprintf_s**, **vsnprintf_s**, and **vsscanf_s** invoke the **va_arg** macro, the representation of **arg** after the return is indeterminate.

⁴⁷¹⁾It is not a runtime-constraint violation for the characters **%n** to appear in sequence in the string pointed at by **format** when those characters are not interpreted as a **%n** specifier. For example, if the entire format string was **%n**.

only if the returned value is both nonnegative and less than **n**.

K.3.5.4.14 The `vsprintf_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdarg.h>
#include <stdio.h>
int vsprintf_s(char * restrict s, rsize_t n, const char * restrict format,
               va_list arg);
```

Runtime-constraints

Neither **s** nor **format** shall be a null pointer. **n** shall neither equal zero nor be greater than **RSIZE_MAX**. The number of characters (including the trailing null) required for the result to be written to the array pointed to by **s** shall not be greater than **n**. The `%n` specifier⁴⁷²⁾ (modified or not by flags, field width, or precision) shall not appear in the string pointed to by **format**. Any argument to **vsprintf_s** corresponding to a `%s` specifier shall not be a null pointer. No encoding error shall occur.

If there is a runtime-constraint violation, then if **s** is not a null pointer and **n** is greater than zero and not greater than **RSIZE_MAX**, then the **vsprintf_s** function sets **s[0]** to the null character.

Description

The **vsprintf_s** function is equivalent to the **vsprintf** function except for the parameter **n** and the previously listed explicit runtime-constraints.

The **vsprintf_s** function, unlike **vsnprintf_s**, treats a result too big for the array pointed to by **s** as a runtime-constraint violation.

Returns

If no runtime-constraint violation occurred, the **vsprintf_s** function returns the number of characters written in the array, not counting the terminating null character. If an encoding error occurred, **vsprintf_s** returns a negative value. If any other runtime-constraint violation occurred, **vsprintf_s** returns zero.

K.3.5.4.15 The `vsscanf_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdarg.h>
#include <stdio.h>
int vsscanf_s(const char * restrict s, const char * restrict format, va_list arg)
;
```

Runtime-constraints

Neither **s** nor **format** shall be a null pointer. Any argument indirected though to store converted input shall not be a null pointer.

If there is a runtime-constraint violation, the **vsscanf_s** function does not attempt to perform further input, and it is unspecified to what extent **vsscanf_s** performed input before discovering the runtime-constraint violation.

Description

The **vsscanf_s** function is equivalent to **sscanf_s**, with the variable argument list replaced by **arg**, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vsscanf_s** function does not invoke the **va_end** macro.⁴⁷³⁾

⁴⁷²⁾It is not a runtime-constraint violation for the characters `%n` to appear in sequence in the string pointed at by **format** when those characters are not interpreted as a `%n` specifier. For example, if the entire format string was `%n`.

⁴⁷³⁾As the functions **vfprintf_s**, **vfscanf_s**, **vprintf_s**, **vscanf_s**, **vsnprintf_s**, **vsprintf_s**, and **vsscanf_s** invoke the **va_arg** macro, the value of **arg** after the return is indeterminate.

Returns

The **vsscanf_s** function returns the value of the macro **EOF** if an input failure occurs before any conversion or if there is a runtime-constraint violation. Otherwise, the **vscanf_s** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

K.3.5.5 Character input/output functions

K.3.5.5.1 The **gets_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
char *gets_s(char *s, rsize_t n);
```

Runtime-constraints

s shall not be a null pointer. **n** shall neither be equal to zero nor be greater than **RSIZE_MAX**. A new-line character, end-of-file, or read error shall occur within reading **n-1** characters from **stdin**.⁴⁷⁴⁾

If there is a runtime-constraint violation, characters are read and discarded from **stdin** until a new-line character is read, or end-of-file or a read error occurs, and if **s** is not a null pointer, **s[0]** is set to the null character.

Description

The **gets_s** function reads at most one less than the number of characters specified by **n** from the stream pointed to by **stdin**, into the array pointed to by **s**. No additional characters are read after a new-line character (which is discarded) or after end-of-file. The discarded new-line character does not count towards number of characters read. A null character is written immediately after the last character read into the array.

If end-of-file is encountered and no characters have been read into the array, or if a read error occurs during the operation, then **s[0]** is set to the null character, and the other elements of **s** take unspecified values.

Recommended practice

The **fgets** function allows properly-written programs to safely process input lines too long to store in the result array. In general this requires that callers of **fgets** pay attention to the presence or absence of a new-line character in the result array. It is recommended to use **fgets** (along with any needed processing based on new-line characters) instead of **gets_s**.

Returns

The **gets_s** function returns **s** if successful. If there was a runtime-constraint violation, or if end-of-file is encountered and no characters have been read into the array, or if a read error occurs during the operation, then a null pointer is returned.

⁴⁷⁴⁾The **gets_s** function, unlike the historical **gets** function, makes it a runtime-constraint violation for a line of input to overflow the buffer to store it. Unlike the **fgets** function, **gets_s** maintains a one-to-one relationship between input lines and successful calls to **gets_s**. Programs that use **gets** expect such a relationship.

K.3.6 General utilities <stdlib.h>

K.3.6.1 General

The header <stdlib.h> defines three types.

The types are

errno_t

which is type **int**; and

rsize_t

which is the type **size_t**; and

constraint_handler_t

which has the following definition

```
typedef void (*constraint_handler_t)(  
    const char * restrict msg,  
    void * restrict ptr,  
    errno_t error);
```

K.3.6.2 Runtime-constraint handling

K.3.6.2.1 The **set_constraint_handler_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1  
#include <stdlib.h>  
constraint_handler_t set_constraint_handler_s(constraint_handler_t handler);
```

Description

The **set_constraint_handler_s** function sets the runtime-constraint handler to be **handler**. The runtime-constraint handler is the function to be called when a library function detects a runtime-constraint violation. Only the most recent handler registered with **set_constraint_handler_s** is called when a runtime-constraint violation occurs.

When the handler is called, it is passed the following arguments in the following order:

1. A pointer to a character string describing the runtime-constraint violation.
2. A null pointer or a pointer to an implementation-defined object.
3. If the function calling the handler has a return type declared as **errno_t**, the return value of the function is passed. Otherwise, a positive value of type **errno_t** is passed.

The implementation has a default constraint handler that is used if no calls to the **set_constraint_handler_s** function have been made. The behavior of the default handler is implementation-defined, and it can cause the program to exit or abort.

If the **handler** argument to **set_constraint_handler_s** is a null pointer, the implementation default handler becomes the current constraint handler.

Returns

The **set_constraint_handler_s** function returns a pointer to the previously registered handler.⁴⁷⁵⁾

⁴⁷⁵⁾ If the previous handler was registered by calling **set_constraint_handler_s** with a null pointer argument, a pointer to the implementation default handler is returned (not null).

K.3.6.2.2 The `abort_handler_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdlib.h>
void abort_handler_s(const char * restrict msg, void * restrict ptr,
                    errno_t error);
```

Description

A pointer to the `abort_handler_s` function shall be a suitable argument to the `set_constraint_handler_s` function.

The `abort_handler_s` function writes a message on the standard error stream in an implementation-defined format. The message shall include the string pointed to by `msg`. The `abort_handler_s` function then calls the `abort` function.⁴⁷⁶⁾

Returns

The `abort_handler_s` function does not return to its caller.

K.3.6.2.3 The `ignore_handler_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdlib.h>
void ignore_handler_s(const char * restrict msg, void * restrict ptr,
                    errno_t error);
```

Description

A pointer to the `ignore_handler_s` function shall be a suitable argument to the `set_constraint_handler_s` function.

The `ignore_handler_s` function simply returns to its caller.⁴⁷⁷⁾

Returns

The `ignore_handler_s` function returns no value.

K.3.6.3 Communication with the environment

K.3.6.3.1 The `getenv_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdlib.h>
errno_t getenv_s(size_t * restrict len, char * restrict value, rsize_t maxsize,
                const char * restrict name);
```

Runtime-constraints

`name` shall not be a null pointer. `maxsize` shall not be greater than `RSIZE_MAX`. If `maxsize` is not equal to zero, then `value` shall not be a null pointer.

⁴⁷⁶⁾Many implementations invoke a debugger when the `abort` function is called.
⁴⁷⁷⁾If the runtime-constraint handler is set to the `ignore_handler_s` function, any library function in which a runtime-constraint violation occurs will return to its caller. The caller can determine whether a runtime-constraint violation occurred based on the library function's specification (usually, the library function returns a nonzero `errno_t`).

If there is a runtime-constraint violation, the integer pointed to by **len** is set to 0 (if **len** is not null), and the environment list is not searched.

Description

The **getenv_s** function searches an *environment list*, provided by the host environment, for a string that matches the string pointed to by **name**.

If that name is found then **getenv_s** performs the following actions. If **len** is not a null pointer, the length of the string associated with the matched list member is stored in the integer pointed to by **len**. If the length of the associated string is less than **maxsize**, then the associated string is copied to the array pointed to by **value**.

If that name is not found then **getenv_s** performs the following actions. If **len** is not a null pointer, zero is stored in the integer pointed to by **len**. If **maxsize** is greater than zero, then **value[0]** is set to the null character.

The set of environment names and the method for altering the environment list are implementation-defined. The **getenv_s** function is not required to avoid data races with other threads of execution that modify the environment list.⁴⁷⁸⁾

Returns

The **getenv_s** function returns zero if the specified **name** is found and the associated string was successfully stored in **value**. Otherwise, a nonzero value is returned.

K.3.6.4 Searching and sorting utilities

K.3.6.4.1 General

These utilities make use of a comparison function to search or sort arrays of unspecified type. Where an argument declared as **size_t nmemb** specifies the length of the array for a function, if **nmemb** has the value zero on a call to that function, then the comparison function is not called, a search finds no matching element, sorting performs no rearrangement, and the pointer to the array can be null.

The implementation shall ensure that the second argument of the comparison function (when called from **bsearch_s**), or both arguments (when called from **qsort_s**), are pointers to elements of the array.⁴⁷⁹⁾ The first argument when called from **bsearch_s** shall equal **key**.

The comparison function shall not alter the contents of either the array or search key. The implementation may reorder elements of the array between calls to the comparison function, but shall not otherwise alter the contents of any individual element.

When the same objects (consisting of **size** bytes, irrespective of their current positions in the array) are passed more than once to the comparison function, the results shall be consistent with one another. That is, for **qsort_s** they shall define a total ordering on the array, and for **bsearch_s** the same object shall always compare the same way with the key.

A sequence point occurs immediately before and immediately after each call to the comparison function, and also between any call to the comparison function and any movement of the objects passed as arguments to that call.

K.3.6.4.2 The **bsearch_s** generic function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdlib.h>
QVoid *bsearch_s(const void *key, QVoid *base, rsize_t nmemb, rsize_t size,
                 int (*compar)(const void *k, const void *y, void *context),
```

⁴⁷⁸⁾Many implementations provide non-standard functions that modify the environment list.

⁴⁷⁹⁾That is, if the value passed is **p**, then the following expressions are always valid and nonzero:

```
((char *)p - (char *)base) % size == 0
(char *)p >= (char *)base
(char *)p < (char *)base + nmemb * size
```



```
void *context);
```

Runtime-constraints

Neither **nmemb** nor **size** shall be greater than **RSIZE_MAX**. If **nmemb** is not equal to zero, then none of **key**, **base**, or **compar** shall be a null pointer.

If there is a runtime-constraint violation, the **bsearch_s** generic function does not search the array.

Description

The **bsearch_s** generic function searches an array of **nmemb** objects, the initial element of which is pointed to by **base**, for an element that matches the object pointed to by **key**. The size of each element of the array is specified by **size**.

The comparison function pointed to by **compar** is called with three arguments. The first two point to the **key** object and to an array element, in that order. The function shall return an integer less than, equal to, or greater than zero if the **key** object is considered, respectively, to be less than, to match, or to be greater than the array element. The array shall consist of: all the elements that compare less than, all the elements that compare equal to, and all the elements that compare greater than the **key** object, in that order.⁴⁸⁰⁾ The third argument to the comparison function is the **context** argument passed to **bsearch_s**. The sole use of **context** by **bsearch_s** is to pass it to the comparison function.⁴⁸¹⁾

Returns

The **bsearch_s** generic function returns a pointer to a matching element of the array, or a null pointer if no match is found or there is a runtime-constraint violation. If two elements compare as equal, which element is matched is unspecified.

The **bsearch_s** generic function is generic in the qualification of the type pointed to by the argument **base**. If this argument is a pointer to a **const**-qualified object type, the returned pointer will be a pointer to **const**-qualified **void**. Otherwise, the argument shall be a pointer to an unqualified object type or a null pointer constant,⁴⁸²⁾ and the returned pointer will be a pointer to unqualified **void**.

The external declaration of **bsearch_s** has the concrete type:

```
void * (const void *, const void *, rsize_t, rsize_t,
        int (*) (const void *, const void *), void *)
```

which supports all correct uses. If a macro definition of the generic function is suppressed to access an actual function, the external declaration with this concrete type is visible.⁴⁸³⁾

K.3.6.4.3 The **qsort_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdlib.h>
errno_t qsort_s(void *base, rsize_t nmemb, rsize_t size,
                int (*compar)(const void *, const void *), void *context),
                void *context);
```

Runtime-constraints

Neither **nmemb** nor **size** shall be greater than **RSIZE_MAX**. If **nmemb** is not equal to zero, then neither **base** nor **compar** shall be a null pointer.

If there is a runtime-constraint violation, the **qsort_s** function does not sort the array.

⁴⁸⁰⁾In practice, this means that the entire array has been sorted according to the comparison function.
⁴⁸¹⁾The **context** argument is for the use of the comparison function in performing its duties. For example, it can specify a collating sequence used by the comparison function.
⁴⁸²⁾If the argument is a null pointer and the call is executed, the behavior is undefined.
⁴⁸³⁾This is an obsolescent feature.

Description

The **qsort_s** function sorts an array of **nmemb** objects, the initial element of which is pointed to by **base**. The size of each object is specified by **size**.

The contents of the array are sorted into ascending order according to a comparison function pointed to by **compar**, which is called with three arguments. The first two point to the objects being compared. The function shall return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second. The third argument to the comparison function is the **context** argument passed to **qsort_s**. The sole use of **context** by **qsort_s** is to pass it to the comparison function.⁴⁸⁴⁾

If two elements compare as equal, their relative order in the resulting sorted array is unspecified.

Returns

The **qsort_s** function returns zero if there was no runtime-constraint violation. Otherwise, a nonzero value is returned.

K.3.6.5 Multibyte/wide character conversion functions

K.3.6.5.1 General

The behavior of the multibyte character functions is affected by the **LC_CTYPE** category of the current locale. For a state-dependent encoding, each function is placed into its initial conversion state by a call for which its character pointer argument, **s**, is a null pointer. Subsequent calls with **s** as other than a null pointer cause the internal conversion state of the function to be altered as necessary. A call with **s** as a null pointer causes these functions to set the **int** pointed to by their **status** argument to a nonzero value if encodings have state dependency, and zero otherwise.⁴⁸⁵⁾

Changing the **LC_CTYPE** category causes the internal object describing the conversion state of these functions to have an indeterminate representation.

K.3.6.5.2 The **wctomb_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdlib.h>
errno_t wctomb_s(int * restrict status, char * restrict s, rsize_t smax,
                wchar_t wc);
```

Runtime-constraints

Let n denote the number of bytes needed to represent the multibyte character corresponding to the wide character given by **wc** (including any shift sequences).

If **s** is not a null pointer, then **smax** shall not be less than n , and **smax** shall not be greater than **RSIZE_MAX**. If **s** is a null pointer, then **smax** shall equal zero.

If there is a runtime-constraint violation, **wctomb_s** does not modify the **int** pointed to by **status**, and if **s** is not a null pointer, no more than **smax** elements in the array pointed to by **s** will be accessed.

Description

The **wctomb_s** function determines n and stores the multibyte character representation of **wc** in the array whose first element is pointed to by **s** (if **s** is not a null pointer). The number of characters stored never exceeds **MB_CUR_MAX** or **smax**. If **wc** is a null wide character, a null byte is stored, preceded by any shift sequence needed to restore the initial shift state, and the function is left in the initial conversion state.

The implementation shall behave as if no library function calls the **wctomb_s** function.

⁴⁸⁴⁾The **context** argument is for the use of the comparison function in performing its duties. For example, it can specify a collating sequence used by the comparison function.

⁴⁸⁵⁾If the locale employs special bytes to change the shift state, these bytes do not produce separate wide character codes, but are grouped with an adjacent multibyte character.

If **s** is a null pointer, the **wctomb_s** function stores into the **int** pointed to by **status** a nonzero or zero value, if multibyte character encodings, respectively, do or do not have state-dependent encodings.

If **s** is not a null pointer, the **wctomb_s** function stores into the **int** pointed to by **status** either *n* or -1 if **wc**, respectively, does or does not correspond to a valid multibyte character.

In no case will the **int** pointed to by **status** be set to a value greater than the **MB_CUR_MAX** macro.

Returns

The **wctomb_s** function returns zero if successful, and a nonzero value if there was a runtime-constraint violation or **wc** did not correspond to a valid multibyte character.

K.3.6.6 Multibyte/wide string conversion functions

K.3.6.6.1 General

The behavior of the multibyte string functions is affected by the **LC_CTYPE** category of the current locale.

K.3.6.6.2 The **mbstowcs_s** function

Synopsis

```
#include <stdlib.h>
errno_t mbstowcs_s(size_t * restrict retval, wchar_t * restrict dst,
    rsize_t dstmax, const char * restrict src, rsize_t len);
```

Runtime-constraints

Neither **retval** nor **src** shall be a null pointer. If **dst** is not a null pointer, then neither **len** nor **dstmax** shall be greater than

RSIZE_MAX/sizeof(wchar_t). If **dst** is a null pointer, then **dstmax** shall equal zero. If **dst** is not a null pointer, then **dstmax** shall not equal zero. If **dst** is not a null pointer and **len** is not less than **dstmax**, then a null character shall occur within the first **dstmax** multibyte characters of the array pointed to by **src**.

If there is a runtime-constraint violation, then **mbstowcs_s** does the following. If **retval** is not a null pointer, then **mbstowcs_s** sets ***retval** to **(size_t)(-1)**. If **dst** is not a null pointer and **dstmax** is greater than zero and not greater than **RSIZE_MAX/sizeof(wchar_t)**, then **mbstowcs_s** sets **dst[0]** to the null wide character.

Description

The **mbstowcs_s** function converts a sequence of multibyte characters that begins in the initial shift state from the array pointed to by **src** into a sequence of corresponding wide characters. If **dst** is not a null pointer, the converted characters are stored into the array pointed to by **dst**. Conversion continues up to and including a terminating null character, which is also stored. Conversion stops earlier in two cases: when a sequence of bytes is encountered that does not form a valid multibyte character, or (if **dst** is not a null pointer) when **len** wide characters have been stored into the array pointed to by **dst**.⁴⁸⁶⁾ If **dst** is not a null pointer and no null wide character was stored into the array pointed to by **dst**, then **dst[len]** is set to the null wide character. Each conversion takes place as if by a call to the **mbrtowc** function.

Regardless of whether **dst** is or is not a null pointer, if the input conversion encounters a sequence of bytes that do not form a valid multibyte character, an encoding error occurs: the **mbstowcs_s** function stores the value **(size_t)(-1)** into ***retval**. Otherwise, the **mbstowcs_s** function stores into ***retval** the number of multibyte characters successfully converted, not including the terminating null character (if any).

All elements following the terminating null wide character (if any) written by **mbstowcs_s** in the array of **dstmax** wide characters pointed to by **dst** take unspecified values when **mbstowcs_s**

⁴⁸⁶⁾Thus, the value of **len** is ignored if **dst** is a null pointer.

returns.⁴⁸⁷⁾

If copying takes place between objects that overlap, the objects take on unspecified values.

Returns

The **mbstowcs_s** function returns zero if no runtime-constraint violation and no encoding error occurred. Otherwise, a nonzero value is returned.

K.3.6.6.3 The **wcstombs_s** function

Synopsis

```
#include <stdlib.h>
errno_t wcstombs_s(size_t * restrict retval, char * restrict dst, rsize_t dstmax,
                  const wchar_t * restrict src, rsize_t len);
```

Runtime-constraints

Neither **retval** nor **src** shall be a null pointer. If **dst** is not a null pointer, then **len** shall not be greater than **RSIZE_MAX/sizeof(wchar_t)** and **dstmax** shall be nonzero and not greater than **RSIZE_MAX**. If **dst** is a null pointer, then **dstmax** shall equal zero. If **dst** is not a null pointer and **len** is not less than **dstmax**, then the conversion shall have been stopped (see the following) because a terminating null wide character was reached or because an encoding error occurred.

If there is a runtime-constraint violation, then **wcstombs_s** does the following. If **retval** is not a null pointer, then **wcstombs_s** sets ***retval** to **(size_t)(-1)**. If **dst** is not a null pointer and **dstmax** is greater than zero and not greater than **RSIZE_MAX**, then **wcstombs_s** sets **dst[0]** to the null character.

Description

The **wcstombs_s** function converts a sequence of wide characters from the array pointed to by **src** into a sequence of corresponding multibyte characters that begins in the initial shift state. If **dst** is not a null pointer, the converted characters are then stored into the array pointed to by **dst**. Conversion continues up to and including a terminating null wide character, which is also stored. Conversion stops earlier in two cases:

- when a wide character is reached that does not correspond to a valid multibyte character;
- (if **dst** is not a null pointer) when the next multibyte character would exceed the limit of *n* total bytes to be stored into the array pointed to by **dst**. If the wide character being converted is the null wide character, then *n* is the lesser of **len** or **dstmax**. Otherwise, *n* is the lesser of **len** or **dstmax-1**.

If the conversion stops without converting a null wide character and **dst** is not a null pointer, then a null character is stored into the array pointed to by **dst** immediately following any multibyte characters already stored. Each conversion takes place as if by a call to the **wcrtomb** function.⁴⁸⁸⁾

Regardless of whether **dst** is or is not a null pointer, if the input conversion encounters a wide character that does not correspond to a valid multibyte character, an encoding error occurs: the **wcstombs_s** function stores the value **(size_t)(-1)** into ***retval**. Otherwise, the **wcstombs_s** function stores into ***retval** the number of bytes in the resulting multibyte character sequence, not including the terminating null character (if any).

All elements following the terminating null character (if any) written by **wcstombs_s** in the array of **dstmax** elements pointed to by **dst** take unspecified values when **wcstombs_s** returns.⁴⁸⁹⁾

If copying takes place between objects that overlap, the objects take on unspecified values.

⁴⁸⁷⁾This allows an implementation to attempt converting the multibyte string before discovering a terminating null character did not occur where required.

⁴⁸⁸⁾If conversion stops because a terminating null wide character has been reached, the bytes stored include those necessary to reach the initial shift state immediately before the null byte. However, if the conversion stops before a terminating null wide character has been reached, the result will be null terminated, but potentially not end in the initial shift state.

⁴⁸⁹⁾When **len** is not less than **dstmax**, the implementation can fill the array before discovering a runtime-constraint violation.

Returns

The `wcstombs_s` function returns zero if no runtime-constraint violation and no encoding error occurred. Otherwise, a nonzero value is returned.

K.3.7 String handling <string.h>

K.3.7.1 General

The header <string.h> defines two types.

The types are

```
errno_t
```

which is type `int`; and

```
rsize_t
```

which is the type `size_t`.

K.3.7.2 Copying functions

K.3.7.2.1 The `memcpy_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <string.h>
errno_t memcpy_s(void * restrict s1, rsize_t s1max, const void * restrict s2,
                 rsize_t n);
```

Runtime-constraints

Neither `s1` nor `s2` shall be a null pointer. Neither `s1max` nor `n` shall be greater than `RSIZE_MAX`. `n` shall not be greater than `s1max`. Copying shall not take place between objects that overlap.

If there is a runtime-constraint violation, the `memcpy_s` function stores zeros in the first `s1max` characters of the object pointed to by `s1` if `s1` is not a null pointer and `s1max` is not greater than `RSIZE_MAX`.

Description

The `memcpy_s` function copies `n` characters from the object pointed to by `s2` into the object pointed to by `s1`.

Returns

The `memcpy_s` function returns zero if there was no runtime-constraint violation. Otherwise, a nonzero value is returned.

K.3.7.2.2 The `memmove_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <string.h>
errno_t memmove_s(void *s1, rsize_t s1max, const void *s2, rsize_t n);
```

Runtime-constraints

Neither `s1` nor `s2` shall be a null pointer. Neither `s1max` nor `n` shall be greater than `RSIZE_MAX`. `n` shall not be greater than `s1max`.

If there is a runtime-constraint violation, the `memmove_s` function stores zeros in the first `s1max` characters of the object pointed to by `s1` if `s1` is not a null pointer and `s1max` is not greater than `RSIZE_MAX`.

Description

The **memmove_s** function copies **n** characters from the object pointed to by **s2** into the object pointed to by **s1**. This copying takes place as if the **n** characters from the object pointed to by **s2** are first copied into a temporary array of **n** characters that does not overlap the objects pointed to by **s1** or **s2**, and then the **n** characters from the temporary array are copied into the object pointed to by **s1**.

Returns

The **memmove_s** function returns zero if there was no runtime-constraint violation. Otherwise, a nonzero value is returned.

K.3.7.2.3 The strcpy_s function**Synopsis**

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <string.h>
errno_t strcpy_s(char * restrict s1, rsize_t s1max, const char * restrict s2);
```

Runtime-constraints

Neither **s1** nor **s2** shall be a null pointer. **s1max** shall not be greater than **RSIZE_MAX**. **s1max** shall not equal zero. **s1max** shall be greater than **strnlen_s(s2, s1max)**. Copying shall not take place between objects that overlap.

If there is a runtime-constraint violation, then if **s1** is not a null pointer and **s1max** is greater than zero and not greater than **RSIZE_MAX**, then **strcpy_s** sets **s1[0]** to the null character.

Description

The **strcpy_s** function copies the string pointed to by **s2** (including the terminating null character) into the array pointed to by **s1**.

All elements following the terminating null character (if any) written by **strcpy_s** in the array of **s1max** characters pointed to by **s1** take unspecified values when **strcpy_s** returns.⁴⁹⁰⁾

Returns

The **strcpy_s** function returns zero⁴⁹¹⁾ if there was no runtime-constraint violation. Otherwise, a nonzero value is returned.

K.3.7.2.4 The strncpy_s function**Synopsis**

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <string.h>
errno_t strncpy_s(char * restrict s1, rsize_t s1max, const char * restrict s2,
    rsize_t n);
```

Runtime-constraints

Neither **s1** nor **s2** shall be a null pointer. Neither **s1max** nor **n** shall be greater than **RSIZE_MAX**. **s1max** shall not equal zero. If **n** is not less than **s1max**, then **s1max** shall be greater than **strnlen_s(s2, s1max)**. Copying shall not take place between objects that overlap.

If there is a runtime-constraint violation, then if **s1** is not a null pointer and **s1max** is greater than zero and not greater than **RSIZE_MAX**, then **strncpy_s** sets **s1[0]** to the null character.

⁴⁹⁰⁾ This allows an implementation to copy characters from **s2** to **s1** while simultaneously checking if any of those characters are null. Such an approach can write a character to every element of **s1** before discovering that the first element was set to the null character.

⁴⁹¹⁾ A zero return value implies that all the requested characters from the string pointed to by **s2** fit within the array pointed to by **s1** and that the result in **s1** is null terminated.

Description

The `strncpy_s` function copies not more than `n` successive characters (characters that follow a null character are not copied) from the array pointed to by `s2` to the array pointed to by `s1`. If no null character was copied from `s2`, then `s1[n]` is set to a null character.

All elements following the terminating null character (if any) written by `strncpy_s` in the array of `s1max` characters pointed to by `s1` take unspecified values when `strncpy_s` returns a nonzero value.⁴⁹²⁾

Returns

The `strncpy_s` function returns zero⁴⁹³⁾ if there was no runtime-constraint violation. Otherwise, a nonzero value is returned.

EXAMPLE The `strncpy_s` function can be used to copy a string without the danger that the result will not be null terminated or that characters will be written past the end of the destination array.

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <string.h>
/* ... */
char src1[100] = "hello";
char src2[7] = {'g', 'o', 'o', 'd', ' ', 'y', 'e'};
char dst1[6], dst2[5], dst3[5];
int r1, r2, r3;
r1 = strncpy_s(dst1, 6, src1, 100);
r2 = strncpy_s(dst2, 5, src2, 7);
r3 = strncpy_s(dst3, 5, src2, 4);
```

The first call will assign to `r1` the value zero and to `dst1` the sequence `hello\0`.

The second call will assign to `r2` a nonzero value and to `dst2` the sequence `\0`.

The third call will assign to `r3` the value zero and to `dst3` the sequence `good\0`.

K.3.7.3 Concatenation functions

K.3.7.3.1 The `strcat_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <string.h>
errno_t strcat_s(char * restrict s1, rsize_t s1max, const char * restrict s2);
```

Runtime-constraints

Let `m` denote the value `s1max - strlen_s(s1, s1max)` upon entry to `strcat_s`.

Neither `s1` nor `s2` shall be a null pointer. `s1max` shall not be greater than `RSIZE_MAX`. `s1max` shall not equal zero. `m` shall not equal zero.⁴⁹⁴⁾ `m` shall be greater than `strlen_s(s2, m)`. Copying shall not take place between objects that overlap.

If there is a runtime-constraint violation, then if `s1` is not a null pointer and `s1max` is greater than zero and not greater than `RSIZE_MAX`, then `strcat_s` sets `s1[0]` to the null character.

Description

The `strcat_s` function appends a copy of the string pointed to by `s2` (including the terminating null character) to the end of the string pointed to by `s1`. The initial character from `s2` overwrites the null character at the end of `s1`.

⁴⁹²⁾This allows an implementation to copy characters from `s2` to `s1` while simultaneously checking if any of those characters are null. Such an approach can write a character to every element of `s1` before discovering that the first element was set to the null character.

⁴⁹³⁾A zero return value implies that all of the requested characters from the string pointed to by `s2` fit within the array pointed to by `s1` and that the result in `s1` is null terminated.

⁴⁹⁴⁾Zero means that `s1` was not null terminated upon entry to `strcat_s`.

All elements following the terminating null character (if any) written by **strcat_s** in the array of **s1max** characters pointed to by **s1** take unspecified values when **strcat_s** returns.⁴⁹⁵⁾

Returns

The **strcat_s** function returns zero⁴⁹⁶⁾ if there was no runtime-constraint violation. Otherwise, a nonzero value is returned.

K.3.7.3.2 The **strncat_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <string.h>
errno_t strncat_s(char * restrict s1, rsize_t s1max, const char * restrict s2,
                 rsize_t n);
```

Runtime-constraints

Let m denote the value **s1max** - **strnlen_s(s1, s1max)** upon entry to **strncat_s**.

Neither **s1** nor **s2** shall be a null pointer. Neither **s1max** nor **n** shall be greater than **RSIZE_MAX**. **s1max** shall not equal zero. m shall not equal zero.⁴⁹⁷⁾ If **n** is not less than m , then m shall be greater than **strnlen_s(s2, m)**. Copying shall not take place between objects that overlap.

If there is a runtime-constraint violation, then if **s1** is not a null pointer and **s1max** is greater than zero and not greater than **RSIZE_MAX**, then **strncat_s** sets **s1[0]** to the null character.

Description

The **strncat_s** function appends not more than **n** successive characters (characters that follow a null character are not copied) from the array pointed to by **s2** to the end of the string pointed to by **s1**. The initial character from **s2** overwrites the null character at the end of **s1**. If no null character was copied from **s2**, then **s1[s1max - m + n]** is set to a null character.

All elements following the terminating null character (if any) written by **strncat_s** in the array of **s1max** characters pointed to by **s1** take unspecified values when **strncat_s** returns.⁴⁹⁸⁾

Returns

The **strncat_s** function returns zero⁴⁹⁹⁾ if there was no runtime-constraint violation. Otherwise, a nonzero value is returned.

EXAMPLE The **strncat_s** function can be used to copy a string without the danger that the result will not be null terminated or that characters will be written past the end of the destination array.

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <string.h>
/* ... */
char s1[100] = "good";
char s2[6] = "hello";
char s3[6] = "hello";
char s4[7] = "abc";
char s5[1000] = "bye";
int r1, r2, r3, r4;
r1 = strncat_s(s1, 100, s5, 1000);
```

⁴⁹⁵⁾This allows an implementation to append characters from **s2** to **s1** while simultaneously checking if any of those characters are null. Such an approach can write a character to every element of **s1** before discovering that the first element was set to the null character.

⁴⁹⁶⁾A zero return value implies that all the requested characters from the string pointed to by **s2** were appended to the string pointed to by **s1** and that the result in **s1** is null terminated.

⁴⁹⁷⁾Zero means that **s1** was not null terminated upon entry to **strncat_s**.

⁴⁹⁸⁾This allows an implementation to append characters from **s2** to **s1** while simultaneously checking if any of those characters are null. Such an approach can write a character to every element of **s1** before discovering that the first element was set to the null character.

⁴⁹⁹⁾A zero return value implies that all the requested characters from the string pointed to by **s2** were appended to the string pointed to by **s1** and that the result in **s1** is null terminated.

```
r2 = strncat_s(s2, 6, "", 1);
r3 = strncat_s(s3, 6, "X", 2);
r4 = strncat_s(s4, 7, "defghijklmn", 3);
```

After the first call **r1** will have the value zero and **s1** will contain the sequence goodbye\0.
After the second call **r2** will have the value zero and **s2** will contain the sequence hello\0.
After the third call **r3** will have a nonzero value and **s3** will contain the sequence \0.
After the fourth call **r4** will have the value zero and **s4** will contain the sequence abcdef\0.

K.3.7.4 Search functions

K.3.7.4.1 The strtok_s function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <string.h>
char *strtok_s(char * restrict s1, rsize_t * restrict s1max,
               const char * restrict s2, char ** restrict ptr);
```

Runtime-constraints

None of **s1max**, **s2**, or **ptr** shall be a null pointer. If **s1** is a null pointer, then ***ptr** shall not be a null pointer. The value of ***s1max** shall not be greater than **RSIZE_MAX**. The end of the token found shall occur within the first ***s1max** characters of **s1** for the first call, and shall occur within the first ***s1max** characters of where searching resumes on subsequent calls.
If there is a runtime-constraint violation, the **strtok_s** function does not indirect through the **s1** or **s2** pointers, and does not store a value in the object pointed to by **ptr**.

Description

A sequence of calls to the **strtok_s** function breaks the string pointed to by **s1** into a sequence of tokens, each of which is delimited by a character from the string pointed to by **s2**. The fourth argument points to a caller-provided **char** pointer into which the **strtok_s** function stores information necessary for it to continue scanning the same string.
The first call in a sequence has a non-null first argument and **s1max** points to an object whose value is the number of elements in the character array pointed to by the first argument. The first call stores an initial value in the object pointed to by **ptr** and updates the value pointed to by **s1max** to reflect the number of elements that remain in relation to **ptr**. Subsequent calls in the sequence have a null first argument and the objects pointed to by **s1max** and **ptr** are required to have the values stored by the previous call in the sequence, which are then updated. The separator string pointed to by **s2** can be different from call to call.
The first call in the sequence searches the string pointed to by **s1** for the first character that is *not* contained in the current separator string pointed to by **s2**. If no such character is found, then there are no tokens in the string pointed to by **s1** and the **strtok_s** function returns a null pointer. If such a character is found, it is the start of the first token.
The **strtok_s** function then searches from there for the first character in **s1** that *is* contained in the current separator string. If no such character is found, the current token extends to the end of the string pointed to by **s1**, and subsequent searches in the same string for a token return a null pointer. If such a character is found, it is overwritten by a null character, which terminates the current token.
In all cases, the **strtok_s** function stores sufficient information in the pointer pointed to by **ptr** so that subsequent calls, with a null pointer for **s1** and the unmodified pointer value for **ptr**, shall start searching just past the element overwritten by a null character (if any).

Returns

The **strtok_s** function returns a pointer to the first character of a token, or a null pointer if there is no token or there is a runtime-constraint violation.

EXAMPLE

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <string.h>
static char str1[] = "?a???b,,,#c";
static char str2[] = "\t \t";
char *t, *ptr1, *ptr2;
rsize_t max1 = sizeof(str1);
rsize_t max2 = sizeof(str2);

t = strtok_s(str1, &max1, "?", &ptr1);    // t points to the token "a"
t = strtok_s(NULLPTR, &max1, ",", &ptr1); // t points to the token "??b"
t = strtok_s(str2, &max2, " \t", &ptr2);   // t is a null pointer
t = strtok_s(NULLPTR, &max1, "#", &ptr1); // t points to the token "c"
t = strtok_s(NULLPTR, &max1, "?", &ptr1); // t is a null pointer
```

K.3.7.5 Miscellaneous functions

K.3.7.5.1 The `memset_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <string.h>
errno_t memset_s(void *s, rsize_t smax, int c, rsize_t n)
```

Runtime-constraints

s shall not be a null pointer. Neither **smax** nor **n** shall be greater than **RSIZE_MAX**. **n** shall not be greater than **smax**.

If there is a runtime-constraint violation, then if **s** is not a null pointer and **smax** is not greater than **RSIZE_MAX**, the `memset_s` function stores the value of **c** (converted to an **unsigned char**) into each of the first **smax** characters of the object pointed to by **s**.

Description

The `memset_s` function copies the value of **c** (converted to an **unsigned char**) into each of the first **n** characters of the object pointed to by **s**. Unlike `memset`, any call to the `memset_s` function shall be evaluated strictly according to the rules of the abstract machine as described in 5.2.2.4. That is, any call to the `memset_s` function shall assume that the memory indicated by **s** and **n** may be accessible in the future and thus contains the values indicated by **c**.

Returns

The `memset_s` function returns zero if there was no runtime-constraint violation. Otherwise, a nonzero value is returned.

K.3.7.5.2 The `strerror_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <string.h>
errno_t strerror_s(char *s, rsize_t maxsize, errno_t errnum);
```

Runtime-constraints

s shall not be a null pointer. **maxsize** shall not be greater than **RSIZE_MAX**. **maxsize** shall not equal zero.

If there is a runtime-constraint violation, then the array (if any) pointed to by **s** is not modified.

Description

The `strerror_s` function maps the number in **errnum** to a locale-specific message string. Typically, the values for **errnum** come from **errno**, but `strerror_s` shall map any value of type **int** to a

message.

If the length of the desired string is less than **maxsize**, then the string is copied to the array pointed to by **s**.

Otherwise, if **maxsize** is greater than zero, then **maxsize-1** characters are copied from the string to the array pointed to by **s** and then **s[maxsize-1]** is set to the null character. Then, if **maxsize** is greater than 3, then **s[maxsize-2]**, **s[maxsize-3]**, and **s[maxsize-4]** are set to the character period (.).

Returns

The **strerror_s** function returns zero if the length of the desired string was less than **maxsize** and there was no runtime-constraint violation. Otherwise, the **strerror_s** function returns a nonzero value.

K.3.7.5.3 The **strerrorlen_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <string.h>
size_t strerrorlen_s(errno_t errnum);
```

Description

The **strerrorlen_s** function calculates the length of the (untruncated) locale-specific message string that the **strerror_s** function maps to **errnum**.

Returns

The **strerrorlen_s** function returns the number of characters (not including the null character) in the full message string.

K.3.7.5.4 The **strnlen_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <string.h>
size_t strnlen_s(const char *s, size_t maxsize);
```

Description

The **strnlen_s** function computes the length of the string pointed to by **s**.

Returns

If **s** is a null pointer,⁵⁰⁰⁾ then the **strnlen_s** function returns zero.

Otherwise, the **strnlen_s** function returns the number of characters that precede the terminating null character. If there is no null character in the first **maxsize** characters of **s** then **strnlen_s** returns **maxsize**. At most the first **maxsize** characters of **s** shall be accessed by **strnlen_s**.

K.3.8 Date and time <time.h>

K.3.8.1 General

The header <time.h> defines two types.

The types are

```
errno_t
```

which is type **int**; and

⁵⁰⁰⁾The **strnlen_s** function has no runtime-constraints. This lack of runtime-constraints along with the values returned for a null pointer or an unterminated string argument make **strnlen_s** useful in algorithms that gracefully handle such exceptional data.

rsizet

which is the type **size_t**.

K.3.8.2 Components of time

A broken-down time is *normalized* if the values of the members of the **tm** structure are in their normal ranges.⁵⁰¹⁾

K.3.8.3 Time conversion functions

K.3.8.3.1 General

Like the **strftime** function, the **asctime_s** and **ctime_s** functions do not return a pointer to a static object, and other library functions are permitted to call them.

K.3.8.3.2 The **asctime_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <time.h>
errno_t asctime_s(char *s, rsizet maxsize, const struct tm *timeptr);
```

Runtime-constraints

Neither **s** nor **timeptr** shall be a null pointer. **maxsize** shall not be less than 26 and shall not be greater than **RSIZE_MAX**. The broken-down time pointed to by **timeptr** shall be normalized. The calendar year represented by the broken-down time pointed to by **timeptr** shall not be less than calendar year 0 and shall not be greater than calendar year 9999.

If there is a runtime-constraint violation, there is no attempt to convert the time, and **s[0]** is set to a null character if **s** is not a null pointer and **maxsize** is not zero and is not greater than **RSIZE_MAX**.

Description

The **asctime_s** function converts the normalized broken-down time in the structure pointed to by **timeptr** into a 26 character (including the null character) string in the form

Sun Sep 16 01:03:52 1973\n\0

The fields making up this string are (in order):

1. The name of the day of the week represented by **timeptr->tm_wday** using the following three character weekday names: Sun, Mon, Tue, Wed, Thu, Fri, and Sat.
2. The character space.
3. The name of the month represented by **timeptr->tm_mon** using the following three character month names: Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, and Dec.
4. The character space.
5. The value of **timeptr->tm_mday** as if printed using the **fprintf** format **"%2d"**.
6. The character space.
7. The value of **timeptr->tm_hour** as if printed using the **fprintf** format **"%.2d"**.
8. The character colon.
9. The value of **timeptr->tm_min** as if printed using the **fprintf** format **"%.2d"**.
10. The character colon.

⁵⁰¹⁾The normal ranges are defined in 7.29.1.

- 11. The value of `timeptr->tm_sec` as if printed using the `fprintf` format `"%.2d"`.
- 12. The character space.
- 13. The value of `timeptr->tm_year + 1900` as if printed using the `fprintf` format `"%4d"`.
- 14. The character new line.
- 15. The null character.

Recommended practice

The `strftime` function allows more flexible formatting and supports locale-specific behavior. If you do not require the exact form of the result string produced by the `asctime_s` function, consider using the `strftime` function instead.

Returns

The `asctime_s` function returns zero if the time was successfully converted and stored into the array pointed to by `s`. Otherwise, it returns a nonzero value.

K.3.8.3.3 The `ctime_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <time.h>
errno_t ctime_s(char *s, rsize_t maxsize, const time_t *timer);
```

Runtime-constraints

Neither `s` nor `timer` shall be a null pointer. `maxsize` shall not be less than 26 and shall not be greater than `RSIZE_MAX`.

If there is a runtime-constraint violation, `s[0]` is set to a null character if `s` is not a null pointer and `maxsize` is not equal zero and is not greater than `RSIZE_MAX`.

Description

The `ctime_s` function converts the calendar time pointed to by `timer` to local time in the form of a string. It is equivalent to

```
asctime_s(s, maxsize, localtime_s(timer, &(struct tm){ 0 })))
```

Recommended practice

The `strftime` function allows more flexible formatting and supports locale-specific behavior. If you do not require the exact form of the result string produced by the `ctime_s` function, consider using the `strftime` function instead.

Returns

The `ctime_s` function returns zero if the time was successfully converted and stored into the array pointed to by `s`. Otherwise, it returns a nonzero value.

K.3.8.3.4 The `gmtime_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <time.h>
struct tm *gmtime_s(const time_t * restrict timer, struct tm * restrict result);
```

Runtime-constraints

Neither `timer` nor `result` shall be a null pointer.

If there is a runtime-constraint violation, there is no attempt to convert the time.

Description
The `gmtime_s` function converts the calendar time pointed to by `timer` into a broken-down time, expressed as UTC. The broken-down time is stored in the structure pointed to by `result`.

Returns
The `gmtime_s` function returns `result`, or a null pointer if the specified time cannot be converted to UTC or there is a runtime-constraint violation.

K.3.8.3.5 The `localtime_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <time.h>
struct tm *localtime_s(const time_t * restrict timer, struct tm * restrict result
);
```

Runtime-constraints
Neither `timer` nor `result` shall be a null pointer.
If there is a runtime-constraint violation, there is no attempt to convert the time.

Description
The `localtime_s` function converts the calendar time pointed to by `timer` into a broken-down time, expressed as local time. The broken-down time is stored in the structure pointed to by `result`.

Returns
The `localtime_s` function returns `result`, or a null pointer if the specified time cannot be converted to local time or there is a runtime-constraint violation.

K.3.9 Extended multibyte and wide character utilities `<wchar.h>`

K.3.9.1 General
The header `<wchar.h>` defines two types.
The types are

```
errno_t
```

which is type `int`; and

```
rsize_t
```

which is the type `size_t`.
Unless explicitly stated otherwise, if the execution of a function described in this subclause causes copying to take place between objects that overlap, the objects take on unspecified values.

K.3.9.2 Formatted wide character input/output functions

K.3.9.2.1 The `fwprintf_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <wchar.h>
int fwprintf_s(FILE * restrict stream, const wchar_t * restrict format, ...);
```


Runtime-constraints

Neither **stream** nor **format** shall be a null pointer. The %n specifier⁵⁰²⁾ (modified or not by flags, field width, or precision) shall not appear in the wide string pointed to by **format**. Any argument to **fwprintf_s** corresponding to a %s specifier shall not be a null pointer.

If there is a runtime-constraint violation, the **fwprintf_s** function does not attempt to produce further output, and it is unspecified to what extent **fwprintf_s** produced output before discovering the runtime-constraint violation.

Description

The **fwprintf_s** function is equivalent to the **fwprintf** function except for the previously listed explicit runtime-constraints.

Returns

The **fwprintf_s** function returns the number of wide characters transmitted, or a negative value if an output error, encoding error, or runtime-constraint violation occurred.

K.3.9.2.2 The fwscanf_s function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
#include <wchar.h>
int fwscanf_s(FILE * restrict stream, const wchar_t * restrict format, ...);
```

Runtime-constraints

Neither **stream** nor **format** shall be a null pointer. Any argument indirected though to store converted input shall not be a null pointer.

If there is a runtime-constraint violation, the **fwscanf_s** function does not attempt to perform further input, and it is unspecified to what extent **fwscanf_s** performed input before discovering the runtime-constraint violation.

Description

The **fwscanf_s** function is equivalent to **fwscanf** except that the c, s, and [conversion specifiers apply to a pair of arguments (unless assignment suppression is indicated by a *). The first of these arguments is the same as for **fwscanf**. That argument is immediately followed in the argument list by the second argument, which has type **size_t** and gives the number of elements in the array pointed to by the first argument of the pair. If the first argument points to a scalar object, it is considered to be an array of one element.⁵⁰³⁾

A matching failure occurs if the number of elements in a receiving object is insufficient to hold the converted input (including any trailing null character).

Returns

The **fwscanf_s** function returns the value of the macro **EOF** if an input failure occurs before any conversion or if there is a runtime-constraint violation. Otherwise, the **fwscanf_s** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

⁵⁰²⁾It is not a runtime-constraint violation for the wide characters %n to appear in sequence in the wide string pointed at by **format** when those wide characters are not a interpreted as a %n specifier. For example, if the entire format string was L"%n%n".

⁵⁰³⁾If the format is known at translation time, an implementation can issue a diagnostic for any argument used to store the result from a c, s, or [conversion specifier if that argument is not followed by an argument of a type compatible with **rsize_t**. A limited amount of checking can be done if even if the format is not known at translation time. For example, an implementation can issue a diagnostic for each argument after **format** that has of type pointer to one of **char**, **signed char**, **unsigned char**, or **void** that is not followed by an argument of a type compatible with **rsize_t**. The diagnostic can warn that unless the pointer is being used with a conversion specifier using the hh length modifier, a length argument is expected to follow the pointer argument. Another useful diagnostic can flag any non-pointer argument following **format** that did not have a type compatible with **rsize_t**.

K.3.9.2.3 The `snwprintf_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <wchar.h>
int snwprintf_s(wchar_t * restrict s, rsize_t n, const wchar_t * restrict format,
...);
```

Runtime-constraints

Neither **s** nor **format** shall be a null pointer. **n** shall neither equal zero nor be greater than **RSIZE_MAX/sizeof(wchar_t)**. The `%n` specifier⁵⁰⁴ (modified or not by flags, field width, or precision) shall not appear in the wide string pointed to by **format**. Any argument to **snwprintf_s** corresponding to a `%s` specifier shall not be a null pointer. No encoding error shall occur.

If there is a runtime-constraint violation, then if **s** is not a null pointer and **n** is greater than zero and not greater than **RSIZE_MAX/sizeof(wchar_t)**, then the **snwprintf_s** function sets **s[0]** to the null wide character.

Description

The **snwprintf_s** function is equivalent to the **swprintf** function except for the previously listed explicit runtime-constraints.

The **snwprintf_s** function, unlike **swprintf_s**, will truncate the result to fit within the array pointed to by **s**.

Returns

The **snwprintf_s** function returns the number of wide characters that would have been written had **n** been sufficiently large, not counting the terminating wide null character, or a negative value if a runtime-constraint violation occurred. Thus, the null-terminated output has been completely written if and only if the returned value is both nonnegative and less than **n**.

K.3.9.2.4 The `swprintf_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <wchar.h>
int swprintf_s(wchar_t * restrict s, rsize_t n, const wchar_t * restrict format,
...);
```

Runtime-constraints

Neither **s** nor **format** shall be a null pointer. **n** shall neither equal zero nor be greater than **RSIZE_MAX/sizeof(wchar_t)**. The number of wide characters (including the trailing null) required for the result to be written to the array pointed to by **s** shall not be greater than **n**. The `%n` specifier⁵⁰⁵ (modified or not by flags, field width, or precision) shall not appear in the wide string pointed to by **format**. Any argument to **swprintf_s** corresponding to a `%s` specifier shall not be a null pointer. No encoding error shall occur.

If there is a runtime-constraint violation, then if **s** is not a null pointer and **n** is greater than zero and not greater than **RSIZE_MAX/sizeof(wchar_t)**, then the **swprintf_s** function sets **s[0]** to the null wide character.

⁵⁰⁴) It is not a runtime-constraint violation for the wide characters `%n` to appear in sequence in the wide string pointed at by **format** when those wide characters are not interpreted as a `%n` specifier. For example, if the entire format string was `L"%n%n"`.

⁵⁰⁵) It is not a runtime-constraint violation for the wide characters `%n` to appear in sequence in the wide string pointed at by **format** when those wide characters are not interpreted as a `%n` specifier. For example, if the entire format string was `L"%n%n"`.

Description

The `swprintf_s` function is equivalent to the `swprintf` function except for the previously listed explicit runtime-constraints.

The `swprintf_s` function, unlike `snwprintf_s`, treats a result too big for the array pointed to by `s` as a runtime-constraint violation.

Returns

If no runtime-constraint violation occurred, the `swprintf_s` function returns the number of wide characters written in the array, not counting the terminating null wide character. If an encoding error occurred or if `n` or more wide characters are requested to be written, `swprintf_s` returns a negative value. If any other runtime-constraint violation occurred, `swprintf_s` returns zero.

K.3.9.2.5 The `swscanf_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <wchar.h>
int swscanf_s(const wchar_t * restrict s, const wchar_t * restrict format, ...);
```

Runtime-constraints

Neither `s` nor `format` shall be a null pointer. Any argument indirected though to store converted input shall not be a null pointer.

If there is a runtime-constraint violation, the `swscanf_s` function does not attempt to perform further input, and it is unspecified to what extent `swscanf_s` performed input before discovering the runtime-constraint violation.

Description

The `swscanf_s` function is equivalent to `fwscanf_s`, except that the argument `s` specifies a wide string from which the input is to be obtained, rather than from a stream. Reaching the end of the wide string is equivalent to encountering end-of-file for the `fwscanf_s` function.

Returns

The `swscanf_s` function returns the value of the macro `EOF` if an input failure occurs before any conversion or if there is a runtime-constraint violation. Otherwise, the `swscanf_s` function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

K.3.9.2.6 The `vfwprintf_s` function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdarg.h>
#include <stdio.h>
#include <wchar.h>
int vfwprintf_s(FILE * restrict stream, const wchar_t * restrict format,
va_list arg);
```

Runtime-constraints

Neither `stream` nor `format` shall be a null pointer. The `%n` specifier⁵⁰⁶⁾ (modified or not by flags, field width, or precision) shall not appear in the wide string pointed to by `format`. Any argument to `vfwprintf_s` corresponding to a `%s` specifier shall not be a null pointer.

If there is a runtime-constraint violation, the `vfwprintf_s` function does not attempt to produce further output, and it is unspecified to what extent `vfwprintf_s` produced output before discovering

⁵⁰⁶⁾It is not a runtime-constraint violation for the wide characters `%n` to appear in sequence in the wide string pointed at by `format` when those wide characters are not a interpreted as a `%n` specifier. For example, if the entire format string was `L"%n%n"`.

the runtime-constraint violation.

Description

The **vfwprintf_s** function is equivalent to the **vfwprintf** function except for the previously listed explicit runtime-constraints.

Returns

The **vfwprintf_s** function returns the number of wide characters transmitted, or a negative value if an output error, encoding error, or runtime-constraint violation occurred.

K.3.9.2.7 The **vfwscanf_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdarg.h>
#include <stdio.h>
#include <wchar.h>
int vfwscanf_s(FILE * restrict stream, const wchar_t * restrict format,
               va_list arg);
```

Runtime-constraints

Neither **stream** nor **format** shall be a null pointer. Any argument indirected though to store converted input shall not be a null pointer.

If there is a runtime-constraint violation, the **vfwscanf_s** function does not attempt to perform further input, and it is unspecified to what extent **vfwscanf_s** performed input before discovering the runtime-constraint violation.

Description

The **vfwscanf_s** function is equivalent to **fwscanf_s**, with the variable argument list replaced by **arg**, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** invocations). The **vfwscanf_s** function does not invoke the **va_end** macro.⁵⁰⁷⁾

Returns

The **vfwscanf_s** function returns the value of the macro **EOF** if an input failure occurs before any conversion or if there is a runtime-constraint violation. Otherwise, the **vfwscanf_s** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

K.3.9.2.8 The **vsnwprintf_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdarg.h>
#include <wchar.h>
int vsnwprintf_s(wchar_t * restrict s, rsize_t n, const wchar_t * restrict format,
                 va_list arg);
```

Runtime-constraints

Neither **s** nor **format** shall be a null pointer. **n** shall neither equal zero nor be greater than **RSIZE_MAX / sizeof(wchar_t)**. The **%n** specifier⁵⁰⁸⁾ (modified or not by flags, field width, or precision) shall not appear in the wide string pointed to by **format**. Any argument to **vsnwprintf_s** corresponding to a **%s** specifier shall not be a null pointer. No encoding error shall occur.

⁵⁰⁷⁾ As the functions **vfwscanf_s**, **vwscanf_s**, and **vswscanf_s** invoke the **va_arg** macro, the representation of **arg** after the return is indeterminate.

⁵⁰⁸⁾ It is not a runtime-constraint violation for the wide characters **%n** to appear in sequence in the wide string pointed at by **format** when those wide characters are not interpreted as a **%n** specifier. For example, if the entire format string was **L"%n"**.

If there is a runtime-constraint violation, then if **s** is not a null pointer and **n** is greater than zero and not greater than **RSIZE_MAX/sizeof(wchar_t)**, then the **vsnwprintf_s** function sets **s[0]** to the null wide character.

Description

The **vsnwprintf_s** function is equivalent to the **vswprintf** function except for the previously listed explicit runtime-constraints.

The **vsnwprintf_s** function, unlike **vswprintf_s**, will truncate the result to fit within the array pointed to by **s**.

Returns

The **vsnwprintf_s** function returns the number of wide characters that would have been written had **n** been sufficiently large, not counting the terminating null character, or a negative value if a runtime-constraint violation occurred. Thus, the null-terminated output has been completely written if and only if the returned value is both nonnegative and less than **n**.

K.3.9.2.9 The vswprintf_s function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdarg.h>
#include <wchar.h>
int vswprintf_s(wchar_t * restrict s, rsize_t n, const wchar_t * restrict format,
va_list arg);
```

Runtime-constraints

Neither **s** nor **format** shall be a null pointer. **n** shall neither equal zero nor be greater than **RSIZE_MAX/sizeof(wchar_t)**. The number of wide characters (including the trailing null) required for the result to be written to the array pointed to by **s** shall not be greater than **n**. The **%n** specifier⁵⁰⁹⁾ (modified or not by flags, field width, or precision) shall not appear in the wide string pointed to by **format**. Any argument to **vswprintf_s** corresponding to a **%s** specifier shall not be a null pointer. No encoding error shall occur.

If there is a runtime-constraint violation, then if **s** is not a null pointer and **n** is greater than zero and not greater than **RSIZE_MAX/sizeof(wchar_t)**, then the **vswprintf_s** function sets **s[0]** to the null wide character.

Description

The **vswprintf_s** function is equivalent to the **vswprintf** function except for the previously listed explicit runtime-constraints.

The **vswprintf_s** function, unlike **vsnwprintf_s**, treats a result too big for the array pointed to by **s** as a runtime-constraint violation.

Returns

If no runtime-constraint violation occurred, the **vswprintf_s** function returns the number of wide characters written in the array, not counting the terminating null wide character. If an encoding error occurred or if **n** or more wide characters are requested to be written, **vswprintf_s** returns a negative value. If any other runtime-constraint violation occurred, **vswprintf_s** returns zero.

K.3.9.2.10 The vswscanf_s function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdarg.h>
```

⁵⁰⁹⁾It is not a runtime-constraint violation for the wide characters **%n** to appear in sequence in the wide string pointed at by **format** when those wide characters are not a interpreted as a **%n** specifier. For example, if the entire format string was **L"%n%n"**.

```
#include <wchar.h>
int vswscanf_s(const wchar_t * restrict s, const wchar_t * restrict format,
               va_list arg);
```

Runtime-constraints

Neither **s** nor **format** shall be a null pointer. Any argument indirected though to store converted input shall not be a null pointer.

If there is a runtime-constraint violation, the **vswscanf_s** function does not attempt to perform further input, and it is unspecified to what extent **vswscanf_s** performed input before discovering the runtime-constraint violation.

Description

The **vswscanf_s** function is equivalent to **swscanf_s**, with the variable argument list replaced by **arg**, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** invocations). The **vswscanf_s** function does not invoke the **va_end** macro.⁵¹⁰⁾

Returns

The **vswscanf_s** function returns the value of the macro **EOF** if an input failure occurs before any conversion or if there is a runtime-constraint violation. Otherwise, the **vswscanf_s** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

K.3.9.2.11 The **vwprintf_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdarg.h>
#include <wchar.h>
int vwprintf_s(const wchar_t * restrict format, va_list arg);
```

Runtime-constraints

format shall not be a null pointer. The **%n** specifier⁵¹¹⁾ (modified or not by flags, field width, or precision) shall not appear in the wide string pointed to by **format**. Any argument to **vwprintf_s** corresponding to a **%s** specifier shall not be a null pointer.

If there is a runtime-constraint violation, the **vwprintf_s** function does not attempt to produce further output, and it is unspecified to what extent **vwprintf_s** produced output before discovering the runtime-constraint violation.

Description

The **vwprintf_s** function is equivalent to the **vwprintf** function except for the previously listed explicit runtime-constraints.

Returns

The **vwprintf_s** function returns the number of wide characters transmitted, or a negative value if an output error, encoding error, or runtime-constraint violation occurred.

K.3.9.2.12 The **vwscanf_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdarg.h>
#include <wchar.h>
```

⁵¹⁰⁾ As the functions **vfwscanf_s**, **vwscanf_s**, and **vswscanf_s** invoke the **va_arg** macro, the representation of **arg** after the return is indeterminate.

⁵¹¹⁾ It is not a runtime-constraint violation for the wide characters **%n** to appear in sequence in the wide string pointed at by **format** when those wide characters are not interpreted as a **%n** specifier. For example, if the entire format string was **L"%n%n"**.


```
int vwscanf_s(const wchar_t * restrict format, va_list arg);
```

Runtime-constraints

format shall not be a null pointer. Any argument indirected though to store converted input shall not be a null pointer.

If there is a runtime-constraint violation, the **vwscanf_s** function does not attempt to perform further input, and it is unspecified to what extent **vwscanf_s** performed input before discovering the runtime-constraint violation.

Description

The **vwscanf_s** function is equivalent to **wscanf_s**, with the variable argument list replaced by **arg**, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** invocations). The **vwscanf_s** function does not invoke the **va_end** macro.⁵¹²⁾

Returns

The **vwscanf_s** function returns the value of the macro **E0F** if an input failure occurs before any conversion or if there is a runtime-constraint violation. Otherwise, the **vwscanf_s** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

K.3.9.2.13 The wprintf_s function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <wchar.h>
int wprintf_s(const wchar_t * restrict format, ...);
```

Runtime-constraints

format shall not be a null pointer. The %n specifier⁵¹³⁾ (modified or not by flags, field width, or precision) shall not appear in the wide string pointed to by **format**. Any argument to **wprintf_s** corresponding to a %s specifier shall not be a null pointer.

If there is a runtime-constraint violation, the **wprintf_s** function does not attempt to produce further output, and it is unspecified to what extent **wprintf_s** produced output before discovering the runtime-constraint violation.

Description

The **wprintf_s** function is equivalent to the **wprintf** function except for the previously listed explicit runtime-constraints.

Returns

The **wprintf_s** function returns the number of wide characters transmitted, or a negative value if an output error, encoding error, or runtime-constraint violation occurred.

K.3.9.2.14 The wscanf_s function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <wchar.h>
int wscanf_s(const wchar_t * restrict format, ...);
```

Runtime-constraints

⁵¹²⁾As the functions **vfwscanf_s**, **wscanf_s**, and **vwscanf_s** invoke the **va_arg** macro, the representation of **arg** after the return is indeterminate.

⁵¹³⁾It is not a runtime-constraint violation for the wide characters %n to appear in sequence in the wide string pointed at by **format** when those wide characters are not a interpreted as a %n specifier. For example, if the entire format string was L"%%n".

format shall not be a null pointer. Any argument indirected though to store converted input shall not be a null pointer.

If there is a runtime-constraint violation, the **wscanf_s** function does not attempt to perform further input, and it is unspecified to what extent **wscanf_s** performed input before discovering the runtime-constraint violation.

Description

The **wscanf_s** function is equivalent to **fscanf_s** with the argument **stdin** interposed before the arguments to **wscanf_s**.

Returns

The **wscanf_s** function returns the value of the macro **EOF** if an input failure occurs before any conversion or if there is a runtime-constraint violation. Otherwise, the **wscanf_s** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

K.3.9.3 General wide string utilities

K.3.9.3.1 Wide string copying functions

K.3.9.3.1.1 The **wcscpy_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <wchar.h>
errno_t wcscpy_s(wchar_t * restrict s1, rsize_t s1max,
                 const wchar_t * restrict s2);
```

Runtime-constraints

Neither **s1** nor **s2** shall be a null pointer. **s1max** shall not be greater than **RSIZE_MAX/sizeof(wchar_t)**. **s1max** shall not equal zero. **s1max** shall be greater than **wcsnlen_s(s2, s1max)**. Copying shall not take place between objects that overlap.

If there is a runtime-constraint violation, then if **s1** is not a null pointer and **s1max** is greater than zero and not greater than **RSIZE_MAX/sizeof(wchar_t)**, then **wcscpy_s** sets **s1[0]** to the null wide character.

Description

The **wcscpy_s** function copies the wide string pointed to by **s2** (including the terminating null wide character) into the array pointed to by **s1**.

All elements following the terminating null wide character (if any) written by **wcscpy_s** in the array of **s1max** wide characters pointed to by **s1** take unspecified values when **wcscpy_s** returns.⁵¹⁴⁾

Returns

The **wcscpy_s** function returns zero⁵¹⁵⁾ if there was no runtime-constraint violation. Otherwise, a nonzero value is returned.

K.3.9.3.1.2 The **wcsncpy_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <wchar.h>
errno_t wcsncpy_s(wchar_t * restrict s1, rsize_t s1max,
                  const wchar_t * restrict s2, rsize_t n);
```

⁵¹⁴⁾This allows an implementation to copy wide characters from **s2** to **s1** while simultaneously checking if any of those wide characters are null. Such an approach can write a wide character to every element of **s1** before discovering that the first element was set to the null wide character.

⁵¹⁵⁾A zero return value implies that all the requested wide characters from the string pointed to by **s2** fit within the array pointed to by **s1** and that the result in **s1** is null terminated.

Runtime-constraints

Neither **s1** nor **s2** shall be a null pointer. Neither **s1max** nor **n** shall be greater than **RSIZE_MAX/sizeof(wchar_t)**. **s1max** shall not equal zero. If **n** is not less than **s1max**, then **s1max** shall be greater than **wcsnlen_s(s2, s1max)**. Copying shall not take place between objects that overlap.

If there is a runtime-constraint violation, then if **s1** is not a null pointer and **s1max** is greater than zero and not greater than **RSIZE_MAX/sizeof(wchar_t)**, then **wcsncpy_s** sets **s1[0]** to the null wide character.

Description

The **wcsncpy_s** function copies not more than **n** successive wide characters (wide characters that follow a null wide character are not copied) from the array pointed to by **s2** to the array pointed to by **s1**. If no null wide character was copied from **s2**, then **s1[n]** is set to a null wide character.

All elements following the terminating null wide character (if any) written by **wcsncpy_s** in the array of **s1max** wide characters pointed to by **s1** take unspecified values when **wcsncpy_s** returns.⁵¹⁶⁾

Returns

The **wcsncpy_s** function returns zero⁵¹⁷⁾ if there was no runtime-constraint violation. Otherwise, a nonzero value is returned.

EXAMPLE The **wcsncpy_s** function can be used to copy a wide string without the danger that the result will not be null terminated or that wide characters will be written past the end of the destination array.

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <wchar.h>
/* ... */
wchar_t src1[100] = L"hello";
wchar_t src2[7] = {L'g', L'o', L'o', L'd', L'b', L'y', L'e'};
wchar_t dst1[6], dst2[5], dst3[5];
int r1, r2, r3;
r1 = wcsncpy_s(dst1, 6, src1, 100);
r2 = wcsncpy_s(dst2, 5, src2, 7);
r3 = wcsncpy_s(dst3, 5, src2, 4);
```

The first call will assign to **r1** the value zero and to **dst1** the sequence of wide characters **hello\0**.

The second call will assign to **r2** a nonzero value and to **dst2** the sequence of wide characters **\0**.

The third call will assign to **r3** the value zero and to **dst3** the sequence of wide characters **good\0**.

K.3.9.3.1.3 The **wmemcpy_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <wchar.h>
errno_t wmemcpy_s(wchar_t * restrict s1, rsize_t s1max,
                  const wchar_t * restrict s2, rsize_t n);
```

Runtime-constraints

Neither **s1** nor **s2** shall be a null pointer. Neither **s1max** nor **n** shall be greater than **RSIZE_MAX/sizeof(wchar_t)**. **n** shall not be greater than **s1max**. Copying shall not take place between objects that overlap.

If there is a runtime-constraint violation, the **wmemcpy_s** function stores zeros in the first **s1max** wide characters of the object pointed to by **s1** if **s1** is not a null pointer and **s1max** is not greater than **RSIZE_MAX/sizeof(wchar_t)**.

⁵¹⁶⁾This allows an implementation to copy wide characters from **s2** to **s1** while simultaneously checking if any of those wide characters are null. Such an approach can write a wide character to every element of **s1** before discovering that the first element was set to the null wide character.

⁵¹⁷⁾A zero return value implies that all the requested wide characters from the string pointed to by **s2** fit within the array pointed to by **s1** and that the result in **s1** is null terminated.

Description

The `wmemcpy_s` function copies `n` successive wide characters from the object pointed to by `s2` into the object pointed to by `s1`.

Returns

The `wmemcpy_s` function returns zero if there was no runtime-constraint violation. Otherwise, a nonzero value is returned.

K.3.9.3.1.4 The `wmemmove_s` function**Synopsis**

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <wchar.h>
errno_t wmemmove_s(wchar_t *s1, rsize_t s1max, const wchar_t *s2, rsize_t n);
```

Runtime-constraints

Neither `s1` nor `s2` shall be a null pointer. Neither `s1max` nor `n` shall be greater than `RSIZE_MAX/sizeof(wchar_t)`. `n` shall not be greater than `s1max`.

If there is a runtime-constraint violation, the `wmemmove_s` function stores zeros in the first `s1max` wide characters of the object pointed to by `s1` if `s1` is not a null pointer and `s1max` is not greater than `RSIZE_MAX/sizeof(wchar_t)`.

Description

The `wmemmove_s` function copies `n` successive wide characters from the object pointed to by `s2` into the object pointed to by `s1`. This copying takes place as if the `n` wide characters from the object pointed to by `s2` are first copied into a temporary array of `n` wide characters that does not overlap the objects pointed to by `s1` or `s2`, and then the `n` wide characters from the temporary array are copied into the object pointed to by `s1`.

Returns

The `wmemmove_s` function returns zero if there was no runtime-constraint violation. Otherwise, a nonzero value is returned.

K.3.9.3.2 Wide string concatenation functions**K.3.9.3.2.1 The `wcscat_s` function****Synopsis**

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <wchar.h>
errno_t wcscat_s(wchar_t * restrict s1, rsize_t s1max,
                 const wchar_t * restrict s2);
```

Runtime-constraints

Let m denote the value `s1max - wcsnlen_s(s1, s1max)` upon entry to `wcscat_s`.

Neither `s1` nor `s2` shall be a null pointer. `s1max` shall not be greater than `RSIZE_MAX/sizeof(wchar_t)`. `s1max` shall not equal zero. m shall not equal zero.⁵¹⁸⁾ m shall be greater than `wcsnlen_s(s2, m)`. Copying shall not take place between objects that overlap.

If there is a runtime-constraint violation, then if `s1` is not a null pointer and `s1max` is greater than zero and not greater than `RSIZE_MAX/sizeof(wchar_t)`, then `wcscat_s` sets `s1[0]` to the null wide character.

Description

The `wcscat_s` function appends a copy of the wide string pointed to by `s2` (including the terminating null wide character) to the end of the wide string pointed to by `s1`. The initial wide character from

⁵¹⁸⁾Zero means that `s1` was not null terminated upon entry to `wcscat_s`.

s2 overwrites the null wide character at the end of **s1**.

All elements following the terminating null wide character (if any) written by **wcscat_s** in the array of **s1max** wide characters pointed to by **s1** take unspecified values when **wcscat_s** returns.⁵¹⁹⁾

Returns

The **wcscat_s** function returns zero⁵²⁰⁾ if there was no runtime-constraint violation. Otherwise, a nonzero value is returned.

K.3.9.3.2.2 The **wcsncat_s** function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <wchar.h>
errno_t wcsncat_s(wchar_t * restrict s1, rsize_t s1max,
                  const wchar_t * restrict s2, rsize_t n);
```

Runtime-constraints

Let *m* denote the value **s1max** - **wcsnlen_s(s1, s1max)** upon entry to **wcsncat_s**.

Neither **s1** nor **s2** shall be a null pointer. Neither **s1max** nor **n** shall be greater than **RSIZE_MAX/sizeof(wchar_t)**. **s1max** shall not equal zero. *m* shall not equal zero.⁵²¹⁾ If **n** is not less than *m*, then *m* shall be greater than **wcsnlen_s(s2, m)**. Copying shall not take place between objects that overlap.

If there is a runtime-constraint violation, then if **s1** is not a null pointer and **s1max** is greater than zero and not greater than **RSIZE_MAX/sizeof(wchar_t)**, then **wcsncat_s** sets **s1[0]** to the null wide character.

Description

The **wcsncat_s** function appends not more than **n** successive wide characters (wide characters that follow a null wide character are not copied) from the array pointed to by **s2** to the end of the wide string pointed to by **s1**. The initial wide character from **s2** overwrites the null wide character at the end of **s1**. If no null wide character was copied from **s2**, then **s1[s1max - m + n]** is set to a null wide character.

All elements following the terminating null wide character (if any) written by **wcsncat_s** in the array of **s1max** wide characters pointed to by **s1** take unspecified values when **wcsncat_s** returns.⁵²²⁾

Returns

The **wcsncat_s** function returns zero⁵²³⁾ if there was no runtime-constraint violation. Otherwise, a nonzero value is returned.

EXAMPLE The **wcsncat_s** function can be used to copy a wide string without the danger that the result will not be null terminated or that wide characters will be written past the end of the destination array.

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <wchar.h>
/* ... */
wchar_t s1[100] = L"good";
wchar_t s2[6] = L"hello";
```

⁵¹⁹⁾This allows an implementation to append wide characters from **s2** to **s1** while simultaneously checking if any of those wide characters are null. Such an approach can write a wide character to every element of **s1** before discovering that the first element was set to the null wide character.

⁵²⁰⁾A zero return value implies that all the requested wide characters from the wide string pointed to by **s2** were appended to the wide string pointed to by **s1** and that the result in **s1** is null terminated.

⁵²¹⁾Zero means that **s1** was not null terminated upon entry to **wcsncat_s**.

⁵²²⁾This allows an implementation to append wide characters from **s2** to **s1** while simultaneously checking if any of those wide characters are null. Such an approach can write a wide character to every element of **s1** before discovering that the first element was set to the null wide character.

⁵²³⁾A zero return value implies that all the requested wide characters from the wide string pointed to by **s2** were appended to the wide string pointed to by **s1** and that the result in **s1** is null terminated.

```

wchar_t s3[6] = L"hello";
wchar_t s4[7] = L"abc";
wchar_t s5[1000] = L"bye";
int r1, r2, r3, r4;
r1 = wcsncat_s(s1, 100, s5, 1000);
r2 = wcsncat_s(s2, 6, L"", 1);
r3 = wcsncat_s(s3, 6, L"X", 2);
r4 = wcsncat_s(s4, 7, L"defghijklmn", 3);

```

After the first call **r1** will have the value zero and **s1** will be the wide character sequence goodbye\0.

After the second call **r2** will have the value zero and **s2** will be the wide character sequence hello\0.

After the third call **r3** will have a nonzero value and **s3** will be the wide character sequence \0.

After the fourth call **r4** will have the value zero and **s4** will be the wide character sequence abcdef\0.

K.3.9.3.3 Wide string search functions

K.3.9.3.3.1 The `wcstok_s` function

Synopsis

```

#define __STDC_WANT_LIB_EXT1__ 1
#include <wchar.h>
wchar_t *wcstok_s(wchar_t * restrict s1, rsize_t * restrict s1max,
                  const wchar_t * restrict s2, wchar_t ** restrict ptr);

```

Runtime-constraints

None of **s1max**, **s2**, or **ptr** shall be a null pointer. If **s1** is a null pointer, then ***ptr** shall not be a null pointer. The value of ***s1max** shall not be greater than **RSIZE_MAX/sizeof(wchar_t)**. The end of the token found shall occur within the first ***s1max** wide characters of **s1** for the first call, and shall occur within the first ***s1max** wide characters of where searching resumes on subsequent calls.

If there is a runtime-constraint violation, the `wcstok_s` function does not indirect through the **s1** or **s2** pointers, and does not store a value in the object pointed to by **ptr**.

Description

A sequence of calls to the `wcstok_s` function breaks the wide string pointed to by **s1** into a sequence of tokens, each of which is delimited by a wide character from the wide string pointed to by **s2**. The fourth argument points to a caller-provided `wchar_t` pointer into which the `wcstok_s` function stores information necessary for it to continue scanning the same wide string.

The first call in a sequence has a non-null first argument and **s1max** points to an object whose value is the number of elements in the wide character array pointed to by the first argument. The first call stores an initial value in the object pointed to by **ptr** and updates the value pointed to by **s1max** to reflect the number of elements that remain in relation to **ptr**. Subsequent calls in the sequence have a null first argument and the objects pointed to by **s1max** and **ptr** are required to have the values stored by the previous call in the sequence, which are then updated. The separator wide string pointed to by **s2** can be different from call to call.

The first call in the sequence searches the wide string pointed to by **s1** for the first wide character that is *not* contained in the current separator wide string pointed to by **s2**. If no such wide character is found, then there are no tokens in the wide string pointed to by **s1** and the `wcstok_s` function returns a null pointer. If such a wide character is found, it is the start of the first token.

The `wcstok_s` function then searches from there for the first wide character in **s1** that *is* contained in the current separator wide string. If no such wide character is found, the current token extends to the end of the wide string pointed to by **s1**, and subsequent searches in the same wide string for a token return a null pointer. If such a wide character is found, it is overwritten by a null wide character, which terminates the current token.

In all cases, the `wcstok_s` function stores sufficient information in the pointer pointed to by **ptr** so that subsequent calls, with a null pointer for **s1** and the unmodified pointer value for **ptr**, shall start

searching just past the element overwritten by a null wide character (if any).

Returns

The `wcstok_s` function returns a pointer to the first wide character of a token, or a null pointer if there is no token or there is a runtime-constraint violation.

EXAMPLE

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <wchar.h>
static wchar_t str1[] = L"?a???b,,,#c";
static wchar_t str2[] = L"\t \t";
wchar_t *t, *ptr1, *ptr2;
rsize_t max1 = wcslen(str1)+1;
rsize_t max2 = wcslen(str2)+1;

t = wcstok_s(str1, &max1, "?", &ptr1);    // t points to the token "a"
t = wcstok_s(nullptr, &max1, ",", &ptr1); // t points to the token "??b"
t = wcstok_s(str2, &max2, " \t", &ptr2);  // t is a null pointer
t = wcstok_s(nullptr, &max1, "#", &ptr1); // t points to the token "c"
t = wcstok_s(nullptr, &max1, "?", &ptr1); // t is a null pointer
```

K.3.9.3.4 Miscellaneous functions
K.3.9.3.4.1 The wcsnlen_s function

Synopsis

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <wchar.h>
size_t wcsnlen_s(const wchar_t *s, size_t maxsize);
```

Description

The `wcsnlen_s` function computes the length of the wide string pointed to by `s`.

Returns

If `s` is a null pointer,⁵²⁴⁾ then the `wcsnlen_s` function returns zero. Otherwise, the `wcsnlen_s` function returns the number of wide characters that precede the terminating null wide character. If there is no null wide character in the first `maxsize` wide characters of `s` then `wcsnlen_s` returns `maxsize`. At most the first `maxsize` wide characters of `s` shall be accessed by `wcsnlen_s`.

K.3.9.4 Extended multibyte/wide character conversion utilities
K.3.9.4.1 Restartable multibyte/wide character conversion functions
K.3.9.4.1.1 General

Unlike `wcrtomb`, `wcrtomb_s` does not permit the `ps` parameter (the pointer to the conversion state) to be a null pointer.

K.3.9.4.1.2 The wcrtomb_s function

Synopsis

```
#include <wchar.h>
errno_t wcrtomb_s(size_t * restrict retval, char * restrict s, rsize_t smax,
wchar_t wc, mbstate_t * restrict ps);
```

⁵²⁴⁾The `wcsnlen_s` function has no runtime-constraints. This lack of runtime-constraints along with the values returned for a null pointer or an unterminated wide string argument make `wcsnlen_s` useful in algorithms that gracefully handle such exceptional data.

Runtime-constraints

Neither **retval** nor **ps** shall be a null pointer. If **s** is not a null pointer, then **smax** shall not equal zero and shall not be greater than **RSIZE_MAX**. If **s** is not a null pointer, then **smax** shall not be less than the number of bytes to be stored in the array pointed to by **s**. If **s** is a null pointer, then **smax** shall equal zero.

If there is a runtime-constraint violation, then **wcrtomb_s** does the following. If **s** is not a null pointer and **smax** is greater than zero and not greater than **RSIZE_MAX**, then **wcrtomb_s** sets **s[0]** to the null character. If **retval** is not a null pointer, then **wcrtomb_s** sets ***retval** to **(size_t)(-1)**.

Description

If **s** is a null pointer, the **wcrtomb_s** function is equivalent to the call

```
wcrtomb_s(&retval, buf, sizeof buf, L'\0', ps)
```

where **retval** and **buf** are internal objects of the appropriate types, and the size of **buf** is greater than **MB_CUR_MAX**.

If **s** is not a null pointer, the **wcrtomb_s** function determines the number of bytes needed to represent the multibyte character that corresponds to the wide character given by **wc** (including any shift sequences), and stores the multibyte character representation in the array whose first element is pointed to by **s**. At most **MB_CUR_MAX** bytes are stored. If **wc** is a null wide character, a null byte is stored, preceded by any shift sequence needed to restore the initial shift state; the resulting state described is the initial conversion state.

If **wc** does not correspond to a valid multibyte character, an encoding error occurs: the **wcrtomb_s** function stores the value **(size_t)(-1)** into ***retval** and the conversion state is unspecified. Otherwise, the **wcrtomb_s** function stores into ***retval** the number of bytes (including any shift sequences) stored in the array pointed to by **s**.

Returns

The **wcrtomb_s** function returns zero if no runtime-constraint violation and no encoding error occurred. Otherwise, a nonzero value is returned.

K.3.9.4.2 Restartable multibyte/wide string conversion functions**K.3.9.4.2.1 General**

Unlike **mbsrtowcs** and **wcsrtombs**, **mbsrtowcs_s** and **wcsrtombs_s** do not permit the **ps** parameter (the pointer to the conversion state) to be a null pointer.

K.3.9.4.2.2 The mbsrtowcs_s function**Synopsis**

```
#include <wchar.h>
errno_t mbsrtowcs_s(size_t * restrict retval, wchar_t * restrict dst,
    rsize_t dstmax, const char ** restrict src, rsize_t len,
    mbstate_t * restrict ps);
```

Runtime-constraints

None of **retval**, **src**, ***src**, or **ps** shall be null pointers. If **dst** is not a null pointer, then neither **len** nor **dstmax** shall be greater than **RSIZE_MAX/sizeof(wchar_t)**. If **dst** is a null pointer, then **dstmax** shall equal zero. If **dst** is not a null pointer, then **dstmax** shall not equal zero. If **dst** is not a null pointer and **len** is not less than **dstmax**, then a null character shall occur within the first **dstmax** multibyte characters of the array pointed to by ***src**.

If there is a runtime-constraint violation, then **mbsrtowcs_s** does the following. If **retval** is not a null pointer, then **mbsrtowcs_s** sets ***retval** to **(size_t)(-1)**. If **dst** is not a null pointer and **dstmax** is greater than zero and not greater than **RSIZE_MAX/sizeof(wchar_t)**, then **mbsrtowcs_s** sets **dst[0]** to the null wide character.

Description

The `mbsrtowcs_s` function converts a sequence of multibyte characters that begins in the conversion state described by the object pointed to by `ps`, from the array indirectly pointed to by `src` into a sequence of corresponding wide characters. If `dst` is not a null pointer, the converted characters are stored into the array pointed to by `dst`. Conversion continues up to and including a terminating null character, which is also stored. Conversion stops earlier in two cases: when a sequence of bytes is encountered that does not form a valid multibyte character, or (if `dst` is not a null pointer) when `len` wide characters have been stored into the array pointed to by `dst`.⁵²⁵⁾ If `dst` is not a null pointer and no null wide character was stored into the array pointed to by `dst`, then `dst[len]` is set to the null wide character. Each conversion takes place as if by a call to the `mbrtowc` function.

If `dst` is not a null pointer, the pointer object pointed to by `src` is assigned either a null pointer (if conversion stopped due to reaching a terminating null character) or the address just past the last multibyte character converted (if any). If conversion stopped due to reaching a terminating null character and if `dst` is not a null pointer, the resulting state described is the initial conversion state.

Regardless of whether `dst` is or is not a null pointer, if the input conversion encounters a sequence of bytes that do not form a valid multibyte character, an encoding error occurs: the `mbsrtowcs_s` function stores the value `(size_t)(-1)` into `*retval` and the conversion state is unspecified. Otherwise, the `mbsrtowcs_s` function stores into `*retval` the number of multibyte characters successfully converted, not including the terminating null character (if any).

All elements following the terminating null wide character (if any) written by `mbsrtowcs_s` in the array of `dstmax` wide characters pointed to by `dst` take unspecified values when `mbsrtowcs_s` returns.⁵²⁶⁾

If copying takes place between objects that overlap, the objects take on unspecified values.

Returns

The `mbsrtowcs_s` function returns zero if no runtime-constraint violation and no encoding error occurred. Otherwise, a nonzero value is returned.

K.3.9.4.2.3 The `wcsrtombs_s` function

Synopsis

```
#include <wchar.h>
errno_t wcsrtombs_s(size_t * restrict retval, char * restrict dst,
    rsize_t dstmax, const wchar_t ** restrict src, rsize_t len,
    mbstate_t * restrict ps);
```

Runtime-constraints

None of `retval`, `src`, `*src`, or `ps` shall be null pointers. If `dst` is not a null pointer, then neither `len` shall be greater than `RSIZE_MAX/sizeof(wchar_t)` nor `dstmax` shall be greater than `RSIZE_MAX`. If `dst` is a null pointer, then `dstmax` shall equal zero. If `dst` is not a null pointer, then `dstmax` shall not equal zero. If `dst` is not a null pointer and `len` is not less than `dstmax`, then the conversion shall have been stopped (see the following) because a terminating null wide character was reached or because an encoding error occurred.

If there is a runtime-constraint violation, then `wcsrtombs_s` does the following. If `retval` is not a null pointer, then `wcsrtombs_s` sets `*retval` to `(size_t)(-1)`. If `dst` is not a null pointer and `dstmax` is greater than zero and not greater than `RSIZE_MAX`, then `wcsrtombs_s` sets `dst[0]` to the null character.

Description

The `wcsrtombs_s` function converts a sequence of wide characters from the array indirectly pointed to by `src` into a sequence of corresponding multibyte characters that begins in the conversion state

⁵²⁵⁾Thus, the value of `len` is ignored if `dst` is a null pointer.
⁵²⁶⁾This allows an implementation to attempt converting the multibyte string before discovering a terminating null character did not occur where required.

described by the object pointed to by **ps**. If **dst** is not a null pointer, the converted characters are then stored into the array pointed to by **dst**. Conversion continues up to and including a terminating null wide character, which is also stored. Conversion stops earlier in two cases:

- when a wide character is reached that does not correspond to a valid multibyte character;
- (if **dst** is not a null pointer) when the next multibyte character would exceed the limit of n total bytes to be stored into the array pointed to by **dst**. If the wide character being converted is the null wide character, then n is the lesser of **len** or **dstmax**. Otherwise, n is the lesser of **len** or **dstmax - 1**.

If the conversion stops without converting a null wide character and **dst** is not a null pointer, then a null character is stored into the array pointed to by **dst** immediately following any multibyte characters already stored. Each conversion takes place as if by a call to the **wcrtomb** function.⁵²⁷⁾

If **dst** is not a null pointer, the pointer object pointed to by **src** is assigned either a null pointer (if conversion stopped due to reaching a terminating null wide character) or the address just past the last wide character converted (if any). If conversion stopped due to reaching a terminating null wide character, the resulting state described is the initial conversion state.

Regardless of whether **dst** is or is not a null pointer, if the input conversion encounters a wide character that does not correspond to a valid multibyte character, an encoding error occurs: the **wcsrtombs_s** function stores the value **(size_t)(-1)** into ***retval** and the conversion state is unspecified. Otherwise, the **wcsrtombs_s** function stores into ***retval** the number of bytes in the resulting multibyte character sequence, not including the terminating null character (if any).

All elements following the terminating null character (if any) written by **wcsrtombs_s** in the array of **dstmax** elements pointed to by **dst** take unspecified values when **wcsrtombs_s** returns.⁵²⁸⁾

If copying takes place between objects that overlap, the objects take on unspecified values.

Returns

The **wcsrtombs_s** function returns zero if no runtime-constraint violation and no encoding error occurred. Otherwise, a nonzero value is returned.

⁵²⁷⁾If conversion stops because a terminating null wide character has been reached, the bytes stored include those necessary to reach the initial shift state immediately before the null byte. However, if the conversion stops before a terminating null wide character has been reached, the result will be null terminated, but does not necessarily end in the initial shift state.

⁵²⁸⁾When **len** is not less than **dstmax**, the implementation can fill the array before discovering a runtime-constraint violation.

Annex L

(normative)

Analyzability

L.1 Scope

This Annex specifies optional behavior that can aid in the analyzability of C programs.

An implementation that defines `__STDC_ANALYZABLE__` shall conform to the specifications in this annex (see also 6.10.10.4).⁵²⁹⁾

L.2 Requirements

If the program performs a trap (3.25), the implementation is permitted to invoke a runtime-constraint handler. Any such semantics are implementation-defined.

All undefined behavior shall be limited to bounded undefined behavior, except for the following which are permitted to result in critical undefined behavior:

- An object is referred to outside of its lifetime (6.2.4).
- A store is performed to an object that has two incompatible declarations (6.2.7),
- A pointer is used to call a function whose type is not compatible with the referenced type (6.2.7, 6.3.3.3, 6.5.3.3).
- An lvalue does not designate an object when evaluated (6.3.3.1).
- The program attempts to modify a string literal (6.4.6).
- The operand of the unary `*` operator has an invalid value (6.5.4.3).
- Addition or subtraction of a pointer into, or just beyond, an array object and an integer type produces a result that points just beyond the array object and is used as the operand of a unary `*` operator that is evaluated (6.5.7).
- An attempt is made to modify an object defined with a const-qualified type through use of an lvalue with non-const-qualified type (6.7.4).
- An argument to a function or macro defined in the standard library has an invalid value or a type not expected by a function with variable number of arguments (7.1.4).
- The `longjmp` function is called with a `jmp_buf` argument where the most recent invocation of the `setjmp` macro in the same invocation of the program with the corresponding `jmp_buf` argument is nonexistent, or the invocation was from another thread of execution, or the function containing the invocation has terminated execution in the interim, or the invocation was within the scope of an identifier with variably modified type and execution has left that scope in the interim (7.13.3.1).
- The value of a pointer that refers to space deallocated by a call to the `free` or `realloc` function is used (7.24.4).
- A string or wide string utility function accesses an array beyond the end of an object (7.26.1, 7.31.4).

⁵²⁹⁾Implementations that do not define `__STDC_ANALYZABLE__` are not required to conform to these specifications.

Annex M

(informative)

Change History

M.1 Attribute Changes

The attribute feature was introduced in the fifth edition of this document, as detailed in the change list in M.2. Table M.1 illustrates what usage of the `__has_c_attribute` preprocessor conditional expression should return in the latest edition, previous values, and the change associated with that value.

Programs and implementations can use Table M.1 to know what values were being used at any specific point in time, leading up to the publication of this document. The value at the bottom of a particular row in Table M.1 is the latest value and corresponds with the behavior for the given attribute described in this document.

Table M.1: `__has_c_attribute` values and associated changes

attribute tokens	value	semantic and/or syntactic changes
deprecated	201904L	Initial introduction.
	202311L	Harmonized for fifth edition.
fallthrough	201904L	Initial introduction.
	201910L	Expanded locations where fallthrough provides diagnostics due to improvements in specification of blocks.
	202311L	Harmonized for fifth edition.
maybe_unused	201904L	Initial introduction.
	202106L	maybe_unused may appertain to labels.
	202311L	Harmonized for fifth edition.
nodiscard	201904L	Initial introduction.
	202003L	Added a form which accepts a string literal for diagnostic purposes, e.g. nodiscard("should have a reason") .
	202311L	Harmonized for fifth edition.
noreturn	202202L	Initial introduction.
	202311L	Harmonized for fifth edition.
reproducible	202207L	Initial introduction.
	202311L	Harmonized for fifth edition.
unsequenced	202207L	Initial introduction.
	202311L	Harmonized for fifth edition.

M.2 Fifth Edition

Major changes in this fifth edition (`__STDC_VERSION__` 202311L) include:

- add new keywords such as **bool**, **static_assert**, **true**, **false**, **thread_local** and others, and allowed implementations to provide macros for the older spelling with a leading underscore followed by a capital letter as well as defining old and new keywords as macros to enable transition of programs easily;
- removed integer width constraints and obsolete sign representations (so-called “1’s complement” and “sign-magnitude”);
- added a one-argument version of **static_assert**;
- removed support for function definitions with identifier lists;
- mandated function declarations whose parameter list is empty be treated the same as a parameter list which only contain a single **void**;

- harmonization with ISO/IEC 9945 (POSIX):
 - extended month name formats for **strftime**
 - integration of functions: **gmtime_r**, **localtime_r**, **memccpy**, **strdup**, **strndup**
- harmonization with floating-point standard ISO/IEC 60559:
 - integration of binary floating-point technical specification ISO/IEC TS 18661-1:2014
 - integration of decimal floating-point technical specification ISO/IEC TS 18661-2:2015
 - integration of floating-point types technical specification ISO/IEC TS 18661-3:2015
 - integration of mathematical functions technical specification ISO/IEC TS 18661-4:2015
 - new maximum and minimum functions for operations recommended in ISO/IEC 60559:2020
- made the **DECIMAL_DIG** macro obsolescent;
- added version test macros to library headers that contained changes to aid in upgrading and portability to be used alongside the **__STDC_VERSION__** macro;
- allowed placement of labels in front of declarations and at the end of compound statement;
- added the attributes feature, which includes the attributes:
 - **deprecated**, for marking entities as discouraged for future use;
 - **fallthrough**, for explicitly marking cases where falling through in switches or labels is intended rather than accidental;
 - **maybe_unused**, for marking entities which can end up not being used;
 - **nodiscard**, for marking entities which, when used, should have their value handled in some way by a program;
 - **noreturn**, for indicating a function shall never return;
 - **reproducible**, for marking function types for which inputs always produce predictable output if given the same input (e.g. cached data) but for which the order of such calls still matter; and,
 - **unsequenced**, for marking function types which always produce predictable output and have no dependencies upon other data (and other relevant caveats);
- added the **u8** character prefix to match the **u8** string prefix;
- mandated all **u8**, **u**, and **U** strings be UTF-8, UTF-16, and UTF-32, respectively, as defined by ISO/IEC 10646;
- separated the literal, wide literal, and UTF-8 literal, UTF-16 literal, and UTF-32 literal encodings for strings and characters and now have a solely execution-based version of these, particularly execution and wide execution encodings;
- added **mbrtoc8** and **c8rtomb** functions missing from `<uchar.h>`;
- compound literals may also include storage-class specifiers as part of the type to change the lifetime of the compound literal (and possibly turn it into a constant expression);
- added the **constexpr** specifier for object definitions and improved what is recognized as a constant expression in conjunction with the **constexpr** storage-class specifier;
- added support for initialization of objects with empty braces;
- added the **typeof** and **typeof_unqual** operations for deducing the type of an expression;
- improved tag compatibility rules, enabling more types to be compatible with other types;

- added bit-precise integer types `_BitInt( N )` and `unsigned _BitInt( N )`, where *N* can be an integer constant expression whose value is from one to `BITINT_MAXWIDTH`, inclusive.
- improved rules for handling enumerations without underlying types, in particular allowing for enumerations without fixed underlying type to have value representations that have a greater range than `int`;
- added a new colon-delimited type specifier for enumerations to specify a fixed underlying type (and which, subject to an implementation's definitions governing such constructs, adopt the fixed underlying type's rules for padding, alignment, and sizing within structures and unions as well as with bit-fields);
- added a new header `<stdbit.h>` and a suite of bit and byte-handling utilities for portable access to many implementations' most efficient functionality;
- modified existing functions to preserve the **const**-ness of the type placed into the function;
- added a feature to embed binary data as faithfully as possible with a new preprocessor directive `#embed`;
- added a `nullptr` constant and a `nullptr_t` type with a well-defined underlying representation identical to a pointer to `void`;
- added the `__VA_OPT__` specifier and clarified language in the handling of macro invocation and arguments;
- mandated support for variably modified types (but not variable length arrays themselves);
- parameter names may be omitted in function definitions;
- ellipses on functions may appear without a preceding parameter in the parameter list of functions and `va_start` no longer requires such an argument to be passed to it;
- Unicode identifiers allowed in syntax following Unicode Standard Annex, UAX #31;
- added the `memset_explicit` function for making sensitive information inaccessible;
- added `memalignment` function to query the alignment of a pointer;
- certain type definitions (i.e. exact-width integer types such as `int128_t`), bit-precise integer types, and extended integer types may exceed the normal boundaries of `intmax_t` and `uintmax_t` for signed and unsigned integer types, respectively;
- names of functions, macros, and variables in this document, where clarified, are potentially reserved rather than reserved to avoid undefined behavior for a large class of identifiers used by programs existing and to be created;
- mandated support for `call_once`;
- allowed `ptrdiff_t` to be an integer type with a width of at least 16, rather than requiring an integer type with a width of at least 17;
- added the `__has_include` feature for conditional inclusion expression preprocessor directives to check if a header is available for inclusion;
- changed the type qualifiers of the `_Imaginary_I` and `_Complex_I` macros;
- added qualifier preserving macros for `bsearch`, `memchr`, `strchr`, `strpbrk`, `strrchr`, `strstr`, `wcschr`, `wcspbrk`, `wcsrchr`, `wmemchr`, and `wcsstr`;
- added (U+0040, COMMERCIAL AT), \$ (U+0024, DOLLAR SIGN), and ` (U+0060, GRAVE ACCENT, "Backtick") into the source and execution character set;
- enhanced the `auto` type specifier for single object definitions using type inference;

- added the **#elifdef** and **#elifndef** conditional inclusion preprocessor directives;
- added the **#warning** preprocessing directive;
- binary integer literals and appropriate formatting for input/output of binary integer numbers;
- digit separators with **'** (single quotation mark);
- removed conditional support for mixed wide and narrow string literal concatenation;
- added support for additional time bases, as well as **timespec_getres**, in **<time.h>**;
- added support for new interface **timegm** to retrieve the broken-down time, in **<time.h>**;
- zero-sized reallocations with **realloc** are undefined behavior;
- added **free_sized** and **free_aligned_sized** functions;
- added an **unreachable** feature which has undefined behavior if reached during program execution;
- added **printf** and **scanf** length modifiers for **intN_t**, **int_fastN_t**, **uintN_t**, and **uint_fastN_t**.

M.3 Fourth Edition

There were no major changes in the fourth edition (**__STDC_VERSION__** 201710L), only technical corrections and clarifications.

M.4 Third Edition

Major changes in the third edition (**__STDC_VERSION__** 201112L) included:

- conditional (optional) features (including some that were previously mandatory)
- support for multiple threads of execution including an improved memory sequencing model, atomic objects, and thread storage (**<stdatomic.h>** and **<threads.h>**)
- additional floating-point characteristic macros (**<float.h>**)
- querying and specifying alignment of objects (**<stdalign.h>**, **<stdlib.h>**)
- Unicode characters and strings (**<uchar.h>**) (originally specified in ISO/IEC TR 19769:2004)
- type-generic expressions
- static assertions
- anonymous structures and unions
- no-return functions
- macros to create complex numbers (**<complex.h>**)
- support for opening files for exclusive access
- removed the **gets** function (**<stdio.h>**)
- added the **aligned_alloc**, **at_quick_exit**, and **quick_exit** functions (**<stdlib.h>**)
- (conditional) support for bounds-checking interfaces (originally specified in ISO/IEC TR 24731-1:2007)
- (conditional) support for analyzability

M.5 Second Edition

Major changes in the second edition (___STDC_VERSION___ 199901L) included:

- restricted character set support via digraphs and `<iso646.h>` (originally specified in ISO/IEC 9899:1990/Amd 1:1995)
- wide character library support in `<wchar.h>` and `<wctype.h>` (originally specified in ISO/IEC 9899:1990/Amd 1:1995)
- more precise aliasing rules via effective type
- restricted pointers
- variable length arrays
- flexible array members
- **static** and type qualifiers in parameter array declarators
- complex (and imaginary) support in `<complex.h>`
- type-generic math macros in `<tgmath.h>`
- the **long long int** type and library functions
- extended integer types
- increased minimum translation limits
- additional floating-point characteristics in `<float.h>`
- remove implicit **int**
- reliable integer division
- universal character names (`\u` and `\U`)
- extended identifiers
- hexadecimal floating constants and **%a** and **%A** **printf**/**scanf** conversion specifiers
- compound literals
- designated initializers
- `//` comments
- specified width integer types and corresponding library functions in `<inttypes.h>` and `<stdint.h>`
- remove implicit function declaration
- preprocessor arithmetic done in **intmax_t**/**uintmax_t**
- mixed declarations and statements
- new block scopes for selection and iteration statements
- integer constant type rules
- integer promotion rules
- macros with a variable number of arguments (___VA_ARGS___)
- the **vscanf** family of functions in `<stdio.h>` and `<wchar.h>`
- additional math library functions in `<math.h>`

- treatment of error conditions by math library functions (**math_errhandling**)
- floating-point environment access in `<fenv.h>`
- ISO/IEC 60559 (also known as IEC 559 or IEEE 754 arithmetic) support
- trailing comma allowed in **enum** declaration
- **%lf** conversion specifier allowed in **printf**
- inline functions
- the **snprintf** family of functions in `<stdio.h>`
- boolean type in `<stdbool.h>`
- idempotent type qualifiers
- empty macro arguments
- new structure type compatibility rules (tag compatibility)
- additional predefined macro names
- **_Pragma** preprocessing operator
- standard pragmas
- **__func__** predefined identifier
- **va_copy** macro
- additional **strftime** conversion specifiers
- LIA compatibility annex
- deprecate **ungetc** at the beginning of a binary file
- remove deprecation of aliased array parameters
- conversion of array to pointer not limited to lvalues
- relaxed constraints on aggregate and union initialization
- relaxed restrictions on portable header names
- **return** without expression not permitted in function that returns a value (and vice versa)

M.6 First Edition, Amendment 1

Major changes in the amendment to the first edition (**__STDC_VERSION__** 199409L) included:

- addition of the predefined **__STDC_VERSION__** macro
- restricted character set support via digraphs and `<iso646.h>`
- wide character library support in `<wchar.h>` and `<wctype.h>`

Bibliography

- [1] ISO/IEC 14882 Programming languages — C++.
- [2] ISO/IEC 646:1991, *Information technology — ISO 7-bit coded character set for information interchange*.
- [3] ISO/IEC 9945-2:1993, *Information technology — Portable Operating System Interface (POSIX) — Part 2: Shell and Utilities*.
- [4] ISO/IEC TR 10176:2003, *Information technology — Guidelines for the preparation of programming language standards*.
- [5] ISO/IEC 10967-1:2012, *Information technology — Language independent arithmetic — Part 1: Integer and floating point arithmetic*.
- [6] ISO/IEC TR 19769:2004, *Information technology — Programming languages, their environments and system software interfaces — Extensions for the programming language C to support new character data types*.
- [7] ISO/IEC TR 24731-1:2007, *Information technology — Programming languages, their environments and system software interfaces — Extensions to the C library — Part 1: Bounds-checking interfaces*.
- [8] ISO 80000-3, *Quantities and units — Part 3: Space and time*.
- [9] IEC 60559:1989, *Binary floating-point arithmetic for microprocessor systems*.
- [10] ISO/IEC/IEEE 60559:2011, *Floating-point arithmetic*.
- [11] IEEE 754-1985 *IEEE Standard for Binary Floating-Point Arithmetic*.
- [12] IEEE 754-2019 *IEEE Standard for Floating-Point Arithmetic*.
- [13] ANSI/IEEE 854-1987, *American National Standard for Radix-Independent Floating-Point Arithmetic*.
- [14] ANSI X3/TR-1-82 (1982), *American National Dictionary for Information Processing Systems, Information Processing Systems Technical Report*.
- [15] “The C Reference Manual” by Dennis M. Ritchie, a version of which was published in *The C Programming Language* by Brian W. Kernighan and Dennis M. Ritchie, Prentice-Hall, Inc., (1978). Copyright owned by AT&T.
- [16] 1984 */usr/group Standard* by the */usr/group Standards Committee*, Santa Clara, California, USA, November 1984.
- [17] The Unicode Consortium. *Unicode Standard Annex, UAX #31, Unicode Identifier and Pattern Syntax [online]*. Edited by Mark Davis and Robin Leroy. Available at <https://www.unicode.org/reports/tr31>.

Index

- ! (exclamation-mark punctuation), 68
- ! (logical negation operator), 82
- != (inequality operator), 88
- != (not-equal punctuation), 68
- # (hash punctuation), 68
- # preprocessing directive, 187
- # punctuation, 163
- ## (hash-hash punctuation), 68
- #define preprocessing directive, 178
- #elif preprocessing directive, 167
- #elifdef preprocessing directive, 168
- #elifndef preprocessing directive, 168
- #else preprocessing directive, 168
- #embed preprocessing directive, 171
- #endif preprocessing directive, 168
- #error preprocessing directive, 9, 186
- #if preprocessing directive, 167
- #ifdef preprocessing directive, 168
- #ifndef preprocessing directive, 168
- #include preprocessing directive, 11, 170
- #line preprocessing directive, 185
- #pragma preprocessing directive, 186
- #undef preprocessing directive, 183, 193
- [*x*], 7
- [*x*], 7
- & (bitwise AND operator), 88
- && (logical AND operator), 17, 89
- &= (bitwise AND assignment operator), 93
- ' ' (space character), 11, 20, 52, 206, 207, 453
- ((opening parenthesis punctuation), 68
- () (cast operator), 84
- () (function-call operator), 75
- () (parentheses punctuation), 130, 154, 155
- (){} (compound-literal operator), 79
-) (closing parenthesis punctuation), 68
- * (asterisk punctuation), 68, 127, 128
- * (indirection operator), 75, 82
- * (multiplication operator), 84, 554
- *= (asterisk-equal punctuation), 68
- *= (multiplication assignment operator), 93
- + (addition operator), 75, 82, 85, 556
- + (plus punctuation), 68
- + (unary plus operator), 82
- + format flag, 332, 419
- ++ (plus-plus punctuation), 68
- ++ (postfix increment operator), 50, 78
- ++ (prefix increment operator), 50, 81
- += (addition assignment operator), 93
- += (plus-equal punctuation), 68
- , (comma operator), 17, 94
- , (comma punctuation), 68, 75, 97, 105, 109, 116, 136
- (minus punctuation), 68
- (subtraction operator), 85, 556
- (unary minus operator), 82, 527
- format flag, 332, 418
- (minus-minus punctuation), 68
- (postfix decrement operator), 50, 78
- (prefix decrement operator), 50, 81
- = (minus-equal punctuation), 68
- = (subtraction assignment operator), 93
- > (minus-greater punctuation), 68
- > (structure/union pointer operator), 76
- . (dot punctuation), 68, 137
- . (structure/union member operator), 50, 76
- ... (ellipsis punctuation), 68, 130, 179
- / (division operator), 84, 554
- / (slash punctuation), 68
- /* */ (comment delimiters), 70
- // (comment delimiter), 70
- /= (division assignment operator), 93
- /= (slash-equal punctuation), 68
- : (colon punctuation), 68, 105
- :> (alternative spelling of []), 69
- :> (colon greater punctuation), 68
- ; (semicolon punctuation), 68, 97, 105, 153, 155, 156
- < (less punctuation), 68
- < (less-than operator), 87
- <: (alternative spelling of []), 69
- <: (less-colon punctuation), 68
- << (left-shift operator), 86
- << (less-less punctuation), 68
- <<= (left-shift assignment operator), 93
- <<= (less-less equal punctuation), 68
- <= (less-equal punctuation), 68
- <= (less-than-or-equal-to operator), 87
- <% (alternative spelling of {}), 69
- <% (less-percent punctuation), 68
- <assert.h> header, 174–176, 192, 195, 220, 476
- <complex.h> header, 25, 30, 101, 139, 189, 191, 192, 196–204, 387, 388, 458, 476, 477, 555, 557, 573, 575, 610, 613, 620, 688, 689
- <ctype.h> header, 192, 205, 206–208, 458, 478
- <errno.h> header, 150, 192, 209, 458, 478, 637
- <fcntl.h> header, 9, 14, 16, 25, 30, 34, 93, 150, 192, 210, 212, 213, 215–222, 238, 458, 478, 523, 527, 530–533, 541, 545, 547, 690
- <float.h> header, 9, 10, 22, 24, 25, 29, 30, 101, 192, 223, 236, 336, 360, 423, 437, 458, 459, 479, 480, 519, 523, 529, 567, 568,

617, 688, 689	512, 686, 688
<inttypes.h> header, 192, 224 , 225, 226, 334, 420, 458 , 480 , 689	<wchar.h> header, 25, 30, 34, 150, 192, 214, 216, 225, 323, 417 , 418, 423, 424, 428–435, 437, 439–451, 460 , 512 , 514, 593 , 594 , 613, 667 , 668–682, 689, 690
<iso646.h> header, 9, 10, 192, 227 , 480 , 689, 690	<wctype.h> header, 192, 452 , 453–457, 460 , 515 , 689, 690
<limits.h> header, 9, 10, 22 , 23, 39, 40, 192, 228 , 481 , 519, 617	= (equal-sign punctuator), 68 , 97, 109, 137
<locale.h> header, 150, 192, 229 , 230, 458 , 481	= (simple assignment operator), 92
<math.h> header, 9, 25, 30, 34, 73, 149, 150, 192, 214–216, 235 , 236, 238–241, 242 – 269 , 270, 271 – 275 , 276, 277 – 282 , 283, 337, 387–389, 423, 458 , 459, 481 , 492, 493, 522, 523, 529, 535 , 536, 541, 545, 547–551, 555, 565, 575 , 588 – 590 , 600, 610, 613, 617, 620, 689	== (equal-equal punctuator), 68
<setjmp.h> header, 192, 284 , 285, 501	== (equality operator), 88
<signal.h> header, 192, 286 , 288, 459 , 501	> (greater punctuator), 68
<stdalign.h> header, 9, 10, 192, 289 , 501 , 688	> (greater-than operator), 87
<stdarg.h> header, 9, 10, 130, 192, 290 , 291–293, 347–349, 429–431, 501 , 646–649, 670–673	>= (greater-equal punctuator), 68
<stdatomic.h> header, 188, 191, 192, 287, 294 , 295, 298, 299, 301–303, 459 , 501 , 606, 688	>= (greater-than-or-equal-to operator), 87
<stdbit.h> header, 9, 192, 305 , 306–312, 459 , 502 , 687	>> (greater greater punctuator), 68
<stdbool.h> header, 9, 10, 192, 313 , 459 , 504 , 690	>> (right-shift operator), 86
<stdckdint.h> header, 192, 314 , 459 , 504	>>= (greater-greater-equal punctuator), 68
<stddef.h> header, 9, 10, 50, 66, 68, 83, 85, 86, 141, 172, 173, 192, 226, 315 , 316, 317, 345, 504 , 638	>>= (right-shift assignment operator), 93
<stdint.h> header, 9, 10, 22–24, 167, 192, 224, 225, 318 , 320, 321, 333, 340, 341, 420, 425, 459 , 504 , 505, 617, 638 , 689	? (question-mark punctuator), 68
<stdio.h> header, 16, 25, 30, 34, 56, 150, 170, 173, 192, 214, 216, 322 , 326–328, 330, 331, 337, 339, 343–355, 404, 418, 423, 424, 427–429, 431–434, 459 , 505 , 506, 613, 638 , 639–650, 668, 670, 671, 688–690	?: (conditional operator), 17, 90
<stdlib.h> header, 9, 25, 30, 34, 192, 194, 214, 216, 357 , 358, 359, 361, 363–375, 459 , 506 , 508, 565, 590 , 591, 592 , 593 , 613, 637, 651 , 652–657, 688	[(opening bracket punctuator), 68
<stdnoreturn.h> header, 9, 10, 147, 192, 377 , 509	[] (array subscript operator), 75, 82
<string.h> header, 9, 173, 176, 177, 192, 378 , 379–386, 460 , 509 , 658 , 659–664	[] (brackets punctuator), 128, 137
<tgmath.h> header, 34, 192, 387 , 390, 510 , 522, 535 , 563 , 594 , 596, 689	# format flag, 332, 419
<threads.h> header, 149, 150, 189, 191, 192, 392 , 393–400, 460 , 511 , 688	% (percent punctuator), 68
<time.h> header, 192, 392, 402 , 403–408, 446, 460 , 511 , 512, 664 , 665–667, 688	% (remainder operator), 84
<uchar.h> header, 66, 68, 192, 412 , 413–416,	%(alternative spelling of #), 69
	%: (percent-colon punctuator), 68
	%:~: (alternative spelling of ##), 69
	%:~: (percent-percent punctuator), 68
	%= (percent-equal punctuator), 68
	%= (remainder assignment operator), 93
	%> (alternative spelling of }), 69
	%> (percent-greater punctuator), 68
	%A conversion specifier, 334, 408, 421
	%B conversion specifier, 333, 408, 420
	%C conversion specifier, 408
	%D conversion specifier, 409
	%E conversion specifier, 334, 421
	%F conversion specifier, 334, 409, 420
	%G conversion specifier, 334, 409, 421
	%H conversion specifier, 409
	%I conversion specifier, 409
	%M conversion specifier, 409
	%R conversion specifier, 409
	%S conversion specifier, 409
	%T conversion specifier, 409
	%U conversion specifier, 409
	%V conversion specifier, 409
	%W conversion specifier, 409
	%X conversion specifier, 333, 409, 420
	%Y conversion specifier, 409
	%Z conversion specifier, 409
	%[conversion specifier, 342, 426
	%% conversion specifier, 336, 343, 423, 427

- %a conversion specifier, 334, 341, 408, 421, 426
- %b conversion specifier, 333, 408, 420
- %c conversion specifier, 335, 341, 408, 422, 426
- %d conversion specifier, 333, 341, 409, 420, 426
- %e conversion specifier, 334, 341, 409, 421, 426
- %f conversion specifier, 334, 341, 420, 426
- %g conversion specifier, 334, 341, 409, 421, 426
- %h conversion specifier, 409
- %i conversion specifier, 333, 420
- %j conversion specifier, 409
- %m conversion specifier, 409
- %n conversion specifier, 336, 343, 409, 422, 427
- %o conversion specifier, 333, 341, 420, 426
- %p conversion specifier, 336, 342, 409, 422, 427
- %r conversion specifier, 409
- %s conversion specifier, 335, 342, 422, 426
- %t conversion specifier, 409
- %u conversion specifier, 333, 341, 409, 420, 426
- %w conversion specifier, 409
- %x conversion specifier, 333, 341, 409, 420, 426
- %y conversion specifier, 409
- %z conversion specifier, 409
- & (address operator), 50, 82
- & (ampersand punctuator), 68
- &= (ampersand-equal punctuator), 68
- && (ampersand-ampersand punctuator), 68
- \ (backslash character), 11, 19, 64
- \ (backslash escape sequence), 65, 189
- \ " (double-quote escape sequence), 65, 67, 189
- \ ' (single-quote escape sequence), 65, 67
- \0 (null character), 19, 66, 67
 - padding of binary stream, 324
- \? (question-mark escape sequence), 65
- \U (universal character names), 56
- \a (alert escape sequence), 21, 65
- \b (backspace escape sequence), 21, 65
- \ (escape character), 64
- \f (form-feed escape sequence), 21, 65, 207
- \n (new-line escape sequence), 21, 65, 207
- \octal digit (octal-character escape sequence), 65
- \r (carriage-return escape sequence), 21, 65, 207
- \t (horizontal-tab escape sequence), 21, 65, 206, 207, 453
- \u (universal character names), 56
- \v (vertical-tab escape sequence), 21, 65, 207
- _Alignas, 53
- _Alignof, 53
- _Atomic, 41
- _Atomic keyword, 53
- _Atomic type qualifier, 120
- _Atomic type specifier, 117
- _BitInt, 23, 39, 47, 49, 53, 59, 60, 103, 104, 687
- _BitInt keyword, 53
- _BitInt type, 103
- _Bool, 53
- _C identifier suffix, 321, 459, 505
- _Complex, 26, 40, 49, 53, 101–104, 196, 553, 555, 567, 573
- _Complex keyword, 53
- _Complex type, 40, 103, 196, 553
- _Complex_I macro, 196, 476, 557, 687
- _DECIMAL_DIG identifier suffix, 29, 336, 360, 423, 437, 529, 530
- _Decimal128, 39
- _Decimal128 keyword, 53
- _Decimal128 type, 103
- _Decimal128x type, 566
- _Decimal32, 39
- _Decimal32 keyword, 53
- _Decimal32 type, 103
- _Decimal32_t type, 235, 492, 615
- _Decimal64, 39
- _Decimal64 keyword, 53
- _Decimal64 type, 103
- _Decimal64_t type, 235, 492, 615
- _Decimal64x type, 566
- _DecimalN type, 565
- _DecimalN_t type, 576
- _DecimalNx type, 567
- _Exit function, 287, 369, 370, 507, 606, 617
- _Float128_t type, 576, 577
- _Float128x type, 566
- _Float16_t type, 576, 577
- _Float32_t type, 576, 577
- _Float32x type, 566
- _Float64_t type, 576, 577
- _Float64x type, 566
- _FloatN type, 565
- _FloatN_t type, 576
- _FloatNx type, 567
- _Generic, 53, 74, 113, 121, 136, 317, 389
- _Generic keyword, 53
- _H__ identifier suffix, 192
- _IOFBF macro, 322, 330, 331, 505
- _IOLBF macro, 322, 331, 505
- _IONBF macro, 322, 330, 331, 505
- _Imaginary keyword, 53
- _Imaginary type, 196, 553
- _Imaginary_I macro, 196, 203, 476, 557, 687
- _MAX identifier suffix, 23, 47, 320, 321, 459, 480, 505, 569
- _MIN identifier suffix, 23, 320, 321, 459, 480, 505, 570
- _Noreturn, 124
- _Noreturn attribute, 143, 147
- _Noreturn keyword, 53
- _PRINTF_NAN_LEN_MAX macro, 323, 505
- _Pragma operator, 189
- _Static_assert, 53
- _Thread_local, 53

<code>__WIDTH</code> identifier suffix, 23, 320, 321, 459, 505	<code>__STDC_VERSION_SETJMP_H__</code> macro, 284, 501
<code>__STDC__</code> identifier prefix, 190	<code>__STDC_VERSION_STDARG_H__</code> macro, 290, 501
<code>__STDC_VERSION__</code> identifier prefix, 192	<code>__STDC_VERSION_STDATOMIC_H__</code> macro, 294, 501
<code>__DATE__</code> macro, 187, 615	<code>__STDC_VERSION_STDBIT_H__</code> macro, 305, 502
<code>__FILE__</code> macro, 169, 187, 195	<code>__STDC_VERSION_STDCCKDINT_H__</code> macro, 314, 504
<code>__LINE__</code> macro, 185, 187, 195, 599	<code>__STDC_VERSION_STDDEF_H__</code> macro, 315, 504
<code>__STDC_ANALYZABLE__</code> macro, 188, 684	<code>__STDC_VERSION_STDINT_H__</code> macro, 318, 505
<code>__STDC_EMBED_EMPTY__</code> macro, 167, 187	<code>__STDC_VERSION_STDIO_H__</code> macro, 322, 505
<code>__STDC_EMBED_FOUND__</code> macro, 166, 187	<code>__STDC_VERSION_STDLIB_H__</code> macro, 357, 507
<code>__STDC_EMBED_NOT_FOUND__</code> macro, 166, 187	<code>__STDC_VERSION_STRING_H__</code> macro, 378, 509
<code>__STDC_ENDIAN_BIG__</code> macro, 305, 306, 502	<code>__STDC_VERSION_TGMATH_H__</code> macro, 387
<code>__STDC_ENDIAN_LITTLE__</code> macro, 305, 306, 502	<code>__STDC_VERSION_TIME_H__</code> macro, 402, 511
<code>__STDC_ENDIAN_NATIVE__</code> macro, 279–281, 305, 502, 588–590, 592–594, 616	<code>__STDC_VERSION_UCHAR_H__</code> macro, 412, 512
<code>__STDC_HOSTED__</code> macro, 187	<code>__STDC_VERSION_WCHAR_H__</code> macro, 417, 512
<code>__STDC_IEC_559_COMPLEX__</code> (obsolete), 24, 188, 189–191, 553	<code>__STDC_VERSION__</code> macro, 188, 685, 686, 688–690
<code>__STDC_IEC_559_COMPLEX__</code> macro, 553	<code>__STDC_WANT_IEC_60559_EXT__</code> macro, 235, 479, 492, 529, 549–551
<code>__STDC_IEC_559__</code> (obsolete), 24, 188, 190, 191, 492, 522	<code>__STDC_WANT_IEC_60559_TYPES_EXT__</code> macro, 477, 480, 493, 508, 514, 568, 573, 588–590, 591, 592–594, 596
<code>__STDC_IEC_60559_BFP__</code> macro, 9, 24, 188, 189, 191, 492, 522, 549–551, 564–566	<code>__STDC_WANT_LIB_EXT1__</code> macro, 478, 504–506, 508, 509, 512, 514, 636, 637, 639–655, 658–680
<code>__STDC_IEC_60559_COMPLEX__</code> macro, 24, 102, 188, 189, 191, 553	<code>__STDC__</code> macro, 168, 187
<code>__STDC_IEC_60559_DFP__</code> macro, 9, 30, 188, 189, 191, 215, 219, 220, 235, 242–265, 266, 267–269, 270, 271–275, 276, 277–280, 359, 361, 390, 437, 479, 480, 486, 492, 508, 514, 520–522, 549–551, 564–566	<code>__TIME__</code> macro, 188, 615
<code>__STDC_IEC_60559_TYPES__</code> macro, 188, 189, 191, 477, 480, 493, 508, 514, 564–566	<code>__VA_ARGS__</code> identifier, 178, 179, 180, 185, 689
<code>__STDC_ISO_10646__</code> macro, 188, 612	<code>__VA_OPT__</code> identifier, 178, 179–181, 687
<code>__STDC_LIB_EXT1__</code> macro, 188, 189, 191, 478, 504–506, 508, 509, 512, 514, 636	<code>__bool_true_false_are_defined</code> (obsolete), 313, 459, 504
<code>__STDC_MB_MIGHT_NEQ_WC__</code> macro, 45, 188, 315	<code>__cplusplus</code> macro, 168, 187
<code>__STDC_NO_ATOMICS__</code> macro, 188, 294, 501	<code>__deprecated__</code> attribute, 143
<code>__STDC_NO_COMPLEX__</code> macro, 189, 196, 476	<code>__func__</code> identifier, 55, 56, 195, 602, 690
<code>__STDC_NO_THREADS__</code> macro, 16, 189, 392, 511	<code>__has_c_attribute</code> , 142, 144–147, 150, 167, 168, 187, 685
<code>__STDC_NO_VLA__</code> macro, 189	<code>__has_c_attribute</code> operator, 187
<code>__STDC_UTF_16__</code> macro, 187	<code>__has_embed</code> , 52, 69, 167, 169, 187
<code>__STDC_UTF_32__</code> macro, 187	<code>__has_embed</code> operator, 187
<code>__STDC_VERSION_ASSERT_H__</code> macro, 195, 476	<code>__has_include</code> , 52, 69, 167, 168, 187, 687
<code>__STDC_VERSION_COMPLEX_H__</code> macro, 196, 476	<code>__has_include</code> operator, 187
<code>__STDC_VERSION_FENV_H__</code> macro, 210, 478	<code>__limit__</code> embed parameter, 165
<code>__STDC_VERSION_FLOAT_H__</code> macro, 223, 479	<code>__nodiscard__</code> attribute, 142
<code>__STDC_VERSION_INTTYPES_H__</code> macro, 224, 480	<code>__explicit</code> identifier suffix, 294, 302, 502
<code>__STDC_VERSION_LIMITS_H__</code> macro, 228, 481	<code>__r</code> identifier suffix, 406
<code>__STDC_VERSION_MATH_H__</code> macro, 235, 481	<code>__t</code> identifier suffix, 318, 319, 321, 459, 493, 505, 576, 577, 688
	<code>wchar_t</code> character constant, 64

- wchar_t** string literal, 67
{ } (braces punctuation), 109, 116, 136, 153
{ } (compound-literal operator), 79
{ (opening brace punctuation), 68
} (closing brace punctuation), 68
] (closing bracket punctuation), 68
^ (bitwise exclusive OR operator), 89
^ (caret punctuation), 68
^= (bitwise exclusive OR assignment operator), 93
^= (caret-equal punctuation), 68
| (bitwise inclusive OR operator), 89
| (vertical-line punctuation), 68
|= (bitwise inclusive OR assignment operator), 93
|= (vertical-line-equal punctuation), 68
|| (logical OR operator), 17, 90
|| (vertical-vertical punctuation), 68
~ (bitwise complement operator), 82
~ (tilde punctuation), 68
0 format flag, 332, 419
- abort** function, 148, 195, 286, 287, 295, 325, 368, 507, 606, 607, 617, 652
abort_handler_s function, 508, 652
abs function, 193, 372, 507
abs macro, 193
absolute-value function
 complex, 201, 562
 integer, 225, 372
 real, 257, 541
abstract declarator, 132
abstract machine, 14
access, 8, 121
access (verb), 3
acos function, 214, 242, 243, 388, 482, 529, 536, 575
acos type-generic macro, 388
acosd128 function, 216, 243, 487
acosd32 function, 216, 242, 487
acosd64 function, 216, 243, 487
acosdN function, 579
acosdNx function, 579
acosf function, 214, 242, 482
acosfN function, 579
acosfNx function, 579
acosh function, 247, 248, 388, 482, 529, 538
acosh type-generic macro, 388
acoshd128 function, 247, 487
acoshd32 function, 247, 487
acoshd64 function, 247, 487
acoshdN function, 580
acoshdNx function, 580
acoshf function, 247, 390, 482
acoshfN function, 580
acoshfNx function, 580
acoshl function, 247, 482
acosl function, 214, 242, 482, 575
acospi function, 245, 482, 529, 537
acospi type-generic macro, 388
acospid128 function, 245, 487
acospid32 function, 245, 487
acospid64 function, 245, 487
acospidN function, 579
acospidNx function, 579
acospiif function, 245, 482
acospiifN function, 579
acospiifNx function, 579
acospil function, 245, 482
acquire fence, 298
acquire operation, 17
active position, 21
add and round to narrower type, 275
addition assignment operator (**+=**), 93
addition operator (**+**), 75, 82, 85, 556
additive expression, 85, 556
address constant, 96
address operator (**&**), 50, 82
address-free, 299
aggregate initialization, 138
aggregate type, 41
alert, 21
alert escape sequence (**\a**), 21, 65
aliasing, 72
alignas, 125
alignas keyword, 53
aligned_alloc function, 365, 366, 367, 507, 600, 609, 617, 688
alignment, 3, 45, 366
 pointer, 41, 51
 structure/union member, 106
alignment header, 289
alignment of memory, 375
alignment specifier, 125
alignof keyword, 53
alignof operator, 81, 82
allocated storage
 order and contiguity, 365
alternative spellings header, 227
and macro, 227, 481
AND operator
 bitwise (**&**), 88
 bitwise assignment (**&=**), 93
 logical (**&&**), 89
AND operator
 logical (**&&**), 17
and_eq macro, 227, 481
anonymous structure, 106
anonymous union, 106
argc (**main** function parameter), 13
argument, 3
 array, 160
 complex, 202

- asinl** function, 243, 482
- asinp** function, 245, 482, 529, 537
- asinp** type-generic macro, 388
- asinp128** function, 245, 487
- asinp32** function, 245, 487
- asinp64** function, 245, 487
- asinpN** function, 579
- asinpNx** function, 579
- asinpif** function, 245, 482
- asinpifN** function, 579
- asinpifNx** function, 579
- asinpil** function, 245, 482
- assert** macro, 145, 175, 177, 195, 220, 476, 605, 615
- assignment
 - compound, 93
 - conversion, 92
 - expression, 91
 - operator, 50, 91
 - simple, 92
- associativity of operator, 72
- asterisk punctuator (*), 127, 128
- at_quick_exit** function, 368, 369, 370, 507, 600, 609, 688
- atan** function, 243, 337, 388, 423, 482, 529, 536, 563
- atan** type-generic macro, 388, 563
- atan2** function, 243, 244, 482, 529, 535, 536, 537, 557
- atan2** type-generic macro, 388
- atan2d128** function, 244, 487
- atan2d32** function, 244, 487
- atan2d64** function, 244, 487
- atan2dN** function, 579
- atan2dNx** function, 579
- atan2f** function, 243, 482
- atan2fN** function, 579
- atan2fNx** function, 579
- atan2l** function, 243, 482
- atan2pi** function, 246, 482, 529, 535, 537
- atan2pi** type-generic macro, 388
- atan2pid128** function, 246, 487
- atan2pid32** function, 246, 487
- atan2pid64** function, 246, 487
- atan2pidN** function, 579
- atan2pidNx** function, 579
- atan2pif** function, 246, 482
- atan2pifN** function, 579
- atan2pifNx** function, 579
- atan2pil** function, 246, 482
- atand128** function, 243, 487
- atand32** function, 243, 487
- atand64** function, 243, 487
- atandN** function, 579
- atandNx** function, 579
- atanf** function, 243, 482

- atanfN** function, 579
- atanfNx** function, 579
- atanh** function, 248, 388, 482, 529, 538, 563
- atanh** type-generic macro, 388, 563
- atanhd128** function, 248, 487
- atanhd32** function, 248, 487
- atanhd64** function, 248, 487
- atanhdN** function, 580
- atanhdNx** function, 580
- atanhf** function, 248, 482
- atanhfN** function, 580
- atanhfNx** function, 580
- atanhl** function, 248, 482
- atanl** function, 243, 390, 482
- atanpi** function, 245, 246, 482, 529, 537
- atanpi** type-generic macro, 388
- atanpid128** function, 246, 487
- atanpid32** function, 246, 487
- atanpid64** function, 246, 487
- atanpidN** function, 579
- atanpidNx** function, 579
- atanpif** function, 246, 482
- atanpifN** function, 579
- atanpifNx** function, 579
- atanpil** function, 246, 482
- atexit** function, 368, 369, 370, 507, 600, 609, 619
- atof** function, 214, 357, 358, 507
- atoi** function, 194, 357, 358, 507
- atol** function, 357, 358, 507
- atoll** function, 357, 358, 507
- atomic lock-free macro, 294, 299
- atomic operation, 17
- atomic type, 14, 41, 42, 50, 76, 78, 93, 117, 188, 300
- ATOMIC_** identifier prefix, 459
- atomic_** identifier prefix, 459
- atomic_bool** type, 300, 302, 501
- ATOMIC_BOOL_LOCK_FREE** macro, 294, 501
- atomic_char** type, 300, 501
- atomic_char16_t** type, 300, 501
- ATOMIC_CHAR16_T_LOCK_FREE** macro, 294, 501
- atomic_char32_t** type, 300, 501
- ATOMIC_CHAR32_T_LOCK_FREE** macro, 294, 501
- atomic_char8_t** type, 300, 501
- ATOMIC_CHAR8_T_LOCK_FREE** macro, 294, 501
- ATOMIC_CHAR_LOCK_FREE** macro, 294, 501
- atomic_compare_exchange_strong** function, 78, 93, 94, 301, 302, 502
- atomic_compare_exchange_strong_explicit** function, 301, 502
- atomic_compare_exchange_weak** function, 301, 302, 502
- atomic_compare_exchange_weak_explicit** function, 301, 502
- atomic_exchange** function, 301, 502
- atomic_exchange_explicit** function, 301, 502
- atomic_fetch_** function, 302, 502
- atomic_flag** type, 294, 295, 303, 501, 502
- atomic_flag_clear** function, 303, 304, 502
- atomic_flag_clear_explicit** function, 303, 502
- ATOMIC_FLAG_INIT** macro, 294, 303, 501
- atomic_flag_test_and_set** function, 303, 502
- atomic_flag_test_and_set_explicit** function, 303, 502
- atomic_init** function, 295, 502
- atomic_int** type, 295, 300, 501
- atomic_int_fast16_t** type, 300, 502
- atomic_int_fast32_t** type, 300, 502
- atomic_int_fast64_t** type, 300, 502
- atomic_int_fast8_t** type, 300, 502
- atomic_int_least16_t** type, 300, 501
- atomic_int_least32_t** type, 300, 501
- atomic_int_least64_t** type, 300, 501
- atomic_int_least8_t** type, 300, 501
- ATOMIC_INT_LOCK_FREE** macro, 294, 501
- atomic_intmax_t** type, 300, 502
- atomic_intptr_t** type, 300, 502
- atomic_is_lock_free** function, 287, 299, 502, 606
- atomic_llong** type, 300, 501
- ATOMIC_LLONG_LOCK_FREE** macro, 294, 501
- atomic_load** function, 301, 302, 502
- atomic_load_explicit** function, 297, 298, 301, 502
- atomic_long** type, 300, 501
- ATOMIC_LONG_LOCK_FREE** macro, 294, 501
- ATOMIC_POINTER_LOCK_FREE** macro, 294, 501
- atomic_ptrdiff_t** type, 300, 502
- atomic_schar** type, 300, 501
- atomic_short** type, 300, 501
- ATOMIC_SHORT_LOCK_FREE** macro, 294, 501
- atomic_signal_fence** function, 299, 502
- atomic_size_t** type, 300, 502
- atomic_store** function, 301, 502
- atomic_store_explicit** function, 297, 298, 301, 502
- atomic_thread_fence** function, 149, 298, 299, 502
- atomic_uchar** type, 300, 501
- atomic_uint** type, 300, 501
- atomic_uint_fast16_t** type, 300, 502
- atomic_uint_fast32_t** type, 300, 502
- atomic_uint_fast64_t** type, 300, 502
- atomic_uint_fast8_t** type, 300, 502
- atomic_uint_least16_t** type, 300, 501

- atomic_uint_least32_t** type, 300, 501
- atomic_uint_least64_t** type, 300, 501
- atomic_uint_least8_t** type, 300, 501
- atomic_uintmax_t** type, 300, 502
- atomic_uintptr_t** type, 300, 502
- atomic_ullong** type, 300, 501
- atomic_ulong** type, 300, 501
- atomic_ushort** type, 300, 501
- atomic_wchar_t** type, 300, 501
- ATOMIC_WCHAR_T_LOCK_FREE** macro, 294, 501
- atomics header, 294, 459
- attribute, 142
 - _Noreturn**, 143, 147
 - __deprecated__**, 143
 - __nodiscard__**, 142
 - deprecated**, 98, 107, 143, 145, 146, 406, 407, 511, 685, 686
 - fallthrough**, 143, 146, 147, 168, 685, 686
 - for function type, 148
 - maybe_unused**, 143, 145, 685, 686
 - nodiscard**, 142, 143, 144, 145, 685, 686
 - noreturn**, 125, 143, 147, 148, 285, 368–370, 398, 501, 507, 509, 511, 685, 686
 - reproducible**, 143, 148, 150, 151, 604, 685, 686
 - unsequenced**, 143, 148, 150, 151, 306–312, 502–504, 604, 685, 686
- attribute declaration, 97
- attribute prefixed token, 142
- attribute token, 142
- auto**, 53, 98, 99, 100, 129, 134, 135, 136, 159, 190, 687
- auto** storage-class specifier, 53, 190
- auto** type specifier, 190
- automatic storage duration, 21, 38

- backslash character (\), 11, 19, 64
- backslash escape sequence (\), 65, 189
- backspace, 21
- backspace escape sequence (\b), 21, 65
- basic character set, 4, 5, 19
- basic type, 40
- behavior, 3
- big-endian, 305
- binary constant, 57
- binary digit, 58
- binary floating type, 566
- binary prefix, 58
- binary resource inclusion, 171
- binary stream, 324, 352–354
- bit, 4
 - high-order, 4, 5
 - low-order, 5
- bit and byte utilities header, 459
- bit and byte utility, 305
- bit-field, 106
- bit-precise integer suffix, **wb** or **WB**, 59
- bit-precise integer type, 39
- bit-precise signed integer type, 39
- bit-precise unsigned integer type, 39
- bitand** macro, 227, 481
- BITINT_MAXWIDTH** macro, 23, 104, 481, 519, 687
- bitor** macro, 227, 481
- bitwise operator, 72
 - AND**, 88
 - AND** assignment (&=), 93
 - complement (~), 82
 - exclusive OR, 89
 - exclusive OR assignment (^=), 93
 - inclusive OR, 89
 - inclusive OR assignment (|=), 93
 - shift, 86
- blank character, 206
- block, 152, 153
 - primary, 152
 - secondary, 152
- block scope, 35
- block structure, 35
- bold type** convention, 35
- bool** keyword, 53
- bool** type, 47, 103
- bool** type conversion, 47
- BOOL_MAX** macro, 481, 519
- BOOL_WIDTH** macro, 22, 481, 519
- boolean type, 47
- boolean type and values header, 313, 459
- boolean type conversion, 46, 47
- bounded undefined behavior, 8
- braces punctuator ({ }), 109, 116, 136, 153
- brackets operator ([]), 75, 82
- brackets punctuator ([]), 128, 137
- branch cut, 197
- break** keyword, 53
- break** statement, 158
- broken-down time, 403, 404–408, 665, 667
- bsearch** macro, 371, 372, 459, 507, 600, 610, 687
- bsearch_s** macro, 508, 653, 654
- btowc** function, 422, 423, 447, 513
- BUFSIZ** macro, 322, 325, 330, 505
- byte, 4, 83
- byte input/output function, 323
- byte-oriented stream, 324

- c** identifier prefix, 387, 388
- C** program, 11
- c16rtomb** function, 414, 415, 512
- c32rtomb** function, 416, 512
- c8rtomb** function, 413, 512, 686
- cabs** function, 201, 387, 388, 477, 557
- cabs** function
 - type-generic macro for, 388
- cabsf** function, 201, 390, 477

- cabsfN** function, 574
- cabsfNx** function, 574
- cabsl** function, 201, 477
- cacos** function, 197, 198, 388, 476, 558
- cacos** function
 - type-generic macro for, 388
- cacosh** function, 197, 476
- cacoshfN** function, 574
- cacoshfNx** function, 574
- cacosh** function, 199, 388, 476, 558, 559
- cacosh** function
 - type-generic macro for, 388
- cacoshf** function, 199, 476
- cacoshfN** function, 574
- cacoshfNx** function, 574
- cacoshl** function, 199, 476
- cacosl** function, 197, 476
- cacospi** function, 458
- calendar time, 402, 403–405, 407, 408, 666, 667
- call by value, 75
- call_once** function, 357, 392, 393, 507, 511, 687
- calloc** function, 365, 366, 367, 507, 600, 609, 617
- canonical representation, 25
- canonicalize** family, 33, 34, 270
- canonicalize** function, 25, 270, 388, 485, 525, 548
- canonicalized128** function, 270, 491
- canonicalized32** function, 270, 491
- canonicalized64** function, 270, 491
- canonicalizedN** function, 585
- canonicalizedNx** function, 585
- canonicalizef** function, 270, 485
- canonicalizefN** function, 585
- canonicalizefNx** function, 585
- canonicalizel** function, 270, 485
- carg** function, 202, 390, 477, 557, 563, 594
- carg** type-generic macro, 388, 563
- cargf** function, 202, 477
- cargfN** function, 574
- cargfNx** function, 574
- cargl** function, 202, 477
- carriage return, 21
- carriage-return escape sequence (`\r`), 21, 65, 207
- carries a dependency, 17
- case** keyword, 53
- case** label, 153, 155
- case mapping function
 - character, 207
 - extensible wide character, 456
 - wide character, 456
- casin** function, 198, 388, 476, 557
- casin** function
 - type-generic macro for, 388
- casinf** function, 198, 476
- casinfN** function, 574
- casinfNx** function, 574
- casinh** function, 199, 388, 476, 557, 559
- casinh** function
 - type-generic macro for, 388
- casinhf** function, 199, 476
- casinhfN** function, 574
- casinhfNx** function, 574
- casinhl** function, 199, 476
- casinl** function, 198, 476
- casinpi** function, 458
- cast, 84
- cast expression, 83
- cast operator (`()`), 84
- catan** function, 198, 388, 476, 557
- catan** function
 - type-generic macro for, 388
- catanf** function, 198, 476
- catanfN** function, 574
- catanfNx** function, 574
- catanh** function, 199, 200, 388, 476, 557, 559
- catanh** function
 - type-generic macro for, 388
- catanhf** function, 200, 476
- catanhfN** function, 574
- catanhfNx** function, 574
- catanhl** function, 200, 476
- catanl** function, 198, 476
- catanpi** function, 458
- cbrt** function, 74, 257, 389, 484, 541
- cbrt** type-generic macro, 388
- cbrtd128** function, 257, 489
- cbrtd32** function, 257, 489
- cbrtd64** function, 257, 489
- cbrtdN** function, 583
- cbrtdNx** function, 583
- cbrtf** function, 74, 257, 389, 484
- cbrtfN** function, 583
- cbrtfNx** function, 583
- cbrtl** function, 74, 257, 389, 484
- ccompoundn** function, 458
- ccos** function, 198, 388, 476, 557
- ccos** function
 - type-generic macro for, 388
- ccosf** function, 198, 476
- ccosf64x** function, 596
- ccosfN** function, 574
- ccosfNx** function, 574
- ccosh** function, 200, 388, 476, 557, 560
- ccosh** function
 - type-generic macro for, 388
- ccoshf** function, 200, 476
- ccoshfN** function, 574
- ccoshfNx** function, 574
- ccoshl** function, 200, 476

- ccosl** function, 198, 476
- ccospi** function, 458
- ceil** function, 236, 262, 484, 523, 544, 545, 546
- ceil** type-generic macro, 388
- ceild128** function, 34, 262, 489
- ceild32** function, 34, 262, 489
- ceild64** function, 34, 262, 489
- ceildN** function, 584
- ceildNx** function, 584
- ceilf** function, 262, 484
- ceilfN** function, 584
- ceilfNx** function, 584
- ceiling, 7
- ceilL** function, 262, 266, 484
- cerf** function, 458
- cerfc** function, 458
- cexp** function, 200, 201, 388, 476, 561, 562
- cexp** function
 - type-generic macro for, 388
- cexp10** function, 458
- cexp10m1** function, 458
- cexp2** function, 458
- cexp2m1** function, 458
- cexpf** function, 201, 476
- cexpfN** function, 574
- cexpfNx** function, 574
- cexpl** function, 201, 476
- cexpm1** function, 458
- change history, 685
- char** keyword, 53
- char** type, 103
- char** type conversion, 46–48
- char16_t** type, 65–68, 138, 187, 300, 412, 414, 512
- char32_t** type, 65–68, 138, 187, 300, 412, 415, 416, 512
- char8_t** type, 65–68, 101, 300, 412, 413, 512
- CHAR_BIT** macro, 23, 42, 105, 171–173, 175, 481, 519
- CHAR_MAX** macro, 24, 231, 232, 481, 519
- CHAR_MIN** macro, 24, 40, 481, 519
- CHAR_WIDTH** macro, 23, 481, 519
- character, 5
 - extended, 19
 - graphic, 20
- character array initialization, 138
- character case mapping function, 207
 - extensible wide character, 456
 - wide character, 456
- character classification function, 205
 - extensible wide character, 455
 - wide character, 452
- character constant, 12, 19, 64
- character display semantics, 21
- character handling header, 205, 230, 458
- character input/output function, 349, 650
 - wide character, 431
- character set, 19
- character string literal, *see* string literal
- character type, 40, 138
- character type conversion, 46
- characteristics of floating types header, 223, 458
- characteristics of integer types header, 228
- checked arithmetic function, 459
- checked integer arithmetic header, 314
- cimag** function, 202, 203, 204, 477, 555, 556, 557, 563, 594
- cimag** type-generic macro, 388, 563
- cimagf** function, 202, 477
- cimagfN** function, 574
- cimagfNx** function, 574
- cimagl** function, 202, 390, 477
- cis** function, 558
- ckd_** identifier prefix, 459
- ckd_add** macro, 314, 504
- ckd_mul** macro, 314, 504
- ckd_sub** macro, 314, 504
- classification function
 - character, 205
 - extensible wide character, 455
 - floating-point, 239
 - wide character, 452
- clearerr** function, 355, 506
- clgamma** function, 458
- clock** function, 402, 403, 406, 511, 617
- clock_t** type, 402, 403, 511, 617
- CLOCKS_PER_SEC** macro, 402, 403, 406, 511
- clog** function, 201, 388, 477, 562
- clog** function
 - type-generic macro for, 388
- clog10** function, 458
- clog10p1** function, 458
- clog1p** function, 458
- clog2** function, 458
- clog2p1** function, 458
- clogf** function, 201, 390, 477
- clogfN** function, 574
- clogfNx** function, 574
- clogl** function, 201, 477
- clogp1** function, 458
- closing, 325
- CMPLX** macro, 102, 196, 202, 203, 204, 477
- CMPLXF** macro, 202, 203, 477, 478, 574
- CMPLXFN** macro, 574
- CMPLXFNx** macro, 574
- CMPLXL** macro, 203, 477
- cnd_** identifier prefix, 460
- cnd_broadcast** function, 393, 394, 395, 511
- cnd_destroy** function, 394, 511
- cnd_init** function, 394, 511
- cnd_signal** function, 394, 395, 511

- cnd_t** type, 392, 393–395, 511
- cnd_timedwait** function, 394, 395, 511
- cnd_wait** function, 394, 395, 511
- code point, 517
- coefficient, 32
- collating sequence, 19
- colon punctuation (:), 105
- comma operator (,), 17, 94
- comma punctuation (,), 75, 97, 105, 109, 116, 136
- command processor, 370
- comment, 11, 52, 70
- comment delimiters (`/* */` and `//`), 70
- common definitions header, 315
- common extension, 618
- common initial sequence, 77
- common real type, 48
- common warning, 12, 597
- comparison
 - pointer, 87
- comparison function, 371, 372, 653, 654, 655
 - string, 381
 - wide string, 441
- comparison macro, 281
- compatible type, 43, 104, 121, 127
- compl** macro, 227, 481
- complement operator (~), 82
- complete, 38
- complete type, 38
- complex arithmetic header, 196, 458
- complex** macro, 139, 196–204, 390, 476–478, 555, 573, 574, 596, 600
- complex number, 40, 553
- complex type, 40, 103, 189, 553
- complex type conversion, 48
- complex type domain, 40
- components of time, 402, 665
- composite type, 44
- compound assignment, 93
- compound literal, 78, 79
- compound literal constant, 95
- compound statement, 153
- compound-literal operator (`() {}`), 79
- compoundn** function, 257, 484, 528, 542
- compoundn** type-generic macro, 388
- compoundnd128** function, 257, 489
- compoundnd32** function, 257, 489
- compoundnd64** function, 257, 489
- compoundndN** function, 583
- compoundndNx** function, 583
- compoundnf** function, 257, 484
- compoundnfN** function, 583
- compoundnfNx** function, 583
- compoundnl** function, 257, 484
- concatenation function
 - string, 380, 660
 - wide string, 441, 677
- conceptual model, 11
- conditional expression inclusion preprocessing directive, 22, 25, 166, 193
- conditional feature, 9, 39, 40, 41, 128, 188, 191, 522, 553, 636, 684
- conditional inclusion, 165
- conditional inclusion preprocessing directive, 166
- conditional operator (`? :`), 17, 90
- conflict, 16
- conformance, 9
- conforming freestanding implementation, 9
- conforming hosted implementation, 9
- conforming implementation, 9
- conforming program, 9
- conj** function, 203, 477, 557, 558–562, 594
- conj** type-generic macro, 388
- conjf** function, 203, 477
- conjfN** function, 574
- conjfNx** function, 574
- conjl** function, 203, 477
- const**, 41
- const** keyword, 53
- const** type qualifier, 120
- const-qualified type, 41, 50, 121
- constant, 57
 - binary, 57
 - character, 64
 - enumeration, 35, 63
 - floating, 59
 - hexadecimal, 57
 - integer, 57
 - octal, 57
- constant expression, 95, 531
- constants
 - as primary expression, 73
- constexpr**, 36–38, 53, 79, 95, 96, 98–103, 137, 190, 287, 531, 532, 686
- constexpr** storage-class specifier, 53
- constraint, 5, 9
- constraint_handler_t** type, 508, 651
- consume operation, 17
- content of structure/union/enumeration, 116
- contiguity of allocated storage, 365
- continue**, 53, 156, 157, 158
- continue** keyword, 53
- continue** statement, 157
- contracted, 73
- contracted expression, 73, 239, 530
- control character, 20, 205
- control wide character, 452
- conversion, 46
 - arithmetic operand, 46
 - array, 49, 50
 - array argument, 160

- array parameter, 160
- boolean, 47
- boolean, characters, and integer, 46
- by assignment, 92
- by **return** statement, 158
- complex type, 48
- explicit, 46
- function, 50
- function argument, 76, 160
- function designator, 49
- function parameter, 160
- imaginary, 553
- imaginary and complex, 553
- implicit, 46
- lvalue, 49
- nullptr_t**, 51
- pointer, 50
- real and complex, 48
- real and imaginary, 553
- real floating and integer, 47, 529
- real floating type, 48
- signed and unsigned integer, 47
- usual arithmetic, *see* usual arithmetic conversion
- void** type, 50
- conversion function
 - multibyte/wide character, 373, 655
 - extended, 447, 680
 - restartable, 412, 448, 680
 - multibyte/wide string, 374, 656
 - restartable, 450, 681
 - numeric, 225, 357
 - wide string, 226, 435
 - single byte/wide character, 447
 - time, 406
 - wide character, 446
- conversion functions
 - time, 665
- conversion specifier, 332, 339, 418, 424
 - %A, 334, 408, 421
 - %B, 333, 408, 420
 - %C, 408
 - %D, 409
 - %E, 334, 421
 - %F, 334, 409, 420
 - %G, 334, 409, 421
 - %H, 409
 - %I, 409
 - %M, 409
 - %R, 409
 - %S, 409
 - %T, 409
 - %U, 409
 - %V, 409
 - %W, 409
 - %X, 333, 409, 420
 - %Y, 409
 - %Z, 409
 - %, 336, 343, 423, 427
 - %a, 334, 341, 408, 421, 426
 - %b, 333, 408, 420
 - %c, 335, 341, 408, 422, 426
 - %d, 333, 341, 409, 420, 426
 - %e, 334, 341, 409, 421, 426
 - %f, 334, 341, 420, 426
 - %g, 334, 341, 409, 421, 426
 - %h, 409
 - %i, 333, 420
 - %j, 409
 - %m, 409
 - %n, 336, 343, 409, 422, 427
 - %o, 333, 341, 420, 426
 - %p, 336, 342, 409, 422, 427
 - %r, 409
 - %s, 335, 342, 422, 426
 - %t, 409
 - %u, 333, 341, 409, 420, 426
 - %w, 409
 - %x, 333, 341, 409, 420, 426
 - %y, 409
 - %z, 409
- conversion state, 373, 412–416, 447, 448–451, 655, 680–683
- conversion state function, 448
- copying function
 - string, 378, 658
 - wide string, 440
- copying functions
 - wide string, 675
- copysign** function, 203, 268, 485, 525, 541, 546, 547, 555, 556
- copysign** type-generic macro, 388
- copysignd128** function, 268, 490
- copysignd32** function, 268, 490
- copysignd64** function, 268, 490
- copysigndN** function, 585
- copysigndNx** function, 585
- copysignf** function, 268, 485
- copysignfN** function, 585
- copysignfNx** function, 585
- copysignl** function, 268, 390, 485
- correctly rounded result, 5
- corresponding real type, 40
- corresponding unsigned integer type, 39
- cos** function, 136, 244, 388, 482, 529, 537, 563
- cos** type-generic macro, 388, 563
- cosd128** function, 244, 487
- cosd32** function, 244, 487
- cosd64** function, 244, 487
- cosdN** function, 579
- cosdNx** function, 579

- cosf** function, 244, 482
- cosfN** function, 579
- cosfNx** function, 579
- cosh** function, 248, 249, 388, 482, 529, 538, 563
- cosh** type-generic macro, 388, 563
- coshd128** function, 248, 488
- coshd32** function, 248, 488
- coshd64** function, 248, 488
- coshdN** function, 580
- coshdNx** function, 580
- coshf** function, 248, 482
- coshfN** function, 580
- coshfNx** function, 580
- coshl** function, 248, 482
- cosl** function, 244, 482
- cospi** function, 246, 482, 529, 538
- cospi** type-generic macro, 388
- cospid128** function, 246, 487
- cospid32** function, 246, 487
- cospid64** function, 246, 487
- cospidN** function, 579
- cospidNx** function, 579
- cospif** function, 246, 482
- cospifN** function, 579
- cospifNx** function, 579
- cospil** function, 246, 482
- cpow** function, 201, 202, 388, 477, 562
- cpow** function
 - type-generic macro for, 388
- cpowf** function, 201, 477
- cpowf128** function, 596
- cpowfN** function, 574
- cpowfNx** function, 574
- cpowl** function, 201, 390, 477
- cpown** function, 458
- cpowr** function, 458
- cproj** function, 203, 477, 557, 594
- cproj** type-generic macro, 388
- cprojf** function, 203, 390, 477
- cprojfN** function, 574
- cprojfNx** function, 574
- cprojl** function, 203, 390, 477
- cr_** identifier prefix, 459, 528, 536
- CR_DECIMAL_DIG** macro, 25, 480, 529, 530
- creal** function, 202, 203, 204, 390, 477, 555, 556, 557, 563, 594
- creal** type-generic macro, 388, 563
- crealf** function, 204, 477
- crealfN** function, 574
- crealfNx** function, 574
- creall** function, 204, 477
- creating, 325
- critical undefined behavior, 8
- crootn** function, 458
- crsqr** function, 458
- csin** function, 198, 199, 388, 476, 557
- csin** function
 - type-generic macro for, 388
- csinf** function, 198, 476
- csinfN** function, 574
- csinfNx** function, 574
- csinh** function, 200, 388, 476, 557, 560
- csinh** function
 - type-generic macro for, 388
- csinhf** function, 200, 476
- csinhfN** function, 574
- csinhfNx** function, 574
- csinhl** function, 200, 476
- csinl** function, 198, 476
- csinpi** function, 458
- csqrt** function, 202, 388, 390, 477, 557, 562
- csqrt** function
 - type-generic macro for, 388
- csqrtf** function, 202, 477
- csqrtfN** function, 574
- csqrtfNx** function, 574
- csqrtl** function, 202, 477
- ctan** function, 199, 388, 476, 557
- ctan** function
 - type-generic macro for, 388
- ctanf** function, 199, 476
- ctanfN** function, 574
- ctanfNx** function, 574
- ctanh** function, 200, 388, 476, 557, 561
- ctanh** function
 - type-generic macro for, 388
- ctanhf** function, 200, 476
- ctanhfN** function, 574
- ctanhfNx** function, 574
- ctanhl** function, 200, 476
- ctanl** function, 199, 476
- ctanpi** function, 458
- ctgamma** function, 458
- ctime** function, 406, 407, 511
- ctime_s** function, 512, 665, 666
- currency_symbol** structure member, 229, 231, 234
- current object, 138
- CX_LIMITED_RANGE** pragma, vi, 186, 187, 197, 476, 554, 605
- D format modifier, 333, 341, 420, 425
- d** identifier prefix, 389
- d-wchar sequence, 361, 438
- d128** identifier prefix, 389
- d32** identifier prefix, 389
- d32add** macro, 33, 216, 389
- d32add** type-generic macro, 389
- d32addd128** function, 275, 491
- d32addd64** function, 275, 491
- d32div** macro, 33, 216, 389
- d32div** type-generic macro, 389
- d32divd128** function, 276, 491

- d32divd64** function, 276, 390, 491
- d32fma** macro, 33, 216, 389
- d32fma** type-generic macro, 389
- d32fmad128** function, 277, 491
- d32fmad64** function, 277, 491
- d32mul** macro, 33, 216, 389
- d32mul** type-generic macro, 389
- d32muld128** function, 276, 491
- d32muld64** function, 276, 491
- d32sqrt** macro, 33, 216, 389
- d32sqrt** type-generic macro, 389
- d32sqrtd128** function, 277, 491
- d32sqrtd64** function, 277, 491
- d32sub** macro, 33, 216, 389
- d32sub** type-generic macro, 389
- d32subd128** function, 276, 390, 491
- d32subd64** function, 276, 491
- d64** identifier prefix, 387, 389
- d64add** macro, 33, 216, 389
- d64add** type-generic macro, 389
- d64addd128** function, 275, 391, 491
- d64div** macro, 33, 216, 389
- d64div** type-generic macro, 389
- d64divd128** function, 276, 491
- d64fma** macro, 33, 216, 389
- d64fma** type-generic macro, 389
- d64fmad128** function, 277, 390, 491
- d64mul** macro, 33, 216, 389
- d64mul** type-generic macro, 389
- d64muld128** function, 276, 491
- d64sqrt** macro, 33, 216, 389
- d64sqrt** type-generic macro, 389
- d64sqrtd128** function, 277, 491
- d64sub** macro, 33, 216, 389
- d64sub** type-generic macro, 389
- d64subd128** function, 276, 491
- dadd** macro, 389, 595
- dadd** type-generic macro, 389
- daddl** function, 275, 390, 486, 524
- data race, 19, 194, 364, 365, 370, 385, 386, 406, 412, 448, 450, 653
- date and time header, 392, 402, 460, 664
- Daylight Saving Time, 402
- DBL_** identifier prefix, 458
- DBL_DECIMAL_DIG** macro, 27, 30, 479, 520
- DBL_DIG** macro, 27, 28, 30, 479, 520
- DBL_EPSILON** macro, 28, 30, 479, 520
- DBL_HAS_SUBNORM** macro, 26, 30, 458, 479
- DBL_IS_IEC_60559** macro, 26, 30, 479
- DBL_MANT_DIG** macro, 27, 30, 102, 479, 520
- DBL_MAX** macro, 28, 30, 479, 520
- DBL_MAX_10_EXP** macro, 28, 30, 479, 520
- DBL_MAX_EXP** macro, 28, 30, 479, 520
- DBL_MIN** macro, 29, 30, 479, 520
- DBL_MIN_10_EXP** macro, 28, 30, 479, 520
- DBL_MIN_EXP** macro, 28, 30, 479, 520
- DBL_NORM_MAX** macro, 28, 479, 520
- DBL_SNAN** macro, 26, 101, 102, 479, 523
- DBL_TRUE_MIN** macro, 29, 30, 479
- DD format modifier, 333, 341, 420, 425
- ddiv** macro, 389, 595, 596
- ddiv** type-generic macro, 389
- ddivl** function, 276, 486, 524, 528, 596
- DEC** identifier prefix, 480
- DEC128_** identifier prefix, 30, 458
- DEC128_EPSILON** macro, 31, 521
- DEC128_MANT_DIG** macro, 31, 521
- DEC128_MAX** macro, 31, 521
- DEC128_MAX_EXP** macro, 31, 521
- DEC128_MIN** macro, 32, 521
- DEC128_MIN_EXP** macro, 31, 521
- DEC128_SNAN** macro, 30
- DEC128_TRUE_MIN** macro, 32, 521
- DEC32_** identifier prefix, 30, 458
- DEC32_EPSILON** macro, 31, 521
- DEC32_MANT_DIG** macro, 31, 521
- DEC32_MAX** macro, 31, 521
- DEC32_MAX_EXP** macro, 31, 521
- DEC32_MIN** macro, 32, 521
- DEC32_MIN_EXP** macro, 31, 521
- DEC32_SNAN** macro, 30, 102
- DEC32_TRUE_MIN** macro, 32, 521
- DEC64_** identifier prefix, 30, 458
- DEC64_EPSILON** macro, 31, 521
- DEC64_MANT_DIG** macro, 31, 521
- DEC64_MAX** macro, 31, 521
- DEC64_MAX_EXP** macro, 31, 521
- DEC64_MIN** macro, 32, 521
- DEC64_MIN_EXP** macro, 31, 521
- DEC64_SNAN** macro, 30, 102
- DEC64_TRUE_MIN** macro, 32, 100, 102, 521
- DEC_** identifier prefix, 458
- DEC_EVAL_METHOD** macro, 25, 30, 62, 95, 235, 480, 520, 552, 567, 576, 613, 615
- DEC_INFINITY** macro, 31, 102, 236, 278, 459, 480, 487
- DEC_NAN** macro, 31, 102, 236, 459, 480, 487
- decimal constant, 57
- decimal digit, 20
- decimal floating type, 39, 566
- decimal re-encoding function, 279
- decimal rounding control pragma, 215
- decimal-point character, 191, 231
- decimal128 suffix, **dl** or **DL**, 62
- decimal32 suffix, **df** or **DF**, 62
- decimal64 suffix, **dd** or **DD**, 62
- DECIMAL_DIG** macro, 25, 27, 458, 479, 520, 686
- decimal_point** structure member, 229, 231
- declaration, 97
 - _Static_assert**, 53
 - function, 130
 - pointer, 127

- static_assert**, 53, 142
- structure/union, 104
- typedef**, 133
- declaration specifier, 97
- declarator, 126
 - abstract, 132
- declarator type derivation, 41, 127
- DECN_EPSILON** macro, 570
- DECN_MANT_DIG** macro, 569
- DECN_MAX** macro, 570
- DECN_MAX_EXP** macro, 569
- DECN_MIN** macro, 570
- DECN_MIN_EXP** macro, 569
- DECN_SNAN** macro, 568
- DECN_TRUE_MIN** macro, 570
- DECNX_EPSILON** macro, 570
- DECNX_MANT_DIG** macro, 569
- DECNX_MAX** macro, 570
- DECNX_MAX_EXP** macro, 569
- DECNX_MIN** macro, 570
- DECNX_MIN_EXP** macro, 569
- DECNX_SNAN** macro, 568
- DECNX_TRUE_MIN** macro, 570
- decodebin** family, 34, 280
- decodebind** family, 280, 281
- decodebind128** function, 280, 492
- decodebind32** function, 280, 492
- decodebind64** function, 280, 387, 389, 492
- decodebindN** function, 587
- decodedec** family, 34, 280
- decodedecd** family, 279, 280
- decodedecd128** function, 280, 492
- decodedecd32** function, 279, 492
- decodedecd64** function, 280, 387, 389, 492
- decodedecdN** function, 587
- decodedefN** function, 589
- default argument promotion, 76
- default initialization, 137
- default** keyword, 53
- default** label, 153, 155
- define** preprocessing directive, 178
- defined**, 167, 168, 174, 187, 604, 605
- defined** operator, 187
- definition, 98
 - escaping, 149
 - function, 159
- dependency-ordered before, 17
- deprecated** attribute, 98, 107, 143, 145, 146, 406, 407, 511, 685, 686
- derived declarator type, 41
- derived type, 40
- designated initializer, 138
- destringizing, 189
- device input/output, 14
- dfma** macro, 389, 595
- dfma** type-generic macro, 389
- dfmal** function, 277, 390, 486, 524
- diagnostic, 12
- diagnostic message, 5, 12
- diagnostics header, 195
- difftime** function, 403, 404, 511
- digit, 20, 205
- digraph, 69
- direct input/output function, 352
- display device, 21
- div** function, 136, 357, 372, 373, 507
- div_t** type, 141, 357, 372, 373, 507
- divide and round to narrower type, 276
- division assignment operator (/=), 93
- division operator (/), 84, 554
- dMadddN** function, 587
- dMadddNx** function, 587
- dMdivdN** function, 587
- dMdivdNx** function, 587
- dMencbindN** function, 590
- dMencdecdN** function, 590
- dMfmadN** function, 587
- dMfmadNx** function, 587
- dMmuldN** function, 587
- dMmuldNx** function, 587
- dMsqrdN** function, 587
- dMsqrdNx** function, 587
- dMsubdN** function, 587
- dMsubdNx** function, 587
- dmul** macro, 389, 595
- dmul** type-generic macro, 389
- dnull** function, 276, 486, 524
- dMxadddN** function, 587
- dMxadddNx** function, 587
- dMxdivdN** function, 587
- dMxdivdNx** function, 587
- dMxfmadN** function, 587
- dMxfmadNx** function, 587
- dMxmuldN** function, 587
- dMxmuldNx** function, 587
- dMxsqrdN** function, 587
- dMxsqrdNx** function, 587
- dMxsubdN** function, 587
- dMxsubdNx** function, 587
- do** keyword, 53
- do** statement, 156
- documentation of implementation, 10
- domain error, 238, 243–248, 252–260, 262–264, 267
- dot operator (.), 76
- double** **_Complex** type, 40
- double** **_Complex** type conversion, 48
- double** **_Imaginary** type, 553
- double** arithmetic, 15
- double** keyword, 53
- double** type, 39, 103
- double** type conversion, 47, 48

- double-quote escape sequence (\"), 65, 67, 189
- double_t** type, 235, 481, 532, 575, 576, 577, 615, 619
- dsqrt** macro, 389, 595
- dsqrt** type-generic macro, 389
- dsqrtl** function, 277, 486, 524
- dsub** macro, 389, 595, 596
- dsub** type-generic macro, 389
- dsubl** function, 276, 486, 524, 596
- dynamic floating-point environment, 210
- E format modifier, 409
- E identifier prefix, 209, 458
- EDOM** macro, 209, 238, 478
- effective type, 72
- effectless function call, 149
- EILSEQ** macro, 209, 326, 413–416, 432, 449–451, 478
- element type, 40
- elif** preprocessing directive, 167
- elifdef** preprocessing directive, 168
- elifndef** preprocessing directive, 168
- ellipsis punctuator (. . .), 130, 179
- else** keyword, 53
- else** preprocessing directive, 168
- else** statement, 154
- embed element width, 171
- embed parameter, 171
 - __limit__**, 165
 - if_empty**, 172, 177, 178
 - limit**, 165, 169, 171, 172, 174, 175–178
 - prefix**, 172, 176, 177
 - suffix**, 172, 176, 177
- embed parameter sequence, 171
- empty initialization, 137
- empty initializer, 137
- empty resource, 171
- empty statement, 153
- encodebin** family, 34, 280
- encodebind** family, 280
- encodebind128** function, 280, 492
- encodebind32** function, 280, 492
- encodebind64** function, 280, 387, 389, 492
- encodebindN** function, 587
- encodedec** family, 34, 279
- encodedecd** family, 279
- encodedecd128** function, 279, 492
- encodedecd32** function, 279, 492
- encodedecd64** function, 279, 387, 389, 492
- encodedecdN** function, 587
- encodefN** function, 588
- encoding, 45
- encoding error, 326, 336, 339, 340, 345–349, 413–417, 423, 424, 428–433, 449–451, 656, 657, 681–683
- end-of-file, 417
- end-of-file indicator, 322, 329, 349–355, 432, 434
- end-of-line indicator, 20
- endif** preprocessing directive, 168
- enum** keyword, 53
- enum** type, 40, 103, 109
- enumerated type, 40
- enumeration, 40, 109
- enumeration constant, 35, 63
- enumeration content, 116
- enumeration member, 109
- enumeration member type, 110
- enumeration specifier, 109
- enumeration tag, 37, 116
- enumerator, 109
- environment, 11
- environment function, 368, 652
- environment list, 370, 653
- environmental consideration, 19
- environmental limit, 21, 284, 325, 326, 328, 337, 364, 368, 369, 423, 640
- EOF** macro, 205, 322, 328, 343, 344, 346–352, 417, 427, 429–431, 433, 447, 448, 505, 605, 643, 644, 646–648, 650, 668, 670, 671, 673–675
- epoch, 405
- equal-sign punctuator (=), 97, 109, 137
- equality expression, 87
- equality operator (==), 88
- ERANGE** macro, 209, 226, 238, 239, 361, 362, 364, 437–440, 478, 615–617
- erf** function, 260, 261, 484, 544
- erf** type-generic macro, 388
- erfc** function, 261, 484, 544
- erfc** type-generic macro, 388
- erfcd128** function, 261, 489
- erfcd32** function, 261, 489
- erfcd64** function, 261, 489
- erfcdN** function, 583
- erfcdNx** function, 583
- erfcf** function, 261, 484
- erfcfN** function, 583
- erfcfNx** function, 583
- erfcl** function, 261, 484
- erfd128** function, 260, 489
- erfd32** function, 260, 489
- erfd64** function, 260, 489
- erfdN** function, 583
- erfdNx** function, 583
- erff** function, 260, 484
- erffN** function, 583
- erffNx** function, 583
- erfl** function, 260, 484
- errno** identifier, 149, 193, 196, 209, 226, 238, 239, 287, 326, 353–357, 361, 362, 364, 386, 413–416, 432, 437–440, 449–451,

- 478, 606, 607, 615–617, 620, 637, 638, 663
- errno_t** type, 478, 506, 508–510, 512, 514, 515, 637, 638, 639–641, 651, 652, 654–657, 658, 659–661, 663, 664, 665, 666, 667, 675–678, 680–682
- error condition, 238
- error function, 260, 544
- error indicator, 322, 329, 349–351, 353, 355, 432
- error** preprocessing directive, 9, 186
- error-handling function, 355, 385, 663, 664
- errors header, 209, 458
- escape character (\), 64
- escape sequence, 19, 21, 64, 190
- escaping definition, 149
- evaluation format, 26, 63, 235
- evaluation method, 26, 73, 532
- evaluation of expression, 14
- exceptional condition, 72
- excess precision, 26, 49, 158
- excess range, 26, 49, 158
- exclusive OR operator
- bitwise (^), 89
 - bitwise assignment (^=), 93
- executable program, 11
- execution character set, 19
- execution environment, 11, 12
- execution sequence, 14, 152
- EXIT_FAILURE** macro, 357, 369, 507
- EXIT_SUCCESS** macro, 357, 369, 398, 507
- exp** function, 249, 250, 251, 302, 388, 390, 483, 528, 539
- exp** type-generic macro, 388
- exp10** function, 250, 483, 528, 539
- exp10** type-generic macro, 388
- exp10d128** function, 250, 488
- exp10d32** function, 250, 488
- exp10d64** function, 250, 488
- exp10dN** function, 582
- exp10dNx** function, 582
- exp10f** function, 250, 483
- exp10fN** function, 582
- exp10fNx** function, 582
- exp10l** function, 250, 483
- exp10m1** function, 250, 483, 528, 539
- exp10m1** type-generic macro, 388
- exp10m1d128** function, 250, 488
- exp10m1d32** function, 250, 488
- exp10m1d64** function, 250, 488
- exp10m1dN** function, 582
- exp10m1dNx** function, 582
- exp10m1f** function, 250, 483
- exp10m1fN** function, 582
- exp10m1fNx** function, 582
- exp10m1l** function, 250, 483
- exp2** function, 250, 251, 483, 528, 539
- exp2** type-generic macro, 388
- exp2d128** function, 251, 488
- exp2d32** function, 250, 488
- exp2d64** function, 250, 488
- exp2dN** function, 582
- exp2dNx** function, 582
- exp2f** function, 250, 483
- exp2fN** function, 582
- exp2fNx** function, 582
- exp2l** function, 250, 483
- exp2m1** function, 251, 483, 528, 539
- exp2m1** type-generic macro, 388
- exp2m1d128** function, 251, 488
- exp2m1d32** function, 251, 488
- exp2m1d64** function, 251, 488
- exp2m1dN** function, 582
- exp2m1dNx** function, 582
- exp2m1f** function, 251, 483
- exp2m1fN** function, 582
- exp2m1fNx** function, 582
- exp2m1l** function, 251, 483
- expd128** function, 249, 488
- expd32** function, 249, 488
- expd64** function, 249, 390, 488
- expdN** function, 582
- expdNx** function, 582
- expf** function, 249, 483
- expfN** function, 582
- expfNx** function, 582
- expl** function, 249, 483
- explicit conversion, 46
- expm1** function, 251, 483, 528, 539
- expm1** type-generic macro, 388
- expm1d128** function, 251, 488
- expm1d32** function, 251, 488
- expm1d64** function, 251, 488
- expm1dN** function, 582
- expm1dNx** function, 582
- expm1f** function, 251, 483
- expm1fN** function, 582
- expm1fNx** function, 582
- expm1l** function, 251, 483
- exponent part, 61
- exponential function
- complex, 200, 561
 - real, 249, 539
- expression, 72, 118
- assignment, 91
 - cast, 83
 - constant, 95
 - evaluation, 14
 - full, 152
 - parenthesized, 73
 - primary, 73
 - unary, 81
 - void, 50

- expression statement, 153
- extended alignment, 45
- extended character, 19
- extended character set, 5, 19, 20
- extended constant expressions, 96
- extended floating type, 566
- extended integer type, 39, 46, 59, 318
- extended multibyte and wide character utilities header, 417, 460
- extended multibyte/wide character conversion utility, 447, 680
- extended signed integer types, 39
- extended unsigned integer type, 39
- extensible wide character case mapping function, 456
- extensible wide character classification function, 455
- extern**, 36, 37, 53, 83, 98, 99, 100, 114, 120–122, 124, 125, 129, 133, 151, 160–162, 176, 177, 194, 603, 619
- extern** storage-class specifier, 36, 53
- external definition, 159
- external identifiers
 - underscore, 192
- external linkage, 36
- external name, 55
- external object definition, 161

- f** identifier suffix, 196, 235, 387, 389, 458
- f32add** macro, 596
- f32addf64** function, 596
- f32addf64x** function, 596
- f32fma** macro, 596
- f32fmaf32x** function, 596
- f32mul** macro, 596
- f32mulf128** function, 596
- f32mulf32x** function, 596
- f32xsqrt** macro, 596
- f32xsqrtf64x** function, 596
- f64div** macro, 596
- f64divf128** function, 596
- f64divf64x** function, 596
- fabs** function, 257, 258, 387, 388, 484, 525, 542, 546, 547, 556, 563
- fabs** type-generic macro, 388, 563
- fabsd128** function, 258, 489
- fabsd32** function, 257, 489
- fabsd64** function, 258, 489
- fabsdN** function, 583
- fabsdNx** function, 583
- fabsf** function, 257, 484
- fabsfN** function, 583
- fabsfNx** function, 583
- fabsl** function, 257, 484
- fadd** function, 275, 486, 524, 595
- fadd** type-generic macro, 389
- faddl** function, 275, 486, 524

- fallthrough** attribute, 143, 146, 147, 168, 685, 686
- fallthrough declaration, 146
- false** keyword, 53
- false** predefined constant, 66
- family
 - canonicalize**, 34, 270
 - decodebin**, 34, 280
 - decodedec**, 34, 280
 - encodebin**, 34, 280
 - encodedec**, 34, 279
 - modf**, 34, 256, 387
 - strto**, 34, 361
 - wcsto**, 34, 437
- fclose** function, 174, 328, 505
- fdim** function, 270, 271, 485, 548
- fdim** type-generic macro, 388
- fdimd128** function, 271, 491
- fdimd32** function, 271, 491
- fdimd64** function, 271, 491
- fdimdN** function, 586
- fdimdNx** function, 586
- fdimf** function, 271, 485
- fdimfN** function, 586
- fdimfNx** function, 586
- fdiml** function, 271, 486
- fdiv** function, 276, 390, 486, 524, 595
- fdiv** type-generic macro, 389
- fdivl** function, 276, 486, 524
- FE_** identifier prefix, 211, 212, 458
- FE_ALL_EXCEPT** macro, 94, 211, 478
- FE_DEC_DOWNWARD** macro, 211, 215, 479, 528
- FE_DEC_DOWNWARD** pragma, 187
- FE_DEC_DYNAMIC** pragma, 187, 215
- fe_dec_getround** function, 212, 219, 479, 528, 575
- fe_dec_setround** function, 212, 215, 220, 479, 528, 575
- FE_DEC_TONEAREST** macro, 211, 212, 215, 479, 528
- FE_DEC_TONEAREST** pragma, 187
- FE_DEC_TONEARESTFROMZERO** macro, 211, 215, 479, 528
- FE_DEC_TONEARESTFROMZERO** pragma, 187
- FE_DEC_TOWARDZERO** macro, 211, 215, 479, 528
- FE_DEC_TOWARDZERO** pragma, 187
- FE_DEC_UPWARD** macro, 211, 215, 479, 528
- FE_DEC_UPWARD** pragma, 187
- FE_DFL_ENV** macro, 212, 478
- FE_DFL_MODE** macro, 211, 219, 478, 527
- FE_DIVBYZERO** macro, 211, 238, 478
- FE_DOWNWARD** macro, 211, 478, 527
- FE_DOWNWARD** pragma, 187
- FE_DYNAMIC** pragma, 187, 213, 479
- FE_INEXACT** macro, 211, 216, 478, 546
- FE_INVALID** macro, 211, 218, 238, 478

- FE_OVERFLOW** macro, 211, 216, 218, 238, 478
FE_SNANS_ALWAYS_SIGNAL macro, 479, 523, 527, 545, 548
FE_TONEAREST macro, 151, 211, 478, 527
FE_TONEAREST pragma, 187
FE_TONEARESTFROMZERO macro, 211, 478, 527
FE_TONEARESTFROMZERO pragma, 187
FE_TOWARDZERO macro, 211, 478, 527, 541, 546
FE_TOWARDZERO pragma, 187
FE_UNDERFLOW macro, 211, 222, 478
FE_UPWARD macro, 9, 211, 478, 527, 545
FE_UPWARD pragma, 187
feature test macro, 192, 210, 235, 318, 357, 387, 402
feclearexcept function, 94, 216, 218, 222, 479, 526, 546
fegetenv function, 221, 222, 479, 527, 606
fegetexceptflag function, 216, 217, 218, 479, 526, 606, 615
fegetmode function, 218, 219, 479, 526
fegetround function, 211, 215, 218, 219, 220, 479, 526, 527, 541, 545, 575
feholdexcept function, 93, 221, 222, 479, 527, 546, 606
femode_t type, 210, 211, 219, 478, 479
fence, 17, 298
 acquire, 298
 release, 298
FENV_ACCESS pragma, vi, 9, 93, 186, 187, 212, 213, 218, 220, 222, 478, 530–535, 541, 545, 547, 599, 605, 606, 613
FENV_DEC_ROUND pragma, vi, 62, 186, 187, 215, 479, 528, 575
FENV_ROUND pragma, vi, 29, 62, 151, 186, 187, 212, 213, 214, 215, 478, 479, 527, 575
fenv_t type, 93, 210, 212, 221, 222, 478, 479, 545
feof function, 344, 349, 355, 432, 506
feraiseexcept function, 216, 217, 479, 532, 599, 615
ferror function, 344, 349, 355, 432, 506
fesetenv function, 213, 221, 479, 527, 606
fesetexcept function, 217, 479, 526
fesetexceptflag function, 216, 217, 479, 526, 606
fesetmode function, 213, 218, 219, 479, 527
fesetround function, 9, 25, 100, 211, 213, 215, 218, 220, 479, 526, 527, 541, 545, 546, 575
fetestexcept function, 216, 218, 479, 526, 546
fetestexceptflag function, 218, 479, 526
feupdateenv function, 94, 213, 221, 222, 479, 527, 546, 606
fexcept_t type, 210, 216–218, 478, 479, 606
fflush function, 328, 329, 505, 608
ffma function, 277, 486, 524, 595
ffma type-generic macro, 389
ffmal function, 277, 486, 524
fgetc function, 323, 326, 349, 350, 352, 505
fgetpos function, 324, 325, 353, 354, 506, 599, 609, 616
fgets function, 323, 349, 350, 506, 609, 650
fgetwc function, 323, 326, 431, 432, 433, 513
fgetws function, 323, 432, 513, 609
field width, 331, 418
file, 325
 access function, 328, 640
 name, 325
 operation, 326, 639
 position indicator, 322, 324, 325, 329, 349, 350, 352–354, 432, 434
 positioning function, 353
file name, 325
file position indicator, 325
file scope, 35, 159
FILE type, 174, 322, 323, 325, 327, 328, 330, 331, 339, 347, 349–355, 418, 424, 429, 432–434, 505, 506, 512–514, 607, 639–642, 646, 647, 667, 668, 670, 671
FILENAME_MAX macro, 322, 505
finite number, 553
fixed underlying type, 109
flag, 331, 418
flexible array member, 107
float _Complex type, 40
float _Complex type conversion, 48
float _Imaginary type, 553
float arithmetic, 15
float keyword, 53
float type, 39, 103
float type conversion, 47, 48
float_t type, 235, 481, 532, 575, 576, 577, 615, 619
floating constant, 59
floating suffix, **f** or **F**, 62
floating type, 40, 190
floating type conversion, 47, 48, 529
floating-point accuracy, 25, 63, 73, 360, 529
floating-point arithmetic function, 235, 535
floating-point classification function, 239
floating-point control mode, 210, 532
floating-point environment, 210, 530, 532
 dynamic, 210
floating-point environment header, 210, 458
floating-point exception, 210, 216, 535
floating-point number, 24, 39
floating-point rounding mode, 25
floating-point status flag, 210, 532
floor, 7
floor function, 236, 262, 484, 523, 545
floor type-generic macro, 388
floorl28 function, 262, 489

- floord32** function, 262, 489
floord64 function, 262, 489
floordN function, 584
floordNx function, 584
floorf function, 262, 484
floorfN function, 584
floorfNx function, 584
floorl function, 262, 484
FLT identifier prefix, 480
FLT_ identifier prefix, 458
FLT_DECIMAL_DIG macro, 27, 29, 479, 520
FLT_DIG macro, 27, 28, 29, 479, 520
FLT_EPSILON macro, 28, 29, 479, 520
FLT_EVAL_METHOD macro, 25, 26, 29, 30, 93–95, 102, 235, 479, 520, 552, 567, 576, 613, 615
FLT_HAS_SUBNORM macro, 26, 30, 458, 479
FLT_IS_IEC_60559 macro, 26, 29, 479
FLT_MANT_DIG macro, 27, 29, 102, 479, 520
FLT_MAX macro, 28, 29, 30, 479, 520
FLT_MAX_10_EXP macro, 28, 29, 30, 479, 520
FLT_MAX_EXP macro, 28, 29, 30, 479, 520
FLT_MIN macro, 29, 30, 479, 520
FLT_MIN_10_EXP macro, 28, 29, 30, 479, 520
FLT_MIN_EXP macro, 28, 29, 479, 520
FLT_NORM_MAX macro, 28, 479, 520
FLT_RADIX macro, 25, 27, 28, 29, 31, 61, 215, 220, 256, 257, 335, 336, 360, 421, 423, 436, 479, 520, 526, 567, 568, 576
FLT_ROUNDS macro, 25, 211, 479, 519, 520, 523, 613
FLT_SNAN macro, 26, 101, 479, 523
FLT_TRUE_MIN macro, 29, 30, 479
FLT_N_DECIMAL_DIG macro, 569
FLT_N_DIG macro, 569
FLT_N_EPSILON macro, 570
FLT_N_MANT_DIG macro, 569
FLT_N_MAX macro, 570
FLT_N_MAX_10_EXP macro, 569
FLT_N_MAX_EXP macro, 569
FLT_N_MIN macro, 570
FLT_N_MIN_10_EXP macro, 569
FLT_N_MIN_EXP macro, 569
FLT_N_SNAN macro, 568
FLT_N_TRUE_MIN macro, 570
FLT_NX_DECIMAL_DIG macro, 569
FLT_NX_DIG macro, 569
FLT_NX_EPSILON macro, 570
FLT_NX_MANT_DIG macro, 569
FLT_NX_MAX macro, 570
FLT_NX_MAX_10_EXP macro, 569
FLT_NX_MAX_EXP macro, 569
FLT_NX_MIN macro, 570
FLT_NX_MIN_10_EXP macro, 569
FLT_NX_MIN_EXP macro, 569
FLT_NX_SNAN macro, 568
FLT_NX_TRUE_MIN macro, 570
fma function, 237, 275, 486, 524, 549
fma type-generic macro, 388
fmad128 function, 275, 491
fmad32 function, 275, 491
fmad64 function, 275, 491
fMaddfN function, 587
fMaddfNx function, 587
fmadN function, 586
fmadNx function, 586
fmaf function, 275, 486
fmafN function, 586
fmafNx function, 586
fmaI function, 237, 275, 486
fmax function, 271, 272, 486, 527, 548
fmax type-generic macro, 388
fmaxd128 function, 271, 491
fmaxd32 function, 271, 491
fmaxd64 function, 271, 491
fmaxf function, 271, 486
fmaximum function, 272, 273, 486, 524, 548
fmaximum type-generic macro, 388
fmaximum_mag type-generic macro, 388
fmaximum_mag_num type-generic macro, 388
fmaximum_num type-generic macro, 388
fmaximum_mag function, 272, 273, 274, 486, 524, 548
fmaximum_mag_num function, 273, 274, 486, 524, 548, 549
fmaximum_mag_numd128 function, 274, 491
fmaximum_mag_numd32 function, 274, 491
fmaximum_mag_numd64 function, 274, 491
fmaximum_mag_numdN function, 586
fmaximum_mag_numdNx function, 586
fmaximum_mag_numf function, 274, 486
fmaximum_mag_numfN function, 586
fmaximum_mag_numfNx function, 586
fmaximum_mag_numI function, 274, 486
fmaximum_magd128 function, 273, 491
fmaximum_magd32 function, 273, 491
fmaximum_magd64 function, 273, 491
fmaximum_magdN function, 586
fmaximum_magdNx function, 586
fmaximum_magf function, 272, 486
fmaximum_magfN function, 586
fmaximum_magfNx function, 586
fmaximum_magI function, 273, 486
fmaximum_num function, 272, 273, 486, 524, 527, 548, 549, 556
fmaximum_numd128 function, 273, 491
fmaximum_numd32 function, 273, 491
fmaximum_numd64 function, 273, 491
fmaximum_numdN function, 586
fmaximum_numdNx function, 586
fmaximum_numf function, 273, 486
fmaximum_numfN function, 586

- fmaximum_numfNx** function, 586
fmaximum_numl function, 273, 486
fmaximumd128 function, 272, 390, 491
fmaximumd32 function, 272, 491
fmaximumd64 function, 272, 491
fmaximumdN function, 586
fmaximumdNx function, 586
fmaximumf function, 272, 486
fmaximumfN function, 586
fmaximumfNx function, 586
fmaximuml function, 272, 486
fmaxl function, 271, 486
fMdivfN function, 587
fMdivfNx function, 587
fMencfN function, 589
fMfafN function, 587
fMfafNx function, 587
fmin function, 271, 272, 486, 527, 548
fmin type-generic macro, 388
fmind128 function, 271, 491
fmind32 function, 271, 491
fmind64 function, 271, 491
fminf function, 271, 486
fminimum function, 272, 273, 274, 486, 524, 548
fminimum type-generic macro, 388
fminimum_mag type-generic macro, 388
fminimum_mag_num type-generic macro, 388
fminimum_mag function, 273, 275, 486, 524, 548
fminimum_mag_num function, 273, 274, 275, 486, 524, 548, 549
fminimum_mag_numd128 function, 274, 491
fminimum_mag_numd32 function, 274, 491
fminimum_mag_numd64 function, 274, 491
fminimum_mag_numdN function, 586
fminimum_mag_numdNx function, 586
fminimum_mag_numf function, 274, 486
fminimum_mag_numfN function, 586
fminimum_mag_numfNx function, 586
fminimum_mag_numl function, 274, 486
fminimum_magd128 function, 273, 491
fminimum_magd32 function, 273, 491
fminimum_magd64 function, 273, 491
fminimum_magdN function, 586
fminimum_magdNx function, 586
fminimum_magf function, 273, 486
fminimum_magfN function, 586
fminimum_magfNx function, 586
fminimum_magl function, 273, 486
fminimum_num function, 272, 273, 274, 486, 524, 527, 548, 549
fminimum_numd128 function, 274, 491
fminimum_numd32 function, 274, 491
fminimum_numd64 function, 274, 491
fminimum_numdN function, 586
fminimum_numdNx function, 586
fminimum_numf function, 274, 486
fminimum_numfN function, 586
fminimum_numfNx function, 586
fminimum_numl function, 274, 486
fminl function, 271, 486
fMmulfN function, 587
fMmulfNx function, 587
fmod function, 266, 267, 485, 546, 547, 615
fmod type-generic macro, 388
fmodd128 function, 267, 490
fmodd32 function, 267, 490
fmodd64 function, 267, 490
fmoddN function, 584
fmoddNx function, 584
fmodf function, 266, 485
fmodfN function, 584
fmodfNx function, 584
fmodl function, 267, 485
fMsqrtfN function, 587
fMsqrtfNx function, 587
fMsubfN function, 587
fMsubfNx function, 587
fmul function, 276, 486, 524, 595
fmul type-generic macro, 389
fmull function, 276, 486, 524
fMxaddfN function, 587
fMxaddfNx function, 587
fMxdivfN function, 587
fMxdivfNx function, 587
fMxfmafN function, 587
fMxfmafNx function, 587
fMxmulfN function, 587
fMxmulfNx function, 587
fMxsqrtfN function, 587
fMxsqrtfNx function, 587
fMxsubfN function, 587
fMxsubfNx function, 587
fopen function, 174, 326, 327, 328, 329, 330, 505, 608, 620, 640
FOPEN_MAX macro, 322, 326, 327, 505, 639
fopen_s function, 506, 639, 640, 641
for keyword, 53
for, 156
form feed, 21
form-feed character, 20, 52
form-feed escape sequence (\f), 21, 65, 207
format conversion of integer types header, 458

FP_FAST_FMXFMAFN macro, 578

- FP_FAST_FMXFMAFNX** macro, 578
FP_FAST_FMXMULFN macro, 578
FP_FAST_FMXMULFNX macro, 578
FP_FAST_FMXSQRTFN macro, 578
FP_FAST_FMXSQRTFNX macro, 578
FP_FAST_FMXSUBFN macro, 578
FP_FAST_FMXSUBFNX macro, 578
FP_FAST_FSQRT macro, 237, 481
FP_FAST_FSQRTL macro, 237, 481
FP_FAST_FSUB macro, 237, 481
FP_FAST_FSUBL macro, 237, 481
FP_ILOGB0 macro, 237, 238, 252, 481
FP_ILOGBNAN macro, 237, 238, 252, 481
FP_INFINITE macro, 236, 481
FP_INT_DOWNWARD macro, 236, 481
FP_INT_TONEAREST macro, 236, 481
FP_INT_TONEARESTFROMZERO macro, 236, 481
FP_INT_TOWARDZERO macro, 236, 481
FP_LLOGB0 macro, 238, 253, 481
FP_LLOGBNAN macro, 238, 253, 481
FP_NAN macro, 236, 481
FP_NORMAL macro, 236, 481
FP_SUBNORMAL macro, 236, 481
FP_ZERO macro, 236, 481
fpclassify macro, 240, 481, 526, 527
fpos_t type, 322, 324, 353, 354, 505, 506
fprintf_s function, 506, 642
fputc function, 21, 323, 326, 350, 351, 353, 506
fputs function, 184, 323, 350, 506
fputwc function, 323, 326, 432, 434, 513
fputws function, 323, 432, 433, 513
frac_digits structure member, 229, 231, 234
fread function, 172, 174, 323, 352, 506, 609
free function, 366, 367, 368, 380, 507, 609, 684
free_aligned_sized function, 367, 507, 688
free_sized function, 366, 367, 507, 688
 freestanding execution environment, 9, 12
freopen function, 324, 325, 330, 505
freopen_s function, 506, 641, 642
frexp function, 251, 252, 483, 539, 588, 599, 600
frexp type-generic macro, 388
frexpd128 function, 252, 488
frexpd32 function, 252, 488
frexpd64 function, 252, 488
frexpdN function, 582
frexpdNx function, 582
frexpf function, 251, 483
frexpfN function, 582
frexpfNx function, 582
frexpl function, 251, 483
fromfp function, 237, 265, 266, 485, 525, 529, 546
fromfp function, 265
fromfp type-generic macro, 388
fromfpd128 function, 265, 490
fromfpd32 function, 265, 490
fromfpd64 function, 265, 490
fromfpdN function, 584
fromfpdNx function, 584
fromfpf function, 265, 485
fromfpfN function, 584
fromfpfNx function, 584
fromfpl function, 265, 485
fromfpx function, 237, 266, 485, 525, 529, 546
fromfpx function, 266
fromfpx type-generic macro, 388
fromfpxd128 function, 266, 490
fromfpxd32 function, 266, 490
fromfpxd64 function, 266, 490
fromfpxdN function, 584
fromfpxdNx function, 584
fromfpxf function, 266, 485
fromfpxfN function, 584
fromfpxfNx function, 584
fromfpxl function, 266, 485
fscanf function, 224, 225, 323, 339, 343–347, 459, 506, 616, 642
fscanf_s function, 506, 642, 643–647
fseek function, 323, 326, 329, 352, 353, 354, 355, 434, 506, 609
fsetpos function, 324, 325, 329, 352, 353, 354, 434, 506, 609, 616
fsqrt function, 277, 486, 524, 595
fsqrt type-generic macro, 389
fsqrtl function, 277, 486, 524
fsub function, 276, 486, 524, 595
fsub type-generic macro, 389
fsubl function, 276, 390, 486, 524, 596
ftell function, 353, 354, 506, 599, 609, 616
 full declarator, 127
 full expression, 152
 fully buffered, 325
 fully buffered stream, 325
 function
 argument, 75, 160
 body, 159
 byte input/output, 323
 call, 75
 library, 193
 declarator, 130
 definition, 130, 159
 designator, 50
 image, 21
 inline, 124
 library, 11, 193
 name length, 22, 55, 190
 no-return, 125
 parameter, 13, 75, 98, 160
 prototype, 13, 35, 130, 160, 235
 prototype scope, 36, 128, 129

- recursive call, 76
- return, 158, 530
- scope, 35
- stateless, 149
- type, 40
- type conversion, 50
- function call
 - effectless, 149
- function prototype, 35
- function prototype scope, 36
- function scope, 35
- function specifier, 124
 - _Noreturn**, 53
 - inline**, 53
- function type, 38
- function type attribute, 148
- function-call operator (**()**), 75
- function-like macro, 179
- fundamental alignment, 45
- Fused multiply-add and round to narrower type, 277
- future direction
 - language, 190
 - library, 458
- fwide** function, 324, 325, 433, 513
- fwprintf** function, 224, 323, 418, 423, 427–429, 431, 460, 512, 611, 616, 668
- fwprintf_s** function, 514, 667, 668
- fwrite** function, 323, 352, 353, 506, 609
- fscanf** function, 224, 323, 424, 427–429, 431, 434, 460, 512, 616, 668
- fscanf_s** function, 514, 668, 670, 671, 675
- gamma function, 260, 544
- general utilities header, 357, 459
- general utility, 651
 - wide string, 435, 675
- general wide string utility, 435, 675
- generic association, 74
- generic parameter, 387
- generic selection, 73, 74
- getc** function, 323, 350, 351, 506
- getchar** function, 16, 323, 351, 506
- getenv** function, 369, 370, 507, 609, 612
- getenv_s** function, 508, 652, 653
- getpayload** function, 492, 524, 550, 551
- getpayloadd128** function, 493, 550
- getpayloadd32** function, 493, 550
- getpayloadd64** function, 493, 550
- getpayloaddN** function, 588
- getpayloaddNx** function, 588
- getpayloadf** function, 492, 550
- getpayloadfN** function, 588
- getpayloadfNx** function, 588
- getpayloadl** function, 492, 550
- gets** (obsolete), 650, 688
- gets_s** function, 506, 650
- getwc** function, 323, 433, 513
- getwchar** function, 323, 433, 434, 513
- gmtime** function, 406, 407, 408, 511
- gmtime_r** function, 408, 512, 686
- gmtime_s** function, 512, 666, 667
- goto** keyword, 53
- goto** statement, 35, 153, 156
- graphic character, 20
- greater-than operator (**>**), 87
- greater-than-or-equal-to operator (**>=**), 87
- grouping** structure member, 229, 231, 232
- H format modifier, 333, 341, 420, 425
- h format modifier, 332, 340, 419, 425
- happens before, 18
- header, 11, 191
- header name, 52, 69, 170
- hexadecimal constant, 57
- hexadecimal digit, 58, 61, 65
- \xhexadecimal digit** (hexadecimal-character escape sequence), 65
- hexadecimal digit sequence, 58
- hexadecimal prefix, 57
- hexadecimal-character escape sequence (**\xhexadecimal digit**), 65
- hh format modifier, 332, 340, 419, 425
- hidden, 36
- high-order bit, 4, 5
- horizontal tab, 21
- horizontal-tab character, 20, 52
- horizontal-tab escape sequence (**\t**), 21, 65, 206, 207, 453
- hosted execution environment, 9, 12, 13
- HUGE_VAL** macro, 235, 239, 361, 437, 481, 535
- HUGE_VAL_D128** macro, 236, 492
- HUGE_VAL_D32** macro, 236, 492
- HUGE_VAL_D64** macro, 236, 492
- HUGE_VAL_DN** macro, 577
- HUGE_VAL_DNX** macro, 577
- HUGE_VAL_FN** macro, 577
- HUGE_VAL_FNX** macro, 577
- HUGE_VALF** macro, 235, 239, 361, 437, 481, 535
- HUGE_VALL** macro, 235, 239, 361, 437, 481, 535
- hyperbolic function
 - complex, 199, 558
 - real, 247, 538
- hypot** function, 258, 484, 523, 528, 542, 557
- hypot** type-generic macro, 388
- hypotd128** function, 258, 489
- hypotd32** function, 258, 489
- hypotd64** function, 258, 489
- hypotdN** function, 583
- hypotdNx** function, 583
- hypotf** function, 258, 484
- hypotfN** function, 583
- hypotfNx** function, 583
- hypotl** function, 258, 484

- I** macro, 203, 557
- idempotent, 149
- identifier, 54, 73
 - continue, 54, 517
 - maximum length, 54
 - name space, 37
 - reserved, 53, 192, 620, 628, 637
 - rule, 620
 - scope, 35
 - start, 54, 517
 - type, 38
- identifier continue, 54, 517
- identifier list, 164
- identifier start, 54, 517
- if** keyword, 53
- if** preprocessing directive, 167
- if** statement, 154
- if_empty** embed parameter, 172, 177, 178
- ifdef**, 9, 102, 163, 166, 167, 168, 215, 219, 220, 242–265, 266, 267–269, 270, 271–275, 276, 277–280, 359, 361, 390, 437, 520, 521, 549–551
- ifdef** preprocessing directive, 168
- ifndef** preprocessing directive, 168
- ignore_handler_s** function, 508, 652
- ilogb** function, 237, 252, 483, 524, 540, 599
- ilogb** type-generic macro, 388
- ilogbd128** function, 252, 488
- ilogbd32** function, 252, 488
- ilogbd64** function, 252, 488
- ilogbdN** function, 582
- ilogbdNx** function, 582
- ilogbf** function, 252, 483
- ilogbfN** function, 582
- ilogbfNx** function, 582
- ilogbl** function, 252, 483
- imaginary** macro, 196, 476, 557
- imaginary number, 553
- imaginary type, 553
- imaginary type domain, 553
- imaxabs** function, 225, 480
- imaxdiv** function, 224, 225, 480
- imaxdiv_t** type, 224, 225, 480
- implementation, 5
- implementation limit, 6, 9, 22, 55, 127, 155, 519
- implementation resource width, 171
- implementation-defined behavior, 3, 9, 611
- implementation-defined value, 7
- implicit conversion, 46
- implicit initialization, 137
- include** preprocessing directive, 11, 170
- inclusive OR operator
 - bitwise (**|**), 89
 - bitwise assignment (**|=**), 93
- incomplete, 38
- incomplete type, 38
- independent, 149
- indeterminate representation, 7
- indeterminately sequenced, 14, 76, 78, 93
- indirection operator (*****), 75, 82
- inequality operator (**!=**), 88
- infinitary, 238
- infinity, 553
- INFINITY** macro, 26, 101, 102, 203, 236, 334, 359–362, 421, 436–438, 459, 479, 481, 523, 555, 556
- initial position, 21
- initial shift state, 20
- initialization, 12, 37, 50, 79, 136, 532
 - in block, 152
- initialized, 12
- initializer, 136
 - permitted form, 95
 - string literal, 50
- inline**, 124
- inline definition, 124
- inline function, 124
- inline** keyword, 53
- inner scope, 36
- input failure, 430, 431, 643, 644, 646–648, 650, 668, 670, 671, 673–675
- input/output
 - device, 14
- input/output function
 - character, 349, 650
 - direct, 352
 - formatted, 331, 642
 - wide character, 418, 667
 - wide character, 431
 - formatted, 418, 667
- input/output header, 322, 459, 638
- INT** identifier prefix, 320, 321, 459, 505
- int** identifier prefix, 318, 319, 459, 505, 688
- int** keyword, 53
- int** type, 39, 47, 59, 103
- int** type conversion, 46–48
- intN_t** type, 318
- INTN_C** macro, 321
- INTN_MAX** macro, 320
- INTN_MIN** macro, 320, 321
- INTN_WIDTH** macro, 320
- int8_t** type, 318
- INT_FAST** identifier prefix, 320, 505
- int_fast** identifier prefix, 319, 505, 688
- INT_LEAST** identifier prefix, 320, 505
- int_least** identifier prefix, 318, 319, 321, 505
- int_curr_symbol** structure member, 229, 232–234
- INT_FASTN_MAX** macro, 320
- INT_FASTN_MIN** macro, 320, 321
- INT_FASTN_WIDTH** macro, 320
- int_fast16_t** type, 300, 319

- int_fast32_t** type, 224, 300, 319
- int_fast64_t** type, 300, 319
- int_fast8_t** type, 300, 319
- int_fastN_t** type, 319
- int_frac_digits** structure member, 229, 232, 234
- INT_LEASTN_MAX** macro, 320
- INT_LEASTN_MIN** macro, 320, 321
- int_leastN_t** type, 318
- INT_LEASTN_WIDTH** macro, 320
- int_least16_t** type, 300, 319
- int_least32_t** type, 300, 318, 319
- int_least64_t** type, 300, 319
- int_least8_t** type, 300, 319
- INT_MAX** macro, 23, 39, 103, 167, 237, 238, 252, 481, 519, 608
- INT_MIN** macro, 23, 39, 237, 238, 481, 519
- int_n_cs_precedes** structure member, 229, 232, 234
- int_n_sep_by_space** structure member, 229, 232–234
- int_n_sign_posn** structure member, 229, 232–234
- int_p_cs_precedes** structure member, 229, 232, 234
- int_p_sep_by_space** structure member, 229, 232–234
- int_p_sign_posn** structure member, 229, 232–234
- INT_WIDTH** macro, 23, 266, 481, 519
- integer arithmetic function, 225, 372
- integer character constant, 64
- integer constant, 57
- integer constant expression, 50, 95, 105, 109, 128, 137, 142, 154, 166, 193
- integer conversion rank, 46
- integer promotion, 15, 47, 82, 86, 155, 332, 419
- integer suffix, 59
- integer type, 40, 318
 - extended, 39, 46, 59, 318
- integer type conversion, 46, 47, 529
- integer types header, 318, 459
- inter-thread happens before, 18
- interactive device, 14, 325, 329
- interchange floating type, 565
- internal linkage, 36
- internal name, 55
- interrupt, 21
- INTMAX_C** macro, 321, 505
- INTMAX_MAX** macro, 226, 320, 505
- INTMAX_MIN** macro, 226, 320, 321, 505
- intmax_t** type, 23, 167, 225, 226, 300, 319, 321, 333, 340, 419, 425, 480, 505, 687, 689
- INTMAX_WIDTH** macro, 320, 505
- INTPTR_MAX** macro, 320, 505
- INTPTR_MIN** macro, 320, 321, 505
- intptr_t** type, 300, 319, 505
- INTPTR_WIDTH** macro, 320, 505
- is** identifier prefix, 458–460
- isalnum** function, 205, 207, 478
- isalpha** function, 205, 452, 478, 618
- isblank** function, 205, 206, 478, 618
- iscanonical** macro, 25, 240, 481, 526, 527
- iscntrl** function, 205, 206, 207, 478
- isdigit** function, 205, 206, 207, 230, 478
- iseqsig** macro, 283, 486, 525, 552
- isfinite** macro, 240, 481, 526, 527, 539, 556
- isgraph** function, 206, 452, 478
- isgreater** macro, 281, 486, 525, 526
- isgreaterequal** macro, 281, 282, 486, 526, 534, 548
- isinf** macro, 240, 241, 481, 526, 527, 541, 555, 556
- isless** macro, 282, 486, 526, 534, 535
- islessequal** macro, 282, 486, 526
- islessgreater** macro, 282, 486
- islower** function, 4, 205, 206, 207, 208, 478, 618
- isnan** macro, 241, 481, 526, 527, 548, 555, 556
- isnormal** macro, 241, 481, 526, 527
- ISO/IEC 10646, 2, 188
- ISO/IEC 2382, 2, 3
- ISO/IEC 60559, 2, 188, 197, 210, 221, 239, 267, 281, 522, 553, 564
- ISO/IEC 9945–2, 229
- ISO 4217, 2, 232
- ISO 80000–2, 2, 3
- ISO 80000–3, 691
- ISO 8601 series, 2, 409
- isprint** function, 21, 206, 478
- ispunct** function, 205, 206, 207, 478, 618
- issignaling** macro, 242, 481, 526, 527
- isspace** function, 191, 205, 206, 207, 478, 618
- issubnormal** macro, 242, 482, 526, 527
- isunordered** macro, 282, 283, 486, 526
- isupper** function, 205, 207, 208, 478, 618
- iswalnum** function, 453, 454, 455, 515
- iswalpha** function, 452, 453, 455, 515, 618
- iswblank** function, 453, 455, 515, 618
- iswcntrl** function, 453, 454, 455, 515
- iswctype** function, 455, 456, 515, 611, 618
- iswdigit** function, 453, 454, 455, 515
- iswgraph** function, 452, 454, 455, 515
- iswlower** function, 453, 454, 455, 456, 515, 618
- iswprint** function, 452, 454, 455, 515
- iswpunct** function, 452, 453, 454, 455, 515, 618
- iswspace** function, 191, 452, 453, 454, 455, 515, 618
- iswupper** function, 453, 454, 455, 456, 515, 618
- iswxdigit** function, 455, 515
- isxdigit** function, 207, 230, 478
- iszero** macro, 242, 482, 526, 527

italic type convention, 3, 35

iteration statement, 155

j format modifier, 333, 340, 419, 425

jmp_buf type, 284, 285, 501, 684

jump statement, 156

keyword, 53, 553, 619

kill_dependency macro, 17, 298, 502

known constant size, 41

L encoding prefix, 64, 65, 67, 138

L format modifier, 333, 341, 420, 425

l format modifier, 332, 340, 419, 425

l identifier suffix, 196, 235, 387, 389, 458

L_tmpnam macro, 323, 328, 505

L_tmpnam_s macro, 506, 638, 639

label

case, 53

default, 53

label name, 35, 37

labeled statement, 153

labs function, 372, 507

language, 35

encoding prefix

L, 64, 65, 67, 138

U, 64, 65, 67, 68, 138

u, 64, 65, 67, 68, 138

u8, 64, 65, 67, 68

future direction, 190

syntax summary, 461

Latin alphabet, 19

LC_ identifier prefix, 229, 458

LC_ALL macro, 229, 230, 233, 481

LC_COLLATE macro, 229, 230, 381, 442, 481

LC_CTYPE macro, 229, 230, 357, 373, 374, 447, 452, 455–457, 481, 610, 611, 655, 656

LC_MONETARY macro, 229, 230, 233, 481

LC_NUMERIC macro, 229, 230, 233, 481

LC_TIME macro, 229, 230, 406, 408, 481

lconv structure type, 229, 230, 481

LDBL_ identifier prefix, 458

LDBL_DECIMAL_DIG macro, 27, 458, 479, 520

LDBL_DIG macro, 27, 28, 479, 520

LDBL_EPSILON macro, 28, 479, 520

LDBL_HAS_SUBNORM macro, 26, 458, 479

LDBL_IS_IEC_60559 macro, 26, 479

LDBL_MANT_DIG macro, 27, 479, 520

LDBL_MAX macro, 28, 479, 520

LDBL_MAX_10_EXP macro, 28, 479, 520

LDBL_MAX_EXP macro, 28, 479, 520

LDBL_MIN macro, 29, 479, 520

LDBL_MIN_10_EXP macro, 28, 479, 520

LDBL_MIN_EXP macro, 28, 479, 520

LDBL_NORM_MAX macro, 28, 479, 520

LDBL_SNAN macro, 26, 479, 523

LDBL_TRUE_MIN macro, 29, 479

ldexp function, 252, 253, 483, 540, 588

ldexp type-generic macro, 388

ldexpd128 function, 252, 488

ldexpd32 function, 252, 488

ldexpd64 function, 252, 488

ldexpdN function, 582

ldexpdNx function, 582

ldexpf function, 252, 483

ldexpfN function, 582

ldexpfNx function, 582

ldexpl function, 252, 483

ldiv function, 136, 357, 372, 373, 507

ldiv_t type, 357, 372, 373, 507

leading underscore in identifier, 192

least significant index, 305

left-shift assignment operator (**<=<**), 93

left-shift operator (**<<**), 86

length

external name, 22, 55, 190

function name, 22, 55, 190

identifier, 54

internal name, 22, 55

length function, 373, 386, 446, 448, 664, 680

length modifier, 332, 339, 418, 424

length of a string, 191

length of a wide string, 191

less-than operator (**<**), 87

less-than-or-equal-to operator (**<=**), 87

letter, 20, 205

lowercase, 20

lexical element, 11, 52

lgamma function, 261, 484, 544

lgamma type-generic macro, 388

lgammad128 function, 261, 489

lgammad32 function, 261, 489

lgammad64 function, 261, 489

lgammadN function, 583

lgammadNx function, 583

lgammaf function, 261, 484

lgammafN function, 583

lgammafNx function, 583

lgammal function, 261, 484

library, 11, 191, 636

constant

memory_order_acq_rel, 296, 297, 298, 301–303, 501

memory_order_acquire, 296, 298, 301, 303, 501

memory_order_consume, 296, 298, 301, 501

memory_order_relaxed, 149, 296, 297, 298, 501

memory_order_release, 296, 298, 301, 302, 501

memory_order_seq_cst, 19, 42, 78, 92, 93, 294, 296, 297, 299, 501

mtx_plain , 392, 396, 511	asind32 , 243, 487
mtx_recursive , 392, 396, 511	asind64 , 243, 487
mtx_timed , 393, 396, 511	asinf , 243, 482
thrd_busy , 393, 397, 511	asinh , 248, 388, 482, 529, 538, 563
thrd_error , 393, 394–401, 511	asinhhd128 , 248, 487
thrd_nomem , 393, 394, 397, 511	asinhhd32 , 248, 487
thrd_success , 393, 394–401, 511	asinhhd64 , 248, 487
thrd_timedout , 393, 395, 396, 511	asinhf , 248, 482
family	asinhhl , 248, 482
canonicalize , 33	asinl , 243, 482
decodebin , 34	asinpi , 245, 482, 529, 537
decodebind , 280, 281	asinpdl128 , 245, 487
decodedec , 34	asinpdl32 , 245, 487
decodedecd , 279, 280	asinpdl64 , 245, 487
encodebin , 34	asinpif , 245, 482
encodebind , 280	asinpil , 245, 482
encodedec , 34	at_quick_exit , 368, 369, 370, 507, 600, 609, 688
encodedecd , 279	atan , 243, 337, 388, 423, 482, 529, 536, 563
modf , 34	atan2 , 243, 244, 482, 529, 535, 536, 537, 557
strfrom , 214, 216	atan2d128 , 244, 487
strfromd , 359	atan2d32 , 244, 487
strto , 33, 214, 216	atan2d64 , 244, 487
strtod , 361, 362	atan2f , 243, 482
wcsto , 33, 214, 216	atan2l , 243, 482
wcstod , 437, 438	atan2pi , 246, 482, 529, 535, 537
function	atan2pid128 , 246, 487
_Exit , 287, 369, 370, 507, 606, 617	atan2pid32 , 246, 487
abort , 148, 195, 286, 287, 295, 325, 368, 507, 606, 607, 617, 652	atan2pid64 , 246, 487
abort_handler_s , 508, 652	atan2pif , 246, 482
abs , 193, 372, 507	atan2pil , 246, 482
acos , 214, 242, 243, 388, 482, 529, 536, 575	atand128 , 243, 487
acosd128 , 216, 243, 487	atand32 , 243, 487
acosd32 , 216, 242, 487	atand64 , 243, 487
acosd64 , 216, 243, 487	atanf , 243, 482
acosf , 214, 242, 482	atanh , 248, 388, 482, 529, 538, 563
acosh , 247, 248, 388, 482, 529, 538	atanhd128 , 248, 487
acoshd128 , 247, 487	atanhd32 , 248, 487
acoshd32 , 247, 487	atanhd64 , 248, 487
acoshd64 , 247, 487	atanhf , 248, 482
acoshf , 247, 390, 482	atanhl , 248, 482
acoshl , 247, 482	atanl , 243, 390, 482
acosl , 214, 242, 482, 575	atanpi , 245, 246, 482, 529, 537
acospi , 245, 482, 529, 537	atanpid128 , 246, 487
acospid128 , 245, 487	atanpid32 , 246, 487
acospid32 , 245, 487	atanpid64 , 246, 487
acospid64 , 245, 487	atanpif , 246, 482
acospif , 245, 482	atanpil , 246, 482
acospil , 245, 482	atexit , 368, 369, 370, 507, 600, 609, 619
aligned_alloc , 365, 366, 367, 507, 600, 609, 617, 688	atof , 214, 357, 358, 507
asctime , 187, 188, 406, 407, 511, 611	atoi , 194, 357, 358, 507
asctime_s , 512, 665, 666	atol , 357, 358, 507
asin , 243, 388, 482, 529, 536, 563	atoll , 357, 358, 507
asind128 , 243, 487	

atomic_compare_exchange_strong , 78, 93, 94, 301, 302, 502	casin , 198, 388, 476, 557
atomic_compare_exchange_strong_explicit , 301, 502	casinf , 198, 476
atomic_compare_exchange_weak , 301, 302, 502	casinh , 199, 388, 476, 557, 559
atomic_compare_exchange_weak_explicit , 301, 502	casinhf , 199, 476
atomic_exchange , 301, 502	casinhl , 199, 476
atomic_exchange_explicit , 301, 502	casinl , 198, 476
atomic_fetch_ , 302, 502	casinpi , 458
atomic_flag_clear , 303, 304, 502	catan , 198, 388, 476, 557
atomic_flag_clear_explicit , 303, 502	catanf , 198, 476
atomic_flag_test_and_set , 303, 502	catanh , 199, 200, 388, 476, 557, 559
atomic_flag_test_and_set_explicit , 303, 502	catanhf , 200, 476
atomic_init , 295, 502	catanhhl , 200, 476
atomic_is_lock_free , 287, 299, 502, 606	catanl , 198, 476
atomic_load , 301, 302, 502	catanpi , 458
atomic_load_explicit , 297, 298, 301, 502	cbirt , 74, 257, 389, 484, 541
atomic_signal_fence , 299, 502	cbirt128 , 257, 489
atomic_store , 301, 502	cbirt32 , 257, 489
atomic_store_explicit , 297, 298, 301, 502	cbirt64 , 257, 489
atomic_thread_fence , 149, 298, 299, 502	cbirtf , 74, 257, 389, 484
btowc , 422, 423, 447, 513	cbirtl , 74, 257, 389, 484
cl6rtomb , 414, 415, 512	ccompoundn , 458
c32rtomb , 416, 512	ccos , 198, 388, 476, 557
c8rtomb , 413, 512, 686	ccosf , 198, 476
cabs , 201, 387, 388, 477, 557	ccosf64x , 596
cabsf , 201, 390, 477	ccosh , 200, 388, 476, 557, 560
cabsl , 201, 477	ccoshf , 200, 476
cacos , 197, 198, 388, 476, 558	ccoshl , 200, 476
cacosf , 197, 476	ccosl , 198, 476
cacosh , 199, 388, 476, 558, 559	ccospi , 458
cacoshf , 199, 476	ceil , 236, 262, 484, 523, 544, 545, 546
cacoshl , 199, 476	ceild128 , 34, 262, 489
cacosl , 197, 476	ceild32 , 34, 262, 489
cacospi , 458	ceild64 , 34, 262, 489
call_once , 357, 392, 393, 507, 511, 687	ceilf , 262, 484
calloc , 365, 366, 367, 507, 600, 609, 617	ceill , 262, 266, 484
canonicalize , 25, 270, 388, 485, 525, 548	cerf , 458
canonicalized128 , 270, 491	cerfc , 458
canonicalized32 , 270, 491	cexp , 200, 201, 388, 476, 561, 562
canonicalized64 , 270, 491	cexp10 , 458
canonicalizef , 270, 485	cexp10m1 , 458
canonicalizel , 270, 485	cexp2 , 458
carg , 202, 390, 477, 557, 563, 594	cexp2m1 , 458
cargf , 202, 477	cexpf , 201, 476
cargl , 202, 477	cexpl , 201, 476
	cexplm1 , 458
	cimag , 202, 203, 204, 477, 555–557, 563, 594
	cimagf , 202, 477
	cimagl , 202, 390, 477
	clearerr , 355, 506
	clgamma , 458
	clock , 402, 403, 406, 511, 617
	clog , 201, 388, 477, 562
	clog10 , 458
	clog10p1 , 458
	cloglp , 458

clog2 , 458	crealf , 204, 477
clog2p1 , 458	creall , 204, 477
clogf , 201, 390, 477	crootn , 458
clogl , 201, 477	crsqr , 458
clogp1 , 458	csin , 198, 199, 388, 476, 557
cnd_broadcast , 393, 394, 395, 511	csinf , 198, 476
cnd_destroy , 394, 511	csinh , 200, 388, 476, 557, 560
cnd_init , 394, 511	csinhf , 200, 476
cnd_signal , 394, 395, 511	csinhl , 200, 476
cnd_timedwait , 394, 395, 511	csinl , 198, 476
cnd_wait , 394, 395, 511	csinpi , 458
compoundn , 257, 484, 528, 542	csqrt , 202, 388, 390, 477, 557, 562
compoundnd128 , 257, 489	csqrtf , 202, 477
compoundnd32 , 257, 489	csqrtl , 202, 477
compoundnd64 , 257, 489	ctan , 199, 388, 476, 557
compoundnf , 257, 484	ctanf , 199, 476
compoundnl , 257, 484	ctanh , 200, 388, 476, 557, 561
conj , 203, 477, 557–562, 594	ctanhf , 200, 476
conjf , 203, 477	ctanhl , 200, 476
conjl , 203, 477	ctanl , 199, 476
copysign , 203, 268, 485, 525, 541, 546, 547, 555, 556	ctanpi , 458
copysignd128 , 268, 490	ctgamma , 458
copysignd32 , 268, 490	ctime , 406, 407, 511
copysignd64 , 268, 490	ctime_s , 512, 665, 666
copysignf , 268, 485	d32addd128 , 275, 491
copysignl , 268, 390, 485	d32addd64 , 275, 491
cos , 136, 244, 388, 482, 529, 537, 563	d32divd128 , 276, 491
cosd128 , 244, 487	d32divd64 , 276, 390, 491
cosd32 , 244, 487	d32fmad128 , 277, 491
cosd64 , 244, 487	d32fmad64 , 277, 491
cosf , 244, 482	d32muld128 , 276, 491
cosh , 248, 249, 388, 482, 529, 538, 563	d32muld64 , 276, 491
coshd128 , 248, 488	d32sqtrtd128 , 277, 491
coshd32 , 248, 488	d32sqtrtd64 , 277, 491
coshd64 , 248, 488	d32subd128 , 276, 390, 491
coshf , 248, 482	d32subd64 , 276, 491
coshl , 248, 482	d64addd128 , 275, 391, 491
cosl , 244, 482	d64divd128 , 276, 491
cospi , 246, 482, 529, 538	d64fmad128 , 277, 390, 491
cospid128 , 246, 487	d64muld128 , 276, 491
cospid32 , 246, 487	d64sqtrtd128 , 277, 491
cospid64 , 246, 487	d64subd128 , 276, 491
cospif , 246, 482	daddl , 275, 390, 486, 524
cospil , 246, 482	ddivl , 276, 486, 524, 528, 596
cpow , 201, 202, 388, 477, 562	decodebind128 , 280, 492
cpowf , 201, 477	decodebind32 , 280, 492
cpowf128 , 596	decodebind64 , 280, 387, 389, 492
cpowl , 201, 390, 477	decodedecd128 , 280, 492
cpown , 458	decodedecd32 , 279, 492
cpowr , 458	decodedecd64 , 280, 387, 389, 492
cproj , 203, 477, 557, 594	dfmal , 277, 390, 486, 524
cprojf , 203, 390, 477	difftime , 403, 404, 511
cprojl , 203, 390, 477	div , 136, 357, 372, 373, 507
creal , 202, 203, 204, 390, 477, 555–557, 563, 594	dmull , 276, 486, 524
	dsqrtl , 277, 486, 524
	dsubl , 276, 486, 524, 596

encodebind128 , 280, 492	f32addf64x , 596
encodebind32 , 280, 492	f32fmaf32x , 596
encodebind64 , 280, 387, 389, 492	f32mulf128 , 596
encodedecd128 , 279, 492	f32mulf32x , 596
encodedecd32 , 279, 492	f32xsqrtf64x , 596
encodedecd64 , 279, 387, 389, 492	f64divf128 , 596
erf , 260, 261, 484, 544	f64divf64x , 596
erfc , 261, 484, 544	fabs , 257, 258, 387, 388, 484, 525, 542, 546, 547, 556, 563
erfcd128 , 261, 489	fabsd128 , 258, 489
erfcd32 , 261, 489	fabsd32 , 257, 489
erfcd64 , 261, 489	fabsd64 , 258, 489
erfcf , 261, 484	fabsf , 257, 484
erfcl , 261, 484	fabsl , 257, 484
erfd128 , 260, 489	fadd , 275, 486, 524, 595
erfd32 , 260, 489	faddl , 275, 486, 524
erfd64 , 260, 489	fclose , 174, 328, 505
erff , 260, 484	fdim , 270, 271, 485, 548
erfl , 260, 484	fdimd128 , 271, 491
exp , 249, 250, 251, 302, 388, 390, 483, 528, 539	fdimd32 , 271, 491
exp10 , 250, 483, 528, 539	fdimd64 , 271, 491
exp10d128 , 250, 488	fdimf , 271, 485
exp10d32 , 250, 488	fdiml , 271, 486
exp10d64 , 250, 488	fdiv , 276, 390, 486, 524, 595
exp10f , 250, 483	fdivl , 276, 486, 524
exp10l , 250, 483	fe_dec_getround , 212, 219, 479, 528, 575
exp10m1 , 250, 483, 528, 539	fe_dec_setround , 212, 215, 220, 479, 528, 575
exp10m1d128 , 250, 488	feclearexcept , 94, 216, 218, 222, 479, 526, 546
exp10m1d32 , 250, 488	fegetenv , 221, 222, 479, 527, 606
exp10m1d64 , 250, 488	fegetexceptflag , 216, 217, 218, 479, 526, 606, 615
exp10m1f , 250, 483	fegetmode , 218, 219, 479, 526
exp10m1l , 250, 483	fegetround , 211, 215, 218, 219, 220, 479, 526, 527, 541, 545, 575
exp2 , 250, 251, 483, 528, 539	feholdexcept , 93, 221, 222, 479, 527, 546, 606
exp2d128 , 251, 488	feof , 344, 349, 355, 432, 506
exp2d32 , 250, 488	feraiseexcept , 216, 217, 479, 532, 599, 615
exp2d64 , 250, 488	ferror , 344, 349, 355, 432, 506
exp2f , 250, 483	fesetenv , 213, 221, 479, 527, 606
exp2l , 250, 483	fesetexcept , 217, 479, 526
exp2m1 , 251, 483, 528, 539	fesetexceptflag , 216, 217, 479, 526, 606
exp2m1d128 , 251, 488	fesetmode , 213, 218, 219, 479, 527
exp2m1d32 , 251, 488	fesetround , 9, 25, 100, 211, 213, 215, 218, 220, 479, 526, 527, 541, 545, 546, 575
exp2m1d64 , 251, 488	fetestexcept , 216, 218, 479, 526, 546
exp2m1f , 251, 483	fetestexceptflag , 218, 479, 526
exp2m1l , 251, 483	feupdateenv , 94, 213, 221, 222, 479, 527, 546, 606
expd128 , 249, 488	fflush , 328, 329, 505, 608
expd32 , 249, 488	
expd64 , 249, 390, 488	
expf , 249, 483	
expl , 249, 483	
expm1 , 251, 483, 528, 539	
expm1d128 , 251, 488	
expm1d32 , 251, 488	
expm1d64 , 251, 488	
expm1f , 251, 483	
expm1l , 251, 483	
f32addf64 , 596	

ffma, 277, 486, 524, 595
ffmal, 277, 486, 524
fgetc, 323, 326, 349, 350, 352, 505
fgetpos, 324, 325, 353, 354, 506, 599, 609, 616
fgets, 323, 349, 350, 506, 609, 650
fgetwc, 323, 326, 431, 432, 433, 513
fgetws, 323, 432, 513, 609
floor, 236, 262, 484, 523, 545
floord128, 262, 489
floord32, 262, 489
floord64, 262, 489
floorf, 262, 484
floorl, 262, 484
fma, 237, 275, 486, 524, 549
fmad128, 275, 491
fmad32, 275, 491
fmad64, 275, 491
fmaf, 275, 486
fmal, 237, 275, 486
fmax, 271, 272, 486, 527, 548
fmaxd128, 271, 491
fmaxd32, 271, 491
fmaxd64, 271, 491
fmaxf, 271, 486
fmaximum, 272, 273, 486, 524, 548
fmaximum_mag, 272, 273, 274, 486, 524, 548
fmaximum_mag_num, 273, 274, 486, 524, 548, 549
fmaximum_mag_numd128, 274, 491
fmaximum_mag_numd32, 274, 491
fmaximum_mag_numd64, 274, 491
fmaximum_mag_numf, 274, 486
fmaximum_mag_numl, 274, 486
fmaximum_magd128, 273, 491
fmaximum_magd32, 273, 491
fmaximum_magd64, 273, 491
fmaximum_magf, 272, 486
fmaximum_magl, 273, 486
fmaximum_num, 272, 273, 486, 524, 527, 548, 549, 556
fmaximum_numd128, 273, 491
fmaximum_numd32, 273, 491
fmaximum_numd64, 273, 491
fmaximum_numf, 273, 486
fmaximum_numl, 273, 486
fmaximumd128, 272, 390, 491
fmaximumd32, 272, 491
fmaximumd64, 272, 491
fmaximumf, 272, 486
fmaximuml, 272, 486
fmaxl, 271, 486
fmin, 271, 272, 486, 527, 548
fmind128, 271, 491
fmind32, 271, 491
fmind64, 271, 491
fminf, 271, 486
fminimum, 272, 273, 274, 486, 524, 548
fminimum_mag, 273, 275, 486, 524, 548
fminimum_mag_num, 273, 274, 275, 486, 524, 548, 549
fminimum_mag_numd128, 274, 491
fminimum_mag_numd32, 274, 491
fminimum_mag_numd64, 274, 491
fminimum_mag_numf, 274, 486
fminimum_mag_numl, 274, 486
fminimum_magd128, 273, 491
fminimum_magd32, 273, 491
fminimum_magd64, 273, 491
fminimum_magf, 273, 486
fminimum_magl, 273, 486
fminimum_num, 272, 273, 274, 486, 524, 527, 548, 549
fminimum_numd128, 274, 491
fminimum_numd32, 274, 491
fminimum_numd64, 274, 491
fminimum_numf, 274, 486
fminimum_numl, 274, 486
fminimumd128, 272, 491
fminimumd32, 272, 491
fminimumd64, 272, 491
fminimumf, 272, 486
fminimuml, 272, 486
fminl, 271, 486
fmod, 266, 267, 485, 546, 547, 615
fmodd128, 267, 490
fmodd32, 267, 490
fmodd64, 267, 490
fmodf, 266, 485
fmodl, 267, 485
fmul, 276, 486, 524, 595
fmull, 276, 486, 524
fopen, 174, 326, 327, 328, 329, 330, 505, 608, 620, 640
fopen_s, 506, 639, 640, 641
fprintf_s, 506, 642
fputc, 21, 323, 326, 350, 351, 353, 506
fputs, 184, 323, 350, 506
fputwc, 323, 326, 432, 434, 513
fputws, 323, 432, 433, 513
fread, 172, 174, 323, 352, 506, 609
free, 366, 367, 368, 380, 507, 609, 684
free_aligned_sized, 367, 507, 688
free_sized, 366, 367, 507, 688
freopen, 324, 325, 330, 505
freopen_s, 506, 641, 642
frexp, 251, 252, 483, 539, 588, 599, 600
frexpd128, 252, 488
frexpd32, 252, 488
frexpd64, 252, 488
frexpf, 251, 483

frexpl , 251, 483	ilogb , 237, 252, 483, 524, 540, 599
fromfp , 237, 265, 266, 485, 525, 529, 546	ilogbd128 , 252, 488
fromfpd128 , 265, 490	ilogbd32 , 252, 488
fromfpd32 , 265, 490	ilogbd64 , 252, 488
fromfpd64 , 265, 490	ilogbf , 252, 483
fromfpx , 237, 266, 485, 525, 529, 546	ilogbl , 252, 483
fromfpxd128 , 266, 490	imaxabs , 225, 480
fromfpxd32 , 266, 490	imaxdiv , 224, 225, 480
fromfpxd64 , 266, 490	isalnum , 205, 207, 478
fromfpxf , 266, 485	isalpha , 205, 452, 478, 618
fromfpxl , 266, 485	isblank , 205, 206, 478, 618
fscanf , 224, 225, 323, 339, 343–347, 459, 506, 616, 642	iscntrl , 205, 206, 207, 478
fscanf_s , 506, 642, 643–647	isdigit , 205, 206, 207, 230, 478
fseek , 323, 326, 329, 352, 353, 354, 355, 434, 506, 609	isgraph , 206, 452, 478
fsetpos , 324, 325, 329, 352, 353, 354, 434, 506, 609, 616	islower , 4, 205, 206, 207, 208, 478, 618
fsqrt , 277, 486, 524, 595	isprint , 21, 206, 478
fsqrtl , 277, 486, 524	ispunct , 205, 206, 207, 478, 618
fsub , 276, 486, 524, 595	isspace , 191, 205, 206, 207, 478, 618
fsubl , 276, 390, 486, 524, 596	isupper , 205, 207, 208, 478, 618
ftell , 353, 354, 506, 599, 609, 616	iswalnum , 453, 454, 455, 515
fwide , 324, 325, 433, 513	iswalpha , 452, 453, 455, 515, 618
fwprintf , 224, 323, 418, 423, 427–429, 431, 460, 512, 611, 616, 668	iswblank , 453, 455, 515, 618
fwprintf_s , 514, 667, 668	iswcntrl , 453, 454, 455, 515
fwrite , 323, 352, 353, 506, 609	iswctype , 455, 456, 515, 611, 618
fwscanf , 224, 323, 424, 427–429, 431, 434, 460, 512, 616, 668	iswdigit , 453, 454, 455, 515
fwscanf_s , 514, 668, 670, 671, 675	iswgraph , 452, 454, 455, 515
getc , 323, 350, 351, 506	iswlower , 453, 454, 455, 456, 515, 618
getchar , 16, 323, 351, 506	iswprint , 452, 454, 455, 515
getenv , 369, 370, 507, 609, 612	iswpunct , 452, 453, 454, 455, 515, 618
getenv_s , 508, 652, 653	iswspace , 191, 452, 453, 454, 455, 515, 618
getpayload , 492, 524, 550, 551	iswupper , 453, 454, 455, 456, 515, 618
getpayloadd128 , 493, 550	iswxdigit , 455, 515
getpayloadd32 , 493, 550	isxdigit , 207, 230, 478
getpayloadd64 , 493, 550	labs , 372, 507
getpayloadf , 492, 550	ldexp , 252, 253, 483, 540, 588
getpayloadl , 492, 550	ldexpd128 , 252, 488
gets_s , 506, 650	ldexpd32 , 252, 488
getwc , 323, 433, 513	ldexpd64 , 252, 488
getwchar , 323, 433, 434, 513	ldexpf , 252, 483
gmtime , 406, 407, 408, 511	ldexpl , 252, 483
gmtime_r , 408, 512, 686	ldiv , 136, 357, 372, 373, 507
gmtime_s , 512, 666, 667	lgamma , 261, 484, 544
hypot , 258, 484, 523, 528, 542, 557	lgammad128 , 261, 489
hypotd128 , 258, 489	lgammad32 , 261, 489
hypotd32 , 258, 489	lgammad64 , 261, 489
hypotd64 , 258, 489	lgammaf , 261, 484
hypotf , 258, 484	lgammal , 261, 484
hypotl , 258, 484	llabs , 372, 507
ignore_handler_s , 508, 652	lldiv , 136, 357, 372, 373, 507
	llogb , 238, 253, 483, 524, 540
	llogbd128 , 253, 488
	llogbd32 , 253, 488
	llogbd64 , 253, 488
	llogbf , 253, 483
	llogbl , 253, 483

llquantexpd128 , 279, 492	logbl , 256, 483
llquantexpd32 , 279, 492	logd128 , 253, 488
llquantexpd64 , 279, 492	logd32 , 253, 488
llrint , 263, 484, 529, 545, 546, 600	logd64 , 253, 488
llrintd128 , 263, 490	logf , 253, 483
llrintd32 , 263, 490	logl , 253, 483
llrintd64 , 263, 490	logp1 , 254, 255, 483, 528, 540
llrintf , 263, 485	logp1d128 , 254, 488
llrintl , 263, 485	logp1d32 , 254, 488
llround , 264, 485, 525, 546, 600	logp1d64 , 254, 488
llroundd128 , 264, 490	logp1f , 254, 483
llroundd32 , 264, 490	logp1l , 254, 483
llroundd64 , 264, 490	longjmp , 284, 285, 369, 370, 501, 606, 609, 684
llroundf , 264, 485	lrint , 263, 484, 529, 545, 546, 600
llroundl , 264, 485	lrintd128 , 263, 490
localeconv , 230, 233, 481, 606	lrintd32 , 263, 490
localtime , 404, 406, 407, 408, 512	lrintd64 , 263, 490
localtime_r , 408, 512, 686	lrintf , 263, 484
localtime_s , 512, 666, 667	lrintl , 263, 484
log , 238, 253, 255, 388, 483, 528, 540	lround , 264, 485, 525, 546, 600
log10 , 253, 254, 483, 528, 540	lroundd128 , 264, 490
log10d128 , 254, 488	lroundd32 , 264, 490
log10d32 , 254, 488	lroundd64 , 264, 490
log10d64 , 254, 488	lroundf , 264, 485
log10f , 254, 483	lroundl , 264, 485
log10l , 254, 483	mblen , 373, 448, 507
log10p1 , 254, 255, 483, 528, 540	mbrlen , 448, 513
log10p1d128 , 254, 488	mbrtoc16 , 413, 414, 512
log10p1d32 , 254, 488	mbrtoc32 , 415, 512
log10p1d64 , 254, 488	mbrtoc8 , 412, 413, 512, 686
log10p1f , 254, 483	mbrtowc , 326, 342, 422, 423, 447, 448, 449, 450, 513, 656, 682
log10p1l , 254, 483	mbsinit , 448, 513
log1p , 254, 255, 483, 528, 540	mbsrtowcs , 447, 450, 514, 681
log1pd128 , 254, 488	mbsrtowcs_s , 515, 681, 682
log1pd32 , 254, 488	mbstowcs , 68, 374, 375, 435, 450, 507
log1pd64 , 254, 488	mbstowcs_s , 509, 656, 657
log1pf , 254, 483	mbtowc , 66, 373, 374, 375, 448, 507
log1pl , 254, 483	memalignment , 9, 375, 507, 687
log2 , 255, 483, 528, 540	memccpy , 378, 509, 686
log2d128 , 255, 488	memcmp , 42, 174, 302, 381, 509
log2d32 , 255, 488	memcpy , 42, 72, 171, 194, 295, 302, 378, 509, 525
log2d64 , 255, 488	memcpy_s , 509, 658
log2f , 255, 483	memmove , 72, 378, 379, 509, 525, 605
log2l , 255, 483	memmove_s , 509, 658, 659
log2p1 , 255, 483, 528, 541	memset , 295, 385, 509, 663
log2p1d128 , 255, 488	memset_explicit , 385, 509, 687
log2p1d32 , 255, 488	memset_s , 510, 663
log2p1d64 , 255, 488	mktime , 404, 511
log2p1f , 255, 483	modf , 256, 387, 388, 483, 541
log2p1l , 255, 483	modfd128 , 256, 489
logb , 252, 253, 255, 256, 483, 524, 539, 541, 556, 588	modfd32 , 256, 489
logbd128 , 256, 489	modfd64 , 256, 489
logbd32 , 256, 488	modff , 256, 483
logbd64 , 256, 489	
logbf , 255, 483	

modfl , 256, 483	pownd64 , 259, 489
mtx_destroy , 395, 511	pownf , 258, 484
mtx_init , 392, 393, 395, 396, 511	pownl , 259, 484
mtx_lock , 396, 511, 610	powr , 259, 484, 529, 543
mtx_timedlock , 396, 511, 610	powrd128 , 259, 489
mtx_trylock , 396, 397, 511	powrd32 , 259, 489
mtx_unlock , 396, 397, 511, 610	powrd64 , 259, 489
nan , 268, 334, 421, 485, 523, 547	powrf , 259, 484
nand128 , 268, 490	powrl , 259, 484
nand32 , 268, 490	printf_s , 506, 643, 644
nand64 , 268, 490	puts , 185, 316, 323, 351, 506
nanf , 268, 485	putwc , 323, 434, 513
nanl , 268, 485	putwchar , 323, 434, 513
nearbyint , 262, 263, 484, 527, 529, 541, 545	qsort , 371, 372, 507, 600
nearbyintd128 , 263, 490	qsort_s , 508, 653, 654, 655
nearbyintd32 , 263, 489	quantized128 , 278, 491
nearbyintd64 , 263, 489	quantized32 , 278, 491
nearbyintf , 262, 484	quantized64 , 278, 491
nearbyintl , 262, 484	quantumd128 , 278, 492
nextafter , 268, 269, 390, 485, 527, 547	quantumd32 , 278, 492
nextafterd128 , 269, 490	quantumd64 , 278, 492
nextafterd32 , 268, 490	quick_exit , 287, 368, 369, 370, 507, 600, 606, 609, 617, 688
nextafterd64 , 268, 490	raise , 286, 287, 288, 295, 368, 501, 606, 607
nextafterf , 268, 485	rand , 357, 364, 365, 507
nextafterl , 268, 485	realloc , 365, 366, 367, 368, 507, 600, 609, 617, 684, 688
nextdown , 270, 485, 524, 548	remainder , 267, 390, 485, 524, 528, 547, 615
nextdownd128 , 270, 490	remainderd128 , 267, 490
nextdownd32 , 270, 490	remainderd32 , 267, 490
nextdownd64 , 270, 490	remainderd64 , 267, 490
nextdownf , 270, 485	remainderf , 267, 485
nextdownl , 270, 485	remainderl , 267, 485
nexttoward , 269, 485, 527, 547	remove , 326, 327, 505, 616, 639
nexttowardd128 , 269, 490	remquo , 267, 485, 524, 528, 547, 599, 615
nexttowardd32 , 269, 490	remquof , 267, 485
nexttowardd64 , 269, 490	remquol , 267, 485
nexttowardf , 269, 390, 485	rename , 327, 505, 616
nexttowardl , 269, 485	rewind , 329, 352, 354, 355, 434, 506
nextup , 269, 270, 485, 524, 548	rint , 263, 484, 524, 529, 545, 546
nextupd128 , 269, 490	rintd128 , 263, 490
nextupd32 , 269, 490	rintd32 , 263, 490
nextupd64 , 269, 490	rintd64 , 263, 490
nextupf , 269, 485	rintf , 263, 484
nextupl , 269, 485	rintl , 263, 484
perror , 355, 356, 506	rootn , 259, 260, 484, 528, 543
pow , 258, 388, 484, 529, 542, 596	rootnd128 , 259, 489
powd128 , 258, 489	rootnd32 , 259, 489
powd32 , 258, 489	rootnd64 , 259, 489
powd64 , 258, 390, 489	rootnf , 259, 484
powf , 258, 484	rootnl , 259, 484
powf32x , 596	round , 236, 264, 485, 523, 545
powf64 , 596	roundd128 , 264, 490
powl , 258, 484, 527	roundd32 , 264, 490
pown , 258, 259, 484, 529, 543	
pownd128 , 259, 489	
pownd32 , 259, 489	

roundd64 , 264, 490	sin , 77, 244, 388, 390, 482, 529, 537, 563
roundeven , 236, 264, 265, 485, 523, 546	sind128 , 244, 487
roundevend128 , 264, 490	sind32 , 244, 487
roundevend32 , 264, 490	sind64 , 244, 487
roundevend64 , 264, 490	sinf , 244, 482
roundevenf , 264, 485	sinh , 249, 388, 482, 529, 538, 563
roundevenl , 264, 485	sinhd128 , 249, 488
roundf , 264, 485	sinhd32 , 249, 488
roundl , 264, 485	sinhd64 , 249, 488
rsqrt , 260, 484, 528, 544	sinhf , 249, 482
rsqrtd128 , 260, 489	sinhl , 249, 482
rsqrtd32 , 260, 489	sinl , 244, 482
rsqrtd64 , 260, 489	sinpi , 247, 482, 529, 538
rsqrtf , 260, 484	sinpid128 , 247, 487
rsqrtl , 260, 484	sinpid32 , 247, 487
samequantumd128 , 278, 492	sinpid64 , 247, 487
samequantumd32 , 278, 491	sinpif , 247, 482
samequantumd64 , 278, 492	sinpil , 247, 482
scalbln , 256, 257, 484, 524, 541, 588	snprintf , 346, 348, 358, 359, 407, 505, 644, 690
scalblnd128 , 256, 489	snprintf_s , 506, 644, 645
scalblnd32 , 256, 489	snwprintf_s , 514, 669, 670
scalblnd64 , 256, 489	sprintf , 346, 349, 505, 645
scalblnf , 256, 484	sprintf_s , 506, 645
scalblnl , 256, 484	sqrt , 151, 260, 388, 484, 524, 544, 549
scalbn , 256, 257, 483, 524, 539, 540, 541, 556, 588	sqrtd128 , 260, 489
scalbnd128 , 256, 489	sqrtd32 , 260, 390, 489
scalbnd32 , 256, 489	sqrtd64 , 260, 489
scalbnd64 , 256, 489	sqrtf , 260, 484
scalbnf , 256, 483	sqrtl , 260, 484
scalbnl , 256, 483	rand , 364, 365, 507
scanf , 33, 214, 216, 323, 345, 346, 348, 505, 525, 688, 689	sscanf , 344, 346, 349, 505
scanf_s , 506, 644, 648	sscanf_s , 506, 645, 646, 649
set_constraint_handler_s , 508, 637, 651, 652	stdc_bit_ceil_uc , 312, 504
setbuf , 322, 325, 326, 328, 330, 505	stdc_bit_ceil_ui , 312, 504
setjmp , 193, 284, 285, 501, 599, 606, 684	stdc_bit_ceil_ul , 312, 504
setlocale , vii, 191, 229, 230, 233, 406, 481, 606, 615	stdc_bit_ceil_us , 312, 504
setpayload , 492, 524, 551	stdc_bit_floor_uc , 312, 504
setpayloadd128 , 493, 551	stdc_bit_floor_ui , 312, 504
setpayloadd32 , 493, 551	stdc_bit_floor_ul , 312, 504
setpayloadd64 , 493, 551	stdc_bit_floor_ull , 312, 504
setpayloadf , 492, 551	stdc_bit_floor_us , 312, 504
setpayloadl , 492, 551	stdc_bit_width_uc , 311, 504
setpayloadsig , 492, 524, 551	stdc_bit_width_ui , 311, 504
setpayloadsigd128 , 493, 551	stdc_bit_width_ul , 311, 504
setpayloadsigd32 , 493, 551	stdc_bit_width_ull , 311, 504
setpayloadsigd64 , 493, 551	stdc_bit_width_us , 311, 504
setpayloadsigf , 492, 551	stdc_count_ones_uc , 310, 503
setpayloadsigl , 492, 551	stdc_count_ones_ui , 310, 503
setvbuf , 322, 325, 326, 328, 330, 331, 505, 608	stdc_count_ones_ul , 310, 503
signal , 15, 16, 134, 286, 287, 369, 370, 501, 606, 607, 612, 616	stdc_count_ones_ull , 310, 503
	stdc_count_ones_us , 310, 503
	stdc_count_zeros_uc , 310, 503
	stdc_count_zeros_ui , 310, 503
	stdc_count_zeros_ul , 310, 503
	stdc_count_zeros_ull , 310, 503

stdc_count_zeros_uc , 310, 503	stdc_trailing_ones_uc , 307, 503
stdc_first_leading_one_uc , 308, 503	stdc_trailing_ones_ui , 307, 503
stdc_first_leading_one_ui , 308, 503	stdc_trailing_ones_ul , 307, 503
stdc_first_leading_one_ul , 308, 503	stdc_trailing_ones_ull , 307, 503
stdc_first_leading_one_ull , 308, 503	stdc_trailing_ones_us , 307, 503
stdc_first_leading_one_us , 308, 503	stdc_trailing_zeros_uc , 307, 502
stdc_first_leading_zero_uc , 308, 503	stdc_trailing_zeros_ui , 307, 502
stdc_first_leading_zero_ui , 308, 503	stdc_trailing_zeros_ul , 307, 502
stdc_first_leading_zero_ul , 308, 503	stdc_trailing_zeros_ull , 307, 502
stdc_first_leading_zero_ull , 308, 503	stdc_trailing_zeros_us , 307, 502
stdc_first_leading_zero_us , 308, 503	strcat , 380, 509
stdc_first_trailing_one_uc , 309, 503	strcat_s , 509, 660, 661
stdc_first_trailing_one_ui , 309, 503	strcmp , 381, 382, 509
stdc_first_trailing_one_ul , 309, 503	strcoll , 9, 230, 381, 382, 509
stdc_first_trailing_one_ull , 309, 503	strcpy , 176, 177, 344, 379, 509
stdc_first_trailing_one_us , 309, 503	strcpy_s , 509, 659
stdc_first_trailing_zero_uc , 309, 503	strcspn , 383, 509
stdc_first_trailing_zero_ui , 309, 503	strdup , 9, 379, 380, 509, 686
stdc_first_trailing_zero_ul , 309, 503	strerror , 9, 356, 385, 386, 509, 609, 610, 618
stdc_first_trailing_zero_ull , 309, 503	strerror_s , 386, 510, 663, 664
stdc_first_trailing_zero_us , 309, 503	strerrorlen_s , 510, 664
stdc_has_single_bit_uc , 311, 503	strfromd , 358, 359, 435, 507, 525
stdc_has_single_bit_ui , 311, 503	strfromd128 , 359, 508
stdc_has_single_bit_ul , 311, 503	strfromd32 , 359, 508
stdc_has_single_bit_ull , 311, 504	strfromd64 , 359, 508
stdc_has_single_bit_us , 311, 503	strfromencf128 , 591, 592
stdc_leading_ones_uc , 306, 502	strfromf , 358, 359, 435, 507
stdc_leading_ones_ui , 306, 502	strfroml , 358, 359, 507
stdc_leading_ones_ul , 306, 502	strftime , 230, 406, 408, 411, 446, 512, 600, 608, 610, 617, 665, 666, 686, 690
stdc_leading_ones_ull , 306, 502	strlen , 380, 386, 509
stdc_leading_ones_us , 306, 502	strncat , 380, 509
stdc_leading_zeros_uc , 306, 502	strncat_s , 509, 661, 662
stdc_leading_zeros_ui , 306, 502	strncmp , 184, 381, 382, 509
stdc_leading_zeros_ul , 306, 502	strncpy , 379, 509
stdc_leading_zeros_ull , 306, 502	strncpy_s , 509, 659, 660
stdc_leading_zeros_us , 306, 502	strndup , 9, 380, 509, 686
	strnlen_s , 510, 659–661, 664
	strspn , 384, 509
	strtod , 63, 268, 340, 341, 345, 358, 359, 435, 507, 525, 530, 531, 599, 616
	strtod128 , 361, 508, 592, 616, 617
	strtod32 , 361, 508, 616, 617
	strtod64 , 361, 362, 508, 616, 617
	strtodf , 268, 345, 358, 359, 507, 599, 616
	strtoimax , 225, 226, 480
	strtok , 9, 384, 385, 509, 610
	strtok_s , 385, 510, 662, 663
	strtoul , 226, 340, 341, 345, 358, 363, 364, 507
	strtold , 268, 345, 358, 359, 507, 599, 616
	strtoll , 226, 345, 358, 363, 364, 507
	strtoul , 226, 341, 345, 358, 363, 364, 507

strtoull, 226, 345, 358, **363**, 364, 507
strtoumax, 225, 226, 480
strxfrm, 9, 230, **382**, 509, 610
swprintf, 428, 430, 512, 669, 670
swprintf_s, 514, **669**, 670
swscanf, 428, 429, 430, 512
swscanf_s, 514, **670**, 673
system, 370, 371, 507, 610, 612, 617
tan, 244, 245, 388, 482, 529, **537**, 563
tand128, 245, 487
tand32, 245, 487
tand64, 245, 487
tanf, 244, 482
tanh, 249, 388, 482, 529, **539**, 563
tanh128, 249, 488
tanh32, 249, 488
tanh64, 249, 488
tanhf, 249, 482
tanh1, 249, 483
tan1, 244, 482
tanpi, 247, 482, 529, **538**
tanpid128, 247, 487
tanpid32, 247, 487
tanpid64, 247, 487
tanpif, 247, 482
tanpil, 247, 482
tgamma, 261, 262, 484, **544**
tgammad128, 262, 489
tgammad32, 261, 489
tgammad64, 261, 489
tgammaf, 261, 484
tgammal, 261, 484
thr_create, 392, **397**, 511
thr_current, 397, 511
thr_detach, 398, 511, 610
thr_equal, 398, 511
thr_exit, 397, **398**, 511, 600
thr_join, 398, 399, 511, 610
thr_sleep, 399, 511
thr_yield, 399, 511
time, 404, **405**, 511, 600
timegm, 404, 405, 511, 688
timespec_get, 402, **405**, 406, 511
timespec_getres, 402, **406**, 511, 688
tmpfile, 327, 369, 505
tmpfile_s, 506, **639**, 640
tmpnam, 323, **327**, 328, 505, 640
tmpnam_s, 506, 638, **639**, 640
tolower, 207, 478
totalorder, 492, 526, **549**, 550
totalorder128, 492, 549
totalorder32, 492, 549
totalorder64, 492, 549
totalorderf, 492, 549
totalorderl, 492, 549
totalordermag, 492, 526, **550**
totalordermag128, 493, 550
totalordermag32, 493, 550
totalordermag64, 493, 550
totalordermagf, 492, 550
totalordermagl, 492, 550
toupper, 207, 208, 478
towctrans, 456, 457, 515, 611, 618
tolower, 456, 457, 515
toupper, 456, 457, 515
trunc, 236, **265**, 485, 523, **546**
truncd128, 265, 490
truncd32, 265, 490
truncd64, 265, 490
truncf, 265, 485
truncl, 265, 485
tss_create, 399, 400, 511, 610, 611
tss_delete, 400, 511, 600, 611
tss_get, 400, 511, 611
tss_set, 400, 401, 511, 611
ufromfp, 237, **265**, 266, 485, 525, 529, **546**
ufromfpd128, 265, 490
ufromfpd32, 265, 490
ufromfpd64, 265, 490
ufromfpx, 265, 485
ufromfpxl, 265, 485
ufromfpxd128, 266, 490
ufromfpxd32, 266, 490
ufromfpxd64, 266, 490
ufromfpxf, 266, 485
ufromfpxl, 266, 485
ungetc, 323, **351**, 352–354, 459, 506, 599, 609, 620, 690
ungetwc, 323, **434**, 435, 513, 599, 620
va_arg, 224, **290**, 291–293, 336, 347–349, 423, 429–431, 501, 607, 647–649, 671, 673, 674
va_copy, 193, 290, **291**, 293, 501, 599, 607, 690
va_end, 193, 290, **291**, 292, 293, 347–349, 429–431, 501, 599, 607, 609, 647–649, 671, 673, 674
va_start, 290, **291**, 292, 293, 347–349, 429–431, 501, 607, 647–649, 671, 673, 674, 687
vfprintf, 323, **347**, 505, 609, 646
vfprintf_s, 506, **646**, 647–649
vfscanf, 323, **347**, 505, 609
vfscanf_s, 506, **647**, 648, 649
vfwprintf, 323, **429**, 512, 609, 671
vfwprintf_s, 514, **670**, 671
vfwscanf, 323, **429**, 430, 434, 512, 609
vfwscanf_s, 514, **671**, 673, 674
vprintf, 323, **347**, 348, 505, 609, 647
vprintf_s, 506, **647**, 648, 649

vscanf , 323, 347, 348 , 505, 609, 689	wcstoull , 226, 428, 439 , 440, 513
vscanf_s , 506, 647 , 648–650	wcstoumax , 226 , 480
vsprintf , 347, 348 , 505, 609, 648	wcsxfrm , 442 , 443, 513, 610
vsprintf_s , 506, 647, 648 , 649	wctob , 447 , 448, 452, 513
vsnwprintf_s , 514, 671 , 672	wctomb , 373, 374 , 375, 448, 507
vsprintf , 347, 348 , 349, 505, 609, 649	wctomb_s , 509, 655 , 656
vsprintf_s , 506, 647, 648, 649	wctrans , 456, 457 , 515, 611
vsscanf , 347, 349 , 505, 609	wctype , 455 , 456, 515, 611
vsscanf_s , 506, 647, 648, 649 , 650	wmemcmp , 443 , 513
vswprintf , 429, 430 , 513, 609, 672	wmemcpy , 440 , 513
vswprintf_s , 514, 672	wmemcpy_s , 515, 676 , 677
vswscanf , 429, 430 , 513, 609	wmemmove , 440 , 441, 513
vswscanf_s , 514, 671, 672 , 673, 674	wmemmove_s , 515, 677
vwprintf , 323, 429, 430 , 431, 513, 609, 673	wmemset , 446 , 513
vwprintf_s , 514, 673	wprintf , 214, 216, 225, 323, 430, 431 , 513, 525, 674
vwsprintf , 323, 429, 431 , 434, 513, 609	wprintf_s , 514, 674
vwsprintf_s , 514, 671, 673 , 674	wscanf , 214, 216, 323, 431 , 434, 513, 525
wcrtomb , 326, 335, 339, 345, 417, 426–428, 449 , 450, 451, 513, 600, 657, 680, 683	wscanf_s , 514, 674 , 675
wcrtomb_s , 515, 680 , 681	future direction, 458
wcscat , 441 , 513	identifier
wcscat_s , 515, 677 , 678	__VA_ARGS__ , 178, 179, 180 , 185, 689
wscmp , 441 , 442, 513	__VA_OPT__ , 178, 179–181, 687
wscoll , 442 , 513	__func__ , 55, 56, 195, 602, 690
wscpy , 440 , 513	errno , 149, 193, 196, 209 , 226, 238, 239, 287, 326, 353–357, 361, 362, 364, 386, 413–416, 432, 437–440, 449–451, 478, 606, 607, 615–617, 620, 637, 638, 663
wscpy_s , 514, 675	identifier prefix
wscspn , 443 , 444, 513	__STDC__ , 190
wcsftime , 230, 446 , 447, 513, 600, 608, 610, 617	__STDC_VERSION__ , 192
wcslen , 441, 446 , 513, 680	ATOMIC_ , 459
wcsncat , 441 , 513	atomic_ , 459
wcsncat_s , 515, 678 , 679	c , 387, 388
wcsncmp , 442 , 513	ckd_ , 459
wcsncpy , 440 , 513	cnd_ , 460
wcsncpy_s , 514, 675 , 676	cr_ , 459, 528, 536
wcsnlen_s , 515, 675–678, 680	d , 389
wcsrtombs , 450, 451, 514, 681	d128 , 389
wcsrtombs_s , 515, 681, 682 , 683	d32 , 389
wcsspn , 444 , 513	d64 , 387, 389
wcstod , 424, 426, 428, 435 , 513, 525, 599, 616	DBL_ , 458
wcstod128 , 435 , 437, 514, 616, 617	DEC , 480
wcstod32 , 435 , 437, 514, 616, 617	DEC128_ , 30, 458
wcstod64 , 435 , 437, 514, 616, 617	DEC32_ , 30, 458
wcstof , 428, 435 , 513, 599, 616	DEC64_ , 30, 458
wcstoimax , 226 , 480	DEC_ , 458
wcstok , 445 , 513, 610, 611	E , 209, 458
wcstok_s , 515, 679 , 680	FE_ , 211, 212 , 458
wcstol , 226, 424, 426, 428, 439 , 440, 513	FLT , 480
wcstold , 428, 435 , 513, 599, 616	FLT_ , 458
wcstoll , 226, 428, 439 , 440, 513	FP_ , 236 , 458
wcstombs , 375, 450, 507	INT , 320 , 321 , 459, 505
wcstombs_s , 509, 657 , 658	int , 318, 319, 459, 505, 688
wcstoul , 226, 426, 428, 439 , 440, 513	INT_FAST , 320 , 505
	int_fast , 319 , 505, 688

INT_LEAST , 320, 505	__STDC_EMBED_EMPTY__ , 167, 187
int_least , 318, 319, 321, 505	__STDC_EMBED_FOUND__ , 166, 187
is , 458–460	__STDC_EMBED_NOT_FOUND__ , 166, 187
LC_ , 229, 458	__STDC_ENDIAN_BIG__ , 305, 306, 502
LDBL_ , 458	__STDC_ENDIAN_LITTLE__ , 305, 306, 502
llquantexpd , 279, 389	__STDC_ENDIAN_NATIVE__ , 279–281, 305, 502, 588–590, 592–594, 616
MATH_ , 459	__STDC_HOSTED__ , 187
mem , 460	__STDC_IEC_60559_BFP__ , 9, 24, 188, 189, 191, 492, 522, 549–551, 564–566
memory_ , 459	__STDC_IEC_60559_COMPLEX__ , 24, 102, 188, 189, 191, 553
memory_order_ , 459	__STDC_IEC_60559_DFP__ , 9, 30, 188, 189, 191, 215, 219, 220, 235, 242–265, 266, 267–269, 270, 271–275, 276, 277–280, 359, 361, 390, 437, 479, 480, 486, 492, 508, 514, 520–522, 549–551, 564–566
mtx_ , 460	__STDC_IEC_60559_TYPES__ , 188, 189, 191, 477, 480, 493, 508, 514, 564–566
PRI , 224, 458	__STDC_ISO_10646__ , 188, 612
quantized , 277, 278, 389	__STDC_LIB_EXT1__ , 188, 189, 191, 478, 504–506, 508, 509, 512, 514, 636
quantumd , 278, 389	__STDC_MB_MIGHT_NEQ_WC__ , 45, 188, 315
samequantumd , 278, 389	__STDC_NO_ATOMICS__ , 188, 294, 501
SCN , 224, 458	__STDC_NO_COMPLEX__ , 189, 196, 476
SIG , 286, 459	__STDC_NO_THREADS__ , 16, 189, 392, 511
SIG_ , 286, 459	__STDC_NO_VLA__ , 189
stdc_ , 459	__STDC_UTF_16__ , 187
str , 458–460	__STDC_UTF_32__ , 187
thrd_ , 460	__STDC_VERSION_ASSERT_H__ , 195, 476
TIME_ , 402, 460	__STDC_VERSION_COMPLEX_H__ , 196, 476
to , 458, 460	__STDC_VERSION_FENV_H__ , 210, 478
tss_ , 460	__STDC_VERSION_FLOAT_H__ , 223, 479
UINT , 320, 321, 459, 505	__STDC_VERSION_INTTYPES_H__ , 224, 480
uint , 318, 319, 459, 505, 688	__STDC_VERSION_LIMITS_H__ , 228, 481
UINT_FAST , 320, 505	__STDC_VERSION_MATH_H__ , 235, 481
uint_fast , 319, 505, 688	__STDC_VERSION_SETJMP_H__ , 284, 501
UINT_LEAST , 320, 505	__STDC_VERSION_STDARG_H__ , 290, 501
uint_least , 318, 319, 321, 505	__STDC_VERSION_STDATOMIC_H__ , 294, 501
wcs , 458–460	__STDC_VERSION_STDBIT_H__ , 305, 502
identifier suffix	__STDC_VERSION_STDCKDINT_H__ , 314, 504
_DECIMAL_DIG , 29, 336, 360, 423, 437, 529, 530	__STDC_VERSION_STDEDEF_H__ , 315,
_r , 406	
_C , 321, 459, 505	
_H , 192	
_MAX , 23, 47, 320, 321, 459, 480, 505, 569	
_MIN , 23, 320, 321, 459, 480, 505, 570	
_WIDTH , 23, 320, 321, 459, 505	
_explicit , 294, 302, 502	
_t , 318, 319, 321, 459, 493, 505, 576, 577, 688	
f , 196, 235, 387, 389, 458	
l , 196, 235, 387, 389, 458	
macro	
_Complex_I , 196, 476, 557, 687	
_IOBF , 322, 330, 331, 505	
_IOLBF , 322, 331, 505	
_IONBF , 322, 330, 331, 505	
_Imaginary_I , 196, 203, 476, 557, 687	
_PRINTF_NAN_LEN_MAX , 323, 505	
_DATE , 187, 615	
_FILE , 169, 187, 195	
_LINE , 185, 187, 195, 599	
_STDC_ANALYZABLE , 188, 684	

504		
__STDC_VERSION_STDINT_H__,	318,	CHAR_MIN, 24, 40, 481, 519
505		CHAR_WIDTH, 23, 481, 519
__STDC_VERSION_STDIO_H__,	322, 505	ckd_add, 314, 504
__STDC_VERSION_STDLIB_H__,	357,	ckd_mul, 314, 504
507		ckd_sub, 314, 504
__STDC_VERSION_STRING_H__,	378,	CLOCKS_PER_SEC, 402, 403, 406, 511
509		CMPLX, 102, 196, 202, 203, 204, 477
__STDC_VERSION_TGMATH_H__,	387	CMPLXF, 202, 203, 477, 478, 574
__STDC_VERSION_TIME_H__,	402, 511	CMPLXL, 203, 477
__STDC_VERSION_UCHAR_H__,	412, 512	compl, 227, 481
__STDC_VERSION_WCHAR_H__,	417, 512	complex, 139, 196–204, 390, 476–478,
__STDC_VERSION__,	188, 685, 686, 688–	555, 573, 574, 596, 600
690		CR_DECIMAL_DIG, 25, 480, 529, 530
__STDC_WANT_IEC_60559_EXT__,	235,	d32add, 33, 216, 389
479, 492, 529, 549–551		d32div, 33, 216, 389
__STDC_WANT_IEC_60559_TYPES_EXT__,		d32fma, 33, 216, 389
477, 480, 493, 508, 514, 568, 573, 588–		d32mul, 33, 216, 389
590, 591, 592–594, 596		d32sqrt, 33, 216, 389
__STDC_WANT_LIB_EXT1__,	478, 504–	d32sub, 33, 216, 389
506, 508, 509, 512, 514, 636, 637, 639–		d64add, 33, 216, 389
655, 658–680		d64div, 33, 216, 389
__STDC__,	168, 187	d64fma, 33, 216, 389
__TIME__,	188, 615	d64mul, 33, 216, 389
__cplusplus,	168, 187	d64sqrt, 33, 216, 389
and,	227, 481	d64sub, 33, 216, 389
and_eq,	227, 481	dadd, 389, 595
assert,	145, 175, 177, 195, 220, 476, 605,	DBL_DECIMAL_DIG, 27, 30, 479, 520
615		DBL_DIG, 27, 28, 30, 479, 520
ATOMIC_BOOL_LOCK_FREE,	294, 501	DBL_EPSILON, 28, 30, 479, 520
ATOMIC_CHAR16_T_LOCK_FREE,	294,	DBL_HAS_SUBNORM, 26, 30, 458, 479
501		DBL_IS_IEC_60559, 26, 30, 479
ATOMIC_CHAR32_T_LOCK_FREE,	294,	DBL_MANT_DIG, 27, 30, 102, 479, 520
501		DBL_MAX, 28, 30, 479, 520
ATOMIC_CHAR8_T_LOCK_FREE,	294, 501	DBL_MAX_10_EXP, 28, 30, 479, 520
ATOMIC_CHAR_LOCK_FREE,	294, 501	DBL_MAX_EXP, 28, 30, 479, 520
ATOMIC_FLAG_INIT,	294, 303, 501	DBL_MIN, 29, 30, 479, 520
ATOMIC_INT_LOCK_FREE,	294, 501	DBL_MIN_10_EXP, 28, 30, 479, 520
ATOMIC_LLONG_LOCK_FREE,	294, 501	DBL_MIN_EXP, 28, 30, 479, 520
ATOMIC_LONG_LOCK_FREE,	294, 501	DBL_NORM_MAX, 28, 479, 520
ATOMIC_POINTER_LOCK_FREE,	294, 501	DBL_SNAN, 26, 101, 102, 479, 523
ATOMIC_SHORT_LOCK_FREE,	294, 501	DBL_TRUE_MIN, 29, 30, 479
ATOMIC_WCHAR_T_LOCK_FREE,	294, 501	ddiv, 389, 595, 596
bitand,	227, 481	DEC128_EPSILON, 31, 521
BITINT_MAXWIDTH,	23, 104, 481, 519,	DEC128_MANT_DIG, 31, 521
687		DEC128_MAX, 31, 521
bitor,	227, 481	DEC128_MAX_EXP, 31, 521
BOOL_MAX,	481, 519	DEC128_MIN, 32, 521
BOOL_WIDTH,	22, 481, 519	DEC128_MIN_EXP, 31, 521
bsearch,	371, 372, 459, 507, 600, 610,	DEC128_SNAN, 30
687		DEC128_TRUE_MIN, 32, 521
bsearch_s,	508, 653, 654	DEC32_EPSILON, 31, 521
BUFSIZ,	322, 325, 330, 505	DEC32_MANT_DIG, 31, 521
CHAR_BIT,	23, 42, 105, 171–173, 175,	DEC32_MAX, 31, 521
481, 519		DEC32_MAX_EXP, 31, 521
CHAR_MAX,	24, 231, 232, 481, 519	DEC32_MIN, 32, 521
		DEC32_MIN_EXP, 31, 521

DEC32_SNAN , 30, 102	FE_UNDERFLOW , 211, 222, 478
DEC32_TRUE_MIN , 32, 521	FE_UPWARD , 9, 211, 478, 527, 545
DEC64_EPSILON , 31, 521	FILENAME_MAX , 322, 505
DEC64_MANT_DIG , 31, 521	FLT_DECIMAL_DIG , 27, 29, 479, 520
DEC64_MAX , 31, 521	FLT_DIG , 27, 28, 29, 479, 520
DEC64_MAX_EXP , 31, 521	FLT_EPSILON , 28, 29, 479, 520
DEC64_MIN , 32, 521	FLT_EVAL_METHOD , 25, 26, 29, 30, 93–95, 102, 235, 479, 520, 552, 567, 576, 613, 615
DEC64_MIN_EXP , 31, 521	FLT_HAS_SUBNORM , 26, 30, 458, 479
DEC64_SNAN , 30, 102	FLT_IS_IEC_60559 , 26, 29, 479
DEC64_TRUE_MIN , 32, 100, 102, 521	FLT_MANT_DIG , 27, 29, 102, 479, 520
DEC_EVAL_METHOD , 25, 30, 62, 95, 235, 480, 520, 552, 567, 576, 613, 615	FLT_MAX , 28, 29, 30, 479, 520
DEC_INFINITY , 31, 102, 236, 278, 459, 480, 487	FLT_MAX_10_EXP , 28, 29, 30, 479, 520
DEC_NAN , 31, 102, 236, 459, 480, 487	FLT_MAX_EXP , 28, 29, 30, 479, 520
DECIMAL_DIG , 25, 27, 458, 479, 520, 686	FLT_MIN , 29, 30, 479, 520
dfma , 389, 595	FLT_MIN_10_EXP , 28, 29, 30, 479, 520
dmul , 389, 595	FLT_MIN_EXP , 28, 29, 479, 520
dsqrt , 389, 595	FLT_NORM_MAX , 28, 479, 520
dsub , 389, 595, 596	FLT_RADIX , 25, 27, 28, 29, 31, 61, 215, 220, 256, 257, 335, 336, 360, 421, 423, 436, 479, 520, 526, 567, 568, 576
EDOM , 209, 238, 478	FLT_ROUNDS , 25, 211, 479, 519, 520, 523, 613
EILSEQ , 209, 326, 413–416, 432, 449–451, 478	FLT_SNAN , 26, 101, 479, 523
EOF , 205, 322, 328, 343, 344, 346–352, 417, 427, 429–431, 433, 447, 448, 505, 605, 643, 644, 646–648, 650, 668, 670, 671, 673–675	FLT_TRUE_MIN , 29, 30, 479
ERANGE , 209, 226, 238, 239, 361, 362, 364, 437–440, 478, 615–617	FOPEN_MAX , 322, 326, 327, 505, 639
EXIT_FAILURE , 357, 369, 507	FP_FAST_D32ADDD128 , 237, 487
EXIT_SUCCESS , 357, 369, 398, 507	FP_FAST_D32ADDD64 , 237, 487
f32add , 596	FP_FAST_D32DIVD128 , 237, 487
f32fma , 596	FP_FAST_D32DIVD64 , 237, 487
f32mul , 596	FP_FAST_D32FMAD128 , 237, 487
f32xsqrt , 596	FP_FAST_D32FMAD64 , 237, 487
f64div , 596	FP_FAST_D32MULD128 , 237, 487
FE_ALL_EXCEPT , 94, 211, 478	FP_FAST_D32MULD64 , 237, 487
FE_DEC_DOWNWARD , 211, 215, 479, 528	FP_FAST_D32SQRTD128 , 237, 487
FE_DEC_TONEAREST , 211, 212, 215, 479, 528	FP_FAST_D32SQRTD64 , 237, 487
FE_DEC_TONEARESTFROMZERO , 211, 215, 479, 528	FP_FAST_D32SUBD128 , 237, 487
FE_DEC_TOWARDZERO , 211, 215, 479, 528	FP_FAST_D32SUBD64 , 237, 487
FE_DEC_UPWARD , 211, 215, 479, 528	FP_FAST_D64ADDD128 , 237, 487
FE_DFL_ENV , 212, 478	FP_FAST_D64DIVD128 , 237, 487
FE_DFL_MODE , 211, 219, 478, 527	FP_FAST_D64FMAD128 , 237, 487
FE_DIVBYZERO , 211, 238, 478	FP_FAST_D64MULD128 , 237, 487
FE_DOWNWARD , 211, 478, 527	FP_FAST_D64SQRTD128 , 237, 487
FE_INEXACT , 211, 216, 478, 546	FP_FAST_D64SUBD128 , 237, 487
FE_INVALID , 211, 218, 238, 478	FP_FAST_DADDL , 237, 481, 577
FE_OVERFLOW , 211, 216, 218, 238, 478	FP_FAST_DDIVL , 237, 481
FE_SNANS_ALWAYS_SIGNAL , 479, 523, 527, 545, 548	FP_FAST_DFMAL , 237, 481
FE_TONEAREST , 151, 211, 478, 527	FP_FAST_DMULL , 237, 481
FE_TONEARESTFROMZERO , 211, 478, 527	FP_FAST_DSQRTL , 237, 481
FE_TOWARDZERO , 211, 478, 527, 541, 546	FP_FAST_DSUBL , 237, 481
	FP_FAST_FADD , 237, 481, 577
	FP_FAST_FADDL , 237, 481, 577
	FP_FAST_FDIV , 237, 481
	FP_FAST_FDIVL , 237, 481
	FP_FAST_FFMA , 237, 481

FP_FAST_FFMAL , 237, 481	isinf , 240, 241, 481, 526, 527, 541, 555, 556
FP_FAST_FMA , 237, 481, 577	isless , 282, 486, 526, 534, 535
FP_FAST_FMA128 , 237, 487	islessequal , 282, 486, 526
FP_FAST_FMA32 , 237, 487	islessgreater , 282, 486
FP_FAST_FMA64 , 237, 487	isnan , 241, 481, 526, 527, 548, 555, 556
FP_FAST_FMAF , 237, 481, 493, 577	isnormal , 241, 481, 526, 527
FP_FAST_FMAL , 237, 481	issignaling , 242, 481, 526, 527
FP_FAST_FMUL , 237, 481	issubnormal , 242, 482, 526, 527
FP_FAST_FMULL , 237, 481	isunordered , 282, 283, 486, 526
FP_FAST_FSQRT , 237, 481	iszero , 242, 482, 526, 527
FP_FAST_FSQRTL , 237, 481	kill_dependency , 17, 298, 502
FP_FAST_FSUB , 237, 481	L_tmpnam , 323, 328, 505
FP_FAST_FSUBL , 237, 481	L_tmpnam_s , 506, 638, 639
FP_ILOGB0 , 237, 238, 252, 481	LC_ALL , 229, 230, 233, 481
FP_ILOGBNAN , 237, 238, 252, 481	LC_COLLATE , 229, 230, 381, 442, 481
FP_INFINITE , 236, 481	LC_CTYPE , 229, 230, 357, 373, 374, 447, 452, 455–457, 481, 610, 611, 655, 656
FP_INT_DOWNWARD , 236, 481	LC_MONETARY , 229, 230, 233, 481
FP_INT_TONEAREST , 236, 481	LC_NUMERIC , 229, 230, 233, 481
FP_INT_TONEARESTFROMZERO , 236, 481	LC_TIME , 229, 230, 406, 408, 481
FP_INT_TOWARDZERO , 236, 481	LDBL_DECIMAL_DIG , 27, 458, 479, 520
FP_INT_UPWARD , 236, 266, 481	LDBL_DIG , 27, 28, 479, 520
FP_LLOGB0 , 238, 253, 481	LDBL_EPSILON , 28, 479, 520
FP_LLOGBNAN , 238, 253, 481	LDBL_HAS_SUBNORM , 26, 458, 479
FP_NAN , 236, 481	LDBL_IS_IEC_60559 , 26, 479
FP_NORMAL , 236, 481	LDBL_MANT_DIG , 27, 479, 520
FP_SUBNORMAL , 236, 481	LDBL_MAX , 28, 479, 520
FP_ZERO , 236, 481	LDBL_MAX_10_EXP , 28, 479, 520
fpclassify , 240, 481, 526, 527	LDBL_MAX_EXP , 28, 479, 520
HUGE_VAL , 235, 239, 361, 437, 481, 535	LDBL_MIN , 29, 479, 520
HUGE_VAL_D128 , 236, 492	LDBL_MIN_10_EXP , 28, 479, 520
HUGE_VAL_D32 , 236, 492	LDBL_MIN_EXP , 28, 479, 520
HUGE_VAL_D64 , 236, 492	LDBL_NORM_MAX , 28, 479, 520
HUGE_VALF , 235, 239, 361, 437, 481, 535	LDBL_SNAN , 26, 479, 523
HUGE_VALL , 235, 239, 361, 437, 481, 535	LDBL_TRUE_MIN , 29, 479
imaginary , 196, 476, 557	LLONG_MAX , 23, 364, 440, 481, 519
INFINITY , 26, 101, 102, 203, 236, 334, 359–362, 421, 436–438, 459, 479, 481, 523, 555, 556	LLONG_MIN , 23, 279, 364, 440, 481, 519
INT_MAX , 23, 39, 103, 167, 237, 238, 252, 481, 519, 608	LLONG_WIDTH , 23, 481, 519
INT_MIN , 23, 39, 237, 238, 481, 519	LONG_MAX , 23, 238, 253, 364, 440, 481, 519
INT_WIDTH , 23, 266, 481, 519	LONG_MIN , 23, 238, 364, 440, 481, 519
INTMAX_C , 321, 505	LONG_WIDTH , 23, 481, 519
INTMAX_MAX , 226, 320, 505	MATH_ERREXCEPT , 238, 239, 361, 362, 437, 439, 481, 535, 536, 615
INTMAX_MIN , 226, 320, 505	math_errhandling , 193, 238, 239, 361, 362, 437–439, 481, 535, 536, 599, 606, 615, 690
INTMAX_WIDTH , 320, 505	MATH_ERRNO , 238, 239, 361, 362, 437–439, 481, 615
INTPTR_MAX , 320, 505	MB_CUR_MAX , 191, 335, 357, 374, 413–416, 449, 450, 507, 655, 656, 681
INTPTR_MIN , 320, 505	MB_LEN_MAX , 23, 191, 357, 481, 519
INTPTR_WIDTH , 320, 505	memchr , 382, 383, 460, 509, 687
iscanonical , 25, 240, 481, 526, 527	NAN , 27, 101, 102, 236, 323, 334, 360–362, 421, 436–438, 459, 479, 481, 523
iseqsig , 283, 486, 525, 552	
isfinite , 240, 481, 526, 527, 539, 556	
isgreater , 281, 486, 525, 526	
isgreaterequal , 281, 282, 486, 526, 534, 548	

NDEBUG, 145, 192, 195, 476
noreturn, 377
not, 227, 481
not_eq, 227, 481
NULL, 50, 229, 315, 322, 357, 378, 402, 417, 481, 504, 505, 507, 509, 511, 512, 616
offsetof, 108, 315, 504, 607
ONCE_FLAG_INIT, 357, 392, 507, 511
or, 227, 481
or_eq, 227, 481
PRIBMAX, 224, 480
PRIBMAX, 224, 480
PRIBPTR, 224, 480
PRIBPTR, 224, 480
PRIdFAST32, 224
PRIdMAX, 224, 480
PRIdPTR, 224, 480
PRIiMAX, 224, 480
PRIiPTR, 224, 480
PRIOiMAX, 224, 480
PRIOiPTR, 224, 480
PRIOuMAX, 224, 480
PRIOuPTR, 224, 480
PRIXMAX, 224, 480
PRIXMAX, 224, 225, 480
PRIXPTR, 224, 480
PRIXPTR, 224, 480
PTRDIFF_MAX, 505
PTRDIFF_MIN, 505
PTRDIFF_WIDTH, 320
putc, 323, 351, 506
putchar, 323, 351, 506
quantize, 33, 215, 524
quantum, 33, 524
RAND_MAX, 357, 364, 365, 507
RSIZE_MAX, 505, 638, 639, 640, 644, 645, 648–650, 652, 654–663, 665, 666, 669, 671, 672, 675–679, 681, 682
samequantum, 524
SCHAR_MAX, 23, 24, 481, 519
SCHAR_MIN, 23, 24, 40, 481, 519
SCHAR_WIDTH, 23, 481, 519
SCNbMAX, 225, 480
SCNbPTR, 225, 480
SCNdMAX, 224, 480
SCNdPTR, 224, 480
SCNiMAX, 224, 480
SCNiPTR, 224, 480
SCNoMAX, 225, 480
SCNoPTR, 225, 480
SCNuMAX, 225, 480
SCNuPTR, 225, 480
SCNxMAX, 225, 480
SCNxPTR, 225, 480
SEEK_CUR, 323, 353, 505
SEEK_END, 323, 326, 353, 505
SEEK_SET, 323, 353, 355, 505, 609
SHRT_MAX, 23, 481, 519
SHRT_MIN, 23, 481, 519
SHRT_WIDTH, 23, 481, 519
SIG_ATOMIC_MAX, 505
SIG_ATOMIC_MIN, 505
SIG_ATOMIC_WIDTH, 321, 505
SIG_DFL, 286, 287, 501, 616
SIG_ERR, 286, 287, 501, 607
SIG_IGN, 286, 287, 501, 612
SIGABRT, 286, 287, 368, 501
SIGFPE, 238, 286, 287, 501, 606, 611, 620
SIGILL, 286, 287, 501, 606, 611
SIGINT, 286, 501
signbit, 241, 242, 481, 526, 527, 547
SIGSEGV, 286, 287, 501, 606, 611
SIGTERM, 286, 501
SIZE_MAX, 41, 505, 638
SIZE_WIDTH, 321, 505
stdc_bit_ceil, 312, 504
stdc_bit_floor, 312, 504
stdc_bit_width, 311, 504
stdc_count_ones, 310, 503
stdc_count_zeros, 310, 503
stdc_first_leading_one, 308, 503
stdc_first_leading_zero, 308, 503
stdc_first_trailing_one, 309, 503
stdc_first_trailing_zero, 309, 503
stdc_has_single_bit, 311, 504
stdc_leading_ones, 306, 502
stdc_leading_zeros, 306, 502
stdc_trailing_ones, 307, 503
stdc_trailing_zeros, 307, 503
strchr, 382, 383, 460, 509, 687
strpbrk, 382, 383, 460, 509, 687
strrchr, 382, 383, 384, 460, 509, 687
strstr, 382, 384, 460, 509, 687
TIME_ACTIVE, 402, 406, 460, 512, 617
TIME_MONOTONIC, 402, 405, 460, 512, 617
TIME_THREAD_ACTIVE, 402, 406, 460, 512, 617
TIME_UTC, 395, 396, 399, 402, 405, 511, 617
TMP_MAX, 323, 327, 328, 505
TMP_MAX_S, 506, 638, 639, 640
TSS_DTOR_ITERATIONS, 392, 398, 511
UCHAR_MAX, 23, 24, 481, 519
UCHAR_WIDTH, 23, 481, 519
UINT64_C, 321
UINT_MAX, 23, 103, 112, 167, 481, 519
UINT_WIDTH, 23, 266, 481, 519
UINTMAX_C, 321, 505
UINTMAX_MAX, 225, 226, 320, 505, 529
UINTMAX_WIDTH, 320, 505

- UINTPTR_MAX**, 320, 505
- UINTPTR_WIDTH**, 320, 505
- ULLONG_MAX**, 23, 102, 112, 364, 440, 481, 519
- ULLONG_WIDTH**, 23, 481, 519
- ULONG_MAX**, 23, 364, 440, 481, 519
- ULONG_WIDTH**, 23, 481, 519
- unreachable**, viii, 315, 316, 504, 607, 688
- USHRT_MAX**, 23, 111, 481, 519
- USHRT_WIDTH**, 23, 481, 519
- WCHAR_MAX**, 417, 505, 512
- WCHAR_MIN**, 417, 505, 512
- WCHAR_WIDTH**, 321, 417, 505
- wcschr**, 443, 460, 513, 687
- wcspbrk**, 443, 444, 460, 513, 687
- wcsrchr**, 443, 444, 460, 513, 687
- wcsstr**, 443, 444, 445, 460, 513, 687
- WEOF**, 417, 432–435, 447, 452, 512, 515, 611
- WINT_MAX**, 505
- WINT_MIN**, 505
- WINT_WIDTH**, 321, 505
- wmemchr**, 443, 445, 446, 460, 513, 687
- xor**, 227, 481
- xor_eq**, 227, 481
- obsolete
 - __STDC_IEC_559_COMPLEX__**, 24, 188, 189–191, 553
 - __STDC_IEC_559__**, 24, 188, 190, 191, 492, 522
 - __bool_true_false_are_defined**, 313, 459, 504
 - gets**, 650, 688
- stream
 - stderr**, 185, 323, 324, 325, 330, 347, 429, 505, 619
 - stdin**, 323, 324, 325, 330, 343–345, 351, 428, 431, 433, 434, 505, 643, 644, 650, 675
 - stdout**, 323, 324, 325, 330, 337–339, 345, 351, 423, 431, 434, 505
- structure member
 - currency_symbol**, 229, 231, 234
 - decimal_point**, 229, 231
 - frac_digits**, 229, 231, 234
 - grouping**, 229, 231, 232
 - int_curr_symbol**, 229, 232–234
 - int_frac_digits**, 229, 232, 234
 - int_n_cs_precedes**, 229, 232, 234
 - int_n_sep_by_space**, 229, 232–234
 - int_n_sign_posn**, 229, 232–234
 - int_p_cs_precedes**, 229, 232, 234
 - int_p_sep_by_space**, 229, 232–234
 - int_p_sign_posn**, 229, 232–234
 - mon_decimal_point**, 229, 231, 234
 - mon_grouping**, 229, 231, 232, 234
 - mon_thousands_sep**, 229, 231, 234
 - n_cs_precedes**, 229, 231, 234
 - n_sep_by_space**, 229, 231, 232, 234
 - n_sign_posn**, 229, 232–234
 - negative_sign**, 229, 231, 232, 234
 - p_cs_precedes**, 229, 231, 234
 - p_sep_by_space**, 229, 231, 232, 234
 - p_sign_posn**, 229, 231, 233, 234
 - positive_sign**, 229, 231, 232, 234
 - thousands_sep**, 229, 231
 - tm_hour**, 403, 404, 407, 409, 665
 - tm_isdst**, 403, 404, 409
 - tm_mday**, 403, 404, 405, 407, 409, 665
 - tm_min**, 403, 404, 407, 409, 665
 - tm_mon**, 403, 404, 405, 407–409, 665
 - tm_sec**, 403, 404, 407, 409, 666
 - tm_wday**, 403, 404, 405, 407–409, 665
 - tm_yday**, 403, 404, 405, 409
 - tm_year**, 403, 404, 405, 407–409, 666
 - tv_nsec**, 403, 405
 - tv_sec**, 403, 405
- structure type
 - lconv**, 229, 230, 481
 - timespec**, 394, 396, 399, 402, 403, 405, 406, 511
 - tm**, 402, 403, 404–408, 417, 446, 511–513, 665, 666, 667
- summary, 476
- term, 191
- type
 - _BitInt**, 103
 - _Complex**, 103, 196
 - _Decimal128**, 103
 - _Decimal128x**, 566
 - _Decimal32**, 103
 - _Decimal32_t**, 235, 492, 615
 - _Decimal64**, 103
 - _Decimal64_t**, 235, 492, 615
 - _Decimal64x**, 566
 - _Float128_t**, 576, 577
 - _Float128x**, 566
 - _Float16_t**, 576, 577
 - _Float32_t**, 576, 577
 - _Float32x**, 566
 - _Float64_t**, 576, 577
 - _Float64x**, 566
 - _Imaginary**, 196
 - atomic_bool**, 300, 302, 501
 - atomic_char**, 300, 501
 - atomic_char16_t**, 300, 501
 - atomic_char32_t**, 300, 501
 - atomic_char8_t**, 300, 501
 - atomic_flag**, 294, 295, 303, 501, 502
 - atomic_int**, 295, 300, 501
 - atomic_int_fast16_t**, 300, 502

atomic_int_fast32_t, 300, 502
atomic_int_fast64_t, 300, 502
atomic_int_fast8_t, 300, 502
atomic_int_least16_t, 300, 501
atomic_int_least32_t, 300, 501
atomic_int_least64_t, 300, 501
atomic_int_least8_t, 300, 501
atomic_intmax_t, 300, 502
atomic_intptr_t, 300, 502
atomic_llong, 300, 501
atomic_long, 300, 501
atomic_ptrdiff_t, 300, 502
atomic_schar, 300, 501
atomic_short, 300, 501
atomic_size_t, 300, 502
atomic_uchar, 300, 501
atomic_uint, 300, 501
atomic_uint_fast16_t, 300, 502
atomic_uint_fast32_t, 300, 502
atomic_uint_fast64_t, 300, 502
atomic_uint_fast8_t, 300, 502
atomic_uint_least16_t, 300, 501
atomic_uint_least32_t, 300, 501
atomic_uint_least64_t, 300, 501
atomic_uint_least8_t, 300, 501
atomic_uintmax_t, 300, 502
atomic_uintptr_t, 300, 502
atomic_ullong, 300, 501
atomic_ulong, 300, 501
atomic_ushort, 300, 501
atomic_wchar_t, 300, 501
bool, 47, 103
char, 103
char16_t, 65–68, 138, 187, 300, **412**, 414, 512
char32_t, 65–68, 138, 187, 300, **412**, 415, 416, 512
char8_t, 65–68, 101, 300, **412**, 413, 512
clock_t, 402, **403**, 511, 617
cnd_t, 392, 393–395, 511
constraint_handler_t, 508, **651**
div_t, 141, 357, 372, 373, 507
double, 103
double_t, 235, 481, 532, **575**, 576, 577, 615, 619
enum, 103
errno_t, 478, 506, 508–510, 512, 514, 515, **637**, **638**, 639–641, **651**, 652, 654–657, **658**, 659–661, 663, **664**, 665, 666, 667, 675–678, 680–682
femode_t, 210, 211, 219, 478, 479
fenv_t, 93, 210, 212, 221, 222, 478, 479, 545
fexcept_t, 210, 216–218, 478, 479, 606
FILE, 174, **322**, 323, 325, 327, 328, 330, 331, 339, 347, 349–355, 418, 424, 429, 432–434, 505, 506, 512–514, 607, 639–642, 646, 647, 667, 668, 670, 671
float, 103
float_t, 235, 481, 532, **575**, 576, 577, 615, 619
fpos_t, 322, 324, 353, 354, 505, 506
imaxdiv_t, 224, 225, 480
int, 47, 59, 103
int8_t, 318
int_fast16_t, 300, **319**
int_fast32_t, 224, 300, **319**
int_fast64_t, 300, **319**
int_fast8_t, 300, **319**
int_least16_t, 300, 319
int_least32_t, 300, 318, 319
int_least64_t, 300, 319
int_least8_t, 300, 319
intmax_t, 23, 167, 225, 226, 300, **319**, 321, 333, 340, 419, 425, 480, 505, 687, 689
intptr_t, 300, **319**, 505
jmp_buf, 284, 285, 501, 684
ldiv_t, 357, 372, 373, 507
lldiv_t, 357, 372, 373, 507
long_double_t, 493, **575**, 576, 577
max_align_t, 45, **315**, 504
mbstate_t, 324–326, 335, 342, 354, **412**, 413–416, **417**, 422, 426, 427, 447–451, 512–515, 611, 617, 680–682
memory_order, **294**, **296**, 298, 299, 301–303, 459, 501, 502
mtx_t, **392**, 394–397, 511
nullptr_t, viii, 41, 47, 50, **51**, 84, 87, 88, 90–92, 291, **315**, **317**, 504, 607, 687
once_flag, 357, **392**, 393, 507, 511
ptrdiff_t, 85, 294, 300, **315**, 316, **320**, 333, 340, 420, 425, 504, 602, 687
rsize_t, 504, 506, 508–510, 512, 514, 515, **638**, **639**, 642–645, 648–650, **651**, 652–657, **658**, 659–663, **665**, 666, **667**, 668, 669, 671, 672, 675–682
sig_atomic_t, 14, **286**, 287, **321**, 501, 598, 606
signed, 103
thrd_start_t, 392, 397, 511
thrd_t, 392, 397, 398, 511
time_t, 402, **403**, 404, 405, 407, 408, 511, 512, 617, 666, 667
tss_dtor_t, 392, 399, 511
tss_t, 392, 399, 400, 511
uint64_t, 266
uint_fast16_t, 300, **319**
uint_fast32_t, 300, **319**
uint_fast64_t, 300, **319**
uint_fast8_t, 300, **319**
uint_least16_t, 300, 318, 319, 412

- uint_least32_t**, 300, 319, 412
- uint_least64_t**, 300, 319, 321
- uint_least8_t**, 300, 319
- uintmax_t**, 23, 167, 225, 226, 300, 320, 321, 333, 340, 419, 425, 480, 505, 687, 689
- uintptr_t**, 300, 319, 505
- unsigned**, 103, 332, 340, 419, 425
- va_list**, 290, 291–293, 347–349, 429–431, 501, 505, 506, 512–514, 607, 609, 646–649, 670–674
- void**, 50, 103
- wchar_t**, 5, 45, 64–67, 138, 188, 226, 300, 315, 321, 332, 335, 337, 340, 342, 345, 357, 373–375, 412, 417, 418, 419, 422–435, 437, 439–446, 449–452, 480, 504, 507, 509, 512–515, 591, 593, 594, 600, 611, 612, 655–657, 667–682
- wctrans_t**, 452, 456, 457, 515
- wctype_t**, 452, 455, 456, 515
- wint_t**, 321, 332, 335, 337, 417, 419, 422, 432–434, 447, 452, 453–456, 512, 513, 515, 611
- use of function, 193
- lifetime, 37
- limit** embed parameter, 165, 169, 171, 172, 174, 175–178
- line, 11
 - preprocessing directive, 164
- line buffered, 325
- line buffered stream, 325
- line number, 185, 187
- line** preprocessing directive, 185
- lines, 324
- linkage, 36, 97, 124, 128, 159, 161, 190
- literal encoding, 45
- little-endian, 305
- ll** format modifier, 333, 340, 419, 425
- llabs** function, 372, 507
- lldiv** function, 136, 357, 372, 373, 507
- lldiv_t** type, 357, 372, 373, 507
- llogb** function, 238, 253, 483, 524, 540
- llogb** type-generic macro, 388
- llogbd128** function, 253, 488
- llogbd32** function, 253, 488
- llogbd64** function, 253, 488
- llogbdN** function, 582
- llogbdNx** function, 582
- llogbf** function, 253, 483
- llogbfN** function, 582
- llogbfNx** function, 582
- llogbl** function, 253, 483
- LLONG_MAX** macro, 23, 364, 440, 481, 519
- LLONG_MIN** macro, 23, 279, 364, 440, 481, 519
- LLONG_WIDTH** macro, 23, 481, 519
- llquantexpd** identifier prefix, 279, 389
- llquantexpd128** function, 279, 492
- llquantexpd32** function, 279, 492
- llquantexpd64** function, 279, 492
- llquantexpdN** function, 587
- llquantexpdNx** function, 587
- llrint** function, 263, 484, 529, 545, 546, 600
- llrint** type-generic macro, 388
- llrintd128** function, 263, 490
- llrintd32** function, 263, 490
- llrintd64** function, 263, 490
- llrintdN** function, 584
- llrintdNx** function, 584
- llrintf** function, 263, 485
- llrintfN** function, 584
- llrintfNx** function, 584
- llrintl** function, 263, 485
- llround** function, 264, 485, 525, 546, 600
- llround** function, 264
- llround** type-generic macro, 388
- llroundd128** function, 264, 490
- llroundd32** function, 264, 490
- llroundd64** function, 264, 490
- llrounddN** function, 584
- llrounddNx** function, 584
- llroundf** function, 264, 485
- llroundfN** function, 584
- llroundfNx** function, 584
- llroundl** function, 264, 485
- local, 148
- local time, 402
- locale, 4
- locale-specific behavior, 4, 618
- localeconv** function, 230, 233, 481, 606
- localization header, 229, 458
- localtime** function, 404, 406, 407, 408, 512
- localtime_r** function, 408, 512, 686
- localtime_s** function, 512, 666, 667
- log** function, 238, 253, 255, 388, 483, 528, 540
- log** type-generic macro, 388
- log10** function, 253, 254, 483, 528, 540
- log10** type-generic macro, 388
- log10d128** function, 254, 488
- log10d32** function, 254, 488
- log10d64** function, 254, 488
- log10dN** function, 582
- log10dNx** function, 582
- log10f** function, 254, 483
- log10fN** function, 582
- log10fNx** function, 582
- log10l** function, 254, 483
- log10p1** function, 254, 255, 483, 528, 540
- log10p1** type-generic macro, 388
- log10p1d128** function, 254, 488
- log10p1d32** function, 254, 488
- log10p1d64** function, 254, 488
- log10p1dN** function, 582

- log10p1dNx** function, 582
- log10p1f** function, 254, 483
- log10p1fN** function, 582
- log10p1fNx** function, 582
- log10p1l** function, 254, 483
- log1p** function, 254, 255, 483, 528, 540
- log1p** type-generic macro, 388
- log1pd128** function, 254, 488
- log1pd32** function, 254, 488
- log1pd64** function, 254, 488
- log1pdN** function, 582
- log1pdNx** function, 582
- log1pf** function, 254, 483
- log1pfN** function, 582
- log1pfNx** function, 582
- log1pl** function, 254, 483
- log2** function, 255, 483, 528, 540
- log2** type-generic macro, 388
- log2d128** function, 255, 488
- log2d32** function, 255, 488
- log2d64** function, 255, 488
- log2dN** function, 582
- log2dNx** function, 582
- log2f** function, 255, 483
- log2fN** function, 582
- log2fNx** function, 582
- log2l** function, 255, 483
- log2p1** function, 255, 483, 528, 541
- log2p1** type-generic macro, 388
- log2p1d128** function, 255, 488
- log2p1d32** function, 255, 488
- log2p1d64** function, 255, 488
- log2p1dN** function, 582
- log2p1dNx** function, 582
- log2p1f** function, 255, 483
- log2p1fN** function, 582
- log2p1fNx** function, 582
- log2p1l** function, 255, 483
- logarithmic function
 - complex, 200, 561
 - real, 249, 539
- logb** function, 252, 253, 255, 256, 483, 524, 539, 541, 556, 588
- logb** type-generic macro, 388
- logbd128** function, 256, 489
- logbd32** function, 256, 488
- logbd64** function, 256, 489
- logbdN** function, 582
- logbdNx** function, 582
- logbf** function, 255, 483
- logbfN** function, 582
- logbfNx** function, 582
- logbl** function, 256, 483
- logd128** function, 253, 488
- logd32** function, 253, 488
- logd64** function, 253, 488
- logdN** function, 582
- logdNx** function, 582
- logf** function, 253, 483
- logfN** function, 582
- logfNx** function, 582
- logical operator
 - AND (&&), 17, 89
 - negation (!), 82
 - OR (| |), 90
 - OR (| |), 17
- logical source line, 11
- logl** function, 253, 483
- logp1** function, 254, 255, 483, 528, 540
- logp1** type-generic macro, 388
- logp1d128** function, 254, 488
- logp1d32** function, 254, 488
- logp1d64** function, 254, 488
- logp1dN** function, 582
- logp1dNx** function, 582
- logp1f** function, 254, 483
- logp1fN** function, 582
- logp1fNx** function, 582
- logp1l** function, 254, 483
- long double** **_Complex** type, 40
- long double** **_Complex** type conversion, 48
- long double** **_Imaginary** type, 553
- long double suffix, **l** or **L**, 62
- long double** type, 39, 103
- long double** type conversion, 47, 48
- long int** type, 39, 103
- long int** type conversion, 46–48
- long integer suffix, **l** or **L**, 59
- long** keyword, 53
- long long int** type, 39, 103
- long long int** type conversion, 46–48
- long long integer suffix, **ll** or **LL**, 59
- long_double_t** type, 493, 575, 576, 577
- LONG_MAX** macro, 23, 238, 253, 364, 440, 481, 519
- LONG_MIN** macro, 23, 238, 364, 440, 481, 519
- LONG_WIDTH** macro, 23, 481, 519
- longjmp** function, 284, 285, 369, 370, 501, 606, 609, 684
- loop body, 155
- low-order bit, 4
- lowercase letter, 20
- lrint** function, 263, 484, 529, 545, 546, 600
- lrint** type-generic macro, 388
- lrintd128** function, 263, 490
- lrintd32** function, 263, 490
- lrintd64** function, 263, 490
- lrintdN** function, 584
- lrintdNx** function, 584
- lrintf** function, 263, 484
- lrintfN** function, 584
- lrintfNx** function, 584

- lrintl** function, 263, 484
- lround** function, 264, 485, 525, 546, 600
- lround** function, 264
- lround** type-generic macro, 388
- lrounddd128** function, 264, 490
- lrounddd32** function, 264, 490
- lrounddd64** function, 264, 490
- lroundddN** function, 584
- lroundddNx** function, 584
- lroundf** function, 264, 485
- lroundfN** function, 584
- lroundfNx** function, 584
- lroundl** function, 264, 485
- lvalue**, 49, 78, 81, 91, 117
- lvalue conversion**, 50, 91–93
- macro
 - atomic lock-free, 294
 - math rounding direction, 236
 - number classification, 236
 - type-generic, 387
- macro argument substitution, 179
- macro definition
 - library function, 193
- macro invocation, 179
- macro name, 178
 - length, 22
 - predefined, 187, 190
 - redefinition, 178
 - scope, 182
- macro parameter, 179
- macro preprocessor, 163
- macro replacement, 178
- magnitude
 - complex, 201
- manipulation function
 - complex, 202
 - real, 268, 547
- matching failure, 430, 431, 671, 673, 674
- math rounding direction macro, 236
- MATH_** identifier prefix, 459
- MATH_ERREXCEPT** macro, 238, 239, 361, 362, 437, 439, 481, 535, 536, 615
- math_errhandling** macro, 193, 238, 239, 361, 362, 437–439, 481, 535, 536, 599, 606, 615, 690
- MATH_ERRNO** macro, 238, 239, 361, 362, 437–439, 481, 615
- mathematics header, 235, 458
- max_align_t** type, 45, 315, 504
- maximal munch, 52
- maximum function, 270, 548
- maybe_unused** attribute, 143, 145, 685, 686
- MB_CUR_MAX** macro, 191, 335, 357, 374, 413–416, 449, 450, 507, 655, 656, 681
- MB_LEN_MAX** macro, 23, 191, 357, 481, 519
- mblen** function, 373, 448, 507
- mbrlen** function, 448, 513
- mbrtoc16** function, 413, 414, 512
- mbrtoc32** function, 415, 512
- mbrtoc8** function, 412, 413, 512, 686
- mbrtowc** function, 326, 342, 422, 423, 447, 448, 449, 450, 513, 656, 682
- mbsinit** function, 448, 513
- mbsrtowcs** function, 447, 450, 514, 681
- mbsrtowcs_s** function, 515, 681, 682
- mbstate_t** type, 324–326, 335, 342, 354, 412, 413–416, 417, 422, 426, 427, 447–451, 512–515, 611, 617, 680–682
- mbstowcs** function, 68, 374, 375, 435, 450, 507
- mbstowcs_s** function, 509, 656, 657
- mbtowc** function, 66, 373, 374, 375, 448, 507
- mem** identifier prefix, 460
- memalignment** function, 9, 375, 507, 687
- member access operators (. and ->), 76
- member alignment, 106
- members, 37
- memccpy** function, 378, 509, 686
- memchr** macro, 382, 383, 460, 509, 687
- memcmp** function, 42, 174, 302, 381, 509
- memcpy** function, 42, 72, 171, 194, 295, 302, 378, 509, 525
- memcpy_s** function, 509, 658
- memmove** function, 72, 378, 379, 509, 525, 605
- memmove_s** function, 509, 658, 659
- memory location, 6
- memory management function, 365
- memory_** identifier prefix, 459
- memory_order_** identifier prefix, 459
- memory_order** type, 294, 296, 298, 299, 301–303, 459, 501, 502
- memory_order_acq_rel** constant, 296, 297, 298, 301–303, 501
- memory_order_acquire** constant, 296, 298, 301, 303, 501
- memory_order_consume** constant, 296, 298, 301, 501
- memory_order_relaxed** constant, 149, 296, 297, 298, 501
- memory_order_release** constant, 296, 298, 301, 302, 501
- memory_order_seq_cst** constant, 19, 42, 78, 92, 93, 294, 296, 297, 299, 501
- memset**
 - explicit, 385
- memset** function, 295, 385, 509, 663
- memset_explicit** function, 385, 509, 687
- memset_s** function, 510, 663
- minimum function, 270, 548
- minus operator
 - unary, 527
- minus operator, unary, 82
- miscellaneous function

- string, 385
- wide string, 446
- miscellaneous functions
 - string, 663
 - wide string, 680
- mktime** function, 404, 511
- modf** family, 34, 256, 387
- modf** function, 256, 387, 388, 483, 541
- modfd128** function, 256, 489
- modfd32** function, 256, 489
- modfd64** function, 256, 489
- modfdN** function, 582
- modfdNx** function, 582
- modff** function, 256, 483
- modffN** function, 582
- modffNx** function, 582
- modfl** function, 256, 483
- modifiable lvalue, 50
- modification order, 17
- modulus
 - complex, 201
- modulus function, 256
- mon_decimal_point** structure member, 229, 231, 234
- mon_grouping** structure member, 229, 231, 232, 234
- mon_thousands_sep** structure member, 229, 231, 234
- most significant index, 305
- mtx_** identifier prefix, 460
- mtx_destroy** function, 395, 511
- mtx_init** function, 392, 393, 395, 396, 511
- mtx_lock** function, 396, 511, 610
- mtx_plain** constant, 392, 396, 511
- mtx_recursive** constant, 392, 396, 511
- mtx_t** type, 392, 394–397, 511
- mtx_timed** constant, 393, 396, 511
- mtx_timedlock** function, 396, 511, 610
- mtx_trylock** function, 396, 397, 511
- mtx_unlock** function, 396, 397, 511, 610
- multibyte character, 5, 20, 64
- multibyte conversion function
 - wide character, 373, 655
 - extended, 447, 680
 - restartable, 412, 448, 680
 - wide string, 374, 656
 - restartable, 450, 681
- multibyte string, 191
- multibyte/wide character conversion function, 373, 655
 - extended, 447, 680
 - restartable, 412, 448, 680
- multibyte/wide string conversion function, 374, 656
 - restartable, 450, 681
- multidimensional array, 75
- multiplication assignment operator (**=**), 93
- multiplication operator (*****), 84, 554
- multiplicative expression, 84, 554
- multiply and round to narrower type, 276
- n-char sequence, 360
- n-wchar sequence, 436
- n_cs_precedes** structure member, 229, 231, 234
- n_sep_by_space** structure member, 229, 231, 232, 234
- n_sign_posn** structure member, 229, 232–234
- name
 - external, 22, 55, 190
 - file, 325
 - internal, 22, 55
 - label, 37
 - structure/union member, 37
- name space, 37
- named constant, 95
- named label, 153
- NaN, 24
- nan** function, 268, 334, 421, 485, 523, 547
- NAN** macro, 27, 101, 102, 236, 323, 334, 360–362, 421, 436–438, 459, 479, 481, 523
- nand128** function, 268, 490
- nand32** function, 268, 490
- nand64** function, 268, 490
- nandN** function, 585
- nandNx** function, 585
- nanf** function, 268, 485
- nanfN** function, 585
- nanfNx** function, 585
- nanl** function, 268, 485
- NDEBUG** macro, 145, 192, 195, 476
- nearbyint** function, 262, 263, 484, 527, 529, 541, 545
- nearbyint** type-generic macro, 388
- nearbyintd128** function, 263, 490
- nearbyintd32** function, 263, 489
- nearbyintd64** function, 263, 489
- nearbyintdN** function, 584
- nearbyintdNx** function, 584
- nearbyintf** function, 262, 484
- nearbyintfN** function, 584
- nearbyintfNx** function, 584
- nearbyintl** function, 262, 484
- nearest integer function, 262, 544
- negation operator (**!**), 82
- negative zero, 268
- negative_sign** structure member, 229, 231, 232, 234
- new line, 21
- new-line character, 11, 20, 52, 165, 185
- new-line escape sequence (**\n**), 21, 65, 207
- nextafter** function, 268, 269, 390, 485, 527, 547

- nextafter** type-generic macro, 388
- nextafterd128** function, 269, 490
- nextafterd32** function, 268, 490
- nextafterd64** function, 268, 490
- nextafterdN** function, 585
- nextafterdNx** function, 585
- nextafterf** function, 268, 485
- nextafterfN** function, 585
- nextafterfNx** function, 585
- nextafterl** function, 268, 485
- nextdown** function, 270, 485, 524, 548
- nextdown** type-generic macro, 388
- nextdownd128** function, 270, 490
- nextdownd32** function, 270, 490
- nextdownd64** function, 270, 490
- nextdowndN** function, 585
- nextdowndNx** function, 585
- nextdownf** function, 270, 485
- nextdownfN** function, 585
- nextdownfNx** function, 585
- nextdownl** function, 270, 485
- nexttoward** function, 269, 485, 527, 547
- nexttoward** type-generic macro, 388
- nexttowardd128** function, 269, 490
- nexttowardd32** function, 269, 490
- nexttowardd64** function, 269, 490
- nexttowardf** function, 269, 390, 485
- nexttowardl** function, 269, 485
- nextup** function, 269, 270, 485, 524, 548
- nextup** type-generic macro, 388
- nextupd128** function, 269, 490
- nextupd32** function, 269, 490
- nextupd64** function, 269, 490
- nextupdN** function, 585
- nextupdNx** function, 585
- nextupf** function, 269, 485
- nextupfN** function, 585
- nextupfNx** function, 585
- nextupl** function, 269, 485
- no linkage, 36
- no-return function, 125
- nodiscard** attribute, 142, 143, 144, 145, 685, 686
- non-canonical, 25
- non-canonical representation, 25
- non-graphic character, 21, 65
- non-local jumps header, 284
- non-stop floating-point control mode, 221
- non-value representation, 7, 42, 51, 76
- noreturn** attribute, 125, 143, 147, 148, 285, 368–370, 398, 501, 507, 509, 511, 685, 686
- noreturn** macro, 377
- norm
 - complex, 201
- normalized, 665
- normalized broken-down time, 665
- normalized floating-point number, 24
- not** macro, 227, 481
- not_eq** macro, 227, 481
- null character (\0), 19, 66, 67
 - padding of binary stream, 324
- NULL** macro, 50, 229, 315, 322, 357, 378, 402, 417, 481, 504, 505, 507, 509, 511, 512, 616
- null pointer, 50
- null pointer constant, 50
- null preprocessing directive, 187
- null statement, 153
- null wide character, 191
- nullptr** keyword, 53
- nullptr** predefined constant, 66
- nullptr_t** type, viii, 41, 47, 50, 51, 84, 87, 88, 90–92, 291, 315, 317, 504, 607, 687
- nullptr_t** type conversion, 51
- number classification macro, 236, 240
- numeric conversion function, 225, 357
 - wide string, 226, 435
- numerical limit, 22
- 0 format modifier, 409
- object, 6
- object representation, 42
- object type, 38
- object types, 38
- object-like macro, 178
- observable, 149
- observable behavior, 14
- observed, 149
- obsolescence, xiii, 190, 458
- octal constant, 57
- octal digit, 58, 65
- octal-character escape sequence (\octal digit), 65
- OFF** pragma, 151, 555, 556
- offsetof** macro, 108, 315, 504, 607
- ON** pragma, 93, 213, 218, 220, 222, 531–533, 541, 545, 547
- on-off switch, 186
- once_flag** type, 357, 392, 393, 507, 511
- ONCE_FLAG_INIT** macro, 357, 392, 507, 511
- opening, 325
- operand, 68, 72
- operating system, 12, 370
- operations on file, 326
- operations on files, 639
- operator, 68, 72
 - _Alignas**, 53
 - _Alignof**, 53
 - _has_c_attribute**, 142, 144–147, 150, 167, 168, 187, 685
 - _has_embed**, 52, 69, 167, 169, 187
 - _has_include**, 52, 69, 167, 168, 187, 687

- additive, 85
- alignas**, 53
- alignof**, 53, 82
- assignment, 91
- associativity, 72
- defined**, 167, 168, 174, 187, 604, 605
- equality, 87
- multiplicative, 84, 554
- postfix, 74
- precedence, 72
- preprocessing, 181, 189
- relational, 86
- shift, 86
- sizeof**, 53, 82
- typeof**, 117
- typeof**, 53
- typeof_unqual**, 53
- unary, 81
- unary arithmetic, 82
- or** macro, 227, 481
- OR operator
 - bitwise exclusive (^), 89
 - bitwise exclusive assignment (^=), 93
 - bitwise inclusive (|), 89
 - bitwise inclusive assignment (|=), 93
 - logical (||), 90
 - logical (|), 17
- or_eq** macro, 227, 481
- order of allocated storage, 365
- order of evaluation, 72, 91, 181
- ordinary identifier name space, 37
- orientation, 324
- orientation of stream, 324, 433
- out-of-bounds store, 8
- outer scope, 36
- over-aligned, 45
- p_cs_precedes** structure member, 229, 231, 234
- p_sep_by_space** structure member, 229, 231, 232, 234
- p_sign_posn** structure member, 229, 231, 233, 234
- padding
 - binary stream, 324
 - bit, 42, 318
 - structure/union, 42, 107
- parameter, 6
 - array, 160
 - ellipsis, 130, 179
 - embed, 171
 - function, 75, 98, 160
 - generic, 387
 - macro, 179
 - main** function, 13
 - program, 13
- parameter type list, 130
- parentheses punctuator (()), 130, 154, 155
- parenthesized expression, 73
- parse state, 324
- perform a trap, 7
- permitted form of initializer, 95
- pererror** function, 355, 356, 506
- phase angle
 - complex, 202
- physical source line, 11
- placemaker, 181
- plus operator
 - unary, 82
- pointer
 - null, 50
- pointer arithmetic, 85
- pointer comparison, 87
- pointer declarator, 127
- pointer operator (->), 76
- pointer to a string, 191
- pointer to a wide string, 191
- pointer to function, 75
- pointer type, 41
- pointer type conversion, 50
- pole error, 238, 247, 248, 253–262
- portability, 9
- positive difference, 271
- positive difference function, 270, 548
- positive_sign** structure member, 229, 231, 232, 234
- postfix decrement operator (--), 50, 78
- postfix expression, 74
- postfix increment operator (++), 50, 78
- pow** function, 258, 388, 484, 529, 542, 596
- pow** type-generic macro, 388
- powd128** function, 258, 489
- powd32** function, 258, 489
- powd64** function, 258, 390, 489
- powdN** function, 583
- powdNx** function, 583
- power function
 - complex, 201, 562
 - real, 257, 541
- powf** function, 258, 484
- powf32x** function, 596
- powf64** function, 596
- powfN** function, 583
- powfNx** function, 583
- powl** function, 258, 484, 527
- pown** function, 258, 259, 484, 529, 543
- pown** type-generic macro, 388
- pownd128** function, 259, 489
- pownd32** function, 259, 489
- pownd64** function, 259, 489
- powndN** function, 583
- powndNx** function, 583
- pownf** function, 258, 484

- pownfN** function, 583
- pownfNx** function, 583
- pownl** function, 259, 484
- powr** function, 259, 484, 529, 543
- powr** type-generic macro, 388
- powrd128** function, 259, 489
- powrd32** function, 259, 489
- powrd64** function, 259, 489
- powrdN** function, 583
- powrdNx** function, 583
- powrf** function, 259, 484
- powrfN** function, 583
- powrfNx** function, 583
- powrl** function, 259, 484
- pp-number, 70
- pragma
 - CX_LIMITED_RANGE**, vi, 186, 187, 197, 476, 554, 605
 - FE_DEC_DOWNWARD**, 187
 - FE_DEC_DYNAMIC**, 187, 215
 - FE_DEC_TONEAREST**, 187
 - FE_DEC_TONEARESTFROMZERO**, 187
 - FE_DEC_TOWARDZERO**, 187
 - FE_DEC_UPWARD**, 187
 - FE_DOWNWARD**, 187
 - FE_DYNAMIC**, 187, 213, 479
 - FE_TONEAREST**, 187
 - FE_TONEARESTFROMZERO**, 187
 - FE_TOWARDZERO**, 187
 - FE_UPWARD**, 187
 - FENV_ACCESS**, vi, 9, 93, 186, 187, 212, 213, 218, 220, 222, 478, 530–535, 541, 545, 547, 599, 605, 606, 613
 - FENV_DEC_ROUND**, vi, 62, 186, 187, 215, 479, 528, 575
 - FENV_ROUND**, vi, 29, 62, 151, 186, 187, 212, 213, 214, 215, 478, 479, 527, 575
 - FP_CONTRACT**, vii, 73, 151, 186, 187, 239, 481, 555, 556, 605, 613
 - OFF**, 151, 555, 556
 - ON**, 93, 213, 218, 220, 222, 531–533, 541, 545, 547
 - STDC**, 93, 151, 186, 190, 197, 212–215, 218, 220, 222, 239, 476, 478, 479, 481, 531–533, 541, 545, 547, 555, 556, 605, 615
- pragma**, 52, 69, 93, 151, 163, 186, 189, 197, 212–215, 218, 220, 222, 239, 476, 478, 479, 481, 531–533, 541, 545, 547, 555, 556, 597, 605, 614, 615
- pragma operator, 189
- pragma** preprocessing directive, 186, 190
- precedence of operator, 72
- precedence of syntax rule, 11
- precision, 43, 46, 331, 418
 - excess, 26, 49, 158
- predefined constant
 - false**, 53, 66
 - nullptr**, 53, 66
 - true**, 53, 66
- predefined macro name, 187, 190
- preferred quantum exponent, 32
- prefix decrement operator (--), 50, 81
- prefix** embed parameter, 172, 176, 177
- prefix increment operator (++), 50, 81
- preprocessing, 165
- preprocessing concatenation, 181
- preprocessing directive, 11, 163, 164
 - ifdef**, 9, 102, 163, 166, 167, 168, 215, 219, 220, 242–265, 266, 267–269, 270, 271–275, 276, 277–280, 359, 361, 390, 437, 520, 521, 549–551
 - pragma**, 52, 69, 93, 151, 163, 186, 189, 197, 212–215, 218, 220, 222, 239, 476, 478, 479, 481, 531–533, 541, 545, 547, 555, 556, 597, 605, 614, 615
 - undef**, 55, 163, 166, 182, 183, 187, 193, 194, 605
- preprocessing file, 11, 163
- preprocessing number, 52, 70
- preprocessing operator
 - #**, 181
 - ##**, 181
 - _Pragma**, 189
- preprocessing parameter, 165
- preprocessing token, 11, 52, 164
- preprocessing translation unit, 11
- preprocessor, 163
- preprocessor parameter, 165
 - prefixed, 165
 - standard, 165
- preprocessor prefixed parameter, 165
- preprocessor standard parameter, 165
- PRI** identifier prefix, 224, 458
- PRIBMAX** macro, 224, 480
- PRIBMAX** macro, 224, 480
- PRIBPTR** macro, 224, 480
- PRIBPTR** macro, 224, 480
- PRICFASTN** macro, 224
- PRICLEASTN** macro, 224
- PRICN** macro, 224
- PRIDFAST32** macro, 224
- PRIDMAX** macro, 224, 480
- PRIDPTR** macro, 224, 480
- PRIMAX** macro, 224, 480
- PRIPTR** macro, 224, 480
- primary block, 152
- primary expression, 73
- printf_s** function, 506, 643, 644
- printing character, 21, 205, 206
- printing wide character, 452
- PRIMAX** macro, 224, 480
- PRIPTR** macro, 224, 480

- PRIuMAX** macro, 224, 480
- PRIuPTR** macro, 224, 480
- PRIXMAX** macro, 224, 480
- PRIXMAX** macro, 224, 225, 480
- PRIXPTR** macro, 224, 480
- PRIXPTR** macro, 224, 480
- program
 - conforming, 9
 - strictly conforming, 9
- program diagnostic, 195
- program execution, 13
- program file, 11
- program image, 12
- program name, 13
- program name (**argv[0]**), 13
- program parameter, 13
- program parameters, 13
- Program semantics, 14
- program startup, 12, 13
- program structure, 11
- program termination, 12, 13, 14
- promotion
 - integer, 15, 47
- promotions
 - default argument, 76
- pseudo-random sequence function, 364
- PTRDIFF_MAX** macro, 320, 505
- PTRDIFF_MIN** macro, 320, 321, 505
- ptrdiff_t** type, 85, 294, 300, 315, 316, 320, 333, 340, 420, 425, 504, 602, 687
- PTRDIFF_WIDTH** macro, 320
- punctuator, 68
- putc** macro, 323, 351, 506
- putchar** macro, 323, 351, 506
- puts** function, 185, 316, 323, 351, 506
- putwc** function, 323, 434, 513
- putwchar** function, 323, 434, 513
- qsort** function, 371, 372, 507, 600
- qsort_s** function, 508, 653, 654, 655
- qualified type, 41
- qualified version of type, 41
- qualifier
 - _Atomic**, 41, 53
 - const**, 41, 53
 - restrict**, 41, 53
 - volatile**, 41, 53
- quantize** macro, 33, 215, 524
- quantized** identifier prefix, 277, 278, 389
- quantized128** function, 278, 491
- quantized32** function, 278, 491
- quantized64** function, 278, 491
- quantizedN** function, 587
- quantizedNx** function, 587
- quantum, 32
- quantum exponent, 32
- quantum exponent function, 277
- quantum function, 277
- quantum** macro, 33, 524
- quantumd** identifier prefix, 278, 389
- quantumd128** function, 278, 492
- quantumd32** function, 278, 492
- quantumd64** function, 278, 492
- quantumdN** function, 587
- quantumdNx** function, 587
- question-mark escape sequence (**\?**), 65
- quick_exit** function, 287, 368, 369, 370, 507, 600, 606, 609, 617, 688
- quiet NaN, 24
- raise** function, 286, 287, 288, 295, 368, 501, 606, 607
- rand** function, 357, 364, 365, 507
- RAND_MAX** macro, 357, 364, 365, 507
- range
 - excess, 26, 49, 158
- range error, 238, 243–255, 257–259, 261–264, 269, 271, 275
- read-modify-write operation, 17
- read-read coherence, 19
- read-write coherence, 19
- real floating type, 40
- real floating type conversion, 47, 48, 529
- real floating types, 566
- real type, 40
- real type domain, 40
- real-floating, 240
- realloc** function, 365, 366, 367, 368, 507, 600, 609, 617, 684, 688
- recommended practice, 6
- recursion, 76
- recursive function call, 76
- redefinition of macro, 178
- reentrancy, 14, 21
 - library function, 194
- referenced type, 41
- register**, 99
- register** storage-class specifier, 53, 159
- relational expression, 86
- relaxed atomic operation, 17
- release fence, 298
- release operation, 17
- release sequence, 17
- reliability of data
 - interrupted, 14
- remainder assignment operator (**%=**), 93
- remainder function, 266, 546
- remainder** function, 267, 390, 485, 524, 528, 547, 615
- remainder operator (**%**), 84
- remainder** type-generic macro, 388
- remainderd128** function, 267, 490
- remainderd32** function, 267, 490
- remainderd64** function, 267, 490

- remainderdN** function, 584
- remainderdNx** function, 584
- remainderf** function, 267, 485
- remainderfN** function, 584
- remainderfNx** function, 584
- remainderl** function, 267, 485
- remove** function, 326, 327, 505, 616, 639
- remquo** function, 267, 485, 524, 528, 547, 599, 615
- remquo** type-generic macro, 388
- remquof** function, 267, 485
- remquofN** function, 584
- remquofNx** function, 584
- remquol** function, 267, 485
- rename** function, 327, 505, 616
- representation
 - canonical, 25
 - non-canonical, 25
- representations of type, 42
 - pointer, 41
- reproducible, 149
- reproducible** attribute, 143, 148, 150, 151, 604, 685, 686
- rescanning and replacement, 182
- reserved identifier, 53, 192, 620, 628, 637
- resource, 166, 171
 - #embed** preprocessing directive, 171
 - empty, 171
- resource width, 171
- restartable multibyte/wide character conversion function, 412, 448, 680
- restartable multibyte/wide string conversion function, 450, 681
- restore calling environment function, 284
- restrict**, 41
- restrict** keyword, 53
- restrict** type qualifier, 120, 122
- restrict-qualified type, 41, 121
- return** keyword, 53
- return** statement, 158
- rewind** function, 329, 352, 354, 355, 434, 506
- right-shift assignment operator (**>>=**), 93
- right-shift operator (**>>**), 86
- rint** function, 263, 484, 524, 529, 545, 546
- rint** type-generic macro, 388
- rintd128** function, 263, 490
- rintd32** function, 263, 490
- rintd64** function, 263, 490
- rintdN** function, 584
- rintdNx** function, 584
- rintf** function, 263, 484
- rintfN** function, 584
- rintfNx** function, 584
- rintl** function, 263, 484
- rootn** function, 259, 260, 484, 528, 543
- rootn** type-generic macro, 388
- rootnd128** function, 259, 489
- rootnd32** function, 259, 489
- rootnd64** function, 259, 489
- rootndN** function, 583
- rootndNx** function, 583
- rootnf** function, 259, 484
- rootnfN** function, 583
- rootnfNx** function, 583
- rootnl** function, 259, 484
- round** function, 236, 264, 485, 523, 545
- round to narrower type, 275
- round** type-generic macro, 388
- roundd128** function, 264, 490
- roundd32** function, 264, 490
- roundd64** function, 264, 490
- rounddN** function, 584
- rounddNx** function, 584
- roundeven** function, 236, 264, 265, 485, 523, 546
- roundeven** type-generic macro, 388
- roundevend128** function, 264, 490
- roundevend32** function, 264, 490
- roundevend64** function, 264, 490
- roundevendN** function, 584
- roundevendNx** function, 584
- roundevenf** function, 264, 485
- roundevenfN** function, 584
- roundevenfNx** function, 584
- roundevenl** function, 264, 485
- roundf** function, 264, 485
- roundfN** function, 584
- roundfNx** function, 584
- rounding, 264
- rounding control pragma, 213
- rounding direction, 236
- rounding mode
 - floating-point, 25
- roundl** function, 264, 485
- RSIZE_MAX** macro, 505, 638, 639, 640, 644, 645, 648–650, 652, 654–663, 665, 666, 669, 671, 672, 675–679, 681, 682
- rsizet** type, 504, 506, 508–510, 512, 514, 515, 638, 639, 642–645, 648–650, 651, 652–657, 658, 659–663, 665, 666, 667, 668, 669, 671, 672, 675–682
- rsqrt** function, 260, 484, 528, 544
- rsqrt** type-generic macro, 388
- rsqrtd128** function, 260, 489
- rsqrtd32** function, 260, 489
- rsqrtd64** function, 260, 489
- rsqrtdN** function, 583
- rsqrtdNx** function, 583
- rsqrtf** function, 260, 484
- rsqrtfN** function, 583
- rsqrtfNx** function, 583
- rsqrtl** function, 260, 484

- runtime-constraint, 6
- Runtime-constraint handling function, 651
- rvalue, 49
- same scope, 36
- samequantum** macro, 524
- samequantumd** identifier prefix, 278, 389
- samequantumd128** function, 278, 492
- samequantumd32** function, 278, 491
- samequantumd64** function, 278, 492
- samequantumdN** function, 587
- samequantumdNx** function, 587
- save calling environment function, 284
- scalar type, 41
- scalbln** function, 256, 257, 484, 524, 541, 588
- scalbln** type-generic macro, 388
- scalblnd128** function, 256, 489
- scalblnd32** function, 256, 489
- scalblnd64** function, 256, 489
- scalblndN** function, 582
- scalblndNx** function, 582
- scalblnf** function, 256, 484
- scalblnfN** function, 582
- scalblnfNx** function, 582
- scalblnl** function, 256, 484
- scalbn** function, 256, 257, 483, 524, 539, 540, 541, 556, 588
- scalbn** type-generic macro, 388
- scalbnd128** function, 256, 489
- scalbnd32** function, 256, 489
- scalbnd64** function, 256, 489
- scalbndN** function, 582
- scalbndNx** function, 582
- scalbnf** function, 256, 483
- scalbnfN** function, 582
- scalbnfNx** function, 582
- scalbnl** function, 256, 483
- scanf** function, 33, 214, 216, 323, 345, 346, 348, 505, 525, 688, 689
- scanf_s** function, 506, 644, 648
- scanlist, 342, 427
- scanset, 342, 426
- SCHAR_MAX** macro, 23, 24, 481, 519
- SCHAR_MIN** macro, 23, 24, 40, 481, 519
- SCHAR_WIDTH** macro, 23, 481, 519
- SCN identifier prefix, 224, 458
- SCNbMAX** macro, 225, 480
- SCNbPTR** macro, 225, 480
- SCNcFASTN** macro, 224
- SCNcLEASTN** macro, 224
- SCNcN** macro, 224
- SCNdMAX** macro, 224, 480
- SCNdPTR** macro, 224, 480
- SCNiMAX** macro, 224, 480
- SCNiPTR** macro, 224, 480
- SCNoMAX** macro, 225, 480
- SCNoPTR** macro, 225, 480
- SCNuMAX** macro, 225, 480
- SCNuPTR** macro, 225, 480
- SCNxMAX** macro, 225, 480
- SCNxPTR** macro, 225, 480
- scope, 35
- scope of identifier, 35, 161
- search function
 - string, 382
 - utility, 371, 653
 - wide string, 443
- search functions
 - string, 662
 - wide string, 679
- secondary block, 152
- SEEK_CUR** macro, 323, 353, 505
- SEEK_END** macro, 323, 326, 353, 505
- SEEK_SET** macro, 323, 353, 355, 505, 609
- selection
 - _Generic**, 53, 74, 113, 121, 136, 317, 389
- selection statement, 154
- self-referential structure, 116
- semicolon punctuator (;), 97, 105, 153, 155, 156
- separate compilation, 11
- separate translation, 11
- sequence point, 14, 76, 89, 90, 94, 121, 122, 152, 193, 194, 331, 371, 418, 516, 653
- sequenced before, 14, 72, 76, 78, 91
- sequenced during a function call, 149
- sequencing of statement, 152
- sequential consistency, 19
- set_constraint_handler_s** function, 508, 637, 651, 652
- setbuf** function, 322, 325, 326, 328, 330, 505
- setjmp** function, 193, 284, 285, 501, 599, 606, 684
- setlocale** function, vii, 191, 229, 230, 233, 406, 481, 606, 615
- setpayload** function, 492, 524, 551
- setpayloadd128** function, 493, 551
- setpayloadd32** function, 493, 551
- setpayloadd64** function, 493, 551
- setpayloaddN** function, 588
- setpayloaddNx** function, 588
- setpayloadf** function, 492, 551
- setpayloadfN** function, 588
- setpayloadfNx** function, 588
- setpayloadl** function, 492, 551
- setpayloadsig** function, 492, 524, 551
- setpayloadsigd128** function, 493, 551
- setpayloadsigd32** function, 493, 551
- setpayloadsigd64** function, 493, 551
- setpayloadsigdN** function, 588
- setpayloadsigdNx** function, 588
- setpayloadsigf** function, 492, 551
- setpayloadsigfN** function, 588

- setpayloadsigfNx** function, 588
- setpayloadsigl** function, 492, 551
- setvbuf** function, 322, 325, 326, 328, 330, 331, 505, 608
- shall, 9
- shift expression, 86
- shift sequence, 191
- shift state, 20
 - initial, 20
- short identifier
 - character, 22
- short int** type, 39, 103
- short int** type conversion, 46–48
- short** keyword, 53
- SHRT_MAX** macro, 23, 481, 519
- SHRT_MIN** macro, 23, 481, 519
- SHRT_WIDTH** macro, 23, 481, 519
- side effect, 14, 42, 50, 72, 78, 91, 139, 153, 210, 213, 350, 351, 433, 434, 530, 533, 534
- SIG** identifier prefix, 286, 459
- SIG_** identifier prefix, 286, 459
- SIG_ATOMIC_MAX** macro, 321, 505
- SIG_ATOMIC_MIN** macro, 321, 505
- sig_atomic_t** type, 14, 286, 287, 321, 501, 598, 606
- SIG_ATOMIC_WIDTH** macro, 321, 505
- SIG_DFL** macro, 286, 287, 501, 616
- SIG_ERR** macro, 286, 287, 501, 607
- SIG_IGN** macro, 286, 287, 501, 612
- SIGABRT** macro, 286, 287, 368, 501
- SIGFPE** macro, 238, 286, 287, 501, 606, 611, 620
- SIGILL** macro, 286, 287, 501, 606, 611
- SIGINT** macro, 286, 501
- sign bit, 43
- signal, 14, 21, 286
- signal** function, 15, 16, 134, 286, 287, 369, 370, 501, 606, 607, 612, 616
- signal handler, 14, 21, 287, 288
- signal handling function, 286
- signal handling header, 286, 459
- signaling NaN, 24, 523
- signals, 286
- signbit** macro, 241, 242, 481, 526, 527, 547
- signed char** type, 39
- signed character, 47
- signed integer type, 39, 47, 59
- signed integer types, 39
- signed** keyword, 53
- signed** type, 39, 103
- signed** type conversion, 46–48
- significand part, 61
- SIGSEGV** macro, 286, 287, 501, 606, 611
- SIGTERM** macro, 286, 501
- simple assignment, 92
- simple assignment operator (=), 92
- sin** function, 77, 244, 388, 390, 482, 529, 537, 563
- sin** type-generic macro, 388, 563
- sind128** function, 244, 487
- sind32** function, 244, 487
- sind64** function, 244, 487
- sindN** function, 579
- sindNx** function, 579
- sinf** function, 244, 482
- sinfN** function, 579
- sinfNx** function, 579
- single-byte character, 20
- single-byte/wide character conversion function, 447
- single-quote escape sequence (\'), 65, 67
- singularity, 238
- sinh** function, 249, 388, 482, 529, 538, 563
- sinh** type-generic macro, 388, 563
- sinhd128** function, 249, 488
- sinhd32** function, 249, 488
- sinhd64** function, 249, 488
- sinhdN** function, 580
- sinhdNx** function, 580
- sinhf** function, 249, 482
- sinhfN** function, 580
- sinhfNx** function, 580
- sinhl** function, 249, 482
- sinl** function, 244, 482
- sinpi** function, 247, 482, 529, 538
- sinpi** type-generic macro, 388
- sinpid128** function, 247, 487
- sinpid32** function, 247, 487
- sinpid64** function, 247, 487
- sinpidN** function, 579
- sinpidNx** function, 579
- sinpif** function, 247, 482
- sinpifN** function, 579
- sinpifNx** function, 579
- sinpil** function, 247, 482
- SIZE_MAX** macro, 41, 505, 638
- SIZE_WIDTH** macro, 321, 505
- sizeof** keyword, 53
- sizeof** operator, 50, 81, 82
- snprintf** function, 346, 348, 358, 359, 407, 505, 644, 690
- snprintf_s** function, 506, 644, 645
- snwprintf_s** function, 514, 669, 670
- sorting utility function, 371, 653
- source character set, 11, 19
- source file, 11
 - name, 185, 187
- source file inclusion, 169
- source line, 11
- source text, 11
- space character (' '), 11, 20, 52, 206, 207, 453
- space* format flag, 332, 419

- spilling, **15**
- sprintf** function, **346**, 349, 505, 645
- sprintf_s** function, 506, **645**
- sqrt** function, 151, **260**, 388, 484, 524, **544**, 549
- sqrt** type-generic macro, **388**
- sqrtd128** function, **260**, 489
- sqrtd32** function, **260**, 390, 489
- sqrtd64** function, **260**, 489
- sqrtdN** function, **583**
- sqrtdNx** function, **583**
- sqrtf** function, **260**, 484
- sqrtfN** function, **583**
- sqrtfNx** function, **583**
- sqrtrl** function, **260**, 484
- square root rounded to narrower type, **277**
- srand** function, **364**, 365, 507
- sscanf** function, 344, **346**, 349, 505
- sscanf_s** function, 506, **645**, 646, 649
- standard attribute, **142**
- standard embed parameter, **172**
- standard error stream, 323, **325**, 355
- standard floating type, **39**
- standard header, **9**, **191**
- standard headers
 - <assert.h>, 174–176, 192, **195**, 220, **476**
 - <complex.h>, 25, 30, 101, 139, 189, 191, 192, **196–204**, 387, 388, **458**, **476**, 477, 555, **557**, **573**, 575, 610, 613, 620, 688, 689
 - <ctype.h>, 192, **205**, 206–208, **458**, **478**
 - <errno.h>, 150, 192, **209**, **458**, **478**, **637**
 - <fcntl.h>, 9, 14, 16, 25, 30, 34, 93, 150, 192, **210**, 212, 213, 215–222, 238, **458**, **478**, 523, 527, 530–533, 541, 545, 547, 690
 - <float.h>, 9, 10, 22, **24**, 25, 29, **30**, 101, 192, **223**, 236, 336, 360, 423, 437, **458**, 459, **479**, 480, 519, 523, 529, **567**, 568, 617, 688, 689
 - <inttypes.h>, 192, **224**, 225, 226, 334, 420, **458**, **480**, 689
 - <iso646.h>, 9, 10, 192, **227**, **480**, 689, 690
 - <limits.h>, 9, 10, **22**, 23, 39, 40, 192, **228**, **481**, 519, 617
 - <locale.h>, 150, 192, **229**, 230, **458**, **481**
 - <math.h>, 9, 25, 30, 34, 73, 149, 150, 192, 214–216, **235**, 236, 238–241, **242–269**, 270, **271–275**, 276, **277–282**, 283, 337, 387–389, 423, **458**, 459, **481**, 492, 493, 522, 523, 529, **535**, 536, 541, 545, 547–551, 555, 565, **575**, **588–590**, 600, 610, 613, 617, 620, 689
 - <setjmp.h>, 192, **284**, 285, **501**
 - <signal.h>, 192, **286**, 288, **459**, **501**
 - <stdalign.h>, 9, 10, 192, **289**, **501**, 688
 - <stdarg.h>, 9, 10, 130, 192, **290**, 291–293, 347–349, 429–431, **501**, 646–649, 670–673
 - <stdatomic.h>, 188, 191, 192, 287, **294**, 295, 298, 299, 301–303, **459**, **501**, 606, 688
 - <stdbit.h>, 9, 192, **305**, 306–312, **459**, **502**, 687
 - <stdbool.h>, 9, 10, 192, **313**, **459**, **504**, 690
 - <stdckdint.h>, 192, **314**, **459**, **504**
 - <stddef.h>, 9, 10, 50, 66, 68, 83, 85, 86, 141, 172, 173, 192, 226, **315**, 316, 317, 345, **504**, **638**
 - <stdint.h>, 9, 10, 22–24, 167, 192, 224, 225, **318**, 320, 321, 333, 340, 341, 420, 425, **459**, **504**, 505, 617, **638**, 689
 - <stdio.h>, 16, 25, 30, 34, 56, 150, 170, 173, 192, 214, 216, **322**, 326–328, 330, 331, 337, 339, 343–355, 404, 418, 423, 424, 427–429, 431–434, **459**, **505**, 506, 613, **638**, 639–650, 668, 670, 671, 688–690
 - <stdlib.h>, 9, 25, 30, 34, 192, 194, 214, 216, **357**, 358, 359, 361, 363–375, **459**, **506**, 508, 565, **590**, 591, **592**, **593**, 613, 637, **651**, 652–657, 688
 - <stdnoreturn.h>, 9, 10, 147, 192, **377**, **509**
 - <string.h>, 9, 173, 176, 177, 192, **378**, 379–386, **460**, **509**, **658**, 659–664
 - <tgmath.h>, 34, 192, 387, 390, **510**, 522, **535**, **563**, **594**, 596, 689
 - <threads.h>, 149, 150, 189, 191, 192, **392**, 393–400, **460**, **511**, 688
 - <time.h>, 192, 392, **402**, 403–408, 446, **460**, **511**, 512, **664**, 665–667, 688
 - <uchar.h>, 66, 68, 192, **412**, 413–416, **512**, 686, 688
 - <wchar.h>, 25, 30, 34, 150, 192, 214, 216, 225, 323, **417**, 418, 423, 424, 428–435, 437, 439–451, **460**, **512**, 514, **593**, **594**, 613, **667**, 668–682, 689, 690
 - <wctype.h>, 192, **452**, 453–457, **460**, **515**, 689, 690
- standard input stream, 323, **325**
- standard integer type, **39**
- standard output stream, 323, **325**
- standard signed integer type, **39**
- standard unsigned integer type, **39**
- state-dependent encoding, **20**, 373, 655
- stateless function, **149**
- statement, **152**
 - break**, 53, **158**
 - compound, **153**
 - continue**, 53, 156, **157**, 158
 - do**, 53, **156**
 - else**, 53, **154**
 - expression, **153**
 - for**, 53, **156**

- goto**, 53, 156
- if**, 53, 154
- iteration, 155
- jump, 156
- labeled, 153
- null, 153
- return**, 53, 158, 530
- selection, 154
- sequencing, 152
- switch**, 53, 154
- while**, 53, 156
- static**, 99
- static assertion, 142
- static storage duration, 37
- static** storage-class specifier, 36, 37, 53
- static**, in array declarator, 128, 130
- static_assert**, 142
- static_assert** storage-class specifier, 53
- STDC** pragma, 93, 151, 186, 190, 197, 212–215, 218, 220, 222, 239, 476, 478, 479, 481, 531–533, 541, 545, 547, 555, 556, 605, 615
- stdc_** identifier prefix, 459
- stdc_bit_ceil** macro, 312, 504
- stdc_bit_ceil_uc** function, 312, 504
- stdc_bit_ceil_ui** function, 312, 504
- stdc_bit_ceil_ul** function, 312, 504
- stdc_bit_ceil_us** function, 312, 504
- stdc_bit_floor** macro, 312, 504
- stdc_bit_floor_uc** function, 312, 504
- stdc_bit_floor_ui** function, 312, 504
- stdc_bit_floor_ul** function, 312, 504
- stdc_bit_floor_ull** function, 312, 504
- stdc_bit_floor_us** function, 312, 504
- stdc_bit_width** macro, 311, 504
- stdc_bit_width_uc** function, 311, 504
- stdc_bit_width_ui** function, 311, 504
- stdc_bit_width_ul** function, 311, 504
- stdc_bit_width_ull** function, 311, 504
- stdc_bit_width_us** function, 311, 504
- stdc_count_ones** macro, 310, 503
- stdc_count_ones_uc** function, 310, 503
- stdc_count_ones_ui** function, 310, 503
- stdc_count_ones_ul** function, 310, 503
- stdc_count_ones_ull** function, 310, 503
- stdc_count_ones_us** function, 310, 503
- stdc_count_zeros** macro, 310, 503
- stdc_count_zeros_uc** function, 310, 503
- stdc_count_zeros_ui** function, 310, 503
- stdc_count_zeros_ul** function, 310, 503
- stdc_count_zeros_ull** function, 310, 503
- stdc_count_zeros_us** function, 310, 503
- stdc_first_leading_one** macro, 308, 503
- stdc_first_leading_one_uc** function, 308, 503
- stdc_first_leading_one_ui** function, 308, 503
- stdc_first_leading_one_ul** function, 308, 503
- stdc_first_leading_one_ull** function, 308, 503
- stdc_first_leading_one_us** function, 308, 503
- stdc_first_leading_zero** macro, 308, 503
- stdc_first_leading_zero_uc** function, 308, 503
- stdc_first_leading_zero_ui** function, 308, 503
- stdc_first_leading_zero_ul** function, 308, 503
- stdc_first_leading_zero_ull** function, 308, 503
- stdc_first_leading_zero_us** function, 308, 503
- stdc_first_trailing_one** macro, 309, 503
- stdc_first_trailing_one_uc** function, 309, 503
- stdc_first_trailing_one_ui** function, 309, 503
- stdc_first_trailing_one_ul** function, 309, 503
- stdc_first_trailing_one_ull** function, 309, 503
- stdc_first_trailing_one_us** function, 309, 503
- stdc_first_trailing_zero** macro, 309, 503
- stdc_first_trailing_zero_uc** function, 309, 503
- stdc_first_trailing_zero_ui** function, 309, 503
- stdc_first_trailing_zero_ul** function, 309, 503
- stdc_first_trailing_zero_ull** function, 309, 503
- stdc_first_trailing_zero_us** function, 309, 503
- stdc_has_single_bit** macro, 311, 504
- stdc_has_single_bit_uc** function, 311, 503
- stdc_has_single_bit_ui** function, 311, 503
- stdc_has_single_bit_ul** function, 311, 503
- stdc_has_single_bit_ull** function, 311, 504
- stdc_has_single_bit_us** function, 311, 503
- stdc_leading_ones** macro, 306, 502
- stdc_leading_ones_uc** function, 306, 502
- stdc_leading_ones_ui** function, 306, 502
- stdc_leading_ones_ul** function, 306, 502
- stdc_leading_ones_ull** function, 306, 502
- stdc_leading_ones_us** function, 306, 502
- stdc_leading_zeros** macro, 306, 502
- stdc_leading_zeros_uc** function, 306, 502

- stdc_leading_zeros_ui** function, 306, 502
- stdc_leading_zeros_ul** function, 306, 502
- stdc_leading_zeros_ull** function, 306, 502
- stdc_leading_zeros_us** function, 306, 502
- stdc_trailing_ones** macro, 307, 503
- stdc_trailing_ones_uc** function, 307, 503
- stdc_trailing_ones_ui** function, 307, 503
- stdc_trailing_ones_ul** function, 307, 503
- stdc_trailing_ones_ull** function, 307, 503
- stdc_trailing_ones_us** function, 307, 503
- stdc_trailing_zeros** macro, 307, 503
- stdc_trailing_zeros_uc** function, 307, 502
- stdc_trailing_zeros_ui** function, 307, 502
- stdc_trailing_zeros_ul** function, 307, 502
- stdc_trailing_zeros_ull** function, 307, 502
- stdc_trailing_zeros_us** function, 307, 502
- stderr** stream, 185, 323, 324, 325, 330, 347, 429, 505, 619
- stdin** stream, 323, 324, 325, 330, 343–345, 351, 428, 431, 433, 434, 505, 643, 644, 650, 675
- stdout** stream, 323, 324, 325, 330, 337–339, 345, 351, 423, 431, 434, 505
- storage duration, 37
- storage order of array, 75
- storage unit (bit-field), 42, 106
- storage-class specifier, 98, 190
 - _Thread_local**, 53
 - auto**, 53, 98, 99, 100, 129, 134, 135, 136, 159, 190, 687
 - constexpr**, 36–38, 53, 79, 95, 96, 98–103, 137, 190, 287, 531, 532, 686
 - extern**, 36, 37, 53, 83, 98, 99, 100, 114, 120–122, 124, 125, 129, 133, 151, 160–162, 176, 177, 194, 603, 619
 - register**, 53, 99
 - static**, 53, 99
 - thread_local**, 37, 53, 99
 - typedef**, 53, 99, 133
- store and load, 15
- str** identifier prefix, 458–460
- strcat** function, 380, 509
- strcat_s** function, 509, 660, 661
- strchr** macro, 382, 383, 460, 509, 687
- strcmp** function, 381, 382, 509
- strcoll** function, 9, 230, 381, 382, 509
- strcpy** function, 176, 177, 344, 379, 509
- strcpy_s** function, 509, 659
- strcspn** function, 383, 509
- strdup** function, 9, 379, 380, 509, 686
- stream, 324, 369
 - binary, 324
 - fully buffered, 325
 - line buffered, 325
 - orientation, 324
 - standard error, 323, 325
 - standard input, 323, 325
 - standard output, 323, 325
 - text, 324
 - unbuffered, 325
- strerror** function, 9, 356, 385, 386, 509, 609, 610, 618
- strerror_s** function, 386, 510, 663, 664
- strerrorlen_s** function, 510, 664
- strfrom** family, 214, 216
- strfromd** family, 359
- strfromd** function, 358, 359, 435, 507, 525
- strfromd128** function, 359, 508
- strfromd32** function, 359, 508
- strfromd64** function, 359, 508
- strfromdN** function, 591
- strfromdNx** function, 591
- strfromencbindN** function, 592
- strfromencdecN** function, 592
- strfromencf128** function, 591, 592
- strfromencfN** function, 592
- strfromf** function, 358, 359, 435, 507
- strfromfN** function, 591
- strfromfNx** function, 591
- strfroml** function, 358, 359, 507
- strftime** function, 230, 406, 408, 411, 446, 512, 600, 608, 610, 617, 665, 666, 686, 690
- stricter, 45
- strictly conforming program, 9
- string, 191
 - comparison function, 381
 - concatenation function, 380, 660
 - conversion function, 230
 - copying function, 378, 658
 - library function convention, 378
 - literal, 12, 19, 50, 67, 73, 138
 - miscellaneous function, 385, 663
 - numeric conversion function, 225, 357
 - search function, 382, 662
- string duplicate function, 379, 380
- string handling header, 378, 460, 658
- string literal
 - wide, 67
- stringizing, 181, 189
- stringizing argument, 181
- strlen** function, 380, 386, 509
- strncat** function, 380, 509
- strncat_s** function, 509, 661, 662
- strncmp** function, 184, 381, 382, 509
- strncpy** function, 379, 509
- strncpy_s** function, 509, 659, 660
- strndup** function, 9, 380, 509, 686
- strnlen_s** function, 510, 659–661, 664
- stronger, 45
- strpbrk** macro, 382, 383, 460, 509, 687
- strrchr** macro, 382, 383, 384, 460, 509, 687

- strspn** function, 384, 509
- strstr** macro, 382, 384, 460, 509, 687
- strto** family, 33, 34, 214, 216, 361
- strtod** family, 361, 362
- strtod** function, 63, 268, 340, 341, 345, 358, 359, 435, 507, 525, 530, 531, 599, 616
- strtod128** function, 361, 508, 592, 616, 617
- strtod32** function, 361, 508, 616, 617
- strtod64** function, 361, 362, 508, 616, 617
- strtodN** function, 591
- strtodNx** function, 591
- strtoencbindN** function, 593
- strtoencdecN** function, 593
- strtoencfN** function, 593
- strtof** function, 268, 345, 358, 359, 507, 599, 616
- strtofN** function, 591
- strtofNx** function, 591
- strtoimax** function, 225, 226, 480
- strtok** function, 9, 384, 385, 509, 610
- strtok_s** function, 385, 510, 662, 663
- strtol** function, 226, 340, 341, 345, 358, 363, 364, 507
- strtold** function, 268, 345, 358, 359, 507, 599, 616
- strtoll** function, 226, 345, 358, 363, 364, 507
- strtoul** function, 226, 341, 345, 358, 363, 364, 507
- strtoull** function, 226, 345, 358, 363, 364, 507
- strtoumax** function, 225, 226, 480
- struct** keyword, 53
- structure
 - arrow operator (->), 76
 - content, 116
 - dot operator (.), 76
 - initialization, 138
 - member alignment, 106
 - member name space, 37
 - member operator (.), 50, 76
 - pointer operator (->), 76
 - specifier, 104
 - tag, 37, 116
 - type, 40, 104
- structure content, 116
- structure or union constant, 96
- strxfrm** function, 9, 230, 382, 509, 610
- subnormal floating-point number, 24
- subscripting, 75
- subtract and round to narrower type, 276
- subtraction assignment operator (-=), 93
- subtraction operator (-), 85, 556
- successful termination, 369
- suffix
 - floating constant, 62
 - integer constant, 59
- suffix** embed parameter, 172, 176, 177
- switch body, 155
- switch **case** label, 153, 155
- switch **default** label, 153, 155
- switch** keyword, 53
- switch** statement, 153, 154
- swprintf** function, 428, 430, 512, 669, 670
- swprintf_s** function, 514, 669, 670
- swscanf** function, 428, 429, 430, 512
- swscanf_s** function, 514, 670, 673
- symbol, 3
- synchronization operation, 17
- synchronize, 149
- synchronize with, 17
- syntactic category, 35
- syntax notation, 35
- syntax rule precedence, 11
- syntax summary
 - language, 461
- system** function, 370, 371, 507, 610, 612, 617
- t format modifier, 333, 340, 420, 425
- tab character, 20, 52
- tag, 114
- tag compatibility, 43
- tag name space, 37
- tags, 37
- tan** function, 244, 245, 388, 482, 529, 537, 563
- tan** type-generic macro, 388, 563
- tand128** function, 245, 487
- tand32** function, 245, 487
- tand64** function, 245, 487
- tandN** function, 579
- tandNx** function, 579
- tanf** function, 244, 482
- tanfN** function, 579
- tanfNx** function, 579
- tanh** function, 249, 388, 482, 529, 539, 563
- tanh** type-generic macro, 388, 563
- tanhd128** function, 249, 488
- tanhd32** function, 249, 488
- tanhd64** function, 249, 488
- tanhdN** function, 580
- tanhdNx** function, 580
- tanhf** function, 249, 482
- tanhfN** function, 580
- tanhfNx** function, 580
- tanhl** function, 249, 483
- tanl** function, 244, 482
- tanpi** function, 247, 482, 529, 538
- tanpi** type-generic macro, 388
- tanpid128** function, 247, 487
- tanpid32** function, 247, 487
- tanpid64** function, 247, 487
- tanpidN** function, 579
- tanpidNx** function, 579
- tanpif** function, 247, 482
- tanpifN** function, 579

- tanpifNx** function, 579
- tanpil** function, 247, 482
- temporary lifetime, 38
- tentative definition, 161
- term, 3
- text stream, 324, 352–354
- tgamma** function, 261, 262, 484, 544
- tgamma** type-generic macro, 388
- tgammad128** function, 262, 489
- tgammad32** function, 261, 489
- tgammad64** function, 261, 489
- tgammadN** function, 583
- tgammadNx** function, 583
- tgammaf** function, 261, 484
- tgammafN** function, 583
- tgammafNx** function, 583
- tgammal** function, 261, 484
- thousands_sep** structure member, 229, 231
- thrd_** identifier prefix, 460
- thrd_busy** constant, 393, 397, 511
- thrd_create** function, 392, 397, 511
- thrd_current** function, 397, 511
- thrd_detach** function, 398, 511, 610
- thrd_equal** function, 398, 511
- thrd_error** constant, 393, 394–401, 511
- thrd_exit** function, 397, 398, 511, 600
- thrd_join** function, 398, 399, 511, 610
- thrd_nomem** constant, 393, 394, 397, 511
- thrd_sleep** function, 399, 511
- thrd_start_t** type, 392, 397, 511
- thrd_success** constant, 393, 394–401, 511
- thrd_t** type, 392, 397, 398, 511
- thrd_timedout** constant, 393, 395, 396, 511
- thrd_yield** function, 399, 511
- thread, 16
- thread of execution, 16, 194, 210, 370, 653
- thread storage duration, 37, 210
- thread_local**, 37, 99
- thread_local** storage-class specifier, 37, 53
- threads header, 392, 460
- time
 - broken down, 403, 404–408, 665, 667
 - calendar, 402, 403–405, 407, 408, 666, 667
 - component, 402, 665
 - conversion function, 406, 665
 - wide character, 446
 - local, 402
 - manipulation function, 403
 - normalized broken down, 665
- time base, 402, 405
- time** function, 404, 405, 511, 600
- TIME_** identifier prefix, 402, 460
- TIME_ACTIVE** macro, 402, 406, 460, 512, 617
- TIME_MONOTONIC** macro, 402, 405, 460, 512, 617
- time_t** type, 402, 403, 404, 405, 407, 408, 511, 512, 617, 666, 667
- TIME_THREAD_ACTIVE** macro, 402, 406, 460, 512, 617
- TIME_UTC** macro, 395, 396, 399, 402, 405, 511, 617
- timegm** function, 404, 405, 511, 688
- timespec** structure type, 394, 396, 399, 402, 403, 405, 406, 511
- timespec_get** function, 402, 405, 406, 511
- timespec_getres** function, 402, 406, 511, 688
- tm** structure type, 402, 403, 404–408, 417, 446, 511–513, 665, 666, 667
- tm_hour** structure member, 403, 404, 407, 409, 665
- tm_isdst** structure member, 403, 404, 409
- tm_mday** structure member, 403, 404, 405, 407, 409, 665
- tm_min** structure member, 403, 404, 407, 409, 665
- tm_mon** structure member, 403, 404, 405, 407–409, 665
- tm_sec** structure member, 403, 404, 407, 409, 666
- tm_wday** structure member, 403, 404, 405, 407–409, 665
- tm_yday** structure member, 403, 404, 405, 409
- tm_year** structure member, 403, 404, 405, 407–409, 666
- TMP_MAX** macro, 323, 327, 328, 505
- TMP_MAX_S** macro, 506, 638, 639, 640
- tmpfile** function, 327, 369, 505
- tmpfile_s** function, 506, 639, 640
- tmpnam** function, 323, 327, 328, 505, 640
- tmpnam_s** function, 506, 638, 639, 640
- to** identifier prefix, 458, 460
- token, 12, 52
- token concatenation, 181
- token pasting, 181
- tolower** function, 207, 478
- totalorder** function, 492, 526, 549, 550
- totalorderd128** function, 492, 549
- totalorderd32** function, 492, 549
- totalorderd64** function, 492, 549
- totalorderdN** function, 587
- totalorderdNx** function, 587
- totalorderf** function, 492, 549
- totalorderfN** function, 587
- totalorderfNx** function, 587
- totalorderl** function, 492, 549
- totalordermag** function, 492, 526, 550
- totalordermagd128** function, 493, 550
- totalordermagd32** function, 493, 550
- totalordermagd64** function, 493, 550
- totalordermagdN** function, 587
- totalordermagdNx** function, 587

- totalordermagf** function, 492, 550
- totalordermagfN** function, 587
- totalordermagfNx** function, 587
- totalordermagl** function, 492, 550
- toupper** function, 207, 208, 478
- towctrans** function, 456, 457, 515, 611, 618
- tolower** function, 456, 457, 515
- toupper** function, 456, 457, 515
- translation environment, 11
- translation limit, 21
- translation phase, 11
- translation unit, 11, 159
- trigonometric function
 - complex, 197, 558
 - real, 242, 536
- true** keyword, 53
- true** predefined constant, 66
- trunc** function, 236, 265, 485, 523, 546
- trunc** type-generic macro, 388
- truncation, 47, 265, 325, 329
- truncation toward zero, 84
- truncd128** function, 265, 490
- truncd32** function, 265, 490
- truncd64** function, 265, 490
- truncdN** function, 584
- truncdNx** function, 584
- truncf** function, 265, 485
- truncfN** function, 584
- truncfNx** function, 584
- truncl** function, 265, 485
- tss_** identifier prefix, 460
- tss_create** function, 399, 400, 511, 610, 611
- tss_delete** function, 400, 511, 600, 611
- TSS_DTOR_ITERATIONS** macro, 392, 398, 511
- tss_dtor_t** type, 392, 399, 511
- tss_get** function, 400, 511, 611
- tss_set** function, 400, 401, 511, 611
- tss_t** type, 392, 399, 400, 511
- tv_nsec** structure member, 403, 405
- tv_sec** structure member, 403, 405
- two's complement, 43
- type, 38
 - aggregate, 41
 - arithmetic, 40
 - atomic, 14, 41, 42, 50, 93, 117, 188, 300
 - basic, 40
 - compatible, 43, 104, 121, 127
 - complex, 40
 - composite, 44
 - const qualified, 121
 - conversion, 46
 - derived, 40
 - derived declarator, 41
 - extended floating, 566
 - integer, 40
 - real, 40
 - restrict qualified, 121
 - scalar, 41
 - typeof, 117
 - volatile qualified, 121
- type category, 41
- type conversion, 46
- type definition, 133
- type domain, 40, 553
- type inference, 134, 619
- type name, 132
- type punning, 76
- type qualifier, 120
- type specifier, 103
 - _BitInt**, 23, 39, 47, 49, 53, 59, 60, 103, 104, 687
 - _Bool**, 53
 - _Complex**, 26, 40, 49, 53, 101–104, 196, 553, 555, 567, 573
 - _Decimal128**, 53
 - _Decimal32**, 53
 - _Decimal64**, 53
 - _Imaginary**, 53
 - bool**, 53
 - char**, 53
 - double**, 53
 - enum**, 53
 - float**, 53
 - int**, 53
 - long**, 53
 - short**, 53
 - signed**, 53
 - struct**, 53
 - union**, 53
 - unsigned**, 53
 - void**, 53
- type-generic macro, 387, 563
- type-generic math header, 387
- typedef**, 99, 133
- typedef** declaration, 133
- typedef** storage-class specifier, 53, 133
- typeof** keyword, 53
- typeof operator, 50, 117
- typeof specifier, 117
- typeof_unqual** keyword, 53
- types
 - atomic, 76, 78
 - character, 138
 - complex, 553
 - imaginary, 553
- U** encoding prefix, 64, 65, 67, 68, 138
- u** encoding prefix, 64, 65, 67, 68, 138
- u8** encoding prefix, 64, 65, 67, 68
- UCHAR_MAX** macro, 23, 24, 481, 519
- UCHAR_WIDTH** macro, 23, 481, 519
- ufromfp** function, 237, 265, 266, 485, 525, 529, 546

- ufromfp** function, 265
- ufromfp** type-generic macro, 388
- ufromfpd128** function, 265, 490
- ufromfpd32** function, 265, 490
- ufromfpd64** function, 265, 490
- ufromfpdN** function, 584
- ufromfpdNx** function, 584
- ufromfpp** function, 265, 485
- ufromfppN** function, 584
- ufromfppNx** function, 584
- ufromfpl** function, 265, 485
- ufromfpx** function, 237, 266, 485, 525, 529, 546
- ufromfpx** function, 266
- ufromfpx** type-generic macro, 388
- ufromfpxd128** function, 266, 490
- ufromfpxd32** function, 266, 490
- ufromfpxd64** function, 266, 490
- ufromfpxdN** function, 584
- ufromfpxdNx** function, 584
- ufromfpxf** function, 266, 485
- ufromfpxfN** function, 584
- ufromfpxfNx** function, 584
- ufromfpxl** function, 266, 485
- UINT** identifier prefix, 320, 321, 459, 505
- uint** identifier prefix, 318, 319, 459, 505, 688
- UINTN_C** macro, 321
- UINTN_MAX** macro, 320
- uintN_t** type, 318
- UINTN_WIDTH** macro, 320
- UINT64_C** macro, 321
- uint64_t** type, 266
- UINT_FAST** identifier prefix, 320, 505
- uint_fast** identifier prefix, 319, 505, 688
- UINT_LEAST** identifier prefix, 320, 505
- uint_least** identifier prefix, 318, 319, 321, 505
- UINT_FASTN_MAX** macro, 320
- UINT_FASTN_WIDTH** macro, 320
- uint_fast16_t** type, 300, 319
- uint_fast32_t** type, 300, 319
- uint_fast64_t** type, 300, 319
- uint_fast8_t** type, 300, 319
- uint_fastN_t** type, 319
- UINT_LEASTN_MAX** macro, 320
- uint_leastN_t** type, 318
- UINT_LEASTN_WIDTH** macro, 320
- uint_least16_t** type, 300, 318, 319, 412
- uint_least32_t** type, 300, 319, 412
- uint_least64_t** type, 300, 319, 321
- uint_least8_t** type, 300, 319
- UINT_MAX** macro, 23, 103, 112, 167, 481, 519
- UINT_WIDTH** macro, 23, 266, 481, 519
- UINTMAX_C** macro, 321, 505
- UINTMAX_MAX** macro, 225, 226, 320, 505, 529
- uintmax_t** type, 23, 167, 225, 226, 300, 320, 321, 333, 340, 419, 425, 480, 505, 687, 689
- UINTMAX_WIDTH** macro, 320, 505
- UINTPTR_MAX** macro, 320, 505
- uintptr_t** type, 300, 319, 505
- UINTPTR_WIDTH** macro, 320, 505
- ULLONG_MAX** macro, 23, 102, 112, 364, 440, 481, 519
- ULLONG_WIDTH** macro, 23, 481, 519
- ULONG_MAX** macro, 23, 364, 440, 481, 519
- ULONG_WIDTH** macro, 23, 481, 519
- unary arithmetic operator, 82
- unary expression, 81
- unary minus operator (-), 82, 527
- unary operator, 81
- unary plus operator (+), 82
- unbuffered, 325
- unbuffered stream, 325
- undef**, 55, 163, 166, 182, 183, 187, 193, 194, 605
- undef** preprocessing directive, 183, 193
- undefined behavior, 4, 9, 600
- underlying type, 109
- underscore
 - leading
 - in identifier, 192
- underspecified, 98
- ungetc** function, 323, 351, 352–354, 459, 506, 599, 609, 620, 690
- ungetwc** function, 323, 434, 435, 513, 599, 620
- Unicode, 412
- Unicode required set, 188
- Unicode Standard
 - Annex, UAX #31, 691
 - Annex, UAX #44, 2
 - Derived Core Properties, 2
- Unicode utilities header, 412
- union
 - arrow operator (->), 76
 - content, 116
 - dot operator (.), 76
 - initialization, 138
 - member alignment, 106
 - member name space, 37
 - member operator (.), 50, 76
 - pointer operator (->), 76
 - specifier, 104
 - tag, 37, 116
 - type, 40, 104
- union content, 116
- union** keyword, 53
- universal character name, 56, 517
- unnormalized floating-point number, 24
- unqualified type, 41
- unqualified version of type, 41
- unreachable, 316
- unreachable** macro, viii, 315, 316, 504, 607, 688

- unsequenced, 14, 72, 91, 149
- unsequenced** attribute, 143, 148, 150, 151, 306–312, 502–504, 604, 685, 686
- unsigned bit-precise integer suffix, **uwb** or **UWB**, 59
- unsigned integer suffix, **u** or **U**, 59
- unsigned integer type, 39, 47, 59
- unsigned** keyword, 53
- unsigned** type, 39, 103, 332, 340, 419, 425
- unsigned** type conversion, 46–48
- unspecified behavior, 4, 9, 598
- unspecified value, 7
- unsuccessful termination, 368, 369
- uppercase letter, 19
- use of library function, 193
- USHRT_MAX** macro, 23, 111, 481, 519
- USHRT_WIDTH** macro, 23, 481, 519
- usual arithmetic conversion, 48, 84, 85, 87–90
- UTF-8 string literal, *see* string literal
- UTF-16 character constant, 64
- UTF-16 string literal, 67
- UTF-32 character constant, 64
- UTF-32 string literal, 67
- UTF-8 character constant, 64
- UTF-8 string literal, 67
- utility
 - bit and byte, 305
 - general, 357, 459, 651
 - wide string, 435, 675
 - Unicode, 412
- va_arg** function, 224, 290, 291–293, 336, 347–349, 423, 429–431, 501, 607, 647–649, 671, 673, 674
- va_copy** function, 193, 290, 291, 293, 501, 599, 607, 690
- va_end** function, 193, 290, 291, 292, 293, 347–349, 429–431, 501, 599, 607, 609, 647–649, 671, 673, 674
- va_list** type, 290, 291–293, 347–349, 429–431, 501, 505, 506, 512–514, 607, 609, 646–649, 670–674
- va_start** function, 290, 291, 292, 293, 347–349, 429–431, 501, 607, 647–649, 671, 673, 674, 687
- value, 7
- value bit, 42
- value of a string, 191
- value of a wide string, 191
- variable argument, 179
- variable arguments header, 290
- variable length array, 127, 128, 189
- variably modified, 127
- variably modified type, 127, 128, 189
- vertical tab, 21
- vertical-tab character, 20, 52
- vertical-tab escape sequence (**\v**), 21, 65, 207
- vfprintf** function, 323, 347, 505, 609, 646
- vfprintf_s** function, 506, 646, 647–649
- vfprintf** function, 323, 347, 505, 609
- vfprintf_s** function, 506, 647, 648, 649
- vfwprintf** function, 323, 429, 512, 609, 671
- vfwprintf_s** function, 514, 670, 671
- vfwscanf** function, 323, 429, 430, 434, 512, 609
- vfwscanf_s** function, 514, 671, 673, 674
- visibility of identifier, 35
- visible, 35
- visible side effect, 18
- void expression, 50
- void** function parameter, 130
- void** keyword, 53
- void** type, 50, 103
- void** type conversion, 50
- volatile**, 41
- volatile access, 14
- volatile** keyword, 53
- volatile** type qualifier, 120
- volatile-qualified type, 41, 121
- vprintf** function, 323, 347, 348, 505, 609, 647
- vprintf_s** function, 506, 647, 648, 649
- vscanf** function, 323, 347, 348, 505, 609, 689
- vscanf_s** function, 506, 647, 648–650
- vsprintf** function, 347, 348, 505, 609, 648
- vsprintf_s** function, 506, 647, 648, 649
- vsprintf_s** function, 514, 671, 672
- vsprintf** function, 347, 348, 349, 505, 609, 649
- vsprintf_s** function, 506, 647, 648, 649
- vsscanf** function, 347, 349, 505, 609
- vsscanf_s** function, 506, 647, 648, 649, 650
- vswprintf** function, 429, 430, 513, 609, 672
- vswprintf_s** function, 514, 672
- vswscanf** function, 429, 430, 513, 609
- vswscanf_s** function, 514, 671, 672, 673, 674
- vwprintf** function, 323, 429, 430, 431, 513, 609, 673
- vwprintf_s** function, 514, 673
- vwscanf** function, 323, 429, 431, 434, 513, 609
- vwscanf_s** function, 514, 671, 673, 674
- warning, 12, 597
- WCHAR_MAX** macro, 417, 505, 512
- WCHAR_MIN** macro, 321, 417, 505, 512
- wchar_t** type, 5, 45, 64–67, 138, 188, 226, 300, 315, 321, 332, 335, 337, 340, 342, 345, 357, 373–375, 412, 417, 418, 419, 422–435, 437, 439–446, 449–452, 480, 504, 507, 509, 512–515, 591, 593, 594, 600, 611, 612, 655–657, 667–682
- WCHAR_WIDTH** macro, 321, 417, 505
- wcrtomb** function, 326, 335, 339, 345, 417, 426–428, 449, 450, 451, 513, 600, 657, 680, 683
- wcrtomb_s** function, 515, 680, 681
- wcs** identifier prefix, 458–460

- wscat** function, 441, 513
- wscat_s** function, 515, 677, 678
- wcschr** macro, 443, 460, 513, 687
- wscmp** function, 441, 442, 513
- wscoll** function, 442, 513
- wscopy** function, 440, 513
- wscopy_s** function, 514, 675
- wscspn** function, 443, 444, 513
- wcsftime** function, 230, 446, 447, 513, 600, 608, 610, 617
- wcslen** function, 441, 446, 513, 680
- wcsncat** function, 441, 513
- wcsncat_s** function, 515, 678, 679
- wcsncmp** function, 442, 513
- wcsncpy** function, 440, 513
- wcsncpy_s** function, 514, 675, 676
- wcsnlen_s** function, 515, 675–678, 680
- wcspbrk** macro, 443, 444, 460, 513, 687
- wcsrchr** macro, 443, 444, 460, 513, 687
- wcsrtombs** function, 450, 451, 514, 681
- wcsrtombs_s** function, 515, 681, 682, 683
- wcsspn** function, 444, 513
- wcsstr** macro, 443, 444, 445, 460, 513, 687
- wcsto** family, 33, 34, 214, 216, 437
- wcstod** family, 437, 438
- wcstod** function, 424, 426, 428, 435, 513, 525, 599, 616
- wcstod128** function, 435, 437, 514, 616, 617
- wcstod32** function, 435, 437, 514, 616, 617
- wcstod64** function, 435, 437, 514, 616, 617
- wcstodN** function, 591
- wcstodNx** function, 591
- wcstoencbindN** function, 594
- wcstoencdecN** function, 594
- wcstoencfN** function, 593
- wcstof** function, 428, 435, 513, 599, 616
- wcstofN** function, 591
- wcstofNx** function, 591
- wcstoimax** function, 226, 480
- wcstok** function, 445, 513, 610, 611
- wcstok_s** function, 515, 679, 680
- wcstol** function, 226, 424, 426, 428, 439, 440, 513
- wcstold** function, 428, 435, 513, 599, 616
- wcstoll** function, 226, 428, 439, 440, 513
- wcstombs** function, 375, 450, 507
- wcstombs_s** function, 509, 657, 658
- wcstoul** function, 226, 426, 428, 439, 440, 513
- wcstoull** function, 226, 428, 439, 440, 513
- wcstoumax** function, 226, 480
- wcsxfrm** function, 442, 443, 513, 610
- wctob** function, 447, 448, 452, 513
- wctomb** function, 373, 374, 375, 448, 507
- wctomb_s** function, 509, 655, 656
- wctrans** function, 456, 457, 515, 611
- wctrans_t** type, 452, 456, 457, 515
- wctype** function, 455, 456, 515, 611
- wctype_t** type, 452, 455, 456, 515
- weaker**, 45
- WEOF** macro, 417, 432–435, 447, 452, 512, 515, 611
- wfN** format modifier, 333, 341, 420, 425
- while** keyword, 53
- while** statement, 156
- white space, 11, 52, 164, 165, 207, 454
- white-space character, 52, 191
- White-space wide character, 191
- wide character, 5
 - case mapping function, 456
 - extensible, 456
 - classification function, 452
 - constant, 64
 - extensible classification function, 455
 - formatted input/output function, 418, 667
 - input function, 323
 - input/output function, 323, 431
 - output function, 323
 - single-byte conversion function, 447
- wide character classification and mapping utilities header, 452, 460
- wide character constant, 64
- wide character input functions, 323
- wide character input/output function, 323
- wide character output function, 323
- wide literal encoding, 45
- wide string, 191
- wide string comparison function, 441
- wide string concatenation function, 441, 677
- wide string copying function, 440, 675
- wide string literal, *see* string literal, 67
- wide string miscellaneous function, 446, 680
- wide string numeric conversion function, 226, 435
- wide string search function, 443, 679
- wide-oriented stream, 324
- width, 42, 43
- WINT_MAX** macro, 505
- WINT_MIN** macro, 321, 505
- wint_t** type, 321, 332, 335, 337, 417, 419, 422, 432–434, 447, 452, 453–456, 512, 513, 515, 611
- WINT_WIDTH** macro, 321, 505
- wmemchr** macro, 443, 445, 446, 460, 513, 687
- wmemcmp** function, 443, 513
- wmemcpy** function, 440, 513
- wmemcpy_s** function, 515, 676, 677
- wmemmove** function, 440, 441, 513
- wmemmove_s** function, 515, 677
- wmemset** function, 446, 513
- wN** format modifier, 333, 340, 420, 425
- wprintf** function, 214, 216, 225, 323, 430, 431,

513, 525, 674	XID_Continue, 54
wprintf_s function, 514, 674	XID_Start, 54
wraparound, 7	xor macro, 227, 481
write-read coherence, 19	xor_eq macro, 227, 481
write-write coherence, 18	
wscanf function, 214, 216, 323, 431, 434, 513, 525	z format modifier, 333, 340, 419, 425
wscanf_s function, 514, 674, 675	zero, 553

This page deliberately left blank

British Standards Institution (BSI)

BSI is the national body responsible for preparing British Standards and other standards-related publications, information and services.

BSI is incorporated by Royal Charter. British Standards and other standardization products are published by BSI Standards Limited.

About us

We bring together business, industry, government, consumers, innovators and others to shape their combined experience and expertise into standards-based solutions.

The knowledge embodied in our standards has been carefully assembled in a dependable format and refined through our open consultation process. Organizations of all sizes and across all sectors choose standards to help them achieve their goals.

Information on standards

We can provide you with the knowledge that your organization needs to succeed. Find out more about British Standards by visiting our website at bsigroup.com/standards or contacting our Customer Services team or Knowledge Centre.

Buying standards

You can buy and download PDF versions of BSI publications, including British and adopted European and international standards, through our website at bsigroup.com/shop, where hard copies can also be purchased.

If you need international and foreign standards from other Standards Development Organizations, hard copies can be ordered from our Customer Services team.

Copyright in BSI publications

All the content in BSI publications, including British Standards, is the property of and copyrighted by BSI or some person or entity that owns copyright in the information used (such as the international standardization bodies) and has formally licensed such information to BSI for commercial publication and use.

Save for the provisions below, you may not transfer, share or disseminate any portion of the standard to any other person. You may not adapt, distribute, commercially exploit or publicly display the standard or any portion thereof in any manner whatsoever without BSI's prior written consent.

Storing and using standards

Standards purchased in soft copy format:

- A British Standard purchased in soft copy format is licensed to a sole named user for personal or internal company use only.
- The standard may be stored on more than one device provided that it is accessible by the sole named user only and that only one copy is accessed at any one time.
- A single paper copy may be printed for personal or internal company use only.

Standards purchased in hard copy format:

- A British Standard purchased in hard copy format is for personal or internal company use only.
- It may not be further reproduced – in any format – to create an additional copy. This includes scanning of the document.

If you need more than one copy of the document, or if you wish to share the document on an internal network, you can save money by choosing a subscription product (see 'Subscriptions').

Reproducing extracts

For permission to reproduce content from BSI publications contact the BSI Copyright and Licensing team.

Subscriptions

Our range of subscription services are designed to make using standards easier for you. For further information on our subscription products go to bsigroup.com/subscriptions.

With **British Standards Online (BSOL)** you'll have instant access to over 55,000 British and adopted European and international standards from your desktop. It's available 24/7 and is refreshed daily so you'll always be up to date.

You can keep in touch with standards developments and receive substantial discounts on the purchase price of standards, both in single copy and subscription format, by becoming a **BSI Subscribing Member**.

PLUS is an updating service exclusive to BSI Subscribing Members. You will automatically receive the latest hard copy of your standards when they're revised or replaced.

To find out more about becoming a BSI Subscribing Member and the benefits of membership, please visit bsigroup.com/shop.

With a **Multi-User Network Licence (MUNL)** you are able to host standards publications on your intranet. Licences can cover as few or as many users as you wish. With updates supplied as soon as they're available, you can be sure your documentation is current. For further information, email cservices@bsigroup.com.

Revisions

Our British Standards and other publications are updated by amendment or revision.

We continually improve the quality of our products and services to benefit your business. If you find an inaccuracy or ambiguity within a British Standard or other BSI publication please inform the Knowledge Centre.

Useful Contacts

Customer Services

Tel: +44 345 086 9001

Email: cservices@bsigroup.com

Subscriptions

Tel: +44 345 086 9001

Email: subscriptions@bsigroup.com

Knowledge Centre

Tel: +44 20 8996 7004

Email: knowledgecentre@bsigroup.com

Copyright & Licensing

Tel: +44 20 8996 7070

Email: copyright@bsigroup.com

BSI Group Headquarters

389 Chiswick High Road London W4 4AL UK